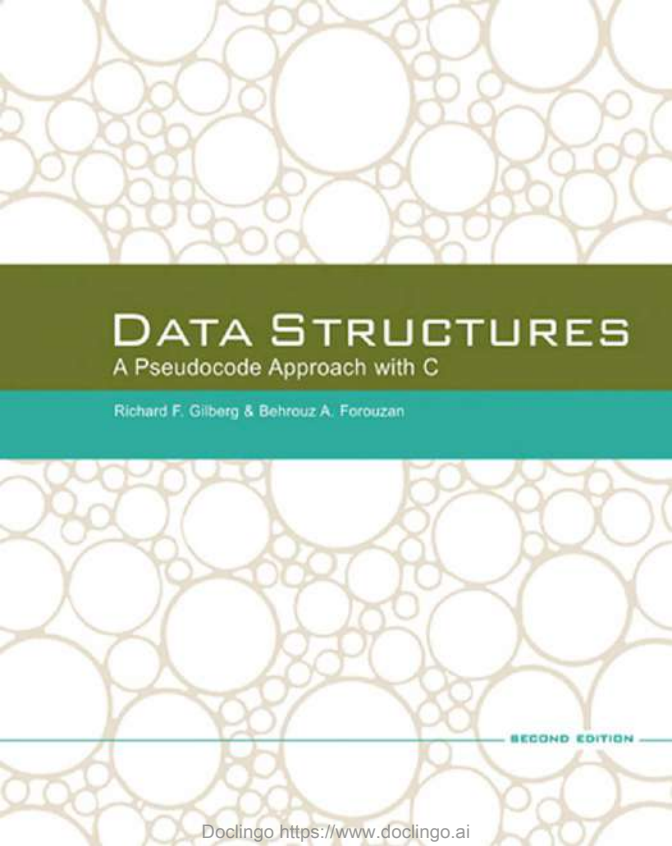




DATA STRUCTURES

A Pseudocode Approach with C

Richard F. Gilberg & Behrouz A. Forouzan

SECOND EDITION



Veri Yapıları: C ile Söзде Kod Yaklaşımı ,İkinci Baskı

Richard F. Gilberg ve Behrouz A. Forouzan

Bu, basılı ders kitabının elektronik versiyonudur. Elektronik hak kısıtlamaları nedeniyle, bazı üçüncü taraf içerikleri bastırılmış olabilir. Editoryal inceleme, bastırılan herhangi bir içeriğin genel öğrenme deneyimini önemli ölçüde etkilemediğini değerlendirmiştir. Yayıncı, sonraki hak kısıtlamaları gerektirirse bu başlıktan içerik kaldırma hakkını saklı tutar. Fiyatlandırma, önceki baskılar, mevcut baskılardaki değişiklikler ve alternatif formatlar hakkında değerli bilgiler için lütfen www.cengage.com/highered adresini ziyaret edin ve ISBN#, yazar, başlık veya anahtar kelime ile ilgili zi çeken materyalleri arayın.

Veri Yapıları: C ile Sözde Kod Yaklaşımı, İkinci Baskı
Richard F. Gilberg
Behrouz A. Forouzan

Kıdemli Ürün Yöneticisi:
Alyssa Pratt

Kıdemli Edinim Editörü:
Amy Yarnevich

Üretim Editörü:
Bobbijo Frasca

Kıdemli Pazarlama Müdürü:
Karen Seitz

Ürün Müdürü Yardımcısı:
Mirella Misiaszek

Editör Asistanı:
Amanda Piantedosi

Kıdemli Üretim Koordinatörü:
Trevor Kallop

Kapak Tasarımcısı:
Abby Scholz

TELİF HAKKI © 2005 Course Technology, Thomson Learning, Inc.'in bir bölümü. Thomson Learning™ burada lisans altında kullanılan bir ticari markadır.

Amerika Birleşik Devletleri'nde basılmıştır

1 2 3 4 5 6 7 8 9 QWT 09 08 07 06 05

Daha fazla bilgi için Course Technology ile iletişime geçin,
25 Thomson Place, Boston, Massachusetts, 02210.

Ya da bizi Dünya Çapında Ağ'da bulabilirsiniz:
www.course.com

TÜM HAKLAR SAKLIDIR. Bu eserin telif hakkı ile korunan hiçbir bölümü, burada belirtilen şekilde, grafik, elektronik veya mekanik, fotokopi, kayıt,

şerit, Web dağıtım veya bilgi depolama ve yerine getirme sistemleri dahil olmak üzere, yayınevinin yazılı izni olmadan, herhangi bir formda veya herhangi bir yöntemle çoğaltılamaz veya kullanılamaz.

Bu metinden veya üründen materyal kullanma izni için, <http://www.thomsonrights.com> adresinde çevrimiçi bir talep gönderin. İzinlerle ilgili elektronik sorular, thomsonrights@thomson.com adresine e-posta ile gönderilebilir. Feragatname

Course Technology, bu yayını gözden geçirme ve içeriğinde zaman zaman değişiklik yapma hakkını saklı tutar.

ISBN-13: 978-0-534-39080-8
ISBN-10: 0-534-39080-3

Adanmışlık Sayfası

Yıllar boyunca verdiği destek için eşim Evelyn'e.
Richard Gilberg

İhtiyacım olduğunda yardım eden Mark Bauer'a.
Behrouz Foruzan

İçindekiler

Bölüm 1 Giriş 1

Bölüm 1 Temel Kavramlar 5

1.1 Sözde kod 5

Algoritma başlığı 6

Amaç, Koşullar ve Döndürme 7

İfade numaraları 7

Değişkenler 8

İfade yapıları 8

Sıra (ardışıklık) 85

eçim (koşullu yapı

) 9Döngü 9

Algoritma analizi 9

Sözde kod örneği 9

1.2 Soyut Veri Türü 10

Atomik ve bileşik veri 10

Veri türü 11

Veri yapısı 11

Soyut Veri Türü 12

1.3 Soyut Veri Türü için Model 14

ADT işlemleri 14

ADT veri yapısı 15

1.4 ADT uygulamaları 15

Dizi uygulamaları 15

Bağlı liste uygulamaları 15

Bağlı listelere işaretçiler 17

1.5 ADT'ler için genel kod 17

void* işaretçisi 18

Fonksiyona işaretçi 23

Fonksiyonlara işaretçilerin tanımlanması 24

Fonksiyonlara işaretçilerin kullanılması 24

1.6 Algoritma Verimliliği 28

Doğrusal döngüler 29

Logaritmik Döngüler 29

İç İçe Döngüler 30

Doğrusal Logaritmik 31

Kare 31

Bağımlı Kare 31

Büyük-O Notasyonu 32

Verimlilik için Standart Ölçüler 33

Büyük-O Analiz Örnekleri 35

Kare Matrisleri Topla 35

Kare Matrisleri Çarp 36

1.7 Anahtar Terimler 38

1.8 Özet 38

1.9 Uygulama Setleri 39

Alıştırmalar 39

Problemler 41

Projeler 42

Bölüm 2 Özyineleme 45

2.1 Faktöriyel—Bir Vaka Çalışması 45

Özyineleme Tanımlandı 46

İteratif Çözüm 47

Özyinelemeli Çözüm 47

2.2 Özyinelemeli Algoritmaların Tasarımı 48

Tasarım Metodolojisi 48

Özyinelemenin Sınırlamaları 50

Tasarım Uygulaması—Ters

Klavve Girişi 50

2.3 Özyinelemeli Örnekler 52En Büyük Ortak Bölgenin Bulunması 52

GCD Tasarımı 53

GCD C Uygulaması 53

Fibonacci Sayıları 54

Tasarım 55

Fibonacci C Uygulaması 56

Önekten Postfiks Dönüşümü 57*Tasarım 58**Önekten Postfiks**C Uygulaması 61***Hanoi Kuleleri 65***Özyinelemeli Hanoi Kuleleri Tasarımı 65**Hanoi Kuleleri**C Uygulaması 69***2.4 Anahtar Terimler 71****2.5 Özet 71****2.6 Uygulama Setleri 72***Alıştırmalar 72**Problemler 74**Projeler 75***Bölüm II Doğrusal Listeler 77****Bölüm 3 Yığınlar 79****3.1 Temel Yığın İşlemleri 80***İtme 80**Çekme 80**Yığın Üstü 81***3.2 Yığın Bağlı Liste***Uygulama 83**Veri Yapısı 83**Yığın Başlığı 83**Yığın Veri Düğümü 83**Yığın Algoritmaları 84**Yığın Oluştur 84**Yığına Ekle 85**Yığından Çık**r 86Yığın Üst**ü 87Boş Yığın 8**8Dolu Yığın 8**9Yığın Sayısı 89**Yığını Yok Et 89***3.3 C Dili Uygulamaları 90***Veri Ekle 92**Yığına Ekle 93**Yığını Yazdır 93**Karakteri Çıkar 94***3.4 Yığın ADT 95***Veri Yapısı 95**ADT Uygulaması 95**Yığın Yapısı 95**Yığın Oluştur 96**Yığına Ekle 97**Yığından Çık**r 98Yığın Üst**ü 99Boş Yığın 10**0Yığın Sayısı 101**Yığını Yok Et 101***3.5 Yığın Uygulamaları 102***Veri Tersine Çevirme 103**Bir Listeyi Tersine Çevir 103**Ondalık Sayıyı İkiliye Dönüştür 106**Parçalama 107**Erteleme 110**Infix'ten Postfix'e Dönüşüm 110**Postfix İfadelerini Değerlendirme 118**Geri İzleme 122**Amaç Arama 122**Sekiz Kraliçe Problemi 125***3.6 Rekürsiyonun Çalışma Şekli 134****3.7 Anahtar Terimler 138****3.8 Özet 138****3.9 Pratik Setleri 139***Alıştırmalar 139**Problemler 141**Projeler 143***Bölüm 4 Kuyruklar 147****4.1 Kuyruk İşlemleri 148Kuyr***uğa Ekle 148**Kuyruktan Çıkar 148**Kuyruk Önü 149**Kuyruk Arkası 150**Kuyruk Örneği 150***4.2 Kuyruk Bağlı Liste Tasarımı 151***Veri Yapısı 152**Kuyruk Başlığı 152**Kuyruk Veri Düğümü 153**Kuyruk Algoritmaları 153**Kuyruk Oluştur 154**Kuyruğa Ekle 154**Kuyruktan Çıkar 155**Kuyruk Verilerini Alma 156**Boş Kuyruk 157**Tam Kuyruk 157**Kuyruk Sayısı 158**Kuyruğu Yok Et 158*

- 4.3 Kuyruk ADT 159
 - Kuyruk Yapısı 159*
 - Kuyruk ADT Algoritmaları 159*
- 4.4 Kuyruk Teorisi 166
- 4.5 Kuyruk Uygulamaları 168
 - Veri Sınıflandırma 168
 - Veri Sınıflandırma Tasarımı 169*
 - Veri Sınıflandırma—C Uygulaması 170*
 - Kuyruk Simülasyonu 175
 - Olaylar 176*
 - Veri Yapıları 176*
 - Çıktı 177*
 - Simülasyon Algoritması 178*
- 4.6 Anahtar Terimler 183
- 4.7 Özet 183
- 4.8 Uygulama Setleri 184
 - Alıştırmalar 184
 - Problemler 187
 - Projeler 188

Bölüm 5 Genel Doğrusal Listeler 193

- 5.1 Temel İşlemler 193
 - Ekleme 193
 - Silme 194
 - Alma 194
 - Geçiş 194
- 5.2 Uygulama 195
 - Veri Yapısı 195
 - Baş Düğüm 196*
 - Veri Düğümü 196*
 - Algoritmalar 197
 - Liste Oluştur 197*
 - Düğüm Ekle 198*
 - Düğüm Sil 202*
 - Liste Ara 205*
 - Düğüm Al 208*
 - Boş Liste 208*
 - Dolu Liste 209*
 - Liste Sayısı 210*
 - Listeyi Gezin 210*
 - Listeyi Yok Et 212*
- 5.3 Liste ADT 213
 - ADT Fonksiyonları 214
 - Liste Oluştur 216*
 - Düğüm Ekle 217*

- İçsel Ekleme Fonksiyonu 217*
- Düğüm Kaldır 219*
- İçsel Silme Fonksiyonu 220*
- Liste Ara 221*
- İçsel Arama Fonksiyonu 222*
- Düğüm Al 224*
- Boş Liste 225*
- Dolu Liste 225*
- Liste Sayısı 226*
- Geçiş 226*
- Listeyi Yok Et 227*

- 5.4 Uygulama 228
 - Veri Yapısı 228
 - Uygulama Fonksiyonları 229
 - Ana Hat 229*
 - Talimatları Yazdır 229*
 - Liste Oluştur 231*
 - Kullanıcı Taleplerini İşle 233*
 - Kullanıcı Seçimini Al 234*
 - Listeyi Yazdır 235*
 - Listeyi Ara 236*
 - Yılı Karşılaştır 237*
 - Eklenme ve Silme Mantiğini Test Etme 238
 - Eklenme Mantiğini Test Etme 238*
 - Silme Mantiğini Test Etme 238*

- 5.5 Karmaşık Uygulamalar 239
 - Dairesel Bağlı Listeler 239
 - Çift Yönlü Bağlı Listeler 240
 - Ekleme 241*
 - Silme 243*
 - Çok Bağlı Listeler 244
 - Ekle 246*
 - Sil 247*

- 5.6 Anahtar Terimler 248

- 5.7 Özet 248

- 5.8 Uygulama Setleri 249
 - Alıştırmalar 249
 - Sorunlar 251
 - Projeler 253

3. Kısım Doğrusal Olmayan Listeler 263

Bölüm 6 Ağaçlara Giriş 265

- 6.1 Temel Ağaç Kavramları 265
 - Terminoloji 266
 - Kullanıcı Temsili 268

6.2 İkili Ağaçlar 270**Özellikler 271***İkili Ağaçların Yüksekliği 271**Denge 272**Tamamlanmış ve Neredeyse Tamamlanmış İkili Ağaçlar 273***İkili Ağaç Geçişleri 273***Derinlik Öncelikli Geçişler 274**Genişlik Öncelikli Geçişler 278***İfade Ağaçları 280***Infix Geçiş 280**Postfix Geçiş 281**Prefix Geçiş 282***Huffman Kodu 282****6.3 Genel Ağaçlar 287****Genel Ağaçlara Ekleme 287***FIFO Ekleme 287**LIFO Ekleme 287**Anahtar sıralı Ekleme 287***Genel Ağaç Silme 288****Genel Ağacı İkili Ağaç Haline Dönüştürme 288****6.4 Anahtar Terimler 290****6.5 Özet 290****6.6 Uygulama Setleri 292***Alıştırmalar 292**Problemler 295**Projeler 296***Bölüm 7 İkili Arama Ağaçları 299****7.1 Temel Kavramlar 299****7.2 BST İşlemleri 301***Geçişler 301**Aramalar 302**En Küçük Düğümü Bul 302**En Büyük Düğümü Bul 303**BST Araması 303**Ekleme 305**Silme 307***7.3 İkili Arama Ağacı ADT 309***Veri Yapısı 311**Baş Yapısı 311**Düğüm Yapısı 311**Algoritmalar 311**Bir BST Oluştur 313**Bir BST Ekle 313**İçsel Ekleme Fonksiyonu 314**Bir BST Sil 315**İçsel Silme Fonksiyonu 316**Bir BST Al 318**İçsel Alma Fonksiyonu 319**Bir BST'yi Gezin 320**İçsel Gezinme Fonksiyonu 321**Bir BST'yi Boşalt 321**Tam BST 322**BST Sayısı 322**Bir BST'yi Yok Et 323**Dahili Yok Etme Fonksiyonu 323***7.4 BST Uygulamaları 324***Tam Sayı Uygulaması 324**Öğrenci Listesi Uygulaması 328**Program Tasarımı 329***7.5 İplikli Ağaçlar 334****7.6 Anahtar Terimler 336****7.7 Özet 336****7.8 Pratik Setleri 337***Alıştırmalar 337**Sorunlar 339**Projeler 339***Bölüm 8 AVL Arama Ağaçları 341****8.1 AVL Ağaç Temel Kavramları 341***AVL Ağaç Denge Faktörü 342**Ağaçları Dengeleme 343**Durum 1: Solun Solu 344**Durum 2: Sağın Sağı 345**Durum 3: Solun Sağı 346**Durum 4: Sağın Solu 347***8.2 AVL Ağaç Uygulamaları 348***AVL Ağacına Ekle 348**AVL Ağaç Ekleme Algoritması 350**AVL Ağaç Sol Denge**Algoritma 351**Dönme Algoritmaları 351**AVL Ağaç Silme Algoritması 352**Sağ Dengeyi Sil 354**Denge Faktörlerini Ayarlama 355***8.3 AVL Ağaç Soyut Veri Türü 356***AVL Ağaç Veri Yapısı 357**Baş Yapısı 357**Düğüm Yapısı 357*

AVL Ağaç Algoritmaları 357*Bir AVL Ağaç Oluştur 359**Bir AVL Ağaç Ekle 360**Bir AVL Ağaç Sil 365**Bir AVL Ağaç Al 370**Bir AVL Ağaçta Gezin 372**Dahili Gezinme Fonksiyonu 373**Bir AVL Ağaç Boşalt 373**Tam AVL Ağaç 374**AVL Ağaç Sayısı 374**AVL Ağaç Yok Et 375**Dahili Yok Etme Fonksiyonu 375***8.4 Uygulama—Kelimeleri Say 376****Veri Yapısı 376****Program Tasarımı 376***Kelimeleri Sayma Programı 377**Liste Oluştur 378**Kelime Al 380**imeleri Karşılaştır 381**limeleri Yazdır 382***8.5 Anahtar Terimler 384****8.6 Özet 384****8.7 Pratik Setleri 384****Alıştırmalar 384****Sorunlar 386****Projeler 386****Bölüm 9 Yığınlar 389****9.1 Temel Kavramlar 389****Tanım 389****Bakım İşlemleri 391***Yeniden Yığınla 391**Yeniden Yığınla Aşağı 392***9.2 Yığın Uygulaması 394****Algoritmalar 396***Yeniden Yığınlama Yukarı 396**Yeniden Yığınlama A**şağı 396**Bir Yığın Ol**uştır 397**Bir Düğümü Yığına Ekle**398**Bir Düğümü Yığından Sil 400***9.3 Yığın ADT 401****Yığın Yapısı 401****Yığın Algoritmaları 402***Bir Yığın Oluştur 403**Bir Yığına Ekle 404**İçsel Yeniden Yığınlama Yukarı Fonksiyon**u 405**İçsel Yeniden Yığınlama Aşağı**Fonksiyonu 406***9.4 Yığın Uygulamaları 407****Seçim Algoritmaları 408****Öncelik Kuyrukları 409***Tasarım 409**Uygulama 411***9.5 Anahtar Terimler 417****9.6 Özet 417****9.7 Uygulama Setleri 418****Alıştırmalar 418****Sorunlar 420****Projeler 420****Bölüm 10 Çok Yönlü Ağaçlar 423****10.1 M-yönlü Arama Ağaçları 423****10.2 B-ağaçları 425****B-ağaç Uygulaması 427***B-ağaç Ekleme 427**Düğüm Ekle 42**9Düğüm Ara 43**2Düğüm Böl 4**33Ekleme Özeti 435**B-ağaç Silme 435**Düğüm Sil 437**Giriş Sil 439**Orta Sil 439**Yeniden Akış 440**Denge 442**Birleştir 444**B-ağacını Gezin 446**B-ağaç Arama 448***10.3 B-ağaç ADT 449****B-ağaç Veri Yapısı 449***Başlık Yapısı 449**Düğüm Yapısı 450***Başlık Dosyası 451****Algoritmalar 453***B-ağaç Arama 453**Dahili Arama Fonksiyonu 454**B-ağaç Gezinme 455**İçsel Gezinme Fonksiyonu 456**B-ağaç Ekle 457*

İçsel Ekleme Fonksiyonu 459
İçsel Bölme Düzümü Fonksiyonu 460
B-ağaç Sil 462
İçsel Silme Fonksiyonu 463
İçsel Orta Silme
Fonksiyonu 465
İçsel Yeniden Akış Fonksiyonu 466
İçsel Sol veya Sağ Ödünç Alma 467
İçsel Düzümleri Birleştirme
Fonksiyonu 469

10.4 Basitleştirilmiş B-ağaçlar 470
 2-3 Ağaç 470
 2-3-4 Ağaç 470

10.5 B-ağaç Varyasyonları 471
 B* ağaç 471
 B+ ağaç 471

10.6 Sözcüksel Arama Ağacı 472
 Denemeler 473
 Trie Yapısı 474
 Trie Arama 474

10.7 Anahtar Terimler 476

10.8 Özet 476

10.9 Uygulama Setleri 477
 Alıştırmalar 477
 Problemler 478
 Projeler 479

Bölüm 11 Grafikler 481

11.1 Temel Kavramlar 481

11.2 İşlemler 483
 Düzüm Ekle 483
 Düzüm Sil 484
 Kenar Ekle 484
 Kenar Sil 485
 Düzüm Bul 485
 Grafiği Gezin 485
Derinlik Öncelikli Gezinme 485
Genişlik Öncelikli Gezinme 487

11.3 Grafik Depolama Yapıları 488
 Komşuluk Matrisi 488
 Komşuluk Listesi 489

11.4 Grafik Algoritmaları 490
 Grafik Oluştur 491
 Düzüm Ekle 492
 Düzüm Sil 493

Arc Ekle 493
 Arc Sil 495
 Düzüm Al 496
 Derinlik Öncelikli Geçiş 496
 Genişlik Öncelikli Geçiş 498
 Grafi Yok Et 499

11.5 Graf ADT 500
 Veri Yapısı 501
Baş Yapısı 501
Düzüm Yapısı 501
Arc Yapısı 501
 Algoritmalar 502
Graf Düzüm Ekle 503
Graf Düzüm Sil 505
Graf Arc Ekle 506
Graf Arc Sil 508
Graf Derinlik Öncelikli Geçiş 509
Graf Genişlik Öncelikli Geçiş 511

11.6 Ağlar 513
 Minimum Yayılma Ağaçları 514
Minimum Yayılma Ağaçları
Örneği 515
Minimum Yayılma Ağaçları Veri
Yapısı 515
Minimum Yayılma Ağaçları
Sözde kod 517
 En Kısa Yol Algoritması 518
En Kısa Yol Kılavuzu Örneği 519
En Kısa Yol Sözde Kodu 520

11.7 Anahtar Terimler 523

11.8 Özet 523

11.9 Uygulama Setleri 524
 Alıştırmalar 524
 Problemler 526
 Projeler 527

Bölüm IV Sıralama ve Arama 531

Bölüm 12 Sıralama 533

12.1 Sıralama Kavramları 533
 Sıralama Sırası 534
 Sıralama Kararlılığı 535
 Sıralama Verimliliği 535
 Geçişler 535
 Sıralamalar ve ADT'ler 536

12.2 Seçim Sıralamaları 537**Düz Seçim Sıralaması 537***Seçim Sıralama Algoritması 537***Yığın Sıralama 539***Yığın Sıralama Algoritması 541***Seçim Sıralama Verimliliği 541***Doğrudan Seçim Sıralama 541**Yığın Sıralama 542***Özet 542****Seçim Sıralama Uygulaması 543***Seçim Sıralama C Kodu 543**Yığın Sıralama C Kodu 544***12.3 Ekleme Sıralamaları 547****Doğrudan Ekleme Sıralama 547***Doğrudan Ekleme Sıralama Örneği 547**Ekleme Sıralama Algoritması 547***Shell Sıralama 549***Shell Sıralama Algoritması 550***Artış Boyutunu Seçme 553****Eklemeli Sıralama Verimliliği 554***Düz Eklemeli Sıralama 554**Shell Sıralama 554***Özet 555****Eklemeli Sıralama Uygulaması 556***Düz Eklemeli Sıralama 556**Shell Sıralama 557***12.4 Değişim Sıralamaları 558****Kabarcık Sıralama 558***Kabarcık Sıralama Algoritması 559***Hızlı Sıralama 560***Hızlı Sıralama Algoritması 562***Değişim Sıralama Verimliliği 567***Balon Sıralama 567**Hızlı Sıralama 567***Özet 568****Değişim Sıralama Uygulaması 569***Balon Sıralama Kodu 569**Hızlı Sıralama Kodu 570***12.5 Dış Sıralamalar 573****Sıralı Dosyaların Birleştirilmesi 573****Sırasız Dosyaların Birleştirilmesi 575****Sıralama Süreci 576***Doğal Birleştirme 577**Dengeli Birleştirme 577**Polifaz Birleştirme 580**Sıralama Aşamaları Yeniden Gözden Geçirildi 580***12.6 Hızlı Sıralama Verimliliği 583***En Kötü Durum 584**En İyi Durum 584**Ortalama Durum 586***12.7 Anahtar Terimler 588****12.8 Özet 588****12.9 Uygulama Setleri 590***Alıştırmalar 590**Problemler 592**Projeler 593***Bölüm 13 Arama 597****13.1 Liste Aramaları 597****Sıralı Arama 597***Sıralı Arama Algoritması 599***Sıralı Aramalardaki Var****yasyonlar 600***Sentinel Araması 600**Olasılık Araması 601**Sıralı Liste Araması 602***İkili Arama 603***Hedef Bulundu 603**Hedef Bulunamadı 605**İkili Arama Algoritması 606**Arama Algoritmalarını Analiz Etme 607**Sıralı Arama 607**İkili Arama 608***13.2 Arama Uygulamaları 609****Sıralı Arama C'de 609****İkili Arama C'de 610****13.3 Hashlenmiş Liste Aramaları 611****Temel Kavramlar 611****Hashleme Yöntemleri 613***Doğrudan Yöntem 613**Çıkarma Yöntemi 615**Modül-bölme Yöntemi 615**Basamak-çıkarma Yöntemi 616**Orta-kare Yöntemi 617**Katlama Yöntemleri 617**Dönme Yöntemi 618**Sözde-rastgele Hashleme 619**Bir Hashleme Algoritması 619***13.4 Çarpışma Çözümü 620****Açık Adresleme 623***Doğrusal Proba 624**Kare Proba 625*

*Sözde-rastgele Çarpışma
Çözümü 626*

Anahtar Ofseti 627

Bağlı Liste Çarpışma Çözümü 628

Kova Hashleme 629

Birleşim Yaklaşımları 630

Hashlenmiş Liste Örneği 630

13.5 Anahtar Terimler 636

13.6 Özet 636

13.7 Pratik Setleri 638

Alıştırmalar 638

Problemler 639

Projeler 640

Ek A ASCII Tabloları 643

A.1 ASCII Kodları (Uzun Form) 643

A.2 ASCII Tablosu (Kısa Form) 648

Ek B Yapı Şemaları 649

B.1 Yapı Şeması Sembolleri 650

Modüller 650

Yapı Şemalarını Okuma 651

Ortak Modüller 652

Koşullu Çağrılar 652

Özel Veya 652

Döngüler 653

Koşullu Döngüler 653

Özyineleme 653

Veri Akışları ve Bayraklar 654

B.2 Yapı Şeması Kuralları 655

Ek C Tam Sayı ve Kesirli Sayı Kütüphaneleri 657

C.1limits.h657

C.2float.h658

Ek D Seçilen C Kütüphaneleri 661

D.1 Fonksiyon Dizini 661D.2

Tür Kütüphanesi 664D.3

Matematik Kütüphanesi 6

64D.4 Standart G/Ç Kütüphanesi 6

68Genel G/Ç 668

Biçimlendirilmiş G/Ç 669

Karakter G/Ç 669

Dosya G/Ç 670

Dize G/Ç 670

Sistem Dosya Kontrolü 670

D.5 Standart Kütüphane 671

Matematik Fonksiyonl

arı 671Bellek Fonksiyonla

rı 671Program Kontrol

ü 671Sistem İletişimi 672Dönü

şüm Fonksiyonları 672D.6 Di

ze Kütüphanesi 672Bellek

Fonksiyonları 672Dize Fo

nksiyonları 673D.7 Za

man Kütüphanesi 673

Ek E Matematiksel Seriler ve Özyinelemeli

İlişkiler 675

E.1 Aritmetik Seriler 675

E.2 Geometrik Seriler 677

E.3 Harmonik Seriler 678

E.4 Özyinelemeli İlişkiler 679

Ek F Yığınlar ve Kuyrukların Dizi Uyg

ulamaları 681

F. 1 Yığın ADT 681

Dizi Veri Yapısı 682

Yığın Oluştur 683

Yığına Ekle 684

Yığından Çık

r 684Yığın Üst

ü 685Boş Yığın 6

86Dolu Yığın 6

86Yığın Sayısı 68

6Yığını Yok Et 687

F. 2Kuyruk ADT 688

Dizi Kuyrukları Uygulaması 689

Kuyruk Oluştur 691

Kuyruğa Ekle 6

92Kuyruktan Ç

ıkar 693Kuyruk Ön

ü 693Kuyruk Arka

sı 694Dolu Kuyru

k 694Boş Kuyruk 69

4Kuyruk Sayısı 695K

uyruğu Yok Et 695

Sözlük 697

Dizin 709

Önsöz

Veri yapılarının incelenmesi hem heyecan verici hem de zordur. Heyecan verici çünkü daha büyük ve daha karmaşık problemleri çözmeyi mümkün kılan geniş bir programlama tekniği yelpazesi sunar. Zordur çünkü veri yapılarının karmaşık doğası, programların tasarımına yaklaşımımızı zı değiştiren birçok kavramı beraberinde getirir.

Veri yapıları çalışması, bol miktarda materyali kapsadığı için, hepsini bir dönem içinde kapsamanın mümkün olmadığını görebilirsiniz. Aslında, veri yapıları o kadar yaygın bir konudur ki, alt düzey, üst düzey ve lisansüstü programlarda onunla ilgili yönler bulacaksınız.

Bu Kitabın Özellikleri

Bu metindeki ana odak noktamız, veri yapılarını bir alt düzey dersinde öğretilen bir giriş konusu olarak sunmaktır. Bu odakla, materyali basit, doğrudan bir şekilde birçok örnek ve şekil ile sunuyoruz. Ayrıca, veri yapılarının matematiksel yönünü önemsizleştiriyoruz ve algoritmaların resmi matematiksel kanıtlarını daha sonraki dersler için bırakıyoruz.

Sözde kod

Sözde kod, bir problemi çözmek için gereken adımların İngilizce benzeri bir sunumudur. Öğrencilerin problem gereksinimlerine odaklanırken detayları gizleyen bir seviyede bir problemi çözmelerine olanak tanıyan rahat söz dizimi ile yazılmıştır. Diğer bir deyişle, öğrencilerin büyük resme odaklanmalarını sağlar.

Mükemmel bir tasarım aracı olmanın yanı sıra, sözde kod da dil bağımsızdır. Sonuç olarak, aynı sözde kod tasarımını farklı dillerde bir algoritmayı uygulamak için kullanabiliriz ve kullanıyoruz. Bu metinde bulacağınız sözde kod söz dizimi yıllar içinde evrim geçirmiştir. Sözde kod evrimimiz sırasında, öğrencilerimiz aynı temel sözde kod algoritmalarını Pascal, C, C++ ve Java'da uygulamışlardır. Bu metinde, tüm kod uygulamalarımız için C kullanıyoruz.

Soyut Veri Türleri

Bu metnin ikinci önemli özelliği, soyut veri türlerinin (ADT'ler) kullanımınıdır. Her veri yapısı bir ADT olarak uygulanmamalıdır. Ancak, uygun olduğunda

öğrencinin çalışması ve kullanımı için C uygulamaları geliştiriyoruz. Özellikle, öğrenciler Yığınlar (Bölüm 3), Kuyruklar (Bölüm 4), Genel Listeler (Bölüm 5), İkili Arama Ağaçları (Bölüm 7), AVL Ağaçları (Bölüm 8), Yığınlar (Bölüm 9), B-Ağaçları (Bölüm 10) ve Grafikler (Bölüm 11) için ADT uygulamaları bulacaklardır.

Görsel Yaklaşım

Çeşitli veri yapılarını tartışırken, önce genel ilkeleri figürler ve diyagramlar kullanarak sunuyoruz, böylece öğrencinin kavramı görselleştirmesine yardımcı oluyoruz. Eşgeri yapıları, birkaç algoritma gerektirecek kadar büyük ve karmaşık ise, dizayn çözümünü sunmak için bir yapı şeması kullanıyoruz. Tasarım veya yapı tamamen anlaşıldıktan sonra, uygun olduğunda, bir sözde kod algoritması sunuyoruz ve ardından C uygulamasını veriyoruz.

Kitabın kısa bir taraması, görsel yaklaşımımızı göstermektedir. 300'den fazla figür, 30 tablo, 120 algoritma, 170 program ve sayısız kod parçası bulunmaktadır. Bu kadar görsel detay, büyük bir kitap oluşturma eğiliminde olsa da, bu materyallerin öğrencilerin kavramları anlamasını ve takip etmesini çok daha kolay hale getirdiği görülmektedir.

Uygulama Setleri

Bölüm sonu materyalleri, öğrencinin öğrendiklerini pekiştirir. Bölümdeki önemli konular madde işaretli listelerle özetlenmiştir. Özeti takiben üç uygulama seti bulunmaktadır.

Alıştırmalar

Bölümdeki materyali kapsayan sorular.

Problemler

Öğrenciden bir sözde kod algoritması geliştirmesini veya bir bilgisayarda çalıştırılacak kısa bir program yazmasını isteyen kısa ödevler. Bu problemler genellikle 2 ila 3 saat içinde geliştirilebilir.

Projeler

Ortalama bir öğrencinin geliştirmesi için 6 ila 9 saat veya daha fazla sürebilecek daha uzun, büyük ödevler.

Sözlük

Kapsamlı bir sözlük sunuyoruz. Metinde kullandığımız tüm anahtar kelimelerin ve diğer teknik terimlerin tanımlarını içerir.

Öğretim Araçları

Aşağıdaki ek materyaller, Kurs Teknolojisi Web sitesinden indirilebilir (<http://www.course.com>). Bilgisayar Bilimleri'ne ve ardından CS2 Veri Yapıları — C'ye gidin.

Eğitmen Materyalleri

Aşağıdaki eğitim materyalleri mevcuttur:

- Alıştırmalar ve problemler için tam çözümler.
- Sınıf için hazır Microsoft PowerPoint sunumları, her bölüm için, şunları içeren:
 - Hedefler
 - Şekiller
 - Tablolar
- Programların çalıştırılması için gereken kaynak kodu ve herhangi bir giriş dosyası, problem çözümleri olarak geliştirilen programlar dahil.

Öğrenci Materyalleri

Aşağıdaki öğrenci materyalleri mevcuttur:

- Tüm tek numaralı alıştırmaların çözümleri.
- Bölümlerdeki programları çalıştırmak için gereken kaynak kodu ve herhangi bir girdi dosyası.

Bu Baskıda Neler Yeni

İlk baskıda bulunan temel veri yapıları bu baskıya taşınmıştır. Ancak, metnin organizasyonunu ve birkaç önemli kavramın sunumunu önemli ölçüde değiştirdiğimizi göreceksiniz. Soyut Veri Türlerinin kullanımı da genişletilmiştir. En önemli değişiklikler aşağıda sıralanmıştır.

1. Kitap dört Bölüme ayrılmıştır:**I. Giriş**

Bu bölüm temel kavramları ve özyinelemeyi kapsamaktadır.

II. Doğrusal Listeler

Bu bölüm yığınlar, kuyruklar ve genel doğrusal listeleri kapsar.

III. Doğrusal Olmayan Listeler

Bu bölüm ağaçlara, ikili arama ağaçlarına, AVL arama ağaçlarına, yığınlara, çok yolları ağaçlara ve grafikleri tanıtmaktadır.

IV. Sıralama ve Arama

Bu bölüm sıralama ve aramayı kapsar.

2. Öğrencilerin ADT'leri genel kod kullanarak anlamalarına ve yazmalarına yardımcı olmak için, Bölüm 1'e iki bölüm ekledik: Bölüm 1.4, "ADT Uygulamaları" ve Bölüm 1.5, "ADT'ler için Genel Kod." Bölüm 1.4, dizileri ve bağlı listeleri ADT veri yapısı uygulamaları olarak tartışmaktadır. Bölüm 1.5, ADT'lerin iki ana aracını, void işaretçisi ve fonksiyon işaretçisini tartışmaktadır.
3. ADT kavramı İkili Arama Ağaçları (Bölüm 7) ve Yığınlar (Bölüm 9) için genişletilmiştir.

4. Sözde kodun seviyesi, daha basit ve daha kavramsal hale getirilmiştir, yani daha az kod odaklıdır. Ayrıca, *end if* ve *end loop* gibi son yapı ifadeleri eklenmiştir.
5. Tüm C programları C-99 uyumlu hale getirilmek üzere gözden geçirilmiştir.
6. Algoritmaların verimliliği, Bölüm 12'de hızlı sıralamanın verimliliği üzerine matematiksel bir tartışmayı içerecek şekilde genişletilmiştir.
7. Ek C, "Tam Sayı ve Float Kütüphaneleri," ve Ek D, "Seçilen C Kütüphaneleri," C-99 standardını yansıtacak şekilde gözden geçirilmiştir.
8. Ek E, "Matematiksel Seriler ve Özyinelemeli İlişkiler," yinelemeli ve özyinelemeli algoritmalar için matematiksel bir arka plan sağlamak üzere eklenmiştir.
9. Yığınlar ve kuyrukların dizi uygulamaları bölümlerden kaldırılmış ve Ek F, "Yığınlar ve Kuyrukların Dizi Uygulamaları"na yerleştirilmiştir.
10. Kurs Teknolojisi web sitesinde bulunan Yardımcı Materyaller, kullanımını kolaylaştırmak için gözden geçirilmiştir.

Teşekkürler

Kurs Teknolojisi Personeli

Kurs Teknolojisi'ndeki editörlerimize ve personele, özellikle Kıdemli Edinim Editörü Amy Yarnevich, Ürün Yöneticisi Alyssa Pratt ve Bobbijo Frasca, Üretim Editörü'ne teşekkür ederiz.

Değerlendiriciler

Özellikle değerlendiricilerimizin katkılarını takdir etmek istiyoruz:

Birinci Baskı

Aşağıdaki profesörler birinci baskının yayımlanmasına katkıda bulunmuştur.

Naguib Attia, *Johnson C Smith Üniversitesi*
 Kevin Croteau, *Francis Marion Üniversitesi*
 Barbara Smith, *Conchise Koleji*
 William E. Toll, *Taylor Üniversitesi*

İkinci Baskı

Aşağıdaki profesörler ikinci baskının yayımlanmasına katkıda bulunmuştur.

Jerry Adams, *Lee Üniversitesi*
 Charles W. Bane, *Tarleton Eyalet Üniversitesi*
 Alfred D. Benoit, *Johnson & Wales Üniversitesi*
 Gerald Burgess, *Wilmington Koleji*

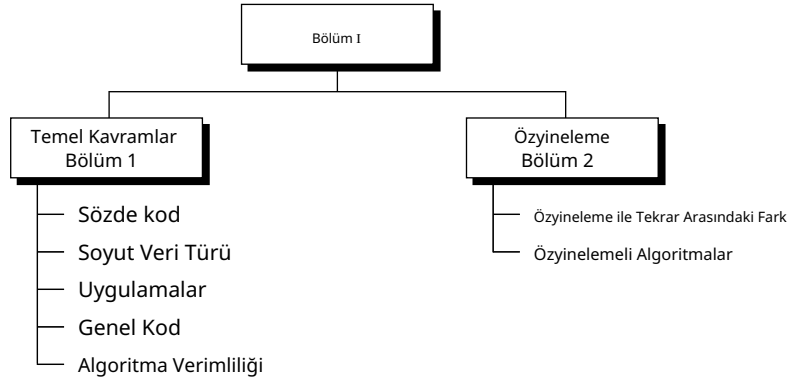
Roman Erenshateyn, *Goldey-Beacom Kolej*
Kamal Fernando, *Wilberforce Üniversitesi*
Barbara Guillott, *Louisiana Eyalet Üniversitesi*
Stefen Howard, *Mars Hill Kolej*
Keenan D. Jackson, *Wichita Eyalet Üniversitesi*
Reza Kamali, *Pennsylvania Teknoloji Kolej*
Herv Podnar, *Southern Connecticut Eyalet Üniversitesi*
Enrico Pontelli, *New Mexico Eyalet Üniversitesi*
Iren Valova, *Massachusetts Dartmouth Üniversitesi*
Colin Ware, *New Hampshire Üniversitesi*
Jesse Yu, *St. Elizabeth Kolej*

Richard F. Gilberg
Behrouz A. Forouzan

Bölüm I

Giriş

Bu metni anlamak için gerekli olan birkaç kavram vardır. Bu kavramları ilk iki bölümde tartışıyoruz. Bölüm 1 “Temel Kavramlar,” kitap boyunca kullandığımız genel materyalleri kapsar. Bölüm 2 “Özyineleme,” özyineleme kavramını tartışmaktadır. Şekil I-1 Bölüm I’in organizasyonunu göstermektedir.



ŞEKİL I-1 Bölüm I İçeriği

Kapsanan Bölümler

Bu bölüm iki bölüm içermektedir.

Bölüm 1: Temel Kavramlar

İlk bölüm, metin boyunca kullandığımız kavramları kapsamaktadır. Bazılarını programlama dersinizden aldığınız materyalin bir gözden geçirmesi olarak bulabilirsiniz. Aşağıda ana kavramlar özetlenmiştir.

Sözde kod

Tüm metinlerimizde, “Tasarım, koddan önce gelir.” temel ilkesini kullanıyoruz. Bu nedenle, metin boyunca algoritma tasarımı, belirli bir dilde onu uygulayan koddan ayırıyoruz. Bu kitapta temel dil C olmasın da rağmen, sözde kod algoritmayı uygulamadan ayırmamıza olanak tanır.

Soyut Veri Türü

Soyut bir veri türü (ADT), bir dizi algoritmayı genel olarak uygular, böylece bunları herhangi bir veri türüne veya yapısına uygulanabilir. Bir ADT uygulamasının güzelliği, algoritmaların basit bir tamsayı veya karmaşık bir kayıt olsun, herhangi bir veri türünü işleyebilmesidir.

ADT Uygulamaları

Genel olarak, bir soyut veri türünü uygulamak için kullanılacak iki temel veri yapısı vardır: diziler ve bağlı listeler. Temel bağlı liste kavramlarını Bölüm 1’de tartışıyoruz ve gerekli oldukça sonraki bölümlerde bunları genişletiyoruz.

ADT'ler için Genel Kod

ADT kavramını uygulamak için genel kod kullanmamız gerekiyor. Her dil genel kodu uygulamak için farklı bir araç seti sağlar. C dili iki araç kullanır: void işaretçisi ve fonksiyon işaretçisi.

Algoritma Verimliliği

Birçok yazar, günümüz bilgisayarlarının ve derleyicilerinin algoritma verimliliğinin akademik bir tartışma haline getirdiğini savunsa da, algoritma verimliliğini anlama ve güvenliğin daha iyi algoritmalar yazmak için bir çerçeve sağladığına inanıyoruz. Belirli algoritmaların verimliliğini geliştirdiğimizde tartışsak da, bu bölümde algoritma verimliliğini tartışmak için temel kavramlar ve araçlar üzerinde duruyoruz.

Bölüm 2: Özyineleme

Bölüm 2'de, genellikle bir giriş programlama kursunda atlanan bir kavram olan özyinelemeyi tartışıyoruz. Veri yapıları hakkında konuşabilmek için özyinelemeyi anlamamız gerekiyor çünkü birçok soyut veri türü doğası gereği özyinelemelidir ve bunları işleyen algoritmalar özyineleme kullanılarak daha iyi anlaşılabilir. Özellikle Bölüm III, "Doğrusal Olmayan Listeler"de özyinelemeli algoritmaları yaygın olarak kullanıyoruz.

Özyineleme ve Tekrar

Bölümün ilk kısmı, özyineleme ve tekrar arasındaki farkları karşılaştırmakta ve her birinin ne zaman kullanıldığını açıklamaktadır.

Özyinelemeli Algoritmalar

Özyinelemeli algoritmalar genellikle şık olsa da, anlaması zor olabilir. Bölümün ikinci kısmında, özyineleme kavramını netleştirmek ve özyinelemeli algoritmalar oluşturmak için bir tasarım anlayışı sağlamak amacıyla birkaç algoritma tanıtıyoruz.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 1

Temel Kavramlar

Bu metin, öğrencinin yapılandırılmış programlama ilkeleri konusunda sağlam bir temel ele sahip olduğunu ve orta düzeyde karmaşıklıkta programlar yazdığını varsayıyor. Metin, tüm uygulama örnekleri için C dilini kullanmasına rağmen, veri yapısı algoritmalarının tasarımı ve mantığı sözde kod üzerine kuruludur. Bu yaklaşım, algoritmalar için dil bağımsız bir ortam yaratır.

Bu bölümde, metnin geri kalanında kullanılan araçlar için bir arka plan oluşturuyoruz; özellikle sözde kod, soyut veri türü, algoritma verimliliği analizi ve genel kod oluşturmak için gerekli kavramlar.

1.1 Sözde Kod

Algoritmaları tanımlamak için birkaç araç kullanılsa da, en yaygın olanlardan biri sözde koddur. Sözde kod, algoritma mantığının İngilizce benzeri bir temsidir. Kısmen İngilizce, kısmen yapılandırılmış koddur. İngilizce kısmı, hata mesajları gibi gereksiz detayları göstermeden, ne yapılması gerektiğini tanımlayan rahat bir sözdizimi sağlar. Kod kısmı, temel algoritmik yapılar olan sıra, seçim ve yineleme için genişletilmiş bir versiyondan oluşur.

Algoritmaları tanımlamak için en yaygın araçlardan biri, kısmen İngilizce, kısmen yapılandırılmış kod olan sözde koddur.

Bu metinde, hem veri yapıları hem de kodu temsil etmek için sözde kod kullanıyoruz. Veri öğelerinin bildirilmesi gerekmez. Bir algoritmada bir veri adını ilk kez kullandığımızda, otomatik olarak bildirilir. Türü bağlama göre belirlenir. Aşağıdaki ifade, `count` adında bir sayısal veri öğesini bildirir ve değerini sıfıra ayarlar.

```
count'u 0 olarak ayarla
```

Öte yandan, verinin yapısı bildirilmelidir. Bir yapı adıyla başlayan ve *end* anahtar kelimesi ve yapının adıyla sona eren basit bir sözdizim sel ifade kullanıyoruz. Yapı içinde, veri öğelerini aşağıda gösterildiği gibi girintileyerek yapısal öğeleri listeliyoruz.

```
düğüm
    veri b
    ağılant
    son düğüm
```

Bu veri tanımı, iç içe bir yapı (*data*) ve bir sonraki düğüme (*link*) işaretçi içeren kendine referans veren bir listedeki bir düğümü tanımlar. Bir öğenin türü, adı ve algoritmadaki kullanımıyla ima edilir.

Daha önce belirtildiği gibi, sözde kod bir algoritmayı tanımlamak için kullanılır. Algoritma ifadeleri hakkında bir tartışmayı kolaylaştırmak için, bunları Algoritma 1-1'de gösterilen hiyerarşik sistem kullanarak numaralandırıyoruz. Aşağıdaki bölümler, bir algoritmanın bileşenlerini tanımlar. Renkli yorumlar, gerektiğinde belgeler veya açıklamalar sağlar.

ALGORİTMA 1-1 Sözde kod örneği

```
Algoritma örnek (sayfaNumarası)
Bu algoritma bir dosyayı okur ve bir rapor yazdırır.
  Ön sayfaNumarası referansla geçildi
  Son Rapor Yazdırıldı
    sayfaNumarası rapordaki sayfa sayısını içerir
  Dönüş Yazdırılan satır sayısı
1 döngü (dosya sonu değilse)
  1 dosyayı oku
  2 eğer (tam sayfa)
    1 sayfa numarasını artır
    2 sayfa başlığını yaz
  3 eğer son
  4 rapor satırını yaz
  5 satır sayısını artır
2 döngüyü sonlandır
3 satır sayısını döndür
son örnek
```

Algoritma başlığı

Her algoritma, adını veren, parametrelerini listeleyen ve eğer ön koşulları ve son koşulları tanımlayan bir başlıkla başlar. Bu bilgi, algoritmayı belgelemek için önemlidir. Bu nedenle, başlık bilgisi, programcının algoritmayı yazmak için bilmesi gereken her şeyi iletecek kadar eksiksiz olmalıdır. Algoritma 1-1'de yalnızca bir parametre vardır, sayfa numarası.

Amaç, Koşullar ve Döndürme

Amacın, algoritmanın ne yaptığını açıklayan kısa bir ifade olmasıdır. Sadece genel algoritma işlemini tanımlamalıdır. Tüm işlemleri tanımlamaya çalışmamalıdır. Örneğin, Algoritma 1-1'de amacın dosyanın açılacağını veya raporun nasıl yazdırılacağını belirtmesine gerek yoktur. Benzer şekilde, arama örneğinde amacın hangi olası dizi aramalarının kullanılacağını belirtmesine gerek yoktur.

Ön koşul, parametreler için herhangi bir ön gereksinimi listeler. Örneğin, Algoritma 1-1'de sample çağırın algoritma sayfa numarasını referans olarak geçmelidir. Bazen ön koşul yoktur, bu durumda yine de hiçbir şeyi gerektirmediğini belirten bir ifade ile ön koşulu listeleriz, aşağıda gösterildiği gibi.

Ön hiçbir şey

Eğer birden fazla girdi parametresi varsa, her biri için ön koşul gösterilmelidir. Örneğin, basit bir dizi arama algoritmasının aşağıdaki başlığı vardır:

```
Algoritma arama (liste, argüman, konum)
Belirli bir öğeyi aramak için diziyi tarar ve indeks konumunu döndürür.
Ön liste, aranacak veri dizisini içerir
    argüman, listede bulunacak veriyi içerir
Son konum, eşleşen indeksi içerir
    -veya- bulunmadıysa belirsizdir
Bulunırsa true, bulunmadıysa false döndür
```

search içinde ön koşul, iki girdi parametresi olan liste ve argümanı başlatılması gerektiğini belirtir. Eğer ikili arama kullanılıyorsa, ön koşul ayrıca dizi verisinin sıralı olması gerektiğini de belirtir.

Son koşul, alınan herhangi bir eylemi ve herhangi bir çıktı parametresinin durumunu tanımlar. Algoritma 1-1'de son koşul iki bölüm içerir. İlk olarak, raporun yazdırıldığını belirtir. İkinci olarak, referans parametresi, sayfaNumarası, rapordaki güncellenmiş sayfa sayısını içerir. Yukarıda gösterilen arama algoritmasında yalnızca bir post koşulu vardır, bu da iki farklı değerden biri olabilir.

Eğer bir değer döndürülürse, bu döndürme koşulu ile tanımlanır. Genellikle yoktur ve döndürme koşuluna ihtiyaç yoktur. Algoritma 1-1'de yazdırılan satır sayısını döndürüyoruz. Arama algoritması, argüman bulunduysa true, bulunmadıysa false döndürür.

İfade Numaraları

İfadeler, her ifadede yalnızca sayı dizisinin sonuncusu gösterilen kısaltılmış ondalık notasyon kullanılarak numaralandırılır. Dosyayı okuyan Algoritma 1-1'deki ifadenin genişletilmiş numarası 1.1'dir.

Sayfa başlığını yazan ifade 1.2.2'dir. Bu teknik, bireysel bir ifadeyi tanımla mamıza olanak tanırken, okunması kolay ifadeler sağlamamıza da yardımcı olur.

Değişkenler

Anlamın anlaşılmasını sağlamak için, akıllı veri adları kullanıyoruz—yani, atanın anlamını tanımlayan adlar. Ancak, bir algoritmada kullanılan değişkenleri tanımlamak gerekli değildir, özellikle adındatanın bağlamını gösterdiği durumlarda.

Bir algoritma veya değişken için ad seçimi, algoritmanın ve kodlanmış uygulamasının daha okunabilir olmasına büyük katkı sağlar. Genel olarak, bu kurallara uymalısınız:

1. Tek karakterli adlar kullanmayın.
2. Uygulama programlarında genel adlar kullanmayın. Genel adlara örnekler `count`, `sum`, `total`, `row`, `column` ve `file`'dir. Herhangi bir boyuttaki bir programda birkaç adet `count`, `sum` ve `total` vardır. Bunun yerine, okuyucunun adın hangi veri parçasına atıfta bulunduğunu tam olarak bilmesi için genel adı akıllı bir nitelik ile genişletin. Örneğin, `studentCount` ve `numberOfStudents`, `count`'dan daha iyidir.
3. Kısaltmalar akıllı veri adları olarak hariç tutulmaz. Örneğin, `stuCnt`, `student count` için iyi bir kısaltmadır ve `numOfStu`, `number of students` için iyi bir kısaltmadır. Ancak, `noStu`, `number of students` için iyi bir kısaltma değildir çünkü çok kolay bir şekilde *no students* olarak okunabilir.

İfade Yapıları

Yapılandırılmış programlama modelini ilk önerdiğinde, Edsger Dijkstra herhangi bir algoritmanın yalnızca üç programlama yapısı: sıra, seçim ve döngü kullanılarak yazılabileceğini belirtti. Bizim sözde kodumuz yalnızca bu üç temel yapıyı içermektedir. Bu yapıların uygulanması, uygulama dilinin zenginliğine dayanır. Örneğin, döngü C dilinde bir *while*, *do...while*, veya *for* ifadesi olarak uygulanabilir.

Sıra

Bir *sıra*, bir algoritma içindeki yürütme yolunu değiştirmeyen bir veya daha fazla ifadedir. Atama ve toplama gibi ifadelerin sıra ifadeleri olduğu açık olsa da, diğer algoritmalara yapılan çağrılar da bir sıra ifadesi olarak kabul edildiği kadar açık değildir. Çağrılar ardışık ifadeler olarak kabul edilmesinin nedeni, her algoritmanın yalnızca bir girişi ve bir çıkışı olduğu yapılandırılmış programlama kavramında yatmaktadır. Ayrıca, bir algoritma tamamlandığında, onu çağıran ifadenin hemen sonrasına geri döner. Bu nedenle, bir algoritma çağırısını bir sıra ifadesi olarak değerlendirebiliriz. Algoritma 1-1'de ifadeler 1.2.1 ve 1.2.2 sıra ifadeleridir.

Seçim

Bir seçim ifadesi bir koşulu değerlendirir ve sıfır veya daha fazla alternatif gerçekleştirir. Değerlendirmenin sonuçları hangi alternatiflerin alındığını belirler.

Tipik seçim ifadesi, bir ifadesinde uygulandığı gibi iki yönlü seçimidir. Çoğu dil çok yönlü seçimler sağlarken, C'deki switch gibi, biz sözde kodda hiçbirini sağlamıyoruz. Seçimin parçaları, aşağıdaki kısa sözde kod ifadesinde gösterildiği gibi girinti ile tanımlanır.

```

1   eğer (koşul)
    1   eylem1
2   aksi takdirde
    1   eylem2
3   eğer son
  
```

Algoritma 1-1'deki İfade 1.2, bir seçim ifadesinin bir örneğidir. Seçimin sonu, İfade 1.3'teki end if ile belirtilir.

Döngü

Bir döngü ifadesi bir kod bloğunu yinelemektedir. Sözde kodumuzda kullandığımız döngü, while döngüsüne çok benzer. Bu bir ön test döngü südür; yani, koşul döngünün gövdesi çalıştırılmadan önce değerlendirilir. Koşul doğruysa, gövde çalıştırılır. Koşul yanlışsa, döngü sona erer.

Algoritma 1-1'de İfade 1, bir döngü örneğidir. Döngünün sonu, İfade 2'deki end loop ile belirtilir.

Algoritma analizi

Seçilen algoritmalar için, bazı belirgin noktalarını açıklayan bir analiz bölümü ile algoritmayı takip ediyoruz. Her kod satırı açıklanmamıştır. Aksine, analiz yalnızca vurgulanması gereken veya bazı açıklamalar gerektiren noktaları inceler. Algoritma analizi genellikle stil veya verimlilik dikkate alımlarını da tanıtır.

Sözde kod örneği

Bir başka sözde kod örneği olarak, bir ortalamadan sapmayı hesaplama için gereken mantığı düşünün. Bu problemde önce bir dizi sayıyı okumalı ve ortalamasını hesaplamalıyız. Ardından, her sayıdan ortalamayı çıkarır ve sayıyı ve sapmasını yazdırırız. Hesaplamanın sonunda, toplamı ve ortalamayı da yazdırırız.

Açık çözüm, verileri okundukça bir dizide yerleştirmektir. Algoritma 1-2, bu basit problem için çağrılabilir bir algorithmada uygulanacak gibi kodu içermektedir.

ALGORİTMA 1-2Seri için Ortalamadan Sapmayı Yazdır

```

Algoritma sapma
  Ön hiçbir şey
  Ortalama ve sapmaları yazdırılmış sayılar
1 döngü (dosya sonu değilse)
  1 sayıyı diziye oku
  2 sayıyı toplamaya ekle
  3 sayımı artır
2 döngüyü sonlandır
3 ortalama toplam / sayım olarak ayarla
4 ortalama yazdır
5 döngü (dizi sonu değil)
  1 devFromAve'yi dizi elemanı - ortalama olarak ayarla
  2 dizi elemanını ve devFromAve'yi yazdır
6 döngüyü sonlandır
son sapma

```

Algoritma 1-2 Analizi Algoritma 1-2'de bahsedilmesi gereken iki nokta var. İlk olarak, hiçbir parametre yoktur. İkincisi, daha önce açıklandığı gibi, değişkenleri tanımlamıyoruz. Bir değişkenin türü ve amacı, adı ve kullanımıyla kolayca belirlenmelidir.

1.2 Soyut Veri Türü

Programlama kavramları tarihine baktığımızda, spaghetti kodu olarak bilinen yapılandırılmamış, lineer programlarla başladık; bu programlarda mantık akışı, bir tabaktaki spaghetti gibi programın içinde dolanıyordu. Ardından, programların her birinin hala lineer kodlama tekniği kullandığı fonksiyonlar halinde organize edildiği modüler programlama kavramı geldi. 1970'lerde, Edsger Dijkstra ve Niklaus Wirth gibi bilgisayar bilimcileri tarafından yapılandırılmış programlamanın temel ilkeleri formüle edildi. Bugün hala geçerlidirler.

Atomik ve Bileşik Veri

Atomik veriler tek bir bilgi parçasından oluşan verilerdir; yani, diğer anlamlı veri parçalarına bölünemezler. Örneğin, 4562 tam sayısı tek bir tam sayı değeri olarak kabul edilebilir. Elbette, bunu basamaklara ayırabiliriz, ancak ayrıştırılan basamakların orijinal tam sayının aynı özelliklerine sahip değildir; bunlar 0 ile 9 arasında değişen dört tek haneli tam sayıdır. Bazı dillerde atomik veriler, sayısal özellikleri nedeniyle skalar veriler olarak bilinir.

Atomik verilerin zıttı bileşik veridir. Bileşik veriler, anlamı olan alt alanlara ayrılabilir. Bir bileşik veri ögesi örneği olarak, telefon numaranızı düşünün. Bir telefon numarası aslında üç farklı parçaya sahiptir. Öncelikle, alan kodu vardır. Sonra, telefon numarası olarak düşündüğümüz şey aslında üç haneli bir değişimden oluşan bir ön ek ve değişim içindeki numara olmak üzere iki farklı veri ögesidir.

dört rakamdan oluşmaktadır. Geçmişte, bu ön ekler Davenport ve Cy press gibi isimlerdi.

Veri Türü

Bir veri türü iki bölümden oluşur: bir veri kümesi ve veriler üzerinde uygulanabilecek işlemler. Böylece, tam sayı türünün belirli bir aralıkta (boş sayılar) değerlerden ve veriler için uygun olan toplama, çıkarma, çarpma, bölme ve diğer işlemlerden oluştuğunu görüyoruz.

Veri Türü
1. Bir değerler kümesi
2. Değerler üzerinde bir dizi işlem

Tablo 1-1 tüm sistemlerde bulunan üç veri türünü göstermektedir.

Tür	Değerler	İşlemler
tam sayı	$-\infty, \dots, -2, -1, 0, 1, 2, \dots, \infty$	$*, +, -, \%, /, ++, --, \dots$
kayan nokta	$-\infty, \dots, 0, 0, \dots, \infty$	$*, +, -, /, \dots$
karakter	$\backslash 0, \dots, 'A', 'B', \dots, 'a', 'b', \dots, \sim$	$<, >, \dots$

TABLO 1-1 Üç Veri Türü

Veri Yapısı

Bir veri yapısı atomik ve bileşik verilerin tanımlı ilişkilerle bir araya getirilmesidir. Bu tanımda *yapı* verileri bir arada tutan bir kural kümesini ifade eder. Başka bir deyişle, verilerin bir kombinasyonunu alıp bunları ilişkilendiren kurallarını tanımlayacak şekilde bir yapıya yerleştirirsek, bir veri yapısı oluşturmuş oluruz. Veri yapıları iç içe geçmiş olabilir. Diğer veri yapılarından oluşan bir veri yapısına sahip olabiliriz. Örneğin, Tablo 1-2'de gösterildiği gibi dizi ve kayıt adlı iki yapıyı tanımlayabiliriz.

Dizi	Kayıt
Elemanlar olarak bilinen veri veya data türlerinin homojen dizisi	Tanımlanmış bir anahtara sahip tek bir yapıda heterojen veri kombinasyonu
Elemanlar arasındaki koordinat ilişkisi	Hiçbir ilişki yok

TABLO 1-2 Veri Yapısı Örnekleri

Çoğu programlama dili birkaç veri yapısını destekler. Ayrıca, modern programlama dilleri, programcıların bir uygulama için yeni veri yapıları oluşturmasına izin verir.

Veri Yapısı

1. Her birinin ya bir veri türü ya da başka bir veri yapısı olduğu bir eleman kombinasyonu
2. Birleşik elemanları içeren bir dizi ilişki veya bağlantı (yapı)

Soyut Veri Türü

Genel olarak konuşursak, programcıların yetenekleri, alet çantalarında ki araçlarla belirlenir. Bu araçlar eğitim ve deneyimle edinilir. Veri yapıları bilgisi, bu araçlardan biridir.

İlk programlamaya başladığımızda, soyut veri türleri yoktu. Bir dosyayı okumak istiyorsak, fiziksel dosya cihazını okumak için kod yazıyorduk. Aynı kodu tekrar tekrar yazdığımızı fark etmek uzun sürmedi. Bu yüzden bugün soyut veri türü (ADT) olarak bilinen şeyi oluşturduk. Bir dosyayı okumak için kod yazdık ve tüm programcıların kullanması için bir kütüphaneye yerleştirdik.

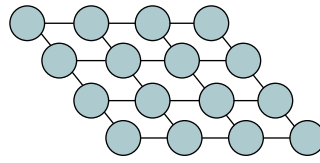
Bu kavram günümüzde modern dillerde bulunmaktadır. Klavyeyi okumak için kullanılan kod bir ADT'dir. Bir veri yapısına, bir karaktere ve o veri yapısını okumak için kullanılabilecek bir dizi işleme sahiptir. ADT'yi kullanarak yalnızca karakterleri okumakla kalmaz, aynı zamanda bunları tam sayılar ve dizeler gibi farklı veri yapıları na da dönüştürebiliriz.

Bir ADT ile kullanıcılar, görevin nasıl yapıldığıyla değil, ne yapabileceğiyle ilgilenir. Diğer bir deyişle, ADT, programcıların işlevleri kullanmasına izin veren ve uygulamayı gizleyen bir tanım setinden oluşur. Belirsiz uygulamalara sahip işlemlerin bu genelleştirilmesine soyutlama denir. Sürecin özünü soyutlarız ve uygulama detaylarını gizli bırakırız.

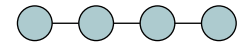
Soyutlama kavramı şunu ifade eder:

1. Bir veri türünün ne yapabileceğini biliyoruz.
2. *Nasıl yapıldığı gizlidir.*

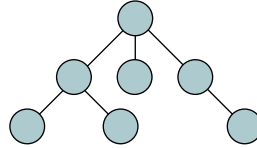
Bir liste kavramını düşünün. En az dört veri yapısı bir listeyi destekleyebilir. Bir matris, bir doğrusal liste, bir ağaç veya bir grafik kullanabiliriz. Listemizi bir ADT'ye yerleştirecek, kullanıcılar kullandığımız yapının farkında olmamalıdır. V eri ekleyip alabiliyorlarsa, veriyi nasıl depoladığımızın bir önemi olmamalıdır. Şekil 1-1, bir listeyi tutmak için kullanılabilecek dört mantıksal yapıyı göstermektedir.



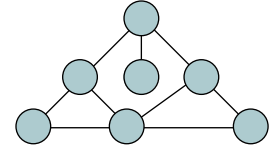
(a) Matris



(b) Doğrusal liste



(c) Ağaç



(d) Grafik

ŞEKİL 1-1 Bazı Veri Yapıları

Başka bir örnek olarak, bir bankanın bekleme sırasını simüle etmesi gereken sistem analistini düşünün; böylece müşterilere verimli bir şekilde hizmet vermek için kaç gişe memuruna ihtiyaç duyulduğunu belirleyebilir. Bu analiz, bir kuyruğun simülasyonu gerektirir. Ancak, kuyruklar genellikle programlama dillerinde mevcut değildir. Bir kuyruk türü mevcut olsa bile, analistimizin simülasyon için bazı temel kuyruk işlemlerine, örneğin kuyruğa ekleme (insertion) ve kuyruktan çıkarma (deleting) ihtiyaç duyar.

Bu soruna iki potansiyel çözüm vardır: (1) analistimizin ihtiyaç duyduğu kuyruğu simüle eden bir program yazabiliriz (bu durumda, çözümümüz yalnızca mevcut uygulama için iyidir) veya (2) herhangi bir kuyruk sorununu çözmek için kullanılacak bir kuyruk ADT'si yazabiliriz. İkincisini seçersek, analistimiz hala bankacılık uygulamasını simüle eden bir program yazmak zorundaadır, ancak bunu yapmak çok daha kolay ve hızlıdır çünkü uygulamaya odaklanabilir, kuyruk yerine.

Artık ADT'yi tanımlamaya hazırız. Soyut veri türü, veri türü için anlamlı olan işlemlerle birlikte paketlenmiş bir veri beyanıdır. Diğer bir deyişle, veriyi ve veriler üzerindeki işlemleri kapsülleriz ve ardından bunları kullanıcıdan gizleriz.

Soyut Veri Türü

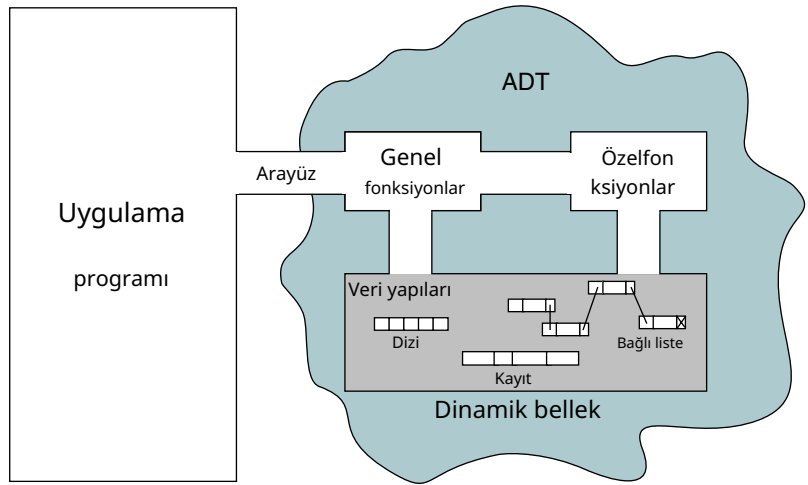
1. Veri beyanı
2. İşlemlerin beyanı
3. Veri ve işlemlerin kapsüllemesi

Uygulamanın gizlenmesinin önemini abartamayız. Kullanıcının ADT'yi kullanmak için veri yapısını bilmesi gerekmemelidir. Kuyruk örneğimize atıfta bulunursak, uygulama programının veri yapısı hakkında hiçbir bilgisi olmamalıdır. Kuyruktaki verilere yapılan tüm referanslar ve manipülasyonlar, tanımlı arayüzler aracılığıyla işlenmelidir.

yapısı. Uygulama programının veriyapısını doğrudan referans almasına izin vermek, birçok uygulamada yaygın bir hatadır. Bu, ADT'nin başka uygulamalara tamamen taşınabilir olmasını engeller.

1.3 Soyut Veri Türü için Model

ADT modeli Şekil 1-2'de gösterilmektedir. Düzensiz bir dış hat ile renklendirilmiş alan ADT'yi temsil eder. ADT'nin içinde modelin iki farklı yönü bulunmaktadır: veri yapıları ve fonksiyonlar (genel ve özel). Her ikisi de tamamen modelin içinde yer almakta ve uygulama programı kapsamının dışındadır. Ancak, veri yapıları ADT'nin ihtiyaç duyulan tüm fonksiyonlarına açıktır ve bir fonksiyon, görevini yerine getirmek için diğer fonksiyonları çağırabilir. Diğer bir deyişle, veri yapıları ve fonksiyonlar birbirinin kapsamındadır.



ŞEKİL 1-2 Soyut Veri Türü Modeli

ADT İşlemleri

Veriler, ADT'nin içine ve dışına kısmen uzanan bir geçit olarak çizilen dış arayüz aracılığıyla girilir, erişilir, değiştirilir ve silinir. Bu arayüz üzerinden yalnızca kamuya açık işlevler erişilebilir. Her ADT işlemi için belirli bir görevi yerine getiren bir algoritma vardır. Uygulamaya yalnızca işlem adı ve parametreleri sunulur ve bunlar ADT'ye erişim sağlar. Sadece bir tek arayüzdür.

ADT Veri Yapısı

Bir liste tamamen program tarafından kontrol edildiğinde, genellikle programlama dersinizde kullanılan basit yapılarla uygulanır. Ancak soyut veri türü, kullanıcıdan uygulamayı gizlemelidir; bu nedenle, yapıya dair tüm veriler ADT içinde korunmalıdır. Yapıyı bir ADT içinde kapsüllemek yeterli değildir. Ayrıca, yapının birden fazla versiyonunun bir arada var olabilmesi de gereklidir. Sonuç olarak, farklı verileri depolayabilmemiz için uygulamayı kullanıcıdan gizlemeliyiz.

Bu metinde, yığınlar, kuyruklar, listeler, ikili arama ağaçları, AVL ağaçları, B-ağaçları, yığınlar ve grafikler için ADT'ler geliştiriyoruz. Önizleme isterseniz, Bölüm 3'teki yığın ADT'sine bakın.

1.4 ADT Uygulamaları

Bir ADT listesini uygulamak için kullanabileceğimiz iki temel yapı vardır: diziler ve bağlı listeler.

Dizi Uygulamaları

Bir dizide, bir listenin ardışıklığı, dizideki elemanların sıralama yapısı (indeksler) ile korunur. Bireysel bir elemanı bulmak için bir diziyi aramak çok verimli olabilirken, eleman ekleme ve silme işlemleri karmaşık ve verimsiz süreçlerdir. Bu nedenle diziler nadiren kullanılır, özellikle liste sık sık değiştiğinde. Ayrıca, doğrusal olmayan listelerin dizi uygulamaları aşırı büyük hale gelebilir, özellikle her eleman için birkaç ardı olduğunda. Ek F, iki ADT için dizi uygulamaları sağlar.

Bağlı Liste Uygulamaları

Bir bağlı liste, her bir ögenin bir sonraki ögenin veya ögelerin konumunu iç erdiği sıralı bir veri koleksiyonudur. Bir bağlı listede, her öge iki kısımdan oluşur: veri ve bir veya daha fazla bağlantı. Veri kısmı, işlenmesi gereken uygulamaya verisini tutar. Bağlantılar verileri bir araya getirmek için kullanılır. Listedeki bir sonraki ögeyi veya ögeleri tanımlayan işaretçiler içerirler.

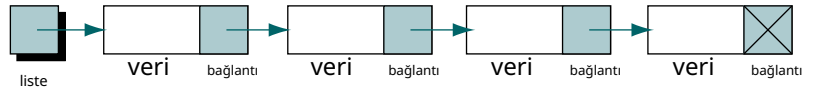
Bir bağlı listeyi doğrusal ve doğrusal olmayan yapılar oluşturmak için kullanabiliriz. Doğrusal bağlı listelerde, her ögenin yalnızca sıfır veya bir ardılı vardır. Doğrusal olmayan bağlı listelerde, her ögenin sıfır, bir veya daha fazla ardılı olabilir.

Bağlı listenin diziye göre en büyük avantajı, verilerin kolayca eklenip silinmesidir. Yeni bir öge eklemek veya bir ögeyi silmek için bağlı listenin ögelerini kaydırmak gerekli değildir. Öte yandan, ögeler artık fiziksel olarak sıralı olmadığından, sıralı aramalarla sınırlıyız: ¹ ikili arama kullanamayız.²

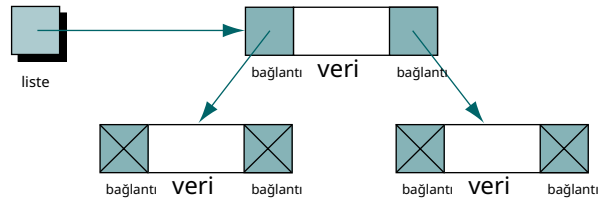
1. Sıralı ve ikili aramalar Bölüm 13'te tartışılmaktadır.

2. Ağaçları incelediğimizde, kolay güncellemeler ve verimli aramalar için birkaç veri yapısını göreceksiniz.

Şekil 1-3(a), bir doğrusal listenin bağlı liste uygulamasını göstermektedir. Son eleman hariç her bir elemandaki bağlantı, kendine özgü ardılına işaret eder; son elemandaki bağlantı ise listenin sonunu belirten bir null işaretçisidir. Şekil 1-3(b), bir doğrusal olmayan listenin bağlı liste uygulamasını göstermektedir. Doğrusal olmayan bir listedeki bir eleman iki veya daha fazla bağlantıya sahip olabilir. Burada her bir eleman, her biri bir ardıla işaret eden iki bağlantı içerir. Şekil 1-3(c), boş bir liste, doğrusal veya doğrusal olmayan bir liste örneğini içermektedir. Boş bir listeyi null liste işaretçisi olarak tanımlıyoruz.



(a) Doğrusal liste



(b) Doğrusal olmayan liste



(c) Boş liste

ŞEKİL 1-3Bağlı Listeler

Bu bölümde, yalnızca bağlı listeler için temel kavramları tartışıyoruz. Bu kavramları gelecekteki bölümlerde genişleteceğiz.

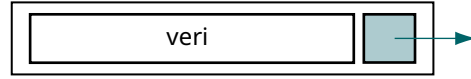
Düğümmler

Bağlı liste uygulamasında, bir listedeki elemanlara düğüm denir. Bir düğüm, iki parçadan oluşan bir yapıdır: veri ve bir veya daha fazla bağlantı. Şekil 1-4, bir doğrusal liste için bir düğüm ve bir doğrusal olmayan liste için diğer düğümü göstermektedir.

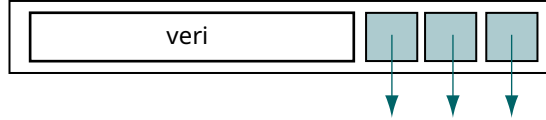
Bağlı listedeki düğümler kendine referans veren yapılar olarak adlandırılır. Kendine referans veren bir yapıda, yapının her bir örneği, aynı yapısal türdeki diğer örneklerle bir veya daha fazla işaretçi içerir. Şekil 1-4'te, oklarla gösterilen renkli kutular, bağlı listeyi kendine referans veren bir yapı haline getiren işaretçilerdir.

Düğümdeki veri kısmı tek bir alan, birden fazla alan veya birkaç alan içeren bir yapı olabilir, ancak her zaman tek bir alan olarak işlev görür. Şekil 1-5, bir doğrusal listenin düğümü için üç tasarımı göstermektedir. Sol üstteki düğüm bir

(a) Doğrusal listede bir düğüm

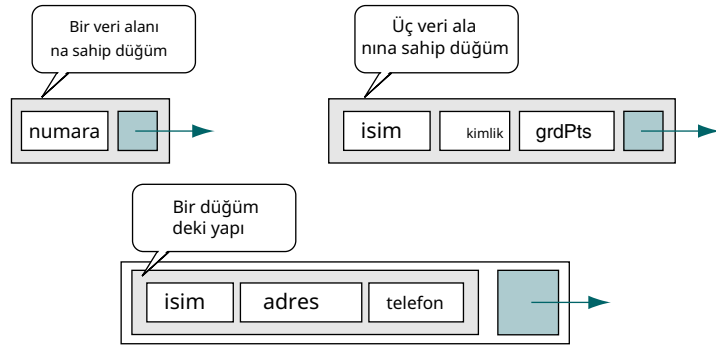


(b) Doğrusal olmayan listede bir düğüm



ŞEKİL 1-4Düğüm

tek bir alan, bir sayı ve bir bağlantı içerir. Sağ üstteki düğüm daha tipiktir. Üç veri alanı içerir: bir isim, bir kimlik ve not puanları (*grdPts*)—ve bir bağlantı. Üçüncü örnek, önerdiğimiz örnektir. Alanlar kendi yapı tanımlarında tanımlanır ve daha sonra bir düğüm yapısı tanımına konulur.



ŞEKİL 1-5Bağlı Liste Düğüm Yapıları

Bağlı Listelere İşaretçiler

Bir bağlı listenin her zaman bir baş işaretçisi olmalıdır. Listeyi nasıl kullandığımıza bağlı olarak, birkaç başka işaretçimiz de olabilir. Örneğin, bir bağlı listede arama yapacaksa, aradığımız veriyi bulduğumuz konuma ek bir işaretçi ihtiyacımız olacak. Ayrıca, birçok yapıda, listenin son düğümüne bir işaretçi ile birlikte bir baş işaretçisi varsa programlama daha verimli olur.

1.5 ADT'ler için Genel Kod

Veri yapılarında soyut veri türleri için genel kod oluşturmalıyız.

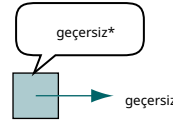
Genel kod bize tek bir kod seti yazma ve bunu herhangi bir veri türüne uygulama imkanı verir. İçin

örneğin, bir yığın yapısını uygulamak için genel işlevler yazabiliriz. Daha sonra genel işlevleri bir tamsayı yığını, bir float yığını, bir double yığını vb. uygulamak için kullanabiliriz. C++ ve Java gibi bazı yüksek seviyeli diller, genel kodu işlemek için özel araçlar sağlarken, C'nin iki özellik aracılığıyla sınırlı bir yeteneği vardır: *void* işaretçisi ve işlev işaretçisi.

void işaretçisi

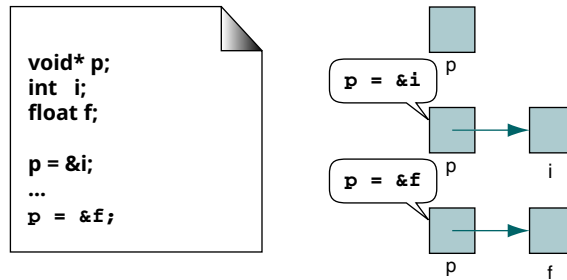
İlk özellik *void* işaretçisidir. C güçlü bir şekilde türendirilmiş olduğundan, atama ve karşılaştırma gibi işlemler uyumlu türler kullanmalı veya uyumlu türlere dönüştürülmelidir.

Tek istisna, bir *void* işaretçisidir; bu işaretçi bir dönüşüm olmadan atanabilir. Başka bir deyişle, bir *void* işaretçisi, derleme veya çalışma zamanında herhangi bir veri türünü temsil etmek için kullanılabilen genel bir işaretçidir. Şekil 1-6, bir *void* işaretçisinin fikrini göstermektedir. Bir *void* işaretçisinin null işaretçi olmadığını unutmayın; bu, genel bir veri türüne (*void*) işaret etmektedir.



ŞEKİL 1-6 void'a işaretçi

Örnek Kavramı göstermek için basit bir program yazalım. Üç değişken içerir: bir tamsayı, bir kayan nokta sayısı ve bir *void* işaretçisi. Programın farklı zamanlarında işaretçi, tamsayı değerinin veya kayan nokta değerinin adresine ayarlanabilir. Şekil 1-7 durumu göstermektedir.



ŞEKİL 1-7 Program 1-1 için İşaretçiler

Program 1-1, bir *void* işaretçisi kullanır; bu işaretçiyi bir tamsayı veya bir kayan nokta sayısını yazdırmak için kullanabiliriz.

PROGRAM 1-1 void'e İşaretçi Göster

```

1  /* void işaretçisini gösterin.
2     Yazarı:
3     Tarih:
4  */
5  #include <stdio.h>
6
7  int main ()
8  {
9  // Yerel Tanımlar
10     void* p;
11     int ben = 7;
12     kayan nokta f = 23,5;
13
14 // İfadeler
15     p = &i;
16     printf ("i içerir: %d", *((int*)p) );p = &f;
17
18
19     printf ("f içerir: %f", *((float*)p));return 0;
20
21
22 }// ana

```

Sonuçlar:
i 7 içerir
f 23.500000 içerir

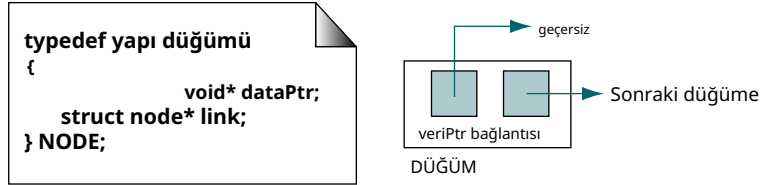
Program 1-1 AnaliziProgram basit, ancak konuyu gösteriyor. İşaretçi **p** bir voidişaretçisi olarak tanımlanmıştır, ancak bir tam sayı veya kayan nokta sayısının adresini kabul edebilir. Ancak,void işaretçileri hakkında çok önemli bir noktayı hatırlamalıyız: bir void işaretçisicast edilmeden dereferans edilemez. Diğer bir deyişle, ***p** kullanmadan önce cast etmemiz gerekir. Bu nedenle, yazdırma işlevinde işaretçiyi kullanmadan önce cast etmemiz gerekiyor.

void işaretçisine dereference edilemez.

ÖrnekBaşka bir örnek olarak, bir sistem işlevine bakalım, *malloc*. Bu işlevbir void işaretçisi döndürür. *malloc* işlevinin tasarımcıları, her türlü veriyi dinamik olarakırmak için ihtiyaç duydular. Ancak, her biri belirli bir veri türüne (int*, float*,double* vb.) işaretçi döndüren birkaç mal-loc işlevi kullanmak yerine, bir void işaretçisi döndüren genel bir işlev tasarladılar(void*). Gerek olmasa da, döndürülen işaretçinin uygun türe cast edilmesi ni öneriyoruz. Aşağıda, bir tam sayıya işaretçi oluşturmak için malloc kullanımını gösterir.

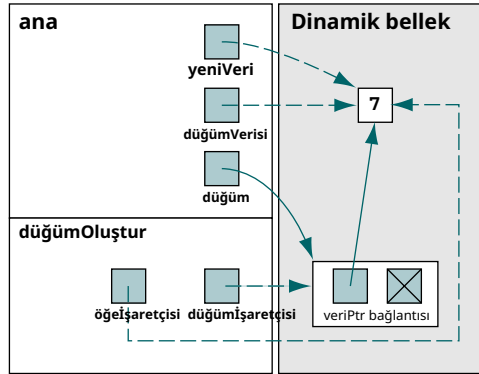
```
intPtr = (int*)malloc (sizeof (int));
```

Örnek Şimdi, ADT'lerimizi uygulamak için kullandığımız şeye benzer bir örneğe bakalım. Bir düğüm yapısı oluşturmak için genel bir işleve ihtiyacımız var. Yapı iki alana sahiptir: veri ve bağlantı. Bağlantı alanı, düğüm yapısına işaretçidir. Ancak veri alanı, herhangi bir türde olabilir: tam sayı, kayan nokta, dize veya hatta başka bir yapı. Düğümde herhangi bir türde veriyi saklayabilmemiz için işlevi genel hale getirmek üzere, dinamik bellek içinde saklanan verilere işaret eden bir *void* işaretçisi kullanıyoruz. Düğüm yapısını Şekil 1-8'de gösterildiği gibi tanımlıyoruz.



ŞEKİL 1-8Düğüm İşaretçisi

Şimdi, herhangi bir türde veri işaretçisini kabul eden bir işlev çağıran programı yazalım ve verileri işaret eden bir düğüm ve bir bağlantı işaretçisi oluşturalım. Bağlantı işaretçisinin nereye işaret edeceğini bilmediğimiz için, onu null yapıyoruz. İşaretçi tasarımı Şekil 1-9'da gösterilmiştir.



ŞEKİL 1-9Programlar 1-2 ve 1-3 için İşaretçiler

Genellikle, ADT'ler kendi başlık dosyalarında saklanır. Bu nedenle, düğüm oluşturma kodunu yazmaya başlıyoruz ve kodu bir başlık dosyasına yerleştiriyoruz. Bu kod Program 1-2'de gösterilmiştir.

PROGRAM 1-2Düğüm Başlık Dosyası Oluştur

1/*Düğüm yapısı oluşturmak için başlık dosyası.

devam ediyor

PROGRAM 1-2Düğüm Başlık Dosyası Oluştur (devam ediyor)

23456789	Yazarı: Tarih:
101112131415161718192021222324	<pre> */ typedef yapı düğümü { void* dataPtr; struct node* link; } NODE; /* ===== createNode ===== Dinamik bellek içinde bir düğüm oluşturur ve veriyi ipucu olarak saklar. Önce itemPtr, saklanacak veriye işaretçidir. Sonra düğüm oluşturuldu ve adresi döndürüldü. */ NODE* createNode (void* itemPtr) { DÜĞÜM* düğümPtr; nodePtr = (NODE*) malloc (sizeof (NODE)); nodePtr->dataPtr = itemPtr; nodePtr->link = NULL; return nodePtr; } // createNode </pre>

Artık veri yapısını ve düğüm oluşturma fonksiyonunu oluşturduğumuza göre, düğümde *void* işaretçilerin kullanımını göstermek için Program 1-3'ü yazabiliriz.

PROGRAM 1-3Düğüm Oluşturma Fonksiyonunu Göster

123456789	<pre> /* Basit genel düğüm oluşturma fonksiyonunu gösterin. Yazarı: Tarih: */ #include <stdio.h> #include <stdlib.h> #include "P1-02.h" int ana (geçersiz) { // Yerel Tanımlar int* yeniVeri; int* düğümVerisi; DÜĞÜM* düğümü; // İfadeler yeniVeri = (int*)malloc (sizeof (int)); *yeniVeri = 7; </pre>
101112131415161718	// Başlık dosyası

devam ediyor

PROGRAM 1-3Düğüm Oluşturma Fonksiyonunu Göster (devamı)

19202122232425

```

düğüm = createNode (yeniVeri);düş
ümVeri = (int*)düğüm->dataPtr;printf ("
Düğümünden veri: %d", *düğümVeri);return 0;
} // ana

```

Sonuçlar:

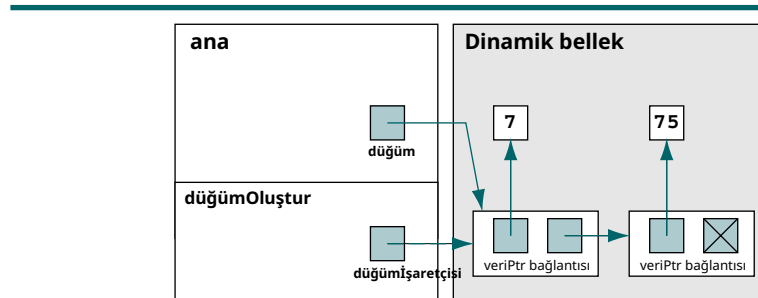
Düğümünden veri: 7

Program 1-3 Analizi Bu programda birkaç önemli kavram bulunmaktadır. Öncelikle, düğümde saklanacak veriler *void* işaretçisi ile temsil edilmektedir. Genellikle bu düğümlerin programda birçok örneği bulunduğundan, veriler dinamik bellek içinde saklanmaktadır. Verilerin tahsis ve saklanması programcının sorumluluğundadır. Bu iki adımı 17 ve 18. ifadelerde gösteriyoruz.

createNode fonksiyonu dinamik bellek içinde bir düğüm yapısı tahsis eder, veriyi *void* işaretçisini düğümde saklar ve ardından düğümün adresini döndürür. 22. ifade *void* işaretçisini düğümünden bir tamsayı işaretçisine saklıyoruz. C dilinin güçlü bir türü olduğu için, bu atama bir tamsayı işaretçisi ne dönüştürülmelidir. Yani, bir *void* işaretçisinde bir adres saklayabilirken, türünü bilmeden, tersine bu doğru değildir. Bir *void* işaretçisini kullanmak için, hatta bir atamada bile, dönüştürülmesi gerekmektedir.

Bir *void* işaretçisine yapılan her referans, işaretçiyi doğru türe dönüştürmelidir.

Örnek ADT yapıları genellikle bir düğümün birkaç örneğini içerir. ADT kavramını daha iyi göstermek için, Program 1-3'ü iki farklı düğüm içerecek şekil de değiştirelim. Bu basit örnekte, ilk düğümü ikinci düğüme işaret ediyoruz. Programın işaretçi yapısı Şekil 1-10'da gösterilmiştir.



ŞEKİL 1-10İki Bağılı Düğüm için Yapı

Şekil 1-10'daki işaretçi değerleri, Program 1-4'ün sonundaki ayarları temsil eder.

PROGRAM 1-4İki Bağlı Düğüm ile Liste Oluştur

```

1  /* İki bağlı düğümden oluşan bir liste oluştur. */
2      Yazarı:
3      Tarih:
4  */
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include "P1-02.h"                                // Başlık dosyası
8
9  int ana (geçersiz)
10 {
11     // Yerel Tanımlar
12     int* yeniVeri;
13     int* düğümVerisi;
14     DÜĞÜM* düğüm;
15
16     // İfadeler
17     yeniVeri = (int*)malloc (sizeof (int));
18     *yeniVeri = 7;
19     düğüm = createNode (newData);
20
21     newData    = (int*)malloc (sizeof (int));
22     *newData    = 75;
23     düğüm->link = createNode (newData);düğüm
24
25     mVerisi = (int*)düğüm->dataPtr;printf ("
26     Düğüm 1'den veri: %d", *düğümVerisi);düğümVerisi = (int*)
27
28     düğüm->link->dataPtr;
29     printf ("Düğüm 2'den veri: %d", *düğümVerisi);return 0;
30
31 }// ana

```

Sonuçlar:

Düğüm 1'den veri: 7
Düğüm 2'den veri: 75

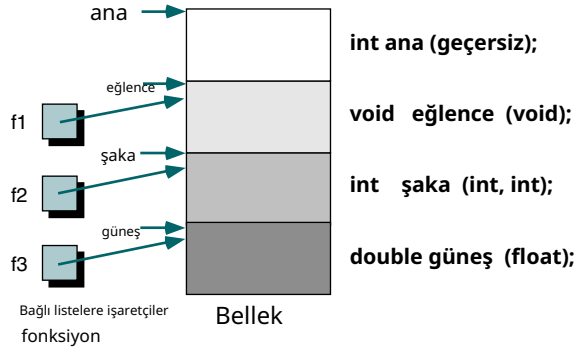
Program 1-4 AnaliziBu program önemli bir noktayı göstermektedir. Programda gösterilen gibi genel bir yapıda, düğümler ve veriler dinamik bellek içinde olmalıdır. Programı incelerken, kodu Şekil 1-10 üzerinden takip edin.

Fonksiyona İşaretçi

C genel kodu oluşturmak için gereken ikinci araç, fonksiyona işaretçidir. Bu bölümde nasıl kullanılacağını tartışıyoruz.

Programınızdaki fonksiyonlar bellek kaplar. Fonksiyonun adı,bellekteki ilk baytına işaret eden bir işaretçi sabitidir. Örneğin, bellek içinde dört fonksiyonunuz olduğunu hayal edin: main, fun, pun ve sun.Bu

ilişki Şekil 1-11'de grafiksel olarak gösterilmektedir. Her fonksiyonun adı bellekteki koduna bir işaretçidir.



ŞEKİL 1-11 Bellekteki Fonksiyonlar

Fonksiyonlara İşaretçilerin Tanımlanması

Diğer tüm işaretçi türlerinde olduğu gibi, fonksiyon değişkenlerine işaretçiler tanımlayabiliriz ve `fun`, `pun` ve `sun` adreslerini onlarda saklayabiliriz. Bir fonksiyona işaretçi tanımlamak için, onu bir prototip tanıımıymış gibi kodlarız, fonksiyonun işaretçisi parantez içinde olur. Bu format Şekil 1-12'de gösterilmektedir. Parantezler önemlidir: onsuz C, fonksiyon dönüş türünü bir işaretçi olarak yorumlar.

Fonksiyonlara İşaretçiler Kullanma

Fonksiyonlara işaretçileri nasıl oluşturup kullanacağınızı gördüğünüze göre, herhangi iki veri parçasının daha büyük olanını döndüren genel bir fonksiyon yazalım. Fonksiyon, önceki bölümde açıklandığı gibi iki `void` işaretçisi kullanır. Fonksiyonumuz, `void` işaretçileri tarafından temsil edilen iki değerden hangisinin daha büyük olduğunu belirlemek zorundadır, ancak hangi tür dönüşümler kullanacağını bilmediği için bunları doğrudan karşılaştıramaz. Sadece uygulama programı veri türlerini bilir.

Çözüm, genel işlevimizi kullanan her program için basit karşılaştırma işlevleri yazmaktır. Ardından, genel karşılaştırma işlevini çağırdığımızda, kullanması gereken belirli karşılaştırma işlevini geçmek için bir işlev işaretçisi kullanırız.

Örnek Görüşmemizde `void` işaretçilerine baktığımız gibi, genel işlevimizi `larger` olarak adlandırdığımız bir başlık dosyasına yerleştiriyoruz, böylece kolayca kullanılabilir. Program arayüzleri ve işaretçiler Şekil 1-13'te gösterilmiştir.

...

// Yerel Tanımlar

void (*f1)(void);int (*f2)(int, int);double (*f3)(float);...

// İfadeler

...

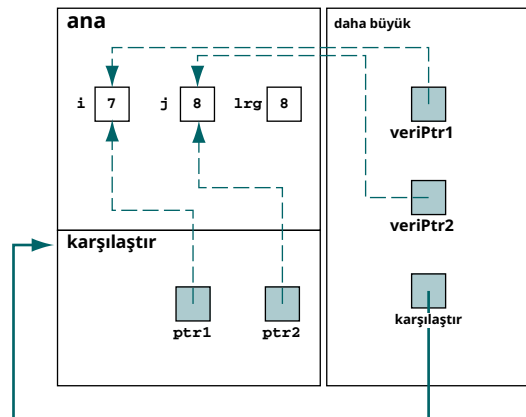
f1 = eğlence;f2 = keli me oyunu; f3 = güne

§;

...

f1: Parametre almayan bir fonksiyona işaretçi; v oid döndürür.

ŞEKİL 1-12Fonksiyonlara İşaretçiler



ŞEKİL 1-13Daha Büyük Fonksiyonun Tasarımı

Kod Program 1-5'te gösterilmiştir.

PROGRAM 1-5Daha Büyük Karşılaştırma Fonksiyonu

```
1234567 /* İki değerin daha büyüğünü belirlemek için genel bir fonksiyon
          void işaretçileri olarak referans edilen değerler. */
          dataPtr1 ve dataPtr2, bilinmeyen bir türdeki değerlere işaret e
          den işaretçilerdir.
          ptrToCmpFun, veri türlerini bilen bir fonksiyonun ad
          residir
          Post verileri karşılaştırıldı ve daha büyük değer döndürüldü
```

devam ediyor

PROGRAM 1-5 Daha Büyük Karşılaştırma Fonksiyonu (devamı)

```

89  */
1011213141516 void* daha büyük (void* dataPtr1, void* dataPtr2,
int (*ptrToCmpFun)(void*, void*))
{
    if ((*ptrToCmpFun) (dataPtr1, dataPtr2) > 0)
        return dataPtr1;

    aksi takdirde
        veriPtr2'yi döndür;
} // daha büyük

```

Program 1-6, genel karşılaştırmamızı nasıl kullanacağımıza ve ona belirli bir karşılaştırm a fonksiyonu geçireceğimize dair bir örnek içermektedir.

PROGRAM 1-6 İki Tam Sayıyı Karşılaştır

```

123456789  /* Genel karşılaştırma işlevlerini ve fonsiyona işaretçiyi gösterin.

Yazarı:
Tarih:

*/
#include <stdio.h>
#include <stdlib.h>
#include "P1-05.h" // Başlık dosyası

int karşılaştırma (void* ptr1, void* ptr2);

int main (void)
{
    // Yerel Tanımlar

    int i = 7 ;
    int j = 8 ;
    int lrg;

    // İfadeler
    lrg = (*(int*) larger (&i, &j, compare)); printf ("Büyük
değer: %d", lrg); return 0;

} // ana
/* ===== compare =====
Tam sayıya özgü karşılaştırma işlevi.
Önce ptr1 ve ptr2 tam sayı değerlerine işaretçilerdir
Sonra ptr1 >= ptr2 ise +1 döner
ptr1 < ptr2 ise -1 döner
*/
int karşılaştı (void* ptr1, void* ptr2)

```

devam ediyor

PROGRAM 1-6 İki Tam Sayıyı Karşılaştır (devam ediyor)

333435363738	<pre> { if (*(int*)ptr1 >= *(int*)ptr2) return 1; başka return -1; } // karşılaştır </pre>
Sonuçlar: Daha büyük değer: 8	

Örnek Şimdi, iki kayan nokta sayısını karşılaştıran bir program yazalım. Daha büyük fonksiyonum uzukullanabiliriz, ancak yeni bir karşılaştırma fonksiyonu yazmamız gerekiyor. Sadece karşılaştırma fonksiyonunu veveri özel ifadelerini *main* içinde değiştirerek Program 1-6'yı tekrarlıyoruz. Sonuç Program 1-7'de gösterilmiştir.

PROGRAM 1-7 İki Kayan Nokta Değerini Karşılaştır

123456789	<pre> /* Genel karşılaştırma işlevlerini ve fonsiyona işaretçiyi gösterin. Yazarı: Tarih: */ #include <stdio.h> #include <stdlib.h> #include "P1-05.h" // Başlık dosyası int karşılaştırma (void* ptr1, void* ptr2); int main (void) { // Yerel Tanımlar float f1 = 73.4; float f2 = 81.7; float lrg; // İfadeler lrg = (*(float*) larger (&f1, &f2, compare));printf ("Büyük d eğer: %5.1f", lrg);return 0; } // ana /* ===== karşılaştır ===== Float özel karşılaştırma fonksiyonu. Önce ptr1 ve ptr2 tam sayı değerlerine işaretçilerdir Sonra ptr1 >= ptr2 ise +1 döner </pre>
1011121314151617181920212223242526272829	

devam ediyor

PROGRAM 1-7 İki Kesirli Değeri Karşılaştır (devamı)

```

303132333435363738
ptr1 < ptr2 ise -1 döner
*/
int karşılaştırma (void* ptr1, void* ptr2)
{
    if (*(float*)ptr1 >= *(float*)ptr2)
        return 1;
    başka
        return -1;
} // karşılaştır

```

Sonuçlar:
Büyük değer: 81.7

1.6 Algoritma Verimliliği

Herhangi bir problem için nadiren tek bir algoritma vardır. Aynı proble mi çözen iki farklı algoritmayı karşılaştırırken, genellikle bir algoritmanın diğerinden bir büyüklük sırası daha verimli olduğunu bulursunuz. Bu durumda, daha verimli algoritmayı tanıyabilmeniz ve seçebilmemiz man tıklıdır.

Bilgisayar bilimcileri algoritmaları ve algoritma verimliliğini kapsamlı bir şe kilde incelemiş olsalar da, bu alana resmi bir isim verilmemiştir. Brassard ve Br atley, “verimli algoritmalar tasarlamak ve analiz etmek için kullanılan temel tek niklerin sistematik çalışması” olarak tanımladıkları algoritmalar terimini icat et tiler.³ Bu kitapta bu terimi kullanıyoruz.

Bir fonksiyon lineer ise—yani, döngü veya özyineleme içermiyorsa—veri mliliği içerdiği talimat sayısının bir fonksiyonudur. Bu durumda, verimliliği bi lgisayarın hızına bağlıdır ve genellikle bir programın genel verimliliğinde bir faktör değildir. Öte yandan, döngü veya özyineleme kullanan fonksiyonlar ve rimlilikte geniş bir çeşitlilik gösterir. Bu nedenle algoritma verimliliği çalışma sı döngülere odaklanır. Analizimiz döngülere yoğunlaşmaktadır çünkü özyin eleme her zaman bir döngüye dönüştürülebilir.

Özel örnekleri incelerken, genellikle algoritmanın verimliliğini işleme si gereken eleman sayısına bağlı olarak tartışıyoruz. Genel format şudur:

$$f(n) = \text{verimlilik}$$

Temel kavramlar bu bölümde tartışılmaktadır.

3. Gilles Brassard ve Paul Bratley, *Algoritmalar Teorisi ve Uygulaması* (Englewood Cliffs, N.J.: Prentice Hall, 1988), xiii.

Doğrusal Döngüler

Basit bir döngü ile başlayalım. Aşağıdaki kodda döngü gövdesinin kaç kez tekrarlandığını bilmek istiyoruz:⁴

```
for (i = 0; i < 1000; i++)
    uygulama kodu
```

i 'nin bir tamsayı olduğunu varsayarsak, cevap 1000 kezdir. İterasyon sayısı doğrudan döngü faktörüne, yani 1000'e orantılıdır. Faktör ne kadar yüksekse, döngü sayısı da o kadar yüksektir. Verimlilik, iterasyon sayısıyla doğrudan orantılıdır, bu nedenle

$$f(n) = n$$

Ancak, cevap her zaman yukarıdaki örnekte olduğu kadar basit değildir. Örneğin, aşağıdaki döngüyü düşünelim. Bu döngüde gövde kaç kez tekrar ediliyor? Burada cevap 500 kezdir. Neden?

```
for (i = 0; i < 1000; i += 2)
    uygulama kodu
```

Bu örnekte iterasyon sayısı döngü faktörünün yarısıdır. Yine de, faktör ne kadar yüksekse, döngü sayısı da o kadar yüksektir. Bu döngünün verimliliği, faktörün yarısına orantılıdır, bu da onu

$$f(n) = n / 2$$

Eğer bu döngü örneklerinden birini çizerseniz, düz bir çizgi elde edersiniz. Bu nedenle, bunlara lineer döngüler denir.

Logaritmik Döngüler

Bir lineer döngüde, döngü güncellemesi ya toplar ya da çıkarır. Bir logaritmik döngüde, kontrol değişkeni her yinelemede çarpılır veya bölünür. Aşağıdaki program segmentlerinde döngülerin gövdesi kaç kez tekrarlanır?

Çarpma Döngüleri

```
for (i = 0; i < 1000; i *= 2)
    uygulama kodu
```

Bölme Döngüleri

```
for (i = 0; i < 1000; i /= 2)
    uygulama kodu
```

Bu problemi anlamanıza yardımcı olmak için, Tablo 1-3 her yineleme için i değerlerini analiz etmektedir. Gördüğünüz gibi, yineleme sayısı her iki durumda da 10'dur. Bunun nedeni, her yinelemede i değerinin çarpma döngüsü için iki katına çıkması ve bölme döngüsü için yarıya inmesidir. Böylece, yineleme sayısı

4. Algoritma verimliliği analizi için, döngü yapılarının net bir şekilde görülebilmesi için C kodu kullanıyoruz.

Çarpma		Bölme	
Yineleme	i'nin Değeri	Yineleme	i'nin Değeri
123456789	1248	123456789	1000 500 250 125 62 31 15 7310
10 (çıkış)	1024	10 (çıkış)	

TABLO 1-3Çarpma ve Bölme Döngülerinin Analizi

çarpan veya bölenin bir fonksiyonudur, bu durumda 2. Yani, döngü aşağıda gösterilen koşul doğru olduğu sürece devam eder.

```

çarp 2Yinelemeler < 1000
böl 1000 / 2Yinelemeler >= 1

```

Analizi genelleştirerek, çarpma veya bölme işlemlerindeki döngülerin yinelemelerinin aşağıdaki formülle belirlendiğini söyleyebiliriz:

$$f(n) = \log n$$

İç İçe Döngüler

Döngülerin içinde döngü barındıran döngülere iç içe döngüler denir. İç içe döngülerini analiz ederken, her döngünün tamamladığı yineleme sayısını belirlemeliyiz. Toplam, iç döngüdeki yineleme sayısı ile dış döngüdeki yineleme sayısının çarpımıdır.

Iterations = dış döngü yinelemeleri x iç döngü yinelemeleri

Şimdi üç iç içe döngüye bakıyoruz: lineer logaritmik, kare ve bağımlı kare.

Doğrusal Logaritmik

Aşağıdaki koddaki iç döngü, çarpan bir döngüdür. Çarpma döngüsünü görmek için, iç döngüdeki güncelleme ifadesine bakın.

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j *= 2)
        application code
```

İç döngüdeki yineleme sayısı bu nedenle $\lceil \log 10 \rceil$. Ancak, dış döngü tarafından kontrol edildiği için, yukarıdaki formül, dış döngünün kaç kez çalıştığı ile çarpılmalıdır, bu da 10'dur. Bu bize verir

$$10 \log 10$$

genelleştirilmiş olarak

$$f(n) = n \log n$$

Kare

Bir kare döngüsünde, iç döngünün çalıştığı sayısı dış döngü ile aynıdır. Aşağıdaki örneği düşünün.

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        application code
```

Dış döngü (için i) 10 kez çalıştırılır. Her bir yinelemesi için, iç döngü (için j) de 10 kez çalıştırılır. Cevap bu nedenle 100'dür, bu da 10×10 , döngülerin karesidir. Bu formül genelleştirilir

$$f(n) = n^2$$

Bağımlı İkinci Dereceden

Bir bağımlı ikinci dereceden döngü, iç döngünün yineleme sayısı dış döngüye bağlıdır. Aşağıdaki örnekte gösterilen iç içe döngüyü düşünün.

```
for (i = 0; i < 10; i++)
    for (j = 0; j < i; j++)
        application code
```

Dış döngü, önceki döngüyle aynıdır. Ancak, iç döngü bir faktörü içi n dış döngüye bağlıdır. İlk yinelemede yalnızca bir kez, ikinci yinelemede iki kez, üçüncü yinelemede üç kez ve bunun gibi çalıştırılır. İç döngünün gövdesindeki yineleme sayısı, aşağıda gösterildiği gibi hesaplanır.

$$1 + 2 + 3 + \dots + 9 + 10 = 55$$

Bu döngünün ortalamasını hesapladığımızda, 5.5 ($55/10$) olur, bu da yineleme sayısı (10) artı 1'in 2'ye bölünmesiyle aynıdır. Matematiksel olarak, bu hesaplama genelleştirilir

$$\frac{(n+1)}{2}$$

İç döngüyü dış döngünün kaç kez çalıştırıldığı ile çarpmak, bize bağımlı bir ikinci dereceden döngü için aşağıdaki formülü verir:

$$f(n) = n \left(\frac{n+1}{2} \right)$$

Big-O Notasyonu

Günümüz bilgisayarlarının hızıyla, bir algoritmanın verimliliğinin kesin ölçü müyle çok fazla ilgilenmiyoruz, daha çok genel büyüklük sırasıyla ilgileniyoruz. Eğer iki algoritmanın analizi, birinin 15 yineleme yaparken diğeri 25 yineleme yaptığını gösteriyorsa, her ikisi de o kadar hızlıdır ki farkı göremeyiz. Öte yandan, biri 15 kez yineleme yapıyorsa diğeri 1500 kez yineleme yapıyorsa, bu durum bizi endişelendirmelidir.

Fonksiyonda yürütülen ifade sayısının, n veri elemanlarının sayısına bağlı olduğunu göstermiş bulunuyoruz ve bu, $f(n)$ olarak ifade edilmektedir. Bir fonksiyon için türetilen denklem karmaşık olsa da, genellikle denklemin baskın bir faktörü sonucun büyüklüğünü belirler.

Bu nedenle, verimliliğin tam ölçüsünü belirlememize gerek yoktur, yalnızca büyüklüğü belirleyen faktörü bilmemiz yeterlidir. Bu faktör, "sırasıyla" anlamında kullanılır ve big-O'dur ve $O(n)$ olarak ifade edilir; yani, n sırasındadır.

Verimliliğin bu basitleştirilmesi big-O analizi olarak bilinir. Örneğin, bir algoritma kare şeklindeyse, verimliliğinin

$$O(n^2)$$

veya n kare sırasına sahip olduğunu söyleriz.

big-O notasyonu, $f(n)$ kullanılarak aşağıdaki adımlarla türetilir:

1. Her terimde, terimin katsayısını 1 olarak ayarlayın.

2. Fonksiyondaki en büyük terimi koruyun ve diğerlerini atın. Terimler, aşağıda gösterildiği gibi en düşükten en yükseğe sıralanmıştır.

$$\log n \quad n \quad n \log n \quad n^2 \quad n^3 \dots n^k \quad 2^n \quad n!$$

Örneğin,

$$f(n) = n \frac{(n+1)}{2} = \frac{1}{2} n^2 + \frac{1}{2} n$$

big-O notasyonunu hesaplamak için önce tüm katsayıları kaldırıyoruz. Bu bize

$$n^2 + n$$

küçük faktörleri kaldırdıktan sonra bize verir

$$n^2$$

big-O notasyonu olarak ifade edilir

$$O(f(n)) = O(n^2)$$

Başka bir örneği ele almak için, polinom ifadesine bakalım

$$f(n) = a_j n^k + a_{j-1} n^{k-1} + \dots + a_2 n^2 + a_1 n + a_0$$

Öncelikle, aşağıda gösterildiği gibi tüm katsayıları ortadan kaldırıyoruz.

$$f(n) = n^k + n^{k-1} + \dots + n^2 + n + 1$$

Bu ifadedeki en büyük terim birincisidir, bu nedenle bir polinom ifadesinin sırasının

$$O(f(n)) = O(n^k)$$

Verimlilik için Standart Ölçüler

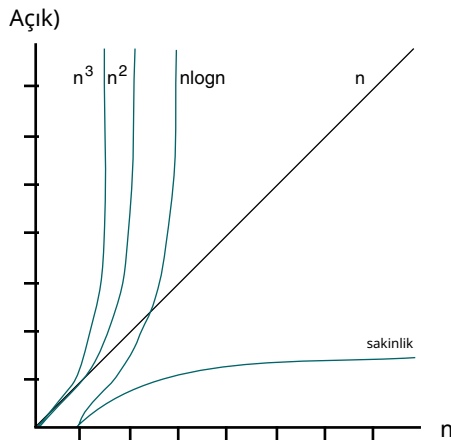
Bilgisayar bilimcileri, algoritma verimliliğinin yedi kategorisini tanımlamıştır. Bunları verimliliğin azalan sırasına göre Tablo 1-4'te listeliyoruz ve ilk beşini Şekil 1-14'te grafiksel olarak gösteriyoruz.

Verim	Büyük-O	İterasyonlar	Tahmini Zaman
LogaritmikDoğrusal	$O(\log n)$	14	mikrosaniye
Doğrusal	$O(n)$	10.000	saniye
Doğrusal logaritmik	$O(n(\log n))$	140.000	saniye
Kare	$O(n^2)$	10.000 ²	dakika
Polinom	$O(n^k)$		saat
Üstel	$O(c^n)$	$2^{10.000}$	çözumsuz
Faktöriyel	$O(n!)$	10,000!	çözumsuz

TABLO 1-4 $n = 10,000$ için Verimlilik Ölçüleri

Herhangi bir verimlilik ölçüsü, yeterince büyük bir örneğinele alındı ğını varsayar. Eğer yalnızca 10 elemanla uğraşıyorsanız ve gereken zam anbir saniyenin kesiriye, iki algoritma arasında anlamlı bir fark yoktur. Öte yandan, eleman sayısı arttıkça, algoritmalar arasındaki fark çarpıcı olabilir.

Verimlilikle neden ilgilenmemiz gerektiği sorusuna bir anlığına geri dönersek, bir problemi üç şekilde çözebileceğiniz durumu düşünün: bir i lineer, diğeri lineer logaritmik ve üçüncüsükareldir. 10,000 eleman içeren bir problem için verimliliklerinin büyüklüğü



ŞEKİL 1-14 Verim Ölçümlerinin Grafiği

öğeler, lineer çözümün bir saniyenin kesirini gerektirdiğini oysa ikincil çözümün dakikalar aldığını göstermektedir (bkz. Tablo 1-4).

Problemi diğer bir açıdan ele alırsak, saniyede bir milyon talimat yürüten bir bilgisayar kullanıyorsak ve döngü 10 talimat içeriyorsa, döngünün her yinelemesi için 0.00001 saniye harcıyoruz. Tablo 1-4, farklı verimlilikler göz önüne alındığında problemi çözmek için gereken zamanın bir tahminini de içermektedir.

Büyük-O Analiz Örnekleri

Tartıştığımız kavramları göstermek için, iki algoritmayı daha inceliyoruz : iki matris toplama ve çarpma.

Kare Matrisleri Toplama

İki kare matrisi toplamak için, karşılık gelen elemanlarını toplarız; yani, birinci matrisin birinci elemanını ikinci matrisin birinci elemanına, birinci matrisin ikinci elemanını ikinci matrisin ikinci elemanına ve bu şekilde devam ederiz. Elbette, iki matrisin aynı boyutta olması gerekir. Bu kavram Şekil 1-15'te gösterilmektedir.

$$\begin{array}{|c|c|c|} \hline 4 & 2 & 1 \\ \hline 0 & -3 & 4 \\ \hline 5 & 6 & 2 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 6 & 1 & 7 \\ \hline 3 & 2 & -1 \\ \hline 4 & 6 & 2 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 10 & 3 & 8 \\ \hline 3 & -1 & 3 \\ \hline 9 & 12 & 4 \\ \hline \end{array}$$

ŞEKİL 1-15 Matrisleri Toplama

İki matrisi toplamak için kullanılan sözde kod, Algoritma 1-3'te gösterilmektedir.

ALGORİTMA 1-3 İki Matrisi Topla

```

Algoritma addMatrix (matrix1, matrix2, size, matrix3)
matrix1'i matrix2'ye ekleyin ve sonuçları matrix3'e yerleştirin
  Önce matrix1 ve matrix2 veriye sahip olmalıdır
  size, matrisin sütun veya satır sayısıdır
  Sonuç olarak matrisler toplandı--sonuç matrix3'te
1  döngü (satır sonu değil)
  1  döngü (sütun sonu değil)
    1  matrix1 ve matrix2 hücrelerini ekle
    2  toplamı matrix3'te sakla
  2  döngüyü sonlandır
2  döngüyü sonlandır
son addMatrix

```

Algoritma 1-3 Analizi Bu algoritmada, bir satırdaki her bir eleman için, bir sütundaki tüm elemanları eklediğimizi görüyoruz. Bu klasik kare döngüsüdür. Dolayısıyla algoritmanın verimliliği $O(n^3)$ (boyut³) veya $O(n^2)$ şeklindedir.

Kare Matrisleri Çarpma

İki kare matris çarpıldığında, ilk matrisin bir satırındaki her bir elemanı, ikinci matrisin bir sütunundaki karşılık gelen elemanla çarpmamız gerekir. Elde edilen matrisin değeri, ürünlerin toplamı olacaktır. Örneğin, yukarıdaki toplama örneğindeki matrisi göz önünde bulundurursak, elde edilen matrisin ilk elemanı—yani, $[0, 0]$ konumundaki eleman—ilk satırdaki (eleman 0) her bir elemanın ilk sütundaki (sütun 0) karşılık gelen elemanla çarpılmasıyla elde edilen ürünlerin toplamıdır. $[0, 1]$ indeks konumundaki elemanın değeri, ilk satırdaki (yine satır 0) elemanların ikinci sütundaki (sütun 1) karşılık gelen elemanla çarpılmasıyla elde edilen ürünlerin toplamıdır. $[1, 2]$ indeks konumundaki elemanın değeri, ikinci satırdaki her bir elemanın üçüncü sütunda karşılık gelen elemanlarla çarpılmasıyla elde edilen ürünlerin toplamıdır. Bir kez daha kare matrislerin aynı boyutta olması gerekir. Şekil 1-16, iki matrisin n asıl çarpıldığını grafiksel olarak göstermektedir.

Bu kavramı genelleştirerek, şunu görüyoruz:

```
matrix3[satır, sütun] =
    matrix1[satır, 0] x matrix2[0, sütun]
    +
    matrix1[satır, 1] x matrix2[1, sütun]
    +
    matrix1[satır, 2] x matrix2[2, sütun]
    ...
    +
    matrix1[satır, s-1] x matrix2[s-1, sütun]
burada s = matrisin boyutu
```

Matrisleri çarpmak için kullanılan sözde kod, Algoritma 1-4'te sağlanmıştır.

ALGORİTMA 1-4 İki Matrisi Çarp

```
Algoritma multiMatrix (matrix1, matrix2, boyut, matrix3)
matrix1'i matrix2 ile çarp ve sonucu matrix3'e yerleştir
    Önce matrix1 ve matrix2 veriye sahip olmalıdır
    matrisin sütun ve satır sayısı
    Post matrisleri çarpıldı--sonuç matrix3 içinde
1 döngü (satır sonu değil)
    1 döngü (sütun sonu değil)
        1 döngü (satır sayısı kadar)
            1 toplamı hesapla
            ffffffff(tüm satır hücreleri) * (tüm sütun hücreleri)
            2 toplamı matrix3'te sakla
```

devam ediyor

ALGORİTMA 1-4 İki Matrisi Çarp (devam ediyor)

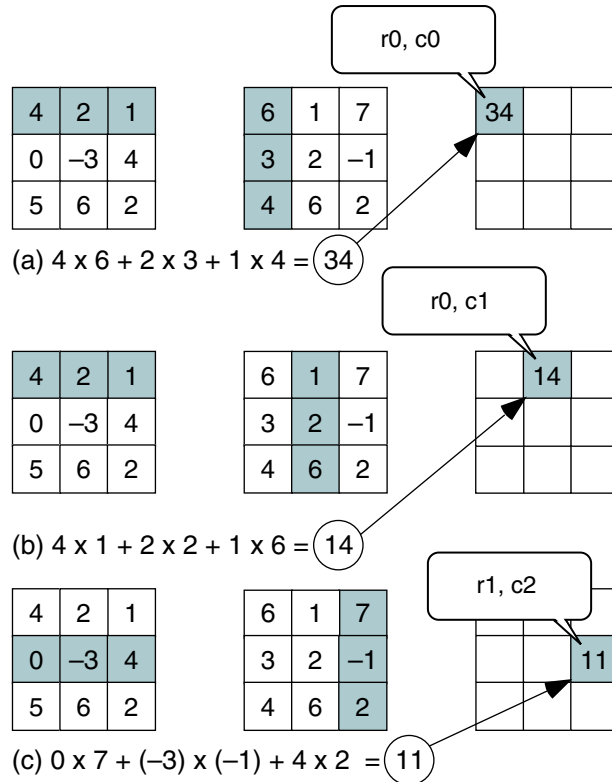
```

2   döngüyü sonlandır
2   döngüyü sonlandır
3   geri dön
son multiMatrix

```

Algoritma 1-4 Analizi Bu algoritmada üç iç içe döngü görüyoruz. Her döngü ilk ele-mandan başladığı için kübik bir döngüye sahibiz. Üç iç içe döngüye sahip döngülerin büyük-O verimliliği $O(n^3)$ (boyut³) veya $O(n^3)$ şeklindedir.

İlk matrisin satır sayısı, ikinci matrisin sütun sayısı aynıysa iki matrisi çarpmak da mümkündür. Bu problemin çözümünü bir alıştırmaya bırakıyoruz (Alıştırma 21).



ŞEKİL 1-16 Matrisleri Çarpma

1.7 Anahtar Terimler

soyut veri türü (ADT)

algoritmalar

atomik veri

büyük-O not

asyonubileşik

veriyapı

veri

veri yapısı

veri türü

bağımlı ikinci dereceden dön

güboş liste

kapsülleme

general kod

akıllı veri adları

lineer döngü

bağlantı

bağlı listelo

garitmik döngüdön

gü

modüler programlama

iç içe döngü

düğüm

void için işaretçi

fonksiyon için işaretçi

sözde kod

kuadratik dön

gügeri dönüş k

oşulukendine

referanslı seçim ifa

desi sıra

spagetti kodu

yapılandırılmış programlama

1.8 Özet

- ▣ Algoritmaları tanımlamak için kullanılan en yaygın araçlardan biri sözde koddur.
- ▣ Sözde kod, bir algoritma için gereken kodun İngilizce benzeri bir temsildir. Hem İngilizce hem de yapılandırılmış kodun bir parçasıdır.
- ▣ Atomik veriler, tekil, parçalanamaz varlıklardır.
- ▣ Atomik veri türleri, bir değerler kümesi ve bu değerlere uygulanan işlemler kümesi ile tanımlanır.
- ▣ Bir veri yapısı, atomik ve bileşik verilerin tanımlı bir ilişki ile bir araya getirilmesidir.
- ▣ Soyut veri türü (ADT), veri türü için anlamlı olan işlemlerle birlikte paketlenmiş bir veri beyanıdır.
- ▣ Bir ADT listesini uygulamak için kullanılan iki temel yapı vardır: diziler ve bağlı listeler.
- ▣ Bir dizide, bir listenin ardışıklığı, elemanların sıralı yapısı ile korunur. Bir diziyi aramak çok verimli olmasına rağmen, ekleme ve silme işlemleri verimli değildir.
- ▣ Bağlı listede ekleme ve silme işlemleri verimli olsa da, arama işlemi verimli değildir çünkü sıralı bir arama kullanmak zorundayız.
- ▣ Bağlı listede, her bir eleman bir sonraki elemanın veya elemanların konumunu içerir.

- ❑ Soyut veri türleri, bir algoritmanın birden fazla veri türü ile kullanılmasını sağlayan genel algoritmalar gerektirir.
- ❑ C dilinin genel kod oluşturulmasına olanak tanıyan iki özelliği vardır: void işaretçisi ve fonksiyon işaretçisi.
- ❑ Bir *void* işaretçisi, herhangi bir veri türünü temsil etmek için kullanılabilen genel bir işaretçidir.
- ❑ Bir *void* işaretçisi dereferans edilemez, bu da *void* işaretçisine yapılan atama dışı referansların doğru türe dönüştürülmesi gerektiği anlamına gelir.
- ❑ Bir fonksiyonun adı, bir fonksiyonun ilk baytına işaret eden bir işaretçi sabitidir.
- ❑ Fonksiyon işaretçisini, genel bir fonksiyonun parametre listesinde bir fonksiyonun adı için yer tutucu olarak kullanabiliriz.
- ❑ Algoritma verimliliği genellikle işlenen eleman sayısının ve kullanılan döngü türünün bir fonksiyonu olarak tanımlanır.
- ❑ Logaritmik bir döngünün verimliliği $f(n) = \log n$ olarak ifade edilir.
- ❑ Doğrusal bir döngünün verimliliği $f(n) = n$ olarak ifade edilir.
- ❑ Doğrusal logaritmik bir döngünün verimliliği $f(n) = n (\log n)$ olarak ifade edilir.
- ❑ Kare bir döngünün verimliliği $f(n) = n^2$ olarak ifade edilir.
- ❑ Bağımlı kare bir döngünün verimliliği $f(n) = n(n + 1)/2$ olarak ifade edilir.
- ❑ Kübik bir döngünün verimliliği $f(n) = n^3$ olarak ifade edilir.
- ❑ Verimliliğin basitleştirilmesi büyük-O notasyonu olarak bilinir.
- ❑ Yedi standart verimlilik ölçüsü $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^k)$, $O(c^n)$, ve $O(n!)$ olarak ifade edilir.

1.9 Pratik SetlerAlı

ştirmalar

1. Yapı şemaları ve sözde kod, iki farklı tasarım aracıdır. Nasıl farklıdır lar ve nasıl benzerler?
2. Farklı sözdizimsel yapılar kullanarak, bir sayıya 1 eklemek için en az iki söz de kod ifadesi yazın. Örneğin, aşağıdaki ifadelerden herhangi biri bir dosya dan veri almak için kullanılabilir:

```
öğrenci dosyasını oku
öğrenci dosyasını öğrenciye oku
(öğrenciDosyasını öğrenciye oku)
```

3. Bir uygulama programındaki algoritmanın, soyut veri türündeki bir algorithmadan nasıl farklı olduğunu açıklayın.
4. Birincil programlama diliniz için atomik veri türlerini tanımlayın.

5. Birincil programlama diliniz için bileşik veri türlerini tanımlayın.
6. Aşağıdaki verimlilikleri en küçüğünden en büyüğüne sıralayın:
 - a. 2^n
 - b. *hayır!*
 - c. n^5
 - d. 10,000
 - ve. $n \log(n)$
7. Aşağıdaki verimlilikleri en küçüğünden en büyüğüne sıralayın:
 - a. $n \log(n)$
 - b. $n + n^2 + n^3$
 - c. 24
 - d. $n^{0.5}$
8. Aşağıdakiler için büyük-O notasyonunu belirleyin:
 - a. $5n^{5/2} + n^{2/5}$
 - b. $6 \log(n) + 9n$
 - c. $3n^4 + n \log(n)$
 - d. $5n^2 + n^{3/2}$
9. Aşağıdaki program segmentinin çalışma zamanı verimliliğini hesaplayın:


```
for (i = 1; i <= n; i++)
    printf("%d ", i);
```
10. Aşağıdaki program segmentinin çalışma zamanı verimliliğini hesaplayın:


```
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++)
        for (k = 1; k <= n; k++)
            print ("%d %d %d\n", i, j, k);
```
11. Eğer algoritmanın `doIt` verimlilik faktörü $5n$ ise, aşağıdaki program segmentinin çalışma zamanını hesaplayın: *efficiency*:


```
for (i = 1, i <= n; i++)
    doIt (...)
```
12. Eğer algoritmanın `doIt` verimliliği $O(n) = n^2$ olarak ifade edilebiliyorsa, aşağıdaki program segmentinin verimliliğini hesaplayın:


```
for (i = 1; i <= n; i++)
    for (j = 1; j < n, j++)
        doIt (...)
```
13. Eğer algoritmanın `doIt` verimliliği $O(n) = n^2$ olarak ifade edilebiliyorsa, aşağıdaki program segmentinin verimliliğini hesaplayın:


```
for (i = 1; i < n; i *= 2)
    doIt (...)
```
14. Bir algoritmanın verimliliği $5n^2$ olarak verildiğinde, eğer bu algoritmadaki bir adım 1 nanosecond (10^{-9} saniye) sürüyorsa, algoritmanın 1000 boyutundaki bir giriş işleme ne kadar sürer?

15. Bir algoritmanın verimliliği n^3 olarak verildiğinde, eğer bu algoritmadaki bir adım 1 nanosecond (10^{-9} saniye) sürüyorsa, algoritmanın 1000 boyutundaki bir giriş işleme ne kadar sürer?
16. Bir algoritmanın verimliliği $5n \log(n)$ olarak verildiğinde, eğer bu algoritmadaki bir adım 1 nanosecond (10^{-9} saniye) sürüyorsa, algoritmanın 1000 boyutundaki bir giriş işleme ne kadar sürer?
17. Bir algoritma, n boyutundaki bir giriş işler. Eğer n 4096 ise, çalışma süresi 512 milisaniedir. Eğer n 16,384 ise, çalışma süresi 2048 milisaniedir. Verimlilik nedir? Büyük-O notasyonu nedir?
18. Bir algoritma, n boyutundaki bir giriş işler. Eğer n 4096 ise, çalışma süresi 512 milisaniedir. Eğer n 16,384 ise, çalışma süresi 8192 milisaniedir. Verimlilik nedir? Büyük-O notasyonu nedir?
19. Bir algoritma, n boyutunda verilen bir giriş işler. Eğer n 4096 ise, çalışma süresi 512 milisaniedir. Eğer n 16,384 ise, çalışma süresi 1024 milisaniedir. Verimlilik nedir? Büyük-O notasyonu nedir?
20. Üç öğrenci aynı problem için algoritmalar yazdı. Üç algoritmayı aşağıda gösterilen iki veri seti ile test ettiler:
 - a. Durum 1: $n = 10$
 - Öğrenci 1 için çalışma süresi: 1
 - Öğrenci 2 için çalışma süresi: 1/100
 - Öğrenci 3 için çalışma süresi: 1/1000
 - b. Durum 2: $n = 100$
 - Öğrenci 1 için çalışma süresi: 10
 - Öğrenci 2 için çalışma süresi: 1
 - Öğrenci 3 için çalışma süresi: 1

Her algoritmanın verimliliği nedir? Hangisi en iyisidir? Hangisi en kötüsüdür? En iyi algoritmanın en iyi çalışma süresine sahip olduğu minimum test durumu sayısı (n) nedir?
21. İki matris çarpabiliriz eğer birinci matrisin sütun sayısı ikinci matrisin n satır sayısı ile aynıysa. $m \times n$ matrisini $n \times k$ matrisine çarpan bir algoritma yazın.
22. İki dizeyi karşılaştırmak için bir karşılaştırma fonksiyonu yazın (bkz. Program 1-6).

Problemler

23. Bir telefon numarasını aramak için bir sözde kod algoritması yazın.
24. Bir şirketteki tüm çalışanlara %3.2 oranında yaşam maliyeti maaş artışı vermek için bir sözde kod algoritması yazın. Maaş dosyasının tüm mevcut çalışanları içerdiğini varsayın.

25. Problem 24'teki sözde kod algoritması için dil-spesifik bir uygulama yazın.
26. Bir ders kitabı veri yapısı için bir sözde kod tanımı yazın.
27. Bir öğrenci veri yapısı için bir sözde kod tanımı yazın.

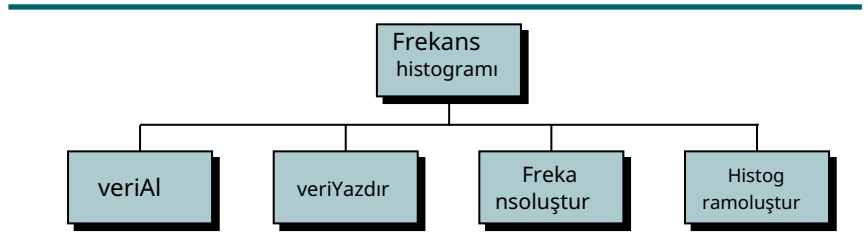
Projeler

28. Kolej kitapçınız, yeni bir ders kitabı envanter sistemi tasarlamanız için sizi ya z stajyeri olarak işe aldı. Aşağıdaki ana süreçleri içermesi gerekmektedir:

- a. Ders kitaplarını sipariş etme
- b. Ders kitaplarını alma
- c. Perakende fiyatını belirleme
- d. Kullanılmış ders kitaplarının fiyatlandırılması
- ve. **Elimizdeki miktarı belirleme**
- f. Ders kitabı satışlarını kaydetme
- g. Ders kitabı iadelerini kaydetme

Envanter sistemi için soyut veri türü algoritma başlıklarını yazın. Her başlık, ad, parametreler, amaç, ön koşullar, son koşullar ve dönüş değer türlerini içermelidir. Analiziniz gerektirdiği takdirde ilave algoritmalar ekleyebilirsiniz.

29. Bir sayısal puanı harf notuna dönüştüren bir algoritma için sözde kodu yazın. Notlandırma ölçeği, %90 veya daha fazlasının A, %80 ile %89'un B , %70 ile %79'un C ve %60 ile %69'un D olduğu tipik mutlak ölçeklerdir. %60'ın altındaki her şey F'dir.
30. Bir tam sayı alan ve ardından tam sayıdaki basamak sayısını ve basamakları arın toplamını yazdıran bir algoritma için sözde kodu yazın. Örneğin, 12,3 45 verildiğinde, 5 basamak olduğunu ve toplamın 15 olduğunu yazdırır.
31. 1 ile 20 arasındaki veri değerleri için bir frekans dizisi oluşturan ve ardından histogramını yazdıran bir program için sözde kodu yazın. Veriler bir dosyadan okunacaktır. Programın tasarımı Şekil 1-17'de gösterilmiştir.



ŞEKİL 1-17 Frekans Histogramı Programı Tasarımı

Alt algoritmaların her biri aşağıda açıklanmıştır.

- a. veriAl algoritması dosyayı okur ve verileri bir dizide saklar.
- b. veriYazdır algoritması dizideki verileri yazdırır.
- c. makeFrequency algoritması, dizideki verileri birer birer inceler ve veri değerine göre frekans dizisindeki karşılık gelen elemana 1 ekler.
- d. makeHistogram algoritması, her bir elemanın oluşumu için yıldızlar kullanarak dikey bir histogram yazdırır. Örneğin, verilerde beş tane 1 ve sekiz tane 2 varsa, şunu yazdırır:
1: *****
2: *****

32. Program 1-4'ü yeniden yazarak bir düğüm listesi oluşturun. Her düğüm iki alandan oluşur. İlk alan, bir öğrenci kimliği (tam sayı) ve bir not ortalaması (ondalık) içeren bir yapıya işaret eden bir işaretçidir. İkinci alan bir bağlantıdır. Veriler bir metin dosyasından okunacaktır.

Sonra, en az 10 öğrencinin bulunduğu bir dosyayı okumak ve yazdığınız fonksiyonu test etmek için bir program yazın. Programınızda Program 1-6'daki genel karşılaştırma kodunu da kullanmanız gerekecek.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 2

Özyineleme

Genel olarak, tekrarlayan algoritmalar yazmanın iki yaklaşımı vardır. Biri yinelemeyi kullanır; diğeri ise özyinelemeyi kullanır. Özyineleme, bir algoritmanın kendisini çağırdığı tekrarlayan bir süreçtir. Ancak, bazı eski dillerin özyinelemeyi desteklemediğini unutmayın.

Bu bölümde özyinelemeyi inceliyoruz. Klasik bir özyinelemeli durumu - faktöriyel - inceleyerek başlıyoruz. Özyinelemenin nasıl çalıştığını açıkladıktan sonra, özyinelemeli algoritmalar geliştirmek için bazı ilkeler geliştiriyoruz ve ardından bunları Fibonacci sayıları gibi başka bir özyinelemeli vaka çalışması geliştirmek için kullanıyoruz. Özyinelemenin teorisini, klasik bir özyinelemeli algoritma olan Hanoi Kuleleri ile bir tartışma ile sonlandırıyoruz. Son bölümde, Fibonacci sayıları, ön ekten son ek dönüşümü ve Hanoi Kuleleri için C uygulamaları geliştiriyoruz.

2.1 Faktöriyel - Bir Vaka Çalışması

Basit bir örnekle başlamak için, faktöriyel hesaplamasını ele alalım. Pozitif bir sayının faktöriyeli, 1'den sayıya kadar olan tam sayı değerlerinin çarpımıdır. Bu tanım Şekil 2-1'de gösterilmektedir.

$$\text{Faktöriyel (n)} = \begin{cases} 1 & \text{eğer } n = 0 \\ n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1 & \text{eğer } n > 0 \end{cases}$$

ŞEKİL 2-1 İteratif Faktöriyel Algoritması Tanımı

Bu tanımın iteratif olduğunu unutmayın. Tekrarlayan bir algoritma, tanım yalnızca algoritma parametrelerini içerdiğinde ve algoritmanın kendisini içermediğinde iteratif olarak tanımlanır. Şekil 2-1'i kullanarak $\text{factorial}(4)$ değerini şu şekilde hesaplayabiliriz:

$$\text{factorial}(4) = 4 \times 3 \times 2 \times 1 = 24$$

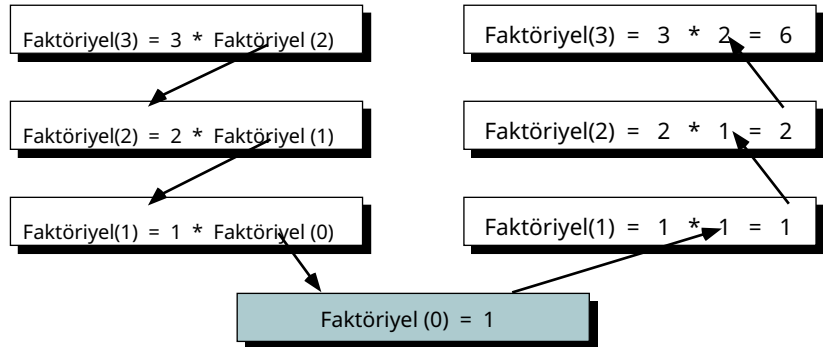
Özyineleme Tanımlandı

Tekrarlayan bir algoritma, algoritmanın tanımın içinde görüldüğü her durumda özyineleme kullanır. Örneğin, faktöriyel algoritması, Şekil 2-2'de gösterildiği gibi özyinelemeli olarak tanımlanabilir.

$$\text{Faktöriyel}(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \times (\text{Faktöriyel}(n - 1)) & \text{eğer } n > 0 \end{cases}$$

ŞEKİL 2-2 Özyinelemeli Faktöriyel Algoritma Tanımı

Şekil 2-2'yi kullanarak $\text{factorial}(3)$ 'ün ayrıştırılması Şekil 2-3'te gösterilmiştir. Şekil 2-3'ü dikkatlice incellerseniz, bir problemin özyinelemeli çözümünün iki yönlü bir yolculuk içerdiğini göreceksiniz: önce problemi üstte n alta ayrıştırıyoruz, sonra alttan üste çözüyoruz.



ŞEKİL 2-3 Faktöriyel (3) Özyinelemeli

Bu örneğe bakarak, özyinelemeli hesaplamaların çok daha uzun ve zor görüldüğü anlaşılıyor. Peki, neden özyinelemeli yöntemi kullanmak isteyelim? Özyinelemeli hesaplama, kullanıldığında daha zor görüne de

kağıt ve kalem, bilgisayarlar kullandığımızda genellikle çok daha kolay ve şık bir çözüm sunar. Ayrıca, yaratıcı ve okuyucu için kavramsal bir basitlik sağlar.

Özyineleme, bir algoritmanın kendisini çağırdığı tekrarlayıcı bir süreçtir.

İteratif Çözüm

Faktöriyel problemini iteratif olarak çözmek için bir algoritma yazalım. Bu çözüm genellikle Algoritma 2-1'de gösterilen bir döngü kullanmayı içerir.

ALGORİTMA 2-1 İteratif Faktöriyel Algoritması

```

Algoritma iteratifFaktöriyel (n)
  Bir döngü kullanarak bir sayının faktöriyelini hesaplar.
  Ön n faktöriyel olarak yükseltilecek sayıdır
  Son n! döndürülür
1  i'yi 1 olarak ayarla
2  factN'yi 1 olarak ayarla
3  döngü (i <= n)
    1 factN'yi factN * i olarak ayarla
    2 i'yi artır
4  döngüyü sonlandır
5  factN'yi döndür
son iterativeFactorial
  
```

Özyinelemeli Çözüm

Şimdi aynı algoritmayı özyinelemeli olarak yazalım. Özyinelemeli çözüm bir döngüye ihtiyaç duymaz; özyineleme kendisi tekrardır. Özyinelemeli versiyonda, faktöriyel algoritmasını n kendisini çağırmasına izin veriyoruz, her seferinde farklı bir parametre seti ile. Özyinelemeli faktöriyel algoritması Algoritma 2-2'de gösterilmiştir.

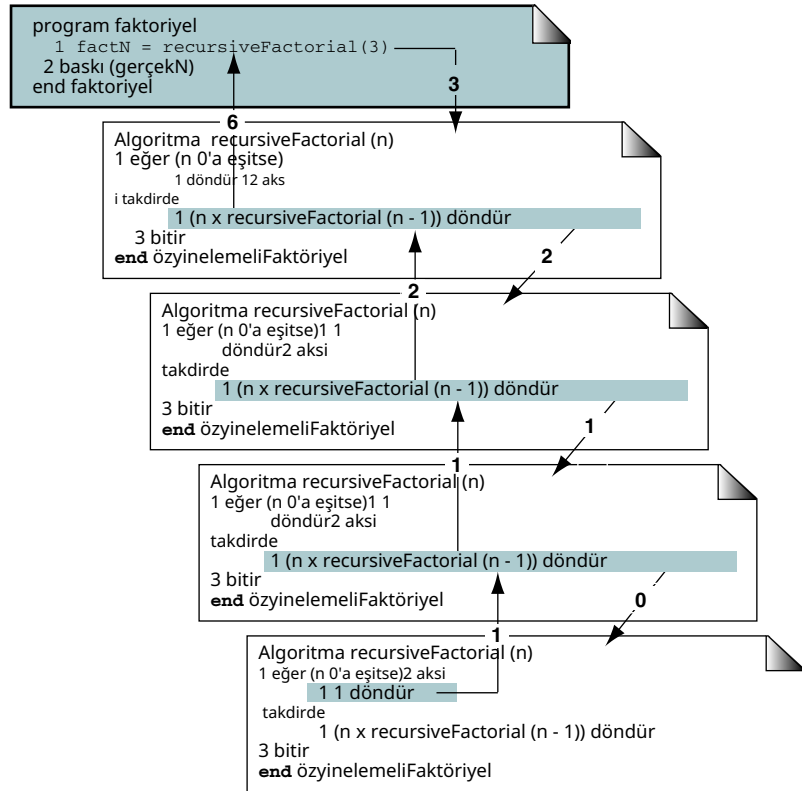
ALGORİTMA 2-2 Özyinelemeli Faktöriyel

```

Algoritma recursiveFactorial (n)
  Bir sayının faktöriyelini özyineleme kullanarak hesaplar.
  Ön n faktöriyel olarak yükseltilecek sayıdır
  Son n! döndürülür
1  eğer (n 0'a eşitse)
    1 1 döndür
2  aksi takdirde
    1 return (n * özyinelemeliFaktöriyel (n - 1))
3  eğer son
end özyinelemeliFaktöriyel
  
```

Algoritma 2-2 AnaliziEğer faktöriyelinin yinelemeli ve özyinelemeli versiyonlarını karşılaştırırsanız, özyinelemeli versiyonun kodunun ne kadar daha basit olduğunu hemen fark edersiniz. Öncelikle, burada bir döngü yok. Özyinelemeli versiyon, ya 1 değerini ya da birinin faktöriyel çağrısı olduğu iki değer çarpımını döndüren basit bir seçim ifadesinden oluşur.

Şekil 2-4, her bir bireysel çağrının özyinelemesini ve parametrelerini izler.



ŞEKİL 2-4Özyinelemeli Bir Algoritmanın Çağrılması

2.2 Özyinelemeli Algoritmalar Tasarlama

Özyinelemenin nasıl çalıştığını gördüğümüze göre, şimdi bir özyinelemeli algoritma tasarlama adımlarına odaklanalım. Öncelikle temel tasarım metodolojisine bakacağız , ardından özyinelemenin sınırlamalarını tartışacağız ve son olarak başka bir özyinelemeli algoritma tasarlayıp uygulayacağız.

Tasarım Metodolojisi

Eğer tüm varsayımsal olarak mümkün olan özyinelemeli algoritmaları inceleysek, hepsinin iki unsuru olduğunu göreceğiz: her çağrı ya *bir kısmını çözer*

problem ya da problemi küçültür problem boyutunu. Algoritma 2-2 ifade 1.1 problemin küçük bir parçasını çözer—faktöriyel(0) 1'dir. İfade-ment 2.1 ise, problemi $n - 1$ ile özyinelemeli olarak çağırarak küçültür. Faktöriyel ($n - 1$) çözümü bilindiğinde, ifade-ment 2.1 genel probleme bir çözüm parçası sağlayarak bir değer döndürür.

2.1 numaralı ifadede gördüğümüz gibi, çözümün genel kısmı, kendini çağıran rekürsif çağrıdır: 2.1 numaralı ifade, problemi çözmek için kendisini çağırır. Bunu Şekil 2-4'te de görüyoruz. Her rekürsif çağrıda, problemin boyutu, 3'ün faktöriyelinden, 2'ye, 1'e ve nihayetinde 0 faktöriyele kadar azaltılır.

Problemi “çözen” ifade, temel durum olarak bilinir. *Herrekürsif algoritmanın bir temel durumu olmalıdır.* Algoritmanın geri kalanı genel durum olarak bilinir. Faktöriyel örneğimizde, temel durum faktöriyel (0); general durum ise $n \times$ faktöriyel ($n - 1$)'dir. Genel durum, problemin boyutunu azaltmak için gereken mantığı içerir.

Her özyinelemeli çağrı, ya problemin bir kısmını çözmeli ya da problemin boyutunu azaltmalıdır.

Faktöriyel probleminde, temel duruma ulaşıldığında, çözüm başlar. Artık cevabın bir kısmını biliyoruz ve bu kısmı bir sonraki, daha genel ifadeye döndürebiliriz. Böylece, Algoritma 2-2'de, faktöriyel(0)'ın 1 olduğunu biliyoruz ve bu değeri döndürüyoruz. Bu, bir sonraki genel durumu çözmemizi sağlar.

faktöriyel(1) $\Rightarrow 1 \times$ faktöriyel(0) $\Rightarrow 1 \times 1 \Rightarrow 1$

Artık faktöriyel(1) değerini daha genel duruma, faktöriyel(2)'ye döndürebiliriz, bunun olduğunu biliyoruz.

faktöriyel(2) $\Rightarrow 2 \times$ faktöriyel(1) $\Rightarrow 2 \times 1 \Rightarrow 2$

Her genel durumu sırayla çözdükçe, bir sonraki daha yüksek genel durumu çözme yeteneği ne sahip oluyoruz, ta ki en genel durumu, orijinal problemi nihayet çözüme kavuşturana kadar.

Bu bölümün amacına dönecek olursak, şimdi bir özyinelemeli algoritma tasarlamak için kuralları belirtmeye hazırız:

1. Öncelikle, temel durumu belirleyin.
2. Sonra genel durumu belirleyin.
3. Temel durumu ve genel durumları bir algortmada birleştirin.

Temel durumu ve genel durumu bir algortmada birleştirirken, mantığa dikkat etmemiz gerekir. Her çağrı, problemin boyutunu azaltmalı ve onu temel duruma yönlendirmelidir. Temel duruma ulaşıldığında, özyinelemeli algortmaya bir çağrı olmadan sona ermelidir; yani, bir dönüş gerçekleştirmelidir.

Özyinelemenin Sınırlamaları

Bu bölümde özyineleme hakkında yalnızca kısa bir açıklama yaptık. Özyineleme, algoritmanın doğal olarak özyinelemeyi destekleyen bir veri yapısı kullandığında en iyi şekilde çalışır. Örneğin, Bölüm 6'da ağaçları inceleyeceğiz. Ağaçlar doğal olarak özyinelemeli bir yapıdır ve özyineleme onlarla iyi çalışır.

Diğer durumlarda algoritma doğal olarak özyinelemeye uygundur. Örneğin, ikili arama algoritması (Bölüm 13'e bakın) doğal bir özyinelemeli algoritmaya uygundur, ayrıca bu bölümde daha sonra tartışacağımız Hanoi Kuleleri algoritması da öyle. Öte yandan, tüm döngü algoritmaları özyineleme ile uygulanamaz veya uygulanmamalıdır, aşağıda tartıştığımız gibi.

Özyinelemeli çözümler, çağrılar kullandıkları için geniş bir ek yük (hem zaman hem de bellek) içerebilir. Her çağrı çalıştırmak için zaman alır. Bu nedenle, bir özyinelemeli algoritma genellikle özyinelemeli olmayan uygulamasından daha yavaş çalışır.¹

Şu sorulardan herhangi birinin yanıtı hayırsa özyineleme kullanmamalısınız:

1. Algoritma veya veri yapısı doğal olarak özyinelemeye uygun mu?
2. Rekürsif çözüm daha kısa ve daha anlaşılır mı?
3. Rekürsif çözüm kabul edilebilir zaman ve alan sınırları içinde mi çalışıyor?

Bölüm 3'te açıklayacağımız nedenlerden dolayı, her çağrı yaptığımızda bellek tahsisatımızdan bir kısmını kullanırız. Eğer özyineleme derin ise—yani, birçok özyinelemeli çağrı varsa—bellek tükenebiliriz.

Zaman ve bellek yükü nedeniyle, faktöriyel gibi algoritmalar büyük sayılar söz konusu olduğunda iteratif olarak geliştirilmelidir. Genel bir kural olarak, özyinelemeli algoritmalar yalnızca verimlilikleri logaritmik olduğunda kullanılmalıdır.

Tasarım Uygulaması—Ters Klavye Girişi

Özyinelemeli algoritmaların tasarım metodolojisini ve sınırlamalarını inceledikten sonra, kavramları pratiğe dökmeye hazırız. Klavyeden veri okuduğumuzu ve verileri ters sırayla yazdırmamız gerektiğini varsayalım.

Listeyi ters sırayla yazdırmanın en kolay yolu, özyinelemeli bir algoritma yazmaktır.

Listeyi ters sırayla yazdırmak için önce tüm verileri okumamız gerektiği açıktır. Bu nedenle, temel durum, son veriyi okuduğumuzdur. Benzer şekilde, genel durum bir sonraki veriyi okumaktır. Soru şu: Ne zaman yazdırıyoruz? Tüm verileri okumadan önce yazdırsak, listeyi sırayla yazdırırız. Son veriyi okuduktan sonra listeyi yazdırsak—yani, özyinelemeyi geri çıkarken yazdırsak—listeyi ters sırayla yazdırırız. Kod, Algoritma 2-3'te gösterilmiştir.

1. Bugünün çoğu derleyicisi, mümkün olduğunda kodu optimize eder. Rekürsiyon bir fonksiyonun sonunda gerçekleştiğinde (kuyruk rekürsyonu olarak bilinir), optimize edilmiş bir derleyici rekürsif kodu basit bir döngüye dönüştürerek fonksiyon çağrısı verimsizliğini ortadan kaldırır.

ALGORİTMA 2-3Ters Yazdır

```

Algoritma printReverse (veri)
  Klavyeden verileri ters sırayla yazdır.
    Ön hiçbir şey
    Son veriler ters sırayla yazdırıldı
  1  eğer (girişin sonu)
    1  geri dön
  2  eğer son
  3  veri oku
  4  tersYazdır (veri)
  Girişin sonuna ulaşıldı: düğümleri yazdır
  5  verileri yazdır
  6  geri dön
  son  tersYazdır

```

Algoritma 2-3 Analizi

Algoritma 2-3'ü incelerken, ifade 5'in girişi bitirene kadar yürütülemeyeceğini unutmayın. İfade 4'ten hemen sonra yürütülmez çünkü ifade 4 bir rekürsif çağrıdır. İfade 5'e yalnızca ifade 1.1'den döndüğümüzde veya algoritmanın sonunda son ifadeden döndüğümüzde ulaşırız.

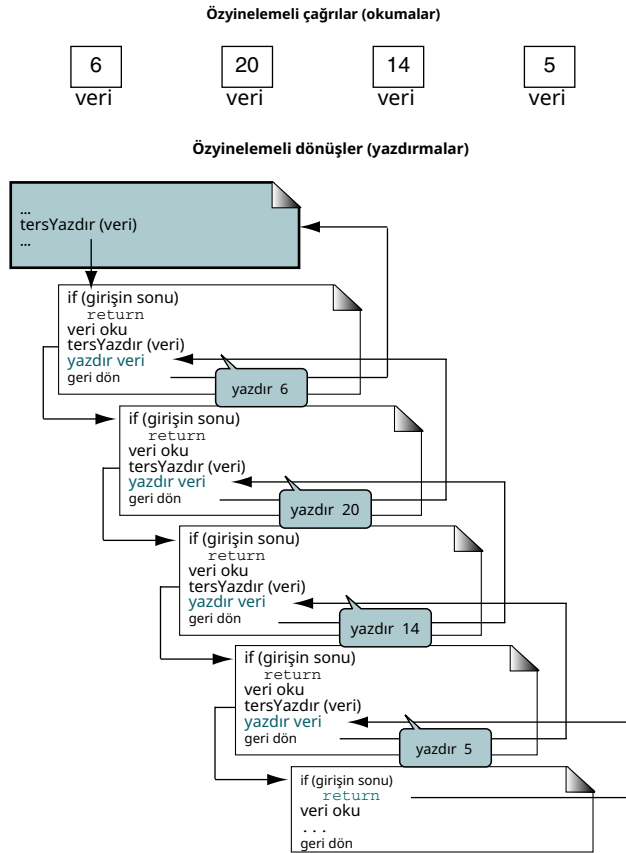
Şekil 2-5, Algoritma 2-3'ün yürütülmesini izler.

Bu algoritma ayrıca rekürsiyondaki yerel değişkenlerin kullanımını gösterir. Sözde k odda yalnızca bir değişken, veri, olmasına rağmen, algoritmayirekürsif bir dilde uyguladığımızda, bilgisayar her çağrı için ayrı bir yerel değişken seti oluşturur. Bu değişkenler, onları yaratan çağrının geri dönüşü gerçekleşene kadar bellekte tutulur ve o zaman geri döndüğünde geri dönüştürülür.

Artık algoritmayı tasarladığımıza göre, gerçekten iyi bir çözüm olup olmadığını belirlemek için onu analiz etmemiz gerekiyor; yani, özyinelemeli algoritma özyineleme için iyi bir aday mı? Bu algoritmayı analiz etmek için, "Özyinelemenin Sınırlamaları" bölümünde geliştirdiğimiz üç soruya döneceğiz.

1. Algoritma veya veri yapısı doğal olarak özyinelemeye uygun mu? Klavyede okunan veriler gibi bir liste, doğal olarak özyinelemeli bir yapı değildir. Ayrıca, algoritma doğal olarak özyinelemeye uygun değildir çünkü logaritmik bir algoritma değildir.
2. Özyinelemeli çözüm daha kısa ve daha anlaşılır mı? Bu sorunun cevabı evet.
3. Rekürsif çözüm kabul edilebilir zaman ve alan sınırları içinde mi çalışıyor? Bir listenin gezintisindeki yineleme sayısı oldukça büyük olabilir çünkü algoritmanın doğrusal bir verimliliği vardır; yani, $O(n)$ şeklindedir.

Bu nedenle, üç sorudan ikisinin cevabının hayır olduğunu görüyoruz. Bu nedenle, algoritmayı özyinelemeli olarak başarıyla yazabilirsek de, bunu yapmamalıyız. Özyineleme için iyi bir aday değildir ve yinelemeli bir döngü olarak daha iyi uygulanır.



ŞEKİL 2-5Klavye Girişini Ters Yazdır

2.3 Özyinelemeli Örnekler

Bu bölümde dört özyinelemeli örneği tartışıyoruz: en büyük ortak bölen, Fibonacci sayıları, ön ekten son ek dönüşümü ve Hanoi Kuleleri. Her örneğe bir tasarımla başlıyoruz ve ardından bunu C dilinde uyguluyoruz.

En Büyük Ortak Bölen

Ortak bir matematiksel işlev, iki sayı için en büyük ortak böleni (EBOB) belirlemektir. Örneğin, 10 ve 25 için en büyük ortak bölen 5'tir.

GCD Tasarımı

İki pozitif tam sayı arasındaki en büyük ortak böleni belirlemek için Öklid algoritmasını kullanıyoruz. İki tam sayı, a ve b verildiğinde, en büyük ortak bölen, Şekil 2-6'daki formül kullanılarak özyinelemeli olarak bulunur.

$$\text{gcd}(a, b) = \begin{cases} a & \text{eğer } b = 0 \\ b & \text{eğer } a = 0 \\ \text{gcd}(b, a \bmod b) & \text{aksi takdirde} \end{cases}$$

ŞEKİL 2-6 En Büyük Ortak Bölgenin Özyinelemeli Tanımı

Öklid algoritması için sözde kod tasarımı, Algoritma 2-4'te gösterilmiştir.

ALGORİTMA 2-4 En Büyük Ortak Bölge için Öklid Algoritması

Algoritma $\text{gcd}(a, b)$
Öklid algoritmasını kullanarak en büyük ortak böleni hesaplar.

Önce a ve b pozitif tam sayılardır, 0'dan büyüktür.
Sonuç en büyük ortak bölen döndürülür.

- 1 eğer (b 0'a eşitse)
- 1 return a
- 2 eğer son
- 3 eğer (a 0'a eşitse)
- 2 dönüş b
- 4 eğer son
- 5 gcd 'yi döndür ($b, a \bmod b$)

gcd 'nin sonu

GCD C Uygulaması

GCD algoritmasının uygulaması, kullanıcıdan iki sayı isteyen bir sürücü kullanır. Sayıları düzenledikten sonra, en büyük ortak böleni döndüren özyinelemeli bir uygulamayı çağırır.

PROGRAM 2-1 GCD Sürücüsü

```
123456 /* Bu program iki sayının en büyük ortak bölenini belirler.
        Yazarı:
        Tarih:
        */
#include <stdio.h>
```

devam ediyor

PROGRAM 2-1 GCD Sürücüsü (devam)

```

789 #include <ctype.h>

// Prototip Bildirimleri
101 int gcd (int a, int b);

12 int main (void)
13 {
14 // Yerel Bildirimler
15     int gcdSonucu;

16 // İfadeler
17     printf("GCD Algoritmasını Test Et");gcdS

18     onucu = gcd (10, 25);
19     printf("10 ve 25'in EBOB'u %d", gcdResult);printf("Test
20     in Sonu");
21     0 değerini döndür;
22 }// ana
23 /* ===== gcd =====
24     Euklides algoritmasını kullanarak en büyük ortak böleni
25     hesaplar.
26     Önce a ve b 0'dan büyük pozitif tam sayılardır
27     Sonuç en büyük ortak bölen döndürülür
28 */
29 int gcd (int a, int b)
30 {
31     // İfadeler
32     if (b == 0)
33         return a;
34     if (a == 0)
35         return b;
36     gcd'yi döndür (b, a % b);
37 } // gcd

```

Sonuçlar:

GCD Algoritmasını Test Et
 10 ve 25'in EBOB'u 5
 Testin Sonu

Fibonacci Sayıları

Fibonacci sayıları İtalyan matematikçi Leonardo Fibonacci'den adını almıştır on üçüncü yüzyılın başlarında yaşamıştır. Bu seride her sayı, önceki iki sayının toplamıdır. Fibonacci serisindeki ilk birkaç sayı şunlardır

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Tasarım

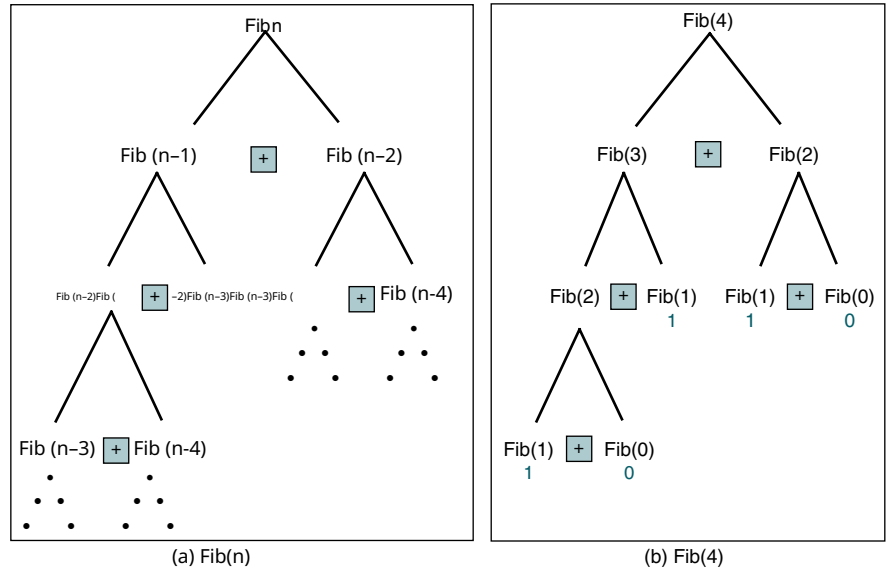
Bir Fibonacci serisine başlamak için, ilk iki sayıyı bilmemiz gerekiyor. Önce ki örnekten görebileceğiniz gibi, bunlar 0 ve 1'dir. Rekürsiyon hakkında ko nuştüğumuz için, bu iki sayıyı temel durumlar olarak tanımalısınız.

Fibonacci serisinin tanımını Şekil 2-7'de gösterildiği gibi genelleşti rebiliriz.

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{eğer } n = 0 \\ 1 & \text{eğer } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{aksi takdirde} \end{cases}$$

ŞEKİL 2-7 Fibonacci Sayıları Rekürsif Tanımı

Fibonacci(4)'ün genelleştirilmesi Şekil 2-8'de gösterilmektedir. Şekil 2-8(a), Fib onacci(4)'ün bileşenlerini genel bir notasyon kullanarak göstermektedir. Şekil 2-8(b), serideki sayıları üretmek için çağrılacak bileşenleri göstermektedir.



ŞEKİL 2-8 Fibonacci Sayıları

Fibonacci C Uygulaması

Fibonacci programımızda, kullanıcıdan kaç sayı gerektiğini sorarak başlıyoruz. Daha sonra, Fibonacci fonksiyonunu belirtilen sayıda çağıran bir döngüsü kullanıyoruz, döngüde her seferinde serideki bir sonraki sayıyı yazdırıyoruz.

C dilinde Fibonacci serisinin uygulanmasında (Program 2-2), algoritmadaki kodun büyük kısmının, generasyon için gerekli sayıların okunması ve sonuçların yazdırılması için ana kod olduğunu belirtmek ilginçtir. Aslında, özyinelemeli algoritmada yalnızca üç ifade vardır.

PROGRAM 2-2 Özyinelemeli Fibonacci Serisi

123456789	/* Bu program bir Fibonacci serisi yazdırır. Yazarı: Tarih: */ #include <stdio.h> // Prototip İfadeleri long fib (long num);
10111213141516171819202122232425262728293031323334353637	int main (void) { // Yerel Bildirimler int seriesSize = 10; // İfadeler printf("Fibonacci serisini yazdır.");for (int loop = 0; loop < seriesSize; loop++) { if (ilmek yapıcı % 5) printf(" %8ld", fib(loop)); else printf("%8ld", fib(loop)); } // for printf(""); 0 değerini döndür; }// ana /* ===== fib ===== nth Fibonacci sayısını hesaplar Önce num Fibonacci sayısını tanımlar Sonra nth Fibonacci sayısını döndürür */ uzun fib (uzun sayı) { // İfadeler eğer (num == 0 num == 1)

devam ediyor

PROGRAM 2-2 Özyinelemeli Fibonacci Serisi (devamı)

38394041	<pre>// Temel Durum num'u döndür; dönüş (fib (sayı - 1) + fib (sayı - 2)); } // fib</pre>
Sonuçlar: Bir Fibonacci serisi yazdırın. 0, 1, 1, 2, 3 5, 8, 13, 21, 34	

Program 2-2 Analizi Eğer kodu takip etmekte zorluk çekiyorsanız, Şekil 2-8'e bakın ve ilk dört Fibonacci sayısını izleyin. İlk 10 sayı algoritmanın sonunda görüntülenir. Eğer çok hızlı bir bilgisayarınız yoksa, 30 ile 35'ten fazla sayı hesaplamaya ça lışmamanızı öneririz.

Fibonacci(5) hesaplamak için kaç çağrı gerekir? Cevap 15'tir. Gördüğünüz gi bi, Tablo 2-1'den, hesapladığımız Fibonacci sayısının boyutunu artırdıkça çağrı s ayısı hızla artmaktadır.

fib(n)	Çağrılar	yalan(n)	Çağrılar
1	1359	11	287
2		12	465
3		13	753
4		14	1219
5	15	15	1973
6	25	20	21,891
7	41	25	242,785
8	67	30	2,692,573
9	109	35	29,860,703
10	177	40	331,160,281

TABLO 2-1 Fibonacci Çağrıları

Tablo 2-1, büyük sayılar için Fibonacci sayılarını hesaplamak üzere bir özyinelemeli çözümün v erimsiz olduğu sonucuna ulaşmamıza yol açıyor.

Önek ile Sonek Dönüşümü

Bir aritmetik ifade üç farklı formatta temsil edilebilir: infiks, sonek ve önek. infiks notasyonunda, operatör iki

operandın arasında gelir, dilbilgisi dersinde öğrendiğimiz cebirsel notasyonun temel formatıdır.

ökl. postfix notasyonunda,² operatör iki operandının ardından gelir ve ön ek notasyonu, iki operandın önünde gelir. Bu formatlar aşağıda gösterilmiştir.

Önek: + A B
İnfiks: A + B
Postfiks: A B +

Bazı yüksek seviyeli diller infiks ve postfix notasyonlarını kullansa da, bu ifadeler doğrudan değerlendirilemez. Bunun yerine, ifadelerin hangi sırayla değerlendirileceğini belirlemek için analiz edilmelidir. Onları analiz etmenin bir yöntemi, özyineleme kullanmaktır.

Önek notasyonunda, operatör operandlardan önce gelir.

İnfiks notasyonunda, operatör operandların arasında gelir.

Postfiks notasyonunda, operatör operandlardan sonra gelir.

Tasarım

Bu bölümde, özyineleme kullanarak ön ek ifadelerini postfix formatına dönüştürüyoruz.³

Algoritmalarla bakmadan önce, temel kavramı gözden geçiriyoruz. Aşağıdaki ön ek ifadesi verildiğinde

*AB

işlemi (*) operandların (A ve B) arkasına taşıyarak dönüştürüyoruz. İfade nin postfix formatı aşağıda gösterilmiştir.

AB*

Algoritmayı mümkün olduğunca basit tutmak için, her operandın yalnızca bir karakter olduğunu ve yalnızca dört operatör olduğunu varsayıyoruz: toplama (+), çıkarma (−), çarpma (*) ve bölme (/).

Belirtilenlere göre, bir ön ek ifadesini dönüştürmek için operatörünü bulmalı ve onu iki operandın arkasına taşımalıyız. Tanıma göre, operatör her zaman ön ek dizisindeki ilk karakterdir. Operatörün ardından iki operandı gelir. Olarak

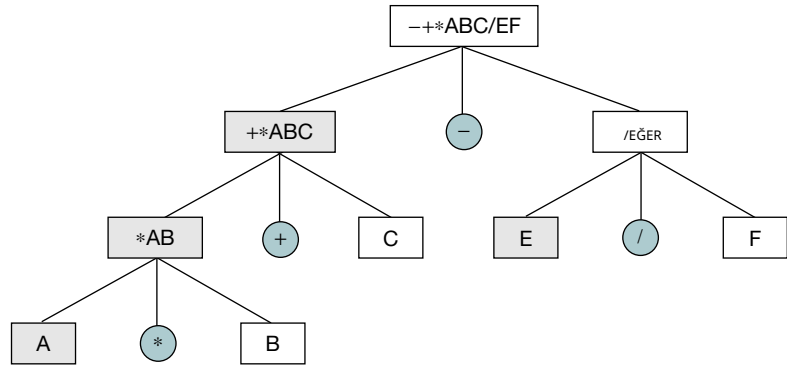
2. Postfix notasyonu, kökenini onurlandırmak için ters Polonya notasyonu (RPN) olarak da bilinir, Polonyalı mantıkçı Jan Lukasiewicz'in.

3. Bu probleme, 3. Bölümde yığınları incelediğimizde geri döneceğiz.

İfade karmaşıklık açısından büyüdüğünde, bir ifadede birden fazla operatör ve operand olabilir. Aşağıdaki örnek ifadesini düşünün, ki bunu tartışmalarımızda kullanacağız.

$$-+*ABC/EF$$

Bu ifade, bir araya getirilmiş birkaç ikili ifadeden oluşan bir karmaşık ifadedir. Bunu ayrıştırmak için soldan başlayıp sağa doğru çalışırız ve bir ikili ifadeyi izole edene kadar devam ederiz, bu durumda $*AB$. Basit bir örnek ifade elde ettiğimizde, bunu postfix hale dönüştürebilir ve çıktıya koyabiliriz. Şekil 2-9 bütün sürecin bir ayrıştırmasını içermektedir. $*AB$ 'yi izole eden adım soldadır.



ŞEKİL 2-9 Parçalama $-+*ABC/EF$

Özyinelemeli bir algoritma tasarlanmanın ilk adımı, temel durumu belirlemektir. Bu nedenle soru, "Temel durum nedir?"dır. Açıkça, bu bir operatör bulmak değildir. O kadar açık olmasa da, temel durum bir operand bulmak olarak ortaya çıkmaktadır—örneğin, alfabe'deki herhangi bir karakterdir.

Operand başka bir ifadeyi içerebileceğinden, genel durum, ikili ifadede operatörü ve sol ve sağ operandları bulmaktır. Bunları tek bir postfix ifade'ne birleştiriyoruz.

Prefix ifadesindeki sol ve sağ operandları bulmak için, bir prefix ifadesinin uzunluğunu belirleyen ikinci bir algoritmaya ihtiyacımız var. Bir ifadenin uzunluğu, ilk alt ifadenin uzunluğu ile ikinci ifadenin uzunluğunun toplamı artı operatör için 1'dir. Bu algoritma için ayrıca özyineleme kullanıyoruz. Her operand bir karakter uzunluğunda olduğundan, temel durumumuz bir operand bulmaktır.

Artık algoritmaları tasarlamaya hazırız. Algoritma 2-5, postfix ifadesini oluşturmak için sözde kodu içermektedir.

ALGORİTMA 2-5 Önek İfadesini Postfix'e Dönüştür

```

Algoritma preToPostFix (preFixIn, postFix)
Bir preFix dizisini postFix dizisine dönüştür.
    Ön preFix geçerli bir preFixIn ifadesidir
        postFix dönüştürülmüş ifade için referanstır
    Son postFix dönüştürülmüş ifadeyi içerir
1  eğer (preFixIn uzunluğu 1 ise)
    Temel durum: bir karakter dizesi bir operatördür
    1  postFix'i preFixIn olarak ayarla
    2  geri dön
2  eğer son
    Eğer bir operatör değilse, bir operatör olmalıdır
3  operatörü preFixIn'in ilk karakterine ayarla
    İlk ifadeyi bul
4  lengthOfExpr'i findExprLen (preFixIn ilk karakterden daha az) olarak ayarla
5  temp'i substring(preFixIn[2, lengthOfExpr]) olarak ayarla
6  preToPostFix (temp, postFix1)
    İkinci postFix ifadesini bul
7  temp'i preFixIn[lengthOfExpr + 1, string'in sonu] olarak ayarla
8  preToPostFix (temp, postFix2)
    Postfix ifadelerini ve operatörü birleştir
9  postFix'i postFix1 + postFix2 + operatör olarak ayarla
10 dön
end preToPostFix

```

Algoritma 2-5 Analizi Algoritma oldukça basittir. En zor kısım önek ifadesini bulmaktır. Uzunluğunu belirleyerek ve ardından geçici bir stringde saklayarak buluyoruz. İfade 5'teki notasyon, ilk konumdan başlayıp ikinci konuma kadar olan alt dizeyi `temp`'ye atayacağını belirtmektedir. Bir önek ifadesini tanımladıktan sonra, onu postfix'e dönüştürmek için Algoritma 2-5'i özyinelemeli olarak çağırıyoruz.

Şimdi, ön ek ifadesinin uzunluğunu belirleyen algoritmaya bakalım. Da ha önce belirtildiği gibi, bu algoritmanın ifadenin uzunluğunu döndürmesi gerekiyor. Yine, temel durum bir operatör bulmaktır. Problemin tanımından, her operatörün bir karakter uzunluğunda olduğunu biliyoruz. Bu nedenle, bir ön ek ifadesinin minimum uzunluğu üçtür: bir operatör için `n` ve her bir operatör için bir tane. İfadeyi özyinelemeli olarak parçalayarak, toplam uzunluğu elde edene kadar birleşik ifadelerinin boyutunu eklemeye devam ederiz.

Kod, Algoritma 2-6'da gösterilmiştir.

ALGORİTMA 2-6 Önek İfadenin Uzunluğunu Bul

```

Algoritma findExprLen (exprIn)
Önek ifadesinin uzunluğunu özyinelemeli olarak belirleyin.
    Önce exprIn geçerli bir önek ifadesidir
    Sonra ifadenin uzunluğu döndürülür
1  eğer (ilk karakter operatörse)
    Genel Durum: İlk karakter operatördür
    İlk önek ifadesinin uzunluğunu bulun
    1  len1'i findExprLen (exprIn + 1) olarak ayarla
    2  len2'yi findExprLen (exprIn + 1 + len2) olarak ayarla
2  aksi takdirde
    Temel durum--ilk karakter operand
    1  len1 ve len2'yi 0 olarak ayarla
3  eğer son
4  len1 + len2 + 1 döndür
son findExprLen

```

Algoritma 2-6 Analizi Bir önek ifade veya bir alt dize içeren bir dizinin ilk karakterini inceliyoruz.

Eğer bir operatörse, iki operandının uzunluğunu özyinelemeli olarak ekliyoruz. Algoritma ya 1 (temel durum) ya da önek ifadede bir ifadenin uzunluğunu döndürür (+*ABC Şekil 2-10'da).

Önekten Postfix C Uygulaması

Tasarım tamamlandığında, Program 2-3'ü yazmaya hazırız. Ana hat, basitçe preToPostFix çağırın ve sonuçları yazdıran bir test sürücüsüdür. İki fonksiyon, önceki bölümde geliştirilen sözde kod ile paraleldir. Sonuçlar sonda gösterilmektedir.

PROGRAM 2-3 Önek'ten Postfix'e

```

123456789 /* Önek ifadesini son ek ifadesine dönüştür.
           Yazarı:
           Tarih:
           */
           #include <stdio.h>
           #include <string.h>

           #define OPERATÖRLER "+-*/"

101112131415 // Prototip Bildirimleri
void preToPostFix (char* preFixIn, char* exprOut);
int findExprLen (char* exprIn);

int ana (geçersiz)
{

```

devam ediyor

PROGRAM 2-3Ön ekten Son ek'e (devam ediyor)

```

1617181920212223242526272829303132333435363738394041424344454647484950515253545556575859606162
// Yerel Tanımlar
char preFixExpr[256] = "-+*ABC/EF";
char postFixExpr[256] = "";

// İfadeler
printf("Ön ekten son ek dönüşümüne başla");preToPostFix (preFi

xExpr, postFixExpr);
printf("Ön ek ifadesi: %-s", preFixExpr);printf("Son ek if
adesi: %-s", postFixExpr);printf("Ön ekten son ek dönüşü

mü sona erdi");return 0;

} // ana

/* ===== preToPostFix =====
Ön ek ifadesini son ek formatına dönüştür.
Ön preFixIn, hata/boşluk içermeyen bir dize ön ek ifa
desidir postFix, son ek için dize değişkenidir S
onuç ifadenin dönüştürüldüğü

*/
void preToPostFix (char* preFixIn, char* postFix)
{
// Yerel Tanımlar
char operator [2];
char postFix1[256];
char postFix2[256];
char temp [256];
int lenPreFix;

// İfadeler
if (strlen(preFixIn) == 1) {

    *postFix = *preFixIn;
    *(postFix + 1) = '\0';
    geri dönmek;
} // yalnızca operand ise

*operator = *preFixIn;
*(operator + 1) = '\0';

// İlk ifadeyi bul
lenPreFix = findExprLen (preFixIn + 1);
strncpy (temp, preFixIn + 1, lenPreFix);
*(temp + lenPreFix) = '\0';
PostFix Akışı (temp, postFix 1);

```

devam ediyor

PROGRAM 2-3Ön ekten Son ek'e (devam ediyor)

63646566676869707172737475767778798081828384858687888990919293949596979899100101

```

// İkinci ifadeyi bul
strcpy (temp, preFixIn + 1 + lenPreFix);
preToPostFix (temp, postFix2);

// postFix'e ekle
strcpy (postFix, postFix1);
strcat (postFix, postFix2);
strcat (postFix, operator);

return;
} // preToPostFix

/* ===== findExprLen =====
Bir ifadede ilk alt dize boyutunu belirleyin.
Önce exprIn, ön ek ifadesini içerir
Sonra ifadenin boyutu döndürülür
*/
int findExprLen (char* exprIn)
{
// Yerel Tanımlar
int len1;
int len2;

// İfadeler
if (strcspn (exprIn, OPERATORS) == 0)
// Genel Durum: İlk karakter operatördür
// İlk ifadenin uzunluğunu bul
{
len1 = findExprLen(exprIn + 1);

// İkinci ifadenin uzunluğunu bul
len2 = findExprLen(exprIn + 1 + len1);
} // if

aksi takdirde
// Temel durum--ilk karakter operand
len1 = len2 = 0;
len1 + len2 + 1'i döndür;
} // findExprLen

```

Sonuçlar:

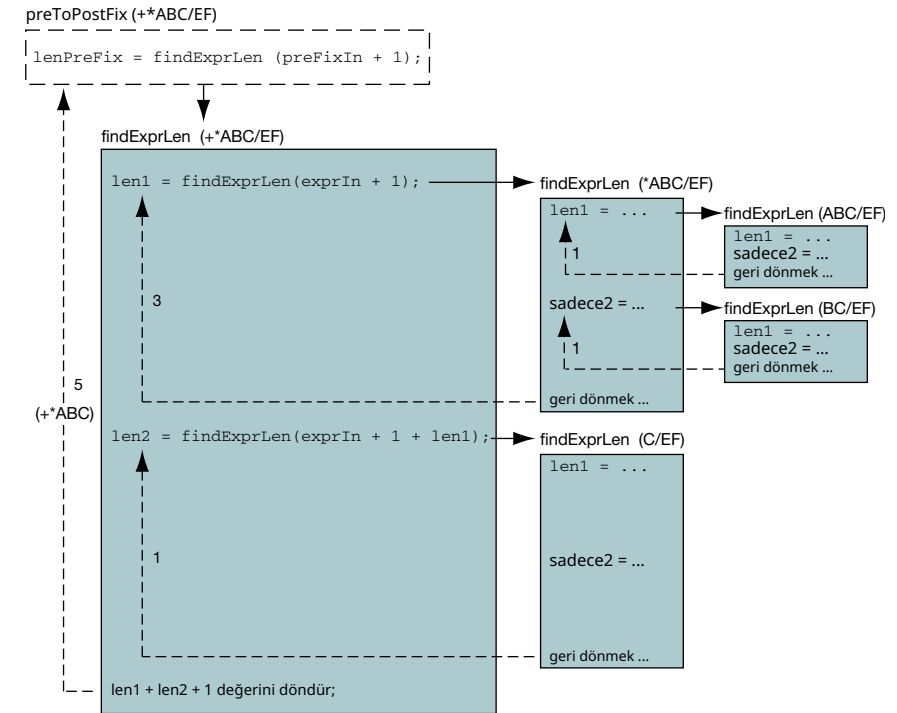
Önekte sonek dönüşümüne başla

Önek ifade: -+*ABC/EF

Postfix ifadesi: AB*C+EF/-

Ön ekten postfix dönüşümünü bitir

Program 2-3 Analizi Program tasarımı oldukça karmaşık ve dikkatli bir inceleme gerektiriyor. Rekürsif çağrılarını Şekil 2-10'da gösterilmiştir. Soldaki ilk ifadeyi belirler(+*ABC) Şekil 2-9'da. Her çağrı, parametre listesinde geçirilen ifadenin bir kısmını gösterir. Orijinal infix ifadesindeki eksi operatörünün çağrıdan önce kaldırıldığını unutmayın



ŞEKİL 2-10 İfade Uzunluğunu Bulmak için Rekürsif Çağrı

İlk karakter bir operatör olduğu için (+), artı operatörünü çıkardıktan sonra rekürsif olarak çağırıyoruz, bırakılan ***ABC/EF**. Yine ilk karakter bir operatör (*) olduğundan, bir kez daha rekürsif olarak çağırıyoruz, bu sefer **ABC/EF** ile. Şimdi ilk karakter bir operand (**A**) olduğundan, fonksiyon ondan çıkıyoruz ve 1 döndürüyoruz. Şimdi ikinci rekürsif çağrıdayız (ifade 95). Yine rekürsif olarak çağırıyoruz, bu sefer ifade **BC/EF** ile, ve bu 1 uzunluğunda bir değer döndürüyor.

Artık ***ABC/EF** ifadesini ayrıştırarak çağrıdan dönüyoruz. Dönüş değeri 3, ilk operand için (A) 1 ve ikinci operand için (B) 1 ve operatör için (*) 1 temsil ediyor.

Değişkenin ****ABC/EF** ayrıştırılmasındaki ikinci özyinelemeli çağrı, ***AB** sonrasındaki ifadeyi geçirir, bu da **C/EF**'yi içerir. İlk karakter bir operand olduğu için, bu bir temel durumdur, fonksiyon sona erer ve 1 döner. Bu, ilk ifadenin, **+*ABC**, uzunluğunu döndürerek izole edilmesini tamamlar, 5. Şekil 2-9'da gösterildiği gibi, işimiz bitmedi. İfadenin ayrıştırılması, tam ön ek ifadesi dönüştürüle ne kadar devam eder.

Hanoi Kuleleri

Hanoi Kuleleri, takip etmesi nispeten kolay, verimli ve karmaşık veri yapıları kullanmayan klasik bir özyineleme problemidir.

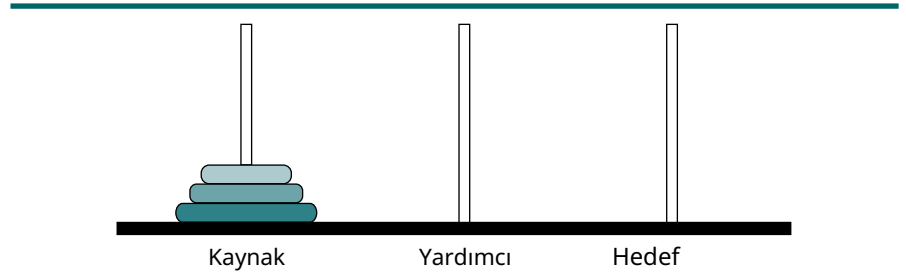
Legende göre, uzak bir dağ manastırındaki rahipler dünyanın ne zaman sona ereceğini tahmin etmeyi biliyorlardı. Üç elmas iğneden oluşan bir setleri vardı. İlk elmas iğnenin üzerinde azalan boyutlarda 64 altın disk yığılmıştı. Rahipler, aşağıdaki kurallara tabi olarak her saat bir disk başka bir iğneye taşıdılar:

1. Sadece bir disk bir seferde taşınabilirdi. Daha büyük bir disk aslaküçük bir diskin üstüne yerleştirilmemelidir.
2. Disklerin ara depolaması için yalnızca bir tane yardımcı iğne kullanılabildi.

Efsaneye göre, tüm 64 disk hedef iğneye aktarıldığında, yıldızlar sönecek ve dünya sona erecekti.

Bugün, tüm diskleri taşımak için $2^{64} - 1$ hareket yapmamız gerektiğini biliyoruz.

Şekil 2-11, yalnızca üç disk ile Hanoi Kulelerini göstermektedir.



ŞEKİL 2-11 Hanoi Kuleleri—Başlangıç Pozisyonu

Bu problem iki nedenle ilginçtir. İlk olarak, özyinelemeli çözüm, genellikle iyi özyinelemeli çözümlerle olduğu gibi, yinelemeli çözümünden çok daha kolay kodlanır. İkinci olarak, çözüm deseni, tartıştığımız basit örneklerden farklıdır. Kulelerin çözümünü incelerken, her temel durumdan sonra, genel durumun bir ayrıştırmasına birkaç adım geri döndüğümü zü not edin. Diğer bir deyişle, problem birkaç alt probleme bölünmüştür ve her biri temel durumla, bir disk hareket ettirerek sona erer.

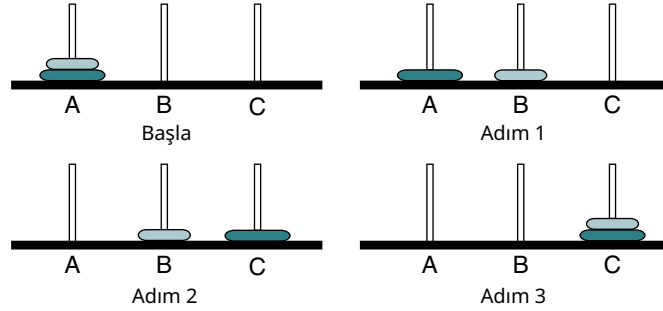
Özyinelemeli Hanoi Kuleleri Tasarımı

Bu problemi çözmek için, hareketleri incelemeliyiz ve bir desen bulup bulamayacağımıza bakmalıyız. Dünyanın sona ermesini istemediğimiz için yalnızca üç disk kullanacağız.

Öncelikle, hareket ettirmemiz gereken sadece bir disk olduğunu hayal edin. Bu çok basit bir durumdur, aşağıdaki sayfada gösterildiği gibi.

Durum 1: Bir disk kaynak iğneden hedef iğneye taşıyın.

Şimdi iki disk hareket ettirmemiz gerektiğini hayal edin. Şekil 2-12, iki disk için adımları izlemektedir.



ŞEKİL 2-12 İki Disk için Kuleler Çözümü

Öncelikle üstteki disk yardımcı iğneye hareket ettiriyoruz. Sonra ikinci diski hedef iğneye hareket ettiriyoruz. Son olarak, ilk disk ikinci diski n üstüne hedef iğneye yerleştiriyoruz. Bu, aşağıda gösterildiği gibi ikinci durumu verir.

Durum 2:1. Bir disk yardımcı iğneye taşıyın.

2. Bir disk hedef iğneye taşıyın.

3. Bir disk yardımcı iğneden hedef iğneye taşıyın.

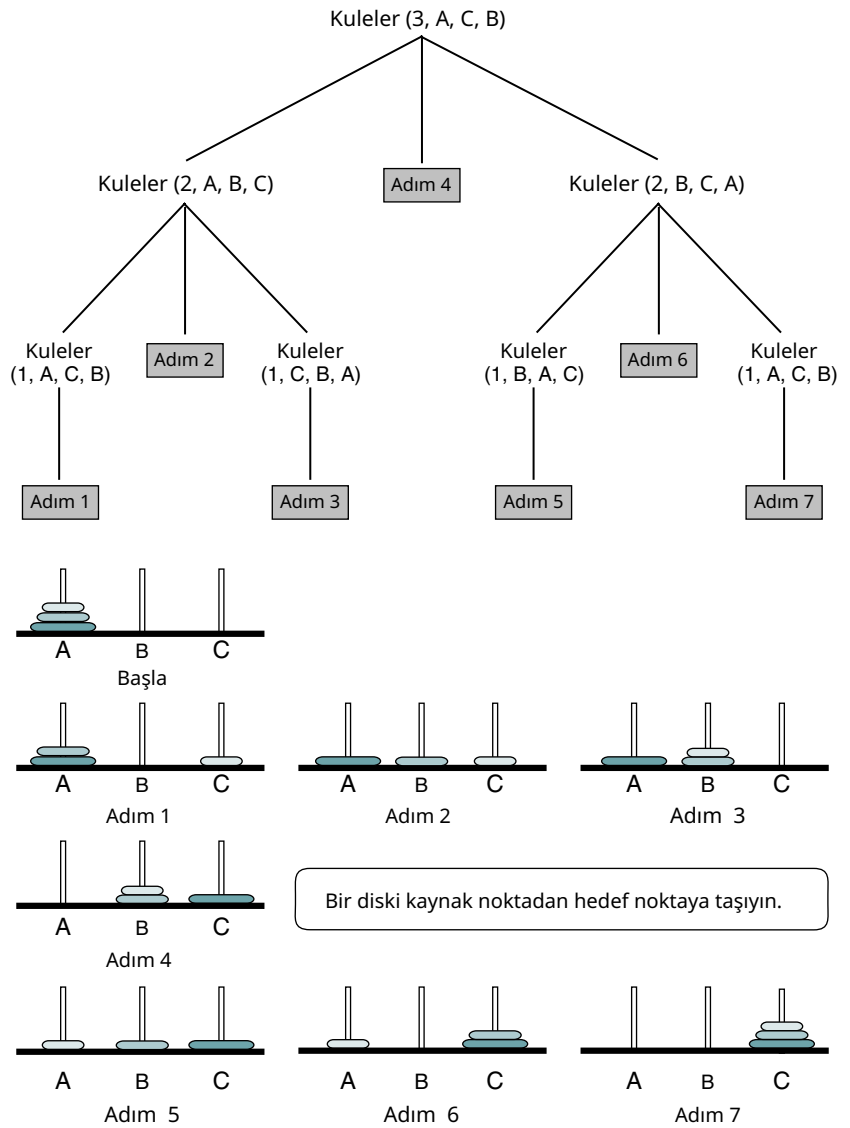
Artık üç disk durumu üzerinde çalışmaya hazırız. Çözümü Şekil 2-13'te gösterilmiştir. İlk üç adım, üstteki iki disk kaynak iğneden yardımcı iğneye taşır. (Bunu nasıl yapacağınızı görmek için Durum 2'ye bakın.) 4. adımda alt disk hedef iğneye taşınır. Artık bir disk yerinde. Bu, Durum 1'in bir örneğidir. Daha sonra, yardımcı iğnedeki iki disk hedef iğneye taşımak için üç adım daha gereklidir.

Bu adımlar aşağıda Durum 3'te özetlenmiştir.

Durum 3:1. İki disk kaynak iğneden yardımcı iğneye taşıyın.

2. Bir disk kaynak iğneden hedef iğneye taşıyın.

3. İki disk yardımcı iğneden hedef iğneye taşıyın.



ŞEKİL 2-13Üç Disk için Kuleler Çözümü

Artık problemi genelleştirmeye hazırız.

1. Kaynaktan yardımcıya $n - 1$ disk taşıyın.

Genel durum

2. Bir disk kaynaktan hedefe taşıyın.

Temel durum

3. Yardımcıdan hedefe $n - 1$ disk taşıyın.

Genel durum

Çözümümüz, taşınacak disklerin sayısı, kaynak iğne, hedef iğne ve yardımcı iğne olmak üzere dört parametreye sahip bir algoritma gerektirir. Yukarıda gösterilen genelleştirmedeki üç hareket, sözde kod kullanılarak

1. Towers (n - 1, kaynak, yardımcı, hedef) çağırısı
2. Bir disk kaynaktan hedefe hareket ettirin
3. Towers (n - 1, yardımcı, hedef, kaynak) çağırısı

Yukarıdaki ikinci çağırıcı dikkatlice inceleyin. İlk diskin hareketini tamamladıktan sonra, kalan diskler yardımcı iğnede bulunmaktadır. Onları yardımcı iğneden hedef iğneye taşımamız gerekiyor. Bu durumda, orijinal kaynak iğne yardımcı iğne haline gelir. (Çağrılan algoritmadaki parametrelerin pozisyonlarının kaynak, hedef ve yardımcı olduğunu unutmayın. Çağırılan algoritma, her çağrı için hangi iğnenin kaynak ve hangi iğnenin hedef olduğunu hatırlamalıdır.)

Artık bu üç çağırıcı bir araya getirebiliriz. Fiziksel hareketler yerine, yapılması gereken hareketleri göstermek için yazdırma ifadeleri kullanıyoruz. Tam sözde kod, Algoritma 2-7'de gösterilmiştir.

ALGORİTMA 2-7 Hanoi Kuleleri

```

Algoritma towers (numDisks, kaynak, hedef, yardımcı)
Diskleri kaynaktan hedefe özyinelemeli olarak hareket ettirin.
    Ön numDisks, taşınacak disk sayısıdır
        kaynak, hedef ve yardımcı kuleler verilmiştir
    Sonra hareketler için adımlar yazdırılır
1  print("Kuleler: ", numDisks, kaynak, hedef, yardımcı)
2  eğer (numDisks 1 ise)
    1  print ("Hareket et ", kaynak, " 'dan ", hedef, " 'e")
3  aksi takdirde
    1  kuleler (numDisks - 1, kaynak, yardımcı, hedef, adım)
    2  yazdır ("Kaydır " kaynak " 'dan " hedef " 'e")
    3  kuleler (numDisks - 1, yardımcı, hedef, kaynak, adım)
4  eğer son
son kuleler
    
```

Algoritma 2-7 Analizi Algoritmaya her girdiğimizde, mevcut parametreleri yazdırıyoruz; bu, hangi kulesinin şu anda kaynak, hangisinin hedef ve hangisinin yardımcı olduğunu takip etmemize yardımcı olur. Kuleler sürekli olarak üçü arasında rollerini değiştirdiğinden, çağrı parametrelerini yazdırma k, onları düz tutmamıza yardımcı olur.

İki ifade hareket talimatlarını yazdırır. İlki ifade 2.1'de yer alır. Bir kulede yalnızca bir disk kaldığında talimatları yazdırır. İkinci hareket talimatı ifade 3.2'de yazdırılır. Daha fazla disk olduğunda kulede bir disk hareket ettirmek için talimatları sağlar ve her geri dönüşte yazdırılır.

İki çağrıdaki parametreleri incelemek önemlidir. İfade 3.1'de kulelerin rollerinin hedeften yardımcıya ve ifade 3.3'te yardımcıdan kaynağa nasıl hareket ettirildiğine dikkat edin.

Algoritmanın çıktısı Şekil 2-14'te gösterilmektedir. Çıktıyı takip etmek için programı izlemek zorunda kalacaksınız.

Calls:	Output:
Towers (3, A, C, B)	
Towers (2, A, B, C)	
Towers (1, A, C, B)	Move from A to C
	Move from A to B
Towers (1, C, B, A)	
	Move from C to B
	Move from A to C
Towers (2, B, C, A)	
Towers (1, B, A, C)	
	Move from B to A
	Move from B to C
Towers (1, A, C, B)	
	Move from A to C

ŞEKİL 2-14 Algoritma 2-7'yi İzleme, Hanoi Kuleleri

Hanoi Kuleleri C Uygulaması

Bir kez daha, rekürsiyonun zarafetini görüyoruz. Sadece 56 satır kod ile ; görece zor bir problemi çözüyoruz. Dahası, tüm işi yapan algoritmanın rekürsif kısmı sadece yedi ifadeden oluşuyor—üç yazdırma ifadesi da hil! Rekürsif algoritmalarla sıkça olduğu gibi, algoritmanın analizi ve tasarımı, onu yazmaktan daha uzun sürüyor.

Kod Program 2-4'te gösterilmektedir. Çıktı Şekil 2-14'te gösterilmektedir.

PROGRAM 2-4 Hanoi Kuleleri

```

1  /* Test Hanoi Kuleleri
2      Yazarı:
3      Tarih:
4  */
5  #include <stdio.h>
6
7  // Prototip İfadeleri
8      void towers (int n,      char source,
9                      char dest, char auxiliary);
10
11 int main (void)
12 {
13     // Yerel Bildirimler
14     int numDisks;
15

```

devam ediyor

PROGRAM 2-4Hanoi Kuleleri (devamı)

```

1617181920212223242526272829303132333435363738394041424344454647484950515253545556
// İfadeler
printf("Lütfen disk sayısını girin: ");
scanf ("%d", &numDisks);

printf("Hanoi Kuleleri'ni başlatın."); towers (numD

isks, 'A', 'C', 'B'); printf("Umarım 64'ü seçmed

iniz "      "ve dünyayı sona erdirmediniz!");

    0 değerini döndür;
} // ana

/* ===== kuleler =====
   Bir diski kaynak noktadan hedef noktaya taşımak için
   yinelemeyi kullanın.
   Önce Kule n diskten oluşur
   Kaynak, hedef ve yardımcı kuleler
   Sonra Taşımlar için adımlar yazdırılır
*/
void kuleler (int n, char kaynak,
              char hedef, char yardımcı)
{
    // Yerel Bildirimler
    statik int adım = 0;

    // İfadeler
    printf("Kuleler (%d, %c, %c, %c)",      n, ka
        kaynak, hedef, yardımcı); if (n == 1)

        printf("Adım %3d: %c'den %c'ye taşı",      ++adım, kaynak,
            hedef);

    aksi takdirde
    {
        kuleler (n - 1, kaynak, yardımcı, hedef); printf("Adım
            %3d: %c'den %c'ye taşı",      ++adım, kaynak, hedef);

        kuleler (n - 1, yardımcı, hedef, kaynak);
    } // eğer ... değilse
    return;
} // kuleler

```


2.4 Anahtar Terimler

temel durum	ek
faktöriyel	postfix
Fibonacci sayıları	prefix
genel durum	özyineleme
en büyük ortak bölen (EBOB)	

2.5 Özet

- Tekrarlayan algoritmalar yazmanın iki yaklaşımı vardır: yineleme ve özyineleme.
- Özyineleme, bir algoritmanın kendisini çağırdığı tekrarlayan bir süreçtir.
- Tekrarlayan bir algoritma, algoritmanın kendisi tanım içinde yer aldığı anda özyinelemeyi kullanır.
- Bir program bir alt yordamı çağırdığında, mevcut modül işlemeyi askıya alır ve çağrılan alt yordam programın kontrolünü devralır. Alt yordam işleme son verdiğinde ve çağrıldığı modüle döndüğünde, modül uyanır ve işlemeye devam eder.
- Bir özyinelemeli algoritmada, her çağrı ya problemin bir kısmını çözer ya da problemin boyutunu azaltır.
- Problemi çözen ifade temel durum olarak bilinir; herözyinelemeli algoritmanın bir temel durumu olmalıdır.
- Özyinelemeli algoritmanın geri kalanı genel durum olarak bilinir.
- Özyinelemeli bir algoritma tasarlamak için genel kural şöyledir:
 1. Temel durumu belirleyin.
 2. Genel durumu belirleyin.
 3. Temel durumu ve genel durumu bir algoritmada birleştirin.
- Aşağıdaki sorulardan herhangi birinin cevabı hayırsa özyineleme kullanmamalısınız:
 1. Algoritma veya veri yapısı özyinelemeye doğal olarak uygun mu?
 2. Özyinelemeli çözüm daha kısa ve daha anlaşılır mı?
 3. Özyinelemeli çözüm kabul edilebilir zaman ve alan sınırları içinde mi çalışıyor?

2.6 Pratik SetlerAlı

ştırmalar

1. Aşağıdaki algoritmayı dikkate alın:

```
algoritma fun1 (x)
1  eğer (x < 5)
    1  dön (3 * x)
2  değilse
    1  dönüş (2 * eglence1 (x - 5) + 7)
3  if sonu
fun1 sonu
```

fun1 çağrıldığında ne dönecektir

- a. eglence1 (4)?
 - b. eglence1 (10)?
 - c. eglence1 (12)?
2. Aşağıdaki algoritmayı dikkate alın:

```
algoritma fun2 (x, y)
1  eğer (x < y)
    1  dönüş -3
2  değilse
    1  dönüş (eglence2 (x - y, y + 3) + y)
3  if sonu
fun2 sonu
```

fun2 çağrıldığında ne dönecektir

- a. eglence2 (2, 7)?
 - b. eglence2 (5, 3)?
 - c. eglence2 (15, 3)?
3. Aşağıdaki algoritmayı dikkate alın:

```
algoritma fun3 (x, y)
1  eğer (x > y)
    1  dönüş -1
2  elseif (x eşit y)
    1  dönüş 1
3  değilse
    1  return (x * fun3 (x + 1, y))
4  viraj sonu
eglencesi3
```

fun3 şu şekilde çağrılırsa ne dönecektir

- eğlence3 (10,4)?
 - eğlence3 (4,3)?
 - eğlence3 (4,7)?
 - eğlenceli3 (0,0)?
4. Bir sayının karekökünü hesaplamak için yöntemlerden biri Newton'un yöntemi'dir. Newton'un yönteminin formülü Şekil 2-15'te gösterilmiştir. Newton'un yönteminin kullanarak bir karekök hesaplamak için bir özyinelemeli algoritmanın sözde kodunu yazın. Algoritmanızı aşağıdaki test durumlarını manuel olarak hesaplayarak doğrulayın: `squareRoot (5, 2, 0.01)` ve `squareRoot (4, 2, 0.01)`. *Not:* formülde, `tol`, *tolerans* için bir kısaltmadır.

kareKök (num, ans, tol) =

$$\left[\begin{array}{ll} \text{ans} & \text{eğer } | \text{ans}^2 - \text{num} | \leq \text{tol} \\ \text{kareKök}(\text{num}, (\text{ans}^2 + \text{num}) / (2 \times \text{ans}), \text{tol}) & \text{aksi takdirde} \end{array} \right]$$

ŞEKİL 2-15Egzersiz 4 için Newton'un Yöntemi

5. Bir seferde k kadar alınan n nesnesinin kombinasyonu, Şekil 2-16'da gösterilen formül kullanılarak özyinelemeli olarak hesaplanabilir. Bu yararlı bir formüldür. Örneğin, bazı eyalet piyangoları oyuncularından bir di zi olası sayıdan altı sayı seçmelerini gerektirir. Bu formül, n nesnesinin olası kombinasyonlarının sayısını, k , hesaplamak için kullanılabilir. Örneğin, 49 sayı için, $C(49, 6)$, yani 13,983,816, farklı altı sayı kombinasyonu vardır. k kadar nesnenin kombinasyonunu hesaplamak için bir özyinelemeli algoritma yazın.

$$C(n, k) = \left[\begin{array}{ll} 1 & \text{if } k = 0 \text{ or } n = k \\ C(n, k) = C(n-1, k) + C(n-1, k-1) & \text{if } n > k > 0 \end{array} \right]$$

ŞEKİL 2-16Egzersiz 5 için Seçim Algoritması

6. Ackerman sayısı, matematiksel mantıkta kullanılan, ŞEKİL 2-17'de gösterilen formül kullanılarak hesaplanabilir. Ackerman sayısını hesaplayan bir özyinelemeli algoritma yazın. Algoritmanızı, aşağıdaki test durumlarını manuel olarak hesaplayarak doğrulayın: `Ackerman(2, 3)`, `Ackerman(2, 5)`, `Ackerman(0, 3)` ve `Ackerman(3, 0)`.

$$\text{Ackerman}(m, n) = \begin{cases} n + 1 & \text{eğer } m = 0 \\ \text{Ackerman}(m - 1, 1) & \text{eğer } n = 0 \text{ ve } m > 0 \\ \text{Ackerman}(m - 1, \text{Ackerman}(m, n - 1)) & \text{aksi takdirde} \end{cases}$$

ŞEKİL 2-17 Problem 6 için Ackerman Formülü

Problemler

7. Bir liste uzunluğunu hesaplayan ve döndüren bir özyinelemeli algoritma yazın.
8. Bir rakamlar dizisini bir tam sayıya dönüştüren bir özyinelemeli algoritma yazın. Örneğin, "43567" 43567'ye dönüştürülecektir.
9. Serinin ilk n elemanlarını toplamak için bir özyinelemeli algoritma yazın.

$$1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/n$$
10. Bir dizinin palindrom olup olmadığını belirlemek için bir özyinelemeli algoritma yazın. Bir dize, ileri ve geri aynı anlamda okunabiliyorsa palindromdur. Büyük harf kullanımı ve boşluklar göz ardı edilir. Örneğin, *annave go dog* palindromlardır. Algoritmanızı aşağıdaki iki palindrom ile ve en az bir palindrom olmayan durumla test edin.

Hanımefendi, ben Adam
Able was I ere I saw Elba
11. Belirtilen bir karakterin bir dizgede olup olmadığını kontrol etmek için özyinelemeli bir algoritma yazın.
12. Belirtilen bir karakterin bir dizgede tüm geçişlerini saymak için özyinelemeli bir algoritma yazın.
13. Belirtilen bir karakterin bir dizgedeki tüm geçişlerini kaldıran özyinelemeli bir algoritma yazın.
14. Bir dizgede bir alt dizgenin tüm geçişlerini bulan özyinelemeli bir algoritma yazın.
15. Bir tam sayıyı ikilik sayıya dönüştüren özyinelemeli bir algoritma yazın.
16. Newton yöntemini kullanarak bir sayının karekökünü hesaplamak için özyinelemeli bir C fonksiyonu yazın. (Bkz. Alıştırma 4.) Fonksiyonunuzu test edin ve 125, 763 ve 997'nin karekökünü yazdırın.
17. Klavyeden bir karakter dizisini okuyup tersine yazdıran özyinelemeli bir algoritma yazın.
18. Bir sepet içindeki toplar gibi n nesnenin, k kadar bir arada alınmasının kombinasyon formülü kullanılarak özyinelemeli olarak hesaplanabilir. (Bkz. Alıştırma 5.) Bir sayıda bulunan kombinasyon sayısını hesaplamak için bir C fonksiyonu yazın. Fonksiyonunuzu test edin ve hesaplayıp yazdırın $C(10, 3)$.

19. Matematiksel mantıkta kullanılan Ackerman sayısı, 6. Egzersizde gösterilen formül kullanılarak hesaplanabilir. Ackerman sayısını hesaplayan bir özyinelemeli C fonksiyonu yazın. Algoritmanızı aşağıdaki test durumlarını hesaplayarak doğrulayın: $Ackerman(5, 2)$, $Ackerman(2, 5)$, $Ackerman(0, 3)$, ve $Ackerman(3, 0)$.
20. Liste elemanlarını ters sırayla yazdıran özyinelemeli algoritma için C fonksiyonunu yazın. (Bkz. Algoritma 2-3, "Ters Yazdır.")

Projeler

21. Eğer bir özyineleme çağrısı algoritmadaki son yürütülebilir ifade ise, buna kuyruk özyinelemesi denir ve kolayca yineleme kullanılarak kaldırılabilir. Kuyruk özyinelemesi, her çağrının dönüş noktasının algoritmanın sonunda olması nedeniyle bu ismi alır. Bu nedenle, her çağrıdan sonra yürütülecek hiçbir ifade yoktur. Bir kuyruk özyinelemesini yinelemeye dönüştürmek için aşağıdaki adımları kullanınız:
 - a. Prosedür çağrısını değiştirmek için bir değişken kullanın.
 - b. Temel durum (veya tamamlayıcı) olarak limit koşulu ile bir döngü kullanın.
 - c. Tüm yürütülebilir ifadeleri döngü içinde kapsayın.
 - d. Özyinelemeli çağrıyı uygun bir atama ifadesine değiştirin.
 - ve. Parametrelere değerleri yeniden atamak için uygun ifadeleri kullanın.
 - f. Adım a'da tanımlanan değişkenin değerini döndürün.

Yinelemeli faktöriyel algoritmasının sürümünü yazın (Algoritma 2-2) ve $factorial(5)$ değerini yazdırarak test edin ve $factorial(20)$.
22. Fibonacci serisi algoritmasının yinelemeli sürümünü, Proje 21'de verilen ipuçlarını kullanarak yazın. Proje 21'deki c adımının, faktöriyel son ifadede iki özyinelemeli çağrı kullandığı için farklı olacağını unutmayın.

Bu sayfa kasıtlı olarak boş bırakılmıştır

İkinci Kısım

Doğrusal Listeler

Bir doğrusal liste , her bir elemanın benzersiz bir ardılı olduğu bir listedir. Doğrusal listenin ardışık özelliği, Şekil II-1'de gösterildiği gibi, tanımının ve kullanımının temelidir.

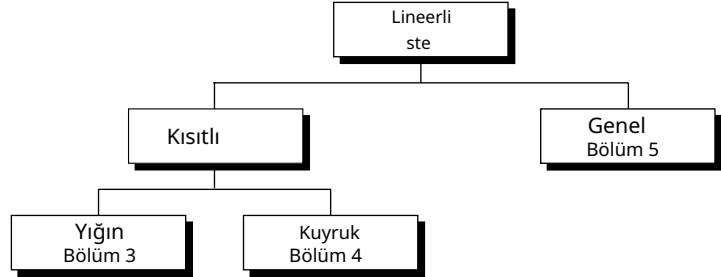


ŞEKİL II-1 Doğrusal Liste

Doğrusal listeler iki kategoriye ayrılabilir: kısıtlı listeler ve genellisteler. Bir kısıtlı listede, verilerin eklenmesi ve silinmesi listenin uçlarıyasınırlıdır. İki kısıtlı liste yapısını tanımlıyoruz: son giren-ilk çıkan (LIFO)liste, yaygın olarak yığın olarak adlandırılır ve ilk giren-ilk çıkan (FIFO) liste,yaygın olarak kuyruk olarak adlandırılır. Yığınları Bölüm 3'te ve kuyruklarıBölüm 4'te tartışıyoruz.

Bir doğrusal listede, her bir elemanın benzersiz bir ardılı vardır.

Genel bir listede, veriler listenin herhangi bir yerine eklenebilir veya silinebilir: başında, ortasında veya sonunda. Genel listeleri Bölüm 5'te tartışıyoruz. Şekil II-2, bu kitapta temsil edilen lineer liste kategorilerini göstermektedir.



ŞEKİL II-2Lineer Liste Kategorileri

Kapsanan Bölümler

Bu bölüm üç bölüm içermektedir:

Bölüm 3: Yığınlar

Bu bölümde, verilerin yalnızca bir uçta eklenip silinebildiği kısıtlı bir lineer liste olan yığını tartışıyoruz.

Bölüm 4: Kuyruklar

Bu bölümde, verilerin yalnızca bir uçta eklenip diğer uçta silinebildiği kısıtlı bir lineer liste olan kuyruğu tartışıyoruz.

Bölüm 5: Genel Lineer Listeler

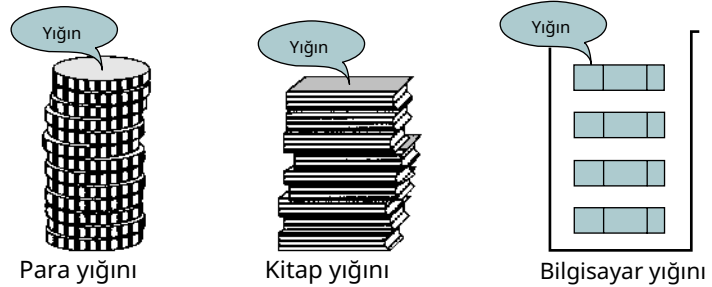
Bu bölümde, verilerin başlangıçta, ortada veya sonda herhangi bir yerde eklenip silinebildiği genel lineer listeleri tartışıyoruz.

Bölüm 3

Yığınlar

Bir yığın, tüm ekleme ve silme işlemlerinin yalnızca bir ucunda, üst olarak adlandırılan bir uca kısıtlandığı doğrusal bir listedir.¹ Eğer bir veri serisini bir yığına eklerseniz ve ardından çıkarırsanız, verilerin sırası tersine döner. {5, 10, 15, 20} olarak girilen veri, {20, 15, 10, 5} olarak çıkar. Bu tersine çevirme özelliği, yığınların son giren-ilk çıkan (LIFO) veri yapısı olarak bilinmesinin nedenidir.

Günlük hayatımızda birçok farklı yığın türü kullanıyoruz. Sıklıkla bir yığın para veya bir yığın tabak hakkında konuşuruz. Üstteki nesneyi yalnızca ekleyip çıkarabileceğiniz her durum bir yığındır. Üstteki nesne dışında herhangi bir nesneyi çıkarmak istiyorsanız, önce üstteki tüm nesneleri çıkarmanız gerekir. Bir yığının grafik temsili Şekil 3-1'de gösterilmiştir.



ŞEKİL 3-1 Yığın

1. Yığın, yapıların uçlarında işlemlerin kısıtlandığı için *kısıtlayıcı veri yapıları* olarak bilinen üç veri yapısından biridir. Diğer ikisi, Bölüm 4'te inceleyeceğimiz kuyruk ve çift uçlu kuyruktur. Bu metinde ele alınmayan çift uçlu kuyruk, her iki uçta da ekleme ve silme işlemlerine izin verir.

Başlığın en üstündeki öğeyi aşağıya taşıma gibi diğer işlemleri gerçekleştirmemizi engelleyen hiçbir şey yoktur, ancak sonuç bir yığın olamayacaktır.

Yığın, tüm ekleme ve silme işlemlerinin yalnızca bir uca, üst olarak adlandırılan bir uca kısıtlandığı son giren ilk çıkar (LIFO) veri yapısıdır.

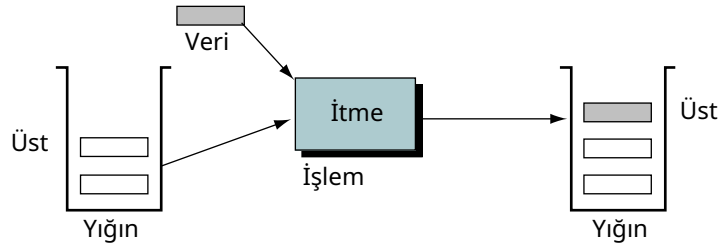
3.1 Temel Yığın İşlemleri

Üç temel yığın işlemi push, pop ve yığın üstüdür. Push, yığına veri eklemek için kullanılır. Pop, bir yığından veriyi kaldırır ve veriyi çağıran modüle geri döndürür. Yığın üstü, veriyi yığından silmeden yığının en üstündeki veriyi döndürür.

İtme

İtme yığın üzerinde bir öğe ekler. İtme işleminden sonra, yeni öğe üstte olur. Bu basit işlemin tek potansiyel sorunu, yeni öğe için yer olduğundan emin olmamızdır. Eğer yeterli yer yoksa, yığın taşma durumundadır ve öğe eklenemez.

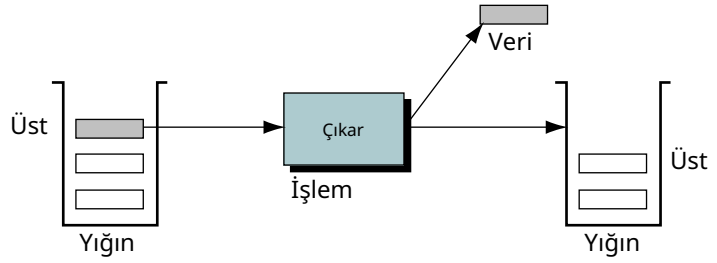
Şekil 3-2, yığın itme işlemini göstermektedir.



ŞEKİL 3-2 Yığın İtme İşlemi

Çıkar

Bir yığının çıkardığımızda, yığının en üstündeki öğeyi kaldırır ve bunu kullanıcıya geri döndürürüz. En üstteki öğeyi kaldırdığımız için, yığındaki bir sonraki daha eski öğe en üstte olur. Yığındaki son öğe silindiğinde, yığın boş durumuna getirilmelidir. Yığın boşken çıkarma işlemi çağrıldığında, taşma durumundadır. Çıkarma yığın işlemi Şekil 3-3'te gösterilmektedir.

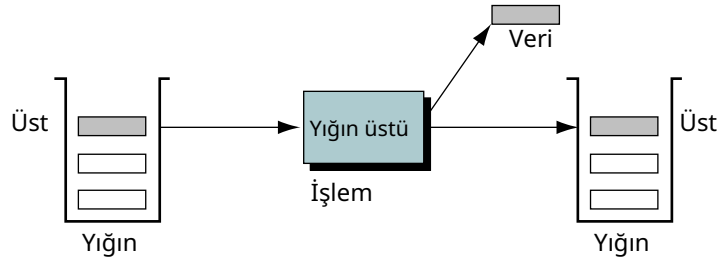


ŞEKİL 3-3Yığın Çıkarma İşlemi

Yığın Üstü

Üçüncü yığın işlemi yığın üstü. Yığın üstü, yığının en üstündeki öğeyi kopyalar; yani, en üstteki elemandaki veriyi kullanıcıya döndürür ancak silmez. Bu işlemi yığın üstünü okumak olarak düşünebilirsiniz. Yığın üstü, yığın boşsa alt akışa da neden olabilir. Yığın üstü işlemi Şekil 3-4'te gösterilmiştir.

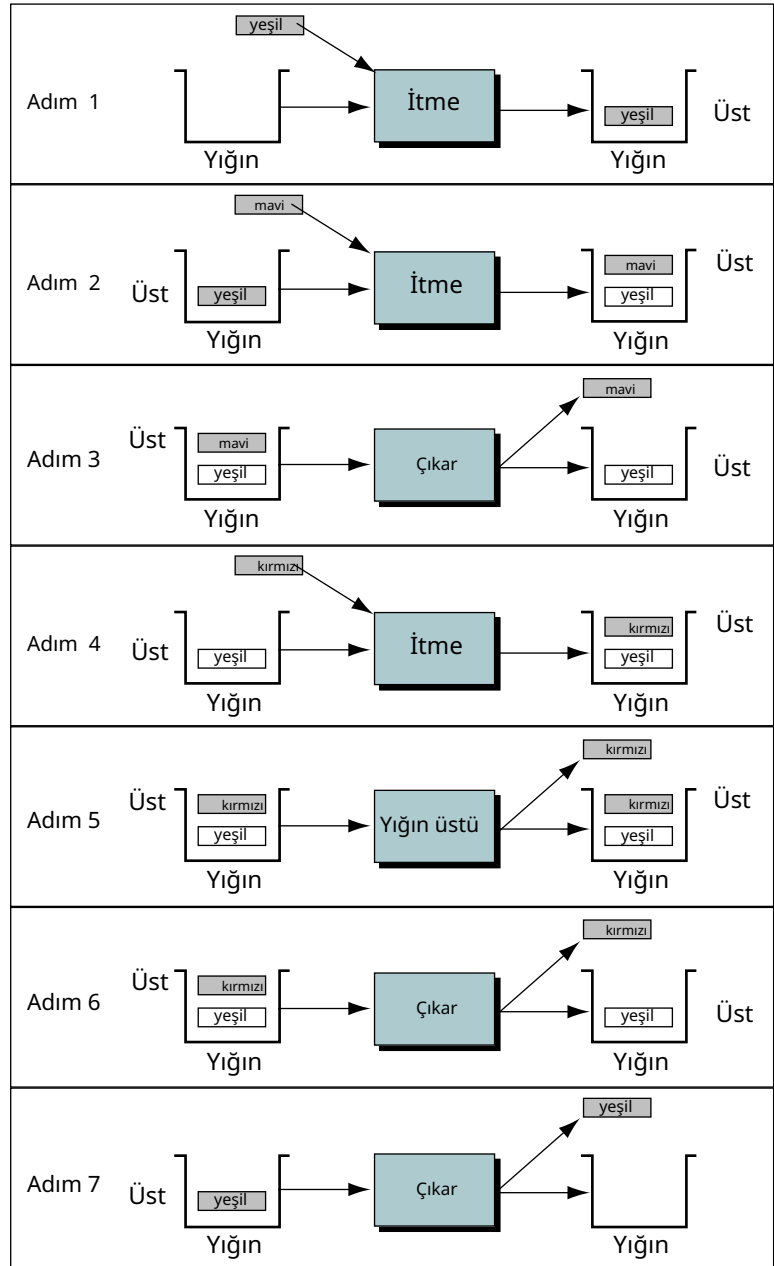
Üç temel yığın işlemi itme, çıkarma ve yığın üstüdür.



ŞEKİL 3-4Yığın Üstü İşlemi

Şekil 3-5, bu üç işlemi bir örnekte izler. Boş bir yığınla başlıyoruz ve yeşil ve mavi öğelerini yığına ekliyoruz. Bu noktada yığın iki giriş içerir. Ardından, yığının en üstündeki mavi öğesini çıkarıyoruz ve yeşil tek giriş olarak kalır. Kırmızı ekledikten sonra, yığın tekrar iki giriş içerir. Bu noktada yığın üstü kullanarak en üstteki girişi, kırmızı, alıyoruz. Yığın üstünün kırmızı öğesini yığından çıkarmadığını unutmayın; hala en üstteki elemandır. Ardından kırmızı öğesini çıkarıyoruz ve yeşil tek giriş olarak kalır. Yeşil çıkarıldığında, yığın tekrar boştur. Bu örneğin, bir yığının son giren ilk çıkar işlemini nasıl gösterdiğine de dikkat edin.

Yeşil ilk itilen olmasına rağmen, en son çıkarılanıdır.



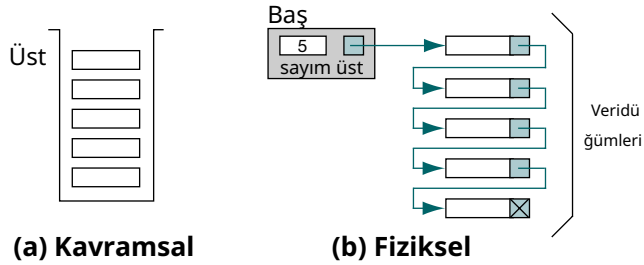
ŞEKİL 3-5 Yığın Örneği

3.2 Yiğın Bağlı Liste Uygulaması

Bir yiğın uygulamak için birkaç veri yapısı kullanılabilir. Bu bölümde yiğın bir bağlı liste olarak uyguluyoruz. Bağlı liste kavramını Bölüm 1, “Bağlı Liste Uygulamaları”nda tanıttık.

Veri Yapısı

Bağlı liste yiğının uygulamak için iki farklı yapıya ihtiyacımız var, bir baş ve bir veri düğümü. Baş yapısı, veri hakkında veri olan meta verileri ve yiğının tepe noktasına bir işaretçi içerir. Veri yapısı, veriyi ve yiğındaki bir sonraki düğüme bir bağlantı işaretçisi içerir. Yiğının kavramsal ve fiziksel uygulamaları Şekil 3-6’da gösterilmiştir.



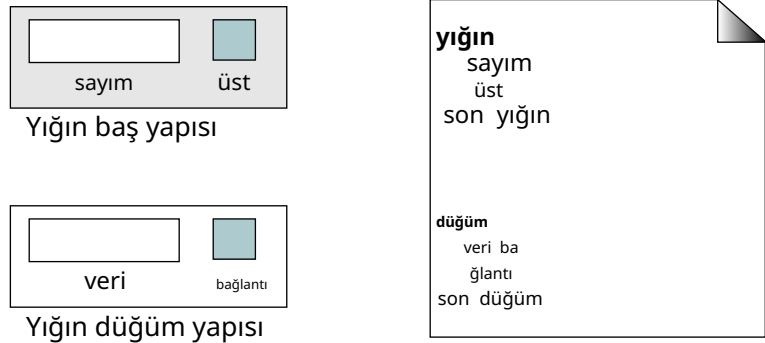
ŞEKİL 3-6 Kavramsal ve Fiziksel Yiğın Uygulamaları

Yiğın Başlığı

Genel olarak, bir yiğın için başlık yalnızca iki niteliğe ihtiyaç duyar: bir üst işaretçi ve yiğında bulunan eleman sayısının bir sayımı. Bu iki öge bir yapıya yerleştirilir. Diğer yiğın nitelikleri de burada yer alabilir. Örneğin, yiğının oluşturulduğu zamanı ve yiğına daha önce yerleştirilen toplam öge sayısını kaydetmek mümkündür. Bu iki meta veri ögesi, kullanıcının belirli bir süre içinde yiğın üzerinden işlenen ortalama öge sayısını belirlemesine olanak tanır. Elbette, bunu yalnızca böyle bir istatistiğin bir nedenle gerekli olması durumunda yaparız. Temel bir baş yapısı Şekil 3-7’de gösterilmiştir.

Yiğın Veri Düğümü

Veri yapısının geri kalanı tipik bir bağlı liste veri düğümüdür. Uygulama, yiğında saklanan verileri belirlese de, yiğın veri düğümü herhangi bir bağlı listede düğümüne benzer. Verinin yanı sıra, diğer veri düğümlerine bir bağlantı işaretçisi içerir, bu da onu kendine referans veren bir veri yapısı yapar. Kendine referans veren bir yapıda, yapının her örneği, aynı yapının başka bir örneğine bir işaretçi içerir. Yiğın veri düğümü ayrıca Şekil 3-7’de gösterilmiştir.



ŞEKİL 3-7Yiğın Veri Yapısı

Yiğın Algoritmaları

Bu bölümde tanımlanan sekiz yiğın işlemi, herhangi bir temel yiğın problemini çözmek için yeterli olmalıdır. Bir uygulama ek yiğın işlemleri gerektiriyorsa, bunlar kolayca eklenebilir. Her işlem için adını, kısa bir tanımını veriyoruz ve ardından an algoritmasını geliştiriyoruz.

Bir yiğının uygulanması, kısmen uygulama diline bağlı olsa da, genellikle C dilinde bir yiğın başı yapısı ile uygulanır. Şekil 3-8'de gösterilen tasarımı kullanıyoruz; bu, en yaygın dört yiğın işlemini gösterir: yiğın oluştur, yiğına ekle, yiğından çıkar ve yiğını yok et. stacktop gibi işlemler, yiğın yapısını değiştirmedikleri için şekilde gösterilmemiştir.

Yiğın Oluştur

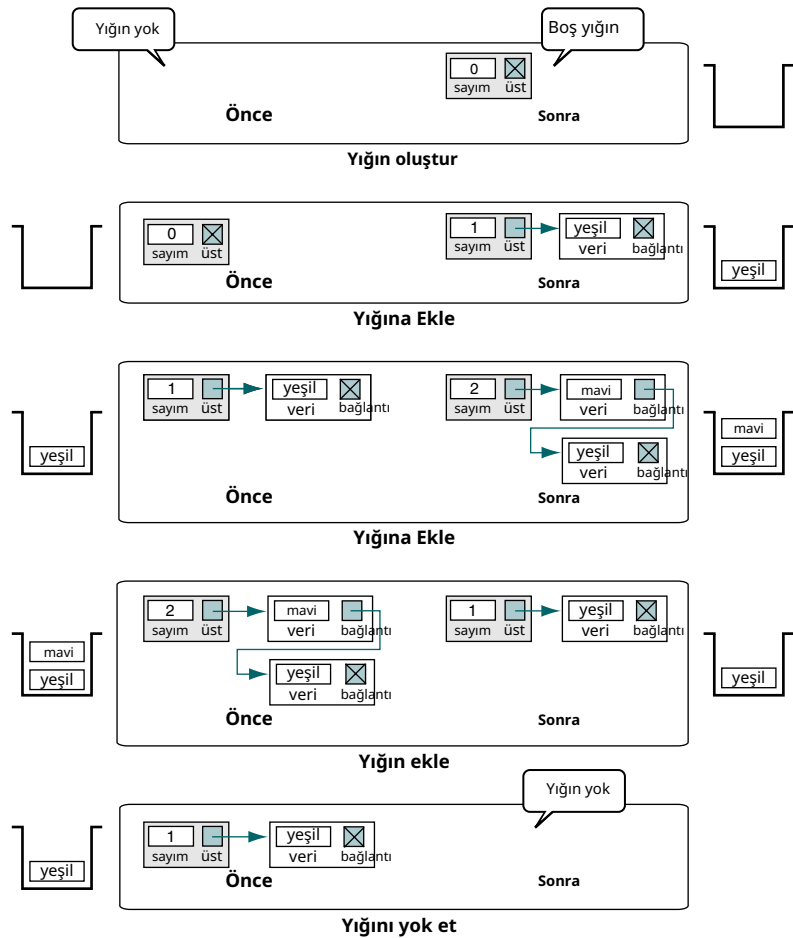
Yiğın yiğın yapısı için bellek ayırır ve meta verilerini başlatır. Pseudocode 3-1'de gösterilmiştir.

ALGORİTMA 3-1Yiğın Oluştur

```

Algoritma yiğınOluştur
Meta veri yapısını oluşturur ve başlatır.
Ön Hiçbir şey
Son Yapı oluşturuldu ve başlatıldı
Yiğın başını döndür
1 yiğın başı için bellek ayır
2 sayacı 0 olarak ayarla
3 üstü null olarak ayarla
4 yiğın başını döndür
son createStack

```

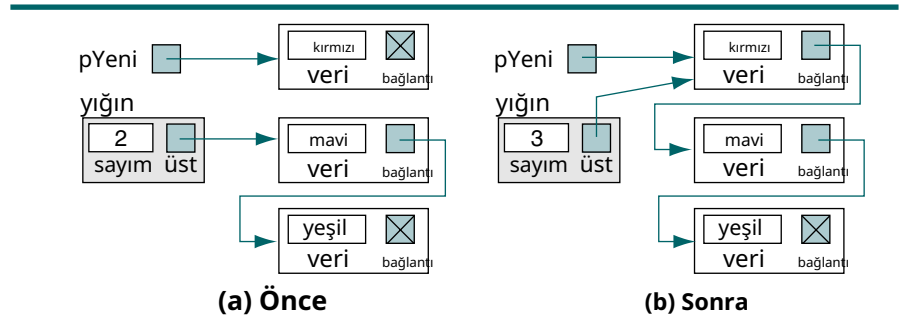


ŞEKİL 3-8 Yığın İşlemleri

Yığına Ekle

Yığına ekleme yığına bir eleman ekler. Yığına veri eklerken yapmamız gereken ilk şey, düğüm için bellek bulmaktır. Bu nedenle dinamik bellekten bir düğüm ayırmalıyız. Bellek ayrıldıktan sonra, veriyi yığın düğümüne atarız ve bağlantı işaretçisini yığın tepe noktasını gösteren düğüme işaret edecek şekilde ayarlarız. Ayrıca yığın tepe işaretçisini güncellemeli ve yığın sayım alanına 1 eklemeliyiz. Şekil 3-9, yığına eklenmesi gereken veriyi tanımlamak için yeni bir işaretçi (p_{New}) kullanıldığı bir yığın ekleme işlemini izler.

Bağlı liste kullanarak ekleme algoritmasını geliştirmek için, üç farklı yığın durumunu analiz etmemiz gerekiyor: (1) boş bir yığına ekleme, (2) verili bir yığına ekleme ve (3) mevcut bellek tükendiğinde yığına ekleme. Bu durumların ilk ikisi Şekil 3-8'de gösterilmektedir. Üçüncüsü bir hata durumudur.



ŞEKİL 3-9Yığın Ekleme Örneği

Bir yığına veri eklediğimizde, yeni düğümün bağlantı işaretçisi, şu anda en üstte bulunan düğüme işaret edecek şekilde ayarlanır ve yığının en üstteki işaretçisi yeni düğüme işaret edecek şekilde ayarlanır. Boş bir yığına ekleme yaptığımızda, yeni düğümün bağlantı işaretçisi null olarak ayarlanır ve yığının en üstteki işaretçisi yeni düğüme işaret edecek şekilde ayarlanır. Ancak, yığının en üstteki işaretçisi null olduğu için, yeni düğümün bağlantı işaretçisini null olarak ayarlamak için bunu kullanabiliriz. Böylece, verili bir yığına ekleme ve boş bir yığına ekleme mantığı aynıdır.

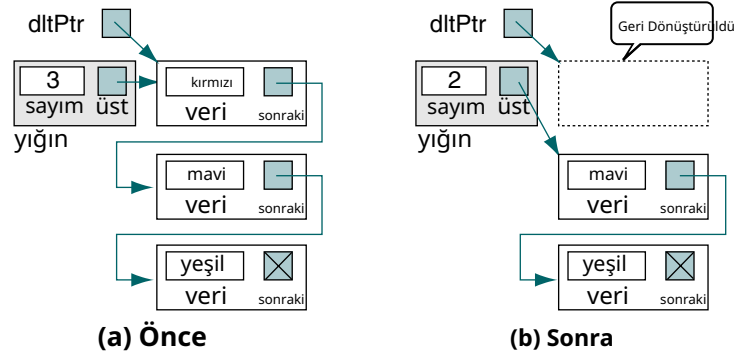
ALGORİTMA 3-2Yığın Tasarımı

```

Algoritma yığınEkle (yığın, veri)
Yığına bir öge ekle (it).
    Ön yığın referansla geçildi
        veri yığına eklenmesi gereken veriyi içerir
    Son veri yığında eklenmiştir
1   yeni düğüm ayır
2   veriyi yeni düğümde sakla
3   mevcut üst düğümü ikinci düğüm yap
4   yeni düğümü üst yap
5   yığın sayısını artır
son   pushStack
  
```

Yığını Çıkar

Yığını çıkarma, yığının en üstündeki düğümdeki veriyi çağırana algoritmaya geri gönderir. Ardından, düğümü mantıksal olarak silmek için işaretçileri ayarlar. Düğüm mantıksal olarak silindikten sonra, fiziksel olarak bellek geri dönüştürülerek silinir, yani dinamik belleğe geri döndürülür. Sayı 1 çıkılarak ayarlandıktan sonra, algoritma durumu çağırana döndürür: eğer çıkarma başarılıysa, true döner; eğer yığın boşsa ve çıkarma çağırılırsa, false döner. Yığını çıkarma işlemleri Şekil 3-10'da izlenmiştir.



ŞEKİL 3-10Yığın Pop Örneği

Pop yığın kodu Algoritma 3-3'te gösterilmiştir.

ALGORİTMA 3-3Yığın Pop

```

Algoritma popStack (yığın, dataOut)
Bu algoritma yığının en üstündeki öğeyi çıkarır ve
onu kullanıcıya geri döndürür.
Ön ffıyığın referansla geçildi
ffdataOut, veri almak için referans değişkenidir
SonffVeriler çağıran algoritmaya geri döndü
Başarılıysa true döndür; underflow ise false döndür
1 eğer (yığın boş)
1 başarıyı false olarak ayarla
2 aksi takdirde
1 dataOut'u üst düğümdeki veriye ayarla
2 ikinci düğümü üst düğüm yap
3 yığın sayısını azalt
4 başarıyı true olarak ayarla
3 eğer sonlandır
4 başarıyı döndür
sonlandır popStack

```

Algoritma 3-3 Analizi Son düğüm silinirken mantığı takip etmek ilginçtir. Bu durumda, sonuç boş bir yığındır. Özel bir mantık gerekmemektedir; ancak, son düğümün null bağlantı işaretçisi olduğu için boş yığın otomatik olarak oluşturulur. Bu işaretçi üstte hareket ettiğinde, yığın n boş olduğunu gösterir. Sayının sıfır olması, sayıyı azalttığımızda otomatik olarak g erçekleşir ve bu da boş bir yığının göstergesidir.

Yığın Üstü

Yığın üstü algoritması (Algoritma 3-4), yığının en üstündeki veriyi çağı ran modüle geri gönderir ve üst düğümü silmez. Bu

uygulamanın bir sonraki pop yığını ile neyin silineceğini bilmesi gerektiğinde dahil edilmiştir.

ALGORİTMA 3-4Yığın Üstü Pseudocode

```

Algoritma stackTop (yığın, dataOut)Bu algori
tma, yığını değiştirmeden yığının tepe noktasından veriyi alır.

Ön   yığın, geçerli bir yığın için meta veri yapısıdır
      dataOut, veri almak için referans değişkenidir
Sonra Veriler çağırılan algoritmaya geri döndürülmüştür
Eğer veri dönerse true, eğer underflow varsa false döner
1  eğer (yığın boş)
    1  başarıyı false olarak ayarla
2  aksi takdirde
    1  dataOut'u üst düğümdeki veriye ayarla
    2  başarıyı true olarak ayarla
3  eğer sonlandır
4  başarıyı döndür
son stackTop

```

Algoritma 3-4 AnaliziYığın üstü için mantık, silme mantığı hariç, pop yığını için olanla neredeyse aynıdır. P op'ta olduğu gibi, algoritmada yalnızca bir potansiyel sorun vardır: yığın boş olabilir. Algoritma başarılı olursa, true döner; eğer yığın boşsa, false döner.

Boş Yığın

Boş yığın veri gizleme yapılandırılmış programlama kavramını uygulamak için sağlanmıştır. Eğer tüm program yığın başı yapısına erişime sahipse, bu gerekli değildir. Ancak, eğer yığın ayrı bir derlenmiş program olarak diğer programları a bağlantı kurmak için uygulanıyorsa, çağırılan program yığın başı düğümüne erişime sahip olmayabilir. Bu durumlarda, yığının boş olup olmadığını belirlemenin bir yolunu sağlamak gereklidir. Boş yığın için pseudocode, Algoritma 3-5'te gösterilmiştir.

ALGORİTMA 3-5Boş Yığın

```

Algoritma emptyStack (yığın)
Yığının boş olup olmadığını belirler ve bir Boolean döner.
Ön yığın, geçerli bir yığın için meta veri yapısıdır
Son yığın durumunu döndürür
Yığın boşsa true, yığın veri içeriyorsa false döndür
1  eğer (yığın sayısı 0 ise)
    1  true döndür
2  aksi takdirde
    1  false döndür
3  eğer sonlandır
son emptyStack

```

Tam Yığın

Tam yığın veri gizleme için başka bir yapılandırılmış programlama uygulamasıdır. Dile bağlı olarak, uygulaması en zor algoritmalardan biri de olabilir. Örneğin, ANSI C, bunu uygulamak için doğrudan bir yol sağlamaz. Tam yığın için pseudocode, Algoritma 3-6'da gösterilmiştir.

ALGORİTMA 3-6 Tam Yığın

```

Algoritma fullStack (yığın)
Yığının dolu olup olmadığını belirler ve bir Boolean döndürür.
  Ön yığın, geçerli bir yığın için meta veri yapısıdır
  Son yığın durumunu döndürür
  Eğer yığın dolu ise true, bellek mevcutsa false döndür
1  eğer (bellek mevcut değilse)
    1  true döndür
2  aksi takdirde
    1  false döndür
3  eğer sonlandır
son fullStack

```

Yığın Sayısı

Yığın sayısı yığında şu anda bulunan eleman sayısını döndürür. Bu, yapılandırılmış programlamanın veri gizleme ilkesinin başka bir uygulamasıdır. Pseudocode, Algoritma 3-7'de gösterilmiştir.

ALGORİTMA 3-7 Yığın Sayısı

```

Algoritma stackCount (stack)
Stack'te şu anda bulunan eleman sayısını döndürür.
  Ön stack, geçerli bir stack için meta veri yapısıdır
  Son stack sayısını döndürür
  Stack'teki eleman sayısının tam sayı olarak döndürülmesi
1  return (stack sayısı)
son stackCount

```

Yığını Yok Et

Yığını yok et bir yığındaki tüm verileri siler. Şekil 3-8, yığını yok etmenin sonuçlarını grafiksel olarak gösterir ve Algoritma 3-8, pseudocode'u içerir.

ALGORİTMA 3-8 Yığını Yok Et

```

Algoritma destroyStack (stack)
Bu algoritma tüm düğümleri dinamik belleğe geri serbest bırakır.
  Ön referansla geçirilen stack
  Son stack boş ve tüm düğümler silindi
1  eğer (stack boş değilse)

```

devam ediyor

ALGORİTMA 3-8Yığın Yok Et (devam ediyor)

```

1  döngü (yığın boş değil)
    1  üst düğümü sil
2  döngüyü sonlandır
2  eğer sonlandır
3  yığın başını sil
son destroyStack

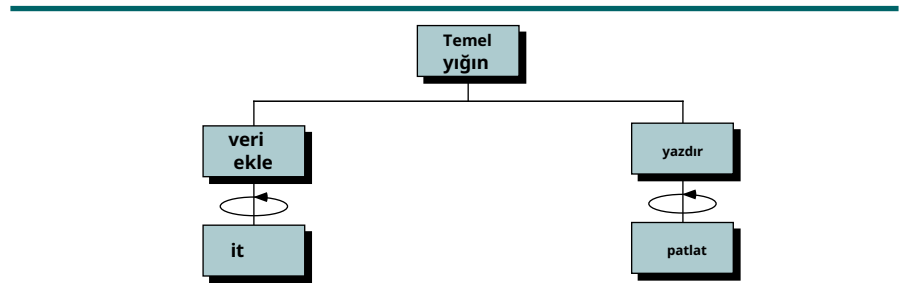
```

Algoritma 3-8 AnaliziBir yığını yok etmenin tek gerekli olduğu durum, artık ona ihtiyacınız olmadığı ve program tamamlanmadığında gerçekleşir. Program tamamlandığında, yığın program sona erdiğinde otomatik olarak temizlenir.

3.3 C Dili Uygulamaları

Daha önce belirtildiği gibi, yığınları uygulamanın iki yaklaşımı vardır. Benzersiz C programları yazabiliriz, bu yazması kolaydır ancak yeniden kullanılabilir değildir, ya da bir soyut veri türü (ADT) oluşturabiliriz. Bu bölümde basit, ancak çok yeniden kullanılabilir olmaya n programlar geliştiriyoruz. Bir sonraki bölümde, yığın ADT'sini geliştireceğiz.

Yığın algoritmalarını göstermek için, bir yığına veri ekleyen bir program yazıyoruz. Basit tutmak için, veriler rastgele büyük harf karakterleridir. Karakterler eklendikten sonra, çıkarılır ve yazdırılır. Yığın boş olduğunda, program sona erer. Yığın başlığı kullanmıyor olsak da, Bölüm 3.2'de tartışılan temel itme ve çıkarma algoritmalarını kullanıyoruz, "Yığın Bağlı Liste Uygulaması." Bu programın tasarımı Şekil 3-11'de gösterilmiştir.



ŞEKİL 3-11 Temel Yığın Programı Tasarımı

Düğüm bildirimi, prototip ifadeleri ve test sürücüsü Program 3-1'de sunulmaktadır.

PROGRAM 3-1 Basit Yığın Uygulama Programı

```

123456789 /* Bu program, yığın itme ve patlatma işlevlerinin temel işlemini
            göstermek için bir test sürücüsüdür.
            Yazarı:
            Tarih:
            */
            #include <stdio.h>
            #include <stdlib.h>
            #include <stdbool.h>

            // Yapı Bildirimleri
            typedef struct node
            {
                char veri;
                struct node* bağlantı;
            } YIĞIN_DÜĞÜM;

            // Prototip Bildirimleri
            void insertData (STACK_NODE** pStackTop);
            void print (STACK_NODE** pStackTop);

            bool push (STACK_NODE** pList, char dataIn);
            bool pop (STACK_NODE** pList, char* dataOut);

            t main (void)
            {
                // Yerel Tanımlar
                STACK_NODE* pStackTop;

                // İfadeler
                printf("Basit Yığın Programına Başlıyor");pStackTop = NUL

                L;
                insertData (&pStackTop);print
                (&pStackTop);printf("Basit Yiğ

                ın Programı Sonu");return 0;

                }// ana

```

Sonuçlar:

Basit Yığın Programına Başlama

Karakterler oluşturuluyor: QMZRH LAJOE

Yığın içerdi: EOJALHRZMQ

Basit Yığın Programı Sonu

Program 3-1 Analizi Programın doğru çalıştığını doğrulamak için, karakterleri üretildikleri gibi yazdırıyoruz. Bu, yığın çıkışının gerçekten doğru olduğunu doğrulamamıza olanak tanır. Bu basit programın, bir yığının LIFO işlemini doğruladığını unutmayın.

Veri Ekle

Veri ekleme fonksiyonu, karakterler oluştururken ve bunları yığına eklerken d öngü yapar. Rastgele karakterler oluşturmak için rastgele sayı üretici kullanıy oruz ve dönen değeri büyük harf aralığına ölçeklendiriyoruz. Bu kod Program 3-2'de geliştirilmiştir.

PROGRAM 3-2Veri Ekle

```

123456789 /* ===== insertData =====
Bu program rastgele karakter verileri oluşturur ve
bunları bağlı liste yığınınına ekler.
Önce pStackTop, ilk düğüme işaret eden bir işaretçidir.
Sonra Yığın oluşturulmuştur.

*/
void insertData (STACK_NODE** pStackTop)
{
// Yerel Tanımlar
Dört aşama;
çok başarılı;

// İfadeler
printf("Karakterler oluşturuluyor: ");
for (int nodeCount = 0; nodeCount < 10; nodeCount++)
{
// Büyük harf karakter oluştur
charIn = rand() % 26 + 'A';
printf("%c", charIn);
başarı = push(pStackTop, charIn);
eğer (!başarı)
{
printf("Hata 100: Bellek Yetersiz"); çıkış (10
0);
}
// eğer } // için
printf("");return;
} // insertData
1011121314151617181920212223242526272829

```

Program 3-2 AnaliziEkleme mantığı basittir. Büyük harflerin üretilmesi biraz açıklama gerektirir. 26 alfabetik karakter olduğu için rastgele sayıyı 0 ile 25 aralığına ölçeklendiriyoruz. Elde edilen rastgele sayıyı büyük harflere eşlemek için, rastgele sayıya karakterin 'A' değerini ekliyoruz.

Ayrıca, bu küçük örnekte bile taşma için test yaptığımızı unutmayın. Taşmanın gerçekleşme olasılığı düşük olsa da, her zaman bunun için test yaparız. Murphy'nin Kanunu'nu hatırlayın.²

2. Murphy'nin Yasası: Eğer bir şey ters gidebiliyorsa, gidecektir.

Yığına Ekle

Veri ekleme fonksiyonu, push fonksiyonunu çağırır. Taşma testi tekniği ve hata işleme kodu ile başlık meta verilerini koruma dışında, bu, Algoritma 3-2'nin doğrudan bir uygulamasıdır. Kod, Program 3-3'te geliştirilmiştir.

PROGRAM 3-3Yığın İt

123456789	/* ===== it ===== Bağlı liste yığınının düğüm ekler. Önce fffpStackTop geçerli yığının işaretçisidir Sonra fffcharIn eklendi Başarı durumunda true döner underflo w durumunda false döner */
1011121314151617181920212223242526	bool push (STACK_NODE** pStackTop, char charIn) { // Yerel Tanımlar STACK_NODE* pNew; bool başarı; // İfadeler Yeni = (STACK_NODE*)malloc(sizeof (STACK_NODE)); if (!pNew) başarı = false; else { Yeni->data = charIn; Yeni->link = *pStackTop; *pStackTop = Yeni; başarı = true; } // else return başarı; } // push

Yığını Yazdır

Yığın oluşturulduktan sonra, çıktıyı doğrulamak için yazdırıyoruz. Yazdırma fonksiyonu, yığın boşalana kadar pop fonksiyonunu çağırır. Yazdırma kodu Program 3-4'te bulunur.

PROGRAM 3-4Yığını Yazdır

1234567	/* ===== yazdır ===== Bu fonksiyon, tek bağlı bir yığın yazdırır. Önce pStackTop geçerli yığının işaretçisidir Sonra yığındaki veriler yazdırılır */ void yazdır (STACK_NODE** pStackTop) {
---------	---

devam ediyor

PROGRAM 3-4Yığın Yazdır (devamı)

```

89 // Yerel Tanımlar
    char printData;

10111213141516 // İfadeler
    printf("Yığın içerdi: ");
    while (pop(pStackTop, &printData))
        printf("%c", printData);
    return;
} // yazdır

```

Karakter Çıkar

Karakter çıkarma fonksiyonu, yığının en üstündeki karakteri ikinci parametrede belirtilen çıkış konumuna kopyalar. Başlık meta verilerini korumak için kullanılan kod hariç, bu, Algoritma 3-3'ün tam bir uygulamasıdır. Uygulama Program 3-5'te gösterilmiştir.

PROGRAM 3-5Pop Yığını

```

1  /* ===== pop =====
2  Bağlı liste yığınından düğüm sil.
3  Önce pStackTop geçerli yığına işaretçidir
4  Sonra charOut silinen veriyi içerir
5  Başarı durumunda true döner      underflo
6  w durumunda false döner
7
8  */
9  bool pop (STACK_NODE** pStackTop, char* charOut)
10 {
11     // Yerel Tanımlar
12     STACK_NODE* pDlt;
13     bool başarı;
14
15     // İfadeler
16     if (*pStackTop) {
17
18         başarı = true;
19         *charOut = (*pStackTop)->data;
20         pDlt = *pStackTop;
21         *pStackTop = (*pStackTop)->bağlantı;
22         ücretsiz (pDlt);
23     } // else
24     else
25         başarı = false;
26         başarıyı döndür;
27 } // pop

```


3.4 Yığın ADT

Önceki bölümdeki algoritmaların her biri birkaç yığın işlemi kullandı. Onları her ihtiyaç duyduğumuzda yazabiliriz, ancak bu kesinlikle zamanımızın verimsiz bir kullanımı olurdu. Bu nedenle, ihtiyacımız olan, bir kütüphaneye koyabileceğimiz ve ihtiyaç duyduğumuzda çağırabileceğimiz bir yığın soyut veri türüdür. Bu bölümde bir yığın soyut veri türü geliştiriyoruz.³

Veri Yapısı

C'de yığın ADT uygulaması basittir. Her düğümde veri depolamak yerine, veriyi bir işaretçi saklarız. Verinin belleğini ayırmak ve adresi yığın ADT'ye iletmek uygulama programının sorumluluğundadır. ADT içinde, yığın düğümü, gerçek veriyi değil, veriyi bir işaretçi içeren herhangi bir bağlı liste düğümüne benzer.

Veri işaretçi türü bilinmediğinden, *void* türünde bir işaretçi olarak saklanır.

Baş düğüm ve veri düğümleri ADT içinde kapsüllenmiştir. Çağırma işlevinin yığın üzerindeki tek görünümü, ADT'deki yığın yapısına işaret eden bir işaretçidir ve bu bir tür tanımlı olarak bildirilmiştir. Bu yığın türüne STACK adını veriyoruz. Bu tasarım, C'nin FILE yapısına çok benzemektedir.

Bir yığın oluşturmak için, programcı aşağıdaki örnekte gösterildiği gibi bir yığının işaretçi tanımları ve ardından yığın oluşturma işlevini çağırır. Yığın yapısının adresi yığın oluşturma tarafından döndürülür ve çağırma işlevindeki işaretçiye atanır.

```
STACK* yığın;
...
yığın = createStack ();
```

Yığın ADT yapısı Şekil 3-12'de bulunmaktadır.

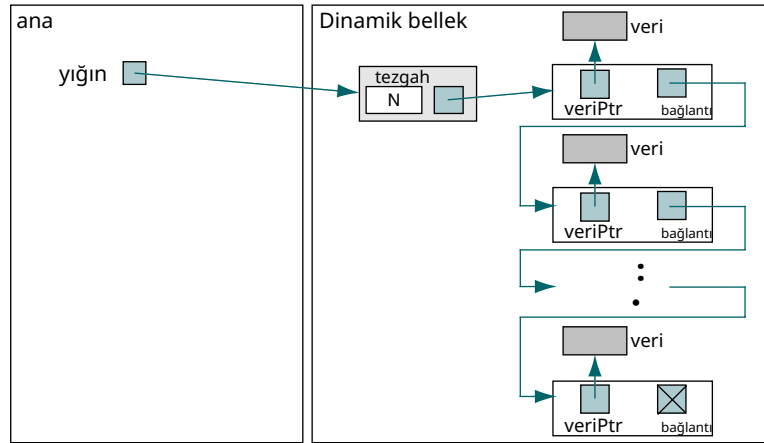
ADT Uygulaması

Soyut bir veri türü oluşturmak için yalnızca bir veri yapısı yeterli değildir : yığını destekleyen işlemler de olmalıdır. Aşağıdaki bölümlerde C işlevlerini geliştiriyoruz.

Yığın Yapısı

Yığın soyut veri türü yapısı Program 3-6'da gösterilmiştir. Düğüm yapısı yalnızca bir veri işaretçisi ve bir bağlantı düğümü işaretçisinden oluşur. Yığın baş yapısı da yalnızca iki öğe içerir: yığının tepe noktasına işaret eden bir işaretçi ve yığında şu anda bulunan giriş sayısını gösteren bir sayım.

3. Bir yığın için dizi uygulaması Ek F'de bulunabilir.



ŞEKİL 3-12Yığın ADT Yapısal Kavramları

Yığın için tür tanımı, Program 3-6'da gösterildiği gibi, herhangi bir işlevin yığın tanımlaması gerektiğinde kolayca yapabilmesi için bir başlık dosyasına dahil edilmiştir.

PROGRAM 3-6Yığın ADT Tanımları

```

1 // Yığın ADT Tür Tanımları
2 typedef struct node
3 {
4     void*    dataPtr;
5     struct node* link;
6 } STACK_NODE;
7
8 typedef yapısı
9 {
10     int sayımı;
11     STACK_NODE* üst;
12 } YIĞIN;

```

Yığın Oluştur

Yığın oluşturma, bir yığın baş düğümü ayırır, üst işaretçiyi null olarak başlatır ve sayım alanını sıfırlar. Dinamik bellek içindeki düğümün adresi daha sonra çağırana döndürülür. Bir yığın oluşturma çağırısı, dönen işaretçi değerini aşağıda gösterildiği gibi bir yığın işaretçisine atamalıdır.

```
yığın = createStack ( );
```

Şekil 3-8, bir yığın oluşturma sonucunu grafiksel olarak göstermektedir. Yığın oluşturma işlevinin kodu Program 3-7'de gösterilmiştir.

PROGRAM 3-7Yığın Oluştur ADT

1	/* ===== createStack =====
2	Bu algoritma boş bir yığın oluşturur.
3	Önce Hiçbir şey
4	Sonra Null yığına işaret eden bir işaretçi döner -veya-
5	Taşma durumunda NULL döner
6	*/
7	STACK* createStack (geçersiz)
8	{
9	// Yerel Tanımlar
10	STACK* yığın;
11	
12	// İfadeler
13	yığın = (STACK*) malloc(sizeof (STACK));
14	eğer (yığın)
15	{
16	yığın->count = 0;
17	yığın->top = NULL;
18	} // eğer
19	yığını döndür;
20	}// createStack

Yığın Düzümü

Verileri bir yığına eklediğimizde yapmamız gereken ilk şey, veriler için bir yer bulmaktır. Bu, heap'ten bellek ayırmamızı gerektirir ve bunu malloc kullanarak yaparız. Bellek ayırdıktan sonra, veriler işaretçisini düğüme atarız ve ardından bağlantıyı, şu anda yığın tepe noktası olarak gösterilen düğüme işaret edecek şekilde ayarlarız. Ayrıca yığın sayım alanına bir eklememiz gerekiyor. Şekil 3-8, yığına birkaç ekleme göstermektedir. Program 3-8, yığın ekleme işleminin bir uygulamasıdır.

PROGRAM 3-8Yığın Ekleme

1	/* ===== pushStack =====
2	Bu fonksiyon bir öğeyi yığına ekler.
3	Ön stack, yığına işaret eden bir işaretçidir
4	dataPtr, eklenmesi gereken veriye işaret eden bir
5	işaretçidir Son Yığına veri eklendi
6	Dönüş başarılıysa true
7	underflow durumunda false
8	*/
9	bool pushStack (STACK* yığını, void* dataInPtr)
10	{
11	// Yerel Tanımlar
12	STACK_NODE* newPtr;
13	
14	// İfadeler
15	newPtr = (STACK_NODE*) malloc(sizeof(STACK_NODE));
16	if (!newPtr)

devam ediyor

PROGRAM 3-8Yiğın İtme (devam ediyor)

```

17181920212223242526
false döndür;

yeniPtr->dataPtr = dataInPtr;

newPtr->link = yiğın->top;
stack->top = yeniPtr;

(yiğın->sayım)++;
true döndür;
} // pushStack

```

Program 3-8 Analiziİtme işleminin ADT uygulaması, yalnızca genel olması açısından geleneksel bir uygulamadan farklıdır. Çünkü bu bir soyut veri türüdür, her türlü veri işaretçisini ele almamız gerekir;bu nedenle, yiğın düğümünde bir *void* işaretçisi kullanıyoruz. Bir *void* işaretçisinin her türlü işaretçiyi saklamak için kullanılabileceğini ve tersine, her işaretçi türüne atanabileceğini hatırlayın. Bir *void* işaretçisi ile yapamayacağımız tek şey, onu dereferans etmektir. Ancak yiğında, gerçek veriye atıfta bulunmaya gerek yoktur, bu nedenle onu asla dereferans etmemiz gerekmez. Yiğın algoritmalarının geri kalanını incelerken, *void* işaretçilerinin kullanımına dikkat edin.

Son nokta ayrıca genel kodu ele alır. Bir ADT yazdığımız için, yiğın taşımasının bir hata olduğunu varsayamayız. Bu karar uygulamaya aittir.

Yapabileceğimiz tek şey durumu rapor etmektir: ya itme başarılı oldu ya da olmadı. Eğer tahsis başarılı olursa, bu normal durumdur, itmenin başarılı olduğunu bildiririz ve true döneriz. Eğer yiğın doluysa, itmenin başarısız olduğunu bildiririz ve false döneriz. O zaman, taşımayı tespit etmek ve yanıt vermek çağırın fonksiyonun sorumluluğundadır.

Yiğın Pop

Pop stack, yiğının en üstündeki düğümdeki veriyi döndürür. Ardından düğümü silerve geri dönüştürür. Sayı 1 çıkarılarak ayarlandıktan sonra, fonksiyon çağırıcıya geri döner. Aşırı boşaltmanın nasıl raporlandığına dikkat edin. 15. ifadede veri işaretçisini NULL olarak ayarlıyoruz. Yiğın boşsa, 24. ifadede veri işaretçisini döndürdüğümüzde NULL döndürüyoruz. Bağlı liste ortamında bir yiğın pop'un grafik örneği için Şekil 3-8'e bakın. Kod Program 3-9'da geliştirilmiştir.

PROGRAM 3-9ADT Pop Yiğın

```

123456789
/* ===== popStack =====
Bu fonksiyon yiğının en üstündeki öğeyi çıkarır.
Önce yiğın, bir yiğın işaretçisidir Sonra Başarılı
ysa kullanıcı verisine işaretçi döndürür Aşırı boşaltma durum
unda NULL döndürür

*/
void* popStack (STACK* yiğını)
{
// Yerel Tanımlar
void* dataOutPtr;
10

```

devam ediyor

PROGRAM 3-9ADT Pop Yığını (devam ediyor)

```

111213141516171819202122232425
STACK_NODE* temp;

// İfadeler
eğer (stack->count == 0)
    dataOutPtr = NULL;

aksi takdirde
{
    temp = yığın->üst;
    dataOutPtr = yığın->üst->dataPtr;
    yığın->top = yığın->üst->bağlantı;
    free (temp);
    (stack->count)--; } // a
ksi takdirde
dataOutPtr'yi döndür;
} // popStack

```

Program 3-9 Analizi Pop işlemi için kod oldukça basittir. İki yerel işaretçi gereklidir, bir çağırıcıya döndürülecek veri işaretçisi için, diğeri ise silinen düğümü serbest bırakmak için kullanılır.

Son düğüm silindiğinde mantığı takip etmek ilginçtir. Bu durumdasonuç b oş bir yığın olur. Özel bir mantık gerektirmez: son düğümün null işaretçisi vardır, bu da üstte atandığında yığının boş olduğunu gösterir.

Null işaretçisi yanlış ve bir adresle işaretçi doğru olduğundan, ayrı bir başarı bayrağına ihtiyacımız yoktur. Bunun yerine, sadece yeni fonksiyon tarafından tahsis edilen adresle işaretçiyi döndürüyoruz. Eğer bellek başarıyla tahsis edildiyse, bir adres içerir, bu da doğrudur. Tahsis başarısız olursa, bir null işaretçisi içerir, bu da yanlıştır.

Yığın üstünü güncelleyen kodu dikkatlice inceleyin. Üst düğümün bağlantı alanına erişmek için iki seviye dolaylılık gerektiğini unutmayın. Bu referanslar kafa karıştırıcıysa, Şekil 3-8'i kullanarak izleyin.

Son olarak, yığından bir düğüm sildiğimizde, pop yığını belleğini serbest bırakır.

Yığın Üstü

Yığın üstü fonksiyonu, üst düğümü silmeden yığının üstündeki veriyi döndürür. Bu, kullanıcının yığın pop edildiğinde neyin silineceğini görmesi ni sağlar. Kod Program 3-10'da geliştirilmiştir.

PROGRAM 3-10Yığın Üstünü Al

```

1234567
/* ===== stackTop =====
   Yığını değiştirmeden yığının tepe noktasından veri alır.

   Önce stack, yığına işaret eden bir işaretçidir
   Sonra Başarılıysa veri işaretçisini döndürür
       yığın boşsa null işaretçisi döndürür
*/

```

devam ediyor

PROGRAM 3-10 Yığın Tepe Noktasını Al (devamı)

89	void* stackTop (STACK* yığını)
	{
101	// İfadeler
11	if (stack->count == 0)
12	return NULL;
13	aksi takdirde
14	return stack->top->dataPtr;
15	} // stackTop

Program 3-10 Analizi Mantık, silme ve geri dönüş dışında pop yığınının neredeyse özdeştir. Kod, ancak ön emli ölçüde farklıdır. Pop'ta olduğu gibi, geri alma işlevinde yalnızca bir potansi yel sorun vardır - yığın boş olabilir. Yığın boşsa, null işaretçisi döndürürüz.

Boş Yığın

Çağırın işlev veri yapısına erişime sahip olmadığından, yığında gerçekten veri olup olmadığını denemeden belirleyemez. Bu nedenle, yığının v eri içerip içermediğini veya boş olduğunu basitçe bildiren boş yığın adı nda bir işlev sağlıyoruz. (Bkz. Program 3-11.)

PROGRAM 3-11 Boş Yığın

1	/* ===== emptyStack =====
2	Bu fonksiyon bir yığının boş olup olmadığını belirler.
3	Önce stack bir yığına işaret eder
4	Sonra boşsa 1; yığında veri varsa 0 döner
5	*/
6	bool emptyStack (STACK* yığını)
7	{
8	// İfadeler
9	return (stack->count == 0);
10	} // emptyStack

Tam Yığın

Tam yığın, destekleyici işlevlerin en karmaşıklarından biridir. Bir sonraki bellek tah sisinin başarılı olup olmayacağını söylemenin basit bir yolu yoktur. Tek yapabilece ğimiz denemektir. Ancak bir tahsis yapmaya çalışarak, yığın alanının bir kısmını ku llanırız. Bu nedenle, bir düğüm için alan tahsis ettikten sonra, program bellek tale p ettiğinde orada olması için hemen serbest bırakırız. Bu mantık Program 3-12'de gösterilmiştir.

PROGRAM 3-12 Tam Yığın

12	/* ===== fullStack =====
	Bu fonksiyon bir yığının dolu olup olmadığını belirler.

devam ediyor

PROGRAM 3-12 Tam Yığın (devam ediyor)

3456789	Heap dolu olarak tanımlanır. Ön yığın, bir yığın baş düğümüne işaretçidir Yığın doluysa doğru döner yığın için yer varsa yanlış döner
1011121314151617181920212223	<pre> */ bool fullStack (STACK* yığını) { // Yerel Tanımlar STACK_NODE* sıcaklığı; // İfadeler eğer ((sıcaklık = (STACK_NODE*)malloc (sizeof(*(yığını->top)))) { free (temp); return false; } // eğer // malloc başarısız oldu return true; } // fullStack </pre>

Yığın Sayısı

Yığın sayısı, yığında bulunan öğelerin sayısını döndürür. Bu sayı yığın baş düğümünde saklandığı için, yığında şu anda kaç öğe bulunduğunu belirlemek için yığını gezinmemize gerek yoktur. Sadece baş sayısını döndürüyoruz.

PROGRAM 3-13 Yığın Sayısı

1	/* ===== stackCount =====
2	Yığında bulunan eleman sayısını döner.
3	Ön yığın, yığına bir işaretçidir
4	Sonuç sayısı döndürüldü
5	*/
6	int stackCount (STACK* yığını)
7	{
8	// İfadeler
9	dön stack->count;
10	} // stackCount

Yığını Yok Et

Yığını yok et, yığındaki düğümleri siler ve bir null işaretçi döndürür. Yığın işaret çisini çağırma alanında NULL olarak ayarlamak, kullanıcının sorumluluğundadır ; bu, dönen değeri yerel yığın işaretçisine atayarak yapılır. Yığın, dinamik bir veri yapısı olarak heap'te uygulandığı için, bellek de yeniden kullanım için serbest bırakılır. Şekil 3-8, bir yığın yok etmenin sonuçlarını grafiksel olarak gösterir.

yığın. İki yığın düğümünün ve yığın başının hepsinin serbest bırakıldığını unutmayın. `destroyStack` için kod, Program 3-14'teki uygulamasıyla aynıdır.

PROGRAM 3-14Yığını Yok Et

```

1  /* ===== destroyStack =====
2  Bu fonksiyon tüm düğümleri yığın alanına serbest bırakır.
3  Önce Bir yığın
4  Sonra null işaretçi döner
5  */
6  STACK* destroyStack (STACK* yığını)
7  {
8  // Yerel Tanımlar
9      STACK_NODE* temp;
10
11 // İfadeler
12 if (yığın)
13 {
14     // Yığındaki tüm düğümleri sil
15     while (stack->top != NULL)
16     {
17         // Veri girişini sil
18         free (stack->top->dataPtr);
19
20         temp = stack->top;
21         yığın->üst = yığın->üst->bağlantı;    serbe
22         st bırak (geçici);
23     } // iken
24
25     // Yığın şimdi boş. Yığın baş düğümünü yok et.
26     serbest bırak (yığın);
27     } // eğer yığın
28     return NULL;
29 } // destroyStack

```

Program 3-14 Analizi Mevcut olmayan bir yığını yok etme çağrısına karşı koruma sağladığımızı unutmayın. Sadece bir yığın varsa a `while` döngüsünü çalıştırıyoruz. Döngüde, önce kullanıcının veri düğümlerini geri dönüştürerek ek ve ardından yığın düğümlerini geri dönüştürerek yığın boyunca ilerliyoruz. Son düğüm serbest bırakıldığında, yığın üstü NULL olur ve döngü sona erer. Tüm veri düğümleri silindikten sonra, baş düğümü de serbest bırakır ve NULL bir yığın işaretçisi döneriz.

3.5 Yığın Uygulamaları

Yığın uygulamaları dört geniş kategoriye ayrılabilir: verileri tersine çevirme, verileri ayrıştırma, veri kullanımını erteleme ve adımları geri alma. Bu uygulamaların her biri için bir veya iki örnek veriyoruz. Verileri tersine çevirmek için bir listeyi tersine çeviriyoruz. Ayrıca, bir ondalık sayıyı ikili karşılığına dönüştürmek için verileri tersine çevirme kullanıyoruz. Ayrıştırma için, bir

kaynak programı. İnfiks notasyonu postfix notasyonuna dönüştürmek ve aynı zamanda bir postfix ifadesini değerlendirmek için ertelemeyi kullanıyoruz. Son olarak, iki veya daha fazla yol arasında seçim yapmak için geri izleme kullanıyoruz. Bu uygulamaların her biri, önceki bölümde tanımlanan yığın ADT'sini kullanır. Aşağıdaki uygulamaları incelemeden önce bu algoritmaları tamamen anladığınızdan emin olmalısınız.

Dört yaygın yığın uygulaması şunlardır: veriyi tersine çevirme, ayrıştırma, erteleme ve geri izleme.

Veri Tersine Çevirme

Veri tersine çevirme, belirli bir veri kümesinin yeniden sıralanmasını gerektirir, böylece ilk ve son öğeler değiştirilir ve ilk ile son arasındaki tüm pozisyonlar da göreceli olarak değiştirilir. Örneğin, {1 2 3 4} {4 3 2 1} haline gelir. İki farklı tersine çevirme uygulamasını inceliyoruz: bir listeyi tersine çevirme ve ondalık sayıyı ikiliye çevirme.

Bir Listeyi Tersine Çevirme

Bir yığın uygulamalarından biri, bir dizi öğeyi tersine çevirmektir. Örneğin, Program 3-15 bir tamsayı listesini okur ve bunları tersine yazdırır.

PROGRAM 3-15 Bir Sayı Dizisini Tersine Çevirme

```
123456789 /* Bu program, klavyeden okunan bir tamsayı listesini tersine
              çevirir ve bunları bir yığına iterek ve birer birer geri alarak.
              Yazarı:
              Tarih:
              */
1011213141516171819202122232425 #include <stdio.h>
#include <stdbool.h>
#include "stacksADT.h"

int main (void)
{
    // Yerel Tanımlar
    bool done = false;
    int* dataPtr;

    STACK* yığın;

    // İfadeler
    // Bir yığın oluştur ve veri için bellek ayır
    yığın = createStack ();

    // Yığını doldur
    while (!done)
    {
```

devam ediyor

PROGRAM 3-15Bir Sayı Serisini Tersine Çevir (devam ediyor)

```

26272829303132333435363738394041424344454647
dataPtr = (int*) malloc (sizeof(int));
printf ("Bir sayı girin: Durdurmak için <EOF>: ");
if ((scanf ("%d" , dataPtr) == EOF
    || fullStack (yığın))
    bitti = doğru;
başka
    pushStack (stack, dataPtr);
} // while

// Şimdi sayıları ters sırayla yazdırprintf
("Sayıların ters listesi:");while (!emptyStack (stack))

{
    dataPtr = (int*)popStack (stack); printf
    ("%3d", *dataPtr);
    ücretsiz (dataPtr);
} // while

// Yığını yok et
destroyStack (stack);
return 0;
} // ana

```

Sonuçlar:

Bir sayı girin: <EOF> durdurmak için: 3
 Bir sayı girin: <EOF> durdurmak için: 5
 Bir sayı girin: <EOF> durdurmak için: 7
 Bir sayı girin: <EOF> durdurmak için: 16
 Bir sayı girin: <EOF> durdurmak için: 91
 Bir sayı girin: <EOF> durdurmak için:

Numaraların ters sıralaması:

91
 16
 7
 5
 3

Program 3-15 AnaliziBu program çok basit. Bir yığın oluşturduktan sonra, bir dizi sayıyı okur ve bunları yığına iter. Kullanıcı dosya sonunu tuşladığında, program yığını boşaltır ve sayılara ters sırayla yazdırır.

Bu basit programda dikkat edilmesi gereken önemli nokta, yığın yapısına doğrudan hiç başvuru yapılmamıştır. Tüm yığın referansları yığın ADT arayüzü üzerinden yapılmıştır. Bu, kapsülleme ve fonksiyon yeniden kullanılabilirliği gibi yapılandırılmış programlama prensiplerinde önemli bir kavramdır.

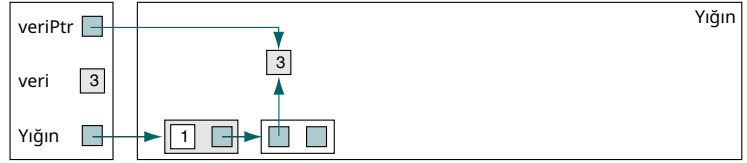
Şekil 3-13, programın çalışmasını takip eder. Program 3-15'i incelerken, işlemlerin izlemek için Şekil 3-13'ü kullanın. Şekildeki her farklı noktada aynı sonuçlara sahip olduğunuzdan emin olun.



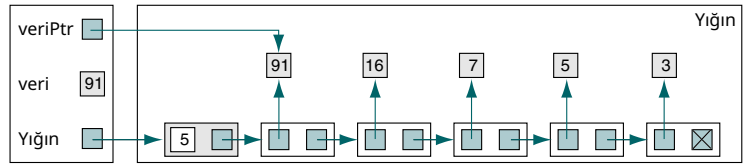
(a) Yığın oluşturulmadan önce



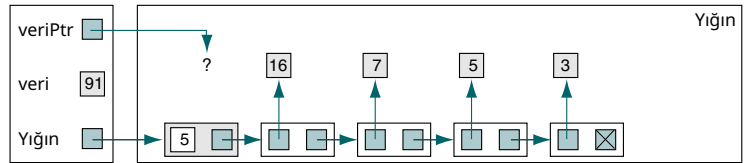
(b) Yığın oluşturulduktan sonra



(c) İlk ekleden sonra



(d) Son ekleden sonra



(e) İlk yazdırmadan sonra



(f) Son yazdırmadan sonra



(g) Yığın yok edildikten sonra

ŞEKİL 3-13 Program 3-15 Adım Adım Uygulama

Onaltılık Sayıyı İkilik Sayıya Dönüştür

Bir diziyi tersine çevirme fikri, ondalık bir sayıyı ikilik bir sayıya dönüştürme gibi klasik problemleri çözmede kullanılabilir. Aşağıdaki basit kod parçası, bir ondalık sayıyı ikilik bir sayıya dönüştürür:

```
1ffoku (sayı)
2ffdöngü (sayı > 0)
    1ffbasamağı sayının 2 ile modunu ayarla
    2ffyazdır (basamak)
    3ffsayıyı sayının 2'ye bölümünün tam sayısı olarak ayarla
3ffdöngüyü bitir
```

Ancak bu kodun bir sorunu var. İkilik sayıyı geriye doğru oluşturuyor. Böylece, 19 11001 yerine 10011 oluyor.

Bu sorunu bir yığın kullanarak çözebiliriz. İkilik basamağı üretildiği anda yazdırmak yerine, onu yığına itiyoruz. Daha sonra, sayı tamamen dönüştürüldükten sonra, yığını pop edip sonuçları birer birer bir satırda yazdırıyoruz. Bu program Program 3-16'da gösterilmiştir.

PROGRAM 3-16Ondalık Sayıyı İkilik Sayıya Dönüştür

123456789	/* Bu program, klavyeden bir tam sayı okur ve onun ikilik karşılığını yazdırır. 0 ve 1'lerin sırasını tersine çevirmek için bir yığın kullanır. Yazarı: Tarih:
1011121314151617181920212223242526	*/ #include <stdio.h> #include "stacksADT.h" int main (void) { // Yerel Tanımlar unsigned int num; int* digit; Yığın* yığını; // İfadeler // Yığın Oluştur stack = createStack (); // bir sayı isteme ve okuma printf ("Bir tam sayı girin: "); scanf ("%d", &num); // 0 ve 1'leri oluştur ve yığına it while (num > 0)

devam ediyor

PROGRAM 3-16 Onaltılık Sayıyı İkiliye Dönüştür (devamı)

```

2728293031323334353637383940414243444546
{
    rakam = (int*) malloc (sizeof(int));
    *digit = sayı % 2;
    pushStack (yığın, rakam);
    num = num / 2;
} // while

// İkili sayı oluşturuldu. Şimdi yazdırın
printf ("İkili sayı: ");
while (!emptyStack (stack))
{
    rakam = (int*)popStack (stack);
    printf ("%1d", *digit);
} // while
printf ("");// Yığın

yok ediliyor destroyStack (stack);return 0;

} // ana

```

Sonuçlar:

Bir tamsayı girin: 45
İkili sayı: 101101

Ayrıştırma

Yığınların bir diğer uygulaması ayrıştırma. Ayrıştırma, veriler bağımsız parçalara ayıran herhangi bir mantıktır. Örneğin, bir kaynak programı makine diline çevirmek için, bir derleyici programınahtar kelimeler, isimler ve tokenler gibi bireysel parçalara ayrılmalıdır.

Yaygın bir programlama problemi, cebirsel bir ifadede eşleşmeyen parantezlerdir. Parantezler eşleşmediğinde, iki tür hata meydana gelebilir: açılış parantezi eksik olabilir veya kapanış parantezi eksik olabilir. Bu iki hata Şekil 3-14'te gösterilmiştir.

?

$$((A + B) / C$$

(a) Açılış parantezi eşleşmiyor

?

$$(A + B) / Q$$

(b) Kapanış parantezi eşleşmiyor

ŞEKİL 3-14 Eşleşmeyen Parantez Örnekleri

Algoritma 3-9'da, tüm parantezlerin düzgün bir şekilde eşleştiğinden emin olmak için bir kaynak programı ayrıştırıyoruz.

ALGORİTMA 3-9 Parantezleri Ayrıştır

```

Algoritma parantezleriAyrıştır
Bu algoritma bir kaynak programı okur ve tüm açılış-kapanış parantezlerini
eşleştiğinden emin olmak için ayrıştırır.
1  döngü (daha fazla veri)
    1  oku (karakter)
    2  eğer (açılış parantezi)
        1  yığınEkle (yığın, karakter)
    3  aksi takdirde
        1  eğer (kapatma parantezi)
            1  eğer (stack boşsa)
                1  yazdır (Hata: Kapanış parantezi eşleşmiyor)
            2  aksi takdirde
                1  popStack(yığın)
            3  eğer sonlandır
        2  eğer sonlandır
    4  eğer sonlandır
2  döngüyü sonlandır
3  eğer (stack boş değilse)
    1  yazdır (Hata: Açılış parantezi eşleşmiyor)
son parseParens

```

Uygulama Program 3-17'de gösterilmiştir.

PROGRAM 3-17 Kaynak Programda Parantezlerin Eşleştiğini Doğrula

```

123456789 /* Bu program bir kaynak programı okur ve
            tüm açılış-kapanış parantezlerinin eşleştiğinden emin olmak için ayrıştırır.
            Yazarı:
            Tarih:
*/
#include <stdio.h>
#include "stacksADT.h"

// Hata Mesajları
const char closMiss[] = "Kapanış parantezi eksik, satır";
const char openMiss[] = "Açılış parantezi eksik, satır";

int main (void)
{
    // Yerel Tanımlar
    STACK* stack;
    char token;
    char* dataPtr;

```

devam ediyor

PROGRAM 3-17Kaynak Programda Parantezlerin Eşleşmesini Doğrula (devam ediyor)

```

192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566
char dosya kimliği[25];
FILE* fpIn;
int lineCount = 1;

// İfadeler
// Yığın Oluştur
yığın = createStack();
printf("Parçalanacak dosya için dosya kimliğini girin: ");
scanf("%s", fileID);

fpIn = fopen(dosya kimliği, "r");
if (!fpIn)
    printf("Açma hatası %s", fileID), exit(100); // kaynak koddan ka

rakterleri oku ve ayırıştır while ((token = fgetc(fpIn)) != EOF)

{
    if (belirteç == '"') lineCo
        unt++; if (belirt
            eç == '(') {

                dataPtr = (char*) malloc(sizeof(char));
                pushStack(stack, dataPtr);
            } //

            eğer aksi halde
            {
                if (belirteç == ')')
                {
                    if (emptyStack(yığın))
                    {
                        printf("%s %d",
                            openMiss, lineCount);
                        return 1;
                    } // doğruysa

                    başka
                        popStack(stack);
                    } // token ==

                    } // aksi takdirde
                } // while

            if (!emptyStack(stack)) {

                printf("%s %d", closMiss, lineCount); return 1;

            } // eğer

// Şimdi yığını yok et
yokEt(yığın);

```

devam ediyor

PROGRAM 3-17 Kaynak Programda Parantezlerin Eşleşmesini Doğrula (devam ediyor)

67686970	<pre> printf ("Parçalama tamam: %d Satır parçalandı.", ffffff fffflineCount); return 0; } // ana </pre>
Sonuçlar: Çalıştırma 1: Parçalanacak dosya için dosya kimliğini girin: no-errors.txt Parçalama tamamlandı: 65 Satır parçalandı. Çalıştır 2: Parçalanacak dosya için dosya kimliğini girin: close-match.txt Kapatma parantezi 46. satırda eksik Çalıştır 3: Parçalanacak dosya için dosya kimliğini girin: open-match.txt Açık parantez 23. satırda eksik	

Program 3-17 Analizi Bir programda açılış parantezi bulduğumuzda, onu yığına ekleriz.

Kapanış parantezini bulduğumuzda, eşleşen açılış parantezini yığından çıkarırız. Programda iki farklı eşleşme hatası olduğunu unutmayın: (1) eşleşen bir açılış parantezi olmadan bir kapanış parantezi (bakınız ifade 49) ve (2) eşleşen bir kapanış parantezi olmadan bir açılış parantezi (bakınız ifade 61).

Programı test etmek için, onu üç farklı dosya üzerinde üç kez çalıştırdık. Butes t, üç durumu da gösterse de, programı tam olarak test etmez. İlk açılış parantezini n eksik olduğu gibi ek test durumları da çalıştırılmalıdır.

Son olarak, hata mesajlarını programın başına nasıl yerleştirdiğimize dikkat edin. Bu, yalnızca iyi belgeler sağlamakla kalmayıp, programı daha okunabilir hale getiren yaygın bir belgelenme tekniğidir.

Erteleme

Bir yığın kullanarak bir listeyi tersine çevirdiğimizde, sonuçları çıkarmaya başlamanın önce tüm liste okunur. Genellikle bir uygulamanın mantığı, verilerin kullanımının daha sonraki bir noktaya ertelenmesini gerektirir. Bir yığın, uygulamanın verilerin kullanımını bir süre ertelemesini gerektirdiğinde faydalı olabilir. Bu bölümde iki yığın erteleme uygulaması geliştiriyoruz: infiks'ten postfiks'e dönüşüm ve postfiks ifade değerlendirme.

İnfiks'ten Postfiks'e Dönüşüm

İnfiks notasyonunun dezavantajlarından biri, operatörlerin değerlendirilmesini kontrol etmek için parantez kullanmamız gerektiğidir. Bu nedenle, parantezleri ve iki operatör öncelik sınıfını içeren bir değerlendirme yöntemine sahibiz. Postfiks ve prefiks notasyonlarında, parantezlere ihtiyacımız yoktur; her biri yalnızca bir değerlendirme kuralı sağlar.

Bazı yüksek seviyeli diller infiks notasyonu kullanmasına rağmen, bu tür ifadeler doğrudan değerlendirilemez. Bunun yerine, değerlendirilmek üzere analiz edilmeleri gerekir.

4. Aritmetik notasyonlar hakkında bir tartışma için, Bölüm 2'deki "Prefix to Postfix Conversion"a bakın.

ifadelerin değerlendirileceği sıra. Yaygın bir değerlendirmetekniği, ifadeleri değerlendirmek için kodu oluşturmada önce postfix notasyonuna dönüştürmektir. Bu kavramı burada gösteriyoruz. Öncelikle infix ifadeleri postfix ifadelerine dönüştürmek için manuel bir yöntemi inceliyoruz ve ardında n bir bilgisayarda uygulanabilir bir algoritma geliştiriyoruz.

Manuel Dönüşüm

Infix ifadeleri postfix ifadelerine manuel olarak dönüştürmek için kurallar şunlardır:

1. İfadeyi, herhangi bir açık parantez ve aritmetik önceliği kullanarak tamamen parantezleyin—çarpma ve bölme, toplama ve çıkarmadan önce yapılmalıdır.
2. Her parantez içindeki infiks notasyonlarını, en içteki ifadelerden başlayarak postfix notasyonuna çevirin. Operatörü ifadenin kapanış parantezinin konumuna taşıyarak postfix notasyonuna dönüşüm yapılır.
3. Tüm parantezleri kaldırın.

Örneğin, aşağıdaki infiks ifade için:

$$A + B * C$$

Adım 1 sonuçları

$$(A + (B * C))$$

Adım 2 çarpan operatörünü C'nin arkasına taşıyır

$$(A + (B C *))$$

ve ardından toplama operatörünü son iki kapanış parantezinin arasına taşıyır. Bu değişiklik, artı işareti için kapanış parantezinin son parantez olması nedeniyle yapılır. Artık şunlara sahibiz

$$(A (B C *) +)$$

Son olarak, adım 3 parantezleri kaldırır.

$$A B C * +$$

Daha karmaşık bir örneğe bakalım. Bu örnek sadeceuzun değil, aynı zamanda varsayılan değerlendirme sırasını geçersiz kılmak için bir parantez setine de sahip.

$$(A + B) * C + D + E * F - G$$

Adım 1 parantezler ekler.

$$(((((A + B) * C) + D) + (E * F)) - G)$$

Adım 2 daha sonra operatörleri taşıyır.

$$(((((A B +) C *) D +) (E F *) +) G -)$$

Adım 3 parantezleri kaldırır.

$$A B + C * D + E F * + G -$$

Algoritmik Dönüşüm

Bu manuel işlemi bir bilgisayarda uygulamak zor olacaktır. Hadi bir yığın ile kolayca uygulanabilen başka bir tekniğe bakalım.

Çok basit bir örnekle başlıyoruz, bir çarpma işlemini dönüştürüyoruz. İki değişkenin çarpımı aşağıda gösterilmektedir, önce infiks notasyonunda ve ardından postfix notasyonunda.

A * B şuna dönüşür A B *

Açıkça operandları okuyabiliriz ve bunları sırayla çıktı olarak verebiliriz. Z. Problem, çarpma operatörünü nasıl ele alacağımızdır; doğru operandı, B'yi okuduğumuz zamana kadar çıktıya koymayı ertelememiz gerekiyor. Bu basit durumda, operatörü bir yığına itiyoruz ve tüm infiks ifadesi okunduktan sonra yığından çıkarıp operatörü postfix ifadesine koyuyoruz.

Şimdi daha karmaşık bir ifadeye bakalım. Yine, infiks ifade solda ve eşdeğer postfix ifade sağda verilmiştir.

A * B + C şuna dönüşür A B * C +

Burada tekrar infiks operatörlerini okuyabiliriz ve bunları post-fix ifadesine kopyalayabiliriz. Operatörleri daha önce yaptığımız gibi yığına koyup, tüm operandlar okunduktan sonra post-fix ifadesine çıkarmaya çalışırsak, yanlış bir sonuç alırız. Bir şekilde iki operatörü doğru operandlarıyla eşleştirmeliyiz. Olası bir kural, bir operatörü yalnızca başka bir operatör alana kadar ertelemektir.

Daha sonra, ikinci operatörü yığa itmeden önce, ilk operatörü çıkarabiliriz ve çıktı ifadesine yerleştirebiliriz. Bu mantık bu durumda işe yarıyor, ancak diğerlerinde işe yaramayacak. Aşağıdaki örneği düşünün.

A + B * C şuna dönüşür A B C * +

Daha önce tartıştığımız gibi, infiks ifadeleri operandları ve operatörleri bir ifadeye gruplamak için bir öncelik kuralı kullanır. İnfiks ifadeyi postfix ifadeye dönüştürürken aynı kuralı kullanabiliriz. Bir operatörü yığa itmeliyiz gerektiğinde, eğer önceliği yığının en üstündeki operatörden daha yüksektse, onu yığa itmeye devam ederiz. Tersine, eğer yığının en üstündeki operatör mevcut operatörden daha yüksek bir önceliğe sahipse, çıkarılır ve çıktı ifadesine yerleştirilir. Yukarıdaki ifadeyle bu kuralı kullanarak aşağıdaki eylemleri gerçekleştireceğiz:

1. Operanı kopyala A çıktı ifadesine.
2. İşlemi yığa ekle + .
3. Operanı kopyala B çıktı ifadesine.
4. İşlemi yığa ekle * . (* önceliği '+' dan yüksektir.)
5. Operanı kopyala C çıktı ifadesine.

6. Pop operatörü * ve çıktıya kopyala ifadesi.
7. Pop operatörü + ve çıktıya kopyala ifadesi.

Mantığı tamamlamak için bir kuralı daha ele almamız gerekiyor. Mevcut operatör, daha düşük veya eşit önceliğe sahip olduğunda, üstteki operatörün yığından çıkarılmasını zorladığında, yeni üstteki operatörü kontrol etmemiz gerekiyor. Eğer bu da mevcut operatörden büyükse, çıktıya çıkarılır. Sonuç olarak, yeni operatörü yığına itmeden önce birkaç operatörü çıktıya çıkarabiliriz.

Algoritmayı resmi olarak geliştirmeden önce bir örnek üzerinde daha çalışalım.

$A + B * C - D / E$

şuna dönüşür

$A B C * + D E / -$

Bu ifadenin dönüşümü Şekil 3-15'te gösterilmiştir. Tüm temel aritmetik operatörlerini kullandığı için, bu tam bir testtir.

	İçerik	Yığın	Sonradan
(a)	$A + B * C - D / E$		
(b)	$+ B * C - D / E$		A
(c)	$B * C - D / E$	+	A
(d)	$* C - D / E$	+	AB
(ve)	$C - D / E$	* +	AB
(f)	$- D / E$	* +	ABC
(g)	D / E	-	ABC*+
(h)	$/ E$	-	ABC*+D
(Ben)	E	/	ABC*+D
(i)		/	ABC*+DE
(k)			ABC*+DE/-

ŞEKİL 3-15 İnfiks Dönüşümleri

İlk operatörü, A, postfix ifadesine kopyalayarak başlıyoruz. Bakın Şekil 3-15(b). Ardından toplama operatörü yığına itilir ve ikinci operatör postfix ifadesine kopyalanır. Bakın Şekil 3-15(d). Bu noktada çarpma operatörünü yığına eklemeye hazırız. Gördüğümüz gibi

Şekil 3-15(e), önceliği yığın üzerindeki toplama operatöründen daha yüksektir, bu nedenle onu yığına itiyoruz. Sonraki operandı kopyaladıktan sonra, c, postfix ifadesine itmemiz gerekiyor, eksi operatörünü yığına itmemiz gerekiyor. Çünkü önceliği çarpma operatöründen daha düşüktür, ancak önce çarpma operatörünü çıkarmalı ve postfix ifadesine kopyalamalıyız. Artı işareti şimdi çıkarılır ve postfix ifadesine eklenir çünkü eksi ve artı aynı önceliğe sahiptir. Eksi daha sonra yığına itilir. Sonuç Şekil 3-15(g)'de gösterilmektedir. Operandı kopyaladıktan sonra D postfix ifadesine, bölme operatörünü yığına itiyoruz çünkü bu, Şekil 3-15(i)'de yığının üstündeki eksi operatöründen daha yüksek önceliğe sahiptir. Postfix ifadesine E kopyaladıktan sonra, elimizde boş bir infix ifadesi ve yığında iki operatör kalır. Şekil 3-15(j)'ye bakın. Bu noktada geriye kalan tek şey, yığını çıkarmak ve her operatörü postfix ifadesine kopyalamaktır. Nihai ifade Şekil 3-15(k)'de gösterilmektedir.

Artık algoritmayı geliştirmeye hazırız. Aşağıda yalnızca gösterilen operatörlerin olduğunu varsayıyoruz. Bunlar standart cebirsel notasyondan uyarlanmıştır.

Öncelik 2: * /
Öncelik 1: + -
Öncelik 0: (

Tasarım Algoritma 3-10'da gösterilmiştir.

ALGORİTMA 3-10 Infix'i Postfix'e Dönüştür

Algoritma inToPostFix (formül)

Infix formülünü postfix'e dönüştür.

Ön formül, sözdizimsel hataların olmadığını sağlamak için düzenlenmiş infix notasyondur. Son postfix formül bir dize olarak biçimlendirilmiştir. Postfix formülü döndür

- 1 createStack (yığın)
- 2 döngü (formüldeki her karakter için)
 - 1 eğer (karakter açık parantez ise)
 - 1 yığınEkle (yığın, karakter)
 - 2 aksi takdirde (karakter kapalı parantez ise)
 - 1 popStack (yığın, karakter)
 - 2 döngü (karakter açık parantez değil)
 - 1 karakteri postFixExpr'e ekle
 - 2 popStack (yığın, karakter)
 - 3 döngüyü sonlandır
- 3 aksi takdirde (karakter operatörse)
 - yığın tepesindeki token ile token'ın önceliğini test et
 - 1 yığınTepesi (yığın, üstToken)
 - 2 döngü (boşYığın değil (yığın)
 - VE karakterin önceliği <= üstToken'ın önceliği
 - 1 yığından çıkar (yığın, tokenDışarı)
 - 2 tokenDışarı'yı postFixExpr'e ekle
 - 3 yığınTepesi (yığın, üstToken)

devam ediyor

ALGORİTMA 3-10 Infix'i Postfix'e Dönüştür (devam ediyor)

```

3   döngüyü sonlandır
4   pushStack (yığın, belirteç)
4   aksi takdirde
   f Karakter operand
   1 Token'i postFixExpr'e ekle
5   eğer sonlandır
3   döngüyü sonlandır
   Girdi formülü boş. Yığını postFix'e çıkar
4   döngü (boşStack (stack) değil)
   1 popStack (yığın, karakter)
   2 Token'i postFixExpr'e ekle
5   döngüyü sonlandır
6   postFix'i döndür
son  inToPostFix

```

Kod Program 3-18'de devam etmektedir.

PROGRAM 3-18 Infix'i Postfix'e Dönüştür

```

123456789 /* Bu program bir infiks formülü postfixformülüne dönüştürür. Infiks
1011121314151617181920212223242526272829 formülü, herhangi bir sözdizim hatası olmadığından emin olmak
        için düzenlenmiştir.
        Yazarı:
        Tarih:
        */
#include <stdio.h>
#include <string.h>
#include "stacksADT.h"

// Prototip Bildirimleri
int priority (char token);
bool isOperator (char token);

int main (void)
{
    // Yerel Tanımlar
    char postfix [80] = {0};
    char temp [2] = {0};
    char token;
    char* dataPtr;
    STACK* yığın;

    // İfadeler
    // Yığın Oluştur
    yığın = createStack();

    // infix formülünü oku ve karakter karakter ayırıştır
    printf("Bir infix formülü girin: ");

```

devam ediyor

PROGRAM 3-18 Infix'i Postfix'e Dönüştür (devam ediyor)

303132333435363738394041424344454647484950515253545556575859606162636465666768697071727374757677

```

while ((belirteç = getchar ())!= ")") {

    if (belirteç == '(')
    {
        dataPtr = (char*) malloc (sizeof(char));
        *dataPtr = token;
        pushStack (stack, dataPtr);    } //
    eğer
    else if (belirteç == ')')
    {
        dataPtr = (char*)popStack (stack);
        while (*dataPtr != '(')
        {
            temp [0]= *dataPtr;
            strcat (postfix , temp);
            dataPtr = (char*)popStack (stack);
        } // while
    } // else if
    else if (isOperator (belirteç))
    {
        // yığının tepesinde belirtecin önceliğini test edin
        dataPtr = (char*)stackTop (stack);
        while (!emptyStack (stack)
            && öncelik (belirteç) <= öncelik (*dataPtr))
        {
            dataPtr = (char*)popStack (stack);
            temp [0] = *dataPtr;
            strcat (postfix , temp);
            dataPtr = (char*)stackTop (stack);
        } // while
        dataPtr = (char*) malloc (sizeof (char));
        *dataPtr = token;
        pushStack (stack , dataPtr);
    } // else if
    başka // karakter operand
    {
        temp[0]= belirteç;
        strcat (postfix , temp);
    } // else
} // token alırken

// Infix formülü boş. Postfix'e geçmek için yığını boşalt
while (!emptyStack (stack))
{
    dataPtr = (char*)popStack (stack);
    temp[0] = *dataPtr;
    strcat (son ek, geçici);
}

```

devam ediyor

PROGRAM 3-18 Infix'i Postfix'e Dönüştür (devam ediyor)

```

78798081 82838485868788899091 9293949596979899100101102103104105106107108109110111112113114115116

        } // while

        // Postfix'i yazdır
        printf ("Postfix formülü: ");
        puts (postfix);

        // Şimdi yığını yok et
        destroyStack (stack);
        return 0;
    } // ana

    /* ===== öncelik =====
        Operatörün önceliğini belirleyin.
        Önceki token geçerli bir operatördür So
        nraki token önceliği döndürüldü
    */
    int öncelik (char token)
    {
        // İfadeler
        if (belirteç == '*' || belirteç == '/')
            return 2;
        if (belirteç == '+' || belirteç == '-')
            return 1;

        0 değerini döndür;
    } // öncelik

    /* ===== isOperator =====
        Token'ın bir operatör olup olmadığını belirleyin.
        Önceki token geçerli bir operatördür
        Sonraki, operatörse true; değilse false döner
    */
    bool isOperator (karakter belirteci)
    {
        // İfadeler
        if (token == '*'
            || token == '/'
            || token == '+'
            || token == '-')
            return true;

        return false;
    } // isOperator

```

Sonuçlar:**Çalıştır 1**

Bir infix formülü girin: 2+4Postfix f
ormülü: 24+Çalıştır 2

Bir infix formülü girin: (a+b)*(c-d)/e
Postfix formülü: ab+cd-*e/

Program 3-18 AnaliziBir infiks ifadesi yalnızca üç nesne içerebilir: bir parantez içindeki operatörler, bir değişken tanımlayıcısı veya bir sabit ve bir operatör. Eğer bir açılış parantezimiz varsa,

onu yığına koyuyoruz. Sonunda bir ifadenin veya alt ifadenin sonunu belirten bir kapanış parantezi ile eşleşir. Değişkenler ve sabitler hemen çıktı dizisine gider; bu nedenle yalnızca operatörlerin daha fazla analize ihtiyacı vardır.

İşlenmesi gereken bir operatör verildiğinde, soru şudur: Yığına mı itilme li yoksa çıktı formülüne mi yerleştirilmeli? Yığın boşsa, operatör yığına itilir ve devam ederiz.

İç içe operatörlerin iki önceliği vardır. Çarpma ve bölme operatörleri, toplama ve çıkarma operatörlerinden daha yüksek bir önceliğe (2) sahiptir (1). Açılış parantezi de operatör yığınının itildiği için, ona 0 önceliği veriyoruz. Yeni mevcut tokenin önceliğini, yığının en üstündeki tokenin önceliği ile 54. durum ifadesinde kontrol ediyoruz.

Yeni operatörün önceliği, yığının en üstündeki operatörün önceliğinden daha düşük veya eşitse, yığının en üstündeki token çıkış dizisine taşınır ve yığının en üstündeki operatör yeniden kontrol etmek için döngüye gireriz. Yeni operatör, yığının en üstündeki operatörden daha yüksek bir önceliğe sahipse, yığına girer ve giriş formülündeki bir sonraki tokeni geçeriz.

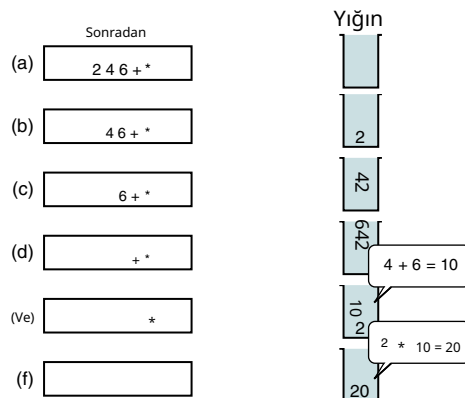
Postfix İfadelerini Değerlendirme

Şimdi, daha önce geliştirdiğimiz postfix ifadeleri değerlendirmek için yığın ertelemesini nasıl kullanabileceğimize bakalım. Örneğin, aşağıda gösterilen ifadeyi göz önünde bulundurarak,

A B C + *

ve A'nın 2, B'nin 4 ve C'nin 6 olduğunu varsayarsak, ifadenin değeri nedir?

İlk fark etmeniz gereken şey, operandların operatörlerden önce gelmesidir. Bu, bu sefer operandların kullanımını ertelemek zorunda kalacağımız anlamına geliyor, operatörlerin değil. Bu nedenle, onları yığına koyuyoruz. Bir operatör bulduğumuzda, yığının en üstündeki iki operandı çıkarıyoruz ve işlemi gerçekleştiriyoruz. Ardından, değeri daha sonra kullanılmak üzere yığına geri itiyoruz. Şekil 3-16, ifademizin çalışmasını izler. (Operand değerlerini yığına itiyoruz, operand isimlerini değil. Bu nedenle, şekilde değerleri kullanıyoruz.)



ŞEKİL 3-16 Postfix İfadesinin Değerlendirilmesi

İfade tamamen değerlendirildiğinde, değeri yığında bulunur. Bakınız Algoritma 3-11. Uygulama Program 3-19'da gösterilmiştir.

ALGORİTMA 3-11 Postfix İfadelerinin Değerlendirilmesi

Algoritma `postFixEvaluate (expr)` Bu algoritma bir postfix ifadesini değerlendirir ve değerini döndürür.

```

Ön   geçerli bir ifade
Son   hesaplanan postfix değeri
İfadenin değeri döndürülür
1 createStack (yığın)
2 döngü (her karakter için)
  1 eğer (karakter işlemci ise)
    1 yığınEkle (yığın, karakter)
  2 aksi takdirde
    1 popStack (yığın, oper2)
    2 popStack (yığın, oper1)
    3 işlemci = karakter
    4 değeri hesapla (oper1, işlemci, oper2) olarak ayarla
    5 pushStack (yığın, değer)
  3 eğer sonlandı
3 döngüyü sonlandır
4 popStack (yığın, sonuç)
5 return (sonuç)
son postFixEvaluate

```

PROGRAM 3-19 Postfix İfadesini Değerlendir

```

1  /* Bu program bir postfix ifadesini değerlendirir ve
2     değeri döndürür. Postfix ifadesi
3     her bir operandın yalnızca bir basamak olduğu geçerli olmalıdır.
4     Yazarı:
5     Tarih:
6
7  */
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include "stacksADT.h"
11
12 // Prototip Bildirimleri
13 bool isOperator (char token);
14 int calc (int işlenen1, int işlenen, int işlenen2);
15
16 int main (void)
17 {
18     // Yerel Tanımlar
19     char    token;
20     int     operand1;
21     int     operand2;
22     int     value;

```

devam ediyor

PROGRAM 3-19 Postfix İfadesini Değerlendir (devamı)

```

222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869
int* veriPtr;
STACK* yığın;

// İfadeler
// Yığın Oluştur
stack = createStack ();

// postfix ifadesini, karakter karakter oku
printf("Girdi formülü: ");
while ((belirteç = getchar ())!= " ") {

    if (!isOperator (belirteç))
    {
        dataPtr = (int*) malloc (sizeof (int));
        *dataPtr = atoi (&token);
        pushStack (stack, dataPtr);
    } // while

    başka
    // karakter işlem
    {
        dataPtr = (int*)popStack (stack);
        işlenen2 = *dataPtr;
        dataPtr = (int*)popStack (stack);
        işlenen1 = *dataPtr;
        değer = calc(işlenen1, belirteç, işlenen2);
        dataPtr = (int*) malloc (sizeof (int));
        *dataPtr = değer;
        pushStack (stack, dataPtr);    } // ak
    si takdirde
} // while

// Sonuç yığında. Çıkart ve yazdır
dataPtr = (int*)popStack (stack);
değer = *dataPtr;
printf ("Sonuç: %d", value);

// Şimdi yığını yok et
destroyStack (stack);
return 0;
} // ana
/* ===== isOperator =====
Operatörü doğrula.
Önceki token doğrulanacak operatördür
Sonuç geçerliyse true, değilse false döndürür
*/
bool isOperator (karakter belirteci)

```

devam ediyor

PROGRAM 3-19 Postfix İfadesini Değerlendir (devamı)

```

707172737475767778798081828384858687888990919293949596979899100101102103104
{
// İfadeler
    if (token == '*'
        || token == '/'
        || token == '+'
        || token == '-')
        return true;
    yanlış döndür;
} // isOperatör
/* ===== calc =====
İki değer ve operatör verildiğinde, formülün değerini belirleyin.

    Önce operand1 ve operand2 değerleridir
    oper kullanılacak operatördür
    Sonuç hesaplamasının sonucunu döndürür
*/
int calc (int işlenen1, int işlenen, int işlenen2)
{
// Yerel Bildirim
    int sonuç;

// İfadeler
    switch (oper)
    {
        case '+': sonuç = operand1 + operand2;
                    break;
        case '-': sonuç = işlenen1 - işlenen2;
                    break;
        case '*': sonuç = işlenen1 * işlenen2;
                    break;
        case '/': sonuç = işlenen1 / işlenen2;
                    break;
    } // anahtar
    sonucu döndür;
} // calc

```

Sonuçlar:

Girdi formülü: 52/4+5*2+

Sonuç 32

Program 3-19 Analizi Bu algorithmadaki tüm hata işleme kısmını atlادık. İfadenin geçerli olduğunu ve değeri değerlendirilecek yeterli bellek olduğunu varsayıyoruz. Bir uygulama açıkça bu sorunlara karşı korunmalıdır.

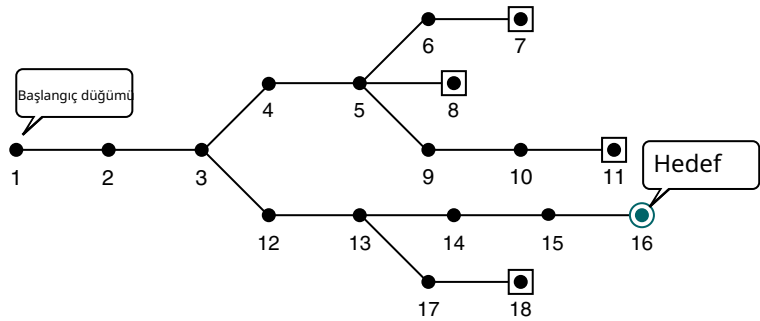
Bu varsayımlar altında, algoritma oldukça basittir. Bir yığın oluşturuyoruz ve ardından bir operatör bulana kadar yığına operandları itiyoruz. Her operatörü bulduğumuzda, en üstteki iki operandı çıkarıyor ve işlemi gerçekleştiriyoruz. Sonuç daha sonra yığına itiliyor ve ifadenin bir sonraki elemanı inceleniyor. En sonunda, yığından değeri çıkarıyor ve geri döndürüyoruz.

Geri izleme

Geri izleme , bilgisayar oyunları, karar analizi ve uzman sistemler gibi uygulamalarda bulunan başka bir yığın kullanım şeklidir. Bu bölümde iki geri izleme uygulamasını inceliyoruz: hedef arama ve sekiz kraliçe problemi.

Hedef Arama

Şekil 3-17, bir hedef arama uygulamasının bir örneğidir. Problemi tasvir etmenin bir yolu, birkaç alternatif yol içeren bir grafik şeklinde adımları düzenlemektir. Şekildeki yolların yalnızca biri istenen bir hedefe ulaşır. Şekle baktığımızda doğru yolu hemen görebilirken, bilgisayarın doğru yolu belirlemek için bir algoritmaya ihtiyacı vardır.



ŞEKİL 3-17Geri İzleme Örneği

1. düğümde başlıyoruz ve bir dallanma düğümü olan 3'e ulaşana kadar sağa hareket ediyoruz. Bu noktada üst yolu alıyoruz. 5. düğümde ulaşana kadar devam ediyoruz, bu noktada tekrar üst yolu alıyoruz. 7. düğümde geldiğimizde daha fazla ilerleyemiyoruz. Ancak, henüz amacımıza ulaşmadık. Bu nedenle 5. düğümde geri dönmeli ve bir sonraki yolu almamız gerekir. 8. düğümde tekrar 5. düğümde geri dönüyoruz ve üçüncü yolu alıyoruz. Yine, 11. düğümde geri dönmek zorundayız, bu sefer 3. düğümde geri. 12. düğümde itibaren yolu takip ettiğimizde nihayet amacımız olan 16. düğümde ulaşırız. 17 ve 18. düğümleri içeren yolu incelemediğimizi unutmayın.

Artık kafanızda bir algoritma oluşturmaya başlamış olmalısınız. Her karar noktasına geldiğimizde, oranın neresi olduğunu hatırlamamız gerekiyor, böylece gerekirse geri dönebiliriz. Geriye dönerken, devam etmeden önce en yakın noktaya geri dönmek istiyoruz; yani, tekrar baştan başlamak istemiyoruz. Bu problem için LIFO veri yapısını, yığını kullanıyoruz.

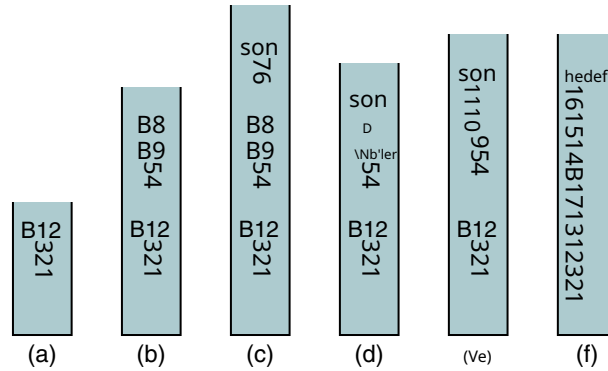
Şimdi yığına ne koyacağımız sorusu var. Eğer sadece hedefi içeren düğümü bulmamız gerekseydi, sadece dal noktası düğümlerini yığına koymaydık. Ancak, işimiz bittiğinde hedefimize giden yolu yazdırmak istiyoruz. Bu nedenle, geçerli yol düğümlerini de yığına koymalıyız.

Yığına iki şey koyduğumuz için, bunları ayırt edebilmemiz gerekiyor. Bu nu bir bayrakla yapıyoruz. Eğer düğüm yolda ise, bir yol belirteci itiyoruz. Eğer bir geri izleme noktasını saklıyorsak, bayrağı bir geri izleme belirteci olarak ayarlıyoruz.

Şekil 3-18, hedefimize giden yolun yığın izlememizi içermektedir, Şekil 3-17'de gösterilmiştir. B, geri izleme düğümünü sakladığımızı belirtmek için kullanıyoruz. Eğer belirteç yoksa, yığın bir yol düğümü içerir.

Önce 1, 2 ve 3'ü yığına itiyoruz. Çünkü 3 hedefimiz değil, üst yolu takip ediyoruz. Ancak burada bir karar verdiğimizizi hatırlamamız gerekiyor. Bu nedenle, dal noktasını, 12'yi, geri izleme belirteci ile birlikte bir yığına itiyoruz, Şekil 3-18(a) de gösterildiği gibi. 5'te başka bir karar vermemiz gerekiyor. Yine, üst yolu takip ediyoruz. Bu sefer hatırlamamız gereken iki devam noktası, 8 ve 9 var. Bu yüzden, her ikisini de geri izleme belirteçleri ile yığına itiyoruz. Bu nokta Şekil 3-18(b)'de gösterilmektedir. 7'de hedefimize ulaşmadan yolun sonuna geldik. Şekil 3-15(c)'e bakın. Devam etmek için, geri izleme noktasında olana kadar yığından eleme yapıyoruz. Ardından, geri izleme noktasını bir yol düğümü olarak yığına itiyoruz ve 8'e doğru ilerliyoruz. Diyagramımızda hemen başka bir çıkmaza giriyoruz, Şekil 3-18(d)'de gösterildiği gibi. 9'a geri döndükten sonra, 11. düğümdeki çıkmaza ulaşana kadar devam ediyoruz. Şekil 3-18(e)'ye bakın. Yine geri dönüyoruz, bu sefer 12. düğüme. 13'e giden yolu takip ederek, yığın içine bir karar düğümü (B17) ekliyoruz ve hedefimizi bulana kadar devam ediyoruz, 16. düğümde.

Artık hedefimize giden yolu biliyoruz, sadece bunu yazdırmamız gerekiyor. Yazdırma döngüsü, yığını pop'layarak yolu yazdırır. Yığında kalan herhangi bir geri izleme noktasını, örneğin Şekil 3-18(f)'deki B17'yi göz ardı ediyoruz.



ŞEKİL 3-18 Geri İzleme Yığın İşlemi

Çözümü algoritmik biçimde sunuyoruz. 12. Bölümde grafiklerin oluşturulması ve depolanmasını incelemeden, C kodunu geliştiremeyiz. Pseudocode çözümü Algoritma 3-12'de bulunmaktadır.

ALGORİTMA 3-12 Hedefe Giden Yolu Yazdır

```

Algoritma seekGoal (harita)
Bu algoritma, istenen bir hedefe giden yolu belirler.
Bir grafiği içeren yol
Yol yazdırıldı
1   createStack (yığın)
2   pMap'i başlangıç noktasına ayarla
3   döngü (pMap null değil VE hedefBulunamadı)
    1   eğer (pMap hedefse)
        1   hedefBulunamadı'yı false olarak ayarla
    2   aksi takdirde
        1   pushStack (yığın, pMap)
        2   eğer (pMap bir dal noktasıysa)
            1   döngü (daha fazla dal noktası)
                1   dalNoktası düğümü oluştur
                2   pushStack (yığın, dalNoktası)
            2   döngüyü sonlandır
        3   eğer sonlandır
        4   bir sonraki düğüme ilerle
    3   eğer sonlandır
4   döngüyü sonlandır
5   eğer (stack boşsa)
    1   print (Hedefinize giden bir yol yok)
6   aksi takdirde
    1   print (Hedefinize giden yol:)
    2   döngü (boşStack (stack) değil)
        1   popStack (yığın, pHarita)
        2   if (pHarita bir branchPoint değilse)
            1   print(harita noktası)
        3   eğer sonlandır
    3   döngüyü sonlandır
    4   print (Yolun Sonu)
7   eğer sonlandır
8   geri dönmek
son aramaHedefi

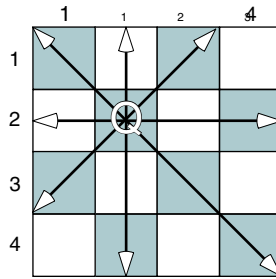
```

Algoritma 3-12 Analizi Bu algoritma, pseudocode'un değerlerinden birini ve kullanıcılarla algoritmalar tartışırken neden popüler olduğunu göstermektedir. Bu problemi bir bilgisayarda çözmek için 11. Bölüm (grafikler) kadar yeterli veri yapısı bilgisine sahip olamayacaksınız. Ancak, bunu çözmek için bir veri yapısına sahip olmasak bile, algoritmayı geliştirebilir ve anlayabiliriz.

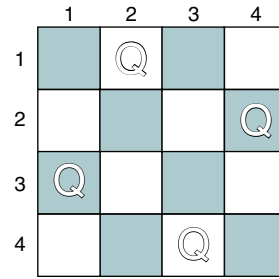
Sekiz Kraliçe Problemi

Bir klasik satranç problemi, sekiz kraliçeyi satranç tahtasına yerleştirmenizi gerektirir; bu şekilde hiçbir kraliçe diğer bir kraliçeyi yakalayamaz. Aslında bu probleme birkaç farklı çözüm vardır. Problemin bilgisayar çözümü, tahtaya bir kraliçe yerleştirmemizi ve ardından yeni kraliçeyi yakalayabilecek bir kraliçe olup olmadığını görmek için tüm saldırı pozisyonlarını analiz etmemizi gerektirir. Eğer varsa, o zaman başka bir pozisyon deneriz.⁵

Örnek bir çözümü göstermek için, dört kraliçeyi 4×4 satranç tahtasına nasıl yerleştireceğimizi analiz edelim. Kraliçenin yakalama kuralları ve bir çözüm Şekil 3-19'da gösterilmiştir.



(a) Kraliçe yakalama kuralları



(b) İlk dört kraliçe çözümü

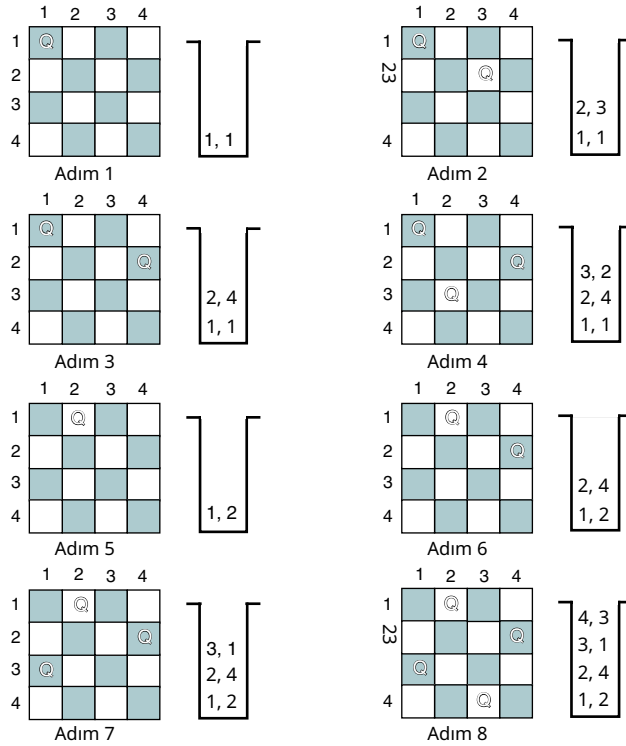
ŞEKİL 3-19Dört Kraliçe Çözümü

Bu problemi bir yığın ve geri izleme mantığı kullanarak çözebiliriz. Herhangi bir satıra yalnızca bir kraliçe yerleştirilebileceğinden, ilk kraliçeyi 1. satır, 1. sütuna yerleştirerek başlıyoruz. Bu konum daha sonra bir yığına itilir ve Şekil 3-20'de gösterilen pozisyonu verir, adım 1.

İlk satıra bir kraliçe yerleştirdikten sonra, ikinci satırda bir pozisyon arıyoruz. Pozisyon 2,1 mümkün değildir çünkü ilk satırdaki kraliçe bu konumu dikey olarak korumaktadır. Benzer şekilde, pozisyon 2,2 çapraz olarak korunmaktadır. Bu nedenle, 2. satırda üçüncü sütuna bir kraliçe yerleştiriyoruz ve bu konumu yığına itiyoruz. Bu pozisyon Şekil 3-20'de, adım 2'de gösterilmiştir.

Şimdi 3. satırda bir pozisyon bulmaya çalışıyoruz, ancak hiçbiri mümkün değil. İlk sütun, 1. satırdaki kraliçe tarafından korunuyor ve diğer üç pozisyon 2. satırdaki kraliçe tarafından korunuyor. Bu noktada, yığınınından çıkararak ikinci satıra geri dönmeli ve ikinci satırdaki kraliçe için bir pozisyon aramaya devam etmeliyiz. 4. sütun korunmadığı için oraya bir kraliçe yerleştiriyoruz ve konumunu yığına ekliyoruz (Şekil 3-20'deki adım 3).

5. Bir kraliçe, aynı satırda, aynı sütunda veya aynı diyagonalde başka bir kraliçeyi saldırabilir.



ŞEKİL 3-20Dört Kraliçe Adım Adım Çözümü

3. satıra tekrar baktığımızda, ilk sütunun hala 1. satırdaki kraliçe tarafından k orunduğunu ancak ikinci sütuna bir kraliçe yerleştirebileceğimizi görüyoruz. Bunu yapıyoruz ve bu konumu yığına ekliyoruz (Şekil 3-20'deki adım 4).

Ancak 4. satıra bir kraliçe yerleştirmeye çalıştığımızda, tüm pozisyonların k orunduğunu buluyoruz. 1. sütun, 1. satırdaki kraliçe ve 3. satırdaki kraliçe tarafından korunuyor. 2. sütun, 2. satırdaki kraliçe ve 3. satırdaki kraliçe tarafından korunuyor. 3. sütun, 3. satırdaki kraliçe tarafından korunuyor ve 4. sütun, hem 1. satırdaki kraliçe hem de 2. satırdaki kraliçe tarafından korunuyor. Bu nedenle, 3. satırdaki kraliçeye geri dönüyoruz ve onun için başka bir yer bulmaya çalışıyoruz. 2. sıradaki kraliçe hem 3. sütunu hem de 4. sütunu koruduğu için, 3. sırada bir kraliçe için hiçbir pozisyon yoktur. Tekrar geri dönüyoruz, yığını çıkarıyoruz ve 2. sıradaki kraliçenin başka gidecek yeri olmadığını görüyoruz, bu yüzden 1. sıradaki kraliçeye geri dönüyoruz ve onu 2. sütuna taşıyoruz. Bu pozisyon Şekil 3-20, adım 5'te gösterilmiştir.

2. sırayı analiz ettiğimizde, 1. sıradaki kraliçenin ilk üç pozisyonu k orduğu için kraliçe için tek mümkün pozisyonun 4. sütun olduğunu görüyoruz. Bu nedenle, kraliçeyi bu konuma yerleştiriyoruz ve konumu yığına ekliyoruz (Şekil 3-20'de adım 6).

Üçüncü satırdaki 1. sütun korumasızdır, bu yüzden oraya bir kraliçe yerleştiriyoruz (Şekil 3-20'de adım7). Dördüncü satıra geçtiğimizde, ilk iki pozisyonun korunduğunu görüyoruz, ilki 3. satırdaki kraliçe tarafından ve ikincisi üç kraliçe tarafından. Ancak üçüncü sütun korumasızdır, bu yüzden bu sütuna dördüncü kraliçeyi yerleştirerek sorunun çözümünü bulabiliriz.

Çözümü genelleştirerek, bir satırda bir kraliçeyi bir pozisyona yerleştirdiğimizi ve ardından bir sonraki satırdaki tüm pozisyonları inceleyerek oraya bir kraliçe yerleştirip yerleştiremeyeceğimizi görüyoruz. Eğer bir sonraki satıra bir kraliçe yerleştiremiyorsak, son yerleştirilen kraliçeye geri döneriz ve onu bir sonraki sütuna yerleştirmeye çalışırız. Eğer bir sonraki sütunda yer yoksa, tekrar geri döneriz. Bir çözüm olduğu göz önüne alındığında,⁶ bu deneme-yanılma yöntemi iyi çalışıyor. Biryığın soyut veri türümüzü kullanan bir pseudocode çözümü Algoritma 3-13'te gösterilmiştir.

ALGORİTMA 3-13 Sekiz Kraliçe Problemi

Algoritma queens8 (tahtaBoyutu)

Bir oyun tahtasında satranç kraliçelerini yerleştirerek hiçbir kraliçenin başka bir kraliçeyi ele geçirememesini sağlayın.

Önce tahtaBoyutu, tahtadaki satır ve sütun sayısıdır

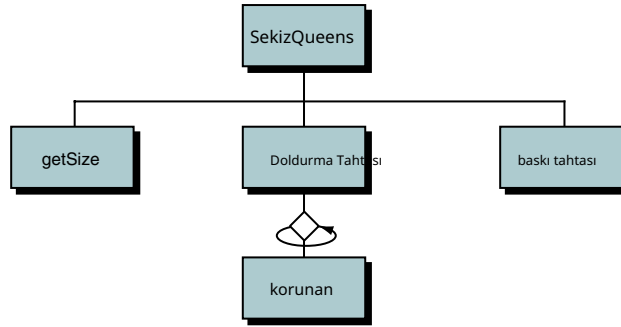
Sonra kraliçelerin pozisyonları yazdırılır

```

1 createStack (yığın)
2 satırı 1 olarak ayarla
3 sütunu 0 olarak ayarla
4 döngü (satır <= tahtaBoyutu)
    1 döngü (sütun <= tahtaBoyutu VE satır <= tahtaBoyutu)
        1 sütun artır
        2 eğer (korunmamış (satır, sütun) değilse)
            1 tahtada satır-sütun kesişiminde kraliçeyi yerleştir
            2 yığın (satır-sütun yığına ekle)
            3 satırı artır
            4 sütunu 0 olarak ayarla
        3 eğer sonlandır
            Satırın sonunda. Önceki konuma geri dön.
        4 döngü (sütun >= tahtaBoyutu)
            1 yığın (satır-sütun yığından çıkar)
            2 tahtada satır-sütun kesişiminden kraliçeyi çıkar
        5 döngüyü sonlandır
    2 döngüyü sonlandır
5 döngüyü sonlandır
6 tahtayı yazdır (yığın)
7 geri dönmek
son queens8
```

6. 4 x 4 pozisyonlardan daha küçük tahtalar için çözüm yoktur. 4 x 4'ten 8 x 8'e kadar olan tüm tahtaların en az bir çözümü vardır.

Şimdi sekiz kraliçe problemini uygulamak için bir C programı geliştiriyoruz. Program, kullanıcıdan tahta boyutunu isteyerek başlar ve ardından kraliçelerin pozisyonlarını saklamak için bir yığın oluşturur. Yığını oluşturduktan sonra, ana satır, tahta boyutuna göre maksimum sayıda kraliçeyi tahtaya yerleştiren bir fonksiyonu çağırır. Ardından sonuçları yazdırır. Tasarım Şekil 3-21'de gösterilmiştir.



ŞEKİL 3-21 Sekiz Kraliçe Tasarımı

Tasarıma göre, yalnızca bir başka fonksiyon, *guarded*, gereklidir. Bu, yeni bir kraliçenin potansiyel yerleştirmesinin etrafındaki her pozisyon incelendiğinde çağrılır.

Bir program tasarlarken, genellikle teknik ve kullanıcı bakış açıları arasındaki farklılıkları çözmek gerekir. Sekiz kraliçe programında, bu farklılık kraliçeleri tahtaya yerleştirmek için kullanılan terminolojide görülmektedir. Kullanıcılar satırları ve sütunları 1'den başlatarak numaralandırır; C ise bunları 0'dan başlatarak numaralandırır. Bu farkı çözmek için, tahta dizisine bir sahte satır ve sütun ekledik ve 9×9 tahtası oluşturduk. İşlem sırasında satır 0 ve sütun 0'ı göz ardı ediyoruz.

Küresel bildirimler ve ana satır kodu Program 3-20'de gösterilmiştir.

PROGRAM 3-20 Sekiz Kraliçe Ana Satırı

123456789	<pre> /* Bu program sekiz kraliçe algoritmasını test eder. Sekiz kraliçe, sekiz kraliçenin bir satranç tahtasında öyle yerleştirildiği klasik bir satranç problemidir ki, hiçbir kraliçe diğer bir kraliçeyi yakal ayamaz. Yazarı: Tarih: */ #include <stdio.h> #include <stdlib.h> #include "P4StkADT.h" // Yapı Bildirimleri </pre>
101112	

devam ediyor

PROGRAM 3-20Sekiz Kraliçe Ana Hattı (devamı)

```

131415161718192021222324252627282930313233343536373839404142434445
typedef yapısı
{
    int satır;
    int sütun;
} POZİSYON;

// Prototip Bildirimleri
int getSize (void);

void fillBoard (STACK* yığın, int tahtaBoyutu);
void printBoard (STACK* yığın, int tahtaBoyutu);

bool guarded (int tahta[][9], int satır,
               int sütun, int tahtaBoyutu);

int main (void)
{
    // Yerel Tanımlar
    int tahtaBoyutu;

    STACK* yığın ;

    // İfadeler
    tahtaBoyutu = getSize ();
    yığın = createStack ();

    fillBoard (yığın, tahtaBoyutu);
    printBoard (yığın, tahtaBoyutu);
    destroyStack (yığın);

    printf("Umarız Sekiz Kraliçe oyununu beğenmişsinizdir.");retu
    rn 0;
} // ana

```

Klavye üzerinden tahta boyutunu okumak için fonksiyonProgram 3-21'de g
österilmiştir.

PROGRAM 3-21Sekiz Kraliçe: Tahta Boyutunu Al

```

1234567
/* ===== getSize =====
   Kullanıcıdan geçerli bir tahta boyutu isteme.
   Önce hiçbir şey
   Sonra geçerli tahta boyutu döndürüldü
*/
int getSize (geçersiz)
{

```

devam ediyor

PROGRAM 3-21 Sekiz Kraliçe: Tahta Boyutunu Al (devam ediyor)

```

89 // Yerel Tanımlar
    int boardSize;

101121314151617181920212223242526272829 // İfadeler
    printf("Sekiz Kraliçe'ye hoş geldiniz. 4 x 4 ile 8 x 8 arasında "
        "bir tahta boyutu seçebilirsiniz. Ben de " "her satıra bi
        r kraliçe yerleştireceğim" "böylece hiçbir kraliçe diğer
        ini" "yakalamayacak. Not: 4 x 4'ten daha küçük"
        "tahtalar için çözüm yoktur.");

    printf("Lütfen tahta boyutunu girin: ");scanf ("%d", &b
    oardSize);
    while (boardSize < 4 || boardSize > 8) {

        printf("Tahta boyutu 3'ten büyük olmalı " "ve 9'dan
        küçük olmalı. Girdiğiniz %d." "Lütfen tek
        rar girin. Teşekkürler.\a\a" "Tahta boyutu
        nuz: ", boardSize);
        scanf ("%d", &boardSize);
    } // while
    boardSize'ı döndür;
} // getSize

```

Programdaki en uzun fonksiyon `fillBoard`. Tahtaya maksimum sayıda kraliçe yerleştirmek için deneme-ve-hata kavramını kullanarak döngü yapar. Bir satırdaki her olası pozisyon `guarded` çağrılarak test edilir. Pozisyon iyi ise, konumu yığına yerleştirilir ve bir sonraki satır analiz edilir. Eğer iyi değilse, yığın çıkarılır ve önceki satırda başka bir pozisyon denemek için geri çekilir. Tüm satırlarda bir kraliçe olduğunda, problem çözülmüş olur. Kod Program 3-22'de gösterilmiştir.

PROGRAM 3-22 Sekiz Kraliçe: Tahtayı Doldur

```

123456789 /* ===== fillBoard =====
    Oyun tahtasında satranç kraliçelerini yerleştirerek hiçbir kraliçenin başka bir kraliçeyi yakalayamayacağı şek
    ilde yerleştirin.
    Önce boardSize kadar satır ve sütun tahtada
    Sonra Kraliçelerin pozisyonları dolduruldu

*/
void fillBoard (STACK* yığını, int boardSize)
{
1011213 // Yerel Tanımlar
    int row;
    int col;
    int pano[9][9] = {0}; // 0 kraliçe yok: 1 kraliçe
                                // satır 0 & sütun 0 kullanılmadı

```

devam ediyor

PROGRAM 3-22 Sekiz Kraliçe: Tahtayı Doldur (devam ediyor)

```

141516171819202122232425262728293031323334353637383940414243444546474849
    POZİSYON* pPos;

    // İfadeler
    satır = 1;
    sutun = 0;

    while (satır <= tahtaBoyutu)
    {
        while (sutun <= tahtaBoyutu && satır <= tahtaBoyutu)
        {
            sutun++;
            if (korunan(tahta, satır, sutun, tahtaBoyutu))
            {
                tahta[satır][sutun] = 1;

                pPos = (POZİSYON*)malloc(sizeof(POZİSYON));
                pPos->satır = satır;
                pPos->col = col;    pu
                shStack(stack, pPos);    row
                ++;

                sütun = 0;
            } // eğer
        } while (sutun >= panoBoyutu)
        {
            pPos = popStack(yığın);
            row = pPos->row;
            col = pPos->col;
            board[row][col] = 0;
            free (pPos);
        } // while col
    } // while row
    return;
} // fillBoard

```

Program 3-22 Analizi Bu geri izleme probleminde, tahtaya yerleştirdiğimiz son kraliçeyi içeren satıra ve sütuna geri dönmemiz gerekiyor. 30 ve 31. ifadelerde, satırı ve sütunu bir yapıda kaydediyoruz ve ardından pozisyonu 33. ifadede pushStack fonksiyonuna geçiriyoruz.

38. ifadede yer alan while döngüsünü dikkatlice inceleyin. Bu, bir satırda pozisyon bulunmadığında geri izlemek için kullanılır. Sütun, tahta boyutuna eşit veya daha büyük olduğunda geri izlememiz gerekiyor; bu, satırdaki tüm pozisyonları incelediğimiz ve gereksinimleri karşılayamayan hiçbir pozisyon bulamadığımızı gösterir. (Eğer korumasız bir pozisyon bulsaydık, 35. ifadede satırı ilerletmiş olacaktık.)

Korunan mantık Program 3-23'te gösterilmiştir. Yeni kraliçenin güvenli olup olmadığını belirlemek için zorlayıcı, deneme-yanılma yöntemi kullanır. Sadece yukarıdan saldıran kraliçeleri test etmemiz gerektiğini unutmayın. Olamaz

mevcut satırın altındaki kraliçeler. Eğer kraliçe güvende değilse, true döneriz—onun mevcut pozisyonu korunmaktadır. Eğer güvendeyse, false döneriz.

PROGRAM 3-23 Sekiz Kraliçe: Korunan

```

123456789      /* ===== korunan =====
                  Sıraları, sütunları, diyagonal çizgileri koruyan kraliçeleri kontrol eder

                  Önceki tahta, kraliçelerin mevcut konumlarını içerir
                  chkRow & chkCol yeni kraliçenin konumudur
                  boardSize, tahtadaki satır ve sütun sayısıdır
                  Sonuç, korunduysa true; değilse false döner

*/
101112131415161718192021222324252627282930313233343536373839
bool korunan (int tahta[][9], int chkRow,
               int chkCol, int tahtaBoyutu)
{
    // Yerel Tanımlar
    int satır;
    int sütun;

    // İfadeler

    // Mevcut sütunda bir kraliçe kontrol et
    sütun = chkCol;
    for (satır = 1; satır <= chkRow; satır++)
        if (tahta[satır][sütun] == 1)
            doğruyu döndür;

    // Sağ-üst diyagonal kontrol et
    for (satır = chkRow - 1, sütun = chkCol + 1;
         satır > 0 && sütun <= tahtaBoyutu;
         satır--, sütun++)
        if (tahta[satır][sütun] == 1)
            return true;

    // Sol-üst diyagonal kontrol et
    for (satır = chkRow - 1, sütun = chkCol - 1;
         satır > 0 && sütun > 0;
         satır--, sütun--)
        eğer (tahta[satır][sütun] == 1)
            doğru döndür;

    yanlış döndür;
} // korumalı

```

printBoard fonksiyonu, tahtadaki kraliçelerin konumunu basitçe ya zdırır. Örnek bir çıktı Program 3-24'ün sonunda gösterilmiştir.

PROGRAM 3-24 Sekiz Kraliçe: Tahtayı Yazdır

```

1 23456789
1011213141516171819202122232425262728293031323334353637383940414243444546

/* ===== printBoard =====
Satranç kraliçelerinin oyun tahtasındaki konumlarını yazdır
Ön yığın kraliçenin konumlarını içerir
tahtaBoyutu satır ve sütun sayısını belirtir
Son Kraliçelerin konumları yazdırıldı
*/

void printBoard (STACK* yığın, int boardSize)
{
// Yerel Tanımlar
int col;

POSITION* pPos;
STACK* pOutStack;

// İfadeler
if (emptyStack(yığın))
{
printf("Bu tahtada hiçbir pozisyon yok"); return;

} // eğer

printf("Kraliçeleri aşağıdaki pozisyonlara yerleştirin:");// Yazdırma

için yığını ters çevir
pOutStack = createStack ();while (!
emptyStack (stack)) {

pPos = popStack (yığın);
pushStack (pOutStack, pPos);
} // while

// Şimdi tahtayı yazdır
while (!emptyStack (pOutStack))
{
pPos = popStack (pOutStack); printf("Satır %d-Sütun %d: |",

pPos->row, pPos->col);

for (col = 1; col <= boardSize; col++)
{
if (pPos->col == col)
printf(" Q |");
else
printf(" |");
} // for
printf(""); } /
/ while

```

devam ediyor

PROGRAM 3-24 Sekiz Kraliçe: Tahtayı Yazdır (devam ediyor)

```

47/48/49      destroyStack(pOutStack);
               return;
               } // baskı tahtası

```

Sonuçlar:

Eight Queens'a hoş geldiniz. 4 x 4 ile 8 x 8 arasında bir tahta boyutu seçebilirsiniz. Daha sonra tahtanın her satırına bir kraliçe yerleştireceğim, böylece hiçbir kraliçe diğer bir kraliçeyi yakalayamaz. Not : 4 x 4'ten daha küçük tahtalar için çözüm yoktur.

Lütfen tahta boyutunu girin: 4

Kraliçeleri aşağıdaki pozisyonlara yerleştirin:

```

Satır 1-Sütun 2:  | Q |  |  |
Satır 2-Sütun 4:  |  |  | Q |
Satır 3-Sütun 1: Q |  |  |  |
Satır 4-Sütun 3:  |  | Q |  |

```

Eight Queens'den keyif aldığınızı umuyoruz.

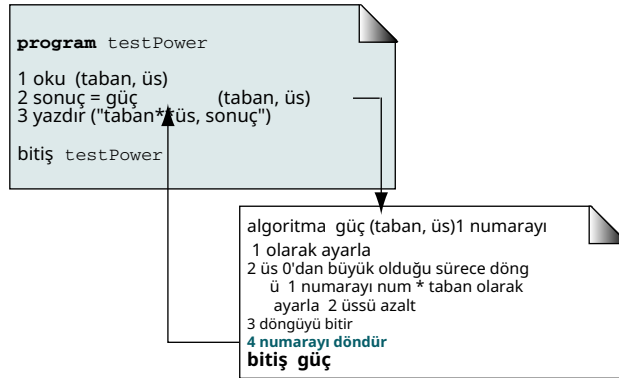
3.6 Rekürsiyonun Çalışma Şekli

Bölüm 2'de rekürsiyonu tartıştık ama fiziksel olarak nasıl çalıştığını açıkla madık. Artık yığınları tartıştığımıza göre, rekürsiyonun fiziksel işlemini tartışabiliriz.

Rekürsiyonun nasıl çalıştığını anlamak için, önce herhangi bir çağrının nasıl çalıştığını keşfetmemiz gerekiyor. Bir program bir alt programı çağırdığında —örneğin, C'deki bir fonksiyon—mevcut modül işlemi askıya alır ve çağrılan alt program programın kontrolünü devralır. Alt program işlemini tamamladığında ve çağrıldığı modüle döndüğünde, modül uyanır ve işlemini sürdürür. Bu etkileşimde önemli bir nokta, referansla çağrı yoluyla değiştirilmediği sürece, çağrılan modüldeki tüm yerel verilerin değişmeden kalmasıdır.

Her yerel değişken, işleme devam edildiğinde, askıya alındığı zamanki durumunda olmalıdır. Benzer şekilde, parametre listesi değiştirilmemelidir. Parametrelerin değeri, bir çağrıdan önce ve sonra aynı olmalıdır, referans parametreleri hariç.

Basit bir çağrının örneğine bakalım ve nasıl çalıştığını görelim. Şekil 3-22, bir sayının x üssü y değerini yazdıran `testPower` adlı bir algoritmayı içermektedir. Fonksiyon çağrısını göstermek için, x^y değerini belirlemek için ayrı bir algoritma olan `power` kullanıyoruz.



ŞEKİL 3-22 Çağrı ve Dönüş

Küçük programımız üç yerel değişken içerir: taban sayısı, base; üs tel, exp; ve cevap, result. Taban ve üstel değerler, power çağrısı ile değiştirilmemelidir.⁷ power sona erdiğinde, x^y değerini döndürür, bu değer yerel değişken olan result içinde saklanır.

Güç çalışmaya başladığında, parametrelerin değerlerini bilmesi gerekir, böylece bunları işleyebilir. İşlemi tamamlandığında geri dönmesi gereken yeri de bilmelidir. Son olarak, güç bir değer döndürdüğü için, sona erdiğinde değer nereye yerleştirileceğini bilmelidir. Fiziksel uygulama derleyici yazarına bağlıdır, ancak bu veriler kavramsal olarak bir yığın çerçevesinde yer alır. Güç çağrıldığında, yığın çerçevesi oluşturulur ve bir sistem yığına itilir. Sonlandığında, yığın çerçevesi çıkarılır, yerel değişkenler değiştirilir, dönüş değeri saklanır ve çağıran algoritma işleme devam edilir.

Bir yığın çerçevesi dört farklı öge içerir:

1. Çağrılan algoritma tarafından işlenecek parametreler
2. Çağıran algoritmadaki yerel değişkenler
3. Çağıran algoritmadaki dönüş ifadesi
4. Dönen değeri alacak ifade (varsa)

Artık bir çağrının nasıl çalıştığını anladığınıza göre, özyinelemenin nasıl çalıştığını görebilirsiniz. Algoritma 3-14, daha önce tartışılan güç algoritmasının özyinelemeli bir versiyonunu içermektedir.

7. Bu tartışmada, özyinelemeli bir derleyici varsayıyoruz. COBOL ve FORTRAN gibi diller, yerel değişkenlerin olmadığı farklı, özyinelemeli olmayan bir tasarım altında çalışır.

ALGORİTMA 3-14Özyinelemeli Güç Algoritması

Algoritma güç (taban, üstel)

Bu algoritma, bir sayının, taban, bir üssün, üstel, kuvvetine yükseltilmiş değerini hesaplar.

Ön taban, yükseltilmesi gereken sayıdır

üstel, üssü temsil eder

Son tabanın üstel kuvvetine yükseltilmiş değeri hesaplandı

Tabanın üstel kuvvetine yükseltilmiş değeri döndürüldü

1 eğer (üstel 0 ise)

1 dön (1)

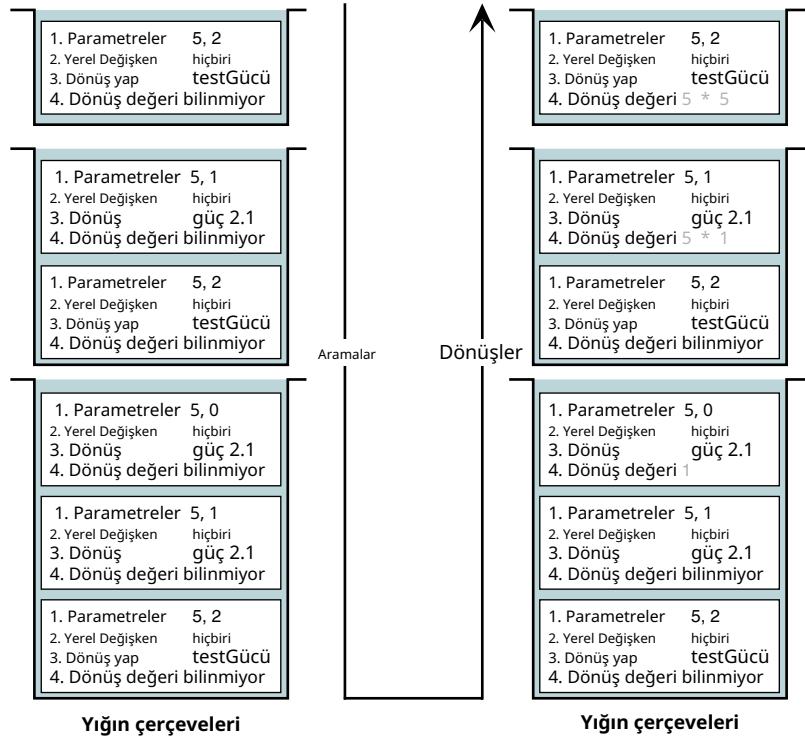
2 aksi takdirde

1 dön (taban * güç (taban, üstel - 1))

3 eğer sonlandır

son güç

Şekil 2-4'te, özyinelemeli faktöriyel algoritması için çağrılarını izledik. Bu örnek mantıksal olarak doğrudur, ancak çağrılarını aşırı basitleştirir. Şekil 3-23 Algoritma 3-14'ün özyinelemeli versiyonunun yürütülmesini izler ve her çağrı için sistem yığınındaki içerikleri gösterir.



ŞEKİL 3-23Güç (5, 2) için Yığın Çerçeveleri

Şekil 3-23'e atıfta bulunarak, güç başlangıçta `testPower` tarafından çağrılır vetaban ve üs sırasıyla 5 ve 2 olarak ayarlanır. İlk yığın çerçevesi orijinal çağrı tarafından oluşturulmuş ve çağırana algoritmadaki bir dönüş konumunu içerir, `testPower`. İkinci çağrıda, taban değişimden kalır ancak 1 olur. Bu çağrı, Şekil 3-23'te ikinci yığın çerçevesini oluşturur. Parametrelerin değiştiğini ve dönüş adresinin artık güç 2.1 olduğunu unutmayın. Üçüncü yinelemede, üstel 0 olur. Bu çağrı üçüncü `stackframe`'de gösterilmektedir.

Bu noktada temel duruma ulaştık ve geri dönmeye başlıyoruz özyineleme. Her dönüş ifadesi yürütüldüğünde, 2.1 ifadesine geri dönüyoruz. güç, önce 1 (temel durum) değeri ile, sonra 5 (5×1) değeri ile, ve nihayet 25 (5×5) değeri ile geri döneriz. Üçüncü kez geri döndüğümüzde, orijinal çağırma algoritmasına geri dönüyoruz, `testPower`.

3.7 Anahtar Terimler

geri izlemeyiğ	patlat
ın oluştıurver	yığın çıkar
i düğümü	it
yığını yok et	yığın ekle
boş yığın	kendine referans veren veri yapısı
dolu yığın	yığın
baş	yığın sayısıyığ
son giren–ilk çıkan (LIFO)	ın çerçevesiy
meta verisi	ıgın üstüü
taşma	st
parsing	alt akış

3.8 Özet

- ▣ Bir yığın, tüm eklemelerin bir uca, üst denilen yere kısıtlandığı doğrusal bir listedir. Yığın ayrıca LIFO listesi olarak da adlandırılır.
- ▣ Bir yığın için sekiz işlem tanımladık: yığın oluştıur, yığına ekle, yığından çıkar, yığın üstü, boş yığın, dolu yığın, yığın sayısı ve yığını yok et.
- ▣ Yığın oluştıur, yığın meta verilerini başlatır.
- ▣ Yığına ekle, yığının üstüne bir öge ekler. Ekledikten sonra, yeni öge üst olur.
- ▣ Her ekleme, yeni öge için yer olduğundan emin olmalıdır. Eğer yer yoksa, yığın taşma durumundadır.
- ▣ Pop yığını, yığının en üstündeki ögeyi kaldırır. Pop işleminden sonra, eğer varsa, bir sonrakiöge en üstte olur. Her pop işlemi, yığında en az bir öge olduğundan emin olmalıdır.Eğer en az bir öge yoksa, yığın bir alt akış durumundadır.
- ▣ Yığın üst işlemi yalnızca yığının en üstündeki ögeyi alır. Her üst yürütme, yığında en az bir öge olduğundan emin olmalıdır. En az bir öge yoksa, yığın bir alt akış durumundadır.
- ▣ Boş yığın, yığının boş olup olmadığını belirler ve eğer boşsa Boolean true döner.
- ▣ Dolu yığın, en az bir öge için yer olup olmadığını belirler ve varsa Boolean false döner.
- ▣ Yığın sayısı, yığında şu anda bulunan öge sayısını döner.

- Yığını yok et, tüm tahsis edilen veri belleğini dinamik belleğe serbest bırakır.
- Bir yığın uygulamalarından biri verileri tersine çevirmektir. Bir yığının doğası (son gir en-ilk çıkan) öğeleri bir yığına eklememize ve bunları ters sırayla çıkarmamıza ola nak tanır.
- Yığınlar genellikle verileri ayırıştırmak, verilerin kullanımını ertelemek ve bir yoldaki adımları geri almak için kullanılır.
- Bir modül bir alt yordamı özyinelemeli olarak çağırdığında, her çağrı da alt yordamın ihtiyaç duyduğu tüm bilgiler yığın çerçevesine itilir . Bilgiler, alt yordamlar birbiri ardına sona erdiğinde ters sırayla çık arılır ve nihayet kontrol başlangıçta çağrılan modüle geri döner.
- Bir yığın, özyinelemeli çağrıları kolaylaştırmak için kullanılır. Bir program bir alt yordamı çağırdığında, sistem a şağıdakileri depolamak için bir yığın çerçevesi oluşturur:
 1. Parametrelerin değeri
 2. Yerel değişkenlerin değeri
 3. Çağırılan modüldeki dönüş adresi
 4. Dönüş değeri

3.9 Pratik SetleriAlı

ştırmalar

1. İki boş tam sayı yığınınız olduğunu hayal edin, s_1 ve s_2 . Aşağıdaki işlemlerden sonra her yığının resmini çizin:

```

1 pushStack (s1, 3)
2 pushStack (s1, 5)
3 pushStack (s1, 7)
4 pushStack (s1, 9)
5 pushStack (s1, 11)
6 pushStack (s1, 13)
7 döngü boşStack değil (s1)
  1 popStack (s1, x)
  2 pushStack (s2, x)
8 döngüyü sonlandır
```

2. İki boş tam sayı yığınınız olduğunu hayal edin, s_1 ve s_2 . Aşağıdaki işlemlerden sonra her yığının resmini çizin:

```

1 pushStack (s1, 3)
2 pushStack (s1, 5)
3 pushStack (s1, 7)
4 pushStack (s1, 9)
5 pushStack (s1, 11)
6 pushStack (s1, 13)
7 döngü boşStack değil (s1)
  1 popStack (s1, x)
  2 popStack (s1, x)
  3 pushStack (s2, x)
8 döngüyü sonlandır

```

3. Manuel dönüşüm kullanarak, aşağıdaki infiks ifadeleri postfix ve prefix x formlarında yazın:
- $D - B + C$
 - $A * B + C * D$
 - $(A + B) * C - D * F + C$
 - $(A - 2 * (B + C) - D * E) * F$
4. Manuel dönüşüm kullanarak, aşağıdaki postfix veya prefix ifadeleri infix'e çevirin:
- $A B * C - D +$
 - $A B C + * D -$
 - $+ - * A B C D$
 - $- * A + B C D$
5. A, B, C ve D değerleri sırasıyla 2, 3, 4 ve 5 ise, aşağıdaki postfix ifadelerinin değerini manuel olarak hesaplayın:
- $A B * C - D +$
 - $A B C + * D -$
6. A, B, C ve D değerleri sırasıyla 2, 3, 4 ve 5 ise, aşağıdaki prefix ifadelerinin değerini manuel olarak hesaplayın:
- $+ - * A B C D$
 - $- * A + B C D$
7. Aşağıdaki infix ifadeleri postfix ifadelerine dönüştürün algoritmik yöntem (bir yığın) kullanarak:
- $D - B + C$
 - $A * B + C * D$
 - $(A + B) * C - D * F + C$
 - $(A - 2 * (B + C) - D * E) * F$

8. Aşağıdaki postfix ifadelerinin değerini belirleyin, değişkenlerin değerleri şunlardır: A 2, B 3, C 4 ve D 5.
- A B C D * - +
 - D C * B A + -

Problemler

- Algoritma 3-15, “Bir Sayı Serisini Tersine Çevir” uygulamak için bir program yazın. Programınızı 1, 3, 5, 7, 9, 2, 4, 6, 8 sayı serisi ile test edin.
- Bir ondalık sayıyı sekizli sayı olarak yazdıran bir algoritmanın pseudocode'unu yazın.
- Bir ondalık sayıyı onaltılık sayıya dönüştüren bir program yazın. (*İpucu:* Kalan 10, 11, 12, 13, 14 veya 15 ise, sırasıyla A, B, C, D, E veya F yazdırın.)
- Algoritma 3-9, “Parantezleri Ayırıştır” uygulamak için bir program yazın, parantezler yerine süslü parantezleri eşleştirin. Uygulamanızda, açılış süslü parantez yerine satır numarasını yığına itiniz. Bir hata oluştuğunda, eşleşmeyen açılış süslü parantez veya eşleşmeyen kapanış süslü parantez için satır numarasını yazdırın. Programınızı kaynak kodunu kendisi üzerinden çalıştırarak test edin (hata olmamalıdır) ve ardından aşağıdaki küçük program ile test edin:

```
Test brace hataları.
} 2. satır kapanış süslü parantezi eşleşmiyor
Parantez yok.
{açılış süslü parantezi aynı satırda eşleşiyor}
Parantez yok.
{açılış süslü parantezi daha sonra eşleşiyor
Parantez yok.
} Kapanış süslü parantezi iki satır yukarıda eşleşiyor.
{{{ 9. Satır. Üç süslü parantez--sadece ikisi eşleşiyor.
} İlk kapanış süslü parantez
} İkinci kapanış süslü parantez.
Programın sonu. Bir açılış süslü parantez kaldı.
```

- Infix'ten postfix notasyonuna (bkz. Algoritma 3-10, “Infix'i Postfix'e Dönüştür”) uygulayan bir program yazın. Program, değişkenler için tek harfli alfabetik karakterlerden, parantezlerden ve +, -, *, ve / operatörlerinden oluşan bir infix dizesini okumalı; dönüşüm algoritmasını çağırmalı; ve ardından elde edilen postfix ifade sini yazdırmalıdır. Bir algoritmayı dönüştürdükten sonra, başka bir infix dizesini dönüştürmek için döngüye girmelidir. Programınızı test etmek için, Programınız a Egzersiz 3'teki ifadeleri dönüştürün.

14. Problem 13'ü çok karakterli değişken tanımlayıcılarına vesayisal sabitlere izin veecek şekilde değiştirin, aşağıdaki ifadede gösterildiği gibi:

+ 18 * faktörü olup olmadığı

15. Postfix değerlendirmesini uygulamak için bir program yazın (bkz. Algoritma 3-11, "Postfix İfadelerin Değerlendirilmesi"). Program, yalnızca çok basamaklı sayısal veriler ve +, -, *, ve /operatörlerinden oluşan bir postfix dizesini okumalı; değerlendirme algoritmasını çağırmalı; ve ardından sonucu yazdırmalıdır. Her değerlendirme den sonra döngüye girmeli ve başka bir değerlendirme işlemi yapmalıdır. Programınızla aşağıdaki ifadeleri değerlendirin:

```
25 7 * 14 - 6 +
1 24 3 + * 41 -
2 37 4 + * 15 -
```

16. Bir yığın uygulamalarından biri geri izlemektir; yani, adımlarını geri takip etmektir. Örneğin, bir liste ögesini okumak istediğimizi hayal edin ve her seferinde negatif bir sayı okuduğumuzda geri izlememiz ve negatif sayının öncesindeki beş sayıyı yazdırmamız ve ardından negatif sayıyı atmamız gerekir.

Bu problemi çözmek için bir yığın kullanın. Sayıları okuyun ve negatif bir sayı okunana kadar (yazdırmadan) yığına itin. Bu noktada okumayı durdurun ve yığından beş ögeyi çıkarın ve yazdırın. Yığında a beşten az öge varsa, bir hata mesajı yazdırın ve programı durdurun.

Beş ögeyi yazdırdıktan sonra, verileri okumaya ve bunları yığına yerleştirmeye devam edin. Dosyanın sonu tespit edildiğinde, bir mesaj ve yığında kalan öğeleri yazdırın.

Programınızı aşağıdaki verilerle test edin:

```
1 2 3 4 5 -1 1 2 3 4 5 6 7 8 9 10 -2 11 12 -3 1 2 3 4 5
```

17. copyStack adında bir algoritmanın pseudocode'unu yazın, bu algoritma bir yığının içeriğini diğerine kopyalar. Algoritma, kaynak yığın ve hedef yığın olmak üzere iki yığın alır. Yığınların sırası aynı olmalıdır. (İpucu: Sıralamayı korumak için geçici bir yığın kullanın.)
18. catStack adında yeni bir ADT fonksiyonu yazın, bu fonksiyon bir yığının içeriğini diğerinin üzerine ekler. Fonksiyonunuzu test etmek için, ADT'yi kullanarak iki yığın oluşturan ve bunları yazdıran bir program yazın. Ardından yığınları birleştirip sonuç yığını yazdırmalıdır.
19. Bir palindrom, geriye ve ileriye aynı sonuçla okunabilen bir dizedir. Örneğin, aşağıdaki bir palindromdur:

Elba'yı görmeden önce bunu yapabildim.

Bir dizenin palindrom olup olmadığını test etmek için bir fonksiyon yazın. Yığına karakterleri birer birer itebilirsiniz. Dizenin sonuna ulaştığınızda, karakterleri çıkarabilir ve yeni bir dize oluşturabilirsiniz. Eğer iki dize tam olarak aynıysa, dize bir palindromdur. Palindromların boşluk, noktalama işaretleri ve büyük/küçük harf duyarlılığını göz ardı ettiğini unutmayın. Programınızı aşağıdaki test durumlarıyla test edin:

Git köpek

Hanımefendi, ben Adamım

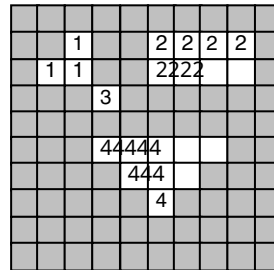
Hanımefendi, ben bir palindrom değilim

20. Bir metin dosyasını, bir satırda bir kez okuyup, okunduğu gibi satırı yazan ve ardından metni ters çevrilmiş olarak yazan bir program yazın. Her ters çevrilmiş satırdan sonra boş bir satır yazın.
21. Bir yığın içeriğini tersine çeviren bir algoritmanın pseudocode'unu yazın (üst ve alt elemanlar yer değiştirir, ikinci ve altıncı eleman yer değiştirir ve bu şekilde tüm yığın tersine çevrilene kadar devam eder). (*İpucu:* Geçici yığınlar kullanın.)
22. İki yığının içeriğinin aynı olup olmadığını kontrol eden bir fonksiyon yazın. Hiçbir yığın değiştirilmemelidir. Fonksiyonunuzun çalıştığını doğrulamak için bir yığının içeriğini yazdıran bir fonksiyon yazmalısınız.

Projeler

23. Bir kare matris verildiğinde, beyaz blokların sayısını ve her bir beyaz bloktaki toplam kare sayısını belirleyen bir program yazın. Tanıma göre, matrisin dış sınırları gölgelenmelidir. Beyaz karelerden oluşan bir blok, yan sınırları başka bir beyaz kareye bitişik olan tüm beyaz karelerden oluşur. Sadece bir köşede temas eden beyaz kareler bitişik sayılmaz.

Şekil 3-24'te, beyaz blokları numaralandırdık. Blok 1 üç kare içerir ve blok 4 dokuz kare içerir. Blok 3'ün yalnızca bir kare içerdiğini unutmayın. Blok 1 ile yalnızca köşeden temas eder.



ŞEKİL 3-24Proje 23: Beyaz Blokları Bul

Bir metin dosyasından kare tanımını alın. Dosya, gölgeli kareler için sıfırlardan ve beyaz kareler için sıfırdan farklı değerlerden oluşan bir kare matris içermelidir. Şekil 3-24 için girdi aşağıda gösterilmiştir. Matris dinamik bellekten tahsis edilmelidir.

```

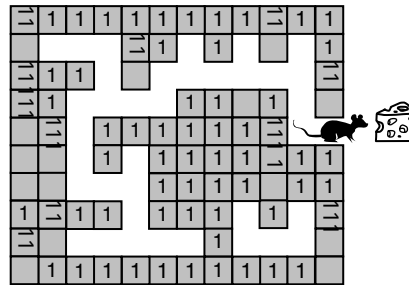
0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 2 2 2 2 0
0 1 1 0 0 2 2 2 2 0
0 0 0 3 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 4 4 4 4 4 0 0
0 0 0 0 4 4 4 0 0 0
0 0 0 0 0 4 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

```

Programın sonunda, beyaz blokların sayısını ve her birindeki kare sayısını gösteren bir rapor yazdırın.

24. Bir labirentte bir fareyi simüle eden bir program yazın. Program, farenin başlangıç noktasından son noktaya kadar aldığı yolu, ziyaret edilen ve geri dönülen tüm noktaları içerecek şekilde yazdırmalıdır. Böylece, bir nokta iki kez ziyaret edilirse, iki kez yazdırılmalıdır; üç kez ziyaret edilirse, üç kez yazdırılmalıdır.

Labirent Şekil 3-25'te gösterilmektedir. Farenin yolculuğuna başladığı giriş noktası, programı çalıştıran kullanıcı tarafından seçilir. Her seferinde değiştirilebilir.



ŞEKİL 3-25 Proje 24 için Fare Labirenti

Labirenti depolamak için destekleyici bir veri yapısı olarak iki boyutlu bir dizi kullanılabilir. Dizinin her bir elemanı siyah veya beyaz olabilir. Siyah bir eleman, farenin giremeyeceği bir kareyi temsil eder. Beyaz bir eleman, farenin kullanabileceği bir kareyi temsil eder.

Dizide siyah bir eleman 1 ile ve beyaz bir eleman 0 ile temsil edilebilir.

Fare labirenti geçerken, elemanları birer birer ziyaret eder. Diğer bir deyişle, fare labirenti bir eleman dizisi olarak düşünmez; yolculuğunun her anında yalnızca bir elemandadır. Bu elemanı `currentSpot` olarak adlandıralım. İki tam sayı alanından oluşan bir yapı ile temsil edilebilir. İlk alan satır ve ikinci alan labirentteki noktanın sütun koordinatıdır. Örneğin, Şekil 3-25'teki çıkış (5,12) konumundadır—yani, satır 5, sütun 12.

Program labirenti oluşturarak başlar. Ardından çıkış noktasını başlatır ve kullanıcıdan giriş noktasının koordinatlarını ister. Program sağlamalıdır. Kullanıcı bir siyah noktanın koordinatlarını girerse, program beyaz bir nokta girilene kadar yeni koordinatlar istemelidir. Fare giriş noktasından başlar ve çıkış noktasına ve ödülüne ulaşmaya çalışır. Ancak bazı başlangıç pozisyonlarının çıkışa ulaşmadığını unutmayın.

Fare yolculuğu boyunca ilerledikçe, yolunu yazdırın. Bir noktaya girdiğinde, program o noktanın sınıfını belirler. Bir noktanın sınıfı aşağıdakilerden biri olabilir:

- Devam Eden**—Bir nokta, yalnızca bir komşusu (son nokta hariç) beyaz nokta ise devam eden bir nokta olarak kabul edilir. Diğer bir deyişle, farenin yalnızca bir seçeneği vardır.
- Kesişim**—Bir nokta, iki veya daha fazla komşusu (son noktayı hariç tutarak) beyaz nokta ise bir kesişim noktasıdır. Diğer bir deyişle, farenin iki veya daha fazla seçeneği vardır.
- Çıkmaz**—Bir nokta, komşularından hiçbiri (son noktayı hariç tutarak) beyaz nokta değilse bir çıkmaz noktasıdır. Diğer bir deyişle, farenin seçebileceği bir nokta yoktur. Geri dönmek zorundadır.
- Çıkış**—Bir nokta, fare labirentten çıkabiliyorsa bir çıkış noktasıdır. Fare bir çıkış bulduğunda, özgürdür ve ödül olarak bir parça peynir alır.

Bu problemi çözmek için iki yığın gerekir. İlk yığın, ziyaret edilen yığın, farenin takip ettiği yolu içerir. Fare bir noktaya geldiğinde, önce çıkış olup olmadığını kontrol eder. Eğer değilse, konumu yığına yerleştirilir. Bu yığın, fare bir çıkmaza girdiğinde ve geri dönmesi gerektiğinde kullanılır. Fare son karar noktasına geri döndüğünde, geri dönüş yolunu da yazdırın.

Fare bir kesişime girdiğinde, alternatifler ikinci bir yığında yer alır. Bu karar noktası, ziyaret edilen yığında yer alan özel bir karar tokeni ile de işaretlenir. Karar tokeninin koordinatları $(-1, -1)$ şeklindedir. Bir yolu seçmek için, alternatiflerden biri alternatifler yığından çıkarılır ve fare yoluna devam eder.

Geri izleme sırasında, eğer fare bir karar belirtecine çarparsa, belirteç atılır ve alternatifler yığınının bir sonraki alternatif seçilir. Bu noktada, bir sonraki alternatif yolun seçildiğini göstermek için konumuna yanına bir yıldız (*) yazdırın.

Eğer fare bir çıkmaz sokağa ulaşır ve her iki yığın da boşsa, fare çıkışı olmay an bir labirentin bir kısmında kilitlenir. Bu durumda, bir tuzağa düşmüş mesajı yazdırın ve çıkış arayışını sonlandırın.

Her denemeden sonra, sonuca bakılmaksızın, kullanıcıya durma veya d evam etme fırsatı verilmelidir.

25. Bölüm 3.4'te tanımlanan yığın ADT'sini uygulayan bir program yazın, "Yığın AD T." Uygulamanızı test etmek için, ADT'deki her bir işlemi test etmek üzere me nü tabanlı bir kullanıcı arayüzü yazın. Test için, aşağıda açıklanan tamsayı verile rini kullanın. Hata koşulları test sonuçlarının bir parçası olarak yazdırılmalıdır. Ö nerilen bir menü aşağıda gösterilmiştir.

- A. Verileri yığına ekle
- B. Verileri çıkar ve yazdır
- C. Yığının en üstündeki veriyi yazdır
- D. Tüm yığını yazdır (üstten alta)
- E. Yığın durumu: Boş
- F. Yığın durumunu yazdır: Dolu
- G. Yığındaki eleman sayısını yazdır
- H. Yığını yok et ve çık

Programınızı aşağıdaki test durumu ile test edin. Ek test durumlar ı ekleyebilirsiniz. Menü işlemi testin sonunda gösterilmektedir.

- a. Yığın durumunu yazdır: Boş [E]
 - b. Veriyi çıkar ve yazdır (hata döndürmelidir) [B]
 - c. Yığına veri ekle: 1 [A]
 - d. Yığına veri ekle: 2 [A]
 - ve. Yığın durumunu yazdır: Boş [E]
 - f. Yazdırma yığını durumu: Dolu [F]
 - g. Yığın üstünde verileri yazdır [C]
 - h. Tüm yığını yazdır (üstten alta) [D]
 - Ben. Yığında eleman sayısını yazdır [G]
 - j. Veriyi çıkar ve yazdır [B]
 - k. Veriyi çıkar ve yazdır [B]
 - l. Veri yazdır ve yazdır (boş döndürmelidir) [B]
 - m. Yığına veri ekle: 3 [A]
 - n. Yığın üstünde verileri yazdır [C]
 - . Yığını yok et ve çık [H]
26. Sekiz kraliçe probleminin tüm çözümlerini bulmak için bir program y azın. Programınız, çözümü olmayan bir yapılandırma arayışını yönete bilmelidir.
27. Proje 25'te kullanıcı arayüzünü, iki farklı türde veriye sahip iki farklı yığınla man ipüle edecek şekilde değiştirin. İlk yığın, tam sayı verilerini içermelidir. İkinci yığın, alfabetik verileri içermelidir. Proje 25'teki test verilerine ek olarak, benzer bir işlem seti ile alfabetik yığını test eden bir alfabe tik test verisi seti oluşturun.

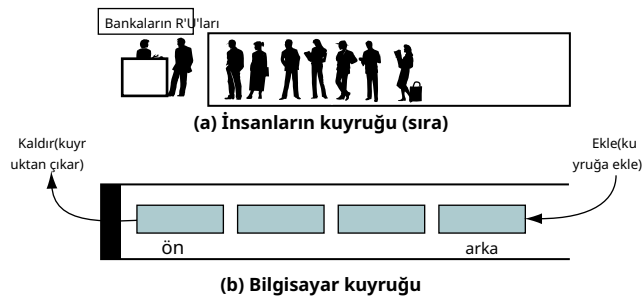
Bölüm 4

Kuyruklar

Bir kuyruk, verilerin yalnızca bir uçta, arka olarak adlandırılan bir uçta eklenebildiği ve diğer uçta, ön olarak adlandırılan bir uçta silinebildiği doğrusal bir listedir. Bu kısıtlamalar, verilerin kuyruğa alındıkları sırayla işlenmesini sağlar. Diğer bir deyişle, bir kuyruk, ilk giren-ilk çıkan (FIFO) bir yapıdır.

Bir kuyruk, bir sıraya eşdeğerdir. Aslında, eğer İngiltere'deyseniz, bir sıraya girmeyeceksiniz, bir kuyruğa gireceksiniz. Bir otobüs durağında otobüsü bekleyen insanların sırası bir kuyruktur, bir telefon operatörü tarafından yanıtlanmak üzere bekletilen çağrılarının listesi bir kuyruktur ve bir bilgisayar tarafından işlenmek üzere bekleyen işlerin listesi bir kuyruktur.

Şekil 4-1, bir kuyruğun iki temsilini gösterir: biri insanların kuyruğu ve diğeri bir bilgisayar kuyruğudur. Hem insanlar hem de veriler kuyruğa arka taraftan girer ve ön tarafa ulaşana kadar kuyrukta ilerler. Kuyruğun önüne geldiklerinde, kuyruktan çıkarlar ve hizmet alırlar.



ŞEKİL 4-1 Kuyruk Kavramı

4.1 Kuyruk İşlemleri

Dört temel kuyruk işlemi vardır. Veriler arka taraftan eklenebilir, ön taraftan silinebilir, ön taraftan alınabilir ve arka taraftan alınabilir. Yığınlar ve kuyruklar arasında birçok benzerlik olmasına rağmen, önemli bir yapısal fark, kuyruk uygulamasının kuyrukta ön ve arka tarafı taşıması gerektiğidir; oysa yığın sadece bir ucu, yani üst kısmı düşünmek zorundadır.

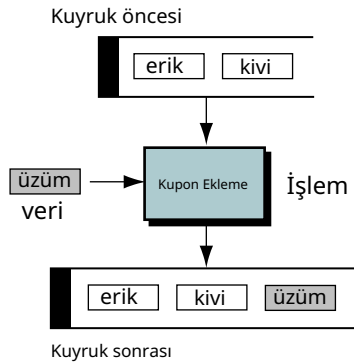
Bir kuyruk, verilerin bir uçtan, arka olarak adlandırılan taraftan eklenebildiği ve diğer uçtan, ön olarak adlandırılan taraftan silinebildiği doğrusal bir listedir. Bu, ilk giren-ilk çıkan (FIFO) yer kısıtlamalı veri yapısıdır.

Kupon Ekleme

Kuyruk ekleme işlemi kupon ekleme olarak bilinir. Veriler kuyruk için eklendikten sonra, yeni eleman arka taraf olur. Yığınlarda gördüğümüz gibi, kupon eklemenin tek potansiyel sorunu, veriler için yerin kalmasıdır. Kuyrukta başka bir eleman için yeterli yer yoksa, kuyruk tasma durumundadır.

Enqueue, bir öğeyi kuyruğun arkasına ekler.

Şekil 4-2, kupon ekleme işlemini göstermektedir.



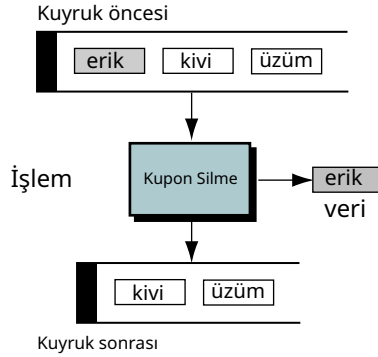
ŞEKİL 4-2 Kupon Ekleme

Kupon Silme

Kuyruk silme işlemi kupon silme olarak bilinir. Kuyruğun önündeki veriler, kullanıcıya geri döner ve kuyruktan silinir. Eğer kuyrukta kupon silme denemesi yapıldığında veri yoksa, kuyruk alt akış durumundadır.

Kupondan çıkarma, kuponun önündeki bir öğeyi siler.

Dequeue işlemi Şekil 4-3'te gösterilmiştir.



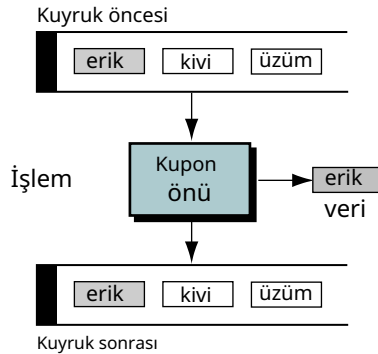
ŞEKİL 4-3 Dequeue

Kupon Önü

Kuponun önündeki veriler kupon önü ile alınabilir. Bu, kuponun önündeki veriyi döndürür ve kuponun içeriğini değiştirmez. Eğer kuponun önünde veri yoksa, kupon bir alt akış durumundadır.

Kupon önü, kuponun önündeki öğeyi alır.

Kupon ön işlemi Şekil 4-4'te gösterilmektedir.



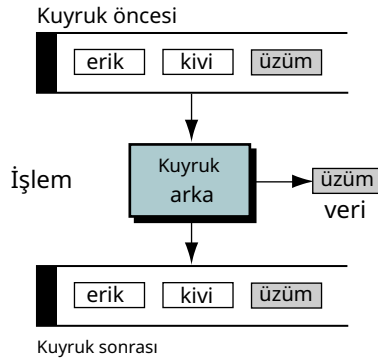
ŞEKİL 4-4 Kupon Önü

Kupon Arkası

Kuponun arkasındaki verileri almak için kupon önüne paralel bir işlem vardır. Bu kupon arkası olarak bilinir. Kupon önünde olduğu gibi, eğer kuponun arkasında veri yoksa, kupon bir alt akış durumundadır.

Kuyruk arka, kuyruğun arkasındaki öğeyi alır.

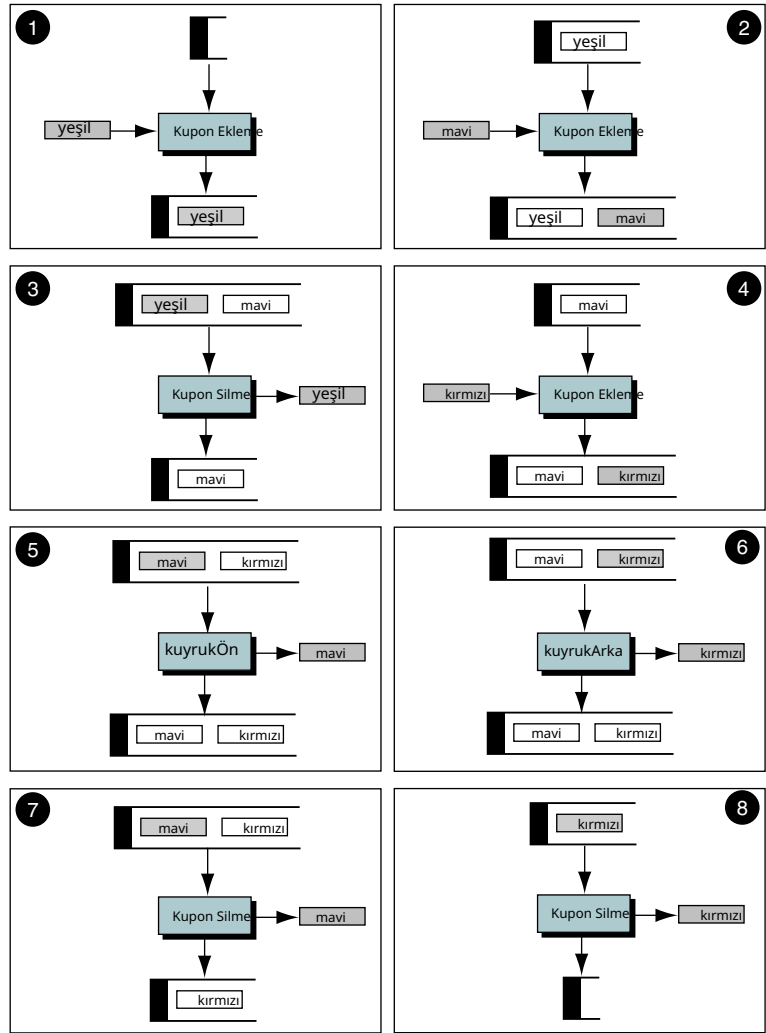
Kuyruk arka işlemi Şekil 4-5'te gösterilmiştir.



ŞEKİL 4-5 Kuyruk Arka

Kuyruk Örneği

Şekil 4-6, bu dört işlemi bir örnekte izlemektedir. Boş bir kuyruk ile başlıyoruz ve yeşil ve mavi ekliyoruz. Bu noktada kuyruk iki öğe içermektedir. Daha sonra, kuyruktaki ön girişini kaldıran dequeue işlemi uyguluyoruz, böylece mavi tek giriş olarak kalıyor. Kırmızı ekledikten sonra kuyruk ön işlemi mavi değerini çağırana döndürüyor ama onu kuyruktaki ön sırada bırakıyor. Sonraki işlem, kuyruk arka, kırmızı değerini döndürüyor ama onu kuyruktan bırakıyor. Bir dequeue işlemi ardından mavi'yi kaldırıyor, böylece kuyruktaki tek giriş olarak kırmızı kalıyor. Son olarak, kırmızıyı kaldırıyoruz, bu da boş bir kuyruk ile sonuçlanıyor.



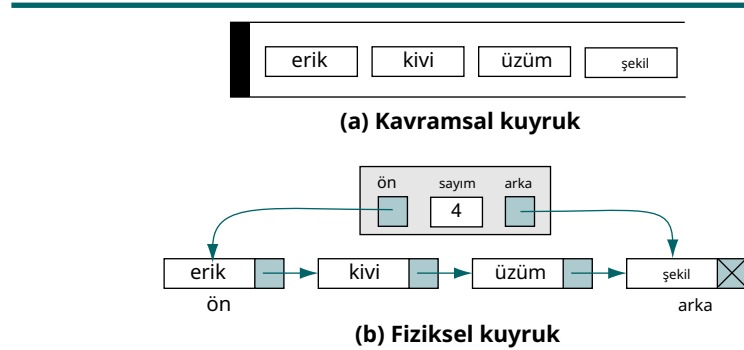
ŞEKİL 4-6 Kuyruk Örneği

4.2 Kuyruk Bağlı Liste Tasarımı

Bir yığın gibi, kuyruklarımızı bağlı liste kullanarak uyguluyoruz. Vurguladığımız gibi, gerçek uygulama farklı olabilir. Örneğin, Ek F'de kuyruk ADT'si için bir dizi uygulaması sağlıyoruz.

Veri Yapısı

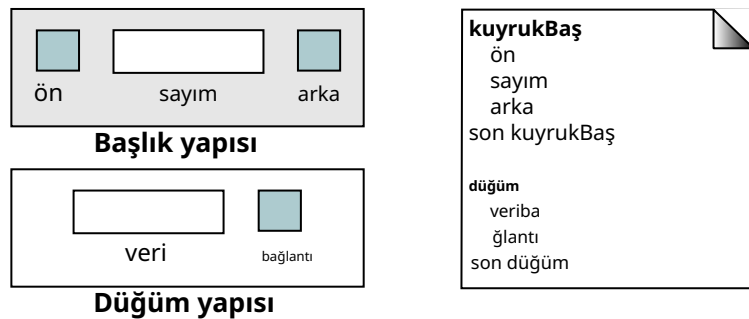
Kuyruğu uygulamak için iki farklı yapıya ihtiyacımız var: bir kuyruk başı yapısı ve bir veri düğümü yapısı. Oluşturulduktan sonra, kuyruk bir baş düğüm ve mevcut durumuna bağlı olarak sıfır veya daha fazla veri düğümüne sahip olacaktır. Şekil 4-7, kuyruk yapımız için kavramsal ve fiziksel uygulamaları göstermektedir.



ŞEKİL 4-7 Kavramsal ve Fiziksel Kuyruk Uygulamaları

Kuyruk Başlığı

Kuyruk, iki işaretçi ve bir sayım gerektirir. Bu alanlar, kuyruk başlığı yapısında saklanır. Kuyruğun başlık düğümünde, kuyruğa daha önce mevcut olan maksimum öge sayısı ve kuyruktan işlenmiş toplam öge sayısı gibi diğer kuyruk özellikleri, bu verilerin bir uygulama için ilgili olması durumunda saklanabilir. Kuyruk başlığı yapısı Şekil 4-8'de gösterilmektedir.



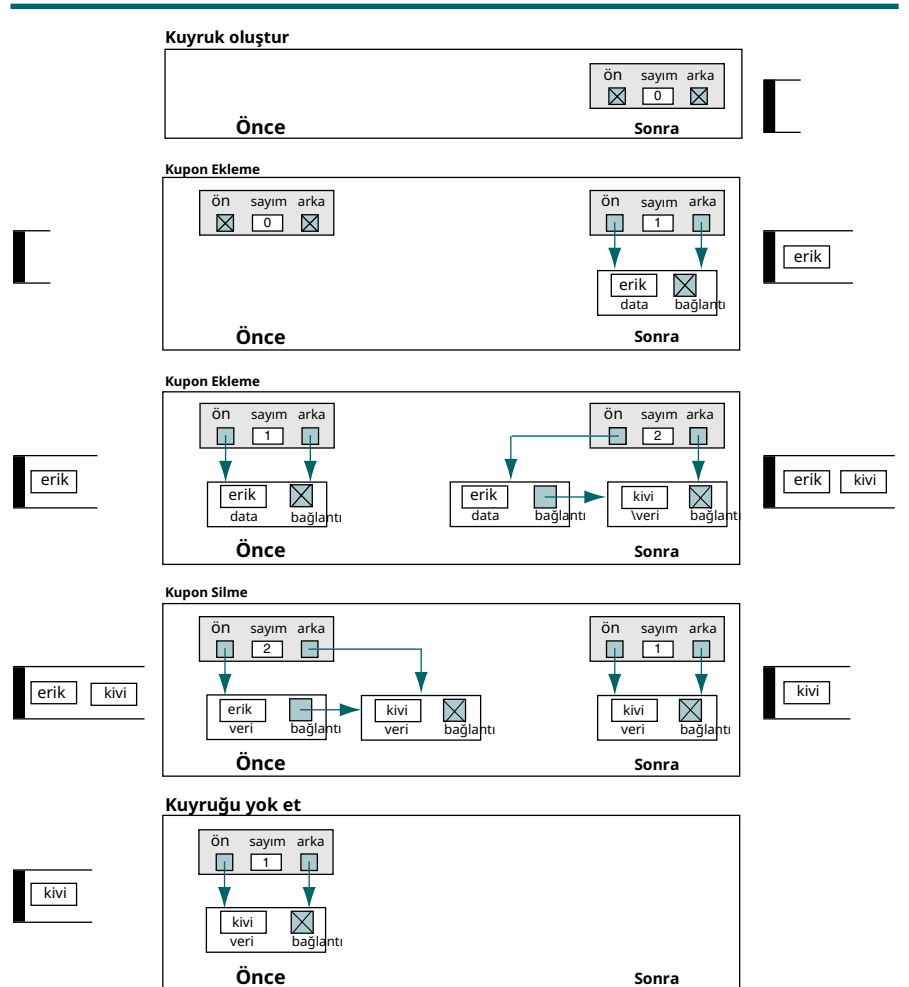
ŞEKİL 4-8 Kuyruk Veri Yapısı

Kuyruk Veri Düzümü

Kuyruk veri düğümü, kullanıcı verilerini ve varsa bir sonraki düğüme işaret eden bir bağlantı alanını içerir. Bu düğümler dinamik bellekte saklanır ve kullanıcı programı tarafından talep edildiği gibi eklenir ve silinir. Yapısı Şekil 4-8'de de gösterilmiştir.

Kuyruk Algoritmaları

Kuyruk işlemleri, bir yığın için olanlarla paraleldir, ancak kuyruğun arka sındaki veriyi görmek için bir algoritma eklenmiştir. Bu işlemleri burada ve sonraki bölümlerde tanımlıyoruz. Dört temel kuyruk işlemi Şekil 4-9'da gösterilmiştir.



ŞEKİL 4-9 Temel Kuyruk Fonksiyonları

Kupon Oluştur

kupon oluştur işlemi oldukça basittir. Tek yapmamız gereken, meta verileri işaretçilerini null olarak ayarlamak ve sayıyı 0 yapmak. Kupon oluşturma için pseudocode, Algoritma 4-1'de gösterilmiştir.

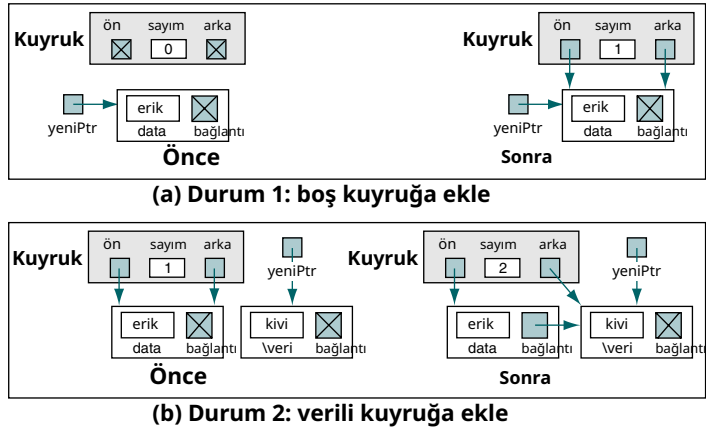
ALGORİTMA 4-1 Kupon Oluştur

```

Algoritma kuponOluştur
Kupon yapısını oluşturur ve başlatır.
  Ön  kupon, bir meta veri yapısıdır
  Son meta veri öğeleri başlatılmıştır
  Kupon başını döndür
1  kupon başını ayır
2  kuyruk önünü null olarak ayarla
3  kuyruk arka değerini null olarak ayarla
4  kuyruk sayısını 0 olarak ayarla
5  kuyruk başını döndür
son createQueue
  
```

Kupon Ekleme

Kupon ekleme, bir yığına veri eklemekten biraz daha karmaşıktır. Ekleme algoritmasını geliştirmek için, iki farklı kupon durumunu analiz etmemiz gerekiyor: boş bir kupona ve verisi olan bir kupona ekleme. Bu işlemler Şekil 4-10'da gösterilmiştir.



ŞEKİL 4-10 Kupon Ekleme Örneği

Boş bir kupona veri eklediğimizde, kuponun ön ve arka işaretçileri yeni düğüme işaret edecek şekilde ayarlanmalıdır. Verisi olan bir kupona veri eklediğimizde, sonuncu

düğüm ve arka işaretçiyi yeni düğüme ayarlayın. Eğer ekleme başarılı olduysa, bir Boolean true döndürürüz; eğer yeni düğüm için yeterli bellek kalmadıysa, bir Boolean false döndürürüz. Pseudocode, Algoritma 4-2'de gösterilmiştir.

ALGORİTMA 4-2 Kuyruğa Veri Ekle

```

Algoritma kuyruk ekle (kuyruk, veriGirişi)
Bu algoritma veriyi bir kuyruğa ekler.
  Ön kuyruk bir meta veri yapısıdır
  Son veriGirişi eklenmiştir
  Başarılıysa true, taşma durumunda false döndür
1  eğer (kuyruk dolu)
    1 false döndür
2  son eğer
3  tahsis et (yeni düğüm)
4  veriGirişini yeni düğüm verisine taşı
5  yeni düğümün sonunu null işaretçisine ayarla
6  eğer (boş kuyruk)
    Null kuyruğa ekleniyor
    1 yeni verinin adresini kuyruk önüne ayarla
7  aksi takdirde
    eski arka ucu yeni düğüme işaret et
    1 arka düğümün sonraki işaretçisini yeni düğümün adresine ayarla
8  son eğer
9  kuyruk arka ucunu yeni düğümün adresine ayarla
10 kuyruk sayısını artır
11 doğru döndür
son kuyruk ekle

```

Kupon Silme

Veri silmekten biraz daha karmaşık olsa da, kuyruktan veri çıkarmak başlangıçta çok benzer. Öncelikle kuyruğun veri içerdiğinden emin olmalıyız. Eğer kuyruk boşsa, alt akış (underflow) oluşur ve false döneriz, bu da dequeue işleminin başarılı olmadığını gösterir.

Çıkarılacak veri olduğunda, veriyi parametre listesi aracılığıyla geri geçiririz ve ardından ön işaretçiyi kuyruğun bir sonraki ögesine ayarlarız. Eğer son ögeyi yeni çıkardıysak, kuyruk ön işaretçisi otomatik olarak son düğümün bağlantı alanından null işaretçiyi atayarak null işaretçisine dönüşür. Ancak, kuyruk şimdi boşsa, arka işaretçiyi de null olarak ayarlamalıyız. Bu durumlar Şekil 4-11'de gösterilmiştir.

Pseudocode, Algoritma 4-3'te gösterilmiştir.

ALGORİTMA 4-3 Kuyruktan Veri Sil

```

Algoritma kuyruk çıkar (kuyruk, öge)
Bu algoritma bir düğümü kuyruktan siler.
  Ön kuyruk bir meta veri yapısıdır

```

devam ediyor

ALGORİTMA 4-3 Kuyruktan Veri Sil (devam ediyor)

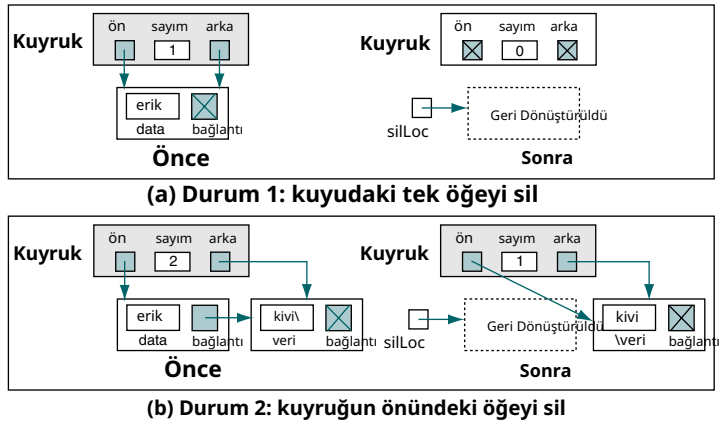
item, çağırın algoritma değişkenine bir referanstır
 Post kuyruk önünde kullanıcıya geri dönen veri
 ve ön eleman silinir
 Başarılıysa true, alt akış varsa false döner

- 1 if (kuyruk boş)
 - 1 false döndür
- 2 son eğer
- 3 ön veriyi item'e taşı
- 4 if (kuyruktaki yalnızca 1 düğüm varsa)

Kuyruktaki tek öğeyi siliyor

 - 1 kuyruk arkasını null olarak ayarla
- 5 son eğer
- 6 kuyruk önünü kuyruk ön sonraya ayarla
- 7 kuyruk sayısını azalt
- 8 doğru döndür

bitiş kuyruktan çıkar



ŞEKİL 4-11 Kuyruk Çıkarma Örnekleri

Kuyruk Verilerini Alma

İki kuyruk alma işlemi arasındaki tek fark - kuyrukön ve kuyruk arka - hangi işaretçinin kullanıldığıdır, ön mü yoksa arka mı. Ön kuyruk üzerinde duralım. Mantığı, verilerin kuyruktan silinmediği dışında, kuyruk çıkarma ile aynıdır. Öncelikle boş bir kuyruk olup olmadığını kontrol eder ve kuyruk boşsa false döner. Kuyruktaki veri varsa, veriyi dataOut üzerinde n geri iletir ve true döner. Pseudocode, Algoritma 4-4'te gösterilmiştir.

ALGORİTMA 4-4 Kuyruğun Önündeki Veriyi Alma

```

Algoritma  kuyrukÖn (kuyruk, veriDışarı) Kuyru
ğun önündeki veriyi, kuyruk içeriğini değiştirmeden alır.

    Ön  kuyruk, bir meta veri yapısıdır
        fveriDışarı, çağıran algoritma değişkenine bir referanstır
    Son  veri çağırana geri iletilir
    Başarılysa true, alt akış varsa false döner
1  if (kuyruk boş)
    1  false döndür
2  son eğer
3  kuyruk önündeki veriyi veriDışarı'ya taşı
4  doğru döndür
end  kuyrukÖn

```

Kuyruk arka kısmını uygulamak için, Algoritma 4-4'ü kopyalayın ve veriyi3. ifadede a rka kısma taşıyın.

Boş Kuyruk

Boş kuyruk kuyruk boşsa doğru, kuyruk veri içeriyorsa yanlış döner. Boş bir kuyruğu test etmenin birkaç yolu vardır. Kuyruk sayısını kontrol etmek en kolay yoldur. Pseudocode Algoritma 4-5'te gösterilmiştir.

ALGORİTMA 4-5 Kuyruk Boş

```

Algoritma  boşKuyruk (kuyruk)
Bu algoritma bir kuyruğun boş olup olmadığını kontrol eder.
    Ön kuyruk, bir meta veri yapısıdır
    Boşsa true, kuyrukta veri varsa false döner
1  eğer (kuyruk sayısı 0'a eşitse)
    1  true döndür
2  aksi takdirde
    1  false döndür
son  boşKuyruk

```

Dolu Kuyruk

Yığın gibi, dolu kuyruk veri gizleme için başka bir yapılandırılmış program lama uygulamasıdır. Dile bağlı olarak, uygulaması en zor algoritmalarından biri olabilir—ANSI C, bunu uygulamak için doğrudan bir yol sağlamaz. Pseudocode, Algoritma 4-6'da gösterilmiştir.

ALGORİTMA 4-6 Dolu Kuyruk

```

Algoritma  doluKuyruk (kuyruk)
Bu algoritma, bir kuyruğun dolu olup olmadığını kontrol eder. Kuyruk

```

devam ediyor

ALGORİTMA 4-6 Dolu Kuyruk (devam ediyor)

```

bir düğüm için bellek tahsis edilemezse doludur.
Ön kuyruk, bir meta veri yapısıdır
Doluysa doğru, başka bir düğüm için yer varsa yanlış döndür
1 eğer (bellek mevcut değilse)
    1 true döndür
2 aksi takdirde
    1 false döndür
3 son eğer
son doluKuyruk

```

Kuyruk Sayısı

Kuyruk sayısı şu anda kuyrukta bulunan eleman sayısını, kuyruk baş düğümündeki sayıyı basitçe döndürerek verir. Kod, Algoritma 4-7'de gösterilmiştir.

ALGORİTMA 4-7 Kuyruk Sayısı

```

Algoritma kuyrukSayısı (kuyruk)
Bu algoritma, kuyruktaki eleman sayısını döndürür.
Ön kuyruk, bir meta veri yapısıdır
Kuyruk sayısını döndür
1 kuyruk sayısı döndür
son kuyrukSayısı

```

Kuyruk Sil

Kuyruk sil kuyruktaki tüm verileri siler. Algoritma 4-8, bir kuyruğu silmek için kodu içerir.

ALGORİTMA 4-8 Kuyruk Sil

```

Algoritma kuyrukYokEt (kuyruk)
Bu algoritma bir kuyruktan tüm verileri siler.
Ön kuyruk bir meta veri yapısıdır
Son tüm veriler silinmiştir
1 eğer (kuyruk boş değilse)
    1 döngü (kuyruk boş değilse)
        1 ön düğümü sil
    2 döngüyü sonlandır
2 son eğer
3 baş yapısını sil
son kuyrukYokEt

```


4.3 Kuyruk ADT

Kuyruk soyut veri tipi (ADT), yığın soyut veri tipinin temel tasarımını takip eder. Kuyruk veri yapısını geliştirerek başlıyoruz ve ardından Bölüm 4.2'de her bir kuyruk algoritmasını geliştiriyoruz.

Kuyruk Yapısı

Kuyruk veri yapısı Program 4-1'de gösterilmiştir. Düğüm yapısı, bir yığın için kullandığımız yapıyla aynıdır. Her düğüm, veriye bir void işaretçi ve kuyruktaki bir sonraki elemana bir bağlantı işaretçisi içerir. Yığın gibi, kullanan program verileri depolamak için bellek ayırmaktan sorumludur.

Program 4-1 ayrıca kuyruk fonksiyonları için prototip ifadelerini içerir. Bir üretim ortamında, bu bir kuyruk başlık dosyası olur, örneğin `queues.h`.

PROGRAM 4-1 Kuyruk ADT Veri Yapıları

123456789	<pre>//Kuyruk ADT Tür Tanımları typedef struct node { void* dataPtr; struct düğümü* sonraki; } QUEUE_NODE; typedef yapısı { QUEUE_NODE* ön; QUEUE_NODE* arka; int sayım; } KUYRUK;</pre>
1011121314151617181920212223242526	<pre>//Prototip Bildirimleri QUEUE* createQueue (void); QUEUE* destroyQueue (QUEUE* kuyruk); bool dequeue (QUEUE* kuyruk, void** itemPtr); bool enqueue (QUEUE* kuyruk, void* itemPtr); bool queueFront (QUEUE* kuyruk, void** itemPtr); bool queueRear (QUEUE* kuyruk, void** itemPtr); int queueCount (QUEUE* kuyruk); bool emptyQueue (QUEUE* queue); bool fullQueue (QUEUE* queue); //Kuyruk ADT Tanımları Sonu</pre>

Kuyruk ADT Algoritmaları

Bu bölüm, her bir kuyruk algoritması için C kodunu içerir. Onları inceleyen, Şekil 4-6, “Kuyruk Örneği” üzerinden izleyin.

Kupon Oluştur

Kuyruk oluşturma, kuyruk başlığı için bir düğüm ayırır. Ardından, ön ve arka işaretçileri null olarak başlatır ve sayacı sıfıra ayarlar. Taşma meydana gelirse, dönüş değeri null'dır; tahsis başarılı olursa, kuyruk başının adresini döndürür. Pseudocode tasarımı Algoritma 4-1'de gösterilmiştir. Kuyruk oluşturma kodu Program 4-2'de geliştirilmiştir.

PROGRAM 4-2 Kuyruk Oluştur

123456789	/*===== createQueue ===== Dinamik bellekten bir kuyruk baş düğümü için bellek ayırır ve adresini çağırana döndürür. Önce hiçbir şey Sonra baş tahsis edilmiştir ve başlatılmıştır Başarılıysa başı döndür; taşma durumunda null döndür */
10111213141516171819202122	QUEUE* createQueue (geçersiz) { //Yerel Tanımlar QUEUE* kuyruk; //İfadeler kuyruk = (QUEUE*) malloc (sizeof (QUEUE)); if (kuyruk) { kuyruk->ön = NULL; kuyruk->arka = NULL; kuyruk->sayı = 0; } // eğer dönüş kuyruğu; } // createQueue

Kupon Ekleme

Enqueue için mantık, Algoritma 4-2'de gösterildiği gibi basittir. Eğer bellek mevcutsa, yeni bir düğüm oluşturur, kuyruk arkasına ekler ve true döndürür. Eğer bellek mevcut değilse, false döndürür. Kod Program 4-3'te gösterilmiştir.

PROGRAM 4-3 Sonraki

123456	/*===== enqueue ===== Bu algoritma verileri bir kuyruğa ekler. Önce kuyruk oluşturulmuştur Sonra veriler eklenmiştir Başarılıysa true, taşma durumunda false döner */
--------	--

devam ediyor

PROGRAM 4-3 Kuyruğa Ekle (devamı)

789	bool enqueue (QUEUE* kuyruk, void* itemPtr)
	{
	//Yerel Tanımlar
	QUEUE_NODE* newPtr;
	//İfadeler
	if (!(yeniPtr =
	(QUEUE_NODE*)malloc(sizeof(QUEUE_NODE))))
	return false;
	newPtr->dataPtr = itemPtr;
	newPtr->next = NULL;
	if (queue->count == 0)
	// Boş kuyruğa ekleme
	queue->front = newPtr;
	aksi takdirde
	kuyruk->arka->sonraki = yeniPtr;
	(sıra->sayım)++;
	kuyruk->arka = yeniPtr;
	return true;
	}// kuyruğa almak
101121314151617181920212223242526272829	

Program 4-3 AnaliziHem ön hem de arka işaretçiyi korumamız gerektiğinden, null bir kuyruğa ekleme yapıp yapmadığımızı kontrol etmemiz gerekiyor. Eğer öyleyse, her iki işaretçiyi de yeni eklenen veriye ayarlamalıyız. Kuyrukta zaten veri varsa, kuyruğun arkasındaki düğümün bir sonraki alanını ve arka işaretçiyi yeni düğüme ayarlamamız gerekiyor. Bu durumda ön işaretçi değişmez. Arka işaretçi her durumda güncellendiğinden, bunu *if* ifadesinden sonra değiştirdik (bkz. ifade 27).

Kupon Silme

Kuyruğu Boşaltma, Algoritma 4-3'te geliştirilen temel tasarımı takip eder. Öncelikle kuyrukta veri olup olmadığını kontrol eder. Eğer varsa, silinen verinin adresini çağıran işlevine bir referans parametresi aracı lığıyla geri gönderir, işaretçileri ayarlar ve kuyruk sayısından bir çıkarır. Silme işlemi başarılıysa true döner; eğer boşsa false döner. Kod Program 4-4'te gösterilmiştir.

PROGRAM 4-4 Sonraki

1234	/*===== kuyruktan çıkar ===== Bu algoritma kuyruktan bir düğümü siler. Önce kuyruk oluşturulmuştur Sonra Kuyruk önüne veri işaretçisi döndürüldü ve
------	--

devam ediyor

PROGRAM 4-4Kuyruktan Çıkarma (devamı)

56789	ön eleman silindi ve geri döndürüldü. Başarılıysa true döndür; aşırı akma durumunda false
10111213141516171819202122232425262728	<pre> */ bool dequeue (QUEUE* kuyruk, void** itemPtr) { //Yerel Tanımlar QUEUE_NODE* deleteLoc; //İfadeler if (!queue->count) return false; *itemPtr = kuyruk->ön->dataPtr; deleteLoc = kuyruk->ön; if (queue->count == 1) // Kuyruкта yalnızca bir öğе siliniyor queue->rear = queue->front = NULL; else kuyruk->ön = kuyruk->ön->sonraki; (kuyruk->sayı)--; ücretsiz (deleteLoc); return true; }// sıradan çıkar </pre>

Program 4-4 AnaliziKuyruktan çıkarma mantığı oldukça basittir. Ana endişe, kuyruktaki son elemanı silmiş olabileceğimizdir; bu durumda onu null durumuna ayarlamamız gerekir. Bu, kuyruk arka kısmının null işaretçisinin ayarlanmasını gerektirir. Her zaman kuyruk önünü silinen düğümdeki bir sonraki işaretçiye a yarladığımızı unutmayın. Eğer son düğüm siliniyorsa, silinen düğümün bir sonraki işaretçisinin null olması garanti edilir (çünkü bu, kuyruktaki son düğümdür). Bu durumda kuyruk ön otomatik olarak null olarak ayarlanır. Eğer kuyruktaki son düğüm değilse, kuyruk ön, kuyruk önündeki yeni düğüme işaret eder.

Dequeue kodu yeterince basit olsa da, çağırısı biraz zorlayıcıdır. Çünkü dequeue v eri parametresi bir *void* işaretçisine işaret eden bir işaretçidir, bu nedenle çağrı veri iş aretçisini *void* olarak dönüştürmelidir. Çağrı aşağıda gösterilmiştir.

```
sıradan çıkar(queue, (void*)&dataPtr)
```

Dequeue bir Boolean başarı bayrağı döndürdüğü için, çağrı bir seçim ifadesinde başarı veya başarısızlığı test etmek için bir ifade olmalıdır.

```
eğer (sıradan çıkar(queue, (void*)&dataPtr))
```

Kupon Önü

Kuyruk ön kısmı, kuyruğun önündeki verinin adresini çağıran işlevine geriaktarır. Dequeue'dan tek farkı, kuyruğun değiştirilmemesidir. Kod Program 4-5'te gösterilmiştir; psödo kod tasarımı Algoritma 4-4'te gösterilmiştir.

PROGRAM 4-5 Kuyruk Ön

123456789	/*===== kuyrukÖn ===== Bu algoritma, kuyruk içeriğini değiştirmeden kuyruğun önündeki veriyi alır. Ön kuyruk, başlatılmış bir kuyruğa işaretçidir Son itemPtr çağırana geri gönderilir Başarılıysa true döndür; aşırı akma durumunda false */
101112131415161718	bool kuyrukÖn (KUYRUK* kuyruk, void** itemPtr) { //İfadeler if (!kuyruk->count) false döndür; aksi takdirde { *itemPtr = kuyruk->ön->dataPtr; return true; } // ak si takdirde }// kuyrukÖn

Kupon Arkası

Kuyruk arka için kod, Program 4-6'da gösterildiği gibi, kuyruk ön ile aynıdır sadece kuyruk arkasındaki verinin adresi çağıran işleve geri gönderilir.

PROGRAM 4-6 Kuyruk Arka

123456789	/*===== kuyrukArka ===== Kuyruğun arkasındaki verileri alır kuyruk içeriğini değiştirmeden. Ön kuyruk, başlatılmış kuyruğa işaretçidir Son Çağırıcıya geri dönen veri Başarılıysa true döndür; aşırı akma durumunda false */
101112	bool kuyrukArka (KUYRUK* kuyruk, void** öğePtr) { //İfadeler if (!kuyruk->sayısı) return true;

devam ediyor

PROGRAM 4-6 Kuyruk Arka (devam ediyor)

```

131415161718
    aksi takdirde
    {
        *itemPtr = kuyruk->arka->dataPtr;
        return false;
    } // aks
    i takdirde }// kuyrukArka

```

Kuyruk Boş

Kuyruk boş, kütüphanedeki en basit işlevlerden biridir. Kuyruk başlığındaki sayımı kullanarak kuyruğun boş olup olmadığını veya veri içerip içermediğini belirler, Program 4-7'de gösterildiği gibi.

PROGRAM 4-7 Boş Kuyruk

```

123456789
/*===== emptyQueue =====
Bu algoritma kuyruğun boş olup olmadığını kontrol eder.
Ön kuyruk, bir kuyruk baş düğümüne işaret eden bir işaretçidirB
oşsa true; veri varsa false döner
*/
bool emptyQueue (QUEUE* kuyruğu)
{
    //İfadeler
    return (queue->count == 0);
10 }// emptyQueue

```

Dolu Kuyruk

Algoritma 4-6'nın tartışmasında belirtildiği gibi, dolu kuyruk, uygulama sı daha zor olan algoritmalarından biridir çünkü C, dinamik bellek içinde k alan alan miktarını test etmenin bir yolunu sağlamaz. Tek yol, bir düğüm ayırmak ve başarıyı test etmektir. Düğüm başarıyla ayrıldıysa, onu se rbest bırakırız ve dolu değil için false döneriz. Eğer hiç bellek kalmadıysa , true döneriz—kuyruk doludur. Dolu kuyruk için kod, Program 4-8'de gösterilmiştir.

PROGRAM 4-8 Dolu Kuyruk

```

12345678
/*===== fullQueue =====
Bu algoritma kuyruğun dolu olup olmadığını kontrol eder. Dolu
bir sonraki düğüm için bellek tahsis edilemezse doludur.
Ön kuyruk, bir kuyruk baş düğümüne işaret eden bir işaretçidir
Doluysa true; bir düğüm için yer varsa false döner
*/
bool fullQueue (QUEUE* kuyruğu)
{

```

devam ediyor

PROGRAM 4-8Dolu Kuyruk (devamı)

9	<code>//Yerel Tanımlar</code>
10	<code>QUEUE_NODE* sıcaklığı;</code>
11	<code>//İfadeler</code>
12	<code>temp = (QUEUE_NODE*)malloc(sizeof(*(queue->rear)));</code>
13	<code>if (temp)</code>
14	<code>{</code>
15	<code> free (temp);</code>
16	<code> return true;</code>
17	<code>} // eğer</code>
18	<code>// Yığın dolu</code>
19	<code>return false;</code>
20	<code>}// doluKuyruk</code>
21	

Kuyruk Sayısı

Kuyruk sayısı, kuyruk baş düğümünde bulunan sayıyı döndürür. Kodunun Program 4-9'da gösterilmiştir.

PROGRAM 4-9Kuyruk Sayısı

1	<code>/*===== kuyrukSayısı =====</code>
2	<code> Kuyruktaki eleman sayısını döndürür.</code>
3	<code> Ön kuyruk, kuyruk baş düğümüne işaretçidir</code>
4	<code> Dönüş kuyruk sayısı</code>
5	<code>*/</code>
6	<code>int kuyrukSayısı(QUEUE* kuyruk)</code>
7	<code>{</code>
8	<code> //İfadeler</code>
9	<code> return kuyruk->count;</code>
10	<code>}// kuyrukSayısı</code>

Kuyruk Sil

Algoritma 4-8'deki kuyruk yok et tasarımı oldukça basittir; Program 4-10'da kiuygulama ise oldukça karmaşıktır. Kuyruktaki tüm elemanları silmemiz ve her birinin düğümünü serbest bırakmamız gerektiği açıktır, ancak veri alanı nı geri dönmeyi unutmak kolaydır. Öncelikle, kuyruğun var olup olmadığını kontrol ederek başlarız. Eğer yoksa, basitçe bir null işaretçi döneriz. Kuyruk geçerliyse, içinden geçerek tüm elemanları sileriz.

PROGRAM 4-10Kuyruğu Yok Et

1	<code>/*===== kuyruğuYokEt =====</code>
2	<code> Bir kuyruğun tüm verilerini siler ve</code>
3	<code> belleğini geri kazanır, ardından kuyruk baş işaretçisini siler ve geri kazanır.</code>
4	<code> Öncelikle Kuyruk geçerli bir kuyruktur</code>

devam ediyor

PROGRAM 4-10 Kuyruğu Yok Et (devam ediyor)

56789	Tüm veriler silindi ve geri dönüştürüldü Null işaretçi döndür
1011121314151617181920212223242526	<pre> */ QUEUE* destroyQueue (QUEUE* sıra) { //Yerel Tanımlar QUEUE_NODE* deletePtr; //İfadeler eğer (queue) { while (sıra->ön != NULL) { ücretsiz (kuyruk->ön->dataPtr); deletePtr = kuyruk->ön; kuyruk->ön = kuyruk->ön->sonraki; ücretsiz (deletePtr); } // while serbest (sıra); } // if return NULL; } // destroyQueue </pre>

Program 4-10 Analizi Olayların normal seyrinde, yok edilen kuyruk boştur. Eğer öyleyse, kuyrukönü null'dır ve yapmamız gereken tek şey kuyruk başlığını serbest bırakmak ve geri dönmektir. Ancak kuyrukta veri varsa, bu durumda önce kuyruk düğümünde saklanan veri işaretçisini kullanarak veri alanını serbest bırakırız, ardından kuyruk düğümünü kendisini sileriz. Sonunda, kuyruksonuna ulaşırız, o zaman kuyruk baş düğümünü serbest bırakır ve geri döneriz.

4.4 Kuyruk Teorisi

Kuyruk teorisi, kuyrukların performansını tahmin etmek için kullanılan bir uygulamalı matematik alanıdır. Bu bölümde birkaç temel kuyruk teorisi kavramını gözden geçiriyoruz. Kuyruk teorisinin matematiğini bu konudaki ileri düzey derslerle bırakıyoruz.

Kuyruk teorisi, kuyrukların performansını tahmin etmek için kullanılan uygulamalı matematik alanıdır.

Kuyruklar, tek sunucu ve çok sunucu türlerine ayrılabilir. Tek sunucu kuyruğu, aynı anda yalnızca bir müşteriye hizmet verebilir. Tek sunucu kuyruğuna bir örnek, çoğu büyük şehirdeki sokak köşelerinde bulunan sıcak yemek satıcısıdır. Öte yandan, çok sunucu kuyrukları aynı anda birçok müşteriye hizmet verebilir. Çok sunucu kuyruğuna bir örnek, birçok bankacının hizmet verdiği tek bir sıranın bulunduğu bir bankadır. Marketlerin, birden fazla tek sunucu kuyruğu veya çoklu kuyruk örnekleri olduğunu unutmayın.

Tüm kuyrukların ortak iki unsuru, bir veya daha fazla hizmete ihtiyaç duyan müşteriler ve hizmeti sağlayan bir sunucudur. Bir müşteri, hizmete ihtiyaç duyan herhangi bir kişi veya şeydir. Verilen örneklerin yanı sıra, müşteriler bilgisayarda çalışmayı bekleyen işler ve ekspres teslimatla gönderilen paketler kadar çeşitli olabilir. Hizmet, gerekli sonucu elde etmek için gereken herhangi bir aktivitedir. Sıcak yemek satıcısı, yiyecekleri bir hizmet olarak sağlar; paket taşıyıcı, paketlerin taşınması ve teslimatını sağlar.

Kuyruk üzerinde en dramatik etkiye sahip olan iki faktör, varış hızı ve hizmet süresidir. Müşterilerin hizmet için kuyruğavarış hızı, varış hızı olarak bilinir. Sağlanan hizmete bağlı olarak, varış hızı rastgele veya düzenli olabilir. Sıcak yemek satıcısının müşterileri muhtemelen rastgele gelir, ancak hız gün boyunca geniş ölçüde değişebilir. Bilgisayarda işlenmesi gereken işler ise, bir iş planlama sistemi tarafından oluşturulan bazı düzenli bir hızda gelebilir.

Hizmet süresi, bir müşteri talebinin işlenmesini tamamlamak için gereken ortalama zamandır. Eğersırada beklerken farklı banka müşterilerini gözlemlediyseniz, her bir müşteriye işlemek için gereken zamanın geniş aralığını not ettiniz. Açıkça, müşterilerin daha hızlı geldiği ve hizmet süresinin daha yüksek olduğu durumlarda, kuyruk daha uzun olur.

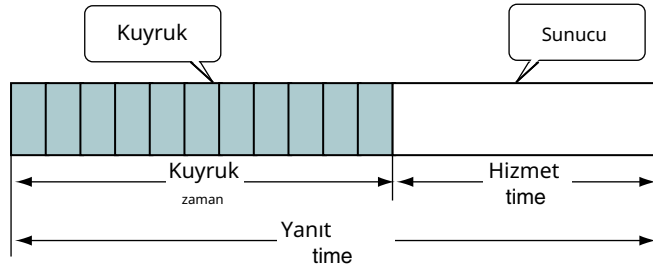
İdeal bir durumda, müşteriler hizmet süresiyle eşleşen bir hızda gelir. Ancak, hepimizin bildiği gibi, durumlar nadiren idealdir. Bazen sunucu, hizmet verilecek müşteri olmadığı için boş kalır. Diğer zamanlarda ise, hizmet bekleyen birçok müşteri olacaktır. Eğer desenleri tahmin edebilirsek, boş sunucuları ve bekleyen müşterileri minimize edebiliriz. Bunu yapmak, uzun süre beklemek zorunda kalan müşterilerin, sokaktaki bir blok ötedeki sıcak yemek satıcısı gibi alternatif bir sunucuya geçme eğiliminde olmaları nedeniyle özellikle önemlidir.

Kuyruk teorisi, bu tür kalıpları tahmin etmeye çalışır. Özellikle, *kuyruk süresi* (müşterilerin kuyrukta beklediği ortalama süre olarak tanımlanır), kuyruk ortalaması ve maksimum kuyruk boyutunu tahmin etmeye çalışır. Bu tahminleri yapmak için iki faktöre ihtiyaç duyar: varış hızı ve ortalama hizmet süresi, bu da boş dönemler arasındaki toplam hizmet süresinin ortalaması olarak tanımlanır.

Kuyruk süresi ve hizmet süresi verildiğinde, *yanıt süresi*'ni biliyoruz,—müşterilerin kuyrukta girdiği noktadan sunucudan çıktıkları ana kadar geçen ortalama süreyi ölçen bir ölçüdür. Yanıt süresi, özellikle çevrimiçi bilgisayar sistemlerinde önemli bir istatistiktir. Bir kuyruk teorisi modeli Şekil 4-12'de gösterilmektedir.

Bir kuyruk modeli oluşturulduktan sonra, sistemde önerilen değişiklikleri incelemek için kullanılabilir. Örneğin, bankacılık kuyruğunda, ortalama hizmet süresini %15 azaltmak için otomasyonu kullanabilirsek, kaç tane daha az işlemlerimiz olurdu? Veya, şu anda kapasitenin altında olan büyüyen bir sistemin modeli verildiğinde, başka bir sunucu eklememiz gerektiği süre ne kadar olacak?

Kuyrukların performansını en çok etkileyen iki faktör, varış oranı ve hizmet süresidir.



ŞEKİL 4-12 Kuyruk Teorisi Modeli

4.5 Kuyruk Uygulamaları

Kuyruklar, daha yaygın veri işleme yapılarından biridir. Onlar, neredeyse her işletim sisteminde ve her ağda ve sayısız diğer alanda bulunur. Örneğin, kuyruklar, müşteri taleplerini, işleri ve siparişleri işlemek gibi çevrimiçi iş uygulamalarında kullanılır. Bir bilgisayar sisteminde, işleri işlemek ve yazdırma kuyrukları gibi sistem hizmetleri için bir kuyruğa ihtiyaç vardır.

Kuyruklar oldukça karmaşık hale gelebilir. Bu bölümde iki kuyruk uygulamasını gösteriyoruz: İlki, birçok problemde yararlı olan bir uygulamadır—veri kategorilendirme. Bu uygulama için hem tasarımı hem de uygulamayı gösteriyoruz. İkincisi, herhangi bir kuyruk uygulamasının performansını incelemek için kullanılabilecek bir kuyruk simülasyonudur. Sadece tasarım geliştirilmiştir; uygulama bölümün sonunda bir proje olarak bırakılmıştır.

Veri Kategorilendirme

Verileri temel sıralarını bozmadan yeniden düzenlemek genellikle gereklidir. Basit bir örnek olarak, bir sayı listesi düşünelim. Sayıları, her gruptaki orijinal sıralarını koruyarak gruplandırmamız gerekiyor. Bu, mükemmel bir çoklu kuyruk uygulamasıdır.

Gösterim için, aşağıdaki sayı listesini düşünün:

3 22 12 6 10 34 65 29 9 30 81 4 5 19 20 57 44 99

Onları dört farklı gruba ayırmak istiyoruz:

- Grup 1: 10'dan az
- Grup 2: 10 ile 19 arasında
- Grup 3: 20 ile 29 arasında
- Grup 4: 30 ve üzeri

Diğer bir deyişle, listeyi aşağıda gösterildiği gibi yeniden düzenlemek istiyoruz.

| 3 6 9 4 5 | 12 10 19 | 22 29 20 | 34 65 30 81 57 44 99 |

Bu sıralama değildir. Sonuç, sıralanmış bir liste değil, belirtilen kurallara göre kategorize edilmiş bir listedir. Her gruptaki sayılar, orijinal sıralarını korumuştur.

Veri Tasarımını Kategorize Etme

Çözüm basittir. Dört kategori için bir kuyruk oluşturuyoruz. Sayıları o kudukça uygun kuyruğa kaydediyoruz. Tüm veriler işlendiğinde, veriler doğru bir şekilde kategorize ettiğimizi göstermek için her kuyruğu yazdırıyoruz. Bu tasarım, Algoritma 4-9'da gösterilmiştir.

ALGORİTMA 4-9 Kategori Kuyrukları

```
Algoritma  kategorize et
  Bir dizi sayıyı dört kuyruk kullanarak dört gruba ayırın.
  Yazarı:
  Tarih:
1 createQueue (q0to9)
2 createQueue (q10to19)
3 createQueue (q20to29)
4 createQueue (qOver29)
5 fillQueues (q0to9, q10to19, q20to29, qOver29)
6 printQueue'lar (q0to9, q10to19, q20to29, qOver29)
son  kategorize et
```

Algoritma 4-9 Analizi Kategori kuyruğunun ana hattı, kuyrukları oluşturur ve ardından bunları doldurup yazdırmak için algoritmaları çağırır. Tamamladığımızda, kuyrukları yok etmediğimize dikkat edin. Bu işlemi işletim sistemine bırakıyoruz. Ancak daha fazla işimiz olsaydı, belleklerini serbest bırakmak için onları silerdik.

Kuyruğu doldurmak için kullanılan sahte kod, Algoritma 4-10'da gösterilmiştir. Kuyrukları yazdırma mantığını, bir sonraki bölümde C uygulamasını yazarken göstereceğiz.

ALGORİTMA 4-10 Kategori Kuyruklarını Doldur

```
Algoritma  fillQueues (q0to9, q10to19, q20to29, qOver29) Bu algoritma
, klavyeden veri okur ve bunları dört kuyruktan birine yerleştirir.
```

```
Ön  tüm dört kuyruk oluşturulmuştur
Sonra kuyruklar verilerle doldurulmuştur
1 döngü (veri sonu değil)
```

devam ediyor

ALGORİTMA 4-10Kategorik Kuyrukları Doldur (devam ediyor)

```

1   oku (sayı)
2   eğer (sayı < 10)
    1   kuyruğa ekle (q0to9, sayı)
3   aksi takdirde eğer (sayı < 20)
    1   kuyruğa ekle (q10to19, sayı)
4   aksi takdirde eğer (sayı < 30)
    1   kuyruğa ekle (q20to29, sayı)
5   aksi takdirde
    1   kuyruğa ekle (qOver29, sayı)
6   bitir
2   döngüyü bitir
bitir fillQueues

```

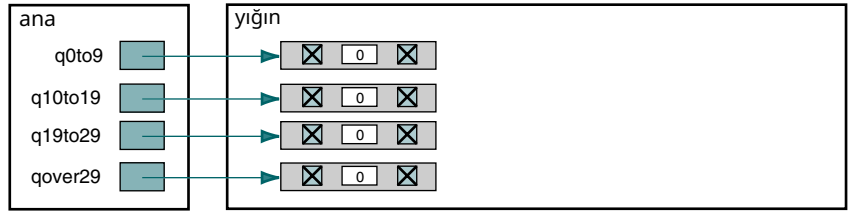
Algoritma 4-10 AnaliziUzun ve zor bir program gibi görünen şey, aslında kısa ve basit bir algoritmadır,kodun yeniden kullanılabilirliği sayesinde. Bu algoritmadaki en zor karar, kategorize etme kodunu yazmanın nasıl olacağını belirlemektir. Basitçe, bir *elseif* yapısı ile uygulanan çok yönlü seçim kullandık.Bu kararı verdikten sonra, algoritma neredeyse kendiliğinden yazıldı.

Veri Kategorize Etme—C Uygulaması

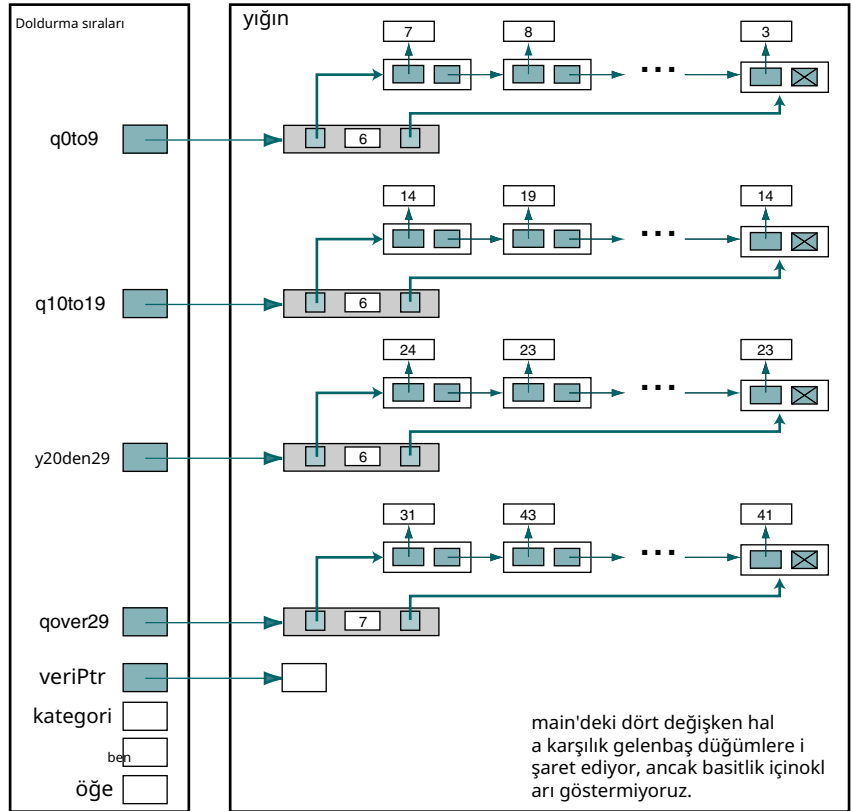
Ancak verileri bir dosyadan okumak yerine, 0 ile 50 arasındaki 25 rastgele sayı üretiyoruz. C fonksiyonları, Bölüm 4.3'te oluşturulan soyut veri tipini kullanır.Programı takip etmenize yardımcı olmak için, oluşturulduktan sonraki yapıları gösteren Şekil 4-13'ü ekliyoruz.

Ana Akış Mantığı

Kategorileme programının ana akış mantığı, kuyrukları oluşturarak başlar.Kuyruklar oluşturulduktan sonra, veriyi oluşturmak ve kuyruğa eklemek için bir fonksiyon ve bunu yazdırmak için başka bir fonksiyon çağrılır. Kod, Program 4-11'de gösterilmiştir.



(a) fillQueues çağırılmadan önce



(b) fillQueues çağırıldıktan sonra

ŞEKİL 4-13 Verileri Kategorize Etme Yapıları

PROGRAM 4-11 Verileri Kategorize Etme Ana Hattı

123	<pre>/*Dört kuyruk kullanarak sayıları dört gruba ayırır. Yazarı: Tarih:</pre>
-----	--

devam ediyor

PROGRAM 4-11 Veri Sınıflandırma Ana Hattı (devam ediyor)

```

456789 */
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "kuyruklar.h"

//Prototip Bildirimleri
void fillQueues (QUEUE*, QUEUE*, QUEUE*, QUEUE*);
void printQueues (QUEUE*, QUEUE*, QUEUE*, QUEUE*);

void printOneQueue (QUEUE* pQueue);

int ana (geçersiz)
{
//Yerel Tanımlar
    QUEUE* q0to9;
    QUEUE* q10to19;
    QUEUE* q20to29;
    QUEUE* qOver29;

//İfadeler
    printf("Veri kategorize etme gösterisine hoş geldiniz""25 rastgele sa
        yı üretiyoruz ve ardından""bunları kuponlar kullanarak kate
        gorilere ayırıyoruz.");q0to9 = createQueue ();

    q10to19 = createQueue ();
    q20to29 = createQueue ();
    qOver29 = createQueue ();

    fillQueues (q0to9, q10to19, q20to29, qOver29);
    printQueues (q0to9, q10to19, q20to29, qOver29);

    return 0;
}// ana

```

Kuponları Doldur

Kuponları doldur, 0 ile 50 arasında 25 rastgele sayı oluşturmak için bir rastgele sayı üretici kullanır. Her sayı oluşturulduğunda, programın doğruluğunu doğrulayabilmemiz için görüntülenir ve ardından her sayı uygun kupona eklenir. Kod Program 4-12'de bulunmaktadır.

PROGRAM 4-12 Verileri Sınıflandırma: Kuyrukları Doldur

```

123456789 /*===== fillQueues =====
Bu fonksiyon rand() kullanarak veri üretir ve
bunları dört kuyruktan birine yerleştirir.
Ön: Tüm dört kuyruk oluşturulmuş olmalıdır
Son: Kuyruklar verilerle doldurulmuş
*/
void fillQueues (QUEUE* q0to9, QUEUE* q10to19,
                QUEUE* q20to29, QUEUE* qOver29)
{
//Yerel Tanımlar
int kategori;
int öge;
int* veriPtr;

//İfadeler
printf("Verileri sınıflandırma:");srand(79
);

for (int i = 1; i <= 25; i++) {

    if (!(dataPtr = (int*) malloc (sizeof (int))))    printf("Queue'l
        eri doldururken taşma\`a",
            çıkış(100);

    *dataPtr = öge = rand() % 51;
    kategori = öge / 10;
    printf("%3d", item);
    if (!(i % 11))
        // Satır dolduğunda yeni satıra geç    p
        rintf("");

    geçiş (kategori)
    {
        durum 0 : sıraya alma (q0to9, dataPtr);
        break;
        durum 1 : kuyruk (q10to19, dataPtr);
        kır;

        durum 2 : enqueue (q20to29, dataPtr);
        break;

        varsayılan: enqueue (qOver29, dataPtr);
        break;

    } // anahtar
} // için
printf("Veri kategorilendirme sonu");return;

} // fillQueues

```

Kupları Yazdır

Kupları yazdırma, her kuplar için bir kupları yazdırma işlevini çağıran basit bir sürücüdür. Kodunun Program 4-13'te gösterildiği gibi.

PROGRAM 4-13 Kupları Yazdır

123456789	/*===== printQueues ===== Bu fonksiyon, her bir kuyruğun verilerini yazdırır. Önce Kuyruklar doldurulmuş Sonra Veriler yazdırılmış ve çıkarılmıştır */
1011121314151617181920212223	void printQueues (QUEUE* q0to9, QUEUE* q10to19, QUEUE* q20to29, QUEUE* qOver29) { //İfadeler printf("Veri 0..9:"); printOneQueue (q0to9); printf("Veri 10..19:"); printOneQueue (q10to19); printf("Veri 20..29:"); printOneQueue (q20to29); printf("Veri 29'dan fazla:"); printOneQueue (qOver29); geri dönmek; } // printQueue'lar

Bir Kupları Yazdır

Bir kuyruk yazdırmak, basit bir yaklaşım kullanır. Her çağrı, bir düğümü kuyruktan çıkararak ve yazdırarak bir kuyruğu yazdırır. Döngü, kuyrukta veri olduğu sürece devam eder. Ancak veri yapısına erişimimiz olmadığından, daha fazla veri yazdırmak için boş bir kuyruk kullanırız. Ayrıca kuyruk sayısını kullanarak sıfırdan büyük olup olmadığını test edebilirdik. Kod, Program 4-14'te gösterilmiştir. Ayrıca bir örnek çalışmadan elde edilen sonuçları içerir.

PROGRAM 4-14 Bir Kuyruk Yazdır

1234567	/*===== printOneQueue ===== Bu fonksiyon bir kuyruktaki verileri yazdırır, bir satıra on giriş. Önce Kuyruk dolduruldu Sonra Veri silindi ve yazdırıldı. Kuyruk boş */ void printOneQueue (QUEUE* pQueue)
---------	---

devam ediyor

PROGRAM 4-14 Bir Kuyruğu Yazdır (devam ediyor)

```

89 {
1011213141516171819202122232425262728 //Yerel Tanımlar
    int lineCount;
    int* veriPtr;

//İfadeler
    satırSayısı = 0;
    while (!emptyQueue (pQueue)) {

        dequeue (pQueue, (void*)&dataPtr);
        if (lineCount++ >= 10)
        {
            lineCount =
                1;    printf ("      ");    } // i
            f
            printf("%3d ", *dataPtr);    } /
        / while !emptyQueue printf(
        "");

        geri dönmek;
    } // printOne Sırası

```

Sonuçlar:

Veri kategorize etme gösterisine hoş geldiniz. 25 rast gele sayı üretiyoruz ve ardından bunları kuyruklar kullanarak kategorilere ayırıyoruz.

Veri kategorize etme:

```

24 7 31 23 26 14 19 8 9 6 43
16 22 0 39 46 22 38 41 23 19 18
14 3 41

```

Veri kategorileme sonu

```

Veri 0..9: 7 8 9 6 0 3
Veri 10..19: 14 19 16 19 18 14
Veri 20..29: 24 23 26 22 22 23
29'dan fazla veri: 31 43 39 46 38 41 41

```

Kuyruk Simülasyonu

Kuyrukların önemli bir uygulaması kuyruk simülasyonu, kuyrukların performansı hakkında istatistikler üretmek için kullanılan bir modelleme etkinliğidir. Tek bir sunucu kuyruğunun modelini oluşturalım—örneğin, bir plaj yürüyüş yolundaki tuzlu su taffy dükkanı. Dükkanın bir penceresi var ve bir görevli aynı anda yalnızca bir müşteriye hizmet verebilir. Dükkan ayrıca taffy kutularını ülke genelinde gönderir. Tuzlu su taffy'nin birçok aroması olduğu için ve bir

Şekerin postalanmasını talep eden müşteri için, müşterilere hizmet verme süresi 1 ile 10 dakika arasında değişmektedir.

Bir gün boyunca mağazanın faaliyetlerini incelemek istiyoruz. Mağaza haftada 7 gün, günde 8 saat açıktır. Bir günü simüle etmek için, 480 dakika (8 saat çarpı 60 dakika) süren bir model oluşturunuz.

Simülasyon, olayların 1 dakikalık aralıklarla başlamasına ve durmasına izin veren bir dijital saat kullanır. Diğer bir deyişle, müşteriler dakikada gelir, kuyrukta tam sayı da dakika bekler ve hizmet almak için tam sayıda dakikaya ihtiyaç duyarlar. Her bir dakikalık işlem süresinde, simülasyon üç olayı kontrol etmelidir: müşterilerin varışı, müşteri işlemlerinin başlaması ve müşteri işlemlerinin tamamlanması.

Etkinlikler

Yeni bir müşterinin gelişi, *yenimüşteri* adını verdiğimiz bir modülde belirlenir. Geliş oranını belirlemek için, mağaza sahibi bir kronometre kullanarak birkaç gün boyunca müşteri kalıplarını inceledi. Sahip, ortalama olarak, her 4 dakikada bir müşterinin geldiğini buldu. Geliş oranını simüle etmek için, 1 ile 4 arasında bir değer döndüren bir rastgele sayı üretici kullanıyor. Eğer sayı 4 ise, bir müşteri gelmiştir. Eğer sayı 1, 2 veya 3 ise, müşteri gelmemiştir.

Bir müşteri, sunucu boş olduğunda işlenmeye başlanır. Bu nedenle, simülasyonun her dakikasında, simülatörün, görevlinin (sunucunun) meşgul mü yoksa boş mu olduğunu belirlemesi gerekir. Simülasyonda bu, server free adında bir modül ile yapılır. Eğer görevli boşsa, sıradaki bekleyen müşteri (kuyruk) hizmet alabilir. Eğer görevli meşgulse, bekleyen müşteriler kuyrukta kalır.

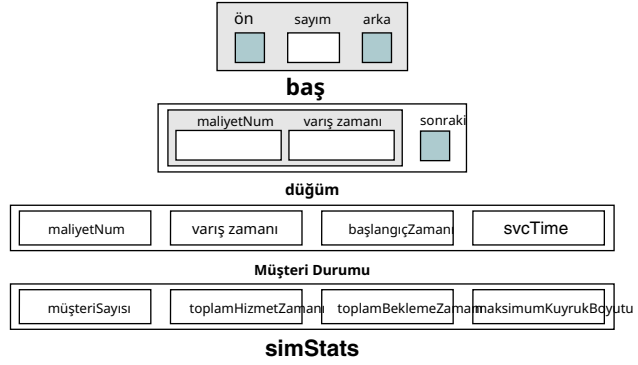
Son olarak, her dakikanın sonunda, simülasyon mevcut müşteri için işlemi tamamlayıp tamamlamadığını belirler. Mevcut müşteri için işlem süresi, müşteri işlemi başlatıldığında bir rastgele sayı üretici tarafından belirlenir. Ardından, her müşteri için, işlemi tamamlamak için gereken dakika sayısı kadar döngü yaparız. Müşteri tamamen hizmet aldığı anda, satışla ilgili istatistikleri toplarız ve sunucuyu boş bir duruma getiririz.

Veri Yapıları

Kuyruk simülasyonu için dört veri yapısı gereklidir: bir kuyruk başı, bir kuyruk düğümü, bir mevcut müşteri durumu ve bir simülasyon istatistikleri yapısı. Bu yapılar aşağıda tanımlanmış ve Şekil 4-14'te gösterilmiştir.

Kuyruk Başlığı

Kuyruk için standart baş düğüm yapısını kullanıyoruz. Bu yapı, iki düğüm işaretçisi—ön ve arka—ve şu anda kuyrukta bulunan eleman sayısını içerir.



ŞEKİL 4-14 Kuyruk Veri Yapıları

Kuyruk Düğümü

Kuyruk düğümü yalnızca iki eleman içerir: müşteri verisi ve bir sonraki düğümün işaretçisi. Müşteri verisi, sıralı bir müşteri numarası ve varış zamanından oluşur. Müşteri numarası, hizmet bekleyen birçok müşteri ile yoğun bir dükkâna girdiğinizde aldığınız numaraya benzer. Varış zamanı, müşterinin dükkâna geldiği ve sıraya girdiği (kuyruğa alındığı) zamandır.

Bir sonraki düğüm işaretçisi, sıradaki müşteriye işaret etmek için kullanılır.

Müşteri Durumu

Müşterinin siparişini işlerken, dört veri parçasını takip etmek için bir müşteri durumu yapısı kullanıyoruz. Öncelikle müşterinin numarasını ve varış zamanını saklıyoruz. Siparişi işlemeye ne zaman başladığımızı bilmemiz gerektiği için, başlangıç zamanını saklıyoruz. Son olarak, işlemi başlattığımızda, müşterinin siparişini doldurmanın ne kadar süreceğini hesaplamak ve saklamak için rastgele bir sayı üretici kullanıyoruz.

Simülasyon İstatistikleri

Simülasyonun sonunda, simülasyonda işlenen toplam müşteri sayısını, toplam ve ortalama hizmet sürelerini, toplam ve ortalama bekleme sürelerini ve bir seferde kuyruksa bulunan en fazla müşteri sayısını rapor etmemiz gerekiyor. Bu verileri saklamak için bir simülasyon istatistikleri yapısı kullanıyoruz.

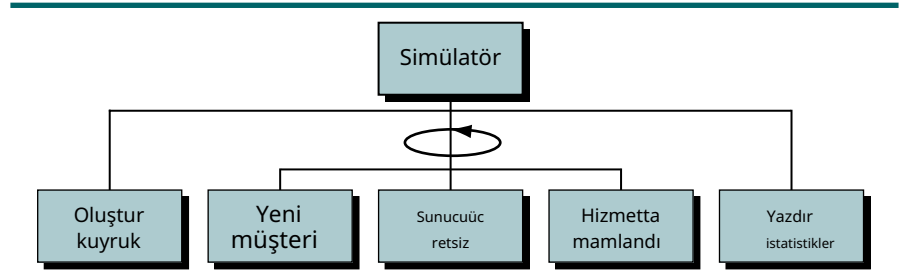
Çıktı

Simülasyonun sonunda, simülasyon sırasında toplanan istatistikleri ortalama kuyruk bekleme süresi ve ortalama kuyruk hizmet süresi ile birlikte yazdırıyoruz. Kuyruğun düzgün çalıştığını doğrulamak için, her müşteri tamamen hizmet aldığı anda aşağıdaki istatistikleri de yazdırıyoruz: varış zamanı, başlangıç zamanı, bekleme süresi, hizmet süresi ve kuyruktaki eleman sayısı.

Simülasyon Algoritması

Simülasyon algoritmasını yazmaya hazırız. Tasarım, Şekil 4-15'te gösterildiği gibi, üç işleme modülünü çağıran bir sürücü modülünden oluşmaktadır.

Daha önce tartışılan gereksinimlerle paraleldir. Yeni müşteri, bir müşterinin gelip gelmediğini ve kuyruğa alınması gerekip gerekmediğini belirler. Sunucu boş, sunucunun boş olup olmadığını belirler ve eğer öyleyse, bir müşteri varsa bir sonraki müşteriye hizmet vermeye başlar. Hizmet tamamlandı, mevcut müşterinin tamamen hizmet almadığını belirler ve eğer öyleyse, gerekli istatistik verilerini toplar ve müşteri hakkında verileri yazdırır. Simülasyonun sonunda, istatistikleri yazdırma istatistikleri modülü kullanılarak yazdırırız.



ŞEKİL 4-15 Kuyruk Simülasyonu Tasarımı

Simülâtör

Simülasyon sürücüsü için sahte kod, Algoritma 4-11'de gösterilmiştir.

ALGORİTMA 4-11 Kuyruk Simülasyonu: Sürücü

Algoritma `taffySimulation`

Bu program, tuzlu su taffy dükkanı için bir kuyruğu simüle eder.

Yazarı:

Tarih:

```

1 createQueue (kuyruk)
2 döngü (saat <= bitişZamanı VEYA dahaFazlaMüşteri)
  1 yeniMüşteri (kuyruk, saat, müşteriNumarası)
  2 sunucuBoş (kuyruk, saat, müşteriDurumu, dahaFazlaMüşteri)
  3 svcComplete (kuyruk, saat, custStatus,
                 runStats, moreCusts)
  4 eğer (queue boş değilse)
    1 moreCusts'i true olarak ayarla
  5 eğer sonlandır
  6 saati artır
3 döngüyü bitir
4 printStats (runStats)
son taffySimulation

```

Algoritma 4-11 AnaliziSürücü kuyruğu oluşturur ve simülasyon tamamlanana kadar döngüye girer. Her döngü yeni bir müşterinin kuyruğa alınması gerekip gerekmediğini test eder, sunucunun boş olup olmadığını kontrol eder böylece yeni bir müşteri başlatılabilir ve mevcut müşterinin hizmetinin tamamlanıp tamamlanmadığını kontrol eder.

Simülasyonun tamamlanıp tamamlanmadığını belirlemek için iki koşulu test etmemiz gerekiyor. Simülasyonu yalnızca tahsis edilen süre boyunca çalıştırmızda ve başka müşteri kalmadığında durdurabiliriz. Eğer simülasyonda daha fazla zaman kalmışsa, işlem bitmemiştir. Ancak, işlem süresinin sonuna ulaştığımızda bile, bir müşteri hizmet alıyorsa veya müşteriler kuyrukta bekliyorsa işlem bitmemiştir. Bu koşullardan birinin doğru olup olmadığını belirlemek için daha fazla müşteri bayrağını kullanıyoruz.

Sunucu boş mantığı, bir çağrı başlattığında daha fazla müşteri bayrağını doğru olarak ayarlar (ifade 2.2). Hizmet tamamlandığında bu bayrağı yanlış olarak ayarlar (ifade 2.3). Sürücü döngüsünde, kuyrukta bekleyen çağrılar varsa bunu doğru olarak ayarlamamız gerekiyor (ifade 2.4). Bu test, kuyrukta bekleyen çağrılarının saat süresi dolduktan sonra işlenmesini sağlar.

Simülasyonun sonunda, sürücü ortalamaları hesaplayan ve tüm istatistikleri yazdıran bir yazdırma algoritmasını çağırır.

Yeni Müşteri

Yeni müşteri için pseudocode, Algoritma 4-12'de gösterilmiştir.

ALGORİTMA 4-12Kuyruk Simülasyonu: Yeni Müşteri

```

Algoritma yeniMüşteri (kuyruk, saat, müşteriNumarası)
Bu algoritma, yeni bir müşterinin gelip gelmediğini belirler.
Ön   kuyruk geçerli bir kuyruğa yapıdır
      saat mevcut saat dakikasıdır
      müşteriNumarası son müşterinin numarasıdır
Son   eğer yeni müşteri geldiyse, kuyruğa yerleştirilir
1 rastgele müşteri gelip gelmediğini belirle
2 eğer (yeni müşteri)
  1 custNum artır
  2 custNum'u custData'da sakla
  3 varış zamanını custData'da sakla
  4 enqueue (kuyruk, custData)
3 son eğer
son yeniMüşteri
  
```

Algoritma 4-12 AnaliziHer dördüncü çağrıda bir müşteri ekleyemeyeceğimizi belirtmek önemlidir. Simülasyon çalışması için, müşterilerin rastgele bir şekilde gelmesi gerekir, tam olarak her 4 dakika da bir değil. Bir müşterinin ne zaman geldiğini rastgele belirleyerek (rastgele sayı üretici kullanarak), birkaç dakika boyunca hiç müşteri olmadan geçebiliriz ve ardından birkaç müşteri peş peşe gelebilir. Bu rastgele dağılım, kuyruk teorisi için esastır.

Sunucu Boş

Yeni bir müşteriye hizmet verip veremeyeceğimizi belirlemek için pseudocode, Algoritma 4-13'te gösterilmiştir.

ALGORİTMA 4-13 Kuyruk Simülasyonu: Sunucu Boş

```

Algoritma serverFree (kuyruk, saat, durum, dahaFazlaMüşteri)
Bu algoritma sunucunun boşta olup olmadığını belirler ve eğer öyleyse
yeni bir müşteriye hizmet vermeye başlar.
  Ön  kuyruk geçerli bir kuyruk için bir yapıdır
      saat mevcut saat dakikasıdır
      durum mevcut/önceki müşteri hakkında veri tutar
  Sonra dahaFazlaMüşteri bir çağrı başlatıldığında doğru olarak ayarlanır
1 eğer (saat > durum başlangıçZamanı + durum hizmetZamanı - 1)
  Sunucu boştur.
  1 eğer (boşKuyruk değilse (kuyruk))
    1 kuyruktan (kuyruk, müşteriVerisi) çıkar
    2 durum müşteriNumarası değerini müşteriVerisi numarasına ayarla
    3 durum varışZamanı değerini müşteriVerisi varışZamanına ayarla
    4 durum başlangıçZamanı değerini saate ayarla
    5 durum hizmetZamanı değerini rastgele hizmet süresine ayarla
    6 moreCusts'i true olarak ayarla
  2 eğer sonlandı
2 son eğer
son sunucuÜcretsiz

```

Algoritma 4-13 Analizi Bu algoritmadaki soru, “Sunucunun boş olduğunu nasıl anlarız?” Müşteri durum kaydı cevabı tutar. Bu, ya mevcut müşteriyi ya da mevcut müşteri yoksa önceki müşteriyi temsil eder. Eğer mevcut bir müşteri varsa, yeni bir müşteri başlatamayız. Kayıt yeni müşteriyi temsil edip etmediğini, saat zamanını müşteri için bitiş zamanı ile karşılaştırarak anlayabiliriz. Bitiş zamanı, başlangıç zamanı artı gerekli hizmet zamanıdır. Eğer saat zamanı hesaplanan bitiş zamanından büyükse, o zaman, eğer kuyruk boş değilse, yeni bir müşteri başlatırız.

Bir soru daha yanıtlanmayı bekliyor: “İlk çağrıyı nasıl başlatırız?” Müşteri durum yapısını oluşturduğumuzda, her şeyi 0 olarak başlatırız. Şimdi, 1. ifadede yer alan formülü kullanarak, 0 başlangıç zamanı artı 0 hizmet zamanı eksi 1'in saat zamanından küçük olması gerektiğini görüyoruz, böylece ilk müşteriyi başlatabiliriz.

Hizmet Tamamlandı

Hizmet tamamlandığına dair mantık, Algoritma 4-14'te gösterilmiştir.

ALGORİTMA 4-14 Kuyruk Simülasyonu: Hizmet Tamamlandı

```

Algoritma svcTamam (kuyruk, saat, durum, istatistikler,
dahaFazlaMüşteri)
Bu algoritma, mevcut müşterinin işleminin tamamlanıp tamamlanmadığını belirler.
  Ön  kuyruk, geçerli bir kuyruk için bir yapıdır
      saat, mevcut saat dakikasıdır
      durum, mevcut/önceki müşteri hakkında verileri tutar
      istatistikler, tamamlanmış simülasyon için verileri içerir

```

devam ediyor

ALGORİTMA 4-14Kuyruk Simülasyonu: Hizmet Tamam (devam ediyor)

```

Hizmet tamamlandıysa, mevcut müşteri için veriler
yazdırıldı ve simülasyon istatistikleri güncellendi
çağrı tamamlandıysa moreCusts false olarak ayarlandı
1 eğer (saat eşit durum başlangıçZamanı + durum svcZamanı - 1)
Mevcut çağrı tamamlandı
1 beklemeZamanını durum başlangıçZamanı - durum varışZamanı olarak ayarla
2 istatistik numCust'ı artır
3 istatistik totSvcZamanı'nı istatistik totSvcZamanı + durum svcZamanı olarak ayarla
4 istatistik totWaitZamanı'nı istatistik totWaitZamanı + beklemeZamanı olarak ayarla
5 queueCount (queue) değerini queueSize olarak ayarla
6 eğer (stats maxQueueSize < queueSize)
1 stats maxQueueSize değerini queueSize olarak ayarla
7 eğer sonlandır
8 yazdır (durum custNum durum arriveTime
fff durum startTime durum svcTime
waitTime queueCount (queue))
9 fmoreCusts değerini false olarak ayarla
2 son eğer
son svcComplete

```

Algoritma 4-14 AnaliziBu algoritmadaki soru, sunucu ücretsiz sorusuna benzer. Analiz etmeye yardımcı olmak içinsimülasyon için birkaç varsayımsal hizmet süresine bakalım. BunlarTablo 4-1'de gösterilmiştir.

Başlangıç zamanı	Hizmet süresi	Tamamlanan süre	Hizmet edilen dakikalar
1	2	2	1 ve 2
3	1	3	3
4	3	6	4, 5 ve 6
7	2	8	7 ve 8

TABLO 4-1Varsayımsal Simülasyon Hizmet Süreleri

Yeni bir müşteri için sunucu boşken test yaptığımızdan, tamamlanmış bir müşteri için test etmeden önce, yalnızca 1 dakikaya ihtiyaç duyan bir müşteriyi aynı döngüde başlatıp bitirmek mümkündür. Bu, yukarıdaki 3. dakikada gösterilmiştir. Hizmet süresinin sonu için doğru hesaplama bu nedenle

$$\text{başlangıç zamanı} + \text{hizmet süresi} - 1$$

Bu hesaplama, Tablo 4-1'deki her bir örnekte doğrulanmaktadır. Bu nedenle, hizmet süresinin sonuna eşit olan saat zamanını test ediyoruz ve eğer onlar

eşitse, mevcut müşteri için gerekli verileri yazdırır ve simülasyonun sonu için gerekli istatistikleri biriktiririz.

İstatistikleri Yazdır

Simülasyondaki son algoritma, tüm simülasyon için istatistikleri yazdırır. Kod, Algoritma 4-15'te gösterilmiştir.

ALGORİTMA 4-15 Kuyruk Simülasyonu: İstatistikleri Yazdır

```
Algoritma printStats (istatistikler)
Bu algoritma simülasyon için istatistikleri yazdırır.
  Ön istatistikler çalışma istatistiklerini içerir
  Son istatistikler yazdırıldı
1 print (Simülasyon İstatistikleri:)
2 print ("Toplam müşteri: " istatistikler numCust)
3 print ("Toplam hizmet süresi: " istatistikler totSvcTime)
4 set avrgSvcTime to istatistikler totSvcTime / istatistikler numCust
5 print ("Ortalama hizmet süresi: " avrgSvcTime)
6 set avrgWaitTime to istatistikler totWaitTime / istatistikler numCust
7 print ("Ortalama bekleme süresi: " avrgWaitTime)
8 print ("Maksimum kuyruk boyutu: " stats maxQueueSize)
end printStats
```


4.6 Anahtar Terimler

geliş hızı	kuyruk sayısı
kuyruk oluştur	kuyruk veri düğümü
müşteri	kuyruk veri yapısı
kuyruktan çıkar	kuyruk ön
kuyruk yok et	kuyruk başı
uyruk boşkuy	kuyruk arkası
ruk ekle	kuyruk simülasyonu
ilk giren-ilk çıkan (FIFO)	arka
ön	hizmet
tam kuyruk	hizmet süresi
çoklu sunucu kuyrukları	tek sunucu kuyruk

4.7 Özet

- Bir kuyruk, verilerin yalnızca bir uçta, arka uçta eklenebildiği ve diğer uçta, ön uçta silinebildiği doğrusal bir listedir.
- Bir kuyruk, ilk giren ilk çıkar (FIFO) yapısıdır.
- Dört temel kuyruk işlemi vardır: ekleme, çıkarma, kuyruk ön ucu, ve kuyruk arka ucu.
 1. Ekleme işlemi, kuyruğun arka ucuna bir öğe ekler.
 2. Çıkarma işlemi, kuyruğun ön ucundaki öğeyi siler.
 3. Kuyruk ön ucu işlemi, kuyruğun ön ucundaki öğeyi silmeden inceler.
 4. Kuyruk arka ucu işlemi, kuyruğun arka ucundaki öğeyi silmeden inceler.
- Kuyruklar, bağlı listeler veya diziler kullanılarak uygulanabilir.
- Bağlı liste kullanarak kuyruğu uygulamak için iki tür yapı kullanılır: bir baş ve bir düğüm.
- Kuyruk teorisi, kuyrukların performansını tahmin etmek için kullanılan uygulamalı matematik alanıdır.
- Kuyruk uygulamaları, tek sunucular ve çoklu sunucular olarak ikiye ayrılabilir.
 1. Tek sunucu kuyruk uygulaması, aynı anda yalnızca bir müşteriye hizmet verir.
 2. Çoklu sunucu kuyruk uygulaması, aynı anda birkaç müşteriye hizmet verir.

- ❑ Kuyrukların performansını en çok etkileyen iki özellik varış hızı ve hizmet süresidir.
 1. Müşterilerin hizmet almak için kuyruğa geldiği hız varış hızı olarak bilinir.
 2. Hizmet süresi, bir müşteri talebinin işlenmesi için gereken ortalama süredir.
- ❑ Kuyruk süresi, müşterilerin kuyrukta beklediği ortalama süre uzunluğudur.
- ❑ Yanıt süresi, müşterilerin kuyruğa girdiği andan sunucudan çıktığı ana kadar geçen ortalama süreyi ölçen bir değerdir. Bu, kuyruk süresi artı hizmet süresidir.
- ❑ Kuyrukların bir uygulaması, bir kuyruğun performansı hakkında istatistikler üretmek için kullanılan bir modelleme etkinliği olan kuyruk simülasyonudur.
- ❑ Kuyrukların bir diğer uygulaması ise kategorilendirmedir. Kuyruklar, verilerin orijinal sıralamasını kaybetmeden farklı gruplara kategorize edilmesi için kullanılır.

4.8 Pratik SetlerAlı

ştırmalar

1. Bir tamsayı yığını, s , ve bir tamsayı kuyruğu, q , olduğunu hayal edin. Aşağıda ki işlemlerden sonra s ve q için bir resim çizin:

```

1 pushStack (S, 3)
2 pushStack (S, 12)
3 ve böylece (S, 5)
4 ve böylece (S, 8)
5 popStack (S, x)
6 pushStack (S, 2)
7 sıraya almak (Q, x)
8 kuyruktan çıkarma (Q, y)
9 pushStack (S, x)
10 pushStack (S, y)

```

2. Kuyrukların Q1 ve Q2 ve yığın S değeri aşağıdaki algoritma segmentinden sonra ne olur?

```

1  S = createStack
2  S1 = createQueue
3  S2 = createQueue
4  kuyruğa alma (Q1, 5)
5  kuyruğa alma (Q1, 6)
6  sıraya alma (Q1, 9)
7  kuyruk (Q1, 0)
8  kuyruk (Q1, 7)
9  kuyruğa alma (Q1, 5)
10 kuyruk (Q1, 0)
11 kuyruk (Q1, 2)
12 kuyruğa alma (Q1, 6)
13 döngü (boşKuyruk değil (Q1))
    1 kuyruktanÇıkar (Q1, x)
    2 eğer (x == 0)
        1 z = 0
        2 döngü (boşYığın değil (S))
            1 yığınÇıkar (S, y)
            2 z = z + y
        3 döngüyü bitir
        4 kuyruk (Q2, z)
    3 aksi takdirde
        1 pushStack (S, x)
    4 eğer son ise bitir
14 döngüyü bitir

```

3. Aşağıdaki kod çalıştırıldığında ve aşağıdaki veriler girildiğinde kuyruk Q içeriği ne olur?

```

1 Q = kuyrukOluştur
2 döngü (dosya sonu değil)
    1 numarayı oku
    2 eğer (numara 0 değilse)
        1 kuyrukEkle (Q, numara)
    3 aksi takdirde
        1 x = kuyrukSonu (Q)
        2 sıraya almak (Q, x)
    4 eğer son ise bitir
3 döngüyü bitir

```

Veriler: 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0.

4. Aşağıdaki kod çalıştırıldığında ve aşağıdaki veriler girildiğinde kuyruk Q1 ve kuyruk Q2 içeriği ne olur?

```

1 S1 = createQueue
2 S2 = createQueue
3 döngü (dosya sonu değil)
  1 numarayı oku
  2 kuyruğa alma (Q1, sayı)
  3 kuyruğa alma (Q2, sayı)
  4 döngü (boş değil Q1)
    1 kuyruktanÇıkar (Q1, x)
    2 kuyruk (Q2, x)
  5 döngüyü bitir
4 döngüyü bitir

```

Veriler 5, 7, 12, 4, 0, 4, 6'dır.

5. Sonraki kod çalıştırıldığında ve aşağıdaki veriler girildiğinde, kuyruk Q1 içeriği ne olur?

```

1 S1 = createQueue
2 S1 = createStack
3 döngü (dosya sonu değil)
  1 numarayı oku
  2 eğer (numara 0 değilse)
    1 pushStack (S1, sayı)
  3 aksi
    1 popStack (S1, x)
    2 popStack (S1, x)
  3 döngü (boş değil S1)
    1 popStack (S1, x)
    2 kuyruğa alma (Q1, x)
  4 döngüyü bitir
4 eğer son ise bitir
4 döngüyü bitir

```

Veriler 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0'dır.

6. Kuyruk Q1 ve kuyruk Q2 içeriğinin gösterildiği gibi olduğunu hayal edin . Aşağıdaki kod çalıştırıldığında Q3 içeriği ne olur? Kuyruk içeriği ön (sol) ile arka (sağ) arasında gösterilmektedir.

1.Ç: 42 30 41 31 19 20 25 14 10 11 12 15

2.Ç: 4 5 4 10 13

```

1 S3 = createQueue
2 sayım = 0
3 döngü (boş Q1 değil ve boş Q2 değil)
    1 sayım = sayım + 1
    2 kuyruktanÇıkar (Q1, x)
    3 kuyruktan çıkarma (Q2, y)
    4 if (y eşit sayım)
        1 kuyruk (Q3, x)
    5 eğer son ise bitir
1 döngüyü bitir

```

Problemler

7. Kuyruk ADT'sindeki algoritmaları kullanarak, bir kuyruğun içeriğini başka bir kuyruğa kopyalayan `copyQueue` adlı bir uygulama algoritması yazın.
8. Problem 7'nin çözümünün çok verimli olmadığını belirlemek için fazla bir analiz gerekmiyor. Bir kuyruğu ADT bilgisi kullanarak kopyalayan yeni bir ADT yöntemi yazmak çok daha verimli olurdu. Problem 7'yi yeni bir ADT algoritması olarak yeniden yazın.
9. Kuyruk ADT'sindeki algoritmaları kullanarak, iki kuyruğu birleştiren `catQueue` adlı bir algoritma yazın. İkinci kuyruk, birinci kuyruğun sonuna eklenmelidir.
10. C fonksiyonu yazın, Problem 9'u uygulamak için.
11. `stackToQueue` adında bir algoritma yazın, bu algoritma bir yığın kullanarak bir kuyruk oluşturur. Kuyruk oluşturulduktan sonra, yığının üstü kuyruğun önünde, yığının tabanı ise kuyruğun arkasında olmalıdır. Algoritmanın sonunda, yığın boş olmalıdır.
12. `queueToStack` adında bir algoritma yazın, bu algoritma bir kuyruk kullanarak bir yığın oluşturur. Algoritmanın sonunda, kuyruk değişmemiş olmalıdır; kuyruğun önü yığının üstü, kuyruğun arkası ise yığının tabanı olmalıdır.
13. Bir dizeyi, dizideki tüm boş karakterleri silerek sıkıştıran bir algoritma yazın. Bunu yapmanın bir yolu, karakterlerden oluşan bir kuyruk kullanmaktır. Diziden boş olmayan karakterleri kuyruğa ekleyin. Dizenin sonuna ulaştığınızda, kuyruktan karakterleri çıkarın ve bunları tekrar dizeye yerleştirin.
14. Bir tamsayı kuyruğu verildiğinde, yalnızca kuyruk ADT'sini kullanarak, kuyruktaki tamsayıların toplamını ve ortalamasını hesaplayan ve yazdıran bir algoritma yazın, kuyruk içeriğini değiştirmeden.
15. Bir tamsayı kuyruğu verildiğinde, kuyruktaki kalan elemanların sırasını değiştirmeden tüm negatif tamsayıları silen bir algoritma yazın.

16. Bir kuyruğun içeriğini tersine çeviren bir algoritma yazın.
17. Sadece kuyruk ADT'sindeki algoritmaları kullanarak, iki kuyruğun içeriğini kontrol eden ve eğer aynı iseler true, değilse false döndüren bir algoritma yazın.
18. Problem 17'yi bir C fonksiyonu olarak uygulayın.

Projeler

19. Bölüm 4.3 "KuyrukADT"de tanımlanan bağlı liste soyut veri türünü kullanarak, ADT'deki her bir işlemi test etmek için menü tabanlı bir kullanıcı arayüzü yazın. İşlem sırasında keşfedilen hatalar, test sonucunun bir parçası olarak yazdırılmaktadır. Aşağıda önerilen bir menü gösterilmektedir.

A. Verileri kuyruğa ekle
 B. Verileri çıkar ve yazdır
 C. Öndeki verileri yazdır
 D. Arkadaki verileri yazdır
 E. Tüm kuyruğu yazdır
 F. Kuyruk durumu: Boş
 G. Kuyruk durumu: Dolu
 H. Eleman sayısını yazdır ve çık

Programınızı aşağıdaki test durumlarıyla test edin. Aşağıda gösterilen testleri gerçekleştirdikten sonra ek test durumları ekleyebilirsiniz.

- a. Kuyruk durumunu yazdır, Boş (F).
- b. Veriyi kuyruktan çıkar ve yazdır (B). Hata döndürmelidir.
- c. Kuyruğa veri ekle: 5 (A).
- d. Verileri kuyruğa ekle: 8 (A).
- ve. Kuyruk durumunu yazdır, Boş (F).
- f. Yazdırma kuyruğu durumu, Dolu (G).
- g. Öndeki verileri yazdır (C).
- h. Arkadaki verileri yazdır (D).
- Ben. Tüm kuyruğu yazdır (E).
- j. Kuyruktaki eleman sayısını yazdır (H).
- k. Verileri kuyruktan çıkar ve yazdır (B).
- l. Verileri kuyruktan çıkar ve yazdır (B).
- m. Verileri kuyruktan çıkar ve yazdır (B). Hata döndürmelidir.
- n. Kuyruğa veri ekle: 14 (A).
- . Öndeki verileri yazdır (C).
- p. Arkadaki verileri yazdır (D).
- s. Verileri kuyruğa ekle: 32 (A).
- r. Öndeki verileri yazdır (C).
- s. Arkadaki verileri yazdır (D).
- t. Kuyruğu yok et ve çık (I).

20. Bir ön ek ifadesini değerlendirmek için bir kuyruk kullanmak bir yoldur. İfadeyi değerlendirmek için, son ifadenin değerini bilene kadar onu tekrar tekrar tarayın. Her taramada, token'ları okuyun ve bunları bir kuyruğa depolayın. Her taramada, bir operatörü, ardından iki operand ile birlikte, hesaplanan değerleri ile değiştirin. Örneğin, aşağıdaki ifade, 159'a değerlendirilen bir ön ek ifadesidir:

$$- + * 9 + 2 8 * + 4 8 6 3$$

İfadeyi tarıyoruz ve bir kuyruğa kaydediyoruz. Tarama sırasında bir operatörün iki operand ile takip edildiği durumlarda, örneğin $+ 2 8$, sonucu, $10'u$, kuyruğun içine koyuyoruz.

İlk taramadan sonra,

$$- + * 9 10 * 12 6 3$$

İkinci taramadan sonra,

$$- + 90 72 3$$

Üçüncü taramadan sonra,

$$- 162 3$$

Dördüncü taramadan sonra,

$$159$$

Bir öncelik ifadesini değerlendiren bir C programı yazın.

21. "Kuyruk Simülasyonu" bölümünde tanımlanan tuzlu şeker dükkanı için C uygulamalarını yazın, 175. sayfadan başlayarak.
22. C ADT'sini kullanarak, yerel pizza dükkanınız gibi küçük bir işletmede bulunabilecek bir telefon sisteminin çalışmasını simüle eden bir program yazın. Sadece bir kişi telefonu cevaplayabilir (tek sunucu kuyruğu), ancak cevaplanmayıp bekleyen sınırsız sayıda çağrı olabilir.

Kupon analizi, iki temel unsuru dikkate alır: bir talep sahibinin hizmet için beklediği süre (kupon bekleme süresi - bu durumda, pizza siparişi veren müşteri) ve hizmet süresi (müşterinin siparişi vermesi için geçen süre). Programınız, telefonun çalışmasını simüle etmeli ve süreç boyunca istatistikler toplamalıdır.

Programın simülasyonu çalıştırmak için iki girdiye ihtiyacı vardır: (1) hizmetin sağlandığı saat cinsinden süre ve (2) operatörün bir sipariş alması için gereken maksimum süre (maksimum hizmet süresi).

Simülasyonu çalıştırmak için dört unsura ihtiyaç vardır: bir zamanlama döngüsü, bir çağrı simülatörü, bir çağrı işleyici ve bir çağrı başlatma fonksiyonu.

- a. Zamanlama döngüsü: Bu, basitçe simülasyon döngüsüdür. Döngünün her y inelemesi gerçek zamanında 1 dakika olarak kabul edilir. Döngü, hizmetin talep edilen süre boyunca çalıştığı sürece devam eder (yukarıdaki girdiye bakın). Ancak, işletim süresi tamamlandığında, simülasyonu sonlandırmadan önce bekleyen tüm çağrılar yanıtlanmalıdır. Zamanlama döngüsünün aşağıdaki alt fonksiyonları vardır:

■ Bir çağrının alınıp alınmadığını belirleyin (çağrı simülatörü)

■ Aktif çağrıyı işleyin

■ Yeni bir çağrı başlatın

Bu sıra, bir çağrının tamamlanmasına ve aynı dakikada başka bir çağrının başlatılmasına olanak tanır.

- b. Arama simülatörü: Arama simülatörü, bir aramanın alınıp alınmadığını belirlemek için rastgele sayı üretici kullanır. Rastgele sayıyı 1 ile 10 gibi uygun bir aralığa ölçeklendirin.

Rastgele sayı, tanımlı bir sabit ile karşılaştırılmalıdır. Değer sabitten küçükse, bir arama alındı; değilse, arama alınmadı. Simülasyon için arama seviyesini %50 olarak ayarlayın; yani, ortalama olarak, her 2 dakikada bir arama alınır. Bir arama alındığında, onu bir kuyruğa yerleştirin.

- c. Aktif aramayı işleme: Eğer bir arama aktifse, tamamlanıp tamamlanmadığını test edin. Tamamlandıysa, mevcut arama için istatistikleri yazdırın ve iş bitiş raporu için gerekli istatistikleri toplayın.
- d. Yeni arama başlat: Eğer aktif arama yoksa, kuyrukta bekleyen bir arama varsa yeni bir arama başlatın. Bir arama başlatmanın, aramanın ne kadar süre beklediğini hesaplaması gerektiğini unutmayın.

İşlem sırasında, her arama tamamlandığında Tablo 4-2'de gösterilen verileri yazdırın. (Not: aynı sonuçları almayacaksınız.)

Saat zamanı	Arama numarası	Varış zamanı	Bekleme zamanı	Başla time	Hizmet time	Kupon boyutu
4	1	2	0	2	3	2
6	2	3	2	5	2	4

TABLO 4-2 Proje 22 için Örnek Çıktı

Simülasyonun sonunda, işleme sırasında toplanan aşağıdaki istatistikleri yazdırın. Her istatistik için uygun bir format kullandığınızdan emin olun, örneğin ortalamalar için bir float.

- a. Toplam çağrılar: işletim süresi boyunca alınan çağrılar
- b. Toplam boşta kalma süresi: çağrılarının hizmet verilmediği toplam süre
- c. Toplam bekleme süresi: tüm çağrılarının bekleme sürelerinin toplamı
- d. Toplam hizmet süresi: tüm çağrılarının hizmet sürelerinin toplamı
- ve. Maksimum kuyruk boyutu: simülasyon sırasında bekleyen en fazla çağrı sayısı
- f. Ortalama bekleme süresi: toplam bekleme süresi/çağrı sayısı
- g. Ortalama hizmet süresi: toplam hizmet süresi/çağrı sayısı

Simülatörü iki kez çalıştırın. Her iki çalışma da 2 saat simüle etmelidir. İlk simülasyonda, maksimum hizmet süresi 2 dakika olmalıdır. İkinci çalışmada, maksimum hizmet süresi 5 dakika olmalıdır.

23. Proje 22'de kuyruk simülasyonunu tekrarlayın, bir bankada bulabileceğiniz gibi birden fazla kuyruk sunucusu kullanın. Sadece bir kuyruk olmalıdır. Sunucu sayısı, klavyeden okunacak şekilde, 2 ile 5 arasında değişebilir.

Birden fazla sunucuyu simüle etmek için, her sunucu için ayrı bir müşteri durumu yapısı sağlamanız gerekir. İşlem döngüsünde, her sunucutamamlanma için test edilmelidir. Benzer şekilde, yeni bir çağrı başlatırken, her boşta kalan sunucu için bir çağrı başlatılmalıdır.

Arama simülatörünü, aynı anda 3 müşteriye kadar gelen müşterileri işleyebilecek şekilde de değiştirmelisiniz. Müşteri sayısını belirlemek için, 0 ile 3 arasında rastgele bir sayı üretin. Eğer sayı 0 ise, hiç müşteri gelmemiştir. Eğer sayı 0 değilse, rastgele sayının belirttiği kadar müşteriyi kuyruğa ekleyin.

24. Bir yığın ve kuyruk test sürücüsü yazın. Bir test sürücüsü, bir kütüphaneye yerleştirilecek işlevleri test etmek için oluşturulmuş bir programdır. Temel amacı, işlevleri tamamen test etmektir; bu nedenle, uygulama kullanımı yoktur.

Test edilecek işlevler yığın oluşturma, kuyruk oluşturma, yığına ekleme, yığından çıkarma, kuyruğa ekleme ve kuyruktan çıkarma olarak belirlenmiştir. Gerekirse diğer yığın ve kuyruk işlevlerini de ekleyebilirsiniz. Tüm veriler tam sayılar olmalıdır. Programda aşağıda açıklandığı gibi iki yığın ve iki kuyruk gereklidir.

- a. Girdi yığını: tüm kullanıcı girdilerini depolamak için kullanılır
- b. Girdi kuyruğu: tüm kullanıcı girdilerini depolamak için kullanılır
- c. Çıktı yığını: girdi kuyruğundan silinen verileri depolamak için kullanılır
- d. Çıktı kuyruğu: girdi yığından silinen verileri depolamak için kullanılır

Kullanıcının ya ekleme ya da silme seçmesini isteyen menü tabanlı bir kullanıcı arayüzü kullanın. Eğer bir ekleme isteniyorsa, sistem kullanıcıdan eklenecek tam sayıyı istemelidir. Veriler daha sonra girdi yığına ve girdi kuyruğuna eklenir. Eğer bir silme isteniyorsa, veriler her iki yapıdan da silinir: girdi yığından çıkarılan veriler çıktı kuyruğuna eklenir ve girdi kuyruğundan çıkarılan veriler çıktı yığına eklenir.

İşlem, girdi yapıları boşalana kadar devam eder. Bu noktada, çıktı yığının içeriğini yazdırın ve tüm verilerini silin. Bu çıktıyı “Çıktı Yığını” olarak etiketleyin, ardından çıktı kuyruğundaki tüm verileri yazdırın ve tüm verileri silin. Bu çıktıyı “Çıktı Kuyruğu” olarak etiketleyin. Çıktınız aşağıda gösterildiği gibi formatlanmalıdır.

```
Çıktı Yığını: 18 9 13 7 5 1
Çıktı Kuyruğu: 7 13 9 18 5 1
```

Programınızı aşağıdaki işlemlerle test edin:

```
1 girdi 1 5 sil 9 girdi 6 13 girdi 8
2 girdi 2 6 girdi 0 10 sil 14 sil
3 sil 7 girdi 5 11 girdi 7 15 sil
4 girdi 3 8 sil 12 sil 16 sil
```

Testinizden elde edilen bilgisayar çıktısına ek olarak, çıktınızın gösterdiği yapısal kavramları tanımlayan kısa bir rapor yazın (bir sayfa dan az).

25. Kuyruklar, ağ sistemlerinde yaygın olarak kullanılır. Örneğin, e-posta gönderilmek üzere beklerken ve alıcının posta kutusuna ulaştıktan sonra kuyruklara yerleştirilir. Ancak, çıkışta bulunan e-posta işleyicisi, kuyruktaki bir veya daha fazla mesajı gönderemezse bir sorun ortaya çıkar. Örneğin, bir mesaj, alıcının sistemi mevcut olmadığı için gönderilemeyebilir.

Ortalama 40 mesajı dakikada işleyen bir e-posta simülatörü yazın. Mesajlar alındıkça, bir kuyruğa yerleştirilir.

Simülasyon için, mesajların ortalama 30 mesaj hızıyla geldiğini varsayın. Unutmayın, mesajlar rastgele gelmelidir, bu nedenle mesajların ne zaman alındığını belirlemek için bir rastgele sayı üretici kullanmanız gerekir (bkz. “Kuyruk Simülasyonu,” sayfa 175’te başlıyor).

Her dakikada, en fazla 40 mesajı kuyruktan çıkarabilir ve gönderebilirsiniz. Kuyruktaki mesajların %25’inin herhangi bir işleme döngüsünde gönderilemeyeceğini varsayın. Yine, belirli bir mesajın gönderilip gönderilemeyeceğini belirlemek için rastgele bir sayı kullanmanız gerekir. Gönderilemiyorsa, mesajı kuyruğun sonuna geri koyun (kuyruklayın).

Simülasyonu 24 saat boyunca çalıştırın, her mesajın kaç kez yeniden sıraya alındığını takip edin. Simülasyonun sonunda, aşağıdaki istatistikleri yazdırın:

- Toplam işlenen mesajlar
- Ortalama varış hızı
- Dakikada gönderilen ortalama mesaj sayısı
- Bir dakikada kuyrukta bulunan ortalama mesaj sayısı
- İlk denemede gönderilen mesaj sayısı, ikinci denemede gönderilen mesaj sayısı ve devamı
- Mesajların kaç kez yeniden sıraya alınması gerektiğinin ortalama sayısı (bu ortalamada ilk seferde gönderilen mesajları dahil etmeyin)

Bölüm 5

Genel Doğrusal Listeler

Genel bir doğrusal liste, işlemlerin, örneğin alma, ekleme, değiştirme ve silme gibi, listenin herhangi bir yerinde, yani listenin başında, ortasında veya sonunda yapılabileceği bir listedir. Günlük hayatımızda birçok farklı türde genel listeler kullanıyoruz. Çalışan listeleri, öğrenci listeleri ve favori şarkılarımızın listelerini kullanıyoruz. Şarkı listemizi işlerken, bir şarkıyı arayabilmemiz, yeni bir şarkı ekleyebilmemiz veya verdiğimiz bir şarkıyı silmemiz gerekiyor. Bu bölüm, genel listeleri nasıl koruyup işleyebileceğimizi anlatmaktadır. Basitlik açısından, bundan sonra genel doğrusal listelere *listeler* olarak atıfta bulunacağız.

5.1 Temel İşlemler

Dört temel liste işlemi ekleme, silme, alma ve gezintidir. Ekleme, listeye yeni bir öğe eklemek için kullanılır. Silme, bir öğeyi listeden kaldırmak için kullanılır. Alma, bir öğeye ait bilgiyi liste yapısını değiştirmeden almak için kullanılır. Gezinti, her öğeye bir işlem uygulanarak listeyi gezmek için kullanılır.

Ekleme

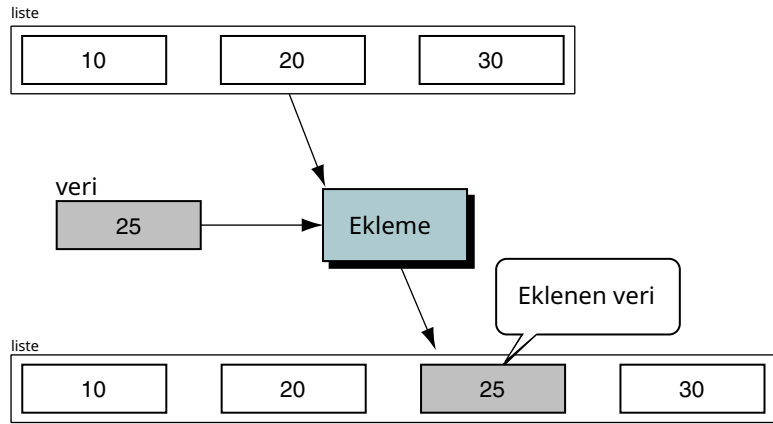
Liste ekleme sıralı veya rastgele olabilir. Sıralı listeler veriye veya mevcutsa, veriyi tanımlayan bir anahtara göre sıralı olarak tutulur. Bir anahtar, veriyi tanımlayan bir yapının içindeki bir veya daha fazla alandır. Anahtar örnekleri Sosyal Güvenlik numarası ve perakende ürün üzerindeki evrensel ürün kodudur.

Rastgele listelerde iki öğe arasında sıralı bir ilişki yoktur.

Rastgele bir listeye veri ekleme konusunda herhangi bir kısıtlama olmamasına rağmen, bilgisayar algoritmaları genellikle veriyi listenin sonuna ekler. Bu nedenle, rastgele listelere bazen kronolojik listeler denir. Rastgele listeleri kullanan uygulamalar, veri toplama uygulamalarında bulunur, bu durumda listeler

kronolojik listelerdir veya uygulamaların rastgelelik gerektirdiği durumlar da, simülasyon çalışmaları veya oyunlar gibi.

Veri, listenin sıralamasının korunması için sıralı listelere eklenmelidir. Sıralamanın korunması, verilerin listenin başına veya sonuna eklenmesini gerektirebilir, ancak çoğu zaman veriler listenin ortasında bir yere eklenir. Verilerin nereye yerleştirileceğini belirlemek için bilgisayar bilimcileri bir arama algoritması kullanır. Eklemeler, Şekil 5-1'de grafiksel olarak gösterilmektedir. Eklenen veriler, gölgeli eleman ile tanımlanır; bu örnekte, revize edilmiş listenin üçüncü elemanıdır.



ŞEKİL 5-1 Ekleme

Silme

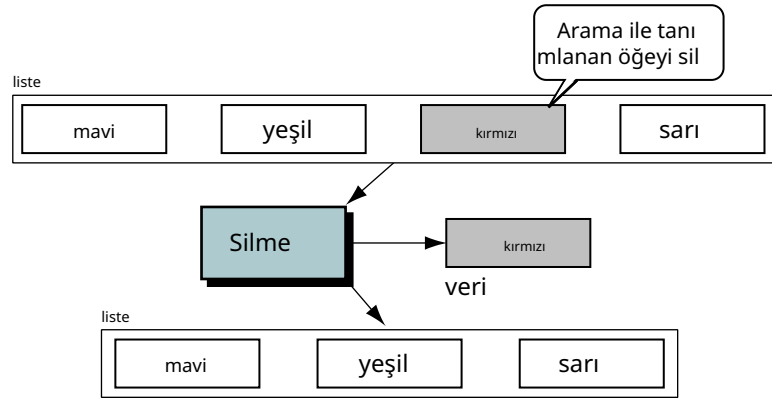
Bir listeden silme, silinecek verilerin bulunması için listenin aranmasını gerektirir. Bulunduktan sonra, veriler listeden çıkarılır. Şekil 5-2, bir listeden silmeyi göstermektedir.

Alım

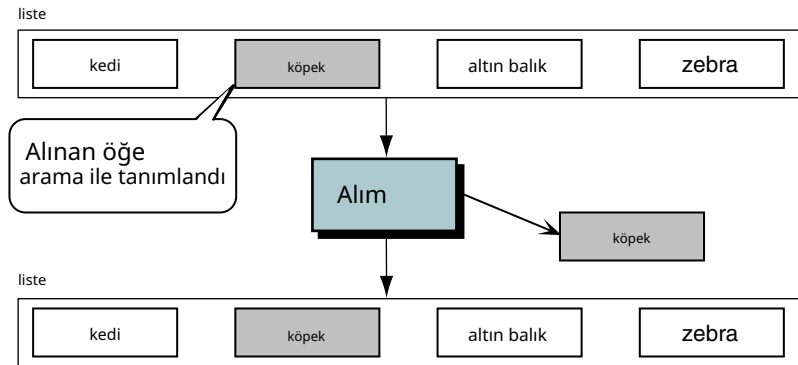
Liste alımı, verilerin bir listede bulunmasını ve çağıran modüle listenin içeriğini değiştirmeden sunulmasını gerektirir. Hem ekleme hem de silme ile olduğu gibi, bir arama algoritması, listeden alınacak verileri bulmak için kullanılabilir. Listedeki veri alımı Şekil 5-3'te gösterilmektedir.

Geçiş

Liste geçişi, bir listedeki her elemanı sırayla işler. Bu, bir arama yerine döngüsel bir algoritma gerektirir. Döngünün her çalıştırılması, listede bir elemanı işler. Tüm elemanlar işlendiğinde döngü sona erer.



ŞEKİL 5-2 Silme



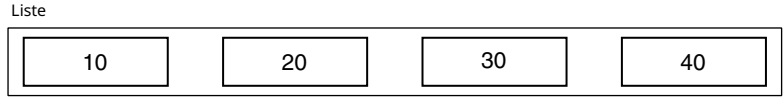
ŞEKİL 5-3 Alım

5.2 Uygulama

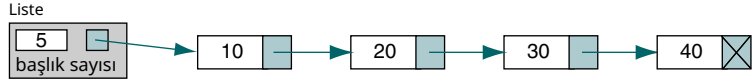
Bir listeyi uygulamak için birkaç veri yapısı kullanılabilir; biz bağlı liste kullanıyoruz. Bir bağlı liste, verilerin listenin başında, ortasında veya sonunda kolayca eklenip silinmesi nedeniyle bir liste için iyi bir yapıdır. Şekil 5-4, bir listenin kavramsal görünümünü ve bağlı liste olarak uygulanışını göstermektedir.

Veri Yapısı

Bir liste uygulamak için, Şekil 5-5'te gösterildiği gibi, bir baş düğüm ve veri düğümü olmak üzere iki farklı yapıya ihtiyacımız var.

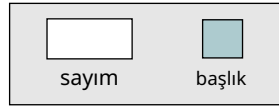


(a) Bir listenin kavramsal görünümü

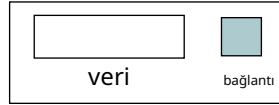


(b) Bağlı liste uygulaması

ŞEKİL 5-4 Bir Listenin Bağlı Liste Uygulaması



(a) Başlık yapısı



(b) Veri düğümü yapısı

```
list
count
head
end list
```

```
node
data
link
end node
```

ŞEKİL 5-5 Baş Düğüm ve Veri Düğümü

Baş Düğüm

Listeyi tanımlamak için yalnızca bir işaretçi gerektiği halde, genellikle baş i şaretçiyi ve liste hakkında diğer verileri saklayan bir baş düğüm yapısı olu şturmak faydalı bulunur. Bir düğüm bir liste hakkında veri içerdiğinde, bu veriler meta veriler olarak bilinir; yani, bunlar listedeki veriler hakkında v erilerdir. Örneğin, Şekil 5-4'teki baş yapısı bir parça meta veri içerir: sayım , listedeki mevcut düğüm sayısını içeren bir tam sayıdır. Liste işlenirken en fazla düğüm sayısı gibi diğer meta veriler, faydalı bir amaç sağladıklarınd a genellikle dahil edilir.

Veri Düğümü

Listenin veri türü tamamen uygulamaya bağlıdır. Aşağıda tipik bir veri tür ü gösterilmektedir. Veri türleri, oluşturulan belirli uygulamaya göre özelle ştirilmelidir. Anahtar ile arama gerektiren uygulamalar için bir anahtar ala nı ekliyoruz.

```

veri
  anahtar
    alan1
    alan2
    ...
    alanN
veri sonu

```

Algoritmalar

Bir liste için 10 işlem tanımlıyoruz, bu da herhangi bir sorunu çözmek için yeterli olmalıdır. Bir uygulama ek liste işlemleri gerektiriyorsa, bunlar kolayca eklenebilir. Her işlem için adını tanımlıyoruz ve kısa bir tanım ve çağrı parametrelerini sağlıyoruz. Daha sonra her işlem için algoritmalar geliştiriyoruz.

Liste Oluştur

Liste oluştur baş yapılandırmasını ayırır ve listenin meta verilerini başlatır. Bu aşamada yalnızca iki meta veri girişi vardır; daha sonra listenin yeteneklerini genişletmek için daha fazlasını ekleyeceğiz. Şekil 5-6, başlatılmadan önce ve sonra başlığı göstermektedir.



ŞEKİL 5-6 Liste Oluştur

Liste oluşturma için sahte kod, Algoritma 5-1'de gösterilmiştir.

ALGORİTMA 5-1 Liste Oluştur

```

Algoritma createList (liste)
  Listenin meta verilerini başlatır.
    Ön liste, referansla geçirilen meta veri yapısı
    Son meta veri başlatıldı
1   ayır (liste)
2   liste başını null olarak ayarla
3   liste sayısını 0 olarak ayarla
son createList

```

Düğüm Ekle

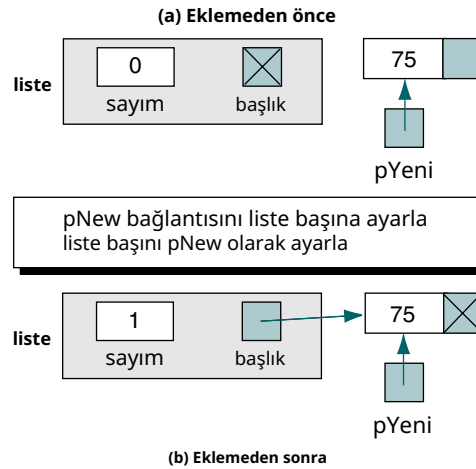
Düğüm ekleme, bir listeye veri ekler. Listeye bir düğüm eklemek için yalnızca mantıksal öncülüne ihtiyacımız var. Önceliği verildiğinde, ekleme için üç adım vardır:

1. Yeni düğüm için bellek ayır ve verileri düğüme taşı.
2. Yeni düğümü, ardılına işaret et.
3. Yeni düğümün öncüsünü yeni düğüme işaret edin.

Bu adımlar basit görünüyor, ancak bunları tam olarak nasıl uygulayacağımızı anlamak için biraz analiz gereklidir. Bir düğümü bir listeye eklemek için, yeni düğümün önceki düğümün konumunu bilmemiz gerekir. Bu düğüm, iki durumdan birinde olabilen bir öncü işaretçi ile tanımlanır: bir düğümün adresini içerebilir veya null olabilir. Önceki işaretçi null olduğunda, eklenen verinin öncüsü olmadığı anlamına gelir. Mantıksal sonuç, ya boş bir listeye ekleme yaptığımız ya da listenin başında olduğumuzdur. Eğer öncü null değilse, ilk düğümün sonra bir yere ekleme yapıyoruz; yani, listenin ortasında veya sonunda. Bu durumların her birini sırayla tartışalım.

Boş Listeye Ekle

Liste baş işaretçisi null olduğunda, liste boşdur. Bu durum Şekil 5-7'de gösterilmiştir. Boş bir listeye bir düğüm eklemek için gereken tek şey, liste baş işaretçisine yeni düğümün adresini atamak ve bağlantı alanının null işaretçi olduğundan emin olmaktır. Yeni düğümün bağlantı alanını ayarlamak için bir sabit kullanabiliriz, ancak liste başında bulunan null işaretçiyi kullanıyoruz. Liste başındaki null işaretçiyi neden kullandığımız bir sonraki bölümde açık olacaktır.



ŞEKİL 5-7 Boş Listeye Düğüm Ekle

Boş bir listeye düğüm eklemek için kullanılan sahte kod ifadeleri burada gösterilmektedir.

pNew bağlantısını liste başına ayarla
liste başını pNew olarak ayarla

(Null işaretçi)
(İlk düğüm)

İfadelerin sırasına dikkat edin. Önce yeni düğümün rakine işaret etmeliyiz; sonra baş işaretçisini değiştirebiliriz. Bu ifadeleri tersine çevirirsek, yeni düğüm kendisine işaret eder, bu da programımızı listeyi işlerken sonsuz bir döngüye sokar.

Başlangıca Ekle

Listeye her zaman ilk düğümünden önce bir düğüm eklememiz gerektiğinde başlangıçta ekleme yaparız. Önceki işaretçiyi test ederek listenin başında ekleme yaptığımızı belirleriz. Eğer bu bir null işaretçi ise, önceki yoktur, yani listenin başındayız.

Listeye bir düğüm eklemek için, yeni düğümü listenin ilk düğümüne işaret ederiz ve ardından baş işaretçisini yeni ilk düğümüne işaret edecek şekilde ayarlarız. Yeni düğümün adresini biliyoruz. Bu noktada soru, yeni düğümü işaret edebilmemiz için listenin içindeki mevcut ilk düğümün adresini nasıl bulabileceğimizdir. Cevap basit: ilk düğümün adresi baş işaretçisinde saklanır. Listenin başına eklemek için kullanılan sahte kod ifadeleri burada gösterilmiştir.

pNew bağlantısını liste başına ayarla
pNew için liste başını ayarla

(Mevcut ilk düğüm)
(Yeni ilk düğüm için)

Bu iki ifadeyi boş bir listeye eklemek için ifadelerle karşılaştırdığımızda, aynı olduklarını göreceğiz. Çünkü mantıksal olarak, boş bir listeye eklemek, bir listenin başına eklemekle aynıdır. Bu nedenle, her iki durumu kapsamak için aynı mantığı kullanabiliriz. Listenin başına eklemek Şekil 5-8'de gösterilmiştir.

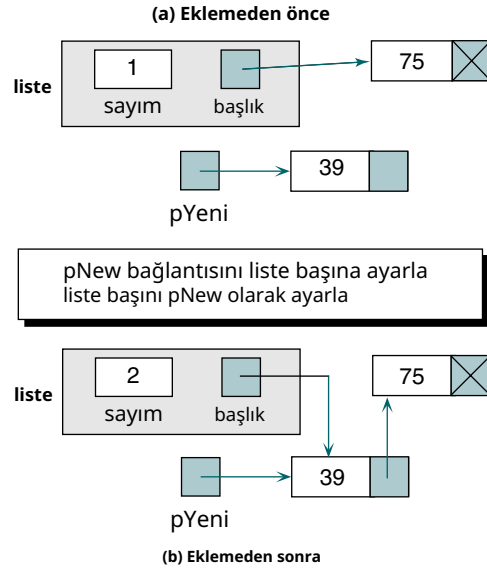
Ortaya Ekle

Listeye ortada herhangi bir yere bir düğüm eklediğimizde, önceki düğümün işaretçisi (pPre) bir adres içerir. Bu durum Şekil 5-9'da gösterilmiştir.

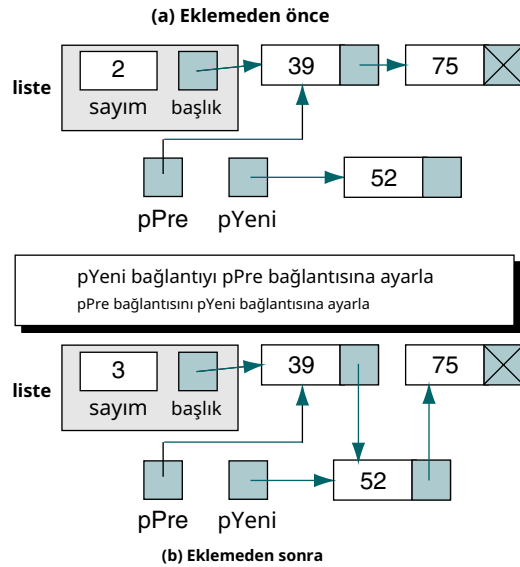
İki düğüm arasında bir düğüm eklemek için, yeni düğümü onun ardıl düğümüne işaret ederiz ve ardından önceki düğümü yeni düğümüne işaret ederiz. Yeni düğümün ardılının adresi, önceki düğümün bağlantı alanında bulunabilir. Listenin ortasında bir düğüm eklemek için kullanılan sahte kod ifadeleri burada gösterilmektedir:

pNew bağlantısını pPre bağlantısına ayarla
pPre bağlantısını pNew'e ayarla

(Yeni, ardıl için)
(Önceki, yeni için)



ŞEKİL 5-8Başlangıçta Düğüm Ekle



ŞEKİL 5-9Ortada Düğüm Ekle

Uçta Ekle

Listeye son eklediğimizde, yalnızca öncüyü yeni düğüme işaret etmemiz gerekir. İşaret edilecek bir ardıl yoktur. Ancak, yeni düğümün bağlantı alanını bir null işaretçisine ayarlamak gereklidir. Bir listeye son eklemek için düğüm ekleme ifadeleri burada gösterilmiştir:

pNew bağlantısını null işaretçisine ayarla
pPre bağlantısını pNew olarak ayarla

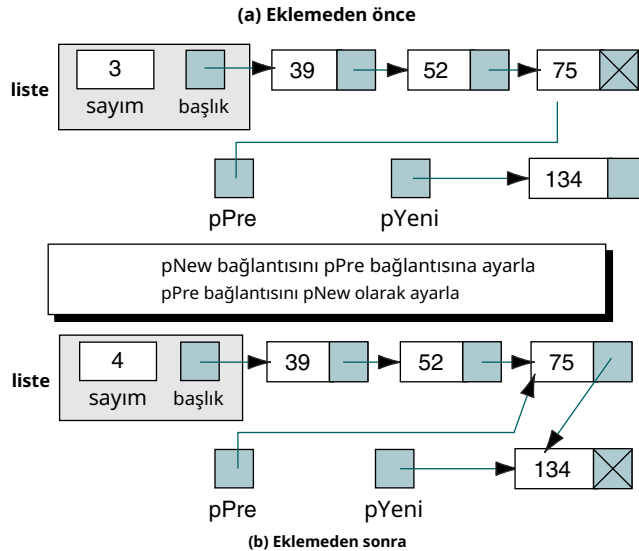
(Önceki, yeni için)

Son ekleme için algorithmada özel bir mantık yerine, mevcut bağlı liste yapısından yararlanabiliriz. Listenin son düğümünün null bağlantı işaretçisine sahip olduğunu biliyoruz. Bu işaretçiyi null işaretçi sabiti yerine kullanırsak, kod tam olarak listenin ortasına ekleme koduyla aynı hale gelir. Gözden geçirilmiş kod aşağıdadır. Bunu listenin ortasına ekleme koduyla karşılaştırın.

pNew bağlantısını pPre bağlantısı olarak ayarla
pPre bağlantısını pNew'e ayarla

(Yeni null'a)
(Önceki, yeni için)

Şekil 5-10, bir listeye son ekleme mantığını göstermektedir.



ŞEKİL 5-10 Uçta Düğüm Ekle

Düğüm Ekleme Algoritması

Şimdi, her şeyi bir araya getiren ve bir düğümü listeye ekleyen algoritmayı yazalım. Listeye, öncüye ve eklenmesi gereken veriye işaret eden bir işaretçimiz var.

eklenmiştir. Yeni düğüm için bellek ayırıyoruz ve bağlantı işaretçilerin uygun şekilde ayarlıyoruz. Algoritma tamamlandığında, başarılıysa true, başarısızsa false döndüren bir Boolean değeri döner. Pseudocode, Algoritma 5-2'de gösterilmiştir.

ALGORİTMA 5-2 Liste Düğümü Ekle

```

Algoritma insertNode (liste, pPre, dataIn)
Listeye yeni bir düğümde veri ekler.
    Ön  liste geçerli bir listeye ait meta veri yapısıdır
        pPre verinin mantıksal öncüsüne işaretçidir
        dataIn eklenmesi gereken veriyi içerir
    Son  veriler sıralı olarak eklenmiştir
    Başarılıysa true, bellek taşması durumunda false döndürün
1 tahsis et (pNew)
2 pNew verisini dataIn olarak ayarlayın
3 eğer (pPre null)
    İlk düğümde önce ekleme veya boş listeye ekleme.
    1 pNew bağlantısını liste başına ayarla
    2 liste başını pNew olarak ayarla
4 aksi takdirde
    Ortada veya sonunda ekleme.
    1 pNew bağlantısını pPre bağlantısı olarak ayarla
    2 pPre bağlantısını pNew olarak ayarla
5 eğer bitir
6 true döndür
son insertNode

```

Algoritma 5-2 Analizi Bu algoritmadaki tüm mantığı, bellek ayırma dışında tartıştık.

Bellek tükendiğinde, ekleme bir overflow durumundadır. Alınan eylem, uygulamaya bağlıdır. Uygulama genellikle programı sonlandırmak zorundadır, bu karar ekleme algoritmasında verilmemelidir. Bu nedenle, başarılı olup olmadığımızı belirten bir Boolean döndürüyoruz ve çağırana modülün sonlandırıp sonlandırmayacağına veya başka bir eylemin uygun olup olmadığına karar vermesine izin veriyoruz.

Düğümü Sil

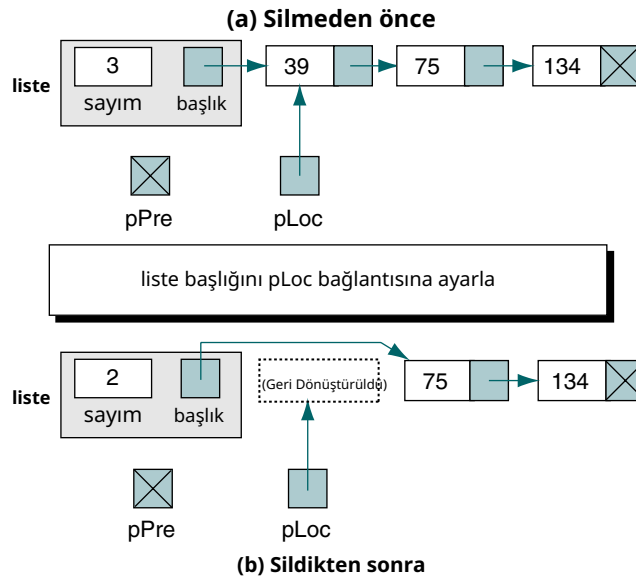
düğüm silme algoritması, çeşitli bağlantı işaretçilerini değiştirerek bir düğüm listeden mantıksal olarak kaldırır ve ardından düğümü dinamik bellekten fiziksel olarak siler. Bir düğümü mantıksal olarak silmek için, birincil düğümü bulmalıyız. Bir düğüm silme, adresini ve öncülünün adresini bilerek bulunur. Konum kavramlarını kısa süre içinde tartışacağız. Silinecek düğümü bulduğumuzda, öncülünün bağlantı alanını silinen düğümün ardılına işaret edecek şekilde değiştirebiliriz. Ardından düğümü dinamik belleğe geri dönüştürüyoruz. Ancak, bir listede tek düğümün silinmesi konusunda endişelenmemiz gerekiyor. Tek düğümün silinmesi boş bir liste ile sonuçlanır, bu nedenle bu durumda başlangıç bağlantısına ayarlandığından emin olmalıyız.

Silme durumları ekleme durumlarıyla paraleldir. Tek düğümü, ilk düğümü, listenin ortasındaki bir düğümü veya bir listenin son düğümünü silebiliriz. Biz

Aşağıda açıklanan bu dört durum yalnızca iki kombinasyona indirgenir: ilk düğüm silinmesi ve diğer herhangi bir düğümün silinmesi. Tüm durumlarda silinecek düğüm bir işaretçi (pLoc) ile tanımlanır.

İlk Düğümü Sil

İlk düğümü sildiğimizde, baş işaretçisini ilk düğümün ardına işaret edecek şekilde sıfırlamalı ve ardından silinen düğüm için belleği geri dönüştürmeliyiz. İlk düğümü sildiğimizi, öncülü test ederek anlayabiliriz. Eğer öncül bir null işaretçisi ise, ilk düğümü siliyoruz demektir. Bu durum Şekil 5-11'de diyagramlanmıştır.



ŞEKİL 5-11 İlk Düğümü Sil

İlk düğümü silmek için ifadeler bir sonraki örnekte gösterilmiştir.

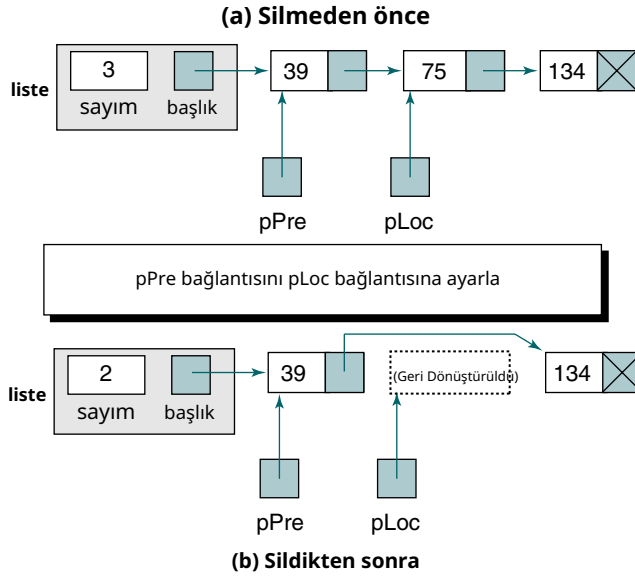
Recycle bir düğümün alanını dinamikbelleğe geri döndürmek için kullanılan sahte kod komutudur.

```
liste başlığını pLoc bağlantısına ayarla
recycle (pLoc)
```

Bu mantığı dikkatlice incelersek, listenin tek düğümünü silerken de geçerli olduğunu not ederiz. Eğer ilk düğüm tek düğümse, o zaman bağlantı alanı bir null işaretçisidir. Bağlantı alanını (bir null işaretçisi) baş işaretçisi taşıdığımızda, sonuç tanım gereği boş bir listedir.

Genel Silme Durumu

İlk düğüm dışında herhangi bir düğümü silmeyi genel durum olarak adlandırıyoruz çünkü listenin ortasında veya sonunda herhangi bir düğümü silmek için aynı mantık geçerlidir. Bu iki durumda da, sadece önceki düğümü silinen düğümün ardıl düğümüne işaret ediyoruz. Mantık Şekil 5-12'de gösterilmiştir.



ŞEKİL 5-12 Genel Silme Durumu

Son düğümü otomatik olarak sileriz. Silinen düğüm listenin son düğümü olduğunda, null işaretçisi önceki düğümün bağlantı alanına taşınır ve böylece önceki düğüm listenin yeni mantıksal sonu haline gelir. Bağlantılar ayarlandıktan sonra, mevcut düğüm geri dönüştürülür.

Silme genel durum sahte kodu burada gösterilmektedir:

```
pPre bağlantısını pLoc bağlantısına ayarla
(pLoc) geri dönüştür
```

Düğüm Silme Algoritması

Bir düğümü silme mantığı Algoritma 5-3'te gösterilmektedir. Algoritmaya bir liste, silinecek düğümün ve silinecek düğümün öncüsüne işaretçi verilir. Silinen düğümün verileri, çağıran programda bir veri dış alanına kopyalanır ve ardından düğümün belleği serbest bırakılmadan önce işaretçiler ayarlanır.

ALGORİTMA 5-3 Liste Düğüm Silme

```

Algoritma deleteNode (liste, pPre, pLoc, dataOut)
Listeden veriyi siler ve çağırana modüle geri döndürür.
  Ön liste, geçerli bir listeye ait meta veri yapısıdır
  Pre, öncü düğüme işaret eden bir işaretli
  pLoc, silinecek düğüme işaret eden bir işaretli
  dataOut, silinen veriyi almak için değişkendir
  Son veriler silinmiş ve çağırana geri döndürülmüştür
1 pLoc verilerini dataOut'a taşı
2 eğer (pPre null)
  İlk düğümü silme
  1 liste başını pLoc bağlantısına ayarla
3 aksi takdirde
  Diğer düğümleri silme
  1 pPre bağlantısını pLoc bağlantısına ayarla
4 eğer bitir
5 geri dön (pLoc)
son deleteNode

```

Algoritma 5-3 Analizi Bu algoritmada iki noktayı tartışmamız gerekiyor. İlk ve en önemli olanı, silinecek düğümün çağrılmadan önce tanımlanması gerektiğidir. Algoritma, önceki ve mevcut işaretçilerin doğru bir şekilde ayarlandığını varsayar. Eğer ayarlanmamışlarsa, algoritma muhtemelen başarısız olacaktır. Başarısız olmasa bile, veriler yanlış olacaktır. (Algoritmanın geçersiz sonuçlar bildirmesinden daha iyi başarısız olmasıdır.)

İkincisi, daha önce bireysel mantık durumlarını tartıştığımızda, her silme ifadeinden sonra geri dönüştürme ifadesini yerleştirmiştik. Uygulamada bunu algoritmanın sonuna taşıdık. Aynı ifadeler, bir seçim ifadesinin hem doğru hem de yanlış bloklarında görüldüğünde, seçim mantığından çıkarılmalıdır. Ortak ifadeleri taşımak, cebirde ortak ifadeleri faktör haline getirmeye benzer. Sonuç, daha küçük ve bakımı daha kolay bir programdır.

Liste Arama

Liste araması, bir listedeki verileri bulmak için birkaç algoritma tarafından kullanılır. Veri eklemek için, yeni verinin mantıksal öncüsünü bilmemiz gerekir. Veri silmek için, silinecek düğümü bulmamız ve onun mantıksal öncüsünü tanımlamamız gerekir. Bir listeden veri almak için, listeyi arayıp veriyi bulmamız gerekir. Ayrıca, birçok kullanıcı uygulaması, verileri bulmak için listelerin aranmasını gerektirir.

Sıralı bir arama kullanmalıyız çünkü düğümler arasında fiziksel bir ilişki yoktur. Klasik sıralı arama, bir öge bulunduğunda onun konumunu ve bulunmadığında son ögenin adresini döndürür. Listemiz sıralı olduğu için, öge bulunduğunda onun konumunu ve bulunmadığında yerleştirilmesi gereken konumu döndürmemiz gerekir. Knuth, bu aramayı "sıralı tabloda sıralı arama" olarak adlandırır. Biz buna basitçe sıralı liste araması diyoruz.

1. Bölüm 13'e bakın.

2. Donald E. Knuth, *Bilgisayar Programlamanın Sanatı*, Cilt 3, *Sıralama ve Arama*, İkinci Baskı (Reading, MA: Addison-Wesley, 1998), 398.

Bir anahtar üzerinde bir listeyi aramak için bir anahtar alanına ihtiyacımız var. Basit listeler için anahtar verdata aynı alan olabilir. Daha karmaşık yapılar için ayrı bir anahtar alanına ihtiyacımız var. Kolaylığınız için, 5-2 Bölümündeki “Veri Düğümü”nden veri düğümü yapısını yeniden üretiyoruz:

```

veri
  anahtar
    alan1 al
    an2
    ...
    alanN
  veri sonu

```

Belirli bir anahtar verildiğinde, sıralı liste araması istenen düğümü listede bulmaya çalışır. Eğer listede bir düğüm hedef değeriyle eşleşiyorsa, arama doğru (true) döner; eğer anahtar eşleşmesi yoksa, yanlış (false) döner. Önceki ve mevcut işaretçiler, Tablo 5-1'deki kurallara göre ayarlanır.

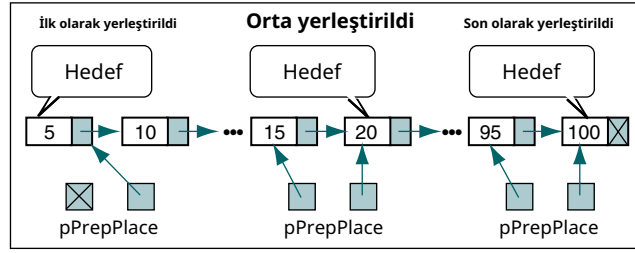
Koşul	pPre	pLoc	İade
Hedef < ilk düğüm	Hükümsüz	İlk düğüm	Yanlış
Hedef = ilk düğüm	Hükümsüz	İlk düğüm	Doğru
İlk < hedef < son	En büyük düğüm < hedef	İlk düğüm > hedef	Yanlış
Hedef = orta düğüm	edefDüğümün öncü	Eşit düğüm	Doğru
Hedef = son düğüm	İlSonun öncülüSo	Son düğüm	Doğru
Hedef > son düğüm	n düğüm	Null	Yanlış

TABLO 5-1 Liste Arama Sonuçları

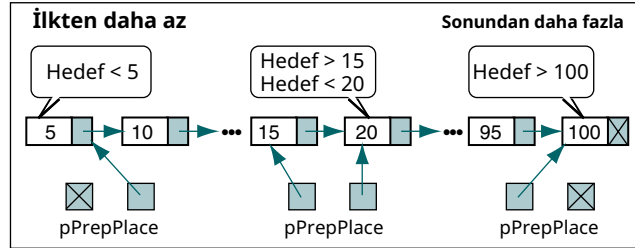
Bu koşulların her biri Şekil 5-13'te de gösterilmektedir.

Başlangıçtan başlayarak, hedef değer mevcut düğümün anahtarından daha büyük olmadığı sürece listeyi sıralı olarak ararız. Bu noktada hedef değer, mevcut düğümün anahtarından ya daha küçük ya da eşittir ve önceki düğüm mevcut düğümün hemen önceki düğümüne işaret etmektedir. Daha sonra mevcut düğümü test ederiz ve hedef değeri liste değeriyle eşitse döndürme değerini doğru (true) olarak ayarlarız, eğer daha küçükse (daha büyük olamaz) yanlış (false) olarak ayarlarız ve aramayı sonlandırırız. Bu arama için psödo kodu Algoritma 5-4'te gösterilmektedir.

Önceki tartışmada, listenin bir anahtar üzerinde sıralı olduğunu varsaydık. Ancak, genellikle listenin bir liste niteliği üzerinde aranması gerekir; anahtar üzerinde değil. Örneğin, bir şirketteki çalışanlar hakkında bir bilgi listesi verildiğinde, mühendislik diplomasına sahip olan veya Japonca konuşan çalışanları bulmak isteyebiliriz.



(a) Başarılı aramalar (doğru döner)



(b) Başarısız aramalar (false döndür)

ŞEKİL 5-13 Sıralı Liste Arama

ALGORİTMA 5-4 Listeyi Ara

Algoritma searchList (liste, pPre, pLoc, hedef)

Listeyi arar ve hedefi içeren düğümün adresini göndermektedir ve mantıksal öncüsünü.

Ön liste, geçerli bir listeye ait meta veri yapısıdır

pPre, öncüyü gösteren işaretçi değişkenidir

pLoc, mevcut düğümü gösteren işaretçi değişkenidir

hedef, aranan anahtardır

Son pLoc, eşit/büyük anahtara sahip ilk düğümü gösterir

-veya- hedef son düğümün anahtarından büyükse null

pPre, anahtarın altında en büyük düğümü işaret eder

-veya- hedef < ilk düğümün anahtarı ise null

Bulunduysa true, bulunmadıysa false döndür

1 pPre'yi null olarak ayarla

2 pLoc'u liste başına ayarla

3 döngü (pLoc null değil VE hedef > pLoc anahtarı)

1 pPre'yi pLoc olarak ayarla

2 pLoc'u pLoc bağlantısına ayarla

4 döngüyü sonlandır

5 eğer (pLoc null ise)

Dönüş değerini ayarla

1 bulundu'yu false olarak ayarla

devam ediyor

ALGORİTMA 5-4 Liste Ara (devamı)

```

6  aksi takdirde
  1  eğer (hedef pLoc anahtarına eşit)
    1  bulundu'yu doğru olarak ayarla
  2  aksi takdirde
    1  bulundu'yu false olarak ayarla
  3  eğer son ise bitir
7  eğer bitir
8  bulundu'yu döndür
son  searchList

```

Algoritma 5-4 Analizi Döngü ifadesini dikkatlice inceleyin. İki test olduğunu unutmayın. İlk test, liste sonuna kadar gitmemizi engeller; ikinci test, hedefi bulduğumuzda veya hedef mevcut değilse, hedeften daha büyük bir düğüm bulduğumuzda döngüyü durdurur. Null işaretçi testinin önce yapılması önemlidir. Eğer döngü listenin sonunda ise, mevcut işaretçi artık geçerli değildir. Anahtarı önce test etmek öngörülemeyen sonuçlar verebilir.³

Arka işaretçimiz olsaydı, aramayı biraz daha verimli hale getirebilirdik. Arka işaretçi, listede son düğümün adresini içeren bir meta veri alanıdır. Bir arka işaretçi ile, hedefin anahtar değerinden daha büyük olmadığından emin olmak için son düğümü test edebilirdik. Eğer hedef daha büyük olsaydı, döngüyü atlayarak, öncü işaretçiyi son düğümüne ve mevcut işaretçiyi null işaretçiye ayarlardık. Hedefin son düğümünden daha büyük olmadığını bildiğimizde, listenin sonuna kadar gitme konusunda endişelenmemize gerek kalmaz.

Bir listeyi anahtar dışında herhangi bir alanda aramak için basit bir sıralı arama kullanırız. Ancak, anahtar dışı aramaların sorunu, birden fazla öğenin genellikle arama kriterlerini karşılamasıdır. Japonca konuşan yalnızca bir çalışmamız olabilir, ancak hiçbir olmayabilir veya birçok çalışmamız olabilir. Basit bir çözüm, kriterleri karşılayan tüm öğelerin bir listesini döndürmektir.

Düğümü Al

Bir listedeki düğümü nasıl bulacağımızı bildiğimize göre, düğümü almayı incelemeye hazırız. Düğümü al, listedeki verileri bulmak için düğümü arar. Veriler bulunursa, verileri çağıran modüldeki çıktı alanına taşır ve doğru döner. Bulunmazsa, yanlış döner. Pseudocode, Algoritma 5-5'te gösterilmiştir.

Boş Liste

İşlem mantığı genellikle bir listede veri olmasına bağlıdır. Boş liste sağlayarak, listede veri olduğunu veya boş olduğunu belirten bir Boolean döndüren basit bir modül sunuyoruz (Algoritma 5-6).

3. Pascal gibi bazı dillerin, sonuçları değerlendirmeden önce koşuldaki her iki ifadeyi de test ettiğini unutmayın. Bu durumlarda döngü sınır testi koşulu iki ayrı ifadeye bölünmelidir.

ALGORİTMA 5-5 Liste Düğümünü Al

```

Algoritma retrieveNode (liste, anahtar, dataOut)
Bir listeden veri alır.
  Ön  liste, geçerli bir listeye ait meta veri yapısıdır
      anahtar, alınacak verinin hedefidir
      dataOut, alınan veriyi almak için değişkendir
  Son  data, dataOut'a yerleştirilir
      -veya- bulunamazsa hata döner
  Başarılysa true, veri bulunamazsa false döner
1 set found to searchList (liste, pPre, pLoc, anahtar)
2 eğer (bulundu)
  1 pLoc verisini dataOut'a taşı
3 eğer bitir
4 geri dönüş bulundu
end getNode

```

ALGORİTMA 5-6 Boş Liste

```

Algoritma boşListe (liste)
Liste boş olup olmadığını belirten Boolean döndürür.
  Ön  liste, geçerli bir listeye ait meta veri yapısıdır
      Liste boşsa true, liste veri içeriyorsa false döndürün
1 eğer (liste sayısı 0'a eşitse)
  1 true döndür
2 aksi takdirde
  1 false döndür
son boşListe

```

Algoritma 5-6 Analizi Program tasarımı öğrenen öğrenciler tarafından sıkça sorulan sorulardan biri “Sadece bir ifade içeren bir modül yazmak neden gerekli? İfadeyi doğrudan kodlamak daha verimli değil mi?”dır. Kodun doğrudan yazılması kesinlikle daha verimlidir, ancak genel bir modül için daha iyi bir tasarım değildir. Kullanıcının veri yapısını bilmediği genel bir kod uyguladığımızı unutmayın. Örneğin, neredeyse tüm sistemlerin dosya sonunu test eden bir algoritması vardır. Bu test oldukça basittir, muhtemelen sadece bir satır kod içerir. Ancak gereklidir, çünkü programcı dosya yapısını bilmediği için des tek olmadan dosya sonu durumunu kontrol edemez.

Tam Liste

İlk bakışta, tam liste boş liste kadar basit görünüyor. Ancak, yığınlar a gördüğümüz gibi, uygulanması nispeten karmaşık bir algoritma haline geliyor. Çok az dil, programcıya dinamik bellek içinde ne kadar bellek kaldığını test etme yeteneği sağlar - C bunu sağlamaz. Pseudocode, Algoritma 5-7'de gösterilmiştir.

ALGORİTMA 5-7 Tam Liste

```

Algoritma fullList (liste)
Listenin dolu olup olmadığını belirten Boolean döndürür.
Ön liste, geçerli bir listeye ait meta veri yapısıdır
Yeni düğüm için yer varsa false döndür; bellek doluysa true döndür
1 eğer (bellek dolu)
1 true döndür
2 aksi takdirde
2 false döndür
3 eğer bitir
4 doğruyu döndür
son tamListe

```

Liste Sayısı

Liste sayısı, başka bir basit, tek satırlık modüldür. Çünkü çağırılan modülün liste yapısına doğrudan erişimi yoktur. Uygulaması Algoritma 5-8'de gösterilmiştir.

ALGORİTMA 5-8 Liste Sayısı

```

Algoritma listeSayısı (liste)
Liste içindeki düğüm sayısını temsil eden tam sayı döndürür.
Ön liste, geçerli bir listeye ait meta veri yapısıdır
Liste içindeki düğüm sayısı için sayıyı döndür
1 döndür (liste sayısı)
son listeSayısı

```

Listeyi Gezinme

Bir listeyi gezen algoritmalar, ilk düğümünden başlayarak her düğümünü sırasıyla inceleyerek son düğüm işlenene kadar devam eder. Gezinme mantığı, düğümdeki bir değeri değiştirmek, listeyi yazdırmak, listede bir alanın toplamını almak veya bir alanın ortalamasını hesaplamak gibi birkaç farklı türde algoritma tarafından kullanılır. Tüm listenin işlenmesini gerektiren herhangi bir uygulama, bir gezinme kullanır.

Listeyi gezmek için, her eleman işlendiğinde düğümünden düğüme hareket eden bir yürüyüş işaretçisine ihtiyacımız var. Bir baş yapısına sahip bir liste olduğunu varsayarsak, aşağıdaki sahte kod, listeyi gezmek için bir yürüyüş işaretçisi kullanır. Her döngü, listeyi gezerken işaretçiyi sırayla bir sonraki düğüme hareket ettirecek şekilde değiştirir.

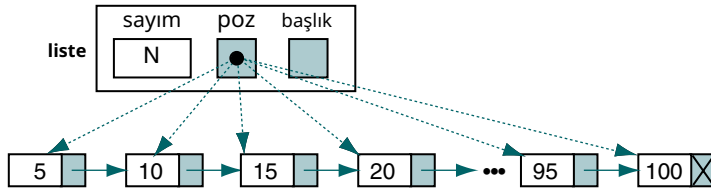
```

pWalker'ı liste başına ayarla
döngü (daha fazla düğüm)
(pWalker verisini) işle pWalker'ı
sonraki bağlantıya ayarla döngüyü
sonlandır

```

Liste içindeki ilk düğüme yürüyüş işaretçisini ayarlayarak başlıyoruz. Sonra, bir döngü kullanarak, tüm veriler işlenene kadar devam ediyoruz. Her bir döngü, bir işlem modülünü çağırır ve ona veriyi iletir, ardından yürüyüş işaretçisini bir sonraki düğüme iletir. Son düğüm işlendiğinde, döngü sona erer.

Listeyi gezme uygulamasını tasarlamak için iki olası yaklaşımımız var. Bir yaklaşımda kullanıcı döngüyü kontrol eder, bir sonraki ögeyi almak için gezme fonksiyonunu çağırır. Diğer yaklaşımda, gezme modülü döngüyü kontrol eder, verileri işlemek için kullanıcı tarafından sağlanan bir algoritmayı çağırır. İlk seçeneği uyguluyoruz çünkü bu, programcıya daha fazla esneklik sağlar. Örneğin, uygulama yalnızca listenin yarısını işlemek istiyorsa, döngü basitçe sona erebilir. İkinci tasarım, verilerin ikinci yarısında gereksiz yere döngü yapar. Şekil 5-14, bir liste gezintisinin grafik temsidir.



ŞEKİL 5-14 Liste Gezintisi

Bir çağrıdan diğerine listede nerede olduğumuzu hatırlamamız gerektiği için, baş yapısına bir mevcut konum işaretçisi eklememiz gerekiyor. Bu, son çağrının ardından işlenen düğümü takip eder. Baş yapısı Şekil 5-14'te gösterilmiştir. Şekil ayrıca, gezinti algoritması çağrıldıkça konum işaretçisinin bir düğümden diğerine nasıl değiştirildiğini göstermektedir.

Her çağrının, listenin başından mı yoksa işlenen son düğümden mi devam ettiğimizi bilmesi gerekir. Bu bilgi, parametre listesi aracılığıyla iletilir. Bir listeyi geçmek için temel mantık, Algoritma 5-9'da gösterilmiştir. Bunu, geçişte bir sonraki düğümü almak için çağrıldığı için getNext olarak adlandırıyoruz.

ALGORİTMA 5-9 Listeyi Geç

Algoritma getNext (liste, nereden, veriDışarı) Bir listeyi geçer. Her çağrı, listede bir ögenin konumunu döndürür.

Ön liste, geçerli bir listeye ait meta veri yapısıdır
nereden, ilk öğeden başlamak için 0'dır
veriDışarı, veri değişkenine referanstır
Son veriDışarı, veri içerir ve doğru döndürülür
-veya- eğer listenin sonundaysak, yanlış döner

devam ediyor

ALGORİTMA 5-9Listeyi Geç (devamı)

```

    Sonraki eleman bulunduğunda true döndür
        listenin sonu ise false döndür
1  eğer (boş liste)
    1 false döndür
2  eğer (fromWhere başlangıçta ise)
    İlkten başla
    1 liste pozisyonunu liste başına ayarla
    2 mevcut liste verilerini dataOut'a taşı
    3 true döndür
3  aksi takdirde
    pos'dan devam et
    1 eğer (liste sonu)
        Liste Sonu
        1 false döndür
    2 aksi takdirde
        1 liste pos'unu bir sonraki düğüme ayarla
        2 mevcut liste verilerini dataOut'a taşı
        3 true döndür
    3 eğer son ise bitir
4  eğer bitir
son getNext

```

Algoritma 5-9 Analizi Bu algoritmada iki ana kod bloğu vardır. 2. ifade, listenin başından başladığımızda işleme sürecini yönetir; 3. ifade, mevcut konumdan devam ettiğimizde işleme sürecini yönetir. Başlangıçta olduğumuzda, baş yapısının konum işaretçisini listenin ilk düğüme ayarlıyoruz ve ilk düğümün verisini çağıran modüle geri iletiyoruz.

Liste geçişine devam ettiğimizde, listede daha fazla veri olduğundan emin olmalıyız. Eğer listenin sonundaysak, başarıyı yanlış olarak ayarlıyoruz ve işlemi sonlandırıyoruz. Daha fazla veri varsa, mevcut konumu bir sonraki düğüme ayarlıyoruz ve verisini çağıran modüle geri iletiyoruz.

Son bir uyarı: Bu tasarım, belirli bir listeyi yalnızca bir kez gezmeyi destekler. Eğer bir uygulama aynı anda birden fazla gezinti gerektiriyorsa, farklı bir tasarım gereklidir.

Listeyi Yok Et

Bir liste artık gerekli olmadığında ancak uygulama tamamlanmadıysa, listeyi edilmelidir. Listeyi yok et hala listede bulunan düğümleri siler ve belleklerini geri dönüştürür. Daha sonra meta verileri null liste durumuna ayarlar. Listeyi yok etme kodu Algoritma 5-10'da gösterilmiştir.

ALGORİTMA 5-10Listeyi Yok Et

```

Algoritma destroyList (pList)
Listede tüm verileri siler.
ffPre liste geçerli bir listeye ait meta veri yapısıdır

```

devam ediyor

ALGORİTMA 5-10Listeyi Yok Et (devam ediyor)

```

ffGönder  Tüm veriler silindi
1 döngüsü (listenin sonunda değil)
  1 liste başını ardıl düğüme ayarla
  2 belleği yığına serbest bırak
2 döngüyü sonlandır
  Listede veri kalmadı. Meta verileri sıfırla.
3 liste konumunu null olarak ayarla
4 liste sayısını 0 olarak ayarla
son destroyList

```

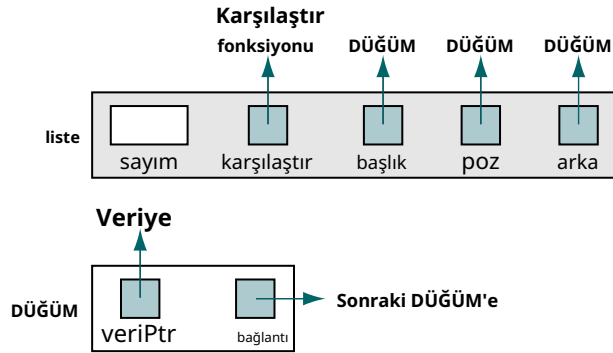
5.3 Liste ADT

Bu bölümde, bağlı liste uygulamasına dayalı bir liste ADT'si oluşturuyoruz. Bir ADT'nin bir veri türü ve verileri manipüle eden işlemlerden oluştuğunu unutmayın. C'de bir ADT'yi uyguladığımızda, uygulama verileri programcı tarafından kontrol edilir. Bu, uygulama programının veriler için bellek ayırmasını ve veri düğümünün adresini ADT'ye geçmesini gerektirir. C, güçlü bir şekilde tiplenmiş olduğu için, veri düğüm işaretçisi bir `void` işaretçisi olarak geçilmelidir. `void` işaretçilerini kullanmak, veri düğüm işaretçilerini ADT'nin veri yapısında verinin detaylarını bilmeden saklamamıza olanak tanır.

Ama, veri düğüm işaretçisini saklayabilmek tüm sorunlarımızı çözmez. Listeler ve birçok diğer veri yapısı, verileri sıralayabilmemizi gerektirir. Bir listede, veriler genellikle anahtar sırasına göre saklanır. ADT'nin bu sıralamayı yapmak için gerekli bilgilere sahip değildir. Ayrıca, her veri türü, iki anahtarı karşılaştırmak için farklı işlevler gerektirir. Bu sorunun çözümü, işlev işaretçileri ile bulunur. Uygulama programcısı, veri yapısındaki iki anahtarı karşılaştırmak için bir işlev yazar, örneğin iki tam sayıyı karşılaştırmak için ve karşılaştırma işlevinin adresini ADT'ye geçirir. ADT, ardından karşılaştırma işlevinin adresini liste başı yapısında meta veri olarak saklar ve verilerin karşılaştırılması gerektiğinde bunu kullanır. Devam etmeden önce, Bölüm 1, "ADT'ler için Genel Kod" ve `void` işaretçileri ile işlev işaretçileri hakkında bir tartışma için C metninizi gözden geçirmeye isteyebilirsiniz.

Liste için, sayım, birkaç işaretçi ve listedeki verileri karşılaştırmak için gereken karşılaştırma fonksiyonunun adresini içeren basit bir baş yapı kullanıyoruz. Baş yapısı Şekil 5-15'te gösterilmiştir.

Veri düğümleri, programcı tarafından oluşturulan bir veri yapısına işaretçi ve listedeki bir sonraki düğüme kendine referans veren bir işaretçi içerir. Bu veri yapıları Program 5-1'de gösterilmiştir. Kullanırken yapı değişkenlerinin her birini açıklayacağız.



ŞEKİL 5-15ADT Yapısı

PROGRAM 5-1 Liste ADT Tür Tanımları

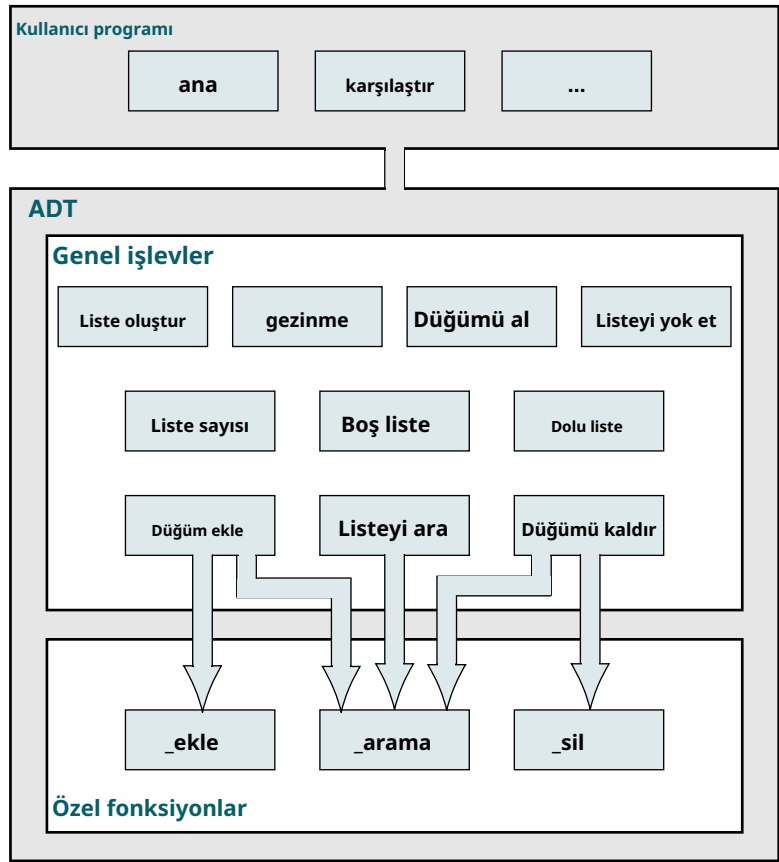
123456789	//Liste ADT Tür Tanımları typedef struct node { void* dataPtr; struct node* link; } NODE;
101112131415	typedef yapısı { int sayımı; DÜĞÜM* konum; DÜĞÜM* kafası; DÜĞÜM* arka; int (*karşılaştır) (void* argüman1, void* argüman2); } LİSTE;

ADT Fonksiyonları

ADT fonksiyonları için prototip bildirimleri, Şekil 5-16'da grafiksel olarak tanımlanmıştır. Bunlar, Bölüm 5.2'de tanımlanan tüm temel algoritmaları içermektedir, "Algoritmalar."

Program 5-2, ADT için prototip ifadelerini içermektedir. Aşağıdaki bölümlerde her bir fonksiyonu tartışacağız. Son üç fonksiyonun alt çizgi ile başlayan tanımlayıcıları olduğunu unutmayın. Bunlar, uygulama kodundaki kullanılmayan iç fonksiyonlardır; yalnızca iç ADT kullanımı içindir. Bununla birlikte, bunları statik fonksiyonlar olarak da tanımladık.⁴ Kod, Program 5-2'de gösterilmiştir.

4. Statik fonksiyonlar bağlayıcıya dışa aktarılmaz. Bunlar, iç fonksiyonların ADT dışında kullanılması sınırlarını engeller.



ŞEKİL 5-16 Liste ADT Fonksiyonları

PROGRAM 5-2 Liste ADT Prototip Bildirimleri

1	//Prototip Bildirimleri
2	LIST* createList (int (*compare)
3	(void* argu1, void* argu2));
4	LIST* destroyList (LIST* list);
5	int addNode (LIST* pList, void* dataInPtr);
6	bool removeNode (LIST* pList,
7	geçersiz* anahtarPtr,
8	void** dataOu
9	tPtr); bool searchList (LIST* pList,
10	void* pArgu,
11	void** pDataOut);

devam ediyor

PROGRAM 5-2 Liste ADT Prototip Bildirimleri (devam ediyor)

1516171819202122232425262728293031323334353637383940

```

bool düğümAl (LIST* pList,
               void* pArgu,
               void** veriDışPtr);

bool gez (LIST* pList,
          int nereden,
          void** veriDışPtr);

int listeSayısı (LIST* pList); bool boşListe (
LIST* pList); bool doluListe (LIST* pList); st
atic int _ekle (LIST* pList,
               NODE
               * pÖnceki,
               void* veriİçinPtr);

static void _sil (LIST* pList,
                 NOD
                 E* pÖnceki,
                 DÜĞÜM* pLoc,
                 void** dataOu
                 tPtr); static bool _search (LIST* pList,
                 NODE** pPre,
                 NODE** pLoc,
                 void* pArgu);

```

//Liste ADT Tanımlamalarının Sonu

Liste Oluştur

Liste oluşturma fonksiyonu, soyut veri türünü boş liste durumunda oluşturur. Tek parametresi, veri yapısındaki iki anahtarı karşılaştırmak için gereken karşılaştırma fonksiyonudur. Baş yapı için bellek ayırır, işaretçileri başlatır (ve liste sayısını, daha sonra kullanılmak üzere karşılaştırma fonksiyonunun adresini saklar) ve yapının adresini çağıran fonksiyona döndürür. Bellek ayırma işlemi başarısız olursa, null işaretçi döndürür. Kod Program 5-3'te gösterilmiştir.

PROGRAM 5-3 Liste Oluştur

1234567

```

/*===== createList =====
Liste başı için dinamik bellek ayırır ve adresini çağır
ana döndürür. Önce karşılaştırma, iki düğümü
karşılaştırmak için kullanılan karşılaştırma fonksiyonunu
n adresidir.
Sonra baş, ayrılmış veya hata döndürülmüştür
Baş düğüm işaretçisi veya taşma durumunda null döner

```

devam ediyor

PROGRAM 5-3 Liste Oluştur (devamı)

```

89  */
10112131415161718192021222324252627  LİSTE* liste oluştur
                                         (int (*karşılaştır) (void* argüman1, void* argüman2))
{
  //Yerel Tanımlar
  LİSTE* listesi;

  //İfadeler
  liste = (LİSTE*) malloc (sizeof (LİSTE));
  if (liste)
  {
    liste->kafa = NULL;
    liste->konum = NULL;
    liste->arka = NULL;
    liste->sayı = 0;
    liste->karşılaştır = karşılaştır;
  } // if

  dönüş listesi;
} // createList

```

Program 5-3 Analizi Karşılaştırma fonksiyonu işaretçisi hariç, bu algoritma temel ve doğrudan-veya. Liste oluşturmak için geçirilen karşılaştırma fonksiyonu işaretçisi, basitçe fonksiyonun adresidir. Ancak Program 5-1'deki veri yapısını incelediğinizde, karşılaştırma fonksiyonunun iki *void* işaretçisini parametre olarak alacak şekilde tamamen bildirildiğini ve bir tamsayı döndüreceğini not edin. Bu türlerden herhangi biri yanlış bildirilirse, derleyici bir hata mesajı döndürür. Bu, programcının karşılaştırma fonksiyonunu gereksinimlerimize uygun şekilde doğru yazdığını garanti eder.

Düğüm Ekle

Düğüm ekle, aslında listeye eklenmesi gereken veriyi alan ve ekleme noktasını bulmak için listeyi arayan daha yüksek seviyeli bir kullanıcı arayüzü fonksiyonudur. Daha sonra, "Liste Düğümü Ekle" algoritması 5-2 ile paralel olan düşük seviyeli silme fonksiyonunu çağırır.

Bu ADT'de, listeye tekrar eden anahtarların eklenmesini önlemeyi seçtik. Bu, üç olası dönüş değerine yol açar: -1 dinamik bellek taşma sınırı, 0 başarıyı ve +1 tekrar eden bir anahtar gösterir. Bu dönüş değerlerini doğru bir şekilde yorumlamak, programcıya aittir. Bir uygulama tekrar eden anahtar eklemeleri gerektiriyorsa, ekleme ve diğer birkaç algoritmanın tekrar eden verileri işlemek için yeniden yazılması gerekecektir. Kod Program 5-4'te gösterilmiştir.

İçsel Ekleme Fonksiyonu

İçsel ekleme fonksiyonu (*_insert*), yeni veriyi listeye fiziksel olarak yerleştirme kten sorumludur. Kodunun Program 5-5'te gösterilmiştir.

PROGRAM 5-4Düğüm Ekle

123456789	/*===== addNode ===== Listeye veri ekler. Ön pList geçerli listeye işaretçidir dataInPtr ekleme verisine işaretçidir Son veri eklendi veya hata Dönüş -1 taşma durumunda 0 başarılıysa 1 eğer a nahtar tekrar ediyorsa
101112131415161718192021222324252627282930	*/ int addNode (LIST* pList, void* dataInPtr) { //Yerel Tanımlar bool bulundu; bool başarılı; NODE* pÖn; NODE* pYer; //İfadeler bulundu = _search (pList, &pÖn, &pYer, dataInPtr); if (bulundu) // Tekrar eden anahtarlar izin verilmez return (+1); başarı = _insert (pList, pPre, dataInPtr); if (!success) // Taşma return (-1); return (0); } // addNode

Program 5-4 AnaliziDüğüm ekleme, yapıyı arayarak ekleme pozisyonunu bulmakla başlar; bu, arama tarafında n döndürülen öncül (pPre) ve mevcut konum (pLoc) işaretçileri ile belirlenir. Yeni verinin anahtarı listede bir düğümle eşleşirse, ekleme reddedilir ve +1 döndürülür. Yeni verinin benzersiz bir anahtar içerdiği varsayılarak, düğüm eklenir.

PROGRAM 5-5Dahili Ekleme Fonksiyonu

12345678	/*===== _insert ===== Veri işaretçisini yeni bir düğüme ekler. Önce pList geçerli bir listeye işaretçi pPre verinin öncüsüne işaretçi dataInPtr eklenecek veri işaretçisi Sonra veriler sıralı olarak eklenmiştir Dönüş boolean, başarılıysa true, false eğer bellek taşması varsa
----------	---

devam ediyor

PROGRAM 5-5 Dahili Ekleme Fonksiyonu (devamı)

```

9  */
101 statik bool _insert (LIST* pList, NODE* pPre,
112                                     void* dataInPtr)
123 {
134 //Yerel Tanımlar
145     NODE* pNew;
156
167 //İfadeler
178     eğer (! (pNew = (NODE*) malloc(sizeof(NODE))))
189         false döndür;
190
191     pNew->dataPtr = dataInPtr;
192     pYeni->bağlantı = NULL;
193
194     eğer (pPre == NULL)
195     {
196         // İlk düğümden önce veya boş listeye ekleme.
197         Yeni->bağlantı = pList->başlık; \ve pList-
198         >başlık = Yeni;
199         eğer (pList->count == 0)
200             // Boş listeye ekleme. Arka ayarla
201             pList->rear = pNew;
202     } // eğer pPre ak
203
204     si takdirde
205     {
206         // Ortaya veya sona ekleme
207         pNew->link = pPre->link;
208         pPre->link = pNew;
209
210         // Şimdi listenin sonuna ekleme kontrolü
211         if (pNew->link == NULL)
212             pList->rear = pNew;
213     } // eğer değil
214
215     (pList->count)++;
216     return true;
217 } // _insert

```

Düğümü Kaldır

Node'u kaldırmak, aynı zamanda yüksek seviyeli, kullanıcı arayüzü işlevidir. Silme işlemini tamamlamak için arama listesini çağırır ve düğümü siler. Node'u kaldırmada iki olası tamamlama durumu vardır: ya başarılı olduk (doğru) ya da silinmesi gereken veriler bulunamadığı için başarısız olduk (yanlış). Kod Program 5-6'da gösterilmiştir.

PROGRAM 5-6Düğümü Kaldır

123456789	<pre> /*===== removeNode ===== Listeden verileri kaldırır. Ön pList geçerli bir listeye işaretçi keyPtr silinecek anahtara işaretçi dataOutPtr veri işaretçisine işaretçi Son Düğüm silindi veya hata döndürüldü. Dön false bulunamadı; true silindi */ bool removeNode f(LIST* pList, void* keyPtr, fvoid** dataOutPtr) { //Yerel Tanımlar bool bulundu; NODE* pÖn; NODE* pYer; //İfadeler bulunan = _search (pList, &pPre, &pLoc, keyPtr); if (bulundu) _delete (pList, pPre, pLoc, dataOutPtr); return bulundu; }// kaldırDüğümü </pre>
101112131415161718192021222324	

Program 5-6 AnaliziDüğümü kaldırmak oldukça kolay görünse de, `dataOutPtr` çağıran fonksiyonda `void` işaretçisine dönüştürülmelidir yoksa bir derleme hatası alırsınız. Tipik bir çağrı burada gösterilmiştir:

```
removeNode (pList, &partNum, (void*)&partPtr);
```

İçsel Silme Fonksiyonu

İçsel silme fonksiyonu (`_delete`) düğümü kaldırma tarafından çağrılır ve tanımlanan düğümü dinamik bellekten fiziksel olarak siler. Veriler silindiğin de, verilerin bir işaretçisi çağıran fonksiyona döndürülür ve çağrıda son parametreyle belirtilen değişken konumuna (`dataOutPtr`) yerleştirilir. Kod Program 5-7'de gösterilmiştir.

PROGRAM 5-7Dahili Silme Fonksiyonu

1234567	<pre> /*===== _sil ===== Bir listeden verileri siler ve veri için çağıran modüle işaretçi döner. Önce pList geçerli listeye işaretçi. pPre öncü düğüme işaretçi pLoc hedef düğüme işaretçi dataOutPtr veri işaretçisine işaretçi </pre>
---------	--

devam ediyor

PROGRAM 5-7 Dahili Silme Fonksiyonu (devamı)

89	Post Veriler silindi ve geri döndürüldü Veri belleği serbest bırakıldı
10111213141516171819202122232425262728293031	<pre> */ void _delete (LIST* pList, NODE* pPre, NODE* pLoc, void** dataOutPtr) { //İfadeler *dataOutPtr = pLoc->dataPtr; if (pPre == NULL) // İlk düğümü silme pList->head = pLoc->link; else // Diğer herhangi bir düğümü silme pPre->link = pLoc->link; // Son düğümü silmek için test if (pLoc->link == NULL) pList->arka = pPre; (pList->count)--; free (pLoc); dönüş; } // _delete </pre>

Arama Listesi

Arama listesi , verilen bir düğümü listede bulup adresini ve öncüsünün adresini çağırarak fonksiyona geri ileten yüksek seviyeli, kullanıcı arayüzü fonksiyonudur. Bir uygulama arayüzü olduğu için yalnızca üç parametreye ihtiyaç duyar: aranacak liste, arama argümanı ve veri işaretçisini alacak işaretçinin adresi. Fonksiyonunu yerine getirmek için, bir iç ADT arama fonksiyonunu çağırır. Arama tamamlandığında, çıktı parametresini bulunan verinin adresine ayarlar ve bulunan Boolean değerini döner. Kod Program 5-8'de gösterilmiştir.

PROGRAM 5-8 Kullanıcı Arayüzü Arama

12345678	<pre> /*===== searchList ===== Arama fonksiyonu için arayüz. Ön pList, başlatılmış listeye işaretçi. pArgu, aranan anahtara işaretçi Son pDataOut, bulunan veriye işaretçi içerir -veya- Bulunamadıysa NULL Boolean true başarılı; false bulunamadı */ </pre>
----------	--

devam ediyor

PROGRAM 5-8Kullanıcı Arayüzü Arama (devam ediyor)

```

9      bool aramaListesi (LIST* pList, void* pArgu,
10111213141516171819202122232425
      void** pDataOut)
      {
        //Yerel Tanımlar
        bool bulundu;

        NODE* pÖn;
        NODE* pYer;

        //İfadeler
        bulundu = _search (pList, &pPre, &pLoc, pArgu);
        if (bulundu)
            *pDataOut = pLoc->dataPtr;
        else
            *pDataOut = NULL;
        return bulundu;
      } // aramaListesi

```

İçsel Arama Fonksiyonu

Gerçek arama işlemi, yalnızca ADT içinde mevcut olan bir içsel arama fonksiyonu (`_search`) ile yapılır. Arama argümanının bir düğümdeki anahtara eşit olup olmadığını belirlemek için, kullanan programcı tarafından oluşturulan karşılaştırma fonksiyonunu kullanır. Arama mantığı Program 5-9'da gösterilmiştir.

PROGRAM 5-9Dahili Arama Fonksiyonu

```

123456789      /*===== _search =====
      Listeyi arar ve hedefi içeren düğümün adresini geri döner ve
      mantıksal öncülünü.
      Önce   pList, başlatılmış listeye işaretçi
            pPre, öncül için işaretçi değişken
            pLoc, düğümü almak için işaretçi değişken
            pArgu, aranan anahtara işaretçi
      Sonra  pLoc, ilk eşit/büyük anahtara işaret eder
            -veya- hedef > son düğümün anahtarı ise null
            pPre, en büyük düğüme işaret eder < key
            -veya- hedef < ilk düğümün anahtarı için null
      Boolean true bulundu; false bulunamadı döndür

      */
      bool _search (LIST* pList, NODE** pPre,
1011121314151617181920
                     NODE** pLoc, void* pArgu)
      {
        //Makro Tanımı
        #define KARŞILAŞTIRMA \
          ( ((*pList->karşılaştırm) (pArgu, (*pLoc->dataPtr)) )

```

devam ediyor

PROGRAM 5-9 Dahili Arama Fonksiyonu (devamı)

```

21222324252627282930313233343536373839404142434445464748495051525354
#define KARŞILAŞTIRMA_SON \
    ((* pList->karşılaştır) (pArgu, pList->rear->dataPtr))

//Yerel Tanımlar
int sonuç;

//İfadeler
*pPre = NULL;
*pLoc = pList->head;
if (pList->count == 0)
    return false;

// Argümanın > listedeki son düğüm için test
if (KARŞILAŞTIRMA_SON > 0)
{
    *pPre = pList->arka;
    *pLoc = NULL;
    return false;
} // eğer

while ( (sonuç = KARŞILAŞTIR) > 0 )
{
    // Arama argüman konumu bulunamadı
    *pPre = *pLoc;
    *pLoc = (*pLoc)->bağlantı;
} // while

if (result == 0)
    // argüman bulundu--başarı
    return true;
    aksi takdirde
    false döndür;
} // _search

```

Program 5-9 Analizi Bu fonksiyonun birkaç yönünü dikkatlice incelemeniz gerekiyor. Öncelikle, fonksiyonun başında tanımladığımız iki makroyu inceleyin. Karşılaştırma kodu oldukça uzun ve kodun mantık a kısmını karıştırma eğilimindedir. Bir makro olarak kodlandığında, fonksiyon mantığı çok da ha kolay takip edilebilir. Listede düğümlere iki farklı işaretçi kullandığımız için iki farklı ma kroya ihtiyacımız var.

Şimdi karşılaştırmayı yakından inceleyelim (burada tekrar gösterilmiştir):

```
(*pList->karşılaştır) (pArgu, (*pLoc)->dataPtr)
```

Karşılaştırmadaki ilk ifade, uygulama programı tarafından yazılan ve oluşturulduğu nda ADT başlığında saklanan karşılaştırma fonksiyonunun adresidir. Bu, fonksiyon a dının eşdeğeridir. Adresi liste başlığı aracılığıyla alıyoruz, pList-> >karşılaştır. İkinci parantez içindeki ifade, fonksiyon için parametre listesidir.

Bu durumda, arama argümanına (**pArgu**) ve veri düğümüne (**dataPtr**) bir *void* işaretçisi içerir. Karşılaştırma fonksiyonu, düğüm için veri yapısını bildiğinden, işaretçileri uygun şekilde dönüştürebilir. Karşılaştırma fonksiyonu, bir arama argümanı liste anahtarından daha küçük olduğunda **-1**, eşit olduğunda **0** veya daha büyük olduğunda **+1** döner. Bu, C'nin dize karşılaştırmaları için kullandığı aynı tasarımdır, bu yüzden buna zaten aşinasınız.

İlk fonksiyon ifadesinde, arama argümanının Program 5-1'de arka olarak tanımlanan listedeki son anahtardan büyük olup olmadığını belirlemek için karşılaştırmayı kullanıyoruz. Eğer büyükse, listeyi aramaya gerek yoktur. Arama argümanı son girişten küçük veya eşitse, *while* döngüsündeki karşılaştırma, arama argümanı liste argümanından büyük olana kadar listenin başından arama yapar. O noktada ya eşleşen bir anahtar bulmuşuzdur ya da arama argümanından büyük en küçük anahtarı bulmuşuzdur.

Düğümü Al

Retrieve node, listeyi arar ve anahtarı arama argümanı ile eşleşen verinin adresini döndürür. Kodunun Program 5-10'da gösterilmiştir.

PROGRAM 5-10 Düğümü Alma

123456789	<pre> /*===== retrieveNode ===== Bu algoritma, liste içeriğini değiştirmeden listede verileri alır. Ön pList, başlatılmış listeye işaretçi. pArgu, alınacak anahtara işaretçi Son Veri (işaretçi) çağırana geri döner Dönüş boolean true başarı; false taşıma */ static bool getNode (LIST* pList, fffvoid* pArgu, f void** dataOutPtr) { //Yerel Tanımlar bool bulundu; NODE* pÖn; NODE* pYer; //İfadeler bulundu = _search (pList, &pPre, &pLoc, pArgu); if (bulundu) { *dataOutPtr = pLoc->dataPtr; return true; } // eğer *dataOutPtr = NULL; return false; } // geri almaDüğümü </pre>
1011121314151617181920212223242526272829	

Boş Liste

Uygulama programcısının liste yapısına erişimi olmadığı için, liste durumunu belirlemek için kullanılabilen üç durum fonksiyonu sağlıyoruz. İlk olarak, boş liste, Program 5-11'de gösterilmiştir.

PROGRAM 5-11 Boş Liste

123456789	/*===== emptyList ===== Boş olup olmadığını belirten boolean döndürür
	Ön pList geçerli bir listeye işaret eden bir işaretçidir Boolean true döndür; false liste veri içeriyor
	*/
	bool emptyList (LIST* pList)
	{
	//İfadeler
1011	return (pList->count == 0);
	}// emptyList

Tam Liste

İkinci durum fonksiyonu, dolu liste, dinamik bellek içinde başka bir düğüm için yeterli alan olup olmadığını belirler. Eklemeden önce mevcut alan olup olmadığını bilmesi gereken uygulamalar için mevcuttur. Kod, Program 5-12'de gösterilmiştir.

PROGRAM 5-12 Dolu Liste

123456789	/*===== fullList ===== Daha fazla veri için yer olmadığını belirten boolean döndürür. Bu liste, başka bir düğüm için bellek tahsis edilemediğinde doludur.
	Ön pList geçerli listeye işaretçi Dönüş boolean true eğer dolu false eğer düğüm için yer var
	*/
	bool fullList (LIST* pList)
	{
	//Yerel Tanımlar
	NODE* temp;
	//İfadeler
	if ((temp = (NODE*)malloc(sizeof(*(pList->head))))
	{
	free (temp);
	return false;
1011121314151617181920	} // eğer

devam ediyor

PROGRAM 5-12 Tam Liste (devam ediyor)

21222324	<pre>// Dinamik bellek dolu true döndür; } // tam Liste</pre>
----------	--

Program 5-12 Analizi Bu, yazılması daha zor olan işlevlerden biridir. C, dinamik bellek içinde boş alanı belirlemek için bir olanak sağlamadığından, bir düğüm ayırabiliriz; eğer çalışırsa, bir düğüm için en az bir yer olduğunu biliriz. Ancak, ayırma işlemi son mevcut alanı kullanabileceğinden, geri dönmeyen önce düğümü serbest bırakmalıyız. Elbette, eğer ayırma başarısız olursa, dinamik bellek içinde daha fazla yer olmadığını biliriz.

Liste Sayısı

Liste sayısı, durum işlevlerinin sonuncusudur. Şu anda listede bulunan düğüm sayısı için bir tamsayı döndürür. Kodu Program 5-13'te gösterilmiştir.

PROGRAM 5-13 Liste Sayısı

123456789	<pre>/*===== listCount ===== Liste içindeki düğüm sayısını döndürür. Pre pList geçerli bir listeye işaret eden bir işaretçidir. Li ste içindeki düğüm sayısı için sayıyı döndürür */ int listCount(LIST* pList) { //Açıklamalar return pList->count; } // listCount</pre>
101112	

Geçiş

Bir zamanlar, her liste geçilmelidir. Programcı, liste yapısına erişim sahibi olmadığından, listeyi geçmek için bir işlev sağlamamız gerekiyor. Bu algoritma Program 5-14'te gösterilmiştir.

PROGRAM 5-14 Listeyi Geç

12345678	<pre>/*===== gezinme ===== Bir listeyi gezer. Her çağrı ya listenin başından başlar ya da listedeki bir sonraki öğenin konumunu döner. Önce pList geçerli bir listeye işaretçi fromWhere ilk öğeden başlamak için 0 dataPtrOut veriye işaretçi adresi Sonra daha fazla veri varsa, bir sonraki düğümün adresi</pre>
----------	---

devam ediyor

PROGRAM 5-14Listeyi Gezinme (devamı)

```

9      Doğru düğüm bulundu; liste sonu ise yanlış döner
10     */
11     bool traverse (LIST* pList,
12                     int fromWhere,
13                     geçersiz** veriPtrOut)
14     {
15         //İfadeler
16         eğer (pList->count == 0)
17             yanlış döner;
18
19         eğer (fromWhere == 0)
20         {
21             // İlk düğümden başla
22             pList->pos = pList->head;
23             *dataPtrOut = pList->pos->dataPtr;
24             doğru döner;
25         } // fromwhere ise ak
26
27         si takdirde
28         {
29             // Mevcut konumdan başla
30             eğer (pList->pos->link == NULL)
31                 yanlış döner;
32
33             aksi takdirde
34             {
35                 pList->pos = pList->pos->link;
36                 *dataPtrOut = pList->pos->dataPtr;
37                 return true;
38             } // eğer değilse
39         } // nereden eğer değilse
40     } // gez

```

Program 5-14 AnaliziGezinme fonksiyonu, gezinmenin sadece başladığını veya bir gezinmenin ortasında olup olmadığını bilmesi gerekir. Bu mantık, **fromWhere** bayrağı aracılığıyla kontrol edilir, C'nin seek fonksiyonundaki konum bayrağına benzer bir tekniktir. Fonksiyon her çağrıldığında, döndürülen mevcut düğümün adresini baş düğümdeki konum işaretçisine kaydeder. Sonraki seferde, eğer listenin başından başlamıyorsa, bir sonraki düğümü bulmak için konum işaretçisini kullanabiliriz. Eğer bir düğüm mevcutsa, true döneriz; eğer listenin sonundaysa, false döneriz.

Listeyi Yok Et

Listeyi yok etme, yalnızca bir listenin silinmesi gerektiği durumlar için gereklidir, böylece yeniden inşa edilebilir. Program tamamlandığında, işletim sistemine geri dönmenden önce listeyi yok etmek gerekli değildir. Çağrıldığında, listeyi yok etme, yalnızca ADT'deki düğümleri değil, aynı zamanda uygulama programcısı tarafından tahsis edilen tüm düğümleri de siler. En sonunda liste baş düğümün siler ve null işaretçi döner. Kod Program 5-15'te gösterilmiştir.

PROGRAM 5-15Listeyi Yok Et

```

123456789 /*===== destroyList =====
Liste içindeki tüm verileri siler ve belleği geri dönüştürür. Ö
n Liste geçerli bir listeyi işaret eden bir işaretçidir.
Son Tüm veriler ve baş yapısı silindi
Null baş işaretçisi döndür
*/
LIST* destroyList (LIST* pList)
{
//Yerel Tanımlar
NODE* deletePtr;

//İfadeler
eğer (pList)
{
while (pList->count > 0) {

f// Önce veriyi sil
free (pList->head->dataPtr);

// Şimdi düğümü sil
deletePtr = pList->başlık;
pList->başlık = pList->başlık->bağlantı;
pList->count--;
ücretsiz (deletePtr);
} // while
serbest (pList);
} // eğer
NULL döndür;
} // destroyList
1011121314151617181920212223242526272829

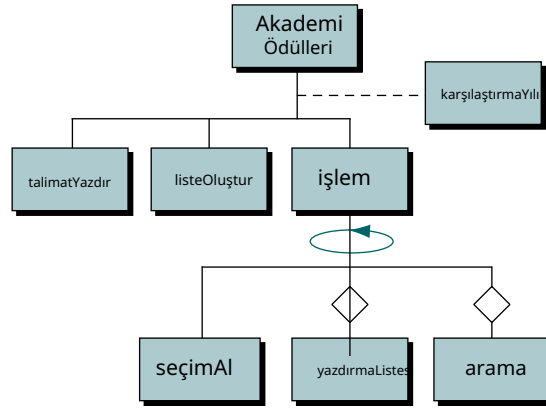
```

5.4 Uygulama

Bir soyutdata türüne sahip olduğumuzda bir listeyi ne kadar kolay uygulayabileceğimizi göstermek için, ödül kazanmış resimlerin ve onların yönetmenlerinin bir listesini uyguluyoruz. Programın üç ana işlevi vardır: talimatları yazdırmak, listeyi oluşturmak ve kullanıcı sorgularını işlemek. Ayrıca, işlem işlevi üç işlev çağırır: kullanıcının seçimini almak, tüm listeyi yazdırmak ve istenen yılı aramak. Bir diğer gerekli işlev, iki yılı karşılaştırmak için bir karşılaştırma işlevidir (anahtarlar). Tam tasarım Şekil 5-17'de gösterilmiştir.

Veri Yapısı

Uygulama veri yapısı üç alan içerir: filmin yapıldığı yıl, filmin adı ve yönetmenin adı. Program 5-16'da gösterilmiştir.



ŞEKİL 5-17 Akademi Ödülleri Listesi Tasarımı

PROGRAM 5-16 Akademi Ödülleri için Veri Yapısı

```

1  /*Akademi Ödülleri için Veri Yapısı
2  Yazarı:
3  Tarih:
4  */
5
6  const kısa STR_MAX = 41;
7
8  typedef struct
9  {
10     short yıl;
11     char  resim [STR_MAX];
12     char  yönetmen[STR_MAX];
13     } RESİM;
  
```

Uygulama Fonksiyonları

Bu bölümde fonksiyonları ve soyut veri tipi ile olan arayüzlerini tanımıyoruz.

Ana Hat

Ana Hat, liste değişkenini tanımlar ve işi yapan üç fonksiyonu çağırır. Soyut veri tipi ile bir arayüzü yoktur. Kodu Program 5-17'de gösterilmiştir.

Yazdırma Talimatları

Yazdırma talimatları, programın nasıl çalıştığını açıklayan basit bir işlevdir. Kodu Program 5-18'de gösterilmiştir.

PROGRAM 5-17Akademi Ödülleri için Ana Hat

```

1  /*Bu program, Akademi Ödülleri Sinema Filmleri'nin bir liste
2     sini tutar ve görüntüler.
3     Yazarı:
4     Tarih:
5  */
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <cType.h>
9  #include <stdbool.h>
10 #include "P5-16.h"           // Veri Yapısı
11 #include "linkListADT.h"
12
13 //Prototip Bildirimleri
14 void printInstr (void);
15 LIST* buildList (void);
16 void process (LIST* list);
17 char getChoice (void);
18 void printList (LIST* list);
19 void arama (LIST* list);
20
21 int cmpYear (void* pYear1, void* pYear2);
22
23 int main (void)
24 {
25     // Yerel Tanımlar
26     LIST* list;
27
28     // İfadeler
29     printInstr ();
30     list = buildList ();
31     process (list);
32
33     printf("En İyi Filmler Sonu"      "Uma
34           rım favorinizi bulmuşsunuzdur!");return 0;
35     return 0;
36 }// ana

```

PROGRAM 5-18Kullanıcı için Yazdırma Talimatları

```

1  /*===== printInstr =====
2     Kullanıcıya talimatları yazdır.
3     Önce hiçbir şey
4     Sonra talimatlar yazdırıldı
5  */
6  void printInstr (geçersiz)
7  {
8  //İfadeler
9     printf("Bu program Akademi Ödüllerini yazdırır "

```

devam ediyor

PROGRAM 5-18 Kullanıcı için Yazdırma Talimatları (devam ediyor)

1011121314	"Yılın En İyi Filmi ve yönetmeni." "Göreviniz yılı girmek; geri kalanını biz yapacağız." "Keyfini çıkarın."); geri dönmek; } // printInstr
------------	--

Liste Oluştur

Liste oluşturma, programdaki iki ana işleme modülünden biridir. Liste verileri bir ir metin dosyasında bulunmaktadır. Bu modül metin dosyasını okur ve verileri listeye ekler. Program 5-19, liste oluşturma için kodu içerir; Şekil 5-18, ikinci ekleden sonraki durumunu diyagramlar. Şekli incelerken, işaretçilere dikkat edin. Öncelikle, programdaki liste işaretçisinin ADT'deki liste başlık yapısını nasıl referans aldığını not edin. Ardından, başlık yapısının listedeki düğümleri nasıl referans aldığını not edin. Ayrıca, her düğümün farklı bir resim yapısını referans aldığını not edin. Son olarak, ADT'nin hem düğümleri hem de resimleri referans alabileceğini, ancak uygulamanın yalnızca resimleri referans alabileceğini not edin.

PROGRAM 5-19 Liste Oluştur

123456789	<pre> /*===== buildList ===== Bir metin veri dosyasını okur ve listeyi yükler Ön dosya şu formatta mevcut: yy 'film' 'yönetmen' Son liste verileri içerir -veya- program sorunlar varsa iptal edilir */ LIST* buildList (geçersiz) { //Yerel Tanımlar FILE* fpData; LIST* listesi; kisa yılIn; int addResult; PICTURE* pPic; //İfadeler liste = createList (cmpYear); if (!list) printf("\aListe oluşturulamıyor"), exit (100); fpData = fopen("pictures.dat", "r"); if (!fpData) printf("\aGiriş dosyası açılırken hata"), exit (1 10); </pre>
1011121314151617181920212223242526	

devam ediyor

PROGRAM 5-19 Liste Oluştur (devam ediyor)

```

27282930313233343536373839404142434445464748495051525354555657585960
while (fscanf(fpData, " %hd", &yılIn) == 1)
{
    pPic = (RESİM*) malloc(sizeof(RESİM));
    if (!(pPic))
        printf("\aBellek Dışı Hata, liste oluşturulurken"),
            exit (100);

    pPic->year = yearIn; // Sekmeleri ve alıntıyı a

    tla
    while ((fgetc(fpData)) != "\n")
        ;
    while ((fgetc(fpData)) != "")
        ;

    fscanf(fpData, " %40[^\n]", %*c", pPic->resim);
    while ((fgetc(fpData)) != "\n")
        ;
    while ((fgetc(fpData)) != "")
        ;

    fscanf(fpData, " %40[^\n]", %*c", pPic->director);

    // Listeye ekle
    addResult = addNode (list, pPic);
    if (addResult != 0)
        if (addResult == -1)
            printf("Bellek taşması, film ekleniyor\a"),
                t (120);
        başka
            printf("Yıl %hd zaten eklenmedi\a",
                    ear);
        while (fgetc(fpData) != "\n")
            ;
    } // while
    dönüş listesi;
} // buildList

```

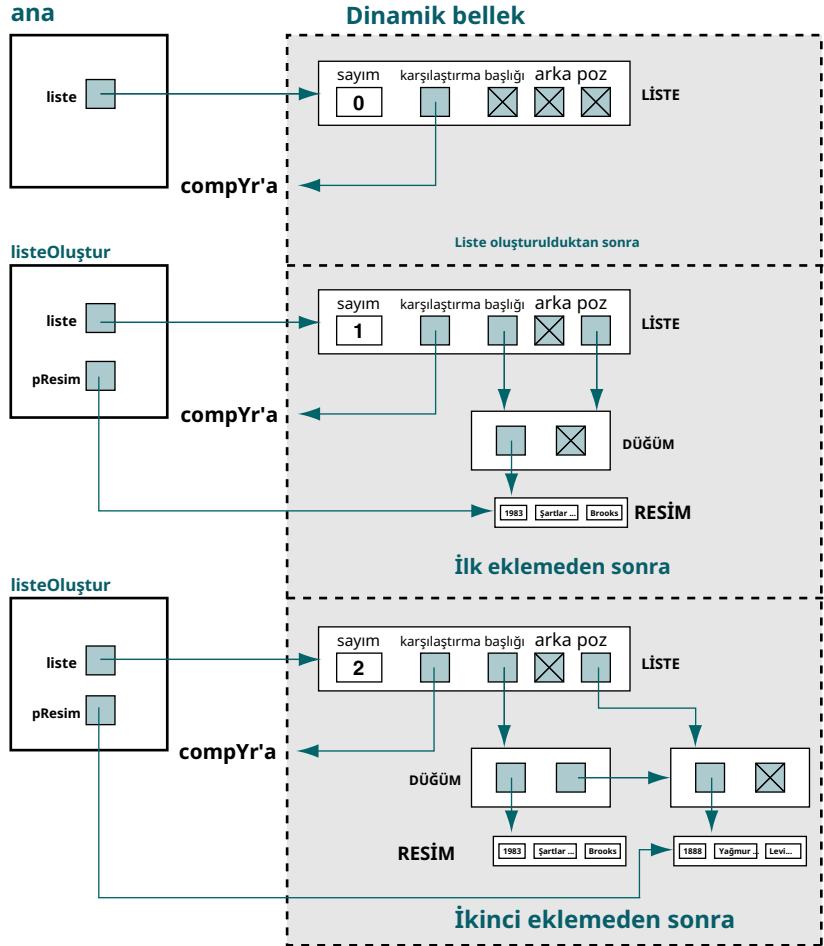
Program 5-19 Analizi Program 5-19, soyut veri türü ile arayüz oluşturan ilk modülü içerir. Bu bazı ilginç tasarım yönlerine sahiptir.

Listeyi oluşturduğumuzda, iki yılı karşılaştıran fonksiyonun adresini geçiriyoruz. U nutmayın, ADT veri türlerine erişim sağlamaz, bu nedenle verileri işleyen herhangi bir fonksiyon için bir fonksiyon yazmalı ve geçirmeliyiz.

Resim ve yönetmen verileri gömülü boşluklar içerdiğinden, dosyayı yılı kısa bir tamsayı olarak veresim başlığını ve yönetmeni sınırlı dizgiler olarak biçimlendiriyoruz. Resim başlığı ve yönetmen için sınırlayıcı çift tırnak işaretidir. Bu biçimlendirme, *scanf* düzen seti format operatörünü kullanmamıza olanak tanır.

Döngü, dosyadan yılı okuyarak kontrol edilir. Eğer yıl yoksa, dosyanın sonundayız. Eğer bir yıl varsa, sırayla resmi ve yönetmeni okumaya devam ederiz.

Verileri resim değişken yapısına okuduktan sonra, listeye ekliyoruz. Node eklemeden üç olası sonuç vardır. Eğer ekleme başarılı olursa, node ekle 0 döner. Bu nedenle, node ekle çağrıldıktan sonra sonuçları kontrol ederiz. Eğer dönen değer 0 değilse, bellek taşmasını kontrol ederiz; bu durumda programdan çıkarız; eğer bir kopya ekleme varsa, sadece bir hata mesajı yazdırır ve devam ederiz.



ŞEKİL 5-18 İkinci Eklemeden Sonra Liste Durumu

Kullanıcı Taleplerini İşle

Liste oluşturulduktan sonra, kullanıcı taleplerini işlemeye hazırız. Kullanıcının üç seçeneği vardır; bunlar, get choice tarafından gösterilen bir menüde yer almaktadır : tüm listeyi yazdırmak, belirli bir yılı aramak veya programdan çıkmak. Listeyi işleme kodu Program 5-20'de gösterilmiştir.

PROGRAM 5-20 Kullanıcı Seçimlerini İşle

1	/*===== işlem =====
2	Kullanıcı seçimlerini işleme
3	Ön liste oluşturulduSonra kullanıcı
4	n tüm seçimleri gerçekleştirildi
5	*/
6	void işlem (LIST* liste)
7	{
8	//Yerel Tanımlar
9	char seçim;
10	
11	//İfadeler
12	do
13	{
14	seçim = getChoice ();
15	
16	switch (seçim)
17	{
18	case 'P': printList (liste);
19	
20	case 'S': search (liste);
21	break;
22	case 'Q': break;
23	} // anahtar
24	} while (seçim != 'Q');
25	return;
26	}// işlem

Program 5-20 AnaliziBu işlevde soyut veri türüne hiçbir çağrı yoktur. İşlev, kullanıcının seçimini almak için bir uygulama işlevini çağırır ve ardından uygun olarak listeyi yazdırır veya arama yapar.

Kullanıcı Seçimini Al

Kullanıcı seçimini almak, yazdığınız birçok işlevle benzer basit bir işlevdir. Bu, kullanıcının seçimini okur ve geçerli olduğunu garanti eder. Seçim geçerli değilse, işlev doğru bir seçim girene kadar döngüye girer. Kod, Program 5-21'de gösterilmiştir.

PROGRAM 5-21 Kullanıcının Seçimini Al

1	/*===== getChoice =====
2	Seçenekler menüsünü yazdırır.
3	Önce hiçbir şey
4	Post menü yazdırıldı ve seçim döndürüldü
5	*/
6	char getChoice (geçersiz)
7	{

devam ediyor

PROGRAM 5-21 Kullanıcının Seçimini Al (devam ediyor)

```

89 //Yerel Tanımlar
    char seçim;
    bool geçerli;

101 //İfadeler
102 printf("===== MENÜ ===== " "Seç
    enekleriniz burada:" " S: Bir yılı
    ara" " P: Tüm yılları yazdır"
    " Q: Çıkış " "Seçiminizi
    girin: ");

    yap
    {
        scanf("%c", &seçim);
        seçim = toupper(seçim);
        switch (seçim)
        {
            durum 'S':
            'P' durumu:
            case 'Q': geçerli = doğru;
                break;
            default: valid = false; prin
                tf("\aGeçersiz seçim" "Lütf
                    en tekrar deneyin: "); b
                reak;
        } // anahtar
    } while (!valid);
    dönüş seçimi;
} // getChoice

```

Listeyi Yazdır

Listeyi yazdırma, listeyi dolaşarak her çağrıda bir filmi yazdırırtipik tr averse fonksiyonuna. Öncelikle, listede veri olup olmadığını kontrol e derizyeterli veri yoksa uygun bir mesaj yazdırırız. Liste verileri içerdiği ni doğruladıktan sonra,ilk filmi okumak için get next çağrılır. Ardında n, kalan verileri işlemek için bir *do...while* döngüsü kullanırız. KodProg ram 5-22'de gösterilmiştir.

PROGRAM 5-22Listeyi Yazdır

```

123 /*===== printList =====
345 Tüm listeyi yazdırır
456 Önce liste oluşturuldu
    Sonra liste yazdırıldı
*/
geçersiz yazdırmaListesi (LIST* listesi)

```

devam ediyor

PROGRAM 5-22Listeyi Yazdır (devam ediyor)

789	{
10111213141516171819202122232425262728	//Yerel Tanımlar
	RESİM* pPic;
	//İfadeler
	// İlk düğümü al
	eğer (listCount (list) == 0) printf(
	"Üzgünüm, listede hiçbir şey yok\");else
	{
	printf("En İyi Filmler Listesi"); traverse (li
	st, 0, (void**)&pPic); yap
	{
	printf("%hd %-40s %s", pPic->y
	ıl, pPic->resim, pPic->yönetmen);
	} iken (traverse (list, 1, (void**)&pPic));
	} // else
	printf("En İyi Filmler Listesi Sonu");}
	// printList

Program 5-22 AnaliziBu işlevdeki ilginç mantık, listenin gezilmesi ile ilgilidir. Soyut veri türündeki gezinti işlevi, listenin başından mı yoksa son konumundan mı başlayacağını belirtmek için bir "nere den" bayrağına ihtiyaç duyar. Bu nedenle, döngüden önce gezintiye ilk filmin verileri ni almak istediğimizi söyleriz. Bir film aldığımızda, geri kalan listeyi bir döngü kullanarak elde ederiz. Çünkü gezinti bir başarı bayrağı döndürdüğünden, bunu *do...while* ifadesinde kullanabiliriz. Tüm filmler görüntülendiğinde, bir liste sonu mesajı gösteririz ve işlemi sonlandırırız.

Arama Listesi

Uygulamamızdaki son işlev, kullanıcının herhangi bir yıl için bir filmi aramasına ve görüntülemesine olanak tanır. Kod Program 5-23'te gösterilmiştir.

PROGRAM 5-23Arama Listesi

123456789	/*===== arama =====Yılı ar
	ar ve yılı, resmi vedirektörü yazdırır.
	Ön liste oluşturuldu
	kullanıcı arama seçeneğini seçti
	Son yıl yazdırıldı veya hata mesajı
	*/
	geçersiz arama (LIST* listesi)
	{

devam ediyor

PROGRAM 5-23Arama Listesi (devam ediyor)

```

10111213141516171819202122232425262728293031
//Yerel Tanımlar
short yıl;
bool bulundu;

RESİM pSrchArgu;
RESİM* pPic;

//İfadeler
printf("Dört haneli bir yıl girin: ");
scanf ("%hd", &yıl);
pSrchArgu.yıl = yıl;

bulundu = searchList (liste, &pSrchArgu,
                                (void**) &pPic);

if (bulundu)
    printf("%hd %-40s %s",
           pPic->yıl, pPic->resim, pPic->yönetmen);
else
    printf("Üzgünüm, ama %d mevcut değil.", yıl);return;

} // arama

```

Yılı Karşılaştır

Liste oluşturma analizi sırasında tartıştığımız gibi, iki yılı karşılaştıran bir karşılaştırma fonksiyonu yazmalıyız. Tasarımımız, C'de bulunan karşılaştırma dizesi fonksiyonuna benzer. Eğer birinci yıl ikinci yıldan küçükse, -1 döneriz; iki yıl eşitse, 0 döneriz; birinci yıl ikinci yıldan büyükse, +1 döneriz. Kod Program 5-24'te gösterilmiştir.

PROGRAM 5-24Yıl Karşılaştırma Fonksiyonu

```

123456789
/*===== cmpYear =====
İKİ YILI PICTURE yapılarında karşılaştırır
Önce year1, ilk yapıya işaret eden bir işaretçidir
year2, ikinci yapıya işaret eden bir işaretçidir
Sonuç olarak iki yıl karşılaştırılır ve sonuç döndürülür
Eğer year1 daha küçükse -1, eşitse 0, daha büyükse +1 döndürün
*/
int cmpYear (void* pYıl1, void* pYıl2)
{
1011121314
//Yerel Tanımlar
int sonuç;
short year1;
short year2;

```

devam ediyor

PROGRAM 5-24Yıl Karşılaştırma Fonksiyonu (devamı)

```

151617181920212223242526
//İfadeler
yıl 1 = ((RESİM*)Yıl 1)->yıl;
yıl2 = ((RESİM*)pYıl 2)->yıl;

if (yıl1 < yıl2)
    sonuç = -1;
else if (yıl1 > yıl2)
    sonuç = +1;
    aksi takdirde
        sonuç = 0;
    dönüş sonucu;
} // cmpYıl

```

Program 5-24 AnaliziYıl karşılaştırma, iki resim yapısını alır ve birincisindeki anahtarı ikincisindeki anahtarla karşılaştırır. Basitlik açısından, resim yapısındaki yılları yerel değişkenlere atarız ve ardından karşılaştırmayı yaparız. Bu adım gereksizdir, ancak ortaya çıkan kod biraz daha anlaşılırdır.

Not edin ki, soyut veri türü karşılaştırmafonksiyonuna bir *void* işaretçi geçirdiğinden , yapıyı bir resim türü olarakcast etmemiz gerekiyor. Bunu yapmazsak, birkomut hatası al ırız.

Ekleme ve Silme Mantiğini Test Etme

Liste algoritmalarını test etmek dikkatli bir planlama gerektirir. Eklememantiğini ve silme mantığının test etmeyi tartışıyoruz.

Ekleme Mantiğini Test Etme

Liste sıralı olduğundan, eklememantiğini doğrulamak için en az dört test durumu gereklidir:

1. *Null bir listeye bir düğüm ekle*—Bu test her zaman otomatik olarak yapılır çünkü ü liste başlangıçta null bir durumda başlar.
2. *İlk veri düğümünden önce bir düğüm ekle*—Bu test otomatik değildir; bu nedenle, girişin bu durumun test edilmesi için düzenlenmesi gerekir.
3. *İki veri düğümü arasında bir düğüm ekle*—Yine, bu test otomatik değildir. Testdur umlarının en az bir düğümün iki mevcut düğüm arasında eklenmesinisağladığında n emin olmamız gerekiyor.
4. *Son düğümünden sonra bir düğüm ekle*—Bu durum, null bir listeye ekleme ile ay nolduğundan, otomatik olarak gerçekleşir. Yine de, yeni verilerinlistenin son d üğümünden sonra eklendiği bir test durumu öneriyoruz.

Silme Mantiğini Test Etme

Silme mantığını test etmek, ekleme mantığını test etmeye benzer:

1. *Null listeye sil*—Silme mantığını tam olarak test etmek için, bir test durumu list enin tüm verilerini silmeli ve ardından yeni veriler eklemelidir.

2. *Listede ilk veri düğümünü sil*—Listelerle çalışırken, hataların en yaygın olduğu yerler listenin ilk ve son elemanlarıdır. Listede ilk veri düğümünü silen bir test durumunun olduğundan emin olun.
3. *İki veri düğümü arasında bir düğüm sil*—Bu en yaygındurum olduğu için genellikle test edilir. Ancak, en az bir silme durumunun ikiveri düğümü arasında bir düğüm sildiğinden emin olmak için test verilerinin analiz etmemiz gerekiyor.
4. *Listede sonundaki düğümü sil*—Bu test otomatik değildir. Listede sonundaki düğümü silen bir silme durumunun olduğundan emin olun.
5. *Var olmayan bir düğümü silmeye çalışın*—Bu test belirgin değildir. Bazı programlar, hedef düğüm mevcut olmadığında bile listede bir düğümü yanlışlıkla silmektedir. Bu nedenle, bulunamayan koşulları da test etmemiz çok önemlidir.
6. *İlk veri düğümünün anahtarından daha küçük bir anahtara sahip bir düğümü silmeye çalışın*—Programdaki ince bir hata, hedef mevcut olmadığında ilk düğümün silinmesi ne neden olabilir. Bu durumun test edildiğinden emin olun.
7. *Son veri düğümünün anahtarından büyük bir anahtara sahip bir düğümü silmeye çalışın*.
8. *Boş bir listeden silmeye çalışın*—Bu, önceki koşulun tersidir. Test durumlarına dahil edilmesi gerekir.

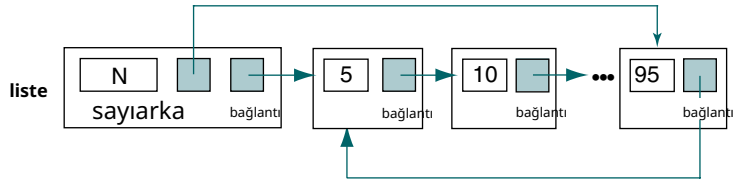
5.5 Karmaşık Uygulamalar

Şu ana kadar kullandığımız uygulama, yalnızca bir ardıl düğüme tek bir bağlantı içerdiği için tek bağlı liste olarak bilinir. Bu bölümde, üç başka yararlı uygulama tanıtıyoruz: dairesel bağlı liste, çift bağlı liste ve çoklu bağlı liste. Tam olarak geliştirilmemişlerdir. Liste ADT'sine dahil edilmek için, karşılaştırma işlevine ve bir konum işaretçisine ek olarak ek işlevler gibi ek yapılara ihtiyaçları olacaktır.

Dairesel Bağlı Listeler

Dairesel bağlı liste uygulamasında, son düğümün bağlantısı, listenin ilk düğümüne işaret eder. Şekil 5-19'da gösterildiği gibi. Dairesel bağlı listeler, listenin ortasındaki düğümlere erişime izin veren listelerden anahtarı baştan başlatmadan kullanılmaktadır. Çoklu bağlı listeyi tartışırken bu yapılardan birini göreceğiz.

Dairesel bağlı listeye ekleme ve silme işlemleri, son düğümün ilk düğüme işaret etmesi dışında, tıpkı tek bağlı listede kullanılan aynı mantık kalıplarını takip eder. Bu nedenle, son düğümü eklerken veya silerken, başlıkta arka işaretçiyi güncellemenin yanı sıra bağlantı alanını da ilk düğüme işaret etmemiz gerekir.



ŞEKİL 5-19 Dairesel Bağlı Liste

Liste içindeki bir düğüme veri yapısı aracılığıyla doğrudan erişebildiğimizden, listede arama yaparken bir sorunla karşı karşıyayız.

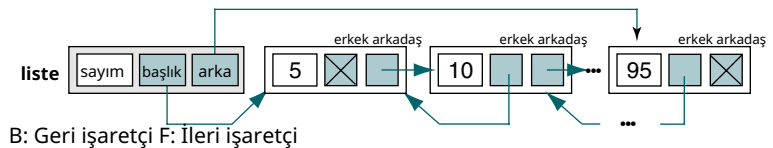
Arama hedefi mevcut düğümden önceyse, onu nasıl buluruz? Tek yönlü bağlı liste uygulamasında, listenin sonuna geldiğimizde arama tamamlanır. Ancak, dairesel liste uygulamasında, aramayı otomatik olarak listenin başından devam ettiririz.

Aramayı devam ettirme yeteneği başka bir sorun ortaya çıkarır. Hedef mevcut değilse, ne olur? Tek yönlü bağlı liste uygulamasında, aradığımız v eriyi bulamazsak, listenin sonuna geldiğimizde veya hedef mevcut düğüm ün verisinden daha küçükse dururuz. Dairesel liste uygulamasında ise, başlangıç düğümünün adresini kaydederek ve ona geri döndüğümüzde dururuz; aşağıdaki kodda gösterildiği gibi:

döngü (hedef pLoc anahtarına eşit değil
VE pLoc bağlantısı startAddress'e eşit değil)

Çift Yönlü Bağlı Listeler

En güçlü uygulamalardan biri çift yönlü bağlı listedir. Çift yönlübağlı li ste, her düğümün hem ardına hem de öncüsüne bir işaretçisahip ol duğu bir bağlı liste yapısıdır. Şekil 5-20, bir çift yönlübağlı listenin sun umudur.



ŞEKİL 5-20 Çift Bağlı Liste

Başlık yapısında üç parça meta veri bulunmaktadır: bir sayı, gezinmeler içi n bir konum işaretçisi ve bir arka işaretçi. Her ne kadar tüm çift bağlı listelerde bir arka işaretçi gerekli olmasa da, bu bazı liste algoritmalarını, örneğin ekleme ve arama, daha verimli hale getirir.

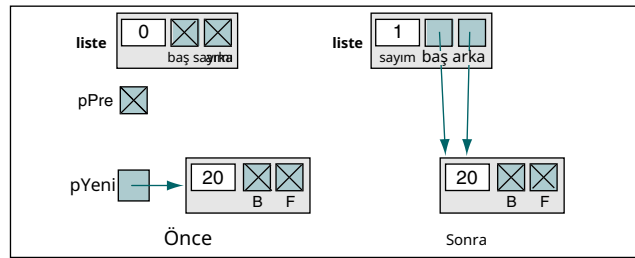
Her düğüm iki işaretçi içerir: bir geri işaretçi öncüsüne ve bir ileri işaretçi ardılına. Şekil 5-20'de bu işaretçiler sırasıyla B ve F olarak adlandırılmıştır.

Çift bağlı listenin bir başka varyasyonu, çift bağlı dairesel bağlı listedir. Bu varyasyonda son ileri işaretçi, listenin ilk düğümüne işaret eder ve ilk düğümün geri işaretçisi son düğümüne işaret eder. Listede yalnızca bir düğüm varsa, hem ileri hem de geri işaretçiler düğümüne kendisine işaret eder.

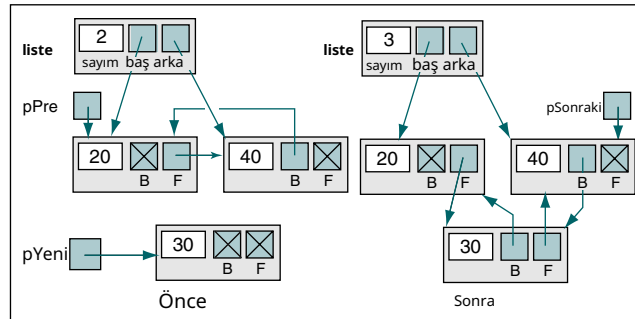
Ekleme

Düğüm ekleme, tek bağlı bir listeye düğüm ekleme temel desenini takip eder, ancak hem ileri hem de geri işaretçileri bağlamamız da gerekir.

Boş bir çift bağlı listenin baş ve arka işaretçileri boştur. Boş bir listeye bir düğüm eklemek için, baş ve arka işaretçileri yeni düğümüne işaret edecek şekilde ayarlıyoruz ve yeni düğümün ileri ve geri işaretçilerini boş ayarlıyoruz. Bir düğümün boş bir listeye eklenmesinin sonuçları Şekil 5-21(a)'de gösterilmiştir.



(a) Boş listeye veya ilk düğümden önce ekleme



(b) İki düğüm arasında ekleme

ŞEKİL 5-21 Çift Bağlı Liste Ekle

Şekil 5-21(b), iki düğüm arasında ekleme durumunu göstermektedir. Yeni düğüm, hem öncüsüne hem de ardılına işaret edecek şekilde ayarlanmalı ve o nlar da yeni düğümüne işaret edecek şekilde ayarlanmalıdır. Ekleme listenin ortasında olduğu için, baş yapısı değişmez.

Listenin sonuna ekleme yapmak, yeni düğümün geriışaretçisinin öncekine iş aret edecek şekilde ayarlanmasını gerektirir. Çünküsonraki yoktur, ileri işaretçi null olarak ayarlanır. Baş yapısındaki arka işaretçiyi zamanda yeni arka düğüme iş aret edecek şekilde ayarlanmalıdır.

Algoritma 5-11, bir dübelli bağılı liste düğümü eklemek için kodu iş erir.

ALGORİTMA 5-11Dübelli Bağılı Liste Ekle

```

Algoritma insertDb1 (liste, veriGiriş i)
Bu algoritma, bir dübelli bağılı listeye veri ekler.
Ön liste, geçerli bir listeye ait meta veri yapısıdır
veriGiriş i, eklenmesi gereken veriyi iş erir
Sonra Veriler sıralı olarak eklenmiştir
Dönüş 0: başarısız--dinamik bellek taşması
1: başarılı
2: başarısız--tekrar eden anahtar sunuldu
1 eğer (tam liste)
1 0 döndür
2 eğer bitir
listede ekleme noktasını bul.
3 bulunanı searchList olarak ayarla
(liste, öncül, ardıl, dataIn anahtar)
4 eğer (bulunmadıysa)
1 yeni düğüm ayır
2 dataIn'i yeni düğüme taşı
3 eğer (öncül null ise)
İlk düğümden önce veya boş listeye ekleniyor
1 yeni düğüm arka iş aretçisini null olarak ayarla
2 yeni düğüm ön iş aretçisini liste başına ayarla
3 liste başını yeni düğüme ayarla
4 aksi takdirde
Listenin ortasına veya sonuna ekleme
1 yeni düğüm ön iş aretçisini öncülün ön iş aretçisine ayarla
2 yeni düğüm arka iş aretçisini öncüye ayarla
5 eğer son ise bitir
null listeye veya listenin sonuna ekleme testi
6 eğer (öncül ön null)
Listenin sonuna ekleme--arka iş aretçiyi ayarla
1 liste arkasını yeni düğüme ayarla
7 aksi takdirde
Listenin ortasına ekleme--başarılıyı yeniye iş aret et
1 başarılıyı yeni düğüme geri ayarla
8 eğer son ise bitir
9 önceki düğümü yeni düğüme ayarla
10 1 döndür
5 eğer bitir
Yinelenen veri. Anahtar zaten mevcut.
6 2 döndür
son insertDb1

```

Algoritma 5-11 Analizi Bu algoritmada birkaç noktaya bakmalıyız. Öncelikle, basit bir başarı veya başarısızlık döndürmek yerine, üç farklı koşulumuz var ve dolayısıyla üç farklı değer döndürüyoruz. Dinamik bellek doluysa, 0 döndürüyoruz, bu bellek taşmasını gösteriyor. Başarılıysak, 1 döndürüyoruz. Son olarak, ekleme, listede zaten bulunan verilerle eşleşiyorsa, 2 döndürüyoruz.

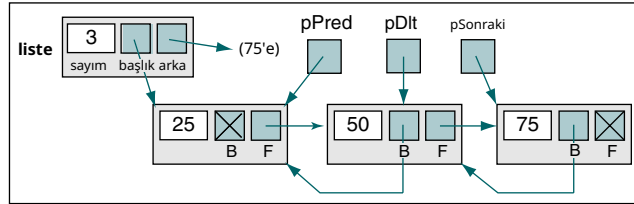
Arama algoritması, hedefin mantıksal öncüsünün konumunu sağlar veya: (1) hedefle eşleşen anahtara sahip düğümün konumu, (2) hedeften büyük olan ilk düğümün konumu, veya (3) hedef son düğümden büyükse null. Arama fonksiyonundaki son parametreyi sonuç olarak adlandırdık çünkü beklediğimiz buydu. (Okuyucu için, sonuç konumundan başka bir şey aradığımızı belirten bir işaretçi adı kullanmak kafa karıştırıcı olabilir.)

Son olarak, yorumlara dikkat edin. Kısa bir algoritma olmasına rağmen, farklı koşulların meydana gelebileceği kafa karıştırıcı olabilir. Yorumlar okuyucu için mantığı netleştirmelidir.

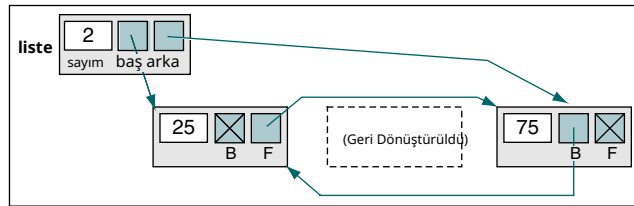
Silme

Silme işlemi, silinen düğümün öncüsünün, mevcutsa, silinen düğümün ardılını işaret etmesini ve ardılın, mevcutsa, öncüyü işaret etmesini gerektirir. Bu oldukça basit mantık Şekil 5-22'de gösterilmiştir.

Düğümü sileceğimiz yeri bulduğumuzda, sadece öncüsünün ve ardılının işaretçilerini değiştirir ve düğümü geri dönüştürürüz.



(a) Silmeden önce



(b) 50 silindikten sonra

ŞEKİL 5-22 Çift Bağlı Liste Silme

Pseudocode, Algoritma 5-12'de gösterilmiştir.

ALGORİTMA 5-12 Çift Bağlı Liste Silme

```

Algoritma deleteDb1 (liste, deleteNode)
Bu algoritma, çift bağlı bir listeden bir düğümü siler.
    Ön   liste geçerli bir listeye ait meta veri yapısıdır
        deleteNode silinecek düğümüne işaret eden bir işaretçidir
    Son   düğüm silindi
1  eğer (deleteNode null)
1  abort ("Çift silmede imkansız durum")
2  eğer bitir
3  eğer (deleteNode back not null)
    Önceki düğümü sonraki düğümüne işaret et
1  önceki düğümü   deleteNode back olarak ayarla
2  önceki düğümün önünü deleteNode fore olarak ayarla
4  aksi takdirde
    baş işaretçisini güncelle
1  silmeDüğümü ön ayarını yap
5  eğer bitir
6  eğer (silmeDüğümü ön null değilse)
    Sonraki noktayı önceki ile bağla
1  sonraki ayarını silmeDüğümü ön olarak ayarla
2  sonraki geri ayarını silmeDüğümü geri olarak ayarla
7  aksi takdirde
    arka noktayı önceki ile bağla
1  silmeDüğümü arka ayar listesi
8  eğer bitir
9  geri dönüşüm (silmeDüğümü)
son   silmeÇift

```

Algoritma 5-12 Analizi Bu algoritmadaki iki noktanın daha fazla yorumlanması gerekiyor. İlk olarak, arama çağıran algoritmada yapıldı. İkili bağlı liste silme algoritmasına girdiğimizde, silinecek düğümün yerini zaten biliyoruz.

İkinci olarak, null silme işaretçisi ile asla çağrılmaması gerektiğinden, bir tane tespit ederse programı durduruyoruz. Bu bir mantık hatasıdır. Bir programda mantık hatası tespit edildiğinde, program durdurulmalı ve düzeltilmelidir.

Çok Bağlantılı Listeler

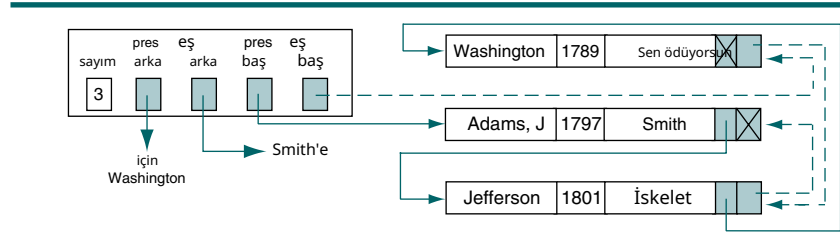
Bir çok bağlantılı liste iki veya daha fazla mantıksal anahtar dizisine sahip bir listedir. Örneğin, ABD'nin ilk 10 başkanının listesini düşünün, bu Liste 5-2'de gösterilmiştir.

Tablo 5-2'deki veriler, başkanın göreve ilk başladığı tarihe (yıl) göre kronolojik olarak sıralanmıştır. İki ek sıralama ilginç olabilir. Veriler, başkanın adı veya eşinin adıyla sıralanabilir. Daha da iyisi, neden bu sıralamalardan herhangi birinde listeyi gezemeyelim? Bu, çok bağlantılı listenin gücüdür: aynı veri seti birden fazla sıralamada işlenebilir. Çok bağlantılı bir listede verilerin kopyalanmadığını anlamak önemlidir. Veriler yalnızca bir kez mevcuttur, ancak bir set veriyi bağlayan birden fazla yol vardır.

Başkan	Yıl	First lady
Washington, George	1789	Custis, Martha Dandridge
Adams, John	1797	Smith, Abigail
Jefferson, Thomas	1801	Skelton, Martha Wayles
Madison, James	1809	Todd, Dorothy Payne
Monroe, James	1817	Kortright, Elizabeth
Adams, John Quincy	1825	Johnson, Louisa Catherine
Jackson, Andrew	1829	Robards, Rachel Donelson
Van Buren, Martin	1837	Hoes, Hannah
Harrison, William H. .	1841	Symmes, Anna
Tyler, John	1841	Christian, Letitia

TABLO 5-2Amerika Birleşik Devletleri'nin İlk 10 Başkanı

Verileri birden fazla dizide işlemek için, her dizi için ayrı bir bağlantı seti oluşturuyoruz. Her bağlantı yapısı tek bağlı, çift bağlı veya dairesel bağlı olabilir. Kavramı göstermek için, başkanların ve ilk hanımlarının tek bağlı çok bağlantılı bir listesini oluşturalım. Şekil 5-23 ilk üç başkan için verileri diyagramlaştırmaktadır.



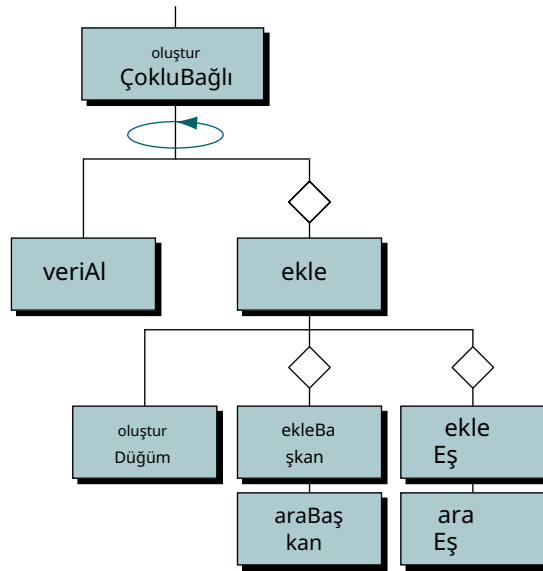
ŞEKİL 5-23Başkanlar Listesinin Çok Bağlantılı Liste Uygulaması

Bir fiziksel listede iki mantıksal liste olduğu için, ikifarklı bağlantı alanı na ihtiyacımız var. Bu nedenle her düğüm iki bağlantı alanı içerir: biribaşk an için ve diğeri eş için. Başkan bağlantıları katıçizgilerle temsil edilir. Eş b ağılantıları kesikçizgilerle temsil edilir. Başkan bağlantılarını takip edersek, listeyi Adams, Jefferson ve Washington üzerinden geçeriz. Eş bağlantıları nı takip edersek, listeyi Custis (Bayan Wash-ington), Skelton (Bayan Jeffers on) ve Smith (Bayan Adams) üzerinden geçeriz. Elbette, gerçek bir uygula mada çok daha fazla veri alanı olacaktır. Yapıyı temsil etmek için sadece ü ç tane kullanıyoruz.

Küçük ayarlamalarla, çok bağlantılı bir listeyi sürdürmek için gereken temel algoritmalar, tek bağlantılı veya çift bağlantılı listeler için kullanılan algoritmalarla aynıdır.

Ekle

Öncelikle çok bağlantılı bir liste oluşturma tasarımına bakalım. Tablo 5-2'de tanımlanan başkan listesi kullanılarak, iki mantıksal liste olduğunu görüyoruz. Bu nedenle, listeye bir düğüm eklediğimizde, onu her bir listeye eklememiz gerekiyor. Ancak, verileri yalnızca bir kez depoladığımızdan emin olmak için dikkatli olmalıyız. Ek düğüm algoritmasının tasarımı Şekil 5-24'te gösterilmiştir.



ŞEKİL 5-24 Çok Bağlantılı Listeye Düğüm Ekleme

Bu tasarım, listenin bir dosyadan oluşturulduğunu varsayıyor. buildMulti-bağlantılı modülü, bir başkan için verileri okuyarak ve listeye bir düğüm ekleyerek döngü yapar. Ardından, bir sonraki başkan için verileri okuyarak tekrar döngü yapar ve dosya tamamen okunana ve liste oluşturulana kadar devam eder.

Her ekleme, buildNode'da veriler için alan ayırarak başlar ve ardından insert president modülünde başkan bağlantılarını ve insert spouse modülünde eş bağlantılarını oluşturur. Farklı anahtarlarla farklı mantıksal listeler olduğu için, her ekleme ve arama için ayrı algoritmalar kullanıyoruz.

Bu tasarımda üç koşullu çağrı vardır. Başarılı bir şekilde veri okuduğumuz sürece insert çağrısını yapıyoruz. Bir hata olduğunda veya dosyanın sonuna geldiğimizde, çağrıyı yapmıyoruz. Benzer şekilde, build node modülündeki bellek tahsis başarılı olduğu sürece eklemeleri çağırıyoruz. Eğer başarısız olursa, verileri ekleyemeyiz.

Eğer tek bağlı ve çift bağlı listelerin ekleme tasarımını çok bağlı listeye için olan mantıkla karşılaştırırsanız, bir ana fark göreceksiniz: Çok bağlı liste eklemesinde, bellek tahsisi arama ve işaretçi bağlantılarından ayırdır çünkü bellek yalnızca bir kez tahsis edilir. Diğer küçük bir varyasyon ise, sayım artışıdır; bu da yapı düğümü modülünde kodlanmalıdır. Sayımı yalnızca bir kez artırıyoruz çünkü yalnızca bir fiziksel düğüm ekliyoruz, iki mantıksal listeye eklemiş olsak bile.

Sil

Silme için ana algoritma varyasyonu, her mantıksal liste için işaretçileri yeniden bağlamamız gerektiğidir. Bu nedenle, bir başkanın kaydını silersek, eşin ve başkanın halef işaretçisini de ayarlamamız gerekir.

Bir başkan silmek için başkanın işaretçilerini takip ettiğimizi varsayarsak, silme mantığı diğer bağlı liste silme işlemlerinin kullandığı aynı deseni takip eder. Ama eşi nasıl sileriz? Bir alternatif, başkan aramasından eşin adını kullanmak ve ardından güncellenmesi gereken işaretçileri bulmak için eş listesinde arama yapmaktır. Listeler uzun olduğunda, bunu yapmak çok verimsiz olabilir.

Standart çözüm, eş bağlantıları için çift yönlü bağlı liste kullanır. Başkan listesi üzerindeki aramamızla eş listesine ortasında ulaştığımızda, eş öncüsünün işaretçilerini, öncüye geri işaretçiyi takip ederek kolayca ayarlayabiliriz. Bu uygulama, çift yönlü bağlı listelerin daha yaygın kullanımlarından biridir.

5.6 Anahtar Terimler

kronolojik listede
iresel bağlı listeliste
oluştur
düğümü sil
işlem
listeyi yok
et\ çift bağlı liste
boş liste
tam liste
eklemeana
htar

meta veri
siçok bağlantılı l
istesıralı list
etaşmar
astgele liste
arka işaretçi
alımara
ma listesit
ek bağlantılı liste
gezinme

5.7 Özet

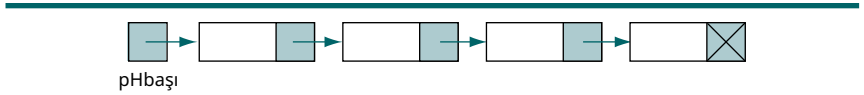
- Genel bir lineer listede, veriler her yerde eklenip silinebilir ve listeyi işlemek için kullanılabilecek işlemler üzerinde herhangi bir kısıtlama yoktur. Bu bölümde genel lineer listelere basitçe *listeler* olarak atıfta bulunuyoruz.
- Listeler, rastgele listeler ve sıralı listeler olarak daha da bölünebilir. Rastgele bir listede, verilerin bir sıralaması yoktur. Sıralı bir listede, veriler, verileri tanımlamak veya kullanımını kontrol etmek için kullanılan bir veya daha fazla alan olan bir anahtara göre düzenlenir.
- Listelerle ilişkili dört yaygın işlem vardır: ekleme, silme, alma ve gezinti.
- Listeler genellikle bağlı listeler kullanılarak uygulanır.
- Bir baş düğüm, listenin ilk düğümüne işaret eden bir baş işaretçi ve son düğümüne işaret eden bir arka işaretçi gibi, liste hakkında meta verileri içeren bir veri yapısıdır. Yapının kullanımı için gereken diğer verileri de içerebilir.
- Bir listeye eklemek istediğimizde, dört durumu göz önünde bulundurmalıyız: boş listeye eklemek, başa eklemek, ortada eklemek ve sona eklemek.
- Bir listeden bir düğümü silmek istediğimizde, iki durumu göz önünde bulundurmalıyız: ilk düğümü silmek veya başka bir düğümü silmek.
- Bir öğeyi bir listede aramak için sıralı liste aramasını kullanırız.
- Bir listeyi geçmek, listeyi düğüm düğüm geçmek ve her düğümü işlemek anlamına gelir. Liste geçişlerine üç örnek, düğüm sayısını saymak, düğümlerin içeriğini yazdırmak ve bir veya daha fazla alanın değerlerini toplamak.
- Bir liste, son düğümün bağlantısının listenin ilk düğümüne işaret ettiği dairesel bağlı liste kullanılarak uygulanabilir.

- ❑ Bir liste, her düğümün hem ardılına hem de öncülüne işaretçi sahip olduğu çift bağlantılı liste kullanılarak uygulanabilir.
- ❑ Bir liste, iki veya daha fazla mantıksal liste oluşturmak için çok bağlantılı liste kullanılarak uygulanabilir.

5.8 Uygulama Setleri

Alıştırmalar

1. Şekil 5-25'te gösterilen listeyi bağlı liste olarak uyguladığımızı hayal edin.



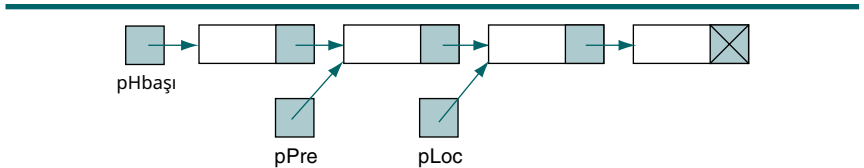
ŞEKİL 5-25 Alıştırma 1 için Bağlı Liste Uygulaması

Bu listeyi ararken aşağıdaki ifadeyi kullanırsak ne olacağını gösterin:

`pHead = pHead->bağlantı`

Bu tür bir ifadeyi kullanmanın sorunu nedir? Metinde tanıttığımız i ki yürüyen işaretçi (p_{Pre} ve p_{Loc}) için bir gerekçe sağlıyor mu?

2. Şekil 5-26'da gösterilen listeyi bağlı liste olarak uyguladığımızı hayal edin. "Liste Arama"da, Bölüm 5.2'de tartışıldığı gibi, arama kriterlerine dayalı olarak hem öncülün (p_{Pre}) hem de mevcut (p_{Loc}) düğümün konumunu geri döndürebilmesi gerekir.



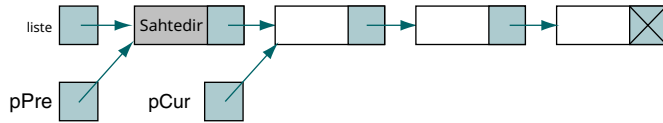
ŞEKİL 5-26 Alıştırma 2 için Bağlı Liste Uygulaması

Aşağıdaki kod, pPre ve pLoc ayarlamak için yaygın bir hata içermektedir. Bu nedir ve nasıl düzeltilmelidir?

```
pLoc = pLoc->link
pPre = pPre->link
```

(*İpucu:* Bu işaretçilerin arama başlangıcındaki içerikleri nelerdir?)

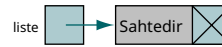
3. Bir listeyi listenin başında bir sahte düğüm kullanarak uyguladığımızı hayal edin. Sahte düğüm herhangi bir veri taşımaz. İlk veri düğümü değildir, boş bir düğümdür. Şekil 5-27, sahte düğümlü bir listeyi göstermektedir.



ŞEKİL 5-27Egzersiz 3 için Bağlı Liste Uygulaması

Kodun, listedeki ilk düğümü (sahte düğümden sonraki düğüm) silmek için yazın.

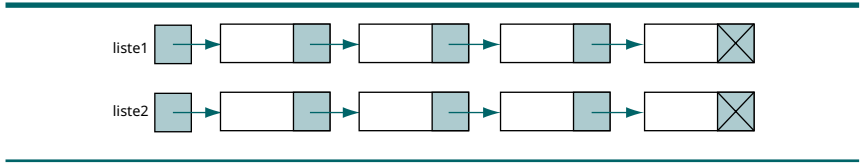
4. Bir liste içinde, sahte düğüm ile uygulanan bir bağlı liste içinde ortada bir düğümü silmek için kodu yazın (bkz. Egzersiz 3). Cevabınızı Egzersiz 3'teki cevapla karşılaştırın. Aynı mı? Ne sonucuna varıyorsunuz? Sahte düğüm, bir liste üzerindeki işlemi basitleştiriyor mu? Nasıl?
5. Şekil 5-28, sahte düğüm ile boş bir listeyi göstermektedir. Bu boş listeye bir düğüm eklemek için kodu yazın.



ŞEKİL 5-28Egzersiz 5 için Liste

6. Sahte düğüm ile bir liste ortasında bir düğüm eklemek için ifadeleri yazın (bkz. Egzersiz 3). Cevabınızı Egzersiz 5'teki cevapla karşılaştırın. Aynı mı? Ne sonucuna varıyorsunuz? Sahte düğüm, bir liste üzerindeki işlemi basitleştiriyor mu? Nasıl?
7. Şekil 5-29'da gösterilen iki listeye sahip olduğumuzu hayal edin. Bu iki listeye aşağıdaki ifadeyi uygularsak ne olur?

```
listel = liste2
```

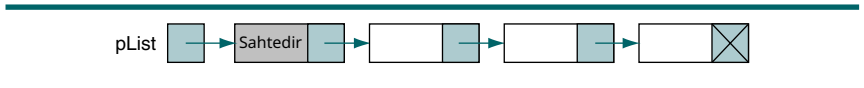


ŞEKİL 5-29Egzersiz 7 için Bağlı Listeler

8. Egzersiz 7'deki iki listeye aşağıdaki ifadeleri uygularsak ne olurdu?

- 1 temp'i list1 olarak ayarla
- 2 döngü (temp bağlantısı null değilse)
 - 1 temp'i temp bağlantısı olarak ayarla
- 3 döngüyü bitir
- 4 temp bağlantısını list2 olarak ayarla

9. Hayal edin ki, Şekil 5-30'da gösterilen listeye sahibiz.



ŞEKİL 5-30Alıştırma 9 için Liste Uygulaması

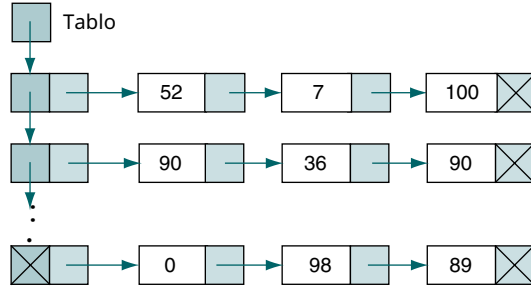
Bu listedeki aşağıdaki ifadeleri uygularsak ne olurdu?

- 1 temp'i pList olarak ayarla
- 2 döngü (temp bağlantısı null değilse)
 - 1 temp'i temp bağlantısı olarak ayarla
- 3 döngüyü bitir
- 4 temp bağlantısını pList olarak ayarla

Problemler

10. Klavyeden bir tam sayı listesi okuyan, bunları bağlı liste uygulaması kullanarak bir liste oluşturan ve sonucu yazdıran bir algoritma yazın.
11. Bağlı liste kullanarak uygulanmış bir listeyi kabul eden, onu dolaşan ve en küçük anahtar değerine sahip düğümdeki veriyi döndüren bir algoritma yazın.
12. Bağlı liste kullanarak uygulanmış bir listeyi dolaşan ve anahtarları negatif olan tüm düğümleri silen bir algoritma yazın.
13. Bağlı liste kullanarak uygulanmış bir listeyi dolaşan ve negatif anahtara sahip bir düğümün ardından gelen düğümü silen bir algoritma yazın.

14. Negatif anahtara sahip bir düğümün hemen öncesindeki düğümü silen ve bağlı liste kullanarak bir listeyi dolaşan bir algoritma yazın.
15. Bağlı liste uygulaması kullanarak iki boyutlu bir dizi oluşturan bir program yazın. İlk sütundaki düğümler yalnızca iki işaretçi içerir, Şekil 5-31'de gösterildiği gibi. Sol işaretçi bir sonraki satıra işaret eder. Sağ işaretçi ise satırdaki verilere işaret eder.



ŞEKİL 5-31 Problem 15 için Liste

16. Metindeki çoğu algoritmayı, Şekil 5-32'de gösterildiği gibi, başında bir sah te düğüm bulunan bir bağlı liste uygulaması kullanarak basitleştirebiliriz.



ŞEKİL 5-32 Problem 16 için Uygulama

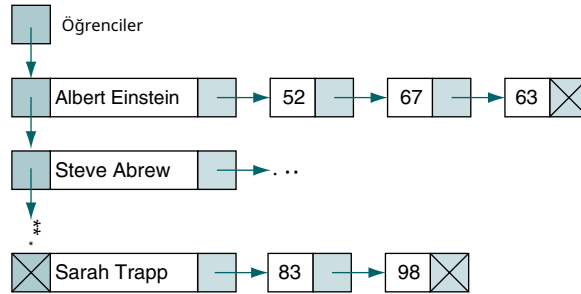
Bir bağlı liste uygulaması kullanarak insertNode (bkz. "Dahili Ekl eme Fonksiyonu," Bölüm 5.4) adında bir algoritma yazın.

17. Bir bağlı liste uygulaması kullanarak deleteNode (Algoritma 5-3, "L iste Düğüm Sil") adında bir algoritma yazın.
18. Bir bağlı liste uygulaması kullanarak searchList (Algoritma 5-4, "L iste Ara") adında bir algoritma yazın.
19. Bir bağlı liste uygulamasında son düğümüne işaretçi döndüren bir al goritma yazın.
20. İki listeyi birleştiren bir algoritma yazın. Bağlı liste uygulamasını ku llanın.
21. Bir listeyi kendisine ekleyen bir algoritma yazın. Bağlı liste uygula masını kullanın.

22. Bir listede iki düğümü değiştiren (takas eden) bir algoritma yazın. Düğümler numara ile tanımlanır ve parametre olarak geçer. Örneğin, 5 ve 8 numaralı düğümleri değiştirmek için `swap (5, 8)` çağrısını yaparsınız. Değişim başarılı olursa, algoritmanın true döndürmesi gerekir. Geçersiz bir düğüm numarası gibi bir hata ile karşılaşır, false döner. Bağlı liste uygulamasını kullanın.
23. İki listeyi birleştirmek için yeni bir ADT algoritması yazın. Bağlı liste uygulamasını kullanın.

Projeler

24. Bir dosyayı okuyan ve bir liste oluşturan bir program yazın. Liste oluşturulduktan sonra, monitörde görüntüleyin. Herhangi bir uygun veri yapısını kullanabilirsiniz, ancak bir anahtar alanı ve veri içermelidir. İki olasılık, en sevdiğiniz CD'lerin bir listesi veya arkadaşlarınızın telefon numaralarıdır. Bağlı liste uygulamasını kullanın.
25. Proje 24'te yazdığınız programı değiştirin. Dosyayı oluşturduktan sonra, program kullanıcıya yeni veri eklemek, mevcut verileri kaldırmak veya tüm verilerin bir listesini yazdırmak için bir menü sunmalıdır.
26. Bir dosyadan öğrenci listesini okumak ve bir liste oluşturmak için bir program yazın. Program, uygulama için bağlı liste kullanılmalıdır. Bağlı listedeki her düğüm, öğrencinin adını, bir sonraki öğrenciye işaret eden bir işaretçi ve notların bağlı listesine işaret eden bir işaretçi içermelidir. Her öğrenci için indört not olabilir. Yapı Şekil 5-33'te gösterilmiştir.



ŞEKİL 5-33 Proje 26 için Veri Yapısı

Program, öğrencilerin adlarını metin dosyasından okuyarak öğrenci listesini başlatmalıdır ve null not listeleri oluşturmalıdır. Daha sonra listeyi döngüye almalı ve kullanıcıdan her öğrenci için notları girmesini istemelidir. Notların istemi, öğrencinin adını içermelidir.

Tüm notlar girildikten sonra, program her öğrenci için notları ve toplam not ile ortalama notu yazdırmalıdır. Ortalama yalnızca mevcut olan notları içermelidir.

Her öğrenci için veriler Tablo 5-3'te gösterilmektedir.

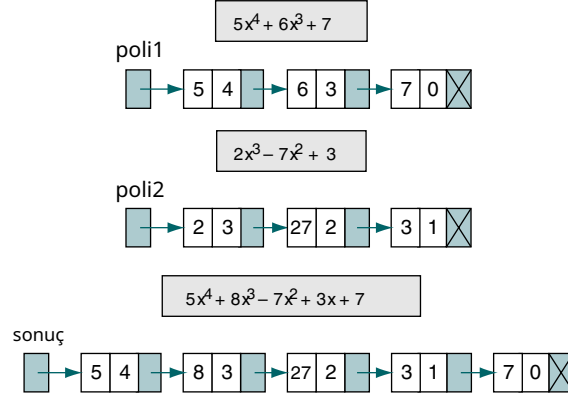
Öğrenci adı	Not 1	Not 2	Not 3	Not 4
Albert Einstein	52	67	63	
Steve Abrew	90	86	90	93
david najske	100	85	93	89
Mike Siyah	81	87	81	85
Andrew Dijkstra'nın	90	82	95	87
Joanne Nguyen	84	80	95	91
Chris Walljasper	86	100	96	89
Fred Albert	70	68		
Dennis Dudley	74	79	77	81
Leo Piringç	95			
Fred Çakmaktaş	73	81	78	74
Frances Dupre	82	76	79	
Dave Işık	89	76	91	83
Hua Tran	91	81	87	94
Sarah Trapp	83	98	94	93

TABLO 5-3Proje 26 için Veriler

27. Proje 26'yı, verileri öğrenci listesine anahtar (öğrenci adı) sırasına göre eklenecek şekilde değiştirin. Veriler, ad-soyad formatında girildiğinden, adı soyad-ad formatına yeniden biçimlendiren ve ardından bir dize karşılaştırması yapan özel bir karşılaştırma algoritması yazmanız gerekecek. Diğer tüm algoritmalar daha önce açıklandığı gibi çalışmalıdır.
28. İki sıralı listeyi birleştiren bir fonksiyon yazın. İki liste birleştirildiğinde, sonuçta elde edilen listedeki veriler de sıralı olmalıdır. İki orijinal liste değişmeden kalmalıdır; yani, birleştirilmiş liste yeni bir liste olmalıdır. Bağlı liste uygulamasını kullanın.
29. Polinomları toplayan ve çıkaran bir program yazın. Her polinom, bir bağlı liste uygulaması ile bir liste olarak temsil edilmelidir. Listede ilk düğüm, polinomdaki ilk terimi temsil eder, ikinci düğüm ikinci terimi, ve bu şekilde devam eder.

5. Birleştirme kavramları hakkında daha fazla bilgi için, Bölüm 12, "Gelişmiş Sıralama Kavramları"na bakın.

Her düğüm üç alan içerir. İlk alan terimin katsayısıdır. İkinci alan terimin gücüdür ve üçüncü alan bir sonraki terime işaret eder. Örneğin, Şekil 5-34'te gösterilen polinomları düşünün. İlk polinomdaki ilk terimin katsayısı 5 ve üssü 4'tür, bu da $5x^4$ olarak yorumlanır.



ŞEKİL 5-34 Proje 29 için Polinom Örneği

Polinomların toplanma kuralları şöyledir:

- Eğer üsler eşitse, katsayılar cebirsel olarak toplanır.
- Eğer üsler eşit değilse, daha yüksek üslü terim yeni polinomda yer alır.
- Eğer üssü 0 ise, x^0 'ı temsil eder, bu da 1'dir. Bu nedenle terimin değeri katsayının değeridir.
- Eğer katsayıların toplamı 0 sonucunu veriyorsa, terim atılır (0 ile çarpılan her şey 0'dır).

Bir polinom, her biri iki tam sayıya sahip bir dizi çizgi ile temsil edilir. İlk tam sayı katsayıyı; ikinci tam sayı ise üssü temsil eder. Böylece Şekil 5-35'teki ilk polinom

5 4
6 3
7 0

İki polinomu toplamak için program, her polinom için katsayıları ve üsleri okur ve bunları bağlı bir listeye yerleştirir. Girdi, ayrı dosyalardan okunabilir veya uygun kullanıcı istemleri ile klavyeden girilebilir. Polinomlar saklandıktan sonra, toplanır ve sonuçlar üçüncü bir bağlı listeye yerleştirilir.

Polinomlar, bir operasyonel birleştirme süreci kullanılarak toplanır. Bioperasyonel birleştirme, iki listeyi birleştirirken bir veya daha fazla işlem gerçekleştirir - bizim durumumuzda toplama. Toplama için her bir polinomdan bir terim alır ve üsleri karşılaştırırız. Eğer iki üs eşitse, kat sayılar toplanarak yeni bir kat sayı oluşturulur. Eğer yeni kat sayı 0 ise, terim atılır; eğer 0 değilse, sonuç polinomunun bağlı listesine eklenir. Eğer üslardan biri diğerinden büyükse, ilgili terim hemen yeni bağlı listeye konulur ve daha küçük üslü terim, diğer listeden gelen bir sonraki terimle karşılaştırılmak üzere tutulur. Eğer bir liste diğerinden önce biterse, uzun listedeki geri kalan kısım, yeni polinom için listeye basitçe eklenir.

İki giriş polinomunu ve bunların toplamını bağlı listelerigezinerek ve bunları sayı setleri olarak göstererek yazdırın. Her polinomu etiketlemeyi unutmayın.

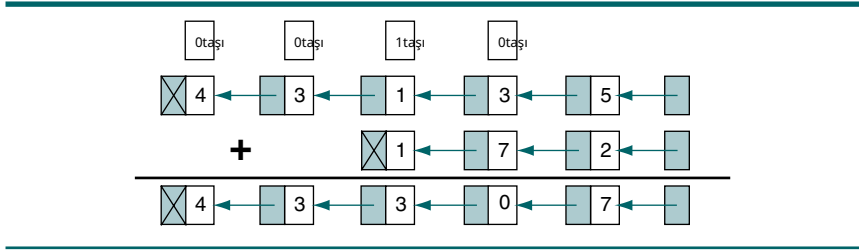
Programınızı Tablo 5-4'te gösterilen iki polinom ile test edin.

Polinom 1		Polinom 2	
Katsayı	Üs	Katsayı	Üs
7	9	-7	9
2	6	2	8
3	5	-5	7
4	4	2	4
2	3	2	3
6	2	9	2
6	0	-7	1

TABLO 5-4 Proje 29 için Metin Verisi

30. Eski kişisel bilgisayarlarda, en büyük tam sayı 32,767 ve en büyük uzun tam sayı 2,147,483,647'dir. Kriptografi ve güvenlik algoritmaları gibi bazı uygulamalar, sınırsız bir tam sayıya ihtiyaç duyabilir. Sınırsız boyutta tam sayıları depolamak ve işlemek için bir bağlı liste kullanmak bir yoldur. Her rakam, listenin bir düğümünde saklanır. Örneğin, Şekil 5-35, beş basamaklı bir sayıyı listede nasıl saklayabileceğimizi göstermektedir.

Şekil 5-35'teki liste sağdan sola hareket ediyormuş gibi temsil edilse de, bir listede fiziksel bir yön yoktur. Problemi netleştirmek için bu şekilde temsil ediyoruz.



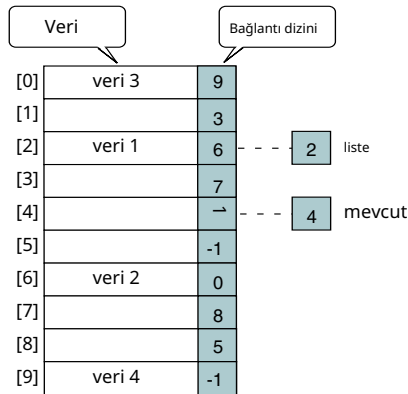
ŞEKİL 5-35Proje 30 için Listede Saklanan Tam Sayı

İki sayıyı toplamak için, ilgili listelerindeki aynı konumdaki karşılıklı gelen rakamı ve önceki toplamın taşmasını ekleriz.

Her toplamada, toplam 10'dan büyükse, 10'u çıkarmamız ve taşı 1 olarak ayarlamamız gerekir. Aksi takdirde, taş 0 olarak ayarlanır.

İki tam sayı listesini eklemek için bir algoritma yazın. Çözümünüzü, aynı mantığın ilk sayıları (birler basamağı) ve sayının geri kalanını ekleyecek şekilde tasarlayın. Diğer bir deyişle, birler basamağını eklemek için özel bir tek seferlik mantık kullanmayın.

31. İki sayıyı eklemek için fonksiyonu yazdıktan sonra, çarpma nispeten kolaydır. Çarpmanın temel tanımını, tekrarlayan toplama olarak kullanabilirsiniz. Diğer bir deyişle, iki sayıyı çarpmamız gerektiğinde, örneğin 45×6 , 45'i altı kez toplarız. Proje 30'da geliştirilen algoritmayı kullanarak iki sayıyı çarpan bir algoritma yazın.
32. Listeyi dinamik bellekte bağlı liste olarak uygulandığı şekilde gösterdik, ancak bunu bir dizide uygulamak da mümkündür. Bu durumda dizi yapısının iki temel alanı vardır: veri ve sonraki indeks konumu. Sonraki indeks alanı, dizi yapısının bir listenin özelliklerini almasına olanak tanır. Bir liste dizisi Şekil 5-36'da gösterilmiştir.



ŞEKİL 5-36Proje 32 için Bağlı Liste Dizisi

Dizide aslında iki liste olduğunu unutmayın. Veri listesielement 2'den başlar ve 6 ve 0 üzerinden geçerek veri sonunaelement 9'da u laşır. İkinci liste, tüm boş veya mevcut elemanlarıtoplar. Element 4'ten başlar, 1, 3, 7 ve 8 üzerinden geçer ve5'te sona erer.

Bir diziye kullanarak bir liste uygulayan bir program yazın. Programınızmenü tabanlı olmalı ve kullanıcıdan veri eklemesini, veri silmesini,liste içeriğini yazdır masını veya belirli bir veri parçasını liste içindearama yapmasını istemelidir.

Yeni veri eklemek için, mevcut listeden bir elementi silmeli ve ardından bunu veri listesine eklemelisiniz. Tersine, veri listesinden bir düğümü sildiğinizde, bunu mevcut listeye eklemelisiniz.

33. Liste ADT'sini çift yönlü bağlı liste olarak uygulanacak şekilde yeniden düzenleyin. Ek bir algoritma olarakgeri dönüşü de dahil edin.
34. Bir stok verilerini işlemek için bir program yazın. Stok verileri, aşağıdaki verileri içeren bir metin dosyasından okunmalıdır: stok kodu, stok adı, yatırılan miktar (xxx.xx), tutul an hisse senedi sayısı ve güncel fiyat. En az 20 hisse senedi hakkında veri toplamak içi n İnterneti veya yerel gazetenizi kullanın. (Hisse senetleri yerine yatırım fonları kullan abilirsiniz.)

Her hisse senedi okunduğunda, onu çift bağlı çok bağlantılı bir listeye ekleyin. İlk mantıksal liste, stok koduna göre sıralanmalıdır. İkinci mantıksal liste, hisse senedinin kazancı veya kaybına göre sıralanmalıdır. Kazanç veya kayıp, hisse se nedinin güncel değeri (tutulan hisse senedi sayısı çarpı güncel fiyat) ile yatırılan miktar arasındaki fark olarak hesaplanır. Test verilerinizde en az bir kayıp dahil edin.

Listeleri oluşturduktan sonra, kullanıcının her mantıksal listeyi ileri veya geri görüntülemesine olanak tanıyan bir menü gösterin (toplam dört seçenek). Her görüntüleme, uygun bir başlık ve sütun başlıkları içermelidir.

Programınızı çalıştırın ve girdi verilerinizin bir listesini ve her görüntüleme seçeneğini n bir çıktısını gönderin.

35. Proje 34'ü, stok kodunu kullanarak bir hisse senedini arama ve hisse senedi verilerini yazdırma, yeni bir hisse senedi için veri ekleme, mevcut bir hisse senedi için verileri d eğiştirme ve program sona erdiğinde verileri bir dosyaya yazma gibi aşağıdaki ek işle mleri içerecek şekilde değiştirin.
36. Bir programlama ekibine atandınız ve bu ekip, Amerika Birleşik Devletleri' ndeki şehirler hakkında nüfus verilerini içeren çift bağlı, çok bağlantılı bir l isteyi sürdürecektir. İlk mantıksal liste, 2000 nüfus v erilerine göre verileri sıralı olarak tutar. İkinci mantıksal liste, aynı verileri 1990 nüfus verilerini kullanarak tutar. Göreviniz, ekleme rutin(ler)ini yazm aktır. Programınızı tamamen test etmek için, listelerin içeriğini yazdıracak rutinler de yazmanız gerekmektedir.

Veri yapısı burada gösterilmektedir:

Mantıksal tanım:
Metropol alanı 50 karakter
Nüfus - 2000 ff f tam sayı
Nüfus - 1900 fff f tam sayı

Girdi, Tablo 5-5'teki nüfus verilerini içeren bir dosyadır.

Metropol alanı	Nüfus sayımı nüfusu	
	2000	1990
New York-Hayır. New Jersey	21,199,865	19,549,649
Los Angeles bölgesi	16,373,645	14,531,529
Chicago bölgesi	9,157,540	8,239,820
Washington-Baltimore	7,608,070	6,727,050
San Francisco bölgesi	7,039,362	6,253,311
Philadelphia-Atlantic City bölgesi	6,188,463	5,892,937
Boston bölgesi	5,819,100	5,455,403
Detroit bölgesi	5,456,428	5,187,171
Dallas-Fort Worth	5,221,801	4,037,282
Houston-Galveston bölgesi	4,669,571	3,731,131
Atlanta bölgesi	4,112,198	2,959,950
Miami-Fort Lauderdale	3,876,380	3,192,582
Seattle bölgesi	3,554,760	2,970,328
Phoenix bölgesi	3,251,876	2,238,480
Minneapolis-St. Paul	2,968,806	2,538,834
Cleveland bölgesi	2,945,831	2,859,644
San Diego bölgesi	2,813,833	2,498,016
St. Louis bölgesi	2,603,607	2,492,525
Denver bölgesi	2,581,506	1,980,140
San Juan, PR, bölgesi	2,450,292	2,270,808

TABLO 5-5 Amerika Birleşik Devletleri'ndeki Yirmi En Büyük Metropol Alanı

Programınızı doğrulamak için, her iki nüfus yılı için listeyi sırayla, hem ileri hem de geri olarak yazdırın. Kağıttan tasarruf etmek için verileri iki taraflı yazdırın, Tablo 5-6'da gösterildiği gibi.

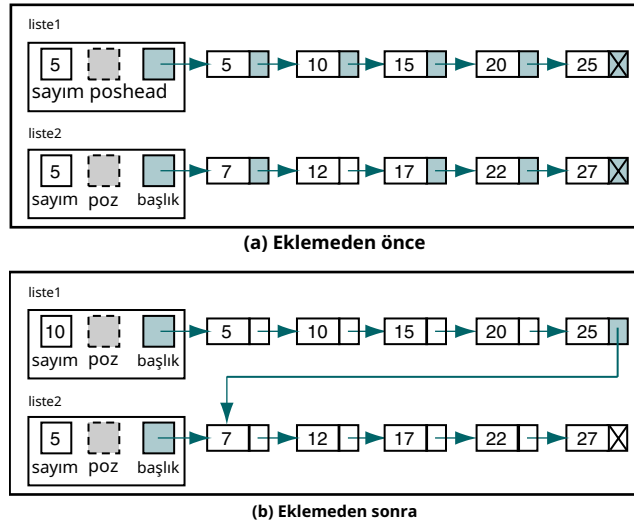
1990 Nüfus Sayımı Verileri	Nüfus	2000 Nüfus Sayımı Verileri	Nüfus
Metropol alanı01 S		Metropol bölgesi01 S	
an Juan, PR, alan02 Den	2450293	an Juan, PR, bölgesi02 De	2270808
ver alanı	581506	nver bölgesi	980140
...		...	
19 Los Angeles alanı	1637365	19 Los Angeles	1453179
20 New York--No. NJ	11199865	20 New York--Hayır. New Jersey	9549649
20 New York--No. NJ	21199665	20 New York--Hayır. NJ	1954969
19 Los Angeles Bölgesi	6373645	19 Los Angeles	4531529
...		...	
02 Denver bölgesi	2581508	02 Denver bölgesi	1980179
01 San Juan, PR, bölgesi	450292	01 San Juan, PR, bölgesi	2270808

TABLO 5-6Yazdırma Örneği

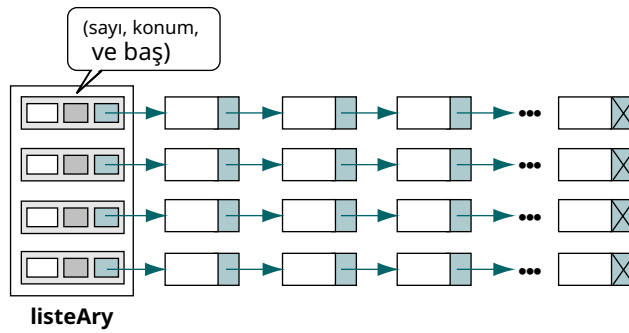
37. Proje 36'yı, listeden bir öğeyi silme yeteneğini ekleyecek şekilde değiştirin.
38. İki bağlı liste oluşturan bir program yazın (ADT kullanmayın) ve ardından ikinci listeyi birincinin sonuna ekleyin. Verilerin görsel temsili Şekil 5-37'de gösterilmiştir.
39. Bir bağlı listeler dizisi oluşturan bir program yazın (ADT kullanmayın). Her bağlı listenin verileri mevcut dosyaların bir setinden okunacak velisteye eklenecektir. Program, kullanıcıdan dosya sayısını ve adlarını sorarak çalışma zamanında belirlenen değişken sayıda dosyayı kabul etmelidir. Veri yapısı Şekil 5-38'de gösterilmiştir.

Program listeleri oluşturduktan sonra, tamamlandıklarını doğrulamak için bunları yazdırın. Programı aşağıdaki dört dosya ile test edin:

- a. 150 110 130 100 140
- b. 200 280 240 220 260
- c. 390 300 330 360
- d. 480 440 400



ŞEKİL 5-37Bağlı Listeleri Ekle



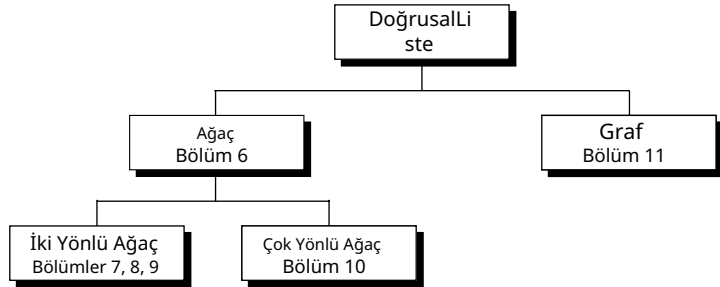
ŞEKİL 5-38Bağlı Listeler Dizisi için Yapı

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm III

Doğrusal Olmayan Listeler

Bir doğrusal olmayan liste, her bir elemanın birden fazla üst ögesi olabileceği bir listedir. Doğrusal olmayan listeler iki kategoriye ayrılabilir: ağaçlar ve grafikler. Bir ağaçta her eleman, kök hariç, yalnızca bir üst ögeye sahip olabilir; bir grafikte ise her eleman birden fazla üst ögeye sahip olabilir. Şekil III-1, bu kitapta doğrusal olmayan listelerin tartışmasının nasıl düzenlendiğini göstermektedir.



ŞEKİL III-1 Doğrusal Olmayan Listelerin Konusunun Gruplandırılması

Bilgisayar bilimlerinde iki tür ağaç tanımlanmıştır: iki yönlü ağaçlar, binay ağaçlar olarak daha iyi bilinir ve çok yönlü ağaçlar. İki yönlü bir ağaçta, her eleman en fazla iki ardıl olabilir; çok yönlü bir ağaçta ise ardıl sayısında herhangi bir sınırlama yoktur.

Doğrusal olmayan bir listede, her bir öğenin birden fazla ardılı olabilir.

Ağaçta, bir öğenin yalnızca bir öncülü olabilir.

Bir grafikte, bir öğenin bir veya daha fazla öncülü olabilir.

Uygulama

Doğrusal olmayan bir liste, doğrusal bir liste gibi, soyut bir veritipi (ADT) olarak uygulanabilir. Bu bölümde tartışılan farklılıkları uygulamak için hem dizileri hem de bağlı listeleri kullanıyoruz. Bir yığın uygulamak için bir dizi kullanıyoruz çünkü daha uygundur, göreceksiniz. Diğer yapıları için bağlı listeleri kullanıyoruz çünkü daha etkilidir.

Bölümler Kapsamı

Bu bölüm altı bölüm içermektedir. İlk beş bölüm, her bir öğenin yalnızca bir öncülü olduğu ağaçlar veya doğrusal olmayan listeleri tartışmaktadır. Son bölüm ise bir veya daha fazla öncülü olan grafikler veya doğrusal olmayan listeleri tartışmaktadır.

Bölüm 6: Ağaçlara Giriş

Bu bölümde, tüm ağaçlarla ilgili genel kavramları tartışıyoruz.

Bölüm 7: İkili Arama Ağaçları

Bu bölümde ikili arama ağacını (BST) tartışıyoruz. BST'ler, uygulanması kolay olan basit yapılardır ancak ağaç dengesiz olduğunda verimsizdir.

Bölüm 8: AVL Ağaçları

Bu bölümde AVL ağaçlarını tartışıyoruz: ikili ağacın dengeli bir varyasyonu. AVL ağaçları, BST'lerden daha verimlidir ancak daha karmaşıktır.

Bölüm 9: Yığınlar

Yığınlar, bir dizi veya bağlı liste yapısında saklanabilen ikili ağaçlardır. Bölüm 9'da yığın oluşturmayı ve iki uygulamayı tartışıyoruz: seçim algoritmaları ve öncelik kuyrukları. Bölüm 12'de üçüncü bir uygulama olan sıralamayı tartışıyoruz.

Bölüm 10: Çok Yönlü Ağaçlar

Çok yönlü ağaçlar veya sınırsız sayıda ardıl içeren ağaçlar, sözlük arama, yazım denetimi ve diğer uygulamalar için kullanılır.

Bölüm 11: Grafikler

Grafikler, karmaşık yönlendirme problemlerini çözmek için kullanılabilir, örneğin, hizmet verdikleri havaalanları arasında havayollarını tasarlamak ve yönlendirmek. Benzer şekilde, grafikler bir bilgisayar ağı üzerinden bir düğümden diğerine mesajları yönlendirmek için de kullanılabilir.

Bölüm 6

Ağaçlara Giriş

Matematikte ağaçların incelenmesi, on dokuzuncu yüzyılın ortalarında Gustav Kirchhoff'a ve birkaç yıl sonra, ağaçları cebirsel formüllerin yapısını incelemek için kullanan Arthur Cayley'e kadar izlenebilir. Cayley'in çalışması, şüphesiz, Grace Hopper'ın 1951'de aritmetik ifadeleri temsil etmek için ağaçları kullanmasına zemin hazırlamıştır. Hopper'ın çalışması, günümüzün ikili ağaç formatlarına güçlü bir benzerlik taşımaktadır.¹

Ağaçlar, cebirsel formülleri temsil etmek için bilgisayar biliminde yaygın olarak kullanılmaktadır; büyük, dinamik listeleri aramak için verimli bir yöntem olarak; ve yapay zeka sistemleri ve kodlama algoritmaları gibi çeşitli uygulamalar için. Bu bölümde, ağaçların bilgisayar bilimi uygulamaları arkasındaki temel kavramları tartışıyoruz. Ardından, sonraki dört bölümde, belirli problemler için ağaçların uygulamasını geliştiriyoruz.

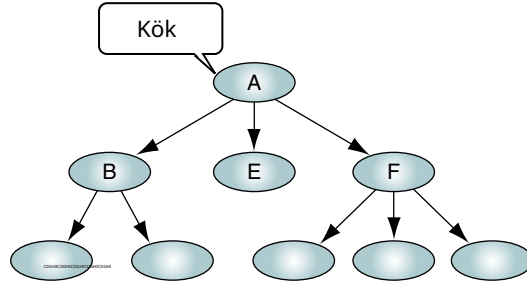
6.1 Temel Ağaç Kavramları

Bir ağaç, düğüm olarak adlandırılan sonlu bir eleman kümesi ve düğümleri bağlayan, dal olarak adlandırılan sonlu bir yönlendirilmiş çizgi kümesinden oluşur. Bir düğümle ilişkili dal sayısı, düğümün derecesidir. Dal düğüme doğru yönlendirilmişse, bu bir indegree dalıdır; dal düğümden uzak yönlendirilmişse, bu bir outdegree dalıdır. Indegree ve outdegree dallarının toplamı, düğümün derecesidir.

Bir ağaç, düğüm olarak adlandırılan sonlu bir eleman kümesi ve düğümleri bağlayan, dal olarak adlandırılan sonlu bir yönlendirilmiş çizgi kümesinden oluşur.

1. Donald E. Knuth, Bilgisayar Programlamanın Sanatı Cilt 1, Temel Algoritmalar, 2. baskı. (Reading, MA: Addison-Wesley, 1972), 405, 458.

Ağaç boş değilse, ilk düğüm kök olarak adlandırılır. Kökün indegree'i tanım gereği sıfırdır. Kök hariç, bir ağaçtaki tüm düğümlerin indegree'i tam olarak bir olmalıdır; yani, yalnızca bir öncülü olabilir. Ağaçtaki tüm düğümler, onlardan ayrılan sıfır, bir veya daha fazla dal alabilir; yani, sıfır, bir veya daha fazla (sıfır veya daha fazla ardıl) çıkış derecesine sahip olabilirler. Şekil 6-1, bir ağacın temsildir.



ŞEKİL 6-1 Ağaç

Terminoloji

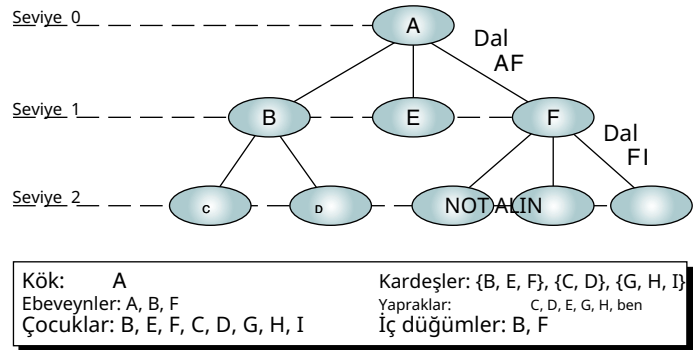
Kök dışında, bir ağacın özelliklerini tanımlamak için birçok farklı terim kullanılmaktadır. Bir yaprak, sıfır çıkış derecesine sahip herhangi bir düğümdür; yani, ardılı olmayan bir düğüm. Kök veya yaprak olmayan bir düğüm, ağacın orta kısmında bulunduğu için iç düğüm olarak bilinir.

Bir düğüm, ardıl düğümleri varsa ebeveyn olarak adlandırılır; yani, çıkış derecesi sıfırdan büyükse. Tersine, bir öncülü olan bir düğüm çocuk olarak adlandırılır. Bir çocuk düğümün indegree'i birdir. Aynı ebeveynin iki veya daha fazla düğümü kardeş olarak adlandırılır. Neyse ki, teyzeler, amcalar, yeğenler ve kuzenler hakkında endişelenmemize gerek yok. Bazı literatür *büyük ebeveyn* terimini kullanmasına rağmen, biz kullanmıyoruz. Daha genel bir terim olan *ata* terimini tercih ediyoruz. Bir ata, kökten düğüme giden yoldaki herhangi bir düğümdür. Bir soy, ebeveyn düğümünün altındaki yoldaki herhangi bir düğümdür; yani, belirli bir düğümden bir yaprağa giden yollardaki tüm düğümler, o düğümün soylarıdır. Şekil 6-2, bu terimlerin kullanımını göstermektedir.

Matematikten veya bilgisayar bilimcileri tarafından oluşturulan birkaç terim, ağaçların ve düğümlerinin özelliklerini tanımlamak için kullanılır. Bir yol, her düğümün bir sonraki ile bitişik olduğu düğümlerin bir dizisidir. Ağaçta ki her düğüme, kökten başlayarak takip edilen benzersiz bir yol ile ulaşılabilir. Şekil 6-2'de kökten yaprağa I giden yol AFİ olarak belirtilmiştir.

İki farklı dal içerir, AF ve FI .

Bir düğümün seviyesi kökten olan mesafesidir. Kök, kendisinden sıfır mesafeye sahip olduğundan, kök seviye 0'dadır. Kökün çocukları seviye 1'de, onların çocukları seviye 2'de ve bu şekilde devam eder. Şekil 6-2'de seviyeler ve kardeşler arasındaki ilişkiye dikkat edin. Kardeşler her zaman aynı seviyededir, ancak bir seviyedeki tüm düğümler mutlaka kardeş değildir. Örneğin, seviye 2'de C ve D kardeşlerdir, G, H ve I de öyledir. Ancak, D ve G kardeş değildir çünkü farklı ebeveynlere sahiptirler.

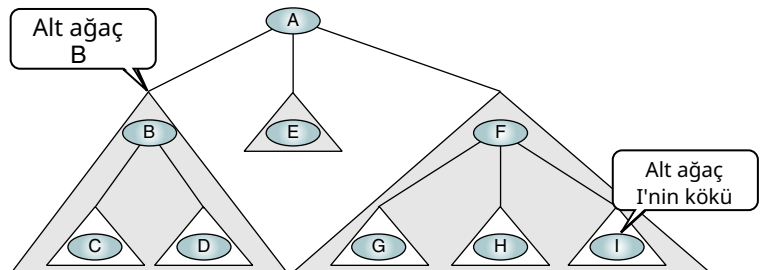


ŞEKİL 6-2 Ağaç Terminolojisi

Ağacın yüksekliği kökten en uzun yoldaki yaprağın seviyesi artı 1'dir. Tanım gereği, boş bir ağacın yüksekliği -1'dir. Şekil 6-2, 0, 1 ve 2 s eviyelerinde düğümler içerir. Yüksekliği 3'tür. Ağaç ters çizildiği için, bazı metinler ağacın yüksekliği yerine derinlik terimini kullanır.

Bir düğümün seviyesi, kökten olan uzaklığıdır. Bir ağacın yüksekliği, kökten en uzun yolda bulunan yaprağın seviyesinin 1 fazlasıdır.

Ağaç, alt ağaçlara bölünebilir. Bir alt ağaç, kök altında bulunan herhangi bir bağlı yapıdır. Alt ağaçtaki ilk düğüm, alt ağacın kökü olarak bilinir ve alt ağacı adlandırmak için kullanılır. Alt ağaçlar ayrıca daha fazla alt ağaçlara bölünebilir. Şekil 6-3'te, BCD bir alt ağaçtır, E ve FGHI de öyledir. Bu tanıma göre, bir tek düğüm de bir alt ağaçtır. Böylece, B alt ağacı iki alt ağaca, C ve D, bölünebilir ve F alt ağacı G, H ve I alt ağaçlarını içerir.



ŞEKİL 6-3 Alt Ağaçlar

Alt ağaçlar kavramı, bir ağacın özyinelemeli tanımına götürür: Bir ağaç, düğüm kümesidir ki ya: (1) boştur ya da (2) belirlenmiş bir düğüme sahiptir, bu düğüme

Kök, hiyerarşik olarak sıfır veya daha fazla alt ağaç inen, bunlar da ağaçlardır.

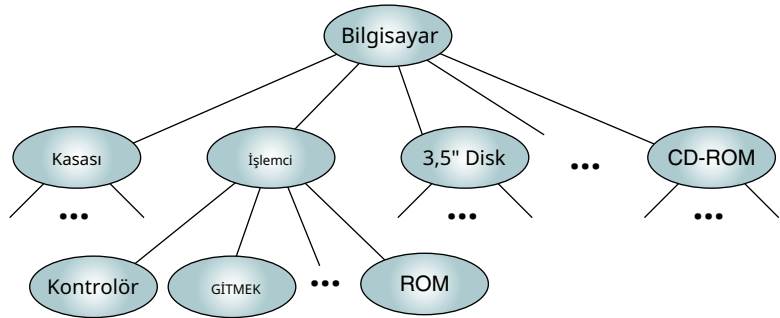
Bir ağaç, ya:

1. Boş, ya da

2. Kök denilen belirli bir düğüme sahip olup, hiyerarşik olarak sıfır veya daha fazla alt ağaç inen, bunlar da ağaçlardır.

Kullanıcı Temsili

Ağaçlar için üç farklı kullanıcı temsili vardır. Birincisi, temelde ağaçları şekille rimizde temsil etmek için kullandığımız notasyon olan organizasyon şeması formatıdır. Bu notasyon için kullandığımız terim genel ağaçtır. Bir bilgisayarın bileşenlerinin genel ağaç temsili Şekil 6-4'te gösterilmektedir; bu, Bölüm 6.3'te tartışılmaktadır.



ŞEKİL 6-4 Genel Ağaç Olarak Bilgisayar Parçaları Listesi

İkinci temsil notasyonu girintili listedir. Bu, en çok bir parçalar listesinin bir öğenin montaj yapısını temsil ettiği malzeme listesi sistemlerinde kullanılır. Şekil 6-4'te bir bilgisayar sisteminin bileşenlerinin grafiksel temsili, bir bilgisayarın çeşitli bileşenleri arasındaki ilişkiyi açıkça göstermektedir, ancak grafiksel temsiller bir veritabanı sisteminden kolayca üretilmez. Bu nedenle, aynı bilgiyi metin tabanlı bir parçalar listesi formatı kullanarak göstermek için malzeme listesi formatı oluşturulmuştur. Bir malzeme listesinde, her montaj bileşeni montajının altında girintili olarak gösterilmektedir. Bazı malzeme listeleri, her bileşenin seviye numarasını bile göstermektedir. Bir malzeme listesi, hangi bileşenlerin her montaj altında monte edildiğini gösterdiği için bazen 'goezinta' (içine girer) listesi olarak adlandırılmaktadır. Tablo 6-1, girintili parçalar listesi formatında bilgisayar malzeme listesini göstermektedir.

Tanıdık olmanız gereken başka bir yaygın malzeme listesi var. Yapılandırılmış bir program yazdığınızda, tüm program ilgili işlevlerin bir montajı olarak düşünülebilir. Yapı şemanız, işlevler arasındaki ilişkiyi temsil eden genel bir ağaçtır.

Parça numarası	Açıklama
301	Bilgisayar
301-1	Ka
...	sa
301-2	...
301-2-1	İşlemci
301-2-2	Kontrolcü
...	ALU
301-2-9	...
301-3	ROM
...	3,5" Disk
301-9	...
...	CD-ROM
...	...

TABLO 6-1 Bilgisayar Malzeme Listesi

Üçüncü kullanıcı formatı parantez içindeki listelemedir. Bu format, c ebirsel ifadelerle birlikte kullanılır. Bir ağaç parantez notasyonunda temsil edildiğinde, her açık parantez yeni bir seviyenin başlangıcını; her kapalı parantez mevcut seviyeyi tamamlar ve ağaçta bir seviye yukarı çıkar. Şekil 6-1'de gösterilen ağacı düşünün. Parantez notasyonu

A (B (C D) E F (G H I))

Genel bir ağacı parantez notasyonuna dönüştürmek için, Algoritma 6-1'deki kodu kullanıyoruz.

ALGORİTMA 6-1 Genel Ağacı Parantez Notasyonuna Dönüştür

```

Algoritma ConvertToParen (root, output)
Genel bir ağacı parantezli notasyona dönüştür.
  Ön root, bir ağaç düğümüne işaret eden bir işaretçidir
  Sonuç, parantezli notasyonu içerir
1 root'u çıktıya yerleştir
2 eğer (root bir ebeveyn ise)
  1 Çıktıya bir açık parantez yerleştir
  2 ParantezeDönüştür (kökün ilk çocuğu)
  3 döngü (daha fazla kardeş)
    1 ParantezeDönüştür (kökün bir sonraki çocuğu)

```

devam ediyor

ALGORİTMA 6-1 Genel Ağacı Parantez Notasyonuna Dönüştür (devam ediyor)

```

4   döngüyü bitir
5   Çıktıya kapanış parantezini yerleştir
3   eğer son ise bitir
4   geri dön
SON ConvertToParen

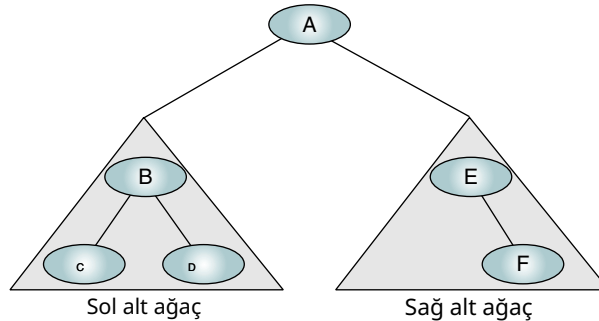
```

Bu algoritmayı Şekil 6-1 ile kullanarak, kökü (A) çıktıya yerleştirerek başlıyoruz. Çünkü A bir ebeveyn, çıktıda bir açılış parantezi ekliyoruz ve ardından alt ağacı B işleyerek önce kökünün (B) çıktıya yerleştiriyoruz. Yine, çünkü B bir ebeveyn, çıktıda bir açılış parantezi yerleştiriyoruz ve ParantezDönüştür çağrısını yineleyerek C'yi ekliyoruz. Ancak bu sefer, kökün çocukları yok. Bu nedenle algoritma sona erer ve 2.4. ifadeye geri döner. D 'yi çıktıya yerleştirdikten sonra, döngü ifadesine geri dönüyoruz ve daha fazla kardeş olmadığını keşfediyoruz. Şimdi çıktıda bir kapanış parantezi yerleştiriyoruz ve 2.4. if adeye geri dönerek B'nin kardeşlerini çıktıya yerleştirmeye devam ediyoruz.

Çocuklarım tamamen işlendiğinde, bir kapanış parantezi ekliyoruz (2.5), bu da bizi bir seviye yukarı çıkararak işlemenin tamamlanmasını sağlar. B'nin kardeşleri ve bir kapanış parantezinin eklenmesi. İşlemenin doğru yapıldığında, parantezlerin dengeli olduğunu unutmayın. Eğer dengeli değilse , dönüşümde bir hata yapmışsınız demektir.

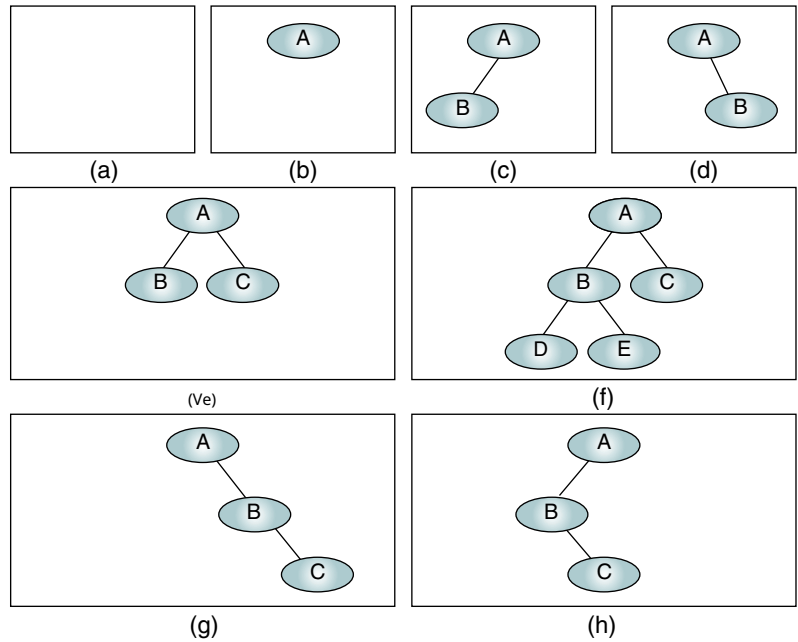
6.2 İkili Ağaçlar

Bir ikili ağaç, hiçbir düğümün iki alt ağacından fazlasına sahip olamayacağı bir ağaçtır ; bir düğüm için maksimum dış derece iki olarak belirlenmiştir. Diğer bir deyişle, bir düğüm sıfır, bir veya iki alt ağaca sahip olabilir. Bu alt ağaçlar, sol alt ağaç ve sağ alt ağaç olarak adlandırılır. Şekil 6-5, iki alt ağacı olan bir ikili ağacı göstermektedir. Her alt ağaç kendisi de bir ikili ağaçtır.



ŞEKİL 6-5 İkili Ağaç

İkili ağaçların yapısını daha iyi anlamak için Şekil 6-6'yı inceleyin.



ŞEKİL 6-6 İkili Ağaçlar Koleksiyonu

Şekil 6-6, ilki boş bir ağaç olan sekiz ikili ağaç içermektedir. Aboş ağaç , düğümü olmayan bir ağaçtır; bu, Şekil 6-6(a)'de gösterilmiştir. Bu şekli incelerken, simetrisinin bir ağaç gerekliliği olmadığını unutmayın.

Bir ikili ağaçtaki bir düğümün en fazla iki alt ağacı olabilir.

Özellikler

Şimdi ikili ağaçları genel ağaçlardan ayıran birkaç özellik tanımlıyoruz.

İkili Ağaçların Yüksekliği

İkili ağaçların yüksekliği matematiksel olarak tahmin edilebilir.

Maksimum Yükseklik

Bir ikili ağaçta N düğüm depolamamız gerektiği göz önüne alındığında, maksimum yükseklik,

$H_{\max}$ şudur

$$H_{\max} = N$$

Örnek Üç düğümün bir ikili ağaçta saklanması verildiğinde, maksimum yükseklik nedir? Bu örnekte N 3'tür. Bu nedenle, maksimum yükseklik 3'tür. Maksimum yüksekliği 3 olan dört farklı ağaç çizilebilir. Maksimum yüksekliğe sahip bir ağaç nadirdir. Tüm ağaçtaki düğümlerin yalnızca bir ardılı olduğunda meydana gelir, bu da Şekil 6-6(g) ve Şekil 6-6(h)'de gösterilmiştir.

Minimum Yükseklik

Ağacın minimum yüksekliği, $H_{\min}$ aşağıdaki formülle belirlenir:

$$H_{\min} = \lfloor \log_2 N \rfloor + 1$$

Örnek İkili ağaçta saklanacak üç düğüm verildiğinde, minimum yükseklik nedir? Yine, N 3'tür. Bu nedenle, minimum yükseklik 2'dir.

Minimum Düğümler. Hesaplamayı tersine çevirebiliriz ve belirli bir yükseklikte ki bir ağaçtaki minimum düğüm sayısını belirleyebiliriz. İkili ağacın yüksekliği verildiğinde, H , ağaçtaki minimum düğüm sayısı şu şekilde verilir:

$$N_{\min} = 2^H$$

Örnek Yüksekliği 3 olan bir ağaç verildiğinde, saklanabilecek minimum düğüm sayısı nedir? $N_{\min}$ formülünü kullanarak, cevap 2^3 , yani 8 düğümdür. Bu durum Şekil 6-6(g) ve 6-6(h)'de görülmektedir.

Maksimum Düğümler

Maksimum düğüm sayısı formülü, her düğümün yalnızca iki soyundan oluşabileceği gerçeğinden türetilmiştir. İkili ağacın yüksekliği verildiğinde, H , ağaçtaki maksimum düğüm sayısı şu şekilde verilir:

$$N_{\max} = 2^{H+1} - 1$$

Örnek Yüksekliği 3 olan bir ağaç verildiğinde, saklanabilecek maksimum düğüm sayısı nedir? $N_{\max}$ 7'dir.

Denge

Kökten bir düğümün uzaklığı, ne kadar verimli bir şekilde bulunabileceğini belirler. Bir ağaçtaki herhangi bir düğümün çocuklarına yalnızca bir dal yolunu takip ederek, istenilen düğüme giden dalı takip ederek erişilebilir. Seviye 1'deki düğümler, kökün çocuklarıdır ve yalnızca bir dalı takip ederek erişilebilir. Aynı şekilde, bir ağacın 2. seviyesindeki düğümlere yalnızca kökten iki dal izleyerek erişilebilir. Bu nedenle, ağacın ne kadar kısa olduğu, istenen düğümü ağaçta bulmayı o kadar kolaylaştırır.

Bu kavram, bize ikili bir ağacın çok önemli bir özelliğine götürüyor—, onun dengesi. Bir ağacın dengeli olup olmadığını belirlemek için, denge faktörünü hesaplarız. İkili bir ağacın denge faktörü, sol ve sağ alt ağaçları arasındaki yükseklik farkıdır. Sol alt ağacın yüksekliğini H_L ve sağ alt ağacın yüksekliğini H_R olarak tanımlarsak, ağacın denge faktörü, B , aşağıdaki formülle belirlenir:

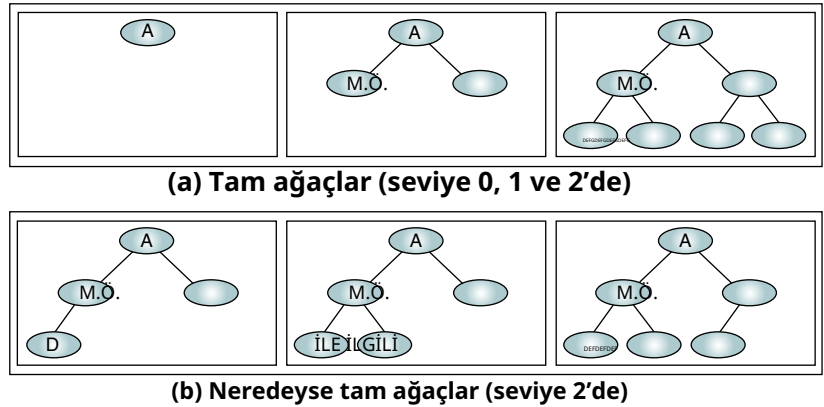
$$B = H_L - H_R$$

Bu formülü kullanarak, Şekil 6-6'daki sekiz ağacın dengeleri (a) 0 tanım gereği, (b) 0, (c) 1, (d) -1, (e) 0, (f) 1, (g) -2 ve (h) 2'dir.

Bir dengeli ikili ağaçta, alt ağaçlarının yüksekliği en fazla bir ile farklılık gösterir (denge faktörü -1, 0 veya +1'dir) ve alt ağaçları da dengelidir. Gördüğümüz gibi, bu tanım Adelson-Veskii ve Landis tarafından AVL ağacının tanımında oluşturulmuştur.

Tamamlanmış ve Neredeyse Tamamlanmış İkili Ağaçlar

Bir tam ağaç yüksekliği için maksimum giriş sayısına sahiptir (formüle bakın $N_{\max}$ "İkili Ağaçların Yüksekliği" bölümünde). Maksimum sayı, son seviye dolu olduğunda ulaşılır (Şekil 6-7'ye bakın). Bir ağaç, düğümleri için minimum yüksekliğe sahip olduğunda neredeyse tam olarak kabul edilir (formüle bakın $H_{\min}$) ve son seviyedeki tüm düğümler solda bulunur. Tam ve neredeyse tam ağaçlar Şekil 6-7'de gösterilmiştir.



ŞEKİL 6-7 Tam ve Neredeyse Tam Ağaçlar

İkili Ağaç Geçişleri

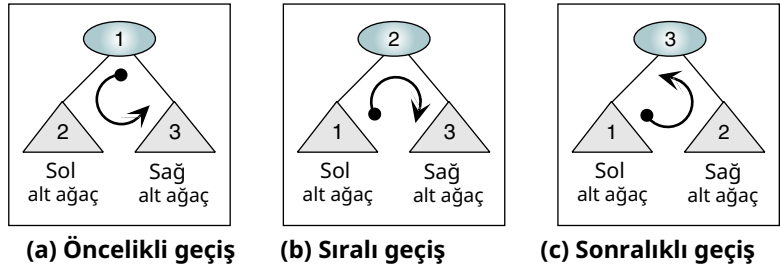
Bir ikili ağaç geçişi ağacın her düğümünün önceden belirlenmiş bir sırada bir kez ve yalnızca bir kez işlenmesini gerektirir. Geçiş sırasına iki genel yaklaşım derinlik öncelikli ve genişlik öncelikli olarak adlandırılır. Derinlik öncelikli geçişte,

işleme, kökten başlayarak bir çocuğa ve o ilk çocuğun en uzak soyunda n geçerek ikinci bir çocuğu işlemeye başlamadan önce devam eder. Diğer bir deyişle, derinlik öncelikli geçişte, bir çocuğun tüm soyuları işlenmeden bir sonraki çocuğa geçmeyiz.

Genişlik öncelikli geçişte, işleme kökten başlayarak yatay olarak tüm çocuklarına, ardından çocuklarının çocuklarına ve bu şekilde devam ederek tüm düğümler işlenene kadar devam eder. Diğer bir deyişle, genişlik öncelikli geçişte, her seviye tamamen işlenmeden bir sonraki seviyeye geçilmez.

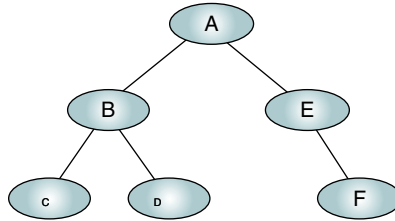
Derinlik Öncelikli Geçişler

Bir ikili ağacın bir kök, bir sol alt ağaç ve bir sağ alt ağaçtan oluştuğu göz önüne alındığında, altı farklı derinlik öncelikli geçiş dizisi tanımlayabiliriz. Bilgisayar bilimcileri, bu dizilerden üçünü literatürde standart isimlerle adlandırmışlardır; diğer üçü isimsizdir ancak kolayca türetilir. Standart geçişler Şekil 6-8'de gösterilmiştir.



ŞEKİL 6-8 İkili Ağaç Geçişleri

Geçişlerin geleneksel tanımı, kök için düğüm(N), sol alt ağaç için sol (L) ve sağ alt ağaç için sağ (R) kullanır. Bir ikili ağaç için farklı geçiş dizilerini göstermek için, Şekil 6-9'u kullanıyoruz.



ŞEKİL 6-9 Geçişler için İkili Ağaç

Öncelikli Geçiş (NLR)

Öncelikli geçişte, kök düğümü önce işlenir, ardından sol alt-ağaç ve sonra sağ alt-ağaç gelir. Adını, önceden gitmek anlamına gelen Latince ön ek *pre*'den alır. Böylece, kök alt-ağaçlardan önce gelir.

Ön sipariş gezintisinde, kök önce işlenir, ardından alt ağaçları.

Ağaçların özyinelemeli özellikleri göz önüne alındığında, ağaç geçişlerini özyinelemeli olarak uygulamak son derece doğaldır. Önce kökü işleriz, sonra sol alt-ağacı, ardından sağ alt-ağacı işleriz. Sol alt-ağaç da özyinelemeli olarak işlenir, sağ alt-ağaç da öyle. Öncelikli geçiş için kod, Algoritma 6-2'de gösterilmiştir.

ALGORİTMA 6-2 İkili Ağaçta Öncelikli Geçiş

```
Algoritma preOrder (kök)
Bir ikili ağacı düğüm-sol-sağ sırasına göre gez.
  ÖN kök, bir ağacın veya alt ağacın giriş düğümüdür
  Son her düğüm sırasıyla işlenmiştir
1 eğer (kök null değilse)
  1 işle (kök)
  2 preOrder (solAltAğaç)
  3 preOrder (sağAltAğaç)
2 eğer son ise bitir
son preOrder
```

Şekil 6-9, her düğümün adlandırıldığı bir ikili ağaç içermektedir. Öncelikli geçiş için işlemesirası bu ağacı şu şekilde işler: Önce kök *A* işlenir. Kökten sonra, sol alt ağaç işlenir. Sol alt ağacı işlemek için önce kökünün, *B*, sonra sol alt ağacını ve sağ alt ağacını sırasıyla işleriz. *B*'nin sol ve sağ alt ağaçları sırasıyla işlendiğinde, *A*'nın sağ alt ağacını, *E*, işlemeye hazırız. Alt ağaç *E*'yi işlemek için önce kökünün ve sonra sol alt ağacını ve sağ alt ağacını işleriz. Sol alt ağaç olmadığı için, hemen sağ alt ağaçla devam ederiz, bu da ağacı tamamlar.

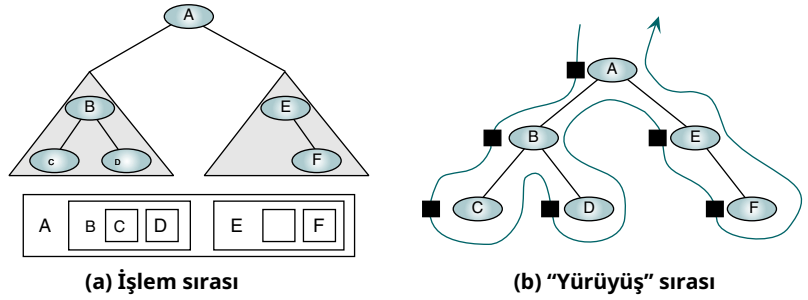
Şekil 6-10, ağacın geçişini görselleştirmenin başka bir yolunu göstermektedir. Ağacın etrafında yürüdüğümüzü hayal edin, kökün solundan başlayarak düğümlere mümkün olduğunca yakın kalmaya çalışıyoruz. Öncelikli geçişte, düğümle ilk karşılaştığımızda (düğümün solunda) düğümü işleriz. Bu, düğümün solunda bir siyah kutu olarak gösterilmektedir. Yol, ağacın etrafında tamamen bir rota izleyen bir çizgi olarak gösterilmektedir ve köke geri dönmektedir.

Şekil 6-11, ağacın özyinelemeli algoritmik geçişini göstermektedir. İlk çağrı, ağacın kökünü, *A*, işler. Daha sonra, alt ağaç *B*'nin kökünü işlemek için yine kendisini özyinelemeli olarak çağırır, bu Şekil 6-11(b)'de gösterilmektedir. Üçüncü çağrı, Şekil 6-11(c)'de gösterilen, *C* düğümünü işler, bu aynı zamanda alt ağaç *C*'dir. Bu noktada, null işaretçisi ile öncelikli çağrısı yaparız, bu da alt ağaç *C*'ye hemen geri dönmemizden olur.

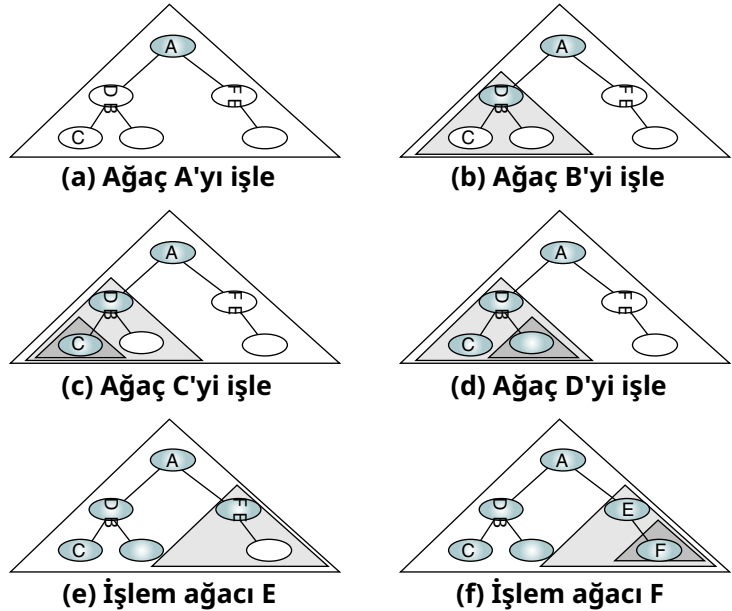
sağ alt ağacını işlemek için. Çünkü C'nin sağ alt ağacı da null olduğundan, düğüm B'ye geri dönüyoruz, böylece sağ ağacını, D, Şekil 6-11(d) de işleyebiliriz. Düğüm D'yi işledikten sonra, bir tanesi D'nin null sol işaretçisi ve diğer null sağ işaretçisi ile olmak üzere iki çağrı daha yapıyoruz. Çünkü alt ağaç B artık tamamen işlenmiştir, ağacın köküne geri dönüyoruz ve sağ alt ağacını, E, Şekil 6-11(e) de işliyoruz.

Bir E'nin null sol alt ağacına yapılan çağrının ardından, Şekil 6-11(f)'de E'nin sağ alt ağacı F'ye çağrıda bulunuyoruz.

Ağaç bu noktada tamamen işlenmiş olsa da, hala iki tane daha çağrıımız var: biri F'nin null sol alt ağacına ve diğeri null sağ alt ağacına. Artık ağaçtan geri çıkabiliriz, önce E'ye ve sonra A'ya dönerek, ağacın gezintisini tamamlıyoruz.



ŞEKİL 6-10 Öncelikli Gezinme—A B C D E F



ŞEKİL 6-11 İkili Ağaçta Algoritmik Geçiş

Sıralı Geçiş (LNR)

inorder traversal önce sol alt ağacı, sonra kökü ve en son sağ alt ağacı işler . ön ekinin anlamı, kökün alt ağaçların arasında işlendiğidir. Algoritmayı yine rekürsif olarak uyguluyoruz, bu da Algoritma 6-3'te gösterilmiştir.

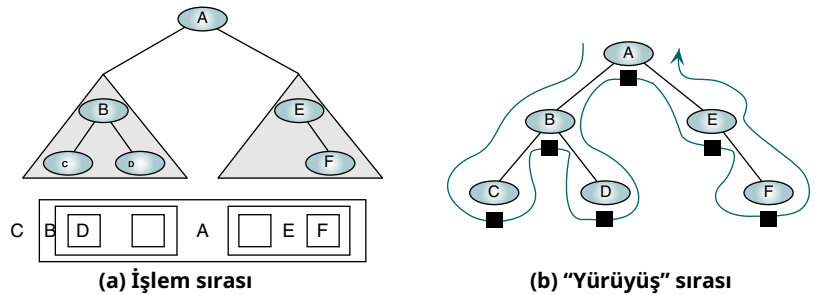
ALGORİTMA 6-3 İkili Ağaçta Inorder Traversal

```

Algoritma inOrder (kök)
Bir ikili ağacı sol-düğüm-sağ sırasına göre geçin.
  Ön kök, bir ağacın veya alt ağacın giriş düğümüdür
  Son her düğüm sırasıyla işlenmiştir
1  eğer (kök null değilse)
    1  inOrder (solAltAğaç)
    2  işle (kök)
    3  inOrder (sağAltAğaç)
2  eğer son ise bitir
son inOrder

```

Sol alt ağacın önce işlenmesi gerektiğinden, herhangi bir düğümü işlemeye başlamadan önce kökten en sol yaprak düğümüne kadar iz sürüyoruz. En sol alt ağacı işledikten sonra, C, ebeveyn düğümünü, B, işliyoruz. Artık sağ alt ağacı, D, işlemeye hazırız. D işlemesi, kökün sol alt ağacının işlenmesini tamamlar ve şimdi kökü, A, işlemek için hazırız, ardından sağ alt ağacını işliyoruz. Sağ alt ağacın, E, sol çocuğu olmadığından, kökünü hemen işleyebiliriz, ardından sağ alt ağacını, F, işliyoruz. Inorder işlemesi için tam sıra Şekil 6-12'de gösterilmiştir.



ŞEKİL 6-12 Sıralı Geçiş—C B D A E F

Ağaçta sıralı sırayla dolaşmak için aynı yolu izliyoruz ama her düğümü ikinci kez (düğümün alt kısmı) karşıladığımızda işliyoruz. Bu işlem yolu Şekil 6-12(b)'de gösterilmiştir.

Sıralı geçişte, kök alt ağaçlarının arasında işlenir.

Posta Siparişi Geçişi (LRN)

Standart gezintilerin sonuncusu *postorder traversal*'dır. Kök düğümü, sol ve sağ alt ağaçlar işlendikten sonra (*post*) işler. En soldaki yaprağı bulup işlemeye başlar. Ardından, sağ kardeşini, alt ağaçlarıyla birlikte (varsa) işler. Son olarak, kök düğümünü işler.

Postorder geçişinde, kök alt ağaçlarından sonra işlenir.

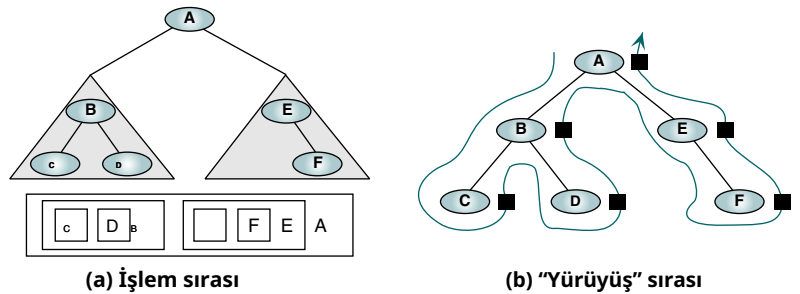
Özinelemeli postorder traversal mantığı Algoritma 6-4'te gösterilmiştir.

ALGORİTMA 6-4 İkili Ağaçta Postorder Traversal

Algoritma postOrder (kök)
İkili bir ağacı sol-sağ-düğüm sırasına göre geçin.
Ön kök, bir ağacın veya alt ağacın giriş düğümüdür
Son her düğüm sırasıyla işlenmiştir

- 1 eğer (kök null değilse)
 - 1 postOrder (sol alt ağaç)
 - 2 postOrder (sağ alt ağaç)
 - 3 işle (kök)
- 2 eğer son ise bitir
gönderiyi bitirSipariş

Postorder traversal için ağaç yürüyüşünde, işleme bloğunu düğümün sağ tarafına taşıyoruz, böylece düğümle üçüncü kez karşılaştığımızda onu işleriz. Postorder traversal Şekil 6-13'te gösterilmiştir. Üç yürüyüşte de aynı yolu izlediğimizi unutmayın; yalnızca işlemenin zamanı değişti.



ŞEKİL 6-13 Postorder Traversal—C D B F E A

Genişlik öncelikli Geçişler

İkili bir ağacın genişlik öncelikli geçişinde, bir düğümün tüm çocuklarını işledikten sonra bir sonraki seviyeye geçiyoruz. Diğer bir deyişle, seviye de n bir kök verildiğinde, seviyede n tüm düğümleri işledikten sonra

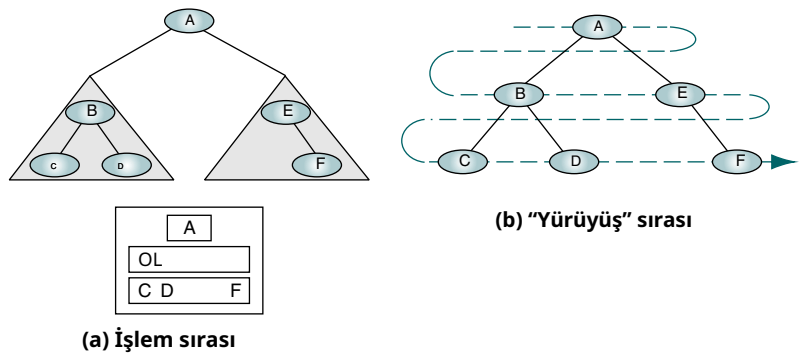
seviye $n + 1$. Bir ağacı derinlik öncelikli sırayla gezmek için bir yığın kullandık. (Unutmam ki, özyineleme bir yığın kullanır.) Bir ağacı genişlik öncelikli sırayla gezmek için bir kuyruk kullanırız. İkili ağacımızın genişlik öncelikli gezintisi için sahte kod, Algoritma 6-5'te gösterilmiştir.

ALGORİTMA 6-5 Genişlik öncelikli Ağaç Gezintisi

```

Algoritma breadthFirst (kök)
Ağaç genişlik öncelikli geçiş kullanılarak işlenir.
  Ön kök işlenecek düğümdür
  Son ağaç işlenmiştir
1 mevcutDüğümü kök olarak ayarla
2 oluşturKuyruk (bfQueue)
3 döngü (mevcutDüğüm null değil)
  1 işle (mevcutDüğüm)
  2 eğer (sol alt ağaç null değilse)
    1 kuyrukla (bfQueue, sol alt ağaç)
  3 eğer son ise bitir
  4 eğer (sağ alt ağaç null değilse)
    1 kuyruk (bfQueue, sağ alt ağaç)
  5 eğer son ise bitir
  6 eğer (boşQueue(bfQueue) değilse)
    1 mevcutDüğümü kuyruktan çıkar (bfQueue) olarak ayarla
  7 aksi takdirde
    1 mevcutDüğümü null olarak ayarla
  8 eğer son ise bitir
4 döngüyü bitir
5 yok etKuyruk (bfQueue)
bitiş genişliktenİlk
  
```

Derinlik öncelikli gezintiler gibi, gezintiyi bir yürüyüşle izleyebiliriz. Bu sefer, yürüyüş yatay bir şekilde ilerler, önce kök seviyesinden, sonra seviye 1'den, sonra seviye 2'den ve ağaç tamamen gezilene kadar devam eder. Genişlik öncelikli gezinti Şekil 6-14'te gösterilmiştir.



ŞEKİL 6-14 Genişlik Öncelikli Geçiş

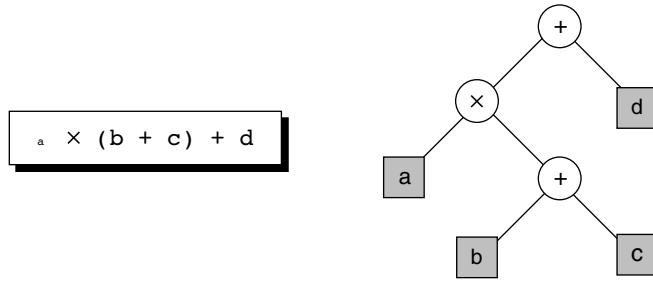
İfade Ağaçları

İkili ağaçların ilginç bir uygulaması ifade ağaçlarıdır. Bir ifade belirli kurallara uyan bir dizi simgedir. Bir simge ya bir operand ya da bir operatör olabilir. Bu tartışmada yalnızca operand-operatör-operand biçimindeki ikili aritmetik operatörleri ele alıyoruz. Standart aritmetik operatörler $+$, $-$, $*$, ve $/$.

Bir ifade ağacı aşağıdaki özelliklere sahip bir ikili ağaçtır:

- Her yaprak bir operandır.
- Kök ve iç düğümler operatörlerdir.
- Alt ağaçlar, kökü bir operatör olan alt ifadeleridir.

Şekil 6-15, bir infiks ifadesi ve onun ifade ağacını içermektedir.



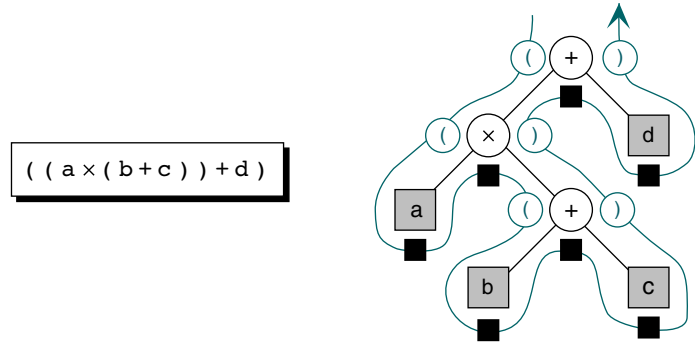
ŞEKİL 6-15 İnfiks İfadesi ve İfade Ağacı

Bir ifade ağacı için, üç standart derinlik öncelikli geçiş, üç farklı ifade formatını temsil eder: infiks, postfix ve prefix. Inorder geçiş infiks ifadesini, postorder geçiş postfix ifadesini ve preorder geçiş prefix ifadesini üretir.

İnfiks Geçiş

Bir ifade ağacının infiks geçişini göstermek için, ağacı gezip ifadeyi yazdıran bir algoritma yazalım. İnfiks ifade ağacını yazdığımızda, her ifadenin başına bir açılış parantezi ve sonuna bir kapanış parantezi eklemeliyiz. Ağacın kökü ve her bir alt ağacı bir alt ifadeyi temsil ettiğinden, bir ağaç veya alt ağaç başlattığımızda açılış parantezini ve tüm çocuklarını işlediğimizde kapanış parantezini yazdırırız. Şekil 6-16, inorder geçiş kullanarak ağaçta yürüdüğümüzde parantezlerin yerleşimini göstermektedir.

İnfiks ifade ağacı geçişi için psödokod, Algoritma 6-6'da gösterilmiştir.



ŞEKİL 6-16 Bir İfade Ağacının Infix Geçişi

ALGORİTMA 6-6 Infix İfade Ağacı Geçişi

Algoritma infix (ağaç)
 Bir ifade ağacı için infix ifadesini yazdırın.
 Ön ağaç bir ifade ağacına işaretçidir
 Son infix ifadesi yazdırılmıştır

- 1 eğer (ağaç boş değilse)
 - 1 eğer (ağaç token bir operand ise)
 - 1 yazdır (ağaç-token)
 - 2 aksi takdirde
 - 1 yazdır (açık parantez)
 - 2 infix (ağaç sol alt ağaç)
 - 3 print (ağaç token)
 - 4 infix (ağaç sağ alt ağaç)
 - 5 print (kapalı parantez)
 - 3 eğer son ise bitir
- 2 eğer son ise bitir

son infix

Postfix Geçişi

Bir ifadenin postfix geçişi, herhangi birikili ağacın temel postorder geçişini kullanır. Parantez gerektirmediğini unutmayın. Pseudocode Algoritma 6-7' de gösterilmiştir.

ALGORİTMA 6-7 Bir İfade Ağacının Postfix Geçişi

Algoritma postfix (ağaç)
 Bir ifade ağacı için postfix ifadesini yazdırın.
 Pre ağaç bir ifade ağacına işaret eder
 Post postfix ifadesi yazdırılmıştır

- 1 eğer (ağaç boş değilse)

devam ediyor

ALGORİTMA 6-7 Postfix İfadesi Ağacı Geçişi (devam ediyor)

```

1 postfix (ağaç sol alt ağaç)
2 postfix (ağaç sağ alt ağaç)
3 print (ağaç token)
2 eğer son ise bitir
bitiş son ek

```

Önek Geçişi

Son ifade ağacı geçişi önek geçiştir. Standart pre-order ağaç geçişini kullanır. Yine, parantezlere gerek yoktur. Pseudocode Algoritma 6-8' de gösterilmiştir.

ALGORİTMA 6-8 Bir İfade Ağacının Ön Ek Geçişi

```

Algoritma prefix (ağaç)
Bir ifade ağacı için prefix ifadesini yazdırın.
Pre ağaç bir ifade ağacına işaretçidir
Post prefix ifadesi yazdırılmıştır
1 eğer (ağaç boş değilse)
1 print (ağaç token)
2 prefix (ağaç sol alt ağaç)
3 prefix (ağaç sağ alt ağaç)
2 eğer son ise bitir
son ön ek

```

Huffman Kodu

Amerikan Standart Bilgi Değişim Kodu (ASCII) sabit-uzunlukta bir koddur; yani, karakter uzunluğu değişmez. Her ASCII karakteri 7 bittten oluşur.² Karakter E, karakter Z'den daha sık geçmesine rağmen, her ikisi de aynı sayıda bit atanmıştır. Bu tutarlılık, her karakterin maksimum bit sayısını kullandığı anlamına gelir.

Huffman kodu, diğer yandan, karakter depolamayı daha verimli hale getirir. Huffman kodunda, daha sıkgeçen karakterlere daha kısa kodlar, daha az sık geçenlere ise daha uzun kodlar atanır. Örneğin, Ev e T, İngilizce dilinde sık geçen iki karakter, her birine bir bit atanabilir. A, O, R ve N, sık geçen ancak E ve T'den daha azsık geçen karakterlerdir ve her birine iki bit atanabilir. S, U, I, D, M, C, v ve G, bir sonraki en sık geçenlerdir ve her birine üç bit atanabilir, vebunun gibi devam eder. Belirli bir metinde, yalnızca bazı karakterler maksimumbit uzunluğuna ihtiyaç duyar. Bir ağ iletiminde kullanıldığında, Huffman kodlu karakterler iletildiğinde,iletimin toplam uzunluğu daha kısa olur.

2. ASCII kodu 8 bitlik bir baytta saklandığında, yalnızca ilk 128 değer ASCII olarak kabul edilir. İkinci 128 karakter özel karakterler ve grafikler için kullanılır ve "genişletilmiş ASCII" olarak kabul edilir.

sabit uzunlukta kodlama yerine; Huffman kodu bu nedenle popüler bir veri sıkıştırma algoritmasıdır.

Her karaktere bit desenleri atamadan önce, her karaktere kullanım sıklığına dayalı bir ağırlık atarız. Örneğimizde, bir metindeki E karakteri nin sıklığının %15 ve T karakterinin sıklığının %12 olduğunu varsayıyoruz. Tablo 6-2, karakterlerin ağırlıklı değerlerini göstermektedir.

Karakter	Ağırlık	Karakter	Ağırlık	Karakter	Ağırlık
ACDEG	10	IKMNO	4	RSTU	7
	3		2		5
	4		3		12
	15		6		5
	2		8		

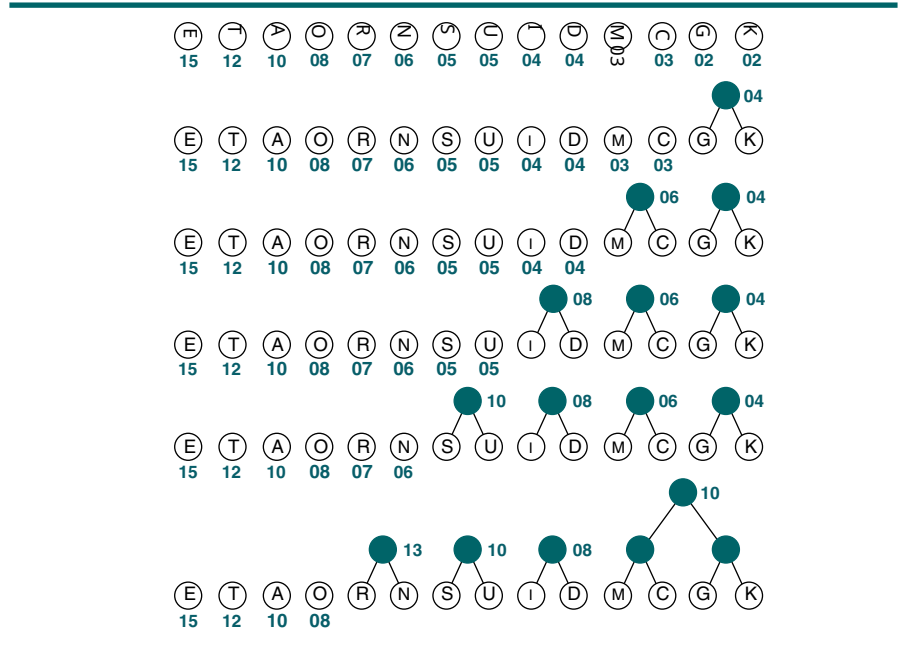
TABLO 6-2Huffman Kodu Örneği için Karakter Ağırlıkları

Her karakterin ağırlığını belirledikten sonra, bu değerlere dayalı bir ağaç inşa ederiz. Bu ağacı oluşturma süreci Şekil 6-17'den 6-20'ye kadar gösterilmektedir. Üç temel adımı takip eder:

1. Öncelikle, tüm karakter setini en yüksekten en düşüğe (veya tersine) sıklıkla rına göre sıralanmış bir satıra düzenleriz. Her karakter artık bir ağacın yaprak seviyesindeki bir düğümdür.
2. Sonra, en küçük birleşik sıklık ağırlıklarına sahip iki düğümü bulur ve bunları bir üçüncü düğüm oluşturacak şekilde birleştiririz, bu da basit bir iki seviyeli ağaç oluşturur. Yeni düğümün ağırlığı, orijinal iki düğümün birleşik ağırlıklarıdır. Bu düğüm, yapraklardan bir seviye yukarıda, diğer düğümlerle birleştirilmek için uygundur. Unutmayın, seçilen iki düğümün ağırlıklarının toplamı, diğer olasılıkların herhangi birinin kombinasyonundan daha küçük olmalıdır.
3. Adım 2'yi, her seviyedeki tüm düğümler tek bir ağaçta birleştirilene kadar tekrar ederiz.

Şekil 6-17, bu sürecin ilk kısmını göstermektedir. Şeklin ilk satırı, değer sırasına göre azalan düzende düzenlenmiş orijinal karakterleri temsil eden yaprak seviyesindeki düğümleri gösterir; ardından, adım 2'de açıklandığı gibi, en küçük değerlere sahip iki düğümü bulup birleştiririz. Bu adım ikinci satırda gösterilmektedir. Gördüğünüz gibi, bu süreç yeni bir düğümün (katı bir daire ile temsil edilen) oluşturulmasıyla sonuçlanır. Bu yeni düğümün frekans değeri (ağırlığı), iki düğümün ağırlıklarının toplamıdır. Üçüncü satırda, iki düğümü daha birleştiririz, ve bu şekilde devam ederiz.

Altıncı satırda, en düşük değerlere sahip düğümler karakterlerin kendileri arasında değil, karakterlerden bir seviye yukarıda bulunur. Onları yapraklardan iki seviye yukarıda bir düğümden birleştiririz.



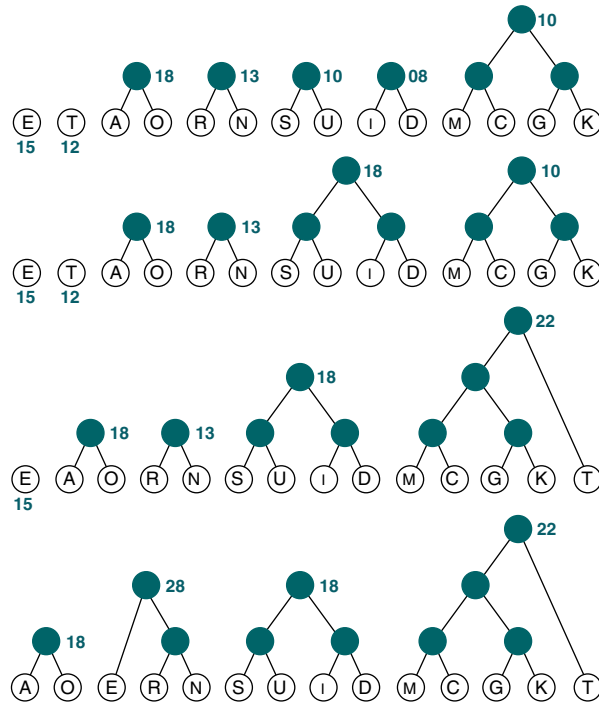
ŞEKİL 6-17 Huffman Ağacı, Kısım 1

Aynı zamanda altıncı sırada, en düşük değerli düğüm 08 (O) ve ikinci en düşük değer 10 (A)'dır. Ancak üç tane 10 var—biri yaprak seviyesinde (A), biri yaprakların bir seviye üstünde (S-U) ve biri yaprakların iki seviye üstünde (M-C-G-K). Hangisini seçmeliyiz? 10'lardan hangisi 8'e bitişikse onu seçiyoruz. Bu karar, dal hatlarının kesişmesini engeller ve ağacın okunabilirliğini korumamıza olanak tanır.

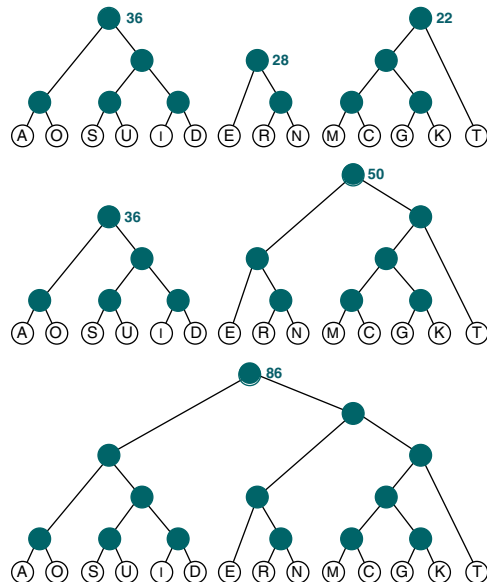
Eğer daha yüksek değerlerden hiçbiri daha düşük değere bitişik değilse, düğümleri netlik için yeniden düzenleyebiliriz (bkz. Şekil 6-18). Şekilde (üçüncü satırda), karakter T'yi ağacın sol tarafından sağ tarafa taşıdık ve bu tarafta ki bir düğümlerle birleştirdik. Aynı nedenle karakter E'yi de taşıyoruz.

Şekil 6-19 işlemin geri kalanını göstermektedir. Gördüğünüz gibi, tamamlanmış ağaç kök seviyesinde tek bir düğümlerle sonuçlanır (değeri 86 olan).

Ağaç tamamlandığında, her karaktere kod atamak için kullanıyoruz. Öncelikle, her dala bir bit değeri atıyoruz (bkz. Şekil 6-20). Kökten (üst düğümden) başlayarak, sol dala 0 ve sağ dala 1 atıyoruz ve bu deseni her düğümlerde tekrarlıyoruz. Hangi dalın 0 ve hangi dalın 1 olacağı tasarımcıya bırakılmıştır—atamalar ağaç boyunca tutarlı olduğu sürece.



ŞEKİL 6-18 Huffman Ağacı, Bölüm 2



ŞEKİL 6-19 Huffman Ağacı, Bölüm 3

6.3 Genel Ağaçlar

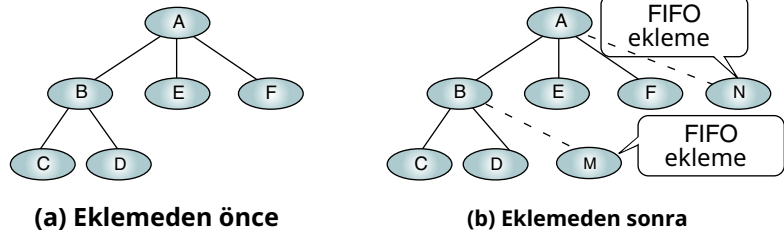
Genel bir ağaç, her düğümün sınırsız bir çıkış derecesine sahip olabileceği bir ağaçtır. Her düğüm, gereksinimlerini karşılamak için gerekli olan kadar çocuğa sahip olabilir. Daha önce tartışılan malzeme listesi, genel bir ağacın bir örneğidir.

Genel Ağaçlara Ekleme

Bir düğümü genel bir ağaca eklemek için, kullanıcı düğümün ebeveynini sağlamalıdır. Ebeveyn verildiğinde, üç farklı kural kullanılabilir: (1) ilk giren ilk çıkar (FIFO) ekleme, (2) son giren ilk çıkar (LIFO) ekleme ve (3) anahtar sıralı ekleme.

FIFO Ekleme

FIFO ekleme kullanıldığında, düğümleri kardeş listesine sonuna ekleriz, tıpkı yeni bir düğümü bir kuyruğun arkasına eklediğimiz gibi. Liste işlendiğinde, kardeşler FIFO sırasına göre işlenir. FIFO sırası, uygulamanın verilerin, girildiği sırayla işlenmesini gerektirdiği durumlarda kullanılır. Şekil 6-21, genel bir ağaca iki FIFO eklemesini göstermektedir. Ebeveyni A olarak verildiğinde, N düğümü F düğümünden sonra seviye 1'e eklenmiştir; ve, ebeveyni B olarak verildiğinde, M düğümü D düğümünden sonra seviye 2'ye eklenmiştir.



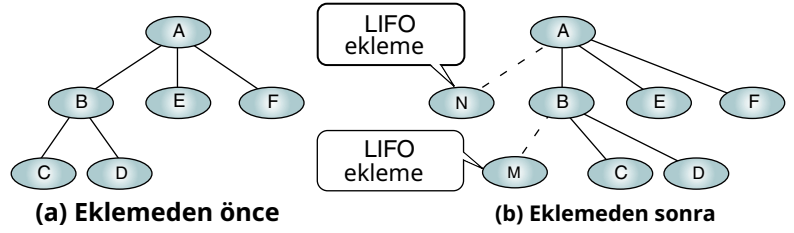
ŞEKİL 6-21 FIFO Genel Ağaç Ekleme

LIFO Ekleme

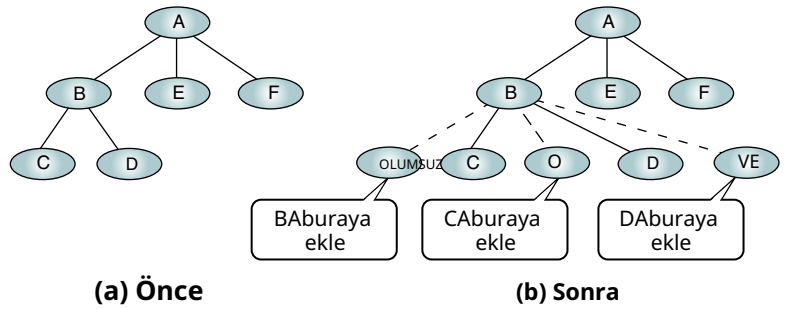
Kardeş listelerini oluşturuldukları ters sırayla işlemek için, LIFO ekleme kullanıyoruz. LIFO ekleme, yeni düğümü kardeş listesinin başına yerleştirir. Bu, bir yığın ile eşdeğerdir. Şekil 6-22, bir LIFO ağacı için ekleme noktalarını göstermektedir.

Anahtar sıralı Ekleme

Belki de en yaygın ekleme kurallarından biri olan anahtar sıralı ekleme, yeni düğümü kardeş düğümler arasında anahtar sırasına yerleştirir. Anahtar sırasına ekleme mantığı, bağlı bir listeye ekleme mantığına benzerdir. Ebeveynin ilk çocuğundan başlayarak, doğru ekleme noktasını bulana kadar kardeş (sağ) işaretçilerini takip ederiz ve ardından öncülerle ve ardıllarla (varsa) bağlantıları kurarız. Şekil 6-23, genel bir ağaç için doğru anahtar sıralı ekleme yerlerini göstermektedir.



ŞEKİL 6-22 LIFO Genel Ağaçlara Ekleme



ŞEKİL 6-23 Anahtar Sıralı Genel Ağaç Ekleme

Genel Ağaç Silme İşlemleri

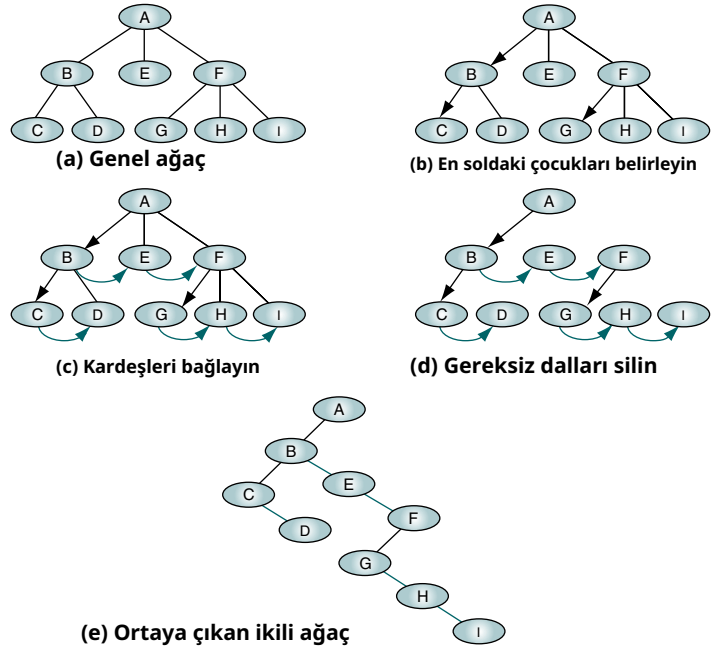
Genel ağaç eklemeleri için standart kurallar geliştiremiyor olsak da, standart silme kuralları geliştirebiliriz. İlk kural şudur: bir düğüm yalnızca bir yapraksa silinebilir. Genel ağaçta bu, bir düğümün çocukları varsa silinemeyeceği anlamına gelir. Kullanıcı çocukları olan bir düğümü silmeye çalışırsa, program, düğümün çocukları silinmeden silinemeyeceğine dair bir hata mesajı verir. Bu durumda, kullanıcı önce çocukları silmekten sorumludur. Alternatif olarak, uygulama, önce çocukları silip ardından istenen düğümü silmek üzere programlanabilir. Bu alternatif kullanılıyorsa, farklı bir kullanıcı seçeneği ile, örneğin düğümü ve çocukları temizle gibi, ve basit düğüm silme seçeneği ile değil, yapılmalıdır.

Genel Ağaçtan İkili Ağaç Oluşturma

İkili ağaçları programlarda temsil etmek, genel ağaçları temsil etmekten çok daha kolaydır. Bu nedenle, genel ağaçları ikili ağaç formatında temsil edebilmek istiyoruz. İkili ağaç formatı, sol ve sağ işaretçilerin anlamını değiştirerek benimsenebilir. Genel bir ağaçta, iki ilişki kullanırız: ebeveyn-çocuk ve kardeş-kardeş. Bu iki ilişkiyi kullanarak, herhangi bir genel ağacı ikili ağaç olarak temsil edebiliriz.

Şekil 6-24'te gösterilen ağacı düşünün. Bunu ikili bir ağaç haline getirmek için, önce ebeveynden ilk çocuğuna giden dalı tanımlarız. Bu dallar, her ebeveynden ikili ağaçta sol işaretçiler haline gelir. Şekil 6-24(b)'te gösterilmiştir. Ardından, en soldaki çocuktan başlayarak kardeşleri birbirine bağlarız; her kardeş için sağdaki kardeşe bir dal kullanırız. Bu dallar, Şekil 6-24(c)'te gösterilen ikili ağaçtaki sağ işaretçileridir. Dönüşüm sürecindeki üçüncü ve son adım, ebeveynden çocuklarına giden tüm gereksiz dalları kaldırmaktır. Ortaya çıkan ikili ağaç, Şekil 6-24(d)'te gösterilmiştir.

Bu geçerli bir ağaç yapısı olmasına rağmen, geleneksel bir ikili ağaç formatına sahip değildir. Bu nedenle, Şekil 6-24(e)'de gösterildiği gibi yeniden çizeriz.



ŞEKİL 6-24 Genel Ağaçların İkili Ağaçlara Dönüştürülmesi

6.4 Anahtar Terimler

ata	inorder gezinti
dengeli	ternal düğüm
dengeli faktör	anahtar-sıralı eklemeyap
dengeli ikili ağaç	rak
ikili ağaç	sol alt ağaçse
ikili ağaç gezintisi	viye
dal	LIFO eklemene
genişlik öncelikli gezinti	redeyse tam ağaçdü
çocuk	ğüm
tam ağaç	boş ağaç
data sıkıştırma	out-değeri
derece	ebeveyn
derinlik	yol
derinlik öncelikli	postfix gezinti
gezintisoy	postorder gezinti
ifadeifade ağac	prefix gezinti
ıFIFO eklemegeneral	preorder gezinti
ağac	sağ alt ağaç
	kök
goezinta	kardeşal
yükseklik	t ağac
Huffman kodu	token
in-değeri	ağac
infix gezinti	

6.5 Özet

- ▣ Bir ağaç, düğüm olarak adlandırılan sonlu bir eleman kümesi ve düğümleri bağlayan yönlendirilmiş çizgiler olarak adlandırılan sonlu bir dal kümesinden oluşur.
 - ▣ Bir düğümle ilişkili dal sayısı, düğümün derecesidir.
- ▣ Dal düğüme doğru yönlendirilmişse, bu bir indegree dalıdır; dal düğümünden uzak yönlendirilmişse, bu bir outdegree dalıdır. Düğümün derecesi, indegree ve outdegree dallarının toplamıdır.
- ▣ Eğer ağaç boş değilse, ilk düğüm kök olarak adlandırılır ve indegree'si sıfırdır.
- ▣ Ağaçtaki tüm düğümler, kök hariç, bir indegree'ye sahip olmalıdır.
- ▣ Bir yaprak, outdegree'si sıfır olan bir düğümdür.
- ▣ Bir iç düğüm, ne kök ne de yaprak olan bir düğümdür.
- ▣ Bir düğüm, bir ebeveyn, bir çocuk veya her ikisi olabilir.
- ▣ Aynı ebeveynin iki veya daha fazla düğümü kardeş olarak adlandırılır.
- ▣ Bir yol, her bir düğümün bir sonraki düğüme bitişik olduğu düğümlerin bir dizisidir.

- Bir ata, belirli bir düğümün kökünden gelen yoldaki herhangi bir düğümdür. Bir soy, belirli bir düğümden bir yaprağa giden tüm yollardaki herhangi bir düğümdür.
- Bir düğümün seviyesi, kökten olan uzaklığıdır.
- Bir ağacın yüksekliği, kökten en uzun yolda bulunan yaprağın seviyesi artı 1'dir; boş bir ağacın yüksekliği -1'dir.
- Bir alt ağaç, kökün altındaki herhangi bir bağılı yapıdır.
- Bir ağaç, ya (1) boş olan ya da (2) hiyerarşik olarak sıfır veya daha fazla alt ağaç indiren kök adı verilen belirli bir düğümden oluşan düğüm kümesi olarak özyinelemeli olarak tanımlanabilir.
- İkili ağaç, hiçbir düğümün iki çocuktan fazlasına sahip olamayacağı bir ağaçtır.
- Bir ikili ağacın minimum ve maksimum yüksekliği, düğüm sayısı ile ilişkilendirilebilir:

$$H_{\min} = \lfloor \log_2 N \rfloor + 1$$

$$H_{\max} = N$$

- Bir ikili ağacın yüksekliği verildiğinde, ağaçtaki minimum ve maksimum düğüm sayısı şu şekilde hesaplanabilir

$$N_{\min} = H$$

$$N_{\max} = 2^H - 1$$

- Bir ikili ağacın denge faktörü, sol ve sağ alt ağaçları arasındaki yükseklik farkıdır.

$$B = H_L - H_R$$

- Bir ikili ağaç, alt ağaçlarının yükseklikleri en fazla 1 farkla farklıysa ve alt ağaçları da dengeliyse dengelidir.
- Tam bir ikili ağaç, yüksekliği için maksimum giriş sayısına sahiptir; bir ağaç, son seviye dolu olduğunda tamdır.
- Neredeyse tamamlanmış bir ikili ağaç, düğümleri için minimum yüksekliğe sahip olan ve son seviyedeki tüm düğümlerin solda bulunduğu bir ağaçtır.
- İkili ağaç gezintisi, ağacın her düğümünü bir kez ve yalnızca bir kez, önceden belirlenmiş bir sırada ziyaret eder.
- İkili ağaç gezintisi için iki yaklaşım vardır: derinlik öncelikli ve genişlik öncelikli.
- Derinlik öncelikli yaklaşımı kullanarak, bir ikili ağacı altı farklı sırada gezinebiliriz; ancak bu sıralardan yalnızca üçü standart isimlere sahiptir: öncelikli, sıralı ve sonralı.
- Öncelikli gezintide, önce kökü işleriz, ardından sol alt ağaç ve sonra sağ alt ağaç gelir.
- Sıralı gezintide, önce sol alt ağaç, ardından kök ve sonra sağ alt ağaç işlenir.

- ❑ Postorder gezintisinde, önce sol alt ağacı, ardından sağ alt ağacı ve sonra kökü işleriz.
- ❑ Genişlik öncelikli yaklaşımda, bir seviyedeki tüm düğümleri işleriz ve sonra bir sonraki seviyeye geçeriz.
- ❑ Genel bir ağaç, her düğümün sınırsız bir çıkış derecesine sahip olabileceği bir ağaçtır.
- ❑ Genel bir ağacı ikili bir ağaç haline getirmek için, her düğümün ilk çocuğun u tanımlarız, kardeşleri soldan sağa bağlarız ve her ebeveyn ile ilk çocuk arasındaki tüm çocuklar arasındaki bağlantıyı sileriz.
- ❑ Genel ağaçlara veri eklemek için üç yaklaşım FIFO, LIFO ve anahtar sıralı olarak adlandırılır.
- ❑ Bir genel ağaçta bir düğümü silmek için, onun bir çocuğu olmadığından emin olmalıyız.
- ❑ Huffman kodu, karakterleri temsil etmek için değişken uzunlukta bir kod kullanan bir kodlama yöntemidir.
- ❑ Huffman kodunda, daha sık meydana gelen karakterlere daha kısa kodlar, daha az sık meydana gelenlere ise daha uzun kodlar atarız.

6.6 Pratik Setleri

İstirmler

1. Aşağıdaki parantezli notasyonun ağaç temsilini gösterin:

a (b (c d) e f (g h))

2. Şekil 6-25'te şunları bulun:

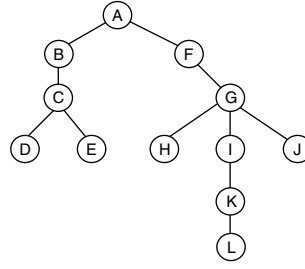
- a. kök
- b. yapraklar
- c. iç düğümler
- d. H'nin ataları
- ve. F'nin soyundan gelenler

3. Şekil 6-25'te şunları bulun:

- a. düğüm F'nin iç derecesi
- b. düğüm B'nin dış derecesi
- c. H'nin kardeşleri
- d. K'nin ebeveyni
- ve. C'nin çocukları

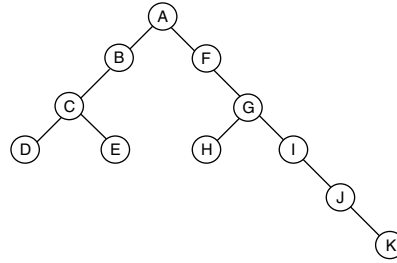
4. Şekil 6-25'te şunları bulun:

- a. ağacın yüksekliği
- b. alt ağaç G'nin yüksekliği
- c. düğüm G'nin seviyesi
- d. düğüm A'nın seviyesi
- ve. alt ağaç E'nin yüksekliği



ŞEKİL 6-25 Alıştırmalar 2, 3, 4, 5, 6 ve 7 için Ağaç

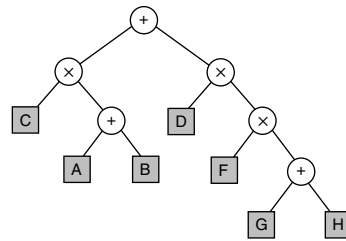
5. Şekil 6-25'te düğüm F'nin alt ağaçlarını gösterin.
6. Şekil 6-25'te ağacın girintili liste temsili gösterin.
7. Şekil 6-25'te ağacın parantezli temsili gösterin.
8. Ön sıralama ve sıralama gezintileri aynı sonucu veren bir ikili ağaç bulun.
9. 28 düğümü olan bir ağacın maksimum ve minimum yükseklikleri nedir?
10. Bir ikili ağaçta, seviye 3'te bulunabilecek maksimum düğüm sayısı nedir? Seviye 4'te? Seviye 12'de?
11. Şekil 6-26'daki ağacın denge faktörü nedir?



ŞEKİL 6-26 Alıştırmalar 11, 14, 15 ve 30 için İkili Ağaç

12. Seviye 4'e kadar tam bir ağaç çizin.
13. Yüksekliği 4 olan bir ağaçta kaç farklı neredeyse tam ağaç bulunabilir?
14. Şekil 6-26'daki ikili ağacın derinlik öncelikli gezintilerini (ön sıralama, sıralama ve son sıralama) gösterin.
15. Şekil 6-26'daki ağacın genişlik öncelikli gezimini gösterin.
16. Aşağıdaki ikili ağaçların kökünü bulun:
 - a. postorder sıralaması ile ağaç: FCB DG
 - b. preorder sıralaması ile ağaç: IBCDFEN
 - c. inorder sıralaması ile ağaç: CBIDFGE

17. Bir ikili ağacın 10 düğümü vardır. Ağacın preorder ve inorder sıralamaları aşağıda gösterilmiştir. Ağacı çizin.
 Ön sipariş: JCBADEFGH
 Sırayla: ABCEDFJGIH
18. Bir ikili ağacın sekiz düğümü vardır. Ağacın postorder ve inorder sıralamaları aşağıda verilmiştir. Ağacı çizin.
 Posta siparişi: FECHGDBA
 Sırayla: FCEABHGD
19. Yedi düğümü olan bir ikili ağaç vardır. Ağacın ön sipariş ve son sipariş gezintileri aşağıda verilmiştir. Ağacı çizebilir misiniz? Eğer hayırsa, açıklayın.
 Ön sipariş: GFDABEC
 Sonraki sipariş: ABDCEFG
20. Neredeyse tamamlanmış bir ikili ağaç dokuz düğüme sahiptir. Ağacın genişlik gezintisi aşağıda verilmiştir. Ağacı çizin.
 Genişlik: JCBADEFG
21. Üç düğümlü (A, B, C) tüm olası benzer olmayan ikili ağaçları çizin.
22. Şekil 6-21(b) ile ilgili ikili ağacı çizin.
23. 42 düğümü olan bir ikili ağacın sahip olabileceği en az seviye sayısı nedir?
24. 42 düğümü olan bir üçlü ağacın, yani çıkış derecesi 3 olan bir ağacın sahip olabileceği en az seviye sayısı nedir?
25. Bir ikili ağacın beşinci seviyesindeki maksimum düğüm sayısı nedir?
26. Şekil 6-27'deki ifade ağacında infiks, prefiks ve postfix ifadeleri bulun.



ŞEKİL 6-27 Alıştırma 26 için İfade Ağacı

27. Aşağıdaki infix ifadesi için ifade ağacını çizin ve prefix ve postfix ifadelerini bulun:

$$(C + D + A \times B) \times (E + F)$$

28. Aşağıdaki prefix ifadesi için ifade ağacını çizin ve infix ve postfix ifa delerini bulun:

$$\times - A B + \times C D / E F$$

29. Aşağıdaki postfix ifadesi için ifade ağacını çizin ve infix ve prefix ifa delerini bulun:

$$A B \times C D / + E F - \times$$

30. ŞEKİL 6-26'da verilen ağaç kullanılarak Algoritma 6-9'daki rekürsif fonksiyonun son ucunu gösterin.

ALGORİTMA 6-9Alıştırma 30 için Ağaç Geçişi

Algoritma ağaçGeçişi (ağaç)

```

1  eğer ağaç null ise
    1  yazdır "Null"
2  aksi takdirde
    1  ağaçGeçişi (sağ alt ağaç)
    2  yazdır "sağ tamamlandı"
    3  ağaçGeçişi (sol alt ağaç)
    4  yazdır (ağaç verisi)
3  sonlandır if
son  ağaçGeçişi

```

31. Bir ikili ağacın yüksekliğine dayalı maksimum düğüm sayısının aşağıdaki rekürsif tanımındaki (Şekil 6-28) eksik faktörü (soru işareti) s ağılayın.

$$N(H) = \begin{cases} 1 & \text{eğer } H = 1 \\ N(H - 1) + ? & \text{eğer } H \geq 2 \end{cases}$$

ŞEKİL 6-28Alıştırma 31 için Rekürsif Tanım

Problemler

32. Bir ikili ağaçtaki düğüm sayısını sayan bir algoritma yazın.
33. Bir ikili ağaçtaki yaprak sayısını sayan bir algoritma yazın.
34. Bir ikili ağaçtan tüm yaprakları silen, kök ve ara düğümleri yerinde bırakan bir algoritma yazın. (*İpucu* : Öncelikli bir geçiş kullanın.)
35. Tam veya neredeyse tam bir ikili ağacın düğüm sayısını verildiğinde, ağacın yüksekliğini bulan bir algoritma yazın.

36. İkili bir ağacın tam olup olmadığını belirleyen bir algoritma yazın.
37. İkili bir ağacın neredeyse tam olup olmadığını belirleyen bir algoritma yazın.
38. İkili ağaç ön sıralama algoritmasını, özyineleme yerine bir yığın kullanarak yeniden yazın.
39. İkili ağaç sıralı gezinti algoritmasını, özyineleme yerine bir yığın kullanarak yeniden yazın.
40. İkili ağaç son sıralama algoritmasını, özyineleme yerine bir yığın kullanarak yeniden yazın.
41. Genel ağaçlar için FIFO ekleme algoritmasını yazın.
42. Genel ağaçlar için LIFO ekleme algoritmasını yazın.
43. Genel ağaçlar için anahtar sıralı ekleme algoritmasını yazın.
44. Genel bir ağaç için silme algoritmasını yazın.
45. Bir ikili ağacın ayna görüntüsünü oluşturan bir algoritma yazın. Tüm sol çocuklar sağ çocuk olur ve tersine.

Projeler

46. Bir ikili ağacın denge faktörünü hesaplamak için bir C fonksiyonu yazın. Eğer ilk olarak kök işaretçisi ile çağrılırsa, tüm ağacın denge faktörünü belirlemelidir. Eğer bir alt ağaç işaretçisi ile çağrılırsa, alt ağacın denge faktörünü belirlemelidir.
47. Bir Huffman ağacı oluşturmak için bir pseudocode algoritması yazın. AlfabeTablo 6-3'te gösterildiği gibi kullanın.

Karakter	Ağırlık	Karakter	Ağırlık	Karakter	Ağırlık
ABCDEFGHI	7	JKLMNOPQR	1	STUVWXYZ	6
	2		1		8
	2		4		4
	3		3		1
	11		7		2
	2		9		1
	2		2		2
	6		1		1
	6		6		

TABLO 6-3Proje 47 için Huffman Karakter Ağırlıkları

48. Proje 47'de geliştirilen Huffman algoritması için C uygulamasını yazın. İnş a edildikten sonra, kodu yazdırın. Ardından, klavyeden karakterleri okuyup bunları Huffman kodunuza dönüştüren bir C programı yazın. Programın ıza Huffman kodunu metne geri dönüştüren bir fonksiyon ekleyin. Klavyeden girilen kodun doğru bir şekilde dönüştürüldüğünü doğrulamak için bunu kullanın.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 7

İkili Arama Ağaçları

Artık arama ağaçlarına dikkatimizi yönlendiriyoruz; bu bölümde ikili arama ağaçları ve Bölüm 8'de AVL ağaçları olmak üzere iki standart ağaç yapısını derinlemesine tartışacağız. Her ikisi de verilerin sıralanması gerektiğinde kullanılır. Temel olarak AVL ağaçları dengelidir, oysa ikili arama ağaçları değildir.

Doğrusal liste yapısının tasarımında iki seçeneğimiz vardı: bir dizi veya bir bağlı liste. Dizi yapısı çok verimli bir arama algoritması, ikili arama ağaçları, ancak ekleme ve silme algoritmaları çok verimsizdir. Öte yandan, bağlı liste yapısı verimli ekleme ve silme ağaçları, ancak arama algoritması çok verimsizdir. İhtiyacımız olan, verimli bir arama algoritması ve aynı zamanda verimli ekleme ve silme algoritmaları sağlayan bir yapıdır. İkili arama ağacı ve AVL ağacı bu yapıyı sağlar.

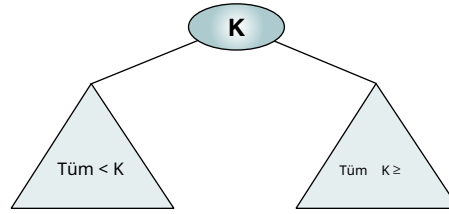
7.1 Temel Kavramlar

Bir ikili arama ağacı (BST), aşağıdaki özelliklere sahip bir ikili ağaçtır:

- Sol alt ağaçtaki tüm öğeler kökten daha küçüktür.
- Sağ alt ağaçtaki tüm öğeler kökten daha büyük veya ona eşittir.
- Her alt ağaç kendisi bir ikili arama ağacıdır.

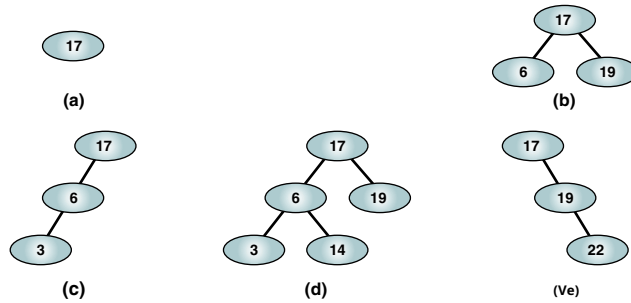
Bir ikili arama ağacında, sol alt ağaç kökten daha küçük anahtar değerlerini içerir ve sağ alt ağaç kökten daha büyük veya ona eşit anahtar değerlerini içerir.

Genel olarak, her düğüm tarafından temsil edilen bilgi, tek bir veri ögesi yerine bir kayıttır. İkili arama ağacı tanımı bir kayda uygulandığında, sıralama özellikleri kaydın anahtarına atıfta bulunur. Şekil 7-1, K'nın anahtar olduğu bir ikili ağacın özelliklerini yansıtır.



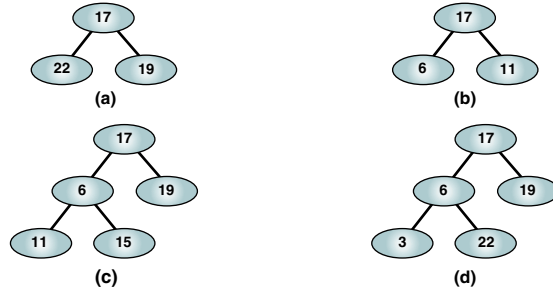
ŞEKİL 7-1 İkili Arama Ağacı

Şekil 7-2, beş ikili arama ağacı içermektedir. Onları incelerken, Şekil 7-2(a) ve (b)'deki ağaçların tam ve dengeli, Şekil 7-2(d)'deki ağacın neredeyse tam ve dengeli olduğunu, Şekil 7-2(c) ve (e)'deki ağaçların ise ne tam ne de dengeli olduğunu not edin.



ŞEKİL 7-2 Geçerli İkili Arama Ağaçları

Şimdi ikili arama ağacı özelliklerine sahip olmayan bazı ikili ağaçlara bakalım. Şekil 7-3'teki ikili ağaçları inceleyin. İlk ağaç, Şekil 7-3(a), birinci kuralı ihlal ediyor: sol alt ağaçtaki tüm öğeler kökten daha küçük olmalıdır. Sol alt ağaçtaki anahtar (22), kök (17)'den büyüktür. İkinci ağaç, Şekil 7-3(b), ikinci kuralı ihlal ediyor: sağ alt ağaçtaki tüm öğeler kökten büyük veya ona eşit olmalıdır. Sağ alt ağaçtaki anahtar (11), kök (17)'den küçüktür. Şekil 7-3(c) üçüncü kuralı ihlal ediyor: her alt ağaç bir ikili arama ağacı olmalıdır. Bu ağaçta sol alt ağaç anahtarı (6), kökten (17) küçüktür ve sağ alt ağaç anahtarı (19), kökten büyüktür. Ancak, sol alt ağaç geçerli bir ikili arama ağacı değildir çünkü birinci kuralı ihlal ediyor: sol alt ağacının (11) kökten (6) büyük. Şekil 7-3(d) de üç kuraldan birini ihlal ediyor. Hangi kuralı gördünüz? (İp: Sol alt ağaçtaki en büyük anahtar nedir?)



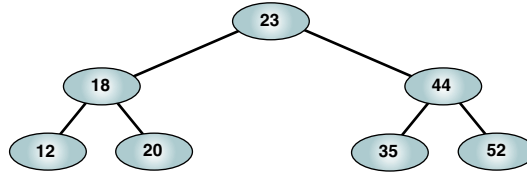
ŞEKİL 7-3 Geçersiz İkili Arama Ağaçları

7.2 BST İşlemleri

Şimdi ikili arama ağaçları üzerinde kullanılan birkaç işlemi inceleyeceğiz. Bölüm 6'da incelediğimiz gezinti algoritmalarıyla başlıyoruz. Ardından bazı basit arama algoritmalarına bakacağız ve bir ikili arama ağacı oluşturan algoritmalarla sonuçlandıracağız.

Gezintiler

İkili ağaç gezinti işlemleri, Bölüm 6'daki işlemlerle aynıdır. Buradaki ilginç işlemde değil, daha çok ürettiği sonuçlardadır. Şimdi Şekil 7-4'teki ağacı gezintiye başlayalım.



ŞEKİL 7-4 İkili Arama Ağacı Örneği

Eğer ağacı öncelikli gezinti kullanarak gezersek, aşağıda gösterilen sonuçları alırız.

23 18 12 20 44 35 52

Bu gezinti geçerli olmasına rağmen, çok faydalı değildir. Daha faydalı olup olmadığını görmek için bir sonralama gezintisi deneyelim.

12 20 18 35 52 44 23

Yine, bu dizinin faydalı olma ihtimali pek yok. Şimdi bir sıralı gezinti deneyelim.

12 18 20 23 35 44 52

Bu gezintinin bazı pratik kullanımları vardır: bir ikili arama ağacının sıralı gezintisi, sıralı bir liste üretir. Eğer ağacı sağ-düğüm-sol sıralaması kullanarak gezerseniz ne olur? Deneyin ve görün.¹

Bir ikili arama ağacının sıralı gezintisi, sıralı bir liste üretir.

Aramalar

Bu bölümde üç arama algoritmasını inceliyoruz: en küçük düğümü bul, en büyük düğümü bul ve istenen düğümü bul (BST araması).

En Küçük Düğümü Bul

Şekil 7-4'teki ikili arama ağacını incelediğimizde, en küçük değere (12) sahip düğümün ağacın en sol yaprak düğümü olduğunu not ediyoruz. Bu nedenle, en küçük düğümü bulma işlemi, bir yaprağa ulaşana kadar sol dalları takip eder. Algoritma 7-1, bir ikili arama ağacında en küçük düğümü bulmak için gereken sahte kodu içermektedir.

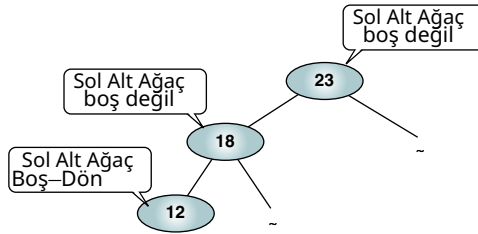
ALGORİTMA 7-1 BST'de En Küçük Düğümü Bul

```
Algoritma findSmallestBST (root)
Bu algoritma, bir BST'deki en küçük düğümü bulur.
Ön root, boş olmayan bir BST veya alt ağaç için bir işaretçidir
En küçük düğümün adresini döndür
1 eğer (sol alt ağaç boşsa)
    1 kayıt (kök)
2 bitiş eğer
3 kayıt enKüçükBST bul (sol alt ağaç)
son enKüçükBSTBul
```

Algoritma 7-1 Analizi Ağaçlarla tipik olarak olduğu gibi, bu algoritma rekursiftir. İlk çağrı, Şekil 7-5'te gösterildiği gibi, ağacın kökü ile başlar. Ardından sol alt ağaçlar boyunca bir yol takip ederiz. Eğer sol

1. Gördüğünüz gibi, sağ-düğüm-sol gezintisi ağacı azalan bir sırada gezer. Bu standart bir gezinti değildir, ancak çok faydalı olabilir.

alt ağaç null değilse, daha sola bakmaya devam etmeliyiz. Bunu, sol alt ağaç ile enKüçükBSTB ul için bir özyinelemeli çağrı ile yapıyoruz. Bu algoritmadaki temel durum, boş bir sol alt ağaç bulduğumuzda gerçekleşir. Bu noktada, 12'yi içeren mevcut düğümün adresini döndürüyoruz. Özyinelemeli çağrı bir dönüş ifadesinin parçası olduğundan, ağaçta yukarı doğru hareket eder ken, en küçük düğüme kadar düğüm adresini döndürmeye devam ediyoruz ve nihayetinde bu nu başlatan modüle geri döndürüyoruz.



ŞEKİL 7-5 BST'de En Küçük Düğümü Bul

En Büyük Düğümü Bul

İkili arama ağacında en büyük düğümü bulma işleminin mantığı, en küçük düğümü bulmanın tersidir. Bu sefer ağaç kökünden başlıyoruz ve ağacın en sağın daki düğüme kadar sağ dalları takip ediyoruz; bu tanıma göre en büyük olmalıdır. Pseudocode, Algoritma 7-2'de gösterilmiştir.

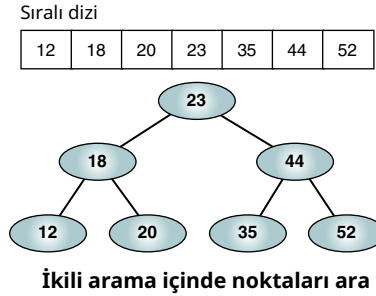
ALGORİTMA 7-2 BST'de En Büyük Düğümü Bul

Algoritma enBüyükBST (kök)
 Bu algoritma, bir BST'deki en büyük düğümü bulur.
 Ön kök, boş olmayan bir BST veya alt ağaca işaret eden bir işaretçidir
 Döndürülen en büyük düğümün adresini döndür

- 1 eğer (sağ alt ağaç boşsa)
 - 1 kayıt (kök)
- 2 sonlandır if
- 3 dön enBüyükBST (sağ alt ağaç)
- son enBüyükBST

BST Arama

Şimdi ikili arama ağaçlarının en önemli özelliği olan ikili ağaç araması inceleyeceğiz; bu, ağaçta belirli bir düğümü bulur. BST aramasının nasıl çalıştığını anlamamıza yardımcı olmak için, Şekil 7-6'da gösterildiği gibi ikili arama algoritmasını tekrar gözden geçirelim. Bu şekil, dizideki orta elemandan olası her arama yolunu izler. 23 ile başlayarak, ikili arama ya 18'i ya da 44'ü inceler; arama anahtarına bağlı olarak. 18'den 12'yi ya da 20'yi inceler; 44'ten 35'i ya da 52'yi inceler. Şekilde de açıkça görüldüğü gibi, tüm olası arama yollarını izlemek, bir ikili arama ağacında gördüğümüz aynı yolları takip eder.



ŞEKİL 7-6BST ve İkili Arama

Şimdi süreci tersine çevirelim. İkili arama ağaçlarında verilen bir düğümü bulalım. Düğüm 20'yi aradığımızı varsayalım. Arama argümanı 20'yi ağaç kökündeki değere karşılaştırarak başlıyoruz. 20, kök değeri 23'ten daha küçük olduğu için ve kök değerinden daha küçük olan tüm değerlerin sol alt ağaçta yer aldığını bildiğimiz için sola gidiyoruz. Şimdi arama argümanını alt ağaçtaki değerle, 18 ile karşılaştırıyoruz. Bu sefer arama argümanı kök değerinden, 18'den büyüktür. Ağaç kökünden daha büyük değerlerin sağ alt ağaçta yer alması gerektiğini bildiğimiz için sağa gidiyoruz ve istediğimiz değeri buluyoruz. Bu mantık Algoritma 7-3'te gösterilmektedir.

ALGORİTMA 7-3BST Arama

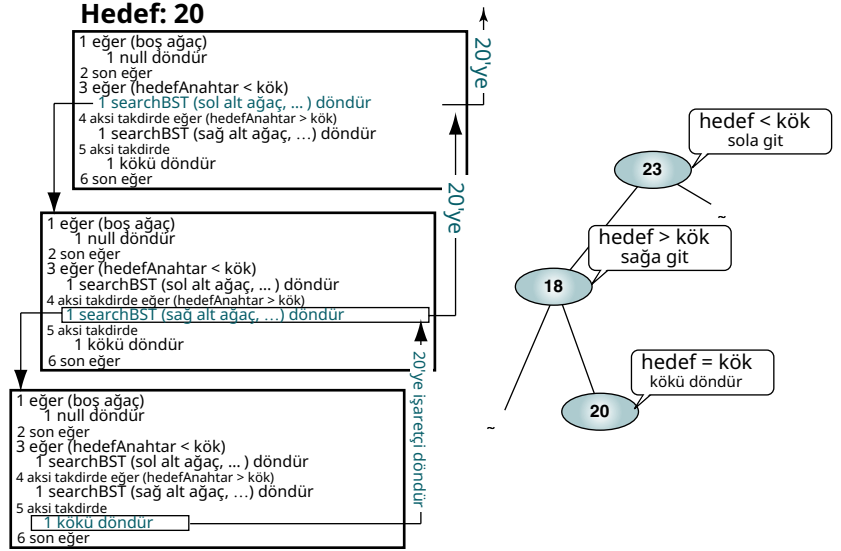
```

Algoritma searchBST (kök, hedefAnahtar)
Verilen bir değeri aramak için bir ikili arama ağacını kullanın.
  Ön   kök, bir ikili ağaç veya alt ağacın köküdür
        hedefAnahtar, talep edilen anahtar değeridir
  Değer bulunursa düğüm adresini döndür
        ağaçta düğüm yoksa null döndür
1  eğer (boş ağaç)
    Bulunamadı
    1  null döndür
2  sonlandır if
3  eğer (hedefAnahtar < kök)
    1  return searchBST (sol alt ağaç, hedefAnahtar)
4  aksi takdirde eğer (hedefAnahtar > kök)
    1  return searchBST (sağ alt ağaç, hedefAnahtar)
5  aksi takdirde
    Hedef anahtar bulundu
    1  kökü döndür
6  sonlandır if
son searchBST
  
```

Algoritma 7-3 Analizi BST aramasını özyineleme kullanarak uyguluyoruz. Bu algorithmada iki temel durum vardır: ya arama argümanını ağaçta buluruz, bu durumda geri döneriz

düğümünün adresi (ifade 5.1), veya arama argümanı mevcut değilse, null döndürüyoruz (ifade 1.1).

İfade 3.1 ve 4.1'deki dönüşleri dikkatlice inceleyin. Bunların, daha önce görüğümüz gibi, düğümün adresi ya da null olan değeri döndürdüğünü not edin. Bu ifadeler, bulunan adresin orijinal talep edene geri iletilmesi için gereklidir. Şekil 7-7, Şekil 7-4'teki ikili arama ağacında, 20 numaralı düğümü ararken izlen en yolu gösterir. Algoritma 7-3'ü kullanarak.



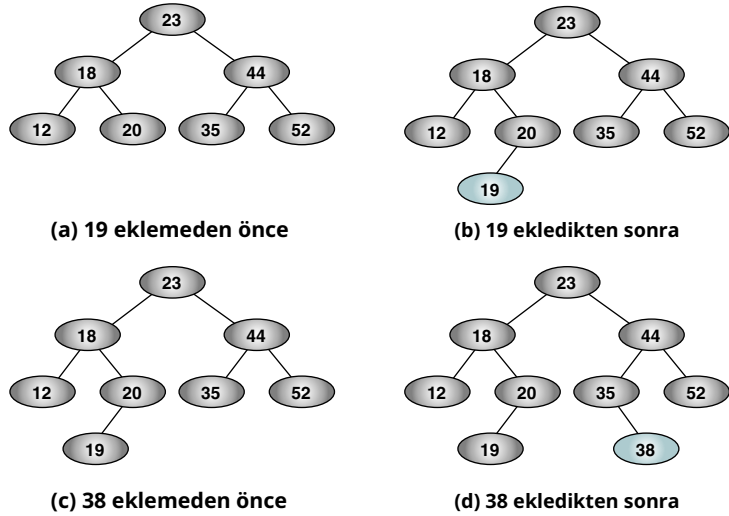
ŞEKİL 7-7 Bir BST'yi Arama

Ekleme

düğüm ekle fonksiyonu bir BST'ye veri ekler. Veri eklemek için yapmamız gereken tek şey, boş bir alt ağaçta dalları takip etmek ve ardından yeni düğümü eklemektir. Diğer bir deyişle, tüm eklemeler bir yaprakta veya yaprak benzeri bir düğümde gerçekleşir—sadece bir null alt ağaç olan bir düğüm.

Tüm BST eklemeleri bir yaprak veya yaprak benzeri düğümde gerçekleşir.

Şekil 7-8, iki düğüm ekledikten sonra ikili arama ağacımızı göstermektedir. Öncelikle 19 numaralı düğümü ekledik. Ekleme noktasını bulmak için, ağacı 23, 18 ve 20 yoluyla null bir sol dalda aradık. Ekleme noktasını bulduktan sonra, yeni düğümü 20'nin sol alt ağacı olarak ekledik. Ardından 38'i ekledik. Bu sefer ağacı 23, 44 ve 35 yoluyla null bir sağ alt ağaçta aradık ve yeni düğümü ekledik.



ŞEKİL 7-8BST Ekleme

Hem 19 hem de 38'in eklemeleri bir yaprak düğümde yapıldı. Eğer kök olan 23'ün bir kopyasını ekleseydik, bu 35'in sol alt ağacı haline gelecekti. Unutmayın ki ikili arama ağaçlarında, eşit değerlere sahip düğümler sağ alt ağaçta bulunur. Bu nedenle ekleme yolu 23, 44 ve 35 olacaktır. Bu durumda eklemeye yaprak benzeri bir düğümde gerçekleşir. 35'in sağ alt ağacı olmasına rağmen, sol alt ağacı boştur. Bu nedenle yeni düğüm, 23, 35'in sol alt ağacı olarak yerleştirilecektir.

Artık ekleme algoritmasını geliştirmeye hazırız. Verileri ağaçta yerleştiren şık bir algoritma yazabiliriz ve bu işlemi özyineleme kullanarak gerçekleştirebiliriz. Ağaç veya alt-ağaç boşsa, verileri kök düğümüne yerleştiririz. Eğer boş bir ağaçta değilsek, hangi dalı takip etmemiz gerektiğini belirleriz ve henüz bir yaprakta olup olmadığımızı belirlemek için özyinelemeli olarak çağırırız. Pseudocode, Algoritma 7-4'te gösterilmiştir.

ALGORİTMA 7-4BST'ye Düğüm Ekle

```

Algoritma addBST (kök, yeniDüğüm)
Yeni verileri içeren düğümü BST'ye özyineleme kullanarak ekleyin.
  Ön  kök, BST'deki mevcut düğümün adresidir
  ffffffyeniDüğüm, verileri içeren düğümün adresidir
  Son yeniDüğüm ağaca eklendi
  Olası yeni ağaç kökünün adresini döndür
1  eğer (boş ağaç)
    1  kökü yeniDüğüm olarak ayarla
    2  yeniDüğüm'ü döndür
2  sonlandır if
  
```

devam ediyor

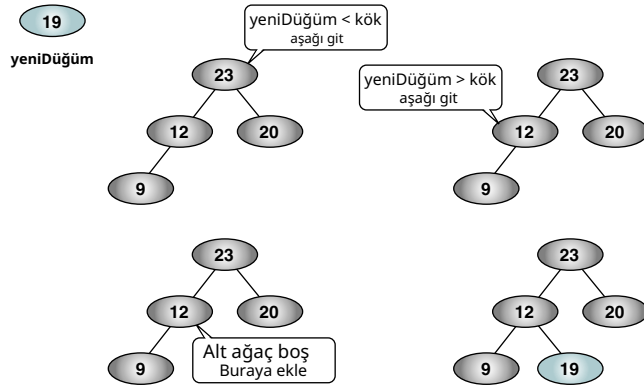
ALGORİTMA 7-4 BST'ye Düğüm Ekle (devamı)

```

Ekleme için null alt ağacı bul
3  eğer (newNode < root)
    1  return addBST(sol alt ağaç, newNode)
4  aksi takdirde
    1  return addBST(sağ alt ağaç, newNode)
5  sonlandır if
son  addBST

```

Algoritma 7-4 Analizi Algoritma oldukça şıktır, ancak yeni düğümün doğru konuma nasıl eklendiğini görmek kolay değildir. Yardımcı olmak için, bir düğümü 7-9. Şekil'de gösterildiği gibi bir ağaca ekleyelim.



ŞEKİL 7-9 Rekürsif BST Ekleme İzleme

19'u ağaca eklemek için, kök ile başlıyoruz. Yeni düğümün anahtarı kök anahtarından daha küçük olduğu için, sol alt ağacı (12) kullanarak **addBST** fonksiyonunu rekürsif olarak çağırıyoruz. Yeni düğümün anahtarı artık kök anahtarından daha büyük, bu yüzden tekrar sağ alt ağacı rekürsif olarak çağırıyoruz, ki bu null'dır. Bu noktada alt ağacın null olduğunu keşfediyoruz, bu yüzden yeni düğümü ekliyoruz, null alt ağacı 12'nin sağ alt ağacı olarak değiştiriyoruz.

Silme

Bir ikili arama ağaçından bir düğümü silmek için önce onu bulmalıyız. Bir düğüm ü sildiğimizde dört olası durum vardır:

1. Silinecek düğümün çocuğu yoktur. Bu durumda yapmamız gereken tek şey düğümü silmektir.
2. Silinecek düğümün yalnızca sağ alt ağacı vardır. Düğümü sileriz ve sağ alt ağacı silinen düğümün ebeveynine ekleriz.
3. Silinecek düğümün yalnızca sol alt ağacı vardır. Düğümü sileriz ve sol alt ağacı silinen düğümün ebeveynine ekleriz.
4. Silinecek düğümün iki alt ağacı vardır. Bir ağacın ortasından bir düğümü silmek mümkündür, ancak sonuç genellikle çok dengesiz hale gelir.

ağaçlar. Bu nedenle, düğümü basitçe silmek yerine, silinen verinin yerini alacak verileri bulmaya çalışarak mevcut yapıyı mümkün olduğunca korumaya çalışıyoruz. Bu, iki şekilde yapılabilir: (1) silinen düğümün sol alt ağacındaki en büyük düğümü bulup verisini silinen düğümün verisinin yerine koyabiliriz veya (2) silinen düğümün sağ alt ağacındaki en küçük düğümü bulup verisini silinen düğümün verisinin yerine koyabiliriz. Hangi mantığı kullanırsak kullanalım, silinebilecek bir yaprak veya yaprak benzeri düğümden veri taşıyacağız. Bu hareketlerin ikisinin de ikili arama ağacının bütünlüğünü koruduğunu kendinize kanıtlayın.

İkili arama ağacı silme için psödo kod, Algoritma 7-5'te gösterilmiştir.

ALGORİTMA 7-5BST'den Düğüm Sil

```

Algoritma deleteBST (root, dltKey)
Bu algoritma, bir BST'den bir düğüm siler.
  Ön  root, silinecek düğüme referanstır
      dltKey, silinecek düğümün anahtarıdır
  Son  düğüm silindi
      eğer dltKey bulunamazsa, root değişmez
  Düğüm silindiye true, bulunamazsa false döndür
1  eğer (boş ağaç)
2    yanlış döndür
3  sonlandır if
4  eğer (dltKey < root)
5    return deleteBST (sol alt ağaç, dltKey)
6 aksi (dltKey > root)
7    return deleteBST (sağ alt ağaç, dltKey)
8 aksi takdirde
9    Silinen düğüm bulundu--yaprak düğüm için test et
10   1 Eğer (sol alt ağaç yoksa)
11     1 sağ alt ağacı kök yap
12     2 true döndür
13   2 aksi takdirde eğer (sağ alt ağaç yoksa)
14     1 sol alt ağacı kök yap
15     2 true döndür
16   3 aksi takdirde
17     Silinecek düğüm bir yaprak değil. Sol alt ağaçta en büyük düğümü
18     bul.
19     1 kökü deleteNode içinde kaydet
20     2 en büyüğü largestBST (sol alt ağaç) olarak ayarla
21     3 en büyük veriyi deleteNode'a taşı
22     4 deleteBST'i döndür (deleteNode'un sol alt ağacı,
23       en büyük anahtar)
24   4 sonlandır if
25   6 sonlandır if
26 son deleteBST

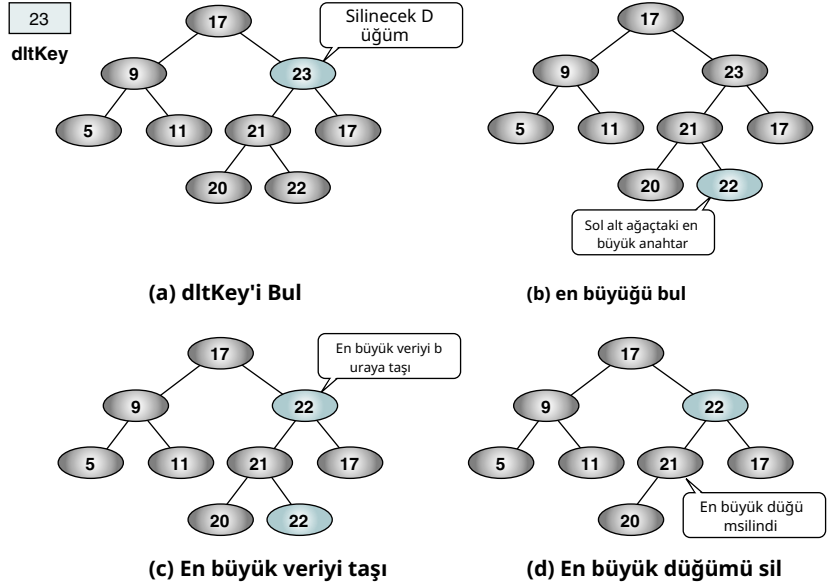
```

Algoritma 7-5 Analizi Bu algoritmayı tam olarak anlamak için dikkatlice incelemeniz gerekiyor. Öncelikle, bunun bir özyinelemeli algoritma olduğunu not edin. İki temel durum vardır: İlk olarak, düğümü bulamıyoruz. Bu durumda

kök işaretçisi null olduğunda. Bu durum 1.1. ifadesinde ele alınmaktadır. İkinci durum noktayı sildikten sonra, ya 5.1.2. ifadesinde ya da 5.2.2. ifadesinde meydana gelir.

313. sayfadaki ilk iki durum, algoritmanın gerçekuygulanmasında bir durum da birleştirilmiştir. Sol alt ağaç null ise, sağalt ağacı ana (**root**) ile basitçe bağlayabiliriz. Sağ alt ağaç null ise, null bir alt ağacıbağlıyoruz ki bu doğrudur. Sağ alt ağaç null değilse, verileri silinen noktaya ait olan ana ile bağlanır, bu Durum 2'dir. Öte yandan, sol alt ağaç null değilse, sağ alt ağacın null olup olmadığını test ederiz. Sağ alt ağaç null ise, sol alt ağaç işaretçisini ana ile taşımak mümkündür.

Bu algoritmadaki en zor mantık, düğüm bir yaprak olmadığında ortaya çıkar. Bu durumu am olarak anlamak için dikkatlice incelemeniz gerekir. Sol alt ağaçtaki en büyük noktayı arayarak başlıyoruz ve verileri silinecek verilerin yerine koymak için taşıyoruz. Daha sonra algoritmayı, i ç veya kök düğümde taşıdığımız verileri içeren yaprak düğümün anahtarını yeni silme hedefi olarak vererek tekrar çağırıyoruz. Bu, bu sefer hedef anahtarı bulduğumuzda, en az bir alt ağacın null olmasını garanti eder. Bu mantığı Şekil 7-10'da izliyoruz.



ŞEKİL 7-10 BST Test Durumlarını Sil

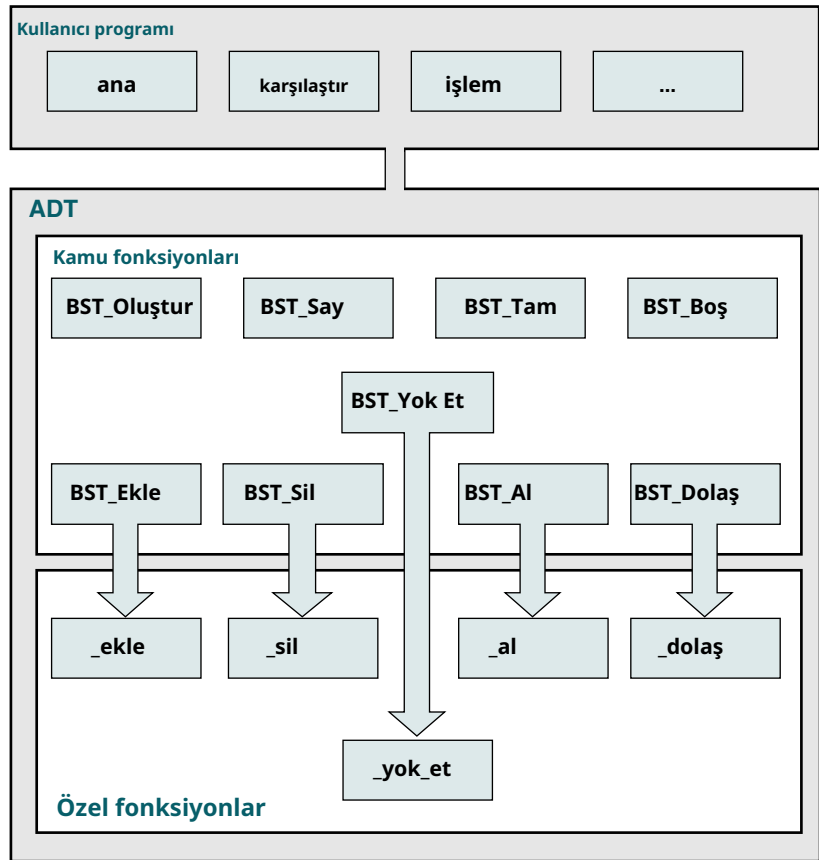
7.3 İkili Arama Ağacı ADT

Hiçbir programlama dilinin ikili arama ağacı için içsel işlemleri yoktur. Onun işlemleri C fonksiyonları kullanılarak simüle edilmelidir. BST soyut veri türü için model, Bölüm 5'te listeler için geliştirdiğimiz ADT'dir. Her iki yapı da uygulama verisine işaretçi içeren düğümleri depolamak için dinamik bellek kullanır. Her iki yapı da her düğümün ardını tanımlamak için işaretçilere ihtiyaç duyar. Liste her ardıl için yalnızca bir işaretçi içerirken, BST ağacı daha fazla işaretçi kullanır—bir sol alt ağaç için ve bir sağ alt ağaç için. Çünkü burada

BST ağacındaki sıralama, listede olduğu gibi, uygulama-dipendent bir arama fonksiyonuna ihtiyacımız var ve bu fonksiyon uygulama programcısı tarafından yazılmalıdır.

İki yapı arasındaki büyük bir fark, yapının gezinimindedir. Listede, listedeki bir sonraki düğümü almak için bir fonksiyon sağladık. BST gezintisinde özyineleme kullandığımız için, bu tasarım mümkün değildir. ADT'nin verileri bir geçişte işlemesi gerektiğinden, tasarım uygulama programcısının geçiş işleme fonksiyonunu geliştirmesini ve adresini ADT'ye iletmesini gerektirir. Herhangi bir uygulama için birden fazla işlem gerekebileceğinden, fonksiyon geçiş başlatıldığında iletilmelidir, ağaç oluşturulurken değil, arama algoritmasında yaptığımız gibi.

Temel tasarım anlaşıldığında, ADT'yi tanımlamaya hazırız. Dokuz kamu fonksiyonu ve beş özel fonksiyon tanımlıyoruz; yani, yalnızca ADT içinde mevcut olan fonksiyonlar. Bu fonksiyonlar, BST ağaçlarını inşa etmek ve sürdürmek için gereken temel seti oluşturur. Özel fonksiyonlar bir alt çizgi karakteri ile başlar. Tasarım Şekil 7-11'de gösterilmiştir ve aşağıdaki bölümlerde açıklanmıştır.



ŞEKİL 7-11 BST ADT Tasarımı

Veri Yapısı

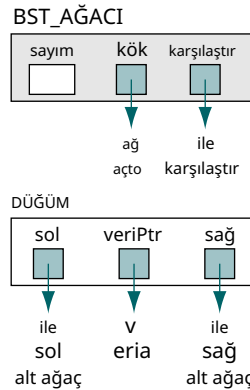
BST, baş için bir ve düğüm için bir olmak üzere iki ayrı veri yapısına ihtiyaç duyar.

Baş Yapısı

Doğrusal liste ile olduğu gibi, BST veri yapısı basit bir baş yapısı kullanır, BST_AĞACI, bir sayım, bir kök işaretçisi ve listeyi aramak için gereken karşılaştırma fonksiyonunun adresini içerir. Uygulama programının ağaç üzerindeki tek görünümü, ağaç oluşturulduğunda dinamik bellekten tahsis edilen baş yapısına bir işaretçidir.

Düğüm Yapısı

BST düğümleri, bir veri işaretçisi ve sol ve sağ alt ağaçlara iki kendine referans veren işaretçi içerir. Bu veri yapıları Şekil 7-12'de gösterilmiştir.



```
typedef yapı
{
    int sayım;
    int (*karşılaştır)
        (void* argu1,
         void* argu2);
    NODE* kök;
} BST_AĞACI;

typedef struct düğüm
{
    void* veriPtr;
    struct düğüm* sol;
    struct düğüm* sağ;
} DÜĞÜM;
```

ŞEKİL 7-12 BST Ağaç Veri Yapısı

Algoritmalar

Bu bölümde temel bir BST ağaç algoritmaları seti ele alınmaktadır. Uygulamaya bağlı olarak, başka algoritmalar da gerekebilir. Örneğin, bazı uygulamalar azalan anahtar gezintilerini içermeyi gerektirebilir. ADT veri yapıları ve prototip bildirimleri Program 7-1'de gösterilmiştir.

PROGRAM 7-1 BST Bildirimleri

1 2 3 4 5	<pre>/* İkili arama ağacı (BST) için başlık dosyası. BST için yapısal tanımlar ve prototipler içerir. Yazarı: Tarih: */</pre>
-----------------------	--

devam ediyor

PROGRAM 7-1 BST Bildirimleri (devam ediyor)

```

6789 #include <stdbool.h>

// Yapı Bildirimleri
typedef struct node
{
    void* dataPtr;
    struct düğümü* sol;
    struct düğümü* sağ;
} NODE;

typedef yapısı
{
    int sayım;
    int (*karşılaştır) (void* argu1, void* argu2);
    NODE* kök;
} BST_TREE;

// Prototip Bildirimleri
BST_TREE* BST_Oluştur
    (int (*karşılaştır) (void* argu1, void* argu2));
BST_TREE* BST_Yık (BST_TREE* ağaç);

bool BST_Ekle (BST_TREE* ağaç, void* veriPtr);
bool BST_Sil (BST_TREE* ağaç, void* dltAnahtar);
void* BST_Ai (BST_TREE* ağaç, void* anahtarPtr);
void BST_Dolaş (BST_TREE* ağaç,
                void (*işlem)(void* veriPtr));

bool BST_Boş (BST_TREE* ağaç);
bool BST_Dolu (BST_TREE* ağaç);
int BST_Sayı (BST_TREE* ağaç);

static NODE* _ekle
    (BST_TREE* ağaç, NODE* kök,
     NODE* yeniPtr);
static NODE* _sil
    (BST_TREE* ağacı, NODE* kökü,
     void* dataPtr, bool* başarı);
static void* _retrive
    (BST_TREE* ağacı,
     void* dataPtr, NODE* root);
static void _traverse
    (DÜĞÜM* kökü,
     void (*process) (void* dataPtr));
static void _destroy (NODE* root);

```

Bir BST oluştur

Soyut veri türü, bir programda birden fazla yapıyı destekleyebilmelidir. Bu , ağaç baş yapısını dinamik bellekte tahsis ederek gerçekleştirilir. Ağaç oluşturma işlemi yapıyı tahsis eder, sayısını sıfıra ayarlar ve kök işaretçilerini null olarak ayarlar ve karşılaştırma fonksiyonunun adresini saklar. Ardından ağaç işaretçisini döndürür. Kod Program 7-2'de gösterilmiştir.

PROGRAM 7-2BST Uygulama Arayüzü Oluştur

123456789	/* ===== BST_Create ===== Dinamik bellek ayırır ve bir BST ağaç başı düğümünün adresini çağırana döner Önce compare, iki düğüm karşılaştırıldığında kullanılan karşılaştırma fonksiyonunun adresidir. Sonra b aş tahsis edildi veya hata döndürüldü. Dönüş ba ş düğüm işaretçisi; taşma durumunda null */
1011213141516171819202122232425	BST_TREE* BST_Oluştur (int (*karşılaştır) (void* argüman1, void* argüman2)) { // Yerel Tanımlar BST_TREE* ağaç; // Beyanlar ağaç = (BST_TREE*) malloc (sizeof (BST_TREE)); if (ağaç) { ağaç->kök = NULL; ağaç->sayı = 0; ağaç->karşılaştır = karşılaştır; } // eğer return ağaç; } // BST_Create

Bir BST Ekle

BST düğüm ekleme fonksiyonu, uygulama programı tarafından çağrılan modüldür. Ağaç yapısına bir işaretçi ve ağaca eklenmesi gereken verilere bir işaretçi alır. Bir düğüm oluşturduktan sonra, fiziksel ekleme işlemi için bir özyineleme ekleme fonksiyonu çağırır. Kod Program 7-3'te gösterilmiştir.

PROGRAM 7-3BST Uygulama Arayüzü Ekle

12345	/* ===== BST_Insert ===== Bu fonksiyon yeni verileri ağaca ekler. Ön ağaç, BST ağaç yapısına işaretçidir Son veri eklendi veya bellek taşması Başarı (true) veya Taşma (false) döndür
-------	---

devam ediyor

PROGRAM 7-3BST Uygulama Arayüzü Ekle (devamı)

6789	*/
	bool BST_Insert (BST_TREE* ağacı, void* dataPtr)
	{
	// Yerel Tanımlar
	DÜĞÜM* yeniPtr;
	// Beyanlar
	newPtr = (NODE*)malloc(sizeof(NODE));
	if (!newPtr)
	false döndür;
	newPtr->right = NULL;
	newPtr->left = NULL;
	newPtr->dataPtr = dataPtr;
	if (tree->count == 0)
	ağaç->kök = yeniPtr;
	else
	_insert(ağaç, ağaç->kök, yeniPtr);
	(ağaç->sayım)++;
	doğruyu döndür;
	// BST_Insert
	}

Program 7-3 AnaliziTüm eklemeler bir yaprak düğümde gerçekleştiğinden, verileri yeni oluşturulan düğümde saklıyoruz ve ağaç işaretçilerini null olarak başlatıyoruz. Ardından, özyinelemeli ekleme fonksiyonunu çağırıyoruz. Döndüğünde, ağaç sayısını güncelliyoruz ve başarıyı döndürüyoruz.

Özyinelemeli ekleme modülünün adını not edin. Sistem yazılımı için kılavuzu takip ederek, adın uygulama programcıları tarafından ADT'yi programlarında kullanırken tekrar edilmemesi için `_insert` olarak adlandırıyoruz.

İçsel Ekleme Fonksiyonu

İlk olarak `BST_Insert` tarafından çağrılan içsel ekleme fonksiyonu, üç parametre gerektirir: ağaç yapısına bir işaretçi, ağacın veya alt ağacın köküne bir işaretçi ve eklenen düğüme bir işaretçi. Fonksiyon Program 7-4'te gösterilmiştir. Algoritma 7-4'teki tasarımı takip eder.

PROGRAM 7-4İçsel Ekleme Fonksiyonu

1234567	/* ===== insert ===== Bu fonksiyon, yeni veriyi BST ağacındaki bir yaprak düğümüne eklemek için özyinelemeyi kullanır. Önce Uygulama, kök ve veri işaretçisini geçiren BST_Insert'i çağırdı Sonra Veriler eklendi [potansiyel olarak] yeni köke işaretçi döndür
---------	---

devam ediyor

PROGRAM 7-4 Dahili Ekleme Fonksiyonu (devamı)

```

89  */
10112131415161718192021222324252627282930
NODE* _insert (BST_TREE* ağacı, NODE* kökü, NODE* newPtr)
{
    // Beyanlar
    eğer (!root)
        // eğer NULL ağaç
        yeniPtr'yi döndür;

    // Ekleme için null alt ağacı bul eğer (tree->compare(ne
    wPtr->dataPtr,

                                root->dataPtr) < 0)

    {
        kök->sol = _insert(ağaç, kök->sol, yeniPtr);
        dönüş kökü;
    } // yeni < node aksi

    takdirde
        // yeni veri >= kök verisi
    {
        kök->sağ = _insert(ağaç, kök->sağ, yeniPtr);
        dönüş kökü;
    } // aksi takdirde yeni veri >= kök verisi

    return root;
} // _insert

```

Program 7-4 Analizi Bu algoritmanın mantığını tam olarak anlamak için dikkatlice incelenmesi gerekir. Doğru ekleme noktasını bulmak için bir yaprak düğümünde özyinelemeli bir arama ile başlar. Bir yaprak düğümü, null olan sağ veya sol alt ağaç işaretçisi ile tanımlanır. Bir yaprak işaretçisi bulunduğumuzda, yeni düğümü döndürürüz, böylece ana işaretçiye (ifade 14) eklenebilir.

Bu bir özyinelemeli fonksiyon olduğu için, bir temel durum olmalıdır. Bunu görebiliyor musun? Temel durum, bir yaprak bulunduğumuzda ve 14. ifadede **newPtr** döndürdüğümüzde gerçekleşir. Bu noktada ağaçtan geri çıkmaya başlıyoruz.

Bir BST'yi sil

Silme işlemi için kullandığımız tasarımı, ekleme için de kullanıyoruz. Uygulama programı arayüzü yalnızca BST baş yapısını görür. Ağaç ve silinecek anahtarları içeren bir değişkene işaretçi geçirir. Silme işlemi başarılı olursa, BST silme fonksiyonu true döner; eğer düğüm bulunamazsa, false döner. Uygulama arayüzü için kod Program 7-5'te gösterilmiştir.

PROGRAM 7-5 BST Uygulama Arayüzünü Sil

```

1234  /* ===== BST_Delete =====
Bu fonksiyon ağaçtan bir düğümü siler ve gerekirse yeniden dengelemektedir.
Önce ağaç başlatıldı--boş ağaç tamamdır

```

devam ediyor

PROGRAM 7-5BST Silme Uygulama Arayüzü (devamı)

56789	dltKey, silinecek anahtarı içeren veri yapısına iş aretçidir Silinen düğüm ve alanı geri dönüştürülür -veya- Bir hata kodu dönd ürülür Başarı (true) veya Bulunamadı (false) döndür
1011121314151617181920212223242526272829	<pre> bool BST_Delete (BST_TREE* ağacı, void* dltKey) { // Yerel Tanımlar bool başarı; NODE* yeniKök; // Beyanlar yeniKök = _delete (tree, tree->root, dltKey, &ba şarı); eğer (başarı) { tree->root = yeniKök; (tree->count)--; eğer (tree->count == 0)// Ağaç şimdi boş tree->root = NULL; } // eğer başarıyı döndür; // BST_Sil </pre>

Program 7-5 Analizi Uygulama arayüzü silme fonksiyonu, kullanıcıdan parametreleri alır ve bunları özyinelemeli silme fonksiyonu için yeniden biçimlendirir. Silme işlemi tamamlandığında, ağaç sayısını günceller ve eğer ağaç boşsa, kökü null olarak ayarlar. Daha sonra silme fonksiyonu başarılıysa true, anahtar bulunamazsa false döner.

İçsel Silme Fonksiyonu

Gerçek iş, Program 7-6'da gösterilen içsel silme fonksiyonu tarafından yapılmaktadır. Algoritma 7-5'teki tasarımı kullanır.

PROGRAM 7-6Dahili Silme Fonksiyonu

123456789	<pre> /* ===== _delete ===== Düğümü ağaçtan siler ve gerekiyorsa ağacı dengeleyerek. Önce ağaç başlatıldı--boş ağaç kabul edilebilir dataPtr silinecek düğümün anahtarını içerir Sonra düğüm silinir ve alanı geri dönüştürülür -veya- eğer anahtar bulunamazsa, ağaç değişmez başarı silindiyse doğrudur; değilse yanlışır Kök işaretçisini döndür </pre>
-----------	--

devam ediyor

PROGRAM 7-6 Dahili Silme Fonksiyonu (devamı)

```

101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657
*/
NODE* _delete (BST_TREE* ağacı, NODE* kökü,
               void* dataPtr, bool* başarı)
{
    // Yerel Tanımlar
    DÜĞÜM* dltPtr;
    DÜĞÜM* exchPtr;
    NODE* yeniKök;
    void* holdPtr;

    // Beyanlar
    eğer (!root)
    {
        *başarı = false;
        return NULL;
    } // eğer

    if (tree->compare(dataPtr, root->dataPtr) < 0)
        root->left = _delete (tree, root->left,
                              dataPtr, başarı);

    else if (tree->compare(dataPtr, root->dataPtr) > 0)
        root->right = _delete (tree, root->right,
                              dataPtr, başarı);

    aksi takdirde
        // Düğüm bulundu-yaprak düğüm testi
        {
            dltPtr = root;
            if (!root->left)
                // Sol alt ağaç yok
                {
                    ücretsiz (kök->dataPtr); // veri belleği
                    newRoot = kök->sağ;
                    ücretsiz (dltPtr); // BST Düğümü
                    *başarı = true;
                    newRoot'u döndür; // temel durum
                } // eğer doğruysa

            başka
            if (!root->right) // Sadece sol alt
                ağaç
                {
                    newRoot = kök->sol;
                    ücretsiz (dltPtr);
                    *başarı = true;
                    newRoot'u döndür; // temel durum
                } // eğer

            // Düğümü sil, iki alt ağaç içerir
            {

```

devam ediyor

PROGRAM 7-6 Dahili Silme Fonksiyonu (devamı)

```

58596061626364656667686970717273
    exchPtr = root->left;
    // Sol alt ağaçta en büyük düğümü bul

    exchPtr = exchPtr->right;

    // Verileri Değiştir
    holdPtr = root->dataPtr;
    root->dataPtr = exchPtr->dataPtr;
    exchPtr->dataPtr = holdPtr;
    kök->sol =

        _delete (tree, root->left,
                exchPtr->dataPtr, success);

    } // düğüm bulundu
    return root;
} // _delete

```

Program 7-6 Analizi Özyinelemeli ekleme ile gördüğümüz gibi, fonksiyonun ilk kısmı silinecek düğümü arar. Eğer silinecek düğümü bulmadan önce bir yaprağa ulaşırsa, anahtar bulunamaz, bu yüzden false döner. Bu, üç temel durumdan ilki.

Silinecek düğüm bulunduğunda, bunun bir yaprak mı yoksa yaprak benzeri bir düğüm mü olduğunu belirleriz. Silme işlemlerinin yalnızca bir yaprakta gerçekleşebileceğini unutmayın. Eğer bir düğümün iki alt ağacı varsa, bunun yerini alacak bir yaprak düğüm aramalıyız. Öncelikle sol alt ağacın olup olmadığını kontrol ederiz. Eğer yoksa, sağ alt ağaç işaretçisini kökün yerini alacak işaretçi olarak kaydederiz. Eğer bir sol alt ağaç varsa, sağ alt ağaç işaretçisini kontrol ederiz. Eğer yoksa, sol alt ağaç işaretçisini kökün yerini alacak işaretçi olarak kaydederiz.

Şu anda sıfır veya bir alt ağaç olduğunu varsayarsak, silinen düğümün belleğini serbest bırakırız ve true döneriz. Bunlar ikinci (ifade 44) ve üçüncü (ifade 53) temel durumlar.

Eğer silinecek düğümün iki alt ağacı varsa, bunun yerini alacak bir düğüm bulmalıyız. Tasarımımız, sol alt ağaçta en büyük düğümü arar. Bunu bulduğumuzda, verilerini silinen verilerin yerine geçirmek için taşıırız ve ardından geçerli bir yaprak düğüm olduğunu bildiğimiz şeyi silmek için silme fonksiyonunu özyinelemeli olarak çağırırız. Bu mantık, ifadeler 57 ile 70 arasında gösterilmektedir.

Bir BST al

Alma fonksiyonu, istenen düğüm bulunana kadar ağacın sol-sağ yapısını takip eder. Bulduğunda, verinin adresi çağırılan fonksiyona döndürülür. Veriler bulunamazsa, bir null işaretçi döndürülür.

Tasarım, yukarıda açıklanan ekleme ve silme algoritmalarına benzer. Bir uygulama arayüzü fonksiyonu, ağaca ve anahtara bir işaretçi sağlar. Daha sonra veriyi bulmak için bir ADT özyinelemeli fonksiyonu çağırırız. ADT alma fonksiyonunun kodu Program 7-7'de gösterilmiştir.

PROGRAM 7-7BST Uygulama Arayüzünü AI

123456789	/* ===== BST_Retrieve ===== Düğüm alma, istenen anahtarı içeren düğümü arar ve datasına işaretçi döner. Ön Ağaç oluşturulmuş (null olabilir) veri, bulunması gereken anahtarı içeren data yapısına işaretçidir Son Ağaç arandı ve veri işaretçisi döndürüldü Dönüş Eşleşen düğümün adresi döndürüldü Bulunamazsa, NULL döner
101112131415161718	*/ void* BST_Retrieve (BST_TREE* ağacı, void* keyPtr) { // Beyanlar if (ağaç->kök) return _retrieve (ağaç, keyPtr, ağaç->kök); else NULL'u döndür; } // BST_Retrieve

Program 7-7 AnaliziVeri alma fonksiyonu oldukça basittir. Sadece, özyinelemeli fonksiyonunek bir parametreye ihtiyaç duyması nedeniyle gereklidir; bu parametre, kullanan uygulama tarafından bilinmemektedir. Basitçe ağaç yapısına ve anahtara işaretçiyi geçirir, ağaç köküne işaretçiyi ekler.

İçsel Alma Fonksiyonu

İçsel alma fonksiyonu için kod Program 7-8'de gösterilmiştir.

PROGRAM 7-8İçsel Alma Fonksiyonu

123456789	/* ===== _retrieve ===== Aranan anahtarı içeren düğümü bulmak için ağacı arar ve verilerini çağıran işlevine döndürür. Önce _retrieve ağaç, dataPtr, kök geçirir dataPtr , bulunacak anahtarı içeren veri yapısına işare tçidir Sonra ağaç arandı; veri işaretçisi döndürüldü Dönüş Eşleşen düğümdeki verinin adresi Bulunamazsa, NULL döndürülür
101112131415	*/ void* _retrieve (BST_TREE* ağaç, void d* dataPtr, NODE* kök) { // Beyanlar eğer (kök)

devam ediyor

PROGRAM 7-8 Dahili Alma Fonksiyonu (devamı)

```

1617181920212223242526272829
{
    if (tree->compare(dataPtr, root->dataPtr) < 0)
        return _retrieve(tree, dataPtr, root->left);
    else if (tree->compare(dataPtr,
        root->dataPtr) > 0)
        return _retrieve(tree, dataPtr, root->right);    else

        // Eşit anahtar bulundu
        return root->dataPtr;
    } // eğer kök

aksi takdirde
    // Ağaçta veri yok
    return NULL;
} // _retrieve

```

Program 7-8 AnaliziAlma fonksiyonu, ağaç yapısında depolanan karşılaştırma fonksiyonunu kullanır ağaç oluşturulduğunda. Arama argümanındaki veri, **dataPtr**, kök verinin anahtarından daha azsa, kendisini sol alt ağaç ile kök olarak çağırır. Arama argümanı kök verinin anahtarından büyükse, kendisini sağ alt ağaç ile kök olarak çağırır. Eğer argüman ne büyük ne de küçükse, eşit olmalıdır, bu yüzden kökün veri işaretçisini döndürür.

İfadelerdeki 18 ve 21 numaralı rekürsif çağrıların işlevlerini dikkatlice inceleyin. Bunlarınrekürsif çağrı t arafından döndürülen adresi döndürdüğünü not edin. Bu şekilde, temel durumlar tarafından döndürülen a dres,ağaç yapısında yukarı doğru iletilir ve nihayetinde arayüz fonksiyonu tarafından orijinaluygulama çağrısı na geri döndürülür.

Bir BST'yi Gezinme

Gezinme fonksiyonu, ağacın sıralı bir gezinmesini kullanır ve düğüm i şlenirken uygulama-bağımlı işlem fonksiyonunu çağırır. Geçiş standart olmasına rağmen, uygulama işleme fonksiyonu standart değildir. Bu nedenle, kullanan uygulama gezinme fonksiyonunu çağırdığında, verileri işleyen fonksiyonun adresini de geçirmelidir. İşlemefonksiyon u yalnızca bir parametre kullanır: işlenecek düğümün adresi. Bu, aslında ağacı gezen ADT iç fonksiyonunu çağırır. Kod Program 7-9'da göst erilmiştir.

PROGRAM 7-9BST Uygulama Arayüzünü Gezinme

```

123456789
/* ===== BST_Gezinme =====
Ağacı sıralı gezinti kullanarak işleyin.
Ön Ağaç oluşturulmuş (null olabilir)
işlem "gezinme" sırasında düğümleri "ziyaret eder"
Son Düğümler LNR (sıralı) sırasına göre işlenmiştir
*/
void BST_Gezinme (BST_TREE* tree,
void (*process) (void* dataPtr))
{

```

devam ediyor

PROGRAM 7-9 BST Uygulama Arayüzünü Gezinme (devamı)

10111213	<pre>// Beyanlar _gez (ağaç->kök, işlem); dön; , // BST_Traverse'ı sonlandır</pre>
----------	---

İçsel Gezinme Fonksiyonu

İçsel arama fonksiyonunun kodu Program 7-10'da gösterilmiştir.

PROGRAM 7-10 İçsel Gezinme Fonksiyonu

123456789	<pre>/* ===== _traverse ===== Ağaçta sıralı gezinti. Bir düğümü işlemek için, gezinti çağrıldığı nda geçirilen fonksiyonu kullanıyoruz. Önce Ağaç oluşturuldu (null olabilir) Sonra Tüm düğümler işlendi */ void _traverse (NODE* root, void (*process) (void* dataPtr)) { // Beyanlar eğer (kök) { _traverse (kök->sol, süreç); süreç (kök->dataPtr); _traverse (kök->sağ, süreç); } // if geri dön; } // _traverse</pre>
101112131415161718	

Bir BST'yi Boşalt

Boş fonksiyon, ağacın sayısını kontrol eder. Eğer sıfırsa, true döner; aksi takdirde, false döner. Kod Program 7-11'de görülmektedir.

PROGRAM 7-11 Boş BST Uygulama Arayüzü

123456789	<pre>/* ===== BST_Empty ===== Ağaç boşsa true, herhangi bir veri varsa false döner. Önce Ağaç oluşturuldu. (Null olabilir) Ağaç boşsa true, herhangi bir veri varsa false döner */ bool BST_Bos (BST_TREE* ağaç) { // Beyanlar return (ağaç->count == 0); } // BST_Bos</pre>
10	

Tam BST

C'de mevcut belleği test etmenin bir yolu olmadığından, aslında bir düğüm ayırmayı denememiz gerekiyor. Başarılı olursak, onu sileriz ve false döneriz—liste dolu değildir. Ayırma başarısız olursa, true döneriz; başka bir düğüm için yeterli bellek yoktur. Kod Program 7-12'de gösterilmiştir.

PROGRAM 7-12 Tam BST Uygulama Arayüzü

123456789	/* ===== BST_Doluluk ===== Başka bir düğüm için yer yoksa, true döner. Ön ağaç oluşturulmuş Başka bir ekleme için yer yoksa true döner yer varsa false
101112131415161718192021	*/ bool BST_Doluluk (BST_AĞAÇ* ağaç) { // Yerel Tanımlar DÜĞÜM* yeniPtr; // Beyanlar newPtr = (DÜĞÜM*)malloc(sizeof (*(ağaç->kök))); if (newPtr) { free (newPtr); return false; } // eğer aksi takdirde return true; } // BST_Doluluk

BST Sayısı

Sayı fonksiyonu, Program 7-13'te gösterildiği gibi, ağaçta şu anda bulunan düğüm sayısını basitçe döner.

PROGRAM 7-13 BST Sayım Uygulama Arayüzü

123456789	/* ===== BST_Sayı ===== Ağaçtaki düğüm sayısını döner. Ön ağaç oluşturulmuş Ağaç sayısını döner
10	*/ int BST_Sayı (BST_AĞAÇ* ağaç) { // İfadeler return (tree->count); } // BST_Count

Bir BST'yi yok et

BST ağaç soyut veri türündeki son işlev yok etme işlevidir. Bu, ağaç artık gerekli ol madığında tüm veri düğümlerini ve ağaç baş yapısını fiziksel olarak silmek ve serbest bırakmak için kullanılır. Silinmesi gereken tüm veri ve düğümleri bulmak için ağacı gezmemiz gerektiğinden, fiziksel silme işlemlerini gerçekleştirmek için bir özyinelemeli işlev çağırıyoruz.

Yok etme işlevinin mantığı daha önce tartıştığımız yok etme işlevle riyle paraleldir. Kod Program 7-14'te gösterilmiştir.

PROGRAM 7-14 BST Uygulama Arayüzünü Yok Et

123456789	/* ===== BST_Destroy ===== Ağaçtaki tüm verileri siler ve belleği geri kazanır. Düğümmler, ağacı sıralı sırayla gezmek için bir rekürsif fonksiyon çağrılarak silinir. Önce geçerli bir ağaca işaret eden bir işaretçi Sonra Tüm veriler ve baş yapısı silindi Dönüş null baş işaretçisi
101112131415161718	*/ BST_TREE* BST_Destroy (BST_TREE* ağacı) { // Beyanlar eğer (ağaç) _destroy (tree->root); // Tüm düğümler silindi. Yapıyı serbest bırak free (tree); NULL değerini döndür; } // BST_Destroy

Program 7-14 Analizi Mantık basit. Öncelikle, ağaç işaretçisini test ederek geçerli bir ağaç olduğundan emin oluyoruz. Eğer geçerliyse—yani, null değilse—fiziksel silme işlemlerini gerçekleştiren rekürsif alt fonksiyonu çağırıyoruz. Döndüğümüzde, ağaç yapısını kendisini siliyoruz ve null işaretçi döndürüyoruz.

İçsel Yok Etme Fonksiyonu

İçsel silme fonksiyonunun kodu Program 7-15'te gösterilmiştir.

PROGRAM 7-15 İçsel Yok Etme Fonksiyonu

12345	/* ===== _destroy ===== Ağaçtaki tüm verileri siler ve belleği geri döndürür. Ayrıca anahtar ve veri düğümleri için belleği geri döndürür. Düğümmler, ağacı sıralı sırayla gezmek için bir özyinelemeli fonksiyonu çağırılarak silinir.
-------	---

devam ediyor

PROGRAM 7-15İşsel Yok Etme Fonksiyonu (devamı)

6789	Ön kök geçerli ağaç/alt ağaç için işaretçidir Son Tüm veriler ve baş yapısı silindi Dönüş null baş işaretçisi
101112131415161718192021	<pre> */ void _destroy (NODE* kökü) { // İfadeler if (root) { yok et (kök->sol); \in bedava (kök->dataPtr); _ yok et (kö k->sağ); bedava (kök); } // if return; } // _destroy </pre>

Program 7-15 AnaliziÖzyinelemeli silme fonksiyonundaki mantık biraz daha karmaşık. Büyük soru şudur:

Verileri ne zaman silmeliyiz ve düğümü ne zaman silmeliyiz? Her birini yalnızca bir kez ve doğru zamanda yaptığımızdan emin olmalıyız.

Veriler, sol alt ağaçtan dönerken silinir. Bu, 16. ifadede yapılır.

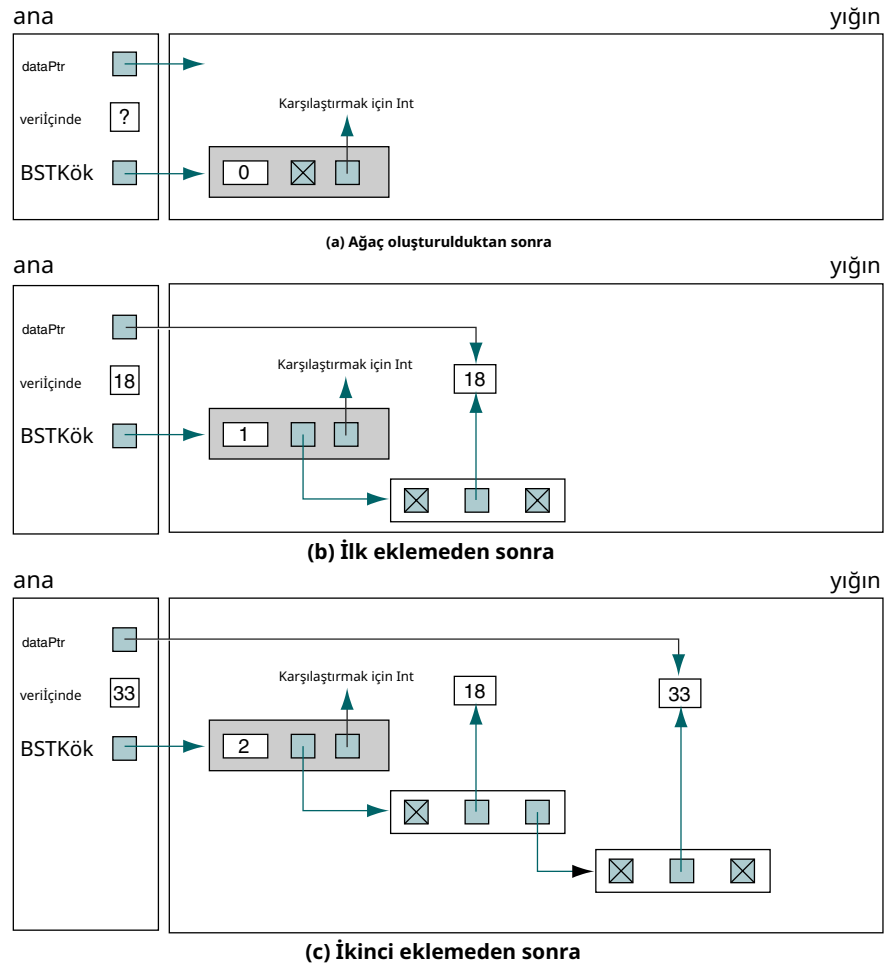
Bu, bir BST'nin verilerini işlemek için mantıksal bir noktadır. Ancak, sağ alt ağacı işlemediğimiz için düğümü silmeye henüz hazır değiliz. Onu silmek için sağ alt ağaç gezintisinden döndüğümüzü beklemeliyiz (bkz. 18. ifade).

7.4 BST Uygulamaları

Artık BST ADT'yi geliştirdiğimize göre, onu kullanan iki uygulama yazalım. İlki yalnızca tam sayı verileri içeren bir BST'dir. İkincisi, bir sınıftaki öğrenci leri öğrenci numaralarına göre listeleyen basit bir uygulamadır.

Tamsayı Uygulaması

BST ağaç tamsayı uygulaması, klavyeden tam sayıları okur ve bunları BST'ye ekleir. Şekil 7-13, null BST'ye yapılan ilk iki eklemeyi, 18 ve 33'ü izler. Programı takip etmenize yardımcı olmak için bunu kullanın.



ŞEKİL 7-13 BST'ye Ekleme

Program çok basit. Bir ağaç oluşturduktan sonra, klavyeden veri okumak için döngüye giriyoruz ve bunu ağaca eklemek için `BST_Insert` çağırıyoruz. Ağaç tamamlandığında, BST gezinti fonksiyonunu çağırarak yazdırıyoruz. Kod Program 7-16'da gösterilmiştir. *main* dışında, karşılaştırma veya yazdırma fonksiyonlarını da içerir.

PROGRAM 7-16BST Tam Sayı Uygulaması

```

123456789  /* Bu program bir BST oluşturur ve yazdırır.
              Yazarı:
              Tarih:
            */
            #include <stdio.h>
            #include <stdlib.h>
            #include "P7-BST-ADT.h"

            // Prototip Bildirimleri
            int compareInt (void* num1, void* num2);
            void printBST   (void* num1);

            int ana (geçersiz)
            {
            // Yerel Tanımlar
                BST_TREE* BSTRoot;
                int*      dataPtr;
                int       dataIn = +1;

            // İfadeler
                printf("BST Gösterimi Başlıyor");BSTRoot = B

                ST_Create (compareInt);// Ağaç Oluştur

                printf("Pozitif tam sayılardan oluşan bir liste girin;");printf("D
                urdurmak için negatif bir sayı girin.");do

                {
                    printf("Bir sayı girin: ");
                    scanf ("%d", &dataIn);
                    if (veriGirişi > -1)
                    {
                        dataPtr = (int*) malloc (sizeof (int));
                        if (!dataPtr)
                        {
                            printf("Eklemede Bellek Taşması"),          exi
                                t(100);
                        } // taşma durumu
                        *dataPtr = dataIn;
                        BST_Insert (BSTRoot, dataPtr);
                    } // geçerli veri
                } while (dataIn > -1);

```

devam ediyor

PROGRAM 7-16BST Tam Sayı Uygulaması (devam ediyor)

```

4647484950515253545556575859606162636465666768697071727374
4647484950515253545556575859606162636465666768697071727374
75767778798081828384

printf("BST içerir:");BST_Traverse (B
STRoot, printBST);printf("BST Gösterimi So

nu");return 0;

} // ana

/* ===== compareInt =====
İki tam sayıyı karşılaştır ve düşük, eşit, yüksek döndür.
Önce num1 ve num2 geçerli tam sayı işaretçileri
Sonra düşük (-1), eşit (0) veya yüksek (+1) döndür
*/
int CompareInt (void* sayı1, void* sayı2)
{
// Yerel Tanımlar
int key1;
int anahtar2;

// Beyanlar
anahtar1 = *(int*)num1;
anahtar2 = *(int*)num2;
eğer (anahtar1 < anahtar2)
dönüş -1;
if (anahtar1 == tuş2)
0 değerini döndür;
+1'i döndür;
} // compareInt

/* ===== printBST =====
BST'den bir tam sayıyı yazdırın.
Ön num1 bir tam sayıya işaretçidir
Son tam sayı yazdırıldı ve satır ilerletildi
*/
void printBST (void* sayı1)
{
// Beyanlar
printf("%4d", *(int*)num1);return;
} // printBST

```

Sonuçlar:

BST Demonstre Başlangıcı
Pozitif tam sayılardan oluşan bir liste girin;
Durdurmak için negatif bir sayı girin.
Bir sayı girin: 18

devam ediyor

PROGRAM 7-16 BST Tam Sayı Uygulaması (devam ediyor)

```

Bir sayı girin: 33
Bir sayı girin: 7
Bir sayı girin: 24
Bir sayı girin: 19
Bir sayı girin: -1

```

BST içerir:

```

  7
 18
 19
 24
 33

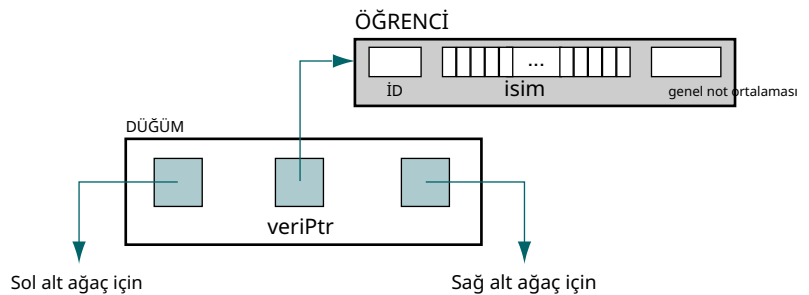
```

BST Gösterimini Bitir

Program 7-16 Analizi Bir uygulamanın bir ADT kullanması gerektiği zamanlar vardır, ancak ADT'nin tasarımı uygulamaya tam olarak uymamaktadır. Bu, o durumlardan biridir. ADT bir anahtar ve data bekler. Ancak, ağacımız yalnızca veriye ihtiyaç duyar. Sorunu, veriyi hem anahtar hem de veri olarak göndererek çözüyoruz.

Öğrenci Listesi Uygulaması

İkinci uygulama bir öğrenci listesi oluşturur. Üç parça veri gerektirir: öğrencinin kimlik numarası, öğrencinin adı ve öğrencinin not ortalaması. Öğrenciler klavye üzerinden eklenir ve silinir. Tekil olarak veya bir liste olarak geri alınabilirler. Öğrenci yapıları ve ADT ile olan ilişkileri Şekil 7-14'te gösterilmiştir.

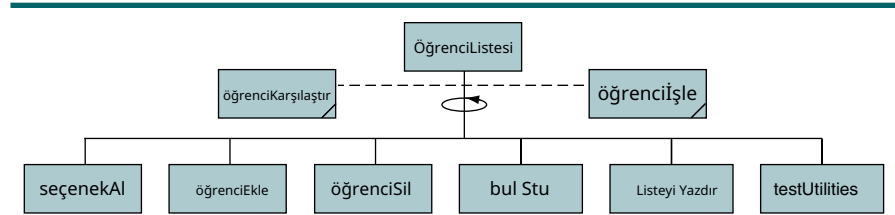


ŞEKİL 7-14 ADT'deki Öğrenci Verileri

Program Tasarımı

Program tasarımı oldukça basittir. Bir menü kullanarak bir öğrencinin verilerini ekleyebilir, silebilir, yazdırabilir veya bir öğrenci listesi yazdırabiliriz. ADT'yi tam olarak kullanabilmek için, üç yardımcı işlevi çağıran bir uygulama dışı işlev olan test yardımcılarını da ekliyoruz.

Bu basit bir örnek olduğu için, işleme kontrolü için *main* içinde bir döngü kullanıyoruz. Döngüye ek olarak, *main* ADT'yi çağırarak dosyayı oluşturur ve sonunda yine ADT'yi çağırarak siler. Şekil 7-15 program tasarımını göstermektedir. Ayrıca ADT tarafından gereken iki işleme işlevini de gösteriyoruz: karşılaştırma ve işleme. Her ne kadar programda doğrudan kullanılmaları da, ADT tarafından çağırılırlar. Dolaylı kullanımlarını kesikli çizgilerle gösteriyoruz. Geleneksel olarak, ADT çağrıları tasarımda gösterilmez.



ŞEKİL 7-15Öğrenci Listesi Tasarımı

Programın veri yapısı, prototip bildirimleri, ana akışı ve bütün kod Program 7-17'de gösterilmiştir. Program uzun olmasına rağmen, kod basit ve anlaşılırdır.

PROGRAM 7-17BST Öğrenci Uygulaması

```

123456789 /* Bu program bir öğrenci listesi oluşturur ve yazdırır.
          Yazarı:
          Tarih:
          */
          #include <stdio.h>
          #include <stdlib.h>
          #include <ctype.h>

          #include "P7-BST-ADT.h"

          // Yapılar
          typedef yapı
          {
              int id;
              karakter adı[40];
              float gpa;
  
```

devam ediyor

PROGRAM 7-17BST Öğrenci Uygulaması (devam ediyor)

```

1718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263
} ÖĞRENCİ;

// Prototip Bildirimleri
char seçenekAl (void);
void öğrenciEkle (BST_AĞACI* liste);
void öğrenciSil (BST_AĞACI* liste);
void öğrenciBul (BST_AĞACI* liste);
void listeYazdır (BST_AĞACI* liste);
void testAraçları (BST_AĞACI* ağaç);
int öğrenciKarşılaştırm (void* stu1, void* stu2);
void öğrenciİşle (void* veriPtr);

int ana (geçersiz)
{
// Yerel Tanımlar
    BST_AĞACI* liste;
    char seçenek = ' ';

// İfadeler
    printf("Öğrenci Listesi Başlıyor");list = B
    ST_Create (compareStu);while ( (option

    = getOption ()) != 'Q') {

        switch (seçenek)
        {
            case 'A': addStu (liste);
                                                    break;

            case 'D': deleteStu (liste);
                                                    break;

            case 'F': findStu (liste);
                                                    break;

            case 'P': printList (liste);
                                                    break;

            case 'U': testUtilities (liste);
                                                    break;

            } // anahtar
        } // while
        liste = BST_Destroy (liste);printf("

        Öğrenci Listesi Sonu");return 0;

    }// ana

/* ===== getOption =====
    Klavye üzerinden işleme seçeneğini okur ve doğrular.
    Önce hiçbir şey

```

devam ediyor

PROGRAM 7-17BST Öğrenci Uygulaması (devam ediyor)

```

646566676869707172737475767778798081828384858687888990919293949596979899100101102103104105106107108109110
Sonra geçerli seçenek döndürüldü

*/
char getOption (geçersiz)
{
// Yerel Tanımlar
    char option;
    bool hata;

// Beyanlar
    printf("===== MENÜ =====");printf(" A Ö
    ğrenci Ekle");
    printf(" D Öğrenci Sil");printf(" F Öğren
    ci Bul");printf(" P Sınıf Listesini Yazdır
    ");printf(" U Araçları Göster");printf(" Q Çı
    k");

    yap
    {
        printf("Seçeneği Girin: "); scanf("
        %c", &option);
        seçenek = toupper(seçenek);
        if (seçenek == 'A' || seçenek == 'D'
            || seçenek == 'F' || seçenek == 'P'
            || seçenek == 'U' || seçenek == 'Q')
            hata = false;

        başka
        {
            printf("Geçersiz seçenek. Lütfen tekrar girin: ");
            hata = true;
        } // eğer
    } while (hata == true);
    return seçenek;
} // getOption

/* ===== addStu =====
    Bir öğrenciyi ağa ekler.
    Önce hiçbir şey
    Öğrenci eklendi (bellek taşması durumunda iptal et)
*/
void addStu (BST_TREE* liste)
{
// Yerel Tanımlar
    ÖĞRENCİ* stuPtr;

// İfadeler
    stuPtr = (ÖĞRENCİ*)malloc (sizeof (ÖĞRENCİ));

```

devam ediyor

PROGRAM 7-17BST Öğrenci Uygulaması (devam ediyor)

```

111112113114115116117118119120121122123124125126127128129130131132133134135136137138139140141142143144145146147148149150151152153154155156157

    if (!stuPtr)
        printf("Eklemede Bellek Taşması"), exit(101);printf("Öğrenci

        kimliğini girin: ");scanf ("%d", &(stuPtr
        ->id));printf("Öğrenci adını girin: ");sc
        anf ("%39s", stuPtr->name);printf("Öğre
        nci not ortalamasını girin: ");scanf ("
        %f", &(stuPtr->gpa));BST_Insert (list, st
        uPtr);

} // addStu

/* ===== deleteStu =====
Ağaçtan bir öğrenciyi siler.
Önce hiçbir şey
Öğrenci silindi veya hata mesajı yazdırıldı
*/
void deleteStu (BST_TREE* listesi)
{
// Yerel Tanımlar
    int id;
    int* idPtr = &id;

// İfadeler
    printf("Öğrenci kimliğini girin: ");
    scanf ("%d", idPtr);

    if (!BST_Delete (list, idPtr)) printf("HAT
        A: Öğrenci Yok: %0d", *idPtr);// deleteStu

/* ===== findStu =====
Bir öğrenciyi bulur ve adını ve gpa'sını yazdırır.
Önce öğrenci kimliği
Öğrenci verileri yazdırıldı veya hata mesajı
*/
void findStu (BST_TREE* listesi)
{
// Yerel Tanımlar
    int id;
    ÖĞRENCİ* stuPtr;

// İfadeler
    printf("Öğrenci kimliğini girin: ");
    scanf ("%d", &id);

```

devam ediyor

PROGRAM 7-17BST Öğrenci Uygulaması (devam ediyor)

```

158159160161162163164165166167168169170171172173174175176177178179180181182183184185186187188189190191192193194195196197198199200201202203204
stuPtr = (ÖĞRENCİ*)BST_Retrieve(list, &id);
if (stuPtr)
{
    printf("Öğrenci kimliği: %04d", id); printf("
    Öğrenci adı: %s", stuPtr->name); printf("Öğrenci gpa:
    %4.1f", stuPtr->gpa); } // if

aksi takdirde
    printf("Öğrenci %d dosyada yok", id);} // findSt
u

/* ===== printList =====
Öğrencilerin listesini yazdırır.
Önce liste oluşturulmuş (null olabilir)
Sonra öğrenciler yazdırıldı
*/
void printList (BST_TREE* listesi)
{
    // Beyanlar
    printf("Öğrenci Listesi:");BST_Traverse
    (list, processStu);printf("Öğrenci Listesi
    Sonu");return;

} //baskıListesi

/* ===== testUtilities =====
Bu fonksiyon, her birini sırayla çağırarak ADT yardımcı programlarını test edery
azdırılan sonuçları.
Önce ağaç oluşturulmuş
Sonra sonuçlar yazdırıldı
*/
void testUtilities (BST_TREE* ağacı)
{
    // Beyanlar
    printf("Ağaç %3d düğüm içeriyor: ", BST_Count
    (tree));
    eğer (BST_Empty(tree))
        printf("Ağaç BOŞ");else

        printf("Ağaç BOŞ DEĞİL");eğer (BST_Full(tree))
    e) printf("Ağaç DOLU\
    a");else

        printf("Ağaç DOLU DEĞİL");dön;

} // testUtilities

```

devam ediyor

PROGRAM 7-17BST Öğrenci Uygulaması (devam ediyor)

```

205206207208209210211212213214215216217218219220221222223224225226227228229230231232233234235236237238239240241242243244
/* ===== compareStu =====
İki öğrenci kimliğini karşılaştır ve düşük, eşit, yüksek döndür.
Önce stu1 ve stu2 geçerli öğrenci işaretçileri
Sonra düşük (-1), eşit (0) veya yüksek (+1) döndür
*/
int compareStu (void* stu1, void* stu2)
{
// Yerel Tanımlar
ÖĞRENCİ s1;
ÖĞRENCİ s2;

// Beyanlar
s1 = *(ÖĞRENCİ*)stu1;
s2 = *(ÖĞRENCİ*)stu2;

eğer ( s1.id < s2.id)
    dönüş -1;

eğer ( s1.id == s2.id)
    0 değerini döndür;

+1'i döndür;
} // compareStu

/* ===== processStu =====
Bir öğrencinin verilerini yazdır.
Pre stu, bir öğrenciyi işaret eder
Post veriler yazdırıldı ve satır ilerletildi
*/
void prosesStu (void* stuPtr)
{
// Yerel Tanımlar
ÖĞRENCİ aStu;

// Beyanlar
aStu = *(ÖĞRENCİ*)stuPtr; printf
("%04d %-40s %4.1f", aStu.id, aStu.name, aStu.gpa);return;

} // süreçStu

```

7.5 İğneli Ağaçlar

İkili ağaç gezinti algoritmaları ya özyineleme ya da

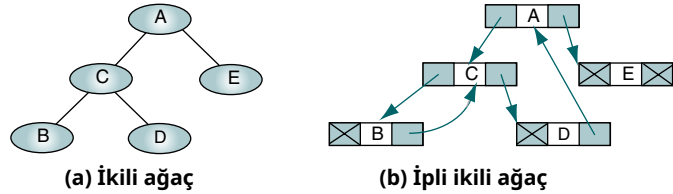
programcı tarafından yazılan yığınlar kullanılarak yazılır. Ağaç sık sık gezilmesi gerekiyorsa, yığınlar kullanmak özyinelemeden daha verimli olabilir. Üçüncü bir alternatif ise

ipli ağaç. İpli bir ağaçta, null işaretçileri, ardıl düğümlerine işaret eden işaretçilerle değiştirilir.

Her çağrı için bir yığın kullanmak, ikili ağaç gezintisini nispeten verimsiz hale getirir, özellikle ağaç sık sık gezilmesi gerekiyorsa. Her adımda, sıradaki düğüme doğrudan erişemediğimiz ve geri izleme kullanmamız gerektiği için rekürsyon veya yığın kullanıyoruz. Ağaç ipli bir ağaçsa, gezinti daha verimlidir.

Örneğin, bir ikili ağacın sıralı gezintisinde, sol alt ağacı, düğümü ve sağ alt ağacı gezmemiz gerekir. Çünkü sıralı gezinti, derinlik öncelikli bir gezintidir, sol işaretçileri en sol yaprağa kadar takip ederiz.

En sol yaprağı bulduğumuzda, aşağıya doğru geçerken geçtiğimiz sağ alt ağaçları işlemek için geri izlemeye başlamalıyız. Bu, özellikle ebeveyn düğümün sağ alt ağacı yoksa verimsizdir. Şekil 7-16(a)'da gösterilen ağacı düşünün.



ŞEKİL 7-16İpli İkili Ağaç

Bu ağacın sıralı gezintisi BCDAE'dir. Ağacı sıralı sırayla gezerken, ilk düğümü almak için sol işaretçileri takip ederiz, B. Ancak, en sol düğümü bulduktan sonra, C'ye geri dönmemiz gerekir, bu yüzden tekrar çağırma veya bir yığın kullanmamız gerekir. B'yi işlerken, B'nin sağ alt ağacının boş olduğu için tekrar çağırmaya veya bir yığına ihtiyacımız yoktur.

Aynı şekilde, düğüm D işleme tamamlandığında, özyineleme veya yığınlar kullanarak, A'ya gitmeden önce C düğümüne geri dönmeliyiz. Ama yine, C üzerinden geçmemize gerek yok. İşlenecek bir sonraki düğüm kök, A'dır.

Bu küçük örnekten, sağ alt ağaçları boş olan düğümlerin özyineleme veya yığınlar kullandığımızda daha fazla iş yarattığı açık olmalıdır. Bu, ipli kavramına yol açar: sağ alt ağaç işaretçisi boş olduğunda, düğümün ardılına işaret etmek için işaretçiyi kullanabiliriz. Diğer bir deyişle, sağ boş işaretçiyi bir iplik olarak kullanabiliriz. İpli ağaç Şekil 7-16(b)'de gösterilmiştir.

İpli bir ağaç inşa etmek için önce standart bir ikili arama ağacı inşa edin. Ardından, ağacı gezerek, boş sağ işaretçileri ardıllarına işaret edecek şekilde değiştirin.

İpli bir ağaç için gezinti basittir. En soldaki düğümü bulduğunuzda, ipli (sağ işaretçi) takip ederek bir sonraki düğüme döngü yaparsınız. Özyineleme veya yığın gerekmez. Bir boş iplik (sağ işaretçi) bulduğunuzda, gezinti tamamlanır.

7.6 Anahtar Terimler

ikili arama ağacı (BST)
 ikili ağaç arama
 ikili ağaç gezintisi
 düğüm sil

en büyük düğümü bul
 en küçük düğümü bul
 düğüm ekle
 ipli ağaç

7.7 Özet

- Bir ikili arama ağacı, aşağıdaki özelliklere sahip bir ikili ağaçtır:
 - Sol alt ağaçtaki tüm öğeler kökten küçüktür.
 - Sağ alt ağaçtaki tüm öğeler kökten büyük veya eşittir.
 - Her alt ağaç kendisi bir ikili arama ağacıdır.
- Bir ikili arama ağacının sıralı gezintisi sıralı bir liste üretir.
- Bir ikili arama ağacında, en küçük değere sahip düğüm en soldaki düğümdür ağaçta. En küçük düğümü bulmak için, null bir sol işaretçiye ulaşana kadar sol dalları takip ederiz.
- Bir ikili arama ağacında, en büyük değere sahip düğüm en sağdakidüğümdür. En büyük düğümü bulmak için, null bir sağ işaretçiye ulaşana kadardaha sağ dalları takip ederiz.
- Bir ikili arama ağacında bir değeri aramak için, önce hedefdeğeri kök ile karşılaştırırız. Hedef değer kökten küçükse, sol alt ağaç içinişlemi tekrarlarız. Hedef değer kökten büyükse, sağ alt ağaç için işlemi tekrarlarız. Hedef değer köke eşitse, arama tamamlanır.
- Bir ikili arama ağacına bir düğüm eklemek için, yeni düğümün değerine bağlı olarak ağacın sol veya sağ dalını takip ederiz, null bir alt ağaç bulana kadar.
- Bir alt ağaçtan bir düğümü silmek için dört durumu dikkate almalıyız:
 1. Silinecek düğümün çocukları yoktur.
 2. Silinecek düğümün yalnızca bir sol alt ağacı vardır.
 3. Silinecek düğümün yalnızca bir sağ alt ağacı vardır.
 4. Silinecek düğümün iki alt ağacı vardır.
- BST'nin gezilmesini kolaylaştırmak için, ipli BST'ler oluşturulmuştur.
- İpli bir BST'de, null işaretçileri belirli bir gezinti sırasındaki başarıcı düğüm işaretçileri ile değiştirilir.

7.8 Uygulama Setleri

Alıştırmalar

1. 5, 9 ve 12 veri elemanları için tüm olası ikili arama ağaçlarını çizin.
2. Aşağıdaki verileri sıralı bir set olarak girerek bir ikili arama ağacı oluşturun:

14 23 7 10 33 56 80 66 70

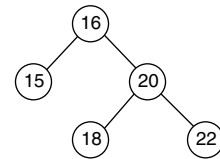
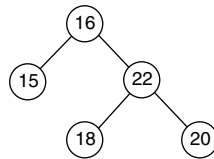
3. Aşağıdaki verileri sıralı bir set olarak girerek bir ikili arama ağacı oluşturun:

7 10 14 23 33 56 66 70 80

4. Aşağıdaki verileri sıralı bir set olarak girerek bir ikili arama ağacı oluşturun:

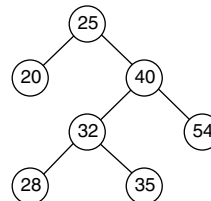
80 70 66 56 33 23 14 10 7

5. Alıştırma 2'de oluşturulan ağaca 44 ve 50'yi ekleyin.
6. Alıştırma 3'te oluşturulan ağaca 44 ve 50'yi ekleyin.
7. Alıştırma 4'te oluşturulan ağaca 44 ve 50'yi ekleyin.
8. Şekil 7-17'deki ağaçlardan hangisi geçerli bir ikili arama ağacıdır ve hangisi değildir? Cevabınızı açıklayın.



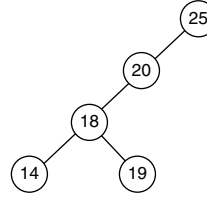
ŞEKİL 7-17 Alıştırma 8 için Şekil

9. Şekil 7-18'deki ikili arama ağacını sıralı bir geçiş kullanarak gezin.



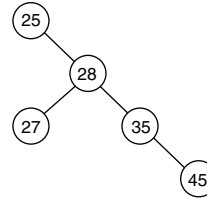
ŞEKİL 7-18 Alıştırma 9 için Şekil

10. Şekil 7-19'deki ikili arama ağacı, null bir ağaçla başlayarak ve klavye eden veri girerek oluşturulmuştur. Veriler hangi sırayla girilmiştir? Birden fazla olası sıra varsa, alternatifleri belirtin.



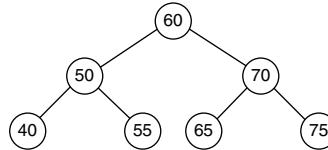
ŞEKİL 7-19 Alıştırma 10 için Şekil

11. Şekil 7-20'deki ikili arama ağacı, null bir ağaçla başlayarak ve klavye eden veri girerek oluşturulmuştur. Veriler hangi sırayla girilmiştir? Birden fazla olası sıra varsa, alternatifleri belirtin.



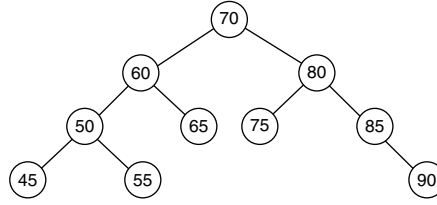
ŞEKİL 7-20 Alıştırma 11 için Şekil

12. Şekil 7-21'deki ikili arama ağacına 44, 66 ve 77'yi ekleyin.



ŞEKİL 7-21 Alıştırma 12 için Şekil

13. Şekil 7-22'deki ikili arama ağacından 60'ı içeren düğümü silin.



ŞEKİL 7-22 Alıştırmalar 13 ve 14 için Şekil

14. Şekil 7-22'deki ikili arama ağacından 85'i içeren düğümü silin.

Problemler

15. Algoritma 7-1, "BST'de En Küçük Düğümü Bul" için bir özyinelemeli olmayan algoritma geliştirin.
16. Algoritma 7-3, "BST'yi Ara" için bir özyinelemeli olmayan algoritma geliştirin.
17. Algoritma 7-1, "BST'de En Küçük Düğümü Bul" için C kodunu yazın.
18. Algoritma 7-4, "BST'ye Düğüm Ekle" için C kodunu yazın.
19. Algoritma 7-5, "BST'den Düğüm Sil" için C kodunu yazın.

Projeler

20. BST algoritmalarını yazarken, algoritmaların verileri doğru bir şekilde işlediğini doğrulamak için ağacı hiyerarşik bir sırayla yazdırabilmeniz gerekir. Ağacı yazdırmak için çağrılacak bir yazdırma fonksiyonu yazın. Yazdırılan çıktı, düğüm seviye numarasını ve verisini içermelidir. Ağacı, Amerikan Köpek Kulübü (AKC) tarafından tanınan birkaç popüler köpek ırkı için Şekil 7-23'te gösterildiği gibi bir malzeme listesi formatında sunun.

1. Labrador
 2. Alman Çoban Köpeği
 3. Cocker Spaniel
 4. Beagle
 4. Daksund
 5. Dalmatıyan
 3. Golden Retriever
2. Rottweiler
 3. Kaniş
 3. Shetland Çoban Köpeği

ŞEKİL 7-23 AKC tarafından Tanınan En Popüler Köpek Irkları

21. Proje 20'deki yazdırma algoritmasını verileri hiyerarşik olarak yazdıracak şekilde yeniden yazın. Verilerin Dikey sunumu Şekil 7-23'te gösterilmiştir ve Şekil 7-24'te yer almaktadır. Sayfayı yan yatırırsanız, veriler ikili ağaç formatında sunulmaktadır.

```

          3. Shetland Çoban Köpeği
        2. Rottweiler
          3. Kaniş
    1. Labrador
        3. Golden Retriever
        2. Alman Çoban Köpeği
            5. Dalmatyan
            4. Daksund
            3. Cocker Spaniel
            4. Beagle
  
```

ŞEKİL 7-24 AKC Verileri Hiyerarşik Olarak Sunulmuştur

22. Bir metin dosyasından isimler ve telefon numaraları içeren bir liste okuyan ve bunları bir BST ağacına ekleyen bir program yazın. Ağaç oluşturulduktan sonra, kullanıcıya belirtilen bir ismi aramasına, yeni bir isim eklemesine, mevcut bir ismi silmesine veya tüm telefon listesini yazdırmasına olanak tanıyan bir menü sunun. İşin sonunda, listedeki verileri dosyaya geri yazın. Programınızı en az 10 isimle test edin.
23. Dizi uygulamasını kullanarak bir ikili arama ağacı için ADT'yi oluşturun. Dizi uygulamasında, işaretçiler alt ağaç elemanlarına indeks haline gelir. Ağaç oluşturduğunuzda, ağaçta saklanacak maksimum düğüm sayısını bilmeniz gerekir. ADT'yi test etmek için, Proje 22'deki programı çalıştırmak için kullanın.
24. Bir iş parçacıklı ikili ağaç işleyen bir program yazın. Program öncelikle ağacı oluşturmalı, ardından iş parçacıklarını kullanarak iteratif bir gezinme ile işlenmelidir.
25. Proje 24'ü öncelikli bir gezinme kullanarak yeniden çalıştırın.
26. Proje 24'ü sonradan bir gezinme kullanarak yeniden çalıştırın.

Bölüm 8

AVL Arama Ağaçları

İkili arama ağacı basit ve anlaşılması kolay olsa da, birana problemi vardır : dengeli değildir. Bu bölümde dengeli olan AVL ağacını tartışıyoruz. AVL ağacı yapısını ve algoritmalarını geliştiriyoruz ve kullanımına dair örnekler veriyoruz. Ardından, bölümün sonunda, AVL ağacını soyut bir veri türü olarak tartışıyoruz ve onu kullanan bir program yazıyoruz.

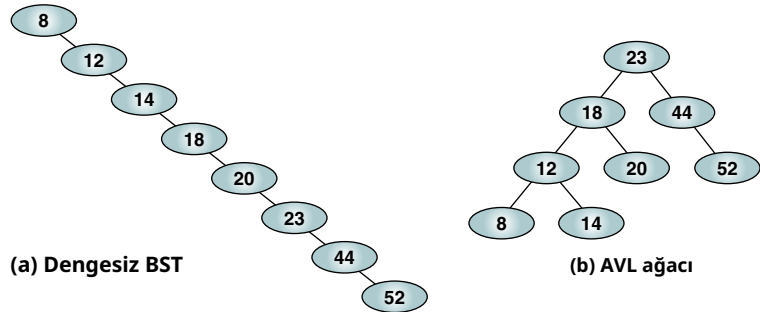
8.1 AVL Ağaç Temel Kavramları

1962 yılında, iki Rus matematikçi, G. M. Adelson-Velskii ve E. M. Landis, adlarını taşıyan dengeli ikili ağaç yapısını oluşturdu—AVL ağacı. AVL ağacı, alt ağaçların yüksekliklerinin en fazla 1 farkla farklı olduğu bir arama ağacıdır. Bu nedenle dengeli bir ikili ağaçtır. Ağaçların dengeli olmasının önemini anlamak için, aynı verileri içeren iki farklı ağaca bakalım. Birincisi, her düğümün önceki düğümünden daha büyük olduğu bir arama ağacıdır. İkincisi ise bir AVL ağacıdır. Bu ağaçlar Şekil 8-1'de gösterilmektedir.

Ağaç 8-1(a) şeklini incelerken, bunun bir ikili ağacın kıyafetleri içinde bir doğru listeye eşdeğer olduğunu hızlıca fark etmelisiniz. 12'yi bulmak için iki test gereklidir. 14'ü bulmak için üç test gereklidir. 52'yi bulmak için sekiz test gereklidir. Diğer bir deyişle, bu belirli ikili arama ağacı için arama çabası $O(n)$ şeklindedir.

Şimdi 8-1(b) şeklinin ağacını düşünün. Bunu bir AVL ağacı olarak etiketlemiş olsak da, her iki ağaç da aynı yapıya dayandığı için bir ikili arama ağacına benziyor. Düğüm 8 ve 14'ü bulmak dört test gerektirir. Düğüm 20 ve 52'yi bulmak üç test gerektirir. Diğer bir deyişle, bu ağacın maksimum arama çabası ya üç ya da dördür. Arama çabası $O(\log n)$ şeklindedir. Bu iki küçük ağacın örneğinde, en kötü arama çabasının sekizden dörde düşürüldüğünü görüyoruz; bu sadece ağacı dengeleyerek sağlanmıştır. 1000 düğümlü bir ağaç için, tamamen dengesiz bir ağacın en kötü durumu 1000'dir, oysa neredeyse

tam bir ağacın en kötü durumu 10'dur. Böylece, bir ağacı dengelemenin önemli performans iyileştirmeleri ne yol açabileceğini görüyoruz.



ŞEKİL 8-1 İki İkili Ağaç

Artık bir AVL ağacını tanımlamaya hazırız. Bir AVL ağacı ya boştur ya da yükseklikleri en fazla 1 farkla değişen iki AVL alt ağacından, T_L ve T_R oluşur, aşağıda gösterildiği gibi.

$$|H_L - H_R| \leq 1$$

Bu formülde, H_L sol alt ağacın yüksekliği ve H_R sağ alt ağacın yüksekliği (çubuk sembolleri mutlak değeri gösterir). AVL ağaçları yükseklikleri ile dengelendiği için, aynı zamanda yükseklik-dengeli ağaçlar olarak da bilinir.

Bir AVL ağacı, yükseklik-dengeli bir ikili arama ağacıdır.

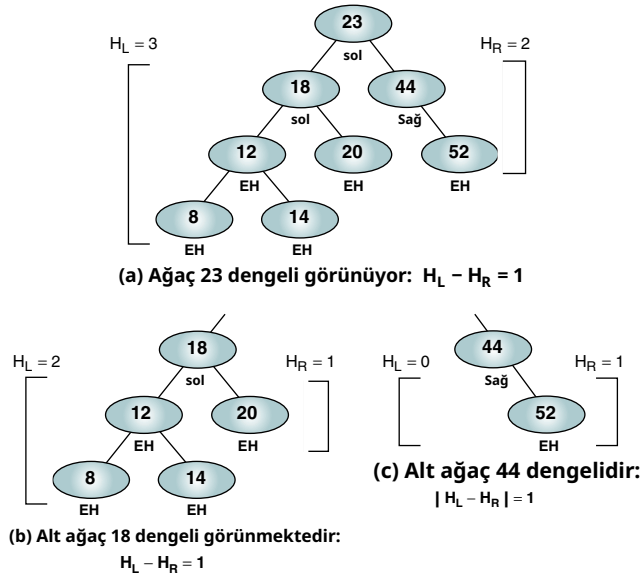
Şekil 8-1(b)'e geri dönersek, köküne baktığımızda ağacın dengeli göründüğünü görüyoruz. Sol alt ağacın yüksekliği (18) 3'tür; sağ alt ağacın yüksekliği (44) 2'dir. Bu nedenle, eğer alt ağaçlar 18 ve 44 dengeliyse, ağaç dengelidir, Şekil 8-2(a)'de gösterildiği gibi.

Şimdi Şekil 8-2(b)'deki sol alt ağaç 18'i inceleyelim. Alt ağaçlarının yükseklikleri yalnızca 1 farkla farklı olduğu için dengeli görünüyor. Eğer alt ağaçları 12 ve 20 dengeliyse, bu dengeli bir ağaçtır. Şekil 8-2(c)'ye baktığımızda, sağ alt ağacın (44) dengeli olduğunu görüyoruz çünkü alt ağaçlarının yükseklikleri yalnızca 1 farkla farklı. Kalan alt ağaçların da dengeli olduğunu kendinize kanıtlayın, bu da ağacı dengeli yapar.

AVL Ağaç Denge Faktörü

6. Bölümde denge faktörünü sol alt ağacın yüksekliğinden sağ alt ağacın yüksekliğini çıkartarak tanımladık. Herhangi bir ağaç için denge faktörü

AVL ağacındaki bir düğüm +1, 0 veya -1 olmalıdır, tanımlayıcıları kullanıyoruz LH sol yüksek (+1) olarak, sol alt ağacın sağ alt ağaçtan daha yüksek olduğunu belirtmek için, EH eşit yüksek (0) olarak, alt ağaçların aynı yükseklikte olduğunu belirtmek içinve RH sağ yüksek (-1) olarak, sol alt ağacın sağ alt ağaçtan daha kısa olduğunu belirtmek için kullanıyoruz. Bu sistemin yapıya daha kolay odaklanmamıza olanak sağladığını görüyoruz; ve çoğu programlama dili, sabit değerler için tanımlayıcılar uygulamak için yöntemler sağladığından, algoritmaları uygularken sorun çıkarmamalıdır. Bu denge faktörleri Şekil 8-2'de gösterilmiştir.



ŞEKİL 8-2AVL Ağacı

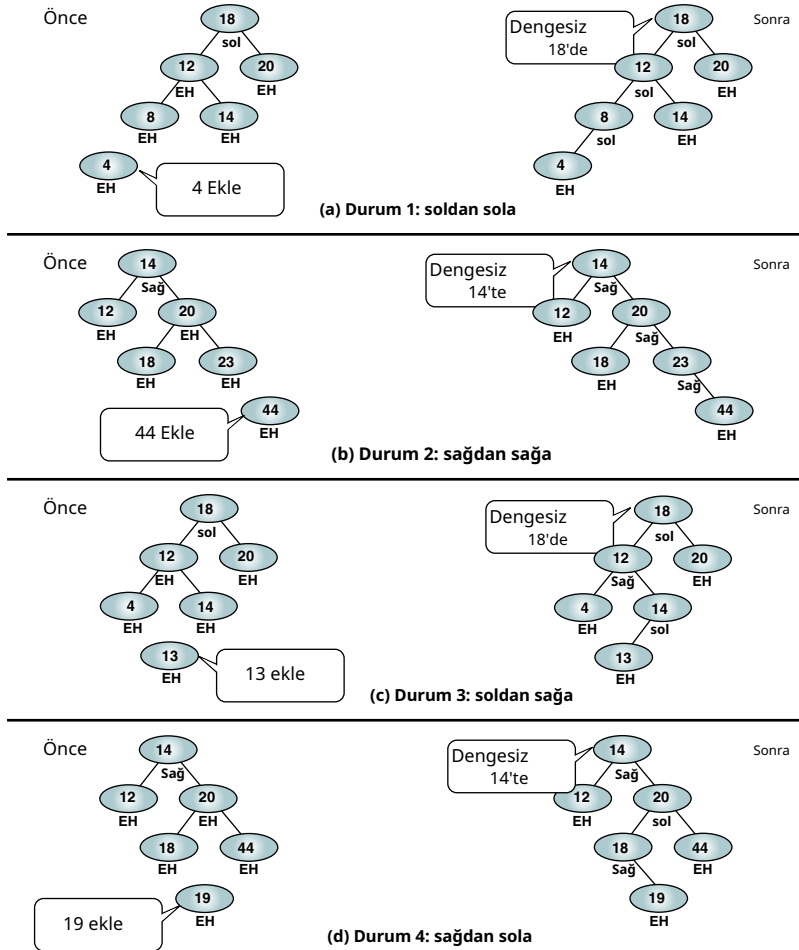
Ağaçları Dengeleme

Bir ağaca düğüm eklediğimizde veya bir düğümü ağaçtan sildiğimizde, ortaya çıka nağaç dengesiz olabilir. Bir ağacın dengesiz hale geldiğini tespit ettiğimizde, ağacı yeniden dengelememiz gerekir. AVL ağaçları, düğümleri ya solaya da sağa döndürerek dengelenir. Bu bölümde temel dengeleme algoritmalarını tartışıyoruz.

Dengelenmeyi gerektiren dört durumu ele alıyoruz. Tüm dengesiz ağaçlar bu dört durumdan birine girer:

1. Solun solunda—Sol yüksek olan bir ağacın alt ağacı da sol yüksek hale gelmiştir.
2. Sağ sağın hakkı—Sağ yüksek olan bir ağacın alt ağacı da sağ yüksek olmuştur.
3. Sol sağın hakkı—Sol yüksek olan bir ağacın alt ağacı sağ yüksek olmuştur.
4. Sağ solun hakkı—Sağ yüksek olan bir ağacın alt ağacı sol yüksek olmuştur.

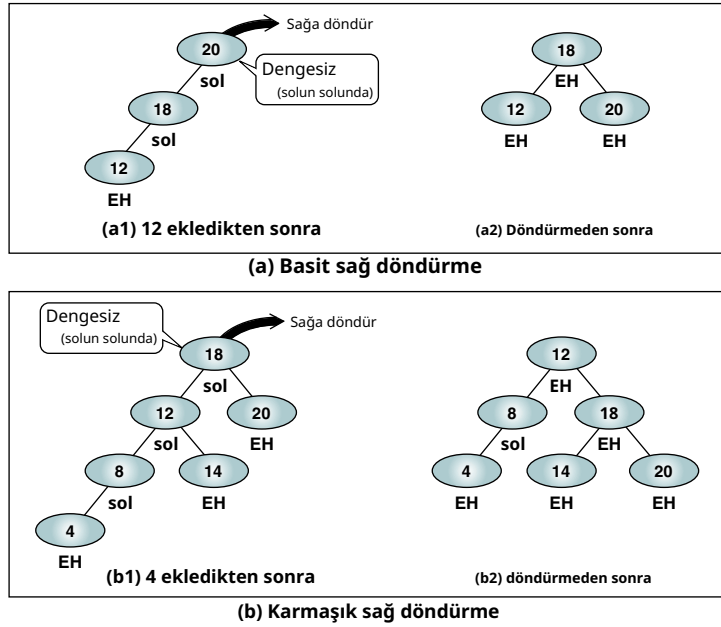
Bu dört durum, eklemelerle oluşturulmuştur ve Şekil 8-3'te gösterilmiştir. Benzer durumlar silmeler için de oluşturulabilir.



ŞEKİL 8-3 Dengesiz AVL Ağaçları

Durum 1: Solun Solu

Dengesiz durum, sol yüksek bir alt ağaç tarafından oluşturulduğunda ağaç dengesini sağlamak için dengesiz düğümü sağa döndürmemiz gerekir. Basit bir durumla başlayalım. Şekil 8-4(a)'da düğüm 20 (ağaç) dengesizdir çünkü sol alt ağaç 18 sol yüksektir ve bu, düğüm 20'nin sol dalındadır, bu da sol yüksektir. Bu durumda, ağacı dengelemek için kök olan 20'yi sağa döndürüyoruz, böylece 18'in sağ alt ağacı haline geliyor.



ŞEKİL 8-4Sola Sola—Tek Dönüş Sağ

Şekil 8-4(b)'deki ağaç daha karmaşık bir problemi sunmaktadır. Alt ağaç 12 dengelidir, ancak tüm ağaç (18) değildir. Bu nedenle, 18'i sağa döndürmemiz gerekiyor, böylece yeni kök olan 12'nin sağ alt ağacı haline geliyor. Ancak bu bir sorun yaratıyor. 12'nin mevcut sağ alt ağacı olan düğüm 14 ile ne yapabiliriz? Ağacı dikkatlice incerseniz, eski kök olan 18'in dönüşte solalt ağacını (12) k aybettiğini göreceksiniz (12 kök haline geliyor). Bu nedenle, 18'in boş sol alt ağacını 14'ü eklemek için kullanabiliriz, bu da kökün sağında bulunan tüm düğümlerin kökten büyük veya ona eşit olması gereken arama ağacı ilişkisini korur.

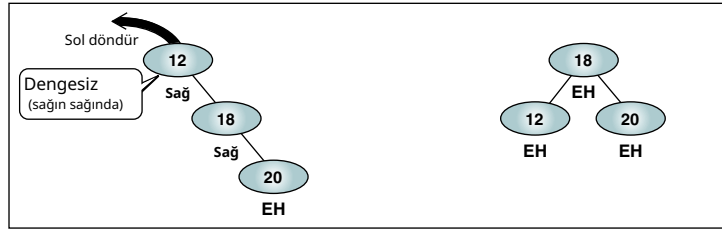
Şekil 8-4'teki her iki durumda da, dönüşten sonra dönen düğüm bileyüksekliktedir. Diğer denge faktörleri de, ağacın karmaşıklığına ve dengesiz durumun nasıl oluşturulduğuna bağlı olarak değişmiş olabilir. Ekleme ve silme algoritmalarını incelediğimizde denge faktörlerinin nasıl değiştiğini göreceğiz.

Durum 2: Sağın Hakkı

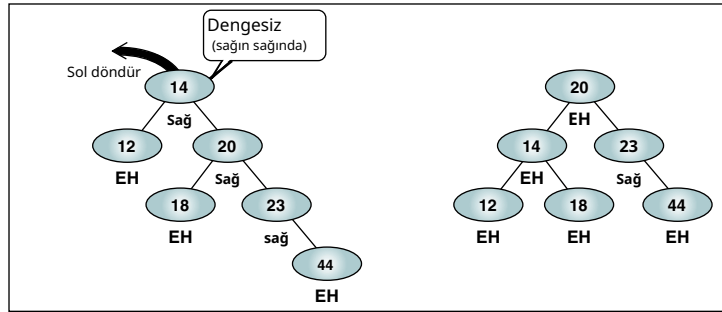
Durum 2, Durum 1'in aynasıdır. Şekil 8-5(a), basit bir sol döndürme içerir. Alt ağaç 18 dengelidir, ancak kök değildir. Bu nedenle kökü sola döndürürüz ve onu yeni kök olan 18'in sol alt ağacı yapıyoruz.

Şimdi Şekil 8-5(b)'de gösterilen daha karmaşık duruma bakalım. Bu durumda sağda yüksek bir kök ve sağda yüksek bir alt ağaç var ve dolayısıyla bir sağ durumu vardır.

sağ, dengesiz durum. Dengesizliği düzeltmek için, kökü sola döndürüyoruz, sağ alt ağaç olan 20'yi yeni kök yapıyoruz. Bu süreçte 20'nin sol alt ağacı, eski kökün sağ alt ağacı olarak bağlanıyor ve arama ağacının sırası korunuyor.



(a) Basit sol döndürme



(b) Karmaşık sol döndürme

ŞEKİL 8-5Sağın Sağı—Tek Dönüşüm Sola

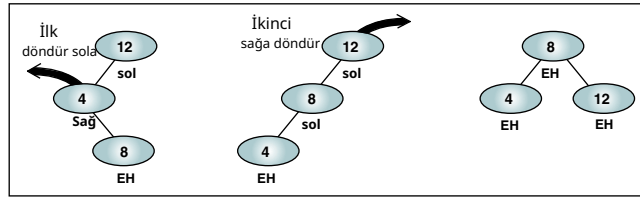
Durum 3: Sola Sağ

İlk iki durum, ağaçları dengelemek için tek dönüşümler gerektiriyordu. Şimdi, ağacı dengelemek için birini sola ve diğerini sağa döndürme gereken iki dengesiz durumu inceliyoruz.

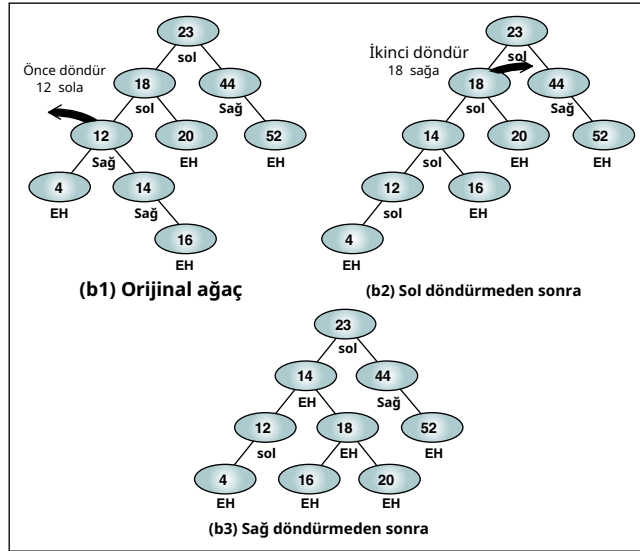
Yine, nispeten basit bir durumla başlayalım. Şekil 8-6(a)'da, kökün sol yüksek ve sol alt ağacın sağ yüksek olduğu dengesiz bir ağaç görüyoruz—solun sağ. Bu ağacı dengelemek için önce sol alt ağacı sola döndürüyoruz, ardından kökü sağa döndürüyoruz ve sol düğümü yeni kök yapıyoruz.

Şimdi karmaşık durumu inceleyelim. Şekil 8-6(b)'de, bir dengesiz ağaç görüyoruz; burada bir düğüm (18) yüksek kalmış ve sol alt ağacı (12) sağda yüksek. Bu ağacı dengelemek için, önce dengesiz olan düğümün (18) sol alt ağacını (12) sola döndürüyoruz, bu da alt ağaç düğümlerini arama ağacı sırasına hizalar. Bu döndürme Şekil 8-6(b2)'de gösterilmiştir. Şimdi sol alt ağacı (18) sağa döndürüyoruz, bu da yeni sol alt ağacın (14) dengelenmesiyle sonuçlanıyor.

Bu örnekte, ortaya çıkan ağaç hala solda yüksek ve yeni dengelenmiş alt ağaç, 14, daha da yüksek.



(a) Basit çift döndürme sağ



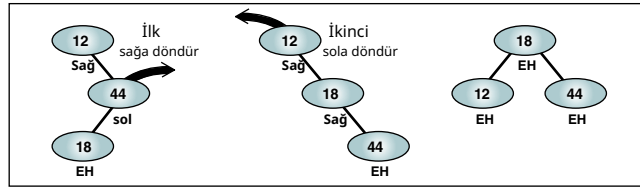
(b) Karmaşık çift döndürme sağ

ŞEKİL 8-6 Solun Sağ—Çift Dönüş Sağ

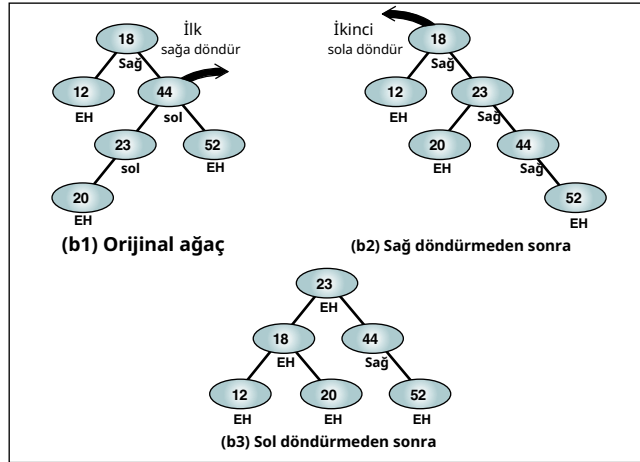
Durum 4: Sağın Solu

Durum 4 de karmaşıktır. Şekil 8-7(a) basit bir durumu göstermektedir. Ağacı dengelemek için önce sağ alt ağacı (44) sağa döndürüyoruz ve ardından kökü (12) sola döndürüyoruz.

Şekil 8-7(b) karmaşık durumu göstermektedir. Bu örnekte alt ağaç 44 dengelidir, ancak ağaç değildir. Dengesiz durum, sağ yüksek bir dalda sol yüksek bir alt ağaç tarafından oluşturulduğu için, çift dönüş yapmamız gerekiyor. Önce sağ alt ağacı (44) sağa döndürerek Şekil 8-7(b2)'deki ağacı oluşturuyoruz. Ardından kökü sola döndürüyoruz, bu da bize Şekil 8-7(b3)'teki dengeli ağacı veriyor. Dikkat edin ki, dengeli ağacın kökü eşit yükseklikte bir denge faktörüne sahiptir.



(a) Basit çift döndürme sağ



(b) Karmaşık çift döndürme sağ

ŞEKİL 8-7Sağın Solu—Çift Dönüş Sağ

8.2 AVL Ağaç Uygulamaları

Bu bölümde AVL ağacı ekleme ve silme algoritmalarını tartışıyoruz, ağacın dengesini korumak için algoritmalar da dahil.

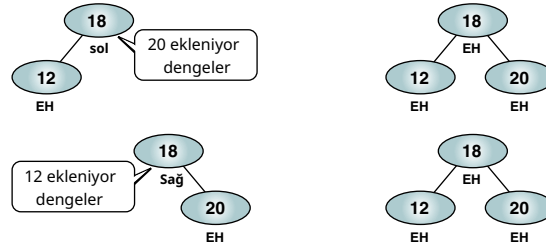
AVL Ağacına Ekle

Arama ve geri alma algoritmaları, herhangi bir ikili ağaç için olduğu gibidir. Doğal olarak, AVL ağaçları arama ağaçları olduğundan, sıralı bir gezinti kullanıyoruz. Ancak, ekleme ve silme algoritmaları yeniden yazılmalıdır.

İkili arama ağacında olduğu gibi, tüm eklemeler bir yaprak düğümde gerçekleşir. Uygun yaprak düğümü bulmak için, kökten başlayarak, yeni verinin anahtarının bir düğümün anahtarından daha az olduğu durumlarda sola, aksi takdirde sağa giderek yolu takip ederiz. Bir yaprak bulduğumuzda, onu ebeveyn düğümüne bağlarız ve ağaçtan geri çıkmaya başlarız. Burada AVL ağacı, ikili arama ağacından farklılık gösterir. *Ağaçtan geri çıkarken, her düğümün dengesini sürekli kontrol ederiz.* Bir düğümün dengesiz olduğunu bulduğumuzda, onu dengeleriz ve sonra ağacın yukarısına devam ederiz.

Tüm eklemeler dengesiz bir durum yaratmaz. Sol yüksek bir düğümün sağ dalına bir düğüm eklediğimizde, otomatik dengeleme gerçekleşir ve düğüm

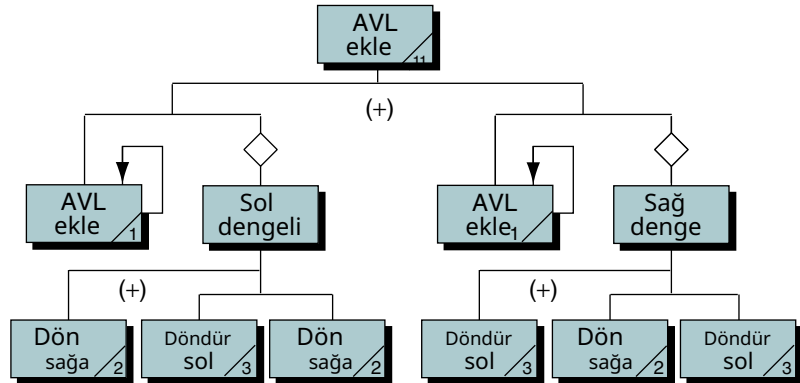
artık eşit yükseklikte, Şekil 8-8'de gösterildiği gibi. Tersine, sağ yüksek bir düğümün sol dalına bir düğüm eklediğimizde, düğüm otomatik olarak dengelenir.



ŞEKİL 8-8 Otomatik AVL Ağaç Dengeleme

Ancak, eklemekten önce yaprak eşit şekilde dengeliyse, yaprağın bulunduğu alt ağaç bir seviye büyür. Bu eklemenin dengesiz bir durum yaratıp yaratmadığı, atalarının düğümlerinin dengesine bağlıdır. Eklemekten geri çıkarken, ağaç büyümüşse, dengeyi sağlamak için döndürmemiz gerekir.

AVL ağaç ekleme algoritmasının tasarımı Şekil 8-9'da gösterilmiştir.¹ Sağ dalda ekleme işleminin, solda ekleme işleminin bir yansıması olduğu açıkça görülmektedir. Yapı şemasında solda, sol dalda ekleme için tasarım bulunmaktadır. Sağda ise sağ dalda ekleme için tasarım bulunmaktadır. Her ikisi de bir yaprağı bulmak için özyinelemeli bir çağrı ile başlar. Yaprığı bulduğumuzda ve özyinelemeden geri çıkmaya başladığımızda, ekleme daha yüksek bir ağaç oluşturduysa ya sol denge ya da sağ denge çağrısı yaparız.



ŞEKİL 8-9 AVL Ağaç Ekleme Tasarımı

1. Yapı şeması notasyonları için Ek B'ye bakın.

Sol alt ağaç düğümünü dengelemek için, ya sağa tek başına ya da iki kez, önce sola ve sonra sağa döndürürüz. Sağ alt ağaç düğümünü dengelemek için, ya sola tek başına ya da iki kez, önce sağa ve sonra sola döndürürüz. Algoritmalara geçmeden önce, bu tasarımı Bölüm 8.1, “AVL Ağaç Temel Kavramları” (Şekil 8-3’ten 8-7’ye) ile karşılaştırın.

AVL Ağaç Ekleme Algoritması

AVL ağaç ekleme psödo kodu Algoritma 8-1’de gösterilmiştir. Bu algoritma, çağrılmadan önce yeni düğüm için alanın ayrılmasını gerektirir. Bu nedenle, çağırılan algoritma belleği yönetir.

ALGORİTMA 8-1 AVL Ağaç Ekleme

```

Algoritma AVLInsert (kök, yeniVeri)
Özyineleme kullanarak, bir düğümü bir AVL ağacına ekleyin.
  Ön   kök, AVL ağacındaki/alt ağaçtaki ilk düğüme işaretçidir
        yeniVeri, eklenecek yeni düğüme işaretçidir
  Sonra yeni düğüm eklenmiştir
  Kök, ağaç boyunca özyinelemeli olarak döndürülür
1  eğer (alt ağaç boşsa)
    Kökten ekle
    1  yeniVeri'yi kökten ekle
    2  kökü döndür
2  sonlandır if
3  eğer (yeniVeri < kök)
    1  AVLInsert (sol alt ağaç, yeniVeri)
    2  eğer (sol alt ağaç daha uzun)
        1  solDenge (kök)
    3  sonlandır if
4  aksi takdirde
    Yeni veri >= kök verisi
    1  AVLInsert (sağ alt ağaç, newPtr)
    2  eğer (sağ alt ağaç daha yüksekse)
        1  sağDenge (kök)
    3  sonlandır if
5  sonlandır if
6  kökü döndür
son AVLInsert

```

Algoritma 8-1 AnaliziDaha önce tartıştiğimiz gibi, algoritma, ikili arama ağacı düğüm ekleme (Algoritma 7-4) ile çok benzer bir şekilde b aşlar. Eğer bir yaprakta isek, kökü yeni düğüme işaret edecek şekilde ayarlarız.

Özyinelemeli çağrıdan döndüğümüzde, iki olasılık vardır: ya ağaç daha uzun hale gelmiştir ya da gelmemiştir. Eğer daha uzun değilse, dengelemeye gerek yoktur. Eğer ağaç daha uzunsa, yine i ki olasılık vardır: ağaç sol yüksekse ve sol tarafa ekleme yaptıysak dengelememiz gerekir (ifade 3.2) veya ağaç sağ yüksekse ve sağ tarafa ekleme yaptıysak dengelememiz gerekir (ifade 4.2).

Son bir not: Bu algoritma, yinelenen eklemeleri kontrol etmez. Eğer bir uygulamayın yinelenenle re izin vermiyorsa, algoritma ekleme yapmadan önce arama yapmalı ve eklemenin yinelenen bir ek leme olmadığından emin olmalıdır.

AVL Ağaç Sol Denge Algoritması

Bir sol alt ağacı dengeleme mantığı ile bir sağ alt ağacı dengeleme mantığı birbirini yansıması olduğundan, yalnızca bir sol alt ağacı dengelemek için mantığı gösterebiliriz. Eğer bunu anlıyorsanız, sağ alt ağacı dengelemek için algoritmayı oluşturabilirsiniz. Bir sol yüksek düğümü dengelemek için kullanılan sahte kod, Algoritma 8-2'de gösterilmiştir.

ALGORİTMA 8-2AVL Ağaç Sol Denge

```

Algoritma solDenge (kök)
Bu algoritma, kök sol yüksek olduğunda (sol alt ağaç sağ alt ağaçtan daha
yüksek olduğunda) girilir.
  Ön kök, [alt] ağacın köküne işaret eden bir işaretçidir
  Son kök güncellenmiştir (gerekirse)
1 eğer (sol alt ağaç yüksekse)
  1 döndürSağ (kök)
2 aksi
  1 sol alt ağaç döndür
  2 döndürSağ (kök)
3 sonlandır if
son solDenge

```

Dönme Algoritmaları

Bir düğümü döndürmek için, kök ve uygun alt ağaç işaretçilerini basitçe değiştiriyoruz. Bu işlem dört adım alır: değişim için üç ve kök işaretçisini sıfırlamak için bir. Sol yüksek bir ağacı döndürme adımları Şekil 8-10'da gösterilmiştir.

Bu şekilde, sadece bir ağacın bir kısmıyla ilgilendiğimizi unutmayın. Bu nedenle, son denge faktörlerini belirleyemeyiz.

Sağ döndürme ve sol döndürme için algoritma 8-3'te gösterilen sahte kod.

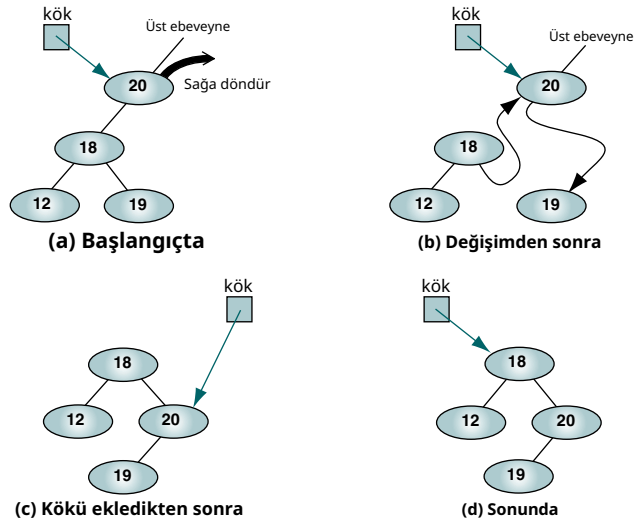
ALGORİTMA 8-3AVL Ağacını Sağ ve Sol Döndür

```

Algoritma sağaDöndür (kök)
Bu algoritma, ağacı sağa döndürmek için işaretçileri değiştirir.
  Ön kök döndürülmesi gereken ağacı gösterir
  Son düğüm döndürüldü ve kök güncellendi
1 sol alt ağacı, sol alt ağacın sağ alt ağacı ile değiştir
2 sol alt ağacı yeni kök yap
son sağaDöndür

Algoritma solaDöndür (kök)
Bu algoritma, ağacı sola döndürmek için işaretçileri değiştirir.
  Ön kök döndürülmesi gereken ağacı gösterir
  Son düğüm döndürüldü ve kök güncellendi
1 sağ alt ağacın sol alt ağacı ile değiştirilmesi
2 sağ alt ağacı yeni kök yapmak
bitiş sola döndür

```



ŞEKİL 8-10 AVL Ağacı Sağ Döndür

AVL Ağacı Silme Algoritması

İkili arama ağacında gördüğümüz gibi, tüm silme işlemleri bir yaprak olduğunda gerçekleşmelidir. AVL ağacı silme (Algoritma 8-4), ikili arama ağacı silme temel mantığını takip eder ve ağacı dengelemek için ek mantık içerir. Ekleme mantığında olduğu gibi, dengeleme ağacın dışına çıktıkça gerçekleşir.

ALGORİTMA 8-4 AVL Ağacı Silme

```

Algoritma AVLDelete (kök, dltAnahtar, başarı)
Bu algoritma bir düğümü AVL ağacından siler ve
gereirse dengelemeyi yapar.
    Ön   kök bir [alt] ağaca işaretçidir
         dltAnahtar silinecek düğümün anahtarıdır
         başarı boolean değişkenine referanstır
    Son   düğüm bulunduysa silinir, bulunmadıysa ağaç değişmez
         başarı true (anahtar bulundu ve silindi) olarak ayarlanır
         veya false (anahtar bulunamadı)
    Yeni [potansiyel] alt ağacın köküne işaretçi döndür
1  eğer Dönüş (boş alt ağaç)
    Bulunamadı
    1  başarıyı yanlış olarak ayarla
    2  null döndür
2  sonlandır if
3  eğer (dltKey < root)
    1  sol-alt-ağacı AVLDelete(sol alt ağaç, dltKey, başarı) olarak ayarla

```

devam ediyor

ALGORİTMA 8-4AVL Ağaç Silme (devam ediyor)

```

2  eğer (ağaç daha kısa)
    1  kökü deleteRightBalance(root) olarak ayarla
3  sonlandır if
4  aksi takdirde (dltKey > kök)
    1  sağ alt ağacı AVLDelete(root->right, dltKey, success) olarak ayarla

    2  eğer (ağaç daha kısa)
        1  kökü deleteLeftBalance (root) olarak ayarla
    3  sonlandır if
5  aksi takdirde
    Silinen düğüm bulundu--yaprak düğüm için test et
    1  kökü kaydet
    2  eğer (sağ alt ağaç yoksa)
        1  başarıyı true olarak ayarla
        2  sol alt ağacı döndür
    3  aksi takdirde (sol alt ağacı yoksa)
        Sağ var ama sol alt ağacı yok
        1  başarıyı true olarak ayarla
        2  sağ alt ağacı döndür
    4  aksi takdirde
        Silinen düğümün iki alt ağacı var
        Yerine geçecek--sol alt ağacın en büyük düğümü
        1  sol alt ağacın en büyük düğümünü bul
        2  en büyük anahtarı kaydet
        3  en büyük veriyi köke kopyala
        4  sol alt ağacı AVLDelete(sol alt ağaç, en büyük anahtar, ba
            şarı) olarak ayarla

    5  eğer (ağaç daha kısa)
        1  kökü dltRightBal (kök) olarak ayarla
    6  sonlandır if
5  sonlandır if
6  sonlandır if
7  kökü döndür
son  AVLDelete

```

Algoritma 8-4 Analizi Temel mantık, ikili arama ağacı silme algoritmasının mantığına paralel olsa da, önemli farklılıklar vardır. Başlangıçta çok benzer bir şekilde başlar; eğer bir null ağaç bulursak, silmek istediğimiz düğüm mevcut değildir. Bu durumda başarıyı false olarak ayarlarız. Ardından null kök döneriz. Null kök döndürmek gereklidir çünkü ağacın veya alt ağacın kökü döndürme sırasında değişmiş olabilir. Bu nedenle, orijinal çağıran algoritma yeni kökü kaydetmeli ve ardından silme anahtarının bulunmadığı durumlarda ağacın kökünü otomatik olarak değiştirmeden önce başarıyı test etmelidir. Bu mantık aşağıda gösterilmiştir.

```

1  yeniTree'yi AVLDelete (kök, dltAnahtar, başarı) olarak ayarla
2  eğer (başarı doğru ise)
    1  kökü yeniTree olarak ayarla
    1  aksi takdirde
    2  hata (silme anahtarı "bulunamadı")
2  sonlandır if

```

Silinecek düğümü bulduğumuzda (ifade 5), silinecek düğümün bir yaprak olup olmadığını belirlememiz gerekiyor. Eğer sağ alt ağaç yoksa, başarıyı doğru olarak ayarlıyoruz ve sol alt ağacı döndürürüz (ki, ikili arama ağacı silme algoritmasında gördüğümüz gibi, bu da null olabilir). Eğer bir sağ alt ağaç varsa ama sol alt ağaç yoksa, başarıyı doğru olarak ayarlıyoruz ve sağ alt ağacı döndürüyoruz.

Eğer bir sol alt ağaç ve bir sağ alt ağaç varsa, silinecek düğümün yerini alacak bir düğüm bulmamız gerekiyor. Bulunduktan sonra, verilerini kök düğüme kopyalıyoruz. Ardından, kök düğümün sol alt ağacında silme işlemini sürdürüyoruz, bu sefer en büyük anahtarın düğümünü silmek için bakıyoruz, bu artık gereksiz. Eğer yaprağından kopyalanan veriyi sildikten sonra ağaç daha kısa olursa, dengeyi sağlamak için sağdan silme işlemini çağırıyoruz.

Sağdan Silme Dengesi

Yine, dengeleme algoritmasının sadece bir tarafını gösteriyoruz, sağdan silme dengesi. Denge faktörlerini sıfırlama mantığını da hariç tutuyoruz. Bu mantığı ADT uygulamasında geliştiriyoruz. Soldan silme dengesinin mantığı, Algoritma 8-5'teki mantığı yansıtır.

ALGORİTMA 8-5AVL Ağaç Silme Sağ Denge

```

Algoritma deleteRightBalance (kök)
Sol dalda bir silme işleminden sonra [alt]ağaç daha kısadır.
Gerekirse, döndürerek ağacı dengeleyin.
  Ön ağaç daha kısadır
  Son denge geri sağlandı
  Yeni kökü döndür
1 eğer (ağaç dengeli değilse)
  Ağaç sol veya eşit yüksekse döndürme gerekmiyor
  1 sağın sağını sağ alt ağaç olarak ayarla
  2 eğer (rightOfRight sol yüksek)
    Çift döndürme gereklidir
    1 leftOfRight'ı rightOfRight'ın sol alt ağacına ayarla
    Önce sağa sonra sola döndür
    2 sağ alt ağaç = rotateRight (rightOfRight)
    3 kök = rotateLeft (kök)
  3 aksi takdirde
    Tek döndürme gerekli
    1 kökü rotateLeft (kök) olarak ayarla
  4 sonlandır if
2 sonlandır if
3 kökü döndür
son deleteRightBalance

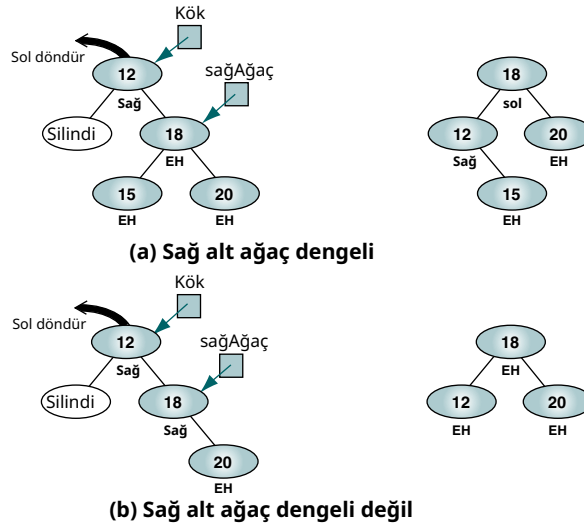
```

Algoritma 8-5 Analizi Ağacı dengelememiz gerekip gerekmediğini belirleyerek başlıyoruz. Unutmayın, bir düğümü sildiğimiz için ağacı yeniden dengelememiz gerekmez. Örneğin, kök sol tarafta yüksekse, şimdi daha da yüksek ve denge faktörünü ayarladıktan sonra çıkabiliriz.

Eğer solda silme işlemi yaptıysak ve ağaç zaten sağda yüksekse, şimdi iki kat sağda yüksektir (silme işlemi sağın sağındadır). Bu nedenle, sağ alt ağacı sola döndürmemiz gerekiyor.

Eğer sağ alt ağaç sol tarafta yüksekse, önce sağa ve sonra sola iki kez döndürmemiz gerekiyor. Bu durum Şekil 8-7'de gösterilmiştir. Eğer sağ alt ağaç sol tarafta yüksek değilse, bunu yapmamız gerekiyor.

yalnızca bir kez döndürün. Bu durum Şekil 8-5'te gösterilmiştir. Soldan sildiğimiz için, kökü sola döndürmemiz gerekiyor. Bu döndürmeler Şekil 8-11'de gösterilmiştir.



ŞEKİL 8-11 AVL Ağaç Silme Dengeleme

AVL Ağaçlarında Tekrar Eden Anahtarlar

AVL ağaçlarında yinelenen anahtarların izin verilmesi durumunda, eşit bir anahtarın sol alt ağaçta döndürülmesi mümkündür. Aşağıdaki örneği dikkate alın.



Bu örnek, ekleme işlemlerinin her zaman sağa yinelenenleri eklediğini, silme işlemlerinin ise onları sola taşıyabileceğini göstermektedir.

Problem, hangi düğümün silineceğini nasıl belirleyeceğimizdir. Rekürsif sıralı arama her zaman sağ dalda ilk düğümü bulur; sol alt ağaçta bir düğüm bulamaz. Çözüm, istenen anahtara sahip düğümün hemen önceki düğümü bulan yin elemeli bir arama yazmaktır.

Denge Faktörlerini Ayarlama

Bir ekleme veya silme işleminden sonra, ağacı dengelediğimizde denge faktörlerini ayarlamamız gerekir. Denge faktörlerindeki ayarlamalar her durum için ayrı ayrı analiz edilmesi gerekse de, genel bir desen vardır:

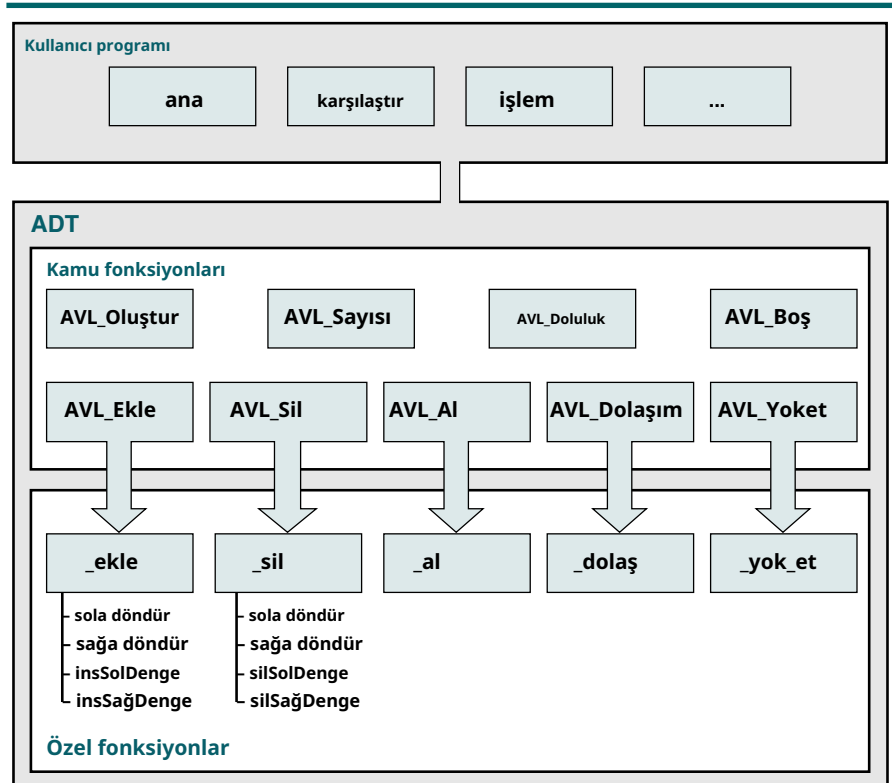
1. Eğer kök bir ekleme öncesinde dengeli ise, şimdi eklemenin yapıldığı tarafta yüksektir.

2. Eğer bir ekleme, dengeli olmayan bir ağacın daha kısa alt ağacında yapıldıysa, kök artık dengelidir.
3. Eğer bir ekleme, dengeli olmayan bir ağacın daha yüksek alt ağacında yapıldıysa, kök döndürülmelidir.

Dengelenme faktörlerini ayarlamak için kodu, AVL ağaç soyut veri tipi uygulamasında tamamen geliştiriyoruz.

8.3 AVL Ağaç Soyut Veri Tipi

Temel tasarım anlaşıldığında, ADT'yi geliştirmeye hazırız. Kullanıcı ara yüzünü sağlayan dokuz işlev tanımlıyoruz. BST'de olduğu gibi, ağacı u uygulamak için gereken birkaç iç işlev vardır. ADT tasarımı Şekil 8-12'de gösterilmiştir.



ŞEKİL 8-12 AVL Ağaç Tasarımı

AVL Ağaç Veri Yapısı

Diğer ADT yapılarında olduğu gibi, AVL ağaç ADT'sini bir baş yapı ve bir düğüm yapısı ile uyguluyoruz.

Baş Yapısı

AVL ağaç baş yapısı, AVL_TREE, bir sayım, bir kök işaretçisi ve listeyi aramak için gereken karşılaştırma fonksiyonunun adresini içerir. Uygulama programının ağaca tek görünümü, ağacın oluşturulduğunda dinamik bellekten ayrılan baş yapısına işaret eden bir işaretçidir.

Veri yapıları Şekil 8-13'te gösterilmiştir.

AVL-AĞACI



DÜĞÜM



typedef yapısı

```
{
    int sayım;
    NODE* kök;
    int (*karşılaştı) (void* argu1, void* argu2);
} AVL_AĞACI;
```

typedef struct düğüm

```
{
    struct düğüm* sol;
    void* veriPtr;
    int denge;
    struct düğüm* sağ;
} NODE;
```

ŞEKİL 8-13 AVL Ağaç Veri Yapısı

Düğüm Yapısı

AVL ağaç düğüm yapısı, ikili arama ağacının aynı tasarımını izler, bir denge faktörünün eklenmesiyle birlikte. Her düğüm, saklanacak veriye ait bir anahtar ve nitelikler, bir sol ve bir sağ alt ağaç işaretçisi ve bir denge faktörü içeren bir void işaretçisi içermelidir.

AVL Ağaç Algoritmaları

Bu bölümde temel bir AVL ağaç algoritmaları seti ele alınmaktadır. Uygu lamaya bağlı olarak, başka algoritmalar da gerekebilir. Örneğin, bazı uygulamalar azalan anahtar gezintilerini içermeyi gerektirebilir. ADT veri yapıları ve Program 8-1'deki prototip bildirimleri ile başlıyoruz.

PROGRAM 8-1 AVL Ağaç Bildirimleri

```

123456789 // Sol Yüksek
// Yüksek Sağ
// Sağ Yüksek

// Yapı Bildirimleri
typedef struct node
{
    void* dataPtr;
    struct node* sol;
    int bal;
    struct node* sağ;
} NODE;

typedef yapısı
{
    int sayımı;
    int (*karşılaştır) (void* argüman1, void* argüman2);
    NODE* root;
} AVL_TREE;

// Prototip Bildirimleri
AVL_TREE* AVL_Create
(int (*karşılaştır)(void* argu1, void* argu2));
AVL_TREE* AVL_Destroy (AVL_TREE* ağacı);

bool AVL_Insert (AVL_TREE* ağacı,
                void* dataInPtr);
bool AVL_Sil (AVL_AĞACI* ağaç, void* silKey);
void* AVL_Getir (AVL_AĞACI* ağaç, void* keyPtr);
void AVL_Dolaş (AVL_AĞACI* ağaç,
                void (*process)(void* dataPtr));
int AVL_Count (AVL_TREE* ağacı);
bool AVL_Empty (AVL_TREE* ağacı);
bool AVL_Full (AVL_TREE* ağacı);

static NODE* _insert
(AVL_TREE* ağacı, NODE* kökü,
NODE* newPtr, bool* daha uzun);
statik NODE* _delete
(AVL_TREE* ağacı,
NODE* root, void* dltKey,
bool* daha kısa, bool* başarı);
static void* _retrive
(AVL_TREE* ağacı,
void* keyPtr, NODE* root);
statik geçersiz _traversal
(DÜĞÜM* kökü,
void (*process)(void* dataPtr));

```

devam ediyor

PROGRAM 8-1 AVL Ağaç Bildirimleri (devam ediyor)

4950515253545556	<pre>static void _destroy (NODE* root); static NODE* rotaryLeft (NODE* root); static NODE* rotaryRight (NODE* root); static NODE* insLeftBal (NODE* root, bool* uzun); static NODE* insRightBal (NODE* root, bool* uzun); static NODE* dltLeftBal (NODE* root, bool* daha kısa); static NODE* dltRightBal (NODE* kökü, bool* daha kısa);</pre>
------------------	---

Bir AVL Ağacı Oluştur

Soyut veri tipi, bir programda birden fazla yapıyı destekleyebilmelidir. Bu, ağaç başı yapısının dinamik bellekte tahsis edilmesiyle gerçekleştirilir. Ağaç oluşturma fonksiyonu yapıyı tahsis eder, sayısını sıfıra ayarlar ve kök işaretçisini null olarak ayarlar ve karşılaştırma fonksiyonunun adresini saklar. Ardından ağaç işaretçisini döndürür. Kod Program 8-2'de gösterilmiştir.

PROGRAM 8-2 AVL Ağaç Oluşturma Uygulama Arayüzü

123456789	<pre>/* ===== AVL_Create ===== AVL ağaç başı düğümü için dinamik bellek ayırır ve adresini çağırana döndürür. Önce compare, iki düğüm karşılaştırıldığında kullanılan k arşılaştırma fonksiyonunun adresidir. Sonra baş tahsis edildi veya hata döndürüldü. Dönüş baş d üğüm işaretçisi; taşma durumunda null */ AVL_TREE* AVL_Create (int (*karşılaştır) (void* argüman1, void* argüman2)) { // Yerel Tanımlar AVL_TREE* ağaç; // Beyanlar ağaç = (AVL_TREE*) malloc (sizeof (AVL_TREE)); if (ağaç) { ağaç->kök = NULL; ağaç->sayı = 0; ağaç->karşılaştır = karşılaştır; } // eğer return ağaç; } // AVL_Create</pre>
10111213141516171819202122232425	

Bir AVL Ağacı Ekle

AVL ağaç ekleme fonksiyonu, uygulama programı tarafından çağrılan modüldür. Ağaç yapısına bir işaretçi ve ağaca eklenmesi gereken verilere bir işaretçi alır. Bir düğüm ayırdıktan sonra, fiziksel ekleme yapmak için bir özyinelem eli ekleme fonksiyonu çağırır. Uygulamanın çağırdığı ekleme fonksiyonu Program 8-3'te gösterilmiştir.

PROGRAM 8-3 AVL Ağaç Uygulama Arayüzü Ekle

123456789	/* ===== AVL_Ekle ===== Bu fonksiyon yeni verileri ağaca ekler. Ön ağaç AVL ağaç yapısına işaretçidir Son veri eklendi veya bellek taşması Başarı (doğru) veya Taşma (yanlış) döndür
101112131415161718192021222324252627	bool AVL_Ekle (AVL_AĞAÇ* ağaç, void* veriPtr) { // Yerel Tanımlar NODE* yeniPtr; bool dahaYüksek; // İfadeler newPtr = (NODE*)malloc(sizeof(NODE)); if (!newPtr) false döndür; yeniPtr->dengesı = EH; yeniPtr->sağ = NULL; yeniPtr->sol = NULL; yeniPtr->veriPtr = veriPtr; ağaç->kök = _ekle(ağaç, ağaç->kök, yeniPtr, &dahaYüksek); (ağaç->sayısı)++; doğruyu döndür; } // AVL_Ekle

Program 8-3 Analizi Tüm eklemelerin bir yaprak düğümde gerçekleştiği için, denge faktörünü eşit yükseklikte ve alt ağaç işaretçilerini null olarak başlatıyoruz. Ardından özyinelemeli ekleme fonksiyonunu çağırıyoruz.

Geri döndüğünde, ağaç sayısını güncelliyoruz ve başarıyı geri döndürüyoruz.

Özyinelemeli ekleme modülünün adını not edin. Sistem yazılımı için kılavuzu takip ederek, adın uygulama programcıları tarafından tekrar edilmemesi için `_insert` olarak adlandırıyoruz.

İçsel Ekleme Fonksiyonu

Başlangıçta `AVL_Insert` tarafından çağrılan iç ekleme işlevi, dört parametre gerektirir: ağaç yapısına bir işaretçi, ağacın veya alt ağacın köküne bir işaretçi, eklenen düğüme bir işaretçi ve ağacın daha yüksek olup olmadığını belirtmek için bir Boolean. Daha yüksek bayrağı, geri dönerken döndürme işlemleri için kullanılır.

eklemeden sonra ağaçtan çıkıyor. Ağaç daha yüksekse, fonksiyon iki alt fonksiyondan birini çağırır: bir sol alt ağacı dengelemek için veya diğerini sağ alt ağacı dengelemek için. Denge faktörlerindeki değişiklikler de ağaçtan çıkarken işlenir. Fonksiyon Program 8-4'te gösterilmiştir. Tasarımı Algoritma 8-1'i takip eder.

PROGRAM 8-4 Dahili Ekleme Fonksiyonu

```

123456789 /* ===== _insert =====
Bu fonksiyon, yeni veriyi AVL ağacındaki bir yaprak düğümüne eklemek için özyinelemeyi kullanır.
Ön Uygulama, kök ve veri işaretçilerini geçiren AVL_Insert'i çağırır.
Veri eklendi.
[potansiyel olarak] yeni köke işaretçi döndür.
*/
1011213141516171819202122232425262728293031323334353637383940
NODE* _insert (AVL_TREE* ağaç, NODE* kök, NODE* yeniPtr, bool* dahaYüksek)
{
// Beyanlar
eğer (!root)
{
// Kökte ekle
kök = yeniPtr;
*dahaYüksek = true;
return kök;
} // NULL ağaç ise

if (ağaç->compare(yeniPtr->dataPtr, kök->dataPtr) < 0)
{
// yeniVeri < kök -- sola git kök->sol
= _insert(ağaç, kök->sol, yeniPtr, dahaYüksek);

eğer (*dahaYüksek)
// Sol alt ağaç daha yüksek
switch (kök->deng)
{
durum LH: // Sol yüksekti--döndür kök = insLeftBal (kök, dahaYüksek);
break;

durum EH: // Dengeliydi--şimdi LH
kök->deng = LH;
break;

durum RH: // Sağ yüksekti--şimdi EH
kök->deng = EH;
}
}
}

```

devam ediyor

PROGRAM 8-4 Dahili Ekleme Fonksiyonu (devam ediyor)

```

4142434445464748495051525354555657585960616263646566676869707172
                                     *dahaYüksek = false;
                                     break;
                                     } // anahtar r
                                     eturn kök; } //
yeni < düğüm aksi t
akdirde
// yeni veri >= kök verisi
{
    kök->sağ = _ekle (ağaç, kök->sağ, yeniPt, dahaYüksek);
    eğer (*dahaYüksek)
        // Sağ alt ağaç daha yüksek
        switch (kök->bal)
        {
            durum sol: // Sol yüksekti--şimdi EH
                kök->bal = EH;
                *dahaYüksek = false;
                break;

            durum EH: // Dengeliydi--şimdi RH
                root->bal = RH;
                mola;

            durum RH: // Doğru yüksek--döndür
                root = insRightBal
                    (root, daha uzun);
                kır;
        } // anahtar
    kökü döndür;
    } // aksi takdirde yeni veri >= root verisi
    return root;
} // _insert

```

Program 8-4 Analizi Bu karmaşık algoritmanın mantığını tam olarak anlamak için dikkatlice incelenmesi gerekmektedir. Doğru ekleme noktasını bir yaprak düğümde bulmak için bir özyinelemeli arama ile başlarız. Ekleme işlemi tamamlandıktan (ifade 16) ve daha yüksek bayrağı ayarladıktan sonra, ağaçtan geri çıkmaya başlarız, giderken dengeyi inceleriz. Sol tarafta ekleme yaptıktan sonra dengeyi kontrol ederiz (ifade 25), sol dengeyi çağırırız. Sağ tarafta ekleme yaptıktan sonra dengeyi kontrol ederiz (ifade 49), sağ dengeyi çağırırız.

Denge faktörlerinin ayarlaması üç işlevde gerçekleşir. Mevcut kökün denge faktörünü ekleme işlevinde ayarlanır. Alt ağaç denge faktörleri sol ve sağ denge işlevlerinde ağaç döndürüldüğünde ayarlanır. Hepsi bu bölümdeki "Denge Faktörlerini Ayarlama" kısmında tartışılan genel kuralları takip eder.

Her düğüm gerekli şekilde ayarlandıktan sonra, kökü geri döneriz. Bu, bir ağacın kökünün veri ekledikçe değişebileceği için gereklidir. Bu, ikili arama ağacı ile AVL ağacı arasındaki en büyük farklardan biridir. İkili arama ağacında, eklenen ilk düğüm her zaman köktür, silinmediği sürece.

Bu bir özinelemeli fonksiyon olduğu için bir temel duruma sahip olmalıdır. Bunu görebiliyor musun? Temel durum, bir yapıya bulduğumuzda ve 18. ifadede kökü döndürdüğümüzde gerçekleşir.

İçsel Sol Denge Fonksiyonu

Ağaç, sol alt ağaçta bir ekleme sonrası daha yüksekse, bir döndürmenin gerekli olup olmadığını kontrol etmemiz gerekir. Yukarıda gördüğümüz gibi, bu yalnızca ağaç zaten sol yüksekse gereklidir. Bu durumda, sol alt ağacı döndürerek dengeyi sağlamak için sol denge fonksiyonunu çağırırız. Bu algoritma Program 8-5'te gösterilmiştir. Tasarımı Algoritma 8-2'de gösterilmektedir.

PROGRAM 8-5 İçsel Ekleme Sol Denge Fonksiyonu

```

123456789 /* ===== insLeftBalance =====
Ağaç sola dengesiz. Bu fonksiyon
ağacı sağa döndürür.
Önce Ağaç sol yüksek
Sonra Denge sağlandı; potansiyel olarak yeni kök döndürülüyor
*/
NODE* insLeftBal (NODE* kökü, bool* daha uzun)
{
// Yerel Tanımlar
NODE* rightTree;
NODE* leftTree;

// İfadeler
leftTree = root->left;
switch (leftTree->bal)
{
durum LH: // Sol Yüksek--Sağa Döndür
root->bal = EH;
solAğaç->bal = EH;

// Sağa Döndür
root = sağaDöndür (root);
*dahaYüksek = false;
kırmak;
durum EH: // Bu bir hata pri
ntf ("\aHata insLeftBal'da"); çıkış (100);

durum RH: // Sağ Yüksek-Çift gerektirir
// döndürme: önce sola, sonra sağa
sağAğaç = solAğaç->sağ;
switch (sağAğaç->bal)
{
durum LH: root->bal = RH; solAğaç->bal = EH;

break;
durum EH: root->bal = EH;
solAğaç->bal = LH;
}
}
}

```

devam ediyor

PROGRAM 8-5İçsel Ekle Sol Denge Fonksiyonu (devam ediyor)

```

38394041424344454647484950515253
        kırmak;
        case RH: root->bal = EH;
            leftTree->bal = LH;          b
            reak;
            } // sağ ağaç değişimi

            rightTree->bal = EH;
            // Soldan Döndür
            root->left = rotateLeft (leftTree);

            // Sağdan Döndür
            root = sağaDöndür (root);
            *dahaYüksek = false;

        } // değişim
        return root;
    } // solDenge

```

Program 8-5 Analizi Fonksiyon, hangi eylemin yapılması gerektiğini belirlemek için sol alt ağacın denge faktörünü inceler.

Eğer alt ağaç sol yüksekse—yani, solun solundaysa—gerekli olan tek şey basit bir sağ döndürmedir.

Eğer sağ yüksekse, solun sağındadır ve çift döndürme gereklidir.

Üçüncü olası denge faktörü, çift yüksek, mümkün değildir. Bu durumun gerçekleşmesi halinde, eylemimiz programı iptal etmek olacaktır çünkü bu imkansız duruma ulaşmak için bir mantık hatası içermesi gerekir.

İç Dönüş Fonksiyonları

Bir alt ağacın döndürülmesi, dört alt ağaç işaretçisi arasında işaretçi değişimi gerektirir. Mantık, iki değeri değiştirme mantığının oldukça basit bir uzantısıdır. Soldan ve sağdan döndürme kodu Program 8-6'da gösterilmiştir. Tasarımı Algoritma 8-3'te gösterilmektedir. Mantığı incelerken, Şekil 8-10'a başvurmanın faydalı olabileceğini görebilirsiniz.

PROGRAM 8-6İç Ekleme Soldan ve Sağdan Döndürme Fonksiyonu

```

123456789
/* ===== rotateLeft =====
   Ağacı sola döndürmek için işaretçileri değiştirir.
   Önce root döndürülecek ağaca işaret eder   Sonra
   Düğüm döndürülür ve yeni kök döndürülür
*/
NODE* solaDondur (NODE* kök)
{
    // Yerel Tanımlar
    NODE* geçiciPtr;

    // Beyanlar
    geçiciPtr = kök->sağ;
    kök->sağ = geçiciPtr->sol;

```

devam ediyor

PROGRAM 8-6 Dahili Ekleme Sola ve Sağa Döndürme Fonksiyonu (devam ediyor)

```

14151617181920212223242526272829303132333435
tempPtr->left = root;

return tempPtr;
} // sola döndür

/* ===== rotateRight =====
Ağacı sağa döndürmek için işaretçileri değiştirin.
Önce root döndürülecek ağaca işaret eder Sonra
Düğüm döndürülür ve yeni kök döndürülür
*/
DÜĞÜM* RotateRight (DÜĞÜM* kökü)
{
// Yerel Tanımlar
NODE* geçiciPtr;

// Beyanlar
tempPtr = kök->sol;
kök->sol = tempPtr->sağ;
tempPtr->sağ = kök;

tempPtr'yi döndür;
} // rotaryRight

```

AVL Ağacını Sil

AVL ağacından silme işlemi, veri eklediğimizde karşılaştığımız birçok aynı sorunla karşılaşır. Ancak ilginç bir şekilde, yalnızca döndürme algoritmaları yeniden kullanılabilir; dengeleme algoritmaları silme işlemi için yeniden yazılmalıdır.

Silme işlemi için kullandığımız tasarımı, ekleme işlemi için de kullanıy oruz. Uygulama programı arayüzü yalnızca AVL ağacı baş yapısını görür. S ilinecek anahtara bir *void* işaretçi geçirir, böylece arama işlevi, ağacın için deki silinecek düğümü bulmak için çağrılabilir. Silme işlemi başarılı olursa, AVL ağacı silme işlevi true döner; düğüm bulunamazsa, false döner.

Uygulama arayüzü için kod Program 8-7'de gösterilmiştir.

PROGRAM 8-7 AVL Ağacı Silme Uygulama Arayüzü

```

123456789
/* ===== AVL_Delete =====
Bu fonksiyon ağaçtan bir düğümü siler ve
gerekirse yeniden dengelemektedir.
Önce ağaç başlatıldı--null ağaç kabul edilebilir
dltKey silinecek anahtara işaretçidir
Sonra düğüm silindi ve alanı geri dönüştürüldü
-veya- Bir hata kodu döndürülür
Başarı (true) veya Bulunamadı (false) döndür

```

devam ediyor

PROGRAM 8-7 AVL Ağaç Silme Uygulama Arayüzü (devam ediyor)

```

10111213141516171819202122232425262728
bool AVL_Delete (AVL_TREE* ağacı, void* dltKey)
{
    // Yerel Tanımlar
    bool dahaKısa;
    bool başarı;
    NODE* yeniKök;

    // Beyanlar
    newRoot = _delete (tree,      tree->root, dltKey,
                        &dahaKısa, &başarı);

    eğer (başarı)
    {
        tree->root = yeniKök;
        (tree->count)--;
        return true;
    } // eğer

    aksi takdirde
    false döndür;
} // AVL_Delete

```

Program 8-7 Analizi Uygulama arayüzü silme fonksiyonu, kullanıcıdan parametreleri alır ve bunları özyinelemeli silme fonksiyonu için yeniden biçimlendirir. Silme işlemi tamamlandığında, fonksiyon ağaç sayısını günceller ve durumu çağıran fonksiyona geri iletir.

İçsel Silme Fonksiyonu

Gerçek iş, Program 8-8'de gösterilen özyinelemeli AVL ağaç silme fonksiyonu tarafından yapılır. Tasarımı, Algoritma 8-4'te gösterilmiştir.

PROGRAM 8-8 Dahili Silme Fonksiyonu

```

123456789
/* ===== _delete =====
Düğümü ağaçtan siler ve gerekiyorsa
ağacı dengeleyerek.
    Ön  ağaç başlatıldı--null ağaç kabul edilebilir.
        dltKey, silinecek düğümün anahtarını içerir
        shorter, ağacın daha kısa olduğunu gösterir
    Post  düğümü silinir ve alanı geri dönüştürülür
        -veya- anahtar bulunamazsa, ağaç değişmez
    Silinmişse true döner; bulunamazsa false döner
        köke işaretçi
*/
10111213141516
NODE* _delete (AVL_TREE* ağacı, NODE* kökü,
               void* dltKey, bool* daha kısa,
               bool* başarılı)
{

```

devam ediyor

PROGRAM 8-8İç Silme Fonksiyonu (devamı)

```

1718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263
// Yerel Tanımlar
NODE* dltPtr;
NODE* exchPtr;
NODE* newRoot;

// İfadeler
if (!root)
{
    *kisa = false;
    *başarı = false;
    return NULL;
} // eğer

if (ağaç->karşılaştırdltKey, root->dataPtr < 0)
{
    kök->sol = _delete (ağaç,
                        kök->sol, dltKey,
                        kısa, başarı);

    if (*kisa)
        root = dltRightBal (root, kısa);
} // eğer daha az
else if (tree->compare(dltKey, root->dataPtr) > 0) {

    kök->sağ = _delete (ağaç,
                        kök->sağ, dltKey,
                        dahaKisa, başarı);

    if (*dahaKisa)
        root = dltLeftBal (root, dahaKisa);
} // eğer büyükse

aksi takdirde
// Eşit düğüm bulundu
{
    dltPtr = root;
    if (!root->right)
        // Sadece sol alt ağaç
    {
        yeniKök = root->left;
        *başarı = true;
        *dahaKisa = true;
        free (dltPtr);
        return yeniKök;
    } // eğer doğruysa

    başka
    if (!root->left) // Sadece sağ alt
        ağaç
        {
            yeniKök = root->right;

```

devam ediyor

PROGRAM 8-8İç Silme Fonksiyonu (devamı)

```

646566676869707172737475767778798081828384
        *başarı = true;          *kıs
        a = true;

        free (dltPtr);
        return newRoot;          // temel durum
    } // eğer

    // Döğümü sil, iki alt ağaç içerir
    {
        exchPtr = kök->sol;
        while (exchPtr->sağ)
            exchPtr = exchPtr->sağ;
        root->dataPtr = exchPtr->dataPtr;
        root->left = _delete (ağaç,
            root->left, exchPtr->dataPtr,
            kısa, başarı);

        if (*kısa)
            root = dltRightBal (root, kısa);    } // aksi
        takdirde
    } // eşit döğüm
    return root;
} // _delete

```

Program 8-8 AnaliziYine uzun, nispeten karmaşık bir uygulama görüyoruz. Bu fonksiyon kesinlikle bir sayfa olan yapılandırılmış programlama kılavuzundan daha uzun olmasına rağmen, parçalanmaya kolayca uygun değildir.

Fonksiyonun ilk kısmının silinecek döğümü aradığını gördüğümüz gibi, eğer silinecek döğümü bulmadan önce bir yaprağa ulaşırsa, başarı bayrağını false olarak ayarlar ve null işaretçi döner. Bu, üç temel durumdan ilki.

Silinecek döğüm bulunduğunda, bunun bir yaprak mı yoksa yaprak benzeri bir döğüm mü olduğunu belirliyoruz. Silme işlemlerinin yalnızca bir yaprakta gerçekleşebileceğini unutmayın. Eğer bir döğümün iki alt ağacı varsa, yerini alacak bir yaprak döğüm aramalıyız. Öncelikle sağ alt ağacı olup olmadığını kontrol ederiz. Eğer yoksa, sol alt ağaç işaretçisini kökün yerini alacak işaretçi olarak kaydederiz. Eğer bir sağ alt ağaç varsa, sol alt ağaç işaretçisini kontrol ederiz. Eğer yoksa, sağ alt ağaç işaretçisini kökün yerini alacak işaretçi olarak kaydederiz. Şu anda sıfır veya bir alt ağacı olduğunu varsayarsak, başarıyı ve kısaı true olarak ayarlıyoruz, silinen döğümün belleğini serbest bırakıyoruz ve yeni kök işaretçisini döndürüyoruz.

Bunlar ikinci (ifade 57) ve üçüncü (ifade 67) temel durumlar.

Eğer silinecek döğümün iki alt ağacı varsa, bunun yerini alacak bir döğüm bulmalıyız. Tasarımımız, sol alt ağacı için en büyük döğümü arar. Bunu bulduğumuzda, verilerini silinen verilerin yerine koyarız ve ardından silinmesi gereken geçerli bir yaprak döğüm olduğunu bildiğimiz döğümü silmek için silme fonksiyonunu özyinelemeli olarak çağırırız. Bu mantık, ifadeler 72 ile 80 arasında gösterilmektedir.

Ağaçtan geri çıkarken, hala dengede olduğundan emin olmalıyız. Eğer sol taraftan bir döğüm sildiysek ve ağaç kısaldıysa, buna sağ denge silme (durum 36 ve 80) diyoruz. Eğer sağ taraftan bir döğüm sildiysek ve ağaç kısaldıysa, buna sol denge silme (durum 44) diyoruz. Elbette, eğer ağaç kısalmadıysa, dengelemeye gerek yoktur.

İçsel Sağ Denge Silme Fonksiyonu

Sağ denge, bir silme işlemi gerçekleştiğinde ve ağacın yeniden dengelenmesi gerektiğinde kullanılır. Yalnızca kök sağ yüksekse yeniden dengelememiz gerekir. Bu durumda, sağ yüksek bir ağacın sol tarafında bir düğüm sildik, bu da sağ alt ağaçta iki seviyede dengesiz hale getirdi. Tersine, eğer kök sol veya eşit yüksekse, döndürmeye gerek yoktur. Kod, Program 8-9'da gösterilmiştir. Tasarımı ise Algoritma 8-5'te gösterilmektedir.

PROGRAM 8-9 İç Silme Sağ Denge Fonksiyonu

123456789	<pre> /* ===== dltRightBal ===== Ağaç soldan bir silme işleminden sonra daha kısadır. Bu fonksiyon denge faktörlerini ayarlar ve gerekirse ağacı sola döndürür. Önce ağaç daha kısa Sonra Denge faktörleri sıfırlanır-dengelenme sağlanır Potansiyel olarak yeni kök döner */ NODE* dltRightBal (NODE* kökü, bool* daha kısa) { // Yerel Tanımlar NODE* rightTree; NODE* leftTree; // İfadeler switch (root->bal) { durum sol: // Sol Silindi--Şimdi dengeli root->bal = EH; kırmak; durum EH: // Şimdi Sağ yüksek root->bal = RH; *shorter = false; break; durumda sağ taraf: // Sağ Yüksek - Sola Döndür rightTree = root->right; if (rightTree->bal == LH) // Çift döndürme gerekli { leftTree = rightTree->left; anahtarı (leftTree->bal) { case LH: rightTree->bal = RH; root->bal = EH; break; </pre>
1011121314151617181920212223242526272829303132333435363738	

devam ediyor

PROGRAM 8-9İçsel Sağ Denge Silme Fonksiyonu (devam ediyor)

```

39404142434445464748495051525354555657585960616263646566676869707172
case EH: kök->top = EH;
        rightTree->top = EH;
        break;
        case RH: root->bal  = LH;
                rightTree->bal = EH;
                break;
        } // anahtar

leftTree->bal = EH;

// Önce Sağa Sonra Sola Döndür
root->right =
    rotasyonRight (rightTree);
root = rotaryLeft (root);
    } // if rightTree->bal == LH
else
{
    // Sadece Tek Dönme
    switch (rightTree->bal)
    {
        durum sol:
            case RH: root->bal  = EH;
                    rightTree->bal = EH;
                    break;
            case EH: kök->bal = RH;
                    rightTree->bal = LH;
                    *daha kısa = false;
                    break;
    } // switch rightTree->bal

    root = rotaryLeft (root);
    } // else
    } // anahtar
return root;
} // dltRightBal

```

Program 8-9 AnaliziYukarıda söylediğimiz gibi, yalnızca sağ ağacın yüksekliği sol ağacın yüksekliğinden iki kat daha büyükse döndürme emiz gerekiyor. Bu nedenle, ağaç silmeden önce sol yüksek veya hatta yüksekse, döndürme gerekli d eğildir. Eğer sağ yüksekse, döndürülmesi gerekir. Eğer sağ alt ağacı sol yüksekse, çift döndürme gereklidir; aksi takdirde, yalnızca tek bir döndürme gereklidir. Her iki durumda da önce denge faktörlerini ayarlıyoruz ve sonra döndürüyoruz. Mantiğı incelerken, Şekil 8-11'e geri dönmenin faydalı olabileceği ni görebilirsiniz.

Bir AVL Ağacı Al

Alma fonksiyonu, istenen düğüm bulunana kadar ağacın sol-sağ yapısını takip eder. Bulunduğunda, verinin adresi çağırılan fonksiyona döndürülür. Veriler bulunamazsa, bir null işaretçi döndürülür.

Tasarım, yukarıda açıklanan ekleme ve silme algoritmalarına benzer. Bir uygulama arayüzü işlevi, ağaca ve bir işaretçiye bir işaretçi sağlar

anahtara. Ardından veriyi bulmak için bir ADT özyinelemeli fonksiyonu çağırıyoruz . ADT alma fonksiyonunun kodu Program 8-10'da gösterilmiştir. Özyinelemeli fonksiyonun kodu ise Program 8-11'de gösterilmiştir.

PROGRAM 8-10 AVL Ağaç Uygulama Arayüzü Alma

123456789	/* ===== AVL_Retrieve ===== Alma düğümü, istenen anahtarı içeren düğümü arar ve verisi ne işaretçi döner. Ön Ağaç oluşturulmuş (null olabilir) veri, bulunması gereken anahtarı içeren data yapısına işaretçidir Son Ağaç arandı ve veri işaretçisi döndürüldü Dönüş Eşleşen düğümün adresi döndürüldü Bulunamazsa, NULL döner
101112131415161718	*/ void* AVL_Retrieve (AVL_TREE* ağacı, void* keyPtr) { // Beyanlar if (ağaç->kök) return _retrieve (ağaç, keyPtr, ağaç->kök); else NULL'u döndür; } // AVL_Retrieve

Program 8-10 Analizi Veri alma fonksiyonu oldukça basittir. Sadece özyinelemeli fonksiyonun farkında ol madığı ek bir parametreye ihtiyaç duyduğu için gereklidir. Basitçe ağaç yapısının a ve anahtara işaretçiyi geçirir, ağaç köküne işaretçiyi ekler.

PROGRAM 8-11 Dahili Alma Fonksiyonu

123456789	/* ===== _retrieve ===== İstenen anahtarı içeren düğümü arar ve verisini çağırarak fonksiyona döner. Önce AVL_Retrieve ağaç, keyPtr, kök geçirir keyPtr, bulunacak anahtarı içeren veri yapısına işaret çidir Sonra ağaç arandı; veri işaret çisi döndürüldü Dönüş Eşleşen düğümün adresi döndürüldü bulunamazsa, NULL döndürüldü
10111213141516	*/ void* _retrieve (AVL_TREE* ağacı, void* keyPtr, NODE* kökü) { // Beyanlar eğer (kök) {

devam ediyor

PROGRAM 8-11 Dahili Alma Fonksiyonu (devamı)

```

17 18 19 20 21 22 23 24 25 26 27 28
    if (tree->compare(keyPtr, root->dataPtr) < 0)    return
        _retrieve(tree, keyPtr, root->left); else if (tree->compare
(keyPtr, root->dataPtr) > 0)    return _retrieve(tree, keyPtr, ro
ot->right);    else

        // Eşit anahtar bulundu
        root->dataPtr'yi döndür;
    } // eğer kök

    aksi takdirde
        // Veri ağaçta yok
        NULL'u döndür;
} // _retrieve

```

Program 8-11 AnaliziAlma fonksiyonu, ağaç oluşturulurken ağaç yapısında saklanan karşılaştırma fonksiyonunu kullanır. Eğer arama argümanı, **keyPtr**, kök verisinin anahtarından daha küçükse, kendisini sol alt ağaç ile kök olarak çağırır. Eğer arama argümanı kök verisinin anahtarından büyükse, kendisini sağ alt ağaç ile kök olarak çağırır. Eğer argüman kökten ne büyük ne de küçükse, eşit olmalıdır, bu yüzden kök verisinin veri işaretçisini döndürür.

18 ve 20. ifadelerdeki özyinelemeli fonksiyon çağrılarını dikkatlice inceleyin. Bunların, özyinelemeli çağrının döndürdüğü adresi döndürdüğünü not edin. Bu şekilde, temel durumların döndürdüğü adres, nihayetinde arayüz fonksiyonu tarafından uygulamaya geri döndürülene kadar ağaç yapısında yukarı doğru iletilir.

Bir AVL Ağacını Gezinme

Gezi, ağacın sıralı gezisini kullanır ve düğüm işlenirken uygulama bağımlı işlem fonksiyonunu çağırır. Gezi standart olmasına rağmen, uygulama işleme fonksiyonu standart değildir. Bu nedenle, kullanan uygulama geziyi çağırdığında, verileri işleyen fonksiyonun adresini de geçirir. İşleme fonksiyonu yalnızca bir parametre kullanır: işlenecek düğümün adresi. Gezinme fonksiyonu için uygulama arayüzü Program 8-12'de gösterilmiştir.

PROGRAM 8-12 AVL Ağaç Uygulama Arayüzünü Gezinme

```

1  /* ===== AVL_Gezinme =====
23  Ağacı sıralı gezinme ile işleyin.
45  Ön  Ağaç oluşturulmuş (null olabilir)
67  işlem "gezinme" sırasında düğümleri "ziyaret eder"
89  Son  Düğümler LNR (sıralı) sırasına göre işlenmiştir
*/
void AVL_Traverse (AVL_TREE* ağacı,
                  void (*process) (void* dataPtr))
{

```

devam ediyor

PROGRAM 8-12 AVL Ağaç Uygulama Arayüzünü Gezinme (devamı)

```

10111213 // Beyanlar
          _traversal (tree->root, process);
          dön;
          , // AVL_Traverse sonu

```

İçsel Gezinme Fonksiyonu

Traverse fonksiyonu, aslında ağacı dolaşan ADT iç fonksiyonunu çağırır. Kod Program 8-13'te gösterilmiştir.

PROGRAM 8-13 İç Dolaşım Fonksiyonu

```

123456789 /* ===== _traversal =====
          Sıralı ağaç dolaşımı. Bir düğümü işlemek için, dolaşım çağrıldığı
          ında geçirilen fonksiyonu kullanıyoruz.
          Önce Ağaç oluşturuldu (null olabilir)
          Sonra Tüm düğümler işlendi
          */
          void _traversal (NODE* root,
                          void (*process) (void* dataPtr))
          {
          // Beyanlar
          eğer (kök)
          {
          _traversal (kök->sol, süreç);
          süreç (kök->dataPtr);
          _traversal (kök->sağ, süreç);
          } // if
          geri dön;
          } // _traversal
101112131415161718

```

Boş bir AVL Ağacı

Boş fonksiyonu, ağaç sayısını kontrol eder. Eğer sıfır, true döner; aksi takdirde, false döner. Kod Program 8-14'te gösterilmiştir.

PROGRAM 8-14 Boş AVL Ağaç Uygulama Arayüzü

```

123456789 /* ===== AVL_Empty =====
          Ağaç boşsa true, herhangi bir veri varsa false döner.
          Önce Ağaç oluşturulmuştur. Null olabilir
          Döner Ağaç boşsa true, herhangi bir veri varsa false
          */
          bool AVL_Empty (AVL_TREE* ağacı)
          {
          // Beyanlar
          return (ağaç->sayı == 0);
          } // AVL_Empty
10

```

Tam AVL Ağacı

Çünkü C dilinde mevcut belleği test etmenin bir yolu yok, aslında bir düğüm ayırmayı denememiz gerekiyor. Başarılı olursak, onu sileriz ve false döneriz—liste dolu değil. Ayırma başarısız olursa, true döneriz—başka bir düğüm için yeterli bellek yok. Kod Program 8-15'te gösterilmiştir.

PROGRAM 8-15 Tam AVL Ağaç Uygulama Arayüzü

123456789	/* ===== AVL_Full ===== Başka bir düğüm için yer yoksa, true döner. Ön Ağaç oluşturuldu Başka bir ekle me için yer yoksa True döner, yer varsa false döner.
101112131415161718192021	*/ bool AVL_Full (AVL_TREE* ağacı) { // Yerel Tanımlar DÜĞÜM* yeniPtr; // Beyanlar newPtr = (DÜĞÜM*)malloc(sizeof (*(ağaç->kök))); if (newPtr) { free (newPtr); return false; } // eğer aksi takdirde true değerini d öndür; } // AVL_Full

AVL Ağaç Sayısı

Program 8-16'da gösterilen sayım fonksiyonu, ağaçta şu anda bulunan düğüm sayısını basitçe döndürür.

PROGRAM 8-16 AVL Ağaç Sayısı Uygulama Arayüzü

123456789	/* ===== AVL_Count ===== Ağaçtaki düğüm sayısını döndürür. Önce Ağaç oluşturulmuş Ağaç sayısını döndürür
10	*/ int AVL_Count (AVL_TREE* ağacı) { // İfadeler return (tree->count); } // AVL_Count

AVL Ağacını Yok Et

AVL ağaç soyut veri tipindeki son fonksiyon, AVL ağacını yok etme fonksiyonudur. Ağaç artık gerekli olmadığına, tüm veri düğümlerini ve ağaç baş yapısını fiziksel olarak silmek ve serbest bırakmak için kullanılır. Silinmesi gereken tüm veri ve düğümleri bulmak için ağacı gezmemiz gerektiğinden, fiziksel silmeleri yapmak için bir özyinelemeli fonksiyon çağırıyoruz.

Yıkım işlevinin mantığı, daha önce gördüğümüz yıkım işlevleriyle paraleldir. Kod, Program 8-17'de gösterilmiştir.

PROGRAM 8-17 AVL Ağaç Yıkım Uygulama Arayüzü

123456789	/* ===== AVL_Destroy ===== Ağaçtaki tüm verileri siler ve belleği geri dönüştürür. Düğümmler, ağacı sıralı sırayla gezmek için bir rekürsif fonksiyon çağrılarak silinir. Önce geçerli bir ağaca işaret eden bir işaretçi Sonra Tüm veriler ve baş yapısı silindi Dönüş null baş işaretçisi
101112131415161718	*/ AVL_TREE* AVL_Destroy (AVL_TREE* ağacı) { // Beyanlar eğer (ağaç) _destroy (tree->root); // Tüm düğümler silindi. Yapıyı serbest bırak free (tree); NULL değerini d öndür; } // AVL_Destroy

Program 8-17 Analizi Mantık basittir. Öncelikle, ağaç işaretçisini test ederek geçerli bir ağaca sahip olduğumuzdan emin oluruz. Eğer geçerliyse—yani, null değilse—fiziksel silme işlemlerini gerçekleştiren özyinelemeli alt işlevi çağırırız. Döndüğümüzde, ağaç yapısını kendisini sileriz ve null işaretçi döneriz.

İçsel Yok Etme Fonksiyonu

Yinelenen silme işlevinin kodu Program 8-18'de gösterilmiştir.

PROGRAM 8-18 Dahili Yok Etme İşlevi

1234567	/* ===== _destroy ===== Ağaçtaki tüm verileri siler ve belleği geri dönüştürür. Düğümmler, ağacı sıralı sırayla gezmek için bir özyinelemeli fonksiyonu çağırarak silinir. Ön kök geçerli ağaç/alt ağaç için işaretçidir Son Tüm veriler ve baş yapısı silindi Dönüş null baş işaretçisi
---------	--

devam ediyor

PROGRAM 8-18 Dahili Yok Etme İşlevi (devamı)

89	*/
	void _destroy (NODE* kökü)
	{
10	// İfadeler
11	if (root)
12	{
13	yok et (kök->sol); \in bedava
14	(kök->dataPtr); _yok et (kö
15	k->sağ); bedava (kök);
16	
17	} // if
18	return;
19	} // _destroy
20	

Program 8-18 AnaliziYinelenen silme işlevindeki mantık biraz daha karmaşıktır. Büyük soru şudur:

Veriyi ne zaman silmeliyiz ve düğümü ne zaman silmeliyiz?. Her birini yalnızca bir kez ve doğru zamanda yaptığımızdan emin olmalıyız.

Veriler, sol alt ağacından dönerken silinir. Bu, bir AVL ağacının verilerini işlemek için gerekli olan mantıksal noktadır. Bu, 15. ifadede yapılır. Ancak, h enüz düğümü silmeye hazır değiliz çünkü sağ alt ağacı işlemedik. Onu silme k için sağ alt ağaç gezintisinden döndüğümüzü beklemeliyiz (bkz. 17. ifade).

8.4 Uygulama—Kelimeleri Say

Bu bölümde, bir belgedeki tüm kelimeleri içeren ve her kelimenin kaç kez kullanıldığını sayan bir ağaç yapısı kullanan bir AVL ağaç uygulaması oluşturuyoruz. Bu program, Bölüm 8.3'te bulunan AVL ağaç soyut veri türünü kullanır, "AVL Ağaç Soyut Veri Türü."

Veri Yapısı

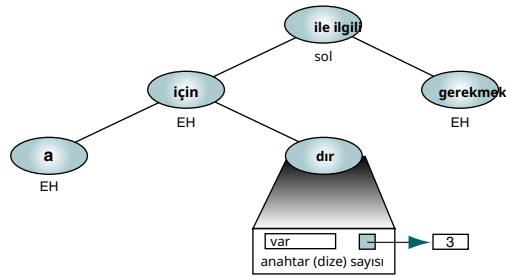
Uygulama veri yapısı Şekil 8-14'te gösterilmiştir. AVL ağacındaki her giriş, belgede geçen bir kelimeyi ve kelimenin belgede kaç kez geçtiğini içeren bir tamsayıya işaretçi içerir.

AVL ağacı, bir düğümü ağaca geri yazma işlevine sahip olsaydı—yani, güncelleme işlevine—sayacı düğümde saklayabilirdik.

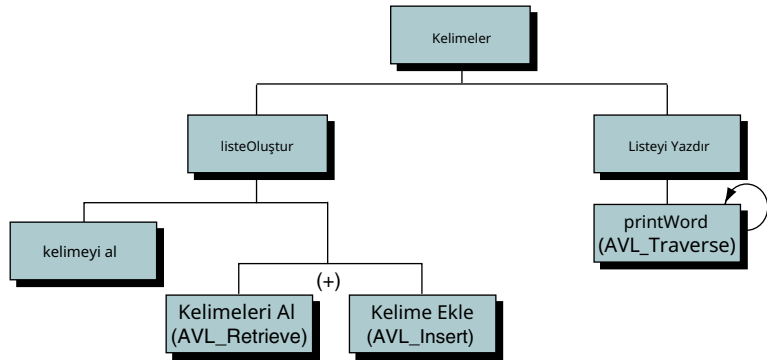
Program Tasarımı

Dosyayı işlerken, bir kelimeyi ayrıştırıyoruz ve ardından ağacı arıyoruz.

Eğer kelime zaten ağaçta varsa, sayacına ekliyoruz. Eğer ağaçta yoksa, ağaca yeni bir giriş ekliyoruz. Şekil 8-15 program tasarımını göstermektedir. Kullanıldığı yerlerde AVL ağacı ADT işlevlerini belirtmişizdir. Her bir işlevin kodu aşağıdaki bölümlerde gösterilmektedir.



ŞEKİL 8-14 Kelime Sayma Veri Yapısı



ŞEKİL 8-15 Kelime Sayma Tasarımı

Kelime Sayma Programı

Programın veri yapısı, prototip bildirimleri ve ana akışı Program 8-19'da gösterilmiştir. İncelediğinizde, özellikle create list işlev çağrısına (if ade 35) dikkat edin. AVL ağaç ADT'sini kullandığımız için, ağaçta saklanan verilerin iki örneğini karşılaştıran bir karşılaştırma işlevi yazmalıyız. Bu karşılaştırma işlevi, ağaç oluşturulduğunda AVL ağacına iletilir.

PROGRAM 8-19 Kelimleri Say

```

12345678 /* Bu program bir dosyadaki kelimeleri sayar.
          Yazar:
          Tarih:
          */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

```

devam ediyor

PROGRAM 8-19 Kelimleri Say (devam ediyor)

```

9  #include <stdbool.h>
10 #include "P8AVLADT.h"
11
12 // Yapılar
13 typedef yapısı
14 {
15     char kelime[51]; // Bir kelime
16     int sayım;
17 } VERİ;
18
19 // Prototip Bildirimleri
20 void listeyiOluştur (AVL_AĞACI* kelimeListesi);
21 void kelimeEkle (AVL_AĞACI* kelimeler);
22 void kelimeSil (AVL_AĞACI* kelimeler);
23 void listeyiYaz (AVL_AĞACI* kelimeListesi);
24 void kelimeyiYaz (void* birKelime);
25 bool kelimeAl (VERİ* birKelime, FILE* fpKelimler);
26 int kelimeleriKiyasla (void* arguPtr, void* listePtr);
27
28 int main (void)
29 {
30     // Yerel Tanımlar
31     AVL_AĞACI* kelimeListesi;
32
33     // İfadeler
34     printf("Belgedeki kelimeleri saymaya başla"); kelimeList
35     esi = AVL_Oluştur (kelimleriKiyasla);
36
37     listeyiOluştur (kelimeListesi
38     ); yazdırListe (kelimeListesi);
39
40     printf("Kelimleri sayma sona erdi");
41     return 0;
42 } // ana

```

Program 8-19 Analizi İyi bir tasarım, kodu ana işlevde minimumda tutar. Bu programda bir merhaba mesajı ile başlıyoruz; `AVL_Create`, `buildList` ve `printList` çağırıyoruz; ve sonra bir hoşça kal mesajı ile bitiriyoruz. İyi bir yönetici gibi, `main` tüm işlemleri alt işlevlere devrediyor.

Liste Oluştur

Liste oluşturma işlevi, dosyadan bir kelime okur, ağacın içinde arar ve kelime bulunursa, kelimenin sayacına 1 ekler. Kelime henüz ağaçta yoksa, bir sayaç oluşturur, 1 olarak ayarlar ve kelimeyi ağaçta ekler. Kod Program 8-20'de gösterilmiştir.

PROGRAM 8-20 Liste Oluştur

```

123456789 /* ===== buildList =====
Dosyayı okur ve dosyada kullanılan tüm kelimelerin listesini iç
eren AVL ağacı oluşturur her kelimenin dosyada kaç kez bulun
duğunun sayısı ile birlikte.
Önce kelimeListesi oluşturulmuştur
Sonra AVL ağacı (liste) oluşturuldu veya hata döndürüldü
*/
void buildList (AVL_TREE* kelimeListesi)
{
// Yerel Tanımlar
char dosyaAdı[25];
FILE* fpKelimeler;

bool başarı;
VERİ* birKelime;
VERİ yeniKelime;

// Beyanlar
printf("İşlenecek dosyanın adını girin: ");
scanf ("%24s", dosyaAdı);

fpWords = fopen (dosyaAdı, "r");
if (!fpWords)
{
printf("%s açılmadı\n", dosyaAdı); printf("Lütfen ismi kontrol
edin ve tekrar deneyin."); exit (100);
} // !fpWords

while (getWord (&newWord, fpWords)) {

aWord = AVL Retrieve(wordList, &(newWord));
if (aWord)
(word->count)++;
else
{
aWord = (DATA*) malloc (sizeof (DATA));
if (!aWord)
{
printf("Hata 120 buildList'te"); exit (120);
}
// eğer
// Listeye kelime ekle
*aWord = newWord;
aWord->count = 1;
başarı = AVL_Insert (wordList, aWord);
eğer (!başarı)
{

```

devam ediyor

PROGRAM 8-20 Liste Oluştur (devam ediyor)

```

495051525354555657
                                printf("Hata 121 buildList'te\");      exit (121);

                                } // taşma testi } // aksi ta
                                kdirde
                                } // iken

                                printf("AVL Ağacı Sonu");return;

                                } // buildList

```

Program 8-20 Analizi Bu fonksiyonu incelerken, dört önemli noktaya dikkat edin. İlk olarak, kullanıcıya dosya adını girmesine izin veriyoruz.

Bu program, herhangi bir metin dosyasını destekleyecek şekilde tasarlanmıştır, ve dosya adını klavyeden okuyarak mümkün olduğunca esnek hale getiriyoruz.

Herhangi bir programda yazılması en zor döngülerden biri, bir dosyayı okuma döngüsüdür. Bu programda basit bir teknik kullanıyoruz: bir kelimeyi ayrıştırabilirsem true, ayrıştıracak kelime yoksa false döndüren bir okuma fonksiyonu çağırıyoruz. Bu, işleme döngüsünün kontrolünü çok basit hale getiriyor.

Kelimeler için kullandığımız “akıllı” veri adlarını not edin. İlk, **aWord**, bir kelimeyi listeden almak için kullanılır. İkincisi, **newWord**, ağaçta yeni bir giriş oluşturmak için kullanılır. İyi adlar bir programı daha okunabilir hale getirir.

Son olarak, bellek hatalarını kontrol ettiğimizi not edin. Yeni kelimenin sayacı için bellek ayırdığımızda, taşma testi yaparız. Yine, yeni kelimeyi ağaca eklediğimizde, ekleme fonksiyonunun dönüşünü test ederiz. Eğer herhangi bir fonksiyon başarısız olursa, bir hata mesajı yazdırır ve programdan çıkarız.

Kelimeleri AI

Kelimeleri alma fonksiyonu dosya girişini ayrıştırır ve dosyadan bir kelime çıkarır. Kodunun Program 8-21’de gösterildiği gibi.

PROGRAM 8-21 Kelimleri AI

```

123456789
/* ===== getWord =====
Dosyadan bir kelime okur.
Önce hiçbir şey
Sonra referans parametreye okunan kelime
*/
bool getWord (DATA* aWord, FILE* fpWords)
{
// Yerel Tanımlar
char strIn[51];
int ioSonuç;
int lastChar;

// Beyanlar
ioResult = fscanf(fpWords, "%50s", strIn);
if (ioResult != 1)
    yanlış döndür;
10111213141516

```

devam ediyor

PROGRAM 8-21 Kelime Al (devam ediyor)

```

17181920212223
// Kelimenin sonundaki noktalama işaretini kopyala ve kaldır.
strcpy(aWord->word, strIn);
lastChar = strlen(aWord->word) - 1;
if (ispunct(aWord->word[lastChar]))
    aWord->word[lastChar] = '\0';
return true;
} // getWord

```

Program 8-21 Analizi Kelime ayrıştırmak, iki veya daha fazlaboşluk arasındaki tüm karakterleri bir dizeye kopyalamak anlamına gelir. Ancak, bir metin belgesini işlediğimiz için, kelimenin sonundaki herhangi bir noktalama işaretini kaldırmamız gerekiyordu. Bunu, son karakterin alfabetik olup olmadığını kontrol ederek yaptık. Eğer değilse, kelimedenden bir boş karakteri kaydırarak sildik.

Uygulamaya bağlı olarak, farklı bir düzenleme kriterleri seti kullanmak gerekli olabilir. Örneğin, bir programdaki kelimeleri ayrıştırmak için, herhangi bir sayıyı ve alt çizgi karakterini kelimenin bir parçası olarak tutmamız gerekiyor. Ayrıca, ayrılmış kelimeleri de ortadan kaldırmak isteyebiliriz. Programın geri kalanını mümkün olduğunca basit tutmak için, bu değişiklikler `getWord` içinde programlanacaktır.

Kelimleri Karşılaştır

Program 8-19'daki tanımımızda belirttiğimiz gibi, AVL ağacı verileri karşılaştırmayı bilmez. Bu nedenle, uygulama programcısının bir karşılaştırma işlevi sağlaması ve bunu ağaç oluşturulurken geçirmesi gerekir. **Program 8-22**, kelime sayma uygulamamız için karşılaştırma işlevini içerir. Karşılaştırmanın kelime üzerinde yapıldığını ve bu nedenle dize karşılaştırma işlevini gerektirdiğini unutmayın.

PROGRAM 8-22 Kelime Karşılaştırma Fonksiyonu

```

123456789
/* ===== compareWords =====
Bu fonksiyon, işaretçilerle tanımlanan iki tamsayıyı karşılaştırır.
Önce arguPtr ve listPtr, DATA'ya işaret eden işaretçilerdir.
Dönüş -1: arguPtr değeri < listPtr değeri
-0: arguPtr değeri == listPtr değeri
+1: arguPtr değeri > listPtr değeri
*/
int compareWords (void* arguPtr, void* listPtr)
{
// Yerel Bildirimler
DATA arguValue;
DATA listValue;

// İfadeler
arguValue = *(DATA*)arguPtr;
listValue = *(DATA*)listPtr;
101112131415161718

```

devam ediyor

PROGRAM 8-22Kelimeleri Karşılaştırma Fonksiyonu (devamı)

```

1920      return (strcmp(arguValue.word, listValue.word));
        }// karşılaştır

```

Kelimeleri Yazdır

Ağaç yapısının uygulanması ADT içinde gizli olduğundan, listeyi yazdırmak için doğrudan ağacı gezemeyiz. Bunun yerine, ağaç gezinti fonksiyonu olan `AVL_Traverse`'ü çağırmalıyız. Şimdi başka bir sorunuz var: gezinti fonksiyonu, ağacı gezerken verileri nasıl işleyeceğini bilmiyor. Bu nedenle, çağırdığımızda işlem fonksiyonunu gezinti fonksiyonuna geçiriyoruz. Program 8-23, gezinti fonksiyonuna yapılan çağrıyı ve bir kelimeyi yazdıran fonksiyonu içerir. Gettysburg Adresi'ndeki kelimelerin sayım sonuçları programın sonunda gösterilmektedir. Alanın tasarrufu açısından, yalnızca kelime listesinin başını ve sonunu gösteriyoruz.

Şekil 8-16, bir boş AVL ağacına “Dört” ve “skor” kelimelerinin ilk iki eklenmesini izlemektedir. Programı takip etmek için bunu kullanın.

PROGRAM 8-23Kelime Listesini Yazdır

```

123456789 /* ===== printList =====
              Her kelime için sayıyla birlikte listeyi yazdırır.
              Önce liste oluşturuldu
              Sonra liste yazdırıldı
*/
void printList (AVL_TREE* wordList)
{
// Beyanlar
printf("Listede bulunan kelimeler");AVL_Traverse (wordList, printWord);printf("Kelime listesinin sonu");return;

} //baskıListesi

1011121314151617181920212223242526 /* ===== printWord =====
              Listedden bir kelimeyi sayısıyla birlikte yazdırır.
              Önce ADT verileri yazdırmak için fonksiyonu çağırır
              Sonra veriler yazdırıldı
*/
void printWord (void* aWord)
{
// Beyanlar
printf("%-25s %3d",
              ((DATA*)aWord)->word, ((DATA*)aWord)->count);
return;
} // printWord

```

Sonuçlar:
Belgedeki kelimeleri saymaya başla

devam ediyor

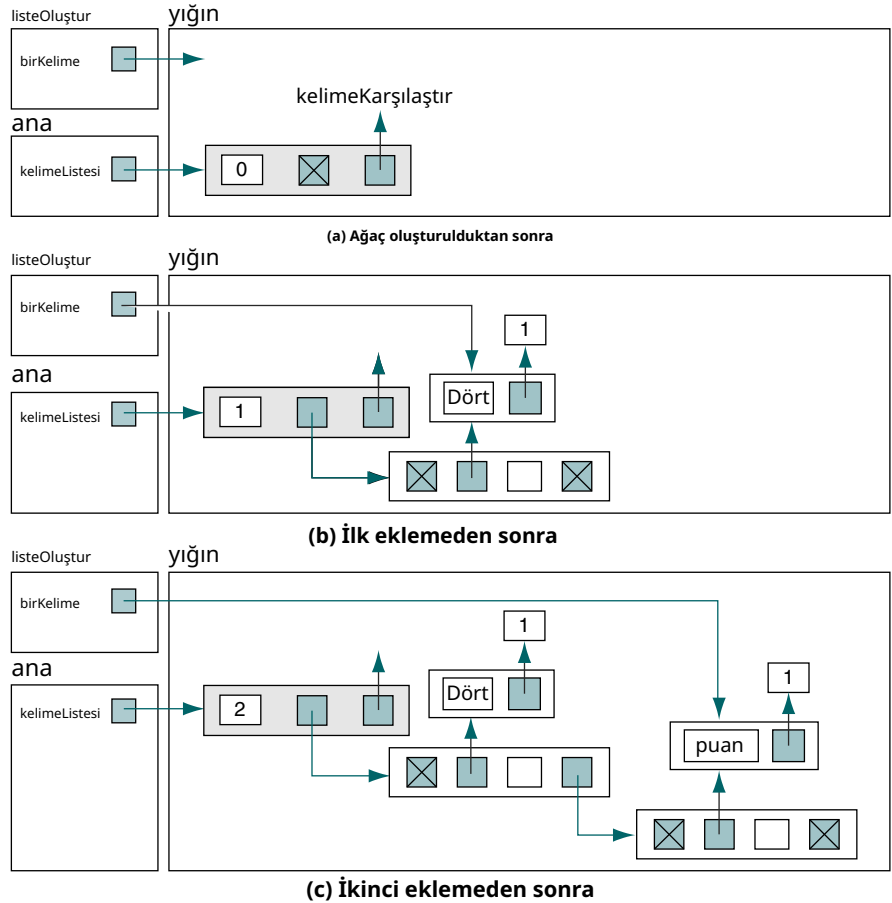
PROGRAM 8-23 Kelime Listesini Yazdır (devam ediyor)

İşlenecek dosyanın adını girin: gtsybrg.txt
AVL Ağacı Sonu

Listede bulunan kelimelerA
ma 1Dört 1Tanr
ı 10 3

...
yıl 1

Kelime listesinin sonu
Kelime sayımını bitir



ŞEKİL 8-16 AVL Ağacına Ekleme

8.5 Anahtar Terimler

AVL ağacı	solun solund
AVL ağacı baş yapısı	asağın solund
AVL ağacı düğüm yapısı	asolun sağınd
yükseklik dengeli ağaçlar	asağın sağında

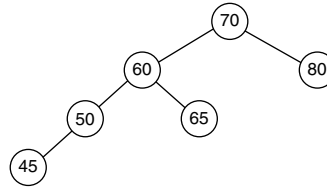
8.6 Özet

- ▣ AVL ağacı, alt ağaçların yüksekliklerinin 1'den fazla farklılık göstermediği bir arama ağacıdır; bu da ağacın dengeli olduğu anlamına gelir.
- ▣ Bir ağacı silme veya ekleme işleminden sonra yeniden dengelemek istediğimizde dört farklı durumu dikkate alırız: solun solunda, sağın sağında, solun sağında ve sağın solunda.
- ▣ Solun solunda, dengesiz bir AVL ağacını dengelemek için denge dışı düğümü sağa döndürmemiz gerekir.
- ▣ Sağın sağında, dengesiz bir ağacı dengelemek için denge dışı düğümü sola döndürmemiz gerekir.
- ▣ Solun sağında, dengesiz bir ağacı çift döndürme ile dengelememiz gerekir: önce sol alt ağacı sola ve sonra denge dışı düğümü sağa döndürmeliyiz.
- ▣ Sağın solunda, dengesiz bir ağacı çift döndürme ile dengelememiz gerekir: önce sağ alt ağacı sağa ve sonra denge dışı düğümü sola döndürmeliyiz.
- ▣ AVL ağacına yeni bir düğüm eklemek, bir ikili arama ağacına yeni bir düğüm eklemekle aynıdır; tek fark, ağaçtan geri çıkarken her düğümün dengesini sürekli kontrol etmemiz ve gerekirse yeniden dengelememiz gerektiğidir.
- ▣ AVL ağacından bir düğüm silmek, bir ikili arama ağacından bir düğüm silmekle aynıdır; tek fark, ağaçtan geri çıkarken her düğümün dengesini sürekli kontrol etmemiz ve gerekirse yeniden dengelememiz gerektiğidir.

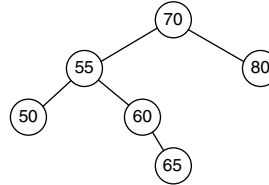
8.7 Pratik SetleriAlı

ştirmalar

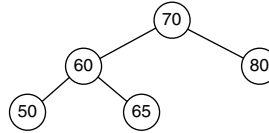
1. Şekil 8-17'deki AVL ağacını dengeleyin. Sonuçta denge faktörlerini gösterin.
2. Şekil 8-18'deki AVL ağacını dengeleyin. Sonuçta denge faktörlerini gösterin.
3. Şekil 8-19'daki AVL ağacına 49 ekleyin. Sonuç bir AVL ağacı olmalıdır. Sonuçtaki ağaçta denge faktörlerini gösterin.
4. Şekil 8-19'daki AVL ağacına 68 ekleyin. Sonuç bir AVL ağacı olmalıdır. Sonuçtaki ağaçta denge faktörlerini gösterin.



ŞEKİL 8-17Alıştırma 1 için Şekil

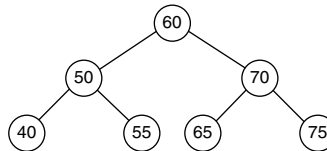


ŞEKİL 8-18Alıştırma 2 için Şekil



ŞEKİL 8-19Alıştırma 3 ve 4 için Şekil

5. Şekil 8-20'deki AVL ağacına 44, 66 ve 77 ekleyin. Sonuç bir AVL ağacı olmalıdır. Ortaya çıkan ağaçtaki denge faktörlerini gösterin.



ŞEKİL 8-20Alıştırma 5 için Şekil

6. Aşağıdaki verileri ardışıl küme olarak girerek bir AVL ağacı oluşturun. Ortaya çıkan ağaçtaki denge faktörlerini gösterin:

14 23 7 10 33 56 80 66 70

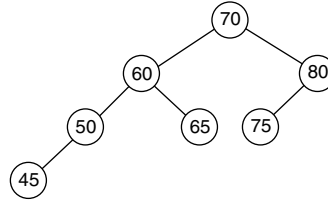
7. Aşağıdaki verileri ardışıl küme olarak girerek bir AVL ağacı oluşturun. Ortaya çıkan ağaçtaki denge faktörlerini gösterin:

7 10 14 23 33 56 66 70 80

8. Aşağıdaki verileri ardışıl küme olarak girerek bir AVL ağacı oluşturun. Ortaya çıkan ağaçtaki denge faktörlerini gösterin:

80 70 66 56 33 23 14 10 7

9. Alıştırma 6'da oluşturulan ağaca 44 ve 50 ekleyin.
 10. Alıştırma 7'de oluşturulan ağaca 44 ve 50 ekleyin.
 11. Alıştırma 8'de oluşturulan ağaca 44 ve 50 ekleyin.
 12. Şekil 8-21'deki AVL ağacından 80 içeren düğümü silin.



ŞEKİL 8-21 Alıştırmalar 12 ve 13 için Şekil

13. Şekil 8-21'deki AVL ağacından 70 içeren düğümü silin.

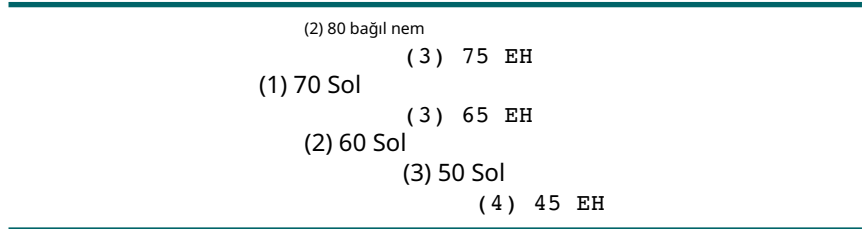
Problemler

14. Algoritma 8-1, "AVL Ağacı Ekle" için bir yinelemeli versiyon yazın.
 15. Algoritma 8-4, "AVL Ağacı Sil" için bir yinelemeli versiyon yazın.

Projeler

16. Problem 14 için C kodunu yazın.
 17. Problem 15 için C kodunu yazın.
 18. Bir metin dosyasından bir isimler ve telefon numaraları listesi okuyun ve bunları bir AVL ağacına ekleyen bir program yazın. Ağaç oluşturulduktan sonra, kullanıcıya belirli bir ismi aramak, yeni bir isim eklemek, mevcut bir ismi silmek veya tüm telefon listesini yazdırmak için bir menü sunun. İşin sonunda, listedeki verileri dosyaya geri yazın. Programınızı en az 10 isimle test edin.

19. AVL ağaç algoritmalarını yazarken, algoritmaların verileri doğru bir şekilde işlediğini doğrulamak için ağacı hiyerarşik bir sırayla yazdırabil memiz gerekiyor. Ağacı yazdırmak için çağrılacak bir yazdırma fonksiyonu yazın. Yazdırılan çıktı, düğüm seviye numarasını parantez içinde, verisini ve denge faktörünü içermelidir. Her boyutta ağacı işleyebilmesi için, veriyi sayfanın solunda kök ile ve sağında soyundan gelenlerle yazdırın. Yan bakıldığında, sol alt ağaçlar sayfanın altında ve sağ alt ağaçlar sayfanın üstünde olmalıdır. Örneğin, Şekil 8-21, Şekil 8-22'de gösterildiği gibi yazdırılacaktır.



ŞEKİL 8-22 Proje 19 için Çıktı

20. AVL ağaç ADT'sinin aramasını, bir düğüm için hemen önceki büyüğünü bulacak şekilde değiştirin. Bulunduktan sonra, bir sonraki düğümü test ederve bulundu veya bulunamadı ve düğümüne bir işaretçi döner (bulunamadıysa null).
21. Bir AVL ağacı için dizi uygulamasını kullanarak ADT'yi oluşturun. Bir dizi uygulamasında, işaretçiler alt ağaç elemanlarına indeks haline gelir. Ağacı oluşturduğunuzda, ağacın içinde saklanacak maksimum düğüm sayısını bilmeniz gerekir. ADT'yi test etmek için, Proje 18'deki programı çalıştırmak için kullanın.
22. Bir belgede kelimeler için bir dizin oluşturun. Her kelime, sayfa numarasıyla belirlenmelidir. Program, her sayfanın sonunda bir form-feed karakteriyle belirtilen sayfaları sayarak sayfa numaralarını takip etmelidir. (*İpucu:* Bölüm 8-4, "Uygulama—Kelimeleri Say.") Programın sonunda dizini yazdırın.
23. Bir dizin genellikle yalnızca anahtar kelimeleri içerir. Bir belgenin dizinini oluşturmak için bir anahtar kelime dosyası kullanan bir program yazın. Program, öncelikle anahtar kelime dosyasını okuyarak kelimeleri bir AVL ağacına ekleyerek başlar. Ardından belgeyi okur ve belgedeki anahtar kelimelerin bir AVL ağaç dizinini oluşturur. Belgenin sonunda anahtar kelime dizinini yazdırın.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 9

Yığınlar

Üçüncü bir ağaç türü yığın (heap)dir. Yığın, sol ve sağ alt ağaçlarının değerleri nin ebeveynlerinden daha az olduğu bir ikili ağaçtır. Bir yığının kökü, ağaçtak i en büyük düğümü tutma garantisine sahiptir; alt ağaçları daha düşük değ erlere sahip verileri içerir. Ancak ikili arama ağacının aksine, bir yığındaki daha düşük değerli düğümler sağ veya sol alt ağaçta yer alabilir. Bu nedenle, ağacı n hem sol hem de sağ dalları aynı özelliklere sahiptir.

Yığınların başka ilginç bir yönü daha vardır: genellikle bağlı liste y erine bir dizide uygulanırlar. Bir yığını bir dizide uyguladığımızda, sol ve sağ alt ağaçların konumunu hesaplayabiliriz. Tersine, bir düğümü n adresi verildiğinde, ebeveyninin adresini hesaplayabiliriz. Bu, çok v erimli bir işlem sağlar.

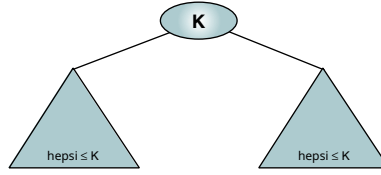
9.1 Temel Kavramlar

Bu bölümde bir yığın ve yığının çalışması için gerekli iki işlemi tanımlı yoruz.

Tanım

Bir yığın, Şekil 9-1'de gösterildiği gibi, aşağıdaki özelliklere sahip bir ikili ağaç yapısıdır:

1. Ağaç tam veya neredeyse tamdır.
2. Her düğümün anahtar değeri, altındaki her düğümün anahtar değ erinden büyük veya ona eşittir.

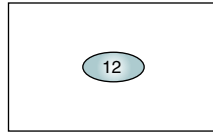


ŞEKİL 9-1Yığın

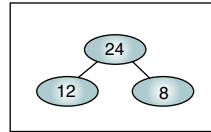
Bazen bu yapı max-yığın olarak adlandırılır. Bir yığının ikinci özelliği—anahtar değeri alt ağaçların anahtar değerlerinden büyüktür—tersine çevrilebilir ve min-yığın oluşturulabilir. Yani, bir düğümdeki anahtar değeri, tüm alt ağaçlarındaki anahtar değerlerinden *daha küçük* olan bir minimum yığın oluşturabiliriz. Genel olarak, *yığın* terimi kendi başına kullanıldığında, bir max-yığına atıfta bulunur.

Bir yığın, bir düğümdeki anahtar değerinin tüm alt ağaçlarındaki anahtar değerlerinden büyük veya onlara eşit olduğu, tamamlanmış veya neredeyse tamamlanmış bir ikili ağaçtır ve alt ağaçlar da yığınlardır.

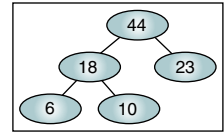
Bir yığının yapısını daha iyi anlamak için, Şekil 9-2'deki yığınları inceleyelim. İki ve üç seviyeli yığınları inceleyin. İki kardeşin sol düğümünün sağ düğümünden daha büyük veya daha küçük olabileceğini unutmayın. Bu sıralamayı daha önce incelediğimiz ikili arama ağacıyla karşılaştırın. Açıkça farklıdır.¹ Son olarak, üç seviyeli yığında, üçüncü seviyenin soldan doldurulduğunu not edin. Bu, neredeyse tam bir ağacın tanımıdır ve bir yığın için bir gerekliliktir.



(a) Sadece kök yığını



(b) İki seviyeli yığın

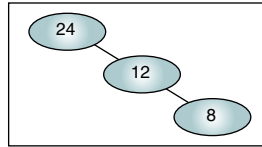


(c) Üç seviyeli yığın

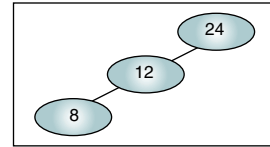
ŞEKİL 9-2Yığın Ağaçları

Bir yığın anlayışımızı tamamlamak için, yığın olmayan bazı yapıları bakalım. Şekil 9-3, yığın olmayan dört yapı örneği göstermektedir. İlk iki yapı, tam veya neredeyse tam ağaçlar olmadıkları için yığın değildir. Üçüncü ve dördüncü örnekler neredeyse tam olsalar da, düğümlerin anahtarları her zaman onların soyundan gelenlerin anahtarlarından büyük değildir.

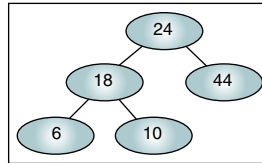
1. Diğer ilginç bir fark, eşit düğüm içermeyen tamamlanmış bir ikili arama ağacında kökün her zaman anahtarların medyan değerini içermesi, oysa yığında en büyük değeri içermesidir.



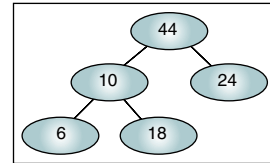
(a) Neredeyse tam değil
(kural 1)



(b) Neredeyse tam değil
(kural 1)



(c) Kök en büyük değil
(kural 2)



(d) Alt ağaç 10 bir yığın değil
(kural 2)

ŞEKİL 9-3 Geçersiz Yığınlar

Bakım İşlemleri

Bir yığın üzerinde iki temel bakım işlemi gerçekleştirilir: bir düğüm eklemekve bir düğüm silmek. Yığın yapısı bir ağaç olmasına rağmen, onu gezmek, aramak veya y azdırmak anlamsızdır. Bu bakımdan, kısıtlı bir veri yapısına çok benzer. Ekleme ve silme işlemlerini gerçekleştirmek için, iki temel algoritmaya ihtiyacımız var: yukarı yeniden yığınlama ve aşağı yeniden yığınlama. Diğer tüm yığın algoritmaları bu iki algoritmaya dayanır. Bu nedenle, önce bunları inceliyoruz.

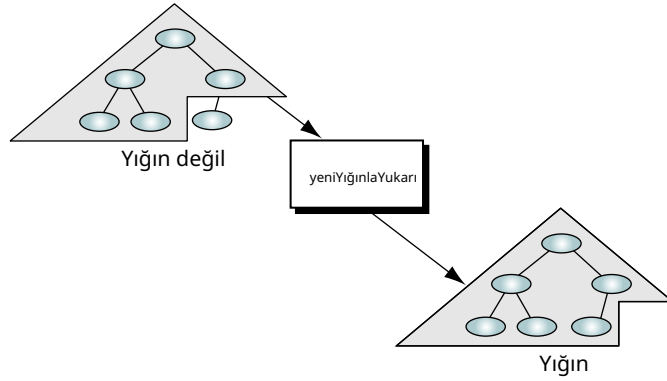
Yukarı Yeniden Yığınlama

İlk $N - 1$ elemanının yığınların sıralama özelliğini sağladığı, ancak son elemanın sağlamadığı neredeyse tamamlanmış bir ikili ağaç olduğunu hayal edin. Başka bir deyişle, son eleman orada olmasaydı yapı bir yığın olurdu. yukarı yeniden yığınlama işlemi, son elemanı ağacın içinde doğru konumuna gelene kadar yukarı doğru yüzdürerek yapıyı yığın haline getirir. Bu yeniden yapılandırmayı Şekil 9-4'te grafiksel olarak gösteriyoruz.

Yeniden yığınlama işleminden önce, yığındaki son düğüm düzensizdi. Yeniden yığınlama işleminden sonra, doğru konumuna yerleşti ve yığın bir düğümle genişletildi.

İkili arama ağacında olduğu gibi, yığınlara eklemeler bir yaprakta gerçekleşir. Ayrıca, yığın tam veya neredeyse tam bir ağaç olduğundan, düğüm son yaprak seviyesinde ilk boş konuma yerleştirilmelidir. Bu, Şekil 9-4'te gördüğümüz durumu yaratır. Yeni düğümün anahtar değeri, ebeveyninin anahtar değerinden büyükse, çocuk ve ebeveyn anahtarları ile verileri değiştirerek ağacın yukarısına doğru yüzdürülür. Veriler, çocuk/ebeveyn anahtarları ve verileri tekrar tekrar değiştirerek sonunda yığındaki doğru yere yükselir.

Yeniden yığınlama işlemi, "bozuk" bir yığını, son ögeyi ağacın yukarısına doğru yüzdürerek yığındaki doğru konumuna gelene kadar yeniden düzenler.



ŞEKİL 9-4Yeniden Yığınlama İşlemi

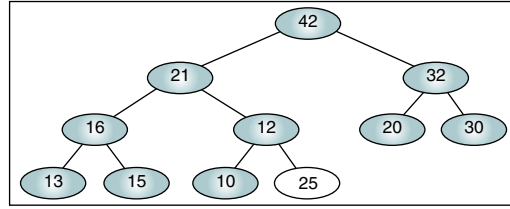
Şekil 9-5, bir yığında yeniden yığınlama işlemini izler. Başlangıçta 25'in ebeveyninin anahtar değeri 12'den büyük olduğunu gözlemliyoruz. 25, 12'den büyük olduğundan, bir yığının tanımından da bildiğimiz gibi, ebeveyninin sol alt ağaç anahtarlarından da büyüktür. Bu nedenle 25 ve 12'yi değiştiriyoruz ve yığındaki mevcut konumunu test etmek için yeniden yığınlama çağrısı yapıyoruz. Yine, 25 ebeveyninin anahtar değeri 21'den büyüktür. Bu nedenle, düğümlerin verilerini tekrar değiştiriyoruz. Bu sefer, yeniden yığınlama çağrıldığında, mevcut düğümün anahtar değerinin ebeveyn anahtarının değerinden küçük olduğunu görüyoruz, bu da doğru konumu bulduğumuzu gösteriyor ve işlem duruyor.

Yeniden Yığınla Aşağı

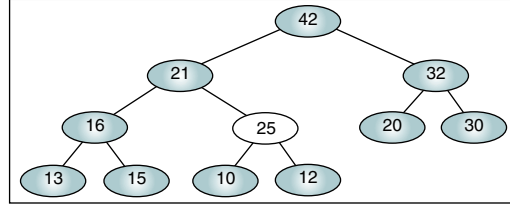
Şimdi ters durumu inceleyelim. Yığın sıralama özelliğini sağlayan, neredeyse tamamlanmış bir ikili ağacımız olduğunu hayal edin, ancak kök pozisyonunda değil. Bu durum, kök ağacın silinmesiyle ortaya çıkar ve iki ayrı yığın bırakır. Durumu düzeltmek için, son ağaç düğümündeki veriyi köke taşırız. A çıkça, bu eylem ağacın yığın özelliklerini yok eder.

Yeniden yığınla aşağı, kökü ağacın doğru pozisyonuna gelene kadar aşağı iterek "bozulmuş" bir yığını yeniden sıralar.

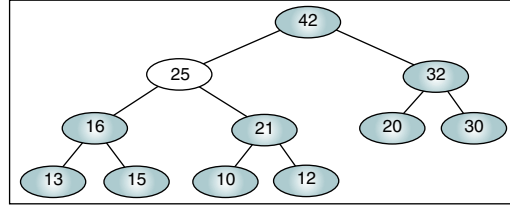
Yığın özelliğini geri kazanmak için, kökü aşağıya doğru iten bir işleme ihtiyacımız var, böylece yığın sıralama özelliğinin sağlandığı bir pozisyona gelir. Bu işleme yeniden yığınla aşağı denir. Yeniden yığınla aşağı işlemi Şekil 9-6'da gösterilmiştir.



(a) Orijinal ağaç: yığın değil

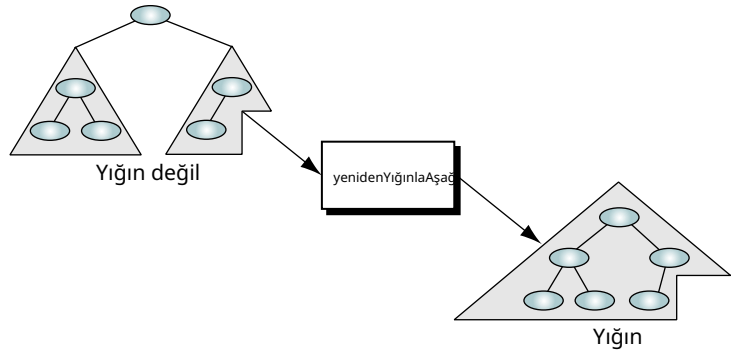


(b) Son eleman (25) yukarı taşındı



(c) Tekrar yukarı taşındı: ağaç bir yığın

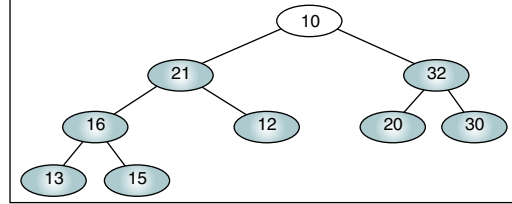
ŞEKİL 9-5Yeniden Yığınlama Yukarı Örneği



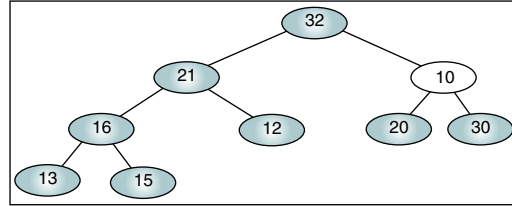
ŞEKİL 9-6Yeniden Yığınlama Aşağı İşlemi

Şekil 9-7, yeniden yığınlama aşağı işleminin bir örneğidir. Başladığımızda, kök (10) alt ağaçlarından daha küçüktür. Onları inceliyoruz ve kök ile değiştirmek için iki sinden daha büyüğünü seçiyoruz, bu durumda 32. Bu işlemi yaptıktan sonra

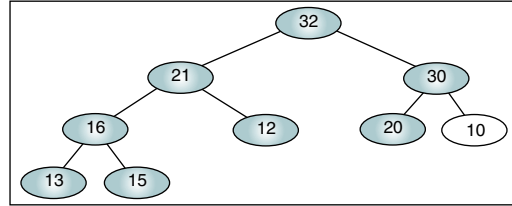
Şekil 9-7(b)'deki değişim sırasında, alt ağaçları kontrol ediyoruz ve 10'un anahtarları arından daha küçük olduğunu görüyoruz. Bir kez daha 10'u alt ağaçların daha büyüğü olan 30 ile değiştiriyoruz. Bu noktada bir yaprağa ulaştığımız ve işlemimiz bitmiştir.



(a) Orijinal ağaç: bir yığın değil



(b) Kök aşağıya hareket etti (sağa)



(c) Tekrar aşağıya hareket etti: ağaç bir yığın

ŞEKİL 9-7Yeniden Yığın Aşağı Örneği

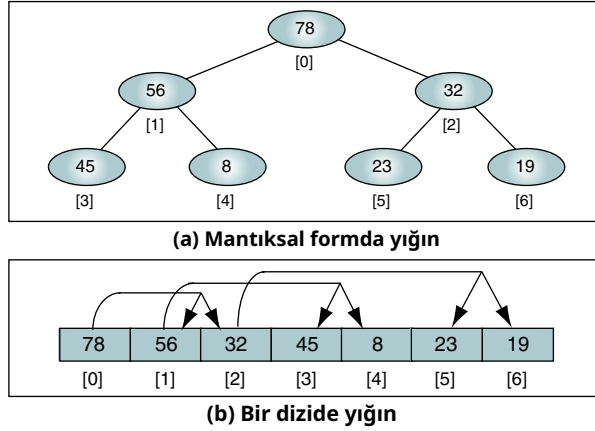
9.2 Yığın Uygulaması

Bir yığın dinamik bir ağaç yapısında inşa edilebilirken, en sık bir dizide uygulanır. Bu uygulama, yığının tanım gereği tam veya neredeyse tam olmasından dolayı mümkündür. Bu nedenle, bir düğüm ile çocukları arasındaki ilişki sabittir ve aşağıda gösterildiği gibi hesaplanabilir.

1. Bir i indeksinde bulunan düğüm için, çocukları şunlardır:
 - a. Sol çocuk: $2i + 1$
 - b. Sağ çocuk: $2i + 2$
2. i indeksinde bulunan bir düğümün ebeveyni $\lfloor (i - 1) / 2 \rfloor$ konumundadır.
3. Sol çocuk için verilen indeks, j , sağ kardeşi, varsa, $j + 1$ 'de bulunur. Tersine, sağ çocuk için verilen indeks, k , sol kardeşi, var olması gereken $k - 1$ 'de bulunur.

4. Tam bir yığının boyutu n olduğunda, ilk yaprağın yeri $\lfloor (n / 2) \rfloor$ 'dir. İlk yaprak elemanının konumuna göre, son yapraksız elemanın konumu bir eksiktir.

Bu ilişkileri göstermek için, Şekil 9-8'i inceleyelim.



ŞEKİL 9-8 Dizilerde Yığınlar

Şekil 9-8'de aşağıdaki ilişkileri görüyoruz:

- 32'nin indeksi 2'dir, bu nedenle sol çocuğu 23'ün indeksi $2 / 2 + 1$, yani 5'tir. Sağ çocuğu 19'un indeksi $2 / 2 + 2$, yani 6'dır (İlişki 1).
- 8'in indeksi 4'tür, bu nedenle ebeveyninin indeksi $\lfloor (4 - 1) / 2 \rfloor$, yani 1'dir (İlişki 2).
- İlk örnekte, sol ve sağ çocukların adresini bulduk. Sağ çocuğu bulmak için, sol çocuğun konumunu (5) kullanıp 1 ekleyebildik (İlişki 3).
- Toplam eleman sayısı 7'dir, bu nedenle ilk yaprak elemanının indeksi, 45, $\lfloor (7 / 2) \rfloor$, yani 3'tür (İlişki 4).
- Son yapraksız elemanın konumu, 32, $3 - 1$, yani 2'dir (İlişki 5).

Son olarak, bu yığın ilişkileri C ve dizileri için sıfır tabanlı indeks adreslemesi kullanan diğer diller için benzersizdir. Bir taban bir indeks adreslemesi kullanan diğer diller için biraz değiştirilmesi gerekecektir.

Bir yığın, her düğüm ile çocukları arasında sabit bir ilişkiyi sağlayan, tamamlanmış veya neredeyse tamamlanmış bir ikili ağaç olması gerektiğinden, bir dizide uygulanabilir.

Şekil 9-8, bir yığını ağaç biçiminde ve dizi biçiminde göstermektedir. Bu iki temsili dikkatlice inceleyin. Mantıksal biçimde, her düğümün

veriler, altındaki verilerden büyüktür. Dizi formatında, bir düğüm için her iki ardılın oklarını takip edin ve dizinin mantıksal (ağaç) formatını doğru bir şekilde temsil ettiğini doğrulayın. Şekil 9-8'de ağaç tamamlanmış ve dizi doludur. Bu nedenle, ağaca başka bir düğüm eklemek için ağaca yeni bir seviye eklememiz ve dizinin boyutunu iki katına çıkarmamız gerekecek çünkü fiziksel dizi her zaman tamamlanmış ağaç yapısını temsil etmelidir.

Algoritmalar

Bir yığın oluşturma için iki yolu vardır. Boş bir dizi ile başlayabilir ve elemanları diziyeye birer birer ekleyebiliriz veya, bir yığın olmayan bir veri dizisi verildiğinde, dizideki elemanları bir yığın oluşturacak şekilde yeniden düzenleyebiliriz. İki temel algoritmayı, yukarıya doğru yığınlama ve aşağıya doğru yığınlama, inceledikten sonra her iki yaklaşımı da gözden geçiriyoruz. Ardından, bir yığından veri silme mantığını inceliyoruz.

Yukarıya Doğru Yığınlama

Yukarıya doğru yığınlama, yeni düğümü ağacın yukarısına taşımak için özineleme kullanır. Bu, daha önce bu bölümde tartıştığımız ilişkileri kullanarak ebeveynin adresini belirleyerek başlar. Yeni verinin anahtarının ebeveyninin anahtarından büyük olması durumunda, düğümleri değiştirir ve kendisini özinelemeli olarak çağırır (Şekil 9-5). Temel durum, ya ebeveyn yoksa, yani yığının kökünde ise ya da düğümler uygun yığın sırasındaysa belirlenir. Mantık, Algoritma 9-1'de gösterilmiştir.

ALGORİTMA 9-1 Yukarıya Doğru Yığınlama

```

Algoritma yenidenYığınla (yığın, yeniDüğüm)
    Yığın dizisinde veriyi çocuğun doğru konumuna taşımak suretiyle yığını yeniden kurar.
    Önce yığın, geçersiz bir yığın içeren bir dizidir
        -yeniDüğüm, yığında yeni verinin indeks konumudur
    Sonra yığın yeniden sıralanmıştır
1   eğer (yeniDüğüm kök değilse)
2       yeniDüğüm'ün ebeveynini ayarla
3       eğer (yeniDüğüm anahtarı > ebeveyn anahtarı)
4           yeniDüğüm ve ebeveyni değiştir
5           yenidenYığınla (yığın, ebeveyn)
6       eğer biterse
7   eğer biterse
    yeniden yığınlamayı sonlandır
    
```

Yeniden Yığınlama Aşağıya Doğru

Yeniden yığınlama aşağıya doğru mantığı biraz daha karmaşıktır. Düğümleri yığın içinde aşağıya doğru iterken, mevcut girişin çocuklarından birinin (bir veya her ikisi) daha küçük olup olmadığını belirlememiz gerekir. Eğer öyleyse, daha büyük girişle değiştirmemiz gerekir (bkz. Şekil 9-7).

Mevcut girişin çocuklarından birinin daha küçük olup olmadığını belirlemek için, öncelikle iki alt ağaç anahtarından hangisinin daha büyük olduğunu belirleriz. Bir kez bildiğimizde

hangisi daha büyükse, mevcut düğümü ona karşılaştırırız. Eğer mevcut düğüm büyük düğüm den daha küçükse, onları değiştiririz ve reheapaşağı çağırısını yinelemeli olarak yaparız. Algoritma 9-2'yi incelerken, çoğupseudocode'un alt ağaç anahtarlarından hangisinin daha büyük olduğunu belirlemek için kullanıldığını göreceksiniz.

ALGORİTMA 9-2 Reheap Aşağı

```

Algoritma reheapDown (yığın, kök, son) Kökteki v
eri yığında doğru konumuna taşımak suretiyle yığını yeniden kura
r.
    Ön yığın bir veri dizisidir kök yığını
        n veya alt yığının köküdür son yığın
        da son elemana işaret eden bir indekstir Son yığın geri
        yüklenmiştir Hangi çocuğun daha büyük
        anahtara sahip olduğunu belirleyin
1  eğer (bir sol alt ağaç varsa)
    1  solAnahtar'ı sol alt ağaç anahtarına ayarla
    2  eğer (bir sağ alt ağaç varsa)
        1  sağAnahtar'ı sağ alt ağaç anahtarına ayarla
    3  aksi takdirde
        1  sağAnahtar'ı null anahtara ayarla
    4  eğer biterse
    5  eğer (solAnahtar > sağAnahtar)
        1  büyükAltAğacı sol alt ağaç olarak ayarla
    6  aksi takdirde
        1  largeSubtree'yi sağ alt ağaç olarak ayarla
    7  eğer biterse
        kök > büyük alt ağaç olup olmadığını test et
    8  eğer (kök anahtar < largeSubtree anahtar)
        1  kök ile largeSubtree'yi değiştir
        2  reheapDown (heap, largeSubtree, son)
    9  eğer biterse
2  eğer biterse
    yeniden yığın aşağı

```

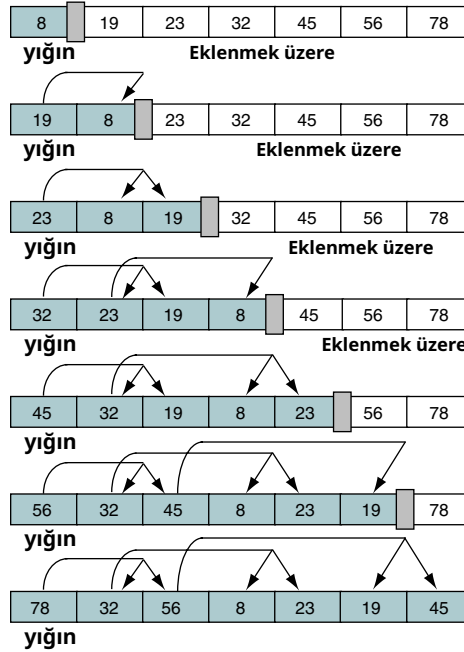
Algoritma 9-2 Analizi. ifadede yalnızca bir alt ağacı test etmemiz gerekiyor çünkü mantıktan şunu biliyoruz: Eğer kök sol alt ağaçtan büyükse ve sol alt ağaç sağ alt ağaçtan büyükse, kök de sağ alt ağaçtan büyüktür. Bu argüman mantıksal olarak "Eğer $A > B$ ve eğer $B > C$ ise, o zaman $A > C$." şeklinde ifade edilir.

Bu algoritmada iki temel durum vardır. İlk olarak, eğer yaprak yoksa, işimiz bitti. Bu durum, 1. ifadede gösterilmiştir (eğer sol bir alt ağaç yoksa, sağ bir alt ağaç olmaz çünkü bir yığın tam bir ağaçtır). İkinci temel durum 1.8. ifadede gösterilmiştir. Eğer kök, daha büyük alt ağaçtan büyükse, geçerli bir yığa sahibiz.

Bir Yığın Oluştur

Doldurulmuş bir dizi eleman verildiğinde, yığını oluşturmak için verileri yeniden düzenlememiz gerekir, böylece yığındaki her düğüm çocuklarından daha büyük olur. Diziyi iki parçaya ayırarak başlarız; soldaki kısım bir yığın ve sağdaki kısım yığına eklenmesi gereken veridir. Başlangıçta kök

(ilk düğüm) heap'teki tek düğümdür ve dizinin geri kalanı eklenmesi gereken veridir. Bu yapı Şekil 9-9'da gösterilmiştir, bu en kötü durum örneğidir —sıralı bir listedir. İlk ve ikinci düğümler arasındaki "duvara" dikkat edin. Şekillerin üstündeki çizgiler düğümlerin çocuklarına işaret eder. Eklemeye algoritmasının her yinelemesi, bir sonraki öğeyi heap'e eklemek için reheap kullanır ve öğeleri ayıran duvarı bir pozisyon sağa kaydırır.



ŞEKİL 9-9 Bir Heap Oluşturma

Heap'e bir düğüm eklemek için, heap boyunca ebeveyn yolunu takip ederiz, düzensiz olan düğümleri değiştiririz. Düğümler düzenli ise, ekleme sonlanır ve bir sonraki düğüm seçilir ve heap'e eklenir. Bu işlem bazen *heapify* olarak adlandırılır.

Yığın oluşturma algoritması çok basittir. Yığını dönüştürmemiz gereken dizi boyunca yürüyerek, ikinci elemandan başlayarak, yığına eklenmesi gereken her dizi elemanı için yeniden yığın oluşturma (reheap up) çağırısı yapıyoruz. Pseudocode, Algoritma 9-3'te gösterilmiştir.

Düğümü Bir Yığın'a Ekle

Yığını oluşturduktan sonra, dizide yer olduğu sürece bir düğüm ekleyebiliriz. Bir düğüm eklemek için, dizideki ilk boş yaprağı bulmamız gerekiyor. Onu, parametre olarak verilen ağaçtaki son düğümün hemen ardından buluyoruz. Bir düğüm eklemek için, yeni veriyi ilk boş yaprağa taşır ve yeniden dengelemeye çağırırız. Bu kavram Şekil 9-10'da gösterilmiştir.

ALGORİTMA 9-3Yığın Oluştur

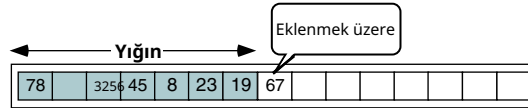
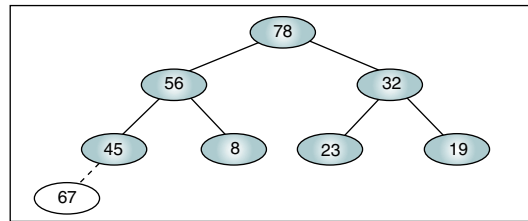
Algoritma buildHeap (yığın, boyut)

Bir dizi verildiğinde, verileri yığın oluşturacak şekilde yeniden düzenleyin.

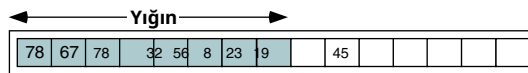
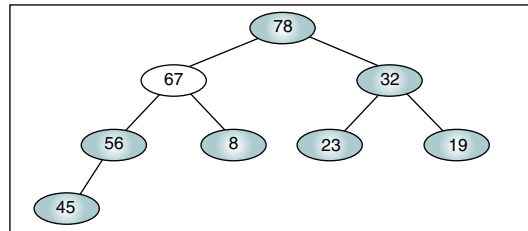
Ön yığın, yığın düzeninde olmayan verileri içeren dizidir
boyut, dizideki eleman sayısıdır

Post dizisi artık bir yığın

- 1 yürüyücüyü 1 olarak ayarla
 - 2 döngü (yürüyücü < boyut)
 - 1 yığınYukarıDöndür(heap, yürüyücü)
 - 2 yürüyücüyü artır
 - 3 döngüyü sonlandır
- son yığınOluştur



(a) Yığın yukarı döndürmeden önce



(b) Yığın yukarı döndürdükten sonra

ŞEKİL 9-10Düğüm Ekle

Yığın ekleme algoritması basittir. Eklenmek istenen veriyi yığın içinde ki ilk yaprağa taşır ve yeniden dengelemeye çağırır.Pseudocode, Algoritma 9-4'te gösterilmiştir.

ALGORİTMA 9-4Yığın Ekle

```

Algoritma insertHeap (yığın, son, veri)
Veriyi yığına ekler.
    Ön yığın geçerli bir yığın yapısıdır
        son yığındaki son düğüme referans parametresidir
        veri eklenmesi gereken veriyi içerir
    Son veri yığına eklenmiştir
    Başarılıysa true; dizi doluysa false döndür
1  eğer (yığın dolu)
    1  yanlış döndür
2  eğer biterse
3  sonu artır
4  verileri son düğüme taşı
5  yığın yukarı döndür (yığın, son)
6  doğru döndür
son yığınEkle

```

Düğümü Bir Yığın'dan Sil

Bir yığından düğüm silerken, en yaygın ve anlamlı mantık kökü silmektir. Aslında, bir yığının mantığı en büyük öğeyi, kökü belirlemek ve çıkar maktır. Silindikten sonra, yığın köksüz kalır. Yığını yeniden kurmak için, son yığın düğümündeki veriyi köke taşı ve aşağı doğru yeniden yığın yaparız. Kavram Şekil 9-11'de gösterilmiştir.

Kökü bir yığından silme mantığı Algoritma 9-5'te gösterilmiştir.

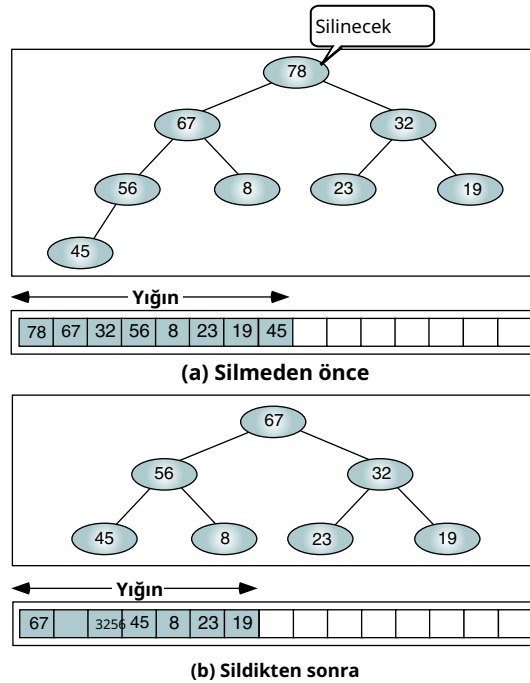
Yığının tepe noktasındaki veriyi işlenmesi için çağıran algoritmaya geri döndürdüğümüz ü unutmayın.

ALGORİTMA 9-5Yığın Düğümünü Sil

```

Algoritma yığınSil (yığın, son, veriDış)
Yığının kökünü siler ve veriyi çağırana geri gönderir.
    Ön yığın geçerli bir yığın yapısıdır
        son yığındaki son düğüme referans parametresidir
        veriDış çıktı alanı için referans parametresidir
    Son kök silindi ve yığın yeniden inşa edildi
        kök verisi veriDış'a yerleştirildi
    Başarılıysa true döndür; dizi boşsa false döndür
1  eğer (yığın boşsa)
    1  yanlış döndür
2  eğer biterse
3  dataOut'u kök verisi olarak ayarla
4  son veriyi köke taşı
5  sonu azalt
6  reheapDown (yığın, 0, son)
7  doğru döndür
son deleteHeap

```



ŞEKİL 9-11 deleteHeap Düşüm

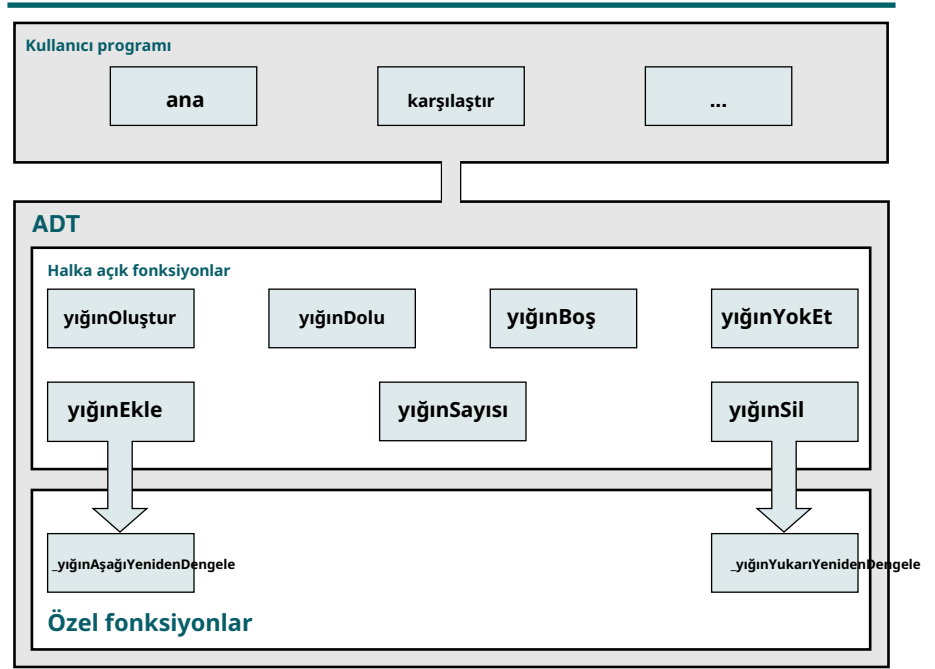
9.3 Yığın ADT

Hiçbir programlama dilinin içsel yığın operatörleri yoktur; bu nedenle, bir yığın kullanmak için yığın işlemlerini yazmalıyız. Bu bölümde bir yığın için soyut bir veri türü oluşturuyoruz.

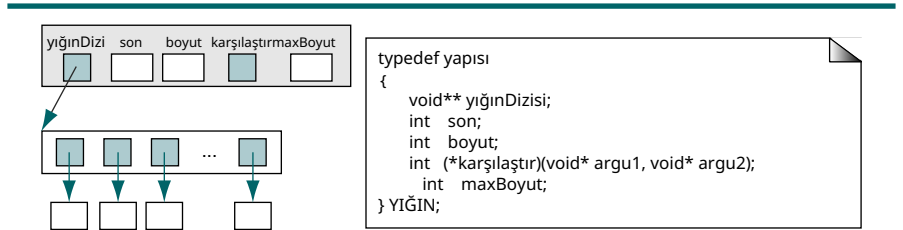
Temel kullanıcı arayüzünü sağlayan dokuz fonksiyon tanımlıyoruz. BST ve AVL ağaçlarında olduğu gibi, ağacı uygulamak için gereken birkaç iç fonksiyon vardır. ADT tasarımı Şekil 9-12'de gösterilmiştir.

Yığın Yapısı

ADT'yi geliştirmeden önce, kullanacağımız veri yapısını belirlememiz gerekiyor. Yukarıda belirttiğimiz gibi, bir dizi kullanıyoruz (bkz. Şekil 9-8). Bu nedenle, ihtiyacımız olan tek şey dizi ve üç parça meta veridir: yığındaki son elemanın indeks konumu, yığının maksimum boyutu ve yığındaki eleman sayısı. Veri yapısı tasarımı Şekil 9-13'te gösterilmiştir.



ŞEKİL 9-12Yığın ADT Tasarımı



ŞEKİL 9-13Yığın ADT Yapısı

Yığın Algoritmaları

Program 9-1, yığın fonksiyonları için veri yapılarını içermektedir.

PROGRAM 9-1Yığın Bildirimi

1234	<pre> /* Yığın ADT için Veri Yapıları Oluşturan: Tarih: */ </pre>
------	--

devam ediyor

PROGRAM 9-1 Yığın Bildirimi (devam ediyor)

56789	#include <stdbool.h>
101112131415161718192021222324252627	<pre> typedef yapısı { void** yığınDizisi; int son; int boyut; int (*karşılaştır)(void* argu1, void* argu2); int maxBoyut; } YIĞIN; // Prototip Tanımları HEAP* yığınOluştur (int maxBoyut, int (*karşılaştır)(void* arg1, void* arg2)); bool yığınEkle (HEAP* yığın, void* veriPtr); bool yığınSil (HEAP* yığın, void** veriDışPtr); int yığınSayısı (HEAP* yığın); bool yığınDolu (HEAP* yığın); bool yığınBoş (HEAP* yığın); void yığınYokEt (HEAP* yığın); static void _yığınYukarıDengele (HEAP* yığın, int çocukKonum); static void _yığınAşağıDengele (HEAP* yığın, int kök); </pre>

Bir Yığın Oluştur

Yığın oluşturma fonksiyonu, yığının maksimum boyutu için bir sayı alır ve yığın yapısının adresini döndürür. Kod, Program 9-2'de gösterilmiştir.

PROGRAM 9-2 Yığın Oluşturma Uygulama Arayüzü

123456789	<pre> /* ===== yığınOluştur ===== Yığın için bellek ayırır ve yığın baş yapısının adresini döndürür. Önce Hiçbir şey Yığın oluşturuldu ve adres döndürüldü bellek taşması durumunda, NULL döndürüldü */ #include <math.h> HEAP* heapCreate (int maxSize, int (*compare)(void* argu1, void* argu2)) { // Yerel Tanımlar HEAP* yığını; </pre>
101112131415	

devam ediyor

PROGRAM 9-2 Yığın Oluşturma Uygulama Arayüzü (devamı)

```

161718192021222324252627282930
// İfadeler
yığın = (HEAP*)malloc(sizeof (HEAP));
if (!heap)
    NULL'u döndür;

heap->last = -1;
heap->compare = compare;

// Yığın boyutunu 2'nin kuvvetine zorla -1
heap->maxSize =
    (int) pow (2, ceil(log2(maxSize)) - 1;
heap->heapAry = (void*)
    calloc(heap->maxSize, sizeof(void*));

return heap;
} // createHeap

```

Bir Yığın Ekle

Yığın ekleme fonksiyonu Algoritma 9-4'ü uygular. Yığına bir giriş ekler veya yığını yeniden dengelemek için yeniden yığın oluşturur. Kodunun Program 9-3'te görüldüğü gibi.

PROGRAM 9-3 Yığın Ekleme Uygulama Arayüzü

```

123456789
/* ===== heapInsert =====
Yığına veri ekler.
Önce Yığın geçerli bir yığın yapısıdır
last son elemanın indeksine işaretçidir
data eklenmesi gereken veridir
Sonra veriler yığına eklenmiştir
Başarılıysa true döner; dizi doluysa false döner
*/
bool heapInsert (HEAP* yığın, void* dataPtr)
{
101112131415161718192021222324
// İfadeler
if (heap->size == 0) // Yığın boş
{
    yığın->boyut = 1;
    yığın->son = 0;
    yığın->heapAry[heap->son] = dataPtr;
    return true;
} // eğer
eğer (heap->last == heap->maxSize - 1)
    return false;
++(yığın->son);
++(yığın->boyut);
heap->heapAry[heap->son] = dataPtr;
_reheapUp (heap, heap->son);

```

devam ediyor

PROGRAM 9-3Yığın Ekleme Uygulama Arayüzü (devamı)

2526	true değerini dö ndür;}// heapInsert
------	---

İçsel Yeniden Yığın Yükseltme Fonksiyonu

Program 9-4, Algoritma 9-1 ile paraleldir.

PROGRAM 9-4İçsel Yeniden Yığın Yükseltme Fonksiyonu

123456789	/* ===== reheapUp ===== Verileri çocuk düğünden yukarı doğru hareket ettirerek yığın doğ ru konumunu yeniden kurar. Önce yığın, geçersiz bir yığın içeren bir dizidir newNode, yığındaki yeni verinin indeksidir Sonra newNode yığına eklenmiştir */
1011121314151617181920212223242526272829303132	void _reheapUp (HEAP* yığını, int childLoc) { // Yerel Tanımlar int parent; void** yığınDizi; void* tutma; // İfadeler // eğer yığın kökünde değilse -- indeks 0 if (childLoc) { heapAry = heap->heapAry; parent = (childLoc - 1)/ 2; if (heap->compare(heapAry[childLoc], heapAry[parent]) > 0) // çocuk ebeveynden büyük -- takas et { hold = heapAry[ana]; heapAry[parent] = heapAry[childLoc]; heapAry[childLoc] = hold; _reheapUp (heap, parent); } // eğer heap[] } // eğer newNode return; } // yeniden yığın

Bir Yığını Sil

Yığın silme fonksiyonu, Algoritma 9-5'i uygular. Yığının tepe noktasındaki öğeyi sil
erve çağırana geri döner. Ardından, yığın yapısını yeniden kurmak için yeniden yığ
ın aşağı çağrılır.Kod Program 9-5'te gösterilmiştir.

PROGRAM 9-5 Yığın Silme Uygulama Arayüzü

123456789	<pre> /* ===== heapDelete ===== Kümenin kökünü siler ve veriyi çağırana geri gönderir. Ön heap geçerli bir heap yapısıdır last heap'teki son düğüme referanstır dataOut çıktı alanına referanstır Sonra last silinir ve heap yeniden inşa edilir silinen veri kullanıcıya geri gönderilir Başarılıysa true; dizi boşsa false döner */ bool heapDelete (HEAP* yığın, void** dataOutPtr) { // İfadeler if (yığın->boyut == 0) // yığın boş yanlış döndür; *dataOutPtr = heap->heapAry[0]; heap->heapAry[0] = heap->heapAry[heap->son]; (heap->last)--; (heap->size)--; _reheapDown (heap, 0); return true; } // heapDelete </pre>
10111213141516171819202122	

İçsel Reheap Aşağı Fonksiyonu

Reheap down algoritması oldukça uzun olmasına rağmen, pseudocode'ayakında n uymaktadır. Ana fark, swap fonksiyonunu çağırmak yerine, swap'ı inline olarak kodlamayı seçmemizdir. Algoritma 9-2 ile paralel olan mantık, Program 9-6'da gösterilmiştir.

PROGRAM 9-6 İçsel Reheap Down Fonksiyonu

123456789	<pre> /* ===== reheapDown ===== Yığın kökündeki veriyi aşağı doğru hareket ettirerek yığını yeniden kurar. Önce yığın veri dizisidir kök yığının veya alt yığının köküdür son yığındaki son elemanın indeksidir Sonra yığın geri yüklenmiştir */ void _reheapDown (HEAP* yığın, int kök) { // Yerel Tanımlar void* hold; void* leftData; void* rightData; int largeLoc; </pre>
101112131415	

devam ediyor

PROGRAM 9-6 İç Yığın Aşağı Fonksiyonu (devam ediyor)

```

16171819202122232425262728293031323334353637383940414243444546474849505152
    int son;

    // İfadeler
    son = yığın->son;
    if ((kök * 2 + 1) <= son)                                     // sol alt ağaç // En az bir çocuk v
    {
        ar
        {
            leftData = heap->heapAry[root * 2 + 1];
            if ((root * 2 + 2) <= last)                          // sağ alt ağaç
                rightData = heap->heapAry[root * 2 + 2];
        }
        yoksa
            sağVeri = NULL;

        // Hangi çocuğun daha büyük olduğunu belirle
        eğer (!sağVeri)
        {
            || heap->karşılaştır (solVeri, sağVeri) > 0)
            {
                büyükKonum = kök * 2 + 1;    } // eğer
                er sağ anahtar yoksa veya solAnahtar daha büyükse   yoksa

                {
                    büyükKonum = kök * 2 + 2;    } //
                    aksi takdirde
                // kök > daha büyük alt ağaç mı
                eğer (heap->karşılaştır (heap->heapAry[kök],
                    heap->heapAry[büyükKonum]) < 0)
                {
                    // ebeveyn < çocuklar
                    tut = heap->heapAry[kök];
                    heap->heapAry[kök] =
                        heap->heapAry[largeLoc];
                    heap->heapAry[largeLoc] = tut;
                    _reheapDown (heap, largeLoc);
                } // eğer kök <

            } // eğer kök
            return;
        } // reheapDown

```

9.4 Yığın Uygulamaları

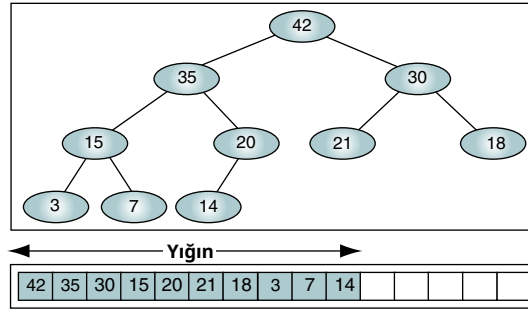
Yığınların üç yaygın uygulaması seçim algoritmaları, öncelik kuyrukları ve sıralamadır. Yığın sıralamasını Bölüm 12'de ve seçim algoritmalarını ve öncelik kuyruklarını burada tartışıyoruz.

Seçim Algoritmaları

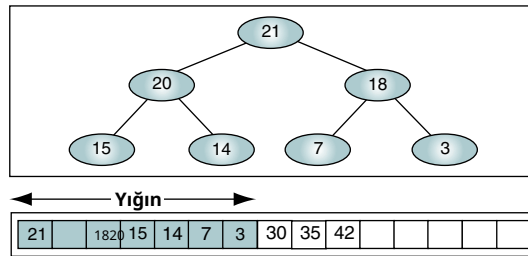
Bir sıralanmamış listede k . elemanı belirleme problemi için iki çözüm vardır. Öncelikle listeyi sıralayıp k . konumdaki elemanı seçebiliriz, ya da bir yığın oluşturup ondan $k - 1$ elemanını silerek istenen elemanı kök olarak bırakabiliriz. k . elemanı seçmek kolaydır; yığının kökünde bulunur. Yığınları incelediğimiz için, ikinci çözümü, bir yığın kullanarak inceleyelim.

Yığın oluşturmak oldukça basittir. Yığın oluşturma algoritmalarını zaten tartıştık. Ancak bir yığın oluşturduktan sonra, istenen öğeyi nasıl seçeceğiz? Yığının üstündeki öğeleri basitçe atmak yerine, silinen öğeyi yığının sonuna yerleştirmek ve yığın boyutunu 1 azaltmak daha iyi bir çözümdür. k ıncı öğe işlendiğinde, geçici olarak çıkarılan öğeler tekrar yığına eklenebilir.

Örneğin, bir listede dördüncü en büyük öğeyi bilmek istiyorsak, Şekil 9-14'te gösterilen yığını oluşturabiliriz. Üç kez silme işlemi yaptıktan sonra, yığının en üstünde dördüncü en büyük öğe olan 21'i elde ederiz. 21'i seçtikten sonra, yığını yeniden dengelemek için yığını restore ederiz, böylece yığın tamamlanır ve başka bir seçim için hazır oluruz.



(a) Orijinal yığın



(b) Üç silme işleminden sonra

ŞEKİL 9-14 Yığın Seçimi

Yığın seçim mantığı Algoritma 9-6'da gösterilmiştir.

ALGORİTMA 9-6Yığın Seçimi

```

Algoritma selectK (yığın, k, yığınSon)
Bir listeden k-ıncı en büyük öğeyi seç
    Ön   yığın, bir yığın uygulamasının dizisidir
        k, istenen öğenin sıralamasıdır
        yığınSon, son öğeye referans parametresidir
    Son   k-ıncı en büyük değer döndürülür
1  eğer (k > yığın boyutu)
    1  yanlış döndür
2  eğer biterse
3  origHeapSize'ı yığınSon + 1 olarak ayarla
4  döngü (k kez)
    1  tempData'yı kök verisi olarak ayarla
    2  deleteHeap (yığın, yığınSon, dataOut) çağırısı yap
    3  tempData'yı heapLast + 1'e taşı
5  döngüyü sonlandır
İstenilen eleman artık heap'in en üstünde
6  root verisini holdOut'a taşıHeap'i
yeniden yapılandır
7  döngü (heapLast < origHeapSize iken)
    1  heapLast'ı artır
    2  reheapUp (yığın, yığınSon)
8  döngüyü sonlandır
9  holdOut'u döndür
son seçK

```

Öncelik Kuyrukları

Bölüm 4'te incelediğimiz kuyruk yapısı, her düğümün kuyrukta sıralı olarak ilerlediği bir linear liste dayanır. Hiçbir öğenin diğerlerinden daha hızlı ilerlemesi mümkün değildir. Bu sistem oldukça adil olabilir, ancak genellikle çok gereksizdir. Çeşitli nedenlerden dolayı, bir öğeyi diğerlerine göre öncelikli endirmek isteriz. Oldukça basit bir örnek olarak, havaalanındaki birinci sınıf hattını düşünün. Çoğu insan uzun bir sırada beklerken, eğer birinci sınıf biletleriniz varsa veya havayolunun yönetici kulübüne ait iseniz, sıra çok kısa, hatta boş olabilir.

Tasarı

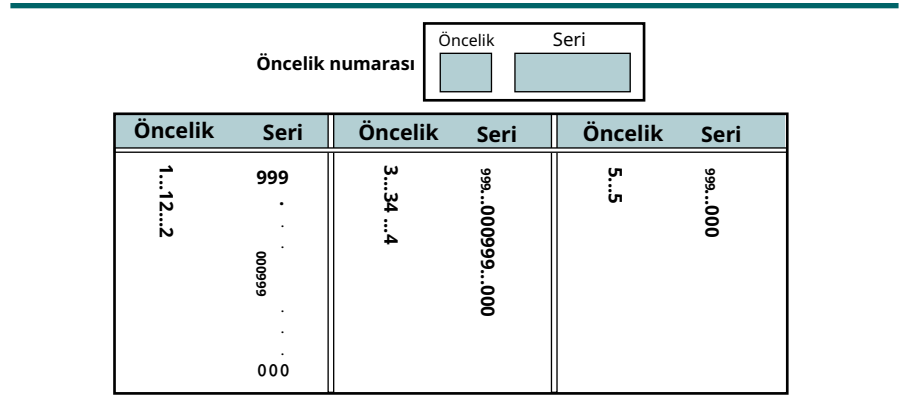
Yığın, bir öncelik kuyruğu için mükemmel bir yapıdır. Bir olay kuyruğuna girdiğinde, diğer olaylara göre konumunu belirleyen bir öncelik numarası atanır. Yeni olay, yalnızca bir yerde, ilk boş yaprakta yığına girebilse de, yine de bir öncelik numarası atanır. Ancak sıraya girdikten sonra, yeni olay hızla yığındaki diğer tüm olaylara göre doğru konuma yükselir. En yüksek önceliğe sahipse, yığının en üstüne çıkar ve işlenecek bir sonraki olay haline gelir.

Düşük bir önceliğe sahipse, yığında nispeten düşük kalır ve sırasını bekler.

Bir öncelik kuyruğundaki anahtar dikkatlice oluşturulmalıdır, böylece kuyruk düzgün çalışır. Yaygın bir teknik, kodlanmış bir öncelik kullanmaktır.

önceliği ve olayın kuyruktaki yerini temsil eden ardışık bir numaradan oluşan sayı. Örneğin, beş öncelik sınıfına sahip bir kuyruk verildiğinde, öncelik numarasının ilk basamağının kuyruk önceliğini, 1'den 5'e kadar temsil ettiği ve sayının geri kalanının öncelik içindeki seri yerleşimi temsil ettiği bir anahtar oluşturabiliriz. Ancak, bir öncelik heap'i kullandığımız için, seri numara azalan sırada olmalıdır; yani, her öncelik içinde 999'dan 0'a kadar.

Eğer herhangi bir öncelik için aynı anda maksimum 1000 olay olacağını varsayıyorsak, en düşük önceliği 1999'dan 1000'e kadar olan ardışık numaralara, ikinci en düşük önceliği 2999'dan 2000'e kadar olan ardışık numaralara, üçüncü en düşük önceliği 3999'dan 3000'e kadar olan numaralara ve bu şekilde devam eden bir yapıyı atayabiliriz. Bu kavram Şekil 9-15'te gösterilmektedir.



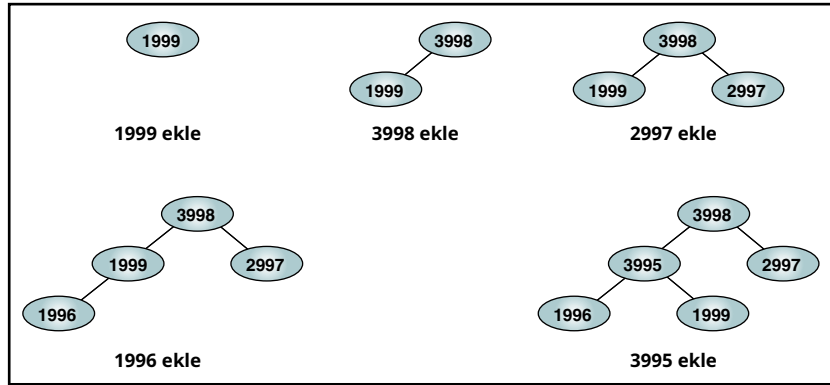
ŞEKİL 9-15 Öncelik Kuyruğu Öncelik Numaraları

Örnek Üç önceliği olan bir öncelik kuyruğuna sahip olduğumuzu varsayalım: yüksek (3), orta (2) ve düşük (1). Gelen ilk beş müşteriden ikincisi ve beşinci yüksek öncelikli müşterilerdir, üçüncüsü orta önceliklidir ve birinci ile dördüncü düşük önceliklidir. Onlara Tablo 9-1'de gösterildiği gibi öncelik numaraları atanmıştır.

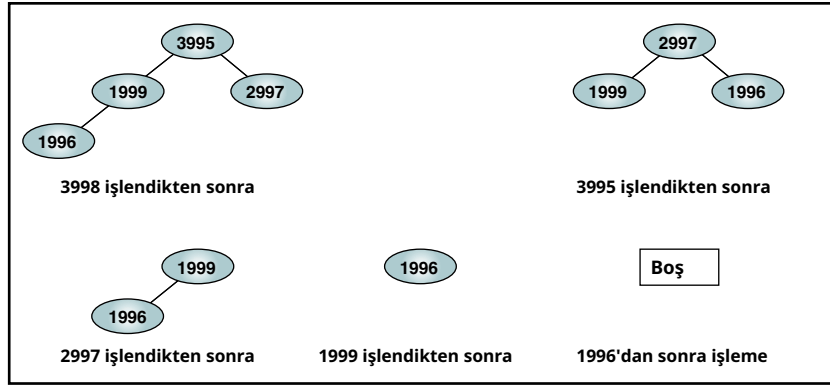
Geliş	Öncelik	Öncelik
1	düşük	1999 (1 & (1000 - 1))
2	yüks	3998 (3 & (1000 - 2))
3	ekortadü	2997 (2 & (1000 - 3))
4	şük	1996 (1 & (1000 - 4))
5	yüksek	3995 (3 & (1000 - 5))

TABLO 9-1 Öncelik Numarası Atamaları

Şekil 9-16, öncelik kuyruğunun nasıl oluşturulduğunu ve işlendiğini izlemektedir, tüm müşterilerin ilk müşteriye hizmet verilmeden önce geldiğini varsayarak. Şekli incelerken, müşterilerin önceliklerine göre ve eşit öncelikler içinde, geliş sırasına göre hizmet aldığını not edin. Böylece müşteri 2 (3998) ilk olarak hizmet alırken, ardından müşteri 5 (3995), müşteri 3 (2997), müşteri 1 (1999) ve müşteri 4 (1996) gelmektedir.



(a) Müşterileri ekleme

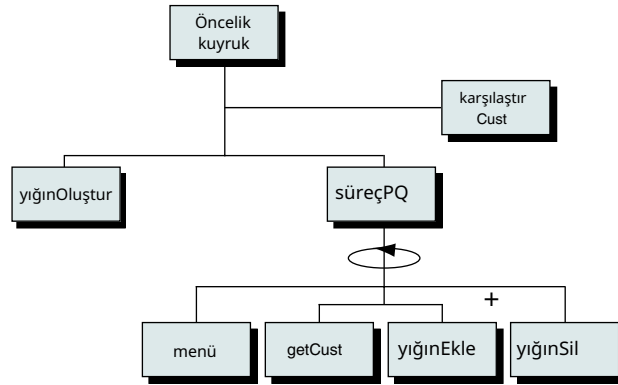


(b) Müşterileri işleme

ŞEKİL 9-16 Öncelik Kuyruğu Örneği

Uygulama

Öncelik kuyruğunun tasarımı Şekil 9-17'de gösterilmiştir.



ŞEKİL 9-17Öncelik Kuyruğu Tasarımı

Tasarıma göre, baş ADT'sinden üç fonksiyon çağırıyoruz: heapoluştur, heap ekle ve heap sil. Öncelik kuyruğunu oluşturduktan sonra, bir menü fonksiyonu kullanarak bir sonraki eylemi belirliyoruz ve ardından ya öncelik kuyruğuna yeni bir müşteri ekliyoruz ya da kuyruğun bir sonraki müşterisini siliyoruz. Bir müşteri silindiğinde, müşterilerin önceliklerine göre ilk giren ilk çıkar prensibine göre işlendiğini doğrulamak için müşteri kimliğini gösteriyoruz. Kod Program 9-7'de gösterilmiştir.

PROGRAM 9-7Öncelik Kuyruğu Uygulaması

```

123456789  /* Yığın kullanarak öncelik kuyruğunu uygulayın.
              Yazarı:
              Tarih:
              */
              #include <stdio.h>
              #include <ctype.h>
              #include <stdbool.h>

              #include "P9-heap.h"

              // Sabit Tanımları
              const int maxQueue = 20;

              // Yapı Tanımları
              typedef struct
              {
                  int id;
                  int öncelik;
                  int seri;
              } CUST;
  
```

devam ediyor

PROGRAM 9-7Öncelik Kuyruğu Uygulaması (devamı)

```

21
22 // Prototip Bildirimleri
23 int compareCust (void* cust1, void* cust2);
24 void processPQ (HEAP* heap);
25 karakter menü (void);
26 MÜŞTERİ* getCust (void);
27
28 int main (void)
29 {
30 // Yerel Tanımlar
31 Yığın* prQueue;
32
33 // İfadeler
34 printf("Öncelik Kuyruğu Gösterimi Başlıyor");prQueue = hea
35
36 pQueue = maxHeap;compareCust);processPQ(prQueue);
37 processPQ (prQueue);
38
39 printf("Öncelik Kuyruğu Gösterimi Bitiyor");return 0;
40 return 0;
41 }
42
43 /* ===== karşılaştırmak =====İki müş
44 erinin önceliğini karşılaştırarakhangisinin daha yüksek önceliğ
45 e sahip olduğunu belirleyin.
46 Önce İki müşteri yapısı verildiğinde
47 Sonra eğer cust1 > cust2 +1 döndür
48 eğer cust1 == cust2 0 döndür
49 eğer cust1 < cust2 -1 döndür
50 */
51 int compareCust (void* cust1, void* cust2)
52 {
53 // Yerel Tanımlar
54 MÜŞTERİ c1;
55 CUST c2;
56
57 // İfadeler
58 c1 = *(CUST*)cust1;
59 c2 = *(CUST*)cust2;
60
61 if (c1.serial < c2.serial)
62 return -1;
63 else if (c1.serial == c2.serial)
64 return 0;
65 return +1;
66 } // compareCust
67

```

devam ediyor

PROGRAM 9-7Öncelik Kuyruğu Uygulaması (devamı)

```

68  /* ===== processPQ =====
69  İki müşterinin önceliğini karşılaştır
70  kimin daha yüksek önceliği olduğunu belirlemek için.
71  Önce İki müşteri yapısı verildiğinde
72  Sonra eğer cust1 > cust2 +1 döndür
73  eğer cust1 == cust2 0 döndür
74  eğer cust1 < cust2 -1 döndür
75  */
76  void prosesPQ (HEAP* prQueue)
77  {
78  // Yerel Tanımlar
79  CUST* cust;
80  bool result;
81  char option;
82  int numCusts = 0;
83
84  // İfadeler
85  Yapmak
86  {
87  seçenek = menü (); switch (seçenek)
88
89  {
90  durum 'e':
91  cust = getCust ();
92  numCusts++;
93  müşteri->seri =
94  cust->priority * 1000 + (1000 - numCusts);
95  result = heapInsert (prQueue, cust);
96  eğer (!result)
97  printf("Hata: yığın içine eklenirken hata oluştu",
98  exit(101));
99  break;
100  durum 'd':
101  result = heapDelete (prQueue, (void**)&cust);
102  eğer (!result)
103  printf("Hata: müşteri bulunamadı"); aksi takdi
104  rde
105  {
106  printf("Müşteri %4d silindi", c
107  cust->id);
108  numCusts--; }
109  // aksi takdirde
110  } // anahtar
111  } while (seçenek != 'q');
112  return;
113  }// süreçPQ
114

```

devam ediyor

PROGRAM 9-7Öncelik Kuyruğu Uygulaması (devamı)

```

115  /* ===== menü =====
116      Menüü görüntüle ve eylemi al.
117      Önce hiçbir şey
118      Sonra eylem okundu ve doğrulandı
119  */
120  karakter menüsü (geçersiz)
121  {
122  // Yerel Bildirimler
123      char option;
124      bool geçerli;
125
126  // İfadeler
127      printf( "===== Menü =====");printf( " e :
128      Müşteri Uçuşu Girişi  " );printf( " d :  Müşteri Uçuşunu Sil  " );
129      printf( " q :  Çıkış.          " );printf( "=====
130      =====");printf( "Lütfen seçiminizi girin:  " );
131
132
133
134      Yapmak
135      {
136          scanf(" %c", &seçenek);
137          seçenek = tolower (seçenek);
138          switch (seçenek)
139          {
140              durum 'e':
141              'd' durumu:
142                  case 'q': geçerli = true;
143                      break;
144
145                  default: printf("Geçersiz seçim. Yeniden Girin: ");
146                      geçerli = false;
147                      kırmak;
148                      } // switch
149          } while (!geçerli);
150      return seçenek;
151  }// menü
152
153  /* ===== getCust =====
154      Klavye üzerinden müşteri verilerini okur.
155      Önce hiçbir şey
156      Post verileri okundu ve yapı içinde döndürüldü
157  */
158  CUST* getCust (geçersiz)
159  {
160  // Yerel Tanımlar
161      CUST* cust;

```

devam ediyor

PROGRAM 9-7Öncelik Kuyruğu Uygulaması (devamı)

```
162
163 // İfadeler
164     cust = (CUST*)malloc(sizeof (CUST));
165     if (!cust)
166         printf("getCust içinde bellek taşması"),    exit (20
167             0);
168
169     printf("Müşteri kimliğini girin:  ");
170     scanf ("%d", &cust->id);
171     printf("Müşteri önceliğini girin: ");
172     scanf ("%d", &cust->priority);
173     müşteri döndür;
174 } // getCust
```

9.5 Anahtar Terimler

yığın

yığınlaştır

maksimum-yığın

minimum-yığın

öncelik kuyruğuyığın

yeniden aşağıyığın y

eniden yukarı

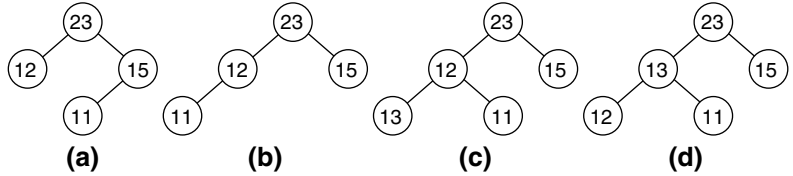
9.6 Özet

- ▣ Bir yığın, her düğümün anahtar değerinin, altındaki her düğümün anahtar değerinden büyük veya ona eşit olduğu, tamamlanmış veya neredeyse tamamlanmış bir ikili ağaç yapısıdır.
- ▣ Bir yığın, bir düğümdeki anahtar değerinin tüm alt ağaçlarındaki anahtar değerlerinden büyük veya eşit olduğu ve her alt ağacın kendisinin bir yığın olduğu tamamlanmış veya neredeyse tamamlanmış bir ikili ağaç olarak özyinelemeli olarak tanımlanabilir.
- ▣ Genel olarak bir yığına uygulanan tek işlemler ekleme ve silmedir.
- ▣ Bir yığından ekleme veya silme yapabilmek için iki algoritmaya ihtiyacımız var: reheap yukarı ve reheap aşağı.
- ▣ Reheap yukarı algoritması, son eleman hariç bir yığın olan bir yığın dışı yapıyı onarır. Son elemanı, o eleman doğru konumuna gelene kadar ağacın yukarısına doğru yükseltir. Reheap yukarı algoritması, bir elemanı yığına eklediğimizde ve bir diziden bir yığın oluşturduğumuzda gereklidir.
- ▣ Reheap aşağı algoritması, iki yığın ve bir kökten oluşan bir yığın dışı yapıyı onarır. Algoritma, kök elemanı doğru konumuna gelene kadar ağacın aşağısına doğru indirir. Bir yığından bir eleman sildiğimizde reheap aşağı algoritmasına ihtiyacımız vardır.
- ▣ Bir yığın, her düğüm ile çocukları arasında sabit bir ilişki olduğu için bir dizide uygulanabilir, çünkü bir yığın tamamlanmış veya neredeyse tamamlanmış bir ikili ağaçtır.
- ▣ Yığınların üç yaygın uygulaması seçim, öncelik kuyruğu ve sıralamadır.
- ▣ Bir yığın, sıralanmamış bir listede *k*ıncı en büyük (veya en küçük) öğeyi seçmek için kullanılabilir.
- ▣ Bir yığın, her öğenin diğer öğelerle olan ilişkisini belirleyen bir öncelik numarasına sahip olduğu bir öncelik kuyruğu uygulamak için de kullanılabilir.
- ▣ Bir yığın ayrıca yığın sıralama adı verilen bir sıralama algoritmasını uygulamak için de kullanılabilir.

9.7 Uygulama Setleri

Alıştırma

1. Şekil 9-18'deki yapılarından hangilerinin yığın olduğunu ve hangilerinin olmadığını gösterin.

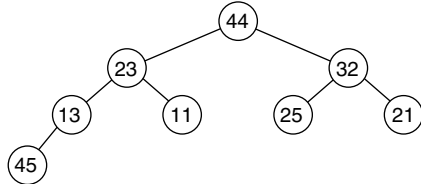


ŞEKİL 9-18 Alıştırma 1 için Yığınlar

2. Aşağıdaki verilerden bir yığın oluşturun:

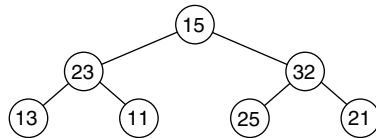
23 7 92 6 12 14 40 44 20 21

3. Şekil 9-19'da gösterilen yığın dışı yapıya yeniden yığınlama yukarı algoritmasını uygulayın.



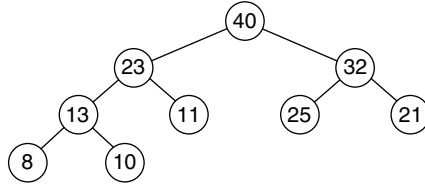
ŞEKİL 9-19 Egzersiz 3 için Yığın Dışı Yapı

4. Şekil 9-20'de gösterilen kısmi yığın yapısına yeniden yığınlama aşağı algoritmasını uygulayın.



ŞEKİL 9-20 Egzersiz 4 için Kısmi Yığın

5. Şekil 9-21'de yığının dizi uygulamasını gösterin.



ŞEKİL 9-21 Egzersizler 5, 6 ve 7 için Yığın

6. Şekil 9-21'deki yığın üzerinde silme işlemini uygulayın. Silmeden sonra yığını onarın.
7. Şekil 9-21'deki yığına 38 ekleyin. Eklemeden sonra yığını onarın.
8. Şekil 9-22'deki yığında 32 ve 27'nin sol ve sağ çocuklarını gösterin. Ayrıca 14 ve 40'ın sol çocuklarını da gösterin.

40	27	32	15	14	20	25	11
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

ŞEKİL 9-22 Alıştırmalar 8 ve 9 için Yığın Dizisi

9. Şekil 9-22'deki yığında 11'in ebeveynini, 20'nin ebeveynini ve 25'in ebeveynini gösterin.
10. Eğer bir düğüm 25 indekseyse, sağ çocuğunun indeksi nedir? Sol çocuğunun indeksi nedir? İndekslerin 0'dan başladığını varsayın.
11. Eğer bir düğüm 37 indekseyse, ebeveyninin indeksi nedir?
12. Aşağıdaki dizilerden hangileri yığınlardır?
- 42 35 37 20 14 18 7 10
 - 42 35 18 20 14 30 10
 - 20 20 20 20 20 20
13. Aşağıdaki yığınlardan hangisinin silineceğini gösterindelete algoritması üç kez çağırıldıktan sonra:

50 30 40 20 10 25 35 10 5

14. Aşağıdaki yığınları 33, 22 ve 8 eklendikten sonra oluşan yığını gösterin:

50 30 40 20 10 25 35 10 5

15. Hem bir yığın hem de bir ikili arama ağacı olan bir ağaç çizin.

16. Aşağıdaki verileri kullanarak bir öncelik kuyruğu oluşturun. İlk sayı önceliktir ve harf veridir.

3-A 5-B 3-C 2-D 1-E 2-F 3-G 2-H 2-I 2-J

17. Kuyruktan üç öğe silindikten sonra Egzersiz 16'daki öncelik kuyruğunu n içeriğini gösterin.
18. Aşağıdaki işlemlerden sonra Egzersiz 16'daki orijinal öncelik kuyruğunu n içeriğini gösterin:

4-K ekle, 3-L ekle, sil, 2-M ekle, sil

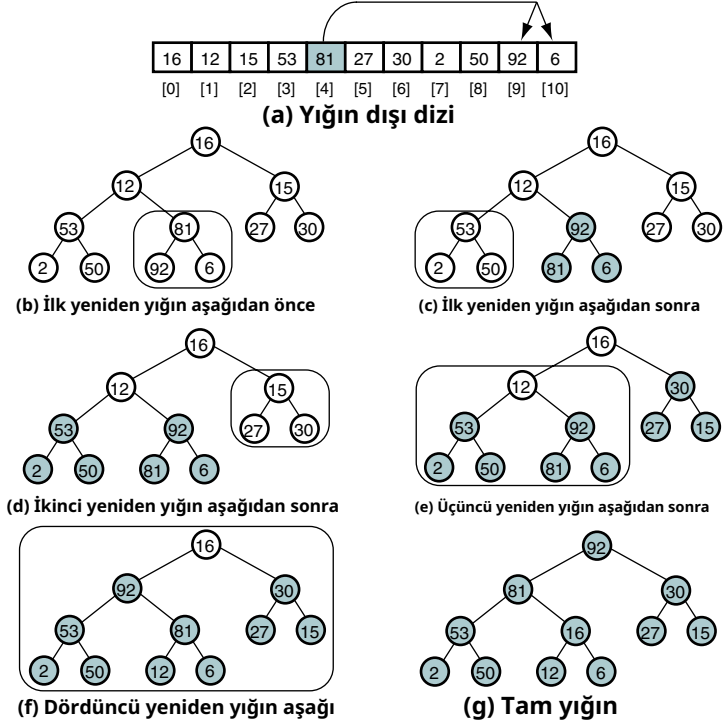
Problemler

19. Algoritma 9-1, "Reheap Up"ı minimum bir yığın oluşturacak şekilde yeniden yazın.
20. Algoritma 9-2, "Reheap Down"ı minimum bir yığın yeniden oluşturacak şekilde yeniden yazın.
21. Algoritma 9-4, "Insert Heap"ı minimum bir yığın oluşturacak şekilde yeniden yazın.
22. Algoritma 9-5, "Delete Heap Node"ı minimum bir yığın yeniden oluşturacak şekilde yeniden yazın.
23. İki yığını birleştirip üçüncü bir yığın üretecek bir algoritma yazın.
24. Problem 19 için C kodunu yazın.
25. Problem 20 için C kodunu yazın.
26. Problem 21 için C kodunu yazın.
27. Problem 22 için C kodunu yazın.
28. C kodunu Problem 23 için yazın.

Projeler

29. Ağaç algoritmalarını incelememiz, çoğu ağaç yapısının oldukça verimli olduğunu göstermiştir. Yığınların verimliliğini inceleyelim. Bölüm 9.3'te geliştirilen yığın ADT'si ni, bir yığın oluşturma karmaşıklığını belirlemek için değiştirin. Bu programda, yığın oluşturma için gerekli veri hareketlerinin sayısını ve rimlilik olarak ölçün.
- Bir desen belirlemek için, programınızı rastgele sayılarla doldurulmuş dizilerle ç alıştırm. Beş farklı dizi boyutu kullanın: 100, 200, 500, 1000 ve 2000. Ardından, bu çalışmalarda geliştirilen sezgileri analiz edin ve hangi büyük-O notasyonunun en iy i şekilde uygulandığını belirleyin. Bulgularınızı uygun tablolar ve grafiklerle kısa bir rapor halinde hazırlayın.
30. Proje 29'u, yalnızca yeniden yığın yukarı ve yeniden yığın aşağı algoritmalarının ve rimliliğini belirlemek için değiştirin. Yine, verileri analiz edin ve verimlilikleri ile ilgili sonuçlarınızı kısa bir rapor halinde hazırlayın.

31. Algoritma 9-3, bir diziden yığın oluşturmak için reheap up algoritmasını kullanır. Bir diziden yığın oluşturmanın başka, biraz daha hızlı bir yolu, dizinin ortasından başlayarak bir döngüde reheap down kullanmaktır $\lfloor N/2 \rfloor - 1$, bu, son boş olmayan alt ağacın köküdür ve ağacın köküne doğru çalışır. Bu kavram Şekil 9-23'te gösterilmiştir.



ŞEKİL 9-23 Son Alt Ağaçtan Yığın Oluşturma (Proje 31)

Yığın oluşturma algoritmasını yeniden yazın ve ardından yukarıdaki yaklaşımları kullanarak C kodunu yazın.

32. Proje 31'e, ortadan yukarıya yığın oluşturma algoritmasının verimliliğini belirlemek için kod ekleyin.
33. Bir havayolu şirketi, aşırı rezervasyonlu uçuşlar için bekleme listesindeki yolcuların önceliğini belirlemek için aşağıda gösterilen formülü kullanır.

$$\text{öncelik numarası} = A / 1000 + B - C$$

burada

A müşteri'nin son bir yıldaki toplam mil sayısıdır

B onun veya onun sık uçan yolcu programındaki yıl sayısını temsil eder

C müşteri'nin uçuşu rezervasyon yaptırdığı zaman varış pozisyonunu temsil eden bir sıra numarasıdır

Tablo 9-2'de gösterildiği gibi, aşırı rezervasyonlu müşterilerin bir dosyasını verildiğinde, dosyayı okuyan ve her müşterinin öncelik numarasını belirleyen bir program yazın. Program daha sonra öncelik numarasını kullanarak bir öncelik kuyruğu oluşturur ve bekleyen müşterilerin öncelik sırasına göre bir listesini yazdırır.

İsim	Mil	Yıllar	Sıra
Bryan Devaux\Amanda Trapp	53,000	53317236432	123456789
\Baclan Nguye	89,000		
n\Sarah Gilley\	93,000		
Warren Rexroa	17,000		
d\Jorge Gonzal	72,000		
es\Paula Hung	65,000		
\Lou Mason St	34,000		
eve Chu\Dave	21,000		
Lightfoot\Joan	42,000		
ne Brown	63,000		
	33,000		10 11

TABLO 9-2 Proje 33 için Veriler

Bölüm 10

Çok Yönlü Ağaçlar

Altıncı bölümde ağaçların genel kavramlarını inceledik. Ayrıca üç tür ikili ağaç inceledik: ikili arama ağaçları (Bölüm 7), AVL ağaçları (Bölüm 8) ve yığınlar (Bölüm 9). İkili ağaçlar boyut olarak büyüdükçe, yükseklikleri önemli hale gelebilir. Örneğin, 1000 girişe sahip bir ağacın yüksekliği en az 10'dur ve 10 000 girişe sahip bir ağacın yüksekliği en az 17'dir. Ağaçlar dengesizse, yükseklikleri önemli ölçüde daha büyük olabilir.

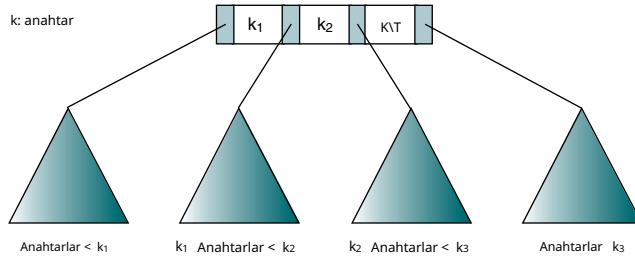
Bu bölümde, dış derecesi 2 ile sınırlı olmayan ancak ikili arama ağaçlarının genel özelliklerini koruyan ağaçları keşfedeceğiz. İkili ağaçtaki her düğüm yalnızca bir girişe sahipken, çok yönlü ağaçlar her düğümde birden fazla girişe sahip olabilir ve dolayısıyla birden fazla alt ağaç içerebilir. Bu yapıları, iç arama ağaçları, yazım denetleyicileri ve harici dosya dizinleri gibi uygulamalarda bulacaksınız.

10.1 m -yönlü Arama Ağaçları

Bir m -yönlü ağaç, her düğümün 0 ile m alt-ağaç içerebildiği bir arama ağacıdır; burada m , B-ağaç sırası olarak tanımlanır. Boş olmayan bir çok yönlü ağaç verildiğinde, aşağıdaki özellikleri tanımlayabiliriz:

1. Her düğüm 0 ile m alt-ağaç içerir.
2. $k < m$ alt-ağaç içeren bir düğüm, k alt-ağaç ve $k - 1$ veri girişi içerir.
3. İlk alt-ağaçtaki anahtar değerleri, ilk girişteki anahtar değerinden daha küçüktür; diğer alt-ağaçlardaki anahtar değerleri, ebeveyn girişindeki anahtar değerine eşit veya daha büyüktür.
4. Veri girişlerinin anahtarları, $\text{anahtar}_1 \leq \text{anahtar}_2 \leq \dots \leq \text{anahtar}_k$ şeklinde sıralanmıştır.
5. Tüm alt-ağaçlar kendileri çok yönlü ağaçlardır.

Şekil 10-1, 4. dereceden bir m -yönlü ağacın diyagramik temsidir.



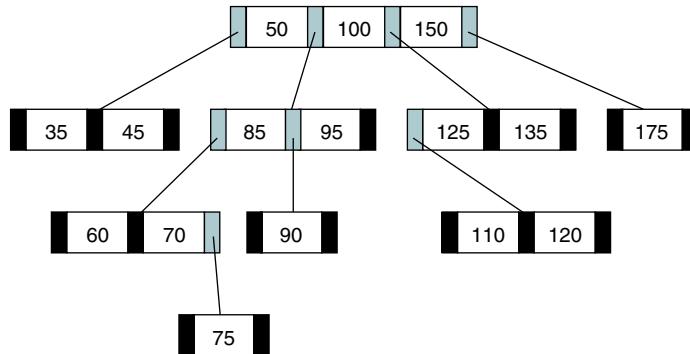
ŞEKİL 10-1 M-yönlü 4. Derece Ağaç

Şekil 10-1'i dikkatlice inceleyin. İlk olarak, ikili arama ağacıyla aynı yapıya sahip olduğunu belirtmek gerekir: bir girişin solundaki alt ağaçlar, girişin anahtarından daha küçük anahtarlar için verileri içerir ve bir girişin sağındaki alt ağaçlar, girişin anahtarından daha büyük veya eşit anahtarlar için verileri içerir. Bu sıralama, ilk ve son alt ağaçlarda en kolay şekilde görülebilir.

Şimdi ikinci alt ağacı inceleyin. Anahtarlarının k_1 'ye eşit veya daha büyük ve k_2 'den daha küçük olduğunu unutmayın. Bu, k_1 için sağ alt ağaç ve aynı zamanda k_2 için sol alt ağaç olarak hizmet eder. Diğer bir deyişle, bir alt ağacın sol veya sağ alt ağaç olup olmadığı, hangi düğüm girişini görüntülediğinize bağlıdır.

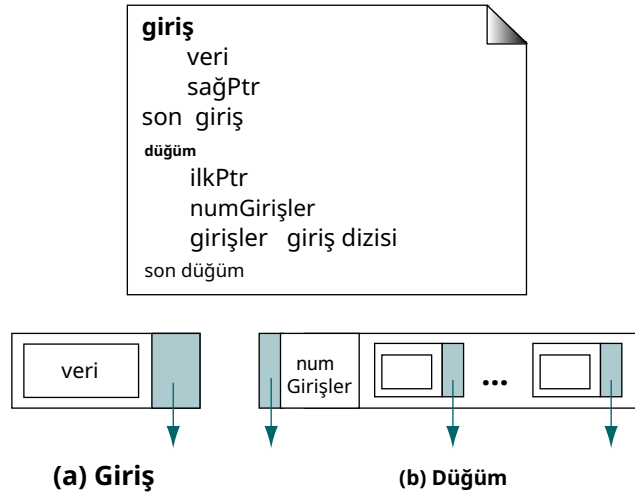
Ayrıca, düğümdeki giriş sayısından bir alt ağacın daha fazla olduğunu unutmayın; her düğümün başında ayrı bir alt ağaç vardır. Bu ilk alt ağaç, düğümdeki ilk girişten daha küçük anahtarları içeren tüm alt ağaçları tanımlar.

Her düğümün değişken sayıda girişi olduğu için, düğümde şu anda kaç giriş olduğunu takip etmenin bir yoluna ihtiyacımız var. Bu, Şekil 10-1'de gösterilmeyen bir giriş sayısı ile yapılır. Şekil 10-2, dört yönlü bir ağaçtır.



ŞEKİL 10-2 Dört Yönlü Ağaç

Artık bir m -yönlü ağaç düğümünün tanımını kodlamaya hazırız. Öncelikle kayıtlar için bir yapı oluşturalım. Kayıt sayısı belirli bir maksimuma kadar değiştiği için, kayıtları saklamak için en iyi yapı bir dizidir. Her kayıt, verinin anahtarını, verinin kendisini (veya başka bir yerde saklanıyorsa datanın bir işaretçisini) ve sağ alt ağacına bir işaretçi tutmalıdır. Şekil 10-3(a) yapıyı göstermektedir.



ŞEKİL 10-3M-yol Düğüm Yapısı

Düğüm yapısı, ilk girişin anahtarından daha küçük girişlere sahip alt ağaca ilk işaretçiyi, düğümde şu anda bulunan giriş sayısını ve girişlerin dizisini içerir. Dizi, $m - 1$ giriş için yer bulundurmalıdır. Düğüm yapısı Şekil 10-3(b)'de gösterilmiştir.

m -yol ağaçları tartışmasını bırakmadan önce, sizinle paylaşmak istediğimiz bir düşünce daha var. İkili arama ağacı, 2. dereceden bir m -yol ağacıdır.

İkili arama ağacı, 2. dereceden bir m -yol ağacıdır.

10.2 B-ağaçları

m -yol ağacı, bir ağacın yüksekliğini büyük ölçüde azaltma potansiyeline sahiptir. Ancak, hala bir ana eksikliği vardır: dengeli değildir. 1970 yılında Seattle, Washington'daki Boeing Şirketi'nde çalışan iki bilgisayar bilimcisi,

yeni bir ağaç yapısı oluşturdular ve buna B-ağacı adını verdiler.¹ A B-ağacı m -yönlü arama ağacıdır ve aşağıdaki ek özelliklere sahiptir:

1. Kök ya bir yapraktır ya da $2 \dots m$ alt ağaçları vardır.
2. Tüm iç düğümler en az $\lceil m/2 \rceil$ boş olmayan alt ağaçlara ve en fazla m boş olmayan alt ağaçlara sahiptir.
3. Tüm yaprak düğümleri aynı seviyededir; yani, ağaç mükemmel bir şekilde dengelenmiştir.
4. Bir yaprak düğüm en az $\lceil m/2 \rceil - 1$ ve en fazla $m - 1$ girişe sahiptir.

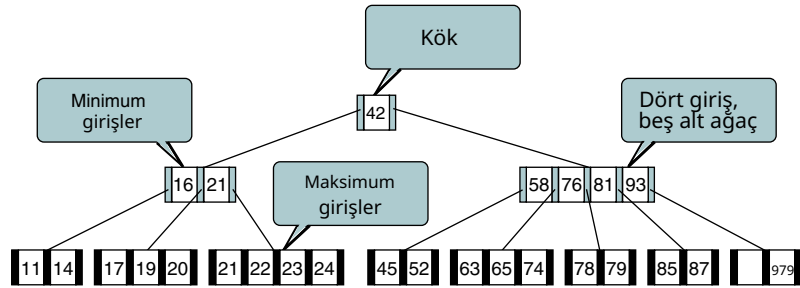
Bir B-ağacının tanımından, bir B-ağacının mükemmel bir şekilde dengelenmiş m -yönlü ağaç olduğu ve her düğümün, kök hariç, en az yarı dolu olduğu açık olmalıdır. Tablo 10-1, farklı derecelerdeki B-ağaçları için bir kök düğümdeki alt ağaçların minimum ve maksimum sayılarını tanımlar.

Sıra	Alt ağaç sayısı		Giriş sayısı	
	Asgari	Maksimum	Asgari	Maksimum
3456	2233	3456	1122	2345
$\dots m$	$\dots$ $\lceil m/2 \rceil$	$\dots m$	$\dots$ $\lceil m/2 \rceil - 1$	$\dots$ $m - 1$

TABLO 10-1 Farklı Derecelerdeki B-ağaçlarındaki Girişler

Şekil 10-4, 5. dereceden bir B-ağacı içermektedir. Tasarımını inceleyelim. Önce likle, Bölüm 10.1'deki m -yönlü kurallara göre, ağaç geçerli bir m -yönlü ağaçtır. Şimdi B-ağaç kurallarına bakalım. Kök bir yaprak değildir - iki alt ağacı vardır (Kural 1). Sol iç düğüm, minimum alt ağaç sayısına (üç) sahiptir ve sağ iç düğüm maksimum alt ağaç sayısına (beş) sahiptir (Kural 2). Tüm yaprak düğümler seviye 2'de bulunmaktadır (Kural 3). Yaprakların hepsi minimum (iki) ve maksimum (dört) giriş arasında (Kural 4) bulunmaktadır. Bu nedenle, ağaç geçerli bir B-ağacıdır.

1. R. Bayer ve E. McCreight, "Büyük Sıralı Dizilerin Organizasyonu ve Bakımı," *Acta Informatica* 1, no. 3 (1972): 173-189. Bu makale neden buna *B-ağaç* dediklerini açıklamamaktadır. İki olasılık vardır: *B* Bayer için veya *B* sadece C Dili adındaki bir sıradır.



ŞEKİL 10-45. Dereceden Bir B-ağacı

B-ağacı Uygulaması

B-ağaçları için dört temel işlem şunlardır: ekleme, silme, gezinti ve arama. Bu bölümde bu dört algoritmayı ve çağırdıkları herhangi bir algoritmayı tartışıyoruz.

B-ağacı Ekleme

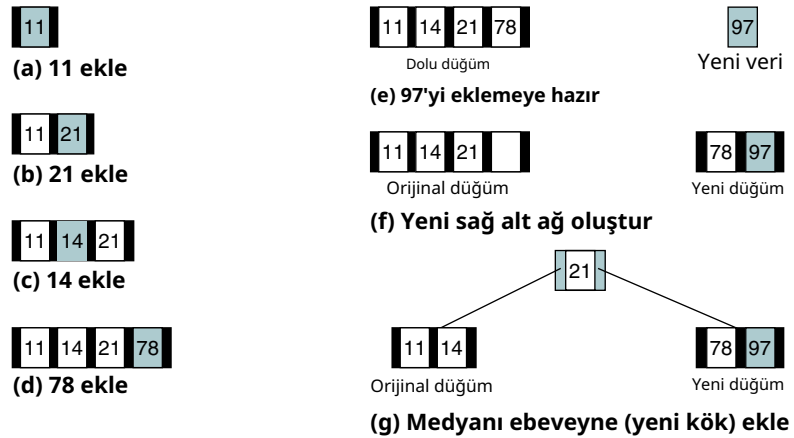
İkili arama ağacında olduğu gibi, B-ağacı eklemesi bir yaprak düğümde gerçekleşir. Bu nedenle, ilk adım, eklenen verinin yaprak düğümünü bulmaktır. Eğer düğüm dolu değilse—yani, $m - 1$ 'den daha az girişi varsa, yeni veriler düğümde sırayla eklenir.

B-ağaçları alttan yukarı doğru büyür.

Yaprak düğüm dolu olduğunda, taşma olarak bilinen bir durumla karşılaşırız. Taşma, yaprak düğümün verilerin yarısını içeren iki düğüme bölünmesini gerektirir. Düğümü bölmek için, mevcut bellekten yeni bir düğüm ayırırız ve ardından dolu düğümün *sonu* kısmındaki verileri yeni düğüm e kopyalarız. Veriler bölündükten sonra, yeni giriş ya orijinal düğüme ya da yeni düğüme, anahtar değerine bağlı olarak eklenir. Ardından, medyan veri girişi ebeveyn düğüme eklenir.

B-ağacını nasıl inşa edeceğimizi anlamamıza yardımcı olmak için, basit bir örnek üzerinden geçelim. 5 dereceli bir B-ağacı yapısı verildiğinde, 11, 21, 14 ve 78 eklemeye başlıyoruz. İlk ekleme, kök haline gelen bir düğüm oluşturur. Sonraki üç ekleme, verileri düğümde artan anahtar sırasına yerleştirir. Bu noktada, Şekil 10-5(f)'deki gibi görünen bir ağacımız var.

97'yi eklemeye çalıştığımızda, düğümün dolu olduğunu keşfederiz. Bu nedenle yeni bir sağ alt ağaç oluşturur ve verilerin üst yarısını ona taşırız, geri kalan veriyi ise orijinal düğümde bırakırız. Bu durum Şekil 10-5(b)'de gösterilmiştir. Yeni değer (97) mantıken verilerin üst yarısına ait olduğu için yeni düğüme yerleştirilmiştir.

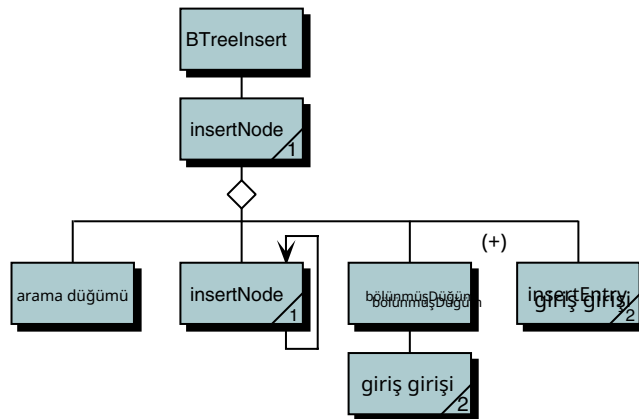


ŞEKİL 10-5B-ağaç Ekleme Genel Görünümü

Yeni düğümü oluşturduktan sonra, medyan değerli veriyi (21) orijinal düğümün ebeveynine ekliyoruz. Orijinal düğüm bir kök olduğundan, yeni bir kök oluşturuyoruz ve 21'i ona ekliyoruz. Bu adım, 97'nin B-ağacına eklenmesini tamamlar ve ağaç şimdi bir seviye büyümüştür. Ortaya çıkan B-ağacı Şekil 10-5(g)'de gösterilmiştir.

B-ağaç ekleme tasarımı Şekil 10-6'da gösterilmektedir. Bir girişi ebeveynine ekleme süreci, ikili arama ağacına ilginç bir karşılık sunar. İkili arama ağacının yukarıdan aşağıya dengesiz bir şekilde büyüdüğünü hatırlayın. B-ağaçları dengeli bir şekilde alttan yukarı doğru büyür.

B-ağacının kök düğümü taşıma yaptığında ve medyan girişi yukarı itildiğinde, yeni bir kök düğüm oluşturulur ve ağaç bir seviye büyür.



ŞEKİL 10-6B-ağacı Ekleme Tasarımı

B-ağaç ekleme psödo kodu Algoritma 10-1'de gösterilmiştir.

ALGORİTMA 10-1 B-ağacı Ekle

```

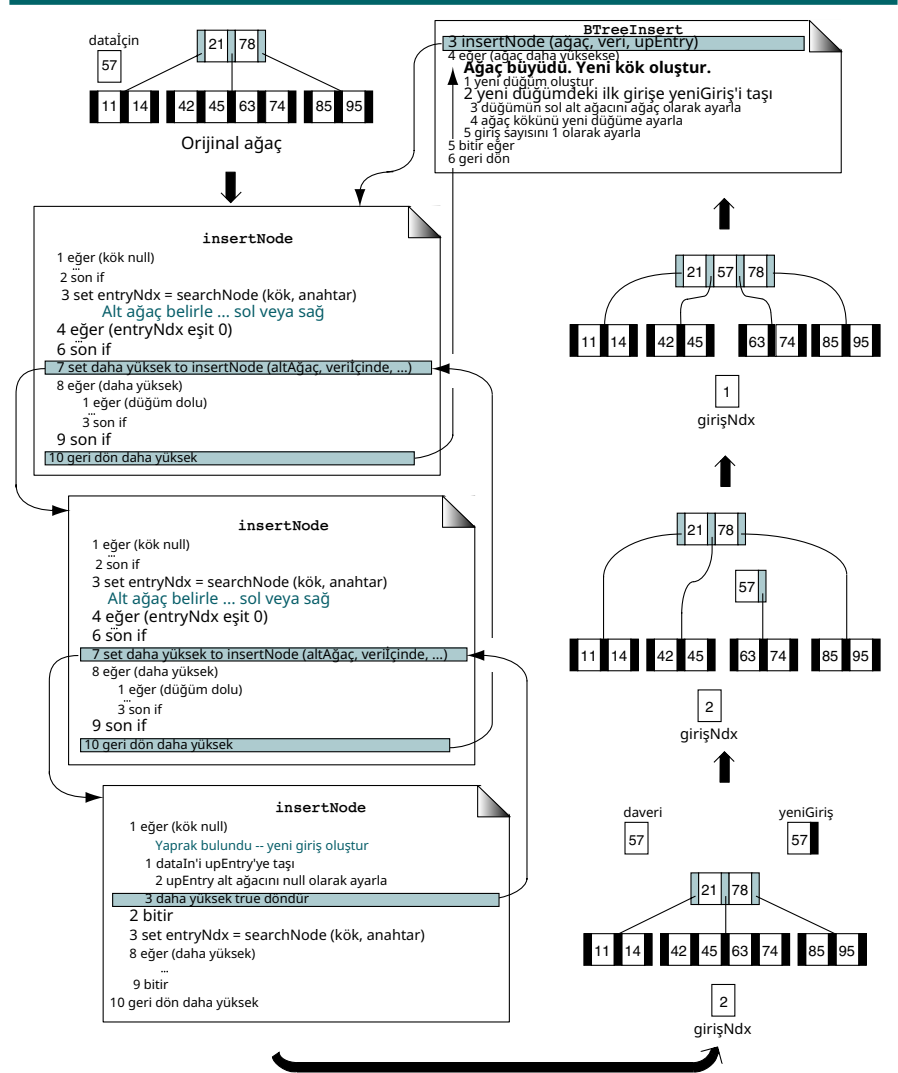
Algoritma BTreeInsert (ağaç, veri)
Veriyi B-Ağacına ekler. Eşit anahtarlar sağ dalda yer alır.
Önceki ağaç B-Ağacı'na referanstır; null olabilir
Sonra veriler eklendi
1  eğer (ağaç null)
    Boş ağaç. İlk düğümü ekle.
    1  yeni düğüm oluştur
    2  düğümün sol alt ağacını null olarak ayarla
    3  verileri yeni düğümdeki ilk girişe taşı
    4  ilk girişin alt ağacını null olarak ayarla
    5  ağaç kökünü yeni düğümün adresine ayarla
    6  giriş sayısını 1 olarak ayarla
2  eğer biterse
    Mevcut ağaçta veri ekle
3  yeniDüğümEkle (ağaç, veri, yukarıGiriş)
4  eğer (ağaç daha yüksek)
    Ağaç büyüdü; yeni kök oluştur
    1  yeni düğüm oluştur
    2  yukarıGirişi yeni düğümdeki ilk girişe taşı
    3  düğümün sol alt ağacını ağaç olarak ayarla
    4  ağaç kökünü yeni düğüm olarak ayarla
    5  giriş sayısını 1 olarak ayarla
    6  eğer biterse
        son BTreeEkle

```

Algoritma 10-1 Analizi B-ağaç ekleme algoritmasının sadece üç süreci vardır. İlk olarak, eğer ağaç null ise, birkök oluşturur. İkincisi, eğer ağaç null değilse, düğüm ekleme algoritmasını çağırır. Gördüğümüz gibi, düğüm ekleme, verileri bir yaprağa ekleyen ve ayrıca herhangi bir taşma girişini üst düğümlerine ekleyen bir özyinelemeli algoritmadır. Ancak, kök taşarsa, medyan girişi ekleme için hiçbir üst düğüm yoktur. Bu nedenle, üçüncü süreç, ağaç taşarsa yeni birkök oluşturur. Taşma olduğunda, düğüm ekleme algoritması medyan girişini B-ağaç ekleme algoritmasına geri iletir, parametre `entry` üzerinden. Bu medyan girişi yeni kök için giriş olur.

Düğüm Ekle

Düğüm ekleme algoritması, B-ağaç eklemenin kalbidir. Karmaşık olduğu için, onu anlamak için bir örnek üzerinden geçmeniz gerekir. 5. dereceden bir B-ağacının inşasını izleyelim. Sürecin ilk kısmının genel bir görünümünü zaten gösterdik. 57'yi ağaca eklemeye hazır olduğumuz yerden devam edelim. B-ağaç ekleme (Algoritma 10-1) çağırısı yaparak ve kökü ve eklenecek yeni veriyi (57) geçirerek başlıyoruz. Şekil 10-7'de gösterildiği gibi.



ŞEKİL 10-7B-ağacı Oluştur, İlk Taşma

Şimdi, Algoritma 10-2'de gösterildiği gibi, düğüm ekleme işlemi üzerinden Şekil 10-7'yi izliyoruz.

ALGORİTMA 10-2B-ağacı Düğüm Ekle

```

Algoritma ekleDüğüm (kök, dataIn, upEntry)
Veri için yaprağı bulmak üzere ağacı özyinelemeli olarak arar. Eğer düğüm
aşırı yüklenirse, medyan anahtarın verisini üst düğüme ekler.
    Ön   kök, ağaç veya alt ağaca referanstır; null olabilir
        dataIn, eklenecek veriyi içerir
        upEntry, giriş yapısı için referanstır
    Son   veri eklendi
        upEntry, ebeveynine eklenmesi gereken taşma girişi
        r
    Boolean (daha yüksek) döndür
1   eğer (kök null)
    Yaprak bulundu -- yeni giriş oluştur
    1   dataIn'i upEntry'ye taşı
    2   upEntry alt ağacını null olarak ayarla
    3   daha yüksek true döndür
2   eğer biterse
    Giriş noktasını (yaprak) ara
3   entryNdx'i searchNode (kök, anahtar) olarak ayarla
    Alt ağacı belirle ... sol veya sağ
4   eğer (entryNdx 0'a eşit)
    1   eğer (data key < key ilk girişte)
        1   alt ağacı düğümün sol alt ağacı olarak ayarla
    2   aksi takdirde
        1   alt ağacı girişin alt ağacı olarak ayarla
    3   eğer biterse
5   aksi takdirde
    1   alt ağacı entryNdx rightPtr olarak ayarla
6   eğer biterse
7   taller'ı insertNode (subTree, dataIn, upEntry) olarak ayarla
    Veri eklendi -- geri çık
8   eğer (taller)
    1   eğer (düğüm dolu)
        1   splitNode (kök, entryNdx, newEntryLow, upEntry)
        2   taller'ı true olarak ayarla
    2   aksi takdirde
        1   insertEntry (kök, entryNdx, upEntry)
        1   root'a upEntry ekle
        2   taller'ı false olarak ayarla
    3   eğer biterse
9   eğer biterse
10  daha uzun dönmek
end insertNode

```

Algoritma 10-2 AnaliziB-ağacı düğüm ekleme (ekleDüğüm) bir özyinelemeli algoritmadır (bkz. ifade 7). Kodu Şekil 10-7 üzerin den takip edelim. Temel durum, null bir alt ağaçtır, bu da yaprak düğümün önceki çağrıda b ulunduğunu gösterir. İlk çağrıda, kök null değildir, bu nedenle ekleDüğüm aramaDüğümü çağırarak doğru alt ağaç dalını belirler, 21'de bulunur ve kendisini 21'in sağ alt ağaç işaretçisi i kullanarak özyinelemeli olarak çağırır.

Üçüncü çağrı, bu ekleme için temel duruma ulaşır. Çünkü root null'dır, **insertNode** yeni veriyi ve yeni giriş için bir null işaretçisini taşır (ifadeler 1.1 ve 1.2) ve yeni girişin eklenmeye hazır olduğunu belirten true döner. Yeni girişin referansla geçirilmesi gerektiğini unutmayın.

Daha sonra, 7. ifadede **insertNode** 'ye geri döneriz ve yeni girişi yaprak düğümüne eklemeye hazırız, geri dönerken. Eğer düğüm doluyorsa, onu böleriz vesonra medyan girişi mevcut düğümün ebeveynine ekleriz (ifadeler 8.1.1). **splitNode** için mantık, Algoritma 10-4'te gösterilmiştir ve kısa süre içinde detaylı olarak tartışılacaktır. Şu an **insertNode** 'nin yeni bir sağ alt ağaç düğümü oluşturduğunu ve verileri ona taşıdığını anlamak yeterli dir. Öte yandan, eğer düğüm dolu değilse, onu mevcut düğümüne basitçe ekleriz.

Node dolu olduğu için, medyan değerli girişi (57) yukarıda 21 ve 78 içeren ebeveyn node'a ekliyoruz. Bu adım, ebeveyn node'da 57 için yer olduğu için eklemeyi tamamlar. Ancak, tüm yolu geri çıkmakla birlikte, rekursiyonu tamamlamamız gerekiyor.

Arama Düğümü

Arama düğümü önce hedefin ilk girişin anahtarından daha küçük olup olmadığını kontrol eder. Eğer öyleyse, giriş 0'ı döndürür. Hedef ilk düğümünden daha küçük değilse, hedefin düğümünü girişlerin sonundan başlayarak başa doğru çalışarak bulur. Bu tasarım, baştan başlamaktan daha verimlidir (aşağıdaki Algoritma 10-3 Analizi'ne bakın). Pseudocode, Algoritma 10-3'te gösterilmiştir.

ALGORİTMA 10-3 B-ağaç Arama Düğümü

```

Algoritma searchNode (nodePtr, hedef)
B-Tree düğümünde anahtar <= hedef olan veri girişi arayın.
  Ön nodePtr, null olmayan düğümüne referanstır
  hedef, bulunacak anahtardır
  Anahtar <= hedef olan girişin indeksini döndür
  -veya- 0, eğer anahtar düğümdeki ilk girişten küçükse
1  eğer (hedef < ilk girişteki anahtar)
1  0 döndür
2  eğer biterse
3  yürüyücüyü giriş sayısı - 1 olarak ayarla
4  döngü (hedef < giriş anahtarı[walker])
1  yürüyücüyü azalt
5  döngüyü sonlandır
6  yürüyücüyü döndür
SON searchNode

```

Algoritma 10-3 Analizi Bu algoritma çok ilginç bir arama kullanır. Giriş dizisinin sonundan başlar ve başa doğru çalışır. Bu arama tekniğini kullanmamızın nedeni, her girişin, mevcut girişin anahtarından büyük veya ona eşit ve bir sonraki girişin anahtarından küçük olan verilerle bir alt ağaç gösterdiği dır. Başlangıçtan arama yapsaydık, hedefimizi aşar ve geri dönmek zorunda kalırdık. Sonundan arama yaparak, hedefe eşit veya daha küçük bir giriş bulduğumuzda, arzu edilen hedefi içeren alt ağacın göstericisini de bulmuş oluruz.

Algoritmanın yakın analizi, 0 indeksini döndürdüğü iki durum olduğunu ortaya koymaktadır. İlk durum, hedefin ilk girişten daha küçük olduğu durumdur (ifade 1). İkinci durum, hedefin ilk girişten büyük veya ona eşit ve ikinci girişten küçük olduğu durumdur. Bu durumda veriler, ilk girişin sağ alt ağacında bulunur.

Bölme Düzümü

Bölme düğümü, tam bir düğüm, düğüme eklenmesi gereken bir giriş ve yeni giriş için indeks konumu alır ve verileri mevcut düğüm, bir medyan girişi ve yeni bir düğüm arasında böler. Kod, Algoritma 10-4'te gösterilmiştir.

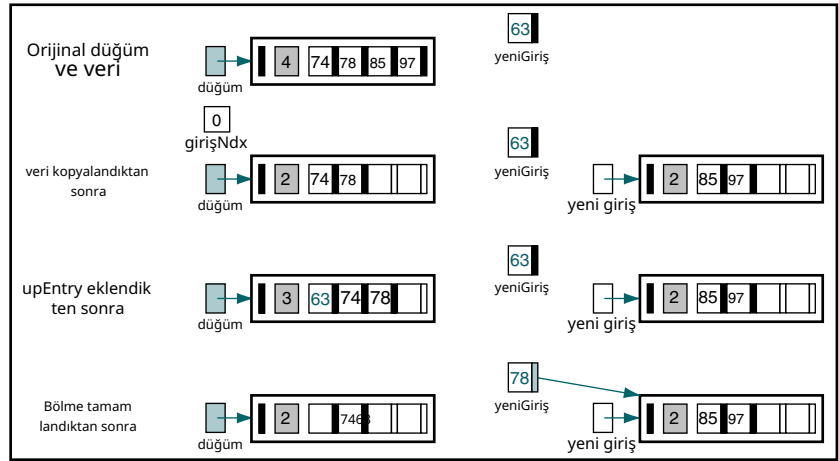
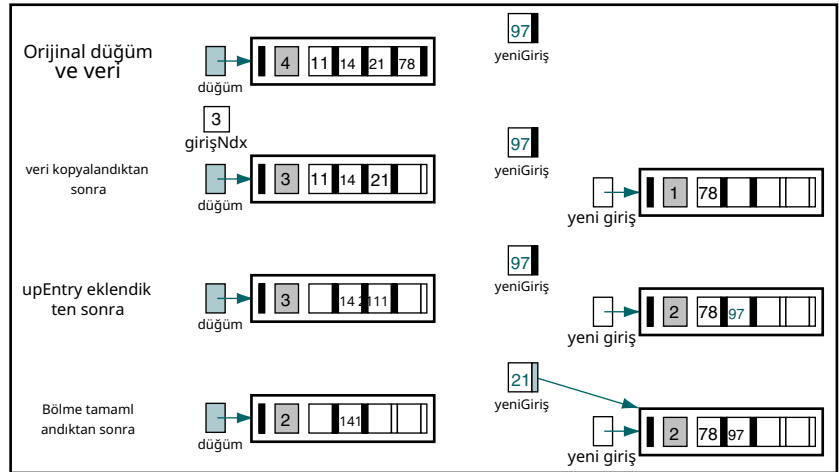
ALGORİTMA 10-4B-ağaç Bölme Düzümü

```

Algoritma splitNode (düğüm, girişNdx, yeniGirişDüşük, yukarıGiriş)
Düğüm taşıma yaptı. Düğümü iki düğüme ayır.
    Ön düğüm, taşınan düğüme referanstır
        entryNdx, ebeveynin indeks konumunu içerir
        newEntryLow, yeni verinin < entryNdx verisi ise doğru
        upEntry, eklenmekte olan girişe referanstır
    Sonra upEntry, ebeveyne eklenmek üzere olan girişi içerir
1 yeni düğüm oluştur
  Sağ alt ağaç düğümünü oluştur
2 yüksek girişleri yeni düğüme taşı
3 eğer (entryNdx < minimum girişler)
  1 Düğümde upEntry ekle
4 aksi takdirde
  1 yeni düğümde upEntry ekle
5 eğer biterse
  Üst için giriş oluştur
6 medyan verileri upEntry'ye taşı
7 ortalama verinin sağ alt ağacı olarak yeni düğüm firstPtr oluştur
8 yeni düğümü upEntry'nin sağ alt ağacı olarak oluştur
bitiş splitNode
  
```

Algoritma 10-4 AnaliziÜç farklı ekleme konumunu analiz etmemiz gerekiyor: yeni anahtar medyan anahtardan daha küçük, yeni anahtar medyan anahtar ve yeni anahtar medyan anahtardan daha büyük. İlk iki durum aynı duruma indirgenir. Şekil 10-8(a), yeni giriş medyandan küçük veya ona eşit olduğunda bir düğümün bölünmesi için adım adım mantığı göstermektedir ve Şekil 10-8(b), yeni giriş medyandan büyük olduğunda adımları göstermektedir.

Her iki durumda da yeni bir düğüm ayırarak başlıyoruz ve orijinal düğümün sonundaki verileri yeni düğümün başına kopyalıyoruz. Ardından, **upEntry** içinde bulunan yeni girişi ya orijinal düğüme ya da yeni düğüme ekliyoruz. Son olarak, medyan girişi **upEntry**'ye kopyalıyoruz. Algoritma 10-4'e girdiğimizde, **upEntry** taşıma düğümüne eklenen verileri içerir; düğüm bölündükten sonra, **upEntry**, üst düğüme döndüğümüzde eklenmesi gereken verileri içerir.

(a) Yeni giriş $\leq$ medyan(b) Yeni giriş $>$ medyan

ŞEKİL 10-8B-ağacı 5. Derece Bölme Düğümü

Yeni anahtar, medyan anahtardan küçük veya eşit olduğunda, yeni veriler sol v eya orijinal düğümde yer alır. Öte yandan, yeni anahtar medyan anahtardan büyük se, yeni veriler yeni düğümde yer alır. Bu nedenle, 3. ifadede yeni girişin konumunu (entryNdx) test ederiz ve ya orijinal düğümü ya da yeni düğümü geçirerek ekleme algoritmasını çağırırız.

Şekil 10-8'i dikkatlice incellerseniz, hangi durumun meydana geldiğine bakılmaksızın, medyan değerinin her zaman aynı yerde olduğunu, minimum giriş sayısı ile tanımlandığını göreceksiniz. Ayrıca, medyan girişindeki sağ işaretçi her zaman yeni düğümün işaretçisi. Bu nedenle, medyan girişini oluşturmak kolaydır.

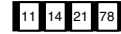
Şekil 10-8'deki test durumları, Şekil 10-9'da daha geniş bir bağlamda bulunmaktadır. Şekil 10-8(a), Şekil 10-9(c)'deki son eklemedir ve Şekil 10-8(b), Şekil 10-9(b) ile aynıdır.

Ekleme Özeti

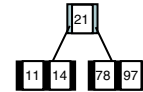
Artık bir B-ağacına veri eklemek için gerekli olan tüm algoritmaları inceledik. Özet olarak, Şekil 10-9'da tam bir B-ağacı inşa ediyoruz. Önceki şekiller de bazı ağaçları zaten gördünüz. Farklı verilerle benzer bir ağaç inşa edebileceğinizden emin olmak için bu şekli dikkatlice inceleyin.

(a) 78, 21, 14, 11 ekle

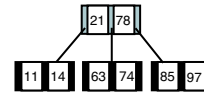
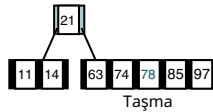
Eklemeden sonraki ağaçlar



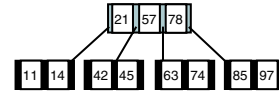
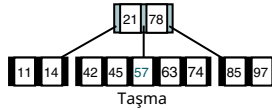
(b) 97 ekle



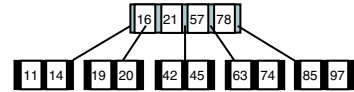
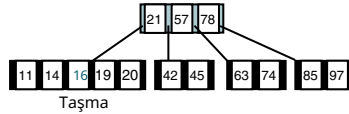
(c) 85, 74, 63 ekle



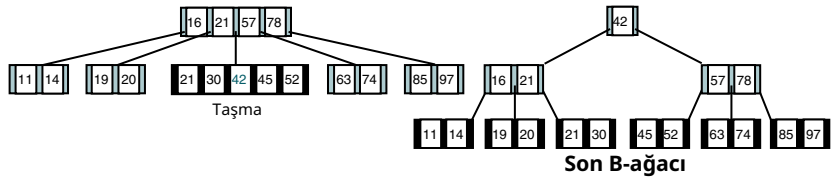
(d) 45, 42, 57 ekle



(e) 20, 16, 19 ekle



(f) 52, 30, 21 ekle

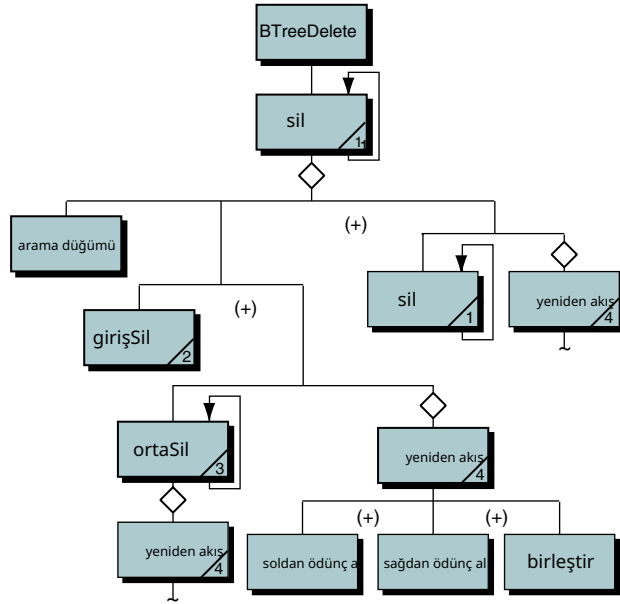


ŞEKİL 10-95. Dereceden Bir B-ağacı Oluşturma

B-ağacı Silme

B-ağacı düğümünden bir veri girişi silerken üç husus dikkate alınmalıdır. Öncelikle, silinecek verilerin gerçekten ağaçta bulunduğundan emin olmalıyız. İkincisi, silme işleminden sonra düğüm yeterli sayıda girişe sahip değilse, yapısal eksikliği düzeltmemiz gerekir. Bir düğümün minimum giriş sayısından daha az girişe sahip olması bir taşma durumudur. Son olarak, ikili arama ağacında görüldüğümüz gibi, yalnızca bir yaprak düğümünden silebiliriz. Bu nedenle, silinecek veriler bir iç düğümdeyse, onların yerini alacak bir veri girişi bulmalıyız.

yer. B-ağacı silme tasarımı Şekil 10-10'da gösterilmiştir. Algoritmalarına geçmeden önce bu tasarımı incelemenizi öneririz.



ŞEKİL 10-10B-ağacı Silme Tasarımı

İki seviye özyineleme olduğunu unutmayın. İlk olarak, Algoritma 10-6, ağacı dolaşarak silinecek düğümü arar ve eğer bir yaprakta bulunursa, basitçe siler. İkincisi için, silinecek veriler bir yaprakta değilse, Algoritma 10-8 özyinelemeli olarak çağrılır. Algoritma yalnızca bir özyineleme ile yazılabilir, ancak takip etmesi daha karmaşıktır.

Şimdi düğüm silme mantığını inceleyin. Silinecek veriyi arar. Eğer bulamazsa, bir hata mesajı yazdırır ve rekursiyonu sonlandırır. Bu birinci temel durumdur. İkinci temel durum, verilerin bulunup silindiği zamandır. Her iki durumda da silme, bir alt akışa neden olup olmadığını belirlemeli ve bir dönüş Boolean'ı ayarlamalıdır—alt akış meydana geldiyse doğru, düğüm iyiye yanlış.

Alt akış işleme, düğüm silme rekursiyondan geri çıkarken gerçekleşir. Her dönüşten sonra, alt akış olup olmadığını kontrol etmek için düğümü kontrol eder. Eğer bir alt akış meydana geldiyse, alt akışı düzeltir ve devam eder. Bu büyük resmi anladıktan sonra, algoritmalara bakmaya hazırız.

Ekleme gibi, B-ağacı silme işlemi de iki algoritma gerektirir: bir kullanıcı arayüzü ve bir özyinelemeli silme fonksiyonu. Yüksek seviyeli kullanıcı arayüzü, özyinelemeli silme işlemini çağırır ve ardından kök alt akışını kontrol eder. Bu, Algoritma 10-5'te gösterilmiştir.

ALGORİTMA 10-5B-ağacı Silme

```

Algoritma BTreeDelete (ağaç, dltAnahtar)
B-ağacıdan hedef anahtara sahip girişi sil.
    Ön ağaç, bir B-ağacına referanstır
        dltAnahtar, silinecek girişin anahtarıdır
    Son veri silindi veya yanlış döndürüldü
    Başarı (bulundu) veya başarısızlık (bulunamadı) döner
1  eğer (ağaç boşsa)
    1  yanlış döndür
2  eğer biterse
3  sil (ağaç, silAnahtar, başarı)
4  eğer (başarı)
    1  eğer (ağaç giriş sayısı sıfır)
        Ağaç daha kısa--kökü sil
        1  ağacı sol alt ağaç olarak ayarla
    2  eğer biterse
5  eğer biterse
6  başarıyı döndür
son BTreeSil

```

Algoritma 10-5 AnaliziB-ağacı silmenin iki sorumluluğu vardır. İlk olarak, silme işlemini başlatır ve özyinelemeli algoritmayı, sil çağırır. Ardından, silme işlemi tamamlandığında, kökün alt akışa uğrayıp uğramadığını belirler. Eğer uğramışsa, yeni kökün adresi düğümün ilk işaretçisinde bulunur.

Düğümü Sil

Silme işleminin kalbi, özyinelemeli silme algoritmasında gösterilmiştir. Eğer silme, null bir alt ağaç ile karşılaşır, silinecek veriyi bulamamış demektir ve alt akış yanlış olarak döner.

Kök null değilse, silme, silinecek verilerin düğümde olup olmadığını kontrol eder. Bu, ekleme araması için kullandığımız aynı algoritmadır (bkz. Algoritma 10-3, "B-ağacı Arama Düğümü"). Arama ya 0 döner, silinecek anahtarın düğümdeki ilk girişten daha küçük olduğunu gösterir, ya da silinecek anahtara eşit veya daha küçük bir düğüm için bir giriş indeksi döner. Silinecek anahtar mevcut düğümde yoksa, silme kendisini yeni bir alt ağaç ile özyinelemeli olarak çağırır. Eğer silinecek anahtar bulunmuşsa, ya deleteEntry ya da deleteMid çağrılarak silinir. Düğüm silindikten sonra, silme alt akışı kontrol eder ve gerekirse alt akışı onarmak için reFlow çağrısını yapar. Pseudocode, Algoritma 10-6'da gösterilmiştir.

ALGORİTMA 10-6B-ağacı Düğüm Sil

```

Algoritma sil (kök, silAnahtar, başarı)
SilAnahtar'ı içeren düğümü özinelemeli olarak bulur; veriyi siler.
  Ön  kök, boş olmayan bir B-Ağacına referanstır
      silAnahtar, silinecek girişin anahtarıdır
      başarı, boolean'a referanstır
  Son  veri silindi--başarı doğru; veya başarı yanlış
      alt akış--doğru veya yanlış
  Başarıyı true veya false olarak döndür
1  eğer (kök null)
    Yaprak düğüm bulundu--deleteKey anahtarı mevcut değil
    1  başarıyı false olarak ayarla
    2  yanlış döndür
2  eğer biterse
3  entryNdx'yi searchNode (root, deleteKey) olarak ayarla
4  eğer (deleteKey bulundu)
    Silinecek girişi buldu
    1  başarıyı true olarak ayarla
    2  eğer (yaprak düğüm)
        1  underflow'yu deleteEntry (root, entryNdx) olarak ayarla
    3  aksi takdirde
        Giriş iç düğümde
        1  eğer (entryNdx > 0)
            1  önceki girişin rightPtr'ini leftPtr olarak ayarla
        2  aksi takdirde
            1  solPtr'yi kök firstPtr olarak ayarla
        3  eğer biterse
        4  underflow'yu deleteMid (kök, entryNdx, solPtr) olarak ayarla
        5  eğer (underflow)
            1  underflow'yu reFlow (kök, entryNdx) olarak ayarla
        6  eğer biterse
    4  eğer biterse
5  aksi takdirde
    1  eğer (deleteKey ilk girişteki anahtardan küçükse)
        1  alt ağacı kök firstPtr olarak ayarla
    2  aksi takdirde
        deleteKey sağ alt ağaçta
        1  alt ağacı entryNdx rightPtr olarak ayarla
    3  eğer biterse
    4  underflow'yu delete (alt ağaç, deleteKey, başarı) olarak ayarla
    5  eğer (underflow)
        1  underflow'yu reFlow (kök, entryNdx) olarak ayarla
    6  eğer biterse

```

devam ediyor

ALGORİTMA 10-6B-ağacı Düğüm Sil (devam ediyor)

```

6  eğer biterse
7  geri akış
   silmeyi sonlandır

```

Algoritma 10-6 Analizi Bu silme algoritması, kendisi de dahil olmak üzere dört farklı algoritmayı çağırır. Rekürsif çağrı 5.4. ifadesi nde bulunur. Rekürsif çağrılardan sonra, taşma testi yapmamız gerektiğini unutmayın. Bu mantık önemlidir. Bir düğüm taşma yaptığında taşma tespit edilse de, mantık akışı ebeveyne döndüğünde işlenir. Taşmayı çözmek için ebeveyni, sol alt ağacı ve sağ alt ağacı ihtiyaç duyarız, bu nedenle bu noktada işlenir.

Girişi Sil

Girişi silme, bir düğümde giriş kaldırır ve sıkıştırır; yani, silinen girişin sağındaki girişleri sola taşır. Girişi sildikten sonra taşma testi yapar ve taşma meydana geldiyse doğru, düğüm iyiye yanlış olan bir Boolean döner. Kod Algoritma 10-7'de gösterilmiştir.

ALGORİTMA 10-7B-ağacı Girişi Sil

```

Algoritma deleteEntry (düğüm, girişİndx)
Yaprak düğümde girişİndx'teki girişi siler.
    Ön  düğüm, silinecek veriyi içeren düğüme referanstır
        girişİndx, düğümdeki girişin indeksidir
    Son  giriş silindi
    Return underflow <Boolean>
1  silmeden sonra girişleri sola kaydır
2  eğer (giriş sayısı minimum giriş sayısından azsa)
    1  doğru döndür
3  aksi takdirde
    1  yanlış döndür
4  eğer biterse
    son  silmeGirişi

```

Orta Sil

Daha önce belirttiğimiz gibi, tüm silme işlemleri bir yaprak düğümde gerçekleşmelidir. Silinecek veriler bir yaprak düğümde değilse, yerine geçecek verileri bulmalıyız. Yerine geçecek veriler için iki seçenek vardır: ya hemen önceki ya da hemen sonraki. Her ikisi de işe yarar, ancak hemen önceki veriyi kullanmak daha verimlidir çünkü bu her zaman bir düğümdeki son giriştir ve silindiğinde kaydırma gerektirmez. Bu nedenle hemen önceki veriyi kullanıyoruz.

Hemen önceki, silinecek girişin sol alt ağacındaki en büyük düğümdür. İlk çağrılan algoritma, delete , sol alt ağacı belirler ve bunu silinecek veriyi içeren düğümle birlikte bir parametre olarak ve dizideki girişinin indeksini geçirir. deleteMid , son girişte sol alt ağacın sağ işaretçisini takip ederek bir yaprak düğüme ulaşana kadar devam eder.

Bir yaprak düğüm bulmak, deleteMid için özyinelemeli temel durumdur. Bu noktada silinecek veriyi yaprağın son girişindeki veri ile değiştirir ve ardından yaprak düğümdeki önceki girişi silmek için kendisini çağırır. deleteMid için pseudocode, Algoritma 10-8'de gösterilmiştir.

ALGORİTMA 10-8B-ağacı Silme Orta

```

Algoritma deleteMid (düğüm, girişİndeksi, altAğaç)
Ağaçtaki iç düğümde giriş siler.
    Ön düğüm, silinecek girişi içeren düğüme referanstır
        girişİndeksi, düğümde silinecek girişin indeksidir
        altAğaç, düğümün alt ağacına referanstır
    Son silme verisi, hemen önceki ile değiştirilir
        ve önceki yaprak düğümde silinir
    Geri dönüş, alt akış doğru veya yanlış
    Silinecek düğümü değiştirmek için girişi bul
1  eğer (en sağdaki alt ağaç yoksa)
    Yaprak bulundu. Veriyi değiştir ve yaprak girişini sil.
    1 öncekinin verisini silinecek girişe taşı
    2 düğüm girişleri minimumdan azsa alt akışı ayarla
2  aksi takdirde
    Bulunamadı. Önceki ögeyi bulmak için sağa geç.
    1 set underflow'u deleteMid olarak ayarla
        (düğüm, girişİndeksi, sağ alt ağaç)
    2 eğer (underflow)
    1 underflow'yu reFlow (kök, entryNdx) olarak ayarla
    3 eğer biterse
3  eğer biterse
4      geri akış
end deleteMid

```

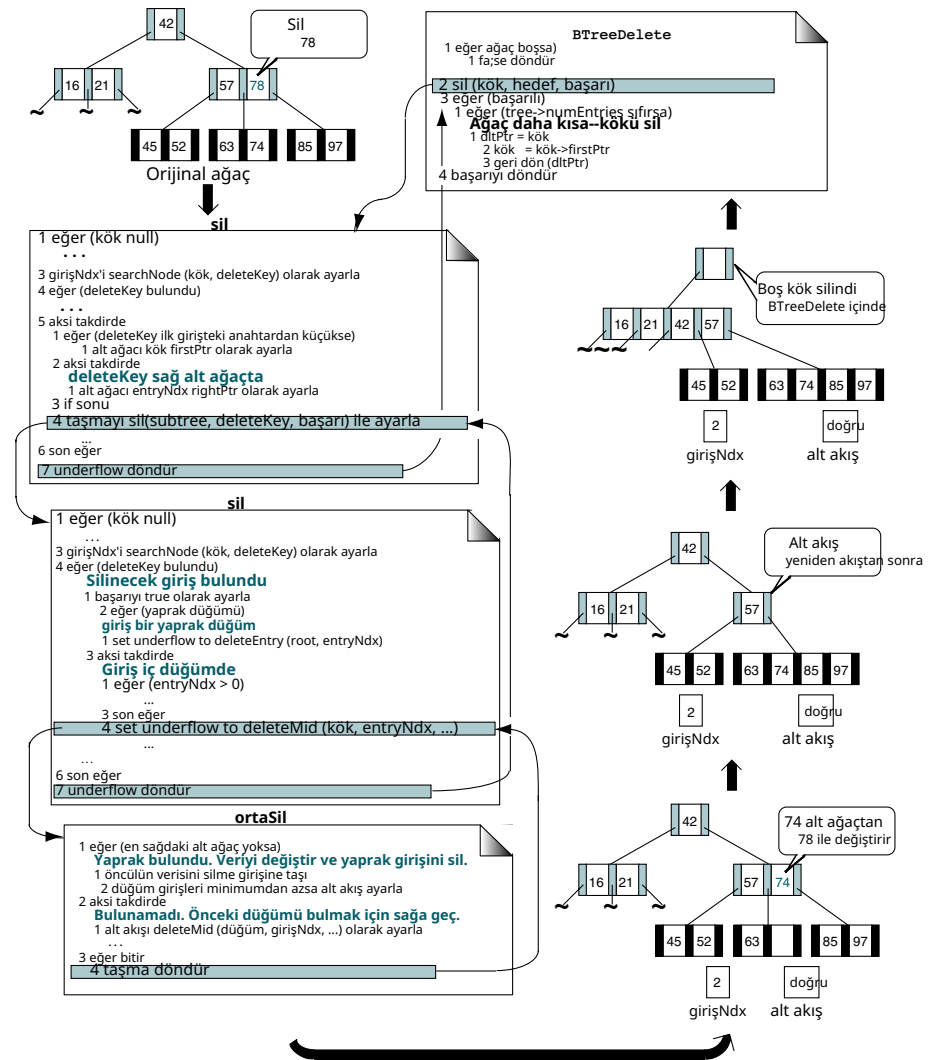
Algoritma 10-8 Analizi Bu algoritma oldukça basittir. Diğer algoritmalar gibi, temel durum ilk olarak ifade 1'de ele alınır. Bir yaprak düğüm için özyinelemeli arama ifade 2'de bulunur. Hem temel durumun hem de genel durumun, taşma olup olmadığını belirlediğini ve kodun sonunda taşma durumunu döndürme ifadesine ilettiğini unutmayın.

Şekil 10-11, B-ağacı silme, silme veya yeni silme algoritmalarının etkileşimini göstermektedir. Devam etmeden önce bunu incelemek isteyebilirsiniz.

Yeniden Akış

Taşma meydana geldiğinde, taşan düğümü en az bir giriş ekleyerek yerine getirmemiz gerekir. Bu işleme yeniden akış denir. Bu, B-ağacı silme işlemi çerçevesinde belki de en zor mantıktır. Bunu anlamak için iki kavramı gözden geçirmemiz gerekiyor: denge ve birleştirme. Denge, ağacın bütünlüğünü yeniden sağlamak için düğümler arasında veriyi kaydırır. Ağacın yapısını değiştirmede için, daha az rahatsız edicidir ve bu nedenle tercih edilir.

Birleştirmesi, bir alt akışta bulunan bir giriş, bir minimal kardeş ve birebeveynin verilerini tek bir düğümde birleştirir. Bu nedenle, Birleştirme, maksimum sayıda girişe sahip bir düğüm ve geri dönüştürülmesi gereken boş bir düğüm ile sonuçlanır.



ŞEKİL 10-11B-ağaç Silme İşlemleri

reFlow'a girdiğimizde, kök, dengesizleşen düğümün ebeveynini içerir, giriş indeksine göre tanımlanır; yani, giriş indeksi, dengesiz durumda alt-ağacı tanımlar. Kökün sol ve sağ alt ağaçlarını inceleyerek, birinin minimum giriş sayısından fazlasına sahip olup olmadığını belirlemeye başlarız. Eğer biri fazlaysa, ağacı dengeleyerek sıralayabiliriz. Ne sol ne de sağ alt ağaç, minimum girişlerden fazlaysa, ebeveynle birleştirmemiz gerekir. Pseudocode, Algoritma 10-9'da gösterilmiştir.

ALGORİTMA 10-9B-ağacı Taşma Yeniden Akış

Algoritma reflow (kök, girişİndeksi)Köktek
i alt ağaçlardan birinde bir alt akış meydana gelmiştir, bu alt akış giriş i
ndeksi parametresi ile tanımlanır. Alt akışı, alt ağaçları dengeleyerek
veya birleştirerek düzeltin.

Ön alt akış, kök alt ağacında meydana gelmiştir
girişİndeksi, alt akış alt ağacının ebeveynini tanımlar

Son alt akış düzeltildi

Düğüm alt akışa uğradıysa alt akışı true olarak döndürün

Önce borç almaya çalışın. Önce sağ alt ağaca bakın.

1 eğer (sağAğaç girişleri minimum girişlerden büyükse)

1 sağdanÖdünçAl (kök, girişİndeksi, solAğaç, sağAğaç)

2 alt akışı false olarak ayarla

2 aksi takdirde

Sağdan dengeleyemiyorum. Soldan denemek.

1 eğer (solAğaç girişleri minimum girişlerden büyükse)

1 soldanÖdünçAl (kök, girişİndeksi, solAğaç, sağAğaç)

2 alt akışı false olarak ayarla

2 aksi takdirde

Ödünç alamıyorum. Girişleri birleştirmeliyim.

1 birleştir (kök, girişİndeksi, solAğaç, sağAğaç)

2 eğer (kök numEntries minimum girişlerden azsa)

1 alt akışı true olarak ayarla

3 aksi takdirde

1 alt akışı false olarak ayarla

4 eğer biterse

3 eğer biterse

3 eğer biterse

4 geri akış

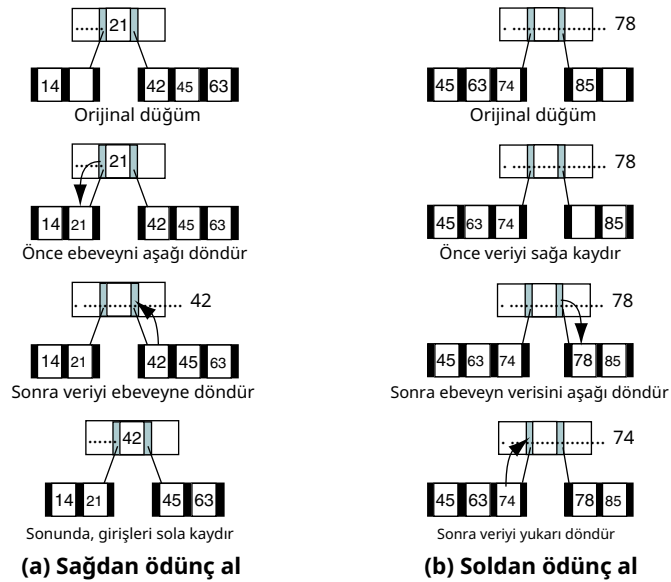
son yeniden akış

Algoritma 10-9 Analizi Algoritmanın ilk kısmı, ağacı dengeleyerek düzeltmenin mümkün olup olmadığını belirler. Bir alt ağacın minimumu n altında olduğunu biliyoruz. Eğer minimumun üzerinde olan birini bulursak, dengeleyebiliriz. Öncelik le sağ alt ağacı kontrol ederiz ve eğer minimumun üzerindeyse, sağdan ödünç alma algoritmasını çağırırız. Eğer değilse, sol alt ağacı kontrol ederiz. Yine, eğer minimumun üzerindeyse, düzeni geri getir mek için soldan ödünç alma algoritmasını çağırırız. Eğer alt ağaçların hiçbirisi ekstra bir girişe sahip de ğilse, onları ayrı bir birleştirme algoritmasını çağırarak birleştirmemiz gerekir.

Alt akış yalnızca düğümler birleştirildiğinde meydana gelebilir. Çünkü borç alma, mini mum giriş sayısından daha fazlasına sahip bir düğümden gerçekleştiği için, alt akış olamaz. Bu nedenle, borç aldığımızda, ister soldan ister sağdan, alt akış false olarak ayarlanır. Öte yandan, düğümleri birleştirdiğimizde, kök düğümden ebeveyn girişini sileriz. Bu nedenle, alt akışın meydana gelip gelmediğini kontrol etmemiz gerekir. Eğer meydana geldiyse, alt akışı true olarak döneriz.

Denge

Bir ağacı, bir kardeşten diğerine ebeveyn aracılığıyla bir giriş döndür erek dengeleriz. reFlow içinde döndürme yönünü belirleriz. Bu nede nle, en basit uygulama, soldan döndürme için bir algoritma ve sağda n döndürme için bir algoritma yazmaktır. Döndürmeler, Şekil 10-12'd e grafiksel olarak gösterilmiştir.



ŞEKİL 10-12Ödünç Alarak Düzeni Yeniden Sağlama

Sol ödünç alma için psödo kod, Algoritma 10-10'da gösterilmiştir ve sağ ödünç alma için psödo kod, Algoritma 10-11'de gösterilmiştir.

ALGORİTMA 10-10B-ağacı Sol Ödünç Alma

Algoritma borrowLeft (kök, girişİndeksi, sol, sağ)
 Kökün sağ alt ağacının
 alt akışa uğradığı belirlenmiştir. Sol alt ağaçtan bir düğüm ödünç al ve
 ebeveyn üzerinden döndür.

Ön kök, alt akışa uğrayan düğümün ebeveynidir
 girişİndeksi ebeveyn girişi
 sol, ekstra düğüm(ler) içeren bir alt ağaçtır
 sağ, alt akışa uğrayan bir alt ağaçtır

Son alt ağaçlar dengelidir
 Yeni veriler için giriş 0'da yer açmak için girişleri sağa kaydır

- 1 tüm elemanları bir sağa kaydır
- 2 Ebeveyn verilerini aşağı taşı ve sağ işaretçiyi sıfırla
- 3 kök verilerini sağdaki ilk girişe taşı
- 4 ilk işaretçiyi ilk girişin sağ alt ağacına taşı
- 4 Taşınan girişin rightPtr'si sağ ağacın ilk işaretçisi olur
- 5 sol son giriş verisini entryNdx'deki köke taşı

son borrowLeft

ALGORİTMA 10-11B-ağacı Sağdan Ödünç Al

Algoritma `borrowRight` (kök, girişNdx, sol, sağ) Kökün sol alt ağacının alt akışa uğradığı belirlenmiştir. Sağ alt ağaçtan bir düğüm ödünç alın ve ana üzerinden döndürün.

Ön kök, alt akışa uğrayan düğümün ebeveynidir
girişİndeksi ebeveyn girişi
alt akışa uğramış bir sol alt ağaç
ekstra düğüm(ler) içeren bir sağ alt ağaç

Sonuç alt ağaçlar dengelidir

Ana ve alt ağaç işaretçisini sol ağaca taşı

1 kök verisini sol son girişe taşı

2 sağ ilk alt ağacı sol son giriş sağ alt ağacına taşı

Sağ veriyi ana düğüme taşı

3 sağ ilk giriş verisini girişNdx'deki köke taşı

Sağ ağaç ilk işaretçisini ayarla ve veriyi kaydır

4 sağ ilkPtr'yi sağ ilk giriş sağ alt ağacına ayarla

5 tüm girişleri sola bir kaydır

son `borrowRight`

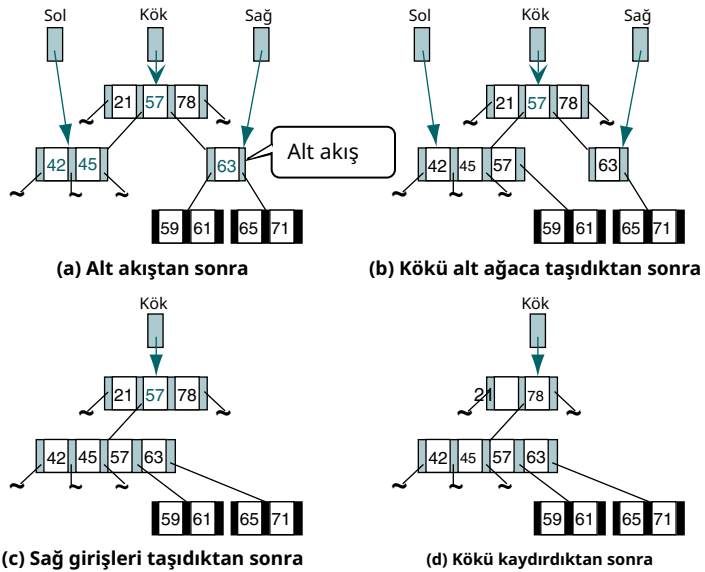
Algoritma 10-10 ve

Algoritma 10-11 Analizi Bu iki algoritmanın kodu oldukça basittir. Sağdan ödünç aldığımızda, ebeveyn düğüm girişinin verisi, ancak alt ağacı değil, sola doğru hareket eder ve sol alt ağacın son girişine haline gelir. Bu nedenle, onun için yeni bir sağ alt ağaç bulmalıyız. Bunu sağ alt ağacın ilk alt ağacında buluyoruz (bkz. Algoritma 10-10, ifade 2). Taşınan ilk alt ağacı değiştirmek için, ilk girişin sağ alt ağacını kullanıyoruz (bkz. ifade 4).

Benzer şekilde, soldan ödünç aldığımızda, sol alt ağacın son girişindeki veri kök girişinin verisine kopyalanır, ancak işaretçisi sağ alt ağacın ilk işaretçisi haline gelir. Birkaç dengeleme durumu oluşturmanızı ve döndürme ve işaretçi manipülasyonu ile rahat olana kadar algoritmaları dikkatlice takip etmenizi öneririz.

Birleştir

Dengeleyemediğimizde, düğümleri birleştirmemiz gerekir. Şekil 10-13, bir alt akış olduğunda birleştirme mantığını göstermektedir. İncelediğinizde, iki alt ağacın ve ana girişin hepsinin sol düğümde birleştirildiğini not edin. Ardından, ana giriş düğümünden silinir, bu da bir alt akışa neden olabilir. Şekilden açık olmayan şey, sağ alt ağaç düğümünün daha sonra geri dönüştürüldüğüdür.



ŞEKİL 10-13B-ağacı Birleştirme

Birleştirme için psödo kod, Algoritma 10-12'de gösterilmiştir.

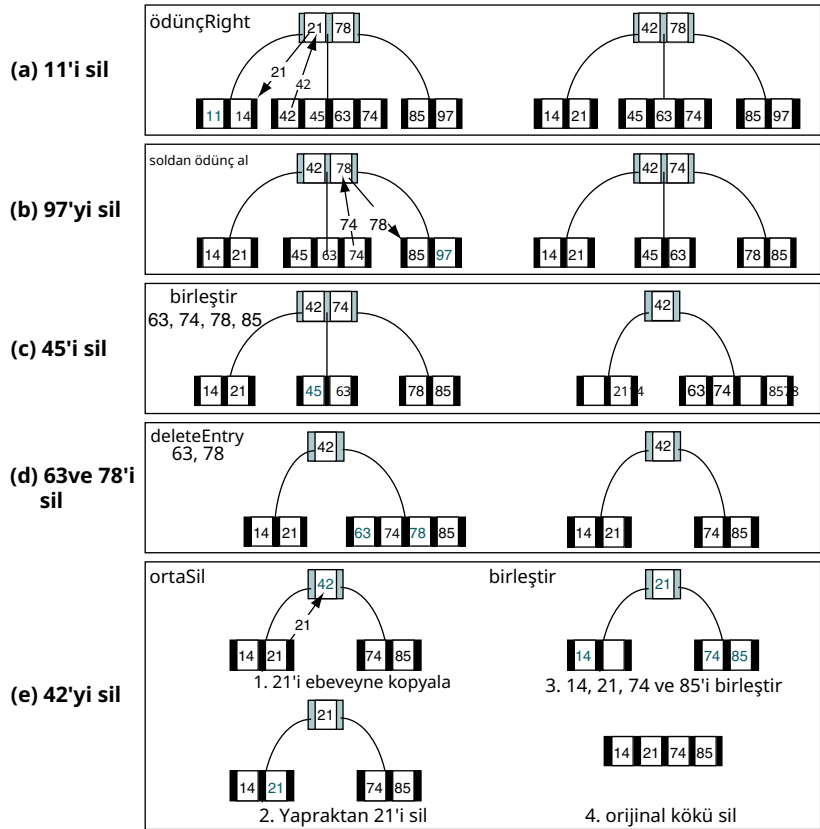
ALGORİTMA 10-12B-ağacı Birleştirme

Algoritma birleştir (kök, girişİndeksi, sol, sağ)
 Aşırı yüklenme meydana geldi ve bir giriş ödünç alamıyoruz.
 İki alt ağaç, ebeveyn ile birleştirilmelidir.
 Ön kök, aşırı yüklenmiş girişin ebeveynini içerir
 girişİndeksi ebeveyn girişini tanımlar
 sol ve sağ alt ağaçlara işaret eden işaretçilerdir
 Son ebeveyn ve alt ağaçlar birleştirildi
 sağ ağaç düğümü geri dönüştürüldü
 Ebeveyni hareket ettirin ve sağ işaretçisini sağ ağaçtan ayarlayın
 1 ebeveyn girişini sol alt ağaçtaki ilk açık girişe taşıyın
 2 sağ alt ağacın ilk alt ağacını
 taşınan ebeveyn sol alt ağacına taşıyın
 Sağ ağaçtan sol ağaca veri taşıyın
 3 sağ alt ağaçtan sol alt ağacın sonuna girişleri taşıyın
 Artık kök verisini sola kaydırın
 4 kök verisini sola kaydırın
 son birleştir

Algoritma 10-12 Analizi Bu algoritmanın en zor kısmı alt ağaç manipülasyonudur. Hangi alt ağacın aşırı yüklenmiş olduğuna bakılmaksızın, tüm düğümleri sol alt ağaçta birleştiriyoruz, böylece sağ alt ağacı geri dönüştürebiliriz.

İfade 2'yi dikkatlice inceleyin. Ebeveyn verileri, sol alt ağacın ilk boş girişine taşındı. Sağ alt ağaç ise sağ alt ağacın ilk alt ağacı haline gelir. İlk alt ağacın, ebeveynin daha büyük ve ilk girişten daha küçük olan girişleri ta nımladığını unutmayın. Bu nedenle, ebeveyn sol alt ağaca taşındığında, bu alt ağaç sağ alt ağaç olmalıdır.

Şekil 10-14, B-ağacı silme işlemlerinin bir özetidir. B-ağacından veri silmek için gereken her bir ana algoritmanın bir örneğini içerir. Konuları anladığınızdan emin olana kadar dikkatlice inceleyin.

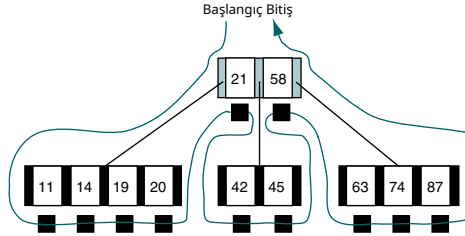


ŞEKİL 10-14B-ağacı Silme Özeti

B-ağacını Gezin

Artık bir B-ağacına sahip olduğumuza göre, onu gezmek için mantığı incele yelim. Bir B-ağacı, ikili arama ağacıyla aynı yapıda inşa edildiğinden, aynı te mel gezinti tasarımı—sıralı gezinti—kullanabiliriz. Ancak, ana fark, yaprak düğümleri hariç, bir düğümdaki tüm verileri işlemememizdir.

aynı anda. Bu nedenle, bir düğüme geri döndüğümüzde ve o noktadan devam ettiğimizde hangi girişin işlendiğini hatırlamamız gerekir. Rekürsiyon ile bumentik oldukça basittir. Şekil 10-15, ağaçta “dolaşırken” girişlerin işlenme sırasını göstermektedir. Bu şekilde her giriş, altında yürüdüğümüzde işlenir. Beklendiği gibi, verilerin ardışık sırayla işlendiğini unutmayın.



ŞEKİL 10-15 Temel B-ağaç Geçişi

Geçiş mantığı Algoritma 10-13'te gösterilmiştir.

ALGORİTMA 10-13 B-ağaç Geçişi

```

Algoritma BTreeTraversal (kök)
Ağacı sıralı geçiş kullanarak işle.
    Ön kök, B-Ağacına işaretçidir; null olmamalıdır
    Son her giriş sırasıyla işlenmiştir
1  taramaSayısını 0 olarak ayarla
2  nextAltAğaç'ı kök sol alt ağaç olarak ayarla
3  döngü (taramaSayısı <= giriş sayısı)
    Alt ağaç için test et
    1  eğer (nextSubTree null değilse)
        1  BTreeTraversal (sonrakiSubTree)
    2  eğer biterse
        Alt ağaç işlendi--bir sonraki girişi al
    3  eğer (scanCount < giriş sayısı)
        1  işle (entry[scanCount])
        2  nextSubTree'yi mevcut girişin sağ alt ağacına ayarla
    4  eğer biterse
    5  scanCount'ı artır
4  döngüyü sonlandır
SON BTreeTraversal
  
```

Algoritma 10-13 Analizi Sıralı geçişte, veriler sol alt ağaçtan sonra ve sağ alt ağaçtan önce işlenir.

Sol alt ağaçlar, 3.1.1. ifadesinde bulunan özyinelemeli çağrıda işlenir.

Bir alt ağacı işledikten sonra geri döndüğümüzde, ana girişin işlenmesine hazırız. Bunu, tarama sayısını kontrol ederek yaparız ve eğer bu sayı, düğümdeki giriş sayısından azsa, verileri işlemek için bir algoritma çağırırız (3.3.1. ifadesi). Biz

o zaman sağ alt ağacı işlemek için bir sonraki alt ağacı sağ alt ağaç olarak ayarlayın ve 3. ifa dede döngüyü devam ettirin.

Döngünün, tarama sayısı düğümdeki giriş sayısına eşit olduğunda sona erebileceğini düşünmek cazip. Bu, bir yaprak düğüm işlenirken doğru olsa da, iç düğümler için doğru değildir. Döngü sona eremez çünkü bir düğümdeki son girişteki verileri işlediğimizde, hala sağ alt ağacını işlememiz gerekir.

B-ağaç Arama

Bir düğümü arayan bir algoritmayı gördük, ancak hedef anahtarı aramak için tüm ağacı dolaşan bir algoritmayı görmedik. Bir ikili arama ağacını aradığımızda, yalnızca hedef veriyi içeren düğümü döndürdük. Ancak, bir B-ağaç düğümünde birden fazla giriş bulunduğundan, düğümü döndürmek yeterli değildir; ayrıca hedefi içeren girişi de döndürmeliyiz. Bu nedenle, Algoritma 10-14, bulunan düğüm ve giriş indeksinin referans parametrelerini kullanır.

ALGORİTMA 10-14B-ağaç Arama

```

Algoritma BTreeSearch (kök, srchKey, düğüm, foundLoc)
Bir B-Ağacını srchKey anahtarı için özyinelemeli olarak arayın.
    Ön   kök, bir ağaç veya alt ağaca referanstır
         srchKey, bulunması gereken veridir
         düğüm, bulunan bir alt ağaca referanstır
         foundLoc, bulunan giriş için indeks referansıdır
    Son  eğer bulunduysa--
         düğüm, bulunan düğümün adresini içerir      f
         oundLoc, düğüm içindeki indeks girişini içerir   eğer
         bulunmadıysa--
         yanlış döner
    Bulunan -- doğru veya yanlış döner
1  eğer (boş ağaç)
1   yanlış döndür
2  eğer biterse
3  eğer (srchKey < ilk giriş)
1   return BTreeSearch (kök ilk alt ağaç,
                           srchKey, düğüm, foundLoc)
4  eğer biterse
    Son (en sağdaki) girişten başlayarak ilk girişe kadar ara
5  foundLoc'u giriş sayısı - 1 olarak ayarla
6  döngü (srchKey < mevcut girişin anahtarı)
1   foundLoc'u azalt

```

devam ediyor

ALGORİTMA 10-14B-ağacı Arama (devam ediyor)

```

7  döngüyü sonlandır
8  eğer (srchKey mevcut girişin anahtarı ile eşitse)
    1  düğümü kök olarak ayarla
    2  doğru döndür
9  eğer biterse
    küçük veya eşit değil, bu yüzden sağ alt ağacı ara
10 return BTreeSearch (mevcut giriş sağ alt ağaç,          srch
                        Key, düğüm, bulunduLoc)
end BTreeSearch

```

Algoritma 10-14 Analizi Şekil 10-15 için mantık, Algoritma 10-3'te gördüğümüz tasarıma benzer, “B-ağacı Düğüm Arama.” Her iki algoritma da düğümün sonundan başına doğru arama yapar. Temel fark, sadece mevcut düğümü değil, alt ağaçları da aramamız gerektiğidir. Bu nedenle algoritmada iki farklı özyinelemeli çağrı gerekir, ilki hedef ilk girdid enküçük olduğunda (ifade 3.1) ve ikincisi mevcut girdiden büyük olduğunda (ifade 10).

İki temel durum vardır: null bir alt ağaçla karşılaşırız, bu hedefin mevcut olmadığı gösterir, ya da hedef girdiyi buluruz. İlk temel durum ifade 1'de ele alınır. Bu durumda basitçe false döneriz.

Eşit temel durumu 8. ifadede gösterilmiştir. Döngüde giriş numarasını ayarladığımız içi n, yalnızca düğüm adresini mevcut kök işaretçisine ayarlamamız ve doğru döndürmemiz gerekiyor.

Son bir nokta: 6. ifadeyi dikkatlice inceleyin. Döngünün dizinin başından çıkmasını ne engelliyor? Bu sorunun cevabı 3. ifadede bulunur. Hedefin anahtarı, ilk girişin anahtarına eşit veya daha büyük olmalıdır; eğer daha küçük olsaydı, ilk işaretçiyi sol alt ağaçta takip ederdik. Hedefin anahtarının ilk girişin anahtarına eşit veya daha büyük olduğunu bildiğimiz için, bu döngüyü durduran bir bekçi haline gelir. Bu nedenle, döngünün başını test etmemize gerek yoktur, bu da döngüyü çok verimli hale getirir - yalnızca bir koşulu test etmesi gerekir.

10.3 B-ağaç ADT

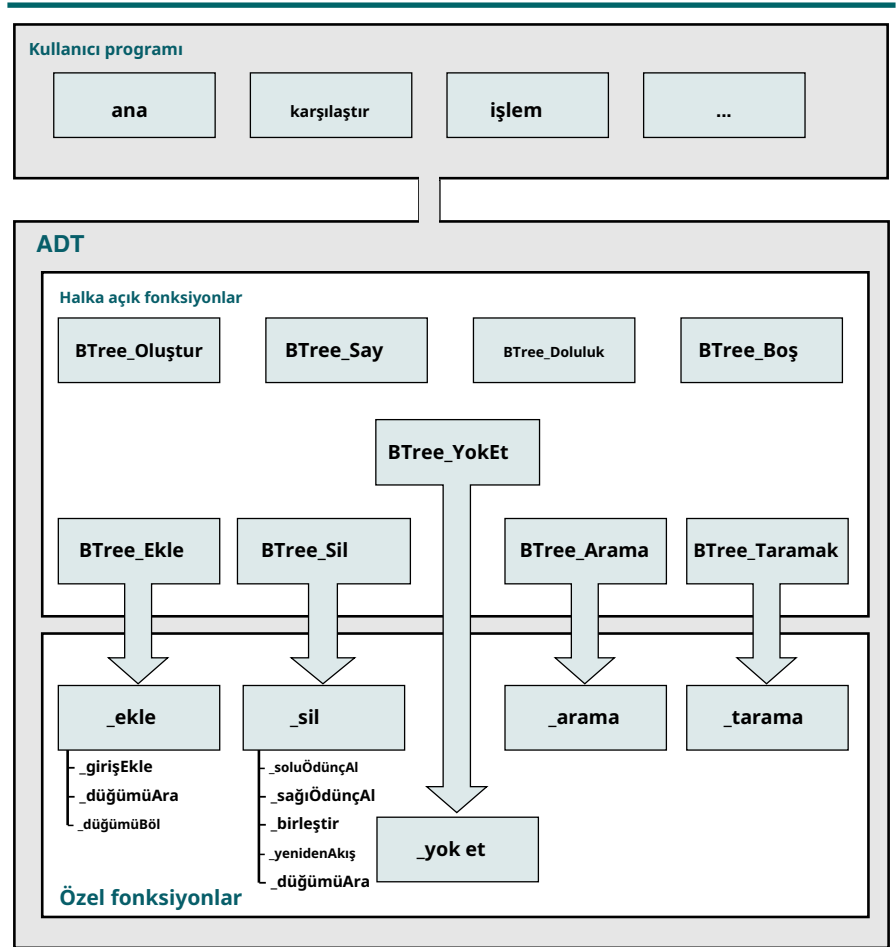
B-ağacının nasıl çalıştığını anladığınızda, B-ağaç uygulamasının oldukça karmaşık olması sürpriz olmamalıdır. Bu bölümde anahtar algoritmalar için kod geliştireyoruz. Bazıları sizin geliştirmeniz için bırakılmıştır. Şekil 10-16, temel bir işlevler listesi sağlar. Sağlanmayan bir işlev güncellemedir; yani, bir girişin içeriğini değiştirmek. Mevcut tasarım altında, bir girişin içeriğini değiştirmek için onu silip yeniden eklememiz gerekecektir.

B-ağacı Veri Yapısı

Diğer ADT yapılarında olduğu gibi, B-ağacı ADT'sini bir başyapısı ve bir düğüm yapısı ile uyguluyoruz.

Baş Yapısı

B-ağacı baş yapısı, `BTREE`, bir sayım, bir kök işaretçisi ve listeyi aramak için gereken karşılaştırma fonksiyonunun adresini içerir. Uygulama



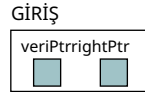
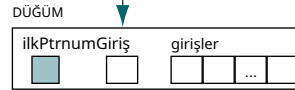
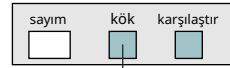
ŞEKİL 10-16B-ağaç Fonksiyon Ailesi

programın ağaca tek görünümü, ağacın oluşturulması sırasında dinamik bellekten tahsis edilen baş yapısına bir işaretçidir. Baş veri yapısı Şekil 10-17'de gösterilmiştir.

Düğüm Yapısı

B-ağaç düğüm yapısı, üç ayrı yapıyı gerektirir: bir ağaç baş yapısı, bir düğüm yapısı ve bir giriş yapısı. Ağaç baş yapısı, kök işaretçisini, gerekli olan herhangi bir meta veriyi ve karşılaştırma fonksiyonuna bir işaretçi içerir. Düğüm yapısı, düğümün sol alt ağacına bir işaretçi, bir dizi giriş ve düğümdeki mevcut giriş sayısını saklar.

Her veri girişi, bir anahtar ve özellikler içeren bir yapıya bir veri işaretçisi ve girişin sağ alt ağacına bir işaretçi içerir. Düğüm yapısı Şekil 10-17'de gösterilmiştir.



```
typedef yapısı
{
    void*      dataPtr;
    struct node* rightPtr;
} GİRİŞ;

typedef yapı düğümü
{
    struct node* ilkPtr;
    int numEntries;
    GİRİŞ girişleri[SİPARİŞ - 1];
} NODE;

typedef yapısı
{
    int sayım;
    NODE* kök;
    int (*karşılaştır) (void* argu1, void* argu2);
} BTREE;
```

ŞEKİL 10-17B-ağaç Veri Yapısı

Başlık Dosyası

B-ağaç ADT'si için başlık dosyası, veri yapıları ve kullanıcı arayüzü için p rototip bildirimlerini içerir. Program 10-1'de gösterilmiştir.

PROGRAM 10-1 B-ağacı Bildirimi

```
123456789 /* ===== B-Ağacı.h =====Bu ba
şlık dosyası AVLAğacı soyut veri türü için fonksiyonları içerir.

Yazarı:
Tarih:

*/
#include <stdlib.h>
#include <stdbool.h>

1011121314151617181920212223 // ===== SÜREKLİLER & MAKROLAR =====
const int SIRA = 5;
const int MIN_GİRİŞLER = (((SIRA + 1) / 2) - 1);

// ===== YAPILAR =====
struct node;

typedef yapısı
{
    void*      dataPtr;
    struct node* rightPtr;
} GİRİŞ;

typedef yapı düğümü
```

devam ediyor

PROGRAM 10-1B-ağacı Bildirimi (devam ediyor)

```

242526272829303132333435363738394041424344454647484950515253545556575859606162636465666768697071
{
    struct node* ilkPtr;
    int      numGiriş;
    GİRİŞ    girişler[ORDER - 1];
} DÜĞÜM;

typedef yapısı
{
    int sayım;
    NODE* kök;
    int (*karşılaştır) (void* argu1, void* argu2);
} BTREE;

// ===== Prototip Bildirimleri =====

// Kullanıcı arayüzleri
BTREE* BTree_Oluştur
    (int (*karşılaştır) (void* argüman1, void* argüman2));
void BTree_Dolaş
    (BTREE* ağaç, void (*işlem) (void* veriPtr));
BTREE* BTree_YokEt (BTREE* ağaç);
void BTree_Ekle (BTREE* ağaç, void* veriGirişPtr);
bool BTree_Sil (BTREE* ağaç, void* silKey);
void* BTree_Ara (BTREE* ağaç, void* veriPtr);
bool BTree_Boş (BTREE* ağaç);
bool BTree_Dolu (BTREE* ağaç);
int BTree_Sayı (BTREE* ağaç);

// Dahili BTree fonksiyonları
static void* _ara
    (BTREE* ağaç, void* hedefPtr,
     NODE* kök);
static int _aramaDüğümü
    (BTREE* ağaç, NODE* düğümPtr,
     void* hedef);
static bool _sil
    (BTREE* ağaç, NODE* kök,
     void* silKeyPtr, bool* başarı);
static bool _ekle
    (BTREE* ağaç, NODE* kök,
     void* veriGirişPtr, ENTRY* yukarıGiriş);
static void _gezin
    (DÜĞÜM* kökü,
     void (*process)(void* dataPtr));
static void _splitNode
    (NODE* root, int entryNdx,
     int compResult, ENTRY* upEntry);
static void _insertEntry

```

devam ediyor

PROGRAM 10-1B-ağacı Bildirimi (devam ediyor)

72737475767778798081828384858687888990	<pre> (NODE* root, int entryNdx, ENTRY upEntry); static bool _deleteEntry (NODE* düğümü, int entryNdx); static bool _deleteMid (NODE* root, int entryNdx, NODE* leftPtr); statik bool _reFlow (NODE* kök, int girişNdx); statik void _borrowLeft (NODE* kök, int girişNdx, NODE* solAğaç, NODE* sağAğaç); statik void _borrowRight (NODE* kök, int girişNdx, NODE* solAğaç, NODE* sağAğaç); statik void _combine (NODE* kök, int girişNdx, NODE* solAğaç, NODE* sağAğaç); statik void _destroy (NODE* kök); </pre>
--	--

Algoritmalar

Bu bölümde bir B-ağacı için temel algoritmaları uyguluyoruz.

B-ağaç Arama

Arama B-ağacı fonksiyonu, bir ADT arayüzüfonksiyonu ve dahili bir özyi nelemeli arama fonksiyonu gerektirdiği için ilginçtir. Çağırın fonksiyon unbakış açısından, sağlaması gereken tek şey ağaç ve bulunacak veridir .Pseudocode algoritması anahtarın düğüm adresinive giriş indeksini dö nerken, ADT'de veri işaretçisini döndürüyoruz. (Uygulamanındüğüm ya pısına erişimi olmadığını unutmayın.) Ancak dahili olarak, herhangi bir a nda hangi ağaç düğümünü işlediğimizi bilmemiz gerekiyor.ADT arayüzü için kod Program 10-2'de gösterilmiştir.

PROGRAM 10-2B-ağacı Arama Uygulama Arayüzü

123456789	<pre> /* ===== BTree_Search ===== Ağaçta istenen anahtarı içeren düğümü arar ve verilerine işaretçi döner. Ön ağaç oluşturulmuş (null olabilir) targetPtr, b ulunacak anahtarı içeren veri yapısına işaretçidir Son ağaç arandı ve veri işaretçisi döndürüldü Veri işaretçisine işaretçi döner */ void* BTree_Search (BTREE* ağaç, void* targetPtr) { </pre>
1011	

devam ediyor

PROGRAM 10-2B-ağaç Arama Uygulama Arayüzü (devamı)

```

12131415161718 // İfadeler
    if (ağaç->kök)
        geri dön _arama
            (ağaç, hedefPtr, ağaç->kök);
    else
        NULL döndür;
} // BTree_Search

```

İç Arama Fonksiyonu

İç arama uygulaması Program 10-3'te gösterilmiştir. Bu, Algoritma 10-1 4 ile paraleldir.

PROGRAM 10-3İç Arama Fonksiyonu

```

123456789 /* ===== _search =====
Aranan anahtarı içeren düğüm için ağacı ara
ve verilerini çağıran işleve döndür.
    Önce BTree_Search ağacı, targetPtr, kök geçirir target
    Ptr, bulunması gereken anahtarı içeren veri yapı
    sına işaretçidir
    Sonra ağaç arandı ve veri işaretçisi döndürüldü
    Eşleşen düğümün adresi döndürüldü
    Bulunamazsa, NULL döndürülür
*/
void* _search (BTREE* ağaç, void* targetPtr, NO
               DE* kök)
{
// Yerel Tanımlar
    int girişNo;

// İfadeler
    if (!kök)
        NULL'u döndür;

    eğer (ağaç->karşılaştıracı(targetPtr,
                                kök->girişler[0].veriPtr) < 0)
        return _arama (ağaç,
                        hedefPtr,
                        kök->ilkPtr);

    girişNo = kök->numGirişler - 1;
    while (ağaç->karşılaştıracı(targetPtr,
                                kök->girişler[girişNo].veriPtr) < 0)
        girişNo--;
    eğer (ağaç->karşılaştıracı(targetPtr,
                                kök->girişler[girişNo].veriPtr) == 0)
        return (kök->girişler[girişNo].veriPtr);
}
101112131415161718192021222324252627282930313233

```

devam ediyor

PROGRAM 10-3 Dahili Arama Fonksiyonu (devam ediyor)

34353637	<pre> return (_search (ağaç, targetPtr, root->girişler[girişNo].rightPtr)); } // _search </pre>
----------	--

Program 10-3 Analizi Öncelikle, fonksiyonun adını not edin. Bir alt çizgi ile başlar. ADT'yi kullanan bir programcının, ADT fonksiyonlarımızdan biriyle aynı adı kullanmaya karar verebileceğini varsaymak makuldür. Bu nedenle, tekrarları önlemek için, tüm dahili adlarımızı alt çizgi ile öne alıyoruz.

B-ağaç yapısını almamız gerekiyor çünkü kullanıcı tarafından yazılan karşılaştırma işlevi ni içeriyor. Karşılaştırma işlevini ilk olarak 21. ifadede ve tekrar 28. ve 31. ifadelere kullanıyoruz. Bu ifadeleri dikkatlice inceleyin. Eğer bir işlev geçişi ile ilgili bilgi sahibi değilseniz, 1. Bölümdeki kavramı gözden geçirmek isteyebilirsiniz.

Arama, anahtar işaretçisinde bulunan arama argümanının düğümdeki ilk girişten daha küçük olup olmadığını test ederek başlar. Eğer küçükse, ilk işaretçiyi sola takip ederiz. Eğer değilse, eşleşen girişi veya aramaya devam etmek için doğru sağ alt ağacı bulmak üzere giriş dizisini aramalıyız. Kullandığımız giriş dizisi araması, sıralı liste aramasının bir uyarlamasıdır (bkz. Bölüm 13). Ana fark, son girişten başlayıp ileri doğru çalışmasıdır. Geriye doğru arıyoruz çünkü her giriş, mevcut girişin anahtarından büyük veya ona eşit olan ve bir sonraki girişin anahtarından küçük olan verilerle bir alt ağaca işaret eder. Başlangıçtan arama yaparsak, hedefimizi aşar ve geri dönmek zorunda kalırız. Sonundan arama yaparak, hedefe eşit veya daha küçük bir giriş bulduğumuzda, aradığımız hedefi içeren alt ağaç işaretçisini de bulmuş oluruz.

B-ağacı Geçişi

B-ağacı geçişi birkaç nedenle ilginçtir. Öncelikle, bir kez daha iki işleve ihtiyacımız var: biri ADT arayüzü için ve diğeri iç geçiş için. İkincisi, her düğümü işlemek için kullanılacak işlevin adını geçmemiz gerekiyor. İşlevin adresini parametre olarak almayı da içeren kullanıcı arayüzü kodu, Program 10-4'te gösterilmiştir.

PROGRAM 10-4 B-ağacı Geçiş Uygulama Arayüzü

123456789	<pre> /* ===== BTree_Geçiş ===== Ağacı sıralı geçiş kullanarak işleyin. Önce Ağaç oluşturulmuştur (null olabilir) ağaç B-Ağacına işaretçidir geçişte düğümleri "ziyaret" etmek için kullanılan işlem Sonra Düğümler LNR sırasına göre işlendi */ void BTree_Geçiş (BTREE* ağaç, void (*işlem) (void* veriPtr)) </pre>
-----------	--

devam ediyor

PROGRAM 10-4B-ağacı Geçiş Uygulama Arayüzü (devamı)

```

10112131415
{
// İfadeler
if (ağaç->kök)
    _traverse (ağaç->kök, süreç);
return;
} // BTree_Traverse sonu

```

İçsel Gezinme Fonksiyonu

Geçiş mantığındaki bir diğer ilginç nokta, ağacı gezerken, öncelikli geçişin bir varyasyonunu kullanarak, her girişi debloke etmemiz gerektiğidir. Ekleme mantığını 10-13. Algoritma, "B-ağacı Gezinmesi" bölümünde inceledik. Kod 10-5. Programda gösterilmiştir.

PROGRAM 10-5 Dahili Gezinme Fonksiyonu

```

123456789
/* ===== _traverse =====
Ağaç, sıralı gezinti kullanılarak gezilir. "İşlemek" için
düğümü, gezinti çağrıldığında geçirilen fonksiyonu kullanarak
Önce ağaç BTree_Traversal'da doğrulandı
root, B-Tree düğümüne işaretçidir
process, bir girişi işlemek için fonksiyondur
Sonra Tüm düğümler işlendi
*/
void _traverse (NODE* kökü,
                void (*işlem) (void* dataPtr))
{
// Yerel Tanımlar
int taramaSayı;
NODE* işaretçi;

// İfadeler

taramaSayı = 0;
işaretçi = root->firstPtr; while (taramaSa
yısı <= root->numEntries)
{
// Alt ağaç için test
if (ptr)
    _traverse (ptr, process);

// Alt ağaç işlendi -- bir sonraki girişi al
if (scanCount < root->numEntries)

```

devam ediyor

PROGRAM 10-5 Dahili Gezinme Fonksiyonu (devamı)

```

2930313233343536
{
    işlem (root->entries[scanCount].dataPtr);
    ptr    = root->entries[scanCount].rightPtr;
} // eğer scanCount
scanCount++;
} // eğer
geri dön;
} // _gezinme

```

B-ağacı Ekle

Ekleme mantığı birkaç algoritma gerektirir. Bu bölümde bunlardan ikisi hariç hepsini geliştiriyoruz. Ekleme, eşleşen düğümün indeks girişini döndüren bir arama gerektirir. Mantık, Program 10-3, “Dahili Arama Fonksiyonu” için gördüğümüzle aynı olduğundan, burada geliştirmiyoruz. Geliştirmedığımız ikinci fonksiyon, giriş dizisine bir giriş ekler.

Uygulama, Algoritma 10-1'de gösterilen tasarıma yakından benzemektedir, “B-ağacı Ekle.” Ana fark, kökün geri döndürülmemesidir—ADT y apısında gizlenmiştir. ADT arayüz fonksiyonu Program 10-6'da gösterilmiştir.

PROGRAM 10-6 B-ağacı Ekle Uygulama Arayüzü

```

123456789
/* ===== BTree_Ekle =====
   Bu fonksiyon yeni verileri ağaca ekler.
   Önce ağaç geçerli B-Ağacı yapısına işaret eder
   Sonra veri eklendi veya bellek O/F ise iptal edilir
*/
void BTree_Ekle (BTREE* ağaç, void* veriInPtr)
{
    // Yerel Tanımlar
    bool dahaUzun;
    NODE* yeniPtr;
    ENTRY yukarıGiriş;

    // İfadeler
    if (ağaç->kök == NULL)
        // Boş Ağaç. İlk düğümü ekle   if (yeniPtr = (NODE*)malloc(sizeof (NODE))
        )
        {
            yeniPtr->ilkPtr    = NULL;
            yeniPtr->numGirişler = 1;
            yeniPtr->girişler[0].veriPtr = veriInPtr;
            yeniPtr->girişler[0].sağPtr = NULL;
            ağaç->kök        = yeniPtr;
            (ağaç->sayı)++;

```

devam ediyor

PROGRAM 10-6B-ağaç Ekleme Uygulama Arayüzü (devam ediyor)

```

242526272829303132333435363738394041424344454647484950515253545556

        for (int i = 1; i < ORDER - 1; i++)    {

            yeniPtr->girişler[i].dataPtr = NULL;
            yeniPtr->girişler[i].rightPtr = NULL;
        } // for *

        geri dönmek;
        } // if malloc

    else
        printf("Hata 100 BTree_Insert\n"),          exit (100);

    uzun = _insert (ağaç, ağaç->kök,
                    dataInPtr, &upEntry);

    if (uzun)
    {
        // Ağaç büyüdü. Yeni kök oluştur
        newPtr = (NODE*)malloc(sizeof(NODE));
        if (newPtr)
        {
            newPtr->entries[0] = upEntry;
            newPtr->firstPtr = ağaç->root;
            newPtr->numEntries = 1;
            ağaç->kök = yeniPtr;
        } // if yeniPtr

        yoksa
            printf("Taşma hatası 101\n"),          exit (
                100);

        } // eğer daha uzun

    (tree->count)++;
    return;
    // BTree_Ekle

```

Program 10-6 Analizi Ekleme ADT arayüzü, şimdiye kadar gördüğümüzden daha karmaşıktır. Dört farklı işlem gerçekleştirir. İlk olarak, null bir ağaca eklemeyi yönetir. Bu kod, 14 ile 34 arasındaki ifadelerde gösterilmiştir.

İkinci işlem, ilk sonrasında tüm düğümlerin eklenmesini içerir. Bu, içsel ekleme fonksiyonuna yapılan tek bir çağrıdır (ifade 36). Bu kod, hem sahte kodda hem de ADT'de aynıdır.

Yeni düğüm eklendikten sonra, ekleme arayüzü ADT'si yeni bir kökün oluşturulup oluşturulmayacağını belirlemelidir. İçsel ekleme fonksiyonu, ağacın yüksekliğinin artıp artmadığını belirten bir Boolean döndürür. Eğer artmışsa, yeni kök, 38'den 52'ye kadar olan ifadelerde oluşturulur. Yine, kod sahte kodla paraleldir.

Son olarak, ekleme başarılı olduysa, B-ağaç sayısı güncellenmelidir. Eğer B-ağaçta tutulmuş en fazla eleman sayısı gibi diğer meta-datalar gerekli olsaydı, bunlar da burada işlenirdi. Bu mantık sahte kodda gerekli değildir.

İçsel Ekleme Fonksiyonu

Ekleme tasarımı Şekil 10-6'da, "B-ağacı Ekleme Tasarımı" gösterilmiştir. Bu bölümd eki fonksiyonları incelemeden önce gözden geçirmek için bir dakikanızı ayırmak is teyebilirsiniz. Fonksiyonlar oldukça uzun olmasına rağmen, Tasarım 10-2, "B-ağacı Düğüm Ekle" ile yakından takip edilmektedir ve takip etmesi kolay olmalıdır. İçsel ekleme fonksiyonunun kodu Program 10-7'de gösterilmiştir.

PROGRAM 10-7İçsel Ekleme Fonksiyonu

123456789	<pre> /* ===== _ekle =====Bu fonksi yon, yeni veriyi B-Ağacındaki bir yaprak düğüme eklemek için özyi nelemeyi kullanır. Ön Uygulama BTree_Insert'i çağırdı, bu, kök ve veri işaretçilerini geçirir Sonra Veriler eklenmiştir Daha yüksek boolean döndür */ */ bool _ekle (BTREE* ağaç, NODE* kök, void* veriGirişi Ptr, ENTRY* yukarıGirişi) { // Yerel Bildirimler int compSonucu; int girdiNdx; bool dahaYüksek; NODE* altAğaçPtr; // İfadeler if (!kök) { // Yaprak bulundu -- yeni girdi oluştur (*yukarıGirdi).veriPtr = veriInPtr; (*yukarıGirdi).sağPtr = NULL; doğruyu döndür; // ağaç daha yüksek } // eğer NULL ağaç girdiNdx = _düğümAra (ağaç, kök, veriInPtr); compSonucu = ağaç->karşılaştırm (veriInPtr, root->entries[entryNdx].dataPtr); if (entryNdx <= 0 && compResult < 0) // düğümün ilk alt ağacında subtreePtr = kök->ilkPtr; else // girişin sağ alt ağacında </pre>
1011121314151617181920212223242526272829303132333435	

devam ediyor

PROGRAM 10-7 Dahili Ekleme Fonksiyonu (devamı)

```

3637383940414243444546474849505152535455565758596061626364
    subtreePtr = root->entries[entryNdx].rightPtr;
    taller = _insert (tree,      subtreePtr,
                      dataInPtr, upEntry);

    // Giriş eklendi -- ağaçtan çık

    {
        if (root->numEntries >= ORDER - 1)    {

            // Yeni düğüm oluşturulması gerekiyor
            _splitNode (root,      entryNdx,
                        compResult, upEntry);

            taller = true;
        } // düğüm dolu

    yoksa
    {
        if (compResult >= 0)
            // Yeni veri >= mevcut giriş -- sonrasına ekle
            _insertEntry(root, entryNdx + 1, *upEntry);
        else
            // Mevcut girişten önce ekle
            _insertEntry(root, entryNdx, *upEntry);
            (root->numEntries)++;
            dahaUzun = false;
        } //
    aksi takdirde } // eğer

    er dahaUzun return dahaUzun;
} // _insert

```

Program 10-7 Analizi Fonksiyonun dört ana bölümü vardır.

- (1) Eğer kök null ise, bu bir temel durumdur, yeni data için yaprak düğümü bulmuştuzdur. Bu noktada yeni girişi oluştururuz ama eklemeyiz.
- (2) Eğer bir yaprakta değilsek, hedefi içeren girişi bulmak için düğümü ararız. Eğer giriş 0 ise, alt ağaç ya düğümün ilk işaretçisinin alt ağacı ya da girişin sol alt ağacı olabilir. Hangisi olduğunu belirledikten sonra, ekleme düğümünü özyinelemeli olarak çağırırız.
- (3) Temel durumdan döndükten sonra, ağacın daha yüksek olup olmadığını veya mevcut düğümde yeni giriş için yer olup olmadığını belirleriz. Eğer ağaç daha yüksekse, düğümü bölmemiz gerekir.
- (4) Düğüm daha yüksek değilse, yeni girişi mevcut girişin ilişkisine bağlı olarak ya öncesine ya da sonrasına ekliyoruz.

İçsel Bölme Düğümü Fonksiyonu

Düğümü bölme kodu Program 10-8'de gösterilmiştir.

PROGRAM 10-8 Dahili Bölme Düzümü Fonksiyonu

```

123456789 /* ===== _splitNode =====
Düğüm dolduğunda düğümü böler.
Önce düğüm taşmış--Düğümü böl entryN
dx yeni verinin indeks konumu compResult yeni
veri < veya > entryNdx anahtar upEntry eklene
cek veriye işaretçi Sonra düğüm bölündü ve upE
ntry, ebeveyne eklenmesi gereken girişi içerir

veya- Bellek taşması durumunda program durduruldu
*/
void _splitNode (NODE* düğümü, int entryNdx,
int compResult, ENTRY* upEntry)
{
// Yerel Tanımlar
int Ndx'ten;
int'den Ndx'e;
DÜĞÜM* sağPtr;

// İfadeler
rightPtr = (NODE*)malloc(sizeof (NODE));
if (!rightPtr)
printf("Taşma Hatası 101 _splitNode\`a"), exit (100);

// Sağ alt ağaç düğümünü oluştur
if (entryNdx < MIN_ENTRIES)
fromNdx = MIN_ENTRIES;
aksi takdirde
fromNdx = MIN_ENTRIES + 1;
toNdx = 0;
rightPtr->numEntries = düğüm->numEntries - fromNdx;
while (fromNdx < düğüm->numEntries)
rightPtr->entries[toNdx++]
= node->entries[fromNdx++];
node->numEntries = node->numEntries
- rightPtr->numEntries;

// Yeni giriş ekle
eğer (entryNdx < MIN_ENTRIES)
{
eğer (compResult < 0)
_insertEntry (node, entryNdx, *upEntry); aksi takdir
de
_insertEntry (node, entryNdx + 1, *upEntry);
} // eğer

```

devam ediyor

PROGRAM 10-8 Dahili Böl Split Düşüm Fonksiyonu (devam ediyor)

```

47484950515253545556575859606162
aksi takdirde
{
    _insertEntry (rightPtr,
                  entryNdx - MIN_ENTRIES,
                  *upEntry);

    (rightPtr->numEntries)++;
    (node->numEntries)--;
} // aksi takdirde

upEntry->dataPtr=node->girişler[MIN_ENTRIES].dataPtr;
upEntry->rightPtr = rightPtr;
rightPtr->firstPtr
                = düğüm->girişler[MIN_ENTRIES].rightPtr;

return;
} // _splitNode

```

Program 10-8 Analizi Bu algoritmada bazı düşünce gerektiren birkaç ilginç mantıksal unsur var onları incelerken. Örneğin, sağ düğüme **upEntry** eklediğimizde (ifadeler 49'dan 53'e), düğümün giriş sayısına bir eklememiz gerekiyor, ancak sola eklediğimizde (ifadeler 42'den 45'e) bunu yapmıyoruz. Sebebi, sol düğümün ebeveynine eklenmesi gereken medyan girişi içermesidir ve bu nedenle bir eklememize gerek yok. Eğer ekleseydik, medyan girişi sildikten sonra onu çıkarmamız gerekecekti.

Aynı şekilde, sol düğüme bir giriş eklediğimizde, yeni veri anahtarının giriş anahtarında n küçük olup olmadığını bilmemiz gerekir. Eğer küçükse, girişi eklemek için giriş indeksini geçiririz; değilse, giriş indeksini bir artırarak geçiririz. Bu farkın nedenini tam olarak anlamak için, iki örnek oluşturmanız ve ekleme mantığını dikkatlice takip etmeniz gerekir.

B-ağacı Silme

B-ağacından silme, eklemekten çok daha fazla iş gerektirebilir. Bu bölümde, silme için gereken yedi işlevden beşini tanımlıyoruz. Daha önce birkaç kez gördüğümüz gibi, silme hem bir ADT arayüz işlevi hem de bir iç el özyinelemeli işlev gerektirir. Ancak, silme arayüz işlevi, ekleme için gördüğümüzden oldukça daha basittir. Kod Program 10-9'da gösterilmiştir.

PROGRAM 10-9 B-ağacı Silme Uygulama Arayüzü

```

123456 /* ===== BTree_Sil =====
        Anahtar hedef ile B-Ağacından girişi sil
        Önce  ağaç başlatılmalıdır. Null ağaç kabul edilebilir
              dltKey silinecek anahtara işaretçidir
        Sonra  Düğüm girişi silindi ve veri alanı serbest bırakıldı
              -veya- Bir hata kodu döndürüldü

```

devam ediyor

PROGRAM 10-9 B-ağaç Silme Uygulama Arayüzü (devam ediyor)

```

789      Başarı (true) veya Bulunamadı (false) döndür
      */
      bool BTree_Delete (BTREE* ağaç, void* dltKey)
      {
      // Yerel Tanımlar
          bool success;
          NODE* dltPtr;

      // İfadeler
          if (!tree->root)
              return false;

          _delete (tree,
                  tree->root,
                  dltKey,
                  &başarı);

          eğer (başarılı)
          {
              (ağaç->sayı)--;
              if (ağaç->kök->numEntries == 0)
              {
                  dltPtr = ağaç->kök;
                  ağaç->kök = ağaç->kök->ilkPtr;
                  ücretsiz (dltPtr);
              } // kök boş
          } // başarı
          return success;
      }, // BTree_Delete

```

Program 10-9 Analizi Özyinelemeli silme fonksiyonu, bir düğümün minimumun altında olup olmadığını belirlemek için bir taşıma Boolean'ı kullanırken, kökün minimumu yoktur. Bu nedenle, bu fonksiyonda tek endişemiz kökteki tüm girişlerin silinmiş olabileceğidir. Bu durumu 27 ile 31. ifadelerde ele alıyoruz.

İçsel Silme Fonksiyonu

İçsel silme fonksiyonu, silinecek girişi bulmak için B-ağacını arar. Eğer bir yaprak düğümde bulunursa, silme işlemi basittir. Ancak, bir iç düğümde bulunursa, bir yaprakta bir yedek bulunmalıdır. Daha önce açıkladığımız gibi, silinecek girişin hemen öncüsünü bulmak için sol alt ağacı ararız. Yerine koyma yapıldıktan sonra, yaprakta olduğunu bildiğimiz girişi sileriz. Pseudocode tasarımı Algoritma 10-6'da gösterilmiştir. Kod Program 10-10'da gösterilmiştir.

PROGRAM 10-10 Dahili Silme Fonksiyonu

```

123456789 /* ===== _delete =====
           Anahtar dltKey ile B-Tree'den girişi sil
           Önce  ağaç başlatılmalı--Null ağaç OK
                   kök, ağaç veya alt ağacın işaretçisi
                   dltKey, silinecek girişin anahtarıdır
                   success, girişin silinip silinmediğini gösterir
           Sonra  düğüm silinir ve alanı geri dönüştürülür
                   -veya- anahtar bulunamazsa, ağaç değişmez.
                   başarı doğru / yanlış
           Alt akış doğru / yanlış
*/
1011213141516171819202122232425262728293031323334353637383940414243444546
bool _sil (BTREE* ağaç,      NODE* kök,
          void* dltAnahtarPtr, bool* başarı)
{
    // Yerel Tanımlar
    NODE* solPtr;
    NODE* altAğaçPtr;
    int girişIndeks;
    int altAkış;

    // İfadeler
    if (!kök)
    {
        *başarı = yanlış;
        return yanlış;
    } // null ağaç

    entryNdx = _searchNode (ağaç, kök, dltKeyPtr);
    if (ağaç->karşılaştırdltKeyPtr,
        kök->girişler[entryNdx].dataPtr) == 0)
    {
        // silinecek girişi bulduk
        *success = true;
        if (root->entries[entryNdx].rightPtr == NULL)
            // giriş bir yaprak düğüm
            underflow = _deleteEntry (root, entryNdx); else

        // giriş bir iç düğümde
        {
            if (entryNdx > 0)
                leftPtr =
                    kök->girişler[entryNdx - 1].rightPtr;
            else
                leftPtr = root->firstPtr;
            alt akış = _deleteMid
                (kök, girişNdx, leftPtr);

```

devam ediyor

PROGRAM 10-10 Dahili Silme Fonksiyonu (devam ediyor)

```

47484950515253545556575859606162636465666768
        if (aşırı akma)
            aşırı akma = _reFlow (kök, girişİndeksi);
        } // aksi takdirde dahili düğüm
    } // aksi takdirde giriş bulundu

    aksi takdirde
    {
        if (ağaç->karşılaştır (silKeyPtr,
            kök->girişler[0].veriPtr) < 0)
            // silme anahtarı < ilk giriş
            altAğaçPtr = kök->ilkPtr;
        else
            // sağ alt ağaçta anahtarı sil
            subTreePtr = root->entries[entryNdx].rightPtr;

        underflow = _delete (tree,          subTreePtr,
                                dltKeyPtr, success);

        eğer (aşırı alt)
            aşırı alt = _reFlow (kök, girişİndeksi);
        } // aksi takdirde bulunamadı *

        return aşırı alt;
    } // _sil

```

Dahili Silme Orta Fonksiyonu

Dahili silme fonksiyonu tarafından çağrılan, delete middle (Program 10-11) s ol alt ağaçtaki öncül düğümü bulur ve silinen düğümün yerine kökteki düğüm mü yerleştirir.

PROGRAM 10-11 Dahili Silme Orta Fonksiyonu

```

123456789
/* ===== _silOrt =====
   B-Tree'deki iç düğümden girişi siler
   Önce Ağaç başlatıldı--null ağaç OK
        düğüm silinecek düğüm verisine işaret eder
        altDüğümPtr kökün alt ağacına işaret eden işaretçidir
        girişİndeksi silinecek giriş
   Sonra öncülün verisi silme verisini değiştirir
        öncül ağaçtan silinir
   Aşırı alt doğru / yanlış döndür
*/
10111213141516
bool _silOrt (DÜĞÜM* kök,          i
                nt girişİndeksi,
                DÜĞÜM* altDüğümPtr)
{
    // Yerel Tanımlar
    int dltNdx;

```

devam ediyor

PROGRAM 10-11 İçsel Silme Orta Fonksiyonu (devam ediyor)

```

171819202122232425262728293031323334353637383940
int sağNdx;
bool altAkış;

// İfadeler
if (altAğaçPtr->ilkPtr == NULL)
{
    // yaprak bulundu. Verileri değiştir ve yaprağı sil
    silNdx = altAğaçPtr->numGirişler - 1;
    kök->girişler[girişNdx].veriPtr =
        altAğaçPtr->girişler[silNdx].veriPtr;
    --altAğaçPtr->numGirişler;
    altAkış = altAğaçPtr->numGirişler < MIN_GİRİŞLER;
} // eğer yaprak

aksi takdirde
{
    // Bulunamadı. Öncel için sağa geçin
    sağNdx = altAğaçPtr->numGirişler - 1;
    altAkış = _ortaSil (kök, girişNdx,
        altAğaçPtr->girişler[sağNdx].sağPtr);

    if (altAkış)
        underflow = _reFlow (altAğaçPtr, sağNdx);
} // aksi takdirde sağa geç

return aşırı alt;
} // _deleteMid

```

Program 10-11 Analizi Orta silme, B-ağaç ADT'sindeki daha karmaşık mantıklardan birini içerir.

34 ile 38 arasındaki ifadelerdeki mantığı dikkatlice inceleyin. Yaprak bulana kadar orta silmeyi özinelemeli olarak çağırıyoruz. Döndüğümüzde, yedek girişi sildiğimizi biliyoruz. Şimdi ağacın dışına çıkmamız gerekiyor, her düğümde aşırı yüklenip yüklenmediğimizi test ediyoruz. Eğer aşırı yüklenmişsek, düğümleri yeniden akıtmalıyız ve ardından ağacın yukarısına doğru ilerlemeliyiz silinecek veriyi ilk içeren düğüme ulaşana kadar. O noktada silmeye geri dönüyoruz, bu da köke ulaşana kadar ağacın dışına çıkmaya devam ediyor.

İçsel Yeniden Akış Fonksiyonu

Bir alt akış meydana geldiğinde, düğümlerin geçerli olduğundan emin olmak için yeniden akış yapmalıyız. Yeniden akış önce sağdan ödünc olarak dengelemeye çalışır, ardından soldan ödünc alma ya çalışır; her ikisi de başarısız olursa, kardeşleri birleştirir. Kod Program 10-12'de gösterilmiştir.

PROGRAM 10-12 İçsel Yeniden Akış Fonksiyonu

```

12345
/* ===== _reFlow =====
Bir alt ağaçta köke doğru bir underflow meydana geldi.
Dengeleyerek veya birleştirerek düzeltin.
Pre kök, underflow ağacı/alt ağacına işaret eder
entryNdx, underflow alt ağacının ebeveynidir

```

devam ediyor

PROGRAM 10-12 Dahili Yeniden Akış Fonksiyonu (devam ediyor)

6789	Post Alt akış düzeltildi Alt akış doğru / yanlış döndür
101112131415161718192021222324252627282930313233343536373839404142434445464748	<pre> */ bool _reFlow (NODE* kökü, int girişNdx) { // Yerel Tanımlar NODE* leftTreePtr; NODE* rightTreePtr; bool altAkış; // İfadeler if (entryNdx == 0) leftTreePtr = root->firstPtr; else leftTreePtr = root->entries[entryNdx - 1].rightPtr; rightTreePtr = root->entries[entryNdx].rightPtr; // Önce borç almayı dene if (rightTreePtr->numEntries > MIN_ENTRIES) { _borrowRight (root, entryNdx, leftTreePtr, rightTreePtr); underflow = false; } // sağdan ödünç alma aksi takdirde { // Sağdan ödünç alınamaz--soldan dene if (leftTreePtr->numEntries > MIN_ENTRIES) { _borrowLeft (root, entryNdx, leftTreePtr, rightTreePtr); underflow = false; } // soldan ödünç alma * başka { // Ödünç alınamaz. Düğümleeri birleştirmek zorundasın. _combine (root, entryNdx, leftTreePtr, rightTreePtr); underflow = (root->numEntries < MIN_ENTRIES); } // aksi takdirde birleştir } // aksi takdirde sağdan ödünç al return underflow; } // _reFlow </pre>

Dahili Sol veya Sağ Borç Alma

Alt akış durumunda, önce ağacı dengelemek için sağ kardeşten bir düğüm b orç almaya çalışırız. Eğer bu işe yaramazsa, soldan borç almaya çalışırız. Bu

iki fonksiyon ayna mantığıdır; yön dışında aynıdırlar. Sağdan ödünç alma mantığı Program 10-13'te gösterilmiştir.

PROGRAM 10-13İçsel Sağdan Ödünç Alma Fonksiyonu

```

123456789
/* ===== _borrowRight =====
   Kök sol alt ağacında underflow. Sağdan ödünç al
   ve döndür.
   Önce kök, underflow düğümünün ebeveyn düğümüdür
       entryNdx ebeveyn girişi
       leftTreePtr underflow olan alt ağaçtır
       rightTreePtr ekstra girişe sahip alt ağaçtır
   Sonra Underflow düzeltili
*/
void _borrowRight (NODE* kökü,
                  int entryNdx,
                  DÜĞÜM* leftTreePtr,
                  NODE* rightTreePtr)
{
    // Yerel Tanımlar
    int toNdx;
    int değiştirici;

    // İfadeler
    // Ebeveyn ve alt ağaç işaretçisini sol ağaçta hareket ettir
    toNdx = leftTreePtr->numEntries;
    leftTreePtr->girişler[toNdx].dataPtr
        = root->girişler[entryNdx].dataPtr;
    leftTreePtr->girişler[toNdx].rightPtr
        = rightTreePtr->firstPtr;
    ++leftTreePtr->numEntries;

    // Sağ veriyi ebeveyne taşı
    root->entries[entryNdx].dataPtr
        = rightTreePtr->entries[0].dataPtr;

    // Sağ ağaç ilk işaretçisini ayarla. Girişleri sola kaydır
    rightTreePtr->firstPtr
        = rightTreePtr->girişler[0].rightPtr;
    kaydırıcı = 0;
    while (shifter < rightTreePtr->numEntries - 1) {

        rightTreePtr->girişler[kaydırıcı]
            = rightTreePtr->girişler[kaydırıcı + 1];
        ++kaydırıcı;
    } // while
    --rightTreePtr->numEntries;
    return;
} // _borrowRight

```

İçsel Birleştirme Düzümleri Fonksiyonu

Bir kardeşten düğüm alamıyorsa, iki düğümü birleştirmemiz gerekir. Düğümler i birleştirme mantığı Program 10-14'te gösterilmiştir.

PROGRAM 10-14İçsel Birleştirme Düzümleri Fonksiyonu

```

123456789 /* ===== _combine =====
Borç olarak düzeltilemeyen alt akış.
İki alt ağacı birleştir.
    Önceki kök, alt akış düğümünün ebeveynini içerir
        entryNdx ebeveyn girişi
            leftTreePtr & rightTreePtr alt ağaç ptr'leridir Sonra Ebeveyn ve alt a
                ğaçlar birleştirildi - sağ düğüm serbest bırakıldı
*/
void _combine (NODE* root,    int  entryNdx,
                NODE* leftTreePtr, NODE* rightTreePtr)
{
    // Yerel Tanımlar
    int'den Ndx'e;
    int Ndx'ten;
    int değiştirici;

    // İfadeler
    // Ebeveyni taşı ve sağ işaretçisini sağ ağaçtan ayarla
    toNdx = leftTreePtr->numEntries;
    leftTreePtr->entries[toNdx].dataPtr
        = root->entries[entryNdx].dataPtr;
    leftTreePtr->entries[toNdx].rightPtr
        = rightTreePtr->firstPtr;
    ++leftTreePtr->numEntries;
    --root->numEntries;

    // sağ ağaçtan sol ağaca veri taşı
    fromNdx = 0;
    Ndx++'a;
    while (fromNdx < rightTreePtr->numEntries)
        leftTreePtr->girişler[toNdx++]
            = rightTreePtr->girişler[fromNdx++];
    leftTreePtr->numEntries += rightTreePtr->numEntries;
    free (rightTreePtr);

    // Şimdi kök verilerini sola kaydır
    shifter = entryNdx;
    while (shifter < root->numEntries) {

        kök->girişler[kaydırıcı] =
            kök->girişler[kaydırıcı + 1];
        kaydırıcı++;
    } // while
    geri dönmek;
} // _combine

```

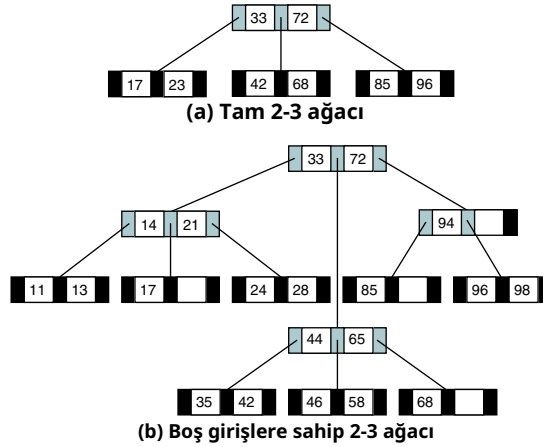
10.4 Basitleştirilmiş B-ağaçları

Bilgisayar bilimcileri, iki özel B-ağacına benzersiz isimler vermiştir:

2-3 ağaçları ve 2-3-4 ağaçları. Bunları kısaca tartışıyoruz. Her ikisi de iç arama ağaçları için iyi bir şekilde uygundur.

2-3 Ağacı

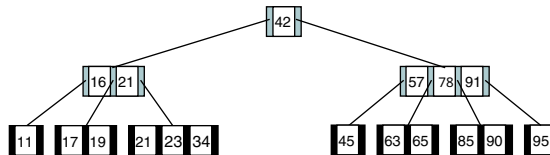
2-3 ağacı, 3. dereceden bir B-ağacıdır. Adını, her kök olmayan düğümün ya da iki ya da üç alt ağaca sahip olmasından alır (kök, sıfır, iki veya üç alt ağaç içerebilir). Şekil 10-18, iki 2-3 ağacı içermektedir. İlki tamdır; yani, yüksekliği için maksimum giriş sayısına sahiptir. İkincisi, iki katından fazla girişe sahiptir, ancak bazı girişler boştur. Ayrıca, alt ağaç 94'ün yalnızca iki torunu olduğunu da unutmayın.



ŞEKİL 10-182-3 Ağaçları

2-3-4 Ağacı

Şekil 10-19, 4. dereceden bir B-ağacı içermektedir. Bu tür bir ağaç bazen 2-3-4 ağacı olarak adlandırılır çünkü her düğüm iki, üç veya dört çocuğa sahip olabilir.



ŞEKİL 10-192-3-4 Ağaç

10.5 B-ağaç Varyasyonları

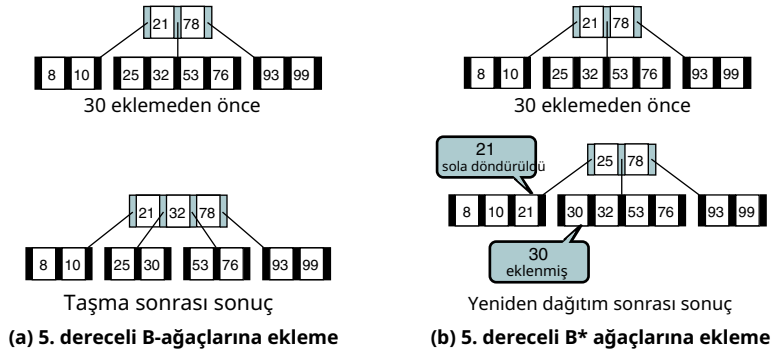
B-ağacı üzerinde iki popüler varyasyon vardır. Manipülasyonları bu metnin kapsamının ötesinde olsa da, varyasyonların farkında olmalısınız.

B*ağacı

B-ağacını büyük sayıda giriş depolamak için kullandığımızda, alan gereksinimleri %50'ye kadar boş giriş olabileceğinden aşırı hale gelebilir.

İlk varyasyon, B* ağacı, büyük ağaçlar için alan kullanımını ele alır. Her düğümün maksimum girişlerin en az yarısını içermesi yerine, minimum iki üç te bir olarak belirlenmiştir.²

B* ağacında, bir düğüm taşma yaptığında, hemen bölünmek yerine, düğümün k ardeşleri arasında veriler yeniden dağıtılır, bu da yeni bir düğüm oluşturmayı geciktirir. Bölünme, yalnızca tüm kardeşler dolu olduğunda gerçekleşir. Ayrıca, düğümler bölündüğünde, iki dolu kardeşten gelen veriler, iki dolu düğüm ve yeni bir düğüm arasında bölünür; sonuç olarak, üç düğüm de iki üçte bir dolu olur. Şekil 10-20, bir B* ağacında 5. dereceden bir düğüm taşma yaptığında yeniden dağıtımın nasıl ele alındığını göstermektedir.



ŞEKİL 10-20 B* ağacı Ekleme

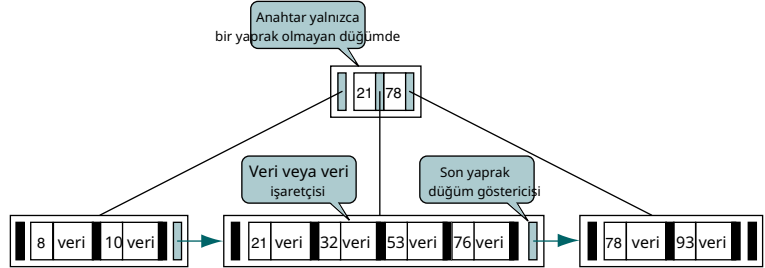
B+ ağacı

Büyük dosya sistemlerinde, verilerin hem rastgele hem de ardışık olarak işlenmesi gerekir. Bu durumlarda en popüler dosya organizasyon yöntemleri verileri rastgele işlemek için B-ağacını kullanır. Ancak, verilerin ardışık olarak işlenmesi gerektiğinde ağaç yapısında yukarı ve aşağı hareket etmek çok fazla işlem süresi alır. Bu verimsizlik, ikinci B-ağaç varyasyonu olan B+ ağacına yol açmıştır.

2. Kök yalnızca bir ağaçta bölündüğünde, iki alt ağacı yalnızca yarı doludur.

B-ağacı ile B+ ağacı arasında iki fark vardır:

1. Her veri girişi, aynı anahtarlara sahip iç düğümler olmasına rağmen , yaprak seviyesinde temsil edilmelidir. Çünkü iç düğümler yalnızca arama için kullanılır, genellikle veri içermezler.
2. Her yaprak düğümünün, sırayla bir sonraki yaprak düğümüne geçmek için kullanılan bir ek işaretçisi vardır. Bu yapı Şekil 10-21'de gösterilmiştir.



ŞEKİL 10-21 B+ ağacı

Verileri rastgele işlediğimizde, ağaç aramasını yalnızca bir yaprakta hedef veriyi bulacak şekilde değiştiriyoruz. Bu nedenle arama süresi biraz artar. Ancak, verileri ardışık olarak işlemek için, yalnızca en soldaki girişi buluyoruz ve ardından verileri, her düğümün bir dizi olduğu bir bağlı listeyi işler gibi işliyoruz.

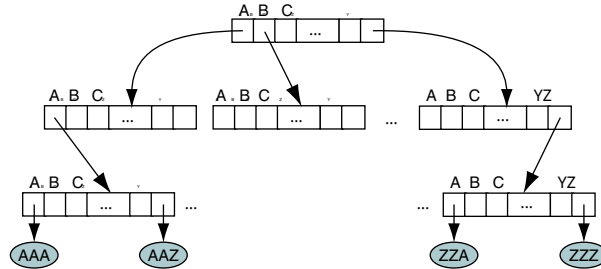
10.6 Sözcüksel Arama Ağacı

Bir ağacı, bir anahtarın tüm değerini kullanarak aramak yerine, anahtarın bir kelime veya sayısal olmayan bir tanımlayıcı (örneğin, bir telefon numarası) gibi karakterlerin bir dizisi olarak düşünebiliriz. Bir ağaçta yer aldığında , her düğüm, sözcüksel ağacın karakterlerinin alabileceği olası değerler için bir yer tutar. Örneğin, bir anahtar tam alfabeyle içerebiliyorsa, her düğümde alfabedeki her bir harf için 26 giriş bulunur. Bu, sözcüksel 26-ary ağaç olarak bilinir.

Her bir giriş, sözcüksel arama ağacı bir sonraki seviyeye işaret eden bir gösterici içerir. Ayrıca, 26-ary ağacının her düğümü 26 gösterici içerir; ilki *A* harfini, ikincisi *B* harfini ve son göstericiye kadar devam eder, bu da *Z* harfini temsil eder. İlk seviyedeki her harf, tam bir değer kümesine işaret etmelidir, bu nedenle ikinci seviye 26×26 girişi içerir; ilk seviyedeki her 26 harf için 26 girişten oluşan bir düğüm vardır. Benzer şekilde, üçüncü seviye de $26 \times 26 \times 26$ girişi içerir. Son olarak, gerçek anahtarları bir yaprakta saklarız.

Eğer bir anahtar üç harften oluşuyorsa, ağaçta en az üç seviye vardır. Eğer bir anahtar 10 harften oluşuyorsa, ağaçta 10 seviye vardır. Çünkü bir sözcüksel ağaç

birçok farklı anahtar içerir, en büyük kelime ağacının yüksekliğini belirler. Şekil 10-22, bir leksikal ağacı göstermektedir.



ŞEKİL 10-22 Leksikal Ağaç Yapısı

Denemeler

Leksikal m -ary ağacının sorunu, birkaç seviyeden sonra çok büyük hale gelmesidir.

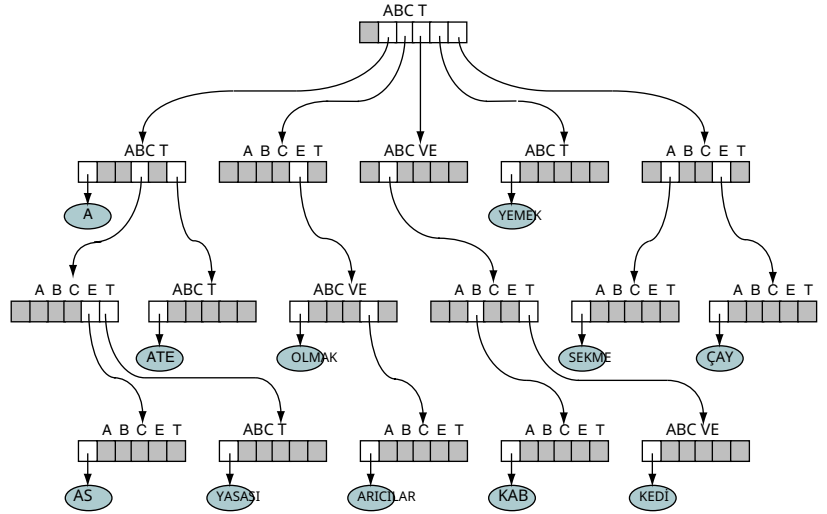
Bunu önlemek için, ağacı budarız; yani, ihtiyaç duyulmayan tüm dalları keseriz. Budanmış bir dalı null işaretçisi ile tanımlarız. Örneğin, eğer hiçbir anahtar X harfi ile başlamıyorsa, o zaman seviye 0'da X işaretçisi null'dır, böylece alt ağacını tamamen ortadan kaldırır. Benzer şekilde, Q harfinden sonra, tek geçerli harf U 'dur (çok az istisna ile ilgilenmeyeceğiz). Bu nedenle, Q dalındaki U harfi dışındaki tüm işaretçiler null olarak ayarlanır, böylece seviye 1'den aşağıya U alt ağacı dışındaki her şey ortadan kaldırılır. Ortaya çıkan yapı *trie* olarak adlandırılır (reTRIEval'ın kısaltmasıdır ve "try" olarak telaffuz edilir).

Örnek olarak, bir trie kullanarak bir yazım denetleyici oluşturalım. Bir yazım denetleyici bir sözlük değildir; yalnızca kelimeleri içerir ve tanımlarını içermez. Yazım denetleyicimizdeki her kelime, trie'deki bir giriştir, bir kelimenin doğru yazımıdır. Fikri göstermek için, yalnızca A, B, C, E ve T harflerini içeren birkaç kelimenin yazımını kontrol eden küçük bir yazım denetleyici trie oluşturalım.

Burada 5-ary bir trie'ye ihtiyacımız var çünkü bu kelimelerde toplam beş karakter var (A, B, C, E ve T). Trie dört seviyeye sahip olmalıdır çünkü her kelimenin en fazla üç karakteri vardır. Trie'miz Şekil 10-23'te gösterilmiştir.

Şekil 10-23, her kelimenin tam yazımını içerse de, A harfi için trie'nin üçüncü ve dördüncü seviyelerini kaldırdık çünkü A altındaki ilk seviyeden tamamen tanımlanabilir. Bu başka bir budama biçimidir. Benzer şekilde, EAT için üçüncü ve dördüncü seviyeleri kaldırdık çünkü E ile başlayan tek kelimedir.

Yazım denetleyicimizi CAB kelimesi için aramak üzere, trie'nin ilk seviyesini C harfinde kontrol ediyoruz. Null olmadığı için, ikinci seviyeye işaretçi takip ediyoruz, bu sefer A harfini kontrol ediyoruz. Yine, A 'da geçerli bir işaretçi var, bu yüzden CA 'yı temsil eden düğüme doğru takip ediyoruz ve B pozisyonunu kontrol ediyoruz. Bu noktada giriş işaretçisini kontrol ediyoruz ve CAB 'ı buluyoruz.



ŞEKİL 10-23Yazım Denetleyici Trie

Şimdi yazım denetleyicimizde *CTA* bulmaya çalışalım. İlk düğümde *C* konumunu kontrol ederek başlıyoruz ve *C* düğümüne geçerli bir işaretçi buluyoruz. Ancak, ikinci seviyedeki *T* konumunu kontrol ettiğimizde, işaretçinin null olduğunu görüyoruz. Ardından, giriş işaretçisini kontrol ediyoruz, bu da null, yani yazım denetleyicimizde geçerli bir kelime *CTA* olmadığını gösteriyor.

Trie Yapısı

Yukarıdaki tartışmadan, bir trie için yapıyı çıkarıyoruz. Her düğümün iki işaretçi türüne ihtiyacı vardır: biri trie'nin alt ağaçlarına ve diğeri veriye. Trie için alfabedeki her harf her düğümde temsil edilmelidir.

Bu, aşağıda gösterilen düğüm yapısını bize verir.

```
denemek
    entryPtr
    ltrPtrs
end trie
```

Trie Arama

Şekil 10-23'teki trie'mizi aramak için bir algoritma yazalım. İki parametreye ihtiyacımız var: biri sözlük trie'si için ve diğeri bakılacak kelime için. Kelime için basit bir dize ve sözlük için yukarıda gösterilen trie yapısını kullanıyoruz. Pseudocode, Algoritma 10-15'te gösterilmiştir.

ALGORİTMA 10-15Trie Arama

```

Algoritma searchTrie (sözlük, kelime)
Kelime için sözlük trie'sinde arama yapın.
  Ön sözlük, ABCET alfabesi ile geçerli bir trie'dir
  Sözlükte kelime varsa true, yoksa false döndür
1 kökü sözlüğe ayarla
2 ltrNdx'i 0 olarak ayarla
3 döngü (kök null değilse)
  1 eğer (kök girişi kelimeye eşitse)
    1 true döndür
  2 eğer biterse
  3 eğer (ltrNdx >= kelime uzunluğu)
    1 false döndür
  4 eğer biterse
  5 chNdx'i kelime[ltrNdx] olarak ayarla
  6 kökü chNdx alt ağ olarak ayarla
  7 arttır ltrNdx
4 döngüyü bitir
5 false döndür
bitiş searchTrie

```

Algoritma 10-15 Analizi Bu algoritmanın basitliği ağaçların gücünü gösterir. Ağaçta aşağıya doğru arama yaparken, öncelikle trie düğümü ile ilişkili kelimeyi hedef kelimemizle karşılaştırırız ve bulup bulmadığımıza bakarız. Eğer bulamazsak, mevcut seviyedeki sözlük kelime uzunluğunu, **ltrNdx** değişkeni ile temsil edilen, kelime uzunluğundan büyük olup olmadığını test ederiz. Eğer büyükse, kelimenin sözlükte olmadığını biliriz.

3.5 ve 3.6 numaralı ifadelerdeki kodun uygulanması, dile bağlı olarak değişir. Çoğu dil, bir harfi bir indekse dönüştürme yöntemine sahiptir. Kelimedeki bir sonraki harf indekse dönüştürüldüğünde (**chNdx**), bunu bir sonraki trie seviyesine geçiş yapmak için kullanabiliriz.

10.7 Anahtar Terimler

2-3 ağacı	leksikal arama ağacı
2-3-4 ağacı	m -yol ağacı
dengeB*	yükbudam
ağacıB	aakış
+ ağacı	
B-ağac	trie
ıB-ağacı sırası	alt akış
birleştir	

10.8 Özet

□ Bir m -yolu ağacı, arama ağacıdır ki:

1. Her düğümün 0 ile m alt ağacı vardır.
2. Verilen bir düğüm $k < m$ alt ağaca sahipse, düğüm k alt ağaç noktası içerir, bunların bazıları null olabilir ve $k - 1$ veri girişi içerir.
3. İlk alt ağaçtaki anahtar değerleri, ilkgirişteki anahtardan daha küçüktür; diğer alt ağaçlardaki anahtar değerleri, ebeveyn girişindeki anahtardan daha büyük veya ona eşittir.
4. Verilerin anahtarları sıralı $\text{anahtar}_1 \leq \text{anahtar}_2 \leq \dots \leq \text{anahtar}_k$
5. Tüm alt ağaçlar çok yolları ağaçlardır.

□ Bir B-ağacı, m -yönlü bir ağaçtır ve:

1. Kök ya bir yapraktır ya da 2, 3, ... m alt ağaçları vardır.
2. Tüm iç düğümler en az $\lceil m / 2 \rceil$ boş olmayan alt ağaçlara ve en fazla m boş olmayan alt ağaçlara sahiptir.
3. Tüm yaprak düğümler aynı seviyededir; yani, ağaç mükemmel bir şekilde dengelenmiştir.
4. Bir yaprak düğüm en az $\lceil m / 2 \rceil - 1$ ve en fazla $m - 1$ girişe sahiptir.

□ B-ağacı ekleme işlemi bir yaprak düğümde gerçekleşir. Dolu bir düğüme ekleme, taşma olarak bilinen bir durumu yaratır. Taşma, yaprak düğümün iki düğüme bölünmesini gerektirir; her biri verilerin yarısını içerir.

□ Bir B-ağacından bir girişi silerken üç nokta dikkate alınmalıdır.

Öncelikle, silinecek verilerin gerçekten ağaçta bulunduğundan emin olmalıyız. İkincisi, silme işleminden sonra düğüm yeterli girişe sahip değilse, yapı eksikliğini (alt akış) düzeltmemiz gerekir. Üçüncüsü, yalnızca bir yaprak düğümden silebiliriz.

□ Bir B-ağacı bir arama ağacı olduğu için, ağacı sıralı gezinti ile gezip her düğümü sırayla ziyaret ederiz.

□ İki özel B-ağacı vardır: 2-3 ağaçları ve 2-3-4 ağaçları.

- 2-3 ağacı, 3. dereceden bir B-ağacıdır. Kök hariç her düğüm ikiveya üç alt ağa sahip olabilir.
- 2-3-4 ağacı, 4. dereceden bir B-ağacıdır. Kök hariç her düğüm iki, üç veya dört alt ağa sahip olabilir.
- B-ağacının iki popüler varyasyonu vardır: B* ağacı ve B+ ağacı.
- B* ağacında, her düğüm her düğüm için izin verilen maksimum öğelerin en az üçte ikisini içerir.
- B+ ağacında, veri girişleri yalnızca yaprak seviyesinde bulunur ve her yaprak düğümü, bir sonraki yaprağa bağlanmak için kullanılan ek bir işaretçi içerir.
- Sözcüksel m -ary ağaçta, anahtar bir karakter dizisi olarak temsil edilir. Her bir giriş, sözcüksel ağacın bir sonraki seviyesine işaret eden bir işaretçi içerir.
- Bir trie, var olmayan karakterlere işaret eden işaretçilerin null işaretçileriyle değiştirildiği bir sözcüksel m -ary ağaçtır.

10.9 Pratik SetlerAlı

ştırmalar

1. Bir:
 - a. 3 seviyeli ağaç yüksekliği 3
 - b. 4 seviyeli ağaç yüksekliği 5
 - c. m -seviyeli ağaç yüksekliği h
2. Bir:
 - a. 3 katlı 5. dereceden B-ağacı
 - b. 5 katlı 5. dereceden B-ağacı
 - c. 5. dereceden B-ağacı ve yüksekliği h
3. Aşağıdaki verileri sırasıyla ekleyerek oluşturulan 3. dereceden B-ağacını çizin:

92 24 6 7 11 8 22 4 5 16 19 20 78

4. Aşağıdaki verileri sırasıyla ekleyerek oluşturulan 4. dereceden B-ağacını çizin:

92 24 6 7 11 8 22 4 5 16 19 20 78

5. Aşağıda gösterilen iki setten gelen verileri sırasıyla ekleyerek oluşturulan iki farklı 3. dereceden B-ağacını çizin. Verilerin sırası farklı B-ağaçları oluşturup oluşturmadığını belirlemek için iki B-ağacını karşılaştırın.

89 78 8 19 20 33 56 44
44 56 33 20 19 8 78 89

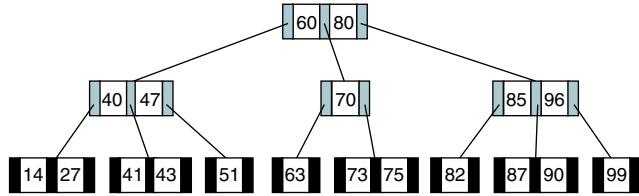
6. Yedi girişi depolayabilen iki farklı 3. dereceden B-ağacı çizin.
7. Aşağıdaki verileri sırasıyla ekleyerek 5. dereceden bir B* ağacı oluşturun:

92 24 6 7 11 8 22 4 5 16 19 20 78

8. Aşağıdaki verileri sırasıyla ekleyerek 5. dereceden bir B+ ağacı oluşturun:

92 24 6 7 11 8 22 4 5 16 19 20 78

9. Tüm 3 bitlik ikili sayılardan (000'dan 111'e kadar) oluşan bir trie çizin.
10. Şekil 10-24'te gösterilen 3. dereceden B-ağacını kullanarak 50, 78, 101 ve 232'yi ekleyin.



ŞEKİL 10-24 Egzersizler 10 ve 11 için B-ağacı

11. Şekil 10-24'te gösterilen 3. dereceden B-ağacını (Egzersiz 10'daki güncellemeler ol madan) kullanarak 63, 90, 41 ve 60'ı silin.

Problemler

12. Bir yığın kullanarak B-ağacı ekleme algoritmasını yeniden yazın.
13. Bir yığın kullanarak B-ağacı silme algoritmasını yeniden yazın.
14. Bir B+ ağacı için arama algoritmasını yazın.
15. Bir trie'yi dolaşan ve tüm kelimelerini leksik sıralamada yazdıran bir algoritma yazın.
16. Bir B-ağacı sayma algoritması için C kodunu yazın.
17. _insertEntry için C kodunu yazın.
18. C kodunu yazın _searchNode.
19. C kodunu yazın _borrowLeft.
20. C kodunu yazın _deleteEntry.

Projeler

21. B-ağaç ADT'sini kullanarak, 100 girişi olan 7. dereceden bir B-ağacı oluşturun. 1 ile 1000 arasında rastgele anahtarlar oluşturmak için bir rastgele sayı üretici kullanın. Ardından, kullanıcının verileri silmesine, eklemesine ve almasına veya ağacı yazdırmasına olanak tanıyan menü tabanlı bir kullanıcı arayüzü oluşturun.
22. B+ ağacı için bir ADT oluşturun. Ağaç yapısında, dosyadaki en soldaki düğümün adresini tanımlayan ek bir meta veri değişkeni sağlayın. Daha sonra, ağacı gezmek için en soldaki düğümün adresini ve sonraki düğüm işaretçileri kullanarak şekilde gezinti fonksiyonunu değiştirin.
23. İncelediğimiz B-ağaç yapısı, bir indeksli dosya oluşturmak için kullanılabilir. Bir indeksli dosya, dosyadaki verileri aramak için bir indeks yapısı içerir. İndekste her kayıt, dosyadaki veri anahtarını ve dosyadaki veri kaydının adresini içerir. İndeks, dosya açıldığında oluşturulabilir veya diskte ayrı bir dosya olarak saklanabilir.
B-ağaç ADT'sini kullanarak dinamik bellek içinde bir dosya indeksi oluşturan bir program yazın. Program başladığında, dosyayı okur ve B-ağaç indeksini oluşturur. İndeks oluşturulduktan sonra, belirli bir kaydı almak, yeni kayıtlar eklemek, kayıtları silmek ve dosyayı gezmek için kullanıcıya menü tabanlı bir arayüz sağlayın; tüm verileri yazdırın. Dosya içinde CD'ler veya kütüphane kitapları koleksiyonu gibi uygun uygulama verilerini kullanabilirsiniz.
24. Proje 23'te dosyayı okuyarak bir B-ağaç indeksi oluşturduk. Program her başladığında dosyayı okumak yerine, indeksi diskte ayrı bir dosya olarak saklayabiliriz. Bu durumda, program başladığında, indeks okunur ve B-ağaç içine eklenir. Program tamamlandığında, güncellenmiş B-ağaç indeksini diskte geri yazılır. Proje 23'teki programı, indeksi diskte saklayacak şekilde değiştirin.
25. B-ağaç dizin projesi, dinamik bellek yerine bir diskte saklanabilir. Dizin dosyasındaki ilk kayıt, kök girişinin dosyadaki konumu ve veriler için karşılaştırma fonksiyonunun adresi dahil olmak üzere dizin hakkında meta-veri içermelidir. B-ağaç ADT'sini diskte B-ağacı sürdürmek için değiştirin. Bu ADT'nin bu versiyonunda, B-ağaç oluşturma fonksiyonu bir dosya açma fonksiyonu ile değiştirilmiştir. Dosya açıldığında karşılaştırma fonksiyonu tanımlanmalıdır.
26. B-ağaç dizin projesinin son bir varyasyonu olarak, Proje 25'i B+ ağaç olarak yeniden çalışın.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 11

Grafikler

Birçok farklı veri yapısını inceledik. Her düğümün tek bir ardılı olduğu doğrusal listelerle ilgili birkaç veri yapısını inceleyerek başladık. Ardından, her düğümün birden fazla ardılı olabileceği ancak yalnızca bir öncülü olabileceği ağaç yapıları üzerinde durduk. Bu son bölümde, diğerlerinden bir ana kavramda farklı olan bir veri yapısına—grafiklere—dikkatimizi çeviriyoruz: her düğüm birden fazla öncül ve birden fazla ardıl olabilir.

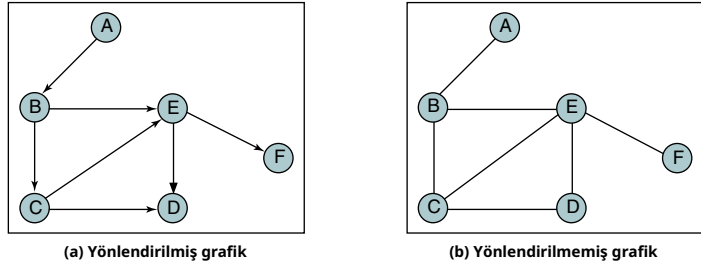
Grafikler çok yararlı yapılardır. Hava yollarının hizmet verdiği havaalanları arasında tasarım ve yönlendirme gibi karmaşık yönlendirme problemlerini çözmek için kullanılabilirler. Benzer şekilde, bir bilgisayar ağı üzerinden bir düğümden diğerine mesajları yönlendirmek için de kullanılabilirler.

11.1 Temel Kavramlar

Bir grafik, düğümler olarak adlandırılan bir dizi düğüm ve çizgiler olarak adlandırılan bir dizi segmentten oluşur; bu segmentler, düğüm çiftlerini bağlar. Diğer bir deyişle, bir grafik, bir düğüm kümesi ve bir çizgi kümesinden oluşur.

Grafikler ya yönlendirilmiş ya da yönlendirilmemiş olabilir. Yönlendirilmiş grafik veya kısaca digraf, her çizginin bir yönü (ok ucu) olduğu bir grafikdir. Yönlendirilmiş bir grafikteki çizgilere yaylar denir. Yönlendirilmiş bir grafikte, iki düğüm arasındaki akış yalnızca belirtilen yönü takip edebilir. Yönlendirilmemiş grafik, herhangi bir çizgide yön (ok ucu) olmayan bir grafikdir; bu çizgilere kenarlar denir. Yönlendirilmemiş bir grafikte, iki düğüm arasındaki akış her iki yönde de gidebilir. Şekil 11-1, hem yönlendirilmiş bir grafiğin (a) hem de yönlendirilmemiş bir grafiğin (b) bir örneğini içermektedir.

Bir yol, her bir düğümün bir sonraki düğüme bitişik olduğu bir düğüm dizisidir. Şekil 11-1'de, {A, B, C, E} bir yol ve {A, B, E, F} başka bir yoldur. Hem yönlendirilmiş hem de yönlendirilmemiş grafiklerin yolları olduğunu unutmayın. Yönlendirilmemiş bir grafikte, her iki yönde de seyahat edebilirsiniz.



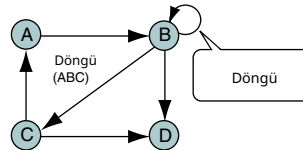
ŞEKİL 11-1 Yönlendirilmiş ve Yönlendirilmemiş Grafikler

Grafikler yönlendirilmiş veya yönlendirilmemiş olabilir. Yönlendirilmiş bir grafikte, yay olarak adlandırılan her çizginin, nasıl geçileceğini belirten bir yönü vardır. Yönlendirilmemiş bir grafikte, çizgi kenar olarak bilinir ve her iki yönde de geçilebilir.

Bir grafikte iki köşe, aralarında 1 uzunluğunda bir yol varsa komşu köşeler (veya komşular) olarak adlandırılır. Şekil 11-1(a)'da, B A'ya komşudur, E ise D'ye komşu değildir; diğer yandan, D E'ye komşudur. Şekil 11-1(b)'de, E ve D komşudur, ancak D ve F komşu değildir.

Bir grafik, düğümlerden oluşan bir koleksiyondur, bunlara köşe denir ve düğüm çiftlerini bağlayan çizgi segmentlerine de yaylar veya kenarlar denir.

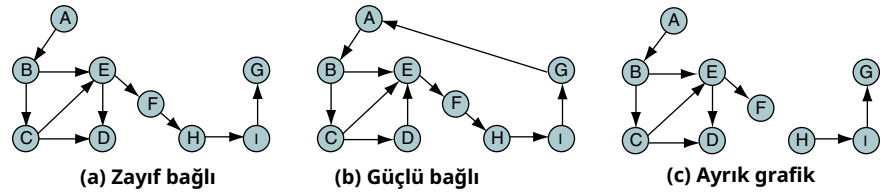
Bir döngü, en az üç köşeden oluşan ve aynı köşe ile başlayan ve biten bir yoldur. Şekil 11-1(b)'de, B, C, D, E, B bir döngüdür. Ancak, Şekil 11-1(a)'daki aynı köşelerin bir döngü oluşturmadığını unutmayın, çünkü bir yönlendirilmiş grafikte bir yol yalnızca yay yönünü takip edebilir, oysa yönlendirilmemiş bir grafikte bir yol kenar boyunca her iki yönde de hareket edebilir. Bir döngü, tek bir yay ile aynı köşe ile başlayan ve biten döngünün özel bir durumudur. Bir döngüde, çizginin uç noktaları aynıdır.



ŞEKİL 11-2 Döngüler ve Döngüler

İki köşe, aralarında bir yol varsa bağlı olarak adlandırılır. Bir graf, yönü göz ardı edildiğinde, herhangi bir köşeden herhangi bir diğer köşeye bir yol varsa bağlıdır. Ayrıca, bir yönlendirilmiş grafik güçlü bir şekilde bağlıdır

Her bir köşeden diğer her bir köşeye bir yol varsa, yönlendirilmiş grafikte. Yönlendirilmiş bir grafik, en az iki köşe birbirine bağlı değilse zayıf bağlı olarak adlandırılır. (Bağlı bir yönlendirilmemiş grafik her zaman güçlü bir şekilde bağlı olur, bu nedenle bu kavram genellikle yönlendirilmemiş grafiklerle kullanılmaz.) Bir grafik, ayırık grafik olarak adlandırılırsa bağlı değildir. Şekil 11-3, bir zayıf bağlı grafiği (a), bir güçlü bağlı grafiği (b) ve bir ayırık grafiği (c) içermektedir.



ŞEKİL 11-3 Bağlı ve Ayırık Grafikler

Bir köşenin derecesi, ona bağlı olan çizgi sayısıdır. Şekil 11-3(a)'da B köşesinin derecesi 3 ve E köşesinin derecesi 4'tür. Bir yönlendirilmiş grafikte bir köşenin çıkış derecesi, köşeden çıkan yayların sayısıdır; giriş derecesi ise köşeye giren yayların sayısıdır. Yine, Şekil 11-3(a)'da B köşesinin giriş derecesi 1 ve çıkış derecesi 2'dir; Şekil 11-3(b)'de E köşesinin giriş derecesi 3 ve çıkış derecesi 1'dir.

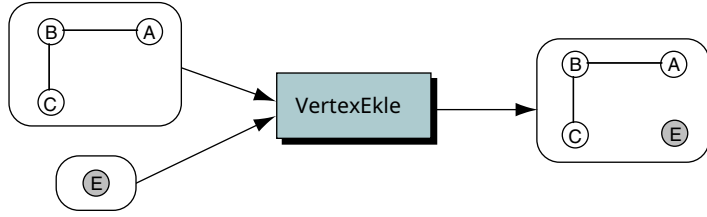
Son bir nokta: bir ağaç, her köşenin yalnızca bir öncüsü olduğu bir grafik tir; ancak, bir grafik ağaç değildir. Bölümün ilerleyen kısımlarında bazı grafiklerin içinde algoritmik olarak belirlenebilen bir veya daha fazla ağaç olduğunu göreceğiz.

11.2 İşlemler

Bu bölümde, bir grafiği sürdürmek için gereken temel modülleri sağlayan altı ilkel grafik işlemini tanımlıyoruz: bir tepe eklemek, bir tepe silmek, bir kenar eklemek, bir kenar silmek, bir tepe bulmak ve bir grafiği gezmek. Gördüğümüz gibi, grafik gezintisi iki farklı gezinti yöntemini içerir.

Tepe Ekle

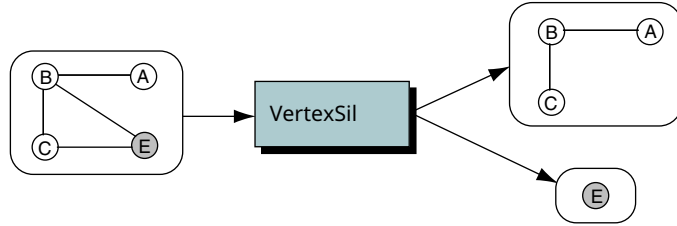
Tepe ekleme, bir grafiğe yeni bir tepe ekler. Bir tepe eklendiğinde, o tepe ayırık olur; yani, listedeki diğer tepelerle bağlantılı değildir. Açıkça, bir tepe eklemek, ekleme sürecindeki ilk adımdır. Bir tepe eklendikten sonra, bağlantı kurulması gerekir. Şekil 11-4, yeni bir tepe eklendikten önce ve sonra bir grafiği göstermektedir.



ŞEKİL 11-4Düğüm Ekle

Düğüm Sil

Düğüm sil, bir düğümü grafikten kaldırır. Bir düğüm silindiğinde, tümbağlantılı kenarlar da kaldırılır. Şekil 11-5, bir düğüm silmenin bir örneğini göstermektedir.



ŞEKİL 11-5Düğüm Sil

Kenar Ekle

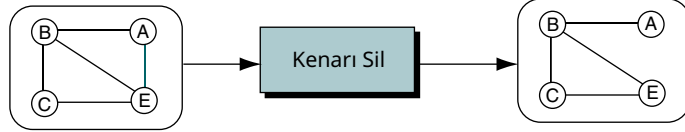
Kenar eklemek, bir düğümü bir hedef düğüme bağlar. Eğer bir düğüm birden fazla kenara ihtiyaç duyuyorsa, her komşu düğüm için bir kenar ekle çağrılmalıdır. Bir kenar eklemek için, iki düğüm belirtilmelidir. Eğer grafik bir yönlü grafikse, düğümlerden biri kaynak olarak ve diğeri hedef olarak belirtilmelidir. Şekil 11-6, grafiğe bir kenar eklemenin {A, E} bir örneğini göstermektedir.



ŞEKİL 11-6Kenar Ekle

Kenar Sil

Kenar sil, bir grafikten bir kenarı kaldırır. Şekil 11-7, grafikten {A, E} kenarını silen bir örneği göstermektedir.



ŞEKİL 11-7 Kenar Sil

VertexBul

Bir tepe noktasını bulmak, belirli bir tepe noktasını aramak için bir grafiği tarar. Eğer tepe noktası bulunursa, verisi döndürülür. Eğer bulunamazsa, bir hata belirtilir. Şekil 11-8'de tepe noktası, C tepe noktasını aramak için grafiği tarar.



ŞEKİL 11-8 Tepe Noktasını Bul

Grafiği Tarama

Verilen bir grafikte tüm tepe noktalarının ziyaret edilmesini gerektiren en az bir uygulama her zaman vardır; yani, grafiğin taranmasını gerektiren en az bir uygulama vardır. Çünkü bir grafikte bir tepe noktasının birden fazla ebeveyni olabilir, bir grafiğin taranması, lineer listelerin ve ağaçların taranmasında bulunmayan bazı sorunlar sunar. Özellikle, her tepe noktasındaki verileri yalnızca bir kez işlediğimizden emin olmalıyız. Ancak, bir tepe noktasına birden fazla yol olduğundan, grafiği tararken ona birden fazla yönden ulaşabiliriz. Bu sorunun geleneksel çözümü, her tepe noktasında bir ziyaret edildi bayrağı eklemektir. Tarama öncesinde, her tepe noktasındaki ziyaret edildi bayrağını *kapalı* olarak ayarlarız. Ardından, grafiği tararken, verilerin işlendiğini belirtmek için ziyaret edildi bayrağını *açık* olarak ayarlarız.

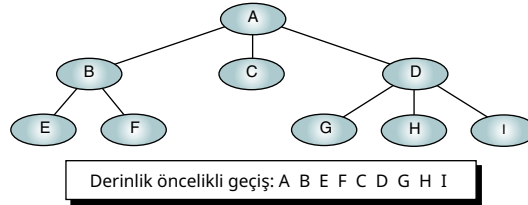
İki standart grafik gezintisi vardır: derinlik öncelikli ve genişlik öncelikli.

Her ikisi de ziyaret edilen bayrağını kullanır.

Derinlik Öncelikli Gezinti

Derinlik öncelikli gezintide, bir tepe noktasının tüm alt düğümlerini işledikten sonra komşu bir tepe noktasına geçeriz. Bu kavram, grafik bir ağaç olduğunda en kolay şekilde görülür.

Şekil 11-9'da, standart derinlik öncelikli gezintilerden biri olan ağaç ön siparişi gezintisi işleme sırasını gösteriyoruz.



ŞEKİL 11-9 Bir Ağacın Derinlik Öncelikli Gezintisi

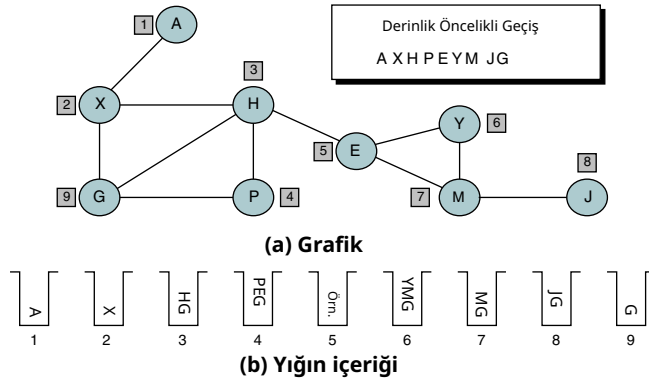
Benzer bir şekilde, bir grafiğin derinlik öncelikli gezintisi, grafiğin ilk tepe noktasını işleyerek başlar. İlk tepe noktasını işledikten sonra, ilk tepe noktasına komşu herhangi bir tepe noktasını seçer ve onu işleriz. Her tepe noktasını işlerken, komşu bir tepe noktasını seçeriz, ta ki komşu girişleri olmayan bir tepe noktasına ulaşana kadar. Bu, bir ağaçta bir yaprağa ulaşmaya benzer. Daha sonra yapının dışına çıkarız, gidişken komşu tepe noktalarını işleriz. Bu mantığın, gezintiyi tamamlamak için bir yığın (veya özyineleme) gerektirdiği açık olmalıdır.

Komşu köşelerin işlenme sırası, grafiğin fiziksel olarak nasıl saklandığına bağlıdır. Bölümde daha sonra ekleme mantığını tartıştığımızda, yayları artan anahtar sırasına göre eklediğimizi göreceksiniz. Ancak bir yığın kullandığımız için, geçiş işlemleri bitişik dikdörtgenleri azalan, yani son giren ilk çıkar (LIFO) sırasına göre işler.

Derinlik öncelikli geçişte, bir düğümün tüm torunları, bitişik bir düğüme geçmeden önce işlenir.

Şekil 11-10'daki grafikte derinlik öncelikli bir geçişi izleyelim. Bir köşenin yanındaki kutudaki sayı, işleme sırasını gösterir. Grafiğin altındaki yığınlar, grafiğin aşağısına doğru ilerlerken yığın içeriğini ve ardından geri çıkarken yığın içeriğini gösterir.

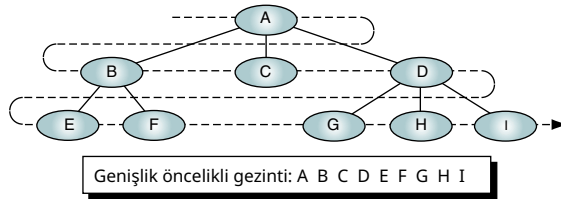
1. İlk köşe olan A'yı yığına iterek başlıyoruz.
2. Daha sonra döngüye giriyoruz, yığını çıkarıyoruz ve köşeyi işledikten sonra tüm bitişik köşeleri yığına itiyoruz. Bu nedenle, adım 2'de X'i işlemek için X'i yığından çıkarıyoruz, işliyoruz ve ardından G ve H'yi yığına itiyoruz; bu da adım 3 için yığın içeriğini Şekil 11-10(b)'de gösterildiği gibi H yapar.
3. Yığın boş olduğunda, geçiş tamamlanır.



ŞEKİL 11-10 Bir Grafiğin Derinlik Öncelikli Geçişi

Genişlik öncelikli Geçiş

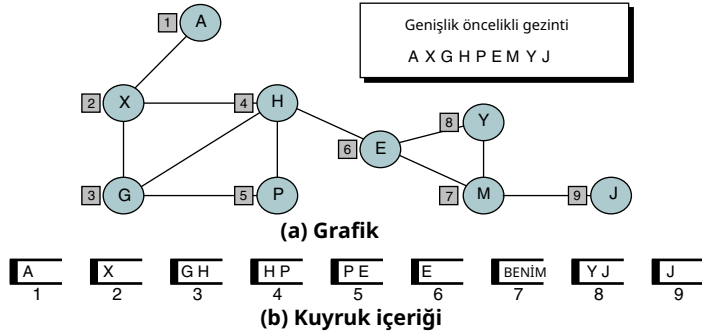
Bir grafin genişlik öncelikli geçişinde bir köşenin tüm komşu köşelerini işleriz sonra ki seviyeye geçmeden önce. Genişlik öncelikli geçişi ilk olarak 6. Bölümde bir ağaçta gördük. Şekil 11-11'deki ağaca baktığımızda, genişlik öncelikli geçişinin seviye 0'dan başladığını ve ardından seviye 1'deki tüm köşeleri işlediğini ve sonra seviye 2'deki köşeleri işlemeye geçtiğini görüyoruz.



ŞEKİL 11-11 Bir Ağacın Genişlik Öncelikli Geçişi

Bir grafin genişlik öncelikli geçişi aynı kavramı takip eder. Başlangıç noktasını (A) seçerek başlarız; onu işledikten sonra, tüm komşu noktalarını (BCD) işleriz. İlk noktanın tüm komşu noktalarını işledikten sonra, ilk komşu noktasını (B) seçeriz ve tüm noktalarını işleriz, ardından ikinci komşu noktasını (C) ve tüm noktalarını işleriz ve bu şekilde devam ederiz.

6. Bölümde, genişlik öncelikli geçişin bir yığın yerine bir kuyruk kullanıldığını gördük. Her noktayı işlerken, tüm komşu noktalarını kuyruğa yerleştiririz. Ardından, işlenecek bir sonraki noktayı seçmek için, kuyruktan bir noktayı sileriz ve onu işleriz. Bu mantığı Şekil 11-12'deki graf üzerinde izleyelim.



ŞEKİL 11-12 Bir Grafin Genişlik Öncelikli Geçiş

1. Öncelikle A köşesini kuyruğa ekleyerek başlıyoruz.
2. Daha sonra döngüye giriyoruz, kuyruğu boşaltıyor ve kuyruğun önündeki köşeyi işliyoruz. Köşeyi işledikten sonra, ona bitişik tüm köşeleri kuyruğa yerleştiriyoruz. Böylece, Şekil 11-12(b)'deki adım 2'de, X köşesini boşaltır, onu işliyoruz ve ardından G ve H köşelerini kuyruğa yerleştiriyoruz. Ardından, G köşesini işlediğimiz adım 3'e hazırız.
3. Kuyruk boş olduğunda, gezinti tamamlanmış olur.

Genişlik öncelikli gezinti sırasında, bir düğümün soyundan önce tüm komşu düğümler işlenir.

11.3 Grafik Depolama Yapıları

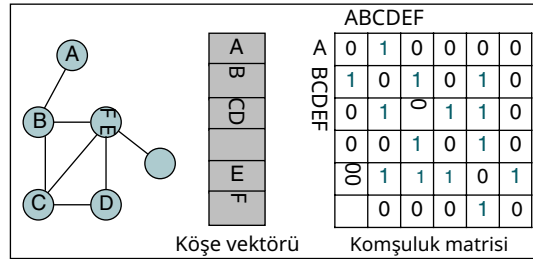
Bir grafiği temsil etmek için iki küme saklamamız gerekiyor. İlk küme grafiğin köşelerini, ikinci küme ise kenarları veya yayları temsil eder. Bu kümeleri saklamak için en yaygın kullanılan iki yapı diziler ve bağlı listelerdir. Diziler bazı basitlik ve işleme verimliliği sunsa da, köşe sayısının önceden bilinmesi gerekir. Bu, önemli bir sınırlamadır.

Komşuluk Matrisi

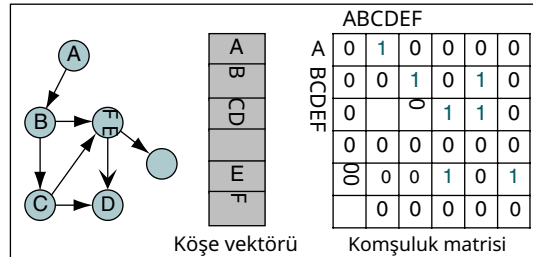
komşuluk matrisi köşeler için bir vektör (tek boyutlu dizi) ve kenarları saklamak için bir matris (iki boyutlu dizi) kullanır (bkz. Şekil 11-13). Eğer iki köşe komşu ise—yani, aralarında bir kenar varsa—matrisin kesişiminde 1 de geri bulunur; eğer aralarında bir kenar yoksa, kesişim 0 olarak ayarlanır.

Eğer grafik yönlendirilmişse, komşuluk matrisindeki kesişim yönü gösterir. Örneğin, Şekil 11-13(b)'de kaynak düğüm B'den hedef düğüm C'ye bir yay vardır. Komşuluk matrisinde, bu yay bir

B'den (solda) C'ye kesişimde 1. Ancak C'den B'ye bir yay olmadığından, C'den B'ye kesişim 0'dır. Öte yandan, Şekil 11-13(a)'da B'den C'ye olan kenar iki yönlüdür; yani, grafikte yönsüz olduğu için her iki yönde de geçiş yapabilirsiniz. Böylece, yönsüz komşuluk matrisinde B'den C'ye olan kesişimde ve C'den B'ye olan kesişimde 1 kullanılır. Diğer bir deyişle, matris, kenarı her iki yönde de kullanabileceğinizi yansıtır.



(a) Yönsüz grafik için komşuluk matrisidir



(b) Yönlendirilmiş grafik için komşuluk matrisi

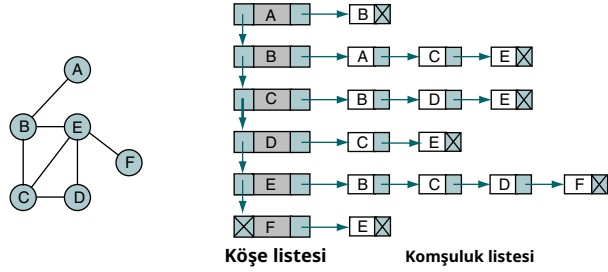
ŞEKİL 11-13 Komşuluk Matrisi

Komşuluk matris temsiliinde, köşeleri depolamak için bir vektör ve kenarları depolamak için bir matris kullanılır.

Program başlamadan önce grafiğin boyutunun bilinmesi gerekliliğinin yanı sıra, komşuluk matrisinde başka bir ciddi sınırlama vardır: İki tepe arasında yalnızca bir kenar saklanabilir. Bu sınırlama, birçok grafiğin matris formatını kullanmasını engellemese de, bazı ağ yapıları tepe noktaları arasında birden fazla satır gerektirir.

Komşuluk Listesi

Komşuluk listesi, kenarları depolamak için iki boyutlu düzensiz bir dizi kullanır. Bir komşuluk listesi Şekil 11-14'te gösterilmiştir.



ŞEKİL 11-14 Komşuluk Listesi

Vertex listesi, listedeki köşelerin tek bağlı bir listesidir. Uygulama ya bağlı olarak, çift bağlı listeler veya dairesel bağlı listeler kullanılabilir. Listenin solundaki işaretçi, köşe girişlerini bağlar. Köşedeki sağdaki işaretçi, köşeden kenarların bağlı listesine bir baş işaret çisidir. Böylece, Şekil 11-14'teki soldaki yönsüz grafikte, B köşesinden A, C ve E köşelerine bir yol vardır. Bu kenarları komşuluk listesinde bulmak için , B'nin köşe listesi girişinden başlayarak bağlı listeyi A'ya, sonra C'ye ve nihayet E'ye geçiyoruz.

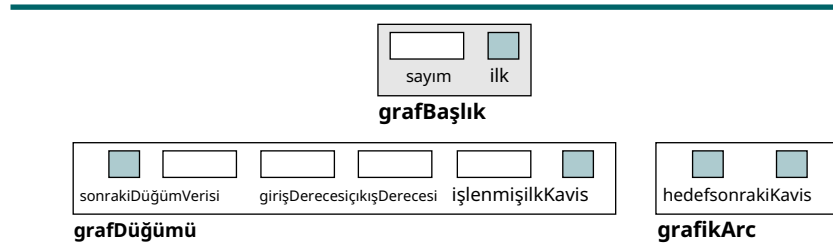
Komşuluk listesinde, köşeleri saklamak için bir bağlı liste ve yayları saklamak için iki boyutlu bir bağlı liste kullanıyoruz.

11.4 Grafik Algoritmaları

Bu bölümde, yönlü bir grafik oluşturmak ve sürdürmek için gereken minimum algoritma setini geliştiriyoruz. Bölüm 11-2, "İşlemler" bölümünde açıklanan işlemlere ek olarak, grafik uygulamalarında gerekli olan create graph gibi birkaç başka işlem de ekliyoruz. Uygulamanın doğası, hangi diğer işlemlerin gerektiğini belirler. Örneğin, köşe sayısını veya bir köşenin indegree veya outdegree'ini döndüren algoritmalar yazmak gerekli olabilir.

Algoritmaları tartışmadan önce, grafiği depolamak için kullandığımız veri yapısını tasarlamamız gerekiyor. En esnek yapı, tek bağlı liste olarak uygulanan komşuluk listesidir. Şekil 11-14'te gösterilen köşe ve komşuluk yapılarının yanı sıra, bir baş yapı da ekliyoruz. Baş yapısı, liste hakkında meta verileri depolar. Algoritmalarımız için grafikteki köşe sayısının sadece bir sayısını saklıyoruz. Saklanabilecek diğer meta veri örnekleri arasında köşe listesinin sonuna bir arka işaretçi ve grafikteki toplam yay sayısının sayısı yer alır. Grafik veri yapısı Şekil 11-15'te gösterilmiştir.

Grafik verilerinin, varsa, köşe düğümünde saklandığını unutmayın. Bu veriler yalnızca köşeye aittir. Daha sonra bir yay hakkında veri saklamamız gereken bir yapı göreceğiz. Bu durumda yay verilerini yay köşesinde saklarız. Grafik yapısı için sahte kod, Algoritma 11-1'de gösterilmiştir.



ŞEKİL 11-15 Grafik Veri Yapısı

ALGORİTMA 11-1 Graf için Veri Yapısı

```

grafBaş
  sayım
  ilk
grafBaş'ı bitir

grafKöşe
  sonKöşe
  data
  girişDerece
  esi çıkışDer
  ecesi işlen
  miş ilkYay
grafKöşe'yi bitirgr

afYay
  hedef sonrak
  iKavis
son grafKavis

```

Veri ve işaretçilerin yanı sıra, her köşe girişi, ona işaret eden yayların sayısını - iç derece - ve ondan çıkan yayların sayısını - dış derece - içerir. İç derece sayısı çok faydalı bir amaca hizmet eder.

Bir köşeyi sildiğimizde, ona işaret eden hiçbir yay olmadığından emin olmalıyız. Eğer varsa, yay işaretçisi kullanılarak silinen köşeye yapılan herhangi bir referans programın başarısız olmasına neden olur. Bu nedenle, silme algoritmamız, ona işaret eden herhangi bir yay varsa bir köşenin silinmesine izin vermez.

Son olarak, yalnızca geçişler için kullanılan bir alanımız var, işlenmiş. Bu, bir köşedeki verilerin işlenmeyi beklediğini veya mevcut geçiş sırasında zaten işlenmiş olduğunu gösterir.

Graf Oluştur

Graf oluştur, bir grafik baş yapısı için meta veri öğelerini başlatır. Kod, Algoritma 11-2'de gösterilmiştir.

ALGORİTMA 11-2Graf Oluştur

```

Algoritma createGraph
Bir grafik yapısının meta veri öğelerini başlatır.
    Önce grafik, meta veri yapısına bir referanstır
    Sonra meta veri öğeleri başlatılmıştır
1  sayacı 0 olarak ayarla
2  ilk değeri null olarak ayarla
3  graf başını döndür
son createGraph
  
```

Tepe Ekle

Vertex ekleme, grafiğe ayırık—yani, bağlantısız—bir vertex ekler.

Vertex ile ilişkili yaylar ayrı olarak eklenmelidir.

pseudocode, Algoritma 11-3'te gösterilmiştir.

ALGORİTMA 11-3Vertex Ekle

```

Algoritma insertVertex (graf, dataIn)
Yeni bir köşe için bellek ayırır ve verileri kopyalar.
    Ön  graf, graf başı yapısına bir referanstır
        dataIn, köşeye yerleştirilecek verileri içerir
    Son  yeni köşe ayrıldı ve veriler kopyalandı
1  yeni köşe için bellek ayır
2  yeni köşede dataIn verisini sakla
3  newNode'daki meta veri öğelerini başlat
4  graf sayısını artır
    Şimdi ekleme noktasını bul
5  eğer (boş grafik)
    1  grafın ilkinin newNode olarak ayarla
6  aksi takdirde
    1  ekleme noktasını ara
    2  eğer (ilk köşeden önce ekliyorsanız)
        1  grafın ilkinin yeni köşe olarak ayarla
    3  aksi takdirde
        1  dizide yeni bir köşe ekle
7  eğer biterse
son insertVertex
  
```

Algoritma 11-3 AnaliziBu, temel tek bağlı liste ekleme durumudur. Yeni vertex için bellek ayırdıktan sonra, verileri taşıyız ve tüm meta veri değerlerini null olarak ayarlarız.

Graf, entegre bir başlık yapısı içermediğinden, boş bir grafiğe eklemeleri ve ilk vertex'ten önceki eklemeleri özel durumlar olarak ele etmemiz gerekir. Null listeye ekleme, ifade 5.1'de ele alınır. İlk vertex'ten önce ekleme, ifade 6.2.1'de ele alınır.

Düğüm Sil

Herhangi bir silme algoritmasında olduğu gibi, bir köşeyi silmek için ilk yapmamız gereken şey onu bulmaktır. Ancak, onu bulduktan sonra, ayrıca ayırık olduğundan emin olmamız gerekir; yani, köşeden çıkan veya köşeye giren hiçbir yay olmaması gerektiğini sağlamalıyız. Eğer varsa, silmeyi reddederiz. Köşe silme için psödo kod, Algoritma 11-4'te gösterilmiştir.

ALGORİTMA 11-4 Köşe Sil

```

Algoritma deleteVertex (graf, anahtar)
Mevcut bir köşeyi yalnızca derecesi 0 ise siler.
  Ön   graf, graf başına referans işaretçisidir
      anahtar, silinecek köşenin anahtarıdır
Sonra köşe silindi (eğer derece sıfırsa)
Başarıyla +1 döndür
      -1 eğer derece sıfır değilse
      -2 eğer anahtar bulunamazsa
1  eğer (boş graf)
    1   dön -2
2  son if
    Silinecek köşeyi bul
3  silinecek köşeyi arayın
4  eğer (bulunamadıysa)
    1   dön -2
5  son if
    Silinecek köşe bulundu. Test derecesi.
6  if (tepe noktası inDegree > 0 VEYA outDegree > 0)
    1   dönüş -1
7  son if
    Köşeyi silmek için uygun
8    köşeyi sil
9    graf sayısını azalt
10 dönüş 1
end deleteVertex

```

Algoritma 11-4 Analizi Bu, temel bir tek bağlı liste silme durumudur. Tek karmaşıklık, bir köşeyi silmemiz için derecesinin 0'dan büyük olmamasıdır. Bu gereklilik, köşedeki girdi ve çıktı derecelerini test ederek ko layca halledilir, bu 6. ifadede gösterilmiştir.

Yay Ekle

Bir köşeye sahip olduğumuzda, onu diğer köşelere bağlayabiliriz. Yay eklemek, grafikte iki noktayı gerektirir: kaynak köşe (`fromPtr`) ve hedef köşe (`toPtr`). Her köşe, fiziksel adresi yerine anahtar değeri ile tanımlanır. Bu tanımlama sistemi, grafikte çalışırken bize daha fazla esneklik sağlar ve algoritmaları uygulamayı kolaylaştıran bir veri yapısı gizleme derecesi sunar. Ekleme mantığı, Algoritma 11-5'te gösterilmiştir.

ALGORİTMA 11-5 Köşe Ekle

```

Algoritma insertArc (graf, fromKey, toKey)
İki köşe arasında bir yay ekler.
    Ön   graf, grafik baş yapısına referanstır
         fromKey, kaynak köşenin anahtarıdır
         toKey, hedef köşenin anahtarıdır
    Sonra yay komşuluk listesine eklendi
    Başarıyla eklenirse +1 döner
         fromKey bulunamazsa -2 döner
         toKey bulunamazsa -3 döner
1 yeni yay için bellek ayır
   Kaynak köşeyi bul
2 fromVertex'i ara ve ayarla
3 eğer (kaynak köşe bulunamazsa)
   1 dön -2
4 son if
   Şimdi hedef köşeyi bul
5 toVertex'i ara ve ayarla
6 eğer (düğüm bulunamadı)
   1 dönüş -3
7 son if
   Başlangıç ve bitiş düğümleri bulundu. Yeni yay ekleniyor.
8 fromVertex çıkışDerecesini artır
9 toVertex girişDerecesini artır
10 yay hedefini toVertex olarak ayarla
11 eğer (fromVertex yay listesi boşsa)
   İlk yayı ekliyorum
   1 fromVertex ilkYay'ını yeni yay olarak ayarla
   2 yeni arc nextArc'ı null olarak ayarla
   3 1 döndür
12 son if
   komşuluk (arc) listesindeki ekleme noktasını bul
13 arc listesindeki ekleme noktasını bul
14 eğer (arc listesinin başına ekle)
   İlk arc'dan önce ekleme
   1 fromVertex firstArc'ı yeni arc olarak ayarla
15 aksi takdirde
   1 arc listesine ekle
16 son if
17 dönüş 1
son insertArc

```

Algoritma 11-5 AnaliziYeni köşe için bellek ayırdıktan sonra, from köşesini ve to köşesini buluyoruz.

Bu mantık, basit bağlı liste aramalarını içerir. Eğer herhangi bir arama başarısız olursa, uygun değeri döneriz, -2 bulunamayan başlangıç noktası için ve -3 bulunamayan bitiş noktası için.

Her iki köşeyi bulduktan sonra, derece sayımlarını güncelliyoruz ve yeni yayının hedef işaretçisini hedef köşeye ayarlıyoruz. Ardından, yeni yayı komşuluk listesine eklemeye hazırız. Komşuluk listesi tasarımı, yay köşelerinin hedef anahtara göre sıralı olmasını gerektirir. Bu nedenle, ekleme noktası için yay listesini arıyoruz. Tekrar eden yayların listeye FIFO sırasına göre yerleştirildiğini unutmayın. Öte yandan, bazı

uygulamalar aynı yayların olmasına izin vermeyebilir. Eğer izin verilmiyorsa, mantığın değiştirilmesi ve ek bir hata kodu oluşturulması gerekir.

Yay Sil

Yay silme algoritması, komşuluk listesinden bir yayı kaldırır. Bir yayı tanımlamak için iki tepe noktasına ihtiyacımız var. Tepeler, anahtarlarıyla tanımlanır. Bu nedenle algoritma önce başlangıç tepe noktasını bulmak için tepe listesinde arama yapar ve ardından hedef tepe noktasını bulmak için komşuluk listesinde arama yapar. Yay bulunduğunda ve silindiğinde, başlangıç ve hedef tepe noktalarındaki derece ayarlanır ve bellek geri kazanılır. Pseudocode, Algoritma 11-6'da gösterilmiştir.

ALGORİTMA 11-6Yay Sil

```

Algoritma deleteArc (graf, fromKey, toKey)
İki tepe noktası arasındaki bir yayı siler.
    Ön   grafik, bir grafik baş yapısına referanstır
         fromKey, kaynak tepe noktasının anahtarıdır
         toKey, varış tepe noktasının anahtarıdır
    Son   tepe noktası silindi
    Başarılıysa +1 döndür
         fromKey bulunamazsa -2 döner
         toKey bulunamazsa -3 döner
1  eğer (boş graf)
    1   dön -2
2  son if
    Kaynak tepe noktasını bul
3  fromVertex'i, anahtarı   fromKey'e eşit olan tepe noktası ile ayarla

4  eğer (fromVertex bulunamadı)
    1   dön -2
5  son if
    Komşuluk listesindeki varış tepe noktasını bul
6  eğer (fromVertex yay listesi null)
    1   dönüş -3
7  son if
8  anahtarı toKey'e eşit olan yay bul ve ara
9  eğer (toKey bulunamadı)
    1   dönüş -3
10 son if
    fromVertex, toVertex ve yay hepsi bulundu. Yayı sil.
11 arc hedefe toVertex ayarla
12 arc sil
13 fromVertex outDegree değerini azalt
14 toVertex inDegree değerini azalt
15 dönüş 1
Arc'ı sonlandır

```

Algoritma 11-6 Analizi Bu algoritmada üç işlem vardır: (1) kaynak tepe noktasını bul, (2) hedef tepe noktasını bul ve (3) yayı sil. Kaynak tepe noktası, tepe listesinde arama yaparak bulunur. Bulduktan sonra, komşuluk listesinde hedefe işaret eden bir yay ararız.

hedef tepe noktası. Doğru arc'ı bulduğumuzda, from ve to tepe noktası girişlerindeki derece alanlarını ayarlıyoruz ve ardından arc'ı siliyoruz.

Tepe Noktası Al

Tepe noktası al, bir tepe noktasında saklanan verileri döndürür. Tepe noktasının anahtarı verildiğinde, veriler çağrıda belirtilen çıktı alanına yerleştirilir. Pseudocode, Algoritma 11-7'de gösterilmiştir.

ALGORİTMA 11-7 Tepe Noktası Al

```

Algoritma retrieveVertex (graf, anahtar, dataOut)
  Anahtarla tanımlanan köşede bulunan veriler çağırana iletilir.
  Ön   graf, bir grafik baş yapısına referanstır
        anahtar, köşe verisinin anahtarıdır
        dataOut, veri değişkenine referanstır
  Sonra köşe verisi dataOut'a kopyalanır
  Başarıyla tamamlanırsa +1 döner
        -2 eğer anahtar bulunamazsa
1  eğer (boş grafik)
    1 dön -2
2  eğer biterse
3    köşeyi ara
4  eğer (köşe bulunduysa)
    1 locnPtr verisini dataOut'a taşı
    2 1 döndür
5  aksi takdirde
    1 dön -2
6  eğer biterse
  son retrieveVertex
  
```

Algoritma 11-7 Analizi Tepe noktası almak, tipik bir bağlı liste arama algoritmasıdır. Arama başarılı olursa, tepe noktasındaki veriler çağrı sırasına göre belirtilen çıktı alanına yerleştirilir ve başarı (+1) döndürülür. Liste null ise veya veriler bulunamazsa, anahtar bulunamadı (-2) döndürülür.

Derinlik Öncelikli Gezinti

Derinlik öncelikli gezinti, Bölüm 11.2, "Genişlik öncelikli gezinti" kısmında tanımlanmıştır. Bir grafikteki tüm köşeleri, bir köşeyi ve onun tüm alt köşelerini işleyerek, k omşu bir köşeyi işlemeye başlamadan önce ziyaret eder.

Derinlik öncelikli gezinti algoritmasını yazmanın iki yolu vardır: onu öz yinelemeli olarak yazabiliriz veya bir yığın kullanarak yazabiliriz. Bu uygulamada yığını kullanmaya karar verdik. Bir köşeye ulaştığımızda, adresini bir yığına itiyoruz. Ardından yığını pop ediyoruz, köşeyi işliyoruz ve tüm komşu köşeleri yığına itiyoruz.

Sorun, bir köşeye grafikte birden fazla yol ile ulaşılabilmesidir. Sonuç olarak, her köşenin yalnızca bir kez işlenmesini sağlamalıyız. Algoritma 11-1'de köşe yapısını oluşturduğumuzda, işlenmiş bir bayrak ekledik. Gezintiye başladığımızda,

işlenmiş bayrağını 0 olarak ayarlıyoruz. Bir köşeyi yığına eklediğimizde, onun işlenmiş bayrağını 1 olarak ayarlıyoruz, bu da onun yığında olduğunu ve sırasını beklediğini gösteriyor. Bu, onu birden fazla kez eklememizi önüyor. Son olarak, köşeyi işlediğimizde, bayrağını 2 olarak ayarlıyoruz. Artık, eğer bir köşeye birden fazla kez ulaşırsak, ya onu daha önce eklediğimizi ya işlediğimizi biliyoruz ve onu ikinci kez eklemememiz veya işlememez gerekiyor. Geçiş mantığı Algoritma 11-8'de gösterilmiştir.

ALGORİTMA 11-8 Derinlik öncelikli Geçiş

```

Algoritma depthFirst (graf)
  Grafın anahtarlarını derinlik öncelikli sırayla işleyin.
  Ön graf, bir grafik baş yapısına işaretçidir
  Post köşeler "işlenmiş"
1  eğer (boş graf)
    1  geri dön
    işlenmiş bayrakları işlenmemiş olarak ayarlayın
2  yürüyüş işaretçisini grafın ilkine ayarlayın
3  döngü (tüm köşeleri üzerinden)
    1  işleneni 0 olarak ayarla
4  döngüyü bitir
  liste içindeki her köşeyi işle
5  yığın oluştur (stack)
6  döngü (köşe listesi üzerinden)
    1  eğer (köşe işlenmemiş ve yığında değilse)
        Yığına ekle ve bayrağı yığına ayarla
        1  yığına ekle (stack, walkPtr)
        2  walkPtr işleneni 1 olarak ayarla
        3  eğer biterse
            yığın tepe noktasındaki köşeyi işle
        4  döngü (boş yığın değilse(stack))
            1  zirveyi yığından çıkarılan değere ayarla(stack)
            2  işle (zirve)
            3  zirveyi işlenmiş olarak ayarla
            işlenmemiş zirveleri komşuluk listesinden it
            4  döngü (yay listesi üzerinden)
                1  eğer (yay hedefi yığında değilse)
                    1  yığını it(stack, hedef)
                    2  hedefi yığında olarak ayarla
                2  eğer biterse
                    3  bir sonraki hedefi al
            5  döngüyü bitir
        5  döngüyü bitir
    2  eğer biterse
        3  bir sonraki köşeyi al
    7  döngüyü bitir
8  yokStack(yığın)
son derinlik ilk

```

Algoritma 11-8 Analizi Derinlik öncelikli geçiş, tüm köşeleri işlenmemiş olarak ayarlayarak başlar. Daha sonra, ifade 6'da köşe listesini döngüye alıyoruz. Eğer grafiğin güçlü bir şekilde

bağlı, bu döngü gerekli değildir. Ancak, ilk köşeden tüm diğer köşelere bir yol yoksaveya ayrık köşeler varsa, tüm köşelerini işlendiğinden emin olmak için köş e listesini döngüye almamız gerekir. Bu, 6. ifadede bu döngünün birincil amacı dır.

Bir köşe işlenmek üzere seçildiğinde, öncelikle daha öncesakiya konulmadığından an veya daha önceki bir köşenin soyundan işlenmediğinden emin oluruz. Eğer değ i lse, 6.1.1. ifadede onuyığına ekleriz. Ardından, yığının en üstündeki köşeyi işleriz ve i şlenmemiş komşu köşeleri yığına ekleriz. Köşeleri tutmak için bir yığın kullandığımız dan, yalnızca derinlik öncelikli sırayla işlemiyoruz, aynı zamandakomşu köşeleri azal an sırayla işliyoruz. (Komşuluk listesini artıran anahtar sırasına göre oluşturduğumuz u unutmayın, bu nedenle komşu köşeler artan sırayla eklenir ve azalan sırayla çıkarıl ır.)

Yığın boş olduğunda, orijinal düğüm e güçlü bir şekilde bağlı olan tüm düğümle ri işlemiş oluruz. Sonra bir sonraki düğüm e geçeriz ve zayıf bağlı veya ayrık düğümle rin işlenmesini sağlamak için 6. ifadeye geri döneriz.

Genişlik öncelikli Geçiş

11.2. Bölümde tanımlanan genişlik öncelikli gezinti, bir düğümü işler ve ardından t üm komşu düğümlerini işler. Derinlik öncelikli gezinti için bir yığın kullandığımız ye rde, bir grafiği genişlik öncelikli olarak gezmek için bir kuyruk kullanırız. Pseudoco de, Algoritma 11-9'da gösterilmiştir.

ALGORİTMA 11-9 Genişlik Öncelikli Gezinti

```

Algoritma breadthFirst (graf)
Grafın anahtarlarını genişlik öncelikli sırayla işler.
  Ön graf, graf başı yapısına işaretçidir
  Sonra işlenen düğümler
1  eğer (boş grafik)
  1  geri dön
2  eğer biterse
  Öncelikle tüm işlenmiş bayrakları işlenmemiş olarak ayarlayın
3  createQueue (kuyruk)
4  döngü (tüm köşeleri üzerinden)
  1  düğümü işlenmemiş olarak ayarlayın
5  döngüyü bitir
  Düğüm listesindeki her düğümü işleyin
6  döngü (tüm köşeleri üzerinden)
  1  eğer (düğüm işlenmemişse)
    1  eğer (düğüm kuyrukta değilse)
      Kuyruğa ekle ve işlem bayrağını kuyrukta (1) olarak ayarla
    1  kuyruğa ekle (kuyruk, yürüyüşPtr)
    2  düğümü kuyrukta eklenmiş olarak ayarla
  2  eğer biterse
    Şimdi kuyruk önündeki düğümün alt öğelerini işle
  3  döngü (boşKuyruk değilse (kuyruk))
    1  düğümü kuyruktan çıkar (kuyruk)
    Düğümü işle ve işlenmiş olarak işaretle

```

devam ediyor

ALGORİTMA 11-9 Genişlik öncelikli Gezinme (devam ediyor)

```

2 işlem (tepe noktası)
3 tepe noktasını işlenmiş olarak ayarla
  Komşuluk listesinden işlenmemiş tepe noktalarını sıraya al
4 döngü (komşuluk listesi üzerinden)
  1 eğer (hedef sıraya alınmamış veya işlenmemişse)
    1 sıraya al (kuyruk, hedef)
    2 hedefi sıraya alınmış olarak ayarla
  2 eğer biterse
  3 bir sonraki varış noktasını al
  5 döngüyü bitir
4 döngüyü bitir
2 eğer biterse
3 bir sonraki köşeyi al
7 döngüyü bitir
8 destroyQueue (kuyruk)
son genişlikÖncelikli

```

Algoritma 11-9 Analizi Genişlik öncelikli bir geçişteki kod, derinlik öncelikli algoritmanın yaptığı gibi başlar, işlenmiş tüm bayrakları işlenmemiş olarak ayarlayarak. Ancak, bu algoritmada üç işlem durumu kullanıldığını unutmayın: işlenmemiş, kuyrukta ve işlenmiş. Bu, bir köşenin kuyrukta birden fazla kez yer almasını engeller. İşlendiğinde, bayrağı 2 olarak ayarlıyoruz. İşlenmiş bayrakları 0 olarak ayarladıktan sonra, Bölüm 4'te geliştirdiğimiz standart algoritmaları kullanarak bir kuyruk oluşturuyoruz.

Köşe listesini döngüye aldığımızda (ifade 6), önce köşenin işlenip işlenmediğini kontrol ediyoruz (ifade 6.1). Eğer işlenmişse, bir sonraki köşeye geçiyoruz (ifade 6.3) ve tekrar kontrol etmek için döngüye geri dönüyoruz. Derinlik öncelikli geçişte gördüğümüz gibi, bu döngü, ayrı köşeleri veya yayıf b ağılı bir grafikteki diğer köşeleri almamıza olanak tanır.

Eğer köşe işlenmemişse, daha önce kuyrukta yer alıp almadığını test ediyoruz (ifade 6.1.1); eğer yer almıyorsa, onu kuyruğa ekliyoruz. Ardından, kuyruğun önünde ki köşenin tüm torunlarını bir döngü ile işliyoruz (ifade 6.1.3) bu döngü, kuyruğun başındaki köşeyi kuyruktan çıkarır ve işler, tüm işlenmemiş komşu köşelerini kuyruğa yerleştirir ve kuyruk boşalana kadar döngüyü tekrarlar.

Öncelikle kuyrukta bulunan tepe noktasının tüm alt öğelerini işledikten sonra, bir sonraki tepe noktasına geçiyoruz ve tepe noktası listesinin işlenmesini tamamlamak için 6. ifadeye geri dönüyoruz.

Grafik Yok Et

Bir grafikten bir tepe noktasını silmeyi tartıştığımızda, tepe noktasının silinmeden önce ayrık olması gerektiğini gördük. Bu kural, grafi yok etme işlemi için geçerli değildir. Grafi yok etme mantığı, bu nedenle önce tepe noktasından tüm yayları silmelidir, ardından tepe noktasını silebilir. Tepe noktasına işaret eden yaylar hakkında endişelenmemize gerek yoktur çünkü bunlar, karşılık gelen tepe noktası silindiğinde silinir. Grafi yok etme tasarımı, Algoritma 11-10'da gösterilmiştir.

ALGORİTMA 11-10Grafı Yok Et

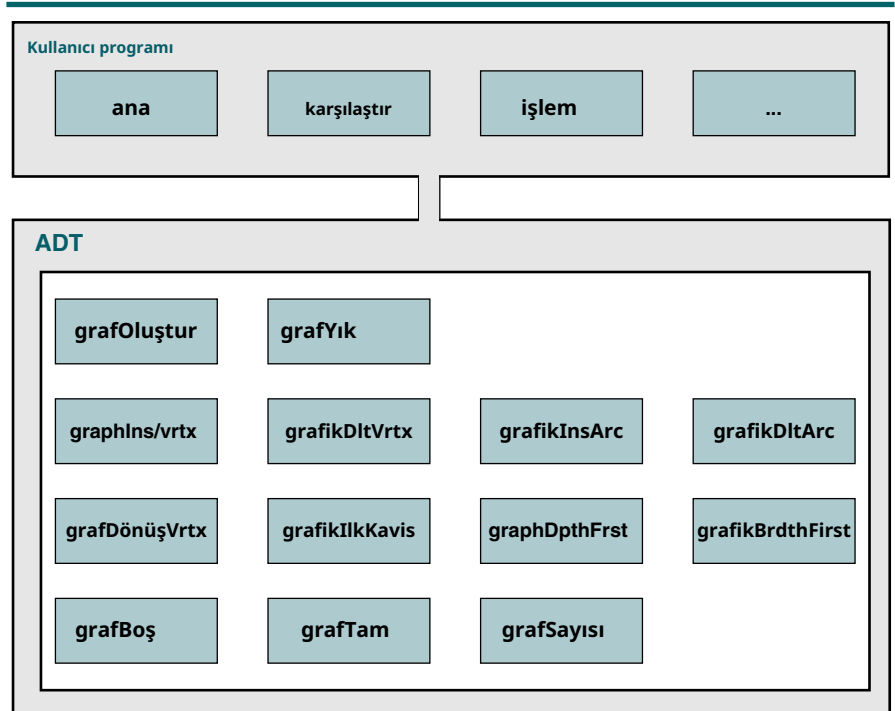
```

Algoritma destroyGraph (graf)
Grafı gezerek tüm düğümleri ve yayları siler.
  Ön Hiçbir şey
  Son Tüm düğümler ve yaylar silindi
1  eğer (boş grafik)
  1  geri dön
2  döngü (daha fazla düğüm)
  1  döngü (düğüm outDegree > 0)
  1  düğüm firstArc sil
  2  düğüm outDegree'den 1 çıkar
  2  döngüyü bitir
3  döngüyü bitir
son destroyGraph

```

11.5 Grafik ADT

Artık bir graf için soyut veri türünü tartışmaya hazırız. Bunu özellikle B-ağaçları için ADT ile karşılaştırıldığında çok basit bulacaksınız. Bu, graf gezintileri için yığın ve kuyruk ADT'leri ile birleştirilmiş basit bağlı liste kavramlarını kullanır. ADT tasarımı Şekil 11-16'da gösterilmiştir.



ŞEKİL 11-16Graf ADT Tasarımı

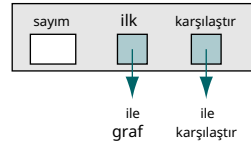
Veri Yapısı

Daha önceki tasarımlarda gördüğümüz gibi, bir grafın üç ayrı veri yapısına ihtiyacı vardır: biri başlık için, biri köşeler için ve biri yaylar için.

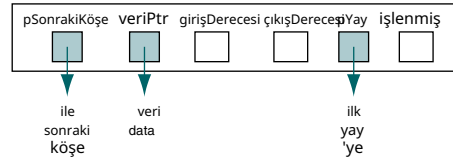
Başlık Yapısı

Graf veri yapısı, bir sayım, ilk köşeye işaretçi ve grafi aramak için gereken karşılaştırma fonksiyonunun adresini içeren basit bir başlık yapısı kullanır. Uygulama programının grafi görme şekli, graf oluşturulduğunda dinamik bellekten tahsis edilen başlık yapısına bir işaretçidir. Başlık yapısı Şekil 11-17'de gösterilmiştir.

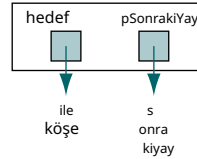
GRAF



KÖŞE



YAY



```
typedef yapışı
{
    int sayımı;
    önce struc vertex*;
    int (*karşılaştır)
        (void* argüman1,
         void* argüman2);
} GRAFİK;
```

```
typedef struct köşe noktası
{
    struct vertex* pNextVertex;
    void* dataPtr;
    int inDegree;
    int outDegree;
    kısa işlendi;
    struct arc* pArc;
} KÖŞE;
```

```
typedef struct arc
{
    struct vertex* hedefi;
    struct arc* pNextArc;
} ARC;
```

ŞEKİL 11-17 Graf Veri Yapısı

Köşe Yapısı

Graf köşe düğümü, köşe hakkında veri depolar. Bir veri işaretçisi ve sıradaki köşeye işaretçi, köşeden ilk yay için bir işaretçi ve üç meta veri niteliği—giriş derecesi, çıkış derecesi ve işlenmiş bayrağı içerir. Köşe yapısı da Şekil 11-17'de gösterilmiştir.

Yay Yapısı

Yay düğümü, bir köşeden diğerine giden yol hakkında veri depolar. Hedef köşeye işaretçi ve ilgili köşeden yoldaki bir sonraki yay için bir işaretçi içerir. Ayrıca, yay ağırlığı gibi yay meta verilerini de içerebilir. Yay yapısı da Şekil 11-17'de gösterilmiştir.

Algoritmalar

Graf veri yapısı, Bölüm 11.3'te tanımlandığı gibi bir komşuluk listesidir, "Komşuluk Listesi." Veri yapıları ve prototip bildirimleri, `graph.h` adını verdiğimiz bir grafik başlık yapısında tanımlanmıştır. Başlık dosyası Program 11-1'de gösterilmiştir.

PROGRAM 11-1 Graf Bildirimi

123456789	<pre> /* ===== graphs.h ===== Bu başlık dosyası, bu metinde geliştirilen grafikler için bildi rimleri ve prototip fonksiyonları içermektedir. Yazarı: Tarih: */ #include "queueADT.h" #include "stackADT.h" #include "stdbool.h" // ===== YAPILAR ===== typedef struct { int sayımı; ilk önce köşeyi* yapılandırın; int (*karşılaştırm) (void* argüman1, void* argüman2); } GRAFİK; typedef struct köşe noktası { struct vertex* pNextVertex; void* dataPtr; int inDegree; int outDegree; kısa işlendi; struct arc* pArc; } DİK AÇI; typedef struct arc { struct vertex* hedefi; struct arc* pNextArc; } ARC; // ===== Prototip Bildirimleri ===== GRAPH* graphCreate (int (*karşılaştırm) (void* argu1, void* argu2)); GRAPH* graphDestroy (GRAPH* graph); </pre>
1011121314151617181920212223242526272829303132333435363738394041	

devam ediyor

PROGRAM 11-1 Grafik Bildirimi (devamı)

```

423444546474849505152535455565758596061
void graphInsVrtx (GRAPH* graph, void* dataInPtr); int graphDltVr
tx (GRAPH* graph, void* dltKey); int graphInsArc (GRAPH* g
raph, void* pFromKey, void* pToKey);

int graphDltArc (GRAPH* grafiği, void* pFromKey,
void* pToKey);

int graphRetrVrtx (GRAPH* grafiği, void* pKey,
void** pDataOut);
int graphFrstArc (GRAPH* grafiği, void* pKey,
void** pDataOut);

void graphDpthFrst (GRAPH* graph,
void (*process) (void* dataPtr));
void graphBrdthFrst (GRAPH* graph,
void (*process) (void* dataPtr));

bool graphEmpty (GRAPH* graph);
bool graphFull (GRAPH* graph);
int graphCount (GRAPH* graph);

```

ADT işlevlerinin çoğunu aşağıdaki bölümlerde tartışıyoruz. Ancak bunl ardan bazıları,¹ çok basit ve diğer ADT'ler için geliştirilmiş paralel işlevlerd ir. Burada geliştirilmemiştir.

Grafik Düğüm Ekle

Bir grafiğin oluşturulmasında, ilk işlem düğümlerin eklenmesidir. Bu, Program 11-2'de gösterildiği gibi düğüm ekleme ile yapılır.

PROGRAM 11-2 Grafik Düğüm Ekle

```

123456789
/* ===== graphInsVrtx =====
Bu fonksiyon grafiğe yeni veri ekler.
Önce graph geçerli bir grafik yapısına işaret eder
Sonra veri eklendi veya bellek O/F ise iptal edilir
*/
void graphInsVrtx (GRAPH* grafiği, void* dataInPtr)
{
// Yerel Tanımlar
VERTEX* newPtr;
VERTEX* locPtr;
VERTEX* predPtr;
1011

```

devam ediyor

1. Özellikle: graphCreate, graphEmpty, graphFull, graphCount, graphRetrVrtx, graphFrstArc, ve graphDestroy dahil değildir.

PROGRAM 11-2Grafik Düğüm Ekle (devamı)

```

12131415161718192021222324252627282930313233343536373839404142434445464748495051
// Beyanlar
newPtr = (VERTEX*)malloc(sizeof (VERTEX));
if (newPtr)
{
    newPtr->pNextVertex = NULL;
    newPtr->dataPtr = dataInPtr;
    newPtr->inDegree = 0;
    newPtr->outDegree = 0;
    newPtr->işlenen = 0;
    newPtr->pArc = NULL;
    (graph->count)++;
} // eğer malloc
else
    printf("Taşma hatası 100\\a"),    exit (100)
    ;

// Şimdi ekleme noktasını bul
locPtr = graph->first;
eğer (!locPtr)
    // Boş grafik. Başlangıca ekle
    graph->first = newPtr;
aksi takdirde
{
    predPtr = NULL;
    while (locPtr && (graph->compare(dat
        aInPtr, locPtr->dataPtr) > 0)) {

        predPtr = locPtr;
        locPtr = locPtr->pNextVertex;
    } // while
    if (!predPtr)
        // İlk köşeden önce ekle
        graph->first = newPtr;
    başka
        predPtr->pNextVertex = newPtr;
    newPtr->pNextVertex = locPtr;
} // else
geri dön;
} // graphInsVrtx

```

Program 11-2 AnaliziGrafik ekleme, Algoritma 11-3'te bulunan sahte kodla paraleldir. Tek önemli

değişiklik, yeni düğüm için bellek ayırmayı 15. ifadede yapmamızdır, daha sonra değil.

Bu, fonksiyonun başında bellek taşması testini kolaylaştırmak için yapıldı.

İşlenmiş bayrağını bu zamanda 0 olarak ayarlamak mantıksal olarak gerekli ol
masa da (bkz.21. ifade), tutarlılık için yapıldı. Geri kalan mantık, düğümü tek bağlı li
steye ekleyen fonksiyondur.

Grafik Düğüm Sil

Bir tepe noktası, derecesi 0 olduğu sürece bir grafikten silinir. Eğer başka bir tepe noktası ona işaret ediyorsa veya o başka bir tepe noktasına işaret ediyorsa, silinemez. Program 11-3, Algoritma 11-4'teki tasarımı takip eder.

PROGRAM 11-3Grafik Tepe Noktası Sil

```

123456789 /* ===== graphDltVrtx =====
Mevcut tepe noktasını yalnızca derecesi sıfırsa siler.
Ön grafik, grafik baş yapısına işaretçidir dltKey, siline
cek tepe noktasının anahtaridir Son Derece sı
fırsa tepe noktası silinir -veya- Bir hata ko
du döndürülür Başarı Durumu +1 ba
şarılysa -1 eğer derece sıfır d
eğerse
-2 eğer dltKey bulunamazsa

*/
int graphDltVrtx (GRAPH* grafiği, void* dltKey)
{
// Yerel Tanımlar
VERTEX* predPtr;
VERTEX* walkPtr;

// İfadeler
if (!graph->ilk)
return -2;

// Silinecek tepe noktasını bul
predPtr = NULL;
walkPtr = grafik->ilk;
while (walkPtr
&& (grafik->karşılaştırd(dltKey, walkPtr->dataPtr) > 0))
{
predPtr = walkPtr;
walkPtr = walkPtr->pNextVertex;
} // walkPtr &&
if (!walkPtr
|| graph->compare(dltKey, walkPtr->dataPtr) != 0)
return -2;

// Bulunan köşe. Dereceyi test et
if ((walkPtr->inDegree > 0)
|| (walkPtr->outDegree > 0))
return -1;

// Silmek için uygun
eğer (!predPtr)
graph->first = walkPtr->pNextVertex; aksi takdi
rde

```

devam ediyor

PROGRAM 11-3Grafik Düğüm Sil (devam ediyor)

```

4344454647
    predPtr->pNextVertex = walkPtr->pNextVertex;
    --graph->count;
    ücretsiz(walkPtr);
    1'i döndür;
    , // graphDltVrtx

```

Grafik Yay Ekle

Bu oldukça uzun işlev aslında oldukça basittir. Bir yay eklemek için, kaynak düğümü ve hedef düğümü bilmemiz gerekir. Kodun büyük bir kısmı kaynak ve hedef düğümleri bulur. Her ikisinin de mevcut olduğundan emin olduktan sonra, basitçe hedef düğüm sırasındaki kaynak komşuluk listesine yeni bir yay ekleriz. Kod, Algoritma 11-5'teki tasarımı takip eden Program 11-4'te gösterilmiştir.

PROGRAM 11-4Grafik Yay Ekle

```

123456789
/* ===== graphInsArc =====
İki tepe arasında bir yay tepe ekler.
    Ön graph, bir grafiğe işaretçidir fromK
    ey, başlangıç tepe anahtarına işaretçidir to
    Key hedef tepe anahtarına işaretçidir Sonra Ya
    y, komşuluk listesine eklendi Başarı duru
    mu döndür +1 başarılıysa -1 b
    ellek taşması durumunda
    -2 if fromKey bulunamazsa
    -3 if toKey bulunamazsa

*/
int graphInsArc (GRAPH* grafiği, void* pFromKey,
                void* pToKey)
{
    // Yerel Tanımlar
    ARC* newPtr;
    ARC* arcPredPtr;
    ARC* arcWalkPtr;
    VERTEX* vertFromPtr;
    VERTEX* vertToPtr;

    // İfadeler
    yeniPtr = (ARC*)malloc(sizeof(ARC));
    if (!newPtr)
        dönüş (-1);

    // Kaynak tepe noktasını bul
    vertFromPtr = graph->first;
    while (vertFromPtr && (graph->compare(pFromKey,
                                         vertFromPtr->dataPtr) > 0))

```

devam ediyor

PROGRAM 11-4Grafik Yay Ekle (devam ediyor)

```

31323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879
{
    vertFromPtr = vertFromPtr->pNextVertex;
} // while vertFromPtr &&
if (!vertFromPtr || (graph->compare(pFromKey,
                                vertFromPtr->dataPtr) != 0))
    return (-2);

// Şimdi tepe noktasına ulaş
versToPte = grafik->ilk;\while (vers
ToPt
&& graph->karşılaştı(pToKey, vertToPtr->dataPtr) > 0)
{
    vertToPtr = vertToPtr->pNextVertex;
} // while vertToPtr &&
if (!vertToPtr ||
    (graph->compare(pToKey, vertToPtr->dataPtr) != 0))
    return (-3);

// Başlangıç ve bitiş noktaları belirlendi. Yeni yay ekle
++vertFromPtr->outDegree;
++vertToPtr->inDegree;
newPtr->hedef = vertToPtr;
if (!vertFromPtr->pArc)
{
    // Bu nokta için ilk yayı ekleme
    vertFromPtr->pArc = newPtr;
    newPtr->pNextArc = NULL;
    return 1;
} // if yeni yay

// Komşuluk (yay) listesinde ekleme noktasını bul
arcPredPtr = NULL;
arcWalkPtr = vertFromPtr->pArc;
while (arcWalkPtr
    && graph->karşılaştı(pToKey,
                        arcWalkPtr->destination->dataPtr) >= 0)
{
    arcPredPtr = arcWalkPtr;
    arcWalkPtr = arcWalkPtr->pNextArc;
} // arcWalkPtr &&

if (!arcPredPtr)
    // İlk yaydan önce ekleme
    vertFromPtr->pArc = newPtr;
else
    arcPredPtr->pNextArc = newPtr;
newPtr->pNextArc = arcWalkPtr;
return 1;
} // graphInsArc

```

Graf Yay Sil

Bu, nispeten basit bir işlem için başka bir uzun işlemdir. Yay ekleme işlemi olduğu gibi, bir yayı kaynak ve hedef köşeleri ile tanımlıyoruz.

Kaynak tepe noktasını bulduğumuzda ve varış noktasına bir yay olduğunu doğruladığımızda, yayı silmek için basit bir bağlı liste silme algoritması kullanıyoruz. Program 11-5'te gösterilen kod, Algoritma 11-6'daki tasarımı takip etmektedir.

PROGRAM 11-5Grafik Yay Sil

```

123456789  /* ===== graphDltArc =====
Mevcut bir yayı siler.
    Ön grafik, grafik baş yapısına işaretçidir fromKey baş
    angıç tepe noktasının anahtarıdır; toKey toKey
    silinecek tepe noktasının varış noktasının anahtar
    ıdır Son Yay silindi
    Başarı durumunda +1 döndür fr
    omKey bulunamazsa -2 döndür
    toKey bulunamazsa -3 döndür

*/
int graphDltArc (GRAPH* grafiği,
                void* Key'den, void* toKey'e)
{
    // Yerel Tanımlar
    VertexPtr'den VERTEX*;
    VERTEX* VertexPtr'ye;
    ARC* preArcPtr;
    ARC* arcWalkPtr;

    // Beyanlar
    eğer (!graph->first)
        -2 döndür;

    // Kaynak tepe noktasını bul
    fromVertexPtr = graph->ilk;
    while (fromVertexPtr && (graph->compare(fromKey,
                                           fromVertexPtr->dataPtr) > 0))
        fromVertexPtr = fromVertexPtr->pNextVertex;

    if (!fromVertexPtr || graph->compare(fromKey,
                                         fromVertexPtr->dataPtr) != 0)
        dönüş -2;

    // Komşuluk listesindeki hedef köşeyi bul
    if (!fromVertexPtr->pArc)
        dönüş -3;

    preArcPtr = NULL;
    arcWalkPtr = fromVertexPtr->pArc;

```

devam ediyor

PROGRAM 11-5Grafik Sil Arc (devam ediyor)

```

40414243444546474849505152535455565758596061
while (arcWalkPtr && (grafik->karşılaştırm(toKey,
    arcWalkPtr->hedef->dataPtr) > 0))
{
    preArcPtr = arcWalkPtr;
    arcWalkPtr = arcWalkPtr->pNextArc;
} // arcWalkPtr && için
if (!arcWalkPtr || (graph->compare(toKey,
    arcWalkPtr->destination->dataPtr) != 0))
    return -3;
toVertexPtr = arcWalkPtr->destination;

// from, toVertex & arcPtr bulundu. Yay sil
--fromVertexPtr->outDegree;
--toVertexPtr->inDegree;
if (!preArcPtr)
    // İlk yayı silme
    fromVertexPtr->pArc = arcWalkPtr->pNextArc;
else
    preArcPtr->pNextArc = arcWalkPtr->pNextArc;
free (arcWalkPtr);
1'i döndür;
// graphDltArc

```

Grafik Derinlik Öncelikli Geçiş

Derinlik öncelikli geçişi incelerken, Şekil11-10, "Bir Grafın Derinlik Öncelikli Geçişi" ne geri dönmek isteyebilirsiniz. Bu algoritmanın ilginç yönlerinden biri, yığının kullanımınıdır. Bir tepe noktasını ve tüm alt noktalarını baştan sona tamamen işlememiz gerektiğini hatırlayın, ardından herhangi bir komşu tepe noktasını işleyiniz. Bu, bir tepe noktasını bir yığına yerleştirmemizi ve ardından işlemek için yığının çıkarmamızı gerektirir. Tasarım, Algoritma 11-8'de gösterilmiştir, kod ise Program 11-6'da bulunmaktadır.

PROGRAM 11-6Graf Derinlik Öncelikli Geçiş

```

123456789
/* ===== graphDpthFrst =====
Grafikte verileri derinlik öncelikli sırayla işleyin.
Önce graph, graf başına işaretçidir
Sonra "işlenmiş" köşeler.

İşlenmiş Bayrağı: 0 = işlenmemiş
                    1 = yığına itildi
                    2 = işlendi
*/
101112
void graphDpthFrst (GRAPH* grafiği,
    void (*process) (void* dataPtr))
{

```

devam ediyor

PROGRAM 11-6Graf Derinlik Öncelikli Geçiş (devam ediyor)

```

131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960
// Yerel Tanımlar bool baş
ar; VERTEX* yürüyüşP
tr; VERTEX* köşePtr;

VERTEX* vertToPtr;
STACK * yığın; ARC
* arcYürüyüşPtr; // İfadel

er
eğer (!graph->first)
    return;

// İşlenmiş bayrakları işlenmemiş olarak ayarla
walkPtr = graph->first;
while (walkPtr) {

    walkPtr->işlenmiş = 0;
    walkPtr = walkPtr->pNextVertex;
} // while

// Listede her bir köşeyi işle
stack = createStack ();
walkPtr = grafik->ilk;
while (walkPtr)
{
    eğer (walkPtr->processed < 2)
    {
        eğer (walkPtr->processed < 1)
        {
            // Yığa ekle ve bayrağı eklenmiş olarak ayarla
            success = pushStack (stack, walkPtr);
            eğer (!success)
                printf("\aYığın taşması 100\a"), exit
                (100);
            walkPtr->processed = 1;
        } // işlenmiş < 1
    } // işlenmiş < 2
    // Yığın tepe noktasındaki kökenlerin işlenmesi
    while (!emptyStack (stack))
    {
        vertexPtr = popStack(stack);
        process (vertexPtr->dataPtr);
        vertexPtr->processed = 2;

        // Komşuluk listesinden tüm köşeleri ekle
        arcWalkPtr = vertexPtr->pArc;
        while (arcWalkPtr)
        {

```

devam ediyor

PROGRAM 11-6 Graf Derinlik Öncelikli Geçiş (devam ediyor)

```

616263646566676869707172737475767778
    vertToPtr = arcWalkPtr->hedef;
    if (vertToPtr->işlenen == 0)
    {
        başarı = pushStack(stack, vertToPtr);
        if (!success)
            printf("\aYığın taşması 101\a"),      exit
            (101);
        vertToPtr->processed = 1;
        } // eğer vertToPtr
    arcWalkPtr = arcWalkPtr->pNextArc;
        } // pWalkArc boyunca

    } // boşYığın değilken
    walkPtr = walkPtr->pNextVertex;
    } // walkPtr boyunca
    Yığın'ı (yığın) yok et; \geri
    dönmek;
    } // graphDpthFirst

```

Grafik Genişlik Öncelikli Geçiş

Yine, bu işlemlerle çalışırken “Grafın Genişlik Öncelikli Geçişi” başlıklı Şekil 11-12’yi incelemenizi öneriyoruz. Bir köşenin tüm komşu köşelerini işlemek istediğimiz için, yapının aşağısına inmeye başlamadan önce bir kuyruk kullanıyoruz. Tasarım, Algoritma 11-9’da, kod ise Program 11-7’de gösterilmiştir.

PROGRAM 11-7 Graf Genişlik Öncelikli Geçiş

```

123456789
/* ===== graphBrdthFirst =====
   Graf verilerini genişlik öncelikli sırayla işleyin.
   Önceki grafik, grafik baş yapısına işaretçidir
   Sonrası grafik işlenmiştir
   İşlenmiş Bayrağı: 0 = işlenmemiş
                     1 = kuyrukta
                     2 = işlenmiş
*/
10111213141516171819
void graphBrdthFirst (GRAPH* grafiği,
                     void (*process) (void* dataPtr))
{
    // Yerel Tanımlar
    bool başarı;
    VERTEX* yürüyüşPtr;
    VERTEX* köşePtr;
    VERTEX* vertToPtr;
    QUEUE* kuyruk;
    ARC* arcWalkPtr;

```

devam ediyor

PROGRAM 11-7Graf Genişlik Öncelikli Geçiş (devam ediyor)

```

202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667
// İfadeler
if (!graph->first)
    return;

// İşlenmiş bayrakları işlenmemiş olarak ayarla
walkPtr = graph->first;
while (walkPtr) {

    walkPtr->işlenmiş = 0;
    walkPtr = walkPtr->pNextVertex;
} // while

// Liste içindeki her bir köşeyi işleme
queue = createQueue ();
walkPtr = grafik->ilk;
while (walkPtr)
{
    if (walkPtr->işlenmiş < 2)
    {
        if (walkPtr->işlenmiş < 1)
        {
            // Kuyruğa ekle ve bayrağı ayarla
            success = enqueue(queue, walkPtr);

            if (!success)
                printf("\aKuyruk taşması 100\a"), exit
                (100);
            walkPtr->processed = 1;
        } // işlenmiş < 1 ise
    } // işlenmiş < 2 ise
    // Kuyruğun önündeki köşenin soylarını işleme
    while (!emptyQueue (queue))
    {
        dequeue(queue, (void**)&vertexPtr);
        süreç (vertexPtr->dataPtr);
        vertexPtr->processed = 2;

        // Komşuluk listesinden köşeleri sıraya ekle
        arcWalkPtr = vertexPtr->pArc;
        while (arcWalkPtr)
        {
            vertToPtr = arcWalkPtr->hedef;
            if (vertToPtr->işlenen == 0)
            {
                başarı = enqueue(queue, vertToPtr);
                if (!success)
                    printf("\aKuyruk taşması 101\a"), e
                    xit (101);
                vertToPtr->işlenen = 1;
            }
        }
    }
}

```

devam ediyor

PROGRAM 11-7Graf Genişlik Öncelikli Geçiş (devam ediyor)

686970717273747576

```

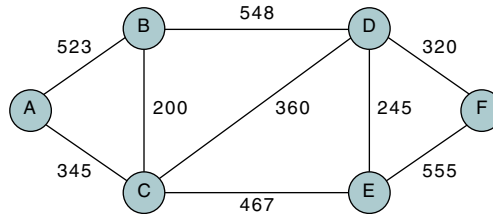
        } // eğer vertToPtr
        arcWalkPtr = arcWalkPtr->pNextArc;
        } // pWalkArc iken
    } // !emptyQueue iken
    walkPtr = walkPtr->pNextVertex;
    } // walkPtr iken
    Kuyruğu yok et(kuyruk); \ger
    i dönmek;
    } // graphBrdthFrst

```

11.6 Ağlar

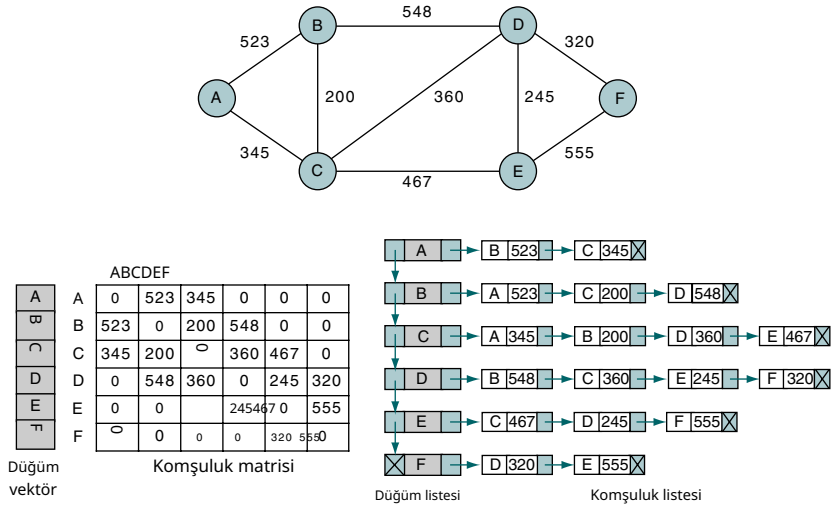
A ağ , kenarlarının ağırlıklı olduğu bir grafikdir. Ayrıca ağırlıklı grafik olarak da bilinir. Ağırlıkların anlamı uygulamaya bağlıdır. Örneğin, bir havayolu, hizmet verdiği şehirler arasındaki rotaları temsil etmek için bir grafik kullanabilir. Bu örnekte, köşeler şehirleri temsil eder ve kenar iki şehir arasındaki bir rota yı temsil eder. Kenarın ağırlığı, iki şehir arasındaki mil sayısını veya uçuşu n fiyatını temsil edebilir. Küçük bir varsayımsal havayolu için bir ağ Şekil 1 1-18'de gösterilmiştir. Bu durumda, ağırlıklar şehirler arasındaki mil mesa fesini temsil eder.

Bir ağ, kenarları ağırlıklı bir grafikdir.



ŞEKİL 11-18Şehir Ağı

Ağırlık bir kenarın bir niteliği olduğundan, kenarı içeren yapıda saklanır. Bir komşuluk matrisinde, ağırlık kesişim değeri olarak saklanır. Bir komşuluk listesinde, komşuluk bağlı listesinde değeri olarak saklanır. Bu iki formatta şehir ağının temsili Şekil 11-19'da gösterilmiştir.



ŞEKİL 11-19Ağırlıkları Grafik Yapılarında Saklama

Şimdi ağların iki uygulamasına dikkat ediyoruz: minimum yayılma ağacı ve bir ağ üzerinden en kısa yol.

Minimum Yayılma Ağacı

Bağlı bir ağdan bir veya daha fazla yayılma ağacı türetebiliriz. Bir yayılma ağacı, grafikteki tüm köşeleri içeren bir ağaçtır.

Bir ilginç algoritma, ağırlıklarının toplamının en az olmasını garanti eden bir ağın minimum yayılma ağacını türetir. Ağdaki ağırlıklar benzersizse, yalnızca bir minimum yayılma ağacı vardır. Eğer tekrar eden ağırlıklar varsa, bir veya daha fazla minimum yayılma ağacı olabilir.

Minimum yayılma ağaçlarının birçok uygulaması vardır, bunların hepsi grafikteki bazı yönleri minimize etme gereksinimi taşır, örneğin grafikteki tüm köşeler arasındaki mesafe. Örneğin, bir bilgisayar ağı verildiğinde, tüm bilgisayarları bağlayan bir ağaç oluşturabiliriz. Minimum yayılma ağacı, tüm bilgisayarları bağlamak için kullanılabilecek en kısa kablo uzunluğu nu sağlar ve herhangi iki bilgisayar arasında bir yol olduğundan emin olur.

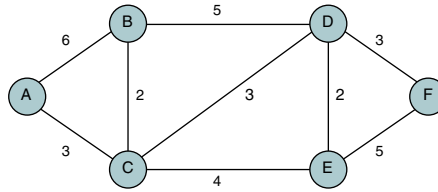
Bir yayılma ağacı, bir grafikteki tüm köşeleri içerir. Minimum yayılma ağacı, çizgilerin toplam ağırlığının grafikteki tüm olası ağaçların en azı olmasını garanti eden bir yayılma ağacıdır.

Güçlü bir şekilde bağlı bir ağda minimum bir yayılma ağacı oluşturmak için—yani, herhangi iki tepe arasında bir yolun bulunduğu bir ağda—minimum yayılma ağacının kenarları, aşağıdaki özelliklerin var olduğu şekilde seçilir:

1. Her tepe dahildir.
2. Yayılma ağacının toplam kenar ağırlığı, herhangi iki tepe arasında bir yol içeren mümkün olan en düşük değerdir.

Minimum Yayılma Ağacı Örneği

Resmi algoritma tanımına geçmeden önce, Şekil 11-20'de gösterilen minimum yayılma ağacını manuel olarak belirleyelim.



ŞEKİL 11-20Yayılma Ağacı

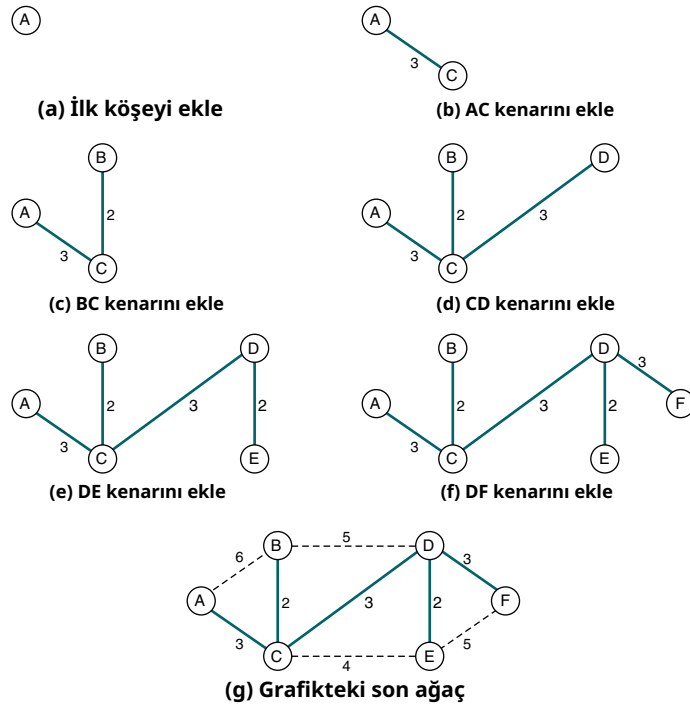
Herhangi bir tepe ile başlayabiliriz. Çünkü tepe listesi genellikle anahtar sıralıdır, A ile başlayalım. Sonra Şekil 11-20'de A ile minimum ağırlıklı kenarı veren tepayı ekleyelim, AC. A ve C ağaçtaki iki tepe olduğuna göre, şimdi minimum ağırlığa sahip kenarı bulalım. AB kenarının ağırlığı 6, BC kenarının ağırlığı 2, CD kenarının ağırlığı 3 ve CE kenarının ağırlığı 4'tür. Bu nedenle minimum ağırlıklı kenar BC'dir. Bu analizde, zaten listede bulunan bir tepeye giden herhangi bir kenarı dikkate almadığımızı unutmayın. Bu nedenle, AC kenarını dikkate almadık.

Bu süreci genelleştirmek için şu kuralı kullanıyoruz: ağaçta mevcut olan tüm verti-çlerden, ağaçta henüz bulunmayan bir verteks için en düşük değere sahip kenarı seçinve bunu ağaca ekleyin. Bu kuralı kullanarak, sırasıyla CD (ağırlık 3), DE (ağırlık 2)ve DF (ağırlık 3) ekliyoruz. Adımlar, Şekil 11-21'de grafiksel olarak gösterilmiştir.

Şekil 11-21(g), orijinal ağ içindeki minimum yayılma ağacını göstermektedir. Ağaçtaki kenarların ağırlıklarını toplarsak, toplam 13'tür. Aynı ağırlıklar olduğu için, farklı bir yayılma ağacı aynı ağırlığa sahip olabilir, ancak hiçbirisi daha düşük bir ağırlığa sahip değildir.

Minimum Yayılma Ağacı Veri Yapısı

Minimum yayılma ağacı algoritmasını geliştirmek için bir depolama yapısına karar vermemiz gerekiyor. En esnek olduğu için, komşuluk listesini kullanıyoruz.



ŞEKİL 11-21 Minimum Yayılma Ağacı Geliştirin

Minimum yayılma ağacını belirlemek için bazı ek unsurlar eklememiz gerekiyor. Her bir köşe düğümünün, köşenin yayılma ağacına eklenip eklenmediğini belirten bir Boolean bayrağına ihtiyacı var. Ayrıca, her bir yay için bir ağaç içi Boolean'a ihtiyaç var. Bu iki alanı `inTree` olarak adlandırıyoruz. Son olarak, yapı bir ağı temsil ettiğinden, her kenarın bir ağırlığı olmalıdır. Ortaya çıkan yapı Algoritma 11-11'de gösterilmektedir.

ALGORİTMA 11-11 Yayılma Ağacı Grafiği için Veri Yapısı

```

grafBaşlık
  sayısı
  ilk
  grafBaşlık sonu

grafDüğüm
  sonrakiDüğüm
  data
  giriş Derecesi
  çıkış Derecesi

```

devam ediyor

ALGORİTMA 11-11 Kapsama Ağaç Grafiği için Veri Yapısı (devam ediyor)

```

girmek
ilkKenar
son graphVertex

graphEdge
  hedef
  ağırlık
  sonrakiKenar
  ar inTree
  son graphEdge

```

Minimum Kapsama Ağaç Pseudocode

Minimum kapsama ağaç algoritması daha önce belirtilen kavramı takip eder . İlk olarak ağaca ilk köşeyi ekleyerek başlar. Daha sonra, tüm köşeler ağaçta olana kadar döngüye girer ve her seferinde ağaca minimum ağırlığa sahip kenarı ekler. Algoritma tamamlandığında, minimum kapsama ağaç, kenarlardan ağaç içi bayraklar ile tanımlanır. Pseudocode, Algoritma 11-12'de gösterilmiştir.

ALGORİTMA 11-12 Bir Grafiğin Minimum Kapsama Ağacı

```

Algoritma yayılmaAğacı (graf)
Bir ağın minimum yayılma ağacını belirleyin.
  önce graf bir ağı içerir
  sonra yayılma ağacı belirlendi
1  eğer (boş graf)
  1  geri dön
2  sonlandır
3  döngü (tüm köşeler üzerinden) InTree
  bayraklarını false olarak ayarlayın.
  1  köşe inTree bayrağını false olarak ayarlayın
  2  döngü (tüm kenarlar üzerinden)
    1  kenar inTree bayrağını false olarak ayarlayın
    2  sonraki kenarı al
  3  döngüyü bitir
  4  bir sonraki köşeyi al
4  döngüyü bitir
Şimdi yayılma ağacını türetin.
5  ilk köşeyi ağaçta ayarla
6  ağaçTamamlandı'yı false olarak ayarla
7  döngü (ağaçTamamlandı değilse)
  1  ağaçTamamlandı'yı true olarak ayarla
  2  minKenar'ı maksimum tamsayı olarak ayarla
  3  minKenarKonumu'nu null olarak ayarla
  4  döngü (tüm köşeler üzerinden)
    Grafiği dolaşarak ağaçtaki köşeleri kontrol et.

```

devam ediyor

ALGORİTMA 11-12 Bir Grafın Minimum Yayılma Ağacı (devam ediyor)

```

1  eğer (köşe ağaçta VE köşe çıkış Derecesi > 0)
1  döngü (tüm kenarlar üzerinden)
1  eğer (varış noktası ağaçta değilse)
    varış noktası ağaçta bayrağını false olarak ayarla
1  ağaçTamamlandı'yı false olarak ayarla
2  eğer (kenar ağırlığı < minEdge)
1  minEdge'i kenar ağırlığı olarak ayarla
2  minEdgeLoc'u kenar olarak ayarla
3  sonlandır
2  sonlandır
3  bir sonraki kenarı al
2  döngüyü bitir
2  sonlandır
3  bir sonraki köşeyi al
5  döngüyü bitir
6  eğer (minEdgeLoc null değilse)
    Ağaçta eklemek için kenar bulundu.
1  minEdgeLoc'u inTree bayrağına true olarak ayarla
2  hedefi inTree bayrağına true olarak ayarla
7  sonlandır
8  döngüyü bitir
SON  spanningTree

```

Algoritma 11-12 Analizi Bu oldukça uzun algoritma iki bölüme kolayca ayrılabilir. İlk bölüm, tüm ağaç içi bayrakları false olarak ayarlayarak grafiği işleme hazırlamaktadır.

İkinci bölüm, tepe grafiği üzerinden döngü yapar (bkz. ifade 7), yayılma ağacına kenarlar ekler. Döngüye başladığımızda, ağaç tamamlandı bayrağını true olarak ayarlıyoruz. Eğer henüz ağaçta olmayan bir tepe bulursak, bunu false olarak ayarlıyoruz (ifade 7.4.1.1.1.1). Grafiğin son geçişinde, yeni kenarlar eklenmez ve ağaç tamamlandı bayrağı true olarak kalır, böylece tepe döngüsü sona erer. Minimum ağırlığa sahip kenarı hatırlamamız gerektiğinden, döngünün başında minimum kenar değişkenini olası herhangi bir ağırlıktan daha büyük bir değere, psödokodda pozitif sonsuzluğa ayarlıyoruz. Aynı zamanda, minimum kenara bir işaretçi null olarak ayarlanır.

Döngü içinde, ağaçta zaten bulunan her bir köşeden kenarlar, minimum değerli kenarı arayan bir iç döngü ile test edilir. Ağaçta olmayan her bir kenar, ağırlığının mevcut minimumdan daha az olup olmadığını belirlemek için test edilir. Eğer öyleyse, ağırlığı ve kenarın konumunu kaydederiz. Döngünün sonunda (ifade 7.6), hem yeni eklenen köşe hem de ona bağlı kenar için bayrakları ayarlarız.

En Kısa Yol Algoritması

Grafiklerle birlikte kullanılan bir diğer yaygın uygulama, bir ağda iki köşe arasındaki en kısa yolu bulmamızı gerektirir. Örneğin, eğer ağ bir havayolu tarafından uçulan rotaları temsil ediyorsa, seyahat ettiğimizde evimiz ile varış noktamız arasındaki en ucuz rotayı bulmak isteriz. Rota grafiğindeki ağırlıklar uçuş ücreti olduğunda, minimum maliyetimiz, başlangıç noktamız ile varış noktamız arasındaki en kısa yoldur. Edsger Dijkstra klasik bir

bu problem için 1959'da bir algoritma.² Algoritması genellikle Dijkstra'nın en kısa yol algoritması olarak bilinir.

Dijkstra algoritması, bir grafikteki herhangi iki düğüm arasındaki en kısa yolu bulmak için kullanılır.

En Kısa Yol Manuel Örneği

Resmi algoritmayı geliştirmeden önce, bir örneği inceleyelim. Aynı grafiği minimum spanning tree için kullandığımız gibi kullanıyoruz. Bu örnekte, A köşesinden grafikteki herhangi bir diğer köşeye en kısa yolu bulmak istiyoruz. Sonuç, A köşesinin kök olduğu bir ağaçtır. Analizimizin sonucu Şekil 11-22'de gösterilmiştir.

Algoritma, minimum spanning tree algoritmasına benzer. Başlangıç noktasını ağaca ekleyerek başlıyoruz. Ardından, başlangıç noktasına bitişik olan tüm köşelere giden yolları inceliyoruz ve minimum ağırlığa sahip yolu ağaca ekliyoruz. Artık ağacımızda iki köşe var, Şekil 11-22(a2)'de gösterildiği gibi.

Ardından, ağaçtaki iki köşe, A ve C'den, bitişik olan tüm köşelere giden yolları inceliyoruz. Ancak, spanning tree'de yaptığımız gibi bir sonraki köşeye en kısa yolu belirlemek yerine, bitişik köşelere toplam yol uzunluğunu belirliyoruz. Her köşeye olan toplam yol uzunluğu, bitişik köşenin yanında T: n notasyonu ile Şekil 11-22'de gösterilmiştir. Şekil 11-22(b1)'i incelediğinizde, toplam ağırlığı 6 olan AB, toplam ağırlığı 5 olan CB, toplam ağırlığı 6 olan CD ve toplam ağırlığı 7 olan CE olmak üzere dört yol olduğunu görüyorsunuz. Minimum yol uzunluğunu, CB (T: 5) seçiyoruz ve bunu ağaçta yerleştiriyoruz. Ortaya çıkan ağaç Şekil 11-22(b2)'de gösterilmiştir.

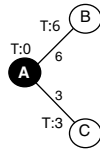
Grafikte üçüncü geçişte, ağaçtaki (A, C ve B) köşelerden ağaçta henüz olmayan köşelere (D ve E) giden yolları inceliyoruz. A'dan hiçbir yol yok. B'den toplam ağırlığı 10 olan bir yol, BD var. C'den toplam ağırlığı 6 olan CD ve toplam ağırlığı 7 olan CE olmak üzere iki yol var. Toplam ağırlığı 6 olan minimum yolu, CD'yi seçiyoruz.

İzlediğimiz adımları genelleştirmeye çalışalım:

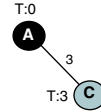
1. İlk köşeyi ağaca ekleyin.
2. Ağaçta zaten bulunan her köşeden, ağaçta olmayan tüm komşu köşelere toplam yol uzunluğunu inceleyin. Minimum toplam yol ağırlığına sahip kenarı seçin ve ağaca ekleyin.
3. Adım 2'yi, tüm köşeler ağaçta olana kadar tekrarlayın.

Bu adımları izleyerek, Şekil 11-22(d2)'de gösterilen CE kenarını ekliyoruz ve son olarak Şekil 11-22(e2)'de gösterilen DF kenarını ekliyoruz.

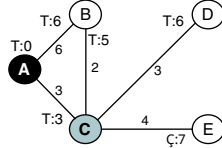
2. E. W. Dijkstra, "Grafiklerle İlgili İki Problem Üzerine Bir Not," *Numerische Mathematik* 1 (1959): 269–271.



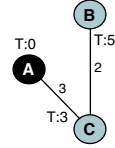
(a1) A1'den olası yollar



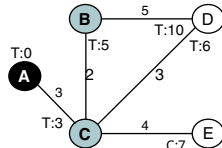
(a2) C'nin eklenmesinden sonraki ağaç



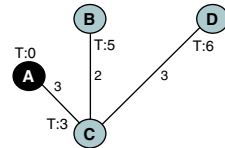
(b1) A ve C'den olası yollar



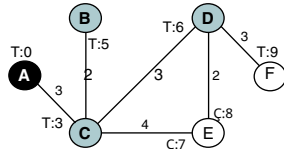
(b2) B'nin eklenmesinden sonraki ağaç



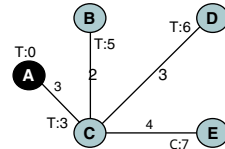
(c1) B ve C'den olası yollar



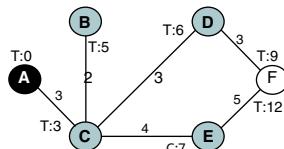
(c2) D'nin eklenmesinden sonraki ağaç



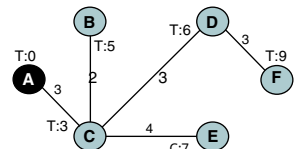
(d1) C ve D'den olası yollar



(d2) E'nin eklenmesinden sonraki ağaç



(e1) D ve E'den olası yollar



(e2) F'nin eklenmesinden sonraki ağaç

T:n A'dan düğüme toplam yol uzunluğu

ŞEKİL 11-22En Kısa Yolun Belirlenmesi

En Kısa Yol Pseudocode

Artık algoritmayı geliştirmeye hazırız. Minimum yolu belirlemek için, düğüm yapısına bir alan daha eklememiz gerekiyor, toplam yol uzunluğu. Buna `pathLength` diyoruz. Pseudocode, Algoritma 11-13'te gösterilmiştir.

ALGORİTMA 11-13En Kısa Yol

Algoritma `shortestPath (graf)Ağ düğümünden diğer düğümlere en kısa yolu belirleyin.`

Ön graf, ağa işaretçidir

Son minimum yol ağacı belirlendi

devam ediyor

ALGORİTMA 11-13 En Kısa Yol (devam ediyor)

```

1  eğer (boş graf)
    1  geri dön
2  sonlandır
3  döngü (tüm köşeler üzerinden)
    InTree bayraklarını ve yol uzunluğunu başlat.
    1  düğüm inTree bayrağını false olarak ayarla
    2  düğüm pathLength değerini maksimum tamsayı olarak ayarla
    3  döngü (tüm kenarlar üzerinden)
        1  kenar inTree bayrağını false olarak ayarlayın
        2  bir sonraki kenarı al
    4  döngüyü bitir
    5  bir sonraki köşeyi al
4  döngüyü bitir
    Şimdi minimum yol ağacını türet.
5  ilk düğüm inTree bayrağını true olarak ayarla
6  düğüm pathLength değerini 0 olarak ayarla
7  ağaçTamamlandı'yı false olarak ayarla
8  döngü (ağaçTamamlandı değilse)
    1  ağaçTamamlandı'yı true olarak ayarla
    2  minKenarKonumu'nu null olarak ayarla
    3  pathLoc değerini null olarak ayarla
    4  yeniPathLen değerini maksimum tamsayı olarak ayarla
    5  döngü (tüm köşeler üzerinden)
        Grafiği dolaşarak ağaçtaki köşeleri kontrol et.
        1  eğer (düğüm ağa dahil VE outDegree > 0)
            1  kenarLoc değerini firstEdge olarak ayarla
            2  minPath değerini pathLength olarak ayarla
            3  minKenar'ı maksimum tamsayı olarak ayarla
            4  döngü (tüm kenarları üzerinden)
                Bu köşeden en küçük yolu bul.
                1  eğer (hedef ağaçta yoksa)
                    1  ağaçTamamlandı'yı false olarak ayarla
                    2  eğer (kenar ağırlığı < minEdge)
                        1  minEdge'i kenar ağırlığı olarak ayarla
                        2  minEdgeLoc'u edgeLoc olarak ayarla
                    3  sonlandır
                2  sonlandır
                3  edgeLoc'u edgeLoc nextEdge olarak ayarla
            5  döngüyü bitir
        En kısa yolu test et.
        6  eğer (minPath + minEdge < newPathLen)
            1  newPathLen'i minPath + minEdge olarak ayarla
            2  pathLoc'u minEdgeLoc olarak ayarla
        7  sonlandır
    2  sonlandır
    3  bir sonraki köşeyi al
6  döngüyü bitir

```

devam ediyor

ALGORİTMA 11-13 En kısa yol (devamı)

```

7  eğer (pathLoc null değilse)
    Ağaç içine eklemek için kenar bulundu.
    1  pathLoc      inTree bayrağını true olarak ayarla
    2  pathLoc varış inTree bayrağını true olarak ayarla
    3  pathLoc varış pathLength'ini newPathLen olarak ayarla
8  sonlandır
9  döngüyü bitir
SON shortestPath

```

Algoritma 11-13 Analizi Minimum yayılma ağacında gördüğümüz gibi, en kısa yol algoritması, ağaç içindeki bayrakları false olarak ayarlayan ve tepe yol uzunluğunu maksimum olası uzunluktan (maksimum tamsayı) daha büyük bir sayıya başlatan birbaşlatma döngüsü ile başlar.

Vertex'leri başlattıktan sonra, tüm vertex'lere en kısa yolu belirlemeye hazırız. Grafikteki ilk vertex ile başlıyoruz, grafiğin ilkişaretçisi tarafından gösterildiği gibi. Başka bir vertex'ten en kısa yolu belirlememiz gerekiyorsa, başlangıç noktasını algoritmaya iletmemiz gerekir.

Grafikteki her döngü (ifade 8), başlangıçta yalnızca A olan herhangi bir bağlı tepeye en kısa yola sahip olan tepeleri ekler. Ağaçtaki her tepe için (ifade 8.5), tüm komşu tepelerini test ederiz ve ağaçta zaten olmayan bir tepeye minimum ağırlığı belirleriz (ifade 8.5.1.4). Her yol kenarını bulduğumuzda, bunun daha önceki herhangi bir yol kenarından daha az olup olmadığını test ederiz (ifade 8.5.1.6); eğer öyleyse, yeni yol uzunluğunu ve kenara işaretçiyi kaydederiz.

Kısa yol kenarını bulduğumuzda, onu ekliyoruz (ifade 8.7) ve grafiği tekrar kontrol etmek için geri dönüyoruz (ifade 8).

11.7 Anahtar Terimler

komşuluk listesi	giriş derecesi
komşuluk matris	hat
bitişik köşeler	döngü
yay	minimum yayılma ağacı
genişlik öncelikli gezinti	şebeke
bağlı	çıkış derecesi
döngü	siyol
derece	yayılma ağa
derinlik öncelikli gezinti	cışiddetle bağlıyö
diyagram	nsüz grafikdüş
yönlendirilmiş grafik (diyagram)	üm (düşümler
disjoint grafik	)düşüm
kenar	listesizayf bağlı
grafik	ağırlıklı grafik

11.8 Özet

- Bir grafik, düğümlerden oluşan bir koleksiyondur, bunlara köşeler denir ve düğüm çiftlerini bağlayan kenar veya yay olarak adlandırılan bir dizi doğru parçasından oluşur.
- Grafikler yönlendirilmiş veya yönlendirilmemiş olabilir. Yönlendirilmiş bir grafikte, ya da digrafta, her bir çizginin bir yönü vardır. Yönlendirilmemiş bir grafikte ise çizgilerin yönü yoktur. Yönlendirilmiş bir grafikteki bir çizgiye yay denir.
- Bir grafikte iki köşe, bir kenar doğrudan onları bağlıyorsa komşu olarak adlandırılır.
- Bir yol, her bir köşenin bir sonraki köşeye komşu olduğu köşelerin bir dizisidir.
- Bir döngü, en az üç köşeden oluşan ve aynı köşeye başlayan ve biten bir yoldur.
- Bir döngü, tek bir yayının aynı düğümle başlayıp bittiği döngünün özel bir durumudur.
- Bir grafiğin bağlı olduğu söylenir eğer, herhangi iki köşe için, birinden diğerine bir yol varsa. Bir grafik bağlı değilse, ayrıık olarak adlandırılır.
- Bir köşenin derecesi, ona bitişik olan köşe sayısıdır. Bir köşenin çıkış derecesi, düğümden çıkan yay sayısıdır; bir köşenin giriş derecesi ise düğüme giren yay sayısıdır.
- Bir grafik için altı işlem tanımlanmıştır: bir köşe eklemek, bir köşeyi silmek, bir kenar eklemek, bir kenarı silmek, bir köşe bulmak ve bir grafiği gezmek.
 - Bir köşe ekleme işlemi, bir grafiğe başka bir köşeye bağlamadan yeni bir köşe ekler.

- Düğüm silme işlemi, bir düğümü bir grafikten kaldırır.
- Kenar ekleme işlemi, bir kaynak düğüm ile bir hedef düğüm arasında bir kenar ekler.
- Kenar silme işlemi, kaynak düğümü hedef düğüme bağlayan bir kenarı kaldırır.
- Bir tepe noktası işlemi, belirli bir tepe noktasını arayarak bir grafiği tarar.
- Bir grafi gezmek, grafikteki tüm düğümleri ziyaret eder ve bunları birbir işlem e alır.
- İki standart grafik gezintisi vardır: derinlik öncelikli ve genişlik öncelikli.
 - Derinlik öncelikli gezintide, bir düğümün tüm soyundan gelenler, komşu bir düğüme geçmeden önce işlenir.
 - Genişlik öncelikli gezintide, bir düğümün tüm komşu köşeleri, bir köşenin soyundan gelenleri işlemeye geçmeden önce işlenir.
- Bir grafi bir bilgisayarda temsil etmek için, iki bilgi seti saklamamız gerekir: ilk set köşeleri, ikinci set ise kenarları temsil eder.
- Bir grafi saklamak için en yaygın yöntemler, komşuluk matris yöntemi ve komşuluk listesi yöntemidir.
 - Komşuluk matris yönteminde, köşeleri saklamak için bir vektör ve kenarları saklamak için bir matris kullanırız.
 - Komşuluk listesi yönteminde, köşeleri saklamak için bir bağlı liste ve kenarları saklamak için iki boyutlu bir bağlı liste kullanırız.
- Bir ağ, çizgileri ağırlıklı olan bir grafikdir.
- Bir kapsayıcı ağaç, grafikteki tüm köşeleri içerir.
- Minimum bir yayılma ağacı, kenarların toplam ağırlığının en az olduğu bir yayılma ağacıdır.
- Grafiklerle birlikte kullanılan bir diğer yaygın algoritma, iki tepe noktası arasında ki en kısa yolu bulur.

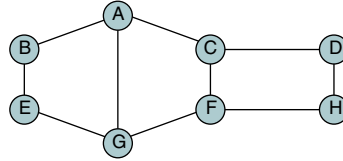
11.9 Pratik Setleri AI

İştirmalar

1. Şekil 11-23'teki grafikte şunları bulun:

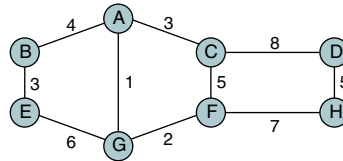
- a. A'dan H'ye tüm döngüsel olmayan yollar
- b. C'den E'ye tüm döngüsel olmayan yollar
- c. B'den F'ye tüm döngüsel olmayan yollar

2. Şekil 11-23'teki grafikte, şunlara komşu olan tüm düğümleri bulun:
- düğüm A
 - düğüm F
 - düğüm G



ŞEKİL 11-23 Alıştırmalar 1'den 8'e kadar olan grafik

- Şekil 11-23'teki grafikte, A, E, F, G ve H düğümlerinin derecesini, çıkış derecesini ve giriş derecesini bulun.
- Şekil 11-23'teki grafiğin derinlik öncelikli gezintisini, A düğümünden başlayarak ak verin.
- Şekil 11-23'teki grafiğin genişlik öncelikli gezintisini, A düğümünden başlayarak ak verin.
- Şekil 11-23'teki grafikte bulunabilen üç kapsayıcı ağaç çizin.
- Şekil 11-23'teki grafiğin komşuluk matrisinin temsilini verin.
- Şekil 11-23'teki grafiğin komşuluk listesinin temsilini verin.
- Şekil 11-24'teki grafiğin minimum yayılma ağacını bulun.



ŞEKİL 11-24 Alıştırmalar 9'dan 12'ye kadar olan grafik

- A noktasından grafikteki diğer tüm noktalara en kısa yolu bulun Şekil 11-24'te.
- Şekil 11-24'teki grafiğin komşuluk matrisini verin.
- Şekil 11-24'teki grafiğin komşuluk listesini verin.
- Şekil 11-25'teki komşuluk matrisinin yönlendirilmiş grafiğini çizin.

	A	B	C	D	E	F
ABCDEF	0	3	4	0	2	1
A	0	0	2	0	0	3
B	0	0	0	2	6	1
C	2	6	1	0	1	2
D	0	0	0	0	0	3
E	0	0	0	0	0	0
F	0	0	0	0	0	0

ŞEKİL 11-25 Alıştırma 13 için Komşuluk Matrisi

14. Bir grafik ilişkileri göstermek için kullanılabilir. Örneğin, aynı kulübe ait olan (düğüm) ve arkadaşlıkları (kenar) gösteren aşağıdaki listede en şunları bulun:

İnsanlar = {George, Jim, Jean, Frank, Fred, John, Susan}
 Arkadaşlık = {(George, Jean), (Frank, Fred),
 (George, John), (Jim, Fred),
 (Jim, Frank), (Jim, Susan),
 (Susan, Frank)}

- John'un tüm arkadaşları
- Susan'ın tüm arkadaşları
- Jean'ın arkadaşlarının tüm arkadaşları
- Jim'in arkadaşlarının tüm arkadaşları

Problemler

- Bir düğümün ayrık olup olmadığını belirleyen bir algoritma yazın.
- Bir düğümü ayrık hale getiren bir algoritma yazın.
- Bir düğüm için derecelerin toplamını bulan bir algoritma yazın, komşuluk listesi temsili kullanarak.
- Hem derinlik öncelikli hem de genişlik öncelikli gezintiler ayrık grafikleri işler. Bir grafiği gezen ve grafik bağlıysa true, ayrık ise false döndüren bir algoritma yazın.
- Belirli bir tepeye giden en az bir yay noktası olup olmadığını belirleyen bir algoritma yazın. (*İpucu:* Bu çok basit bir algoritmadır.)
- Zayıf bağlı bir grafikte, bir köşeden başlayıp başka birine ulaşmak mümkün olmayabilir. Grafiği, bir kaynak köşesini ve bir hedef köşesini verildiğinde, kaynak ile hedef arasında en az bir yol olup olmadığını belirleyen bir algoritma yazın.
- Algoritma 11-8, "Derinlik Öncelikli Geçiş," grafiği geçmek için bir yığın kullanır. Algoritmayı özyineleme kullanacak şekilde yeniden yazın. (*İpucu:* Kullanılan algoritma ile arayüz oluşturmak için bir, ve özyineleme için bir algoritmaya ihtiyacınız var.)

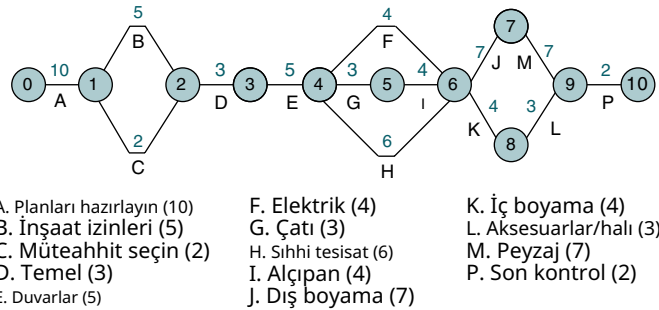
Projeler

22. Algoritma 11-12, "Bir Grafiğin Minimum Yayılma Ağaçları" için metinde veril en ADT'yi kullanarak C kodunu yazın.
23. Algoritma 11-13, "En Kısa Yol" için metinde verilen ADT'yi kullanarak C k odunu yazın.
24. Bir grafiğin minimum yayılma ağacını yazdıran bir algoritma yazın. Son unda, yayılma ağacının ağırlığını yazdırın. Önerilen bir rapor formatı aş ğıdaki örnekte gösterilmiştir.

Kaynak Köşe	Hedef Köşe	Ağırlık
ABAB		2
klıma		4
BDBD		3
İLE İLGİLİ		1
Yayılma ağının toplam ağırlığı: 10		

25. Veri yapısı olarak bir komşuluk matrisini kullanarak grafik ADT'sini gözden geçirin.
26. Büyük projeleri yönetmek için kullanılan araçlardan biri kritik yol yöntemi (CPM). CPM'de yönetici, bir projenin tüm aşamalarının bir ağını ol uşturur ve ardından projenin kritik yönlerini belirlemek için ağı değerle ndirir.

Bir CPM ağında, her bir köşe bir olaydır, örneğin bir görevin başlang ıcı veya tamamlanması. Köşeleri bağlayan yaylar, etkinliğin süresini temsil eder. Metindeki örneklerin aksine, bunlar ayrıca etkinliğin adını da saklar. Kavramı daha iyi anlamak için, bir ev inşa etmek için olası bir CPM planına bakalım. Bu projenin ağı Şekil 11-26'da gösterilmektedir.



ŞEKİL 11-26 Proje 26: Bir Ev İnşa Etme Adımları

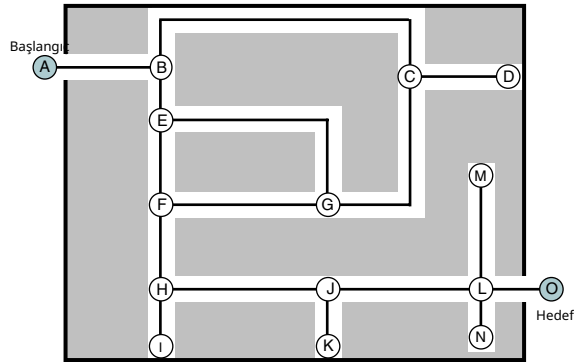
Planda, bina planını hazırlamanın 10 gün (A) ve onaylatmanın 5 gün (B) alacağını görüyoruz. Ayrıca, müteahhidi seçene kadar inşaata başlayamayız (C).

Planımız için başlangıçtan sona en kısa yolu inşa edebildik, ancak bunun pek bir değeri olmazdı. Öte yandan, eğer belirleseydik

maksimum yol—yani, en büyük ağırlık toplamına sahip yol—planım ızdaki hangi adımların kritik olduğunu bileceğiz. Eğer kritik bir adım bir gün bile kayarsa, bitiş tarihimiz kayar. Ancak, kritik yolu değiştirmediği sürece, bitiş tarihimiz kaymadan kritik olmayan tarihleri kaydırabiliriz.

Algoritma 11-12, “Bir Grafın Minimum Yayılma Ağaçları”nı değiştirin ve graf üzerindeki maksimum yolu belirleyin. Ardından proje yöneticisinin aşığıdaki soruları yanıtlamasına olanak tanıyan bir menü sağlayın:

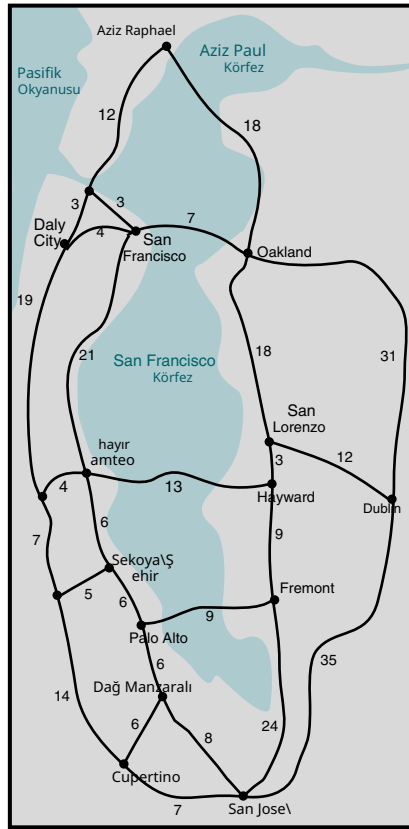
- a. En kısa olası tamamlama süresi (SPCT) nedir? SPCT, grafın baştan sona kadar olan en uzun yoludur.
 - b. Her bir etkinlik için en erken başlangıç zamanı (EST) nedir? EST, etkinliğ e kadar olan maksimum yayılma ağacındaki ağırlıkların toplamıdır.
 - c. Her bir etkinlik için en geç başlangıç zamanı (LST) nedir? LST, tüm pr oje için SPCT'den, projenin geri kalan kısmı için SPCT'yi (mevcut etkin likten başlayarak) çıkarmaktır.
 - d. Her bir etkinlik için kayma süresi nedir? Kayma süresi $LST - EST$ 'dir.
 - ve. Bir etkinlik kritik yol ögesi midir? (Kritik yol ögelerinin kayma süresi sı fırdır.)
 - f. Proje için kritik yol nedir? (Kritik yol, maksimum yayılma ağacını oluşturan alt grafi temsil eder.)
27. Grafik, labirent problemini çözmek için kullanılabilecek başka bir yapıdır (bkz. Bölüm 3'teki Proje 24). Her başlangıç noktası, çıkmaz, hedef ve karar noktası bir düğümle temsil edilebilir. Düğümler arasındaki yollar, labirentteki bir olası yolu temsil eder. Bir grafik labirenti Şekil 11-27'de gösterilmektedir.



ŞEKİL 11-27 Proje 27 için Grafik Labirenti

Bir farenin labirentteki hareketini simüle eden bir program yazın, grafik ve derinlik öncelikli tarama kullanarak. Program tamamlandığında, labirentteki yolu yazdırın.

28. Silicon Valley bölgesinde bir bilgisayar şirketi (bkz. Şekil 11-28), şehirler arasında en kısa rotada teslimat araçlarını yönlendirmeye ihtiyaç duymaktadır. Veri yapılarını inceledikten sonra, bunun Dijkstra'nın en kısa yol algoritması için bir uygulama olduğunu kabul ediyorsunuz. Teklifinizi göstermek için, bunu bilgisayarınızda uygulamaya karar veriyorsunuz. Bunu yapmak için aşağıdaki görevleri tamamlamanız gerekiyor:
- Şekil 11-28'deki haritayı bir ağa dönüştürün ve yönetime sunun.
 - Yaygın grafik ADT'sini yollarındaki ağırlıkları depolayacak şekilde değiştirin.
 - Başlangıç ve varış noktası verildiğinde aralarındaki en kısa rotayı gösteren etkileşimli bir program yazın.



ŞEKİL 11-28 Silicon Valley Bölgesi Haritası

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm IV

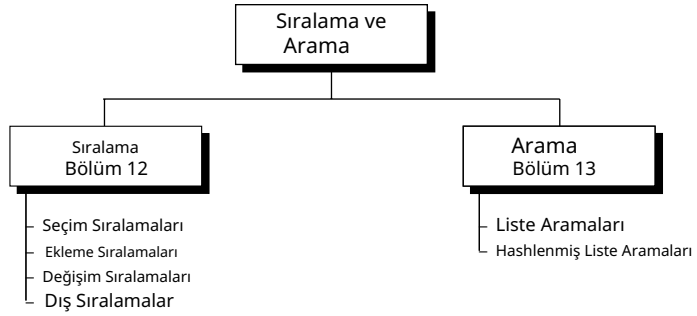
Sıralama ve Arama

Kitabın son bölümü, soyut veri türleri olarak kabul edilmeyen ancak çok yakın ilişkili iki konuyu ele alıyor: sıralama ve arama.

Sıralama ve arama, bir giriş programlama kursunda kısaca tartışılmalıdır, tartışma genellikle ileri düzey kavramlara girmemektedir. Bu bölümde, ileri düzey sıralama ve arama algoritmalarını tartışıyoruz.

Şekil IV-1, bu bölümde yer alan bölümleri ve her bölümde tartışılan konuları göstermektedir.

İki bölümde, sıralama ve aramanın karmaşıklığı hakkında tartışmalar buluyoruz. Bölüm 1'de geliştirilen fikirler, her sıralama ve arama algoritmasına uygulanmakta ve bazı resmi olmayan ama basit kanıtlar verilmektedir. Karmaşıklık analizi, bu bölümde geliştirilen hızlı sıralamalar ile giriş programlama kursunda tartışılan yavaş sıralamalar arasındaki farkı anlamamıza olanak tanır.



ŞEKİL IV-1

Uygulama

Bu kitabın bu bölümünde geliştirilen sıralama ve arama algoritmaları, C dilinde bağımsız işlevler olarak doğrudan uygulanmaktadır. Soyut veri türleri, genel iş evler geliştirmek için kullanılabilir, ancak bu bölümde bunları kullanmıyoruz. Bunun yerine, algoritmayı basit tutarak algoritma mantığına odaklanmamıza olanak tanıyoruz.

Bölmeler Kapsandı

Bu bölüm iki bölümü içermektedir.

Bölüm 12: Sıralama

Bu bölümde sıralamayı tartışıyoruz. Bölüm, sıralama kavramlarının genel bir tartışmaıyla başlar. Ardından dört sıralama kategorisini tanıtlıyoruz: seçim sıralaması, ekleme sıralaması, değişim sıralaması ve diş sıralama.

Bölüm 13: Arama

Bu bölümde aramayı tartışıyoruz. Temel sıralı arama ile başlıyoruz, birkaç varyasyonunu da dahil ediyoruz ve ikili aramayı ele alıyoruz. Hash'li liste aramaları ve çıkışma çözümü ile ilgili bir tartışma ile sonlandırıyoruz.

Bölüm 12

Sıralama

Bilgisayar bilimlerinde en yaygın uygulamalardan biri, verilerin değerlerine göre düzenlendiği süreç olan sıralamadır. Verilerle çevriliyiz. Eğer veriler sıralı olmasaydı, tek bir bilgi parçasını bulmak için saatler harcardık. İçsel bir düzeni olmayan bir telefon rehberinde birinin telefon numarasını bulmanın zorluğunu hayal edin.

Sıralamanın tarihi, bilgisayar biliminin köklerine kadar uzanır. Herman Hol-lerith'in elektrikli tablo makinesi, 1890 ABD Nüfus Sayımı'nı saymak için kullanıldı ve modern sıralama makinelerinden biri olarak kabul edildi. Genel amaçlı bilgisayarlar ilk kez kullanılmaya başlandığında sıralama da sahnedeydi. Knuth'a göre, "Bir sıralama rutinine dair kanıtlar, saklanan program bilgisayarı için yazılan ilk programdır."¹ Bilgisayar bilimcileri son 30 yılda büyük bir yeni algoritma geliştirmemiştir (bu kitapta yer alan en yeni algoritma 1964'te geliştirilen heap sıralamasıdır), ancak sıralama, günümüzde bilgisayarda en önemli kavramlardan biri olmaya devam etmektedir.

12.1 Kavramları Sıralama

Bu bölümde altı iç sıralama türünü tartışıyoruz: ekleme sıralaması, balon sıralaması, s eçim sıralaması, shell sıralaması, yığın sıralaması ve hızlı sıralama. İlk üçü yalnızca çok küçük listeleri sıralamak için yararlıdır, ancak son üçünün temelini oluştururlar; bu s on üçü ise genel amaçlı sıralama kavramlarıdır. İç sıralamaları tartıştıktan sonra, dış s ıralamalarda kullanılan temel kavramları tanıtaçız.

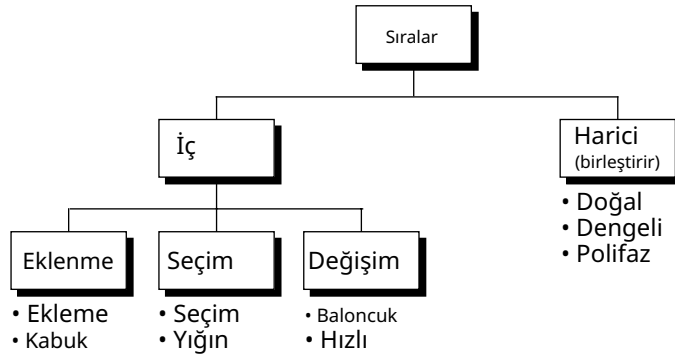
Sıralama, en yaygın veri işleme uygulamalarından biridir.

1. Donald E. Knuth, *Bilgisayar Programlamanın Sanatı*, cilt 3, *Sıralama ve Arama*, İkinci Baskı (Reading, MA: Addison-Wesley, 1998), 385.

Sıralamalar genellikle iç veya dış sıralamalar olarak sınıflandırılır. Bir iç sıralamada, sıralama işlemi sırasında tüm veriler birincil bellek içinde tutulur. Bir dış sıralama, sıralanan veriler için birincil belleği ve birincil belleğe sığmayan veriler için ikincil depolamayı kullanır. Örneğin, 20.000 kayıttan oluşan bir dosya, yalnızca 1000 kaydı tutan bir dizi kullanılarak sıralanabilir. Sıralama işlemi sırasında, her seferinde yalnızca 1000 kayıt bellekte bulunur; diğer 19.000 kayıt ise ikincil depolamada bir dosya da tutulur.

Sıralama algoritmaları, ya içsel ya da dışsal olarak sınıflandırılır.

İçsel sıralama algoritmaları, sıralama için genel yaklaşımlarına bağlı olarak birkaç farklı sınıflandırmaya ayrılmıştır. Knuth, beş farklı sınıflandırmaya belirlemiştir: ekleme, seçim, değiştirme, birleştirme ve dağıtım sıraları. Bu metinde ilk üçünü ele alıyoruz. Dağıtım sıraları, ilginç olmalarına rağmen, bilgisayarlarda minimal kullanıma sahiptir. Farklı sıralar Şekil 12-1'de gösterilmektedir.



ŞEKİL 12-1 Sıralama Sınıflandırmaları

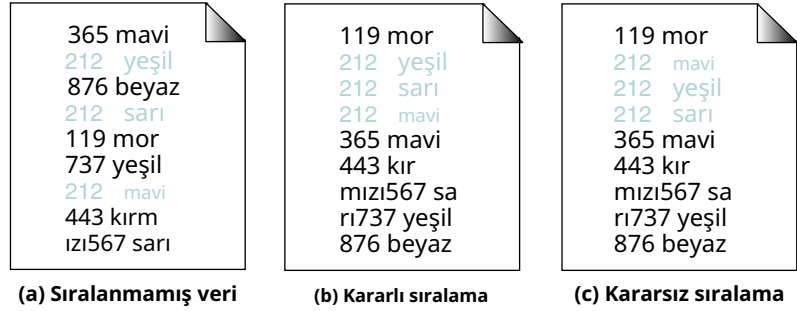
Sıralama Düzeni

Veriler, ya artan sırada ya da azalan sırada sıralanabilir. Sıralama düzeni, sıralanan verilerin artan veya azalan sırasını tanımlar. Sıralamanın düzeni belirtilmemişse, artan olduğu varsayılır. Artan sırada sıralanan yaygın veri örnekleri sözlük ve telefon rehberi dir. Azalan sırada sıralanan yaygın veri örnekleri, bir spor etkinliğinde kazanılan oyunların yüzdeleri veya onur öğrencileri için not ortalamalarıdır.

2. Knuth'a bakınız, Bilgisayar Programlamanın Sanatı, 73-179.

Sıralama Kararlılığı

Sıralama kararlılığı, eşit anahtarlara sahip verilerin çıktıdaki görelî giriş sırasını koruduğunu belirten bir sıralama özelliğidir. Kararlılık en iyi bir örnekte görülür. Şekil 12-2(a)'da sıralanmamış veriler, aynı anahtara (212) sahip üç girdi içerir. Veriler kararlı bir sıralama ile sıralandığında, Şekil 12-2(b)'deki sıra garanti edilir. Yani, 212 yeşil, çıktıda üçlünün ilki olarak garanti edilir, 212 sarı ikinci olarak garanti edilir ve 212 mavi son olarak garanti edilir. Sıralama kararlı değilse, aynı anahtara sahip kayıtlar herhangi bir sırada, Şekil 12-2(b)'de gösterilen kararlı sırayı da içerecek şekilde ortaya çıkabilir. Şekil 12-2(c), kararsız bir sıralamada meydana gelebilecek altı farklı diziden bir örnektir. Bu örnekte, mavi, eşit anahtarların sonuncusu olmasına rağmen ilk sırada çıkmaktadır. Bu metinde tartışacağımız sıralama algoritmlerinden düz ekleme sıralaması ve baloncuk sıralaması kararlıdır; diğerleri kararsızdır.



ŞEKİL 12-2 Sıralama Kararlılığı

Sıralama Verimliliği

Sıralama verimliliği, bir sıralamanın görelî verimliliğinin bir ölçüsüdür. Genellikle, sıralanmamış bir listeyi sıralamak için gereken karşılaştırma ve hareket sayısının bir tahminidir. Bu bölümde ele aldığımız her bir iç sıralamanın sıralama verimliliğini tartışıyoruz. Ancak genel olarak, en iyi sıralama algoritmaları $n \log n$; yani, $O(n \log n)$ sıralamalarıdır.³ İncelediğimiz sıralamalardan üçü $O(n^2)$. En iyisi, hızlı sıralama, $O(n \log n)$ dir.

Geçişler

Sıralama sürecinde, veriler birçok kez geçer. Verilerin her geçişi sıralama geçişi olarak adlandırılır. Algoritmaya bağlı olarak, sıralama

3. Bir proje olarak, ilginç bir sıralama olan radix sıralamasını tartışıyoruz—bu $O(n)$ olarak ifade edilir. Ancak, geniş overhead'i genel kullanımını sınırlar.

geçiş, tüm listeyi veya listenin sadece bir bölümünü geçebilir. Ayrıca, bir sıralama geçişinin bir karakteri, bir veya daha fazla öğenin sıralı bir listede yerleştirilmesidir.

Sıralamalar ve ADT'ler

Sıralama algoritmaları, Bölüm 1.5'te tartışılan teknikleri kullanarak bir ADT uygulamasına kolayca uyarlanabilir, "ADT'ler için Genel Kod." Önceki bölümlerdeki ADT'ler ile sıralama ADT'leri arasında bir önemli fark vardır: sıralama ADT'lerinde veri türü tanımlanır ve uygulamada doldurulur. Daha sonra bir sıralama için ADT'ye iletilir.

Bilinmeyen türde ve boyutta verileri sıralamak için ADT'nin aşağıdaki prototip ifadesinde gösterildiği gibi dört parametreye ihtiyacı vardır:

```
bool sortADT (geçersiz* ary,
              int sizeofElem,
              int numElem,
              int (*karşılaştır)(void* arg1, void* arg2));
```

Sıralanacak veriler *void* işaretçi olarak alınır. Bir elemanı dizide bulmak için diziyi karakterler dizisi olarak ele alabiliriz ve bir elemandan diğerine geçmek için işaretçi aritmetiği kullanabiliriz. Örneğin, dizinin beşinci elemanı na erişmek için aşağıdaki ifadeyi kullanırız:

```
(char*)ary + (walker * sizeofElem)
```

Listeyi fiziksel olarak sonlandırmak için eleman sayısı girişi (*numElem*) gereklidir. Ve nihayetinde, tüm anahtar sıralı ADT'lerde gördüğümüz gibi, bir karşılaştırma fonksiyonu gereklidir.

Tüm sıralama algoritmaları, dizi elemanlarının değiştirilmesini gerektirir. Fiziksel tür bilmediğimiz için, değişimler her bir elemanı bir karakter dizisi olarak ele almalı ve bunları birer birer hareket ettirmelidir. Aşağıdaki kod parçası, iki elemanı nasıl değiştireceğimizi göstermektedir:

// En küçüğü tutma alanına taşı

```
*(holdData + i) = *(smallest + i);
// Mevcut olanı en küçük konuma taşı
movePtr = (char*)ary + (current * sizeofElem);
for (int i = 0; i < sizeofElem; i++)
    *(smallest + i) = *(movePtr + i);
// Tutma alanındaki mevcut konumu taşı
for (int i = 0; i < sizeofElem; i++)
    *(movePtr + i) = *(holdData + i);
```

Bu bölümde, sıralama tasarımına odaklanabilmemiz için tam sayı verileri kullanarak sıralama algoritmalarını uyguluyoruz. ADT uygulamalarını bölümün sonunda problemler ve projeler için bırakıyoruz.

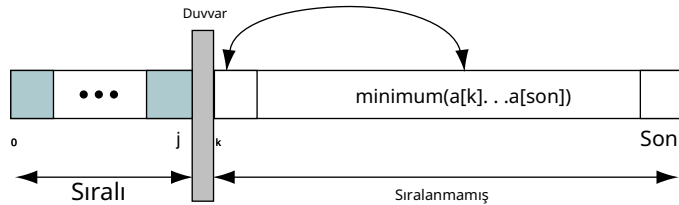
12.2 Seçim Sıralamaları

Seçim sıralamaları, tüm sıralamalar arasında en sezgisel olanlardandır. Sıralanacak bir veri listesi verildiğinde, en küçük öğeyi seçip sıralı bir listeye yerleştiririz. Bu adımlar, tüm verileri sıralayana kadar tekrarlanır. Bu bölümde iki seçim sıralamasını inceliyoruz: düz seçim sıralaması ve yığın sıralaması.

Seçim sıralamasının her geçişinde, en küçük öğe sıralanmamış alt listeden seçilir ve sıralanmamış listenin başındaki öğe ile değiştirilir.

Düz Seçim Sıralaması

Düz seçim sıralamasında, liste her an iki alt listeye ayrılır, sıralı ve sıralanmamış, bunlar hayali bir duvarla ayrılır. Sıralanmamış alt listeden en küçük öğeyi seçeriz ve bunu sıralanmamış verilerin başlangıcındaki öğe ile değiştiririz. Her seçim ve değişimden sonra, iki alt liste arasındaki duvar bir öğe hareket eder, sıralı öğelerin sayısını artırır ve sıralanmamış olanların sayısını azaltır. Sıralanmamış alt listeden sıralı alt listeye her seferinde bir öğe taşıdığımızda, bir sıralama geçişini tamamladığımızı söyleriz. Dolayısıyla, n öğeden oluşan bir listemiz varsa, verileri tamamen yeniden düzenlemek için $n - 1$ geçişe ihtiyacımız vardır. Seçim sıralaması, Şekil 12-3'te grafiksel olarak sunulmuştur.

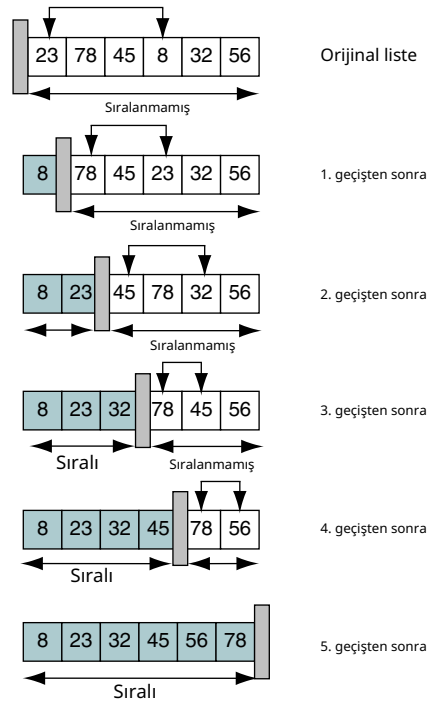


ŞEKİL 12-3 Seçim Sıralama Kavramı

Şekil 12-4, altı tam sayımızı sıralarken izler. Sıralı ve sıralanmamış alt listeler arasındaki duvarın her geçişte nasıl hareket ettiğini gösterir. Şekli incelediğinizde, dizinin beş geçişten sonra sıralandığını, dizideki öğe sayısından bir eksi k olduğunu görürsünüz. Böylece, sıralamayı kontrol etmek için bir döngü kullanıyorsak, döngümüz dizideki öğe sayısından bir daha az yinleme yapar.

Seçim Sıralama Algoritması

Eğer sıralama hakkında hiçbir şey bilmiyorsanız ve bir listeyi kağıt üzerinde sıralamanız isteniyorsa, kesinlikle listenin en küçük öğesini bulmak için tararsınız ve ardından onu ikinci bir listeye kopyalarsınız. Daha sonra, listenin kalan en küçük öğesini bulma ve onu yeni listeye kopyalama işlemini tekrarlıyorsunuz, ta ki tüm öğeleri sıralı listeye kopyalayana kadar.



ŞEKİL 12-4Seçim Sıralama Örneği

İki alan kullanma hariç, seçim sıralamasının tam olarak böyle çalıştığıdır. Listeye ilk öğeden başlayarak, algoritma listeyi en küçük öğeyi bulmak için tarar ve bunu listenin başındaki öğe ile değiştirir. Her seçim ve değişim bir sıralama geçişidir. İndeksi (duvar) ilerlettikten sonra, sıralama, liste tamamen sıralanana kadar devam eder. Pseudocode, Algoritma 12-1'de gösterilmiştir.

ALGORİTMA 12-1Seçim Sıralaması

Algoritma `selectionSort (liste, son)` Sırasız dizinin en küçük öğesini seçerek liste dizisini sıralar ve bunu sırasız listenin başındaki öğe ile değiştirir.

Ön liste en az bir öğe içermelidir
`son`, listedeki son öğenin indeksini içerir
 Son liste en küçükten büyüğe yeniden düzenlenmiştir

- 1 şu anki değeri 0 olarak ayarla
- 2 döngü (`son` öğe sıralanana kadar)
 - 1 en küçüğü şu anki olarak ayarla

devam ediyor

ALGORİTMA 12-1 Seçim Sıralaması (devamı)

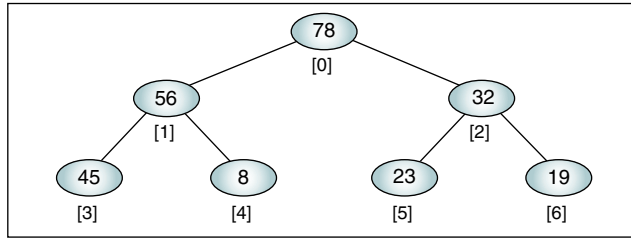
```

2  yürüyüşçüyü mevcut + 1 olarak ayarla
3  döngü (yürüyüşçü <= son)
    1  eğer (yürüyüşçü anahtar < en küçük anahtar)
        1  en küçük yürüyüşçü olarak ayarla
    2  yürüyüşçüyü artır
4  döngüyü bitir
Seçilen en küçük: mevcut eleman ile değiştir.
5  değiştir (mevcut, en küçük)
6  mevcutı artır
3  döngüyü bitir
son seçimSıralama

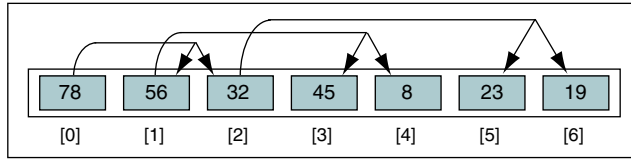
```

Yığın Sıralaması

Bölüm 9'da yığınları inceledik. Bir yığının, kökünde en büyük (veya en küçük) öğeyi içeren bir ağaç yapısı olduğunu hatırlayın. (Yığın sıralamasını incelemenden önce Bölüm 9'u gözden geçirmek isteyebilirsiniz.) Hızlı bir gözden geçirme olarak, Şekil 12-5 bir yığının ağaç biçiminde ve dizi biçiminde göstermektedir.



(a) Ağaç biçimindeki yığın



(b) Dizisi biçimindeki yığın

ŞEKİL 12-5 Yığın Temsilleri

Yığın sıralama algoritması, düz seçim sıralamasının geliştirilmiş bir versiyonudur. Düz seçim sıralama algoritması, sıralanmamış öğeleri tarar ve en küçük öğeyi seçer. n öğesi arasında en küçük öğeyi bulmak $n - 1$ karşılaştırmaya gerektirir. Seçim sıralamasının bu kısmı onu çok yavaş hale getirir.

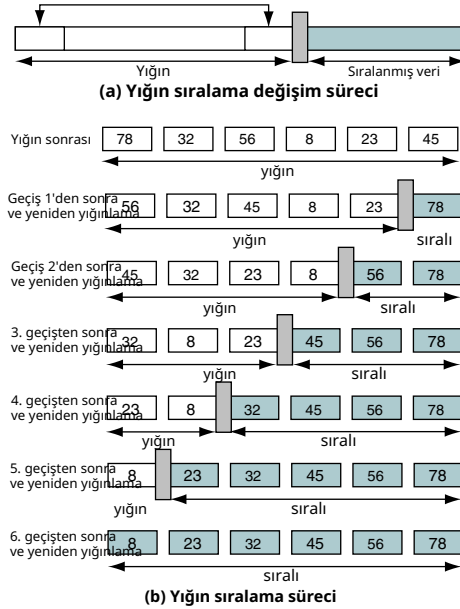
Yığın sıralaması da sıralanmamış liste kısmından bir öğe seçer, ancak bu en büyük öğedir. Ancak, yığın bir ağaç yapısı olduğundan, en büyük anahtarı bulmak için tüm listeyi taramamız gerekmez. Bunun yerine,

yığını yeniden düzenleyerek, en büyük öğeyi köke taşır. Dalları takip etme yeteneği, yığın sıralamasını düz seçim sıralamasından çok daha hızlı hale getirir.

Yığın sıralaması, sıralanacak diziyi bir yığına dönüştürerek başlar. Diziyi, her sıralama için yalnızca bir kez yığına dönüştürülür. Ardından, yığındaki en büyük öğe olan kökü, sıralanmamış listenin son öğesi ile değiştiririz. Bu değişim, en büyük öğenin sıralı listenin başına eklenmesiyle sonuçlanır. Daha sonra yığını yeniden inşa etmek için aşağı doğru yeniden yığın yaparız ve tekrar değiştiririz. Yeniden yığın yapma ve değiştirme işlemi, tüm liste sıralanana kadar devam eder.

Yığın sıralama, en büyük öğenin (kök) seçildiği ve sıralanmamış listedeki son öğe ile değiştirildiği seçim sıralamasının geliştirilmiş bir versiyonudur.

Yığın sıralama süreci Şekil 12-6'da gösterilmektedir. Öncelikle düzensiz listeyi bir yığa dönüştürüyoruz. Dizinin gerçekten bir yığın olduğunu kendiniz doğrulamalısınız.



ŞEKİL 12-6 Yığın Sıralama Süreci

En büyük öğe (78) yığının tepe noktasında olduğu için, onu yığındaki son öğe ile değiştirebiliriz ve yığın duvarını bir öğe sola kaydırabiliriz. Bu değişim, en büyük öğeyi dizinin sonundaki doğru konumuna yerleştirir, ancak yığını bozar. Bu nedenle yığını yeniden inşa ederiz. Daha küçük yığın artık en büyük öğesi (56) tepededir. Biz

56'yı yığının sonundaki eleman (23) ile değiştirin, bu 56'yı sıralı dizideki doğru konumuna yerleştirin. Yeniden yığınla ve değişim işlemleri bütün diziyi sıralayana kadar devam eder.

Yığın Sıralama Algoritması

Algoritma 12-2, yığın sıralama için sahte kodu içerir. Bu, Bölüm 9'da tanımlanan iki algoritmayı kullanır: Algoritma 9-1, "Yukarı Yeniden Yığınla," ve Algoritma 9-2, "Aşağı Yeniden Yığınla."

ALGORİTMA 12-2 Yığın Sıralama

```

Algoritma heapSort (heap, last)
Bir diziyi, bir yığın kullanarak sıralayın.
  Önce yığın dizisi doldurulur
    last dizideki son elemanın indeksidir
  Sonra yığın dizisi sıralanmıştır
  Yığın oluştur
1  yürüyücüyü 1 olarak ayarla
2  döngü (yığın oluşturuldu)
  1 reheapUp (yığın, yürüyücü)
  2 yürüyüşçüyü artır
3  döngüyü bitir
  Yığın oluşturuldu. Şimdi sıralayın.
4  sıralı son olarak ayarla
5  döngü (tüm veriler sıralanana kadar)
  1 eşit değişimi (yığın, 0, sıralı)
  2 sıralı azalt
  3 yığınAşağı (yığın, 0, sıralı)
6  döngüyü bitir
son yığınSıralama
  
```

Algoritma 12-2 Analizi Yığın-dizi yapısı nispeten karmaşık olmasına rağmen, yığın sıralama algoritması basittir.

Algoritma, diziyi bir yığın haline getirmek için reheap up algoritmasını kullanarak başlar. Daha sonra, diziyi yığının tepe noktasındaki elemanı yığının sonundaki elemanla değiştirerek sıralar ve reheap down algoritmasını kullanarak yığını yeniden inşa eder.

Seçim Sıralama Verimliliği

Bu bölümde seçim sıralamaları için sıralama verimliliğini inceliyoruz.

Düz Seçim Sıralaması

Düz seçim sıralaması için kod, Algoritma 12-1, "SeçimSıralama"da bulunur. Aşağıda gösterilen iki döngüyü içerir.

```

2  döngü (son eleman sıralanana kadar)
  ...
3  döngü (yürüyücü <= son)
  
```

Dış döngü $n - 1$ kez çalıştırılır. İç döngü de $n - 1$ kez çalıştırılır. Bu, klasik bir kare döngü örneğidir. Dolayısıyla, büyük-O notasyonu kullanarak, araması $O(n^2)$ olur.

Düz seçim sıralama verimliliği $O(n^2)$.

Yığın Sıralaması

Yığın sıralama sahte kodu Algoritma 12-2'de gösterilmiştir. Yığını başlangıçta oluşturmak için gereken çabayı göz ardı edersek, sıralama iki döngü içerir. İlk, basit bir yinelemeli döngüdür; ikincisi ise bir özyinelemeli döngüdür:

```
5  döngü (tüm veriler sıralanana kadar)
    ...
    3  yığınAşağı (yığın, 0, sıralı)
```

Dış döngü, dizinin sonundan başlar ve yığın içinde her seferinde bir eleman ilerleyerek ilk elemana ulaşana kadar devam eder. Bu nedenle n kez döner. İç döngü, bir ikili ağaçta kökten bir yaprağa veya ebeveyn ve çocuk verileri yığın sırasına gelene kadar bir dalı takip eder. Verilerin yaprağa ulaşmadan önce sıralı olma olasılığı çok küçüktür, bu yüzden bunu göz ardı ediyoruz. Bu analizin zor kısmı, dış döngülerin her biri için yığınküçüldüğünde, kökten bir yaprağa giden yolun kısalmasıdır. Yine, en büyük yığınlar hariç, bu faktör oldukça önemsizdir ve büyük-O analizinde ortadan kaldırılır; bu nedenle, biz de bunu göz ardı ediyoruz.

Bir ikili ağacın dallarını kökten bir yaprağa takip etmek logn döngüleri gerektirir. Yığın sıralama için sıralama çabası, dış döngü ile iç döngünün çarpımıdır, bu nedenle

$n(\lg n)$

Orijinal yığını oluşturma işlemini dahil ettiğimizde, büyük-Onotasyonu aynıdır. Yığının oluşturulması, veriler üzerinden $n \log n$ döngü gerektirir. Sıralama çabasına dahil edildiğinde, bir katsayı haline gelir ve bu da nihai sıralama çabasını belirlemek için düşürülür.

Yığın sıralama verimliliği $O(n \log n)$.

Özet

Analizimiz, yığın sıralamanın, $O(n \log n)$, düz seçim sıralamasından daha etkili olduğunu sonucuna varıyor, $O(n^2)$. Tablo 12-1, bu iki sıralamanın karşılaştırmasını sunmaktadır. Algoritmanın uygulanmasına bağlı olarak, çalışma süresi etkilenebilir.

n	Döngü sayısı	
	Düz seçim	Yığın
25	625	116
100	10,000	664
500	250,000	4482
1000	1,000,000	9965
2000	4,000,000	10,965

TABLO 12-1Seçim Sıralamalarının Karşılaştırması

Seçim Sıralaması Uygulaması

Artık seçim sıralama algoritmalarını C dilinde uygulamaya geçiyoruz. Öncelikli e düz seçim sıralamasını, ardından ise yığın sıralamasını uyguluyoruz.

Seçim Sıralaması C Kodu

Seçim sıralaması için kod Program 12-1'de gösterilmiştir. Tasarımı Algoritma 12-1' de bulunmaktadır.

PROGRAM 12-1Seçim Sıralaması

```

123456789 /* ===== selectionSort =====
Sırasını [1...son] sıralar, dizinin sıralanmamış kısmındaki en küçük ö
ğeyi seçerek ve bunu sıralanmamış listenin başındaki öge ile d
eğiştirerek. */
    Ön liste en az bir öge içermelidir
        son, son liste ögesinin indeksini içerir
    Son liste en küçükten büyüğe sıralanmıştır
*/
geçersiz seçimSıralama (int liste[ ], int son)
{
// Yerel Bildirimler
    int enKüçük;
    int veriTut;

// Beyanlar
    for (int mevcut = 0; mevcut < last; mevcut++)
    {
        enKüçük = mevcut;
        for (int yürüyücü = mevcut + 1;
            yürüyücü <= last;
            yürüyücü++)
            if (list[ yürüyücü ] < list[ enKüçük ])

```

devam ediyor

PROGRAM 12-1 Seçim Sıralaması (devam ediyor)

```

232425262728293031
enKüçük = yürüyücü;

// Seçilen en küçük: mevcut ile değiştir
tutVeri    = liste[ mevcut ];
liste[mevcut] = liste[ enKüçük ];
liste[enKüçük] = tutVeri;
} // mevcut için

return;
} // seçimSıralama

```

Yığın Sıralama C Kodu

Yığın sıralama programı, yığın sıralama adını verdiğimiz bir sürücü işlevi ve daha önce tartıştığımız iki işlev, yığın yukarı ve yığın aşağı gerektirir. Bunları sıralama süreci için burada özelleştirdik.

Yığın Sıralama Fonksiyonu

Yığın sıralama fonksiyonu, sıralanmamış verilerden oluşan bir dizi ve dizideki son elemana bir indeks alır. Ardından bir yığın oluşturur ve bunu yeniden yığın yukarı ve yeniden yığın aşağı kullanarak sıralar. Yığın sıralama algoritmaları Program 12-2'de gösterilmiştir.

PROGRAM 12-2 Yığın Sıralama

```

123456789
/* ===== yığınSıralama =====
   Bir diziyi, [liste0 .. son], bir yığın kullanarak sırala.
   Önce liste en az bir öge içermelidir
       son, listede son elemana indeks içerir
   Sonra liste en küçükten en büyüğe sıralanmıştır
*/
void heapSort (int list[ ], int last)
{
// Yerel Tanımlar
    int sorted;
    int veriTut;

// Beyanlar
// Yığın Oluştur
    for (int yürüteç = 1; yürüteç <= son; yürüteç++)
        reheapUp (liste, yürüteç);

// Yığın oluşturuldu. Şimdi sıralayın.
    sıralanmış = son;
    while (sıralanmış > 0)
    {
        holdData = liste[0];
        list[0] = liste[sıralanmış];
        list[sıralanmış] = holdData;

```

devam ediyor

PROGRAM 12-2Yığın Sıralama (devam ediyor)

```

252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172
sıralanmış--;
reheapDown (liste, 0, sıralı);
} // iken
geri dönmek;
} // yığınSort

/* ===== reheapUp =====
Yığın dizisini yeniden kurar, çocuk verilerini doğru konuma taşır.

Önce yığın, geçersiz bir yığın içeren dizidir.
newNode, yığında yeni verinin indeks konumudur.
Sonra newNode yığına eklenmiştir.
*/
void reheapUp (int* heap, int newNode)
{
// Yerel Bildirimler
int parent;
int tutma;

// Beyanlar
// eğer yığının kökünde değilse
if (newNode)
{
ebeveyn = (yeniNode - 1) / 2;
if (heap[newNode] > heap[parent] )
{
// çocuk, ebeveynden büyüktür
hold = heap[parent];
heap[parent] = heap[newNode];
heap[newNode] = hold;
reheapUp (heap, parent);
} // eğer heap[]
} // eğer newNode
return;
} // yeniden yığın

/* ===== reheapDown =====
Kökteki veriyi aşağı doğru hareket ettirerek heap'i yeniden kurar.
Doğru konumuna yerleştirir. */
Önce heap, bir veri dizisidir
kök, heap veya alt heap'in köküdür
son, heap'teki son elemanın indeksidir
Sonuç heap geri yüklenmiştir
*/
void reheapDown (int* yığın, int kök, int son)
{
// Yerel Deklarasyonlar
int hold;

```

devam ediyor

PROGRAM 12-2Yığın Sıralama (devam ediyor)

```

73     int solAnahtar;
74     int sagAnahtar;
75     int buyukCocukAnahtar;
76     int buyukCocukIndeksi;
77
78     // İfadeler
79     eğer ((kök * 2 + 1) <= son) // En az bir çocuk var
80
81     {
82         solAnahtar = heap[kök * 2 + 1]; eğer
83         ((kök * 2 + 2) <= son)     sagAnahtar
84         ar = heap[kök * 2 + 2]; aksi takdirde
85
86         sagAnahtar = -1;
87
88         // Hangi çocuğun daha büyük olduğunu belirle
89         eğer (solAnahtar > sagAnahtar)
90         {
91             buyukCocukAnahtar = solAnahtar;
92             buyukCocukIndeksi = kök * 2 + 1;
93         } // eğer solAnahtar
94     aksi takdirde
95     {
96         buyukCocukAnahtar = sagAnahtar;
97         buyukCocukIndeksi = kök * 2 + 2; } // aksi takdirde
98
99     // Kökün daha büyük alt ağaçtan büyük olup olmadığını test et eğer (h
100    eap[kök] < heap[buyukCocukIndeksi])
101    {
102        // ebeveyn < çocuklar
103        tut = heap[kök];
104        heap[kök] = heap[buyukCocukIndeksi];
105        heap[buyukCocukIndeksi] = tut;
106        reheapDown (heap, largeChildIndex, last);
107        } // eğer kök <
108    } // eğer kök
109    return;
110 } // reheapDown

```

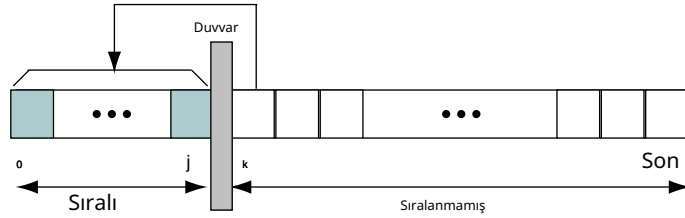
Program 12-2 AnaliziHeap sıralama uygulaması, pseudocode'a yakından uymaktadır. Bir indeks kullanarak,diziyi dolaşıyoruz, heap oluşturmak için reheap çağrısı yapıyoruz. Heap oluşturulduktan sonra,heap'in en üstündeki elemanı heap'teki son elemanla değiştiriyoruz ve heap boyutunu 1 azaltıyoruz.Ardından, kök elemanı ağacın doğru konumuna taşımak için reheap down çağrısı yapıyoruz.

12.3 Ekleme Sıralamaları

Ekleme sıralaması, kart oyuncuları tarafından kullanılan en yaygın sıralama tekniklerinden biridir. Her kartı aldıklarında, onu ellerindeki doğru sıraya yerleştirirler.⁴ Bu kavram, bilgisayar sıralamasına da iyi bir şekilde uzanır. Bir ekleme sıralamasının her geçişinde, bir veya daha fazla veri parçası, sıralı bir listede doğru konumlarına yerleştirilir. Bu bölümde iki ekleme sıralamasını inceleyeceğiz: düz ekleme sıralaması ve kabuk sıralaması.

Doğrudan Ekleme Sıralaması

Doğrudan ekleme sıralamasında, liste iki parçaya ayrılır: sıralı ve sıralanmamış. Her geçişte, sıralanmamış alt listeden ilk eleman, uygun yere eklenerek sıralı alt listeye aktarılır. Eğer elimizde n elemanlı bir liste varsa, verileri sıralamak en fazla $n - 1$ geçiş alır. Bu kavram Şekil 12-7'de gösterilmiştir. Bu şekilde, sıralı ve sıralanmamış liste bölümleri arasında görsel bir duvar yerleştirilmiştir.



ŞEKİL 12-7 Ekleme Sıralama Kavramı

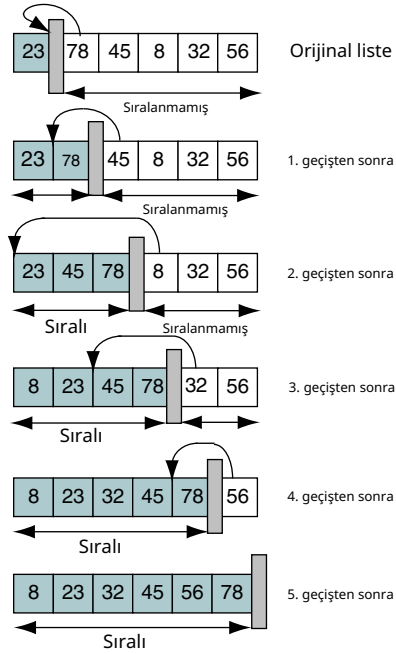
Doğrudan Ekleme Sıralaması Örneği

Şekil 12-8, altı sayıdan oluşan bir liste üzerinden ekleme sıralamasını izlemektedir. Bu verilerin sıralanması beş sıralama geçişi gerektirir. Her geçiş, sıralanmamış alt listeden bir eleman çıkarıldıkça duvarı bir eleman sağa hareket ettirir ve sıralı alt listeye eklenir.

Ekleme Sıralama Algoritması

Ekleme sıralamasının tasarımı örnekteki deseni takip eder. Dış döngü nün her çalıştırılması, sıralanmamış listeden ilk elemanı sıralı listeye ekler. İç döngü, sıralı listeyi yüksek uçtan başlayarak doğru ekleme konumunu aramak için adım adım ilerler. Pseudocode, Algoritma 12-3'te gösterilmiştir.

4. Kart sıralama, sıralamak için iki veri parçası kullanan bir sıralama örneğidir: takvim ve rütbe.



ŞEKİL 12-8 Ekleme Sıralama Örneği

ALGORİTMA 12-3 Düz Ekleme Sıralaması

Algoritma insertionSort (liste, son)

Liste dizisini ekleme sıralaması kullanarak sırala. Dizi, sıralı ve sıralanmamış listelere ayrılır. Her geçişte, sıralanmamış listedeki ilk eleman, sıralı listeye eklenir.

Ön liste en az bir eleman içermelidir

son, listedeki son elemanın indeksidir

Son liste yeniden düzenlenmiştir

- 1 geçerli olanı 1 olarak ayarla
- 2 döngü (son öğe sıralanana kadar)
 - 1 geçerli elemanı tutucuya taşı
 - 2 yürüyücüyü geçerli - 1 olarak ayarla
 - 3 döngü (yürüyücü >= 0 VE tutucu anahtar < yürüyücü anahtar)
 - 1 yürüyücü elemanını bir eleman sağa taşı
 - 2 azaltma yürüyücüsü
 - 4 döngüyü bitir
 - 5 tutmayı yürüyücü + 1 elemana taşı
 - 6 mevcut artır
- 3 döngüyü bitir

son insertionSort

Algoritma 12-3 Analizi Bu basit algoritmada keşfedilmesi gereken iki tasarım kavramı vardır. İcraatlarının bir noktasında, tüm sıralama algoritmaları iki öğeyi değiştirmek zorundadır. Her değişim üç ifade alır, bu da birçok öğenin değiştirilmesi gerektiğinde sıralama verimliliğini büyük ölçüde etkileyebilir. Bu nedenle, değişimi algoritmanın farklı bölümlerine ayırarak verimliliği artırın. Döngünün ilk adımı, sıralanan verileri tutma alanına taşıyarak değişimi başlatır. Herhangi bir değişimdeki bu tipik ilk hareket, ifade 2.1'de gösterilmiştir. İç döngü (ifade 2.3), doğru ekleme konumunu bulana kadar öğeleri sağa kaydırır. Her kaydırma, bir değişimdeki ikinci adımı tekrarlar. Son olarak, doğru konum bulunduğunda, tutma alanı diziyi geri taşınır (ifade 2.5). Bu, değişim mantığındaki üçüncü adımdır ve değişimi tamamlar.

Doğrudan ekleme sıralamasında, liste her an sıralı ve sıralanmamış alt listelere ayrılmıştır. Her geçişte, sıralanmamış alt listenin ilk elemanı sıralı alt listeye eklenir.

İç döngünün işleyişini incelemeniz gereken bir diğer nokta. Sıralamanın mümkün olduğu kadar verimli olması için, sıralı listenin yüksek ucundan başlayarak sıralı alanın başına doğru çalışıyoruz. İlk ekleme için, bu yaklaşım en fazla bir öğenin kaydırılması gerektirir. Son ekleme için, en az sıfır ve en fazla $n - 1$ öğenin kaydırılması gerektirir. Sonuç olarak, her sıralama geçişinde listenin yalnızca bir kısmı incelenebilir.

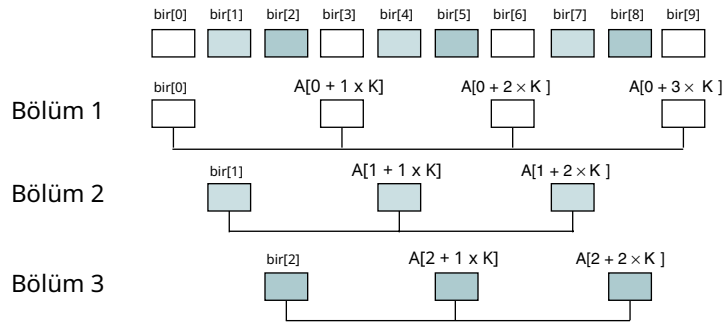
Shell Sıralaması

shell sıralama algoritması, yaratıcısı Donald L. Shell'in adını taşıyor doğrudan ekleme sıralamasının geliştirilmiş bir versiyonudur. Hızlı sıralama algoritmalarının dan biriydi.5

Shell sıralaması, verileri sıralamak için azalan bölümlerin kullanıldığı doğrudan ekleme sıralamasının geliştirilmiş bir versiyonudur.

Shell sıralamasında, N öğeden oluşan bir liste K segmentine bölünür; burada K artış olarak bilinir. Her segment, en az tam sayı N/K içerir; eğer N/K bir tam sayı değilse, bazı segmentler bir ekstra öğe içerecektir. Şekil 12-9, bir shell sıralamasındaki segmentlerin grafiksel temsilini içermektedir. Segmentlerin liste boyunca dağıldığını unutmayın. Şekil 12-9'da artış 3'tür; birinci, dördüncü, yedinci ve onuncu öğeler segment 1'i oluşturur; ikinci, beşinci ve sekizinci öğeler segment 2'yi oluşturur; üçüncü, altı ve dokuzuncu öğeler segment 3'ü oluşturur. Veriler üzerinden her geçişten sonra, her segmentteki veriler sıralanır. Böylece, Şekil 12-9'da gördüğümüz gibi, üç farklı sıralı liste vardır. İki segment olduğunda, iki sıralı liste vardır; yalnızca bir segment olduğunda, liste sıralanır.

5. Shell algoritması ilk olarak *ACM İletişimleri*, cilt 2, sayı 7 (1959): 30–32'de yayımlandı.



ŞEKİL 12-9 Segmentli Dizi

Shell Sıralama Algoritması

Veri üzerinden her geçiş, dizideki ilk elemanla başlar ve diziyi ilerletir ken her segmentteki bitişik elemanları karşılaştırır.

Şekil 12-9'da, ilk segmentten A[0] ve A[3] elemanlarını karşılaştırarak başlıyoruz, ardından ikinci segmentten A[1] ve A[4] elemanlarını, ardından üçüncü segmentten A[2] ve A[5] elemanlarını karşılaştırıyoruz. Daha sonra ilk segmentten A[3] ve A[6], ikinci segmentten A[4] ve A[7] elemanlarını karşılaştırıyoruz ve nihayetinde A[6] ve A[9] elemanlarını karşılaştırıyoruz. Elemanlar sıralı değilse, yer değiştirirler. Bu karşılaştırmaların her birini görene kadar Şekil 12-9'u dikkatlice inceleyin. Ayrıca, ilk segmentin dört elemanı olduğunu, diğer iki segmentin ise yalnızca üç elemanı olduğunu not ettiğinizden emin olun. Segmentlerdeki eleman sayısı, liste boyutunun (10) artışı (3) tam olarak bölünmemesi nedeniyle değişir.

Bir segmentteki bitişik anahtarları karşılaştırmak için, mevcut indekse artışı ekliyoruz, aşağıda gösterildiği gibi.

liste[cur] : liste[cur + artışı]

Veri üzerinden her geçişten sonra, artışı azaltılır ve son geçişte 1 olur. Son artışın 1 olması gerektiği kesin olmakla birlikte, artışların boyutu sıralamanın verimliliğini etkiler. Bu konuyu daha sonra ayrı olarak tartışacağız. Azalan artışı 6, 10 elemanlı bir dizi ve 5, 2 ve 1 artışları için Şekil 12-10'da gösterilmiştir.

6. Knuth bu sıralamaya azalan artışı sıralaması adını vermiştir, ancak daha çok kabuk sıralaması olarak bilinir.

(a) İlk artış: $K = 5$

Yürüteç						Mevcut													
77	62	14	9	30		21	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									
21	62	14	9	30		77	80	25	70	55									

(b) İkinci artış: $K = 2$

21	62	14	9	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										
14	9	21	62	30	77	80	25	70	55										

(c) Üçüncü artış: $K = 1$

14	9	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										
9	14	21	25	30	55	70	62	80	77										

(d) Sıralanmış dizi

9	14	21	25	30	55	62	70	77	80										
---	----	----	----	----	----	----	----	----	----	--	--	--	--	--	--	--	--	--	--

ŞEKİL 12-10 Shell Sıralamasında Azalan Artışlar

Veri üzerinden her geçişten sonra, her segmentteki öğeler sıralanır. Geçişin sonunda her segmentin sıralı olmasını sağlamak için, herhangi bir değişim yapıldığında bir artış geri düşeriz ve komşu elemanları test ederiz. Eğer sıralı değillerse, onları değiştiririz ve tekrar geri düşeriz. Gerekirse, iki öğenin sıralı olduğunu bulana kadar değişim yapmaya ve geri düşmeye devam ederiz. Artık shell sıralamasının tüm öğelerine sahibiz. Pseudocode'u Algoritma 12-4'te gösterilmiştir.

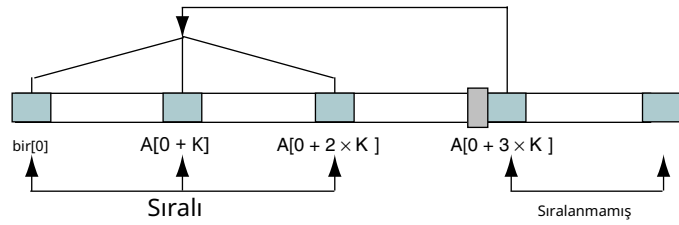
ALGORİTMA 12-4 Shell Sıralaması

```

Algoritma shellSort (liste, son) Liste dizisi
ndeki veriler yerinde sıralanır. Sıralama s
onrası, anahtarları sıralı olacaktır, liste[0] <= liste[1] <= ... <= liste[
son].
    Ön liste, kayıtlardan oluşan sırasız bir dizidir
        son, dizideki son kaydın indeksidir
    Son liste, liste[i].anahtarına göre sıralıdır.
1  incre'yi son / 2 olarak ayarla
    Anahtarları "arttırma" elemanları kadar karşılaştır.
2  döngü (incre 0 değilse)
    1  mevcut'u incre olarak ayarla
    2  döngü (son öğe sıralanana kadar)
        1  geçerli elemanı tutucuya taşı
        2  yürüyüşçüyü mevcut - incre olarak ayarla
        3  döngü (yürüyüşçü >= 0 VE tutulan anahtar < yürüyüşçü anahtarı)
            Daha büyük elemanı listede yukarı taşı.
            1  yürüyüşçü elemanını bir artırma sağa taşı
            Bir bölüme geri dön.
            2  yürüyüşçüyü yürüyüşçü - incre olarak ayarla
        4  döngüyü bitir
        Kaydı uygun göreceli konuma yerleştir.
        5  tutmayı walker + incre elemanına taşı
        6  mevcutı artır
    3  döngüyü bitir
    Geçişin sonu--bir sonraki artış hesapla.
    4  incre'yi incre / 2 olarak ayarla
3  döngüyü bitir
son shellSort

```

Algoritma 12-4 Analizi Shell sıralamasını dikkatlice inceleyelim, böylece nasıl bir ekleme sıralaması olarak nitelendirildiğini görebiliriz. Ekleme sıralamalarının yeni verileri listenin sıralı kısmındaki doğru konumlarına yerleştirdiğini hatırlayın. Bu kavram shell sıralamasında bulunur: listenin sıralı kısmı, üyeleri artış boyutuyla ayrılmış bir segmenttir. Bunu daha net görmek için, Şekil 12-11'e bakın. Bu şekilde segment, bir dizideki renkli öğeler olarak gösterilmiştir. Yeni eleman, $A[0 + 3 \times K]$, segmentin sıralı kısmında doğru konumuna yerleştiriliyor.



ŞEKİL 12-11 Shell Sıralamada Sıralı Segment

Ayrıca, Algoritma 12-4'teki kodu Algoritma 12-3, "Düz Ekleme Sıralaması" ile karşılaştırdığınızda, artış dışında, kodun neredeyse aynı olduğunu görüyorsunuz.

Shell sıralamanın en önemli kısımlarından biri, segmentin sıralı olmasını sağlamak içi n geri dönmektir. Bu mantık 2.2.3.2 ifadesinde gerçekleşir. Bu mantık dahil edilmezse, el de edilen liste tamamen sıralı değildir.

Artırım Boyutunu Seçme

Öncelikle, hiçbir artırma boyutunun tüm durumlar için en iyi olmadığını kabul edin. Sıralamadaki öncelikli hususlar, sıralamayı minimum geçiş (artırma) sayısı ile tamamlamak ve birden fazla segmentte görünen eleman sayısını en aza indirmektir. Bir elemanın birden fazla segmentte olmadığını garanti etmenin bir yöntemi asal sayıları kullanmaktır. Ne yazık ki, asal sayıların dinamik hesaplanması nispeten yavaş bir süreçtir.

Çoğu metin, artırmayı liste boyutunun yarısı olarak ayarlayıp her geçişte 2'ye bölerek, Algoritma 12-4'te önerdiğimiz basit seriyi kullanır. Ancak Knuth, artırmanın liste boyutunun üçte birinden büyük olmaması gerektiğini önermektedir. Diğer bilgisayar bilimcileri, artırmaların 2'nin bir kuvveti eksi 1 ve ya bir Fibonacci serisi olmasını önermiştir. Bu varyasyonlar, biraz daha verimli bir sıralama ile sonuçlanabilir, ancak nispeten karmaşıktır. Bölme-yöntemi ile 2'ye yaklaşımın basit bir varyasyonu, artırma çift olduğunda 1 eklemektir. Bunu yapmak, birden fazla segmentte görünen eleman sayısını azaltma eğilimindedir.

Daha karmaşık artırma ayarlama algoritmaları kullanabilirsek de, bir shell sıralamanın verimliliği, bir sonraki bölümde tartışılan hızlı sıralamanın verimliliğine asla yaklaşmaz. Bu nedenle, en verimli sıralamayı elde etme amacı varsa, çözüm hızlı sıralamayı kullanmak olmalıdır; shell sıralamadaki artırma boyutunu optimize etmeye çalışmak yerine. Öte yandan, shell sıralama çok daha basit bir sıralama olup, aynı zamanda makul derecede verimlidir.

Ekleme Sıralama Verimliliği

Sıralama algoritmaları, belirli bir sıralama için sıralama çabasını belirler. Sıralama çabası, bir sıralamanın göreceli verimliliği olarak tanımlanır. Birkaç farklı şekilde belirlenebilir, ancak biz sıralamadaki döngü sayısını kullanıyoruz. Diğer yaygın bir ölçüm, listeyi sıralamak için gereken hareket ve karşılaştırmaların sayısıdır. Elbette, en iyi ölçüm, sıralamanın gerçekten çalıştırılması için geçen zamandır. Ancak zaman, program uygulamasının verimliliği ve kullanılan bilgisayarın hızı ile değişir. Bu nedenle, farklı sıralamaları analiz etmek için ilk iki ölçüm daha anlamlıdır. Şimdi düz ekleme sıralama ve shell sıralama algoritmalarını analiz ederek göreceli verimliliklerini belirleyelim.

Doğrudan Ekleme Sıralaması

Algoritma 12-3, "Düz Ekleme Sıralaması"na atıfta bulunarak, aşağıdaki iki döngüyü bu luyoruz:

```

2   döngü (son eleman sıralanana kadar)
    ...
2   yürüyücüyü geçerli - 1 olarak ayarla
3   döngü (yürüyücü >= 0 VE tutucu anahtar < yürüyücü anahtar)
```

Dış döngü, listedeki son elemana kadar 1'den $n - 1$ kez çalıştırılır. Her dış döngü için, iç döngü, tutulan anahtar ile yürüyen anahtar arasındaki ilişkiye bağlı olarak 0'dan `current` kadar çalıştırılır. Ortalama olarak, iç döngünün sıralı listenin yarısındaki verileri işlemesini bekle yebiliriz. İç döngü, dış döngü tarafından kontrol edilen `current` ayarın a bağlı olduğundan, matematiksel olarak ifade edilen bağımlı bir kar e döngüye sahibiz.

$$f(n) = n\left(\frac{n+1}{2}\right)$$

Büyük-O notasyonunda bağımlı ikinci dereceden döngü $O(n^2)$ şeklindedir.

Doğrudan ekleme sıralama verimliliği $O(n^2)$ şeklindedir.

Shell Sıralaması

Şimdi Algoritma 12-4, "Shell Sıralama"ya bakalım. Bu algoritma aşağıda gösteril en iç içe döngüleri içerir:

```

2   döngü (artış 0 değil)
    ...
2   döngü (son eleman sıralanana kadar)
    ...
3   döngü (yürüyücü >= 0 VE anahtar tut < yürüyücü anahtarı)
```

Her döngüde artışı 2'ye böldüğümüz için, dış döngü logaritmik ol up $\log n$ kez çalıştırılır. İlk iç döngü, dış döngülerin her biri için $n - ar tış kadar çalışır; ilk seferde dizinin %50'sini ($n - (n / 2)$) döngüye alır, i kinci seferde dizinin %75'ini ($n - (n / 4)$) döngüye alır ve bu şekilde tū m elemanları döngüye alana kadar devam eder. Dış döngü ve ilk iç döngü için toplam yineleme sayısı aşağıda gösterilm iştir:$

$$\log n \times \left[\left(n - \frac{n}{2} \right) + \left(n - \frac{n}{4} \right) + \left(n - \frac{n}{8} \right) + \dots + 1 \right] = n \log n$$

En içteki döngüyü analiz etmek en zordur. İlk sınır, dizinin başında n düşmemizi engeller. İkinci sınır, döngüye girip girmeyeceğimizi belirler: veriler düzensiz olduğunda yalnızca döngüye gireriz.

Bazen iç döngü sıfır kez çalıştırılır; bazen bir ile artış sayısı arasında herhangi bir yerde çalıştırılır. Üçüncü faktör için bir formül türetebilirs ek, toplam sıralama çabası üç döngünün çarpımı olacaktır. İlk iki dön günün birleşik verimliliği $O(n \log n)$ şeklindedir. Ancak, üçüncü döngü yü de dahil etmemiz gerekiyor. Dolayısıyla, sonucun $O(n \log n)$ 'den da ha büyük bir şey olduğunu görebiliriz.

Knuth⁸, shell sıralaması için sıralama çabasının matematiksel olarak türetilmeyeceği ni söylüyor. Ampirik çalışmalarından, ortalama sıralama çabasının $15n^{1.25}$ olduğunu tahmi n ediyor. Knuth'un analizini büyük-O notasyonuna indirgersek, shell sıralamasının $O(n^{1.25})$ olduğunu görüyoruz.

Shell sıralama verimliliği $O(n^{1.25})$ dir.

Özet

Analizimiz, yığın sıralamasının tartıştığımız diğer sıralamalardan daha verimli olduğu sonucuna varıyor. Düz ekleme ve düz seçim sıralamaları her iki si de $O(n^2)$ sıralamalarıdır, shell sıralaması $O(n^{1.25})$ ve yığın sıralaması $O(n \log n)$ dir. Tablo 12-2, sıralamaların karşılaştırmasını sunmaktadır. Sıkı bir matematiksel büyük-O notasyonu yorumuna göre, yığın sıralaması, sırala nacak 2000 öğeye yaklaştıkça shell sıralamasını verimlilikte geçmektedir. Ancak iki noktayı hatırlayın: İlk olarak, büyük-O yuvarlanmış bir yaklaşık değ erdir; tüm katsayılar ve birçok faktör çıkarılmıştır. İkincisi, büyük-O algoritm aların analitik bir değerlendirmesine dayanır, kodun değerlendirmesine değ i l. Algoritmanın uygulanmasına bağlı olarak, gerçek çalışma süresi etkilenebil ir.

8. Knuth, *Bilgisayar Programlamanın Sanatı*, 382.

n	Döngü sayısı		
	Düz ekleme Düz seçim	Takvim	Yığın
25	625	55	116
100	10,000	316	664
500	250,000	2364	4482
1000	1,000,000	5623	9965
2000	4,000,000	13,374	10,965

TABLO 12-2 Ekleme ve Seçim Sıralamalarının Karşılaştırması

Ekleme Sıralaması Uygulaması

Bu bölümde düz ekleme sıralaması ve takvim sıralaması için C uygulamaları yazıyoruz.

Doğrudan Ekleme Sıralaması

Düz ekleme sıralamasının uygulaması, Algoritma 12-3'teki sahte kodu çok yakından takip etmektedir. Sıralamayı test etmek için rastgele tam sayılardan oluşan bir dizi oluşturduk. Kod Program 12-3'te gösterilmiştir.

PROGRAM 12-3 Ekleme Sıralaması

123456789	/* ===== insertionSort ===== Ekleme Sıralaması kullanarak sıralama. Liste, sıralı/sırasız listeye bölünmüştür. Her geçişte, sırasız listedeki ilk eleman, sıralı listeye eklenir. Önceki liste en az bir eleman içermelidir son, listedeki son elemanın indeksini içerir Sonra liste yeniden düzenlenmiştir */
10111213141516171819	void insertionSort (int list[], int last) { // Yerel Tanımlar int hold; int yürüteç; // Beyanlar for (int geçerli = 1; geçerli <= son; geçerli++) { basılı tutun = liste[geçerli]; for (yürüteç = geçerli - 1;

devam ediyor

PROGRAM 12-3 Ekleme Sıralaması (devamı)

```

2021222324252627
        yürüteç >= 0 && basılı tut < liste[yürüteç];
        yürüteç--)

                list[walker + 1] = list[walker];

        list [walker + 1] = hold;
    } // mevcut için
    geri dönmek;
} // insertionSort

```

Program 12-3 Analizi Her iki döngüyü de *for* ifadesi kullanarak uyguluyoruz. Sıralamaya başladığımızda, sıralı listeye ilk öğeyi içerir. Bu nedenle, döngü başlangıcını sıralanmamış liste için 1. konuma ayarlıyoruz (ifade 16). İç döngüde, sınırlayıcı koşul dizinin başlangıcıdır, bu ifade 20'de gösterilmiştir.

Shell Sıralaması

Shell sıralama uygulaması, aynı zamanda bir tamsayı dizisi kullanan, Program 12-4'te gösterilmiştir. Tasarımı Algoritma 12-4'te gösterilmektedir.

PROGRAM 12-4 Shell Sıralama

```

123456789
/* List[1], list[2], ..., list[last] yerinde sıralanmıştır
   anahtarların sıralı olması için, list[1].key <=
   liste[2].anahtar, <= ... <= liste[son].anahtar.
   Önce list, düzensiz bir tamsayı dizisidir
   last, dizideki son elemanın indeksidir
   Sonra list sıralıdır
*/
void shellSort (int list [], int last)
{
    // Yerel Tanımlar
    int hold;
    int artışı;
    int yürüyücü; // İfadeler i

    ncre = son / 2;

    while (incre != 0)
    {
        for (int curr = incre; curr <= son; curr++)
        {
            tut = liste [curr];
            yürüyücü = curr - incre;    while (yürüyücü >= 0 &
            & tut < liste [yürüyücü])
            {
                // Daha büyük öğeyi listede yukarı taşı
                liste [yürüyücü + incre] = liste [yürüyücü];
                // Bir bölüme geri dön
            }
        }
    }
}

```

devam ediyor

PROGRAM 12-4 Shell Sıralaması (devamı)

```

28293031323334353637
yürüteç = ( yürüteç - artış );
    } // while
// Tutmayı uygun pozisyona yerleştir
    list [walker + incre] = hold;
    } // for walk
// Geçişin sonu--bir sonraki artışı hesapla.
    artış = artış / 2;
    } // while
    dönüş;
} // kabukSort

```

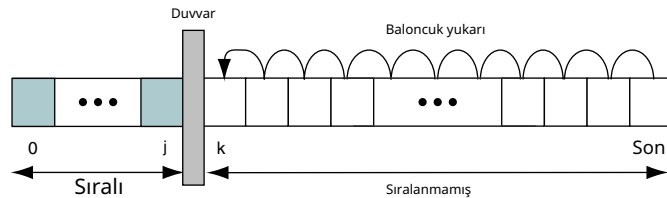
Program 12-4 Analizi Doğrudan ekleme sıralamasından daha karmaşık olmasına rağmen, shell sıralaması kesinlikle zor bir algoritma değildir. Doğrudan ekleme sıralamasına göre yalnızca iki ek karmaşıklık vardır. İlk olarak, tek bir dizi yerine, bir dizi bölümümüz var. İkincisi, her değişim yapıldığında, geri dönüp bölümün sırasını doğrulamamız gerekir.

12.4 Değişim Sıralamaları

Üçüncü sıralama kategorisi, değişim sıralamaları, bilgisayar bilimlerinde öğretilen en yaygın sıralama olan balon sıralaması ve en verimli genel amaçlı sıralama olan hızlı sıralamayı içerir. Değişim sıralamalarında, tüm liste sıralanana kadar düzensiz olan elemanları değiştiririz. Neredeyse her sıralama yöntemi bir tür değişim kullanmasına rağmen, bu bölümdeki sıralamalar bunu yaygın olarak kullanır.

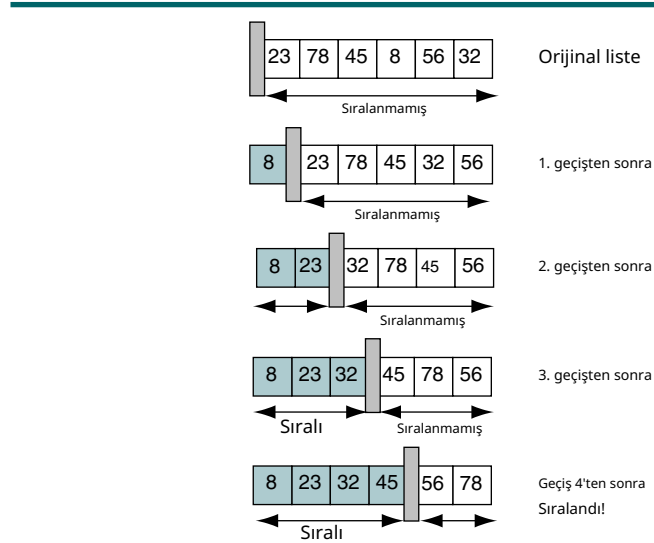
Balon Sıralaması

Balon sıralaması'nda, liste herhangi bir anda iki alt listeye ayrılır: sıralı ve düzensiz. En küçük eleman düzensiz alt listeden baloncuk gibi yükseltilir ve sıralı alt listeye taşınır. En küçüğü sıralı listeye taşıdıktan sonra, duvar bir elemanı sağa kaydırır, sıralı elemanların sayısını artırır ve düzensiz olanların sayısını azaltır (Şekil 12-12). Bir eleman düzensiz alt listeden sıralı alt listeye her geçtiğinde, bir sıralama geçişi tamamlanmış olur. n eleman içeren bir liste verildiğinde, balon sıralaması verileri sıralamak için en fazla $n - 1$ geçiş gerektirir.



ŞEKİL 12-12 Balon Sıralama Kavramı

Şekil 12-13, duvarın her geçişte bir öğeyi nasıl hareket ettirdiğini gösterir. İlk geçişte bakarak, 32 ile 56'yı karşılaştırıyoruz. 32, 56'dan küçük olduğu için, ikisini değiştiriyoruz ve bir öge aşağı iniyoruz. Ardından 32 ve 8'i karşılaştırıyoruz. 32, 8'den küçük olmadığı için bu öğeleri değiştirmiyoruz. Bir öge aşağı iniyoruz ve 45 ile 8'i karşılaştırıyoruz. Sıralama dışındalar, bu yüzden onları değiştiriyoruz ve tekrar aşağı iniyoruz. 8'i aşağı indirdiğimiz için, şimdi 78 ile karşılaştırılıyor ve bu iki öge değiştiriliyor. Son olarak, 8, 23 ile karşılaştırılıyor ve değiştiriliyor. Bu değişim serisi, 8'i ilk konuma yerleştiriyor ve duvar bir pozisyon yukarı taşınıyor.



ŞEKİL 12-13 Baloncuk Sıralama Örneği

Baloncuk Sıralama Algoritması

Eklemeli ve seçim sıralamaları gibi, baloncuk sıralaması oldukça basittir. Verilerdeki her geçişte, en küçük öge, dizinin sıralanmamış segmentinin başına baloncuk gibi yükseltilir. Bu süreçte, sıralı olmayan yan yana öğeler değiştirilir ve veriler kısmen sıralanır. En küçük öge ile karşılaşıldığında, otomatik olarak sıralanmamış listenin başına baloncuk gibi yükseltilir. Sıralama, ardından sıralanmamış liste üzerinden bir geçiş yaparak devam eder. Baloncuk sıralaması için kod, Algoritma 12-5'te gösterilmiştir.

ALGORİTMA 12-5 Balon Sıralama

Algoritma `balonSıralama (liste, son)` Bir diziyi balon sıralama kullanarak sırala. Komşu elemanları karşılaştırılır ve liste tamamen sıralanana kadar değiştirilir.

devam ediyor

ALGORİTMA 12-5 Balon Sıralama (devamı)

```

Ön liste en az bir öge içermelidir
son, listedeki son ögenin indeksini içerir
Sonuç liste, düşükten yükseğe doğru sıralanmıştır.
1 şu anki değeri 0 olarak ayarla
2 doğruyu false olarak ayarla
3 döngü (geçerli <= son VE doğru false)
    Her yineleme bir sıralama geçiştir.
    1 yürüyücüyü sona ayarla
    2 doğruyu true olarak ayarla
    3 döngü (yürüyücü > geçerli)
        1 eğer (yürüyücü verisi < yürüyücü - 1 verisi)
            Herhangi bir değişim, listenin sıralı olmadığı anlamına gelir.
            1 sıralı false olarak ayarla
            2 değiştir (liste, yürüyücü, yürüyücü - 1)
        2 sonlandır
        3 azaltma yürüyücüsü
    4 döngüyü bitir
    5 mevcutı artır
4 döngüyü bitir
son bubbleSort

```

Algoritma 12-5 AnaliziEğer başka balon sıralama algoritmalarını incelediyse, bu sıralama versiyonunda hafif bir iyileşme fark etmiş olabilirsiniz. Eğer bir geçişte (ifade 3) bir değişim yapılmazsa, listenin zaten sıralı olduğunu ve sıralamanın durabileceğini biliyoruz. İfade 3.2'de bir Boolean, sıralı, true olarak ayarlanır. Geçiş sırasında herhangi bir zamanda bir değişim yapılırsa, sıralı false olarak değiştirilir, bu da geçiş başladığında listenin sıralı olmadığını gösterir.

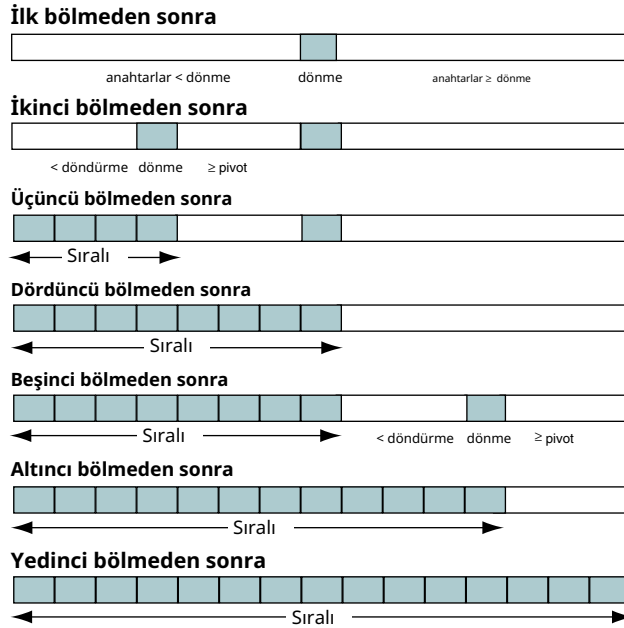
Fark ettiğiniz bir diğer fark, yüksek uçtan başlayıp aşağı doğru baloncuk yapmamızdır. Tarihsel bir not olarak, balon sıralama başlangıçta listenin en yüksek elemanını "baloncuk yaparak yukarı" çıkarmak için yazılmıştır. Verimlilik açısından, en büyük elemanın aşağı doğru baloncuk yapması veya en küçük elemanın yukarı doğru baloncuk yapması arasında bir fark yoktur. Ancak tutarlılık açısından, üç temel sıralama yöntemimiz (ekleme, seçim ve değişim) aynı şekilde çalışıyorsa sıralamalar arasındaki karşılaştırmalar daha kolaydır. Bu nedenle, her geçişte en düşük anahtarı baloncuk yapmayı seçtik.

Hızlı Sıralama

Bubble sıralamada, ardışık öğeler her geçişte karşılaştırılır ve muhtemelen değiştirilir bu da bir öğeyi doğru konumuna taşımak için birçok değişikliğin gerekebileceği anlamına gelir. Hızlı sıralama, 1962'de C. A. R. Hoare tarafından geliştirilen bir değişim sıralamasıdır. Bubble sıralamadan daha etkilidir çünkü tipik bir değişim, uzak olan öğeleri içerir, bu nedenle bir öğeyi doğru konumlandırmak için daha az değişim gereklidir.

Hızlı sıralamanın her yinelemesi, pivot olarak bilinen bir öğeyi seçer ve listeyi üç gruba ayırır: anahtarları pivotun anahtarından daha küçük olan öğelerin bir bölümü, pivot öğesi, listenin nihai doğru konumuna yerleştirilen ve pivotun anahtarına eşit veya daha büyük olan öğelerin bir bölümü. Sıralama, ardından sol tarafı hızlı sıralayarak devam eder.

partisyon, sağ partisyonu hızlı sıralama ile takip eder. Bu partisyonlama Şekil 12-14'te gösterilmiştir.



ŞEKİL 12-14 Hızlı Sıralama Partisyonları

Hoare'ın orijinal algoritması, pivot anahtarını listedeki ilk eleman olarak seçmiştir. 1969'da, R. C. Singleton, pivot anahtarını üç elemanın ortası n değeri olarak seçerek sıralamayı geliştirmiştir: sol, sağ ve listenin ortasında bir eleman. Medyan değeri belirlendikten sonra, sol eleman ile değiştirilir. Singleton'ın hızlı sıralama varyasyonunu uyguluyoruz.

Hızlı sıralama, bir pivot anahtarının dizideki doğru konumuna yerleştirildiği ve diğer elemanların liste boyunca geniş bir şekilde yeniden düzenlendiği bir değişim sıralamasıdır.

Knuth, sıralama partisyonu küçük hale geldiğinde, partisyonun sıralamasını tamamlamak için düz bir ekleme sıralaması kullanılmasını önermektedir.⁹ Minimum partisyon boyutunu optimal olarak seçmek için matematik oldukça karmaşık olsa da, partisyon boyutunun nispeten küçük olması gerekir; 16 öneriyoruz.

9. Knuth, *Bilgisayar Programlamanın Sanatı*, "Algoritma Q," 115.

Hızlı Sıralama Algoritması

Artık algoritmayı geliştirmeye hazırız. Tartışmamızdan tahmin edebileceğiniz gibi, hızlı sıralama için bir özyinelemeli algoritma kullanıyoruz. Temel algoritmaya ek olarak, bir pivot elemanını belirlemek ve düz ekleme sıralaması için iki destekleyici algoritma gereklidir. Öncelikle bu iki algoritmayı tartışıyoruz.

Düz Ekleme Modülü

Düz ekleme sıralaması, daha önce geliştirilen algoritmanın küçük bir varyasyonudur ve her zaman 0 konumundan başlayan bir listeyi sıralar. Hızlı sıralamada sıralanacak parçanın dizinin herhangi bir yerinde bulunabileceğinden, 0'dan farklı konumlarda başlayan parçaları sıralayabilmemiz gerekir. Bu nedenle, sıralanacak konumun başlangıç konumunu belirtmek için bir parametre ekliyoruz. Değiştirilmiş kod, Algoritma 12-6'da gösterilmiştir.

ALGORİTMA 12-6 Hızlı Sıralama'nın Düz Ekleme Sıralama Modülü

```

Algoritma quickInsertion (liste, ilk, son)
  Dizi listesini ekleme sıralaması kullanarak sırala. Liste,
  sıralı ve sıralanmamış listelere ayrılır. Her geçişte,
  sıralanmamış listedeki ilk eleman,
  sıralı listeye eklenir. Bu, hızlı sıralama ile kullanmak için
  değiştirilmiş özel bir ekleme sıralama versiyonudur.
    Önce liste en az bir eleman içermelidir
      ilk, listedeki ilk elemanın indeksidir
      son, listedeki son elemanın indeksidir
    Sonra liste yeniden düzenlenmiştir
1  mevcut değeri ilk + 1 olarak ayarla
2  döngü (mevcut bölüm sıralanana kadar)
    1  geçerli elemanı tutucuya taşı
    2  yürüyücüyü geçerli - 1 olarak ayarla
    3  döngü (yürüyücü >= ilk VE tutma anahtarı < yürüyücü anahtarı)
        1  yürüyücü elemanını bir eleman sağa hareket ettir
        2  azaltma yürüyücüsü
    4  döngüyü bitir
    5  tutmayı yürüyücü + 1 elemanına taşı
    6  mevcutı artır
3  döngüyü bitir
son quickInsertion

```

Üçün Medianını Belirle

Median konumunu seçmek için üç test gereklidir. Öncelikle sol ve orta elemanları test ediyoruz; eğer sıralama dışındalarsa, onları değiştiriyoruz. Sonra sol ve sağ elemanları test ediyoruz; eğer sıralamadan çıkmışlarsa, onları değiştiriyoruz. Son olarak, orta ve sağ elemanları test ediyoruz; eğer sıralamadan çıkmışlarsa, onları değiştiriyoruz. Bu noktada üç eleman sıralı hale gelmiştir.

sol eleman $\leq$ orta eleman $\leq$ sağ eleman

Bu nedenle, bu mantığın, eğer a b 'den küçükse, ve b c 'den küçükse, o zaman a c 'den küçüktür mantıksal önermesine dayandığını görüyoruz. Son olarak, algoritmayı terk etmeden önce, sol ve orta elemanları değiştiriyoruz, böylece medyan değerli elemanı dizinin soluna yerleştiriyoruz. Pseudocode, Algoritma 12-7'de gösterilmiştir.

ALGORİTMA 12-7 Medyan Sol

```

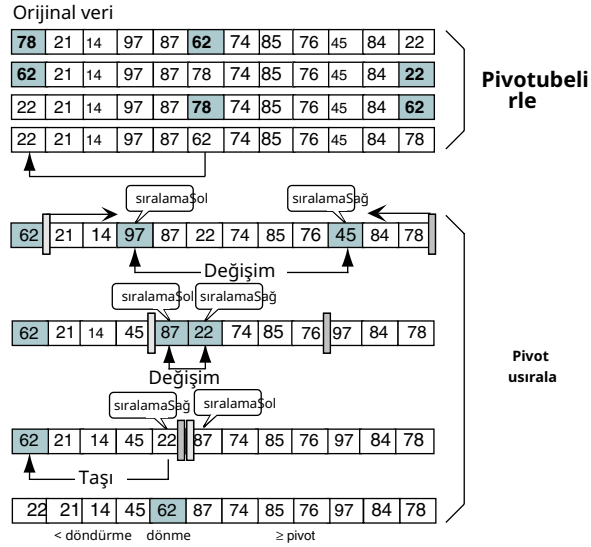
Algoritma medyanSol (sıralamaVerisi, sol, sağ)
Bir dizinin medyan değerini bul ve ilk
(sol) konumuna yerleştir.
    Ön  sıralamaVerisi en az üç eleman içeren bir dizidir
        sol ve sağ dizinin sınırlarıdır
    Sonra medyan değeri bulundu ve sola yerleştirildi
        sıralamaVerisi'ni yeniden düzenle, böylece medyan değeri ortada yer alsın.
1  mid'i (left + right) / 2 olarak ayarla
2  eğer (left key > mid key)
    1  (sortData, left, mid) değiştir
3  sonlandır
4  eğer (left key > right key)
    1  (sortData, left, right) değiştir
5  sonlandır
6  eğer (mid key > right key)
    1  (sortData, mid, right) değiştir
7  sonlandır
    Medyan ortada. Sol ile değiştir.
8  (sortData, left, mid) değiştir
uç medyanSol

```

Algoritma 12-7 Analizi Bir medyan değerini belirleme mantığı çok hızlı bir şekilde anlaşılabilir hale gelebilir. Bu algoritmanın güzelliği basitliğindedir. Medyan değeri belirlemek için üç eleman üzerinde çok basit bir sıralama yaparak medyanı ortada yerleştirir. Ardından ortadaki elemanı sol elemanla değiştirir.

Hızlı Sıralama Algoritması

Şimdi hızlı sıralama algoritmasına dikkatimizi veriyoruz. Diğer dizi uygulamalarında faydalı bulacağınız ilginç bir program tasarımı içerir. Pivot eleman için doğru konumu belirlemek için dizinin iki ucundan ortasına doğru çalışıyoruz. Pivot elemanı belirlemek için üç elemanın medyan değerini kullandığımız için, pivot dizinin ortasına yakın bir yerde olabilir, ancak bu garanti edilmez. Hızlı sıralama, pivotun konumu dizinin ortası olduğunda en verimlidir. Uçlardan ortaya doğru çalışma te



ŞEKİL 12-15 Hızlı Sıralama Pivotu

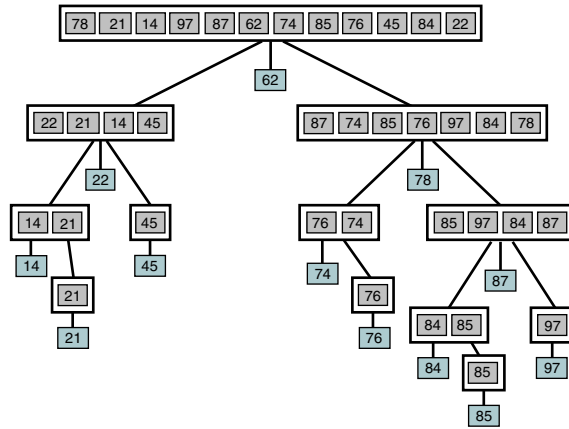
Şekil 12-15'i incelerken, değişimlerin başlamasından önce, medyan elemanın ortada olduğunu ve medyanı belirlemek için kullanılan üç elemandan en küçüğünün doğru konumda olduğunu not edin. Medyan sol algoritması çağırıldıktan sonra, medyan sol konumda ve en küçük eleman ortada yer alır. Pivot anahtar daha sonra işlemi kolaylaştırmak için bir bekleme alanına taşınır. Bu hareket aslında pivot anahtarı doğru konumuna yerleştiren bir değişimin ilk parçasıdır.

Sıralamayı takip etmeye yardımcı olmak için, bir tarafta pivotun hemen ardından bir duvar ve diğer tarafta bir duvar hayal edin (bkz. Şekil 12-15). Sol duvardan başlıyoruz ve pivotun sağında yer alması gereken bir eleman ararken sağa doğru hareket ediyoruz. Element 97'de bir tane buluyoruz. Solda sağa ait bir eleman bulduktan sonra, sağ duvardan başlıyoruz ve pivotun solunda yer alması gereken bir eleman ararken sola doğru hareket ediyoruz. Element 45'te bir tane buluyoruz. Bu noktada iki elementi değiştiriyoruz, duvarları 87. elemanın soluna ve 76. elemanın sağına taşıyoruz ve işlemi tekrarlıyoruz.

Bu sefer sağa hareket ettiğimizde, ilk sol eleman, 87, sağda yer alıyor. Sağdan aşağıya indiğimizde, 22. elemanın solda yer aldığını görüyoruz. Yine, sol ve sağ elemanları değiştiriyoruz ve duvarı hareket ettiriyoruz. Bu noktada duvarların kesiştiğini not ediyoruz. Böylece pivot elemanın için doğru konumu bulmuş olduk. Pivotun bulunduğu alandaki veriyi dizinin ilk elemanına taşıyoruz ve ardından pivotu diziye geri getiriyoruz, pivot anahtarını tutma alanına taşıdığımızda başlattığımız değişimi tamamlıyoruz. Liste artık pivot elemanının (62) dizideki diğer tüm elemanlara göre doğru konumda olacak şekilde yeniden düzenlendi.

Pivotun solundaki elemanlar pivotun hepsinden küçüktür ve pivotun sağındaki elemanlar pivotun hepsinden büyüktür. Herhangi bir eşit v eri pivotun sağında bulunur.

Pivot anahtarını doğru konumuna yerleştirdikten sonra, sol bölümü sıralamak için hızlı sıralama işlemini yineleyerek çağırıyoruz. Sol bölüm tamamsıralandığında, sağ bölümü sıralamak için hızlı sıralama işlemini yineleyerek çağırıyoruz. Hem sol bölüm hem de sağ bölüm sıralandığında, liste tamamsıralanmış olur. Şekil 12-15'teki veriler, yalnızca Şekil 12-16'daki hızlı sıralamakullanılarak tamamen sıralanmıştır.



ŞEKİL 12-16 Hızlı Sıralama İşlemi

Hızlı sıralama pseudocode'u Algoritma 12-8'de gösterilmiştir.

ALGORİTMA 12-8 Hızlı Sıralama

```

Algoritma quickSort (liste, sol, sağ)
Bir dizi, liste, özyineleme kullanarak sıralanır.
  Ön liste sıralanacak veri dizisidir
  sol ve sağ, sırasıyla listenin ilk ve son
  elemanlarını tanımlar
  Son liste sıralanmıştır
1  eğer ((sağ - sol) > minBoyut)
    Hızlı sıralama
    1  medyanSol (liste, sol, sağ)
    2  pivotu sol elemana ayarla
    3  sıralamaSol'u sol + 1 olarak ayarla
    4  sortRight'ı sağa ayarla
    5  döngü (sortLeft <= sortRight)
        Sağda olması gereken soldaki anahtar bul

```

devam ediyor

ALGORİTMA 12-8 Hızlı Sıralama (devam ediyor)

```

1  döngü (sortLeft anahtarı < pivot anahtarı)
    1  sortLeft'i artır
2  döngüyü bitir
    Solda olması gereken sağdaki anahtarı bul
3  döngü (sortRight anahtarı >= pivot anahtarı)
    1  sortRight'i azalt
4  döngüyü bitir
5  eğer (sortLeft <= sortRight)
    1  değiştir(list, sortLeft, sortRight)
    2  sortLeft artır
    3  sortRight'i azalt
6  sonlandır
6  döngüyü bitir
    sonraki geçiş için hazırla
7  sortLeft - 1 elemanını sola hareket ettir
8  pivot elemanını sortLeft - 1 elemanına hareket ettir
9  eğer (sol < sortRight)
    1  quickSort (list, sol, sortRight - 1)
10 sonlandır
11 eğer (sortLeft < sağ)
    1  quickSort (list, sortLeft, sağ)
12 sonlandır
2  aksi takdirde
    1  insertionSort (liste, sol, sağ)
3  sonlandır
son quickSort

```

Algoritma 12-8 Analizi Ortalama noktaya iki uçtan çalışan tasarıma ek olarak, bu algoritmanın iki yönü daha fazla tartışmayı gerektiriyor. Sıralama sol ve sıralama sağ elemanlarını belirlemek için kullanılan döngüler (ifade 1.5.1 ve 1.5.3) yalnızca bir koşulu test eder. Bir diziye işleyen çoğu döngü, dizinin sonunu da test etmelidir, ancak bu algoritmada pivot anahtarın seçimine dayalı olarak testi atlayabiliriz. Pivot anahtarın, üç elemanın medyan değeri olması garanti edilir: ilki, sonuncusu ve ortadaki. Bu nedenle, medyan anahtar, en soldaki anahtardan daha az olamaz ve en sağdaki anahtardan daha büyük olamaz. En kötü durumda, en soldaki veya en sağdaki anahtara eşittir.

En kötü durumu varsayalım: bir bölümdeki tüm öğelerin aynı anahtar değeri vardır. Bu durumda, pivot anahtar en soldaki ve en sağdaki anahtarlarla eşittir. Soldan başlayıp listeyi yukarı doğru hareket ettirmeye başladığımızda, pivot anahtar **sortLeft** anahtarına eşit olduğu için hemen dururuz. Daha sonra sağdan başlayıp aşağı doğru hareket ettiğimizde, tüm anahtarlar eşit olduğunda listenin başını geçiyoruz. Ancak, iki öğe birbirini geçtiği için, öğeleri değiştirmeyiz. Bunun yerine, 1.7. ifadeye geçeriz. Bu durumda, dizinin başından çıkmış olan sağ sıralama indeksini asla kullanmayız.

Hoare'ın orijinal algoritması özyinelemeli değildi. Bir yığın tutması gerektiğinden, yığının boyutunun minimumda tutulmasını sağlamak için mantık ekledi. Sadece sol bölümü sıralamak yerine, hangi bölümün daha büyük olduğunu belirledi ve daha küçük bölümü sıralarken büyük olanı yığına koydu. Yığın boyutunu minimize etmekle ilgilenmiyoruz, çünkü iki nedenimiz var. İlk olarak, pivot anahtarını medyan değer olarak seçtik.

Bu nedenle, iki parçanın boyutları genellikle aynı olmalıdır, böylece rekürsif çağrılarının sayısını en aza indirmiş oluruz. Daha önemlisi, rekürsiyon kullandığımız için, stack boyutunu önceden belirlememize gerek yoktur. Sadece sistemden stack alanı sağlamasını isteriz.

Değişim Sıralama Verimliliği

Değişim sıralamalarında, Knuth'un en iyi genel amaçlı sıralama olarak adlandırdığı sıralamayı buluyoruz: hızlı sıralama. Nedenini görmek için sıralama çabalarını belirleyelim.

Balon Sıralaması

Bubble sort'un kodu Algoritma 12-5, "Bubble Sort"da gösterilmiştir. Düzgün ekleme ve düzgün seçim sıralamalarında gördüğümüz gibi, verileri sıralamak için iki döngü kullanır. Döngüler aşağıda gösterilmiştir:

```
3  döngü (geçerli <= son VE sıralı yanlış)
  ...
  3  döngü (yürüyücü > geçerli)
```

Dış döngü iki koşulu test eder: mevcut indeks ve sıralı bayrağı. Listenin son geçişe kadar sıralı olmadığını varsayarsak, dizi üzerinden n kez döneriz. İç döngüdeki döngü sayısı, dış döngüdeki mevcut konuma bağlıdır. Bu nedenle, ortalama olarak listenin yarısı kadar döner. Toplam döngü sayısı, both döngülerin çarpımıdır, bu da bubble sort verimliliğini

$$n\left(\frac{n+1}{2}\right)$$

Büyük-O notasyonunda balon sıralama verimliliği $O(n^2)$.

Balon sıralama verimliliği $O(n^2)$.

Hızlı Sıralama

Hızlı sıralama kodu, Algoritma 12-8, "Hızlı Sıralama"da gösterilmiştir. Algoritmaya hızlı bir bakış, beş döngü olduğunu ortaya koymaktadır (üç yinelenmeli döngü ve iki özyinelenmeli döngü). Algoritma ayrıca düz ekleme sıralamasını bir alt yordam olarak içermektedir. Hızlı sıralamanın verileri tamamen sıralamak için kullanılabileceği—yani ekleme sıralamasının ortadan kaldırmanın mümkün olduğu—göz önüne alındığında, ekleme sıralamasının kullanılmadığı varsayımıyla hızlı sıralama kısmını analiz ediyoruz. Hızlı sıralama döngüleri aşağıda gösterilmiştir:

```
5  döngü (sıralamaSol <= sıralamaSağ)
  1  döngü (sıralamaSol anahtar < pivot anahtar)
  ...
```

devam ediyor

```

2   döngüyü bitir
3   döngü (sortRight anahtar >= pivot anahtar)
   ...
4   döngüyü bitir
5   ...
6   döngü sonu
   ...
9   eğer (sol < sortRight)
1  quickSort (list, sol, sortRight - 1)
10  sonlandır
11  eğer (sortLeft < sağ)
1  quickSort (list, sortLeft, sağ)
12  eğer (sortLeft < sağ)

```

Hızlı sıralamadaki her geçişin listeyi üç parçaya böldüğünü hatırlayın: pivot anahtardan daha küçük elemanların listesi, pivot anahtar ve pivot anahtardan daha büyük elemanların listesi. İlk döngü (ifade 5), sıralanan dizinin kısmındaki her elemanı inceler; ifade 5.1, listenin sol kısmında döner; ifade 5.3, listenin sağ kısmında döner.

Bu nedenle, listeyi n kez döngüye alıyorlar.

Benzer şekilde, iki özyinelemeli döngü, dizinin her bir kısmını işler, ya pivot anahtarın solunda ya da sağında. Soru, ne kadar sıklıkla çağrıldıklarıdır. Hızlı sıralamanın en verimli olduğunu söylediğimizi hatırlayın pivot anahtar dizinin ortasında olduğunda. Bu yüzden bir medyan değeri kullandık. Bunun merkezine göre nispeten yakın bir yerde olduğunu varsayarsak, listeyi yaklaşık aynı boyutta iki alt listeye böldüğümüzü görüyoruz. Çünkü 2 ile bölüyoruz, döngü sayısı logaritmik. Bu nedenle toplam sıralama çabası, ilk döngünün özyinelemeli döngülerle çarpımıdır, yani $n \log n$.

Hızlı sıralama verimliliği $O(n \log n)$.

Algoritmalar, liste küçük olduğunda neden düz ekleme sıralamasını kullandığımızı açıklamaz. Cevap, algoritma kodunda ve özyineleme yükünde yatmaktadır. Liste yeterince küçük hale geldiğinde, düz ekleme sıralamasını kullanmak daha verimli hale gelir.

Özet

Analizimiz, hızlı sıralamanın verimliliğinin yığın sıralamasıyla aynı olduğu sonucuna varıyor. Bu doğrudur çünkü büyük- O notasyon gerçek sıralama verimliliğinin yalnızca bir yaklaşık değeridir. Her ikisi de $n \log n$ sırasındadır, ancak daha doğru formüller geliştirirsek, gerçek verimliliklerini yansıtmak için, hızlı sıralamanın aslında daha verimli olduğunu göreceğiz. Tablo 12-3, bu bölümde tartıştığımız altı türü özetlemektedir.

n	Döngü sayısı		
	Doğrudan ekleme Doğrudan seçim Balon sıralamaları	Takvim	Yığın ve hızlı
25	625	55	116
100	10,000	316	664
500	250,000	2364	4482
1000	1,000,000	5623	9965
2000	4,000,000	13,374	10,965

TABLO 12-3 Sıralama Karşılaştırmaları

Küçük listeler için düz ekleme sıralamasını ve büyük listeler için hızlı sıralamayı kullanmanızı öneririz. Shell sıralaması ilginç bir sıralama olup, sıralama tarihindeki önemli bir rol oynamıştır, ancak çoğu durumda bunu önermiyoruz. Dış sıralamayı tartıştığımızda göreceğiniz gibi, yığın sıralaması gibi algoritmalar özel durumlarda önemli bir niş rol oynamakta ve bu nedenle sıralama araç setinizde yer almalıdır.

Değişim Sıralama Uygulaması

Baloncuk sıralama ve hızlı sıralama algoritmaları için C koduna bakalım.

Balon Sıralama Kodu

Balon sıralama kodu Program 12-5'te gösterilmiştir.

PROGRAM 12-5 Balon Sıralama

```

123456789 /* ===== bubbleSort =====
Listeyi balon sıralama kullanarak sırala. Komşu elemanlar
karşılaştırılır ve liste sıralanana kadar değiştirilir.
Önce liste en az bir öge içermelidir
son son liste elemanının indeksini içerir
Sonra liste artan sırada sıralanmıştır
*/
void bubbleSort (int list [], int last)
{
// Yerel Tanımlar
int temp;

```

devam ediyor

PROGRAM 12-5 Bubble Sort (devamı)

```

131415161718192021222324252627282930
// Beyanlar
// Her yineleme bir sıralama geçiştir
for (int current = 0, sorted = 0;
    current <= last && !sorted;
    current++)
    for (int walker = son, sıralanmış = 1;
        walker > geçerli;
        yürüteç--)
        eğer (list[walker] < list[walker - 1]) // Herhangi bir değişim, listenin sıralı olmadığı anlamına
            gelir
            {
                sıralanmış = 0;
                sıcaklık = liste[yürüteç];
                liste[yürüteç] = liste[yürüteç - 1];
                liste[yürüteç - 1] = sıcaklık;
            } // eğer

    geri dönmek;
} // bubbleSort

```

Hızlı Sıralama Kodu

Bu bölümde daha önce tanımlanan üç hızlı sıralama algoritmasını uyguluyoruz. Sıralama dizisi bir kez daha basit bir tam sayı dizisidir.

İlk fonksiyon hızlı sıralama, Program 12-6'da gösterilmiştir. Düz ekleme sıralamasının değiştirilmiş versiyonu Program 12-7'de gösterilmiştir ve medyan fonksiyonu Program 12-8'de gösterilmiştir.

PROGRAM 12-6 Hızlı Sıralama

```

123456789
/* ===== quickSort =====
   Dizi, sortData[left..right] özyineleme kullanılarak sıralanır.
   Önce sortData, sıralanacak verilerin bir dizisidir
       left, sortData'nın ilk elemanını tanımlar
       right, sortData'nın son elemanını tanımlar
   Sonra sortData dizisi sıralanmıştır
*/
void quickSort (int sortData[ ], int sol, int sağ)
{
    #define MIN_SIZE 16

    // Yerel Tanımlar
    int sortLeft;
    int sortRight;
    int pivot;
    int hold;
1011121314151617

```

devam ediyor

PROGRAM 12-6 Hızlı Sıralama (devamı)

```

181920212223242526272829303132333435363738394041424344454647484950515253
// Beyanlar
if ((sağ - sol) > MIN_SIZE)
{
    medianLeft (sortData, sol, sağ);
    pivot = sortData [sol];
    sortLeft = sol + 1;
    sortRight = sağ;
    while (sortLeft <= sortRight)
    {
        // Sağda olması gereken soldaki anahtarı bul
        while (sortData [sortLeft] < pivot)
            sortLeft = sortLeft + 1; // Sol
        // da olması gereken anahtarı sağda bul

        sortRight = sortRight - 1; // i
        f (sortLeft <= sortRight) {

            hold = sortData[sortLeft];
            sortData[sortLeft] = sortData[sortRight];
            sortData[sortRight] = hold;

            sortLeft = sortLeft + 1;
            sortRight = sortRight - 1;

        } // eğer
    } // iken
    // Sonraki geçiş için hazırlık
    sortData [left] = sortData [sortLeft - 1];
    sortData [sortLeft - 1] = pivot;
    if (sol < sortRight)
        quickSort (sortData, left, sortRight - 1);
    if (sortLeft < right)
        quickSort (sortData, sortLeft, right);
    } // eğer sağ
    aksi takdirde
    quickInsertion (sortData, left, right);
    dön;
} // hızlıSıralama

```

PROGRAM 12-7 Hızlı Sıralama için Değiştirilmiş Düz Ekleme Sıralaması

```

12345678
/* ===== quickInsertion =====
Veri[1...son] sıralama sırası kullanarak sıralanır. Veri sıralı ve sıral
anmamış listelere ayrılır. Her geçişte, sıralanmamış listedeki ilk ele
man sıralı listeye eklenir. Bu, hızlı sıralama ile kullanmak için değiştiril
miş bir sıralama sırası özel versiyonudur.

Önce veri en az bir eleman içermelidir
ilk, verideki ilk elemanın indeksidir

```

devam ediyor

PROGRAM 12-7 Hızlı Sıralama için Değiştirilmiş Düz Sıralama Sırası (devamı)

```

9          son, verideki son elemanın indeksidir
          Sonra veri yeniden düzenlenmiştir
*/
void quickInsertion (int data[], önce int, int son)
{
// Yerel Tanımlar
    int hold;
    int yürüteç;

// Beyanlar
    for (int geçerli = ilk + 1;
        geçerli <= son;
        geçerli++)
    {
        tut = data[current]; yürüyücü = current - 1; while (
        yürüyücü >= first    && tut < data[yürüyücü])

        {
            data[yürüyücü + 1] = data[yürüyücü];
            yürüyücü = yürüyücü - 1;
        } // while
        data[yürüyücü + 1] = tut;
    } // for
    geri dönmek;
} // quickInsertion

```

PROGRAM 12-8 Hızlı Sıralama için Medyan Solu

```

1  /* ===== medianLeft =====
2  Bir dizinin medyan değerini bul
3  sortData[left..right] ve bunu
4  location sortData[left] içine yerleştir.
5  Önce sortData en az üç elemanlı bir dizi
6  left ve right dizinin sınırlarıdır
7  Medyan değeri sortData[left] içine yerleştir
8  */
9  void medianLeft (int sortData[], int left, int right)
10 {
11 // Yerel Tanımlar
12     int mid;
13     int tutma;
14
15 // Beyanlar
16     // sortData'yı medyanın orta konumda olacak şekilde yeniden düzenle
17     mid = (left + right) / 2;

```

devam ediyor

PROGRAM 12-8 Hızlı Sıralama için Medyan Solu (devam ediyor)

```

1819202122232425262728293031323334353637383940414243
if (siralamaVerileri[sol] > siralamaVerileri[orta])
{
    hold = sortData[sol];
    sortData[left] = sortData[mid];
    sortData[mid] = hold;
} // eğer
if (verileri sirala[sol] > Verileri sirala[sağ])
{
    hold = sortData[sol];
    sortData[left] = sortData[right];
    sortData[right] = hold;
} // eğer
if (siralamaVerileri[orta] > siralamaVerileri[sağ])
{
    hold = sortData[mid];
    sortData[mid] = sortData[right];
    sortData[right] = hold;
} // eğer

// Medyan ortadadır. Sol ile değiştir
hold = sortData[left];
sortData[left] = sortData[mid];
sortData[mid] = hold;

geri dönmek;
} // medyanSol

```

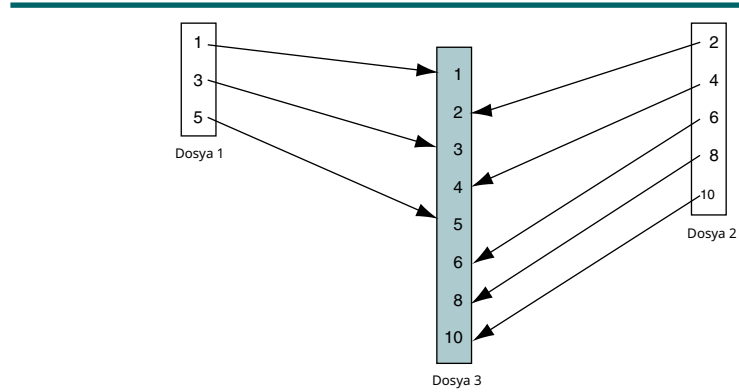
12.5 Dış Sıralamalar

Şimdiye kadar incelediğimiz tüm algoritmalar iç sıralamalardı—yani, sıralama süreci sırasında tüm verilerin birincil bellek içinde bulunmasını gerektiren sıralamalar. Şimdi dış sıralamaya—sıralama süreci sırasında verilerin bir kısmının ikincil bellek içinde saklanmasına izin veren sıralamalara—dikkatimizi çeviriyoruz.

Terim *dış sıralama* biraz yanıltıcı bir terimdir. Büyük dosyaları sıralamak için harcanan işin çoğu sıralama değil, aslında birleştirmedir. Gördüğümüz gibi, dış sıralamalar, veri bloklarını dahili olarak sıralayarak ve bunları dosyalara yazarak başlar. Tüm veriler dahili sıralama aşamasından geçtikten sonra, tamamen sıralanmış bir dosyanın oluşturulması dosyaların birleştirilmesiyle yapılır. Bu nedenle, dış sıralamayı anlamak için önce birleştirme kavramını anlamamız gerekir.

Sıralı Dosyaları Birleştirme

Bir birleştirme, belirli bir anahtara göre sıralanmış iki dosyayı, aynı belirli anahtar üzerinde bir sıralı dosya haline getirme sürecidir. Basit bir örnek Şekil 12-17'de gösterilmiştir.



ŞEKİL 12-17 Basit Birleştirme

Dosya 1 ve dosya 2, dosya 3'e birleştirilecektir. Dosyaları birleştirmek için, dosya 1'deki ilk kaydı dosya 2'deki ilk kayıtla karşılaştırır ve daha küçük olanı, 1, dosya 3'e yazarız. Ardından, dosya 1'deki ikinci kaydı dosya 2'deki ilk kayıtla karşılaştırır ve daha küçük olanı, 2, dosya 3'e yazarız. İşlem, tüm veriler sıralı bir şekilde dosya 3'e birleştirilene kadar devam eder. Bu iki dosyayı birleştirmek için kullanılan mantık, nispeten basit olup, Algoritma 12-9'da gösterilmiştir.

ALGORİTMA 12-9 Dosyaları Birleştir

```

Algoritma dosyalarıBirleştir
İki sıralı dosyayı bir dosyada birleştir.
  Ön giriş dosyaları sıralıdır
  Son çıkış dosyasında giriş dosyaları sıralı olarak birleştirilir
1 dosyaları aç
2 oku (dosya1 kayda1)
3 oku (dosya2 kayda2)
4 döngü (dosya1 sonu değil VEYA dosya2 sonu değil)
  1 eğer (kayda1 anahtarı <= kayda2 anahtarı)
    1 yaz (kayda1 dosya3'e)
    2 oku (dosya1 kayda1)
    3 eğer (dosya1 sonu)
      1 kaydı1 anahtarını sonsuzluğa ayarla
    4 sonlandır
  2 aksi takdirde
    1 yaz (kaydı2 dosya3'e)
    2 oku (dosya2'yi kaydı2'ye)
    3 eğer (dosya2 sonu)
      1 kaydı2 anahtarını sonsuzluğa ayarla
    4 sonlandır
3 sonlandır

```

devam ediyor

ALGORİTMA 12-9 Dosyaları Birleştir (devamı)

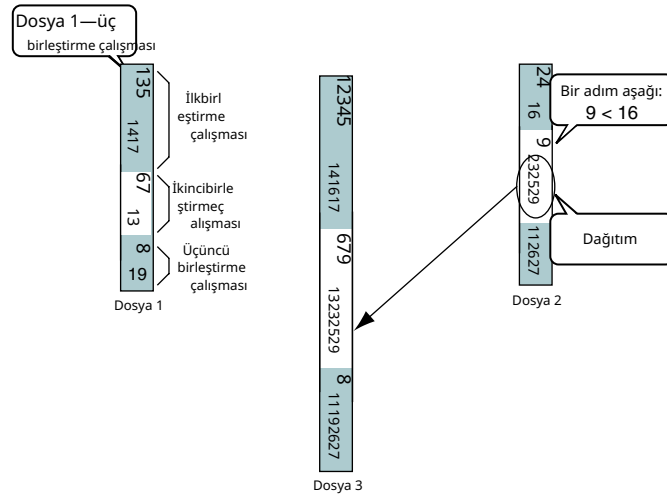
- 5 döngüyü bitir
 - 6 dosyaları kapat
- bitir birleştirDosyaları

Algoritma 12-9 Analizi Birleştirme dosyaları nispeten basit bir algoritma olmasına rağmen, tartışmaya değer bir nokta var.

Bir dosya sona ulaştığında, ikinci dosyada daha fazla veri olabilir. Bu nedenle, her iki dosya da sona ulaşana kadar işlemeye devam etmemiz gerekiyor. Her dosya için dosya sonu işlemlerini ele almak üzere ayrı kod blokları yazabiliriz, ancak daha basit bir yöntem var. Bir dosya sona ulaştığında, anahtar değerini yapay olarak yüksek bir değere ayarlıyoruz. Algoritmada bu değer sonsuzluk olarak tanımlanır (bkz. ifadeler 4.1.3.1 ve 4.2.3.1). İki anahtarı karşılaştırdığımızda (if ifade 4.1), sona ulaşan dosya yüksek bir değere zorlanır ve diğer dosya işlenir. Yüksek değere genellikle bir **sentinel** denir. Bu mantığın tek sınırlaması, sentinel değerinin geçerli bir veri değeri olamayacağıdır.

Sırasız Dosyaları Birleştirme

Ancak bir birleştirme sıralaması içinde, genellikle Algoritma 12-9'da gösteril en durumdan farklı bir durumla karşılaşırız. Dosyalar sıralı olmadığından, ve riler sırayla akar ve ardından bir sıra kesintisi olur ve ardından başka bir veri serisi sırayla gelir. Bu durum Şekil 12-18'de gösterilmektedir.



ŞEKİL 12-18 Dosyaları Birleştirme Örneği

Bir dosyadaki ardışık sıralı veri serisine birleştirme çalışması denir. Şekil 12-18'de üç dosyanın da üç birleştirme çalışması vardır. Onları görmek kolay olsun diye, her dosyadaki birleştirme çalışmalarını aralıklı ve renklendirilmiş olarak gösterdik. Bir adım düşüşü meydana gelir

bir birleştirme dosyasındaki kayıtların sıralı düzeninin bozulduğunda. Her bir birleştirme çalışmasının sonu bir adım aşağı ile belirlenir. Örneğin, Şekil 12-18'deki birleştirme çalışmasındaki anahtar 16'nın ardından ikinci birleştirme çalışmasındaki anahtar 9 geldiğinde dosya 2'de bir adım aşağı vardır. Bir dosyanın sonu da bir adım aşağı olarak kabul edilir.

Bir dosyanın birleştirilmesinde, alternatif birleştirme girişi sonrası ardışık bir dizi kaydın birleştirme çıktı dosyasına kopyalanması süreci rolloout olarak bilinir. Şekil 12-18'de, kayıt 13 çıktı dosyasına kopyalandıktan sonra bir adım aşağı inildiğinde, dosya 2'deki ikinci birleştirme çalışmasındaki kalan kayıtlar (23, 25 ve 29) birleştirme çıktısına aktarılır.

Şimdi Şekil 12-18'deki birleştirme sürecine yakından bakalım. Tamamen sıralı olmayan dosyaları birleştirirken, her dosyadan karşılık gelen birleştirme çalışmaları çıktı dosyasında birleştirilir. Böylece, dosya 1'deki birleştirme çalışması 1, dosya 2'deki birleştirme çalışması 1 ile birleştirilerek dosya 3'teki birleştirme çalışması 1'i üretir. Benzer şekilde, giriş dosyalarındaki birleştirme çalışmaları 2 ve 3, çıktıdaki birleştirme çalışmaları 2 ve 3'ü üretmek için birleştirilir.

Bir giriş birleştirme dosyasında bir adım düşüşü tespit edildiğinde, alternatif dosyadaki birleştirme çalışması iki dosyayı senkronize etmek için geri alınmalıdır. Böylece, birleştirme çalışması 2'de, kayıt 13 ile kayıt 8 arasındaki adım düşüşü tespit edildiğinde, üçüncü birleştirme çalışmasına başlayabilmemiz için dosya 2'de kalan üç kaydı geri almamız gerekir.

Son olarak, birleştirme çıktısının tamamen sıralı bir dosya olmadığını belirtmek önemlidir. Bu özel durumda, iki tane daha birleştirme çalışması gereklidir. Tam süreci görmek için daha yüksek seviyeli bir örneğe geçiyoruz.

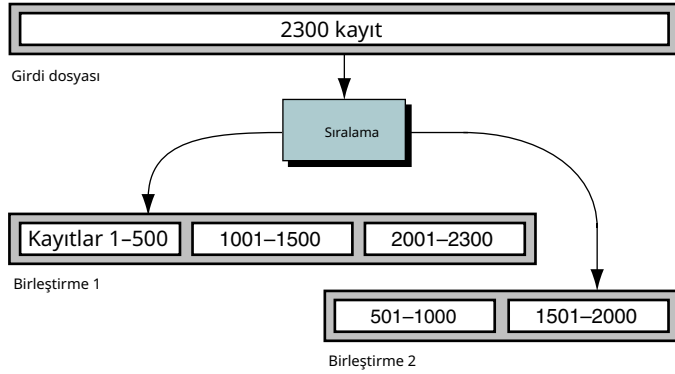
Sıralama Süreci

2300 kayıttan oluşan bir dosyanın sıralanması gerektiğini varsayalım. Dış bir sıralamada, mümkün olduğunca çok kaydı sıralayarak iki veya daha fazla birleştirme dosyası oluşturuyoruz. Kayıt boyutunun ve sıralama programımız için mevcut belleğin maksimum sıralama dizisi boyutunun 500 kayıt olmasına izin verdiğini varsayarsak, ilk 500 kaydı okuyup sıralayarak bir birleştirme çıktı dosyasına yazmaya başlıyoruz. İlk 500 kaydı sıralarken, kalan 1800 kaydı ikinci depolamada tutuyoruz. İlk birleştirme çalışmasını yazdıktan sonra, ikinci 500 kaydı okuyor, sıralıyor ve alternatif bir birleştirme çıktı dosyasına yazıyoruz. Sıralama sürecine devam ediyoruz, 500 sıralı kaydı (kayıt 1001'den 1500'e) birinci birleştirme çıktı dosyasına ve diğer 500 sıralı kaydı (kayıt 1501'den 2000'e) ikinci birleştirme çıktı dosyasına yazıyoruz. Son olarak, son 300 kaydı sıralayıp birinci çıktı birleştirme dosyasına yazıyoruz. Bu noktada, Şekil 12-19'da gördüğümüz durumu oluşturmuş oluyoruz. Verilerin birleştirme çalışmaları

arı haline ilk işlenmesi sıralama aşaması olarak bilinir.

Dış sıralamanın sıralama aşamasını tamamladıktan sonra, birleştirme aşamasına geçiyoruz. Girdi birleştirme dosyalarının her birinin tam okunması ve bir veya daha fazla çıktı birleştirme dosyasına birleştirilmesi ayrı bir birleştirme aşaması olarak kabul edilir. Ne kadar birleştirme çalışması oluşturulduğuna bağlı olarak, sıfır veya daha fazla birleştirme aşaması vardır. Tüm veriler bir seferde belleğe sığarsa veya dosya baştan sıralanmışsa, yalnızca bir birleştirme çalışması vardır ve ilk birleştirme çıktı dosyasındaki veriler

tamamen sıralanmıştır. Ancak bu durum bir istisnadır; genellikle birkaç birleştirme aşaması gereklidir.



ŞEKİL 12-19 Dış Sıralamada Sıralama Aşaması

Bilgisayar bilimcileri yıllar içinde birçok farklı birleştirme kavramı geliştirmiştir. Temsili olan üç tanesini sunuyoruz: doğal birleştirme, dengeli birleştirme ve çok aşamalı birleştirme.

Doğal Birleştirme

Bir doğal birleştirme sabit sayıda girdi birleştirme dosyasını bir birleştirme çıktı dosyasına sıralar. Her bir birleştirme aşaması arasında, birleştirme çalışmaları yeniden birleştirmek için girdi dosyalarına yeniden dağıtmak amacıyla bir dağıtım aşaması gereklidir. Şekil 12-20, doğal iki yönlü birleştirme kullanılarak sıralanacak 2300 kaydımızın diyagramıdır—yani, iki girdi birleştirme dosyası ve bir çıktı birleştirme dosyası ile doğal bir birleştirme.

Bu süreci diyagramlamak için, bir şablon dosyası için klasik sembolü kullanıyoruz: bantsembolü. Ancak, verilerin ya bir bant dosyasında ya da birdisk dosyasında olabileceğini kabul edin. Tek gereklilik, bunun bir sıralı dosya olmasıdır.

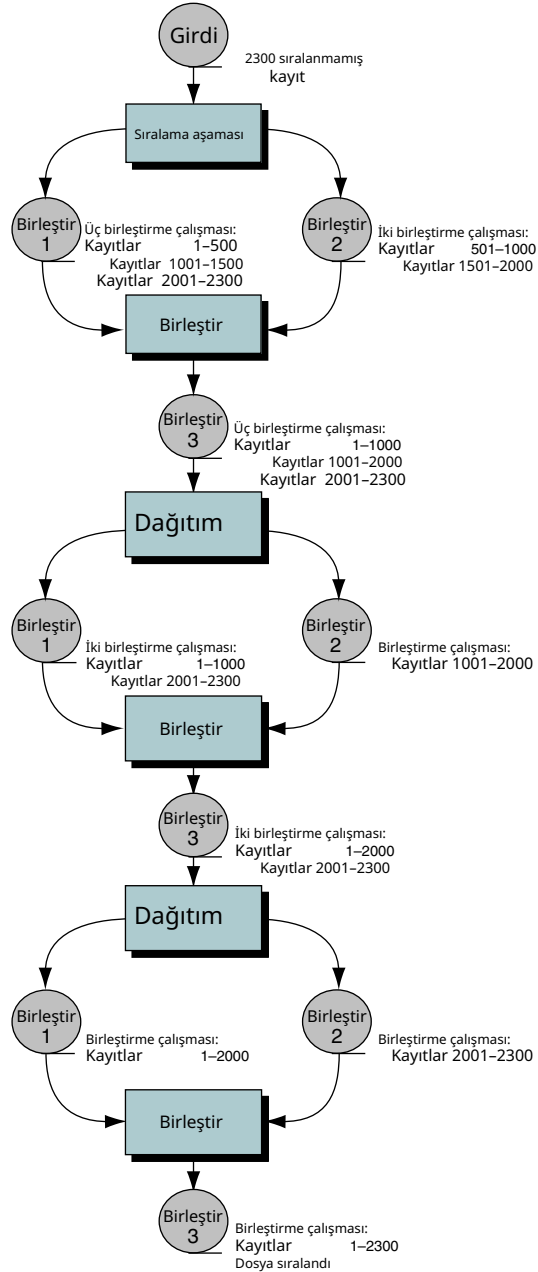
Doğal bir birleştirmede, tüm birleştirme çalışmaları tek bir dosyaya yazılır. Bu nedenle, dosya tamamen sıralı değilse, birleştirme çalışmaları her bir birleştirme aşaması arasında dua birleştirme dosyasına dağıtılmalıdır. Bu işlem çok verimsizdir, özellikle kayıtları okumak ve yazmak, tüm veri işleme işlemlerinin en yavaşları arasındadır. Bu nedenle, sorumuz, birleştirme sürecini nasıl daha verimli hale getirebiliriz. Cevap, dengeli bir birleştirmede bulunur.

Doğal bir birleştirmede, her aşama sabit sayıda girdi dosyasını tek bir çıktı dosyasına birleştirir.

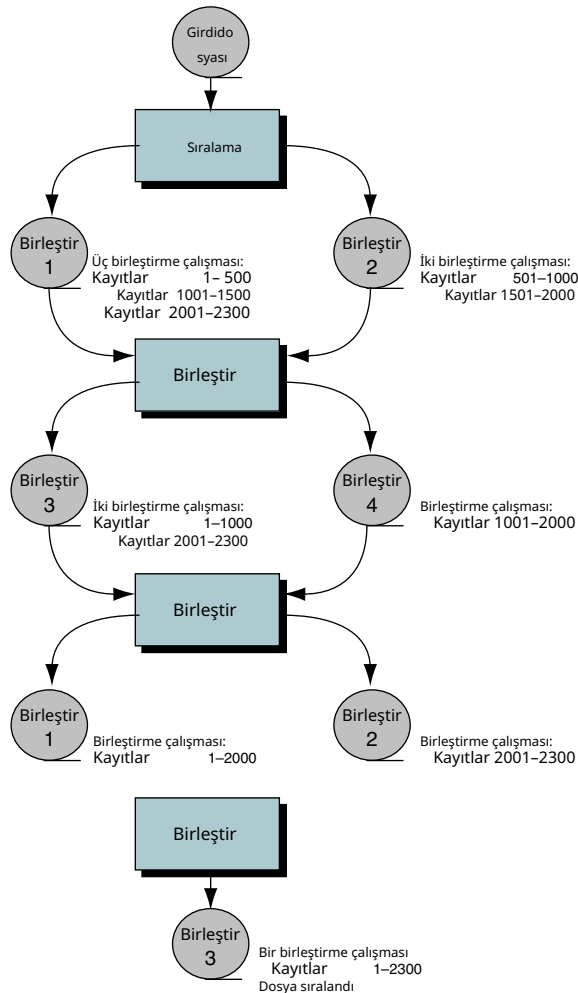
Dengeli Birleştirme

Dengeli birleştirme, sabit sayıda girdi birleştirme dosyası ve aynı sayıda çıktı birleştirme dosyası kullanır. Herhangi bir sayıda birleştirme dosyası kullanılabilir,

dörtten fazla olması alışılmadık. Her bir birleştirme aşamasında birden fazla birleştirme dosyası oluşturulduğundan, dağıtım aşamasına gerek yoktur. Şekil 12-21, 23 00 kaydımızı dengeli bir iki yönlü birleştirme ile sıralar.



ŞEKİL 12-20Doğal İki Yönlü Birleştirme Sıralaması



ŞEKİL 12-21 Dengeli İki Yönlü Birleştirme Sıralaması

Dengeli iki yönlü bir birleştirme için dört birleştirme dosyası gereklidir. İlkbirleştirme aşaması, dosya 1'deki ilk birleştirme çalışmasını dosya 2'deki ilk birleştirme çalışmasıyla birleştirir ve bunu dosya 3'e yazar. Ardından, dosya 1'deki ikinci birleştirme çalışması, dosya 2'deki ikinci birleştirme çalışmasıyla birleştirilir ve dosya 4'e yazılır. Bu noktada dosya 2'deki tüm birleştirme çalışmaları işlenmiştir, bu nedenle dosya 1'deki kalan birleştirme çalışmasını birleştirme dosyası 3'e dağıtıyoruz. Bu 300 kayıtlık dağıtım boşa harcanmış bir çabadır. Bu adımı ortadan kaldırmak ve birleştirme işlemini mümkün olduğunca verimli hale getirmek istiyoruz. Eğer çok aşamalı birleştirme kullanırsak bunu yapabiliriz.

Dengeli birleştirme, aynı sayıda girdi ve çıktı birleştirme dosyası kullanarak dağıtım aşamasını ortadan kaldırır.

Çok Aşamalı Birleştirme

Çok aşamalı birleştirmede, sabit sayıda girdi birleştirme dosyası bir çıktı birleştirme dosyasına birleştirilir ve girdi birleştirme dosyaları, girdileri tamamen birleştirildiğinde hemen yeniden kullanılır. Çok aşamalı birleştirme, tartıştığımız birleştirme sıralarının en karmaşığıdır.

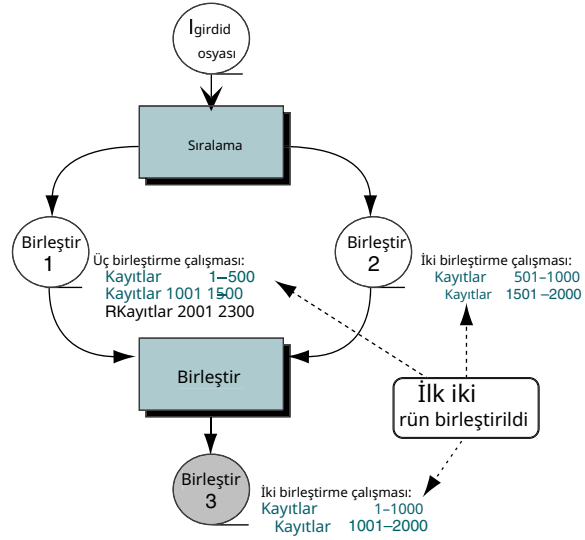
Polifaz birleştirmede, sabit sayıda girdi dosyası bir çıktı dosyasına birleştirilir. Her girdi dosyasındaki veriler tamamen birleştirildiğinde, hemen çıktı dosyası haline gelir ve önceki çıktı dosyası bir girdi dosyası olur.

Bu süreci göstermek için, Şekil 12-22'yi dikkatlice inceleyin. İşlem, natürel iki yönlü birleştirme için olduğu gibi başlar. Dosya 1'deki ilk birleştirme çalışması, dosya 2'deki ilk birleştirme çalışmasıyla birleştirilir. Ardından, dosya 1'deki ikinci birleştirme çalışması, dosya 2'deki ikinci birleştirme çalışmasıyla birleştirilir. Bu noktada birleştirme 2 boştur ve ilk birleştirme aşaması tamamlanmıştır. Bu nedenle birleştirme 2'yi kapatıyoruz ve çıktığı olarak açıyoruz, birleştirme 3'ü kapatıyoruz ve girdi olarak açıyoruz. Dosya 1'deki üçüncü birleştirme çalışması, ardından dosya 3'teki ilk birleştirme çalışmasıyla birleştirilir ve birleştirilen veriler birleştirme 2'ye yazılır. Birleştirme 1 boş olduğundan, birleştirme aşaması 2 tamamlanmıştır. Bu nedenle birleştirme 1'i kapatıyoruz ve çıktığı olarak açıyoruz, birleştirme 2'yi kapatıyoruz ve girdi olarak açıyoruz. Her bir girdi dosyasında yalnızca bir birleştirme çalışması olduğundan, bu iki birleştirme çalışması birleştirme 1'e birleştirildiğinde sıralama tamamlanır.

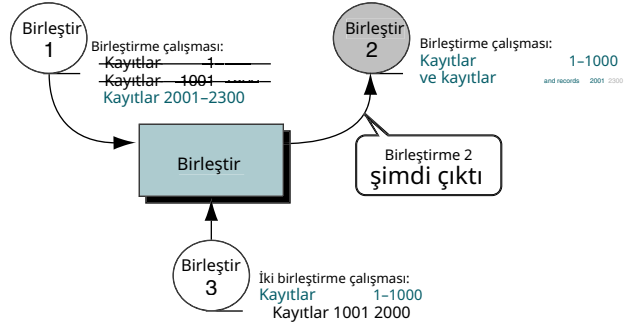
Sıralama Aşaması Yeniden Gözden Geçirildi

Polifaz sıralaması ile birleştirme aşamalarını mümkün olduğunca geliştirdik. Şimdi sıralama aşamasına geri dönüyoruz ve bunu geliştirmeye çalışıyoruz. Sıralama aşamasında mümkün olan en hızlı sıralamayı kullandığımızı varsayalım. İç sıralama hızını iki katına çıkarmayı başarsak bile, pek bir şey kazanılmaz. Bugünün modern bilgisayarları pikosaniye hızlarında çalışırken ve dosya işleme mikro saniye hızların da gerçekleşirken, sıralama hızını artırmanın pek bir faydası yoktur. Aslında, tam tersi doğrudur. Sıralama hızını biraz yavaşlatırsak, daha yavaş bir sıralama kullanarak, genel hızı artırabiliriz. Bunu nasıl yaptığımıza bakalım.

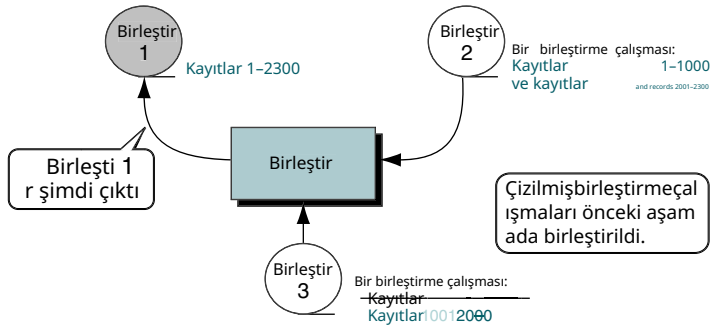
Ağaç sıraları olarak adlandırılan bir sıralama sınıfı, sıralama tamamlanmadan önce verileri birleştirme dosyasına yazmamıza olanak tanır. Böylece ağaç sıralarını kullanarak daha uzun birleştirme çalışmaları yazabiliriz, bu da birleştirme çalışmaları sayısını azaltır ve dolayısıyla sıralama sürecini hızlandırır. Bir ağaç sıralaması olan yığın sıralamasını incelediğimiz için, bunun daha uzun birleştirme çalışmaları yazmak için nasıl kullanılabileceğine bakalım.



(a) İlk birleştirme aşaması tamamlandı



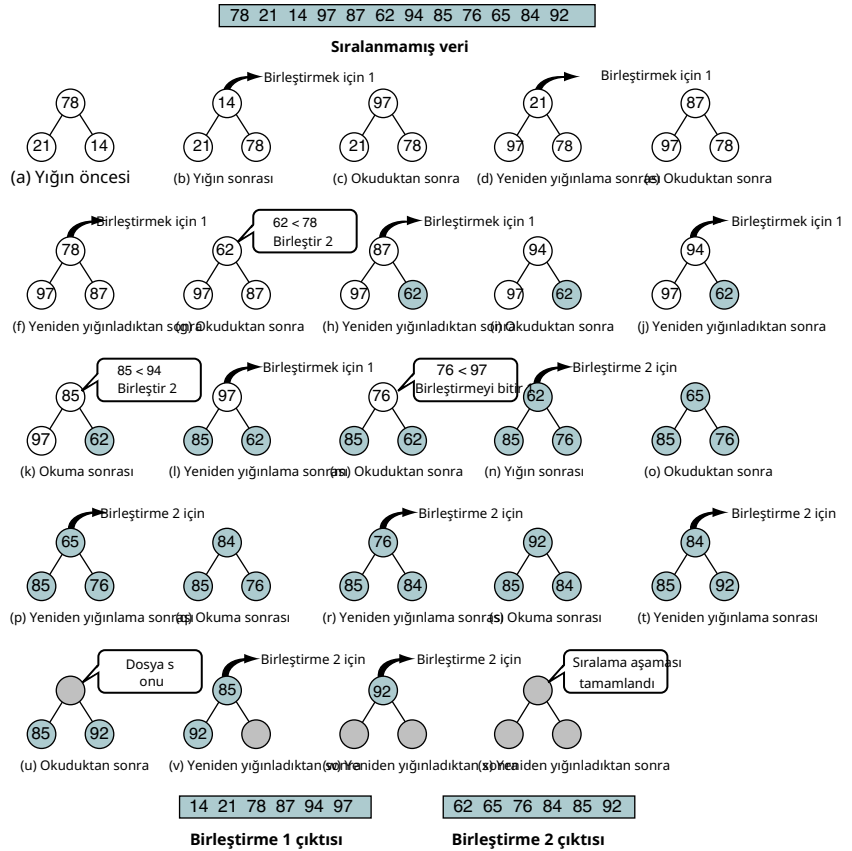
(b) İkinci birleştirme aşaması tamamlandı



(c) Üçüncü birleştirme aşaması; sıralama tamamlandı

ŞEKİL 12-22 Polifazlı Birleştirme Sıralaması

Şekil 12-23, yığın sıralamasını uzun birleştirme işlemleri yazmak için nasıl kullanabileceğimizi göstermektedir. Sıralanacak 12 elemanlık bir liste verildiğinde ve üç düğümlü bir yığın kullanarak, verileri iki birleştirme işlemine getiriyoruz. Sıralama dizisini doldurarak başlıyoruz ve sonra bunu bir minimum yığın haline getiriyoruz. Yığını oluşturduktan sonra, kök düğümde bulunan en küçükelementi ilk birleştirme dosyasına yazıyoruz ve dördüncü elementi (97) kök düğümde okuyoruz. Yine yığını yeniden yapılandırıyoruz ve bu sefer 21'i birleştirme dosyasına yazıyoruz. 87'yi okuduktan ve yığını yeniden inşa ettikten sonra, 78'i birleştirme dosyasına yazıyoruz.



ŞEKİL 12-23 Yığın Sıralama Kullanarak Dış Sıralama Aşaması

62'yi okuduktan sonra Şekil 12-23(g)'de gösterilen yığını elde ediyoruz. Bu noktada okuduğumuz veri (62), birleştirme dosyasına yazdığımız son elemandan (78) daha küçüktür. Bu nedenle, mevcut birleştirme çalışmasına birleştirme 1'de yerleştirilemez. Tanıma göre, bir sonraki birleştirme çalışmasına aittir. Bu nedenle, yığındaki son elemanı a(87) değiştiriyoruz ve yığın boyutundan 1 çıkarıyoruz, böylece yığın sadece iki elemanlı oluyor. 62'yi içeren üçüncü elemanın şu anda yığına olmadığını gölgeleyiyoruz. Yığını yeniden düzenledikten sonra 87'yi birleştirme 1'e yazıyoruz. Ardından 94'ü okuyoruz, yığını yeniden düzenliyoruz ve bunu birleştirme 1'e yazıyoruz. 85'i okuduğumuzda,

1. birleştirmedeki en büyük anahtardan (94) daha az olduğunu keşfediyoruz, bu nedenle yığınlardan da çıkarılması gerekir. Yığınlardaki son elemanla değiştirildikten sonra, bizim durumumuz Şekil 12-23(l)'de gösterilmiştir.

1. birleştirmeye 97 yazdıktan sonra, 76'yı okuyoruz. 76, birleştirme dosyasına yazılan son elemenden daha az olduğu için, bir sonraki birleştirme çalışmasına da aittir. Bu noktada dizideki tüm elemanlar ikinci birleştirme çalışmasına aittir. Bu nedenle, birinci birleştirme çalışması tamamlanmıştır, Şekil 12-23(m)'de gösterildiği gibi.

Yığını yeniden inşa ettikten sonra, ilk elemanı birleştirme dosyası 2'ye yazıyoruz. Okumaya, yeniden yığınlamaya ve yazmaya devam ediyoruz, ta ki 84'ü birleştirme dosyasına yazana kadar. Bu noktada sıralanacak tüm verileri okuduk ve giriş dosyası dosyanın sonundadır, Şekil 12-23(u)'de gösterildiği gibi. Bu nedenle, son elemanı yığınaaktarır, yığın boyutundan 1 çıkarır ve yeniden yığınlarız. 85'i birleştirme çalışmasına yazdıktan sonra, yığındaki son elemanı köke taşır, yeniden yığınlar ve onubirleştirme dosyasına yazarız. Yığın artık boş olduğundan, sıralama tamamlanmıştır.

Bu örnekte oluşturulan birleştirme çalışmaları incelendiğinde, her ikisinin dedizi boyutunun iki katı uzunlukta olduğu görülmektedir. Genel bir kural olarak, heuristik çalışmalarımız, orta lama birleştirme çalışması boyutunun sıralama için kullanılan dizinin boyutunun iki katı olduğunu göstermiştir. Bu daha uzun birleştirme çalışmaları, bir birleştirme geçişini ortadan kaldırır, bu da önemli bir zaman tasarrufudur.

12.6 Hızlı Sıralama Verimliliği

Hızlı sıralama, günümüzde bilinen en iyi genel amaçlı sıralama algoritması olarak kabul edilmektedir. Bu nedenle, düşük bir sıralama çabası göstermesini bekleriz. Bölüm 12.4'te, hızlı sıralamanın verimliliğini büyük-O notasyonu kullanarak tartışık ve gerçekten hızlı olduğunu gösterdik. Bu bölümde, size rekürsif algoritmaları matematiksel olarak analiz etmeniz için bir arka plan sağlıyoruz.

Hızlı sıralamanın karmaşıklığını hesaplamak için, n elemanlı bir diziyi sıralamak için gereken karşılaştırma sayısının $f(n)$ olduğunu hayal edin. Aşağıdakileri gözlemliyoruz:

- Sıfır veya bir elemanlı bir dizi zaten sıralıdır. Bu, $f(0) = f(1) = 0$ anlamına gelir.
- Eğer pivotu i indeksinde seçersek, iki alt dizi elde ederiz. Sol alt dizinin (i) elemanı vardır ve sağ alt dizinin ($n - 1 - i$) elemanı vardır. Sol alt diziyi sıralamak için gereken karşılaştırma sayısı $f(i)$ ve sağ alt diziyi sıralamak için gereken karşılaştırma sayısı $f(n - 1 - i)$ olup, i 0 ile $n - 1$ arasında olabilir.
- Diziyi sürekli olarak alt dizilere bölmek için n karşılaştırmaya ihtiyacımız var.

Artık karşılaştırma sayısı için genel formülü yazabiliriz:

$$\begin{aligned} f(n) &= 0 & \text{eğer } n < 2 \\ f(n) &= f(i) + f(n - i - 1) + n & \text{eğer } n \geq 2 \end{aligned}$$

Bu formül, hızlı sıralamanın karmaşıklığını üç durumda bulmak için kullanılabilir: en kötü durum, en iyi durum ve ortalama durum.

En Kötü Durum

Modern hızlı sıralama için en kötü durum yoktur. Singleton'un 1969'daki modifikasyonu bunu etkili bir şekilde ortadan kaldırmıştır. Ancak orijinal sıralamada, en kötü durum verilerin zaten sıralandığı durumdaydı. Bu durumda i her zaman 0'dır, bu da dizinin pivot hücreğine ve dizinin geri kalanına bölündüğü anlamına gelir. Liste sıralandığında, ister artan ister azalan, formül şu şekilde basitleştirilebilir:

$$\begin{aligned} f(n) &= f(i) + f(n - i - 1) + n \\ f(n) &= f(n - 1) + n \end{aligned}$$

Çünkü $f(i) = f(0)$ ve $i = 0$

Bu, $f(n)$ değerini $f(n - 1)$ cinsinden tanımlayan bir özyinelemeli hesaplamadır. Özyinelemedeki tüm faktörleri değerlendirirsek, şunu gözlemleriz:

$$\begin{aligned} f(n) &= f(n - 1) + n \\ f(n - 1) &= f(n - 2) + n - 1 \\ f(n - 2) &= f(n - 3) + n - 2 \\ f(n - 3) &= f(n - 4) + n - 3 \\ f(n - 4) &= f(n - 5) + n - 4 \\ &\dots \\ f(1) &= f(0) + 1 \\ f(0) &= 0 \end{aligned}$$

Tüm sağ terimleri sol terimlerle toplarsak, ilk satırın sağ tarafındaki $f(n - 1)$ i ikinci satırın sol tarafındaki $f(n - 1)$ ile iptal edilir; ikinci satırın sağ tarafındaki $f(n - 2)$ üçüncü satırın sol tarafındaki $f(n - 2)$ ile iptal edilir ve bu şekilde devam eder. Geriye kalan şudur:

$$f(n) = n + (n - 1) + (n - 2) + (n - 3) + \dots + 1 + 0$$

Bu aritmetik seridir,¹⁰ bu da $n(n + 1) / 2$ olarak değerlendirilir. Parantezleri ortadan kaldırdığımızda, terimleri çarptığımızda şunu elde ederiz:

$$f(n) = n \frac{(n + 1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

Ama bu n^2 , bu nedenle

$$O(n) = n^2$$

En Kötü Durum

En İyi Durum

En iyi durumda, diziler her zaman eşit

boyutlarda iki alt diziye bölünür. Diğer bir deyişle, pivot anahtar tam olarak ortada yer alır.

10. Ek E'ye bakın.

dizi. Bunun gerçekleşmesi için, orijinal dizideki eleman sayısı n üzeri bir eksi bir olmalıdır. Diğer bir deyişle, eleman sayısı $n = 2^k - 1$ veya tersine, $k = \log_2(n + 1)$. Yani, bu gereksinimi karşılayan bir dizi verildiğinde, $i = 2^{k-1} - 1$ ve

$$\begin{aligned} f(n) &= f(i) + f(n - i - 1) + n \\ f(2^k - 1) &= f(2^{k-1} - 1) + f(2^k - 1 - (2^{k-1} - 1) - 1) + 2^k - 1 \\ f(2^k - 1) &= 2f(2^{k-1} - 1) + 2^k - 1 \end{aligned}$$

Bu özyinelemeli tanımları çözmek için, k yerine $k - 1$ 'i tekrar tekrar koyabiliriz ve eşitliğin her iki tarafını 2 ile çarpabiliriz. Bu bize şunu verir:

$$\begin{aligned} f(2^k - 1) &= 2f(2^{k-1} - 1) + 2^k - 1 \\ 2f(2^{k-1} - 1) &= 2^2f(2^{k-2} - 1) + 2^{k-1} - 1 \\ 2^2f(2^{k-2} - 1) &= 2^3f(2^{k-3} - 1) + 2^{k-2} - 1 \\ &\dots \\ 2^{k-1}f(2^1 - 1) &= 2^k f(2^1 - 1) + 2^{k-1}(2^1 - 1) \end{aligned}$$

Şimdi sağ tarafın hepsini ve sol tarafın hepsini topluyoruz; farklı taraflardan eşit terimleri iptal ettikten sonra, şunu elde ediyoruz:

$$\begin{aligned} f(2^k - 1) &= 2^{k-1}f(1) + 2^0(2^k - 1) + 2^1(2^{k-1} - 1) + \\ &\quad 2^2(2^{k-2} - 1) + \dots + 2^{k-1}(2^1 - 1) \end{aligned}$$

Ama, $f(1) = 0$, bu yüzden

$$\begin{aligned} f(2^k - 1) &= (2^k - 1) + (2^k - 2^1) + (2^k - 2^2) + \dots + (2^k - 2^{k-1}) \\ f(2^k - 1) &= (k - 1)2^k - (2^0 + 2^1 + 2^2 + \dots + 2^{k-1}) \end{aligned}$$

$(2^0 + 2^1 + 2^2 + \dots + 2^{k-1})$ terimi, ¹¹ ile sadeleşen bir geometrik seridir.

$$(2^k - 1) / (2 - 1) \text{ or } 2^k - 1$$

Yani bizde var

$$f(2^k - 1) = (k - 1)2^k - (2^k - 1)$$

Eğer $n = 2^k - 1$ veya $k = \log_2(n + 1)$ ifadesini değiştirirsek, elde ederiz

$$\begin{aligned} f(n) &= (\log(n + 1) - 1)(n + 1) - (n) \\ f(n) &= (n + 1) \log(n + 1) - n - 1 - n \\ &= (n + 1) \log(n + 1) - 2n - 1 \end{aligned}$$

11. Ek E'ye bakın.

Ama, bu $n \log n$, bu nedenle

$$O(n) = n \log n$$

En İyi Durum

Ortalama Durum

Ortalama durumda, i her yerde olabilir. Her bir bölümde şunlar mümkündür: (1) i başlangıçta. Singleton varyasyonu ile bu oldukça olası değildir. İki anahtarın eşit ve üçüncü bir anahtardan küçük veya ona eşit olması gerekir. (2) i dizinin ortasında bir yerde. Bu en olası olanıdır. Medya n değeri seçmek için kullanılan üç değer, listeyi rastgele temsil eder. Ve nihayet, (3) i dizinin sonunda. Yine, (1)'de belirtilen aynı nedenlerden dolayı, bu oldukça olası değildir. Bu nedenle, i ortada bir yerde olduğunu varsayıyoruz ve bir ortalama alıyoruz.

Ortalama alırsak, şunu görüyoruz:

$$\begin{aligned} f(n) &= f(i) + f(n - i - 1) + n \\ f(n) &= (1/n)\{f(0) + f(1) + \dots + f(n-1)\} + \\ &\quad (1/n)\{f(n-1) + f(n-2) + \dots + f(0)\} + n \end{aligned}$$

İlk iki terimin tam olarak aynı olduğunu gözlemleyebiliriz (parantez içindeki fonksiyonlar ters sırada listelenmiştir). Yani, şunları elde ediyoruz:

$$f(n) = (2/n)\{f(0) + f(1) + \dots + f(n-1)\} + n$$

Bu özyineleme tanımını çözmek için, denklemin her iki tarafını n ile çarpıyoruz.

$$nf(n) = 2\{f(0) + f(1) + \dots + f(n-1)\} + n^2$$

Şimdi n 'i $n-1$ ile değiştiriyoruz, bu da bize şu denklemi veriyor:

$$(n-1)f(n-1) = 2\{f(0) + f(1) + \dots + f(n-2)\} + (n-1)^2$$

Şimdi yeni denklemi önceki denklemin üzerinden çıkarıyoruz, bu da bize şunu veriyor:

$$nf(n) - (n-1)f(n-1) = 2f(n-1) + n^2 - (n-1)^2$$

Sadeleştirerek, şunu elde ediyoruz:

$$nf(n) = (n+1)f(n-1) + 2n - 1$$

Her iki tarafı $n+1$ ile bölüyoruz ve şunu elde ediyoruz:

$$\frac{1}{n+1} f(n) = \frac{1}{n} f(n-1) + \frac{2}{n+1} - \frac{1}{n(n+1)}$$

Şimdi n yerine $n - 1, n - 2, \dots, 2$ sürekli olarak yerleştiriyoruz

$$\begin{aligned}
 \frac{1}{n+1} f(n) &= \frac{1}{n} f(n-1) + \frac{2}{n+1} - \frac{1}{n(n+1)} \\
 \frac{1}{n} f(n-1) &= \frac{1}{n-1} f(n-2) + \frac{2}{n} - \frac{1}{(n-1)n} \\
 \frac{1}{n-1} f(n-2) &= \frac{1}{n-2} f(n-3) + \frac{2}{n-1} - \frac{1}{(n-2)(n-1)} \\
 &\vdots \\
 \frac{1}{3} f(2) &= \frac{1}{2} f(1) + \frac{2}{3} - \frac{1}{(2)(3)}
 \end{aligned}$$

Şimdi her iki tarafı topluyoruz ve benzer terimleri iptal ediyoruz

$$\begin{aligned}
 (1/(n+1))f(n) &= \\
 (1/2)f(1) + \\
 (2/(n+1) + 2/n - 1) + \dots + 2/3 - \\
 \{(1/n(n+1) + 1/(n-2)n) + 1/(n-2)(n-1) + \dots + 1/6\}
 \end{aligned}$$

Ama, $f(1) = 0$, bu yüzden

$$\begin{aligned}
 1/(n+1)f(n) &= \\
 (2/(n+1) + 2/n - 1) + \dots + 2/3 - \\
 \{(1/n(n+1) + 1/(n-2)n) + 1/(n-2)(n-1) + \dots + 1/6\} \\
 f(n) &= \\
 (n+1)(2/(n+1) + 2/n - 1) + \dots + 2/3 - \\
 \{(n+1)(1/n(n+1) + 1/(n-2)n) + 1/(n-2)(n-1) + \dots + 1/6\}
 \end{aligned}$$

İfade $(2/(n+1) + 2/n - 1) + \dots + 2/3$ doğal bir logaritmadır; ikinci ifade önemsizdir çünkü $1/n$ sırasındadır ve büyük-O notasyonu kuralları gereği göz ardı edilebilir. Bu nedenle, sonuç

$$f(n) = (n+1) \ln n$$

ve büyük-O notasyonu içinde

$$O(n) = (n \log n)$$

12.7 Anahtar Terimler

dengeli birleştirmek	hızlı sıralama
abarcık sırala	açarak yaymak
madağıtım aşaması	seçim sıralaması
ğişim sıralamaha	sentinel
rici sıralamayığı	kabuk sıralaması
n sıralama\i	sıralama
ç sıralamabirleş	sıralama verimliliği
tirme	sıralama sırası
birleştirme aşaması	sıralama geçişi
irleştirme çalıştır	sıralama aşaması
masıbirleştirme sı	sıralama kararlılığı
ralamasiminimum yığınd	adım düşürme
oğal birleştirmedöndür	düzgün ekleme sıralaması
me	düzgün seçim sıralaması
çok fazlı birleştirme	

12.8 Özet

- Bilgisayar bilimlerinde en yaygın uygulamalardan biri sıralamadır.
- Sıralamalar genellikle iç veya dış olarak sınıflandırılır.
 - Bir iç sıralamada, tüm veriler sıralama süreci boyunca birincil depolama da tutulur.
 - Bir dış sıralama, sıralanan veriler için birincil depolamayı ve birinci l belleğe sığmayan veriler için ikincil depolamayı kullanır.
- Veriler artan veya azalan sırada sıralanabilir.
- Sıralama kararlılığı, eşit anahtarlara sahip verilerin çıktıda görelî giriş sıralarını koruduğunu belirten bir sıralama özelliğidir.
- Sıralama verimliliği, bir sıralamanın göreceli verimliliğinin bir ölçüsüdür.
- Sıralama sürecinde verilerin her bir geçişi bir geçiş olarak adlandırılır.
- Dahili sıralama üç geniş kategoriye ayrılabilir: ekleme, seçim ve değişim.
- Bu bölümde iki ekleme sıralama yöntemi tartışılmıştır: düz ekleme sıralaması ve shell sıralaması.
 - Düz ekleme sıralamasında, liste herhangi bir anda iki alt listeye ayrılır: sıralı ve sıralanmamış. Her geçişte, sıralanmamış alt listenin ilk elemanı, sıralı alt listeye uygun yere eklenerek aktarılır.

- Kabuk sıralama algoritması, düz ekleme sıralamasının geliştirilmiş bir versiyonudur, burada işlem, listeyi sıralamak için farklı artışlar kullanır. Her artışta liste segmentlere ayrılır ve bu segmentler birbirinden bağımsız olarak sıralanır.
- Bu bölümde iki seçim sıralama yöntemi tartışılmaktadır: düz seçim sıralaması ve yığın sıralaması.
 - Düz seçim sıralamasında, liste her an iki alt listeye ayrılır: sıralı ve sıralanmamış. Her geçişte, işlem sıralanmamış alt listeden en küçük öğeyi seçer ve bunu sıralanmamış alt listenin başındaki öğe ile değiştirir.
 - Yığın sıralaması, düz seçim sıralamasının geliştirilmiş bir versiyonudur. Yığın sıralamasında, yığındaki en büyük öğe, sıralanmamış alt listenin son öğesi ile değiştirilir. Ancak, bu sıralama yönteminde en büyük öğeyi seçmek çok daha kolaydır çünkü en büyük öğe yığının köküdür.
- Bu bölümde iki değişim sıralama yöntemi tartışılmaktadır: balon sıralaması ve hızlı sıralama.
 - Balon sıralamasında, liste her an iki alt listeye ayrılır: sıralı ve sıralanmamış. Her geçişte, en küçük öğe sıralanmamış alt listeden yukarı doğru kabarcık yaparak sıralı alt listeye taşınır.
 - Hızlı sıralama, listenin sürekli olarak daha küçük alt listelere ayrıldığı ve sıralama dışındaki öğeler arasında değişimlerin gerçekleştiği değişim sıralamanın yeni bir versiyonudur. Hızlı sıralamanın her geçişi bir pivot seçer ve listeyi üç gruba ayırır: pivotun anahtarından daha küçük olan öğelerin bir bölümü, nihai doğru konumuna yerleştirilen pivot öğesi ve pivotun anahtarına eşit veya daha büyük olan öğelerin bir bölümü. Sıralama, ardından sol bölümü hızlı sıralayarak ve ardından sağ bölümü hızlı sıralayarak devam eder.
- Doğrudan ekleme, doğrudan seçim ve balon sıralamanın verimliliği $O(n^2)$.
- Shell sıralamanın verimliliği $O(n^{1.25})$, yığın ve hızlı sıralamanın verimliliği ise $O(n \log n)$.
- Dış sıralama, sıralama sürecinde verilerin bir kısmının ikincil depolama alanında saklanmasına olanak tanır.
- Dış sıralama iki aşamadan oluşur: sıralama aşaması ve birleştirme aşaması.
- Birleştirme aşaması, üç yöntemden birini kullanır: doğal birleştirme, dengeli birleştirme ve çok aşamalı birleştirme.
 - Doğal birleştirmede her aşama, sabit sayıda girdi dosyasının bir çıktı dosyasında birleştirir. Doğal birleştirme, her birleştirme aşaması arasında bir dağıtım süreci gerektirir.

- Dengeli birleştirmede, dağıtım süreçleri, aynı sayıda girdi ve çıktı dosyası kullanılarak ortadan kaldırılır.
 - Çok aşamalı birleştirmede, bir dizi girdi dosyası bir çıktı dosyasında birleştirilir. Ancak, ilk olarak tükenen girdi dosyası hemen çıktı dosyası olarak kullanılır. Önceki aşamadaki çıktı dosyası, bir sonraki aşamadaki girdi dosyalarından biri haline gelir.
- Dış sıralamada sıralama aşamasının verimliliğini artırmak için, minimum yığın sıralaması gibi bir ağaç sıralaması kullanabiliriz. Bu sıralama, sıralama süreci tamamlanmadan önce birleştirme dosyasına yazmamıza olanak tanır, bu da daha uzun birleştirme işlemleri sonucunu doğurur.

12.9 Uygulama Setleri

Alıştırmalar

1. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. İlk iki eleman doğrudan ekleme sıralaması kullanılarak sıralanmıştır. Doğrudan ekleme sıralama algoritmasının üç geçişinden sonra dizideki elemanların değeri ne olacaktır?

3 13 7 26 44 23 98 57

2. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. Dizi, bir artış geçişinden sonra içeriğini gösterin. Artış faktörü $k = 3$.

23 3 7 13 89 7 66 2 6 44 18 90 98 57

3. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. İlk iki eleman doğrudan seçim sıralaması kullanılarak sıralanmıştır. Seçim sıralama algoritmasının üç geçişinden sonra dizideki elemanların değeri ne olacaktır?

7 8 26 44 13 23 98 57

4. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. Yığın sıralama algoritmasının üç geçişinden sonra dizideki elemanların değeri ne olacaktır?

44 78 22 7 98 56 34 2 38 35 45

5. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. İlk iki eleman baloncuk sıralaması kullanılarak sıralanmıştır. Baloncuk sıralama algoritmasının üç geçişinden sonra dizideki elemanların değeri ne olacaktır? En küçük elemanı yukarı doğru baloncuk yapan baloncuk sıralama versiyonunu kullanın.

7 8 26 44 13 23 57 98

6. Aşağıda gösterilen elemanları içeren bir dizi bulunmaktadır. Hızlı sıralama kullanarak, ilk pivot doğru konumuna yerleştirildikten sonra dizinin içeriğini gösterin. O noktada var olan üç alt listeyi tanımlayın.

44 78 22 7 98 56 34 2 38 35 45

7. İki geçişten sonra bir sıralama algoritması, aşağıdaki dizi:

47 3 21 32 56 92

aşağıda gösterildiği gibi yeniden düzenlenmiştir.

3 21 47 32 56 92

Hangi sıralama algoritması kullanılıyor (doğrudan seçim, baloncuk veya doğrudan ekleme)? Cevabınızı savunun.

8. İki geçişten sonra bir sıralama algoritması, aşağıdaki dizi:

80 72 66 44 21 33

aşağıda gösterildiği gibi yeniden düzenlenmiştir.

21 33 80 72 66 44

Hangi sıralama algoritması kullanılıyor (doğrudan seçim, baloncuk veya doğrudan ekleme)? Cevabınızı savunun.

9. İki geçişten sonra bir sıralama algoritması, aşağıdaki dizi:

47 3 66 32 56 92

aşağıda gösterildiği gibi yeniden düzenlenmiştir.

3 47 66 32 56 92

Hangi sıralama algoritması kullanılıyor (doğrudan seçim, baloncuk veya doğrudan ekleme)? Cevabınızı savunun.

10. Aşağıdaki iki dosyayı birleştirirken her bir birleştirme aşamasından sonra sonucu gösterin:

6 12 19 23 34 · 8 11 17 20 25 · 9 10 15 25 35

13 21 27 28 29 · 7 30 36 37 39

11. Aşağıdaki dosyadan başlayarak, dış sıralama ve doğal birleştirme yöntemi kullanılarak oluşturulan tüm dosyaların içeriğini gösterin (bir sıralama aşaması dahil etmeyin):

37 9 23 56 4 5 12 45 78 22 33 44 14 17 57 11 35 46 59

12. Dengelemiş birleştirme yöntemini kullanarak Egzersiz 11'i yeniden çalışın.
13. Yedi elemanlı bir dizi boyutu ve bir ekleme sıralaması ile polifazik birleştirme yöntemini kullanarak Egzersiz 11'i yeniden çalışın.

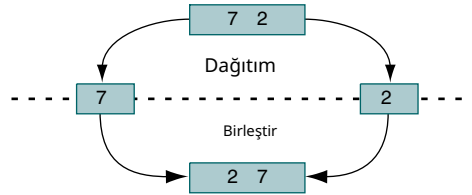
Problemler

14. Program 12-3, "Ekleme Sıralaması"nı, 1000 rastgele sayının sıralanması için gereken veri hareketlerinin sayısını sayacak şekilde değiştirin. Bir veri hareketi, bir veri elemanının dizideki bir konumdan başka birine, bir bekleme alanına veya bir bekleme alanından diziye geri hareketidir. Diziyi sıralamadan önce ve sonra gösterin. Programın sonunda, diziyi sıralamak için gereken toplam hareketleri gösterin.
15. Shell sıralaması kullanarak Problem 14'ü tekrarlayın (bkz. Program 12-4).
16. Seçim sıralaması kullanarak Problem 14'ü tekrarlayın (bkz. Program 12-1).
17. Yığın sıralaması kullanarak Problem 14'ü tekrar edin (bkz. Program 12-2).
18. Program 12-5'e bakarak balon sıralama ile Problem 14'ü tekrarlayın.
19. Hızlı sıralama kullanarak Problem 14'ü tekrar edin (bkz. Program 12-8).
20. Bubble sıralama algoritmasını (Program 12-5) aşağıdaki gibi değiştirin: Her geçişte iki yönlü kabarcık kullanın. İlk kabarcıkta, en küçük eleman yukarı kabarcık yapar; ikinci kabarcıkta, en büyük eleman aşağı kabarcık yapar. Bu sıralama, shaker sıralama olarak bilinir.
21. "Sıralamalar ve ADT'ler" konusundaki teknikleri kullanarak, Seçim Sıralaması (Program 12-1) kullanarak bir ADT oluşturun. Ardından, bir tamsayı dizisi ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.
22. Shell sıralamasının artan fikrini bir seçim sıralamasına uygulayan bir algoritma yazın. Algoritma önce düz bölüm sıralamasını öğelere uygulayarak $n/2$ eleman aralığında (ilk, orta ve son). Daha sonra bunu $n/3$ eleman aralığına, ardından elemanlara $n/4$ aralığında ve devamına uygular.

23. Seçim sıralama algoritmasının (bkz. Program 12-4) özinelemeli bir versiyonunu yazın.
24. Tek bağlı bir liste kullanarak (Program 12-3) ekleme sıralama algoritmasını yeniden yazın.

Projeler

25. Birleştirme sıralaması, böl ve fethet paradigmasının bir örneğidir. Tartışmalarımız da, birleştirmeyi yalnızca harici bir sıralama yöntemi olarak kullandık. Ayrıca dahili sıralama için de kullanılabilir. Nasıl çalıştığını görelim. Eğer yalnızca iki elemandan oluşan bir listeye sahipseniz, listeyi iki yarıya bölebilir ve ardından birleştirebiliriz. Diğer bir deyişle, birleştirme sıralaması tamamen iki işleme, dağıtım ve birleştirme b aşımıdır. Bu temel işlem Şekil 12-24'te gösterilmiştir.

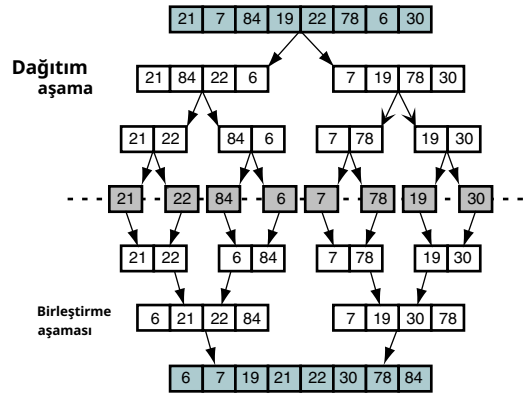


ŞEKİL 12-24 Proje 25 için Böl ve Birleştir

İki elemandan daha uzun bir liste verildiğinde, dağıtım ve birleştirme işlemlerini tekrarlayarak sıralama yapabiliriz. Başlarken girdi listesinin içinde kaç eleman olduğunu bilmediğimiz için, listeyi başlangıçta ilk elemanı bir çıkış listesine, ikinci elemanı ikinci bir çıkış listesine yazarak dağıtabiliriz ve ardından girdi listesi iki çıkış listesine bölünene kadar alternatif olarak birinci listeye ve ardından ikinci listeye yazmaya devam edebiliriz. Çıkış listeleri daha sonra dengeli bir iki yönlü birleştirme kullanılarak sıralanabilir. Bu işlem Şekil 12-25'te gösterilmiştir. Sıralama aşaması kullanmadan sıralama yaptığı için “sırasız sıralama” olarak adlandırılabilir.

Bu yaklaşımı kullanarak 500 rastgele sayıdan oluşan bir diziyi sıralamak için bir C programı yazın. Sıralamadan önce ve sonra verileri yazdırın.

26. Rastgele sayıların bir dizisini sıralayan bir program yazın, shellsıralaması ve hızlı sıralama kullanarak. Her iki sıralama da aynı veriyi kullanmalıdır. Her sıralama i kez çalıştırılmalıdır. İlk sıralama için, diziyi 1 ile 999 arasında rastgele sayılara doldurun. İkinci sıralama için, diziyi neredeyse sıralı bir liste ile doldurun. Neredeyse sıralı listenizi, sıralı rastgele sayı listesindeki 19 ve 20. elemanları ters çevirerek oluşturun. Her sıralama için, bu listeyi sıralamak için gerekli karşılaştırma ve hareket sayısını sayın.



ŞEKİL 12-25 Sırasız Sıralama

Programı üç kez çalıştırın, bir kez 100 öğeden oluşan bir dizi ile, bir kez 500 öğeden oluşan bir dizi ile ve bir kez 1000 öğeden oluşan bir dizi ile. İlk çalıştırma için yalnızca (100 eleman), sıralanmamış veriyi 10'ar 10'ar matrisler halinde (her biri 10 sayıdan oluşan 10 satır) yazdırın. Tüm çalıştırmalar da veriyi sıralamak için gerekli karşılaştırma sayısını ve hareket sayısını yazdırın.

İstatistiklerinizin mümkün olduğunca doğru olduğundan emin olmak için, her döngü sınır koşulu testini ve sıralama algoritmalarınızdaki her seçim ifadesini analiz etmelisiniz. Aşağıdaki notlar bu analize yardımcı olmalıdır:

- Tüm döngüler, gövdesinde bir sayım artırımı gerektirir.
- Ön test döngüleri (*while* ve *for*) de, son testi saymak için döngüden önce (önerilir) veya sonra bir sayım artırımı gerektirir.
- C'nin Boolean ve/veya ifadeleri değerlendirmek için kısayol kuralını kullandığınız unutmayın. Onları saymanın en iyi yolu, aşağıda gösterildiği gibi bir virgül ifadesi kullanmaktır. Seçim ifadeleri için benzer kodu kullanın.

```
while ((count++, a) && (count++, b))
```

Ürettiğiniz istatistikleri analiz edin ve bu iki tür hakkında ne keşfettiğinize dair kısa bir rapor yazın (bir sayfadan az). Rastgele veriler için bir tane ve nerdeyse sıralı veriler için bir tane olmak üzere, Tablo 12-4'teki verileri dahil edin. Oranı bir ondalık basamağa kadar hesaplayın.

- Proje 26'yı tekrarlayın, yığın sıralaması ekleyin ve bilgisayarınızın iç saatini kullanın. Her sıralama algoritması için, sıralama çağrısından önceki son ifade olarak saati başlatın ve sıralamadan sonraki ilk ifade olarak saati okuyun. Metindeki önerilen algoritmalarla çalışma sürelerini karşılaştıran kısa bir rapor yazın. Sonuçlarınız algoritmalarla uyuşmuyorsa, daha iyi bir resim elde etmek için dizi boyutunu 1000 eleman artışlarıyla artırın.

Liste Boyutu	Kabuk	Hızlı	Oran (kabuk/hızlı)
Kıyaslar 100 500 1000 Hareketler 100 500 1000			

TABLO 12-4 Proje 26 için Sıralama İstatistikleri Formatı

28. Radix sıralama—aynı zamanda rakam, cep ve kova sıralama olarak da bilinir—kısaca anahtarlar için çok verimli bir sıralama yöntemidir. Aslında, yalnızca büyük k-O notasyonunu göz önünde bulundurursak, bu $O(n)$ ile, en iyilerden biridir. Radix sıralamaları, delikli kart döneminde elektronik muhasebe makinelerinde (EAM'ler) kartları sıralamak için yaygın olarak kullanıldı.

Bir radix sıralamasında, liste üzerinden her geçiş verileri bir basamağa sıralar. İlk geçiş verileri birler (en az anlamlı) basamağa göre sıralar. İkinci geçiş verileri onlar basamağına göre sıralar. Üçüncü geçiş verileri yüzler basamağına göre sıralar ve liste tamamen sıralanana kadar en anlamlı basamağa göre verileri sıralamaya devam eder.

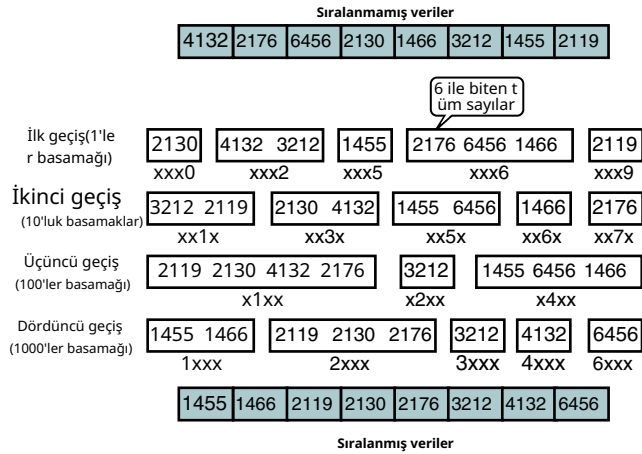
EAM sıralamalarında delikli kartlar ceplere ayrılmıştır. Günümüz sistemlerinde bunları bir bağlı listeye ayırırız, her basamak için ayrı bir bağlı liste ile. Bir cep veya kova yaklaşımını kullanarak, Şekil 12-26'daki sekiz dört basamaklı sayıyı sıralarız.

Bu sıralamayı analiz ederseniz, k sıralama geçişi olduğunu göreceksiniz, burada k anahtarın basamak sayısını temsil eder. Her sıralama geçişi için n işlemimiz var, burada n sıralanacak elemanların sayısını temsil eder. Bu bize kn verimliliği sağlar, bu da büyük-O notasyonunda $O(n^2)$ olarak ifade edilir.

1000 rastgele basamağı sıralamak için radix sıralamasını kullanan bir program yazın. Sıralamadan önce ve sonra verileri yazdırın. Her sıralama kovası bir bağlı liste olmalıdır. Sıralamanın sonunda veriler orijinal dizide olmalıdır.

29. Proje 27'yi radix sıralamasını içerecek şekilde değiştirin. Sıralama sürelerinin bir tablosunu oluşturun ve farklı sıralama sürelerinin sıralama verimliliği ile olan ilişkisini açıklayan kısa bir makale yazın.

12. Not: Verimlilik için, radix sıralamaları küçük listeler veya çok sayıda basamak içeren anahtarlar için önerilmez.



ŞEKİL 12-26Radix Sıralama Örneği Proje 27 için

30. Bölüm 12.1'in sonunda "Sıralamalar ve ADT'ler"de tartışılan teknikleri kullanarak, düz ekleme sıralaması (Program 12-3) kullanarak bir ADT oluşturun. Daha sonra, bir tamsayı dizisi ile bir kez ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.
31. Bölüm 12.1'in sonunda "Sıralamalar ve ADT'ler" de tartışılan teknikleri kullanarak, balon sıralaması (Program 12-5) kullanarak bir ADT oluşturun. Daha sonra, bir tamsayı dizisi ile bir kez ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.
32. Bölüm 12.1'in sonunda "Sıralamalar ve ADT'ler" de tartışılan teknikleri kullanarak, kabuk sıralaması (Program 12-4) kullanarak bir ADT oluşturun. Daha sonra, bir tamsayı dizisi ile bir kez ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.
33. Bölüm 12.1'in sonunda "Sıralamalar ve ADT'ler" de tartışılan teknikleri kullanarak, yığın sıralaması (Program 12-2) kullanarak bir ADT oluşturun. Daha sonra, bir tamsayı dizisi ile bir kez ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.
34. "Sıralamalar ve ADT'ler" başlıklı bölüm 12.1'in sonunda tartışılan teknikleri kullanarak, hızlı sıralama (Program 12-6) kullanarak bir ADT oluşturun. Ardından, bir tamsayı dizisi ve bir kayan nokta sayısı dizisi ile bir kez test etmek için bir test sürücüsü yazın.

Bölüm 13

Arama

Bilgisayar bilimlerinde en yaygın ve zaman alıcı işlemlerden biri arama, bir hedefin bir nesne listesi içindeki konumunu bulmak için kullanılan süreçtir. Bu bölümde birkaç arama algoritmasını inceliyoruz. Liste aramayı başlıyoruz ve iki temel arama algoritması hakkında tartışıyoruz: sıralı arama—üç ilginç varyasyon dahil—ve ikili arama.

Liste arama kavramlarını gözden geçirdikten sonra, verilerin listede konumunu hesaplamak için veri anahtarının algoritmik olarak manipüle edildiği hashed liste aramalarını tartışıyoruz. Neredeyse tüm hashed liste algoritmalarının ayrılmaz bir parçası çarpışma çözümlemesidir, bunu son bölümde tartışıyoruz.

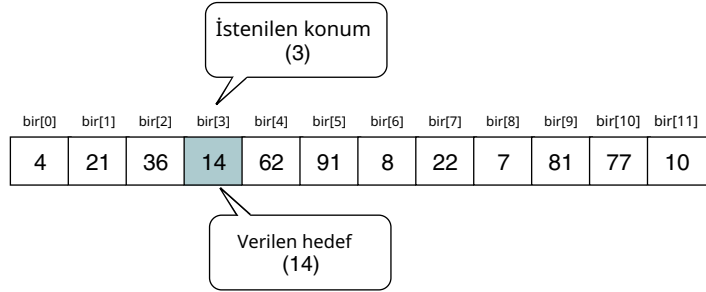
Bir dizi yapısı kullanarak liste arama algoritmalarını tartışsak da, aynı kavramlar bağlı liste aramalarında da bulunabilir. Sıralı arama, sıralı liste varyasyonu ile birlikte, bir listede veri bulmak için en yaygın olarak kullanılır (Bölüm 5). İkili arama ağacı (Bölüm 7) aslında bir ağaç yapısının ikili aramasının verimliliğini sağlamak için inşa edilmiş bir yapıdır. Bu aramalar kendi bölümlerinde ele alınmaktadır.

13.1 Liste Aramaları

Bir listeyi aramak için kullanılan algoritma, büyük ölçüde listenin yapısına bağlıdır. Bu bölümde dizilerle çalışan aramaları inceliyoruz. Diziler için iki temel arama türü vardır: sıralı arama ve ikili arama. Sıralı arama, herhangi bir dizide bir öğeyi bulmak için kullanılabilir. Öte yandan, ikili arama, sıralı bir liste gerektirir. Temel arama kavramı Şekil 13-1'de gösterilmektedir.

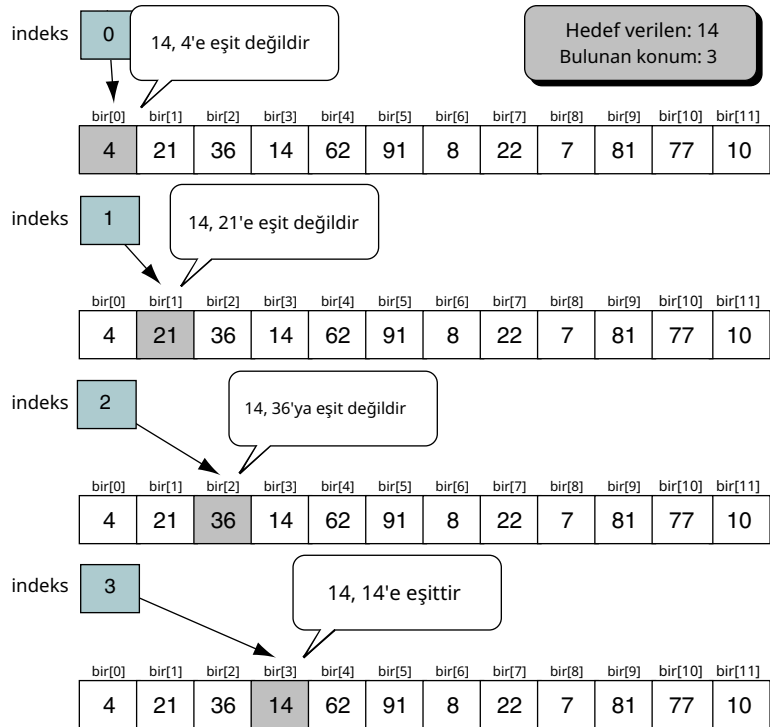
Sıralı Arama

Sıralı arama, liste sıralı olmadığında kullanılır. Genel olarak, bu tekniği yalnızca küçük listeler veya sık aranmayan listeler için kullanırsınız. Diğer durumlarda, önce listeyi sıralamalısınız ve ardından daha sonra tartışılacak olan ikili arama ile aramalısınız.



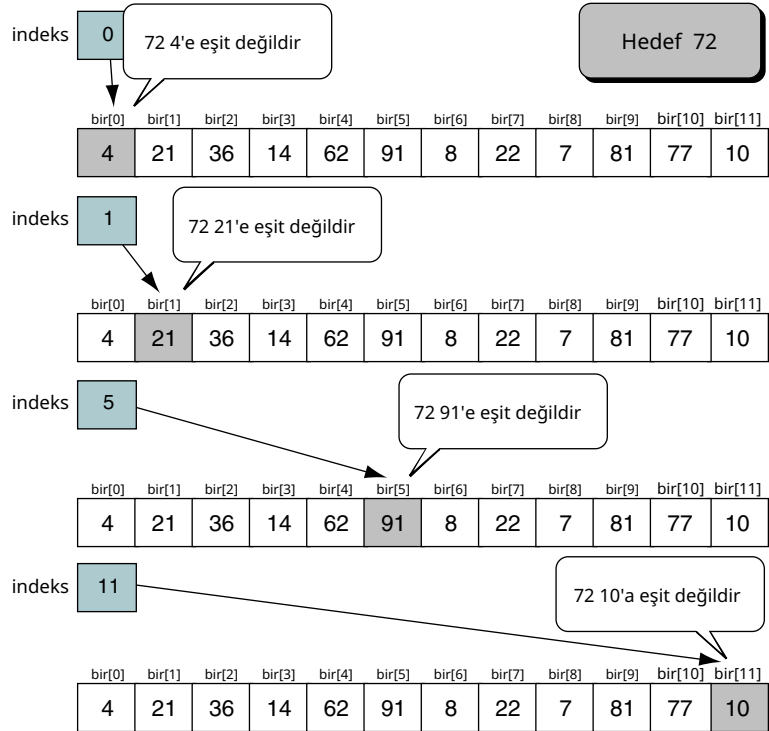
ŞEKİL 13-1Arama Kavramı

Sıralı aramada, hedefi listenin başında aramaya başlarız ve hedefi bulana kadar veya listenin içinde olmadığından emin olana kadar devam ederiz. Bu yaklaşıma bize iki olasılık sunar: ya buluruz ya da listenin sonuna ulaşırız. Şekil 13-2'de 14 değerini bulmak için adımları izliyoruz. Önce 0. indeksteki veriyi kontrol ediyoruz, sonra 1. ve ardından 2. indeksi kontrol ediyoruz ve 14'ü dördüncü öğede (indeks 3) buluyoruz.



ŞEKİL 13-2Başarılı Bir Sırasız Liste Araması

Ama hedef listedeyse ne olur? Bu durumda, listenin sonuna ulaşana kadar her bir öğeyi incelememiz gerekecek. Şekil 13-3, 72 hedefinin aramasını izlemektedir. Listenin sonunda, targetin mevcut olmadığını keşfe diyoruz.



Not: Tüm test noktaları gösterilmemiştir.

ŞEKİL 13-3 Sırasız Listede Başarısız Arama

Sıralı Arama Algoritması

Sıralı arama algoritması, çağıran algoritmaya iki şey söylemelidir: İlk olarak, aradığı veriyi buldu mu? Ve ikinci olarak, bulduysa, hedef veri ler hangi indekste bulunuyor? Bu soruları yanıtlamak için arama algoritması dört parametreye ihtiyaç duyar: (1) aradığımız liste, (2) listenin son öğesine giden bir indeks,¹ (3) hedef ve (4) bulunan öğenin indeks konumunun saklanacağı adres. Verilerin bulunup bulunmadığını çağıran algoritmaya söylemek için, bir Boolean döndürüyoruz—bulduysak true, bulamadıysak false.

Son öğeye giden indeksi geçmeden bir sıralı arama algoritması yazabilirsek de, eğer bunu yaparsak arama nasıl olduğunu bilmek zorunda kalır.

1. Son öğeye giden indeksin alternatifi olarak, listenin boyutu geçirilebilir.

Listede birçok öge var. Fonksiyonu mümkün olduğunca esnek hale getirmek için, dizideki son veri değerinin indeksini geçiriyoruz. Algoritmayı genelleştirerek son ögeye indeks geçmek de iyi bir yapılandırılmış tasarım tekniğidir. Bu bilgiyle artık Algoritma 13-1'i kodlamaya hazırız.

ALGORİTMA 13-1 Sıralı Arama

```

Algoritma seqSearch (liste, son, hedef, locn)
Hedefi düzensiz bir eleman listesinde bul.
    Ön   liste en az bir eleman içermelidir
        son, listedeki son elemanın indeksidir
        hedef, bulunması gereken veriyi içerir
        locn, çağrılan algoritmadaki indeksin adresidir
    Sonra bulunduğunda: indeks locn'da saklanır & bulundu true
        bulunmadığında: son locn'da saklanır & bulundu false
    Dönüş bulundu true veya false
1  looker'ı 0 olarak ayarla
2  döngü (looker < son VE hedef list[looker] ile eşit değil)
    1  looker'ı artır
3  döngüyü bitir
4  locn'ı looker olarak ayarla
5  eğer (hedef list[looker] ile eşitse)
    1  bulundu'yu true olarak ayarla
6  aksi takdirde
    1  bulundu'yu false olarak ayarla
7  eğer bitir
8  bulundu'yu döndür
bitir seqSearch

```

Algoritma 13-1 AnaliziSadece bir yorumumuz var. Arama için konum parametresini kullanabiliydik, bunun yerine arayıcı indeksini oluşturduk. Ancak, bir parametreyi çalışma değişkeni olarak kullanmak genellikle e iyi bir fikir değildir çünkü bu, algorithmada daha sonragerekli olabilecek herhangi bir ba şlangıç değerini yok eder.

Sıralı Aramalardaki Varyasyonlar

Sıralı arama algoritması üzerinde üç yararlı varyasyon vardır: (1) gözetmen arama, (2) olasılık araması ve (3) sıralı liste araması. Aşağıdaki bölümlerde her birine kısaca bakacağız.

Gözetmen Arama

Arama algoritmasını dikkatlice incerseniz, döngünün ikikoşulu test ettiğini göreceksiniz: listenin sonu ve hedefin bulunmaması. Knuth, "Bir programın iç döngüsü iki veya daha fazla koşulu test ettiğinde, testi sadece bir koşula indirmeye çalışmalıyız." der.² Eğer hedefin listede bulunacağını biliyorsak, listenin sonu için testi ortadan kaldırabiliriz. Bu

2. Donald E. Knuth, *Bilgisayar Programlamanın Sanatı*, cilt 3, *Sıralama ve Arama*, İkinci Baskı (Reading, MA: Addison-Wesley, 1998), 398.

bir hedefin gerçekten listede olduğunu garanti etmenin tek yolu, onu oraya koymaktırkendimiz. Bir hedef, dizinin sonuna ekstra bir eleman (gözcü girişi) ekleyereklisteye konur ve hedef gözcüye yerleştirilir. Daha sonra döngüyü optimize edebilirizve döngü tamamlandıktan sonra gerçek veriyi mi yoksa gözcüyü mü bulduğumuzu belirleyebiliriz.³ Açık dezavantaj, geri kalan işlemlerinlistenin sonundaki gözcü elemanına asla bakmamaya dikkat etmesi gerektiğidir. Gözcü araması için psödo kod, Algoritma 13-2'de gösterilmiştir.

ALGORİTMA 13-2Gözcü Arama

```

Algoritma GözcüArama (liste, son, hedef, konum)
Hedefi düzensiz bir öge listesinde bul.
    Ön liste, gözcü için sonunda öge içermelidir
        son, listedeki son veri ögesinin indeksidir
        hedef, bulunması gereken veriyi içerir
        konum, çağrılan algoritmadaki indeksin adresidir
    Son eğer bulunduysa--eşleşen indeks konumda saklanır & bulundu doğru olarak
        ayarlanır
    eğer bulunmadıysa--son konumda saklanır & bulundu yanlış
    Bulunan doğru veya yanlış olarak döner
1 hedefi liste[last + 1] olarak ayarla
2 looker'ı 0 olarak ayarla
3 döngü (hedef liste[looker] ile eşit değil)
    1 looker'ı artır
4 döngüyü bitir
5 eğer (looker <= last)
    1 bulundu'yu true olarak ayarla
    2 locn'i looker olarak ayarla
6 aksi takdirde
    1 bulundu'yu false olarak ayarla
    2 locn'i last olarak ayarla
7 eğer bitir
8 bulundu'yu döndür
son SentinelSearch

```

Olasılık Araması

Sıralı aramanın daha yararlı varyasyonlarından biri, olasılık araması olarak bilinir. Olasılık aramasında, dizideki veriler, en olası arama öğeleri dizinin başında ve en az olası olanlar ise sonunda olacak şekilde düzenlenir. Bu, çoğu arama için hedef olan nispeten az sayıda öge olduğunda özellikle faydalıdır. Olasılık sıralamasının zamanla doğru olmasını sağlamak için, her aramada bulunan öğeyi dizide hemen önündeki öge ile değiştiririz. Olasılık aramasının tipik bir uygulaması Algoritma 13-3'te gösterilmiştir.

3. Bir döngüyü yalnızca bir teste indirmek her zaman mümkün değildir. Mantiği tatmin etmek için iki veya daha fazla teste ihtiyaç duyan birçok döngü göreceğiz.

ALGORİTMA 13-3 Olasılık Arama

Algoritma OlasılıkArama (liste, son, hedef, konum) Her bir elemanın hedef olma olasılığına göre sıralanmış bir listede hedefi bul--en olası olan ilk , en az olası olan son.

Ön liste en az bir öge içermelidir
son, listedeki son ögenin indeksidir
hedef, bulunması gereken veriyi içerir
locn, çağırılan algoritmadaki indeksin adresidir
Son eğer bulunduysa--eşleşen indeks locn'de saklanır,
bulundu doğru ve öge öncelikte yukarı taşınır.
eğer bulunmadıysa--son konumda saklanır & bulundu yanlış
Bulunan doğru veya yanlış olarak döner

```

1 listede hedefi bul
2 eğer (hedef listede)
  1 bulundu'yu true olarak ayarla
  2 locn'yi hedefi içeren ögenin indeksine ayarla
  3 eğer (hedef ilk öğeden sonra)
    1 hedefi içeren öğeyi bir konum yukarı taşı
  4 eğer bitir
3 aksi takdirde
  1 bulundu'yu false olarak ayarla
4 eğer bitir
5 bulundu'yu döndür
son OlasılıkArama

```

Sıralı Liste Arama

Genel olarak, anahtar (hedef) üzerinde sıralı bir liste ararken ikili arama öneriyoruz; ancak, liste küçükse, ardışık bir arama kullanmak daha verimli olabilir. Bununla birlikte, sıralı bir listeyi ardışık olarak ararken, hedefin listede olmadığını belirlemek için listenin sonuna kadar aramak gereklidir. Test ettiğimiz mevcut öğeden daha küçük veya eşit olduğunda durabiliriz. Ayrıca, hedef son öğeden büyük olduğunda arama döngüsünü atlayarak sentinel kavramını da dahil edebiliriz. Diğer bir deyişle, hedef son öğeden küçük veya eşit olduğunda, son öge bir sentinel haline gelir ve listenin sonunu test etme gereğini ortadan kaldırır.

Dizi uygulamaları ile kullanılabilir de, sıralı liste araması daha yaygın olarak bağlı liste uygulamalarını ararken kullanılır.

Bir sıralı dizide arama yapmak için kullanılan pseudocode, Algoritma 13-4'te bulunmaktadır.

ALGORİTMA 13-4 Sıralı Liste Arama

Algoritma SıralıListeArama (liste, son, hedef, locn)
Hedefi, hedefe göre sıralanmış bir listede bul.
Ön liste en az bir öge içermelidir
son, listedeki son ögenin indeksidir
hedef, bulunması gereken veriyi içerir

devam ediyor

ALGORİTMA 13-4 Sıralı Liste Arama (devamı)

```

    locn, çağırılan algoritmadaki indeksin adresidirPost  eğer
    bulunduysa--eşleşen indeks locn'da saklanır-bulundu doğru  eğer bul
    unmadıysa--locn, hedeften büyük olan ilk elemanın indeksi
    dir veya locn son ile eşit ve bulundu yanlıştırBulundu
    doğru veya yanlış olarak döndür
1  eğer (hedef listenin son elemanından küçükse)
    1  hedeften küçük veya ona eşit olan ilk elemanı bul
    2  öge indeksini locn olarak ayarla
2  aksi takdirde
    1  locn'yi son olarak ayarla
3  eğer bitir
4  eğer (hedef listede)
    1  bulundu'yu true olarak ayarla
5  aksi takdirde
    1  bulundu'yu false olarak ayarla
6  eğer bitir
7  bulundu'yu döndür
son SıralıListeArama

```

İkili Arama

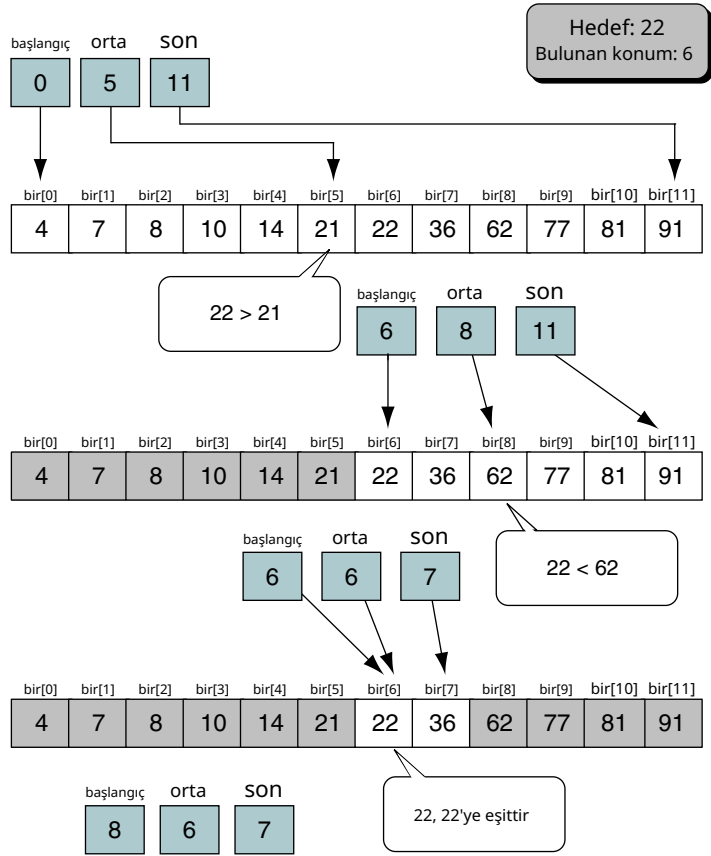
Sıralı arama algoritması çok yavaştır. Eğer 1000 elemanlı bir dizimiz v arsa, en kötü durumda 1000 karşılaştırma yapmamız gerekir. Dizi sıralı değilse, sıralı arama tek çözümdür. Ancak, dizi sıralıysa, ikili arama adı verilen daha verimli bir algoritma kullanabiliriz. Genel olarak, liste büyümeye başladığında ikili arama kullanmalıyız. *büyük* tanımının belirsiz olmasına rağmen, listenin 16'dan fazla eleman içerdiği durumda ikili aramaları düşünmenizi öneririz.

İkili arama, hedefin listenin ilk veya ikinci yarısında olup olmadığını belirlemek için dizinin ortasındaki elemandaki verileri test ederek başlar. Eğer ilk yarıda ise, ikinci yarıyı kontrol etmemize gerek yoktur. Eğer ikinci yarıda ise, ilk yarıyı test etmemize gerek yoktur. Diğer bir deyişle, sadece bir karşılaştırma ile listenin yarısını daha fazla değerlendirmeden eliyoruz. Bu süreci tekrarlarız, her testte kalan listenin yarısını ortadan kaldırarak, hedefi bulana veya listenin içinde olmadığını belirleyene kadar.

Listenin ortasını bulmak için üç değişkene ihtiyacımız var: birisi listenin başlangıcını, birisi listenin ortasını ve birisi listenin sonunu tanımlamak için. Burada iki durumu analiz ediyoruz: hedef listenin içinde ve hedef listenin içinde değil.

Hedef Bulundu

Şekil 13-4, sıralı bir dizide 22 hedefi için ikili aramayı izlemektedir.



ŞEKİL 13-4Başarılı İkili Arama Örneği

Üç indeksimizi tanımlayıcı olarak başlangıç, orta ve son olarak adlandırıyoruz. başlangıç 0 ve son 11 olarak verildiğinde, orta şu şekilde hesaplanabilir:

$$\text{mid} = \lfloor (\text{begin} + \text{end}) / 2 \rfloor = \lfloor (0 + 11) / 2 \rfloor = 5$$

İndeks konumu 5'te, hedefin listedeğerinden büyük olduğunu keşfediyoruz ($2 > 21$). Bu nedenle, 0'dan 5'e kadar olan dizi konumlarını eleme şansımız var. (orta otomatik olarak elenmiştir.) Aramamızı daraltmak için orta

+ 1'yi başlangıç olarak ayarlıyoruz ve aramayı tekrarlıyoruz.

Sonraki döngü, başlangıç için yeni değerle orta hesaplar (6) ve orta no ktanın artık 8 olduğunu belirler.

$$\text{orta} = \lfloor (6 + 11) / 2 \rfloor = \lfloor 17 / 2 \rfloor = 8$$

Hedefi orta değeri ile ikinci kez test ettiğimizde, hedefin liste değerinden daha küçük olduğunu keşfederiz ($22 < 62$). Bu sefer, liste sonunu son değerini orta - 1 olarak ayarlayarak ve orta'yi yeniden hesaplayarak ayarlıyoruz.

$$\text{orta} = \lfloor (6 + 7) / 2 \rfloor = \lfloor 13 / 2 \rfloor = 6$$

Bu adım, 8'den 11'e kadar olan elemanları dikkate almaktan etkili bir şekilde çıkarır. Artık hedefimizle eşleşen değere sahip olan indeks konumu 6'ya ulaştık. Algoritmadaki a ramayı durdurmak için (bkz. sayfa 607), başlangıç değerini son'dan büyük olacak şekilde zorlarız.

Hedef Bulunamadı

Daha ilginç bir durum, hedefin listede olmadığı durumdur. Arama algoritma mızı, tüm olası konumları kontrol ettiğimizde duracak şekilde inşa etmeliyiz. Bunu, başlangıç ve bitiş indekslerinin kesişip kesişmediğini test ederek ikili a ramada yaparız; yani, başlangıç bitişten büyük olduğunda işlemiz bitmiştir. Şi mdi, iki koşulun ikili arama algoritmasını sonlandırdığını görüyoruz: hedef bulunur veya bulunmaz. Bulunduğunda döngüyü sonlandırmak için , başlangıcı bitişten büyük olacak şekilde zorlarız. Hedef listede yoksa, başlangıç otomatik olarak bitişten büyük hale gelir.

Hedefin bulunmadığını bir örnekle gösterelim. Şekil 13-5'te, dizid e mevcut olmayan 11 hedefini arıyoruz.

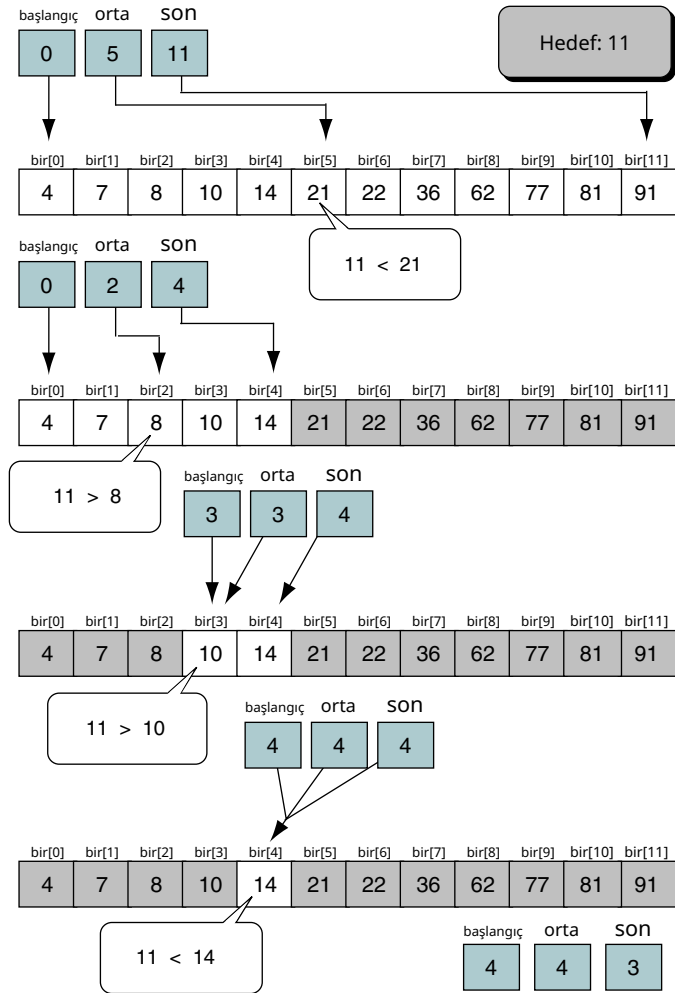
Bu örnekte döngü, başarılı aramada gördüğümüz gibi aralığı daraltmay a devam eder, ta ki 3 ve 4 indeks konumlarındaki verileri inceleyene kadar. Bu başlangıç ve bitiş ayarları, orta indeksini 3 olarak belirler.

$$\text{mid} = \lfloor (3 + 4) / 2 \rfloor = \lfloor 7 / 2 \rfloor = 3$$

3 numaralı indeks konumundaki test, hedefin liste değerinden büyük olduğunu gösteriyor, bu yüz den başlangıcı orta + 1, yani 4 olarak ayarlıyoruz. Şimdi 4 numaralı konumdaki verileri test ediyoruz ve $1 < 14$ olduğunu keşfediyoruz.

$$\text{mid} = \lfloor (4 + 4) / 2 \rfloor = \lfloor 8 / 2 \rfloor = 4$$

Bu noktada, hedefin iki bitişik değer arasında olması gerektiğini keşfettik; diğer bir deyişle, listede yer almıyor. Bunu algoritmik olarak görüyoruz çünkü end , mid - 1 olarak ayarlanmış, bu da begin 'nin end'den büyük olduğu anlamına geliyor, aradığımız ız değer listede olmadığına gösteren bir sinyal.



ŞEKİL 13-5 Başarısız İkili Arama Örneği

İkili Arama Algoritması

Algoritma 13-5, tanımladığımız ikili arama algoritmasının uygulanma sırası içermektedir. Sıralı arama için gördüğümüz aynı tasarım doğrultusunda inşa edilmiştir. İlk üç parametre, listeyi ve aradığımız hedefi tanımlar ve sonuncusu, bulduğumuz indeksin yerleştirileceği adresi içerir. Dikkate değer bir nokta: döngüyü bulunamadı durumu ile sonlandırdığımızda, dönen indeks öngörülemezdir - bu, düğümün hedefte ki değerden büyük veya küçük olduğunu gösterebilir.

ALGORİTMA 13-5 İkili Arama

```

Algoritma ikiliArama (liste, son, hedef, konum)
İkili Arama kullanarak sıralı bir listeyi arayın
    Ön liste sıralıdır; en az 1 değere sahip olmalıdır
        son, listedeki en büyük öğenin indeksidir
        hedef, aranan öğenin değeridir
        konum, çağrılan algoritmadaki indeksin adresidir
    Son BULUN
DU: konum, hedef öğeye atanmış indeksi bulundu, doğru olarak
    ayarlandı
        BULUNMADI: konum = hedefin altındaki veya üstündeki öğe bulundu, yanlış
            olarak ayarlandı
Doğru veya yanlış olarak bulundu döndür
1 başlangıcı 0 olarak ayarla
2 sonu son olarak ayarla
3 döngü (başlangıç <= son)
    1 orta değeri (başlangıç + son) / 2 olarak ayarla
    2 eğer (hedef > liste[mid])
        Üst yarıda ara
        1 başlangıcı (mid + 1) olarak ayarla
    3 aksi takdirde eğer (hedef < liste[mid])
        Alt yarıda ara
        1 sonu mid - 1 olarak ayarla
    4 aksi takdirde
        Bulundu: zorla çıkış
        1 başlangıcı (son + 1) olarak ayarla
    5 eğer bitir
4 döngüyü bitir
5 lokasyonu mid olarak ayarla
6 eğer (hedef liste[mid] ile eşitse)
    1 bulundu'yu true olarak ayarla
7 aksi takdirde
    1 bulundu'yu false olarak ayarla
8 eğer bitir
9 bulundu'yu döndür
bitir binarySearch
  
```

Arama Algoritmalarını Analiz Etme

Bahsedilen arama algoritmalarından hangisi en iyisidir? Bir uygulama genellikle hangi algoritmanın kullanılacağını belirler, ancak algoritmaları analiz ederek hangisinin en verimli olduğunu belirliyoruz.

Sıralı Arama

Ardışık arama için temel döngü aşağıda gösterilmiştir.

```

2 döngü (bakar < son VE hedef liste[bakar] ile eşit değilse)
    1 artış bakıcı
  
```

Bu, doğrusal bir algoritmanın klasik bir örneğidir. Aslında, bazı literatürde bu arama doğrusal arama olarak bilinir. Algoritma doğrusal olduğu için, etkinliği $O(n)$ dir.

Ardışık aramanın verimliliği $O(n)$ dir.

Gözcü arama için arama etkinliği, temel olarak sıralı arama ile aynıdır. Gözcü arama döngüde birkaç talimat tasarrufu sağlasa da, dizaynı aynıdır. Bu nedenle, bu da bir $O(n)$ aramasıdır. Benzer şekilde, sıralı liste aramasının etkinliği de $O(n)$ dir. Bir aramanın başarılı olma olasılığını biliyorsak, sıralı bir listeyi aramak için daha doğru bir formül oluşturabiliriz. Ancak, bu geliştirilmiş doğruluk, formülde bir katsayı olarak ortaya çıkar ki, hatırlarsanız, büyük-O analizinde düşürülür.

Her bir elemanın listede olasılığını bilmeden olasılık aramasının etkinliğini genelleştirmek mümkün değildir. Öte yandan, eğer bir listenin ilk birkaç elemanının olasılığı %90'dan fazlaysa, bu, listeyi sürdürmek için gereken ek yükü dikkate alsak bile, çok etkili bir arama olabilir. Ancak genel olarak, büyük listeler için ikili aramayı öneriyoruz.

İkili Arama

İkili arama, listeyi sürekli olarak ikiye bölerek bir öğeyi bulur. Döngüsü:

```

3  döngü (başlangıç <= son)
  1  mid'i (başlangıç + bitiş) / 22  eğer (hedef > liste[mid])
    1  başlangıcı (mid + 1) olarak ayarla
  3  aksi takdirde eğer (hedef < liste[mid])
    1  bitiş mid - 1 olarak ayarla
  4  aksi takdirde
    1  başlangıcı (bitiş + 1) olarak ayarla
  5  eğer bitir
  6  döngüyü bitir

```

Bu döngü açıkça bölmektedir ve bu nedenle logaritmik bir döngüdür. Verimlilik böylece $O(\log n)$ olup, bu da tüm ölçümler arasında en verimli olanlardan biri olarak tanınmalısınız.

İkili aramanın verimliliği $O(\log n)$ dir.

İki aramayı karşılaştırdığımızda, listeyi sıralamak için gereken süreyi göz ardı ederek, ikili aramanın herhangi bir önemli boyuttaki listeyi aramak için açıkça daha verimli olduğunu görüyoruz (bkz. Tablo 13-1). Bu nedenle, ikili arama, en küçük listeler (yani, 16'dan az öğe içeren listeler) hariç tüm listeler için önerilmektedir.

Liste boyutu	İterasyonlar	
	İkili	Sıralı
16	4	16
50	6	50
256	8	256
1000	10	1000
10,000	14	10,000
100,000	17	100,000
1,000,000	20	1,000,000

TABLO 13-1 İkili ve Sıralı Aramaların Karşılaştırması

13.2 Arama Uygulamaları

Bu bölümde sıralı ve ikili aramalar için C kodunu geliştiriyoruz.

Sıralarda gördüğümüz gibi, herhangi bir türdeki data için genel bir arama algoritması yazabiliriz. Arama algoritmalarına odaklanmamızı sağlamak için, yalnızca tam sayı verileri kullananarak geliştiriyoruz.

Sıralı Arama C'de

Program 13-1'deki sıralı arama, Algoritma 13-1'deki sahte kod uygulamasıyla paraleldir.

PROGRAM 13-1 Sıralı Arama

123456789	<pre> /* Hedefi boyut elemanlarından oluşan düzensiz bir listede bul. Önce liste en az bir öge içermelidir son, listedeki son elemanın indeksidir hedef, bulunması gereken veriyi içerir locn, çağıran işlevdeki indeksin adresidir Sonra BULUNDU: eşleşen indeks locn adresinde saklanır true (bulundu) döndür BULUNAMADI: son locn adresinde saklanır false (bulunamadı) döndür */ bool seqSearch (int liste[], int son, int hedef, int* locn) { // Yerel Tanımlar int looker; </pre>
101112131415	

devam ediyor

PROGRAM 13-1 Sıralı Arama (devamı)

```

161718192021222324
// İfadeler
    bakan = 0;
    while (looker < son && hedef != liste[looker])
        viewer++;

    *locn = bakan;
    return ( hedef == liste[arayan] );
} // seqArama

```

Program 13-1 Analizi Bu program basit olmasına rağmen, bazı tartışmaları hak ediyor. Öncelikle, neden bir `while` ifadesi kullandık, `for` döngüsü yerine? Dizinin sınırlarını bilmemize rağmen, bu hala bir olay kontrollü döngüdür. Aradığımızı bulana kadar veyalistenin sonuna ulaşana kadar arama yapıyoruz. Bir şeyi bulmak bir olaydır, bu yüzden bir olay döngüsü kullanıyoruz.

Sonra, döngünün limit ifadesinde iki test olduğunu unutmayın. Dizinin sonu iç in testi önce kodladık. Bu durumda, hangi testin önce olduğu yürütme açısından fark etmez, ancak diğer arama döngülerinde edebilir. Bu nedenle, limit testini önce kodlama alışkanlığını geliştirmelisiniz çünkü bu birindeksi değer kullanmaz ve bu nedenle daha güvenlidir.

Adresle çağırma kullanımı `locn` tartışmayı da hak ediyor. Bulunan konumu çağıran programdaki değişkene geri iletmemiz gerektiğinden, aramaya adresini iletmemiz gerekiyor.

Bu fonksiyonun ne kadar özlü olduğunu fark edin. Aslında, kod satırlarından daha fazla belge satırı var. Tüm arama bir `while` ifadesinde yer alıyor. Bu kısa kod ile “Fonksiyonu neden yazalım? Neden sadece gerekli olduğu yere bir satır kod koymuyoruz?” diye sormak isteyebilirsiniz. Cevap, her fonksiyonun yalnızca bir şey yapması gerektiği ve yeniden kullanılabilirlik kavramında yatıyor. Arama sürecini kendi fonksiyonunda izole ederek, aramaya ihtiyaç duyan süreçten ayırıyoruz. Bu, yapılandırılmış programlamaya daha iyi bir yaklaşım ve ayrıca kodu programın diğer bölümlerinde yeniden kullanılabilir ve arama gerektiren diğer programlara taşınabilir hale getiriyor.

C'de İkili Arama

İkili arama algoritmasının C uygulaması Program 13-2'de gösterilmiştir.

PROGRAM 13-2 İkili Arama

```

123456789
/* Sıralı bir listeyi İkili Arama kullanarak arayın.
   Önceki liste en az bir eleman içermelidir
       son, listedeki en büyük elemanın indeksidir
       hedef, aranan elemanın değeridir
       locn, çağıran işlevdeki indeksin adresidir
   Sonuç BULUNDU: locn = hedef elemana indeks
                   true (bulundu) döndür
   BULUNMADI: locn = hedefin altındaki/yukarıdaki indeks
                   false (bulunamadı) döndür

```

devam ediyor

PROGRAM 13-2İkili Arama (devamı)

```

10111213141516171819202122232425262728293031323334353637
*/
bool ikiliArama (int liste[ ], int son,          int hedef, int* ko
                                num )
{
    // Yerel Tanımlar
    int baslangic;
    int orta;
    int son;

    // İfadeler
    baslangic = 0;
    son = son;while (baslangic
    <= son)
    {
        orta = ( baslangic + son ) / 2;
        if ( hedef > liste[ orta ] )
            // üst yarıda ara
            baslangic = orta + 1;
        else if ( hedef < liste[ orta ] )
            // alt yarıda ara
            son = orta - 1;

        aksi takdirde
            // bulundu: zorla çıkış
            begin = end + 1;
    } // döngü sonu
    *locn = orta;
    return (hedef == liste [orta]);
} // ikiliArama

```

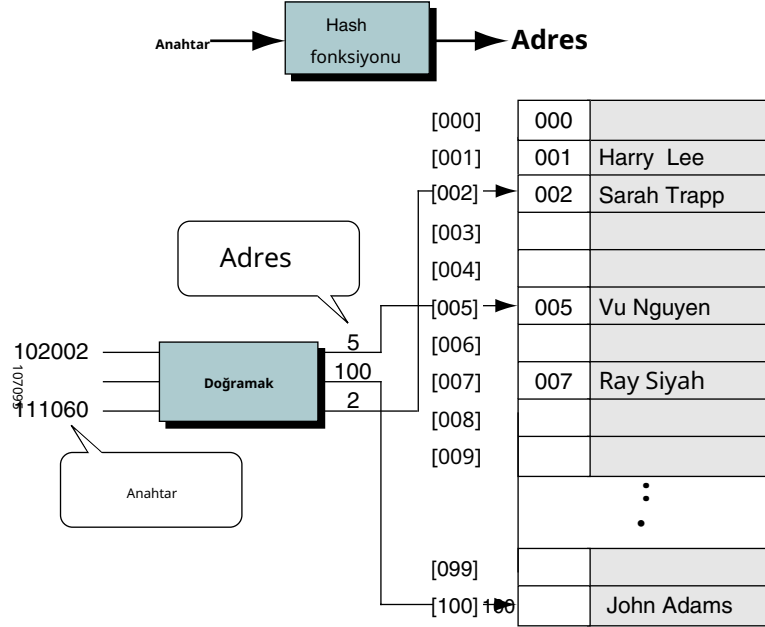
13.3 Hashlenmiş Liste Aramaları

Bölüm 13.1, “Liste Aramaları”da tartışılan arama teknikleri, verileri bulmadan önce birkaç test gerektirir. İdeal bir aramada, verilerin tam olarak nerede olduğunu bilirdik ve doğrudan oraya giderdik. Bu, hashlenmiş bir aramanın amacıdır: verileri yalnızca bir test ile bulmak. Tartışmamız için bir veri dizisi kullanıyoruz. Genel kavram, disk üzerinde saklanan dosyalar gibi başka yapılara kolayca genişletilebilir. Ancak, bu yapılar bu metnin kapsamının ötesindedir ve tartışmaları diğer kitaplara bırakılmıştır.

Temel Kavramlar

Bir hashlenmiş aramada, anahtar, algoritmik bir işlev aracılığıyla, dataların konumunu belirler. Bir diziyi aradığımız için, anahtarı bulmamız gereken verileri içeren indekse dönüştürmek için bir hashleme algoritması kullanıyoruz. Hashlemeyi tanımlamanın bir diğer yolu, anahtarların bir listede adreslere eşlendiği bir anahtar-adres dönüşümü olarak tanımlamaktır. Bu eşleme dönüşümü

Şekil 13-6'da gösterilmiştir. Şeklin üst kısmında hashing kavramının genel bir temsili bulunmaktadır. Şeklin geri kalanı, üç anahtarın listede üç farklı adrese nasıl hash'lenebileceğini göstermektedir.



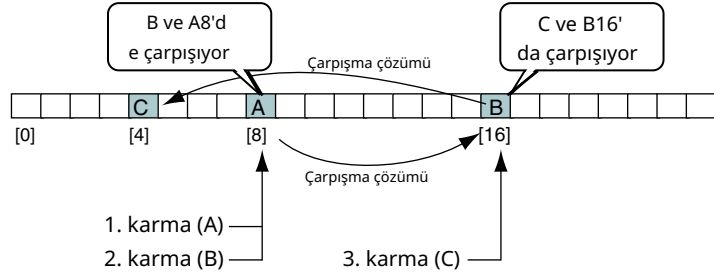
ŞEKİL 13-6 Hash Kavramı

Genel olarak, hash'li bir liste için anahtarların sayısı, verilerin depolama alanından daha fazladır. Örneğin, öğrencilerin Sosyal Güvenlik numaralarının son dört hanesi ile tanımlandığı bir sınıf için 50 öğrenciden oluşan bir diziye sahipsek, dizideki her eleman için 200 olası anahtar vardır ($10,000 / 50$). Dizideki her indeks konumu için birçok anahtar bulunduğundan, birden fazla öğrenci dizide aynı konuma hash'lenebilir. Aynı konuma hash'lenen anahtarlar kümesine eşanlımlılar diyoruz.

Hashing, anahtar ile adres arasında bir eşleme sürecidir.

Eğer listemize eklediğimiz gerçek veriler iki veya daha fazla eşanlımlı içeriyor sa, çakışmalar yaşayabiliriz. Çakışma, bir hashing algoritması bir ekleme anahtarı için bir adres ürettiğinde ve o adresin zaten dolu olduğu durumlarda meydana gelir. Hashing algoritması tarafından üretilen adres ev adresi olarak bilinir. Tüm ev adreslerini içeren bellek ana alan olarak bilinir.

İki anahtar bir ev adresinde çakıştığında, çakışmayı çözmek için anahtarlardan birini ve verilerini başka bir yere yerleştirmemiz gerekir. Çakışma çözümleme kavramı Şekil 13-7'de gösterilmiştir.



ŞEKİL 13-7 Çakışma Çözümleme Kavramı

Şekil 13-7'de anahtar A hash'lenir ve listede 8. konuma yerleştirilir. Daha sonra bir zamanda, Anahtarını hash'leriz, bu da Anın eşanlamıdır. Eşanlamı oldukları için, birbirine aynı ev adresine hash'lenirler ve bir çakışma meydana gelir. Çakışmayı, anahtar B 'yi 16. konuma yerleştirerek çözüyoruz. Anahtar C'yi hash'lediğimizde,evin adresi 16'dır. B ve C eşanlamı olmasa da, daha önceki çakışmayı çözdüğümüzde anahtar B 'yi oraya yerleştirdiğimiz için 16. konumda çakışıyorlar. Bu nedenle, anahtar C için başka bir konum bulmalıyız, bu konumu 4. konuma yerleştiriyoruz.

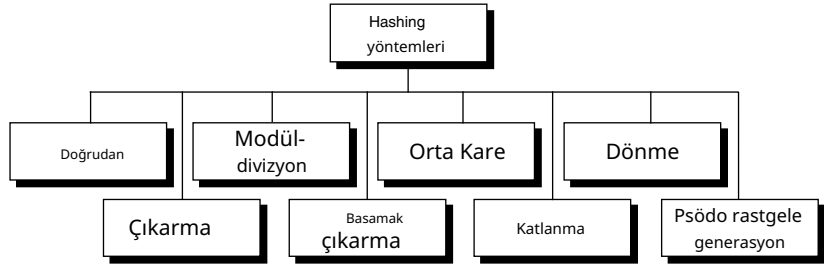
A hashedliste bir öğeyi bulmamız gerektiğinde, onu listeye eklemek için kullandığımız aynı algoritmayı kullanmamız gerektiği açık olmalıdır. Sonuç olarak, önce anahtarı hash'leriz ve ev adresini kontrol ederiz istenen öğeyi iç erip içermediğini belirlemek için. Eğer içeriyorsa, arama tamamlanmıştır. Eğer içer miyorsa, çarpışma çözümleme algoritmasını kullanarak bir sonraki konumu belirle memiz ve öğeyi bulana kadar devam etmemiz gerekir veyalistenin içinde olmadığı nı belirlememiz gerekir. Her adres hesaplama ve başarı test bir prob olarak bilinir.

Hashing Yöntemleri

Şimdi birkaç hashing yöntemini incelemeye hazırız. Farklı yöntemlere b aktıktan sonra, birkaçını içeren basit bir hashing algoritması oluşturaca gız. İncelediğimiz hashing teknikleri Şekil 13-8'de gösterilmektedir.

Doğrudan Yöntem

Doğrudan hashing yönteminde anahtar, herhangi bir algoritmik manipülasyon olmadan adrestir. Bu nedenle veri yapısı, her olası anahtar için bir öge içermelidir. Doğrudan hashing kullanabileceğiniz durumlar sınırlıdır, ancak eşanlamlıların olmamasını ve dolayısıyla çarpışmaların olmamasını garanti ettiği için çok güçlü olabilir. İki uygulamaya bakalım.



ŞEKİL 13-8 Temel Hashing Teknikleri

Öncelikle, ayın günlerine göre aylık satışları toplamamız gereken durumu düşünelim. Her satış için tarih ve satış miktarına sahibiz. Bu durumda, 31 akümülatörden oluşan bir dizi oluşturuyoruz. Ayın satış kayıtlarını okurken, ayın gününü dizi için anahtar olarak kullanıyor ve satış miktarını ilgili akümülatöre ekliyoruz. Akümülasyon kodu aşağıdaki örnekte gösterilmiştir.⁴

```
günlükSatış[indirim.gün] = günlükSatış[indirim.gün]
+ satış.tutarı;
```

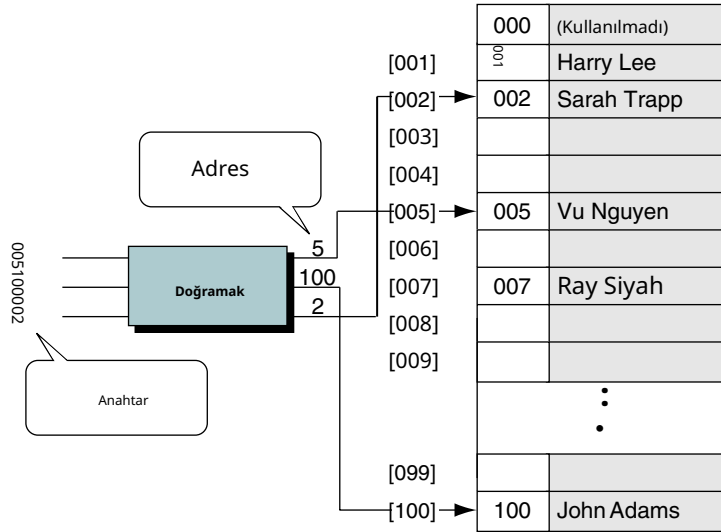
Şimdi daha karmaşık bir örneği düşünelim. Küçük bir organizasyonun 100'den az çalışanı olduğunu hayal edin. Her çalışana 1 ile 100 arasında bir çalışan numarası atanmıştır. Bu durumda, 101 çalışan kaydından oluşan bir dizi oluştursak (0 konumu kullanılmaz), çalışan numarası herhangi bir bireysel kaydın adresi olarak doğrudan kullanılabilir. Bu kavram Şekil 13-9'da gösterilmektedir.

Şekil 13-9'u incelerken, dizideki her öğenin bir çalışanın kaydını içermediğini unutmayın. Her öğe günlük satış örneğimizde kullanılmış olsa da, genellikle karma listelerde bazı boş öğeler vardır. Aslında, daha sonra göreceğimiz gibi, doğrudan hashing dışında tüm hashing teknikleri, çakışma sayısını azaltmak için bazı öğelerin boş olmasını gerektirir.

Gördüğümüz gibi, bu ideal yöntem olmasına rağmen, uygulanması çok sınırlıdır. Örneğin, bu yöntemi kullanarak Anahtar olarak Sosyal Güvenlik numarasını kullanamayız çünkü Sosyal Güvenlik numaraları dokuz basamaktır. Diğer bir deyişle, Sosyal Güvenlik numarasını anahtar olarak kullanırsak, 1.000.000.000 giriş kadar büyük bir diziye ihtiyacımız var, ancak bunlardan 100'den azını kullanırız.

Şimdi, büyük bir olası anahtar popülasyonunu küçük bir adres alanına eşleyen hashing tekniklerine dikkat ediyoruz.

4. C dizileri 0 konumundan başladığı için dizimizde 32 eleman olmalıdır. Eleman 0 kullanılmaz.



ŞEKİL 13-9 Çalışan Numaralarının Doğrudan Hashing'i

Çıkarma Yöntemi

Bazen anahtarlar ardışık olabilir ancak 1'den başlamaz. Örneğin, bir şirketin yalnızca 100 çalışanı olabilir, ancak çalışan numaraları 1001'den başlayıp 1100'e kadar gidebilir. Bu durumda, anahtardan 1000 çıkararak adresi belirleyen basit bir hashing fonksiyonu olan çıkarma hashing'i kullanırız.

Bu örneğin güzelliği, basit olması ve çakışma olmayacağını garanti etmesidir. Sınırlamaları, doğrudan hashleme ile aynıdır: yalnızca anahtarların yoğun bir şekilde doldurulmuş bir listeye eşlendiği küçük listeler için kullanılabilir.

Doğrudan ve çıkarma hash fonksiyonları, çakışma olmadan bir arama çabası garanti eder. Bunlar birebir hashleme yöntemleridir: yalnızca bir anahtar her adrese hashlenir.

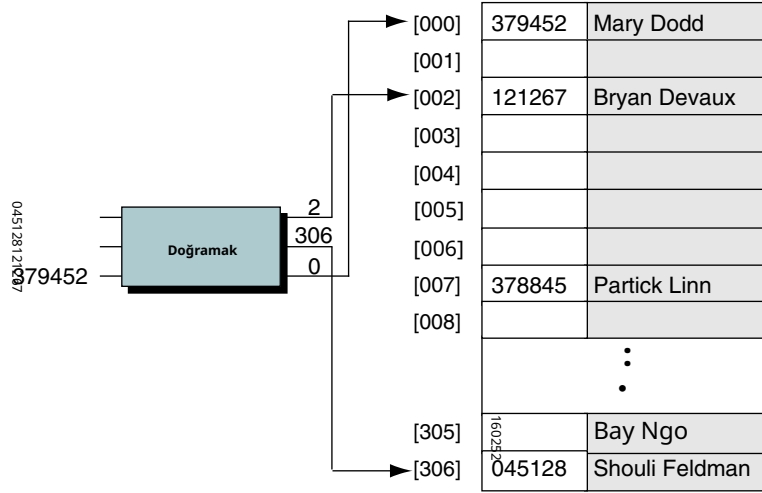
Modül-bölme Yöntemi

Ayrıca bölüm kalanı olarak bilinen modül-bölme yöntemi, anahtar dizisi boyutuna böler ve adres için kalanı kullanır. Bu yöntem, listSize dizideki eleman sayısını temsil eden basit hashleme algoritmasını aşağıda gösterir:

$$\text{adres} = \text{anahtar} \text{ MODÜL } \text{listSize}$$

Bu algoritma herhangi bir liste boyutuyla çalışır, ancak asal bir sayı olan liste boyutu, diğer liste boyutlarına göre daha az çakışma üretir. Bu nedenle, mümkün olduğunca dizinin boyutunu asal bir sayı yapmaya çalışmalıyız.

Küçük şirketimiz büyümeye başladıkça, yakında 100'den fazla çalışanımız olacağını fark ediyoruz. Geleceği planlayarak, 1.000.000'a kadar çalışan numaralarını işleyebilen yeni bir çalışan numaralandırma sistemi oluşturuyoruz. Ayrıca, 300 çalışana kadar veri alanı sağlamak istediğimize karar veriyoruz. 300'den büyük ilk asal sayı 307'dir. Bu nedenle, 0'dan 306'ya kadar adres aralığına sahip bir liste (dizi) boyutu olarak 307'yi seçiyoruz. Yeni çalışan listemiz ve bazı hashlenmiş adresleri Şekil 13-10'da gösterilmiştir.



ŞEKİL 13-10Modül-bölme Hashing

Demonstrasyon yapmak için, Bryan Devaux'un çalışan numarasını, 121267'yi hashleyelim.

$$121267/307 = 395 \text{ kalan } 2$$

$$\text{Dolayısıyla: hash}(121267) = 2$$

Rakam-çıkarma Yöntemi

Rakam çıkarma seçilen rakamlar anahtardan çıkarılır veadres olarak kullanılır. Örneğin, altı haneli çalışan numaramızı üç haneli bir adrese (000–999) hashlemek için, ilk, üçüncü ve dördüncü rakamları (soldan) seçebiliriz ve bunları adres olarak kullanabiliriz. Şekil 13-10'daki anahtarları kullanarak, bunları aşağıda gösterilen adreslere hashleyebiliriz:

379452	⇒	394
121267	⇒	112
378845	⇒	388
160252	⇒	102
045128	⇒	051

Midsquare Yöntemi

Midsquare hashing'de anahtar kare alınır ve adres, kare alınmış sayının ortasında n seçilir. Bu yöntemin en belirgin sınırlaması anahtarın boyutudur. Altı basamaklı bir anahtar verildiğinde, ürün 12 basamaklı olacaktır ki bu da birçok bilgisayarın maksimum tam sayı boyutunun ötesindedir. Çoğu kişisel bilgisayar dokuz basamaklı bir tam sayıyı işleyebildiğinden, kavramı dört basamaklı anahtarlarla gösterelim. 9452 anahtarı verildiğinde, dört basamaklı bir adres (0000-9999) kullanılarak midsquare adres hesaplaması aşağıda gösterilmiştir.

9452² 89340304: adres 3403

Orta kare yönteminin bir varyasyonu olarak, anahtarın bir kısmını, örneğin ortadaki üç rakamı seçebiliriz ve ardından tüm anahtar yerine bunları kullanabiliriz. Bunu yapmak, anahtarın kare alma işlemi için çok büyük olduğu durumlarda yöntemin kullanılmasını sağlar. Örneğin, Şekil 13-10'daki anahtarlar için ilk üç rakamı seçebiliriz ve ardından aşağıda gösterildiği gibi orta kare yöntemini kullanabiliriz. (Üçüncü, dördüncü ve beşinci rakamı adres olarak seçiyoruz.)

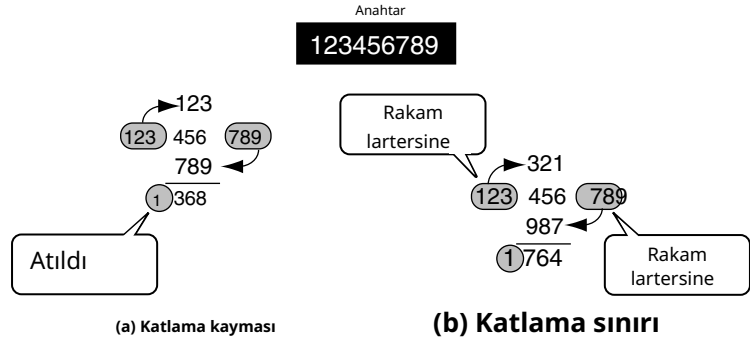
379452: 379² = 143641 ➡ 364
 121267: 121² = 014641 ➡ 464
 378845: 378² = 142884 ➡ 288
 160252: 160² = 025600 ➡ 560
 045128: 045² = 002025 ➡ 202

Orta kare yönteminde aynı rakamların üründen seçilmesi gerektiğini unutmayın. Bu nedenle, ürünün tam altı rakam olması için yeterli baştaki sıfırlara sahip olduğunu kabul ediyoruz.

Katlanma Yöntemleri

İki katlanma yöntemi kullanılır: katlama kaydırma ve katlama sınırı. Katlama kaydırmada anahtar değeri, gerekli adresin boyutuyla eşleşen parçalara bölünür. Ardından, sol ve sağ parçalar kaydırılır ve orta parçayla eklenir. Örneğin, Sosyal Güvenlik numaralarını üç haneli adreslere eşlemek istediğimizi hayal edin. Dokuz haneli Sosyal Güvenlik numarasını üç haneli sayıya böleriz ve bunlar toplanır. Elde edilen toplam 999'dan büyükse, baştaki rakamı atarız. Bu yöntem Şekil 13-11(a)'de gösterilmiştir.

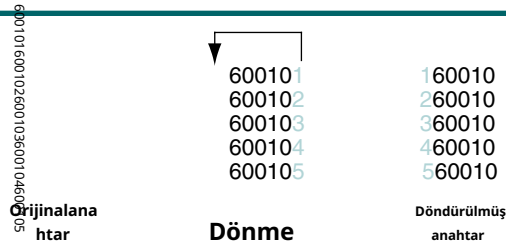
Katlama sınırında, sol ve sağ sayılar, aralarındaki sabit bir sınırdan ve merkez sayının üzerinde katlanır. Böylece iki dış değer tersine çevrilir; bu, Şekil 13-11(b)'de gösterildiği gibi, 123'ün 321'e ve 789'ın 987'ye katlandığıdır. İki katlanma yönteminin farklı hashlenmiş adresler verdiğini belirtmek ilginçtir.



ŞEKİL 13-11 Hash Fold Örnekleri

Dönme Yöntemi

Dönme hashing genellikle tek başına kullanılmaz, daha ziyade diğer hashing yöntemleriyle birleştirilir. Anahtarların sıralı olarak atıldığı durumlarda en faydalıdır, bu durumları genellikle çalışan numaraları ve parça numaralarında görürüz. Basit bir hashing algoritması, hashing anahtarları son karakter dışında aynı olduğunda eş anlamlılar yaratma eğilimindedir. Son karakterin anahtarın önüne döndürülmesi bu etkiyi en aza indirir. Örneğin, büyük bir şirkette kullanılabilecek altı haneli bir çalışan numarasını düşünelim (bkz. Şekil 13-12).



ŞEKİL 13-12 Dönme Hashing

Dönmüş anahtarları dikkatlice inceleyin. Tüm anahtarlar artık 60010 ile bittiği için, modül bölme ile iyi çalışmayacakları açıktır. Öte yandan, orijinal anahtar ve iki haneli bir adres üzerinde basit bir fold shift hash kullansaydık, adresler 62 ile başlayarak sıralı olurdu. Dönmüş anahtar üzerinde bir shift hash kullanmak, 26, 36, 46, 56, 66 adres serisini oluşturur; bu da verilerin adres alanında daha eşit bir şekilde dağılmasını sağlamak için istenen etkiye sahiptir. Dönme genellikle folding ve sahte rastgele hashing ile birleştirilerek kullanılır.

Psödo-rastgele Hashleme

Psödo-rastgele hashlemede anahtar, bir psödo-rastgele sayı üretici için tohum olarak kullanılır ve elde edilen rastgele sayı, modül bölme kullanılarak mümkün olan adres aralığına ölçeklendirilir (bu bölümde daha önce “Modül bölme yöntemi”ne bakın). Psödo-rastgele sayıları rastgele sayılarla karıştırmayın. Belirli bir tohum verildiğinde, psödo-rastgele sayı üreticileri her zaman aynı sayı serisini üretir. Bu, onları hashlemede kullanmamıza olanak tanır.

Aşağıda yaygın bir rastgele sayı üretici gösterilmektedir.

$$y = ax + c$$

Pseudorastgele sayı üreticini bir hashing yöntemi olarak kullanmak için, x 'i anahtar olarak ayarlıyoruz, a katsayısı ile çarpıyoruz ve ardından c sabitini ekliyoruz. Sonuç, liste boyutuna bölünüyor ve kalan, hashlenmiş adres oluyor. Maksimum verimlilik için, a ve c faktörleri asal sayılar olmalıdır. Bu kavramı Şekil 13-10'dan bir örnekle gösterelim. Hesaplamayı makul tutmak için, a ve c faktörleri için sırasıyla 17 ve 7 kullanıyoruz. Ayrıca, örnekteki liste boyutu asal sayı 307'dir.

$$\begin{aligned} y &= ((17 * 121267) + 7) \bmod 307 \\ y &= (2061539 + 7) \bmod 307 \\ y &= 2061546 \bmod 307 \\ y &= 41 \end{aligned}$$

Bu pseudorastgele sayı üreticini çarpışma çözümlemesini tartışırken tekrar göreceğiz.

Tüm hash fonksiyonları, doğrudan hashing ve çıkarma hashing'i hariç, çoktan bire işlevlerdir: birçok anahtar bir adrese hash'lenir.

Bir Hashing Algoritması

Hashing yöntemleri konusundaki tartışmamızı sonlandırmadan önce, tam bir hashing algoritmasını tanımlamamız gerekiyor. Bir hashing yöntemi, bir anahtar bir dizideki bir adrese hash'lerken iyi çalışabilir, ancak büyük dosyalara hash'leme genellikle daha karmaşıktır. Genellikle, algoritmanın ürettiği eş anlamlıların sayısını ve çarpışma serisinin uzunluğunu belirlemek için hash'lenecek anahtarların popülasyonunun kapsamlı bir analizini gerektirir. Bu tür bir analizin incelenmesi, bu metnin kapsamının dışındadır.

Bu bölümde, büyük bir dosya için kullanılabilecek algoritmayı sunuyoruz. Bu, endüstriyel bir veritabanı sisteminde anahtarları hash'lemek için kullanılan bir algoritmanın basitleştirilmiş halidir. İncelediğinizde, üç farklı hashing yöntemi kullandığını not edin: katlama kaydırma, döndürme ve modül-bölme.

30 bayta kadar alfanümerik bir anahtarımız olduğunu varsayalım ve bunu 32 bitlik bir adrese hash'lememiz gerekiyor. İlk adım, alfanümerik anahtar bir sayıya dönüştürmektir; her karakterin Amerikan Standart Kodlama Sistemi (ASCII) değerini bir toplayıcıya ekleyerek.

bu adres olacaktır. Her karakter eklendikçe, değerlerin dağılımını maksimize etmek için adresin bitlerini döndürüyoruz. Anahtar içindeki karakterler tamamen hash'lenince, adresin mutlak değerini alıyoruz ve ardından dosya için adres aralığına haritalıyoruz. Bu mantık Algoritma 13-6'da gösterilmiştir.

ALGORİTMA 13-6 Hashleme Algoritması

Algoritma hash (anahtar, boyut, maxAddr, addr) Bu algoritma, boyutu karakter olan alfanümerik bir anahtarı tam sayısal bir adrese dönüştürür.

Ön anahtar hash'lenmesi gereken bir anahtardır

boyut anahtarın karakter sayısıdır

maxAddr liste için maksimum olası adrestir

Son addr hash'lenmiş adresi içerir

1 looper'ı 0 olarak ayarla

2 addr'ı 0 olarak ayarla

Anahtarı hashle

3 anahtardaki her karakter için

1 eğer (karakter boşluk değilse)

1 karakteri adrese ekle

2 addr'ı 12 bit sağa döndür

2 eğer bitir

4 döngüyü bitir

Negatif adres için test

5 eğer (addr < 0)

1 addr = mutlak(addr)

6 eğer bitir

7 addr = addr mod maxAddr

son hash

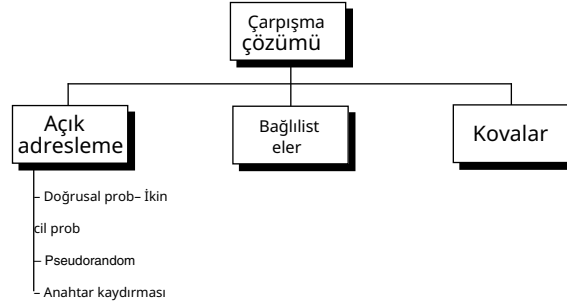
Algoritma 13-6 Analizi Bu algorithmada tartışmaya değer iki nokta var. İlk olarak, 3.1.2 numaralı ifadede döndürme, genellikle bir montaj dili talimatıyla gerçekleştirilebilir. Algoritma yüksek seviyeli bir dilde yazıldığında, döndürme, bir dizi bit düzeyinde işlemle gerçekleştirilir. Bizim amacımız için, adresin sonundaki 12 bitin adresin başındaki 12 bit olacak şekilde kaydırılması yeterlidir ve baştaki bitler sağdaki bit konumlarının aplayacak şekilde kaydırılır.

İkincisi, bu algoritma aslında daha önce tartışılan üç hashleme yöntemini kullanır. Bir eysel karakterleri adrese eklediğimizde katlama kaydırma kullanıyoruz. Her eklemekten sonra adresi döndürdüğümüzde döndürme kullanıyoruz. Son olarak, hash'lenmiş adresi mevcut adresler aralığına haritalarken modül-bölme kullanıyoruz.

13.4 Çarpışma Çözümü

Doğrudan ve çıkarma yöntemleri hariç, hashing için kullanılan yöntemlerin hiçbiri birebir eşleme değildir. Bu nedenle, yeni bir anahtarı bir adrese hashlediğimizde, bir çarpışma yaratabiliriz. Bununla başa çıkmak için birkaç yöntem vardır.

çarpışmalar, her biri hashing algoritmasından bağımsızdır. Yani, her hashing yöntemi, çarpışma çözümü yöntemleriyle birlikte kullanılabilir. Bu bölümde Şekil 13-13'te gösterilen çarpışma çözümü yöntemlerini tartışıyoruz.



ŞEKİL 13-13 Çarpışma Çözümü Yöntemleri

Ancak çarpışma çözümü yöntemlerini tartışmadan önce, üç tane daha kavramı kapsamamız gerekiyor. Hashing algoritmalarının doğası gereği, bir liste de her zaman bazı boş elemanlar olmalıdır. Aslında, birtam listeyi, bir eleman hariç tüm elemanların veri içerdiği bir liste olarak tanımlıyoruz. Genel bir kural olarak, bir hashlenmiş listenin %75'ten fazla dolmasına izin verilmemelidir. Bu kılavuz bizi ilk kavramımıza yönlendiriyor: yük faktörü. yük faktörü, bir hashlenmiş listenin içindeki eleman sayısının, listenin tahsis edilen fiziksel bölge sayısına bölünmesiyle elde edilen ve yüzde olarak ifade edilen bir değerdir. Geleneksel olarak, yük faktörüne alfa (α) sembolü atanır. k 'nın listenin içindeki dolu bölge sayısını ve n 'nin listenin tahsis edilen toplam eleman sayısını temsil ettiği formül

$$\alpha = \frac{k}{n} \times 100$$

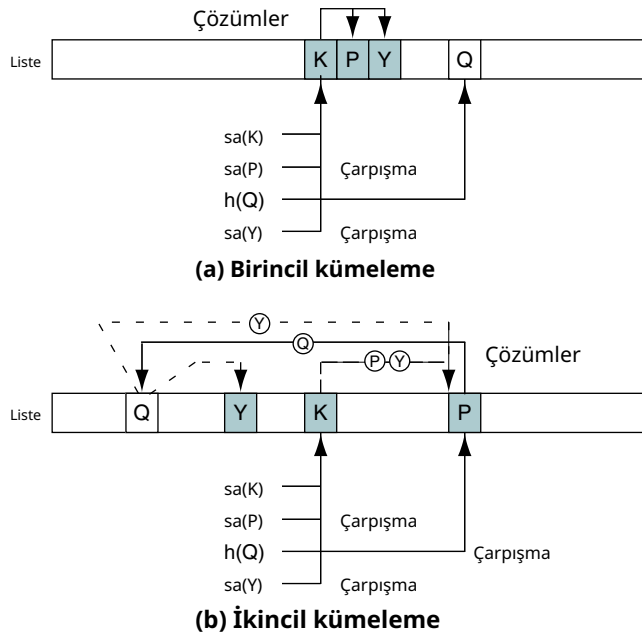
Veriler bir listeye eklendikçe ve çarpışmalar çözüldükçe, bazı hash algoritmaları verilerin liste içinde gruplandırılmasına neden olma eğilimindedir. Verilerin hashlenmiş bir liste içinde dengesiz bir şekilde birikme eğilimi kümelenme olarak bilinir, bu da ikinci kavramımızdır. Kümelenme bir endişe kaynağıdır çünkü genellikle çarpışmalarla oluşur. Liste yüksek bir kümelenme derecesine sahipse, bir öğeyi bulmak için gereken sorgu sayısı artar ve listenin işleme verimliliğini azaltır.

Bilgisayar bilimcileri iki farklı küme türü tanımlamıştır. Birincisi, birincil kümelenme, verilerin bir ana adres etrafında kümelendiği durumdur. Birincil kümelenme kolayca belirlenebilir. Örneğin, Amerika Birleşik Devletleri'nde bulunan nüfus kümelerinin düşünün. Eğer bir gece ülke genelinde uçuysanız, karanlıkta kasabaların ve şehirlerinin ışıklarıyla belirlediğini fark etmişsinizdir. Eğer tüm ülke bir hashlenmiş liste olsaydı, ve

ışıklar her biri bir veri ögesini temsil ediyorsa, ana adres etrafında kümelenme—liste mizdeki bir ev adresi etrafında kümelenmeye bakıyor olacağız.

İkincil kümelenme verilerin bir çarpışma yolu boyunca gruplandığında meydana gelir bir liste boyunca. Bu tür bir kümelenmeyi tanımlamak kolay değildir. İkincil kümelenmede veriler tüm liste boyunca geniş bir şekilde dağılmıştır, bu nedenle liste iyi dağıtılmış gibi görünür. Ancak, veriler iyi bir şekilde seyredilen bir çarpışma yolu boyunca yer alıyorsa, istenen bir veri ögesini bulma süresi artabilir.

Şekil 13-14 her iki tür kümelenmeyi diyagramlar. Şekil 13-14(a)'da K, P ve Y'nin K'nın ev adresi etrafında kümelenmiş olduğunu görüyoruz. Bu örnek te çarpışma çözümü ev adresine dayanmaktadır. Öte yandan Q, K'nın ev adresinden uzakta bulunan kendi adresine hash'lenir. Şekil 13-14(b)'de aynı dört anahtar listeye eklenmiştir. Bu örnekte çarpışma çözümü ev adresine dayanmamaktadır. Çarpışmaları tüm listeye yayar. Böylece K, P ve Y'nin hala aynı ev adresine hash'lenen eşanlamlılar olduğunu görsek de, K'nın ev adresi etrafında kümelenmemişlerdir; dosya boyunca dağılmışlardır. Ayrıca P'nin Q'nun ev adresine yerleştirildiği için ikincil bir çarpışma oluşturulduğunu da unutmayın.



ŞEKİL 13-14Kümelenme

İkincil kümeleme etkilerini daha iyi anlamak için, aşırı bir örneği düşünün: Her anahtarı aynı ana adrese hashleyen bir hash algoritmamız olduğunu varsayalım. Listeye eklenen ilk ögeyi bulmak

İkinci öğeyi bulmak yalnızca iki prob alır. Analojiyi sonuçlandırarak, listeye eklenen *n*inci öğeyi bulmak *n*prob alır, veriler liste adresleri arasında geniş bir şekilde dağıtılmış olsa bile.

Bu tartışmadan, hem birincil hem de ikincil kümelenmeyi en aza indirmek için hash algoritmalarımızı tasarlamamız gerektiği açık olmalıdır. Ancak, doğrudan ve çıkarma yöntemleri hariç, çarpışmaları ortadan kaldırmayacağımızı unutmayın.

Hashing yöntemlerinin sürprizlerinden biri, bir çarpışmanın meydana gelmesi için bir listeye ne kadar az öğe eklenmesi gerektiğidir. Bu kavram, karışmayı teşvik etmek için kullanılan yaygın bir parti oyununu hatırlarsak daha kolay anlaşılır. Ev sahibi, konuların bir listesini hazırlar ve her misafir, her konuyu karşılayan bir veya daha fazla misafir bulmak zorundadır. Bir konu genellikle doğum günleridir ve misafirlerin aynı doğum gününe sahip iki kişiyi bulmalarını gerektirir (yıl sayılmaz). 23'ten fazla parti misafiri varsa, bunlarda *n* ikisinin aynı doğum gününe sahip olma olasılığı %50'den daha iyidir.⁵ Bu olguyu hash algoritmalarımıza genelleterek, 365 adresli bir listeye sahipsek, ilk 23 eklemede %50'den fazla bir çarpışma bekleyebiliriz.

Son konseptimiz, verileri depolamak için bir yer ararken incelenen eleman sayısının sınırlı olması gerektiğidir. Listenin tüm elemanlarını incelemenin geleneksel sınırı üç zorluk ortaya çıkarır. İlk olarak, arama sıralı değildir, bu nedenle listenin sonunu bulmak, her elemanın test edildiği anlamına gelmez. İkincisi, her elemanı incelemek, hedefi bir arama çabası olan bir algoritma için aşırı zaman alıcı olacaktır. Üçüncüsü, bazı çarpışma çözümleme teknikleri, bir listede tüm elemanları fiziksel olarak inceleyemez. (Bir örnek için, sonraki bölümde "Kare Proba"ya bakın.)

Bu nedenle bilgisayar bilimcileri genellikle hashing algoritmalarında bir çarpışma sınırı koyarlar. Sınır aşıldığında ne olacağı uygulamaya bağlıdır. Basit bir çözüm programı iptal etmektir. Daha şık bir çözüm, verileri listeden ayrı bir alanda depolamaktır (bölümün ilerleyen kısımlarında "Bağlı Liste Çarpışma Çözümü"ne bakın). Her ne olursa olsun, listede veri ararken aynı algoritmik sınırı kullanmak önemlidir.

Artık bazı çarpışma çözümleme yöntemlerine bakmaya hazırız. Genel olarak, çarpışmaları çözmek için iki farklı yaklaşım vardır: açık adresleme ve bağlı listeler. Üçüncü bir kavram olan kovalar, çarpışmaları ertelemekte ancak önlememektedir.

Açık Adresleme

İlk çarpışma çözümleme yöntemi, açık adresleme, çarpışmaları ana alan içinde çözer—yani, tüm ev adreslerini içeren alan. Bu

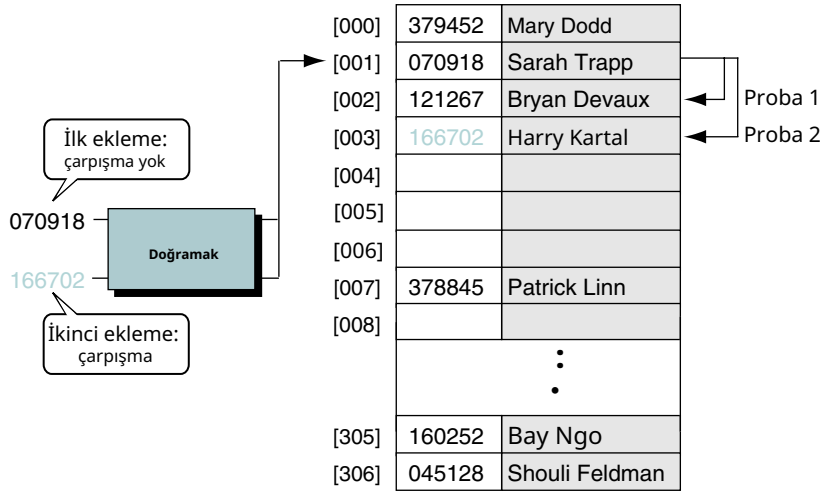
5. Bu matematiksel gerçek, 1930'larda von Mises tarafından ilk kez belgelenmiş ve vonMises doğum günü paradoksu olarak bilinmektedir.

teknik, çarpışmaların verilerin ayrı bir taşıma alanına yerleştirilerek çözüldüğü bağlı liste çözümlemesine karşıt bir yöntemdir.

Bir çarpışma meydana geldiğinde, ana alan adresleri yeni verilerin yerleştirilebileceği açıkveya boş bir eleman aramak için taranır. Dört farklı yöntemi tartışıyoruz: lineer tarama, kuadratik tarama, çift hashing ve anahtar ofseti.

Lineer Tarama

En basit olan lineer tarama yönteminde, veriler ev adresinde saklanamıyorsa çarpışmayı mevcut adrese 1 ekleyerek çözüyoruz. Örneğin, Şekil 13-10'daki modül-bölme yöntemi örneğine iki eleman daha ekleyelim. Sonuçlar Şekil 13-15'te gösterilmektedir. Anahtar 070918'i eklediğimizde, boş bir eleman buluyoruz ve çarpışma olmadan ekliyoruz. Ancak anahtar 166702'yi eklemeye çalıştığımızda, 001 konumunda bir çarpışma yaşıyoruz. Çarpışmayı çözmek için adrese 1 eklemeye çalışıyoruz ve yeni veriyi 002 konumuna ekliyoruz. Ancak, bu adres de dolu. Bu nedenle, adrese bir 1 daha ekliyoruz ve bu sefer yeni veriyi yerleştirebileceğimiz boş bir konum, 003 buluyoruz.



ŞEKİL 13-15 Doğrusal Proba Çarpışma Çözümü

Basit bir doğrusal probaya alternatif olarak, 1 ekleyebilir, 2 çıkarabilir, 3 ekleyebilir, 4 çıkarabiliriz ve boş bir eleman bulana kadar bu şekilde devam edebiliriz. Örneğin, bir çarpışma 341. konumda meydana geldiğinde, 342, 340, 343, 339 ve benzer konumları deneyerek boş bir eleman bulana kadar devam ederiz.

Her iki yöntemde de doğrusal probanın kodu, bir sonraki çarpışma çözümleme adresinin listenin sınırları içinde olduğundan emin olmalıdır. Böylece, bir anahtar listenin son konumuna hashlenirse, 1 eklemek,

listenin ilk ögesi. Benzer şekilde, anahtar listenin ilk ögesine hashlenirse, 1 çıkarmak, listenin son ögesinin adresini üretmelidir.

Doğrusal sorgulamalar iki avantaja sahiptir. İlk olarak, uygulaması oldukça basittir. İkincisi, veriler genellikle ev adreslerine yakın kalma eğilimindedir. Bu eğilim, ev adresine yakın olmanın önemli olduğu uygulamalarda, örneğin bir disk adresine hashlediğimizde, önemli olabilir. Öte yandan, doğrusal sorgulamalar genellikle birincil kümelenmeye yol açar. Ayrıca, veriler silindikten sonra arama algoritmasını daha karmaşık hale getirme eğilimindedirler.

Kareli Sondaj

Birincil kümeleme, ikincil kümeleme olmaksızın, mevcut adrese 1'den farklı bir değer ekleyerek ortadan kaldırılabilir. Kolayca uygulanabilir bir yöntem, kuadratik sorgu kullanmaktır. Kuadratik sorguda, artış çarpışma sorgu numarasının karesidir. Böylece ilk sorgu için 1^2 ekleriz, ikinci çarpışma sorgusu için 2^2 ekleriz, üçüncü çarpışma sorgusu için 3^2 ekleriz ve bu şekilde devam ederiz, ya boş bir eleman bulana kadar ya da mümkün olan elemanları tüketene kadar.

Adres listesinin sonuna ulaşmamamız için, yeni adres için kare toplamının modülünü kullanıyoruz. Bu dizilim, basitlik açısından 1. konumda bir çarpışma ve 100'lük bir liste boyutu varsayarak Tablo 13-2'de gösterilmiştir.

Probn umarası	Çarpışma konumu	Prob ² ve artış	Yeni adres
1	1	$1^2 = 1$	1 + 1 → 02
2	2	$2^2 = 4$	2 + 4 → 06
3	6	$3^2 = 9$	6 + 9 → 15
4	1531569241	$4^2 = 16$	15 + 16 → 31
5		$5^2 = 25$	31 + 25 → 56
6		$6^2 = 36$	56 + 36 → 92
7		$7^2 = 49$	92 + 49 → 41
8		$8^2 = 64$	41 + 64 → 05
9	5	$9^2 = 81$	5 + 81 → 86
10	86	$10^2 = 100$	86 + 100 → 86

TABLO 13-2 İkincil Çarpışma Çözüm Artışları

İkincil probun potansiyel bir dezavantajı, prob numarasını kareye almak için gereken zamandır. Ancak, her prob ile 2 artan bir artış faktörü kullanarak çarpan faktörünü ortadan kaldırabiliriz. Önceki artışa artış faktörünü eklemek, bir sonraki artışı verir; bu, Tablo 13-2'deki son sütundan görebileceğiniz gibi, probun karesine eşdeğerdir.

İkili probun bir sınırlaması vardır: listedeki her bir eleman için yeni bir adres oluşturmak mümkün değildir. Örneğin, Tablo 13-2'de yalnızca 59 prob benzersiz adresler oluşturabilir. Listedeki diğer 41 konum probelenmemiştir. İlk tekrarlanan adres prob 10'da bulunur. Daha fazla örnek görmek için

tekrarlanan adresler için, tabloyu birkaç prob ile genişletin. Bu sorunun çözümü, asal bir sayı olan bir liste boyutu kullanmaktır. Liste boyutu asal bir sayı olduğunda, listenin en az yarısı erişilebilir, bu da makul bir sayıdır.

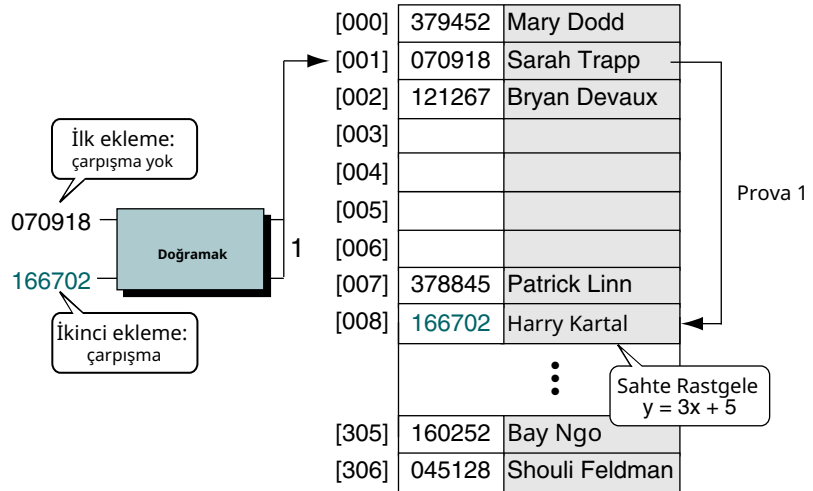
Psödo-rastgele Çarpışma Çözümü

Son iki açık adresleme yöntemi topluca çift hashleme olarak bilinir. Heryö ntemde, aritmetik bir prob fonksiyonu kullanmak yerine, adres yeniden h ashlenir. Tartışmadan anlaşılacağı gibi, her iki yöntem de birincil kümelen meyi önler.

Psödo-rastgele çarpışma çözümü çarpışmayı çözmek için psödo-rastgele bir sayı kullanır. Daha önceki bölümde "Psödo-rastgele Hashing" kısmında bir hashleme yöntemi olarak psödo-rastgele sayı üretici gördük. Şimdi bunu bir çarpışma çözümleme yöntemi olarak kullanıyoruz. Bu durumda, rastgele sayı hesaplamasında faktör olarak anahtar kullanmak yerine, çarpışma adresini kullanıyoruz. Şekil 13-15'te oluşturduğumuz çarpışmayı düşünün. Şimdi çarpışmayı aşağıdaki psödo-rastgele sayı üretici ile çözüyoruz, burada $a = 3$ ve $c = 5$ 'tir:

$$\begin{aligned} y &= (ax + c) \bmod \text{listSize} \\ &= (3 \times 1 + 5) \bmod 307 \\ &= 8 \end{aligned}$$

Bu örnekte çakışmayı, yeni veriyi eleman008'de (Şekil 13-16) yerleştire rek çözüyoruz. Örneğimizi sığdırmak için katsayıları küçük tutmalıyız. Dah a iyi bir faktör seti, a için 1663 gibi büyük bir asal sayı kullanır.



ŞEKİL 13-16 Sahte Rastgele Çakışma Çözümü

Sahte rastgele sayılar nispeten basit bir çözümdür, ancak bir önemli sınırlamaları vardır: tüm anahtarlar yalnızca bir çakışma çözüm yolu izler. (Bu eksiklik, doğrusal ve kare denemelerde de görülür.) Sahte rastgele çakışma çözümü önemli ikincil kümelenmelere neden olabileceğinden, farklı anahtarlar için farklı çakışma yolları üreten bir yöntem aramalıyız.

Anahtar Ofseti

Anahtar ofseti, farklı anahtarlar için farklı çakışma yolları üreten bir çift hashing yöntemidir. Sahte rastgele sayı üretici, önceki adresin bir fonksiyonu olarak yeni bir adres üretirken, anahtar ofseti yeni adresi eski adresin ve anahtarın bir fonksiyonu olarak hesaplar. En basit versiyonlarından biri, anahtarın liste boyutuna bölünmesinin bölümünü adrese ekleyerek bir sonraki çakışma çözüm adresini belirler; bu aşağıdaki formülde gösterilmiştir.

$$\begin{aligned} \text{telafi etmek} &= \lfloor \text{anahtar} / \text{listeBoyutu} \rfloor \\ \text{adres} &= ((\text{offset} + \text{eski adres}) \bmod \text{listeBoyutu}) \end{aligned}$$

Örneğin, anahtar 166702 ve liste boyutu 307 olduğunda, modulo-bölme hashing yöntemi 1 adresi üretir. Şekil 13-16'da gösterildiği gibi, bu 070918 eş anlamlısı 1 adresinde bir çakışma üretir.

Anahtar ofsetini kullanarak bir sonraki adresi hesapladığımızda, aşağıda gösterildiği gibi 237 elde ederiz.

$$\begin{aligned} \text{offset} &= \lfloor 166702 / 307 \rfloor = 543 \\ \text{address} &= ((543 + 001) \bmod 307) = 237 \end{aligned}$$

Eğer 237 de bir çakışma olsaydı, aşağıda gösterildiği gibi bir sonraki adresi bulmak için süreci tekrar ederiz.

$$\begin{aligned} \text{telafi etmek} &= \lfloor 166702 / 307 \rfloor = 543 \\ \text{adres} &= ((543 + 237) \bmod 307) = 166 \end{aligned}$$

Anahtar ofsetinin etkisini gerçekten görmek için, hepsi aynı ana adresine hashlenen birkaç farklı anahtarı hesaplamamız gerekiyor. Tablo 13-3'te, adres 001'de çakışan üç anahtar için sonraki iki çakışma prob adresini hesaplıyoruz.

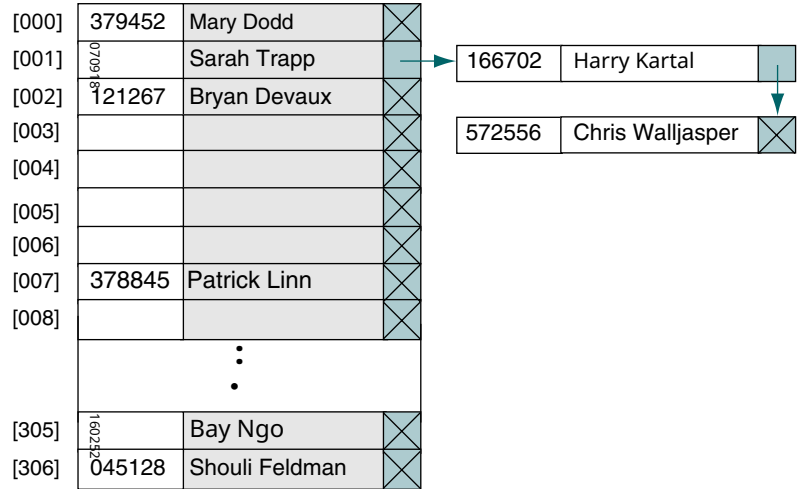
Anahtar	Ev adresi	Anahtar ofseti	Prova 1	Prova 2
166702	1	543	237	166
572556	1	1865	024	047
067234	1	219	220	132

TABLO 13-3 Anahtar-ofset Örnekleri

Her anahtarın çakışmasını hem birinci hem de ikinci denemeler için farklı bir adreste çözdüğünü unutmayın.

Bağlı Liste Çakışma Çözümü

Açık adreslemenin büyük bir dezavantajı, her çakışma çözümlemesinin gelecekteki çakışma olasılığını artırmasıdır. Bu dezavantaj, çakışma çözümlemesinin ikinci yaklaşımında ortadan kaldırılır: bağlı listeler. Bağlı liste, her bir elemanın bir sonraki elemanın konumunu içerdiği sıralı bir veri koleksiyonudur. Örneğin, Şekil 13-17'deki dizi elemanı 001, Sarah Trapp, bir sonraki eleman olan Harry Eagle'a işaret eden bir gösterici içerir; bu gösterici de üçüncü eleman olan Chris Walljasper'a işaret eder (bağlantı yeri çizildiğinde, bağlantı null olur). Bağlı listelerin bakımını Bölüm 5'te inceledik.



ŞEKİL 13-17 Bağlı Liste Çarpışma Çözümü

Bağlı liste çarpışma çözümü, çarpışmaları depolamak ve tüm eşanlılıları bir bağlı listede bir araya getirmek için ayrı bir alan kullanır. İki depolama alanı kullanır: birincil alan ve taşma alanı. Birincil alandaki her öge, taşma alanındaki taşma verilerinin bağlı listesine bir bağlantı başı işaretçisi olan ek bir alana sahiptir. Bir çarpışma meydana geldiğinde, bir öge birincil alana depolanır ve taşma alanındaki karşılık gelen bağlı listeye zincirlenir. Taşma alanı herhangi bir veri yapısında olabilir, ancak genellikle dinamik bellek içinde bir bağlı liste olarak uygulanır. Şekil 13-17, adres 001 için üç eşanlılı ile Şekil 13-16'daki bağlı listeyi göstermektedir.

Bağlı liste verileri herhangi bir sırada depolanabilir, ancak son giren ilk çıkar (LIFO) sırası veya anahtar sırası en yaygın olanıdır. LIFO sırası

eklemeler için en hızlısıdır çünkü bağlı liste verileri eklemek için taranmak zorunda değildir. Taşmaya eklenen öge, bağlı listenin başına yerleştirilir ve birincil alandaki düğüme bağlanır. Anahtar sıralı listeler, birincil alandaki anahtarın en küçük olduğu durumlarda daha hızlı arama geri alımı sağlar. Hangi sıranın (LIFO veya anahtar sırası) kullanılacağı uygulamaya bağlıdır.

Kova Hashing

Çarpışma problemlerini ele almanın bir diğer yaklaşımı, anahtarların birden fazla veri örneğini barındıran kovalara hash'lendiği kova hashing'dir. Bir kova birden fazla veriyi tutabildiğinden, çarpışmalar, kova dolana kadar ertelenir. Örneğin, Şekil 13-17 listemizde her adresin üç çalışanın verilerini tutacak kadar büyük olduğunu varsayalım. Bu varsayım altında, dördüncü bir çalışmanı bir adrese eklemeye çalışana kadar bir çarpışma meydana gelmez. Bu kavramın iki sorunu vardır. İlk olarak, birçok kova her zaman boş veya kısmen boş olduğundan, önemli ölçüde daha fazla alan kullanır. İkincisi, çarpışma problemini tamamen çözmez. Bir noktada bir çarpışma meydana gelir ve çözülmesi gerekir. Bu olduğunda, tipik bir yaklaşım, bir sonraki ögenin bazı boş alanlara sahip olduğunu varsayarak lineer bir probe kullanmaktır. Şekil 13-18, kova yaklaşımını göstermektedir.

[000]	Kova 0	379452	Mary Dodd
[001]	Kova 1	070918	Sarah Trapp
		166702	Harry Kartal
		367173	Ann Giorgis
[002]	Kova 2	121262	Bryan Devaux
		572556	Chris Walljasper
[307]	Kova 307	045128	Shouli Feldman

Doğrusal probe
buraya yerleştirildi

ŞEKİL 13-18 Kova Hashing

Şekil 13-18'deki ikinci kovayı inceleyin. Üç girişin verilerini içerdiğini ve bunların hepsinin adres 1'e hash'lendiğini not edin. Dördüncü anahtar, örneğimizde 572556, listeye eklendiğinde çarpışma yaşamıyoruz. Bir çarpışma nihayet meydana geldiğinde—yani, kova dolduğunda—çarpışmanın herhangi biri

çözüm yöntemleri kullanılabilir. Örneğin, Şekil 13-18'de 572556'yı eklediğimizde bir çarpışma meydana gelir çünkü kova 1 doluydu. Daha sonra, bunu konum 2'ye eklemek için bir doğrusal prob kullanırız. Ayrıca, verimlilik için anahtarları bir kovada artan anahtar sırasına göre yerleştirdiğimizi unutmayın.

Kombinasyon Yaklaşımları

Çarpışmaları çözmek için birkaç yaklaşım vardır. Hashing yöntemlerinde gördüğümüz gibi, karmaşık bir uygulama genellikle birden fazla adım kullanır. Örneğin, büyük bir veritabanı uygulaması bir kovaya hash'ler. Kova doluyorsa, çarpışmayı çözmek için üç gibi belirli sayıda doğrusal prob kullanır ve ardından bağlı liste taşıma alanını kullanır.

Hash'lenmiş Liste Örneği

Hash'lenmiş aramaları göstermek için, telefon numaralarını arayan bir program oluşturulalım. Girdiğimiz isimler ve telefon numaraları içeren basit bir metin dosyasıdır. Program başladığında, dosyayı okur ve verileri bir diziye yükler. Daha sonra telefon numaralarını bulmak için bir hash kullanırız. Hashing için Algoritma 13-6'nın bir varyasyonunu ve çarpışma çözümü için bir doğrusal prob kullanırız.

53'lük bir liste boyutu kullanıyoruz çünkü bu bir asal sayıdır. Ayrıca, eş anlamlı olan iki isim seti de ekliyoruz. Julie ve Chris her ikisi de konum 38'e hash'leniyor; Wayne ve Wyan her ikisi de hash listemizdeki son konum olan 52'ye hash'leniyor. Bu nedenle, listenin sonunda çarpışma çözümlememizi test edebiliriz. Kodumuz Program 13-3'te gösterilmiştir.

PROGRAM 13-3 Hashlenmiş Liste Arama

```

123456789 /* Hashing kullanarak telefon arama programı.
           Girdi, isimler ve telefon numaraları içeren bir dosyadır.
           Yazarı:
           Tarih:
1011121314151617181920 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Küresel Bildirimler
typedef struct
{
    karakter adı [31];
    karakter telefonu[16];
} LISTING;

const int cMax_Size = 53;

// Prototip Bildirimleri
void buildList (LISTING phoneList[], int* last);

```

devam ediyor

PROGRAM 13-3 Hashlenmiş Liste Arama (devam ediyor)

```

212223242526272829303132333435363738394041424344454647484950515253545556575859606162636465666768
void hashSearch (LISTING* telefonList, int last);
int hashKey (char* key, int last);
int çarpışma (int last, int locn);

int main (void)
{
// Yerel Tanımlar
LISTING phoneList[cMax_Size];
int last;

// İfadeler
printf("Telefon Listesine Başla"); last = c

Max_Size - 1;
buildList (phoneList, &last);hashSear
ch(phoneList, last);printf("Telefon Lis

tesine Son Ver");return 0;

} // ana

/* ===== buildList =====
Telefon numarası dosyasını oku ve diziye yükle.
Önce phoneList doldurulacak bir dizidir
last son yüklenen elemanın indeksidir
Sonra dizi dolduruldu
*/
void buildList (LISTING phoneList[], int* last)
{
// Yerel Tanımlar
DOSYA* fpPhoneNums;
LISTELEMEListeleme;
int yerel;
int cntCol;
niyet etmek;

// İfadeler
fpPhoneNums = fopen ("P13-03.TXT", "r");
if (!fpPhoneNums)
{
printf("Telefon dosyası açılmıyor\"); exit (1
00);
} // eğer

// Anahtarları null olarak ayarla
for (int i = 0; i <= *last; i++)
phoneList[i].name[0] = '\0';

```

devam ediyor

PROGRAM 13-3 Hashlenmiş Liste Arama (devam ediyor)

```

69707172737475767778798081828384858687888990919293949596979899100101102103104105106107108109110111112113114115116
        while (!feof(fpPhoneNums))
        {
            fscanf(fpPhoneNums, " %30[^\n]*%c %[^;\n]*%c",
                    aListing.name, aListing.phone);
            locn = hashKey(aListing.name, *last); if (ph

oneList[locn].name[0] != '\0')    {

            // Çakışma
            bitiş = *son;
            cntCol = 0;
            while (telefonListesi[locn].isim[0] != '\0'
                    && cntCol++ <= *son)
                locn = çarpışma(*son, locn);

            if (telefonListesi[locn].isim[0] != '\0')
            {
                printf("Liste dolu. Hepsi okunamadı.");      re
                turn;
            } // tam liste ise
        } // çakışma ise
        phoneList[locn] = aListing;
    } // iken
    geri dönmek;
} // yapıListesi

/* ===== hashKey =====
Verilen anahtar, listedeki konuma hash anahtarı.
Önceki telefonList hash dizisidir      s
on listedeki son indekstir      anaht
ar hash'lenmesi gereken stringdir
Sonuç hash konumunu döndürür

*/
int hashKey (char* anahtarı, int last)
{
    // Yerel Tanımlar
    int  addr;
    int  keyLen;

    // İfadeler
    keyLen = strlen(key);
    addr = 0;

    for (int i = 0; i < keyLen; i++)
        if (key[i] != ' ')
            addr += anahtar[i];
    return (addr % son + 1);
} // hashKey

```

devam ediyor

PROGRAM 13-3 Hashlenmiş Liste Arama (devam ediyor)

```

117118119120121122123124125126127128129130131132133134135136137138139140141142143144145146147148149150151152153154155156157158159160161162163164
/* ===== çarpışma =====
    Bir çarpışma meydana geldi. Çöz.
    Önce phoneList hashlenmiş listedir
        last listedeki son elemanın indeksidir
        locn çarpışmanın adresidir
    Sonra listedeki bir sonraki adresi döner
*/
int çarpışma (int last, int locn)
{
    // İfadeler
    return locn < last ? ++locn : 0;
} // çarpışma

/* ===== hashSearch =====
    Kullanıcıdan isim iste ve dizide ara.
    Önce phoneList başlatılmıştır
    Sonra Kullanıcı çıkış istedi
*/
void hashSearch (LISTING* telefonList, int last)
{
    // Yerel Tanımlar
    char srchName[31];
    char more;
    int locn;
    int maxSrch;
    int cntCol;

    // İfadeler
    do
    {
        printf("İsim girin: ");
        scanf ("%s", srchName);

        locn = hashKey (srchName, last);
        if (strcmp(srchName, telefonListesi[locn].name) != 0)
        {
            // çarpışma olarak değerlendir
            maxSrch = last;
            cntCol = 0;
            while (strcmp (srchName,
                           telefonListesi[locn].isim) != 0
                && cntCol++ <= maxSrch)
                locn = collision(last, locn);
        } // eğer

        // Başarıyı test et
        if (strcmp (srchName, telefonListesi[locn].isim) == 0)

```

devam ediyor

PROGRAM 13-3 Hashlenmiş Liste Arama (devam ediyor)

```

165166167168169170171172173174175
printf("%-32s (%02d) %-15s", telefonLi
      stesi[locn].isim, locn,

      telefonListesi[locn].telefon);

başka
printf("%s bulunamadı", srchName); printf("

Başka bir numara aramak ister misiniz <E/H>? "); scan
f (" %c", &more);
} while (more == 'Y' || more == 'y');
} // hashSearch

```

Sonuçlar:

Telefon Listesine Başlaİsim gi
rin: Julie
Julie (38) (555) 916-1212

Başka bir numara aramak ister misiniz <E/H>? eİ
sim girin: Chris
Chris (39) (555) 946-2859

Başka bir numara aramak ister misiniz <E/H>? eİ
sim girin: Wyan
Wyan (52) (555) 866-1234

Başka bir numara aramak ister misiniz <E/H>? eİ
sim girin: Wayn
Wayn (0) (555) 345-0987

Başka bir numara aramak ister misiniz <Y/N>? yİ
sim girin: Bill
Bill bulunamadı

Başka bir numara aramak ister misiniz <Y/N>? n
Telefon Listesini Sonlandır

Program 13-3 Analizi Bu programda tartışılması gereken birkaç nokta var, ancak mümkün olduğunca basit tutmaya çalıştık. Dosyayı yüklerken, anahtarı hashlemek için bir fonksiyon kullanıyoruz. Aynı fonksiyonu listeyi aramak için de kullanıyoruz. Anahtarı hashledikten sonra, bir çakışma olup olmadığını test ediyoruz. Eğer hashlenmiş konumdaki isim boş bir dize değilse, bir çakışma var demektir. Çakışmayı çözmek için, boş bir konum bulana kadar döngüye giriyoruz. İki neden için yalnızca bir ifade olmasına rağmen, ayrı bir çakışma fonksiyonu kullanıyoruz. İlk olarak, çakışma çözümü, programdaki birden fazla fonksiyon tarafından kullanıldığı için kendi fonksiyonuna yerleştirilmesi gereken ayrı bir süreçtir. İkincisi, daha sonra çakışma çözümleme algoritmasının anahtar ofsetine değiştirmeye karar verirsek, kodu kolayca izole edip değiştirebiliriz.

Arama işlevinde, arama argümanını kullanıcıdan alırız ve ardından konumunu belirleme için hash fonksiyonunu çağırırız. Eğer arama argümanı konumdaki isimle eşleşmezse, bir çakışma olduğunu varsayız. Bu durumda, eşleşen bir anahtar bulana kadar veya listedeki her konumu inceleyene kadar döngü devam eder.

Bu basit arama algoritmasının en büyük sorunu, bir anahtarın listede olmadığını belirlemek için tüm konumlarda arama yapması gerektiğidir. Çok fazla veri içeren bir üretim programında, biraz farklı bir yaklaşım kullanırdık. Bir konumun daha önce hiç kullanılmadığını belirlemek için bir bayrak kullanırsak, daha önce veri içermiş ama boş olan bir konumdan ziyade, boş bir konum bulduğumuzda aramayı durdurabilirdik.

13.5 Anahtar Terimler

ikili arama
 kova
 kova hashing
 kümeleme
 çarpışma
 çarpışma çözümü
 jit çıkarımı
 doğrusal hashing
 bölüm kalanı
 çift hashing
 katlama sınırı
 katlama kaydırması
 hashlenmiş arama
 ev adresi
 anahtar ofseti
 doğrusal sorgulama
 doğrusal arama
 bağlı liste çarpışma çözümü

yük faktörü
 orta kare hashing
 modül-bölme
 açık adresleme
 aşırı yük alanı
 birincil kümelenme
 asal alan
 olasılık araması
 sorgulama
 sahte rastgele çarpışma çözümü
 sahte rastgele hashing
 kuadratik sorgulama
 amadönme hashi
 ngarama
 ikincil kümelenme
 sıralı arama
 eş anlamlı

13.6 Özet

- Arama, bir hedefin bir nesne listesi içindeki konumunu bulma süreci dir.
- Diziler için iki temel arama yöntemi vardır: sıralı ve ikili arama.
- Sıralı arama genellikle bir liste sıralı olmadığında kullanılır. Liste başında n başlar ve veriyi bulana kadar veya listenin sonuna ulaşana kadar arama yapar.
- Sıralı aramanın bir varyasyonu, bekçi aramasıdır. Bu yöntemde aramayı sonlandıran koşul, hedefin liste sonuna yapay olarak eklenmesiyle yalnızca bir taneye indirilir.
- Sıralı aramanın ikinci varyasyonu olasılık araması olarak adlandırılır. Bu yöntemde liste, en olası elemanlar listenin başında ve en az olası olanlar ise sonunda olacak şekilde sıralanır.
- Sıralı arama, sıralı bir listeyi aramak için de kullanılabilir. Bu durumda, hedef mevcut elemandan küçük olduğunda aramayı sonlandırabiliriz.
- Bir dizi sıralıysa, ikili arama adı verilen daha verimli bir algoritma kullanılabiliriz.
- İkili arama algoritması, önce ortadaki öğeyi kontrol ederek listeyi arar. Hedef ortadaki öğede yoksa, algoritma listenin üst yarısını veya alt yarısını, değerine bağlı olarak ortadan kaldırır.

orta eleman. Süreç, hedef bulunana kadar veya azaltılmış liste uzunluğu sıfıra dönüşene kadar devam eder.

- Sıralı bir listenin verimliliği $O(n)$.
- İkili aramanın verimliliği $O(\log n)$.
- Hashlenmiş bir aramada, anahtar, algoritmik bir dönüşüm aracılığıyla verinin konumunu belirler. Bu, anahtar-adres dönüşümüdür.
- Doğrudan, çıkarma, modül-bölme, rakam çıkarma, orta kare, katlama, döndürme ve sahte rastgele üretim dahil olmak üzere birkaç hashleme fonksiyonu vardır.
 - Doğrudan hashlemede anahtar, herhangi bir algoritmik manipülasyon olmadan adrestir.
 - Çıkarma hashlemede anahtar, sabit bir sayı çıkarılarak bir adrese dönüştürülür.
 - Modül-bölme hashlemede anahtar, liste boyutuna (bir asal sayı olması önerilir) bölünür ve kalan artı 1 adres olarak kullanılır.
- Rakam çıkarma hashlemede seçilen rakamlar anahtardan çıkarılır ve adres olarak kullanılır.
- Orta kare hashlemede anahtar kare alınır ve adres, sonucun ortasından seçilir.
- Katlama kaydırma hashlemede anahtar, gerekli adresin boyutuna uyan parçalara bölünür. Ardından parçalar toplanarak adres elde edilir.
- Katlama sınır hashlemede anahtar, gerekli adresin boyutuna uyan parçalara bölünür. Ardından sol ve sağ parçalar ters çevrilir ve orta parçaya eklenerek adres elde edilir.
- Dönüşümlü hashing'de, anahtarın en sağdaki rakamı sola döndürülerek bir adres belirlenir. Ancak, bu yöntem genellikle diğer yöntemlerle birlikte kullanılır.
- Pseudorandom üretim hashing'inde, anahtar bir tohum olarak kullanılarak bir pseudorandom sayı üretilir. Sonuç daha sonra adresi elde etmek için ölçeklenir.
- Doğrudan ve çıkarma yöntemleri dışında, hashing'de çakışmalardan kaçınılmazdır. Çakışmalar, yeni bir anahtarın zaten işgal edilmiş bir adrese hashlenmesi durumunda meydana gelir.
- Kümeleşme, verilerin hashlenmiş bir liste üzerinde düzensiz bir şekilde birikme eğilimidir.
 - Birincil kümeleşme, verilerin bir ana adres etrafında birikmesi durumunda meydana gelir.
 - İkincil kümeleşme, verilerin listede bir çakışma yolu boyunca birikmesi durumunda meydana gelir.
- Bir çakışmayı çözmek için, bir çakışma çözümleme yöntemi kullanılır.
- Çakışmaları çözmek için üç genel yöntem kullanılır: açık adresleme, bağlı listeler ve kovalar.

- Açık adresleme yöntemi, lineer prob, kuadratik prob, pseudorandom rehashing ve anahtar-offset rehashing olarak alt bölümlere ayrılabilir.
 - Doğrusal prob yönteminde, çarpışma gerçekleştiğinde yeni veriler bir sonraki mevcut adrese kaydedilir.
 - Kare yönteminde, artış çarpışma prob numarasının karesidir.
- Pseudorastgele yeniden hashleme yönteminde, adresi yeniden hashlemek için rastgele bir sayı üretici kullanılır.
- Anahtar-ofset yeniden hashleme yönteminde, adresi yeniden hashlemek için bir ofset kullanılır.
- Bağlı liste tekniğinde, ayrı alanlar çakışmaları depolar ve tüm eşanlam lıları bir bağlı listede zincirler.
- Kova hashlemede bir kova birden fazla veri örneğini barındırır.

13.7 Uygulama Setleri

Alıştırımlar

1. Bir dizi aşağıda gösterilen elemanları içerir. İkili arama algoritmasını kullanarak 88'i bulmak için izlenen adımları takip edin. Her döngü yinelemesinde, son dahil, ilk, son ve orta'nın içeriklerini gösterin.

18 13 17 26 44 56 88 97

2. Bir dizi aşağıda gösterilen elemanları içerir. İkili arama algoritmasını kullanarak 20'yi bulmak için izlenen adımları takip edin. Her döngü yinelemesinde, son dahil, ilk, son ve orta'nın içeriklerini gösterin.

18 13 17 26 44 56 88 97

3. Modül-bölme yöntemi ve doğrusal deneme kullanarak, aşağıda gösterilen anahtarları 19 elemanlı bir dizide saklayın. Kaç çakışma meydana geldi? Tüm anahtarlar eklendikten sonra listenin yoğunluğu nedir?

224562 137456 214562

140145 214576 162145

144467 199645 234534

4. Çakışmalar için bir bağlı liste yöntemi kullanarak Alıştırma 3'ü tekrarlayın. Bu alıştırma 3'teki sonuçları Alıştırma 3'te elde ettiğiniz sonuçlarla karşılaştırın.

5. Sayı çıkarma yöntemini (birinci, üçüncü ve beşincirakamlardan) ve ikinci dereceden sorgulamayı kullanarak Egzersiz 3'ü tekrarlayın.
6. Çakışmalar için bağlı liste yöntemini kullanarak Egzersiz 5'i tekrarlayın. Bu egzersizdeki sonuçları, Egzersiz 5'te elde ettiğiniz sonuçlarla karşılaştırın.
7. Midsquare yöntemini kullanarak, hashing için ortadaki iki rakamıkullanarak Egzersiz 3'ü t ekrarlayın. Bir çakışma meydana gelirse yeniden hashleme için bir sahte rastgele sayı üre teci kullanın. $a = 3$ ve $c = -1$ faktörlerini kullanın.
8. Çakışmalar için bir anahtar-ofset yöntemi kullanarak Egzersiz 7'yi tekrarlayın. Bu e gzersizdeki sonuçları, Egzersiz 7'de elde ettiğiniz sonuçlarla karşılaştırın.
9. İki rakamlı bir seferde katlayarak ve ardından katlanmış toplam üzerinde mod bölme işlemi yapa rak fold shift yöntemini kullanarak Egzersiz 3'ü tekrarlayın.
10. Fold boundary yöntemini kullanarak Egzersiz 9'u tekrarlayın.
11. Hashing için döndürme yöntemini kullanarak Egzersiz 3'ü tekrarlayın. Öncelikle en sağdaki raka mları iki sola döndürün ve ardından rakam çıkarma işlemi yapın (birinci, üçüncü ve beşinci raka mlar). Çakışmaları çözmek için lineer sorgulama yöntemini kullanın.
12. Çakışmalar için bir anahtar-ofset yöntemi kullanarak Egzersiz 11'i tekrarlayın. Bu e gzersizdeki sonuçları, Egzersiz 11'de elde ettiğiniz sonuçlarla karşılaştırın.

Problemler

13. 1 ile 200 arasındaki 100 rastgele tam sayıdan oluşan bir dizi oluşturan b ir program yazın ve ardından, ardışık arama kullanarak diziyi 100 kez, a yını aralıktaki rastgele oluşturulmuş hedeflerle arayın. Programın sonu nda, aşağıdaki istatistikleri görüntüleyin:
 - a. Tamamlanan arama sayısı
 - b. Başarılı arama sayısı
 - c. Başarılı aramaların yüzdesi
 - d. Her arama başına ortalama test sayısı

Her arama başına ortalama test sayısını belirlemek için, her arama için test sayısını saymanız gerekir.

Programınızı çalıştırdıktan sonra, beklenen verimlilik (big-O) ile hesapladığınız s onuçlar arasındaki benzerlikler veya farklılıklar hakkında bir paragraf yazın.
14. Sıralı liste araması kullanarak Problem 13'ü tekrarlayın. Bir psödo-rastgele sayı üretici kullanmak yerine, 1 ile başlayarak bir dizi sayı oluşturun ve sırayla 1 ekleyip ardından 2 ekleyerek serideki sonraki sayıları oluşturun, aşağıda gösterildiği gibi.

1 3 4 6 7 9 10 12 13 15 6 ... 145 147 148 150

Arama argümanları için, 1 ile 150 arasındaki 100 sayıyı oluşturun

15. Problem 14'ü ikili arama kullanarak tekrarlayın.

Projeler

16. Program 13-3'ü, hashlenmiş liste aramasının verimliliğini belirlemek için değiştirin. Programınızı en az 50 isim ve en az 10 arama ile çalıştırın. Listede olmayan en az üç arama argümanı ekleyin.
17. Programınızı Problem 16'dan dört kez çalıştırın. Çalışmalar arasındaki tek farklar yükleme faktörü olmalıdır. İlk program için %60 yükleme faktörü kullanın ve ardından her bir sonraki çalışma için %10 artırın. Arama verimliliğini çizen bir grafik çizin ve farklar hakkında kısa bir rapor yazın.
18. Program 13-3'ü çarpışmaları çözmek için sahte rastgele sayı üretimi kullanacak şekilde değiştirin. İki yöntem arasındaki farklar hakkında kısa bir rapor yazın.
19. Geçen ay satılan envanter parçalarının ve miktarlarının bir listesini oluşturmak için bir hashing algoritması kullanan bir program yazın. Hash'lenmiş listeyi oluşturduktan sonra, kullanıcının aşağıdaki seçeneklerden birini seçmesine olanak tanıyan basit bir menü tabanlı kullanıcı arayüzü yazın:
 - a. Bir envanter ögesini arayın ve satılan miktarını raporlayın
 - b. Envanter parçalarını ve satılan miktarlarını yazdırın
 - c. Hashing algoritmasının verimliliğini analiz edin

Parça verileri, Tablo 13-4'te gösterildiği gibi bir metin dosyasında bulunmaktadır. anahtar üç haneli parça numarasıdır. Miktar, geçen ay içinde satılan birimleri temsil eder.

Parça numarası	Miktar
112	12
130	30
156	56
173	17
197	19
150	50
166	66
113	13
123	12
143	14
167	16
189	18
193	19
117	11
176	76

TABLO 13-4 Hashing Problemi için Veriler

Programınızdan üç çıktı gereklidir.

- a. Aşağıdaki aramaları test edin ve uygun mesajları döndürün. İsterseniz diğer parça numaralarını da test edebilirsiniz, ancak aşağıdaki testlerin önce tamamlanması gerekmektedir:
 - 112 için ara
 - 126 için ara
 - 173 için ara
- b. İstenildiğinde, bu veri seti için hashing algoritmasının verimliliğini analiz edin. Çıktınız aşağıda gösterilen rapor formatını takip etmelidir.

```
Doldurulan Prime Alan Yüzdesi: xx%
Bağlantılı listelerdeki ortalama düğüm sayısı:nn
En uzun bağlantılı liste          nn
```

- c. Listenin tüm içeriğinin çıktısı aşağıdaki formatı kullanmalıdır:

```
Ev Adresi Prime Alan Taşma Listesi
0 130/30
1
2 112/12
3 123/12 143/14, 173/17, 193/19 .
.
.
```

20. Sıralı arama ADT'si oluşturun. Aranacak dizi, uygulama programı tarafından kendi alanında tutulacaktır. Hedef herhangi bir türde olabilir ve bir yapının içinde yer alabilir. ADT arayüzü için prototip şu dur:

```
bool seqSearch (geçersiz* ary,
               int sizeofElem,
               int numElem,
               int (*karşılaştır)(void* arg1, void* arg2));
```

burada ary aranacak verileri içerir, sizeofElem dizideki bir elemanın boyutudur, numElem dizideki eleman sayısıdır ve compare iki dizi elemanını karşılaştırmak için uygulama fonksiyonudur. Karşılaştırma fonksiyonu $arg1 < arg2$ ise -1, $arg1 = arg2$ ise 0 ve $arg1 > arg2$ ise +1 döner.

21. Proje 20'yi ikili arama kullanarak yeniden yazın.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Ek A

ASCII Tabloları

Bu ek, Bilgi Değişimi için Amerikan Standart Kodu'nu (ASCII) içermektedir. Tablo A-1, ondalık, onaltılık, sekizlik ve sembolik kodları, uygun olduğunda İngilizce bir yorum ile göstermektedir. Tablo A-2, tüm değerlerin onaltılık matrisini vermektedir.

A.1 ASCII Kodları (Uzun Form)

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
0	00	00	HÜKÜMSÜZ	NULL değeri
1	01	01	SOH	Başlık başlangıcı
2	02	02	STX	Metin başlangıcı
3	03	03	VEŚAİRE	Metin sonu
4	04	04	EOT	İletim sonu
5	05	05	ENYA	Soru
6	06	06	ACK	Teşekkür
7	07	07	BEL	Zil çal
8	08	10	BS	Geri sil
9	09	11	HT	Yatay sekme
10	0A	12	LF	Satır beslemesi

devam ediyor

TABLO A-1 ASCII Kodları

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
11	0B	13	VT	Dikey sekme
12	0C	14	FF	Form beslemesi
13	0D	15	CR	Taşıma dönüşü
14	0E	16	BU YÜZDEN	Çıkış yap
15	0F	17	VE	Giriş yap
16	10	20	BUNA GÖRE	Veri bağlantısı kaçışı
17	11	21	DC1	Aygıt kontrol 1
18	12	22	DC2	Aygıt kontrol 2
19	13	23	DC3	Aygıt kontrol 3
20	14	24	DC4	Aygıt kontrol 4
21	15	25	İSTEK	Negatif onay
22	16	26	SEN	Eşzamanlı boшта
23	17	27	ETB	İletim sonu bloğu
24	18	30	YAP	İptal Et
25	19	31	İÇİNDE	Ortamin sonu
26	1A	32	ALT	Yerine Geçme
27	1B	33	KAÇ	Kaçış
28	1C	34	FS	Dosya ayırıcı
29	1D	35	GS	Grup ayırıcı
30	1E	36	RS	Kayıt ayırıcı
31	1F	37	ABD	Birlik ayırıcı
32	20	40	SP	Boşluk
33	21	41	!	
34	22	42	"	Çift tırnak
35	23	43	#	
36	24	44	\$	

devam ediyor

TABLO A-1ASCII Kodları (devam ediyor)

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
37	25	45	%	
38	26	46	&	
39	27	47	'	Apostrof
40	28	50	(	
41	29	51	)	
42	2A	52	*	
43	2B	53	+	
44	2C	54	,	Virgül
45	2D	55	–	Eksi
46	2E	56	.	
47	2F	57	/	
48	30	60	0	
49	31	61	1	
50	32	62	2	
51	33	63	3	
52	34	64	4	
53	35	65	5	
54	36	66	6	
55	37	67	7	
56	38	70	8	
57	39	71	9	
58	3A	72	:	İki Nokta
59	3B	73	;	Noktalı Virgül
60	3C	74	<	
61	3D	75	=	
62	3E	76	>	
63	3F	77	?	

devam ediyor

TABLO A-1ASCII Kodları (devam ediyor)

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
64	40	100	@	
65	41	101	A	
66	42	102	B	
67	43	103	C	
68	44	104	D	
69	45	105	E	
70	46	106	F	
71	47	107	G	
72	48	110	H	
73	49	111	I	
74	4A	112	J	
75	4B	113	K	
76	4C	114	L	
77	4D	115	M	
78	4E	116	N	
79	4F	117	O	
80	50	120	P	
81	51	121	Q	
82	52	122	R	
83	53	123	S	
84	54	124	T	
85	55	125	U	
86	56	126	V	
87	57	127	W	
88	58	130	X	
89	59	131	Y	
90	5A	132	Z	

devam ediyor

TABLO A-1ASCII Kodları (devam ediyor)

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
91	5B	133	[	Açık parantez
92	5C	134	\	Tersten eğik çizgi
93	5D	135	]	Kapalı parantez
94	5E	136	^	Şapka
95	5F	137	_	Alt çizgi
96	60	140	`	Açık aksan
97	61	141	a	
98	62	142	b	
99	63	143	c	
100	64	144	d	
101	65	145	e	
102	66	146	f	
103	67	147	g	
104	68	150	h	
105	69	151	i	
106	6A	152	j	
107	6B	153	k	
108	6C	154	l	
109	6D	155	m	
110	6E	156	n	
111	6F	157	o	
112	70	160	p	
113	71	161	q	
114	72	162	r	
115	73	163	s	
116	74	164	t	
117	75	165	u	

devam ediyor

TABLO A-1ASCII Kodları (devam ediyor)

Ondalık	Onaltılık	Sekizlik	Sembol	Yorumlama
118	76	166	v	
119	77	167	w	
120	78	170	x	
121	79	171	y	
122	7A	172	z	
123	7B	173	{	Açık süslü parantez
124	7C	174		Çubuk
125	7D	175	}	Kapatma parantezi
126	7E	176	~	Tilde
127	7F	177	OF	Sil

TABLO A-1ASCII Kodları (devam ediyor)

A.2 ASCII Tablosu (Kısa Form)

Sağ Sol	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	HÜKÜM	SOH	STX	VESAYET	EOT	ENYA	ACK	BEL	BS	HT	LF	VT	FF	CR	BU YÜZDE	END
1	BUNA	DC1	DC2	DC3	DC4	İSTEK	SEN	ETB	YAP	İÇİNDE	ALT	KAÇ	FS	GS	RS	ABD
2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	OF

TABLO A-2Kısa ASCII Tablosu (Onaltılık)

Ek B

Yapı Grafikler

Bu ek, bu kitapta kullanılan yapı grafik kavramlarını ve stillerini belgelemektedir. Temel sembololoji ile ilgili bazıkılavuzlar içermektedir. Açıkça, bu bir tasarım süreci eğitimi değildir.

Yapı şeması, bir program için birincil tasarım aracıdır. Bir tasarım aracı olarak, programınızı yazmaya başlamadan önce kullanılır. Kodlamaya başlamadan önce tasarım yapmanın önemini anlamana yardımcı olabilecek bir benzetme vardır.

Bir ev inşa etmeye karar verdiğiniz varsayın. Tam olarak ne istediğinizi düşünmek için çok zaman harcayacaksınız. Kaç odaya ihtiyacı var? Aile odası mı yoksa büyük bir oda mı istiyorsunuz? Çamaşırhane evin içinde mi yoksa garajda mı olmalı? Herkesin ne istediğinizi anlamasını sağlamak için, her şeyi ayrıntılı bir şekilde tanımlayan resmi planlar hazırlarsınız. Çocuğunuz için bir oyuncak ev veya arka bahçeniz için bir alet kulübesi gibi küçük bir şey inşa etseniz bile, bazı taslaklar veya planlar yaparsınız.

Evde ne istediğinizi belirlemek, büyük bir sistemin gereksinimlerini belirlemeye benzer. Bir dizi plan hazırlamak, bir programın tasarımında ki yapı şemasına paraleldir. Hepsi önceden planlama gerektirir; yalnızca ayrıntı düzeyi değişir.

Profesyoneller yapı şemasını başka bir amaç için kullanır. Bir proje ekibi ortamında çalıştığınızda, programınızı yazmaya başlamadan önce tasarımınızın gözden geçirilmesi gerekir. Bu gözden geçirme sürecine *yapılandırılmış yürüyüş* denir. Gözden geçirme ekibi, projenizin alanından sorumlu sistem analisti, kullanıcı topluluğunun bir temsilcisi, bir sistem test mühendisi ve projeden bir veya iki programcıdan oluşur.

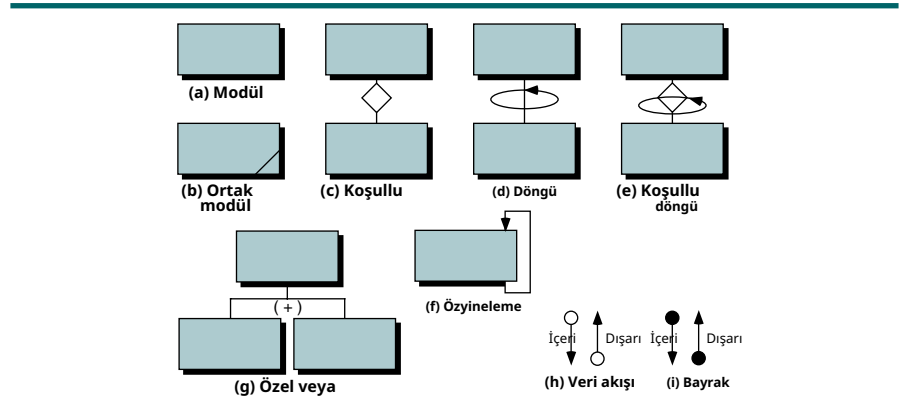
Tasarım yürüyüşünüz üç amaca hizmet eder. İlk olarak, tasarımınızı ekibe ileterek programınızın sistemle nasıl uyumlu olduğunu anlamana sağlar. Herhangi bir eksiklik veya iletişim hatası varsa, burada tespit edilmelidir. Programınızla arayüz oluşturma gereken programcıları davet ederseniz, yürüyüş aynı zamanda programlar arası iletişim bağlantılarının doğru olduğunu da garanti eder.

İkincisi, tasarımınızı doğrular. Tasarımınızı oluştururken, programınızı yazmak için birkaç alternatif yaklaşımı göz önünde bulundurmuş olacaksınız. İnceleme ekibi, düşündüğünüz farklı tasarımları görmeyi ve anlamayı bekler ve önerdiğiniz tasarımı neden seçtiğinizi duymak ister. Tasarımın bazı yönlerine meydan okuyacaklar ve belki de düşünmediğiniz yaklaşımlar önereceklerdir. İncelemenin sonucu, mümkün olan en iyi tasarımıdır.

Üçüncüsü, test mühendisinin programınızın test edilebilirliğini değerlendirme fırsatını verir. Bu adım, nihai programın sağlam ve mümkün olduğunca hata sız olmasını sağlar.

B.1 Yapı Şeması Sembolleri

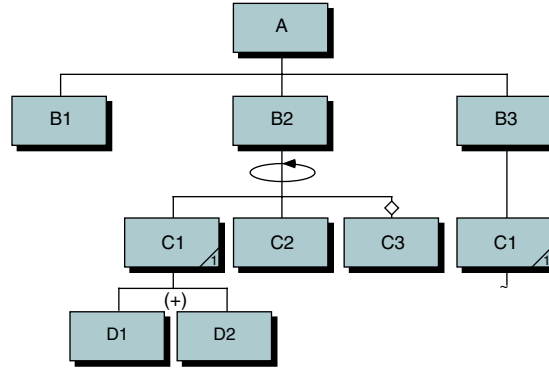
Şekil B-1, bir yapı şeması yazmak için kullanılan çeşitli sembolleri göstermektedir. Bu bölümde bu sembollerin her birini tanımlıyoruz. Sembollerin kendilerini tartışmanın yanı sıra, bir yapı şemasının nasıl okunacağı ve yapı şemalarınızda takip etmeniz gereken birkaç kuralı da ele alıyoruz.



ŞEKİL B-1Yapı Şeması Sembolleri

Modüller

Bir yapı şemasındaki her dikdörtgen (bkz. Şekil B-2) yazacağınız bir modülü temsil eder. Uygulama dilinin bir parçası olan, okuma, yazma ve karekök gibi modüller yapı şemanızda gösterilmez. Dikdörtgendeki isim, modüle verdiğiniz isimdir. Anlamlı olmalıdır. Akıllı isimler olarak bilinen yazılım mühendisliği prensibi, bir programda kullanılan isimlerin kendini belgeleyen olması gerektiğini belirtir; yani, okuyucuya niyet edilen kullanımını iletmelidir. Akıllı isimler, programınızdaki hem modüller hem de veri isimleri için kullanılmalıdır.



ŞEKİL B-2Yapı Şeması

Tüm isimlerin tanımlayıcı olması gerektiği halde, belirli bir program uygulamasından ziyade bir yapı şemasının formatına odaklanmak istediğimiz için kendi kuralımızı ihlal edeceğiz. Şekil B-2'de gördüğünüz isimler, tartışma için çeşitli semboller tanımlar.

Yapı şemaları yalnızca modül akışını gösterir; kod içermezler.

Bu noktada önemli bir yapı şeması kuralını tartışmak faydalıdır: bir yapı şemasında hiçbir kod bulunmaz. Bir yapı şeması yalnızca program içindeki modül akışını gösterir. Bu bir blok diyagramı veya akış diyagramı değildir. Programınızın haritası olarak, yapı şeması yalnızca modüllerin mantıksal akışını gösterir. Algoritma tasarımı, her modülün işini tam olarak nasıl yaptığını gösterir. Başka bir bakış açısı, bir yapı şemasının büyük resmi gösterdiği; detaylar algoritma tasarımına bırakılmıştır.

Yapı Şemalarını Okuma

Diğer semboller tartışmadan önce, yapı şemalarının nasıl okunacağına bakalım. Yapı şemaları *yukarıdan aşağıya, soldan sağa okunur*. Böylece, Şekil B-2'ye atıfta bulunarak, Modül A (en üstteki ilk dikdörtgen) üç alt modülden oluşur, B1, B2 ve B3. Soldan sağa kuralına göre, programdaki ilk çağrı B1'dir. B1 tamamlandıktan sonra, program B2'yi çağırır. B2 tamamlandığında, program B3'ü çağırır. Diğer bir deyişle, bir yapı şemasındaki aynı seviyedeki modüller soldan sağa doğru sırayla çağrılır.

Yukarıdan aşağıya kavramı B2 ile gösterilmektedir. B2 çağrıldığında, sırasıyla C1, C2 ve C3'ü çağırır. Ancak, Modül C2, C1 tamamlanmadan çalışmaya başlamaz. C1 çalışırken, D1 veya D2'yi çağırır. Diğer bir deyişle, C1'den D2'ye kadar olan tüm modüller, Modül C2 başlamadan önce çağrılmalıdır.

Ortak Modüller

Şekil B-2'de 10 modül bulunmaktadır. Ancak bunlardan ikisi aynı modüldür (c1). Bir modül tasarımınızda birden fazla yerde kullanıldığında, tekrarlanır. Başka bir deyişle, bir program içinde birkaç farklı yerden ortak bir modülü çağırmanız gerekiyorsa, her çağrı yapının şeması içinde bir dikdörtgenle temsil edilir. Ortak bir modülü tanımlamak için, dikdörtgenin sağ alt köşesine bir çizgi çizin; Şekil B-1(b)'ye bakın. Birden fazla modül birden fazla kez kullanılıyorsa, her modül için benzersiz bir numara eklemek faydalıdır.

Bu tekniği göstermek için, bu örnekte gerekli olmasa da Şekil B-2'de C1 numaralandırılmıştır.

Ortak modüller hakkında son bir nokta: Ortak modüller birkaç başka modülü çağırdığında, tasarımları oldukça büyük hale gelebilir. Ortak bir modül her çağrıldığında tüm alt modülleri yeniden çizmek yerine, modüllerin çağrıldığını belirtmek için dikdörtgenin altına bir çizgi ve bir kesme (~) sembolü ekleyebilirsiniz. Bu kavram da Şekil B-2'de gösterilmektedir. Bu varsayımsal tasarımda, Modül C1 hem Modül B2 hem de Modül B3 tarafından çağrılmaktadır.

Ancak c1 tasarımını tekrarlamak yerine, B3'ten çağırdığımızda diğer modülleri çağırdığını kesme sembolü ile belirtiriz.

Koşullu Çağrılar

Şekil B-2'de, Modüller B1, B2 ve B3 her zaman çağrılır. Ancak, genellikle bir modül bazen çağrılır ve bazen atlanır. Buna *koşullu çağrı* diyoruz. Bu, Modül B2'nin Modül C3'ü çağırdığı Şekil B-2'de gösterilmektedir. Bir modülü koşullu olarak tanımlamak için, bir seçim ifadesini belirtmek için akış diyagramında kullanılan aynı sembol olan küçük bir elmas kullanıyoruz. Bu tasarımı uygulayan kod, aşağıda gösterildiği gibi basit bir if ifadesidir.

```
1 if (koşul)
  1 C3'ü ara
2 eğer bitir
```

Özel Veya

İki veya daha fazla modül genellikle özel olarak çağrılır. Örneğin, Şekil B-2'deki programımızın tasarımı, ya D1 ya da D2'yi çağırmamızı gerektirir, ancak her ikisini birden değil. Bu tasarım, Modül c1'de aşağıdaki ifadelerle uygulanabilir:

```
1 if (koşul)
  1 D1'i ara
2 aksi takdirde
  2 D2'yi ara
3 eğer bitir
```

Koddan, ya D1 ya da D2 çağrıldığıının ama her ikisinin de çağrılmadığının açık olduğu görülmektedir. Bu tasarımı yapı şemasında tanımlamak için bir yol bulmamız gerekiyor. Özel çağrıları belirtmek için kullandığımız sembol (+) dir. Birden fazla çağrı özel olarak gruplandırıldığında, örneğin çok yönlü bir seçimde (C'deki bir *switch* veya iç içe *if* ifadeleri gibi), aynı sembolü kullanırız.

Döngüler

Çoğu zaman bir tasarım, bir modülün bir döngü içinde çağrılmasını gerektirir. Örneğin, Şekil B-2'de, B2 bir dosyayı işlerken C1, C2 ve C3 çağırıldığını varsayalım. Bu nedenle, tüm veriler okunana kadar döngü yapmalıdır. Döngü sembolü bir ovaldir. Ovalimizde bir ok kullanıyoruz, ancak bu zorunlu değildir. Döngümüzün uygulama kodu aşağıdaki kod parçasına benzer bir şey olacaktır:

```

1  döngü (dosya sonu değil)
    1  C1'i çağır
    2  C2'yi çağır
    3  if (koşul)
        1  C3'ü ara
    4  eğer bitir
2  döngüyü bitir

```

Koşullu Döngüler

Bazen koşullu olarak döngü yaparız. Örneğin, bir arama yapmadan önce null bağlı listeyi test ettiğimiz aşağıdaki kod parçasını düşünün. Bu kod için yapı şeması sembolü, koşullu elmas ile döngü ovalini birleştirir; bu Şekil B-1(e)'de gösterilmiştir.

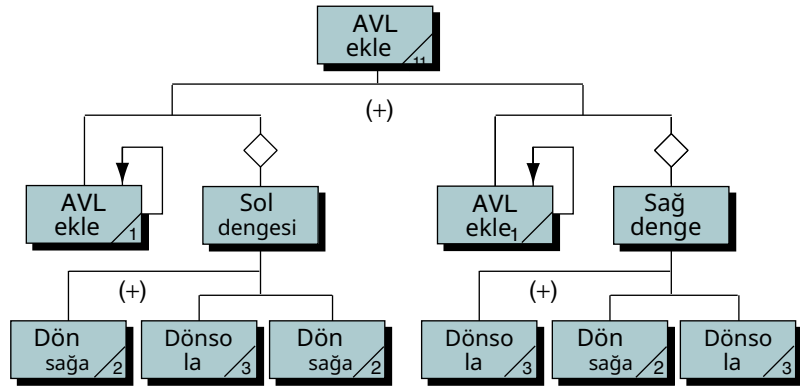
```

1  eğer (liste->ilk null değilse)
    1  yürüteç = liste->ilk
    2  döngü (walker null değil)
        1  ...
2  eğer bitir

```

Özyineleme

Modül sembollerinin sonuncusu, özyinelemeli bir modülü temsil eder. Bu, Şekil B-1(f)'de gösterilmiştir. Özyineleme sembolünün bir kullanımını görmek ve yapı şemasının son bir örneği olarak, Bölüm 8'de gösterilen AVL ağacı ekleme tasarımını düşünün. Bu, Şekil B-3'te tekrarlanmaktadır. Bu yapı şeması, bir döngü hariç tüm olası yapı şeması sembollerini içerir. Ayrıca bazı ilginç tasarım noktalarını sunar.



ŞEKİL B-3Yapı Şeması Tasarımı

Modülde, her ikisi de ikinci seviyede olmak üzere iki özyinelemeli çağrı olduğunu unutmayın. İlk çağrının bir parametre olarak sol alt ağacı ve ikinci çağrının bir parametre olarak sağ alt ağacı olduğu için iki çağrıya ihtiyacımız var. Bu şekilde, dikdörtgenin altına kadar gelmeyen özyineleme sembolünün isteğe bağlı bir varyasyonunu kullandık. Ayrıca, modülün adını tanımlayan üst dikdörtgende özyineleme sembolünün genellikle kullanılmadığını, ancak bunun yanlış olarak kabul edilmediğini de belirtmelisiniz.

Bu tasarımda, ağacın ya sol ya da sağ dalına bir düğüm ekliyoruz. (Henüz ağaçları incelememişseniz, aşağıdaki tartışmayı anlamakta biraz zorlanabilirsiniz.) Bu, özel bir veya tasarım olduğu için, iki dikey çizgi arasında (+) yerleştiriyoruz.

Sol tarafa ekleme tasarımına baktığımızda, bir özyinelemeli çağrı ve ardından ağacı dengelemek için bir koşullu çağrı olduğunu görüyoruz. Ekleme den sonra ağaç hala dengedeyseniz, dengelemeye gerek yoktur. Bu iki çağrının bir arada gruplandığını göstermek için, onları yatay bir çizgi ile bağlarız; bu da özel veya tasarımını ayırır.

Şimdi *Sol denge* tasarımına bakalım. Yine, bu sefer sağa tek bir dönüşü veya sola ve sağa çift döngü olduğunu gösteren bir özel veya görüyoruz. Yine, tasarım adımlarını gruplamak ve özel veya mantığını izole etmek için yatay çizginin kullanımına dikkat edin.

Veri Akışları ve Bayraklar

Bu noktada, veri akışları ve bayraklar hariç tüm sembolleri kapsadık. Genel bir kural olarak, bunları kullanmıyoruz çünkü hızla bozuluyorlar ve genellikle bakım yapılmıyor. Tamlik açısından burada dahil ediyoruz.

Veri akışları ve bayraklar, modüle iletilen ve modülden alınan parametreleri temsil eder. Giriş parametreleri, modülleri bağlayan dikey çizginin solunda gösterilir ve çıkış parametreleri çizginin sağında gösterilir. Tek olan

Aralarındaki sembolik fark, veri akışlarının boş bir daireye sahip olmasıdır; oysa bayraklar dolu bir daireye sahiptir. Her iki durumda da, veri akışlarını n ve bayrakların adları yapı grafiğinde yer almalıdır.

B.2 Yapı Grafiği Kuralları

Bu bölümde açıklanan kurallar aşağıda özetlenmiştir:

1. Bir yapı grafiğindeki her dikdörtgen, programcı tarafından yazılmış bir modül ü temsil eder. Dil çeviricisinin bir parçası olarak sağlanan standart modüller d ahiil değildir.
2. Dikdörtgen içindeki isim, modülün amacını ileten akıllı bir isimdir. Bu, modülün kodlanmasında kullanılan isimdir.
3. Yapı şeması yalnızca modül akışını içerir. Hiçbir kod belirtilmemiştir.
4. Ortak modüller, modülün dikdörtgeninin sağ alt köşesinde çapraz hat veya gölgeleme ile gösterilir.
5. Ortak çağrılar, programda bulundukları her yerde bir yapı içinde gösterilir. Eğer alt modül çağrıları içeriyorlarsa, tam yapı yalnızca bir kez gösterilmelidir.
6. Veri akışları ve bayraklar isteğe bağlıdır. Kullanıldıklarında, adlandırılmalıdırlar.
7. Giriş akışları ve bayrakları dikey çizginin solunda; çıkış akışları ve bayrakları sağında gösterilir.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Ek C

Tamsayı ve Float Kütüphaneleri

Bu ek, daha önemli iki C kütüphanesini belgeler. Burada gösterilen değerlerin yalnızca temsilci olduğunu ve donanımdan donanıma değiştiğini unutmayın. Kütüphaneler, genellikle onaltılık olarak ifade edilen biçimlendirilmemiş sayılar (virgül yok) içerir. Okunabilirlik için ondalık biçimlendirilmiş sayılar kullanıyoruz.

C.limits.h

Tablo C-1, tam sayı türleri için donanım spesifik değerleri içermektedir.

Tanımlayıcı	Anlam	Minimum Değer
CHAR_BIT	bir char'daki bit sayısı	8
SCHAR_MIN	kısa char minimum	-127
SCHAR_MAX	kısa char maksimum	127
UCHAR_MAX	unsigned char maksimum	255
CHAR_MIN	char minimum	Bakınız SCHAR_MIN
CHAR_MAX	char maksimum	Bakınız SCHAR_MAX
SHRT_MIN	kısa int minimum	-32,767
SHRT_MAX	kısa int maksimum	32,767
USHRT_MAX	unsigned short maksimum	65,535

devam ediyor

TABLO C-1 Sınırlar Kütüphanesinin Kısmi İçeriği

Tanımlayıcı	Anlam	Minimum Değer
INT_MIN	int minimum\i	-32,767
INT_MAX	nt maksimum\	32,767
UINT_MAX	unsigned int maksimum	65,535
LONG_MIN	uzun minimum	-2,147,483,647
LONG_MAX	uzun maksimum	2,147,483,647
ULONG_MAX	signed uzun maksimum	4,294,967,295
LLONG_MIN	long long int minimum	$-(2^{63} - 1)$
LLONG_MAX	long long int maksimum	$2^{63} - 1$
ULLONG_MAX	signed long long maksimum	$2^{64} - 1$

TABLO C-1Limit Kütüphanesinin Kısmi İçeriği (devam ediyor)

C.2float.h

Tablo C-2, kayan nokta türleri için donanım spesifik değerler içermektedir.

Tanımlayıcı	Anlam	Minimum Değer
FLT_DIG	kesirli basamak sayısı	6
DBL_DIG		15
LDBL_DIG		10
DECIMAL_DIG	ondalık basamaklar float değeri temsil etmek için gerekli	10
FLT_MANT_DIG	mantissa boyutu	yok
DBL_MANT_DIG		yok
LDBL_MANT_DIG		yok
FLT_MIN_EXP	negatif üslü için en büyük tam sayı (float radix)	yok
DBL_MIN_EXP		yok
LDBL_MIN_EXP		yok

devam ediyor

TABLO C-2Float Kütüphanesinin Kısmi İçeriği

Tanımlayıcı	Anlam	Minimum Değer
FLT_MIN_10_EXP DBL_MIN_10_EXP LDBL_MIN_10_EXP	negatif üslü için en büyük tam sayı(10 tabanı)	-37 -37 -37
FLT_MAX_EXP DBL_MAX_EXP LDBL_MAX_EXP	pozitif üslü için en büyük tam sayı (float radix)	37 37 37
FLT_MAX_10_EXP DBL_MAX_10_EXP LDBL_MAX_10_EXP	pozitif üstel için en büyük tamsayı(10 tabanı)	37 37 37
FLT_MAX DBL_MAX LDBL_MAX	mümkün olan en büyük kayan nokta sayısı	10^{37} 10^{37} 10^{37}
FLT_MIN DBL_MIN LDBL_MIN	mümkün olan en küçük kayan nokta sayısı	10^{-37} 10^{-37} 10^{-37}

TABLO C-2Float Kütüphanesinin Kısmi İçeriği(devamı)

Bu sayfa kasıtlı olarak boş bırakılmıştır

Ek D

Seçilen C Kütüphaneleri

Bu ek bölümde, C Dilinde bulunan standart işlevlerin çoğunu listelemekt eyiz. İlgili işlevlerin bir arada gruplandığı kütüphanelerine göre grupland ırdık. Ayrıca, sizin için kolaylık olması açısından D.1 Bölümünde alfabetik olarak listeledik. Tüm işlevlerin metinde yer almadığını ve bu ek bölümd e yer almayan kütüphanelerde işlevler olduğunu unutmayın.

D.1 İşlev Dizini

Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane
<i>_Çıkış</i>	671	<i>stdlib</i>	<i>forit</i>	670	<i>stdio</i>	<i>kalan</i>	667	<i>matematik</i>
<i>iptal</i>	671	<i>stdlib</i>	<i>getc</i>	669	<i>stdio</i>	<i>geri kalan</i>	667	<i>matematik</i>
<i>karın kasları</i>	671	<i>stdlib</i>	<i>getchar</i>	669	<i>stdio</i>	<i>eşit</i>	667	<i>matematik</i>
<i>aco</i>	664	<i>matematik</i>	<i>alır</i>	670	<i>stdio</i>	<i>kalıntı</i>	667	<i>matematik</i>
<i>aosf</i>	664	<i>matematik</i>	<i>gmtime</i>	674	<i>zaman</i>	<i>Yine de</i>	667	<i>matematik</i>
<i>acosl</i>	664	<i>matematik</i>	<i>oyynamak</i>	666	<i>matematik</i>	<i>kaldır</i>	670	<i>stdio</i>
<i>asctime</i>	674	<i>zaman</i>	<i>mantık</i>	666	<i>matematik</i>	<i>yeniden adlandır</i>	670	<i>stdio</i>
<i>tuz</i>	664	<i>matematik</i>	<i>kayıt</i>	666	<i>matematik</i>	<i>geri sar</i>	670	<i>stdio</i>
<i>asinf</i>	664	<i>matematik</i>	<i>buz salonu</i>	664	<i>C tipi</i>	<i>koşar</i>	667	<i>matematik</i>
<i>asil</i>	664	<i>matematik</i>	<i>sülfa</i>	664	<i>C tipi</i>	<i>rintf</i>	667	<i>matematik</i>
<i>amaç için</i>	664	<i>matematik</i>	<i>isascii</i>	664	<i>C tipi</i>	<i>rintl</i>	667	<i>matematik</i>
<i>atanf</i>	664	<i>matematik</i>	<i>kontrol</i>	664	<i>C tipi</i>	<i>yuvarla</i>	667	<i>matematik</i>

devam ediyor

Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane
atan1	664	matematik	rakam m1	664	C tipi	yuvarlaf	667	matematik
O bize geldi	665	matematik	grafik mi	664	C tipi	yuvarlal	667	matematik
atan2f	665	matematik	küçükharf	664	C tipi	ölçekbn	667	matematik
atan2l	665	matematik	yazdır	664	C tipi	ölçekbnf	667	matematik
çıkış	671	stdlib	noktalama	664	C tipi	ölçekbnl	667	matematik
atof	672	stdlib	boşluk	664	C tipi	ölçeklenebilirlik	667	matematik
römork	672	stdlib	aşam yemeği	664	C tipi	ölçekblnf	667	matematik
atol	672	stdlib	isxdigit	664	C tipi	ölçekblnl	667	matematik
atol	672	stdlib	laboratuvarlar	671	stdlib	tarama	669	stdio
kalloc	671	stdlib	Labs	671	stdlib	günah	668	matematik
tavan	665	matematik	ldexp	666	matematik	sınıf	668	matematik
tavan	665	matematik	ldexpf	666	matematik	günah	668	matematik
hücre	665	matematik	ldexpl	666	matematik	doğmak	668	matematik
hata temizle	668	stdio	ldiv	671	stdlib	Singhf	668	matematik
saat	673	zaman	lliv	671	stdlib	Sinhala	668	matematik
kosinüs	665	matematik	tüysüz	666	matematik	sürat koşusu	669	stdio
kosinüs f	665	matematik	tüysüz	666	matematik	snprintf	669	stdio
çünkü	665	matematik	tüysüz	666	matematik	kare	668	matematik
para	665	matematik	yazı	666	matematik	sqrtf	668	matematik
coshf	665	matematik	lrintf	666	matematik	karekök	668	matematik
koshl	665	matematik	lrintl	666	matematik	küçük parça	671	stdlib
ctime	674	zaman	baskı	666	matematik	sscanf	669	stdio
fark zamanı	673	zaman	llrintf	666	matematik	berbat	673	sicim
div	671	stdlib	llrintl	666	matematik	strchr	673	sicim
çıkış	671	stdlib	yerel zaman	674	zaman	strcmp	673	sicim
_Çıkış	671	stdlib	kayıt	666	matematik	strcpy	673	sicim
deneyim	665	matematik	logf	666	matematik	strcspn	673	sicim
ifade	665	matematik	günlük	666	matematik	strftime	674	zaman
açıkla	665	matematik	log2	666	matematik	streyn	673	sicim
deney1	665	matematik	log2f	666	matematik	strncat	673	sicim
expm1f	665	matematik	log2l	666	matematik	strncmp	673	sicim

devam ediyor

Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane	Fonksiyon	Sayfa	Kütüphane
<i>deney1l</i>	665	matematik	<i>günlük10</i>	666	matematik	<i>strncpy</i>	673	<i>sicim</i>
<i>exp2</i>	665	matematik	<i>log10f</i>	666	matematik	<i>stripbrk</i>	673	<i>sicim</i>
<i>deneyim2f</i>	665	matematik	<i>log10l</i>	666	matematik	<i>strchr</i>	673	<i>sicim</i>
<i>deneyim2l</i>	665	matematik	<i>çevrede</i>	666	matematik	<i>strspn</i>	673	<i>sicim</i>
<i>muhteşem şeyler</i>	665	matematik	<i>çevre</i>	666	matematik	<i>strstr</i>	673	<i>sicim</i>
<i>muhteşem</i>	665	matematik	<i>yuvarlak</i>	666	matematik	<i>strtod</i>	672	<i>stdlib</i>
<i>muhteşem</i>	665	matematik	<i>her tarafta</i>	666	matematik	<i>strof</i>	672	<i>stdlib</i>
<i>fclose</i>	668	<i>stdio</i>	<i>her tarafta</i>	667	matematik	<i>strtok</i>	673	<i>sicim</i>
<i>feof</i>	668	<i>stdio</i>	<i>her tarafta</i>	667	matematik	<i>çöktü</i>	672	<i>stdlib</i>
<i>korku</i>	668	<i>stdio</i>	<i>malloc</i>	671	<i>stdlib</i>	<i>anlatılmış</i>	672	<i>stdlib</i>
<i>fgetc</i>	669	<i>stdio</i>	<i>hafıza</i>	672	<i>sicim</i>	<i>strtoll</i>	672	<i>stdlib</i>
<i>fgets</i>	670	<i>stdio</i>	<i>memcmp</i>	672	<i>sicim</i>	<i>yatak</i>	672	<i>stdlib</i>
<i>zemin</i>	665	matematik	<i>memcpy</i>	673	<i>sicim</i>	<i>sistem</i>	672	<i>stdlib</i>
<i>zemin</i>	665	matematik	<i>hareket et</i>	673	<i>sicim</i>	<i>bronzluk</i>	668	matematik
<i>zemin</i>	665	matematik	<i>mktime</i>	673	zaman	<i>tanf</i>	668	matematik
<i>fmod</i>	665	matematik	<i>modf</i>	667	matematik	<i>Beklemek</i>	668	matematik
<i>fmodf</i>	665	matematik	<i>vs</i>	667	matematik	<i>şüpheli</i>	668	matematik
<i>fmodl</i>	665	matematik	<i>modfal</i>	667	matematik	<i>tanhf</i>	668	matematik
<i>açık</i>	669	<i>stdio</i>	<i>yakındaki</i>	667	matematik	<i>gezinti</i>	668	matematik
<i>serbest açık</i>	669	<i>stdio</i>	<i>yakındakiintf</i>	667	matematik	zaman	674	zaman
<i>fprintf</i>	669	<i>stdio</i>	<i>yakınıntl</i>	667	matematik	<i>geçiciDosya</i>	670	<i>stdio</i>
<i>fputc</i>	669	<i>stdio</i>	<i>güç</i>	667	matematik	<i>geçicişim</i>	670	<i>stdio</i>
<i>fput'lar</i>	670	<i>stdio</i>	<i>güç</i>	667	matematik	<i>kısalt</i>	668	matematik
<i>korku</i>	670	<i>stdio</i>	<i>kümes hayvanı</i>	667	matematik	<i>kısaltma</i>	668	matematik
<i>ücretsiz</i>	671	<i>stdlib</i>	<i>baskı</i>	669	<i>stdio</i>	<i>trunc1</i>	668	matematik
<i>frexp</i>	666	matematik	<i>putc</i>	669	<i>stdio</i>	<i>ton</i>	664	<i>C tipi</i>
<i>frexpf</i>	666	matematik	<i>putchar</i>	669	<i>stdio</i>	<i>daha düşük</i>	664	<i>C tipi</i>
<i>frexpl</i>	666	matematik	<i>koyar</i>	670	<i>stdio</i>	<i>büyük harf</i>	664	<i>C tipi</i>
<i>fscanf</i>	669	<i>stdio</i>	<i>rastgele</i>	671	<i>stdlib</i>	<i>gres vb.</i>	669	<i>stdio</i>
<i>fseek</i>	670	<i>stdio</i>	<i>yeniden tahsis</i>	671	<i>stdlib</i>			
<i>söyle</i>	670	<i>stdio</i>	<i>kalan</i>	667	matematik			

D.2 Tür Kütüphanesi

Aşağıdaki fonksiyonlar `cctype.h` içinde bulunur.

<i>buz salonu</i>	int	isalnum (int a_char);
<i>sülfa</i>	int	isalpha (int a_char);
<i>isascii</i>	int	isascii (int a_char); ^a
<i>kontrol</i>	int	isctrl (int a_char);
<i>rakam mı</i>	int	isdigit (int a_char);
<i>grafik mi</i>	int	isgraph (int a_char);
<i>küçük harf</i>	int	islower (int a_char);
<i>yazdır</i>	int	sürat koşusu (int a_char);
<i>noktalama</i>	int	ispunct (int a_char);
<i>boşluk</i>	int	isspace (int a_char);
<i>akşam yemeği</i>	int	isupper (int a_char);
<i>isxdigit</i>	int	isxdigit (int a_char);
<i>ton</i>	int	toint (int a_char);
<i>daha düşük</i>	int	tolower (int a_char);
<i>büyük harf</i>	int	toupper (int a_char);

a. Standart C değil. Çoğu uygulamada yer alan geleneksel uzantı.

D.3 Matematik Kütüphanesi

Aşağıdaki fonksiyonlar `math.h` içinde bulunur.

<i>aco</i>	çift	acos (çift sayı);
<i>acosf</i>	batmadan yüzmek	acosf (float sayı);
<i>acosl</i>	uzun çift	acosl (long double sayı);
<i>tuz</i>	çift	asin (çift sayı);
<i>asinf</i>	batmadan yüzmek	asinf (float sayı);
<i>asil</i>	uzun çift	asinl (long double sayı);
<i>amaç için</i>	çift	atan (çift sayı);
<i>atanf</i>	batmadan yüzmek	atanf (float sayı);
<i>atanl</i>	uzun çift	atanl (long double sayı);

devam ediyor

<i>O bize geldi</i>	çift	atan2 (double sayı1, double sayı2);
<i>atan2f</i>	batmadan yüzmek	atan2f (float sayı1, float sayı2);
<i>atan2l</i>	uzun çift	atan2l (long double sayı1, long double sayı2);
<i>tavan</i>	çift	ceil (double sayı);
<i>tavan</i>	batmadan yüzmek	ceilf (float sayı);
<i>hücre</i>	uzun çift	ceil (long double sayı);
<i>kosinüs</i>	çift	cos (çift sayı);
<i>kosinüs f</i>	batmadan yüzmek	cosf (float sayı);
<i>çünkü</i>	uzun çift	cosl (long double sayı);
<i>para</i>	çift	cosh (double sayı);
<i>coshf</i>	batmadan yüzmek	coshf (float sayı);
<i>koshl</i>	uzun çift	coshl (long double sayı);
<i>deneyim</i>	çift	exp (double sayı);
<i>ifade</i>	batmadan yüzmek	expf (float sayı);
<i>açıkla</i>	uzun çift	expl (long double sayı);
<i>deney1</i>	çift	expm1 (double sayı);
<i>expm1f</i>	batmadan yüzmek	expm1f (float sayı);
<i>deney1l</i>	uzun çift	expm1l (uzun çift sayı);
<i>exp2</i>	çift	exp2 (çift sayı);
<i>deneyim2f</i>	batmadan yüzmek	exp2f (float sayı);
<i>deneyim2l</i>	uzun çift	exp2l (uzun çift sayı);
<i>muhteşem şeyler</i>	çift	fabs (çift sayı);
<i>muhteşem</i>	batmadan yüzmek	fbsf (float sayı);
<i>muhteşem</i>	uzun çift	fbsl (uzun çift sayı);
<i>zemin</i>	çift	floor (çift sayı);
<i>zemin</i>	batmadan yüzmek	floorf (float sayı);
<i>zemin</i>	uzun çift	floorl (uzun çift sayı);
<i>fmod</i>	çift	fmod (çift sayı1, çift sayı2);
<i>fmodf</i>	batmadan yüzmek	fmodf (kayan sayı1, kayan sayı2);
<i>fmodl</i>	uzun çift	fmodel (uzun çift sayı 1, ve uzun çift sayı 2);

devam ediyor

<i>frexp</i>	çift	frexp (çift sayı, int* üssü);
<i>frexpf</i>	batmadan yüzmek	frexpf (float sayı, int* üssü);
<i>frexpl</i>	uzun çift	frexpl (uzun çift sayı, int* üssü);
<i>oyynamak</i>	çift	ilogb (çift sayı);
<i>mantık</i>	batmadan yüzmek	ilogbf (değişken sayı);
<i>kayıt</i>	uzun çift	ilogbl (uzun çift sayı);
<i>ldexp</i>	çift	ldexp (çift sayı, int kuvvet);
<i>ldexpf</i>	batmadan yüzmek	ldexpf (float sayı, int güç);
<i>ldexpl</i>	uzun çift	ldexpl (long double sayı, int güç);
<i>tüysüz</i>	çift	lint (double sayı);
<i>tüysüz</i>	batmadan yüzmek	lintf (float sayı);
<i>tüysüz</i>	uzun çift	lintl (long double sayı);
<i>yazı</i>	uzun	lrint (double sayı);
<i>lrintf</i>	uzun	lrintf (float sayı);
<i>lrintl</i>	uzun	lrintl (long double sayı);
<i>baskı</i>	uzun uzun	llrint (çift sayı);
<i>llrintf</i>	uzun uzun	llrintf (float sayı);
<i>llrintl</i>	uzun uzun	llrintl (long double sayı);
<i>kayıt</i>	çift	log (double sayı);
<i>logf</i>	batmadan yüzmek	logf (float sayı);
<i>günlük</i>	uzun çift	logl (long double sayı);
<i>log2</i>	çift	log2 (double sayı);
<i>log2f</i>	batmadan yüzmek	log2f (float sayı);
<i>log2l</i>	uzun çift	log2l (long double sayı);
<i>günlük10</i>	çift	log10 (double sayı);
<i>log10f</i>	batmadan yüzmek	log10f (float sayı);
<i>log10l</i>	uzun çift	log10l (uzun çift sayı);
<i>çevrede</i>	uzun	lround (çift sayı);
<i>çevre</i>	uzun	lroundf (float sayı);
<i>yuvarlak</i>	uzun	lroundl (uzun çift sayı);
<i>her tarafta</i>	uzun uzun	llround (çift sayı);

devam ediyor

<i>her tarafta</i>	uzun uzun	llroundf (float sayı);
<i>her tarafta</i>	uzun uzun	llroundl (uzun çift sayı);
<i>modf</i>	çift	modf (çift sayı, çift* integral);
<i>VS</i>	batmadan yüzmek	modff (float sayı, float* integral);
<i>modfal</i>	long double	modfl (long double sayı, long double* integral);
<i>yakındaki</i>	çift	yakint (double sayı);
<i>yakintf</i>	float	yakintf (float sayı);
<i>yakintl</i>	long double	yakintl (long double sayı);
<i>güç</i>	çift	pow (double taban, double kuvvet);
<i>güç</i>	batmadan yüzmek	powf (float taban, float kuvvet);
<i>kümes hayvanı</i>	long double	powl (long double taban, long double kuvvet);
<i>kalan</i>	çift	kalan (double sayı1, double sayı2);
<i>kalanf</i>	float	kalanf (float sayı1, float sayı2);
<i>kalanl</i>	long double	kalanl (uzun çift sayı1, uzun çift sayı2);
<i>eşit</i>	çift	remquo (çift sayı1, çift bölüm);
<i>kalıntı</i>	batmadan yüzmek	remquof (kayan sayı1, kayan bölüm);
<i>Yine de</i>	uzun çift	remquol (uzun çift sayı1, uzun çift bölüm);
<i>koşar</i>	çift	rint (çift sayı);
<i>rintf</i>	batmadan yüzmek	rintf (değişken sayı);
<i>rintl</i>	uzun çift	rintl (uzun çift sayı);
<i>yuvarla</i>	çift	yuvarlak (çift sayı);
<i>yuvarlaf</i>	batmadan yüzmek	yuvarlak (değişken sayı);
<i>yuvarlal</i>	uzun çift	yuvarlak (uzun çift sayı);
<i>ölçekbn</i>	çift	scalebn (çift sayı, int faktör);
<i>ölçekbnf</i>	batmadan yüzmek	Scalebnf (float sayısı, int faktörü);
<i>ölçekbnl</i>	long double	scalebnl (long double sayı, int faktörü);
<i>ölçeklenebilirlik</i>	çift	Scalebln (çift sayı, uzun faktör);
<i>ölçekbnlf</i>	batmadan yüzmek	Scalebnlf (float sayısı, uzun faktör);
<i>ölçekbnll</i>	uzun çift	ölçekbnll (uzun çift sayı, uzun faktör);

devam ediyor

<i>günah</i>	çift	sin	(çift sayı);
<i>sinf</i>	batmadan yüzmek	sinf	(float sayı);
<i>günah</i>	uzun çift	sinl	(long double sayı);
<i>doğmak</i>	çift	sinh	(çift sayı);
<i>Singhf</i>	batmadan yüzmek	sinhf	(float sayı);
<i>Sinhala</i>	uzun çift	sinhl	(long double sayı);
<i>kare</i>	çift	sqrt	(çift sayı);
<i>sqrtf</i>	batmadan yüzmek	sqrtf	(float sayı);
<i>karekök</i>	uzun çift	sqrtrl	(long double sayı);
<i>bronzluk</i>	çift	tan	(çift sayı);
<i>tanf</i>	batmadan yüzmek	tanf	(float sayı);
<i>Beklemek</i>	uzun çift	tanl	(uzun çift sayı);
<i>şüpheli</i>	çift	tanh	(çift sayı);
<i>tanhf</i>	batmadan yüzmek	tanhf	(float sayı);
<i>gezinti</i>	uzun çift	tanhl	(uzun çift sayı);
<i>kısalt</i>	çift	trunc	(çift sayı);
<i>kısaltma</i>	batmadan yüzmek	truncf	(float sayı);
<i>trunc1</i>	uzun çift	truncl	(uzun çift sayı);

D.4 Standart G/Ç Kütüphanesi

Sistem girdi/çıkı kütüphanesini (`stdio.h`) okunan veya yazılan veri t ürüne göre ayırdık.

Genel G/Ç

Genel girdi/çıkı, tüm dosyalara uygulanan fonksiyonları içerir.

<i>hata temizle</i>	geçersiz	clearerr (DOSYA* fp);
<i>fclose</i>	int	fclose (DOSYA* fp);
<i>feof</i>	int	feof (DOSYA* fp);
<i>korku</i>	int	ferror (DOSYA* fp);

devam ediyor

<i>açık</i>	DOSYA*	fopen (const char* dahili_adı, const char* dosya_modu);
<i>serbest açık</i>	DOSYA*	freopen (const char* dahili_adı, const char* dosya_modu, FILE* akışı);

Biçimlendirilmiş G/Ç

Metin verilerini iç bellek formatlarına dönüştür.

<i>fprintf</i>	int	fprintf (FILE* fileOut, const char* format_string, ...);
<i>baskı</i>	int	printf (const char* format_string, ...);
<i>sürat koşusu</i>	int	sprintf (char* to_loc, const char* format_string, ...);
<i>snprintf</i>	int	snprintf (char* to_loc, size_t n, const char* format_string, ...);
<i>fscanf</i>	int	fscanf (FILE* fileIn, const char* format_string, ...);
<i>tarama</i>	int	scanf (const char* format_string, ...);
<i>sscanf</i>	int	sscanf (const char* from_loc, const char* format_string, ...);

Karakter G/Ç

Bir karakteri bir seferde oku ve yaz.

<i>fgetc</i>	int	fgetc (DOSYA* fp);
<i>fputc</i>	int	fputc (int char_out, FILE* fp);
<i>getc</i>	int	getc (DOSYA* fp);
<i>getchar</i>	int	getchar (boş);
<i>putc</i>	int	putc (int char_out, FILE* fp);
<i>putchar</i>	int	putchar (int char_out);
<i>gres vb.</i>	int	ungetc (int char_out, FILE* fp);

Dosya G/Ç

Bu fonksiyonlar ikili dosyalarla çalışır.

<i>korku</i>	size_t	fread (void* alan_çinde, size_t boyut, size_t sayı, FILE* fp);
<i>fseek</i>	int	fseek (DOSYA* fp, uzun konum, int from_loc);
<i>söyle</i>	uzun	ftell (DOSYA* fp);
<i>forit</i>	size_t	fwrite (const void* out_data, size_t boyut, size_t sayısı, FILE* fp);
<i>geri sar</i>	geçersiz	geri sarma (DOSYA* fp);

String G/Ç

Dizeleri oku ve yaz.

<i>fgets</i>	karakter*	fgets (char* string, int boyut, FILE* fp);
<i>fput'lar</i>	int	fputs (const char* dize, FILE* fp);
<i>alır</i>	karakter*	alır (char* string);
<i>koyar</i>	int	koyar (const char* dize);

Sistem Dosya Kontrolü

Diskte dosya oluşturan ve silen sistem komutları.

<i>kaldır</i>	int	kaldır (const char* dosya_adi);
<i>yeniden adlandır</i>	int	yeniden_adlandır (const char* eski_ad, const char* yeni_ad);
<i>geçiciDosya</i>	DOSYA*	geçici_dosya (void);
<i>geçiciİsim</i>	karakter*	geçici_ad (char* dosya_adi);

D.5 Standart Kütüphane

Aşağıdaki fonksiyonlar `stdlib.h` içinde bulunur.

Matematik Fonksiyonları

Aşağıdaki matematik fonksiyonları `stdlib` içinde bulunur.

<i>karın kasları</i>	<code>int</code>	<code>abs (int sayı);</code>
<i>div</i>	<code>div_t</code>	<code>div (int pay, int bölen);</code>
<i>laboratuvarlar</i>	<code>uzun</code>	<code>labs (long sayı);</code>
<i>Labs</i>	<code>long long llabs (long sayı);</code>	
<i>ldiv</i>	<code>ldiv_t</code>	<code>ldiv (long pay, long bölen);</code>
<i>lliv</i>	<code>lldiv_t</code>	<code>lldiv (uzun uzun pay, uzun uzun bölen);</code>
<i>rastgele</i>	<code>int</code>	<code>rand (boş);</code>
<i>küçük parça</i>	<code>geçersiz</code>	<code>srand (işaretsiz tohum);</code>

Hafıza Fonksiyonları

Aşağıdakiler bellek tahsis fonksiyonlarıdır.

<i>kalloc</i>	<code>geçersiz*</code>	<code>calloc (boyut_t eleman_sayısı, boyut_t eleman_boyutu);</code>
<i>ücretsiz</i>	<code>geçersiz</code>	<code>ücretsiz (geçersiz*);</code>
<i>malloc</i>	<code>geçersiz*</code>	<code>malloc (boyut_t byte_sayısı);</code>
<i>yeniden tahsis</i>	<code>geçersiz*</code>	<code>realloc (void* stge_ptr, boyut_t eleman_boyutu);</code>

Program Kontrolü

Aşağıdaki fonksiyonlar program akışını kontrol eder.

<i>iptal</i>	<code>geçersiz</code>	<code>iptal et (geçersiz);</code>
<i>çıkış</i>	<code>int</code>	<code>atexit (void (*) fonksiyon_adı (void));</code>
<i>çıkış</i>	<code>geçersiz</code>	<code>çıkış (int çıkış_kodu);</code>
<i>_Çıkış</i>	<code>geçersiz</code>	<code>_Çıkış (int çıkış_kodu);</code>

Sistem İletişimi

Aşağıdaki fonksiyon işletim sistemi ile iletişim kurar.

<i>sistem</i>	int	system(const char* sistem_komutu);
---------------	-----	------------------------------------

Dönüşüm Fonksiyonları

Aşağıdaki fonksiyonlar verileri bir türden diğerine dönüştürür.

<i>atof</i>	çift	atof (const char* dize);
<i>atol</i>	uzun	atol (const char* dize);
<i>atoi</i>	int	atoi (const char* dize);
<i>strtod</i>	çift	strtod (const karakter* dizi, karakter** sonraki_dizge);
<i>strtof</i>	batmadan yüzmek	strtof (const karakter* dizi, karakter** sonraki_dizge);
<i>çöktü</i>	uzun	strtol (const char* str, char** next_str, int base);
<i>anlatılmış</i>	uzun çift	strtold (const char* str, char** next_str);
<i>strtoll</i>	uzun uzun	strtoll (const char* str, char** next_str, int taban);
<i>yatak</i>	işaretsiz uzun	strtoul (const char* str, char** next_str, int taban);
<i>strtoull</i>	işaretsiz uzun uzun	strtoull (const char* str, char** next_str, int taban);

D.6 Dize Kütüphanesi

Aşağıdaki fonksiyonlar `string.h` içinde bulunur.

Hafıza Fonksiyonları

Aşağıdaki fonksiyonlar bellekte blok verileri işler.

<i>hafıza</i>	geçersiz*	memchr (const void* mem, int a_char, boyut_t bayt);
<i>memcmp</i>	int	memcmp (const void* mem1, const void* mem2, boyut_t bayt);

devam ediyor

<i>memcpy</i>	geçersiz*	memcpy (void* to_mem, const void* fr_mem, size_t bayt);
<i>hareket et</i>	geçersiz*	memmove (void* to_mem, const void* fr_mem, size_t bayt);

String Fonksiyonları

Aşağıdaki fonksiyonlar dizeleri işler.

<i>berbat</i>	karakter*	strcat (char* to_str, const char* fr_str);
<i>strchr</i>	karakter*	strchr (const char* str, int a_char);
<i>strcmp</i>	int	strcmp (const char* str1, const char* str2);
<i>strcpy</i>	karakter*	strcpy (char* to_str, const char* fr_str);
<i>strlen</i>	size_t	strlen (const char* str);
<i>strncat</i>	karakter*	strncat (char* to_str, const char* fr_str, size_t bayt);
<i>strncmp</i>	int	strncmp (const char* str1, const char* str2, size_t bayt);
<i>strncpy</i>	karakter*	strncpy (char* to_str, const char* fr_str, size_t bayt);
<i>strpbrk</i>	karakter*	strpbrk (const char* str1, const char* str2);
<i>strcspn</i>	size_t	strcspn (const char* str1, const char* str2);
<i>strrchr</i>	karakter*	strrchr (const char* str, int a_char);
<i>strspn</i>	size_t	strspn (const char* str1, const char* str2);
<i>strstr</i>	karakter*	strstr (const char* str1, const char* str2);
<i>strtok</i>	karakter*	strtok (karakter* p1, const karakter* p2);

D.7 Zaman Kütüphanesi

Aşağıdaki fonksiyonlar `time.h` içinde bulunur.

<i>saat</i>	saat_t	saat (boş);
<i>difftime</i>	double	difftime (time_t time_start, time_t time_end);
<i>mktime</i>	zaman_t	mktime (struct tm* takvim_zaman);

devam ediyor

<i>zaman</i>	<code>zaman_t</code>	<code>zaman (zaman_t* num_zaman);</code>
<i>asctime</i>	<code>karakter*</code>	<code>asctime (const struct tm* takvim_zaman);</code>
<i>ctime</i>	<code>karakter*</code>	<code>ctime (const zaman_t* num_zaman);</code>
<i>gmtime</i>	<code>struct tm*</code>	<code>gmtime (const zaman_t* num_zaman);</code>
<i>yerel_zaman</i>	<code>struct tm*</code>	<code>localtime (const zaman_t* num_zaman);</code>
<i>strftimesize_t</i>		<code>strftime (char* str, size_t maxsize, const char* biçimi, const struct tm* zaman_gösterici);</code>

Matematiksel Seriler ve

Özyinelemeli İlişkiler

Bir algoritmanın verimliliğinin nasıl hesaplandığını anlamak için, matematiksel bir serinin nasıl çalıştığına dair temel bir anlayışa sahip olmanız gerekir. Çünkü algoritmik tasarımın büyük bir kısmı özyineleme ile ilgilidir, bu nedenle bir seri üzerindeki özyinelemenin etkisi de anlaşılmalıdır.

Bu ek, matematiksel serilerin ve özyinelemeli ilişkilerin arkasındaki dört temel kavramı keşfetmektedir: aritmetik seriler, geometrik seriler, harmonik seriler ve özyinelemeli ilişkiler.

E.1 Aritmetik Seriler

Matematikte bir aritmetik seri, Şekil E-1'de gösterildiği gibi tanımlanır.

$$S_a = \sum_{i=1}^n i \cdot x = x + 2x + \dots + (n-1)x + nx$$

ŞEKİL E-1 Aritmetik Seriler

Bu seriye gayri resmi bir çözüm sunmak için, önce normal sırasıyla ve ardından terimlerin ters sırasıyla iki kez yazıyoruz. Eşit terimler elde etmek için, Şekil E-2'de gösterildiği gibi bunları topluyoruz.

$$\begin{aligned}
 \text{Açık} &= \quad + \quad + \dots + (n-1)x + \quad \\
 \text{Açık} &= \quad nx + (n-1)x + \dots + \quad 2x + \quad x \\
 2S_a &= \underbrace{(n+1)x + (n+1)x + \dots + (n+1)x + (n+1)x}_{n \text{ terimler}}
 \end{aligned}$$

Sonuç: $\boxed{\text{Açık} = \frac{n(n+1)x}{2}}$

ŞEKİL E-2 Aritmetik Seri Çözümü

Örnek Aşağıdaki serinin toplamını bulun:

$$1 + 2 + 3 + 4 + \dots + 100$$

Bu, $x = 1$ ve $n = 100$ olan aritmetik seridir. Tüm sayıları toplamak mümkün olsa da, seri formülünü uygulamak çok daha kolaydır.

$$S_a = 100(101) / 2 = 5050$$

Örnek Bu örnekte x değerinin 1 olmadığı bir seri kullanıyoruz.

$$5 + 10 + 15 + 20 + \dots + 50$$

Bu seride, her sayının 5 arttığını görüyoruz, bu da $x = 5$ anlamına geliyor. Genel formülü kullanıyoruz

$$S_a = xn(n+1) / 2 = 5(10)(11) / 2 = 275$$

Örnek Son örneğimiz için, $x \neq 1$ değilken ve $n+1$ ile başlamıyorsa bir serinin toplamını buluyoruz. Aşağıdaki toplamı bulun:

$$100 + 105 + 110 + 115 + \dots + 200$$

Formülümüzü kullanabilmek için, bu seriyi iki serinin birleşimi olarak yazmamız gerekiyor.

$$(5 + 10 + 15 + \dots + 200) - (5 + 10 + 15 + \dots + 95)$$

İlk seri (5'ten 200'e kadar) $x = 5$ ve $n = 40$ olan aritmetik bir seridir. İkinci seri $x = 5$ ve $n = 19$ olan aritmetik bir seridir. Formülümüzü kullanarak, şunu görüyoruz ki

$$S_a = (5 + 200)(40) / 2 - 5(19)(20) / 2 = 3150$$

E.2 Geometrik Seriler

Matematikte geometrik bir seri Şekil E-3'te gösterildiği gibi tanımlanır.

$$S_g = \sum_{i=0}^n x^i = 1 + x + x^2 + \dots + x^{(n-1)} + x^n$$

ŞEKİL E-3 Geometrik Seriler

Yine, Şekil E-4'te gösterildiği gibi bu seriye gayri resmi bir çözüm sunuyoruz.

Given

$$S_g = 1 + x + x^2 + \dots + x^{(n-1)} + x^n$$

We multiply both sides by x

$$x \cdot S_g = x + x^2 + x^3 + \dots + x^n + x^{(n+1)}$$

We add and subtract 1 from right side

$$x \cdot S_g = \underbrace{1 + x + x^2 + x^3 + \dots + x^n}_{S_g} + x^{(n+1)} - 1$$

After substituting part of the right side with S_g

$$x \cdot S_g = S_g + x^{(n+1)} - 1$$

After reordering and factoring

$$S_g = \frac{x^{(n+1)} - 1}{x - 1}$$

ŞEKİL E-4 Geometrik Seri Çözümü

Örnek Aşağıdaki serinin toplamını bulun:

$$1 + 2 + 4 + 8 + \dots + 1024$$

Bunun, $x = 2$ olan bir geometrik seri olduğunu kabul ediyoruz (seri 2 'nin kuvvetleridir) ve $n = 10$ 'dur ($2^{10} = 1024$ 'tür). Geometrik formülü kullanarak,

$$S_g = (x^{n+1} - 1) / (x - 1) = (2^{11} - 1) / (2 - 1) = 2047$$

ÖrnekAşağıdaki serinin toplamını bulun:

$$1 + 1/2 + 1/4 + 1/8 + \dots$$

Bu, $x = 1/2$ olan geometrik formülün bir durumudur. Geometrik serinin bir özelliği, $x = 1$ 'den küçükse ve n sonsuza yaklaşıyorsa, yakınsar. Bu durumda $x^{n+1} \rightarrow 0$ olur çünkü 1 'den küçük bir sayının sonsuz kuvveti 0 'dır. Bu, $x = 1/2$ 'den küçük ve sonsuz olduğunda gösterilen bu örnekte gösterilmektedir.

$$\begin{aligned} S_g &= (x^{n+1} - 1) / (x - 1) \\ &= (-1) / (x - 1) \\ &= 1 / (1 - x) \\ &= 1 / (1 - 1/2) \\ &= 2 \end{aligned}$$

E.3 Harmonik Seri

Şekil E-5, bir harmonik seriyi göstermektedir. Bu seri yakınsar değil, bu da H_n değerinin n sonsuza yaklaştığında 0 'a yaklaşmadığı anlamına gelir. Bu serinin tam hesaplaması bu metnin kapsamının ötesindedir, ancak şekil üzerinde gösterilen yaklaşık değeri sağlıyoruz.

$$H_n = \sum_{i=1}^n \frac{1}{i} = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Result: $H_n \approx \ln n$

ŞEKİL E-5 Harmonik Seri

ÖrnekAşağıdaki serinin toplamını bulun.

$$1 + 1/2 + 1/3 + \dots + 1/100$$

Cevap $\ln 100$ veya 4.60 .

E.4 Özyinelemeli İlişkiler

Özyinelemeli algoritmaların karmaşıklığını bulmak için, bir özyinelemeli ilişkiyi çözmeye ihtiyacımız gerekiyor. Özyinelemeli bir ilişki, bir fonksiyonun değerinin kendi terimleri cinsinden tanımlandığı bir ilişkidir. Örneğin,

$$f(n) = f(n - 1) + 1$$

bir özyinelemeli ilişkidir çünkü $f(n)$ değerinin $f(n - 1)$ değerine bağlıdır. Bu tür ilişkileri çözmek için genel bir formül olmasa da, genel yaklaşım, n değeri için $f(n)$ değeri bulmaktır. $n = 1, n = 2, n = 3$ gibi terimleri geliştirmemizi gerektirir. Bu süreçte, sağ tarafta yalnızca $f(n)$ ve sol tarafta yalnızca sabitler olacak şekilde $f(i)$ ile ilgili terimleri ortadan kaldırmaya çalışırız. Bir ilişkinin çözülebilmesi için, bazı başlangıç değerlerinin f fonksiyonuna sağlanması gerekir. En yaygın verilen değer $f(0)$ veya $f(1)$ 'dir. Kavramı örnekler aracılığıyla heuristik olarak geliştiriyoruz.

Örnek Şekil E-6, bir özyinelemeli ilişkinin özyinelemeli çözümünü göstermektedir.

$$f(n) = \begin{cases} 0 & \text{eğer } n = 0 \\ f(n - 1) + 1 & \text{Aksi takdirde} \end{cases}$$

n 'yi -1 ile tekrar tekrar değiştiriyoruz

$$f(n) = f(n - 1) + 1$$

$$f(n - 1) = f(n - 2) + 1$$

$$f(n - 2) = f(n - 3) + 1$$

...

$$f(1) = f(0) + 1$$

Denklemleri topladığımızda, bazı terimler iptal edilir

$$f(n) = f(0) + \underbrace{1 + 1 + \dots + 1}_n$$

Temel durum, $f(0) = 0$, bu yüzden şunları elde ediyoruz

$$f(n) = n$$

ŞEKİL E-6 İlk Özyinelemeli İlişkinin Çözümü

Örnek Şekil E-7, başka bir özyinelemeli ilişkiyi ve çözümünü göstermektedir.

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ f(n-1) + n & \text{Otherwise} \end{cases}$$

We substitute n with $n-1$ repeatedly

$$\begin{aligned} f(n) &= f(n-1) + n \\ f(n-1) &= f(n-2) + n-1 \\ f(n-2) &= f(n-3) + n-2 \\ &\dots \\ f(1) &= f(0) + 1 \end{aligned}$$

When we add the equations, some terms are canceled

$$f(n) = \underbrace{f(0)}_0 + \underbrace{1 + 2 + \dots + n}_{\text{Arithmetic series}}$$

Result:
$$f(n) = \frac{n(n-1)}{2}$$

ŞEKİL E-7 İkinci Özyinelemeli İlişki için Çözüm

Örnek Şekil E-8, üçüncü bir özyinelemeli ilişkiyi göstermektedir.

$$f(n) = \begin{cases} 0 & \text{if } n = 1 \\ f(n/2) + 1 & \text{Otherwise} \end{cases}$$

Where n is 2^k

We substitute n with 2^k and repeatedly replace k by $k-1$

$$\begin{aligned} f(2^k) &= f(2^{k-1}) + 1 \\ f(2^{k-1}) &= f(2^{k-2}) + 1 \\ &\dots \\ f(2^1) &= f(2^0) + 1 \end{aligned}$$

If we add right sides and left sides, some terms are canceled

$$\begin{aligned} f(2^k) &= \underbrace{f(1)}_0 + \underbrace{1 + 1 + \dots + 1}_{k \text{ times}} \\ f(n) &= k = \log_2 n \end{aligned}$$

The value of k is $\log_2 n$

$$f(n) = \log_2 n$$

ŞEKİL E-8 $n \log n$ Serisi için Özyinelemeli Çözüm

Dizi Uygulamaları Yığı nlar ve Kuyruklar

3. Bölümde dinamik bellek ve bağlı liste uygulaması kullanarak bir yığın ADT'si geliştirdik. 4. Bölümde benzer şekilde bir kuyruk ADT'si geliştirdik. Ancak, bazen bir dizi uygulaması kullanmak isteriz veya buna ihtiyaç duyarız. Bu ek bölümde yığın ve kuyruk dizi uygulamalarını tartışıyoruz.

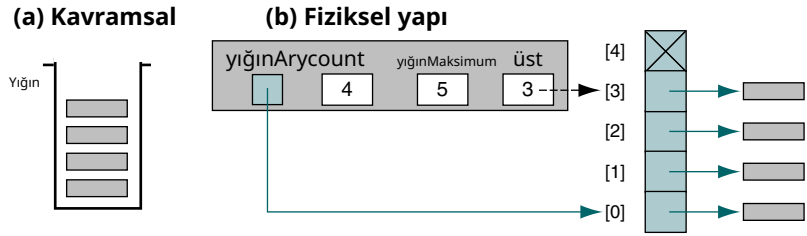
F. 1 Yığın ADT

Eğer bir yığının maksimum boyutu program yazılmadan önce hesaplanabiliyorsa, bir yığın için dizi uygulaması, bağlı liste kullanan dinamik uygulamadan daha verimlidir. Ayrıca, bir dizi yığını, bir yığın için daha kolay anlaşılır ve doğal bir görüntüdür.

Bir yığını bir dizide uygularken, taban ilk yığın elemanında, C'de indeks 0'da bulunur. Üst, veriler eklendikçe ve silindikçe dizide yukarı ve aşağı hareket eder. Yığına bir eleman eklemek için, üstü 1 artırır ve yeni veri için dizi indeksi olarak kullanırız. Yığından bir eleman çıkarmak için, üstteki indeks konumundaki veriyi kopyalar ve ardından üstten 1 çıkarırız.

Bir ek meta veri öğesi gereklidir: yığın içindeki maksimum eleman sayısı. Maksimum boyutu 5 eleman olan dizi uygulaması için yapı Şekil F-1'de gösterilmiştir.

Bu bölümde, bağlı liste uygulaması yerine bir dizi kullanarak sekiz temel yığın algoritmasını yeniden tasarlıyoruz.



ŞEKİL F-1 Yığın Dizi Uygulaması

Dizi Veri Yapısı

Dizi uygulaması için veri yapısında üç fark vardır:

ilk olarak, yığın üstü bir işaretçi yerine bir indeks; ikinci olarak, yığında izin veril en maksimum eleman sayısını saklamamız gerekir; ve üçüncüsü, bir sonraki al anlara ihtiyacımız yoktur. Yığın bir LIFO yapısı olduğundan, her elemanın bir önc ekine fiziksel bir komşuluğu vardır.

Veri yapısı önemli ölçüde farklı olsa da, bir yığının dizi uygulaması , bağlı liste uygulamasında kullanılan aynı temel algoritmaları gerekti rir. ADT bildirimi Program F-1'de gösterilmiştir. Algoritmalar, takip ed en bölümlerde geliştirilmiştir.

PROGRAM F-1 Yığın Dizi Tanımı

123456789	<pre>// Dizi Uygulaması için Yığın Tanımları typedef struct { void** yığın; int count; int stackMax; int üst; } YIĞIN;</pre>
1011121314151617181920	<pre>// Prototip Bildirimleri YIĞIN* createStack (int maxSize); YIĞIN* destroyStack (YIĞIN* kuyruk); bool pushStack (YIĞIN* yığın, void* itemPtr); void* popStack (YIĞIN* yığın); void* stackTop (YIĞIN* yığın); int stackCount (YIĞIN* yığın); bool emptyStack (YIĞIN* yığın); bool fullStack (YIĞIN* yığın); // Yığın ADT Tanımlarının Sonu</pre>

Yığın Oluştur

Yığın oluşturmanın ATD uygulaması Program F-2'de gösterilmiştir. Uygulamada programı, yığın oluşturulduğunda yığının maksimum boyutunu belirtir.

PROGRAM F-2Yığın Oluştur

```

1  /* ===== createStack =====
2  Bu algoritma, baş yapısını ve diziyi dinamik bellekten ayırarak
3  tamamı boş bir yığın oluşturur.
4  Ön   maxSize, maksimum eleman sayısıdır
5  Son   yığın baş yapısına işaretçi döner
6        -veya- taşma durumunda NULL
7  */
8  STACK* createStack (int maxSize)
9  {
10 // Yerel Tanımlar
11     STACK* yığın;
12
13 // İfadeler
14     yığın = (STACK*) malloc( sizeof (STACK));
15     if (!yığın)
16         NULL'u döndür;
17
18     // Baş tahsis edildi. Yığını başlat ve tahsis et.
19     yığın->count = 0;
20     yığın->top = -1;
21     stack->stackMax = maxSize;
22     stack->stackAry = (void**)calloc (stack->stackMax,
23                                     sizeof(void*));
24     if (!stack->stackAry)
25     {
26         ücretsiz (yığın); NU
27         LL değerini dönd
28         ür; } // if
29     yığını döndür;
30 } // createStack

```

Program F-2 AnaliziYığın oluşturma işleminde iki olası hata vardır; her ikisi de bellek tahsisi ile ilgilidir. 14. ifadede yığın başı y apısı için bellek tahsis ediyoruz. Eğer bu tahsis başarısız olursa, null işaretçi döndürüyoruz. Ard ndan, 22. ifadede, *calloc* kullanarak *void* işaretçilerin dizisini tahsis ediyoruz ve adresini yığın başı yapısında saklıyoruz. Yine, eğer bu tahsis başarısız olursa, null işaretçi döndürüyoruz. Baş yapıdaki yığın dizisinin, *void* işaretçilerin bir dizisine işaret eden bir işaretçi olduğunu unutmay ın. Bu, işaretçi türünü dönüştürdüğümüzde, bunun *void* işaretçiye işaret eden bir işaretçi olara k dönüştürülmesi gerektiği anlamına gelir.

Yeni oluşturulan yığın tanım gereği boştur, bu nedenle yığın üst indeksini-1 olarak ayarlıyoruz. Null bir yığın için -1 değerini kullanmalıyız çünkü içinde yalnızca bir öge bulunan bir yığınsıfır yığın üst üne sahiptir.

Yığına Ekle

Yığın yığını uygulamasının mantığı çok basittir. Öncelikle, yığındaki elemanın sayısını maksimum yığın boyutuyla karşılaştırarak taşmayı test ederiz. Eğer yığın doluysa, false döneriz, bu da eklemenin başarısız olduğunu gösterir. Eğer yer varsa, verileri yığına kopyalarız, üst indeksi ve yığın sayısını artırırız ve başarı için true döneriz. Fonksiyon kodu Program F-3'te gösterilmiştir.

PROGRAM F-3Yığın İt

123456789	/* ===== pushStack ===== Bu fonksiyon bir öğeyi yığına ekler. Önce yığın, yığın başı yapısına işaretçidir dataInPtr eklenmek üzere işaretçidir Sonra, başarı durumunda true; taşma durumunda false döner */
101112131415161718	bool pushStack (STACK* yığını, void* dataInPtr) { // İfadeler if (stack->count == stack->stackMax) return false; (yığın->sayım)++; (yığın->üst)++; yığın->stackAry[yığın->top] = dataInPtr; \dönüş doğru; } // pushStack

Yığından Çıkar

Yığın çıkarımının dizi uygulaması da oldukça basittir ve bağlı liste uygulamasıyla paraleldir. Kod Program F-4'te gösterilmiştir.

PROGRAM F-4Yığından Çıkar

123456789	/* ===== popStack ===== Bu fonksiyon, yığının en üstündeki öğeyi çıkarır. Önce stack, yığın başı yapısına işaretçidir. Sonra, başarılıysa kullanıcı verisine işaretçi döner. NULL, eksi akış durumunda munda
1011	void* popStack (STACK* yığını) { // Yerel Bildirimler void* dataPtrOut;

devam ediyor

PROGRAM F-4Yığını Çıkar (devam ediyor)

```

121314151617181920212223
// İfadeler
if (yığın->sayı == 0)
    dataPtrOut = NULL;

aksi takdirde
{
    dataPtrOut = yığın->stackAry[yığın->top];
    (yığın->sayı)--;
    (stack->top)--; } // a
ksi takdirde

dataPtrOut'u döndür;
} // popStack

```

Program F-4 AnaliziVeri yapıları ile çalışırken, yapının içindeki tek öğeyi sildiğinizde ne olacağını her zaman test etmelisiniz. Dizi yığımıza göre, yığındaki tek öğe silindiğinde, yığının doğru bir şekil de null durumuna ayarlandığından emin olmalıyız. Null bir yığın, -1 üst indeksine sahip bir yığın olarak tanımlanmıştır. Yığında yalnızca bir öğe varsa, üst 0'dır. 0'dan 1 çıkardığımızda -1 elde ederiz, bu da null yığın için belirlediğimiz işaretidir.

Yığın Üstü

Yığın üstü basit bir işlemdir. Yığında veri olup olmadığını kontrol ederiz ve yığının boş değilse üst öğedeki veri işaretçisini, boşsa bir null işaretçisini döndürürüz. Kod Program F-5'te gösterilmiştir.

PROGRAM F-5Yığın Üstü

```

123456789
/* ===== yığınÜstü =====Bu işlemin amacı, yığının üstündeki veriyi alır.

Önce yığın, yığına işaretçidir
Sonra başarılıysa veri işaretçisini döndürür
-veya- yığın boşsa null işaretçisini döndürür

*/
void* stackTop (STACK* yığını)
{
101112131415
// İfadeler
if (stack->count == 0)
    return NULL;

aksi takdirde
    return stack->stackAry[stack->top];
} // stackTop

```

Boş Yığın

Boş yığın mantığı, uygulama yapısına bakılmaksızın aynıdır. Bu, baş y apısındaki yığın sayısını test eder ve 0 ise doğru, değilse yanlış döner. Kod Program F-6'da gösterilmiştir.

PROGRAM F-6Boş Yığın

```

123456789 /* ===== emptyStack =====
          Bu fonksiyon bir yığının boş olup olmadığını belirler.
          Önce yığın, yığın başı yapısına işaretçidir
          Sonra boşsa true; yığında veri varsa false döner
*/
bool emptyStack (STACK* yığını)
{
    // İfadeler
    return (stack->count == 0);
10 } // emptyStack

```

Dolu Yığın

Yığının dolu olup olmadığını belirlemek için, sayıyı dizi için ayrılan maksimum eleman sayısı ile karşılaştırırız. Eğer eşitse, yığın doludur; değilse, yığın daha fazla veri için yer vardır. Kod Program F-7'de gösterilmiştir.

PROGRAM F-7Tam Yığın

```

123456789 /* ===== fullStack =====
          Bu fonksiyon bir yığının dolu olup olmadığını belirler.
          Önce yığın, bir yığın başı yapısına işaretçidir
          Sonra doluysa true; boş eleman varsa false döner
*/
bool fullStack (STACK* yığını)
{
    // İfadeler
    return (yığın->top == yığın->yığınMax);
10 } // fullStack

```

Yığın Sayısı

Boş yığın gibi, yığın sayısı için mantık tüm uygulamalarda aynıdır. Yığın başı yapısında bulunan yığın sayısını basitçe döndürüyoruz. Kod Program F-8'de gösterilmiştir.

PROGRAM F-8Yığın Sayısı

123456789	/* ===== stackCount ===== Yığın içindeki eleman sayısını döndürür. Önce yığın, yığın baş yapısına işaretçidir Sonra döndürülen sayı */ int stackCount(STACK* yığını) { // İfadeler dön stack->count; } // stackCount
10	

Yığını Yok Et

Bir dizi kullanarak uygulanmış bir yığını yok etmek için önce diziyapısını serbest bırakmalı ve ardından yığın başı yapısını serbest bırakmalıyız. Daha sonra null işaretçi döndürürüz, ki bu da çağırın fonksiyon tarafından uygulama yığın işaretçisinde saklanmalıdır. Kod P program F-9'da gösterilmiştir.

PROGRAM F-9Yığını Yok Et

123456789	/* ===== destroyStack ===== Bu fonksiyon tüm belleği yığın için serbest bırakır. Önce yığın, yığın baş yapısına işaretçidir Sonra null işaretçi döndürür */ STACK* destroyStack (STACK* yığını) { // İfadeler eğer (stack) { // Veri belleğini serbest bırak for (int i = 0; i < stack->count; i++) free (stack->stackAry[i]); // Yığın dizisini serbest bırak free (stack->stackAry); // Şimdi yığın başı için belleği serbest bırak free (stack); } // eğer yığ in NULL döndü r
10111213141516171819202122	

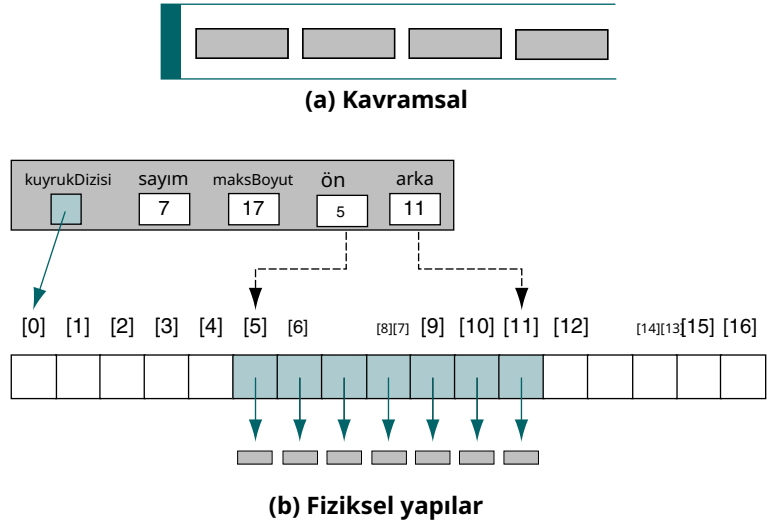
Program F-9 Analizi Tek endişemiz yığın işaretçisinin null olabileceğidir. Eğer öyleyse, sadece null işaretçi döndürmekten başka bir şey yapmayız. Yığın işaretçisi null değilse, önce dizi belleğini serbest bırakırız, ardından baş yapısı belleğini serbest bırakır ve null işaretçi döndürürüz.

F. 2Kuyruk ADT

Bağlı liste uygulaması çok popüler olmasına rağmen, bir kuyruğu uygulamanın tek yolu değildir. Kuyruklar ayrıca dizilerde ve çok büyük kuyruklar için dosyalar da da uygulanabilir. Bir dosya, diskte saklanan bir dizi olarak düşünülebileceğinden, dizi ve dosya uygulamaları benzer çözümlere sahiptir. Bu bölümde kuyruk ADT'sini bir dizi kullanarak yeniden yazıyoruz.

Boş bir kuyruğa ekleme, dizinin ilk elemanına yerleştirilir, bu da hem ön hem de arka olur. Sonraki eklemeler, arka sonrasındaki dizi konumuna yerleştirilir; yani, her ekleme, kuyruk verilerini mevcut kuyruk arkasından sonraki elemana kaydeder. Böylece, eğer kuyruktaki son eleman 11 numaralı dizi konumunda saklanıyorsa, bir sonraki ekleme için veri 12 numaralı elemana yerleştirilir.

Dequeues, kuyruğun önünde gerçekleşir. Bir eleman kuyruğunda silindiğinde kuyruğun önü bir sonraki konuma ilerler; yani, kuyruk önü, kuyruk önüne bir eklenerek güncellenir. Şekil F-2(a) kavramsal bir kuyruğu gösterirken; Şekil F-2(b) bir süre çalıştıktan sonra fiziksel bir kuyruğu göstermektedir. Gösterilen noktada, veriler dizinin önünden ortasına göç etmiştir.



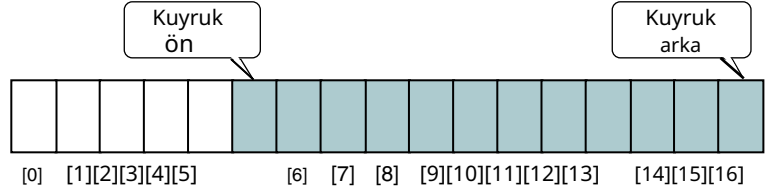
ŞEKİL F-2 Dizide Saklanan Kuyruk

Dizide bir kuyruk uyguladığımızda, işaretçiler yerine indeksler kullanırız. Böylece, Şekil F-2'deki kuyruğun önü 5 ve arkası 11'dir.

Dizide bir kuyruk uyguladığımızda, dolu bir kuyruğun tanımı değişir. Dolu bir kuyruk, her elemanın dolu olduğu şekilde tanımlanır. Çünkü dizilerde bir

Sınırlı sayıda eleman olduğunda, kuyruk sayısını maksimum eleman sayısı ile karşılaştırarak kuyruğun dolu olduğunu belirleyebiliriz.¹

Veriler, kuyruk hizmet süresinden daha hızlı geldiğinde, kuyruk dizinin sonuna doğru ilerleme eğilimindedir. Bu, dizinin sonundaki son elemanın dolu olduğu ancak dizinin başında boş elemanlar bulunduğu için kuyruğun dolu olmadığı duruma yol açar. Bu durum Şekil F-3'te gösterilmiştir.



ŞEKİL F-3 Son Elemanı Doldurulmuş Dizi Kuyruğu

Veriler dizinin sonunda gruplandığında, verileri kuyrukladığımızda a yeni eleman için bir yer bulmamız gerekiyor. Bir çözüm, dizinin sonundaki tüm elemanları başa kaydırmaktır. Örneğin, Şekil F-3'te eleman 5, eleman 0'a, eleman 6, 1'e, 7, 2'ye ve bu şekilde devam ederek eleman 16, 11'e kaydırılır.

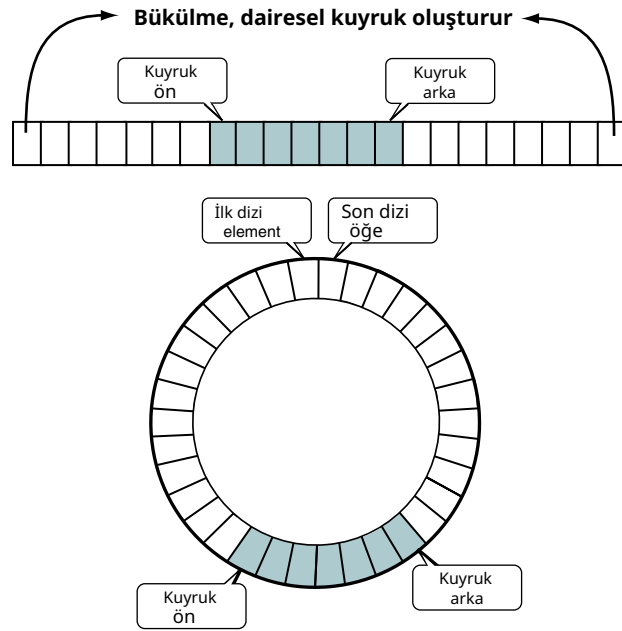
Daha verimli bir alternatif, dairesel bir dizi kullanmaktır. Dairesel bir dizide, son eleman mantıksal olarak ilk elemandan sonra gelir. Bu, son elemanı test ederek ve bir eklemek yerine, indeksi sıfıra ayarlayarak yapılır. Yukarıdaki dizi kuyruğumuza göre, 16'dan sonraki eleman 0'dır. Dairesel bir kuyruk Şekil F-4'te gösterilmiştir.

Dairesel bir kuyruk anlayışıyla, soyut veri tipini yeniden yazmaya hazırız. İki uygulama arasındaki çağrı dizisindeki tek fark, kuyruk oluşturma fonksiyonunda maksimum kuyruk boyutunun eklenmesidir.

Dizi Kuyrukları Uygulaması

Kuyruk başı için veri yapısında iki değişiklik gereklidir. Dinamik bellekten ayırdığımız kuyruk dizisinin adresini saklamamız gerekiyor. Ayrıca dizideki maksimum eleman sayısını da saklamamız gerekiyor. Revize edilmiş veri yapısı Program F-10'da gösterilmiştir. Kuyruk fonksiyonları için prototip bildirimlerini de ekliyoruz. Veri yapısını incelerken, Program 4-1'deki bağlı liste yapısıyla "Kuyruk ADT Veri Yapıları"nı karşılaştıralım.

1. C dilinde kuyruğu uyguluyor olsak da, *realloc* ile dizinin boyutunu dinamik olarak değiştirme olasılığı nı göz ardı ediyoruz.



ŞEKİL F-4Dairesel Kuyruk

PROGRAM F-10Kuyruk Dizi Tanımı

```

123456789 // Kuyruk ADT Tanımları
10111213141516171819202122 typedef struct
{
    void** queueAry;
    int maxSize;
    int count;
    int front;
    int rear;
} KUYRUK;

// Prototip Bildirimleri
QUEUE* createQueue (int maxSize);
QUEUE* destroyQueue (QUEUE* kuyruk);

bool enqueue (QUEUE* kuyruk, void* itemPtr);
void* dequeue (QUEUE* kuyruk);
void* queueFront (QUEUE* queue);
void* queueRear (QUEUE* queue);
int queueCount (QUEUE* queue);
bool emptyQueue (QUEUE* queue);
bool fullQueue (QUEUE* queue);
// Kuyruk ADT Tanımlarının Sonu

```

Kuyruk Oluştur

Kuyruk oluşturmanın dizi uygulaması, dinamik bellek içinde bir kuyruk baş yapısı ve dizinin kendisi için bellek ayırır ve baş yapısının adresini çağırana döner. Kuyruk yapısı için yeterli bellek yoksa, bir null işaretçi döner. Kuyruk oluşturma kodu Program F-11'de gösterilmiştir.

PROGRAM F-11 Kuyruk Oluştur

123456789	/* ===== createQueue ===== Dinamik bellekten bir kuyruk baş düğümü için bellek ayırır ve adresini çağırana döndürür. Ön hiçbir şey Son baş tahsis edildi ve başlatıldı Başarılıysa başın adresini döndür; taşıma durumunda null
10111213141516171819202122232425262728	*/ QUEUE* createQueue (int maxSize) { // Yerel Tanımlar QUEUE* kuyruk; // İfadeler kuyruk = (QUEUE*) malloc (sizeof (QUEUE)); if (kuyruk) { // baş yapı oluşturuldu. Şimdi kuyruk ayır kuyruk->queueAry = (void**)calloc(maxSize, sizeof(void*)); kuyruk->count = 0; kuyruk->front = -1; kuyruk->rear = -1; kuyruk->maxSize = maxSize; } // eğer return kuyruk; } // createQueue

Program F-11 Analizi Kuyruk oluşturma işleminin en zor kodu 19. ifadede bulunur. Dizi işaretçisini bir void işaretçisi olarak dö nüştürmemiz gerekir. Bunun nedeni, verileri doğrudan ADT'ye depolayamamamızdır. Çağırana i şlev, dinamik bellek içinde bir veri işaretçisi geçmelidir. Daha sonra bunu kuyruk dizisinde bir v oid işaretçisi olarak depolayabiliriz. Dizi, void işaretçilerin bir dizisi olduğundan, adı bir void iş aretçisine sabit bir işaretçidir. Bu, calloc tarafından döndürülen işaretçinin bir void işaretçisine işaretçi olması gerektiği anlamına gelir.

Not edilmesi gereken ikinci bir nokta, ön ve arka indeksleri -1 olarak ayarlamamızdır. K uyruk için null indeks olarak -1'i seçtik, böylece kuyruk ekleme mantığını mümkün olduğunca basit hale getirebiliriz. Kuyruğa bir eleman eklediğimizde, arka indekse bir ekleriz. Kuyruk null olduğunda, bu otomatik olarak arka indeksi sıfıra ayarlar. Ön kuyruk i çin başlangıç değeri herhangi bir şey olabilir, ancak tutarlılık için -1 yapıyoruz.

Kuyruk ekle

Kuponun dizi uygulaması için parametre listesi, bağlı liste uygulamasıyla aynıdır. Ancak, mantık önemli ölçüde farklıdır. Öncelikle, tahsis edilecek bir bellek yoktur. Verileri depolamak için hangi dizi elemanında veri işaretçisini saklayacağımızı belirlememiz yeterlidir.

Tam bir kuyruk testi de değiştirilmiştir. Kuyruğun dolu olup olmadığını belirlemek için, kuyruk sayısını maksimum kuyruk boyutuyla karşılaştırırız. Eğer eşitse, dolu bir kuyruğumuz vardır.

Ekleme işlemini tamamlamak için, arka indekse bir ekleriz. Eğer arka son elemana ulaşılmışsa, kuyruğun ilk elemanına geri dönmemiz gerekir. Kod Program F-12'de gösterilmiştir.

PROGRAM F-12 Kuyruk Ekle

123456789	/* ===== kuyruk ekleme ===== Bu algoritma verileri bir kuyruğa ekler. Önce kuyruk oluşturuldu Sonra veriler eklendi Başarılıysa true döner; taşma durumunda false döner */
10111213141516171819202122232425262728	bool enqueue (QUEUE* kuyruk, void* itemPtr) { // İfadeler if (queue->count == kuyruk->maxSize) return false; (sıra->arka)++; if (queue->rear == queue->maxSize) // Kuyruk 0. elemana sarar queue->rear = 0; kuyruk->kuyrukAry[kuyruk->arka] = itemPtr; if (sıra->sayım == 0) { // Boş kuyruğa ekleme queue->front = 0; queue->count = 1; } // eğer aksi takdirde (sıra->sayım)++; return true; } // sıraya ekle

Program F-12 Analizi Bu işlevde iki noktayı dikkatlice incelemeniz gerekiyor. Öncelikle, 14 ile 16. ifadelerdeki geri sarmayı mantığının nasıl ele alındığını not edin. İşlevin bu noktasında, kuyruğun dolu olmadığını biliyoruz. Ancak, arka indeks maksimum kuyruk boyutuna eşit olduğunda, ilk elemana geri dönmemiz gerekir. (Unutmayın, C sıfır indeksleme kullanır, bu nedenle yeni arka, sayı ya eşit olduğunda, son eleman doldurulmuştur.)

İkincisi, sıfır sayısı ile tanımlanan bir boş kuyruğa ekleme yapıyorsak, ön indeksini 0 olarak ayarlamalıyız. Arka indeks, boş arka işaretçisine bir eklediğimizde otomatik olarak 0 olarak ayarlanır; bu işaretçi, kuyruk boş olduğunda -1 olarak ayarlanır.

Kuyruktan Çıkarma

Kuyruktan çıkarma, kuyruğun önündeki veriyi siler ve onu çağıran işleve geri döndürür. Kodunun Program F-13'te gösterildiği gibi.

PROGRAM F-13 Kuyruktan Çıkarma

123456789	/* ===== kuyruktan çıkarma ===== Bu algoritma bağlı listeden bir düğümü siler. Önce kuyruk oluşturulmuştur Baş arılıysa kullanıcı verisine işaretçi döner NULL eksi akış durumunda */
101112131415161718192021222324252627	void* kuyruktan çıkarma (QUEUE* kuyruğu) { // Yerel Bildirimler void* dataPtrOut; // İfadeler if (!kuyruk->say) NULL'u döndür; dataPtrOut = kuyruk->kuyruk->ön]; (kuyruk->ön)++; if (queue->front == queue->maxSize) // kuyruk önü 0. elemana sarıldı queue->front = 0; eğer (queue->count == 1) // Kuyruktaki tek öge silindi queue->rear = queue->front = -1; (queue->count)--; return dataPtrOut; } // kuyruktan çıkar

Program F-13 Analizi Bu işlevde iki kod parçasına dikkat edin. İlk olarak, bir kez daha kuyruğun sonundan geri sarmamız gerekiyor. Kuyruktan çıkarma işlemi için, veriyi kuyruktan sildikten sonra bir geri sarma testi yapıyoruz. Eğer ön, dizinin son elemanındaysa, yeni ön değeri ilk elemana, 0'a ayarlamalıyız.

İkinci nokta, kuyruktaki son ögeyi sildiğimizde hem ön hem de arka değerlerini -1 olarak sıfırlamamız gerektirir. Bu, 21'den 23'e kadar olan ifadelerde yapılmaktadır.

Kuyruk Önü

Kuyruk ön mantığı, kuyruktan çıkarma ile paraleldir, tek farkı kuyruk durumunun değiştirilmemesidir. Mantığı Program F-14'te gösterilmiştir.

PROGRAM F-14 Kuyruk Önü

123456789	/* ===== queueFront ===== Kuyruğun önündeki veriyi alır kuyruk içeriğini değiştirmeden. Önce kuyruk, başlatılmış kuyruğa işaretçidir Sonra başlar ılıysa kullanıcı verisine işaretçi döner; NULL eğer alt akı ş varsa
1011121314	*/ void* kuyrukFront (QUEUE* kuyruk) { // İfadeler if (!kuyruk->say) NULL'u döndür; dönüş kuyruğu->queueAry[kuyruk->ön]; } // kuyrukFront

Kuyruk Arkası

Kuyruk arkası, kuyruğun arkasındaki veriye bir işaretçi döndürür ve kuyruğun içeriğini değiştirmez. Mantığı Program F-15'te gösterilmiştir.

PROGRAM F-15 Kuyruk Arkası

123456789	/* ===== queueRear ===== Kuyruğun arkasındaki veriyi alır kuyruk içeriğini değiştirmeden. Önce kuyruk, başlatılmış kuyruğa işaretçidir Sonra başlar şarılıysa kullanıcı verisine işaretçi döner; NULL eğer alt akış varsa
1011121314	*/ void* kuyrukArka (QUEUE* kuyruk) { // İfadeler if (!kuyruk->say) NULL'u döndür; dönüş kuyruğu->kuyrukAry[sıra->arka]; } // kuyrukArka

Dolu Kuyruk

Dolu kuyruk mantığı, bağlı listeye göre dizi uygulamasında çok daha basittir. Sadece sayımı maksimum kuyruk boyutuyla test ediyoruz. Eğer eşitlerse, kuyruk doludur. Kod Program F-16'da gösterilmiştir.

Boş Kuyruk

Boş kuyruk mantığı, bağlı listeye göre dizi uygulamasında çok daha basittir. Sadece sayımı maksimum kuyruk boyutuyla test ediyoruz. Eğer eşitlerse, kuyruk doludur. Kod Program F-17'de gösterilmiştir.

PROGRAM F-16Dolu Kuyruk

123456789	/* ===== doluKupon ===== Bir kuponun dolu olup olmadığını kontrol eder. Kupon doludur eğer dizi doluysa. Ön kupon, bir kupon baş düğümüne işaretçidir Doluysa doğru; daha fazla yer varsa yanlış döner
1011	*/ bool doluKupon (KUPO* kupon) { // İfadeler return (kupon->sayı == kupon->maksBoyut); } // doluKupon

PROGRAM F-17Boş Kuyruk

123456789	/* ===== boşKupon ===== Bir kuponun boş olup olmadığını kontrol eder. Ön kupon, bir kupon baş düğümüne işaretçidir Boşsa doğru; kupon içinde veri varsa yanlış döner
10	*/ bool boşKupon (KUPO* kupon) { // İfadeler return (kupon->sayı == 0); } // boşKupon

Kuyruk Sayısı

Kuyruk sayısı, kuyruktaki eleman sayısını döndürür. KodProgram F-18 'de gösterilmiştir.

PROGRAM F-18Kuyruk Sayısı

123456789	/* ===== kuyrukSayısı ===== Kuyruktaki eleman sayısını döndürür. Ön kuyruk, bir kuyruk baş düğümüne işaretçidir Kuyruk sayısını döndür
10	*/ int kuyrukSayısı (KUYRUK* kuyruk) { // İfadeler dönüş kuyruk->sayısı; } // boşKuyruk

Kuponu Yok Et

Dizi uygulaması için son işlev kuponu yok etmektir. Kuponsayısı ve boş kupon işlevleri, bağlı liste uygulaması ve dizi uygulaması için aynıdır , bu yüzden bunları tekrarlamayacağız.

Kuyruğu yok etmek için, diziyi ve baş yapılarını serbest bırakırız ve null işaretçi döndürürüz. Kod Program F-19'da gösterilmiştir.

PROGRAM F-19 Kuyruğu Yok Et

```

123456789 /* ===== destroyQueue =====
           Bu fonksiyon kuyruğu ve dizi belleğini siler.
           Önce kuyruk geçerli bir kuyruktur
           Sonra tüm veriler silinmiş ve geri dönüştürülmüştür
           Null işaretçi döndür
*/
101112131415161718192021222324252627 QUEUE* destroyQueue (QUEUE* kuyruk)
{
// Yerel Tanımlar
int locn = kuyruk->ön;

// İfadeler
if (kuyruk)
{
// Veri belleğini serbest bırak while (kuyru
k->sayı != 0)
{
free (kuyruk->kuyrukAry[locn++]);
kuyruk->sayı--;
if (locn == kuyruk->maxBoyut)
locn = 0;
} // while
serbest (queue->queueAry);
serbest (queue);
} // eğer
NULL döndür;
} // destroyQueue

```

2-3 ağacı: 3. dereceden bir B-ağacı.

2-3-4 ağacı: 4. dereceden bir B-ağacı.

A

soyut veri tipi (ADT): bir veri bildirimi veri tipinde anlamlı olan işlemlerle birlikte paketlenmiştir.

komşuluk listesi: bir grafiği temsil etmenin bir yöntemi kenarları depolamak için köşeleri saklamak üzere bir bağlı liste ve duyuşal, bağlı liste dizisi kullanır.

komşuluk matris: bir grafiği temsil etmenin bir yöntemi kenarlar için köşeleri saklamak üzere bir vektör ve kenarları saklamak için bir matris (kare iki boyutlu dizi) kullanır.

komşu köşeler: bir grafikte bir kenar ile bağlanmış olan iki köşe.

ADT: soyut veri türünü görün.

algorithm: bir problemi çözmek için gerekli mantıksal adımlar; bir modül veya bir modülün parçası.

algorithmics: Brassard ve Bratley tarafından oluşturulan, verimli algoritmalar oluşturmak için kullanılan tekniklerin incelenmesini ifade eden terim.

ancestor: bir ağacın köküne giden yol üzerinde ki herhangi bir düğüm.

arc: bir grafikte yönlendirilmiş bir çizgi; kenar ile karşılaştır.

array: aynı veri türündeki elemanların sabit boyutlu, sıralı bir koleksiyonu.

arrival rate: müşterilerin kuyruğa geldiği oran.

ascending sequence: listedeki her elemanın, öncükilerin anahtarlarından büyük veya onlara eşit olduğu bir liste sırası.

ascending sort: bir listeyi artan sırada sıralayan bir sıralama.

ASCII: Bilgi Değişimi için Amerikan Standart Kodu. Bir baytın ilk 128 değeri için kontrol karakterlerini ve grafik karakterlerini tanımlayan bir kodlama şeması.

atomik veri: anlamlı bir şekilde alt bölümlere ayrılamayan veridir.

AVL ağacı: denge faktörünü kullanarak dengeyi kontrol eden bir yükseklik dengeli ikili arama ağacıdır.

AVL ağacı baş yapısı: bir AVL ağacı uygulamasında ağacın kök işaretçisi, karşılaştırma işlevi, işaretçisi ve düğüm sayısı gibi ağacın meta verilerini içeren düğümdür.

AVL ağaç düğümü yapısı: AVL ağacındaki bir düğüm uygulamasının ağaç verilerini ve yapısal meta verileri, sol ve sağ dallar gibi, düğüm için denge faktörünü içerir.

B

geri izleme: genellikle bir yığın ile veya rekürsion yoluyla uygulanan, bir veri yapısı üzerindeki yolu hatırlayan ve bu yolu ters sırayla izleyebilen bir algoritmik süreçtir.

denge: düğümün alt ağaçları arasındaki yükseklik farkını temsil eden bir ağaç düğümü atribütüdür.

denge faktörü: AVL ağacında, ağacın dengesi için kullanılan meta veridir.

dengeli ikili ağaç: alt ağaçların yüksekliğinin birden fazla farklı olmadığı ve alt ağaçların dengeli olduğu bir ikili ağaçtır.

dengeli birleştirme: sabit sayıda girdi birleştirme dosyası ve aynı sayıda çıktı birleştirme dosyası kullanılarak bir birleştirmedir.

temel durum: bir rekürsif fonksiyonda, "problemi çözen" ifadedir.

büyük-O notasyonu: yalnızca baskın faktörün dikkate alındığı bir algoritmanın verimliliğinin bir ölçüsüdür.

malzeme listesi: hangi bileşenlerin hangi parçalara montajlandığını gösteren bir ürünün ağaç temsildir. Bir yapı şeması, malzeme listesinin bir örneğidir.

ikili arama: bir hedefin listenin sürekli olarak yarıya bölünmesiyle bulunduğu bir arama algoritması.

ikili arama ağacı (BST): (1) sol alt ağacın anahtarları kök anahtarından daha küçük, (2) sağ alt ağacın anahtarları kök anahtarına eşit veya daha büyük ve (3) alt ağaçların hepsi ikili arama ağaçları olan bir ikili ağaç.

ikili ağaç: hiçbir düğümün iki çocuktan fazlasına sahip olamayacağı bir ağaç; maksimum 2 dış dereceye sahip bir ağaç.

ikili ağaç araması: aşağıdaki özelliklere sahip bir ikili ağaç: (1) Sol alt ağaçtaki tüm öğeler kökten daha küçüktür. (2) Sağ alt ağaçtaki tüm öğeler kökten daha büyük veya eşittir. Ve (3) her alt ağaç kendisi bir ikili arama ağacıdır.

ikili ağaç gezintisi: her düğümü bir kez ve yalnızca bir kez belirlenmiş bir sırayla işleyen bir ikili ağaç işleme dizisi.

bool: Boolean türü için C anahtar kelimesi.

Boolean: yalnızca doğru veya yanlış olabilen bir tür.

dal: iki bitişik düğümü bağlayan bir ağaçtaki bir çizgi.

genişlik öncelikli gezinti: mevcut (kardeş) düğüme bitişik düğümlerin, onların soyundan önce işlendiği bir grafik gezintisi.

BST: *ikili arama ağacına bakınız.*

B*tree: en düğümün en az üçte iki dolu olduğu bir B-ağacıdır.

B+tree: tüm verilerin yaprak seviyesinde temsil edildiği bir B-ağacı ve her yaprak düğümün ağaçtaki bir sonraki ardışık yaprak düğüme bir işaretçi sahip olduğu bir B-ağacıdır.

B-tree: kök hariç tüm düğümlerin minimum girdi sayısına sahip olduğu bir m-yönlü ağaçtır, ve yapraklar hariç her düğüm girişi bir null olmaya n alt ağaç içerir.

B-tree order: bir B-ağaç düğümünden maksimum alt ağaç sayısıdır.

bubble sort: her geçişte veriler arasında (en düşük) elemanı sıralanmamış listenin başına taşıyan (kabarcık) bir sıralama algoritmasıdır.

buck et: bir hashing algoritmasında, birden fazla data örneğini barındırabilen bir düğümdür.

bucket hashing: anahtarların birden fazla data örneğini barındıran düğümlere hash'lendiği bir hashing tekniğidir.

C

child: bir öncüsü olan bir ağaç veya grafikteki bir düğümdür.

chronological list: zamanla düzenlenmiş bir liste; yani veriler alındıkları sıraya göre saklanır; ayrıca ilk giren-ilk çıkar (FIFO) ve son giren-son çıkar (LIFO) ile de görülür.

circularly linked list: son düğümün bağlantısının listeki ilk düğüme işaret ettiği bir bağlı listedir.

kümeleme: verilerin bir hashlenmiş liste üzerinde dengelessiz bir şekilde birikme eğilimi.

çarpışma: bir hashing algoritmasının bir ekleme için bir adres üretmesi ve o adresin zaten dolu olması durumunda meydana gelen bir olay.

çarpışma çözümü: bir çarpışmadan sonra alternatif bir adres belirleyen algoritmik işlem.

birleştirmek: bir düğümden bir alt akıştan, minimal bir kardeşten ve bir ebeveyninden verileri birleştiren bir B-ağacı alt akış dengeleme tekniği.

tam ağaç: yüksekliği için maksimum düğüm sayısına sahip, kısıtlı bir dış dereceye sahip bir ağaç.

tam ikili ağaç: dış derecesi 2 olan bir tam ağaç.

bileşik veri: diğer veri yapıları üzerine inşa edilen veriler; yani, ayrık atomik öğelere ayrılabilen veriler.

bağlı grafik: bir grafik, yön bastırıldığında, herhangi bir köşeden diğer herhangi bir köşeye bir yol varsa bağlıdır.

inşa etmek: sıralama, seçim veya döngü gibi temel bir programlama tasarım ifadesi.

müşteri: bir kuyrukta, bir kişi veya şey hizmete ihtiyaç duyar.

döngü: uzunluğu 1'den büyük olan ve aynı köşede başlayan ve biten bir grafik yolu.

D

veri: bağlı listede, uygulama bilgilerini depolayan yapının kısmı.

veri sıkıştırma: verileri depolamak için gereken bit sayısını azaltan herhangi bir veri depolama yöntemi. Huffman koduna bakın.

veri gizleme: verilerin yalnızca bir işlevin işlemini tamamlamak için gerektiğinde mevcut olduğu yapılandırılmış programlama ilkesidir; gerekli olmayan veriler görünümünden "gizlenir"; ayrıca bakınız kapsülleme.

veri adı: bir programda verilere verilen bir tanımlayıcı.

veri düğümü: bir veri yapısında, uygulama verilerini veya uygulama verilerine bir işaretçi içeren bir düğüm; başlık düğümü ile karşılaştırın.

veri yapısı: tanımlı ilişkileri olan bir küme haline getirilen atomik ve bileşik veri türlerinin agregasyonu.

veri türü: değerleri manipüle etmek için tanımlanan bir dizi değer ve işlemden oluşan adlandırılmış bir küme, örneğin karakter ve tam sayı.

veri doğrulama: bir dış kaynaktan okunan verilerin doğrulanması ve geçerliliğinin kontrol edilmesi süreci.

derece: bir grafikte bir düğüme bağlı olan hat sayısı.

silme: bir listede, verileri kaldıran herhangi bir işlem.

bağımlı ikinci dereceden döngü: iç döngünün yineleme sayısının dış döngüye bağlı olduğu herhangi bir döngü.

derinlik (ağaç için): yükseklik için bakınız.

derinlik öncelikli gezinti: bir düğümün tüm soyundan gelenlerin, herhangi bir komşu düğümden (kardeşlerden) önce işlendiği bir gezinti.

kuyruktan: bir öğeyi kuyruktan sil.

soy: mevcut düğümden bir yaprağa giden yoldaki herhangi bir düğüm.

azalan sıra: bir listedeki her öğenin anahtarının, öncekinden küçük veya eşit olduğu bir liste sırası.

azalan sıralama: bir listeyi azalan sırada sıralayan bir sıralama.

rakamsal çıkarım: seçilen rakamların anahtardan çıkarıldığı ve adres olarak kullanıldığı bir hashin yöntemi.

yönlü grafik: yönlendirilmiş bir grafik.

doğrudan hashing: anahtarın değiştirilmeden kullanıldığı bir hashing yöntemi.

yönlendirilmiş grafik: yönün çizgilerde (yaylar) belirtildiği bir grafik.

bağımsız grafik: bağlı olmayan bir grafik.

dağıtım aşaması: doğal bir birleştirme sıralamasında, birleştirme işlemlerini yeniden birleştirmek için giriş dosyalarına yeniden dağıtan veri üzerinden geçiş.

bölüm kalanı: anahtarın dizi boyutuna bölündüğü ve adres için kalanın kullanıldığı bir hashing yöntemi.

çift hashing: çakışma adresinin bir sonraki adresi belirlemek için hash'lendiği bir çakışma çözümleme yöntemi.

çift bağlı liste: her düğümün hem ardılları hem de öncülüne işaretçi taşıdığı bir bağlı liste yapısı; tek bağlı liste ile karşılaştırın.

dinamik bellek: programın çalışması sırasında belirli bir şekilde kullanılabilen bellek (genellikle *yığın* olarak adlandırılır).

E

kenar: yönsüz bir grafikteki bir çizgi.

boş liste: tanımlanmış ancak data içermeyen bir liste. Ayrıca null liste olarak da bilinir.

kapsülleme: verilerin, fonksiyonların ve nesnelerin, metin dosyaları veya doğrusal listeler gibi, uygulamadan ayrı olarak tutulduğu tasarım kavramı; ayrıca veri gizleme için bakınız.

kuyruk ekleme: bir öğeyi kuyruğa eklemek.

değişim sıralaması: tüm liste sıralanana kadar uygun olmayan öğeleri değiştiren sıralama algoritması.

üstel verimlilik: işlenen öğe sayısının kuvvetine bağlı olarak çalışma süresinin bir fonksiyonu olan program/modül verimliliği kategorisi, örneğin $O(n) = c^n$.

ifade ağacı: bir ifadenin ikili ağaç temsili.

dış sıralama: sıralanan veriler için birincil belleği kullanırken, birincil belleğe sığmayan veriler için ikincil depolama kullanan bir sıralama.

F

faktöriyel: değeri 1'den verilen sayıya kadar olan tam sayıların çarpımı olan aritmetik fonksiyon.

faktöriyel verimliliği: bir modülün verimliliğinin bir ölçüsü çalışma süresinin eleman sayısının faktöriyel ile orantılı olduğu, $O(n) = n!$ şeklinde olduğu durumlarda.

Fibonacci sayıları: Leonardo Fibonacci tarafından keşfedilen bir sayı dizisi olup, her sayı önceki iki sayının toplamıdır.

FIFO: ilk giren-ilk çıkan.

FIFO ekleme: verileri bir listeye yerleştiren bir ekleme yöntemi, böylece sonraki bir geçiş verileri ilk giren-ilk çıkan sırasına göre işler.

ilk giren-ilk çıkan (FIFO): verilerin alındığı sıraya göre işlendiği bir veri yapısı işleme sırası. Bir kuyruk.

katlama sınırı: anahtar belirlemek için eklenmeden önce sol ve sağ katlamaların tersine çevrildiği bir hashing algoritması.

katlama kaydırması: anahtarın parçalar halinde bölündüğü ve anahtarı belirlemek için eklendiği bir hashing algoritması.

ön: bir listede, ilk öğeyi tanımlayan bir referans. Bir kuyrukta, çıkarılacak bir sonraki öğe.

G

genel durum: özzyinelemede, problemi azaltan durum.

genel liste: verilerin listede herhangi bir yere eklenip silinebildiği bir liste.

genel ağaç: sınırsız dış dereceye sahip bir ağaç.

genel kod: herhangi bir veri türünü işlemek için kullanılacak bir kod seti kullanan bir program tasarımı.

goezinta: malzeme listesi için "içine girer" anlamına gelen bir argo terim.

grafik: her bir öğenin sıfır, bir veya daha fazla öncülü ve sıfır, bir veya daha fazla ardılı olabileceği lineer olmayan bir liste.

en büyük ortak bölen (EBOB): matematikte, iki sayının en büyük tam böleni.

H

hashlenmiş liste: verilerin konumunun bir hashing algoritmasıyla belirlendiği bir liste.

hashlenmiş arama: hashlenmiş listede verileri bulmak için kullanılan yöntemlerden herhangi biri.

hashleme: bir anahtarın, algoritmik bir dönüşüm aracılığıyla, verinin konumunu doğru dan belirlediği bir anahtar-adres dönüşümü.

baş: bir ADT'de, ADT'nin metadata'sını içeren bir yapı; bu, ilk düğümün adresi ve ADT'deki öğe sayısını içerir.

baş işaretçisi: bağlı bir listenin ilk öğesini tanımlayan bir işaretçi.

başlık düğümü: *bkz. baş.*

yığın: kökü ağaçtaki en büyük düğüm olan ve alt ağaçların da yığın olduğu bir ikili ağaç; *bkz. minimum yığın.*

yığın bellek: bir program çalışırken veriler için dinamik olarak alan ayırmak üzere kullanılabilecek bir bellek havuzu; ayrıca dinamik bellek'a da bakın.

yığın sıralama: bir dizinin sıralanmamış alanındaki en büyük öğeyi belirlemek için bir yığın kullanan bir seçim sıralama algoritması.

heapify: yığın işleme algoritması, en büyük öğeyi yığının tepesine çıkarır.

height: kökten son seviyeye kadar olan yolun uzunluğunu belirten bir ağaç niteliği; kökten en uzun yolda bulunan yaprağın seviyesi artı 1.

height-balanced tree: alt ağaçlarının yüksekliği a racılığıyla dengenin kontrol edildiği dengeli bir ağaç. AVL ağacı, yükseklik dengeli bir ağaçtır.

home address: bir hashlenmiş listede, hashing algoritması tarafından üretilen ilk adres.

Huffman code: daha sık meydana gelen karakterlere daha kısa kodlar ve daha az sık meydana gelenlere daha uzun kodlar atayan bir sıkıştırma algoritması.

I

indegree: bir ağaçta veya grafikte, bir düğüme girilen çizgi sayısı.

index: bir dizideki bir öğenin adresi.

infix: operatörün iki operandın arasına yerleştirildiği ikili aritmetik notasyon.

infix traversal: bir ifade ağacında, operatörün iki operandın arasına yerleştirildiği geçiş. Inorder'a bakın.

information hiding: kullanıcının veri yapısını veya işlemlerinin uygulanmasını bilmediği yapılandırılmış bir programlama kavramı.

inorder: bir ikili ağaçta, kök sol alt ağacın ardından ve sağ alt ağaçtan önce işlenir; bir LNR geçişi.

inorder traversal: *inorder'a bakın.*

insert: bir veri yapısına veya bir dosyaya bir elemanın eklenmesi.

insertion: bir listede, verileri ekleyen herhangi bir işlem.

insertion sort: sıralama algoritması, listenin sıralanmamış kısmından ilk elemanın, listenin sıralı kısmındaki verilere göre uygun pozisyona yerleştirildiği bir sıralama yöntemidir.

intelligent data name: verinin anlamını veya algoritmanın amacını tanımlayan bir veri veya algoritma adı.

internal node: kök ve yapraklar hariç, herhangi bir ağaç düğümü; bir ağacın ortasında bir düğüm.

internal sort: sıralama işlemi sırasında tüm verilerin birincil depolamada tutulduğu bir sıralama.

K

key: veriyi tanımlamak veya kullanımını kontrol etmek için kullanılan bir veri yapısındaki bir veya daha fazla alan.

key offset: mevcut adresin ve anahtarın bir fonksiyonu olarak bir sonraki adresin belirlendiği, hashlenmiş liste çamaşıma çözümüleme yöntemi.

key-sequenced insertion: genel bir ağaçta, yeni düğümü kardeş düğümler arasında anahtar sırasına yerleştiren ekleme yöntemi.

L

son giren-ilk çıkan (LIFO): verilerin geri alındığı sıranın tersine işlenerek işlendiği bir veri yapısı işleme sırası; bir yığın.

yaprak: çıkış derecesi 0 olan bir grafik veya ağaç düğümü.

yaprak alt ağacı: ikili ağaçta, bir düğümden sol dalda bulunan alt ağaç.

sol yüksek: AVL ağacında, sol alt ağacının sağ alt ağacından daha yüksek olduğu bir ağaç veya alt ağaç.

solun solunda: sol yüksek bir düğümün sol high alt ağacına sahip olduğu dengesiz AVL ağacının geçici durumu.

solun sağında: sağ yüksek bir düğümün sol high alt ağacına sahip olduğu dengesiz AVL ağacının geçici durumu.

seviye: bir ağaç veya grafikte, bir düğümün kökten uzaklığını belirten bir özellik.

leksikal arama ağacı: her düğümün yalnızca bir karakter barındırdığı ve tam anahtarın ağaç boyunca bir yol izlenerek oluşturulduğu leksikografik arama ağacı.

leksikografik: sözlüğe dayalı bir veri sırası; *ayrıca* artan sıra için bakınız.

LIFO: son giren - ilk çıkan için bakınız.

LIFO ekleme: verileri bir listede yerleştiren bir ekleme yöntemi, böylece sonraki bir geçiş verileri son giren - ilk çıkan sırasına göre işler.

çizgi: iki köşeyi bağlayan bir grafik elemanı grafikte; *ayrıca bakınız* yay ve kenar.

doğrusal verimlilik: bir

modülün verimliliğinin bir ölçüsü, burada çalışma süresi işlenen eleman sayısı ile orantılıdır, $O(n) = n$.

lineer liste: her elemanın, sonuncusu hariç, benzersiz bir ardılı olan bir liste yapısı.

lineer döngü: işlenmekte olan eleman sayısının bir fonksiyonu olan bir döngü; *ayrıca bakınız* z lineer verimlilik.

lineer sorgulama: bir çarpışma çözümleme algoritması sonraki aday adresi belirlemek için 1 ekleyerek veya çıkararak belirler.

lineer arama: bir lineer listede verileri bulmak için kullanılan herhangi bir arama algoritması.

bağlantı: bir liste yapısında, listedeki bir sonraki elemanı tanımlayan alan.

bağlı liste: elemanların sıralamasının bağlantı alanları tarafından belirlendiği bir yapı.

bağlı liste çarpışma çözümü: eş anlamlılar için ayrı bir alan kullanan bir hashlenmiş liste çarpışma çözümü yöntemi, bu alan bir bağlı listede tutulur.

liste: ana bellekte bulunan sıralı bir veri kümesi.

liste gezintisi: bir listedeki her elemanı sırayla işleyen liste işleme algoritması.

LNR: sol, düğüm, sağ; *bakınız* sıralı.

yükleme faktörü: bir hashlenmiş listede, listenin içindeki veri düğümlerinin sayısının, listenin içindeki fiziksel elemanların sayısına oranı, yüzde olarak ifade edilir.

logaritmik verimlilik: işlenen eleman sayısının logaritması ile orantılı olan bir modülün verimliliğini ölçen bir ölçü, örneğin $O(n) = \log n$.

logaritmik döngü: işleme alınan eleman sayısının logaritmasının eğriliğine bağlı olan bir döngü; *ayrıca bakınız* logaritmik verimlilik.

mantıksal veri: değerleri yalnızca doğru veya yanlış olabilir veri; *ayrıca bakınız* Boolean.

döngü: bir kod bloğunu yineleyen bir ifade.

LRN: sol, sağ, düğüm; *ayrıca bakınız* postorder.

M

m: bir m -yönlü ağaçtaki sıralama.

m -yönlü ağaç: her düğümde birden fazla data girişi ve alt ağaç içeren bir arama ağaç yapısı; alt ağaçların maksimum sayısı,

m -yönlü ağacın sıralaması olarak bilinir.

max-heap: *ayrıca bakınız* heap.

birleştirme: iki veya daha fazla ardışık dosyayı bir ortak anahtar ve yapı formatına dayalı olarak bir ardışık dosyada birleştirmek.

birleştirme aşaması: bir dış sıralamada, dataları iki veya daha fazla dosyadan birleştiren bir geçiş.

birleştirme çalışması: birleştirme dosyasında ardışık sıralı verilerin bir seti.

birleştirme sıralaması: dosyaların tekrarlayan birleştirilmesiyle data sıralayan herhangi bir sıralama tekniği.

meta veriler: veri yapısının kendisi içinde saklanan liste veya diğer veri yapıları hakkında veri.

orta kare hashing: anahtarın kare alındığı ve adresin kare alınmış sayısının ortasında seçildiği bir hashing yöntemi.

minimum heap: bir ikili ağaçtır, kökü ağaçtaki en küçük düğümdür ve alt ağaçlar da min-heap'tir; heap ve max-heap ile karşılaştırılır.

minimum spanning tree: ağırlıkların toplamının graf içindeki tüm olası ağaçların minimumu olduğu bir spanning tree.

modular programming: programın bir dizi modül şeklinde yazıldığı program tasarımıdır; structured programming ile karşılaştırılır.

modulo-division: anahtar dizi boyutuna bölün bir hashing yöntemidir ve adres için kalanı kullanır.

multilinked list: iki veya daha fazla mantıksal anahtar dizisine sahip bir fiziksel bağlı liste yapısıdır.

multiserver queue: aynı anda birçok müşteriye hizmet verebilen bir kuyruk tasarımıdır.

N

natural merge: sabit sayıda girdi birleştirme dosyası ve bir çıktı birleştirme dosyası ile bir birleştirmedir.

nearly complete tree: sınırlı bir dış dereceye sahip, düğümleri için minimum yüksekliğe sahip ve yaprak seviyesinin sol taraftan doldurulduğu bir ağaçtır.

nested loop: verimliliği kontrol eden bir döngünün verimliliğine bağlı olan bir döngüdür.

network: bakınız weighted graph.

NLR: düğüm, sol, sağ; öncelikli sıraya bakınız.

düğüm: bir veri yapısında, listeyi işlemek için kullanılan hem veri hem de yapısal öğeleri içeren bir öğe.

doğrusal olmayan liste: her bir öğenin birden fazla ardılı olabileceği bir liste.

boş ağaç: düğümü olmayan bir ağaç.

O

nesne: nesne yönelimli programlamada, bir sınıfın herhangi bir örneği, üyeleri ve yöntemleri dahildir.

açık adresleme: yeni adresin ana alanda olduğu bir çarpışma çözümleme yöntemi.

sıra: m yönlü bir ağaçta, bir düğüm için izin verilen maksimum alt ağaç sayısı.

sıra (sıralama): sıralama sırası için bakınız.

sıralı liste: öğelerin anahtar değerlerinin artan veya azalan sırayla yerleştirildiği bir liste.

çıkış derecesi: bir ağaçta veya grafikte bir düğümden çıkan çizgi sayısı.

taşma: dolu bir listeye veri eklenmeye çalışıldığında ortaya çıkan durum.

taşma alanı: bir karmaşık listede, bağlı bir listede sözcüklerin saklandığı ana alanda ayrı bir alan.

P

ebeveyn: dış derecesi 0'dan büyük olan bir ağaç veya grafik düğümü; yani, ardılları olan.

parsing: verileri bağımsız parçalara ayıran mantık.

yol: bir ağaç veya grafikte, her düğümün bir sonrakisi ile komşu olduğu düğümlerin bir dizisi.

pivot: quicksort'ta, verileri düzenlemek için kullanılan anahtar değeri, pivot'tan daha küçük anahtara sahip tüm verilerin pivot'un solunda ve pivot'tan daha büyük veya eşit anahtara sahip tüm verilerin pivot'un sağında olduğu dizide yer alır.

pointer: bir bellek konumunun adresini içeren bir sabit veya değişken.

pointer to function: bir işlevin giriş noktasını tanımlayan bir işaretçi. Bir işlevin adresini parametre olarak göndermek için kullanılır.

pointer to void: herhangi bir türün işaretçisini saklayabilen genel bir işaretçi türü.

polynomic efficiency: bir modülün verimliliğinin bir ölçüsü, çalışma süresinin, en yüksek faktöre yükseltilmiş element sayısı ile orantılı olduğu durumlarda, $O(n) = n^k$.

polyphase merge: sabit sayıda girdi birleştirme dosyasının bir çıktı birleştirme dosyasında birleştirildiği bir birleştirme ve her girdi birleştirme dosyası, girdisi tamamen birleştirildiğinde hemen yeniden kullanılır.

pop: yığın silme işlemi.

postfix: operatörün operandlarının ardından yerleştirildiği bir ikili aritmetik notasyon.

postfix traversal: bir ifade ağacında, operatörün duasının operandından sonra yerleştirildiği bir geçiş. Postorder'a bakın.

postorder: bir ikili ağaçta, sol alt ağaç sağ alt ağaçtan önce işlenir v ve her ikisi de kökten önce işlenir; bir LRN geçişi.

postorder traversal: postorder'a bakın.

prefix: operatörün operandlardan önce yerleştirildiği bir ikili aritmetik notasyon.

prefix traversal: bir ifade ağacında, operatörün iki operandından önce yerleştirildiği bir geçiş. Preorder'a bakın.

preorder: bir ikili ağaçta, kök sol alt ağaçtan önce ve sol alt ağaç sağ alt ağaçtan önce işlenir; bir NLR geçişi.

preorder traversal: preorder'a bakın.

primary clustering: bir hashlenmiş listede evi adresi etrafında veri birikimi; ikincil kümeleme ile karşılaştırın.

prime area: bir hashlenmiş listede, evi adreslerini içeren bellek.

priority queue: elemanların öncelik numaralarına göre gruplara ayrıldığı ve en yüksek öncelikli elemanların önce işlenerek çıktığı bir kuyruk. Aynı önceliğe sahip öğeler, ilk giren ilk çıkar (FIFO) sırasına göre işlenir.

probability search: listenin, arama hedefi olma olasılığına göre sıralandığı bir arama; en olası hedefler önceliklidir.

probe: bir hashing algoritmasında, bir adresin hesaplanması ve başarı için bir test; bir arama algoritmasında, arama argümanı için test içeren döngünün bir yinelemesi.

prune: bir trie'de, gerekli olmayan tüm dalları kaldırma süreci.

pseudocode: gevşek tanımlı bir sözdizimini izleyen İngilizce benzeri ifadeler ve bir algoritmanın yada bir fonksiyonun tasarımını iletmek için kullanılır.

pseudorandom collision resolution: bir çarpışma çözümleme yöntemi, çarpışmadan sonra bir sonraki adresi belirlemek için bir psödo-rastgele sayı üretici kullanır.

pseudorandom hashing: bir hashing yöntemi anahtarı bir psödo-rastgele sayı üreticinde değişken faktör olarak kullanarak adresi belirler.

pseudorandom numbers: rastgele özelliklere sahip tekrarlanabilir bir sayı serisi.

push: yığın ekleme işlemi.

Q

quadratic efficiency: bir modülün verimliliğinin ölçüsü, çalışma süresinin eleman sayısının karesine proporsiyonel olduğu durumdur. İkinci dereceden efficiency, $O(n) = n^2$ gibi polinomik faktörlerden biridir.

quadratic loop: her döngünün lineer verimliliğe sahip olduğu iç içe geçmiş bir döngü.

quadratic probe: çarpışma çözümleme yöntemi, artışın çarpışma prob numarasının karesi olduğu yöntemdir, $1^2, 2^2, 3^2$ serisini verir.

kuyruk: verilerin yalnızca bir uçta, arka olarak adlandırılan yerde eklenebildiği ve diğer uçta, ön olarak adlandırılan yerde silinebildiği doğrusal bir listedir.

kuyruk veri düğümü: kullanıcı verilerini içeren ve bir sonraki düğüme işaret eden bir bağlantı alanı bulunan düğüm.

kuyruk veri yapısı: kuyruk yapısında kuyruk meta verilerini içeren düğüm, ön ve arka işaretçileri veya kuyruk sayısı gibi.

kuyruk simülasyonu: bir kuyruğun performansı hakkında istatistikler üretmek için kullanılan bir modelleme etkinliği.

hızlı sıralama: C. A. R. Hoare tarafından geliştirilen ve listeyi üç gruba tekrar tekrar bölerek sıralayan değişim sıralaması: anahtarları pivotun anahtarından daha az olan elemanların bir bölümü, listenin nihayet doğru konumuna yerleştirilen pivot elemanı ve pivotun anahtarına eşit veya daha büyük olan elemanların bir bölümü.

R

rastgele liste: verilerin herhangi bir sıralaması olmayan bir liste.

rastgele sayı: tüm üyelerin seçilme olasılığını aynı olduğu bir kümeden seçilen bir sayı.

arka: bağlı listede, son ögeyi tanımlayan bir işaretçi; bir kuyrukta, yapıya en son eklenen öge.

arka işaretçi: bir listede, listedeki son girişi gösteren bir işaretçi.

özyineleme: bir algoritmanın kendisini çağırdığı tekrarlayan bir süreç.

yeniden akış: B-ağacında, bir alt taşmanın düzeltilmesini sağlayan modül, verileri bir düğümden diğerine taşıyarak veya düğümleri birleştirerek gerçekleştirir.

yığın aşağı yeniden düzenleme: yığın işleme sırasında, bir girişin yığın ağaç yapısı boyunca aşağı doğru akmasını sağlayan mantık, yığındaki uygun konumuna gelene kadar devam eder.

yığın işleme sırasında, bir girişin yığın ağaç yapısında yukarı doğru akmasını sağlayan mantık, yığındaki uygun konuma ulaşana kadar devam eder.

kısıtlı liste: verilerin yalnızca listenin uçlarına eklenebileceği veya silinebileceği ve işleme yalnızca verilerin uçlardaki işlemlerle sınırlı olduğu bir listedir.

geri alma: bir listedeki bir ögenin konumu ve geri dönüşü.

dönüş koşulu: algoritma belgelerinde, geri dönen herhangi bir değerin açıklaması algoritması.

sağ yüksek: bir AVL ağacında, bir ağaç veya alt ağaç sağ alt ağacının yüksekliğinin sol alt ağacından daha büyük olduğu durum.

solun sağında: dengesiz bir AVL ağacının geçici bir durumu, burada sol yüksek bir düğümün sağ yüksek alt ağacı vardır.

sağ sağ: dengesiz bir AVL ağacının geçici bir durumu, burada sağ yüksek bir düğümün sağ yüksek bir alt ağacı ile birlikte bulunur.

sağ alt ağaç: bir ikili ağaçta, bir düğümden sağ dalda bulunan alt ağaç.

dağıtım: bir ardışık birleşim girdi verilerinin, alter natifte bir adım düşüşünden sonra birleşim çıktısına kopyalandığı bir birleştirme işlemi. birleşim girişi.

kök: bir ağacın ilk düğümü.

dönüşüm: bir anahtarın son kısmının anahtarın önüne kopyalandığı bir hashing yöntemi.

S

arama: belirli bir değer içeren bir veya daha fazla ögeyi bulmak için bir listeyi inceleyen süreç.

arama argümanı: bir aramada aranan anahtar değeri.

ikincil kümelenme: bir hashlenmiş liste boyunca bir çarpışma yolu boyunca verilerin birikimi; birincil kümelenme ile karşılaştırılır.

seçim sıralaması: sıralanmamış bir listenin son kısmındaki en küçük değerin seçilip, sıralı kısmın sonuna yerleştirildiği sıralama algoritması.

seçim ifadesi: bir koşulu değerlendiren ve sıfır veya daha fazla alternatifi yürütüldüğü bir ifade.

kendine referans veren veri yapısı: kendisine bir referans içeren bir yapı.

gözcü: bir listenin veya bir dosyanın sonunu koruyan bir bayrak. Gözcü genellikle bir anahtar alanı için maksimum değerdir ve geçerli bir veri değeri olamaz.

gözcü araması: arama argümanının listenin sonundaki ekstra bir ögeye yerleştirildiği bir arama algoritması.

dizi: yapılandırılmış programlamada, bir algoritmadaki çalışma yolunu değiştirmeyen bir veya daha fazla açıklama.

ardışık dosya: verilerin dosyadaki ilk girişten seri olarak işlenmesi gereken bir dosya yapısı.

ardışık arama: aramanın ilk elemandan başlayarak ve aranan değere eşit bir elemanın değeri bulunduğu veya listenin sonuna geldiğinde sona erdiği bir doğrusal liste ile kullanılan bir arama tekniği.

hizmet: bir kuyrukta, gerekli sonucu elde etmek için gereken herhangi bir etkinlik.

hizmet süresi: bir kuyrukta, bir müşteri talebinin işlenmesi için gereken ortalama süre.

kabuk sıralama: bölümlere ayrılmış bir liste kullanan bir ekleme sıralamasıdır. Her sıralama geçişi, verileri ilgili bölümlerinde sıralar. Her geçişten sonra, bölüm boyutu azaltılır ve sıralama, liste tamamen sıralanana kadar tekrarlanır.

kardeşler: ortak bir ebeveyni olan iki veya daha fazla ağaç düğümü.

tek sonucu kuyruğu: yalnızca bir müşteriyi aynı anda işleyebilen bir kuyruk.

tek bağlı liste: her elemanın veri ve yalnızca bir referans içerdiği düğüm koleksiyonu; bu referans, sonraki elemanın konumudur; çift bağlı liste ile kıyasla.

sıralama: bir listeyi veya dosyayı sıralayan işlem.

sıralama verimliliği: sıralama performansının bir ölçüsü.

sıralama sırası: bir listedeki veya dosya içindeki verilerin düzeni, ya artan ya da azalan.

sıralama geçişi: bir sıralamada, verilerin bir kez geçişi.

sıralama aşaması: bir dış sıralamada verilerin üzerinden geçişin ilk aşaması, burada bir sıralama algoritması verileri daha fazla işleme için birleştirme çalışmaları olarak sıralar.

sıralama kararlılığı: sıralama çıktısında eşit anahtar a sahip giriş verilerinin göreceli düzenlerini koruduğu bir sıralamanın özelliği.

spagetti kodu: mantık akışının programda bir tabaktaki spaghetti gibi dolaştığı, eski, yapılandırılmamış bir kodlama stili.

yayılma ağacı: grafikteki tüm köşeleri içeren bir bağlı grafikten çıkarılan bir ağaç.

kararlılık: bkz. sıralama kararlılığı.

yığın: verilerin yalnızca bir uçta, top olarak adlandırılan yerde eklenip silinebildiği kısıtlı bir veri yapısı.

stackframe: bir işlev çağırısında kullanılan mantıksal bir yapı olup, çağırılan işlev için parametreleri, yerel değişken değerlerini, çağırılan işlevdeki dönüş ifadesini ve dönüş değerini alacak değişkenin adresini içerir.

stepdown: birleştirme dosyasındaki verilerin aradığı sıralamasının broken olduğu bir olaydır; birleştirme çalışmasının sonu. Dosyanın sonu da bir stepdown olarak kabul edilir.

doğrudan ekleme sıralaması: sıralanmamış alt listeye ait ilk elemanın, uygun konumda sıralı alt listeye eklenmesiyle her geçişte gerçekleştirilen bir sıralama.

güçlü bağlı grafik: her düğümden diğer her düğüme bir yolun bulunduğu bir grafik; zayıf bağlı grafik ile karşılaştır.

yapı şeması: bir programı hiyerarşik algoritmalar veya fonksiyonlar akışı olarak temsil eden bir tasarım ve dokümantasyon aracı.

yapılandırılmış programlama: bir programın bir giriş ve bir çıkışa sahip modüllere ayrıldığı program tasarım tekniği.

çıkarma hashing'i: anahtardan yalnızca sabit bir değer çıkarıldığı bir hashing yöntemi.

alt ağaç: ağacın kökünün altındaki herhangi bir bağlı yapı.

eşanlımlı: hashed listede, aynı ana adresine hashlenen iki veya daha fazla anahtar.

T

iplikli ağaç: null (yaprak) işaretçilerinin, ardıl düğümlerine işaret eden işaretçilerle değiştirildiği bir ağaç yapısı.

token: bir işlemi veya bir bayrağı temsil eden herhangi bir sözdizimsel yapı, örneğin artı işareti (+).

üst: bir yığında, çıkarılacak bir sonraki eleman.

geçiş: bir yapıda her elementin bir kez ve sadece bir kez işlendiği algoritmik bir süreçtir.

ağaç: her düğümün yalnızca bir öncülü olduğu lineer olmayan bir liste.

trie: null alt ağaçların budandığı, yani null alt ağaçların silindiği bir sözcüksel arama ağacı.

U

underflow: bir veri yapısından veri silme girişimi yapıldığında yapının boş olması durumunda meydana gelen bir olay.

yönsüz grafik: çizgiler üzerinde yön belirtisi olmayan bir grafik.

V

düğüm: bir grafikteki bir düğüm.

düğüm listesi: bir grafikte, düğümlerin listesi.

W

zayıf bağlı grafik: en az bir düğümün diğer herhangi bir düğüme giden yolu olmayan bir grafik; **güçlü bağlı grafik** ile kıyasla.

ağırlıklı grafik: çizgilerinin ağırlıklandırıldığı bir grafik.

Bu sayfa kasıtlı olarak boş bırakılmıştır

Semboller

_Çıkış 671

Sayısal

2-3 ağaç 470
2-3-4 ağaç 470

A

a_karakter 664
iptal 671
mutlak 671
soyut veri türü 10–15, 213
atomik veri 10
B-ağaç 449
tanım 13
modeli 14
işlemler 14
yığın dizisi
uygulama 681
yapı 15
soyutlama 12
Ackerman'ın sayısı 73, 74
düğüm ekle 217
BST ağacı 313
Adelson-Velskii, G. M. 341
komşuluk listesi 489
komşuluk matris 488
koşu köşeler 482

ADT *bkz* Soyut Veri Türü
algoritma verimliliği 28–37
algoritmalar 28
büyük-O analizi
32büyük-O nota
syonu 32ağımlı kare
31üstel 34
faktöriyel 34
doğrusal 34
doğrusal logaritmik 29, 31, 34doğr
usal döngüler 29
logaritmik 31, 34
iç içe döngüler 30
polinom 34
kare 31, 34
standart ölçüler 33
algoritmalar 28
değişim sıralamaları 565
ekleme sıralamaları 552
radix sıralama 595
ayrıca bakınız
Big-Oseçim sıralar
ı 541ata 266
yay 481
aritmetik ifade
infix 57
postfix 57, 58
prefix 57, 58
varış oranı 167
ASCII 282, 643–648

asctime 674
atan2f 665
atan2l 665
atanf 664
atanl 664
atexit 671
atof 672
römork 672
672 sırasında
atomik veri 10
AVL Ağacı 341–348
algoritmalar 257–375
dengede 351, 354
denge faktörü 342, 355
sayım 374
oluştur 35
9data yapısı 357d
efinasyon 342
sil 352, 365tasa
rım 349im
ha et 375b
oş 373dol
gu 374
ekle 348–350, 360
solun solu 344
sağın solu 347
al 370
solun sağı 346
sağın sağı 345
döndür 351

AVL ağaç soyut veri tipi

356–376

say 374

oluştur 359

data yapısı 357

sil 365

yok et 375

boş 373

dolu 374

ekle 360

al 370

gezin 372

B

B* ağacı 471

B+ ağacı 471

geri izleme 122s

ekiz kraliçe 125h

edef arama 122

geri işaretçi 240deng

e faktörü 273deng

eli birleştirme 577d

engeli ikili ağaç 273teme

l durum 49

Bayer, R. 426

büyük-o notasyonu 32–33ikili

arama 607baloncuk sırala

ma 567yığın sıralama

542ekleme sıralama

554hızlı sıralama 568radi

x sıralama 595seçim

sıralama 542sıralı ar

ama 607shell sıralama 55

5

malzeme listesi 268

ikili arama 603, 606, 608

ikili arama ağacı

tanım 299

sil 307

en büyük bul 303

en küçük bul 302

ekle 305

arama 303, 304ikili a

ğaç 270–286denge 272, 2

73denge faktörü 273t

am 273yükseklik 271in

order geçişi 277s

ol işaretçi 270

neredeyse tam 273

null 271

postorder geçişi 278pr

eorder geçişi 275sağ i

şaretçi 270geçiş

273ikili ağaç

geçişigenişlik öncel

ikli 274, 278derinlik ö

ncelikli 273

sıralı 277

posta siparişi 278

ön sipariş 275

dal 265

Brassard, Gilles 28

Bratley, Paul 28

genişlik öncelikli gezinti 274, 278,

487, 498

algoritma 279

BST 299

sınıf listesi uygulaması 328t

am sayı uygulaması 324B

ST soyut veri türü

309–324

düğüm ekle 313

say 322

oluştur 313

veri yapısı 311, 500

yok et 323

boş 321

dolu 322

düğüm kaldır 315

al 318

gez 320

BST ağacı

soyut veri türü 309

düğüm ekle 313

say 322

oluştur 313

veri yapısı 311, 500

veri yapısı bildirimi 311

sil 315

yok et 323

boş 321

dolu 322

geri al 318

gezin 320B-a

ğacı 425–4722-3

ağacı 4702-3-

4 ağacı 470

ADT 449–469

denge 440, 442birl

eştir 440, 444veri y

apısı 449, 451tanım 426

sil 435

sil denge 440, 442

sil birleştir 440, 444

sil giriş 439

orta sil 439düğü

mü sil 437şemay

ı ekle 428düğüm

ü ekle 429taşma

427yeniden akış

440arama 448a

rama düğümü 4

32böl düğümü 4

33gezinme 446a

lt akış 435data y

apısı 449

kabarcık sıralama 558, 5
67paket 629

C

calloc 671
veri sınıflandırma 168
Cayley, Arthur 265
ceil 665
tavan 665
hücre 665
karakter sınıflandırma fonksiyonu
a_char 664
sülfat 664
İsascii 664
isntrl 664
rakam 664
grafik 664
daha düşük 664
sürat 664
nokta 664
isspace 664
üst 664
isx rakamı 664
karakter dönüştürme fonksiyonu
toint 664
alçak 664
toupper 664
karakter G/Ç
fgetc 669
getc 669
getchar 669
putc 669
putchar 669
ungetc 669
çocuk 266
kronolojik liste 193daire
sel bağlı liste 239sınıf listes
i uygulaması 328
clearerr 668

saat 673
asctime 674
ctime 674
fark zamanı 673
mktime
zaman 674
kümeleme 621
çarpışma 612
çarpışma çözümü 620–635
kova 629
çift hashing 626an
ahtar ofseti
627lineer prob
624bağlı liste
628 açık adreslem
e 623 taşma ala
nı 628asal ala
n 612sahte rastge
le 619kare prob 62
5tam ağaç 273ka
rmaşık lineer liste yapıları
dairesele bağlı liste 239
çift bağlı liste 240ço
klu bağlı liste 244
bileşik veri 10ba
ğlantılı grafik 482c
osf 665
coshf 665
koşl 665
maliyeti 665
oluştur
graf 482
liste 216
kuyruk 154, 160, 691
yığın 84, 96, 683
zaman 674
döngü 482

D

veri sıkıştırma 283
veri akışı 654
veri düğümü 83
veri yapısı 11
veri türü 10
derece 265, 483
sil
AVL ağacı 352, 365
ikili arama ağacı 307
BST ağacı 315
B-ağacı 435–437
çift bağlı liste 243
yığın 392, 400
çok bağlı liste 247
düğüm 202
kuyruk 148
yığın 80, 86sil
e 194bağımlı iki
nci dereceden
efficiency 34
derinlik 267
derinlik öncelikli gezinti 274,
485, 496
kuyruktan çıkar 148, 155, 161, 693
soy 266
tasarım 650
özyinelemeli algoritmalar tasarlama
48–52
listeyi yok et 212, 22
7kuyruk yok et 158, 165, 695y
ığın yok et 101, 687zam
an farkı 673
basamak çıkarma hashing 616ikiyö
nlü grafik 481
Dijkstra, Edsger 8, 10, 519
en kısa yol algoritması 518doğru
dan hashing 613
yönlendirilmiş grafik
481ayrık grafik 483

dağıtım aşaması 577
böl 671
bölüm kalanı hashing 615
çift hashing 626
çift bağlı liste 240
geri işaretçi 241
silme 243
ileri işaretçi 241ekle
me 241arka iş
aretçi 240tekrarl
anan anahtar 217

E

kenar 481
efficiency
gör algoritma verimliliği
eight queens problem 125
boş liste 16, 208, 225boş
kuyruk 157, 694boş yığı
n 88, 100, 686kapsülleme
13
kuyruk ekle 148, 154, 160, 692
Öklid algoritması 53
değişim sıralamaları 558–573
baloncuk 558, 567
hızlı 560, 567–568
çıkış 671
deneyim 665
deneyim2 665
deneyim2f 665
deneyim2l 665
deneyim 665
açıklama 665
deney1 665
expm1f 665
deney1l 665
üstel verimlilik 34
ifade 280

ifade ağaçları 280–282infi
x geçişi 280postfix
geçişi 281prefix geçi
şi 282geçişler 280h
arici sıralamal
ar 534, 573–583dengeleme b
irleştirme 577birleşti
rme 573
minimum yığın sıralaması 582
doğal birleştirme 577
polifaz birleştirme 580

F

muhteşem 665
fabsf 665
muhteşem 665
faktöriyel 45va
ka çalışması 45ite
ratif çözüm 47özyinelemel
i çözüm 47faktöriyel verimli
liği 34
fclose 668
feof 668
korku 668
fgetc 669
670'i buluyor
Fibonacci sayıları 54, 55, 700
Fibonacci, Leonardo 54, 700
FIFO 77, 147
dosya
clearerr 668
fclose 668
feof 668
ferror 668
fgetc 669
fgets 670
fopen 669
fprintf 669
fputc 669

çıkışlar 670
korku 670
fscanf 669
670'i arayın
ftell 670
foritis 670
kaldır 670yeniden ad
landır 670geri sar 67
o
tmpfile 670
tmpnam 670
en büyük düğümü bul 303en küçük d
üğümü bul 302bayrak 654
kat 665
katf 665
katl 665
fmod 665
fmodf 665
fmodl 665
katlama sınır hashing 617katlama kayd
ırma hashing 617
fopen 669
ileri işaretçi 240
fprintf 669
fputc 669
çıkışlar 670
korku 670
ücretsiz 671
frexp 666
frexpf 666
frexpl 666
fscanf 669
670'i arayın
ftell 670
tam liste 208, 225
tam kuyruk 157, 164, 694t
am yığın 89, 100, 686par
ametre olarak işlev 23
fwrite 670

G

gcd 52
 genel durum—özyineleme 49
 general liste 78
 genel ağaç 287–289
 malzeme listesi 268
 ikiliye dönüştür 288
 tanım 287
 silme 288
 FIFO ekleme 287
 anahtar-sıralı ekleme 287
 LIFO ekleme 287
 genel ağaç 268g
 eneric kod, ADT'ler için 17
 getc 669
 getchar 669
 gets 670
 amaç arama 122
 gezinta 268
 ayrıca bakınız malzeme liste
 sigraf
 komşu köşeler 482
 yay 481
 genişlik öncelikli gezinti 498
 bağlı 482
 oluştur 482
 döngü 482
 tanım 481
 derece 483
 yay sil 495
 köşe sil 493
 derinlik öncelikli gezinti 496
 diyagram 481
 yönlendirilmiş
 481ayrık 48
 3kenar 48
 1giriş derecesi
 483yay ekle 49
 3köşe ekle 492

çizgi 481
 döngü 482
 minimum yayılan ağaç 514
 ağ 513
 derece 483
 yol 481
 tepe noktasını al 4
 96en kısa yol 518
 yayılan ağaç 514g
 üçlü bağlı 482yapı 490
 –491yönlendirilme
 miş 481tepe n
 oktası 481
 zayıf bağlı 483
 ağırlıklı 513
 graf algoritmaları 490–500
 breadth-first traversal 498
 create 491
 arc sil 495vert
 ex sil 493
 depth-first traversal 496
 arc ekle 493
 vertex ekle 492
 retrieve vertex 496
 graf işlemleri 483–488kenar e
 kle 484
 genişlik öncelikli geçiş
 487, 498
 oluştur 482
 arc sil 495
 kenar sil 485ver
 tex sil 483, 493
 depth-first traversal 485, 496
 vertex bul 485
 yayınla yay 493
 nokta ekle 483,492
 nokta al 496
 geçiş 485

grafik depolama yapıları
 488–490
 komşuluk listesi 489
 komşuluk matris 488
 nokta listesi 490
 grafik yapısı 490–491
 en büyük ortak bölen 52

H

hashli arama 611–620
 kova 629
 kümeleme 621
 çarpışma 612
 tanım 611raka
 m çıkarma 616doğr
 udan 613
 bölüm kalanı 615çift
 hashleme 626katlam
 a sınırı 617katlama k
 aydırması 617ana ad
 res 612anahtar kayd
 ırması 627lineer sor
 gulama 624bağlı list
 e çarpışmaçözüm 62
 8yük faktörü 621or
 ta kare 617modül-bö
 lüm 615açık adresle
 me 623birincil küme
 leme 621asal alan 6
 12
 prob 613
 psödo-rastgele 619
 psödo-rastgele çarpışma
 çözüm 626
 kuadratik prob 625
 dönüş 618
 ikincil kümelenme 622
 çıkarma yöntemi 615
 eşanlam 612

baş 83**yığın 389**

oluştur 421

çocuk konumu 394

data yapısı 394, 401

tanım 389

sil 400

silYığın 405

ilk yaprak 395

yığınlaştıır 398

ekle 398

ekleYığın 404

son yapraksız 395

maks 390

dk 390

ebeveyn konumu 394

yığın aşağı 392, 396

yığın yukarı 391, 396

yığınAşağı 406

yığınYukarı 405

sağ kardeş 394

yapı 390, 401

yığın algoritmaları 396–407

yığın oluştur 397, 421

yığını sil 400

yığı ekle 398

yığın aşağı 396

yığın yukarı 396

yığın uygulamaları 407–416

öncelik sırası 409

ayrıca bakınız yığın

seçim 408yığın s

ıralama 539, 542

minimum 582

yığınlaştıır 398

yükseklik 267

ikili ağacın yüksekliği 271

yükseklik dengeli ağaç 342

ayrıca bakınız AVL ağacı

Hollerith, Herman 533

ev adresi 612

Hopper, Grace 265

Huffman kodu 282kav

ram 282–283kodlam

a adımları 283ağırlık a

taması 283

I

ilogb 666

ilogbf 666

ilogbl 666

artış boyutu 553giriş d

erecesi 265, 483

infix 57

infix gezintisi 280

inorder gezintisi 277

ekle

AVL ağacı 348, 360

ikili arama ağacı 305

çift bağlı liste 241

liste 198

çok bağlantılı liste 246

düğüm ekle 217

ekleme 193

ekleme sıraları 547–558

kabuk 549, 554

doğrudan ekleme 547, 554ta

m sayı uygulaması 324a

kıllı isimler 8, 650iç düğü

m 266iç sıralama 5

34

isalnum 664

sülfa 664

İsascii 664

isntrl 664

rakam 664

grafik 664

daha düşük 664

sürat 664

ispunct 664

isspace 664

isupper 664

isxdigit 664

K

anahtar 196

anahtar ofseti 627

Kirchhoff, Gustav 265

Knuth, Donald E. 205, 265,
533, 534, 550, 553, 555,
561, 567, 600**L**

laboratuvarlar 671

Landis, E. M. 341

Idexp 666

Idexpf 666

Idexpl 666

ldiv 671

yaprak 266

sol alt ağaç 270

seviye 266

leksikal arama ağacı 472–473

budama 473

arama 474

yapı 474kütü

phane

sınırlar.h 657

LIFO 77, 79, 80

data yapısı 79özyinele

menin sınırlamaları 50limits kütüp

hanesi 657

doğrusal verimlilik 34doğ

rusal liste 77

FIFO 77

genel liste 78

LIFO 79

sınırlı liste 77

doğrusal liste aramaları 597–609

ikili 603, 610

olasılık 601

gözcü 600

ardışık 597, 609doğrus

al liste *bkz* listedo

ğrusal logaritmik verimlilik 34doğru

sal sorgulama 624

doğrusal arama 608ba

ğlantı 15

bağlı liste 15, 628

çarpışma çözümü 628

data 15

boş 16

bağlantı 15

düğüm 16

işaretçi 17te

rsine yazdır 50kendi

ne referans 16

lint 666

lintf 666

tüysüz 666

liste

düğüm ekle 217

kronolojik liste 193

say 210, 226

oluştur 216

sil 202

ilk düğümü sil 203

genel durumu sil 204

düğümü sil 202

silme 194

yok et 227

boş 225

dolu 225

başında ekle 199

sonunda ekle 201

ortasında ekle 199

boş listeye ekle 198

düğüm ekle 198, 217

ekleme 193ana

htar 193

meta verisi 196

aşırı yük 202

rastgele listeler 193

düğüm kaldır 219

alma 194

düğüm al 224

arama 205, 221

tek bağlı 239

test 238

geçiş 194geçm

ek 210, 226liste

ADT 213–228dü

ğüm ekle 217lis

te oluştur 216d

üğüm sil 202list

e yok et 227boş

liste 225dolmu

ş liste 225

düğüm ekle 176

liste sayısı 226

düğüm kaldır 219

düğüm al 224

arama listesi 221

geçmek 226

liste algoritmaları 197–213

liste oluştur 197

düğüm sil 202liste yok

et 212boş liste 208dol

muş liste 208düğüm

ekle 198liste sayı

sı 210düğüm al 208ar

ama listesi 205liste

geçiş 210

llabs 671

lldiv 671

llround 666

llroundf 666

llroundl 666

yükleme faktörü 621

log 666

log10f 666

log10l 666

günlük2 666

log2l 666

logaritmik verimlilik 34

logf 666

günlük 666

döngü 8, 482

lroundf 666

Lukasiewicz, Jan 58

M

malloc 671

maks-heap 390

McCreight, E. 426

üç anahtarın medyanı 562–563

memchr 672

memcmp 672

memcpy 673

memmove 673

birleştirme 573

dengeli 577da

ğıtım aşaması 577birleşti

rme aşaması 576m

inimum heap sıralaması 58

2doğal 577

polifaz 580

yayılma 576

çalıştır 575

gözetmen 575sıra

lama aşaması 576adı

m düşürme 575met

a veri 83, 196orta kare ha

shing 617

min-heap 390

minimum yığın sıralama 582
 minimum yayılma ağacı 514
 mktime 673
 667'ye karşı
 Modfal 667
 modüler programlama 10
 modulo-bölme hashing 615
 çok bağlantılı liste 244
 sil 247
 ekle 246
 çok sunucu kuyruğu 166
 m-yönlü arama ağacı 423-425
 sıra 423
 yapı 425

N

doğal birleştirme 577
 yakın int 667
 yakınıntf 667
 yakınıntl 667
 neredeyse tamamlanmış
 ağaç 273iç içe geçmiş ver
 i yapısı 11ağ 513-
 522tanım 513
 minimum yayılma ağacı 514
 en kısa yol 518
 yayılma ağacı 514
 ağırlıklı grafik 513
 Newton'un yöntemi 73
 düğüm 16, 20, 265
 null ağacı 271

O

açık adresleme 623
 sıra 423
 sıralı liste arama 205
 çıkış derecesi 265, 483

taşma 80
 AVL ağacı 380
 B-ağacı 427
 liste 202
 kuyruk 148
 yığın 80
 taşma alanı 628

P

palindrom 74eb
 eveyne 266
 parsing 107
 yol 266, 481ışar
 etçi
 bağlantılı liste 17
 arka 208
 işlev 23, 24
 to void 18
 polinom verimliliği 34polifa
 zlı birleştirme 580
 pop 80
 pop yığını 684
 son durum 7
 postfix 57, 58
 postfix gezintisi 281
 postorder gezintisi 278
 powf 667
 powl 667
 ön koşul belgeleri 7
 prefix 57, 58
 prefix gezintisi 282
 preorder gezintisi 275
 birincil kümeleme 621
 asal alan 612, 623-624, 628
 printf 669
 öncelik kuyruğu 409
 olasılık araması 601
 probe 613

program verimliliği. *Bak*
 algoritmalar
 programlama yapıları 8
 prune trie 473
 pseudocode 5
 algoritma başlığı 6zeki v
 eri adları 8döngü 9

son durum 7
 ön koşul 7
 dönüş durumu 7
 seçim ifadesi 9
 sıra 8
 ifade yapıları 8ifade
 numaraları 7değiş
 kenler 8
 psödo rastgele çarpışma
 çözümü 626
 psödo rastgele hashing 619
 itme 80
 itme yığını 85, 97, 684
 putc 669
 putchar 669670 d
 eğerini verir

Q

kare verimlilik 34
 kare sorgulama 625
 kuyruk
 say 158, 165, 695o
 luştur 154, 160, 69
 1data düğüm
 ü 153data yapısı
 152anım 14
 7çıkır 148, 155, 161, 693y
 ok et 158, 165, 695b
 oş 157, 164, 694sır
 aya al 148, 154, 160, 692F
 IFO yapısı 147

ön 147, 148, 156, 163
 dolu 157, 164, 694
 baş 152
 öncelik 409
 kuyruk ön 149, 693
 kuyruk arka 150, 694
 arka 147, 150, 164
 simülasyon 13, 175
 zaman 167

kuyruk ADT

kuyruk oluştur 160, 691data ya
 pısı 159, 689dekuyruğa alma 16
 1, 693yok et 165, 695boş
 kuyruk 164, 694kuyruğ
 a alma 160, 692dolmuş kuyruk
 164, 694kuyruk sayısı 16
 5, 695kuyruk ön 163, 693ku
 yruk arka 163, 694kuyruk ADT
 uygulaması

159–166

oluştur 160deku
 yruğa alma 161y
 ok et 165kuyruğ
 a alma 160dolm
 uş kuyruk 164ku
 yruk sayısı 165k
 uyruk boş 164ku
 yruk ön 163kuyr
 uk algoritmalarıku
 yruk oluştur 154
 dekuyruğa alma
 155kuyruğu yok
 et 158boş kuyru
 k 157kuyruğa al
 ma 154dolmuş k
 uyruk 157kuyru
 k sayısı 158kuyr
 uk ön 156

kuyruk uygulamaları 168–182

data sınıflandırma 168
 kuyruk simülasyonu 175
 simülasyon algoritması 178
 kuyruk sayısı 165, 695
 kuyruk veri yapısı 159
 kuyruk boş 164

kuyruk ön 156, 163, 693kuyr
 uk işlemleri 148–151dekuyru

ğa alma 148

kuyruğa alma

148kuyruk ön 148

kuyruk arka 150

kuyruk arka 163, 694k

uyruk simülasyonu 13, 175al

goritma 178

kuyruk zamanı 167

kuyruk—dizi uygulaması

688–696

kuyruk oluştur 691d
 ata yapısı 689dekuyr
 uğa alma 693kuyruğ
 u yok et 695boş kuyr
 uk 694kuyruğa alma
 692dolmuş kuyruk 6
 94kuyruk sayısı 694k
 uyruk ön 693kuyruk
 arka 694kuyruk teori
 si 166–168varış oranı
 167

çoklu sunucu kuyruğu 166

kuyruk süresi 167

yanıt süresi 167

hizmet süresi 167

tek sunucu kuyruğu 166

hızlı sıralama 560, 561, 562, 567

R

radix sıralama 595

rastgele 671

yeniden tahsis 671

arka işaretçi 208, 239, 240

özyineleme 46

temel durum 49

tanımlı 46

tasarım soruları 50

tasarım algoritmaları 48

Fibonacci sayıları 54, 700

genel durum 49

nasıl çalışır 134sınır

lamalar 50yığın

çerçevesi 135kuyr

uk 50, 75

Hanoi kuleleri 65özyi

nelemeli ilişki 679geri d

önüşüm 203

yığın aşağı 392, 396

yığın yukarı 396

yenidenYığınl

a 391kalan 6

67kalanf 667k

alanl 667kaldı

r 670

düğümü kaldır 219

remquo 667

kalıntı 667

667

yeniden adlandır 670

yanıt süresi 167

kısıtlı liste 77

kısıtlayıcı veri yapısı 77

kuyruk 147

yığın 77

geri alma 194

düğümü geri al 208, 224

dönüş koşulu 7
 geri sar 670
 sağ alt ağaç 270
 rintf 667
 rint 667
 dağıtım 576
 kök 266
 dönme hashing 618
 roundf 667
 yuvarlak 667

S

skalar veri 10
 ölçekbIn 667
 ölçekbInf 667
 ölçekbInl 667
 ölçekbn 667
 ölçekbnf 667
 tarama 669
 arama 597
ayrıca bakınız hashlenmiş arama
 ikili 603, 606, 609 ikili arama ağa
 cı 303, 304
 efficiency 607-608
 doğrusal 608
 bağlı liste 205
 liste 205
 sıralı 205, 602 ola
 sılık 601
 sentinel 600
 ardışık 597, 607, 609
 ikincil kümelenme 622
 seçim sıralamaları 537-546
 yığın 539, 542
 doğrudan seçim 537,
 541, 542
 seçim ifadesi 9 kendine
 referans veren yapı 16, 83
 sentinel 575
 sentinel arama 600

dizi 8 ardış
 ık arama 597,
 607, 609
 hizmet süresi 167 sha
 ker sıralama 592 kab
 uk sıralama 549, 552, 554
 Shell, Donald L. 549
 en kısa yol algoritması 518 kardeş 26
 6
 sınıf 668
 tek sunucu kuyruğu 166
 Singleton, R. C. 561
 tek bağlı liste 239
 Singhf 668
 Sinhala 668
 668'de
 sırala 533
 algoritmalar 541, 552, 562
 baloncuk 558, 567
 verimlilik 535
 çaba 554
 harici 534 yı
 ğın 539, 542 e
 kleme 547, 554 iç
 534 birleştirir
 me aşaması 576
 sıra 534
 geç 535
 aşama 576
 hızlı 560, 567-568
 radix 595
 seçim 537, 541, 542
 shaker 592
 kabuk 549, 552, 554
 stabilite 535
 spagetti kodu 10
 yayılma ağacı 514
 minimum 514
 sprintf 669
 sqrtf 668

SQL 668
 karekök
 Newton'un yöntemi 73
 srand 671
 sscanf 669
 yığın 79
 dizi uygulaması
 681-687
 say 89, 101, 686
 oluştur 96-97
 veri düğümü 83
 yok et 89, 101
 boş 100
 dolu 100
 baş 83
 taşma 80
 çıkar 80, 86, 98
 it 80, 84
 üst 79, 81, 99, 685
 alt akış 80
 yığın soyut veri türü 95-102
 yığın oluştur 96
 veri yapısı 95
 yığını yok et 101
 boş yığın 100
 dolu yığın 100
 uygulama 83, 95
 yığını çıkar 97
 yığını it 97
 yığın sayısı 101
 yığın üstü 99
 yığın algoritmaları 84-90
 yığın oluştur 84
 yığını yok et 89
 boş yığın 88
 dolu yığın 89
 yığını çıkar 86
 yığını it 85
 yığın sayısı 89
 yığın üstü 81, 87

yığın uygulamaları 102-134
 geri izleme 122
 ondalık sayıyı ikiliye çevirme 106
 postfix değerlendirme
 ifadeler 118infix'ten
 postfix'edönüşü
 m 110
 parsing 107
 erteleme 110ters
 liste 103veriler
 i tersine çevirme
 103yığın veri yapısı 83
 ADT yapısı 95yığı
 n LIFO yapısı 79yığın çe
 rçevesi 135
 verimlilik için standa
 rt ölçüler 33
 adım adım 575
 doğrudan ekleme modülü 562doğr
 udan ekleme sıralaması 547, 554doğr
 udan seçim sıralaması 537,
 541, 542
 strcat 673
 strchr 673
 strcmp 673
 strcpy 673
 strcspn 673
 string
 G/Ç—fgets 670
 G/Ç—fputs 670
 G/Ç—gets 670
 G/Ç—puts 670
 memchr 672
 memcmp 672
 memcpy 673
 memmove 673
 strcat 673
 strchr 673
 strcmp 673
 strcpy 673
 strlen 673

strncat 673
 strncmp 673
 strncpy 673
 strpbrk 673
 strrchr 673
 strspn 673
 strstr 673
sayfa 672
strof 672
 672 çöktü
 strtold 672
 strtoll 672
Sokak 672
Streç 673
 strncat 673
 strncmp 673
 strncpy 673
 güçlü bir şekilde bağlı grafik 482
 strpbrk 673
 strrchr 673
 strspn 673
 strstr 673
sayfa 672
strof 672
 672 çöktü
 strtold 672
 strtoll 672
Sokak 672
 yapı—kendine referans veren 16
yapı şeması 649-655
 ortak modüller 652
 koşullu çağrılar 652
 koşullu döngüler 653
 data akışları 654
 özel veya 652ba
 yarak 654
 döngü 65
 3okuma 651ö
 zyineleme 653k
 urallar 655
 semboller 650

yapılandırılmış programlama 10
 alt ağaç 267
 eşanlamı 612
 sistem 670

T

kuyruklu özyineleme 50, 75
 tanf 668
 tanhf 668
 gezinti 668
 668 içerir
 test sürücüsü
 191test 23
 8iplikli ağaç 334, 335za
 man 674
 tmp dosyası 670
 tmpnam 670
 toint 664
 belirteç 280
alt 664
top 79
 toupper 664
 Hanoi Kuleleri 65alg
 oritma 68geçi
 ş 194geçiş list
 esi 266ağaç 265,
 3042-3 ağacı
 4702-3-4 ağac
 ı 470ata 266ikil
 i 270

 ikili arama ağacı 299
 dal 265
 B-ağ 425
 çocuk 266
 tanım 265
 derece 265
 derinlik 267
 soy 266
 en küçük düğümü bul 302

genel 268yü
kseklik 267
giriş derecesi
265iç düğüm 266ya
prak 266
seviye 266
düğüm 265
çıkış derecesi 265
ebeveyn 266
yol 266
kök 266
kardeş 266
alt ağaç 267
ipli 334, 335
trie
arama 474
yapı 474

trunc 668
truncf 668
trunc1 668
2-3-4 ağaç 470
tip 10

U

taşma
B-ağaç 435
kuyruk 148, 149, 155
yığın 80
yönsüz grafik 481
ungetc 669

V

düğüm 481e
kle 483düğü
m listesi 490Von
Mises doğum günü par
adoks 623

W

zayıf bağlı grafik 483
ağırlıklı grafik 513
Wirth, Niklaus 10